DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY · THE LAZY PROGRAMMER

PyTorch: Deep Learning and Artificial Intelligence

ANNs, CNNs, RNNs, NLP, recommenders, transfer learning, GANs and deep RL in PyTorch

Volume 2 of 3 · Sections 39–93

55 SECTIONS

530 FIGURES

440 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 39 Convolutional Arithmetic 4

LECTURE 40 Loading in the Data 15

LECTURE 41 Fashion MNIST 23

LECTURE 42 Fashion MNIST and Color Image 26

LECTURE 43 Why Data Is Important 28

LECTURE 44 Batch Normalization 39

LECTURE 45 Number of Parameters and Predicted 46

LECTURE 46 Shape of a Sequence 50

LECTURE 47 All Machine Learning Interfaces Are the Same 72

LECTURE 48 Validation Predictions and Sine Wave 84

LECTURE 49 Perfect Forecast for a Sine Wave? 87

LECTURE 50 Calculating Our Savings 93

LECTURE 51 Forward Function 116

LECTURE 52 RNN 127

LECTURE 53 Hidden and Sequence Length 129

LECTURE 54 Why Do We Need Fancy RNNs? 131

LECTURE 55 Return of the Neuron 150

LECTURE 56 Simple RNN and Linear Model 163

LECTURE 57 RNN for Images 165

LECTURE 58 Image and Notebook 172

LECTURE 59 Time Series and Stock Prices 173

LECTURE 60 Return Intuition 178

LECTURE 61 Only Appears to Work Well 181

LECTURE 62 Other Ways to Forecast 188

LECTURE 63 Dealing with Text 193

LECTURE 64 Forward Function 211

LECTURE 65 Multiple Words to Single Words (Tokenization) 215

LECTURE 66 Why Bad Programmers Need the Latest Version 226

LECTURE 67 Field Objects 236

LECTURE 68 Data Frame 246

LECTURE 69 Index Dictionary 249
LECTURE 70 1-D Convolution: Example 253
LECTURE 71 Training Corpus and Idx 268
LECTURE 72 Pre Process and Single Sentence 270
LECTURE 73 List of Integers 272
LECTURE 74 Ratings Dataset Must Be Incomplete 273
LECTURE 75 User Movie Rating 289
LECTURE 76 IDs and Num Samples 298
LECTURE 77 Software Engineer and TensorFlow Model 300
LECTURE 78 Movie IDs 303
LECTURE 79 Transfer Learning Intuition 305
LECTURE 80 ResNet 318
LECTURE 81 Large Image Datasets 321
LECTURE 82 Pros and Cons 328
LECTURE 83 Food and Data Augmentation 334
LECTURE 84 Data Augmentation 336
LECTURE 85 See for Yourself 338
LECTURE 86 Model Architecture 357
LECTURE 87 Loss and Optimizers 360
LECTURE 88 Reinforcement Learning 366
LECTURE 89 Large State Spaces 375
LECTURE 90 Policy as a Function or Mapping 402
LECTURE 91 The Markov Assumption 412
LECTURE 92 Future Rewards 425
LECTURE 93 Expressing the Problem as an Equation 430

LECTURE 39

Convolutional Arithmetic

In this lecture we're going to fill in the missing detail from the last lecture. How do we calculate the dimensionality of the initial feature vector into the dense layers of a CNN. For this we need to know about a topic called convolutional arithmetic. Basically these are the concepts that allow us to calculate what the output size of a convolution will be given some input size some filter size and various options such as padding and strides as a reminder.

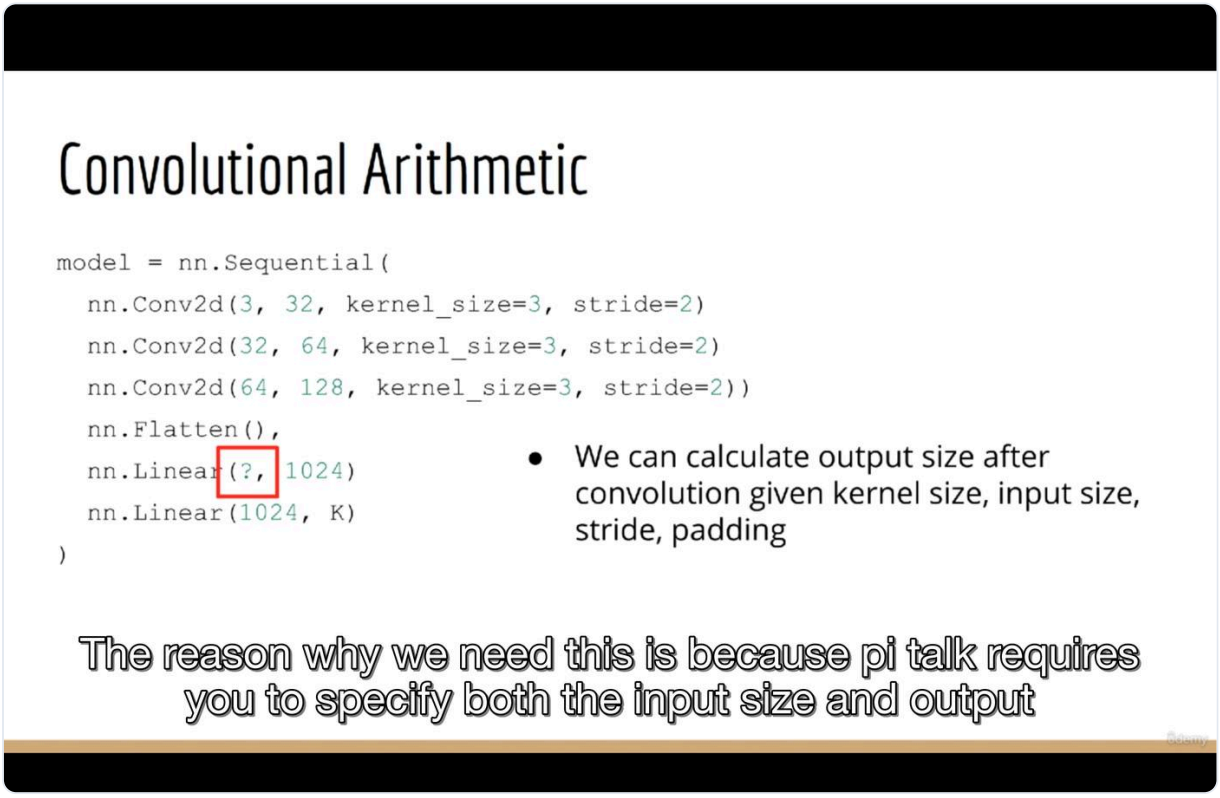


FIGURE 1.1 | Convolutional Arithmetic

The reason why we need this is because PyTorch requires you to specify both the input size and output size of each layer. Even though we implicitly already know this information.

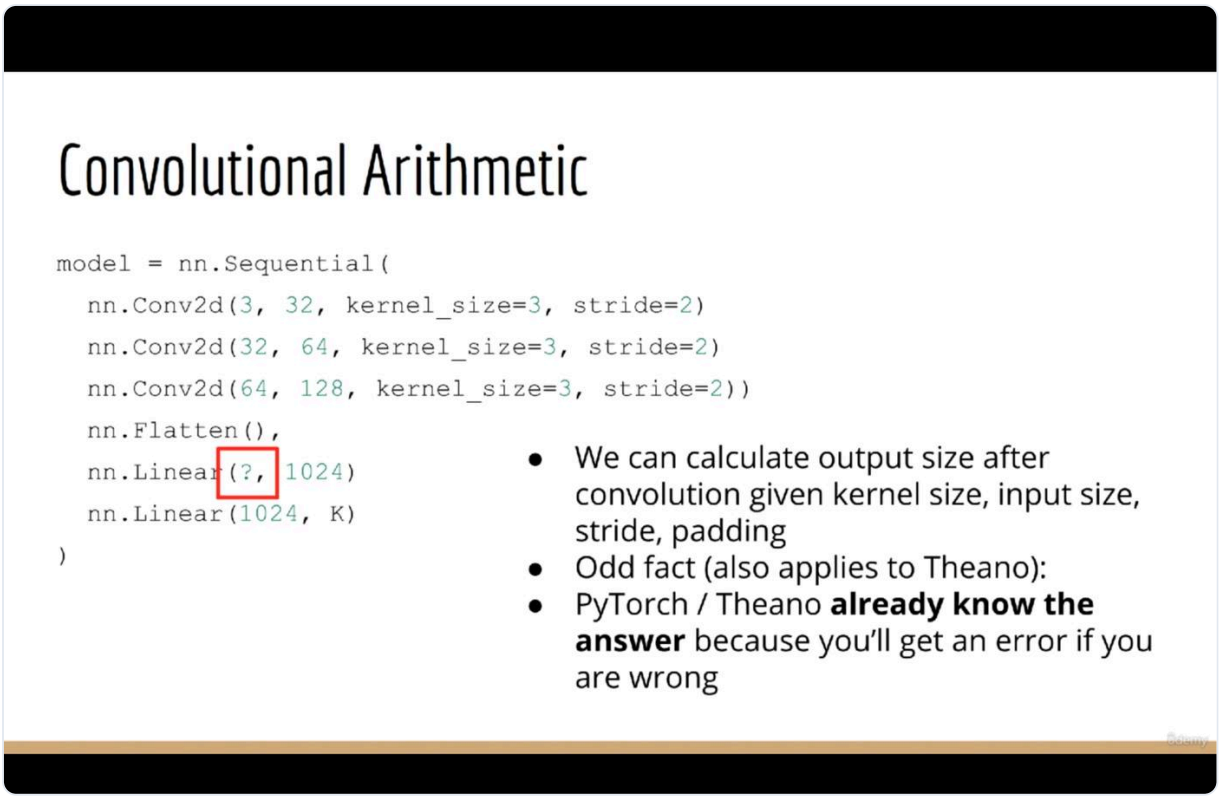


FIGURE 1.2 | Convolutional Arithmetic

What's weird is that these calculations are well-defined. So there isn't really a great reason to require us to do them ourselves. Libraries such as Keras make things super easy by automatically inferring the input size of each layer.

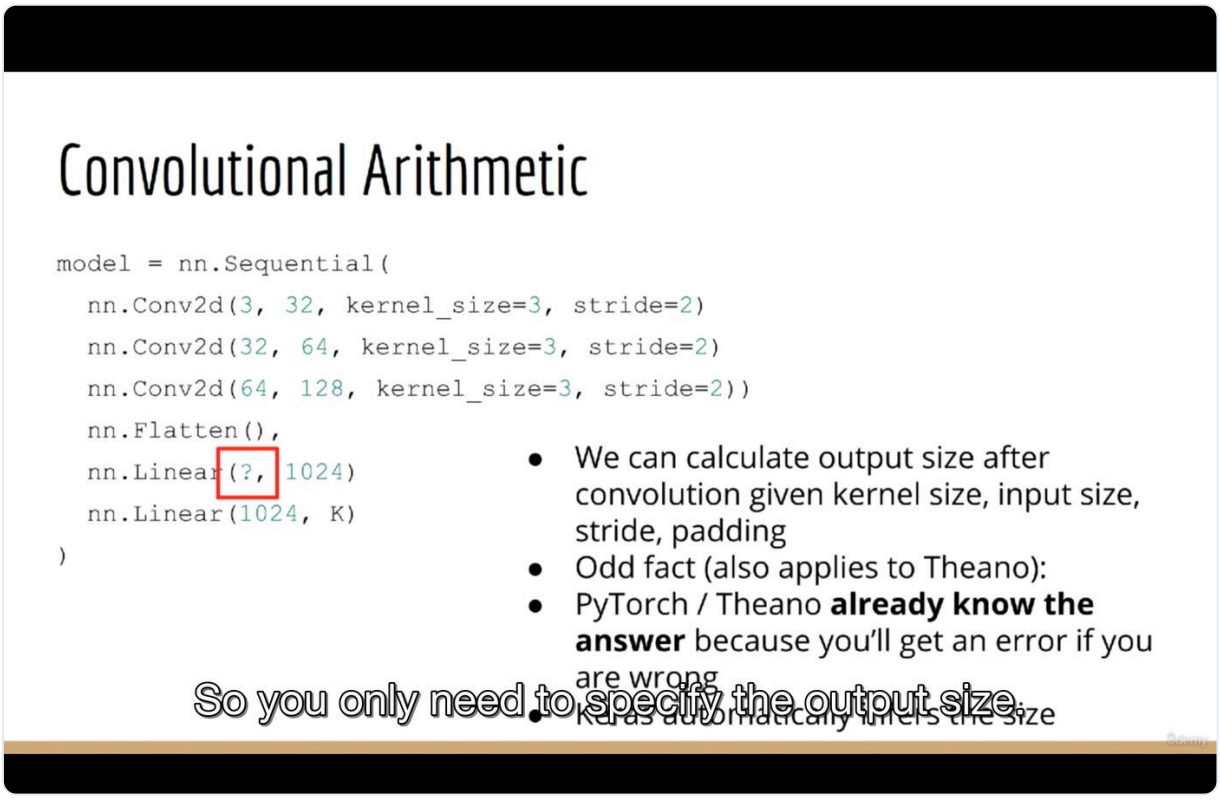


FIGURE 1.3 | Convolutional Arithmetic

So you only need to specify the output size. The reason I mentioned Keras a lot is because Keras is usually a student's first introduction to deep learning.

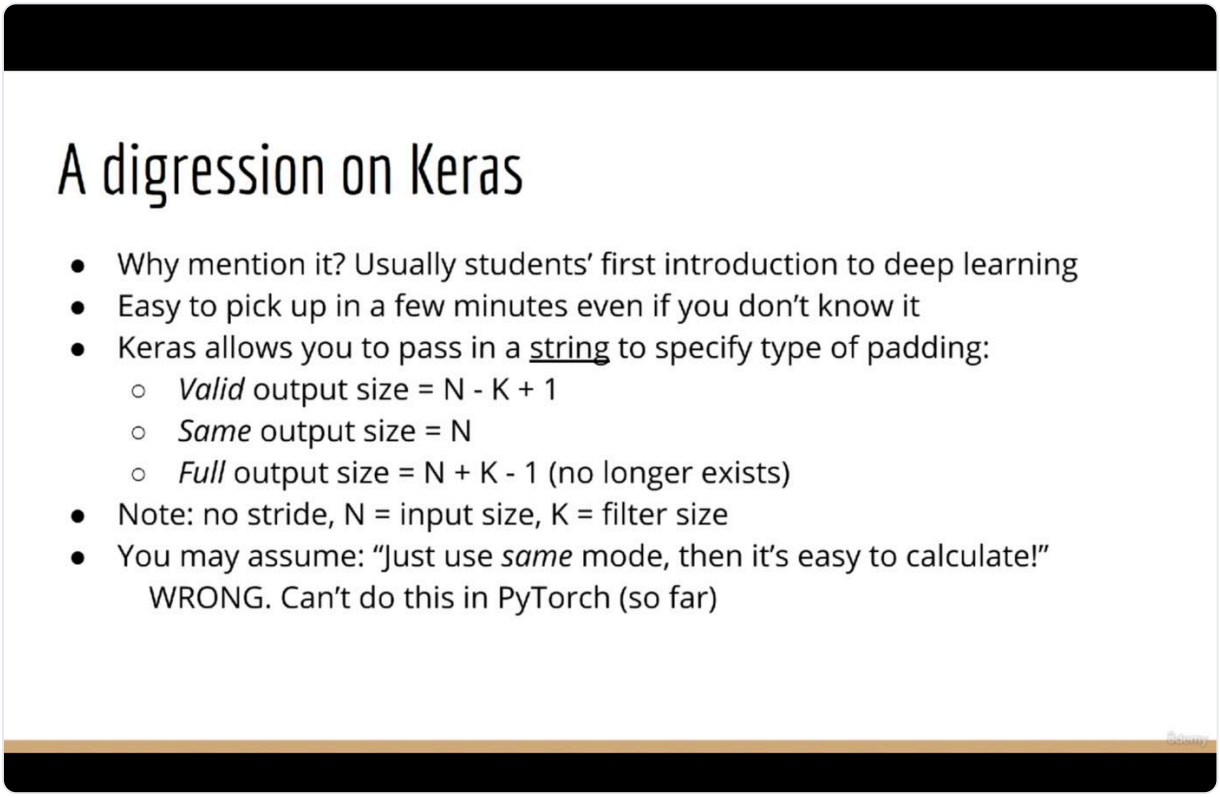


FIGURE 1.4 | A Digression on Keras

And even if you do not know Keras you could easily pick it up in a few minutes provided you have some base level of programming skill so another great thing about Keras is that it allows you to pass in a string to specify the type of padding that is used for convolution as a reminder. We generally have three modes of convolution in valid mode the output size is N minus K plus 1 in the same mode the output size is just N the input size and in full mode which no longer exists by the way the output size is N plus K minus 1. And remember that these are just the basic form of convolution with no strides. So you might think well that's great.

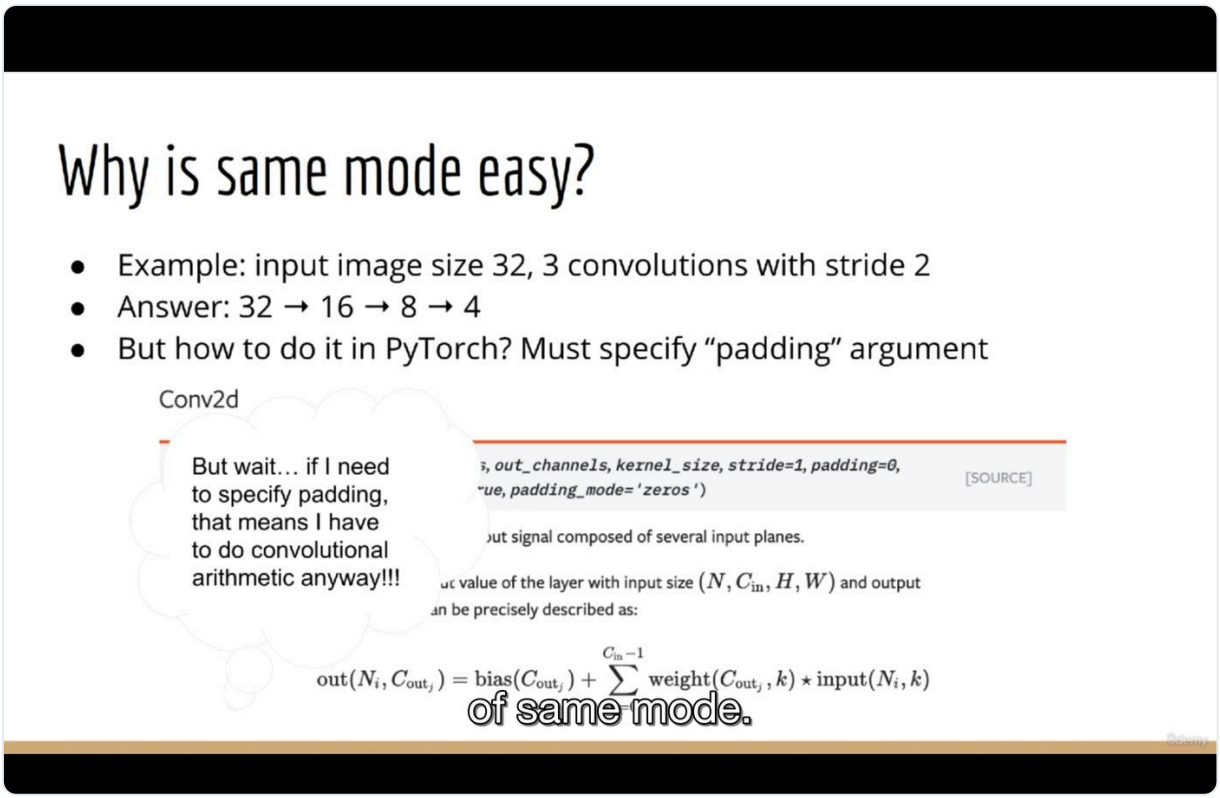


FIGURE 1.5 | Why Is Same Mode Easy?

If I just use same mode for all my convolutions then calculating the output size is super easy but way passing in the mode of convolution as a string exists only in Keras not PyTorch the reason I mentioned this is because it's a natural question to ask why not use the same mode so that it's easy to compute the size of the image after each convolution. If we do that then let's say we have an input image of size 32 that goes through three convolutions with a stride of two after the first convolution the image shrinks down to 16 after the second convolution as shrinks down 8 and after the third convolution it shrinks down to 4 easy. Well let's ask ourselves how would we do the equivalent of same mode in PyTorch.

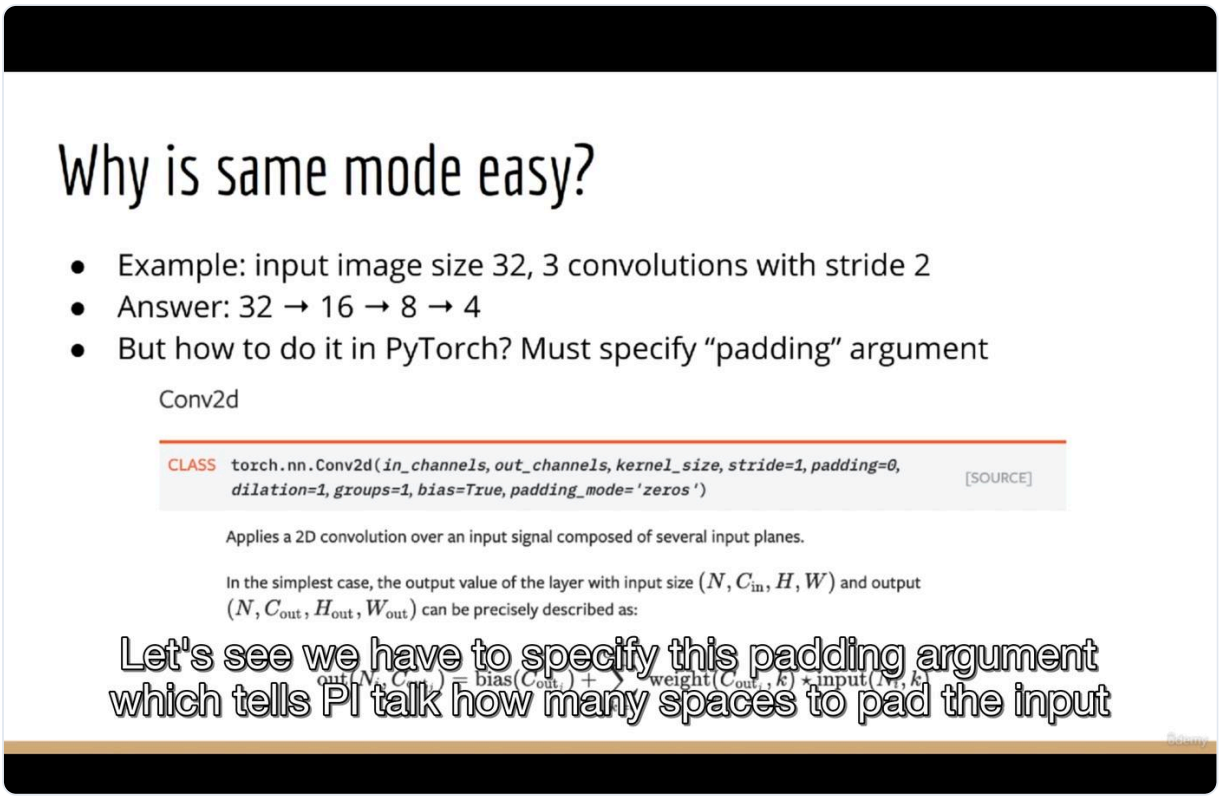


FIGURE 1.6 | Why Is Same Mode Easy?

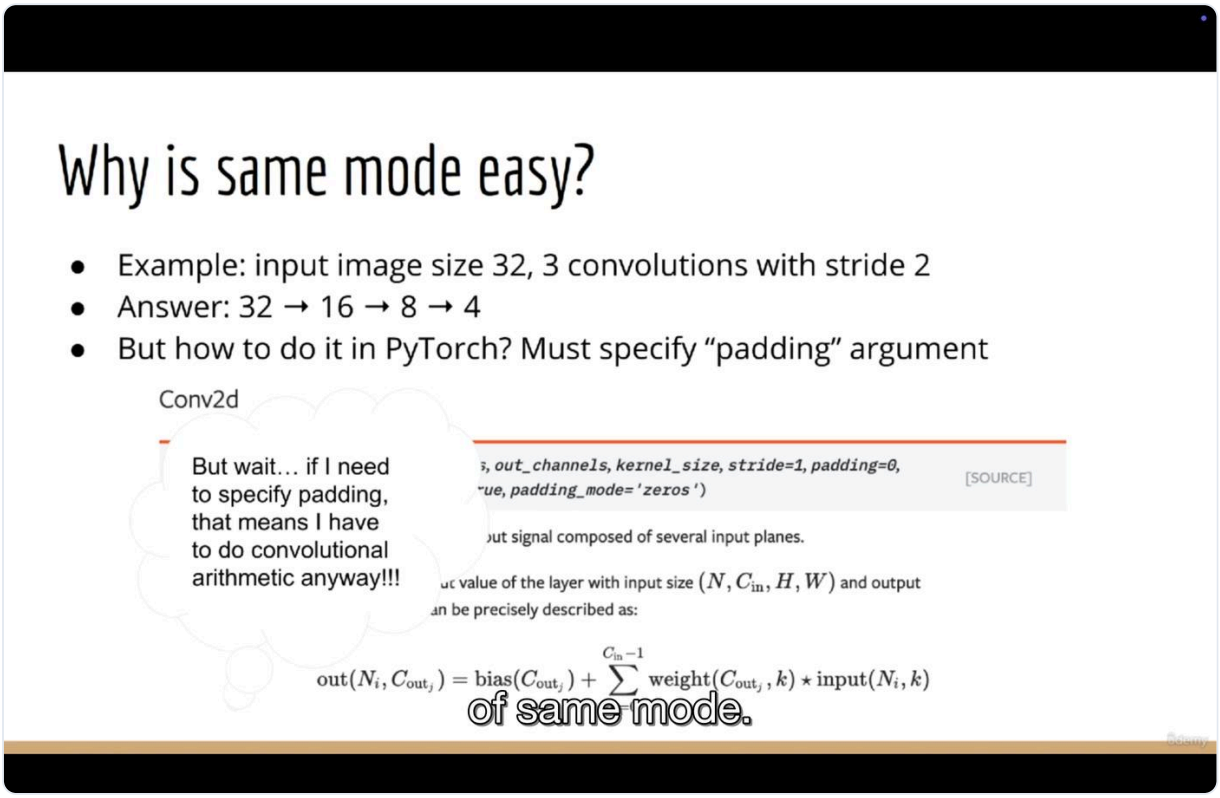
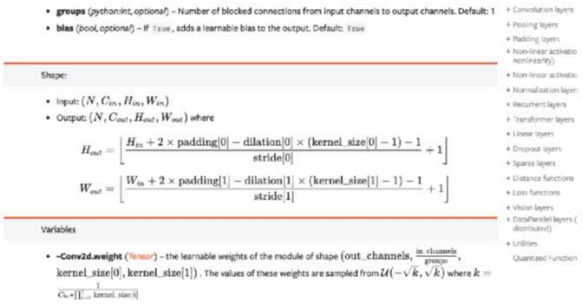


FIGURE 1.7 | Why Is Same Mode Easy?

Conclusion: You must do convolutional arithmetic

- See extra_reading.txt
  - Convolution arithmetic tutorial
  - http://deeplearning.net/software/theano_versions/dev/tutorial/conv_arithmetic.html
- Easy way:
  - Just use the formula from PyTorch documentation



- **groups** (*python:int, optional*) – Number of blocked connections from input channels to output channels. Default: 1
- **bias** (*bool, optional*) – If `True`, adds a learnable bias to the output. Default: `True`

Shape:

- Input: $(N, C_{in}, H_{in}, W_{in})$
- Output: $(N, C_{out}, H_{out}, W_{out})$ where

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

Variables

- **~Conv2d.weight** (*Tensor*) – the learnable weights of the module of shape $(\text{out_channels}, \frac{\text{in_channels}}{\text{groups}}, \text{kernel_size}[0], \text{kernel_size}[1])$. The values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{1}{C_{in} * \prod_{i=0}^{1} \text{kernel_size}[i]}$

FIGURE 1.8 | Convolution Arithmetic Tutorial

Example

- Assume dilation = 1 always (outside scope of this course)
- padding = 1, kernel_size = 3, and stride = 2, H_{in} = 32
- H_{out} = floor[(32 + 2*1 - 1*(3 - 1) - 1) / 2 + 1]
- H_{out} = floor[31 / 2 + 1]
- H_{out} = floor[15.5 + 1]
- H_{out} = floor[16.5]

- Input: $(N, C_{in}, H_{in}, W_{in})$
- Output: $(N, C_{out}, H_{out}, W_{out})$ where

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

point five.

FIGURE 1.9

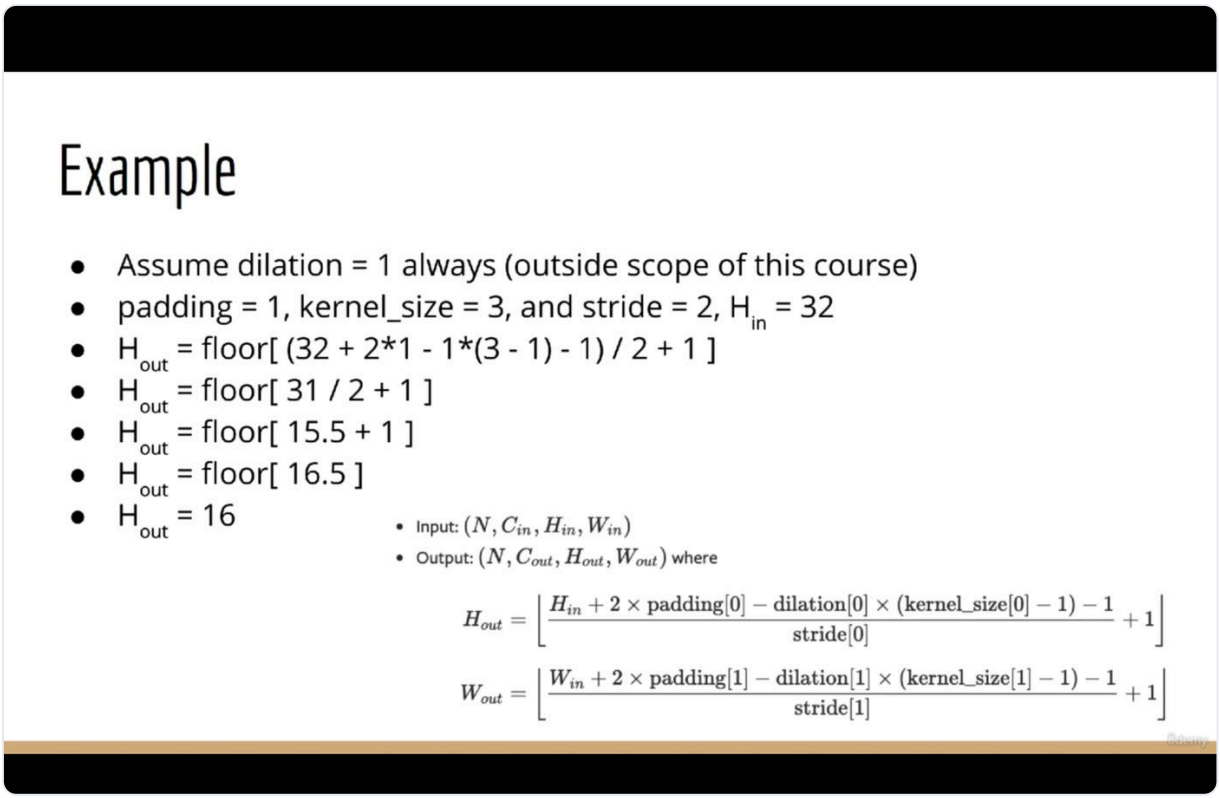


FIGURE 1.10

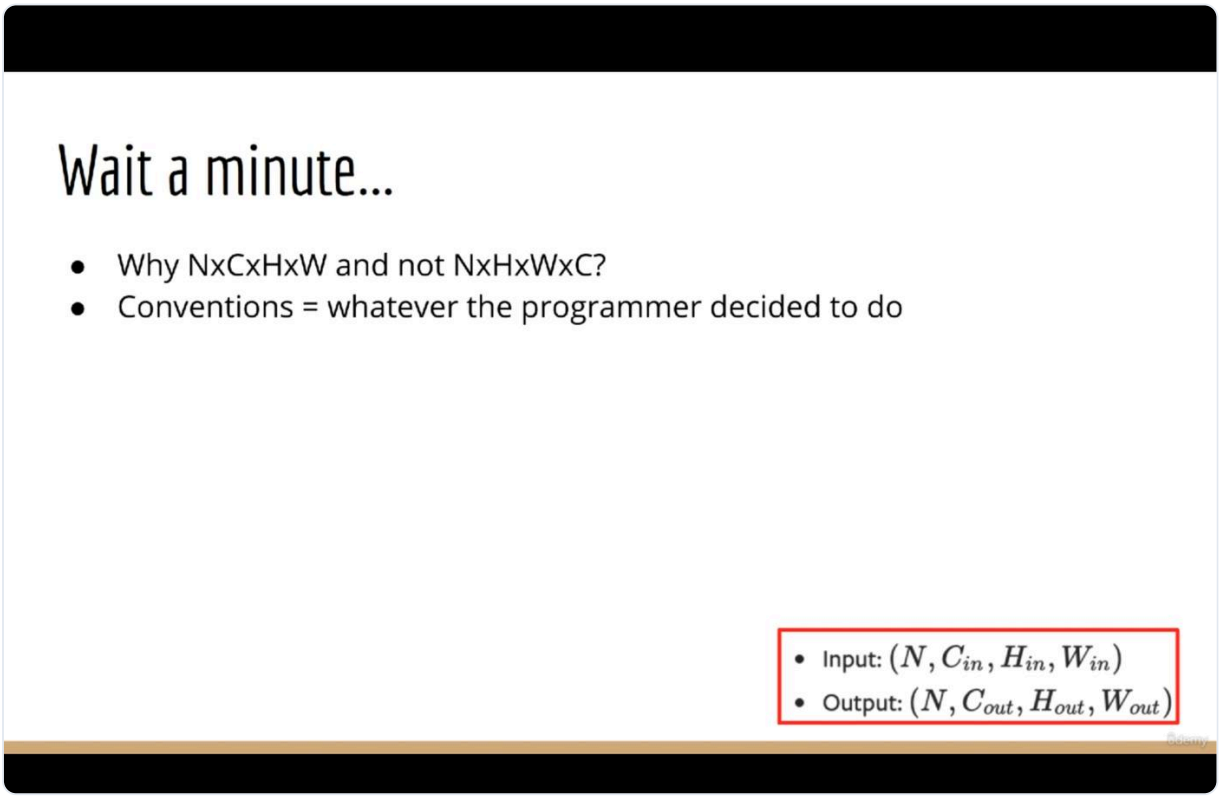


FIGURE 1.11

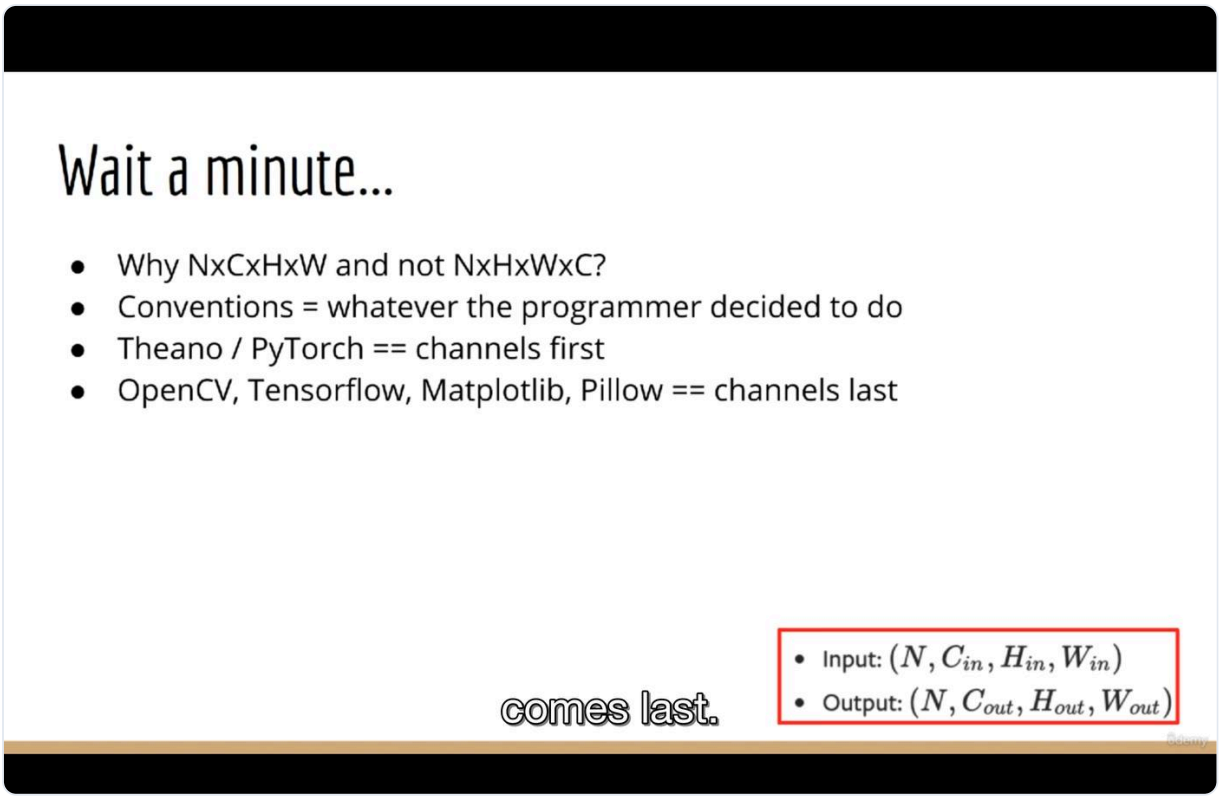


FIGURE 1.12

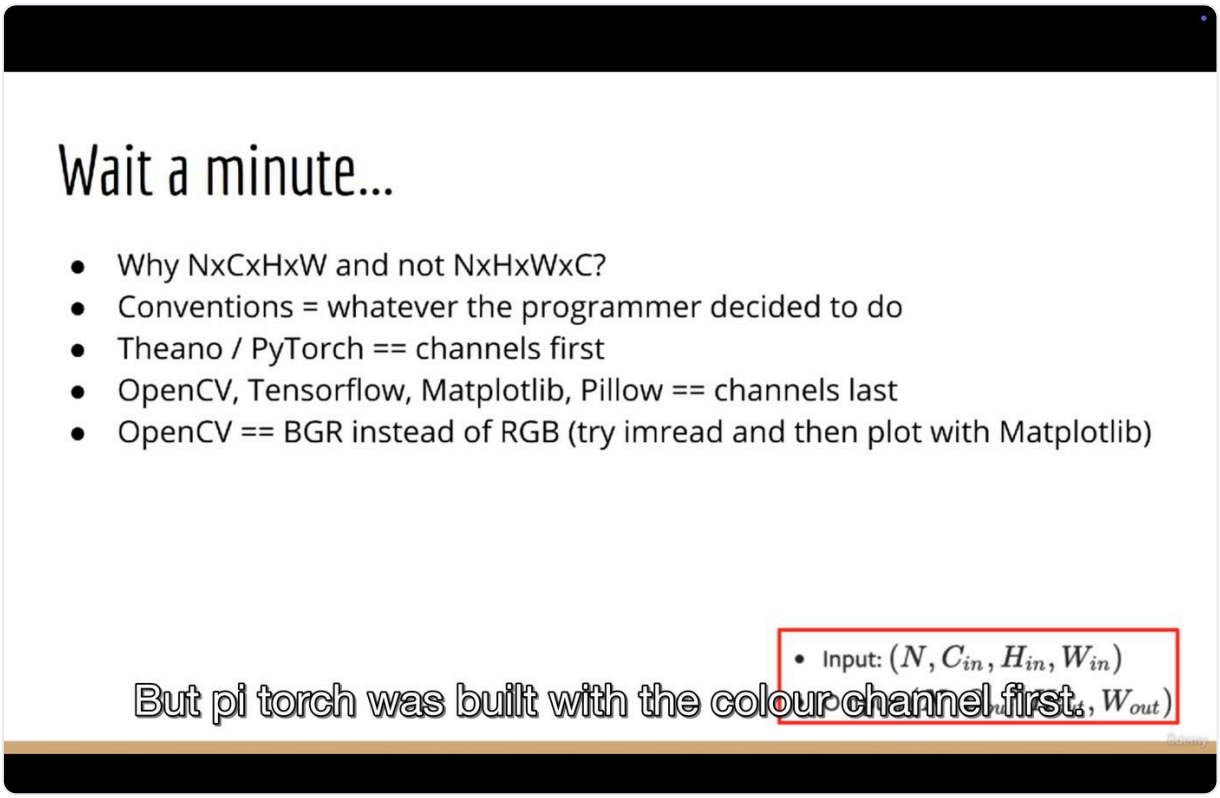


FIGURE 1.13

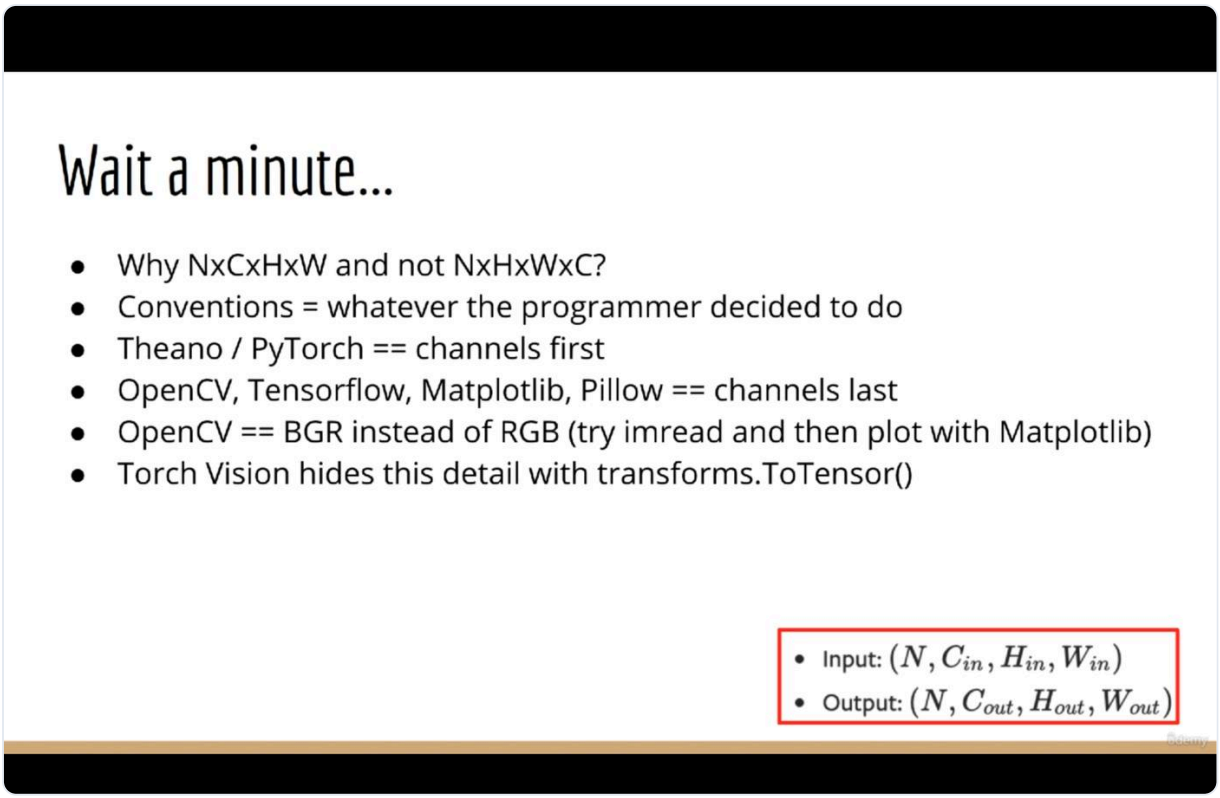


FIGURE 1.14

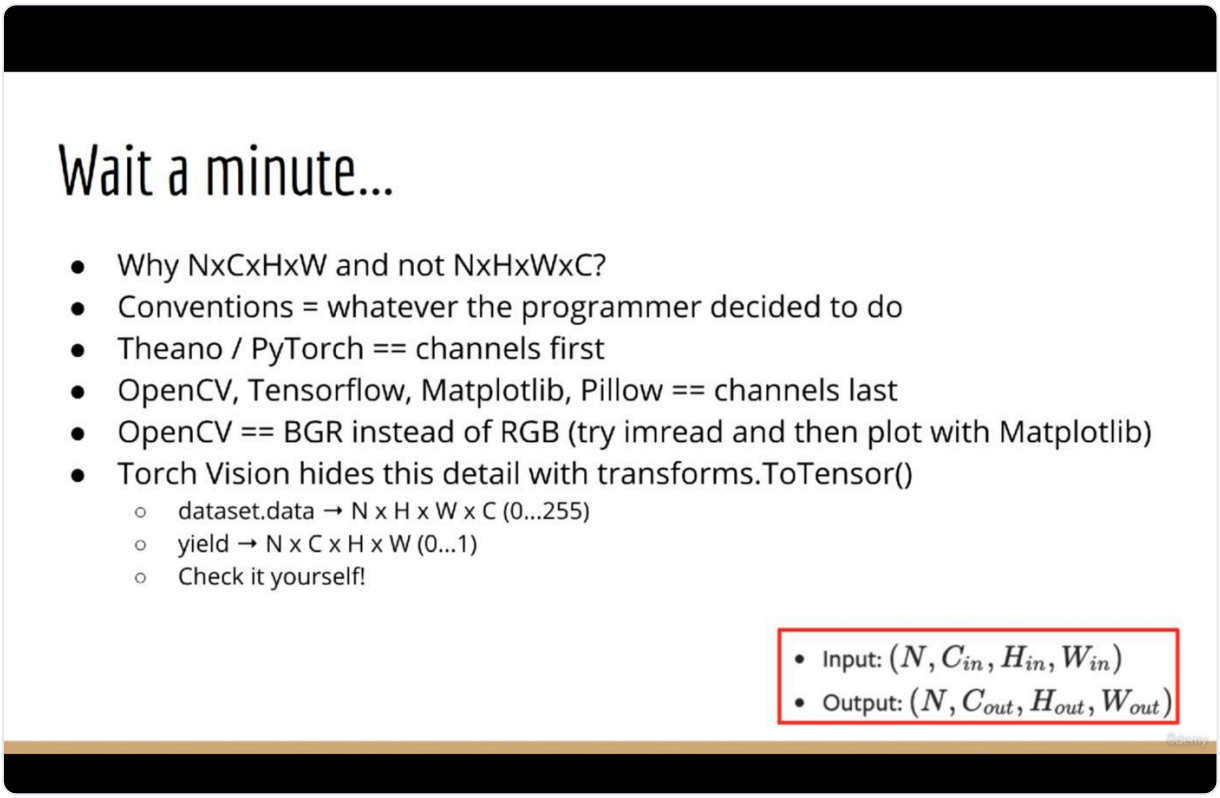


FIGURE 1.15

Let's see we have to specify this padding argument which tells PyTorch how many spaces to pad the input image on each side but way in order to specify this argument correctly so that you achieve the equivalent of same mode. You need to do convolutional arithmetic to calculate the correct amount of padding. In other words you still need to do convolutional arithmetic whether you have same mode or not. So there's no benefit to using same mode because either way you still have to do convolutional

arithmetic. In fact this might be a little harder because you have to do convolution arithmetic in reverse. The only good reason to use same mode would be if you really believe it benefits your model's predictions.

OK so now you're convinced that there is no way out of doing convolutional arithmetic and this course is not my goal to go in-depth about this topic. If you want to gain a better intuition about convolutional arithmetic I've left a link to deep learning dot net in the file extra reading DST. Which leads you to a tutorial about convolutional arithmetic and how the calculations change when you have padding strides and so forth. It's pretty non trivial however an easy way to get around this is to simply use the formula that PyTorch provides in their documentation. With this formula you can plug in all your values and calculate the output dimensions. You don't need to understand convolutional arithmetic to use this formula.

You just need to know how to multiply add and divide. Let's do a simple example to make sure you know how to use this formula since the formulas for both the height and width dimensions are the same. We only need to do one firstly the concept of dilation is outside the scope of this course and we will use the default value of one next. Let's assume we have padding equals one kernel size equals three and stride equals two. Let us set H in to 32 which is common for deep learning image datasets. If we plug these values into our formula we get thirty two plus two times one minus one times three minus one minus one all divided by two then we plus one and then take the floor.

So that gives us thirty one over two plus one which is fifteen point five plus one which is sixteen point five. Then we take the floor of sixteen point five and we get sixteen so H out equals sixteen. Now it's easy to make mistakes with these and I wouldn't be surprised if I did. So if you want to double check my work please do alternatively you could just implement this in code you may have noticed something a little curious on the previous slide which is that in the PyTorch documentation the input size and the output size are specified as N by color by height by width. This is contrary to what we've been working with so far which is data sets of size N by height by width by color.

Why is there this discrepancy. Well it has to do with the conventions of the library. Remember that when you're a programmer building a library you have full control over how it works. So you get to decide what the conventions are how you want to structure the data and so forth. It turns out that some developers decided to put the color channel first while other developers decided to put the color channel last. In our case libraries such as Theano and PyTorch decided to put the color channel first. Libraries like OpenCV TensorFlow matplotlib and Pillow use the convention that the colour Channel comes last. Although OpenCV does other weird things like using the BGR colour ordering instead of RGB so it seems that putting the colour channel last has now become more common.

But PyTorch was built with the colour channel first. We can't just switch it around one day and expect everyone to update their code. Therefore PyTorch still uses the convention that the colour channel comes first. This detail is hidden from you. If you use utilities such as those from the torchvision library since we haven't worked with colour images yet in this course we haven't had the opportunity to uncover these hidden details but now we do. As you recall if you inspect the data attribute of a dataset loaded in from torchvision it gives you the untransformed version of the data. These will be N by height by width by colour but when you yield batches of data from the data loader this data is transformed in whatever way is needed before it goes into the neural network.

And so when you inspect these batches of data yielded by the data loader you will see that they appear as N by colour by height by width so in fact PyTorch still uses the convention that the colour channel comes first. But it comes with interfaces that work with the more commonly used convention that the

colour channel comes last.

LECTURE 40

Loading in the Data

At this point we finally understand how to fully build a convolutional neural network in python. Now we're ready to outline the upcoming code as usual. I'm going to repeat the steps of our analysis just to be sure. Step number one is to load in the data for the following examples.

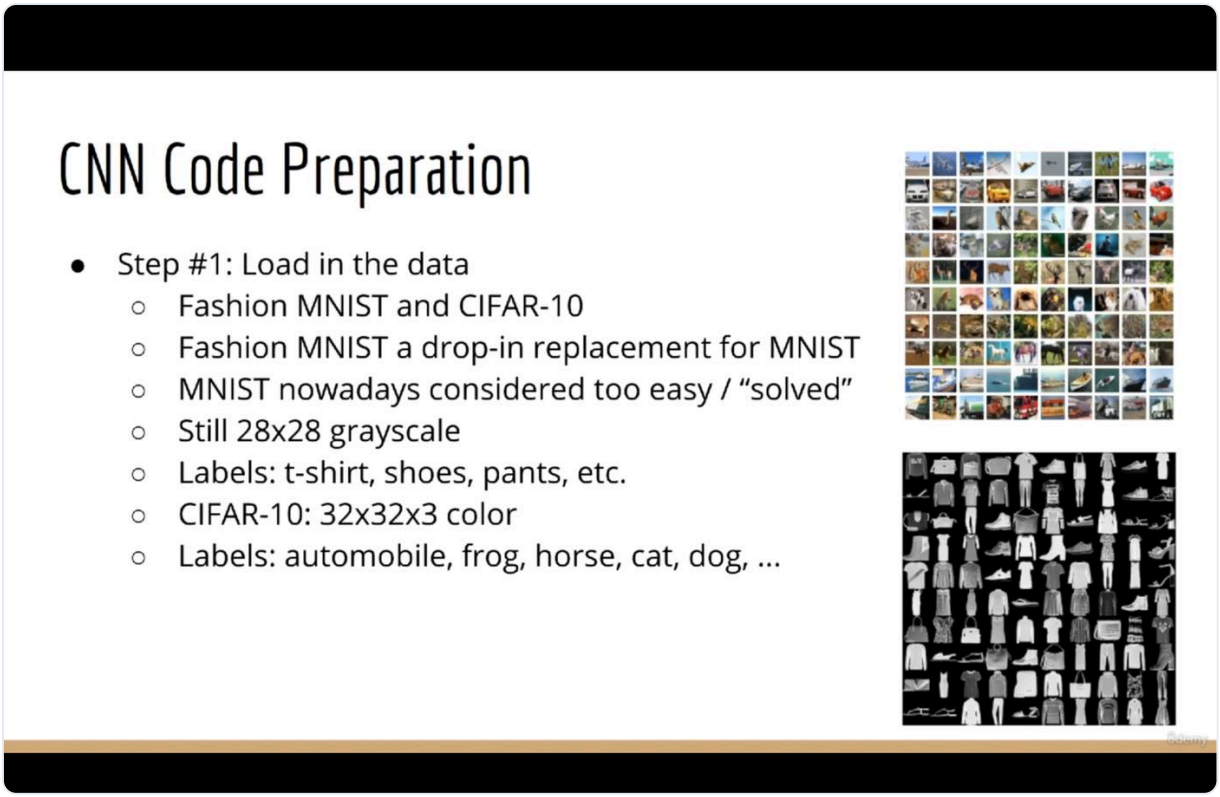


FIGURE 2.1

We'll be using more difficult datasets in particular Fashion MNIST and CIFAR-10 Fashion MNIST was designed to be a drop in replacement for the MNIST data set which today is considered to be too easy for modern machine learning algorithms one might consider it to be a solved problem. Thus Fashion MNIST was born and it was designed to be a standard benchmark just like MNIST but with the goalpost further ahead just like MNIST it is a data set of 28 by 28 grayscale images and it even has the same number of samples. The difference is that instead of simply handwritten digits we have images of different types of clothing. For example t shirts shoes pants and so forth.

Your job is to classify these images into the correct category. The CIFAR-10 data set is older but more difficult. This dataset contains color images so the data takes up more ram and the images are also a bit bigger 32 by 32.

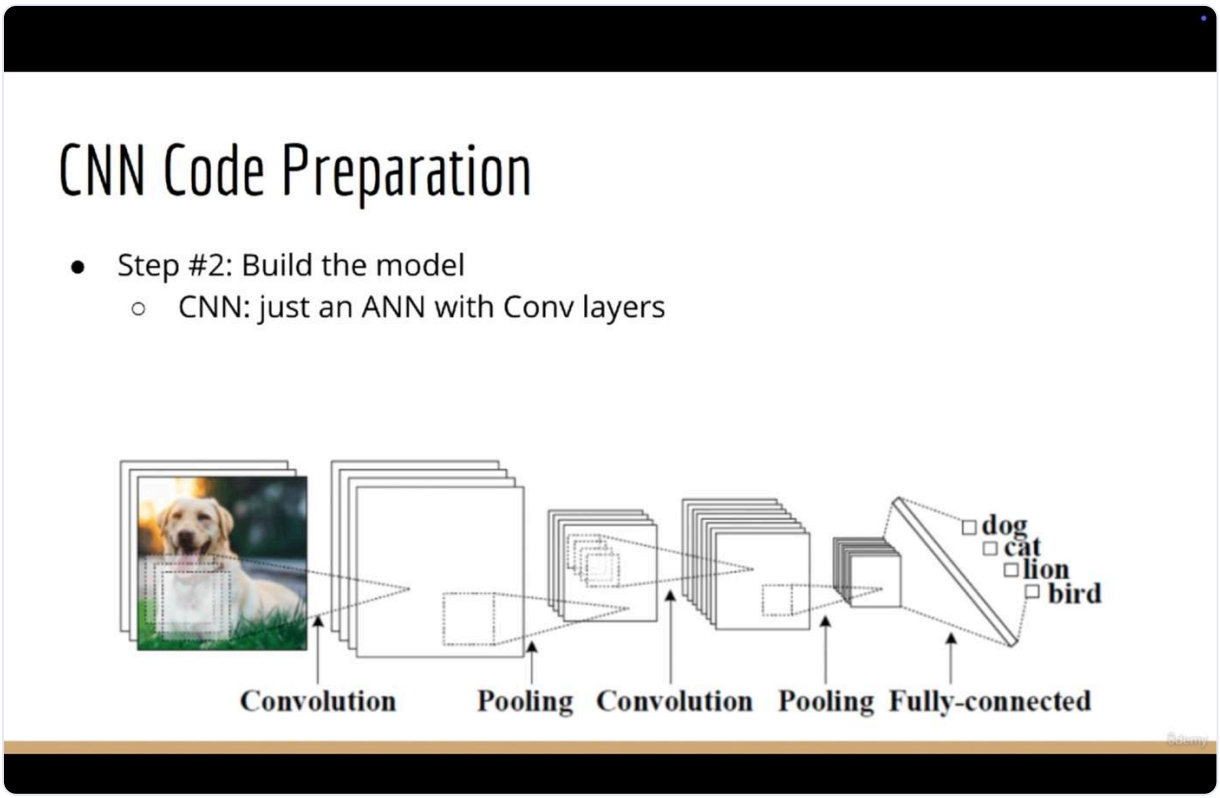


FIGURE 2.2

It contains classes such as automobile frog horse cat and dog. You can already imagine that with such tiny images it might be very easy to mistake a cat for a dog or a dog for a horse step number two is to build a model. Luckily we just discussed that in painstaking detail over the past few lectures. In case you forgot we'll be using convolutional neural networks step number three is to train the model. Step number four is to evaluate the model and step number five is to use the model to make predictions just like the ANN section. Nothing here changes.

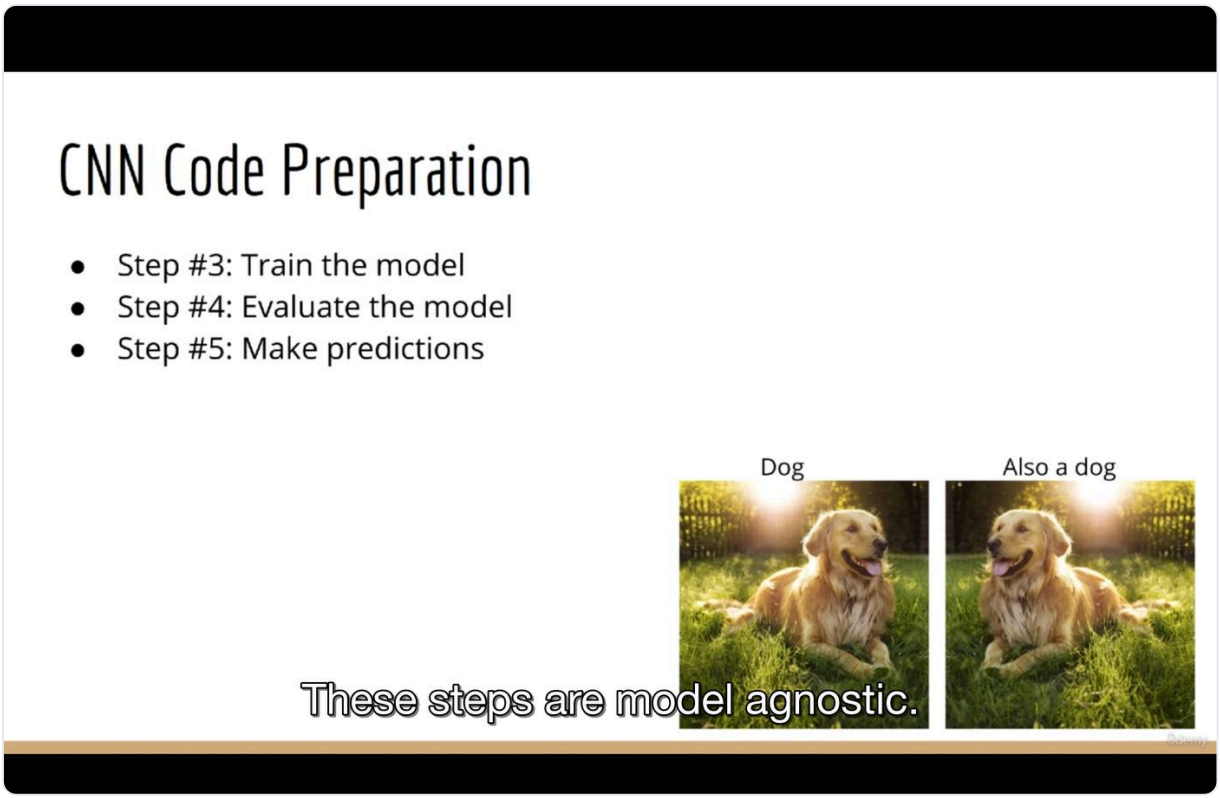


FIGURE 2.3

These steps are model agnostic. As per my rule all machine learning interfaces are the same one new thing you'll learn about later on in this section is what to do if your data set is too large to fit into memory or if you want to augment your data. For example we know that a dog facing left is a dog but a dog facing right is also a dog. So should our CNN not be able to recognize both of these orientations as a dog. The answer is yes. We also know that deep learning loves data. The more data the better.

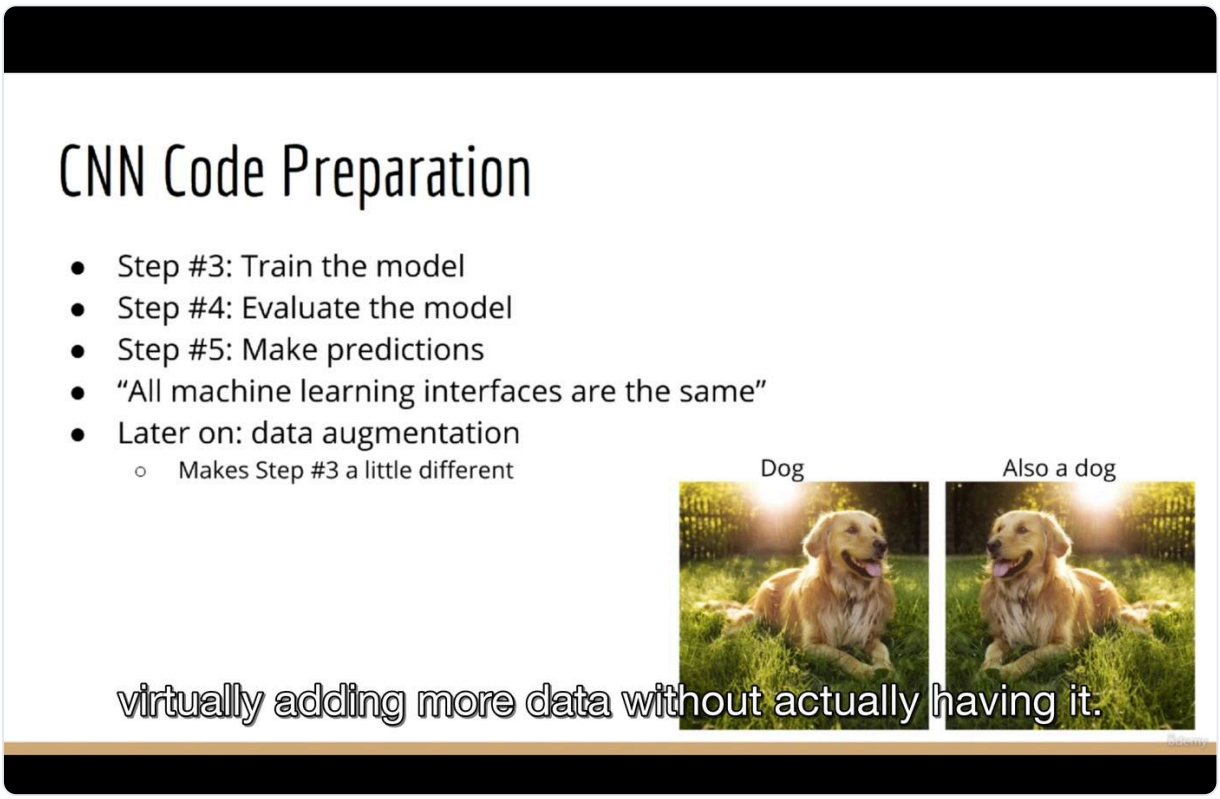


FIGURE 2.4

This is why deep learning has come to prominence in recent years so data augmentation is a method of virtually adding more data without actually having it. We'll discuss this more later on. But for our initial example we're going to keep things simple so let's elaborate on step number one loading in the data.

FIGURE 2.5 | Loading in the Data

Luckily both the data sets we'll be using for these examples are also included in torchvision. It's generally going to follow the same pattern as before you can even probably try to guess the function names and you might be right for Fashion MNIST we load in the data by calling torchvision dot datasets dot FashionMNIST for CIFAR-10 and we load in the data by calling torchvision datasets dot CIFAR10. Nothing too surprising there.

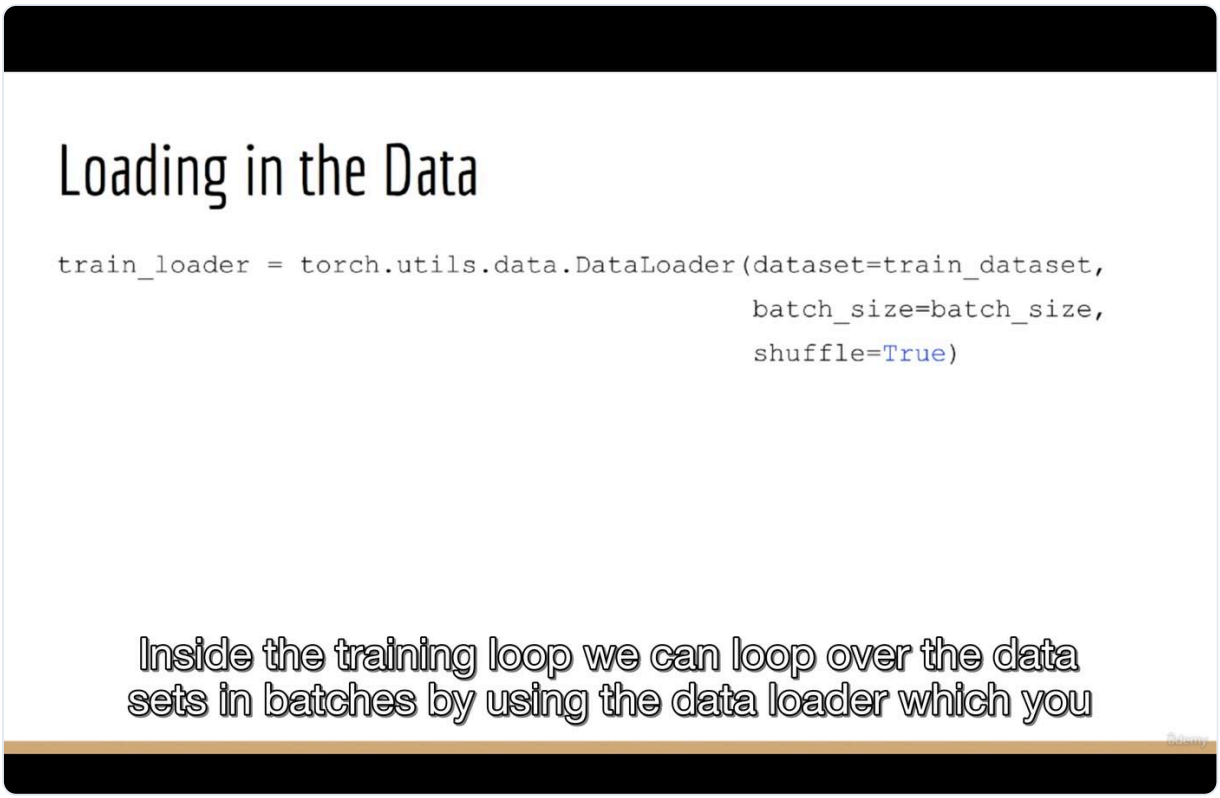


FIGURE 2.6

Inside the training loop we can loop over the data sets in batches by using the data loader which you guessed it looks exactly the same as before. So what changes. Other than changing the model and changing the data functions. Nothing really. So this is a great time to remind you of my two rules.

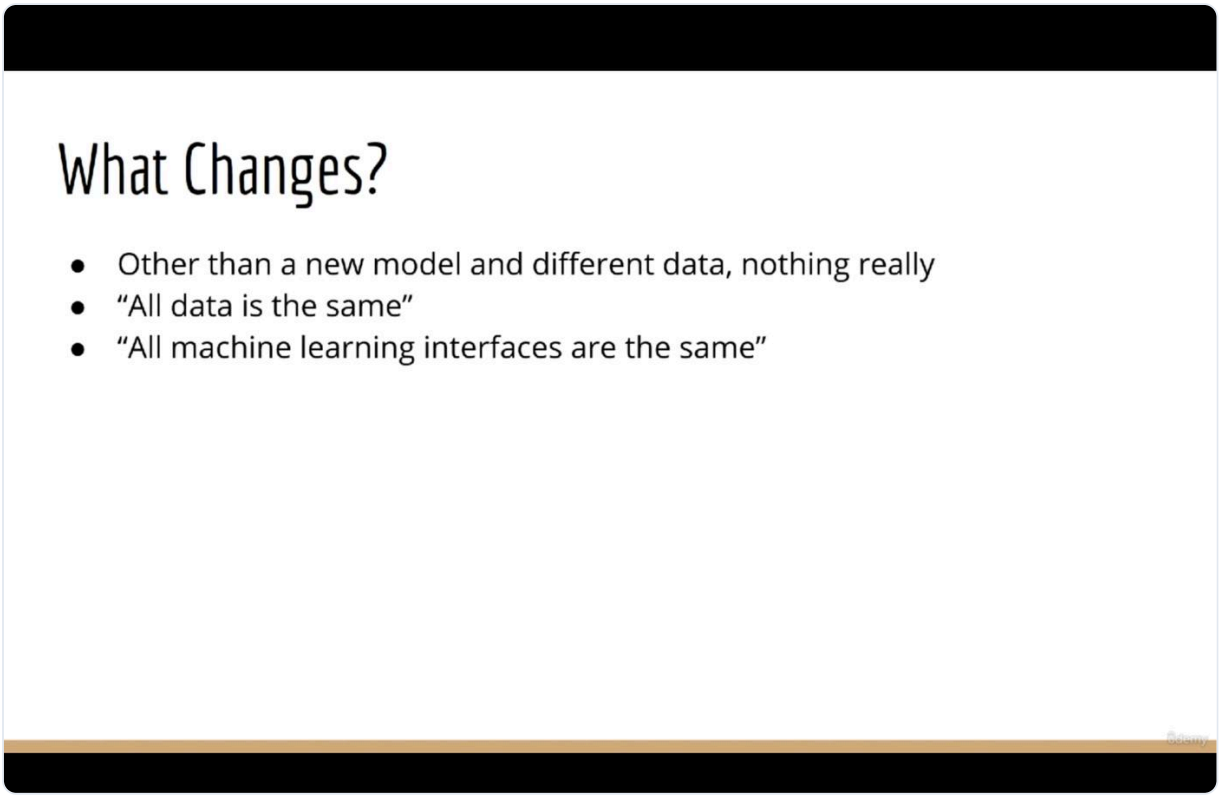


FIGURE 2.7 | What Changes?

All data is the same and all machine learning interfaces are the same so just as a quiz. Let's think about which of these steps will be the same and which will be different.

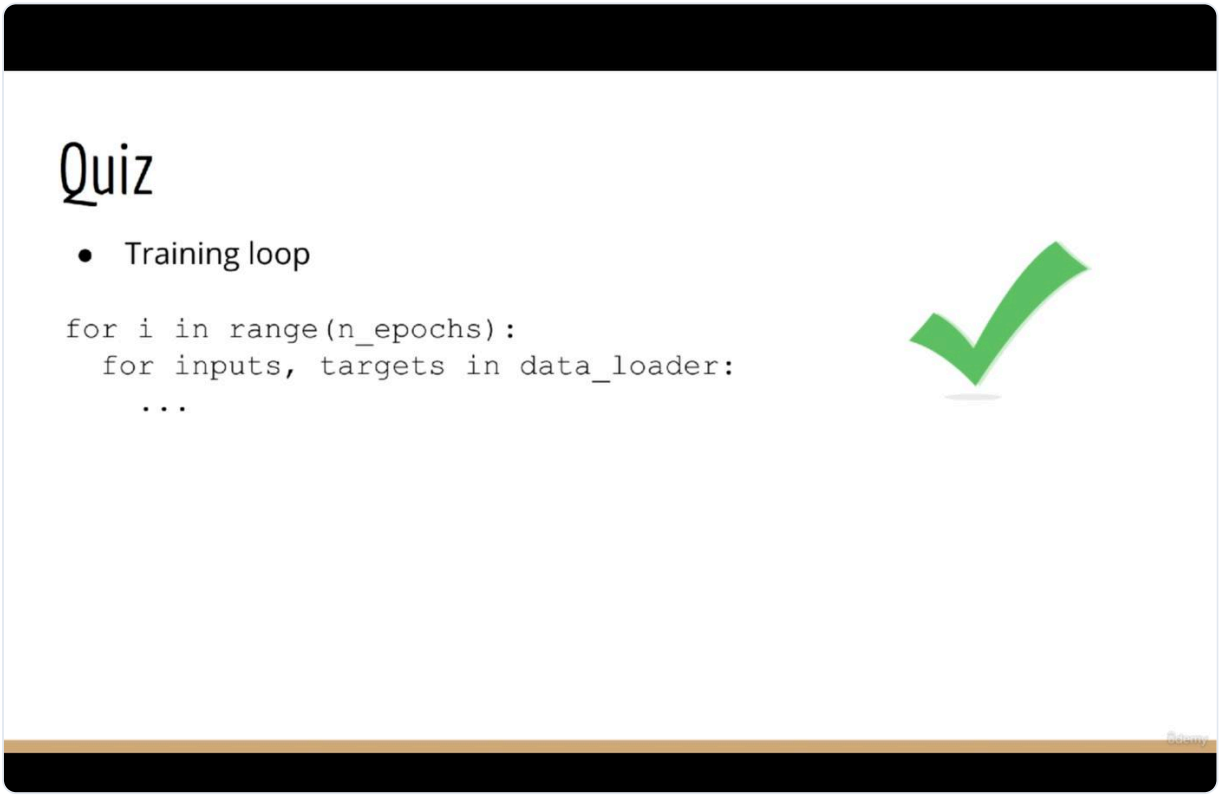


FIGURE 2.8

First the training loop as you know we'll be doing Batch gradient descent. Will this be the same or different.

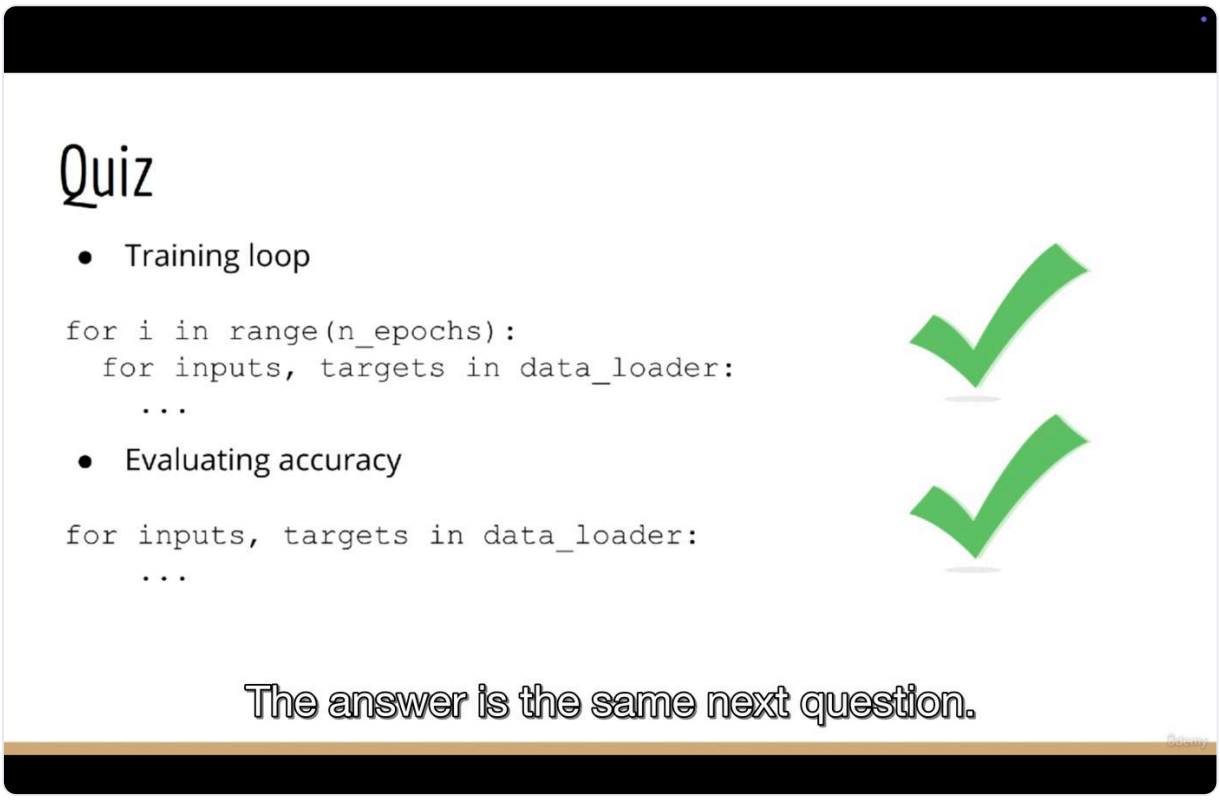


FIGURE 2.9

FIGURE 2.10

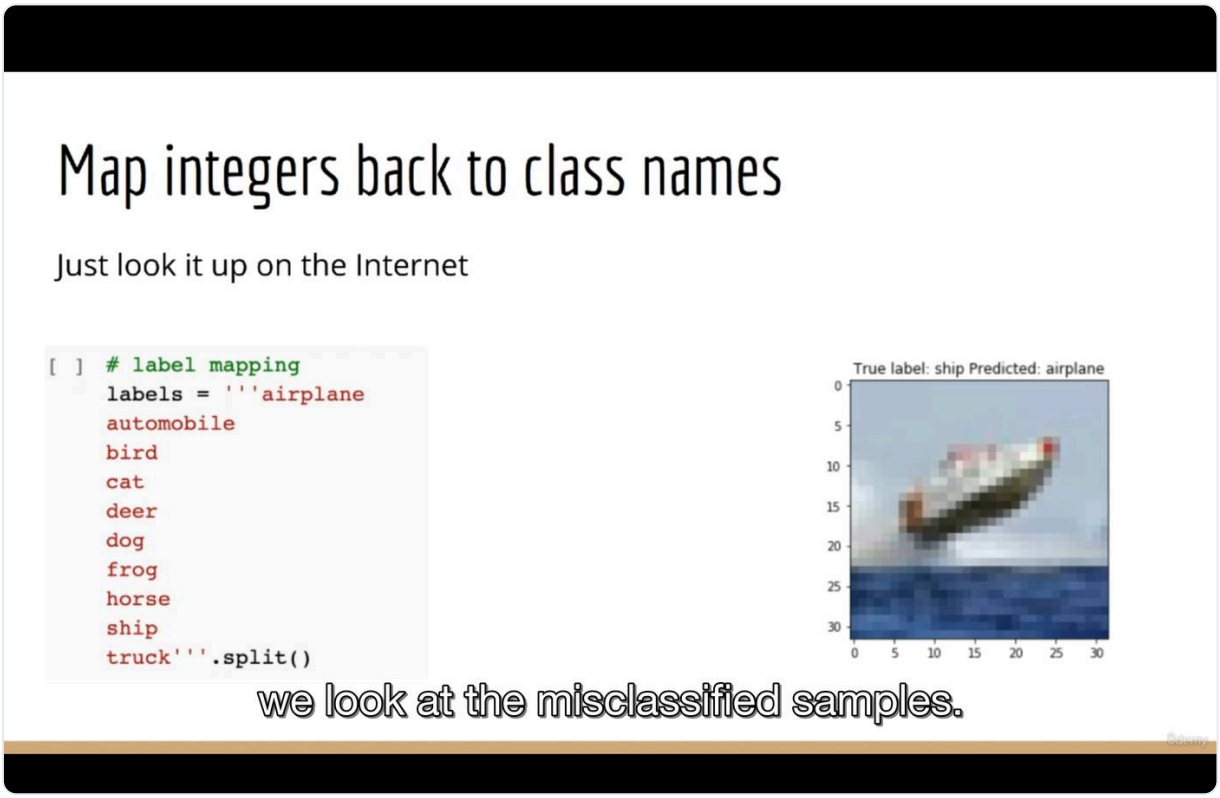


FIGURE 2.11 | Just Look It Up on the Internet

The answer is the same next question. Let's consider evaluating the accuracy of the model. Will this be the same or different. The answer is the same next question. Let's consider plotting the confusion matrix given the model predictions and model targets. Will this be the same or different. The answer is the same the only minor non-trivial thing I want to highlight is that the classes are going to be encoded as integers from 0 up to 9 inclusive. But how do we know which integer corresponds to which class.

For example what if zero is dress and one is t shirt. Is that obvious. Of course not. I just did the easy thing and look them up on the Internet and hardcoded them as strings this way when we look at the misclassified samples.

It won't say something like 0 is misclassified as one which doesn't really mean anything but rather t shirt is misclassified as dress which is more interpretable.

LECTURE 41

Fashion MNIST

See the notebook `6.PyTorch-CNN-Fashion-MNIST.ipynb` for the code.

In this lecture, we are going to look at a CoLab notebook that does image classification using PyTorch with a convolutional neural network on the Fashion MNIST data set. This lecture is going to walk you through a prepared CoLab notebook, although a very good exercise, which I always recommend is once you know how this is done, to try and recreate it yourself with as few references as possible. As usual, you can look at the title of the notebook to determine what notebook we are currently looking at. All right, so let's start by recognizing everything that should seem familiar. First, we have our imports. There should be nothing unexpected here. Notice how we're also importing the datetime module.

This is useful since as your data grows and your models grow, training is going to take longer and longer. When training starts to take a non-trivial amount of time, we naturally start to ask, so how long is this taking? And so we can use the datetime module to get a sense of how long each iteration of the training loop takes. In more high level libraries like Keras, this all happens for you, but in PyTorch, if you want these features, you have to build them yourself. Next, we load in the data, which is, as we mentioned, the Fashion MNIST data set, since it is included in the torchvision library, nothing has to change in the way we load in this data.

Next, we're going to check a few of the data sets attributes to see how it compares to. Remember, it's supposed to be a drop in replacement. As you can see, the maximum value is 255, which is what we expect. The shape of the data is sixty thousand by twenty eight by twenty eight, which is what we expect. And the targets contain integers that look like they're between zero and nine inclusive, which is also what we expect. Next, we load in the test data set and then we get the number of classes by casting all the targets into a set and then taking the length of that set. As expected, the data set contains 10 classes. The next step is to define our CNN model.

As promised, we're going to start looking at how to build models using inheritance rather than stacking layers together in a sequential. On the first line, you can see that the CNN class inherits from the nn.Module class. This makes our class an official PyTorch module, meaning it has all the useful functions and attributes that a built in PyTorch module has, even doing automatic differentiation inside the constructor. The first thing we do is call the parent classes constructor. Next, we create our layers. The first set of layers is a series of convolution layers, followed by the ReLU activation function. We can wrap this in a sequential to make it easier to call in the future.

Notice that the first convolution layer has only one input channel. That's because, as you know, the MNIST and Fashion MNIST data sets are greyscale. As you recall, we need to do some convolutional arithmetic in order to calculate the arguments into the first linear layer. So I've left some useful links here on this topic. If you want to check them out, I would recommend you do the calculation yourself to make sure that what I've written below is correct. Next, we have a series of dense layers, along with some dropouts and a ReLU activation. Again, we wrap this in a sequential so it's easier to call later. Next, we have the forward function. This is where we actually pass a given input through the CNN to get the neural networks output.

As you can see, it's made up of just three lines of code thanks to our sequential objects. First, we pass our input into the convolution sequential. Next, we reshape those outputs in order to flatten them using the view function. Notice how there's no need to specify both dimensions of the flattened output. However, it may be easier to specify just the first dimension and not the second one. The first dimension simply refers to the size which we can obtain by using out of zero. This is because if we have N samples in, we also must have N samples out. Clearly, the number of samples we pass into the input of a neural network must also be the number of predictions that we get out.

The minus one means that PyTorch will automatically calculate the leftover dimension, given whatever dimensions are left. Next, the third step is to pass the data through the dense layers and finally we return the output. The next step is to instantiate the model we pass in one argument into the constructor, which is the number of classes K. In the next block of code, I show you how to build an equivalent CNN using the sequential module instead of building a custom class, as you can see, it's much simpler, but still requires you to do convolutional arithmetic to determine the first linear layer's input size. You might want to try both models and confirm that they are equivalent.

The next step is to move our model to the GPU, which you already know how to do. The next step is to create our loss and optimizer, which you already know how to do. The next step is to create our data loaders, which, again, you already know how to do. The next step is to create a training function, which, again, you already know how to do. The only minor difference here is that I have some timing code and when each epoch completes, I print the duration of that epoch. So you can see here I'm printing out the epoch duration. But now you see how important it is to understand what we were doing previously. We'll be seeing that exact same code again and again.

All right, so training completed successfully at around six seconds per epoch. Next, we plot the loss per iteration, as usual. So this appears to look at. Next, we get the train and test accuracy using the same code we saw before. As you can see, we do pretty well, about 94 percent on the train set and about 89 percent on the test set, but this is definitely less than the accuracy we got on the original MNIST data set. Next, we have our confusion matrix code. So if we scroll down, we can see what our model is most confused by. Unfortunately, the axes are labeled with integers, so it's hard to tell what's what, but we can see if we look at the labels below what each of them means.

So it seems we often confuse zero with six and six with zero, zero corresponds to t shirt and top, while six corresponds to shirt. So it definitely makes sense that our model would confuse these. In fact, one might argue that they are the same thing and that even humans would confuse these. That's similar to what we saw with MNIST. We can also see that six gets confused with four a lot. As we saw six means shirt, and now we can see that four means coat. This also makes sense given the very low resolution of the images, basically the little white blobs on a black background. We would expect that a coat has a similar shape to a shirt. We also see that two gets confused with four and six as well, pretty often two stands for pullover, which again is pretty much a shirt, especially when it's just a 28 by 28 blob.

On the other end of this, we see that seven often gets confused with nine and vice versa. Seven corresponds to sneaker and nine corresponds to ankle boot. So, again, considering that the images are just little blobs, this makes total sense. Let's now look at some misclassified samples to check if our reasoning holds. So the first one we see is a T-shirt top getting predicted as a shirt, which makes sense. Here's a shirt predicted as a pullover that makes sense. Here is a shirt predicted as a coat, I wouldn't really consider that to be a coat. Use a T-shirt predicted as a shirt, that makes sense. It's a pullover, I think we saw this one. Here's a sandal predicted as an ankle boot that makes sense.

Here is a coat predicted as a pullover. That makes sense. So you can see that all the things that are getting misclassified are generally the same. OK, so as a final exercise for this lecture, what you should try is mix and match the models and data that we've seen so far. Previously, we used an ANN and now we are using a CNN on Fashion MNIST. You should have some intuition that a CNN should perform better than an ANN, but also that Fashion MNIST is a harder classification problem than MNIST. So what will happen if you try to use an ANN on Fashion MNIST? What kind of results do you expect to see what will happen if you try to use a CNN on MNIST? What kind of results do you expect to see?

As always, remember my rule machine learning is experimentation and not philosophy. Whenever you want to know the outcome of running some computer code, never try to guess the outcome with your mind. Well, in this case, it's OK because I've asked you to in order to check whether your intuition is correct. But what you really want to do is write these scripts in code and see the answer for yourself.

LECTURE 42

Fashion MNIST and Color Image

In this lecture, we are going to look at a CoLab notebook that does image classification using PyTorch with the convolutional neural network on the CIFAR-10 dataset. This lecture is going to walk you through a prepared CoLab notebook, although a very good exercise, which I always recommend is once you know how this is done, to try and recreate it yourself with as few references as possible. As usual, you can look at the title of the notebook to determine what notebook we are currently looking at. All right, so there are a few major differences at the start of this code. First, you can see that the imports are almost the same as before, except that we also import torch dot nn dot functional.

You'll see how that will be used very shortly. I'd like to introduce these new things when there's not much else going on so that you can learn about different ways of doing the same thing, which could be useful in the future. Next, you'll notice that we are importing the CIFAR-10 data set, which is a color image data set and is even more difficult than Fashion MNIST, however, since the data set is included in torchvision, loading it in is just as easy as it was for the previous two data sets. Next, let's look at the raw data by calling the data attribute, as you can see, it's slightly different from MNIST and Fashion MNIST, which return torch Tensors.

Instead, CIFAR-10 is stored as a regular NumPy array. And the type is uint8. Next, let's check the shape of the training data, we can see that we have 50000 training samples in each training sample is a 32 by 32 color image. Next, let's check the targets. This is, again, different from MNIST. Instead of being stored as a tensor, it's stored as a list. Next, let's get the number of classes K by adding all the targets to a set and then taking the length of that set of. Next, let's create our data loader objects, which is the same as how we did it before. As mentioned previously, here's a good opportunity to look at one of the quirks of PyTorch. If we look at the shape of the data yielded by our data loader, we see something interesting.

The batch that's yielded by the data loader has shaped one by three by 32 by 32. This should seem strange to you because now the color channel comes before the spatial dimensions instead of after. PyTorch hides this detail from you by getting you to use data loaders and special data set functions. Next, let's look at our CNN custom model class. You'll notice that this is built a little differently from our previous CNN. What I'm trying to do is show you different ways of accomplishing the same thing. One thing that's different is the number of input color channels, which is now three instead of one since our data is in color. Next, you'll notice that in the constructor, I've only defined layers that involve parameters, so there are no ReLU activations.

So where did they go? Well, remember that none of these layers are actually called until we get to the forward function. So let's look at the forward function, we can see here that instead of treating the ReLUs as modules or layers, we can treat them like functions which can be accessed from torch dot nn dot functional. That's the new thing we imported earlier, if you recall. In fact, dropout can be applied this way as well. You can also see here an alternative way of using the view function, which, as you recall, flattens the data. Instead of putting the minus one in the last dimension, we can put minus one in the first dimension and then specify the second dimension explicitly.

This isn't any extra work for us because we had to calculate this value anyway inside the constructor. So you can see 128 times, three times three, both in the constructor and in the forward function. All right, so next is a bunch of stuff we've seen before, so I'm going to skip over it a lot faster. So we instantiate our

model, we move the model to the GPU, we create a loss and optimizer. We do our batch gradient descent loop. And we can see that our model trains quite fast at around six or seven seconds per epoch. If we look at the loss per iteration, it seems to look OK. Next, we check the model accuracy. So we get about 75 percent on the train set and a 66 percent on the test set.

Of course, this is going to vary depending on several different factors, so when you run this, you might get a slightly different answer. But clearly, this is a much more difficult data set than MNIST or Fashion MNIST. Next, we look at the confusion matrix. In fact, there are so many things that we predict wrong that it's kind of difficult to look at, we can see that we confuse one and nine a lot. One corresponds to automobile and nine corresponds to truck. This makes sense and fits our intuition. We can see that three and five get confused a lot. Three corresponds to cat and five corresponds to a dog. That makes sense, especially when you consider that these are very small pictures.

We can also see that zero and eight get confused a lot, zero corresponds to airplane and eight corresponds to ship, that makes less sense and perhaps it could simply point to the fact that our model is just not powerful enough. So if we scroll down to the bottom and we look at some misclassified samples, maybe we can make sense of these results. So here's a horse predicted as an airplane that does not make sense at all. Here's a bird predicted as a deer, that kind of makes sense. Here is a truck predicted as an automobile, that kind of makes sense. Here is a cat predicted as a bird, looks like a small blob, so that kind of makes sense. Here's a frog predicted as a deer that doesn't really make sense.

Here's a dog predicted as a cat, that kind of makes sense. All right, so it seems that a lot of these wrong predictions, the neural network should be able to get right. So in the coming lectures, we're going to see how we can improve this model.

LECTURE 43

Why Data Is Important

In this lecture we are going to discuss how to improve our model using a process known as data augmentation. Consider this picture of a cat. You and I can agree that it's a cat now consider this other picture of a cat. It's pretty obvious that this is the same cat.

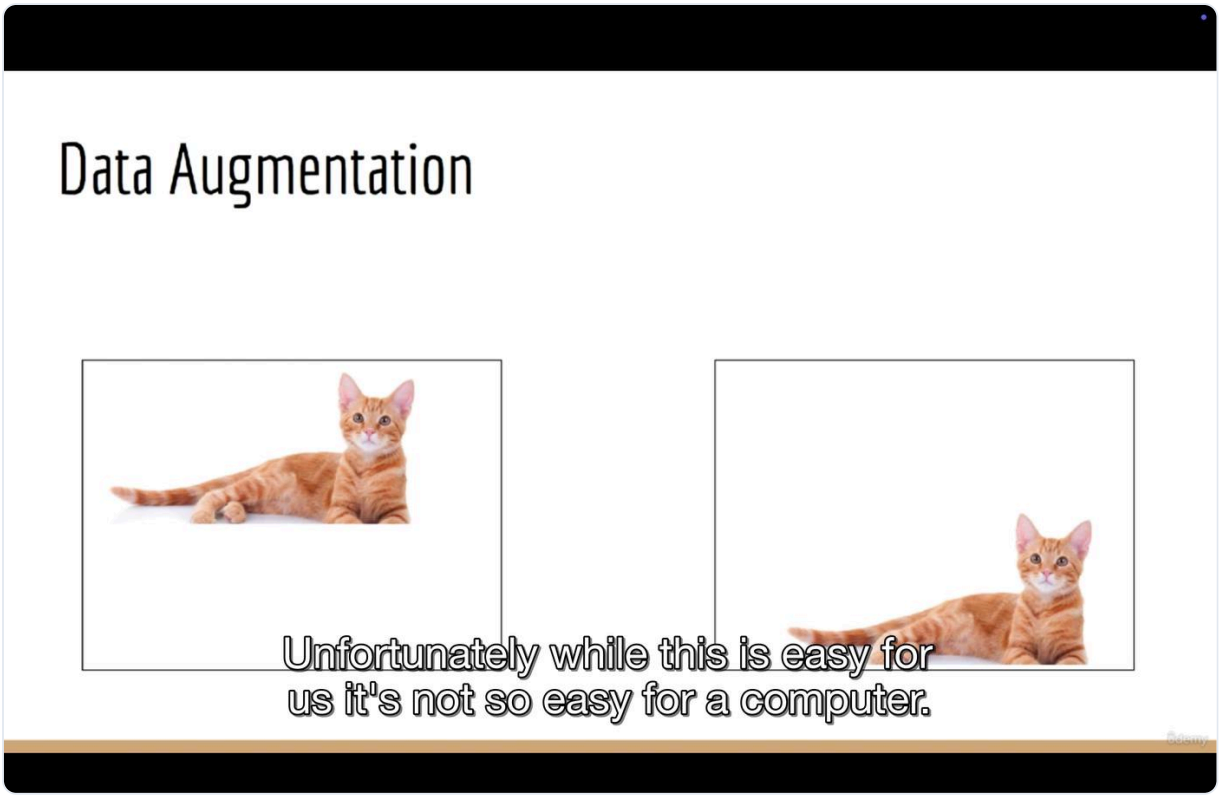


FIGURE 5.1 | Data Augmentation

Unfortunately while this is easy for us it's not so easy for a computer. Now the neural network may be able to identify this because of convolution but perhaps there's a way to help the neural network learn this translational invariance using data augmentation as you may have heard one of the reasons. Deep Learning has been so successful these days is that we have so much more data deep learning continues to improve as it trains on more and more data. While other machine learning algorithms seem to hit a plateau. One disadvantage of tabular data is that you just have what you have.

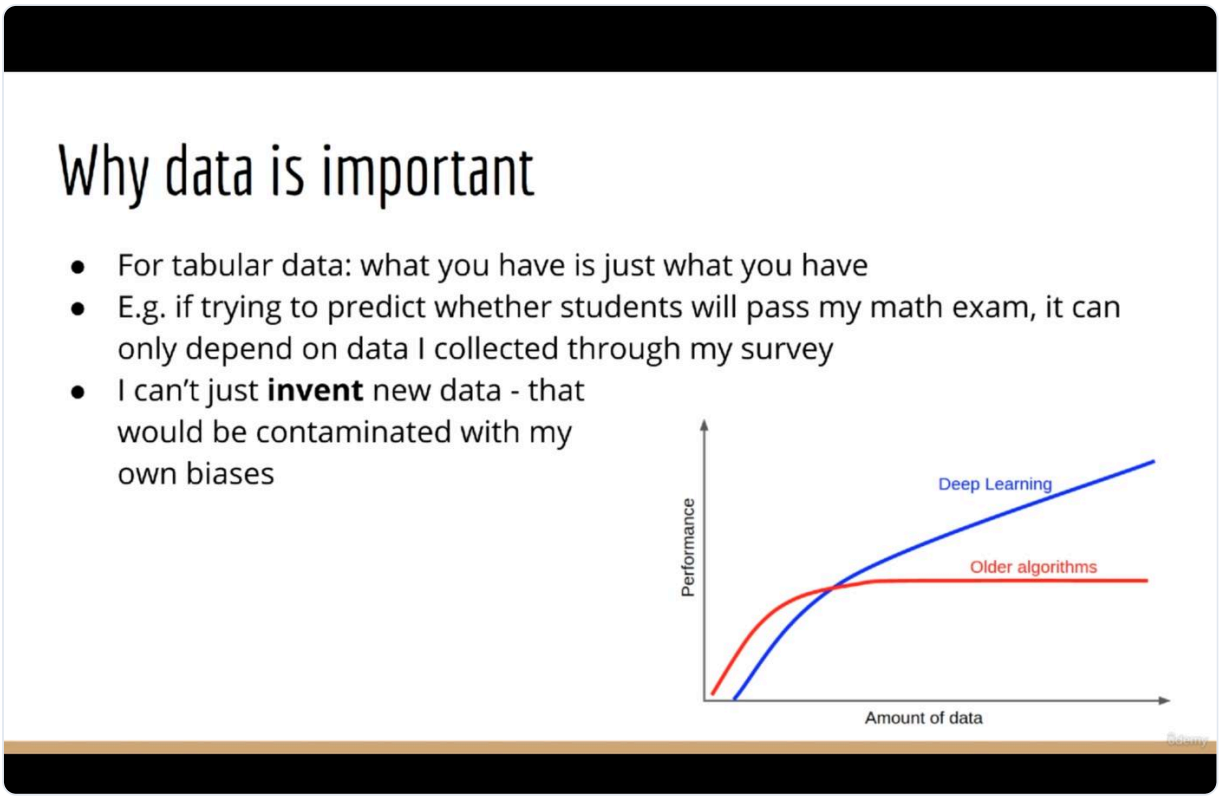


FIGURE 5.2 | Why Data Is Important

If we're doing classification to see if you'll pass my math exam and I base that on how many hours you study and how many hours you play video games that can only depend on the data I collected through my survey. I can't just make up new data which would necessarily include my own biases about student performance. I can't invent a new student who studied this much in play video games this much and then decide if he passed or failed. That wouldn't make any sense however.

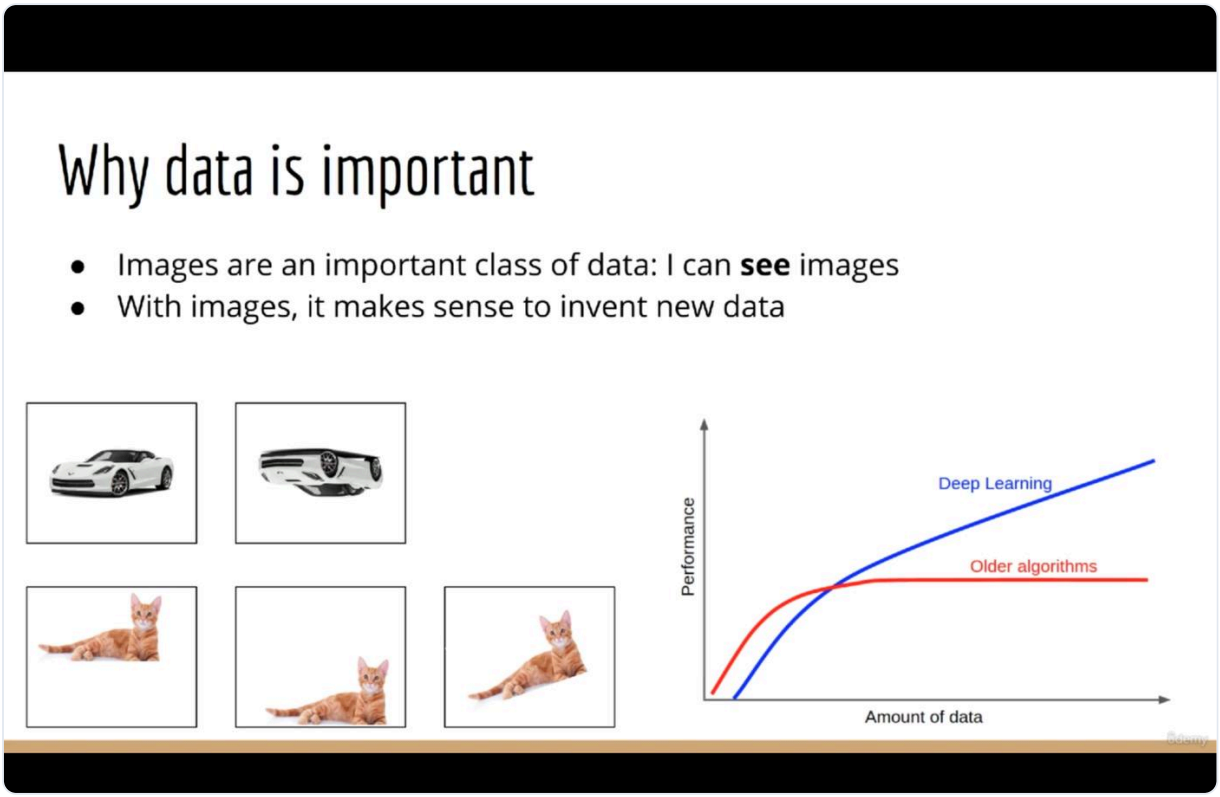


FIGURE 5.3 | Why Data Is Important

Images are an important class of data we can see them. We know that both these images are cats. I know that if I rotate this cat a little bit it's still a cat. I know that if I flip a car upside down or left to right it's still a car. Although you should probably be worried if you encounter an upside down car. In other words what I'm trying to say is with images it is ok to invent new data. Now here's one problem with this approach.

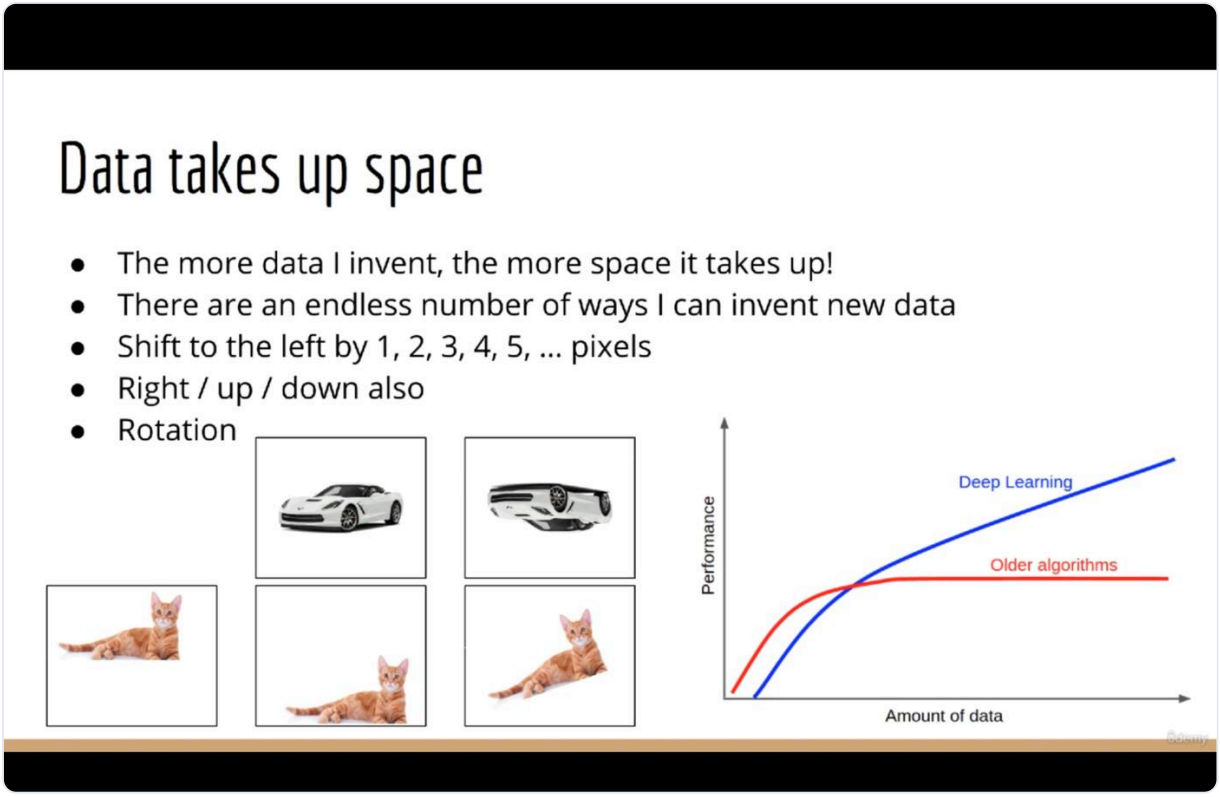


FIGURE 5.4 | Data Takes Up Space

Data takes up space so the more new data I invent the more space it takes up and on top of that there are an endless number of ways I can invent new data. I can shift the image to the left one pixel two pixels three pixels four pixels five pixels and so forth and I can also go in the other direction. I can also rotate one degree two degrees three degrees and so on. Do I really want to store all these combinations on my hard drive. Probably not. Also Google Colab probably wouldn't have space for that anyway. So what can you do luckily none of this work is work you have to do yourself. Have you ever considered how to rotate an image. Well it's not as straightforward as it sounds.

Instead we can use the torchvision API to do all this work for us.

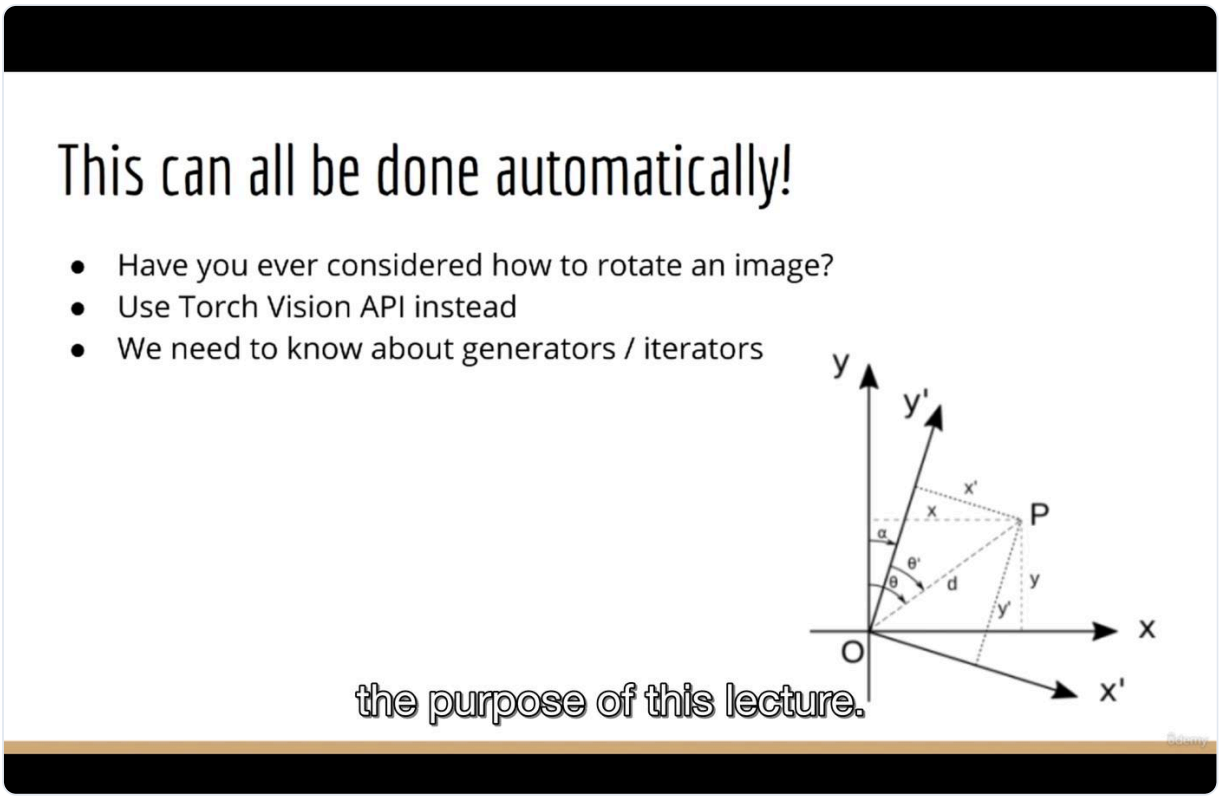


FIGURE 5.5

To understand how it's going to work we have to discuss the concept of generators and iterators for the purpose of this lecture. We won't really differentiate between the two consider what you do when you want to write a for loop using indices 0 up to let's say 10. We would do. For i in range 10. Now in python 2 you would not want to use range 10 because that would return an actual list of numbers from zero up to 9.

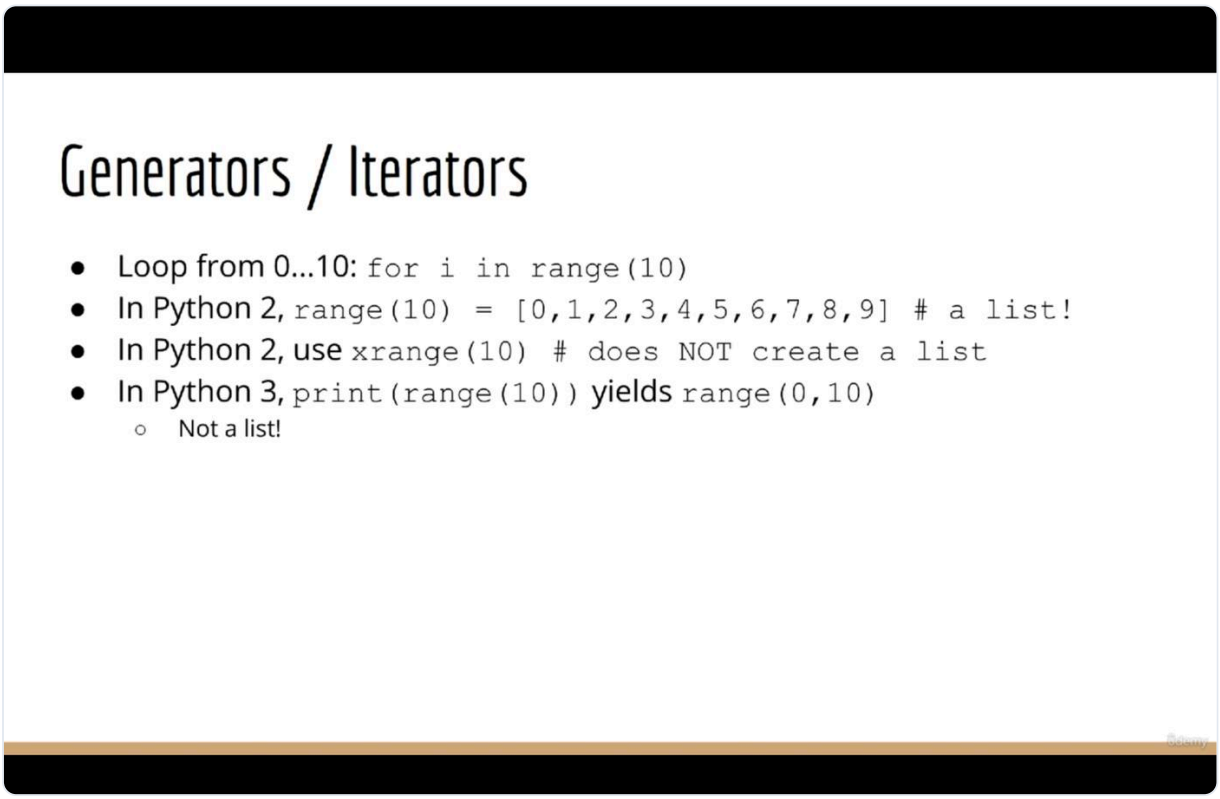


FIGURE 5.6 | Generators / Iterators

Instead you would use X range which does not create a list but rather generates those numbers on the fly. That's why in Python 3 This is the default behavior for the range function. If you print out range 10 you will not see a list of numbers from 0 up to 9. As you might expect conceptually that's what it is but physically that's not what it is. Instead it just says range 010 which is kind of not telling us much. The range is a range. Well it appears to be some kind of object on which you can call instance methods and so forth like you might not have known is that you can actually write your own generators. For example suppose you wanted to write a loop where the loop variable was a randomly generated number.

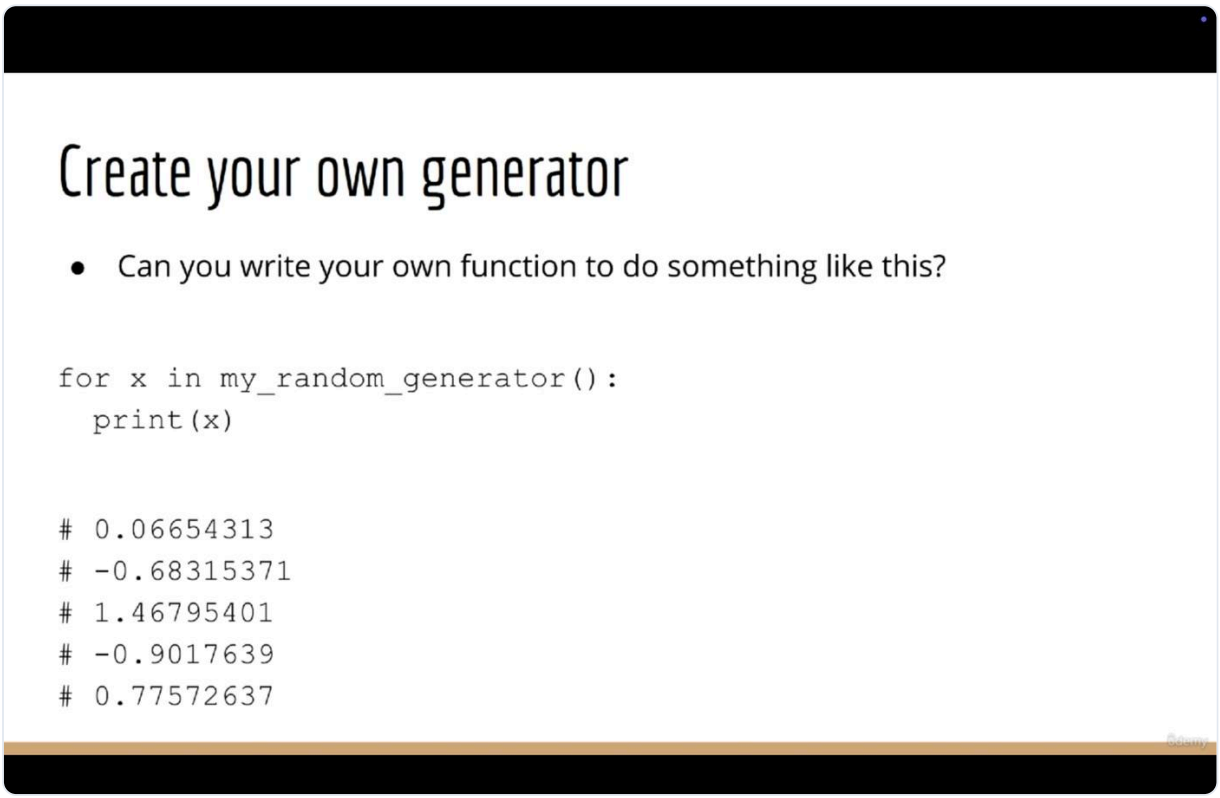


FIGURE 5.7 | Create Your Own Generator

So for x in my random generator would be a loop where on each iteration of the loop X would be a random number. Can we do this in Python The answer is yes we can accomplish exactly this using the yield command.

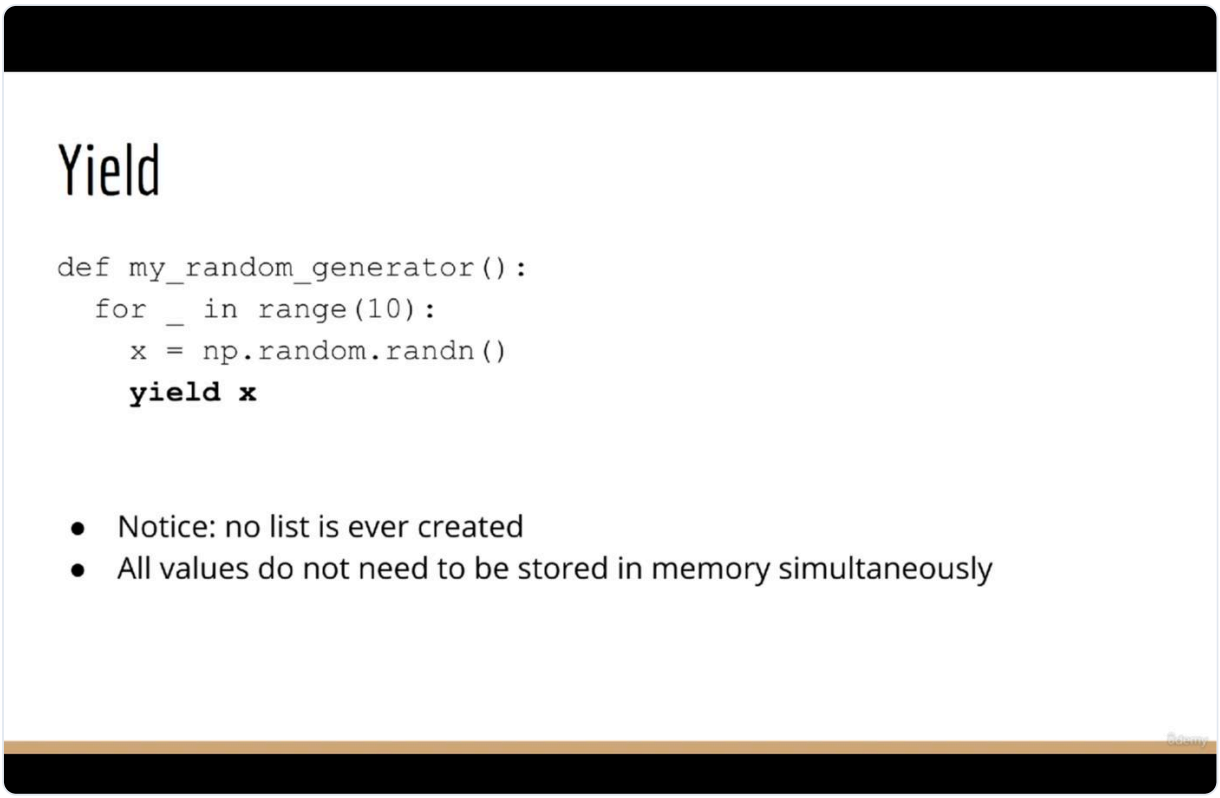


FIGURE 5.8

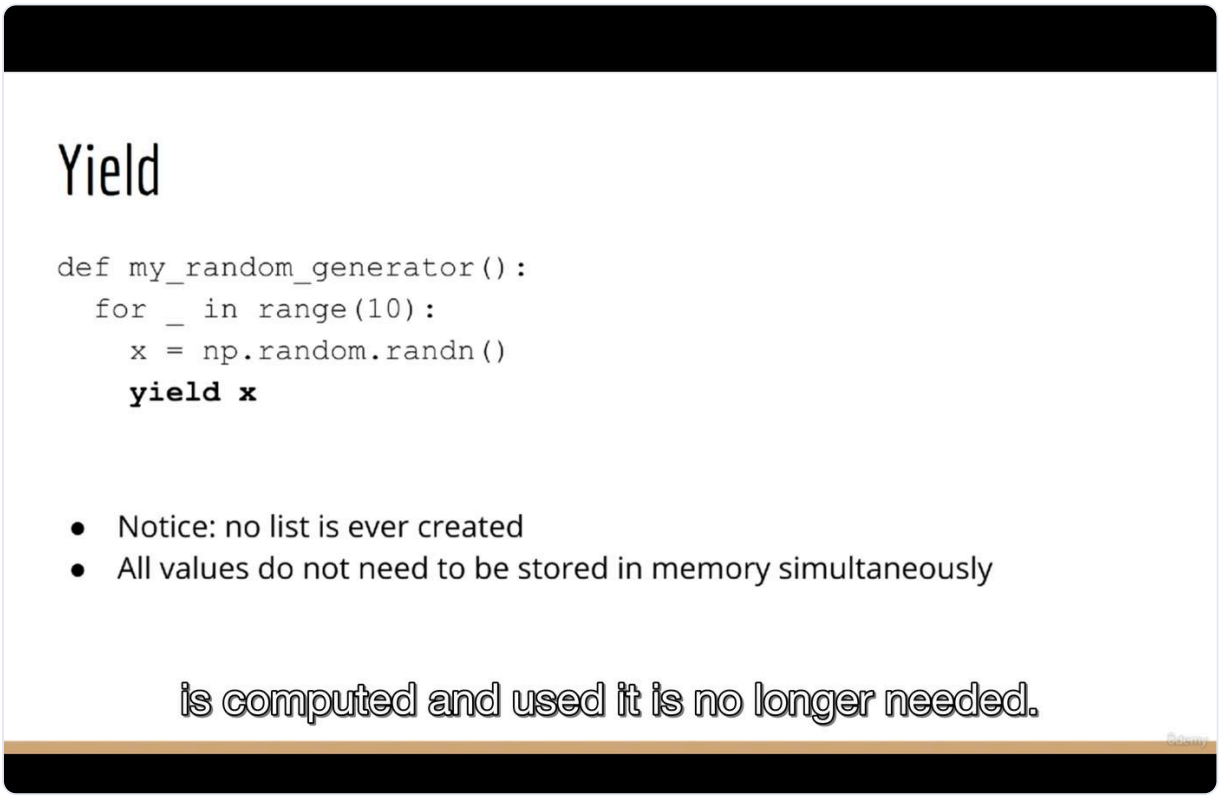


FIGURE 5.9

So here is how you would write my random generator function that generates 10 random numbers on the fly as you can see the code to generate the entire list of values does not have to be computed beforehand and more importantly the entire list of values does not have to be stored in memory. Instead this takes up a constant amount of memory because as soon as the variable for each iteration is computed and used it is no longer needed.

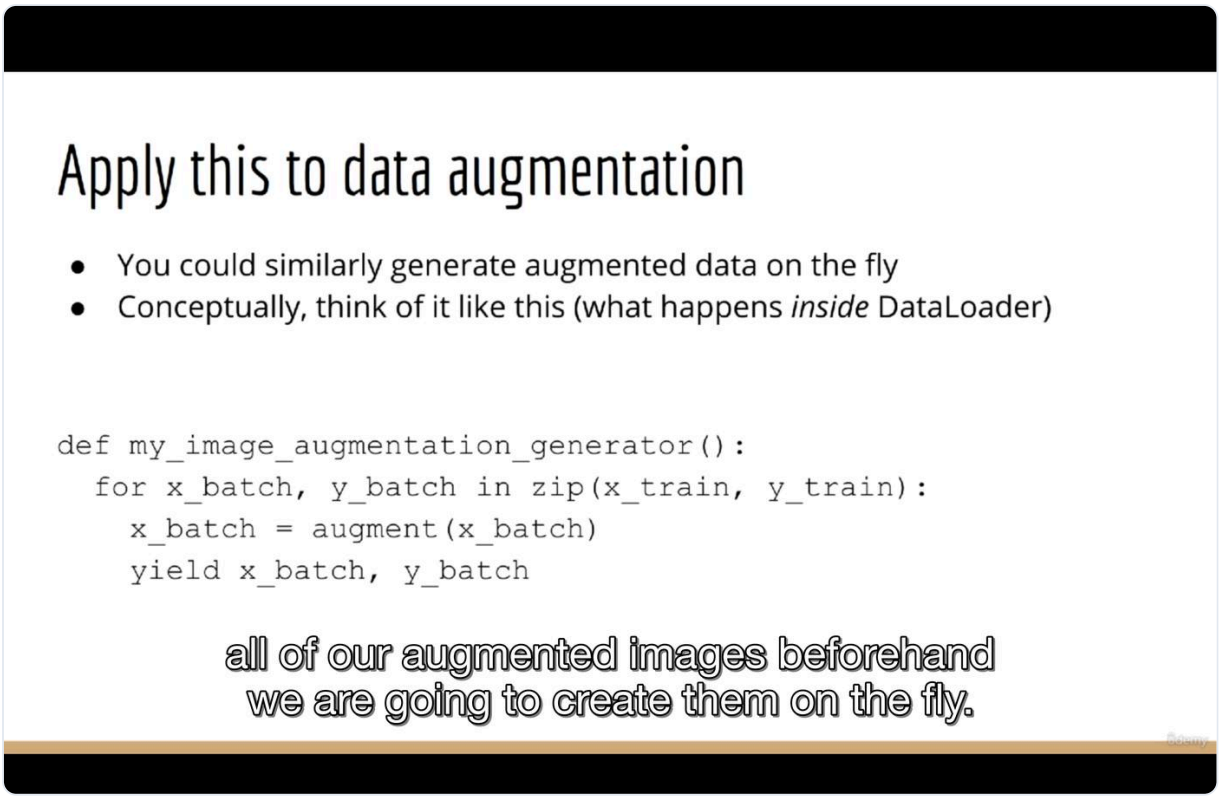


FIGURE 5.10 | Apply This to Data Augmentation

So in PyTorch when we want to do data augmentation it's a similar concept instead of pre calculating all of our augmented images beforehand we are going to create them on the fly.

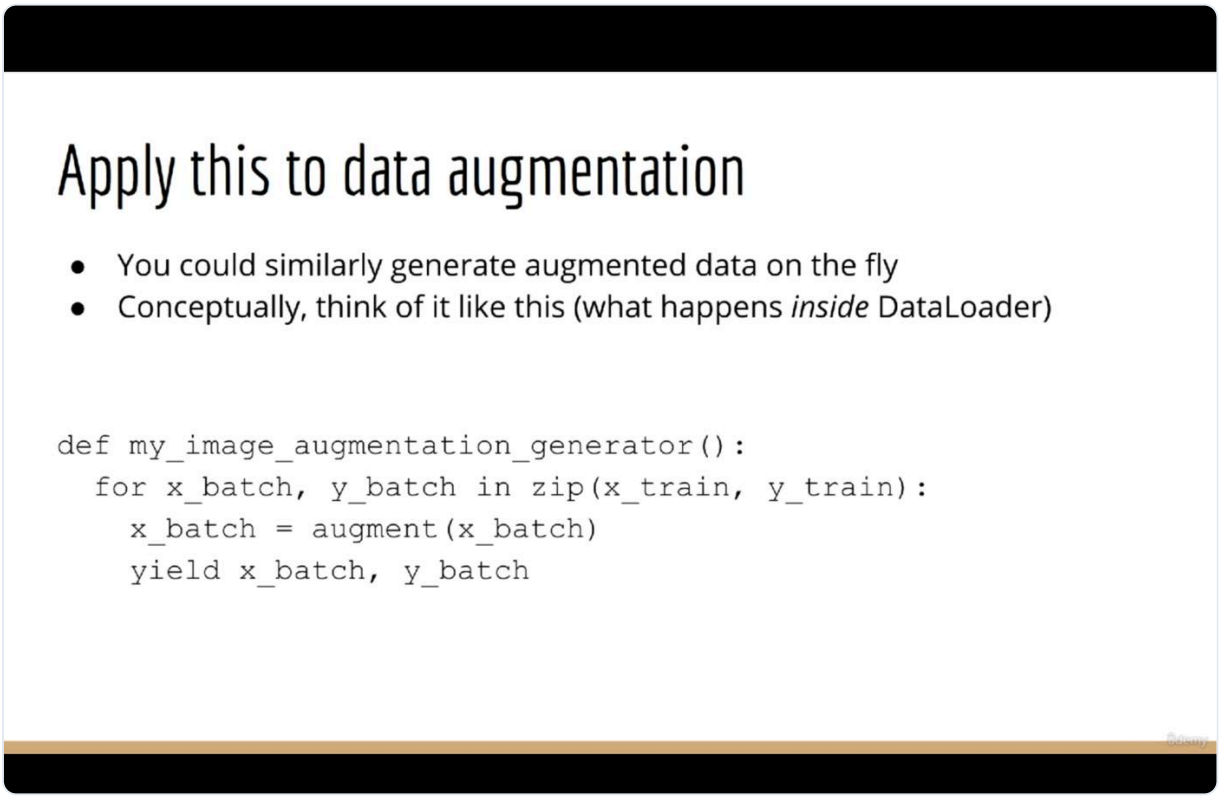


FIGURE 5.11 | Apply This to Data Augmentation

You can imagine that it looks something like this for each batch of data we augment X batch using some randomly generated values then we yield x batch and Y batch. In other words only a small slice of X train is ever augmented. On each iteration. More importantly it's done randomly so you get something different each time which helps with generalization what you should notice is that this looks curiously like our data loader which we have already used for generating batches. When we do Batch gradient descent so how does this actually work in PyTorch. What we've learned so far in this lecture is just conceptual. Practically speaking the answer lies in the functionality we have already been making use of specifically you know that when we load in the CIFAR dataset it's just a function and this function accepts as input an argument called transform.

At this point you should ask yourself. I wonder what this does.

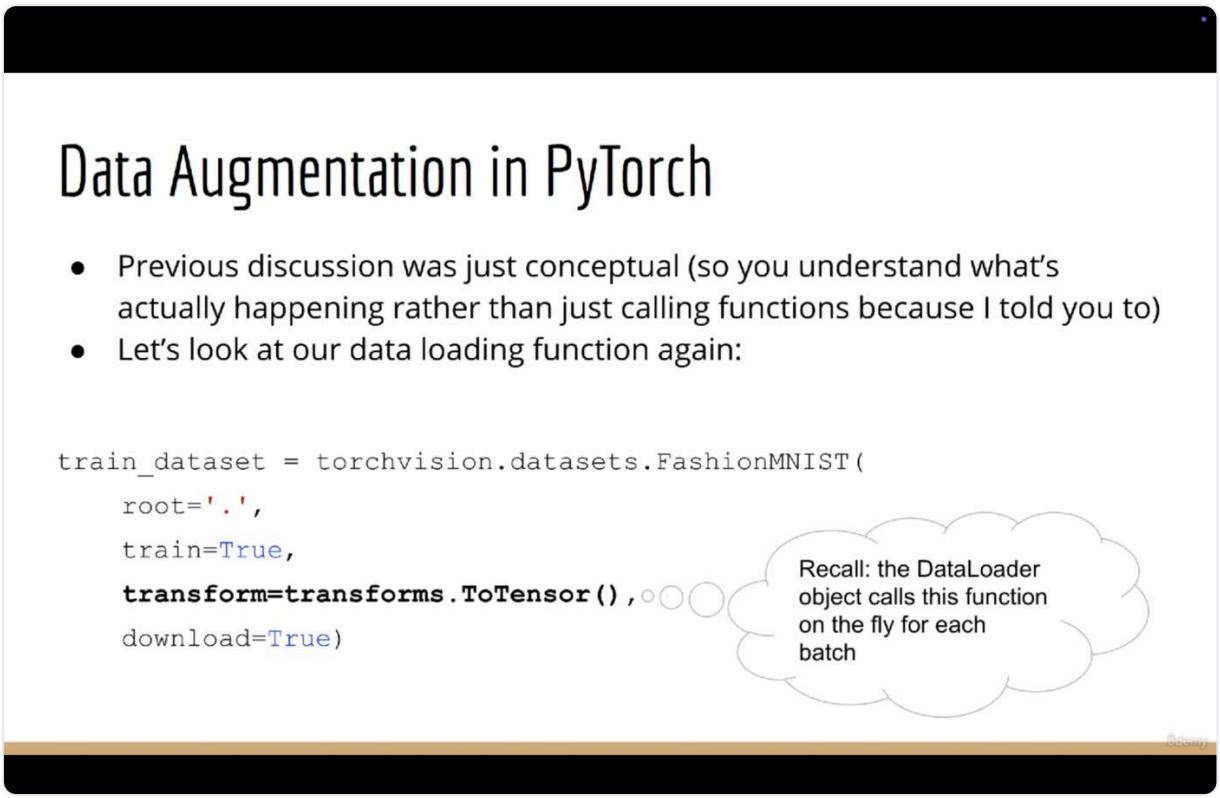


FIGURE 5.12 | Data Augmentation in PyTorch

In addition as you may recall we learned in the last section the data loader object yields your data set in batches as you loop over it applying the transform function on the fly so to be explicit. Well we have to do is modify the transform argument and instruct torchvision on how to augment the data specifically torchvision comes with a whole collection of different transformations you can apply such as slightly changing the colour cropping flipping both horizontally and vertically rotating and so forth. I like the idea of changing color because I don't think people mentioned it that often yet it seems very important. You would want your model to perform well under different lighting conditions and different colors.

I would recommend checking out the documentation and experimenting with each of them in different combinations to get a feel for what has the most impact on your particular dataset. As I mentioned early on in this course PyTorch gives us a lot of building blocks from which we can compose more complex things just like how we can compose complex neural networks from simple layers.

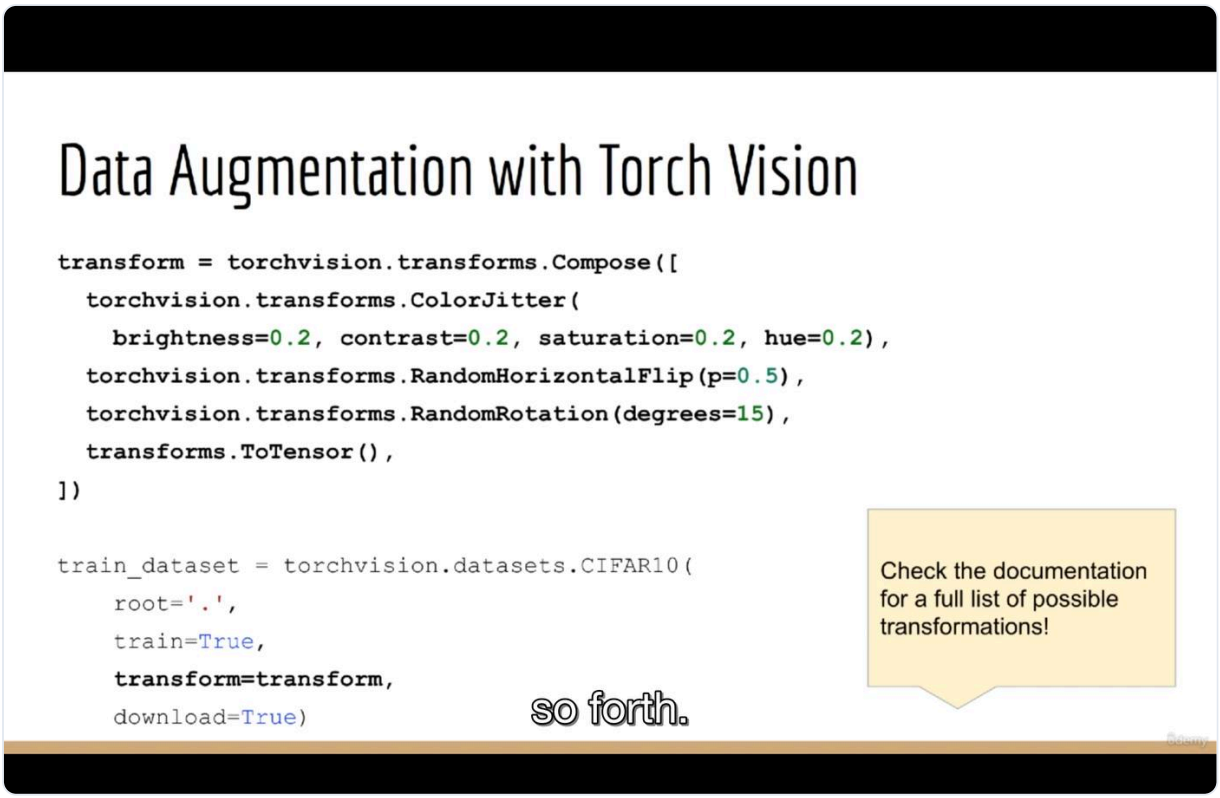


FIGURE 5.13

We can also compose complex transformations made up from these basic ones. All we have to do is wrap them in a torchvision dot transforms dot Compose object and pass this into the transform argument when we load in the data now because all of the data augmentation transformations are specified in the data function itself. That means everything that happens after that does not need to change specifically consider what the data loader would look like exactly the same as before. All the instructions for transforming the data belong to the data set object itself not the data loader. If you recall the only thing that the data loader does is call the transform function on the data set.

Similarly we should expect that the training loop also does not change. We still have a nested for loop to do batch gradient descent.

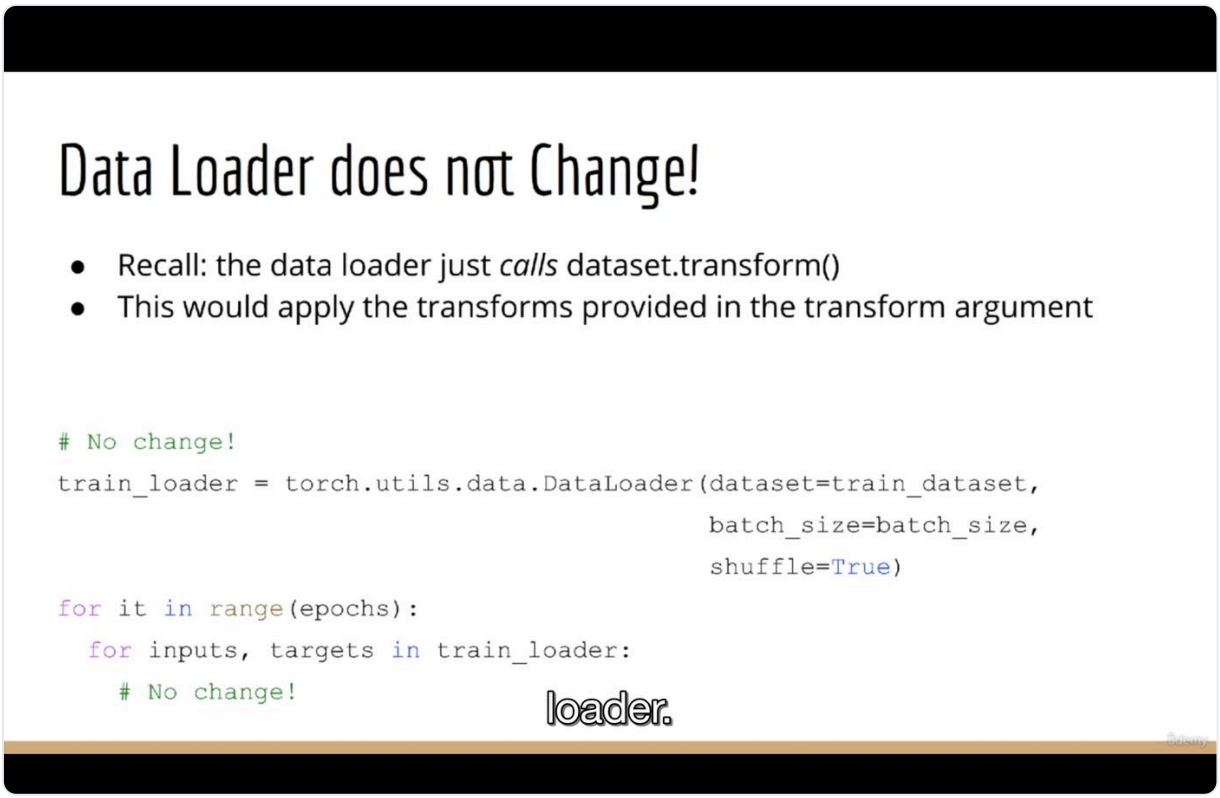


FIGURE 5.14

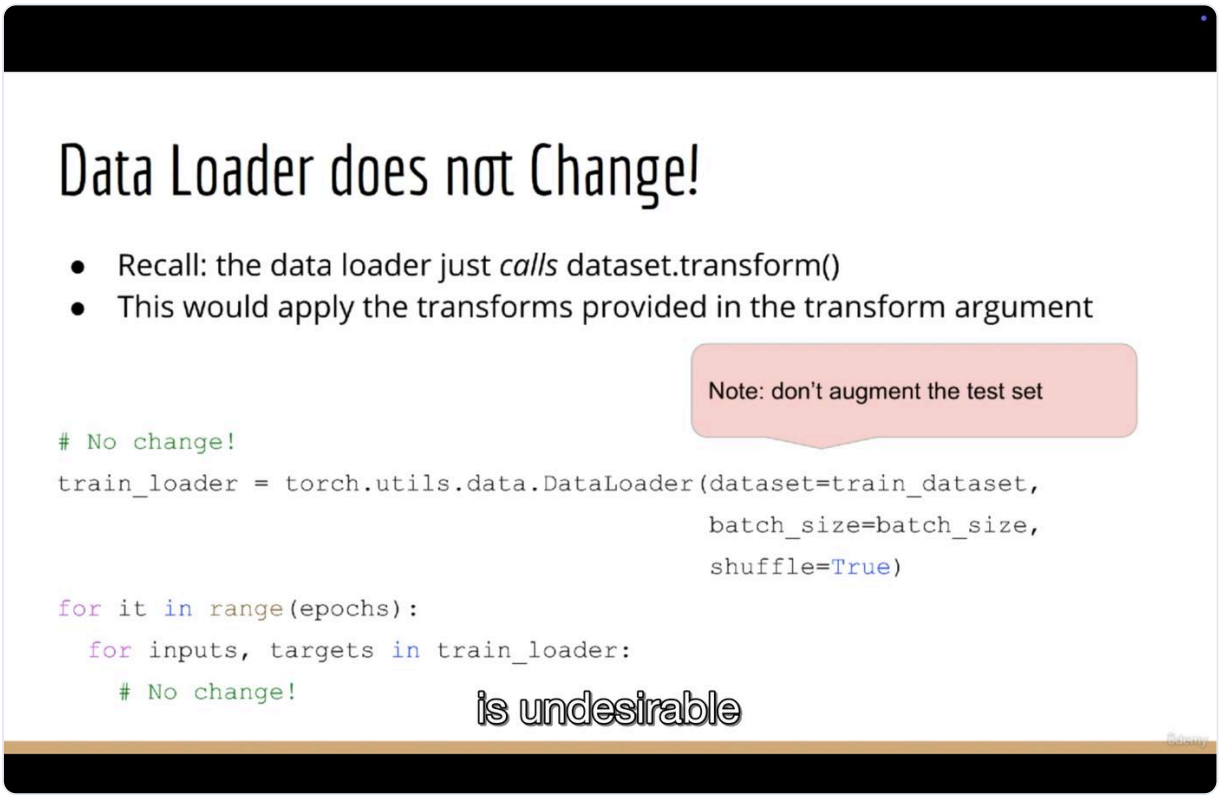


FIGURE 5.15

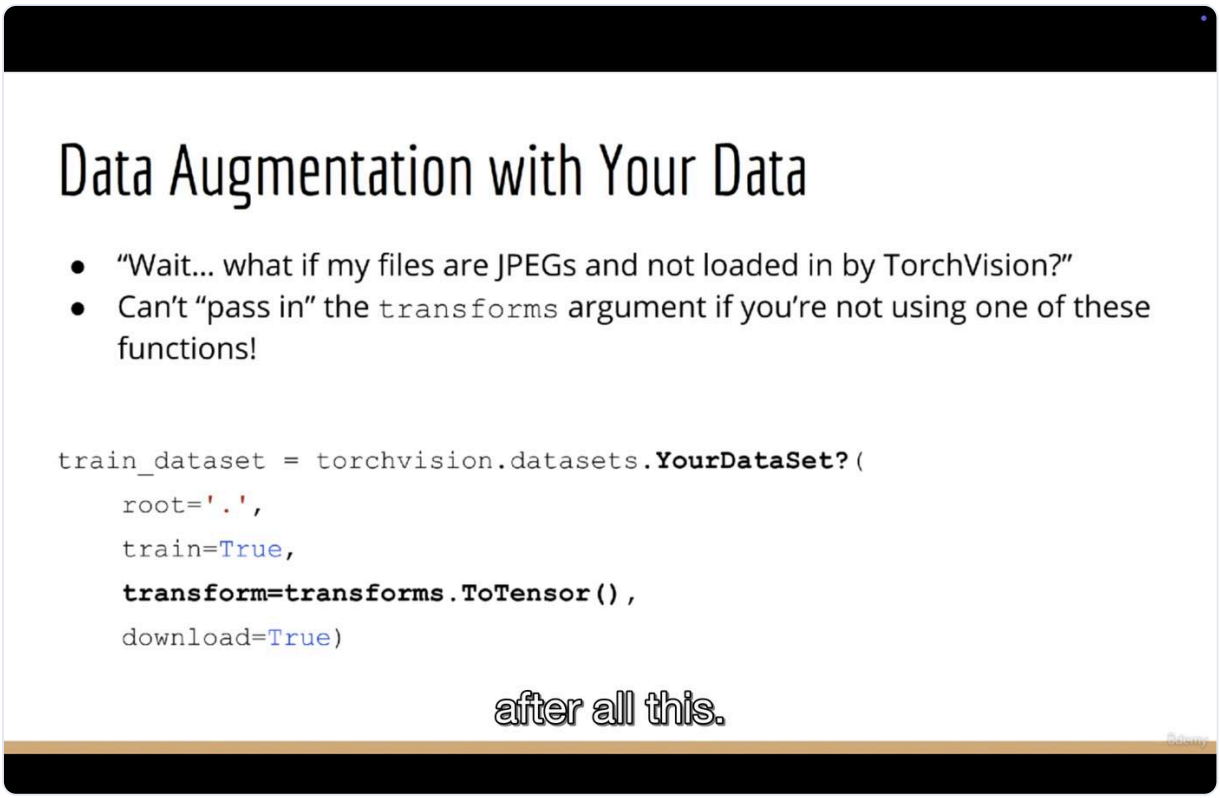


FIGURE 5.16 | Data Augmentation with Your Data

The first loop is still for i in range epochs and the second loop is still for inputs targets in data loader. The only thing to be careful about is that while you do want to augment the training set you probably don't want to augment the test set otherwise you get a different answer each time you evaluate it which is undesirable after all this. You might still have one remaining question. So far we've loaded all our image data from pre-built functions in torchvision but what if we have our own image dataset. Sure I can load images into a NumPy array using a library like Pillow but once I've done that how can I apply these transformations that torchvision provides.

It seems like in order to use them I must have a torchvision data set object. Luckily we will cover this later in the course but for now you'll just have to wait and see.

LECTURE 44

Batch Normalization

In this lecture we are going to introduce a new type of layer which has been shown to be very helpful in training convolutional neural networks here's something to consider.

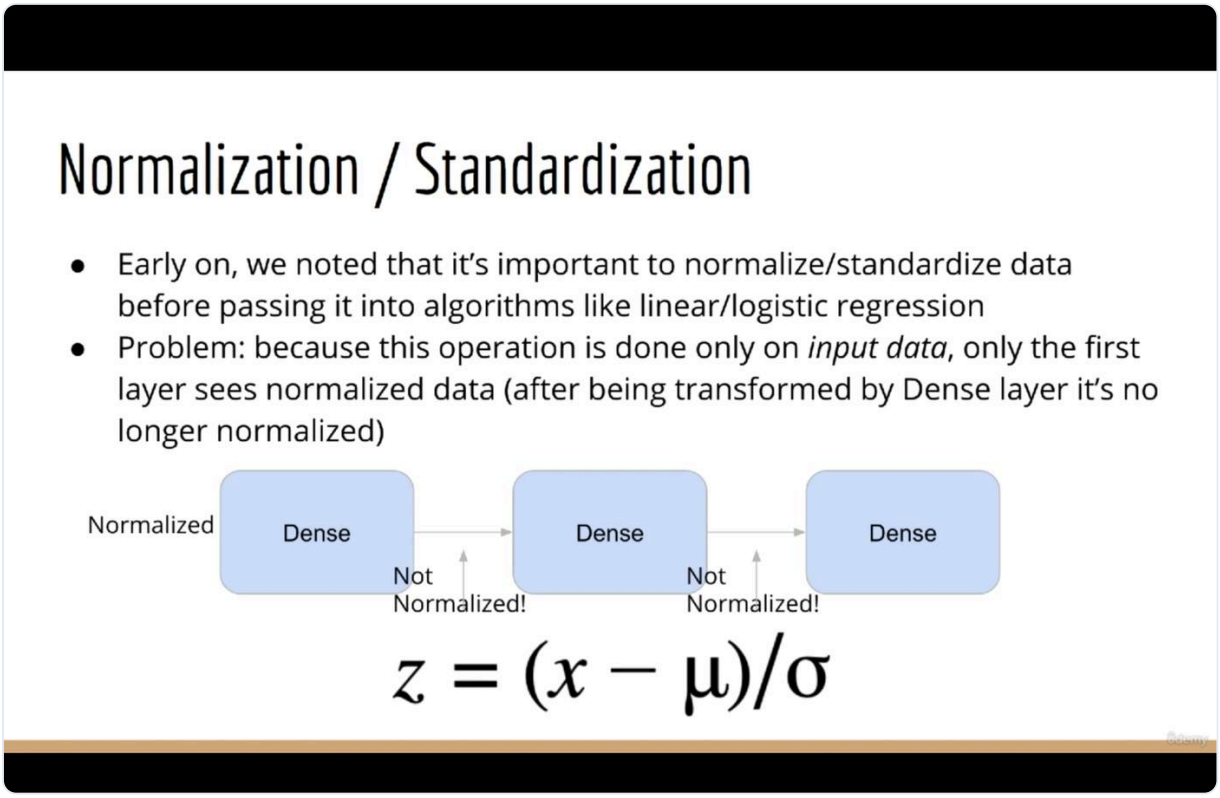


FIGURE 6.1

Recall that early on we looked at why it's important to normalize your data before passing it into a model such as linear regression or logistic regression. We do this by subtracting the mean and dividing by the standard deviation but because this is an operation on the data we can only do it at the very first layer of the neural network. What happens if the data becomes not normalized at some later layer then that layer has to deal with unnormalized data which is not as good as normalized data. So this begs the question how can we make the data at every layer normalized. The answer is batch normalization. If you recall inside the PyTorch training loop we're actually doing Batch gradient descent on the outer loop.

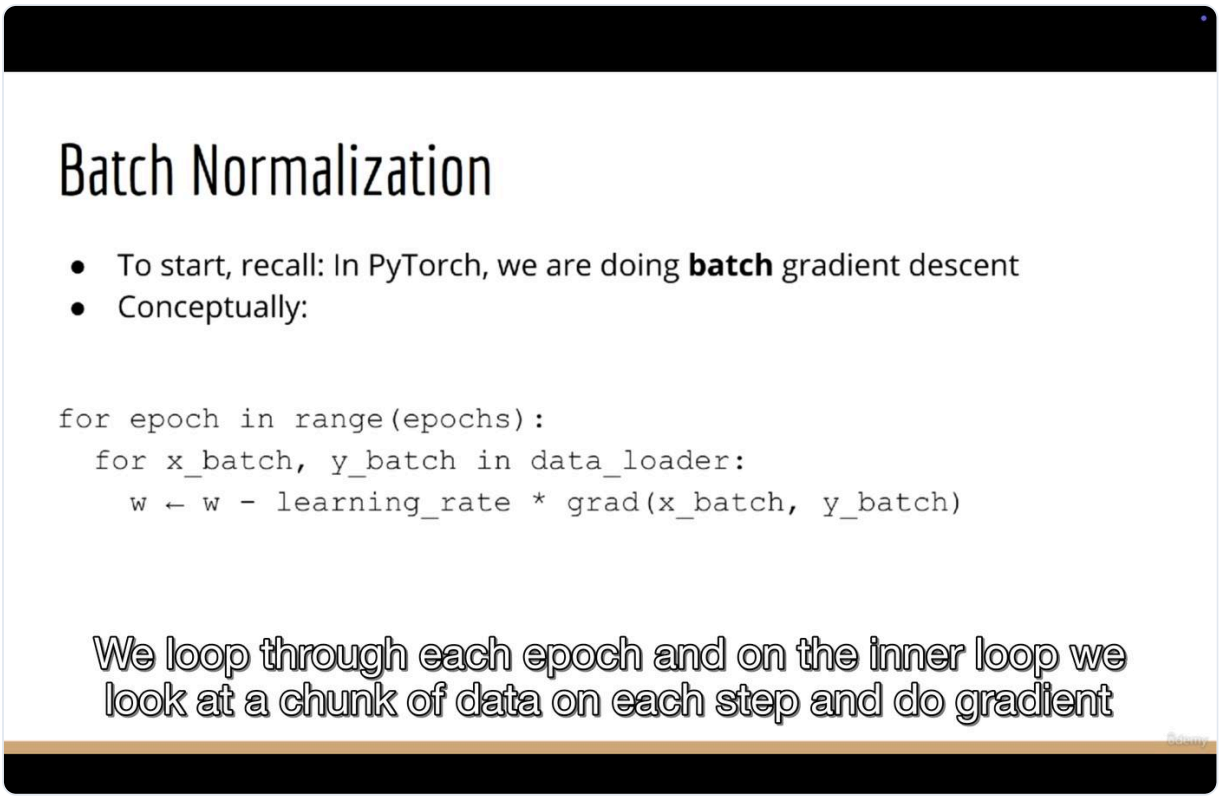


FIGURE 6.2 | Batch Normalization

We loop through each epoch and on the inner loop we look at a chunk of data on each step and do gradient descent with respect to just this chunk of data. This works because we know that a batch of data will have approximately the same statistics as the full dataset. So what if we did something like this. We inject a layer that does normalization for us for example in an ANN instead of dense dense dense we have batch norm dense batch norm dense batch norm dense at each of these layers. All it does is take the batch of data subtract its mean and divide by its standard deviation.

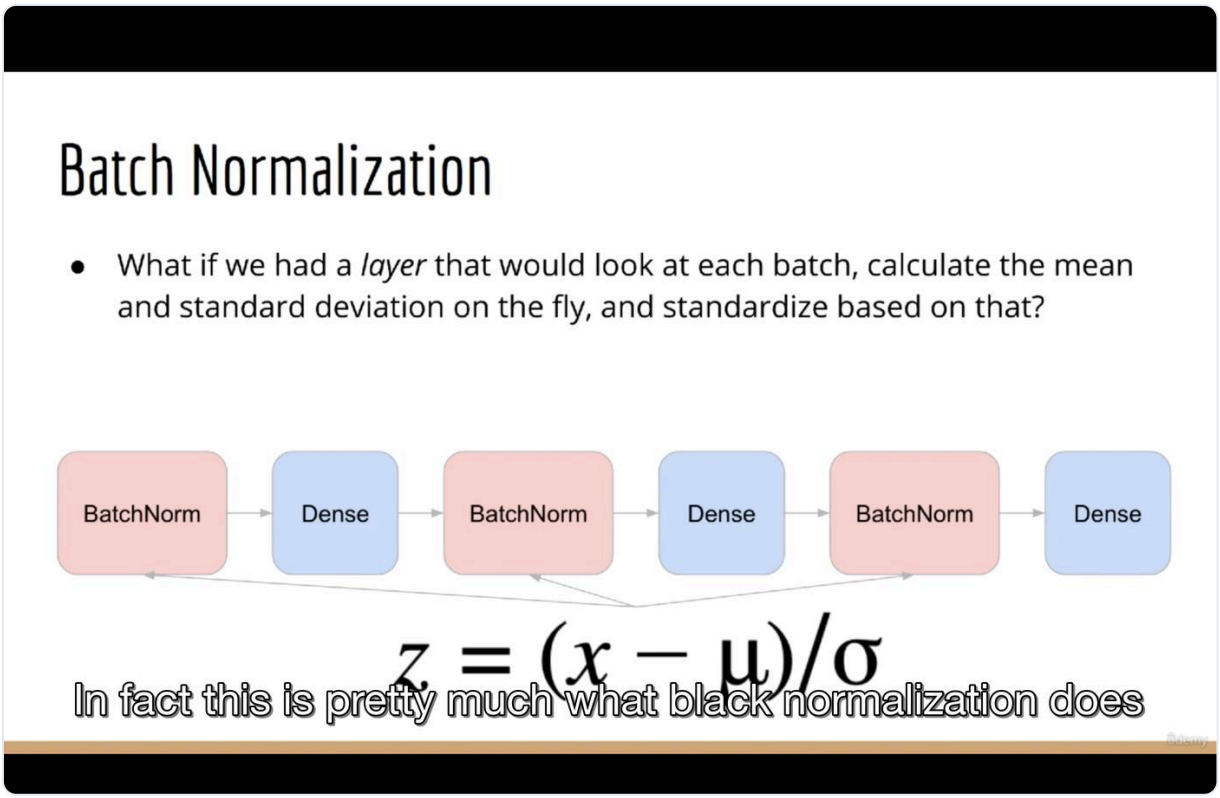


FIGURE 6.3 | Batch Normalization

In fact this is pretty much what batch normalization does but there's something extra that batch normalization does. Here's another question to consider. How do we know that normalization is actually good. In fact we don't. Perhaps there's a better amount of shifting and a better amount of scaling that we can use that leads to a more optimal model. And so this is the full description of the batch normalization layer it first does normalization using the batch mean and the batch standard deviation. Then it re scales the data and re shifts the data so that the final output has an optimal location and an optimal scale.

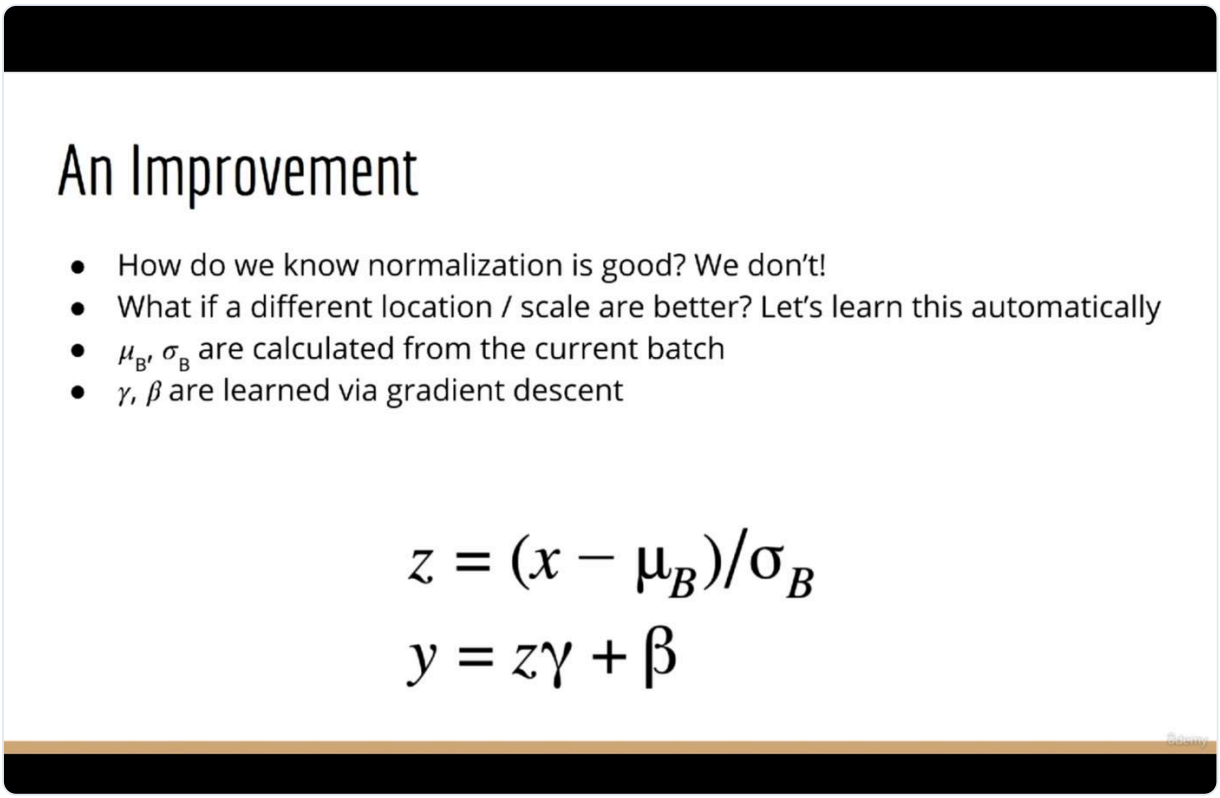


FIGURE 6.4

Of course these new parameters gamma and beta are learned automatically using a gradient descent. One alternative perspective on batch normalization is that it acts as a regularizer. As you know we use regularization to prevent over fitting with methods such as dropout. So how does batch normalization do regularization. The idea is because each batch of data is slightly different. It's going to have a different mean and standard deviation. It's not the true mean and standard deviation of the entire dataset these differences are like noise which slightly change what the neural network sees on each iteration.

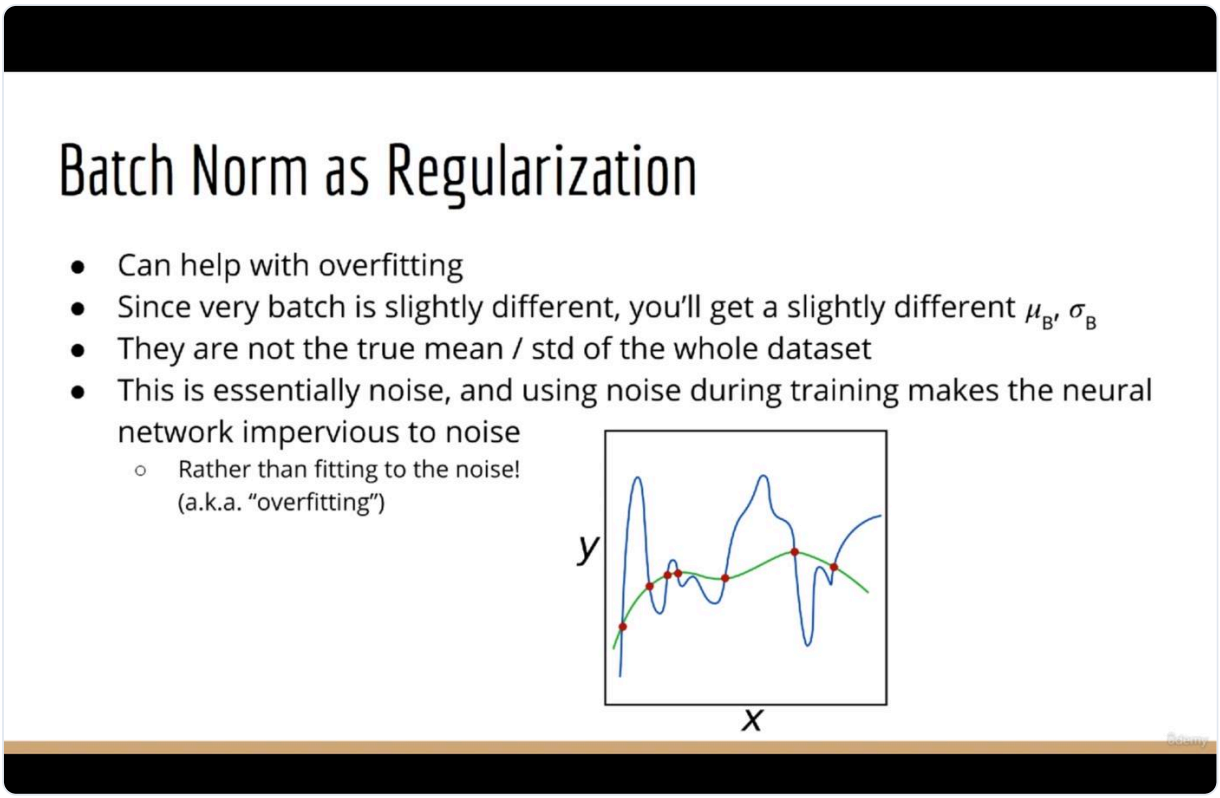


FIGURE 6.5 | Batch Norm as Regularization

In other words because the neural network always has to learn noisy images it becomes impervious to the noise the final thing I want to mention in this lecture is that while we discussed the batch norm in the context of a feedforward neural network batch norm is not commonly applied in between dense layers as you recall. This is the CNN section of the course. And so the reason we're talking about batch normalization now is because batch norm is mostly used with convolutions now adding batch norm also adds some hyperparameter choices. In particular you have to decide whether or not you should use batch norm and if you decide you should then you have to decide where to put it.

Luckily we can just follow the conventions in this regard batch norm layers are usually applied after convolution layers well after or before depends on which way you look at it. So maybe a better way of looking at it is that they go in between convolution layers. One common pattern would be like this convolution to batch norm to convolution to batch norm back to convolution back to batch norm and then flatten and then your dense layers.

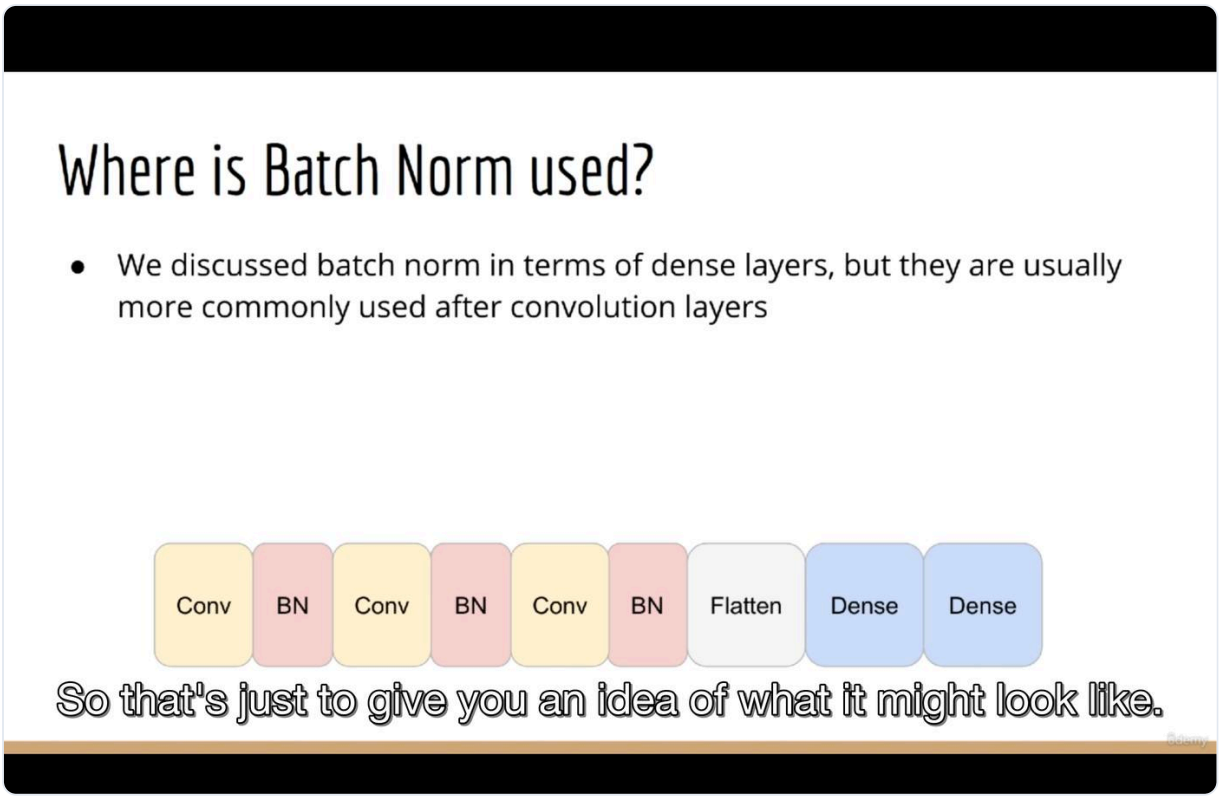


FIGURE 6.6 | Where Is Batch Norm Used?

So that's just to give you an idea of what it might look like. You're always encouraged to try things on your own but also to read about the most popular CNNs such as VGG ResNet and Inception to see how they're doing things. This is the approach we're going to take in the next lecture when we improve our CIFAR-10 results. By the way if you want to read more about batch norm you can check out the paper included an extra reading that T T.

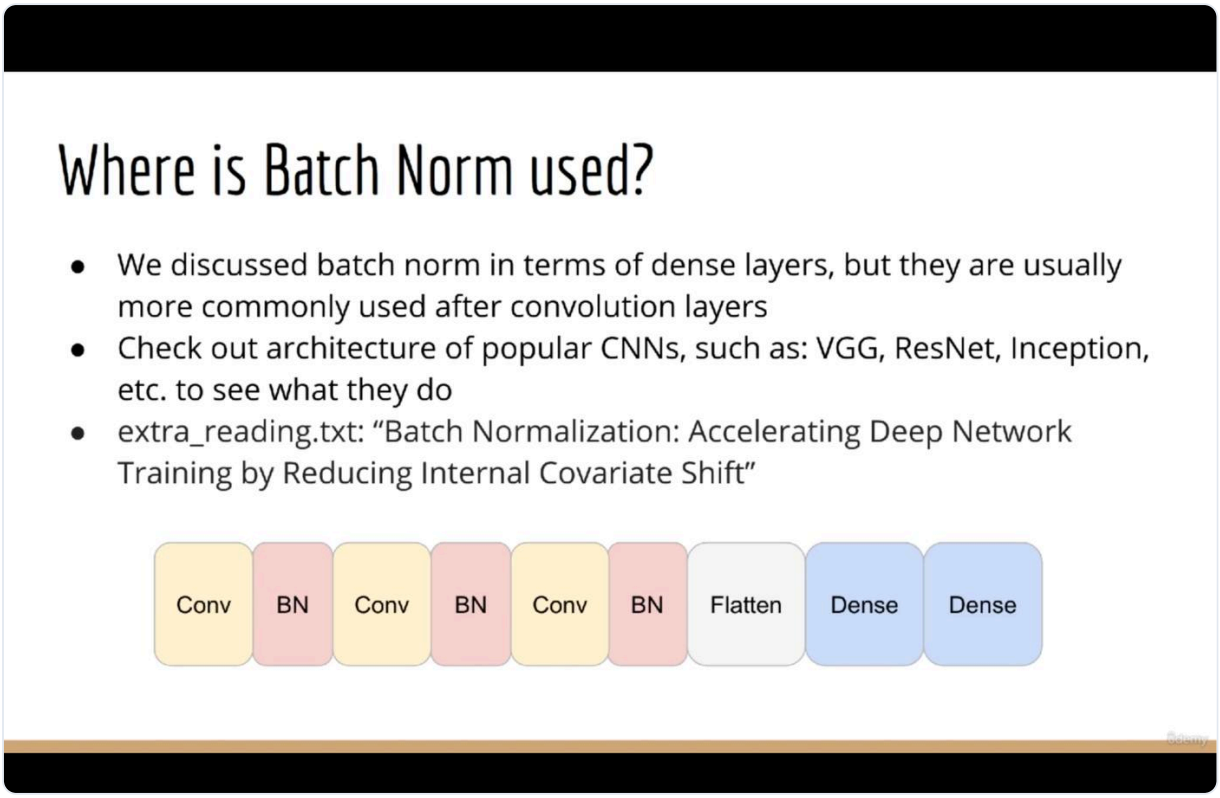


FIGURE 6.7 | Where Is Batch Norm Used?

The title is Batch Normalization Accelerating Deep Network Training by Reducing Internal Covariate Shift.

LECTURE 45

Number of Parameters and Predicted

Check the notebook `8.PyTorch-CIFAR-Improved.ipynb` for the code.

In this lecture, we you're going to improve our previous results using some of the techniques we just learned about, including data augmentation and batch normalization. This lecture is going to walk you through a prepared CoLab notebook, although a very good exercise, which I always recommend is once you know how this is done, to try and recreate it yourself with as few references as possible. As usual, you can look at the title of the notebook to determine what notebook we are currently looking at. One important feature of this notebook is that it has many options for you to choose from. We've added quite a few things since the last script. So as an exercise, one thing you want to try is experiment with what happens if only some of these options are used and not others.

For example, check the results when we don't have data augmentation, check the results when we do not have batch norm, check the results if we remove one layer of convolutions. This is part of the process of improving your results and hyperparameter optimization. OK, so at the top, you'll see that when we load in the data set, we specify a transforms dot Compose instead of just transforms dot ToTensor. This is a list of things I tried. And you definitely want to play around with these to see if you can improve the results. There are tons of things here and each of these things has its own parameters. So test them out and see what happens. So pretty much everything in this script is the same as the previous CIFAR-10 script, so let's just scroll down to where we build the model.

And so here we're just inspecting the data. Here's the data loader. All right, so here's what we define, the model. The first thing you'll see is that I've taken away the strided convolutions in this scenario for these small images. Normal convolution followed by max pooling actually seems to work better. But also, we're taking inspiration from the VGG network, where they do multiple convolution layers before they do pooling. Now VGG is designed for much larger images. So it's also a much larger network. They actually have five groups of convolutions and poolings, and each group has multiple convolutions, too, in the first couple layers and three in the following layers.

So that's another thing we've done differently. First convolution with no strides and now multiple convolutions before doing pooling. Initially, you might want to try only this by itself to see how it works. The next thing you can see that we have added is a batch norm layer after every convolution, not much to say here since we just described how and why it works. But as I mentioned before, because I added so many things to the script, you want to turn some off and turn them on at different combinations so you can see for yourself the effect each of them has. You'll notice that for these convolutions I've used, padding equals one.

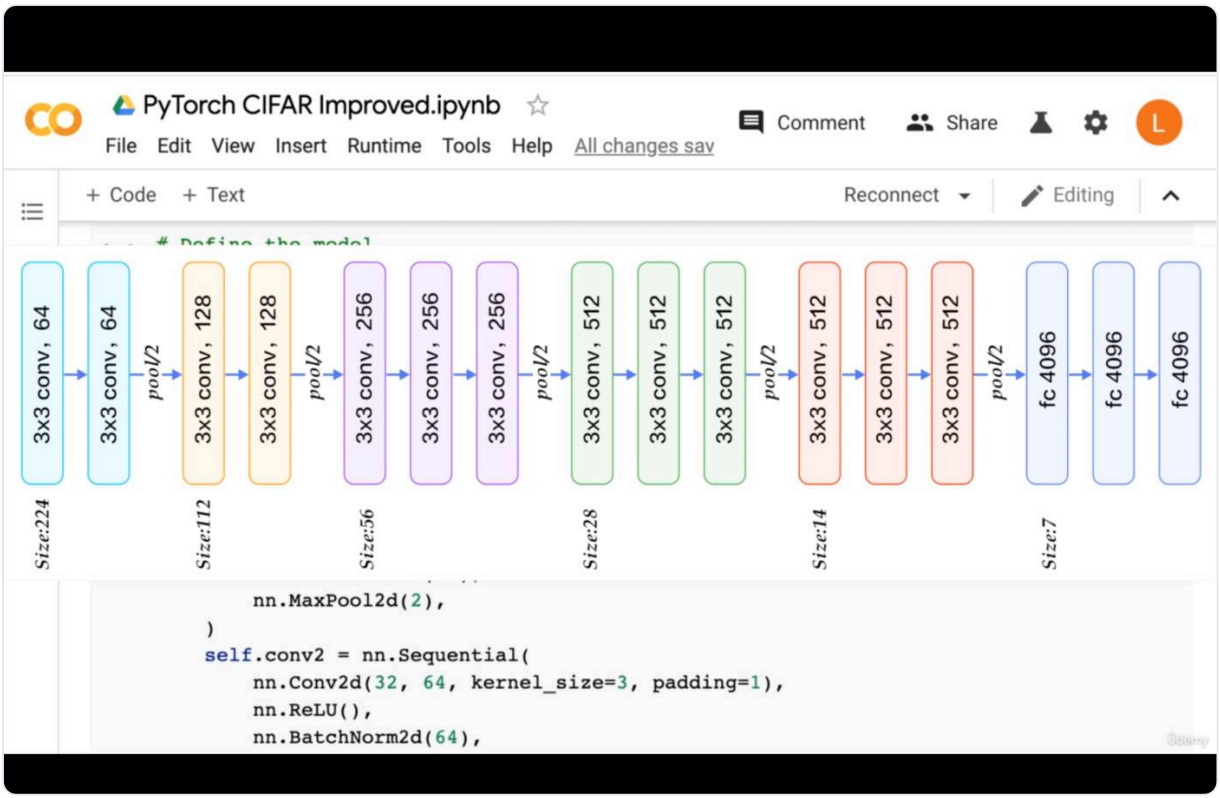


FIGURE 7.1

This is because I wanted to show you how to achieve same padding such that the output image is always equal in size to the input image. Basically, we can take the formula from the PyTorch documentation and plug in values for the kernel size and stride. Then we use the fact that we want H out to be equal to H in and then we can solve for P. In this case I get P equals one. So that means if we want the same mode convolution then we have to use padding equals one. This makes convolutional arithmetic a little easier for us after each convolution, the size of the image remains the same. And after the max pool, the image is downsampled by two. So basically, you can take 32 and divide it by two, three times.

We end up with four, so the size of the input of the first linear layer is 128 times four times four. Finally, we have our final dense layers with the dropout, which is the same as before. All right, so let's scroll down so we instantiate the model, move the model to the GPU, create the loss and optimizer to batch gradient descent. Observe our losses, so these epochs take a little longer, around 15 seconds, and also notice that we did 80 epochs instead of just 15. All right, so here is the loss per iteration, something you may have noticed in the previous script is that it had some overfitting and you can see here that the overfitting effect is less.

So let's look at the accuracy. And you can see that we reach much higher accuracy than before, so now we get about 94 percent on the train set and 87 percent on the test set. Next, let's look at the confusion matrix. All right, so this is encouraging because a lot of these off diagonal numbers are a lot smaller than before. You'll notice that a few of them are still quite high, such as when we confuse threes and fives, which are cats and dogs. I think this is the most difficult to discriminate in this data set because in a 32 by 32 image, cats and dogs are going to look extremely similar. We also still see a lot of ones and nines being confused, which correspond to automobiles and trucks.

Again, that seems to make a lot of sense. OK, so let's take a look at a few misclassified samples. So here is a cat predicted as a truck that doesn't really make sense. Here's a truck predicted as an automobile. I think we can agree that that might be a truck. Here's a cat predicted as a horse, we can kind of see how

that might be a horse or a dog. Here's a frog predicted as a cat. So this frog is kind of in the same poses a cat might stand in. Is a horse predicted as a deer? So, again, we can kind of see why that might be.

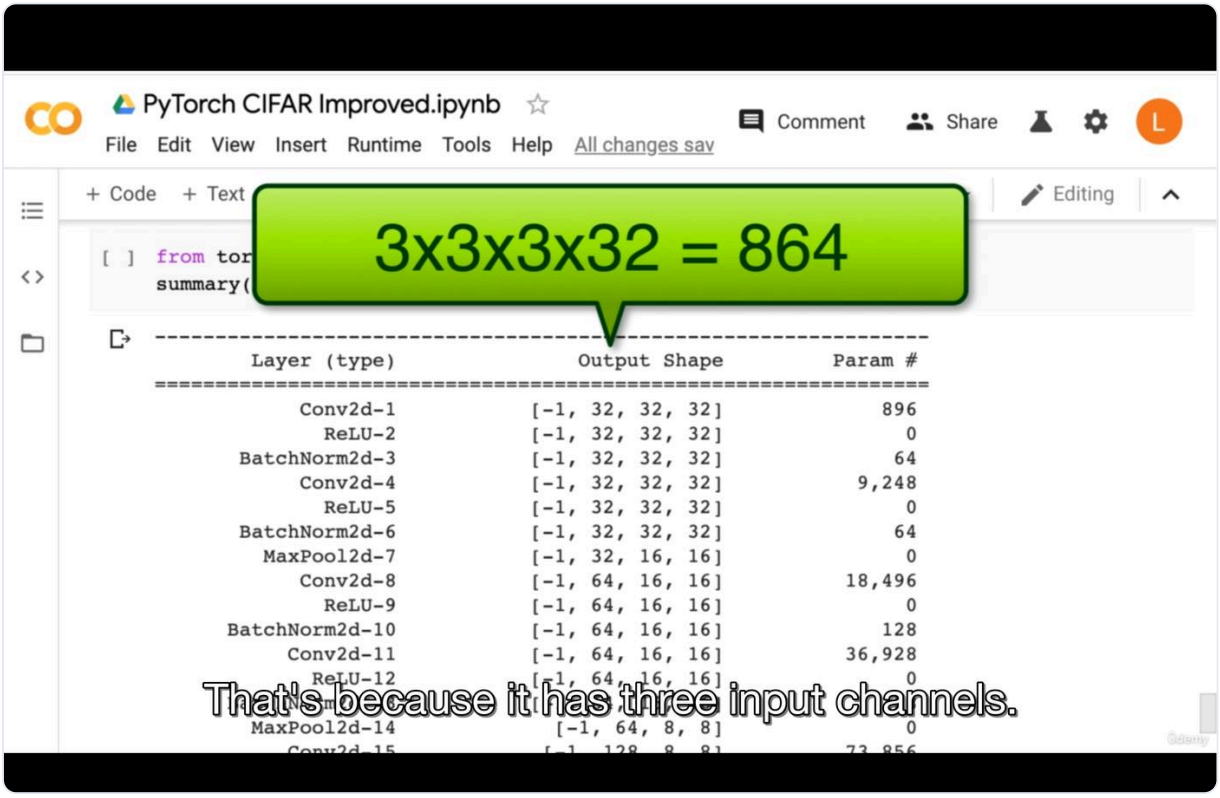


FIGURE 7.2

Here's a truck predicted as a ship. Personally, I don't think it looks like anything. Here's an automobile predicted as an airplane. So that seems kind of wrong. It looks like a car. Here's a horse predicted as a deer that seems to make sense. Here is a dog predicted as a horse, that kind of makes sense. So are these four legged animals are very hard to distinguish, at least for a neural network. Here's a bird predicted as a horse. If we look closely, it kind of looks like a bird. We can definitely tell it looks more like a bird than a horse, but if you weren't told the label, you might not even know what it is. So I think it's clear that while we improve the performance of the neural network, it still doesn't reach human level image recognition capability.

One conclusion you can draw from this lecture could be summarized as bigger is better. We've seen that by virtually adding a lot more data to our data set and using a much larger neural network than what we've seen so far in this course, we were able to significantly improve the performance of our network on this data set. This is a very common theme and deep learning, and it's often surprising because it seems so simple. The fact that it's so simple actually makes it seem less profound than it really is. Some people believe that instead of better algorithms, we just need to scale up our data and our models and our compute. And this by itself will lead to breakthroughs in AI.

As noted, our final neural network in this lecture is much larger than what we've worked with so far in general, CNN's can get pretty deep, even hundreds of layers. This is a big jump from the last section when all we had was just two dense layers at this scale. It helps to be able to see a summary of your model in some way. And so the torchsummary library allows you to do just that by calling the summary function and passing in the model and input size, you get a table that shows every layer in your model,

along with some other useful information, such as the output shape and the number of parameters in each layer. So first, we have our initial conv2d layer, which has 896 parameters, and its output shape is 32 by 32 by 32.

This is because we use the same mode convolution. We can do a quick sanity check to make sure that the number of parameters makes sense. Our filter is three by three by three by 32. That's because it has three input channels. Its size is three by three and it has 32 output channels. And so three times, three times, three times 32 is 864.

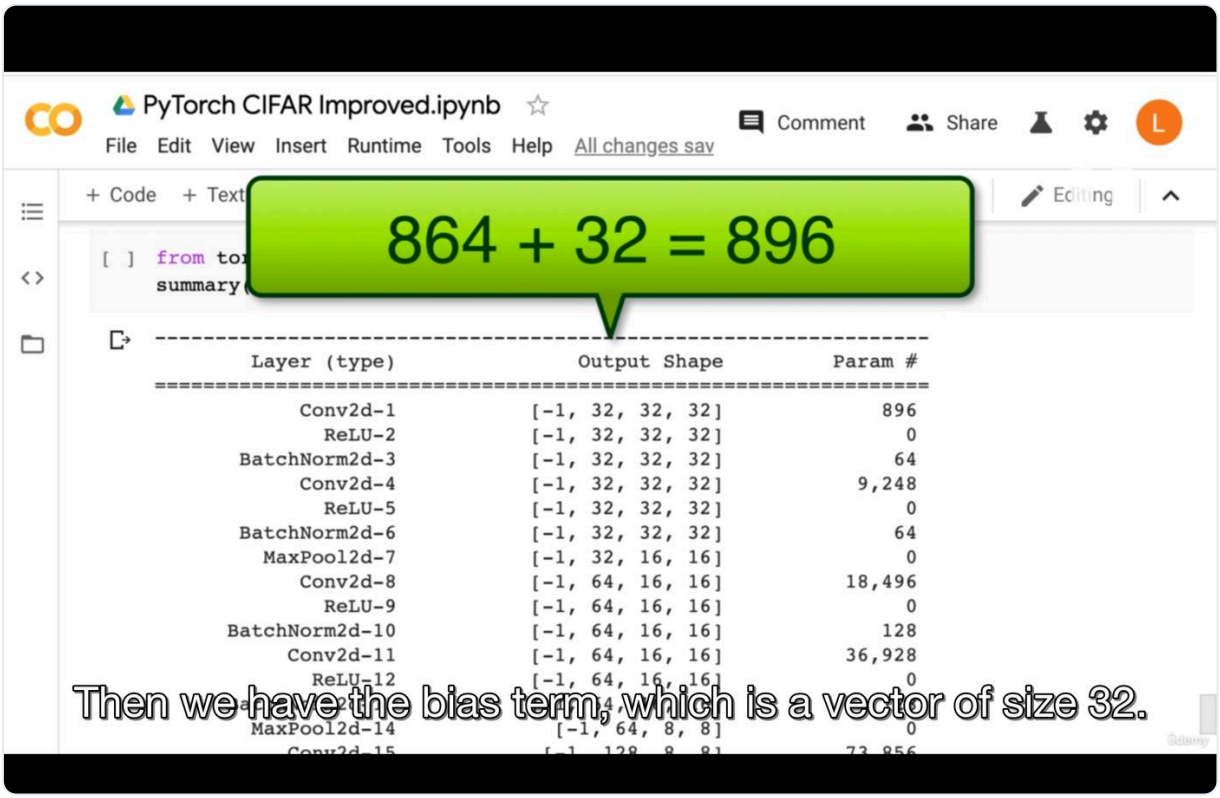


FIGURE 7.3

Then we have the bias term, which is a vector of size 32. So 864 plus 32 is 896, which checks out. If we scroll down to the end, we can see the total number of parameters in the model for this neural network, it's about two point four million. So, again, a big jump from what we were working with previously.

LECTURE 46

Shape of a Sequence

In this section of the course we are going to focus on a new topic sequence data although deep learning initially became popular for being really great at working with images. It was not long until Deep Learning also proved its dominance with sequences as well.

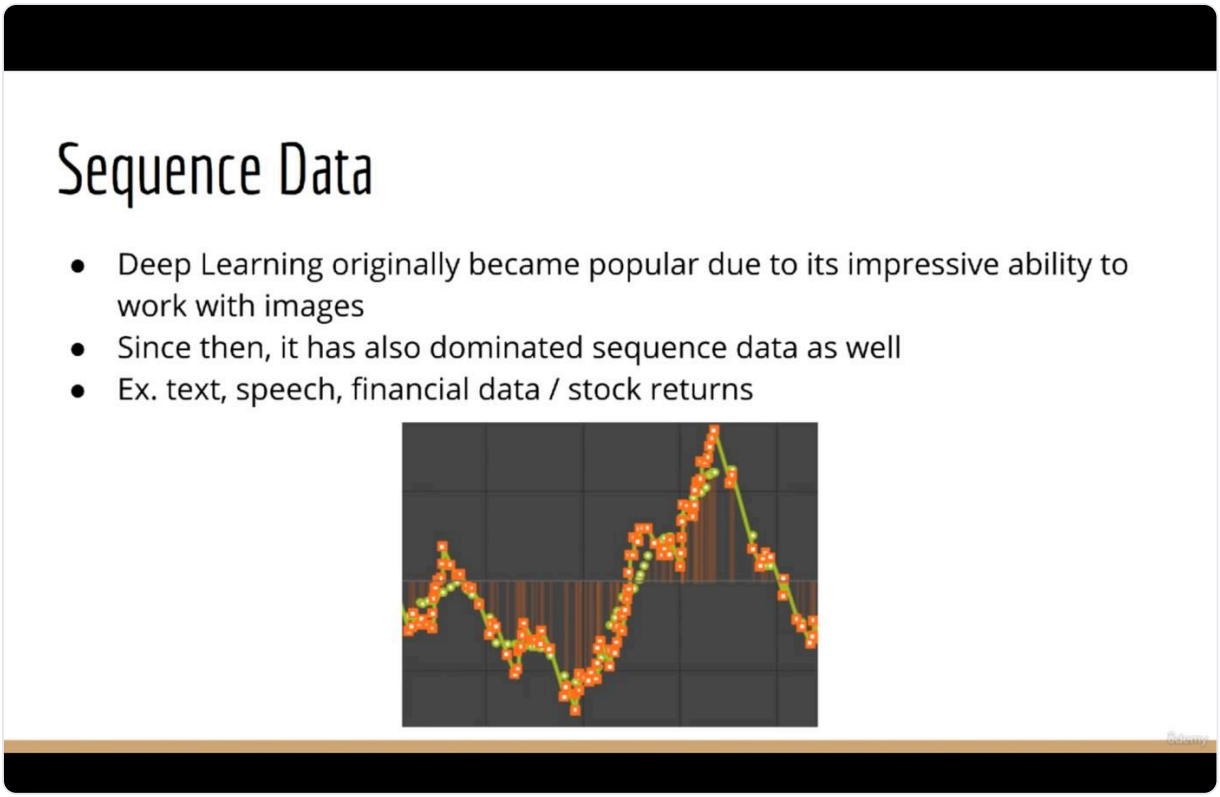


FIGURE 8.1 | Sequence Data

Some examples of that are text speech and even financial data such as stock returns. In this lecture we are going to discuss why we should model sequences and also look at some examples so let's start with the simplest kind of sequence data. At least this is what most people think of when they think of a sequence that is a time series signal a time series signal can be any continuous value measurement taken periodically.

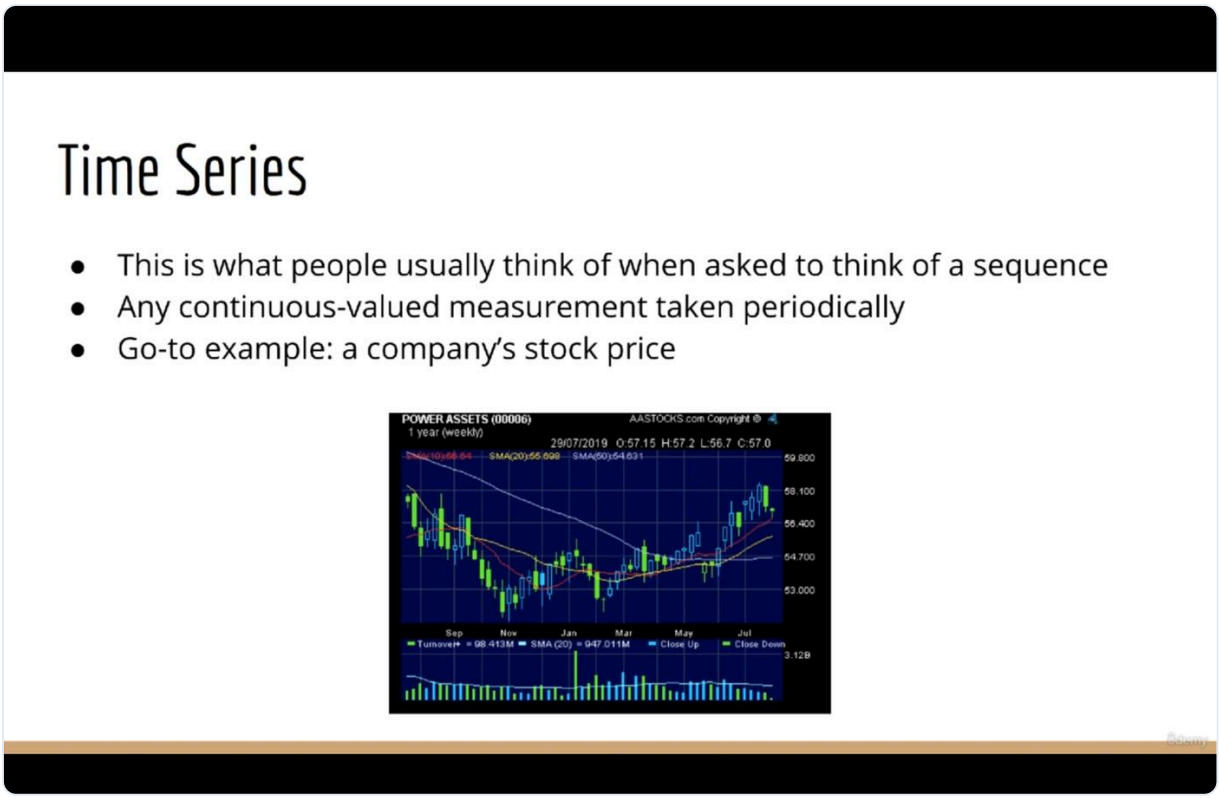


FIGURE 8.2 | Time Series

For example a company's stock price is probably the go to example Another famous example is the airline passengers data set. This is actually more practical than you might imagine.

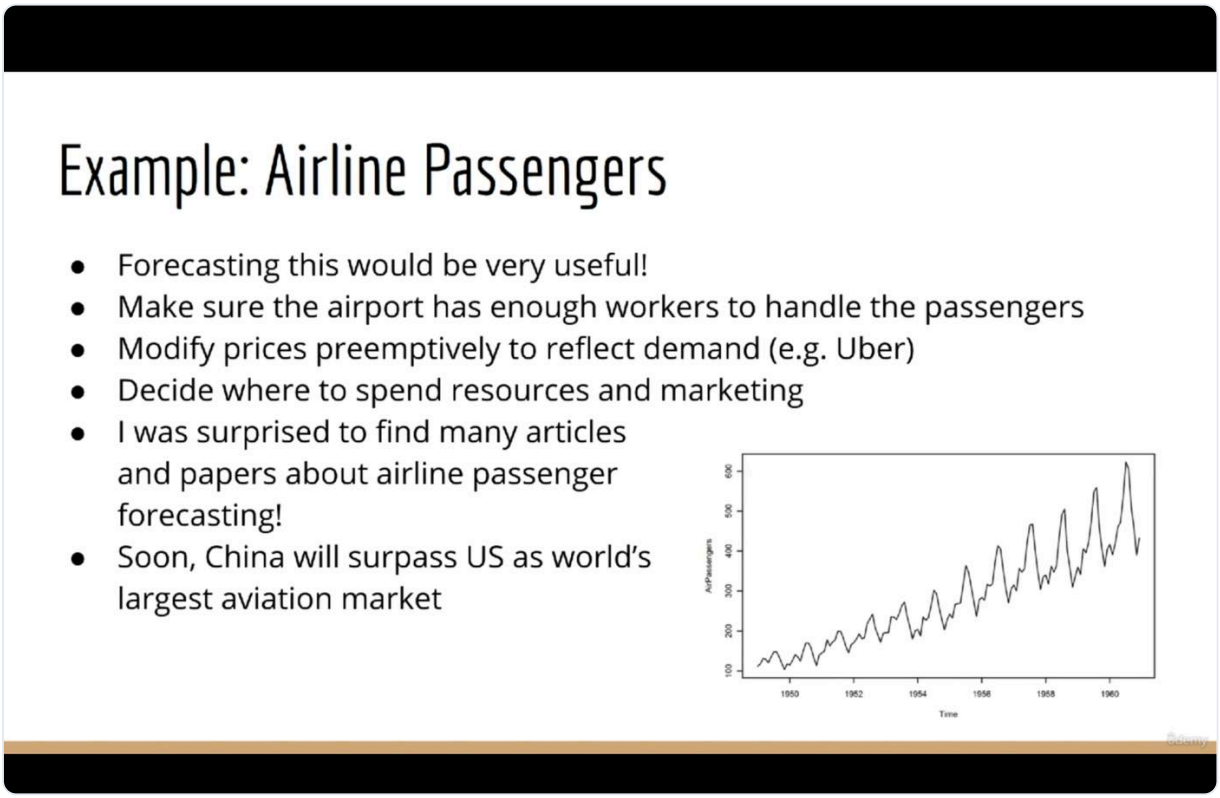


FIGURE 8.3 | Example: Airline Passengers

If you can forecast the number of passengers you will have on your airline you can do all sorts of things such as making sure the airport has enough workers to handle all the passengers. You can modify your prices to reflect demand. In fact that's what people do. Uber surge pricing is an example of that. It could also help airlines decide where to put their resources and marketing actually forecasting airline passengers is a surprisingly big deal. There are all sorts of articles and papers written on this topic. I just found out that China will soon surpass the United States as the world's largest aviation market. Another example of Time series is weather tracking.

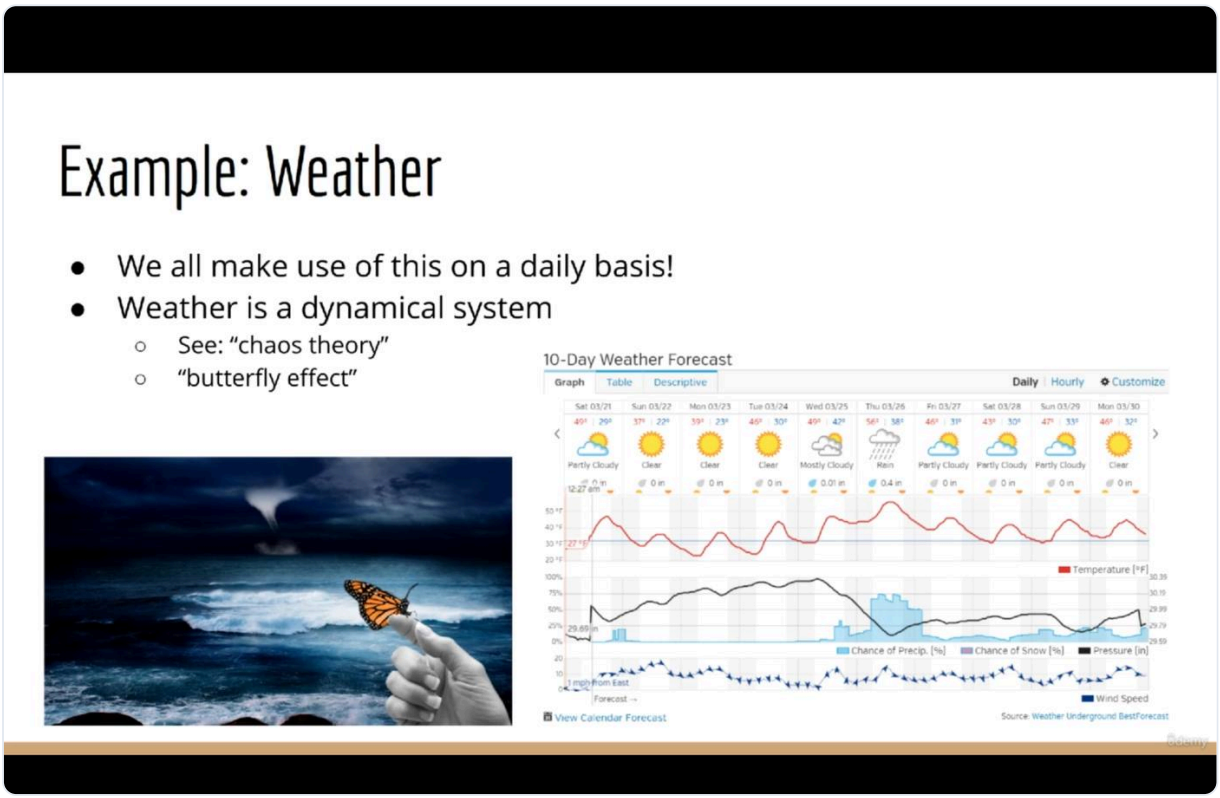


FIGURE 8.4 | Example: Weather

This is something we all probably make use of on a daily basis. You don't want to go outside without your umbrella. If it's going to rain. One interesting fact about weather is that it's a dynamical system that probably sounds complicated but some terms you probably have heard of are Chaos Theory and The Butterfly Effect. This is that even if we have the exact deterministic equations to describe a weather system our forecast will still become more and more incorrect. The further into the future we try to predict that's pretty counterintuitive because you would think if you have the exact equation for something then you should be able to calculate all the future values precisely but due to the butterfly effect this is not actually true.

As the saying goes a butterfly flapping its wings in Tokyo can cause a tornado in America small in decisions like numerical round off error and your computer will ultimately lead it to your weather forecast being completely wrong eventually. This is actually really relevant to us because when you think of time series and RNNs you automatically think How can I use a powerful model like an RNN to do time series forecasting another great example is speech or audio for which the obvious application would be speech recognition. Now I'm not sure how many of you are into audio engineering or music production but if you are then you should definitely recognize this as a time series.

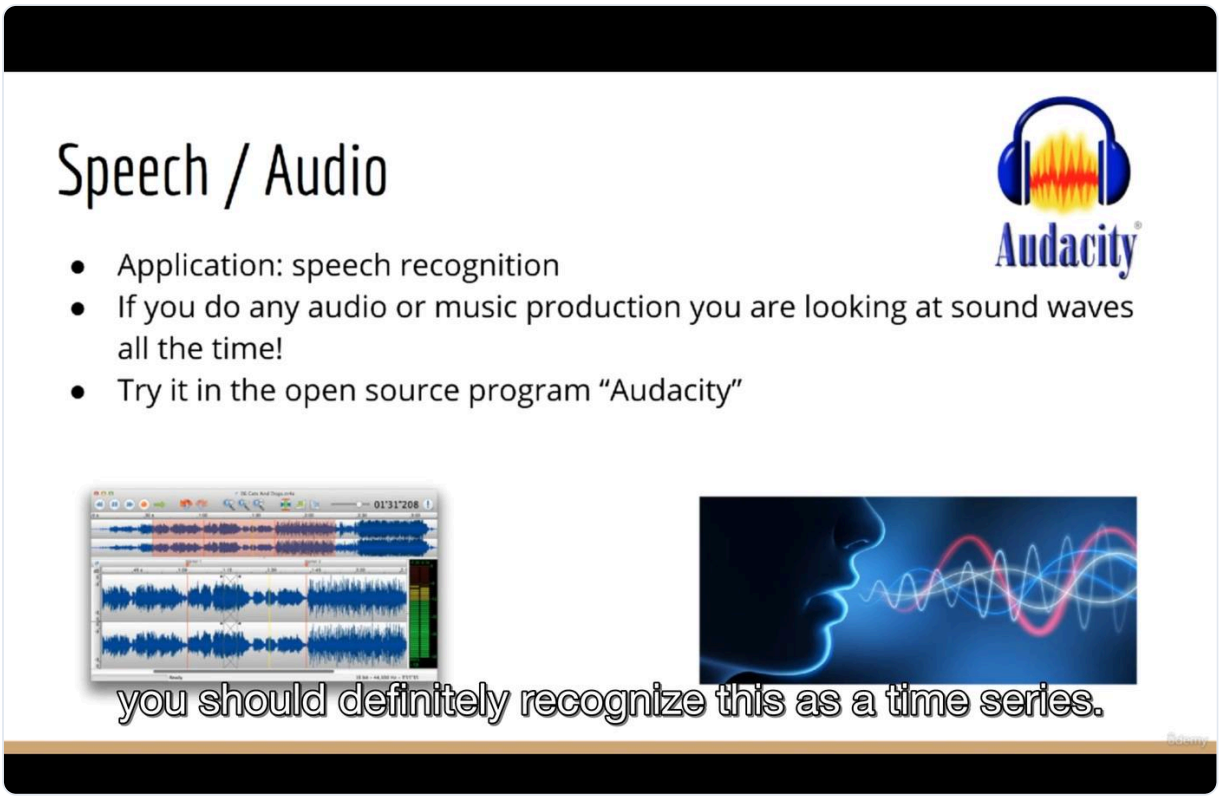


FIGURE 8.5

Just open up your favorite audio file in the open source program audacity and you'll see how it's just a time series of sound amplitude. So all in all time series are everywhere. Another type of sequential data is text which of course deep learning also excels at. The interesting thing about a text data is that it can be treated as a sequence but in machine learning. You don't have to.

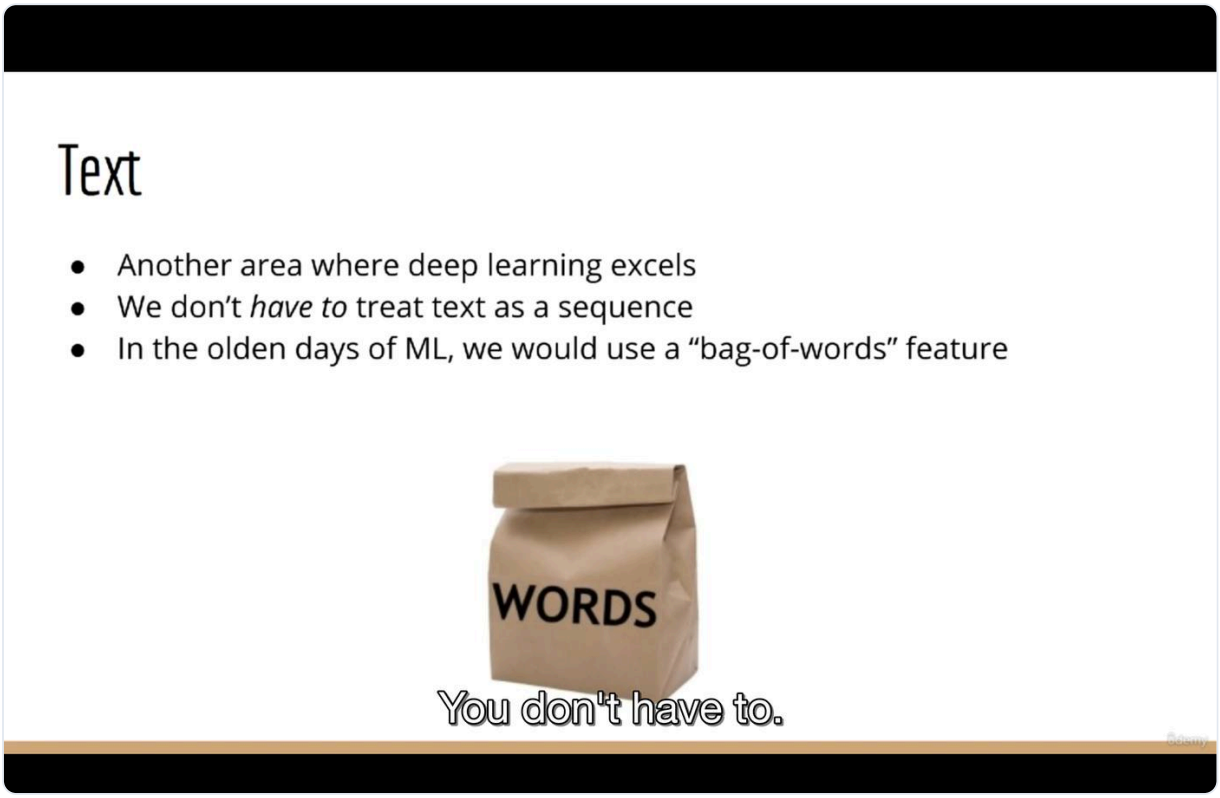


FIGURE 8.6

In the olden days of machine learning we couldn't deal with sequences very well. And so we resorted to more naive models the typical way of doing this would be to create what is called a bag of words feature vector basically it works like this. Suppose you're doing document classification so for example you take an email and you want to know is it spam and not spam. So what do you do. Well first let's take a big long feature vector with length equal to the number of words in the English language.

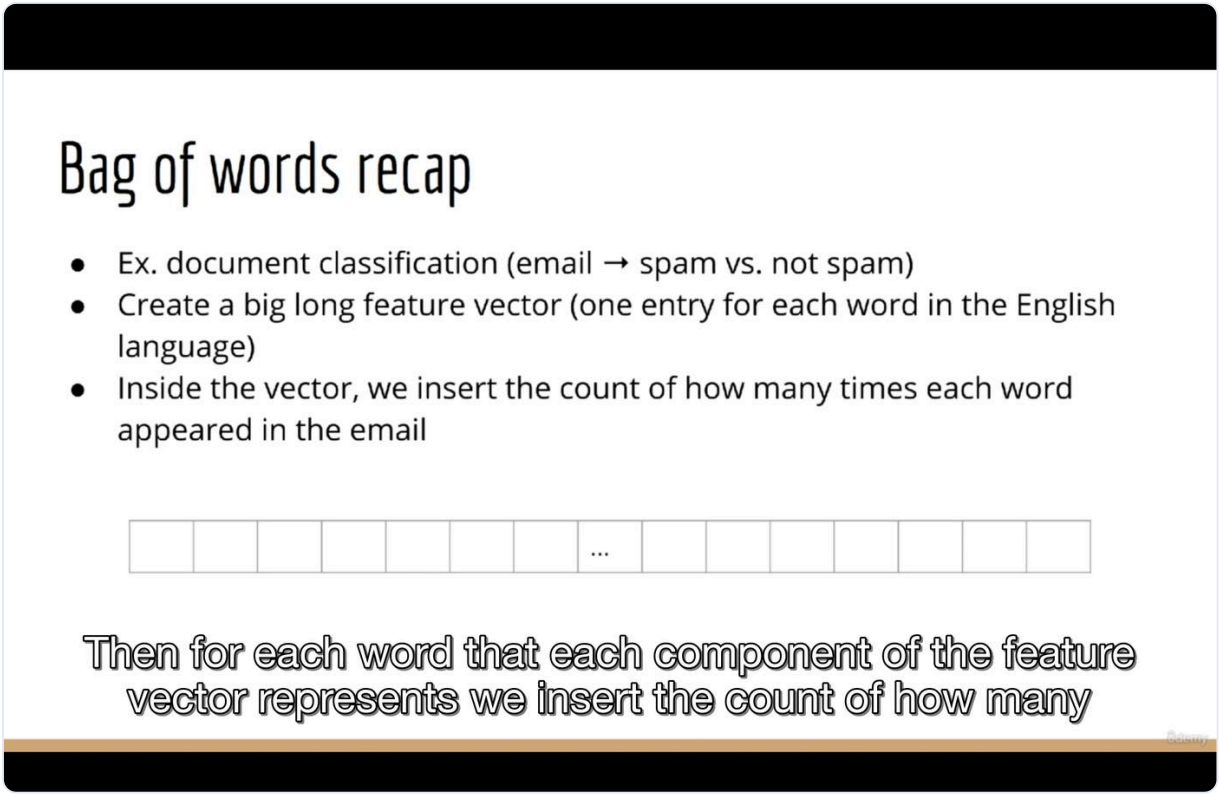


FIGURE 8.7 | Bag of Words Recap

Then for each word that each component of the feature vector represents we insert the count of how many times that word appeared in our email so suppose for simplicity's sake. There are only five words in the English language insurance loan pickles backpack and football. So now let's say our email contained the word insurance three times.

FIGURE 8.8 | Bag of Words Example

The word loan one time and the rest of the words zero times then our feature vector would be 3 1 0 0 0.

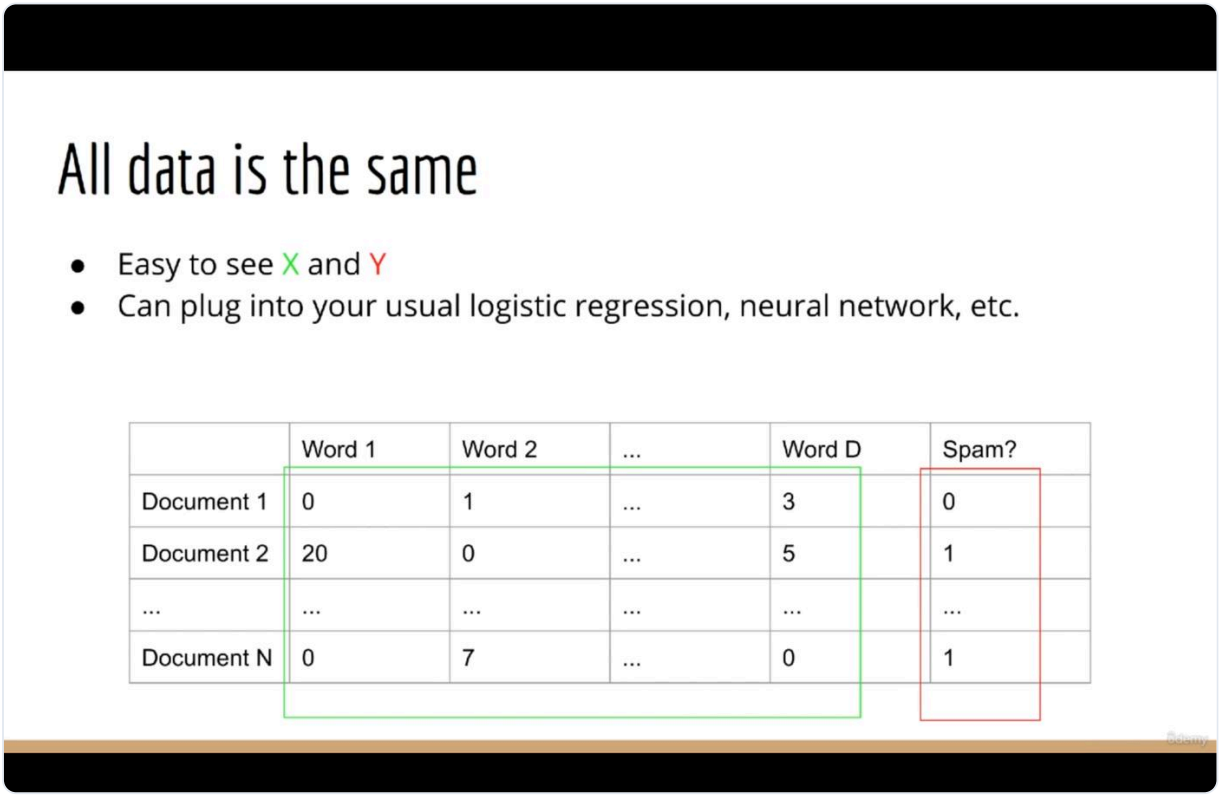


FIGURE 8.9 | All Data Is the Same

In this way we can convert every email in our dataset into a feature vector and then we get back to the situation where we can say all data is the same as our input x. We again have a table of numbers. They just happen to represent word counts for the labels is just 0 and 1 for spam or not spam. As usual you

can run any binary classifier on this such as logistic regression or a neural network but there's a problem with these bag of words representations when we only use word counts we lose information about the order of the words. Here's a simple example of why it matters.

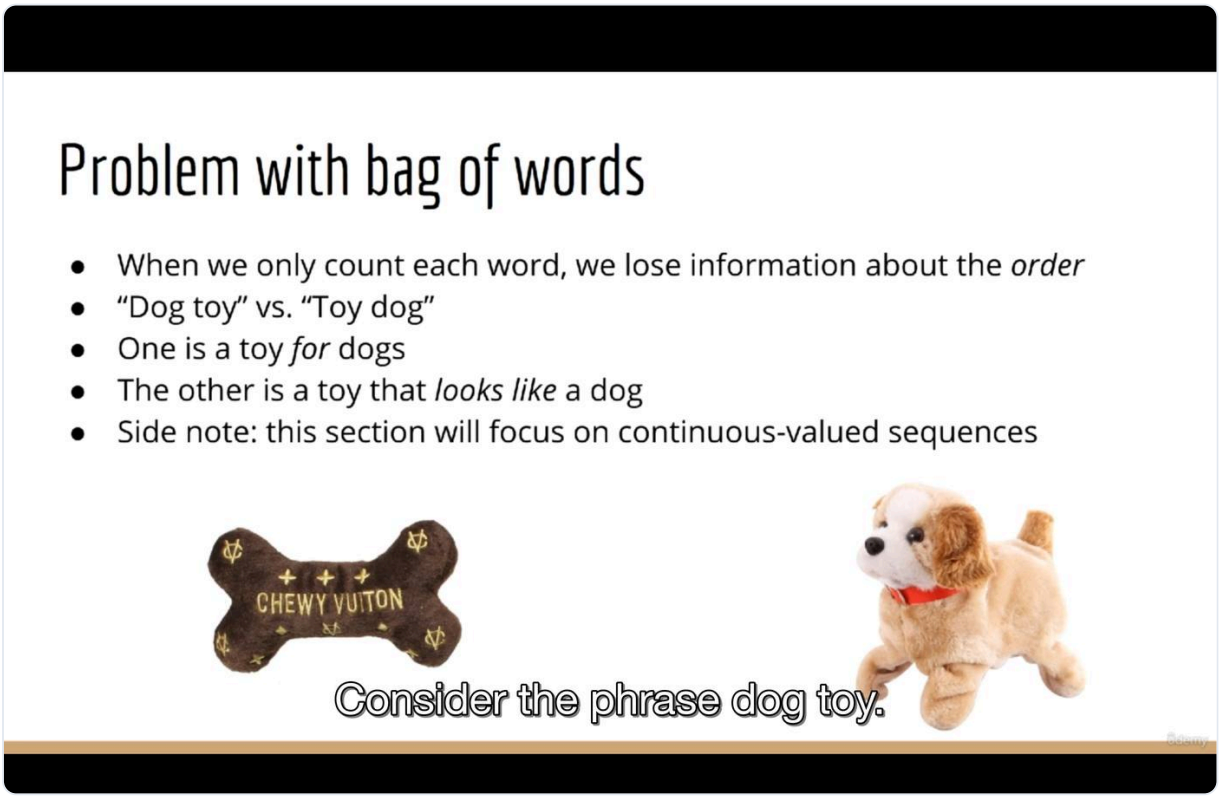


FIGURE 8.10 | Problem with Bag of Words

Consider the phrase dog toy. Now consider the phrase toy dog. Clearly these two phrases mean something completely different. And yet a bag of words model would treat them the same if you want to learn about how to deal with text that would be a major topic of the next section. But this section of focus mostly on modeling continuous valued sequences of data usually when we think of a sequence we think of something like what you see here.

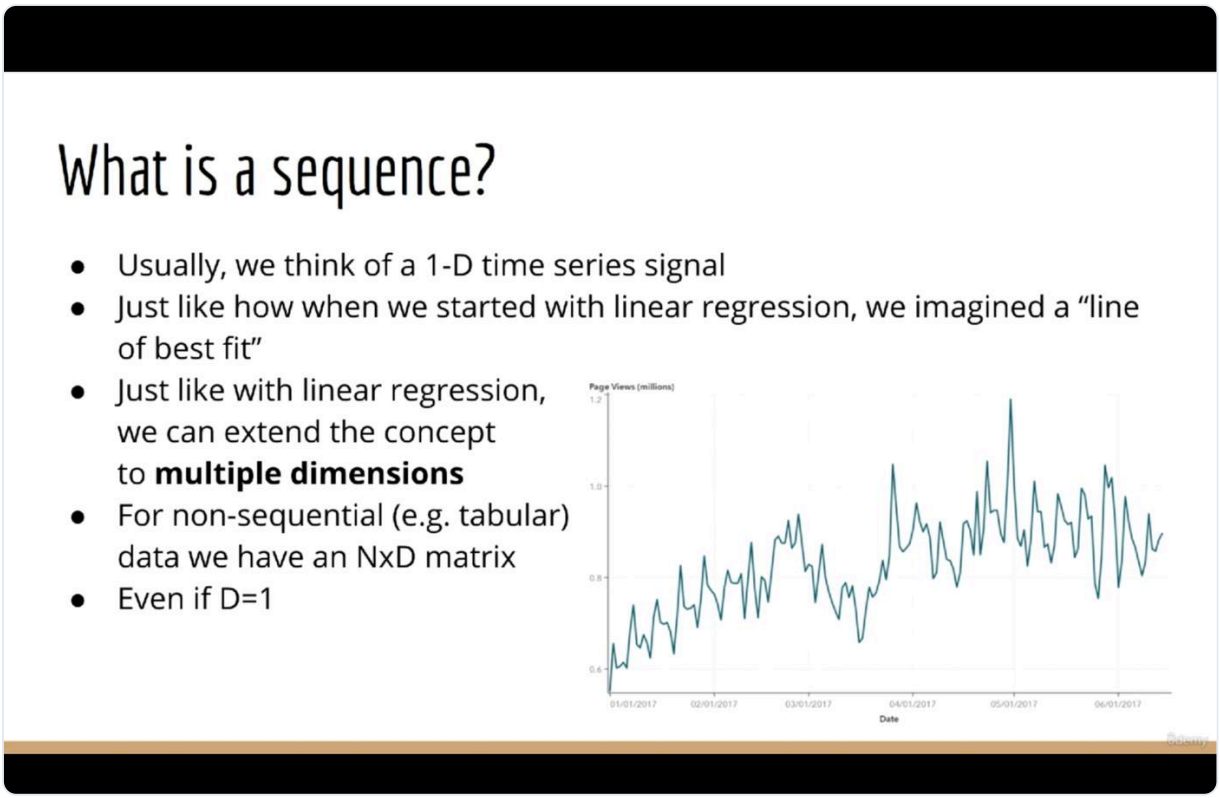


FIGURE 8.11 | What Is a Sequence?

This is just like how we started with linear regression in the simplest way to look at it is a line of best fit. This is the most basic thing we can do because we can see it so analogously for sequences we are going to think of one dimensional sequences because we can see them. And while this is technically a sequence we can extend this concept a little bit. Recall that when we were working with a non sequential data like what you saw with linear regression that in general the data was of shape N by D. And N is the number of samples and D is the number of features as usual even when we are doing one dimensional linear regression we still use a two dimensional array to represent the input data.

We still say it is N by D. It is just that D is 1 so far in time series are sequential data. Let's think about what the shape of the data should be. First consider just a basic sequence like a stock. How can we represent the length of the sequence. Should it be n should it be D. In fact it's not really either of these we don't want to consider each point in the sequence to be a sample nor do we want each point in the sequence to be considered a feature as a side note. We could do that but that is not general enough for RNNs.

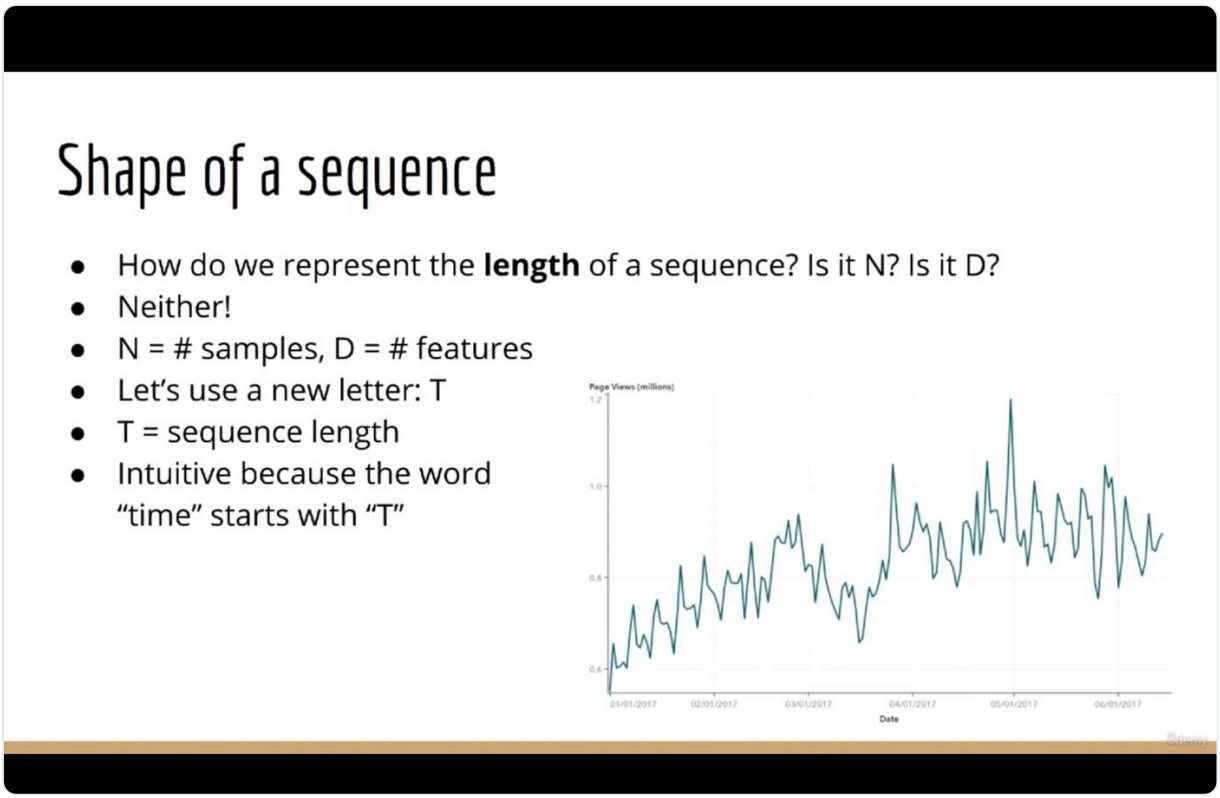


FIGURE 8.12 | Shape of a Sequence

So it's clear we need a new letter to represent the length of a sequence. How about the letter Big T. That seems pretty intuitive because the word time starts with a letter T so it makes sense. If we're dealing with a time series or any other continuous valued signal that the length of the signal should be denoted with a new letter T. Considering what you just learned. I'm going to tell you that the shape of our data when we're dealing with RNNs will have a shape N by T by D. Let's break down what this means. We know that N is the number of samples and D is the number of features and you've just learned that t is a number of steps in a sequence as you can see this is a three dimensional signal and is the first dimension t is the second dimension and d is the third dimension.

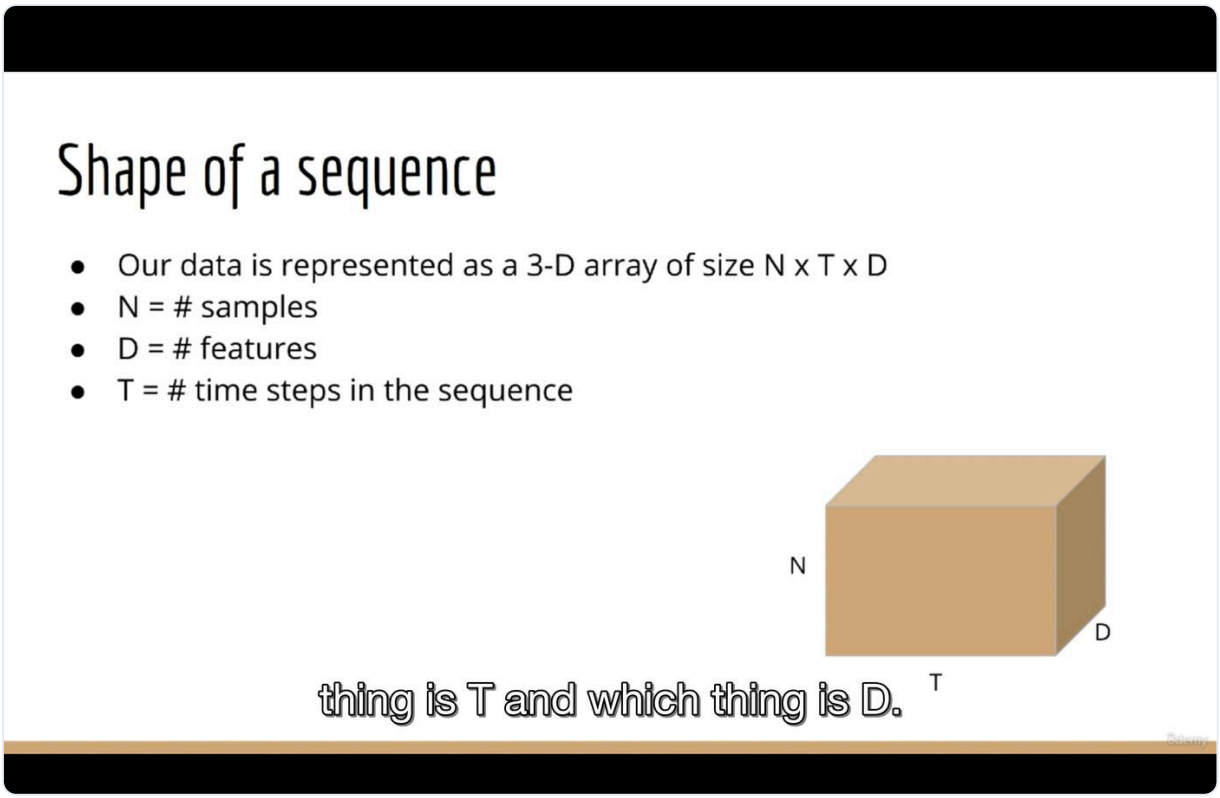


FIGURE 8.13 | Shape of a Sequence

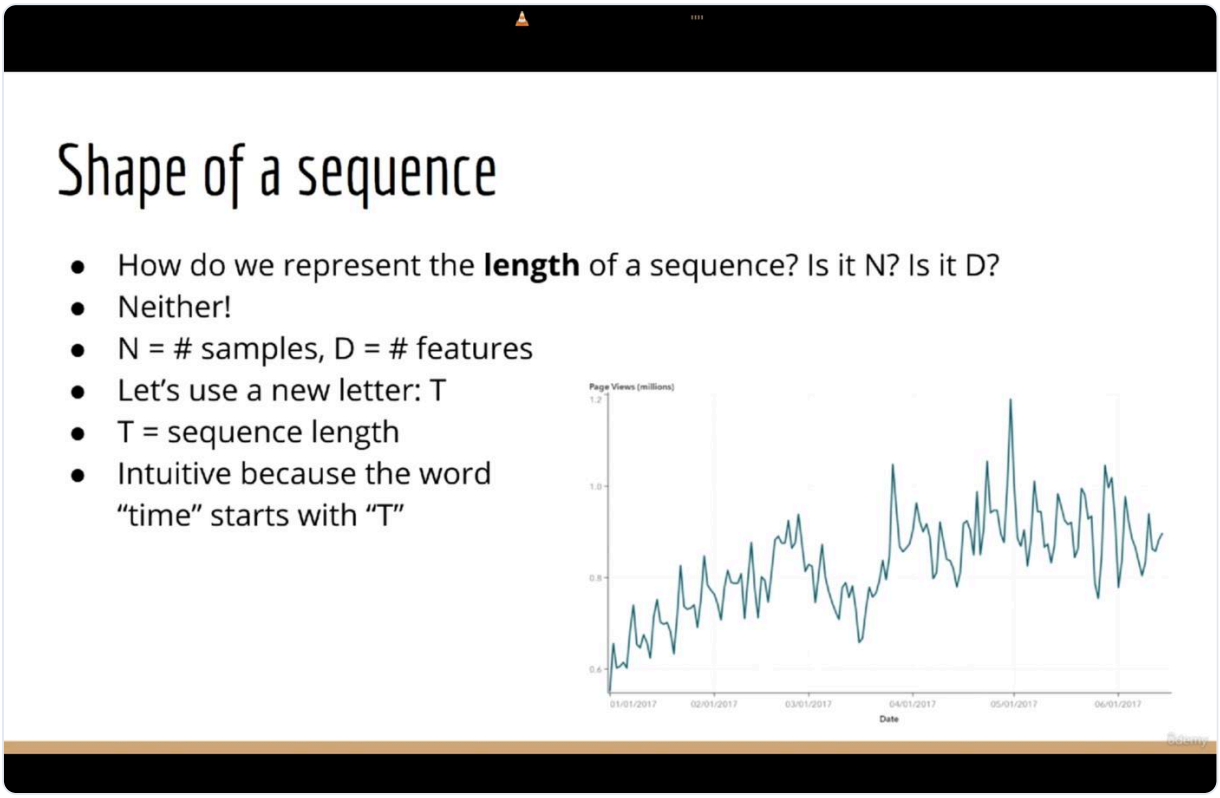


FIGURE 8.14 | Shape of a Sequence

When you're dealing with sequence data sometimes students can get confused about which thing is n which thing is T and which thing is D.

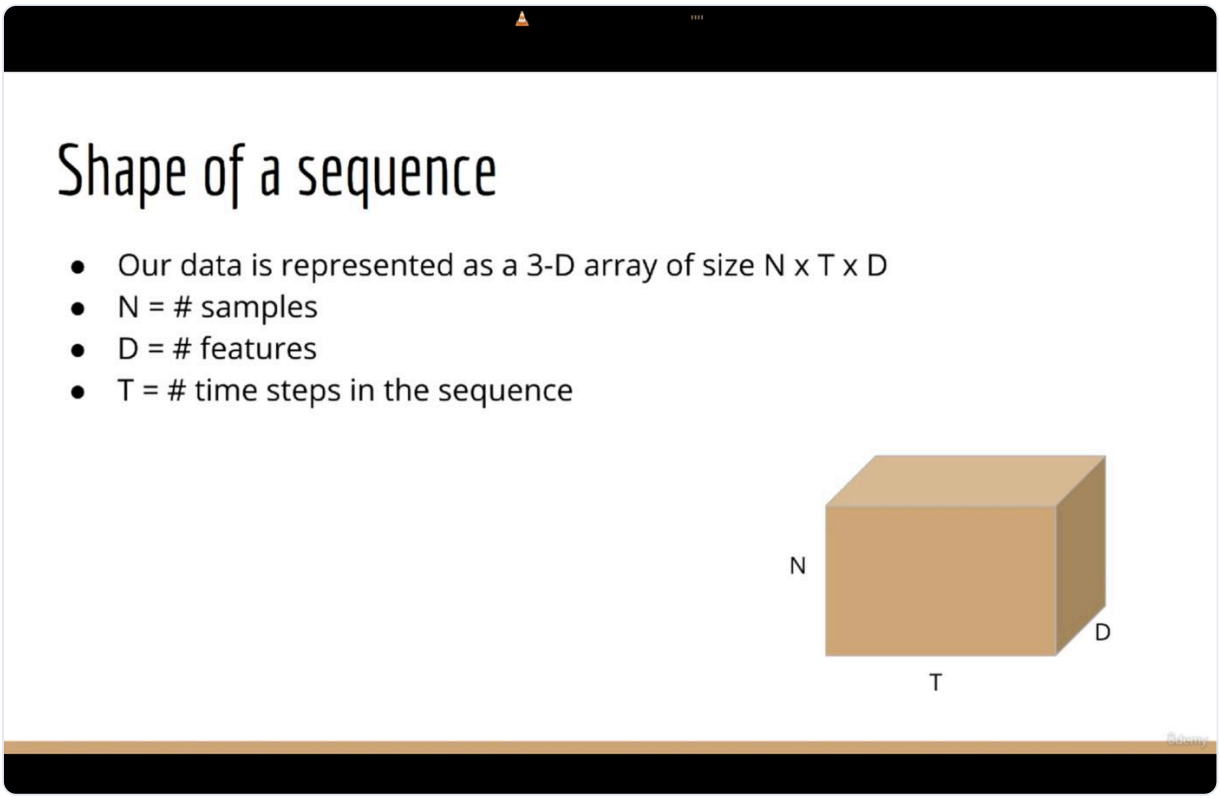


FIGURE 8.15 | Shape of a Sequence

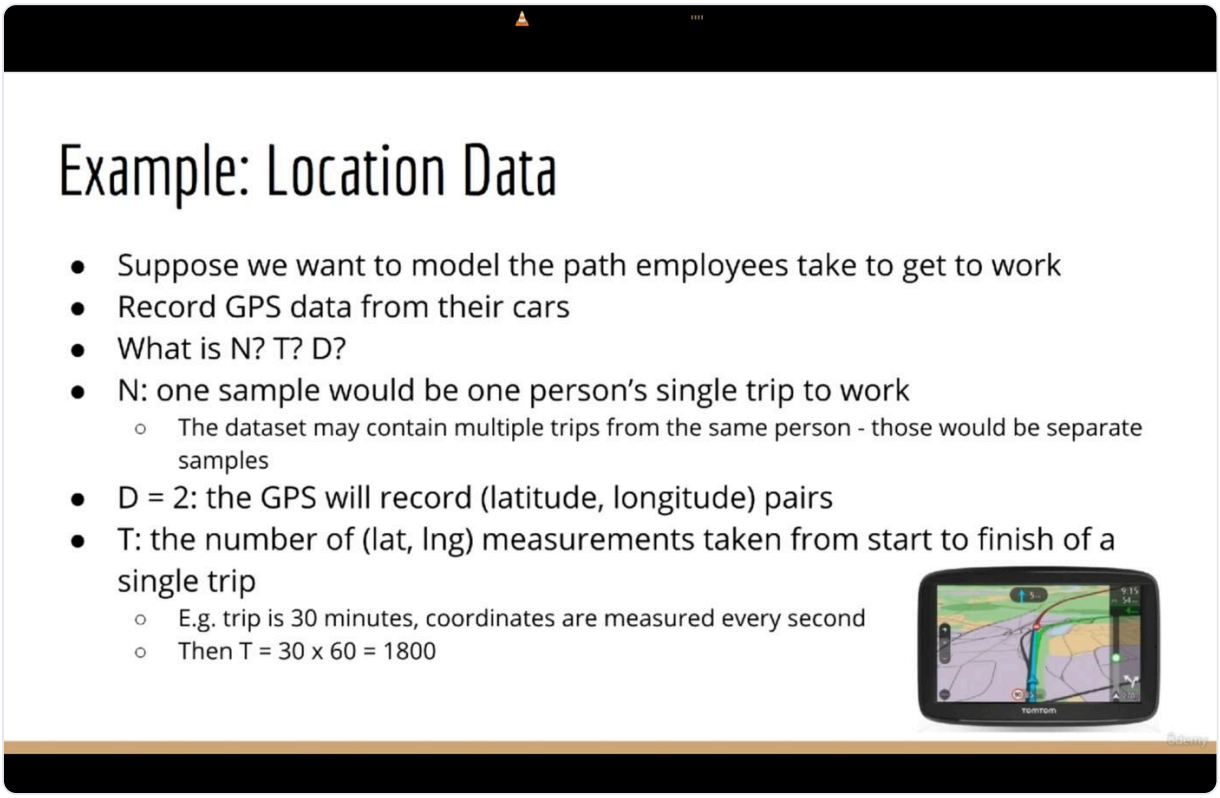


FIGURE 8.16 | Record GPS Data from Their Cars

So I think the best way to understand this is by example let's suppose we want to model the path that people take to get to work so you can record data from the G.P.S. In everyone's car. In this scenario what is n what is T and what is D. Well since we're modeling people's trips to work one sample will consist of one person's single trip to work. You might record some data over multiple days in which case that person could have multiple trips to work that would count as multiple samples. All right. So what's D. In

this case D is probably two because the G.P.S. Will give you both latitude and longitude measurements. Okay so what is T. T is the number of lat long measurements taken from the time the person embarks on their journey to work until they arrive at work.

So let's say it takes a person 30 minutes to get to work and you record at lat long coordinates every second in that case the number of steps you would have is 30 times 60 which is eighteen hundred. So the length of your sequence t would be eighteen hundred now you might be wondering doesn't it take each person a different amount of time to get to work and indeed you would be correct.

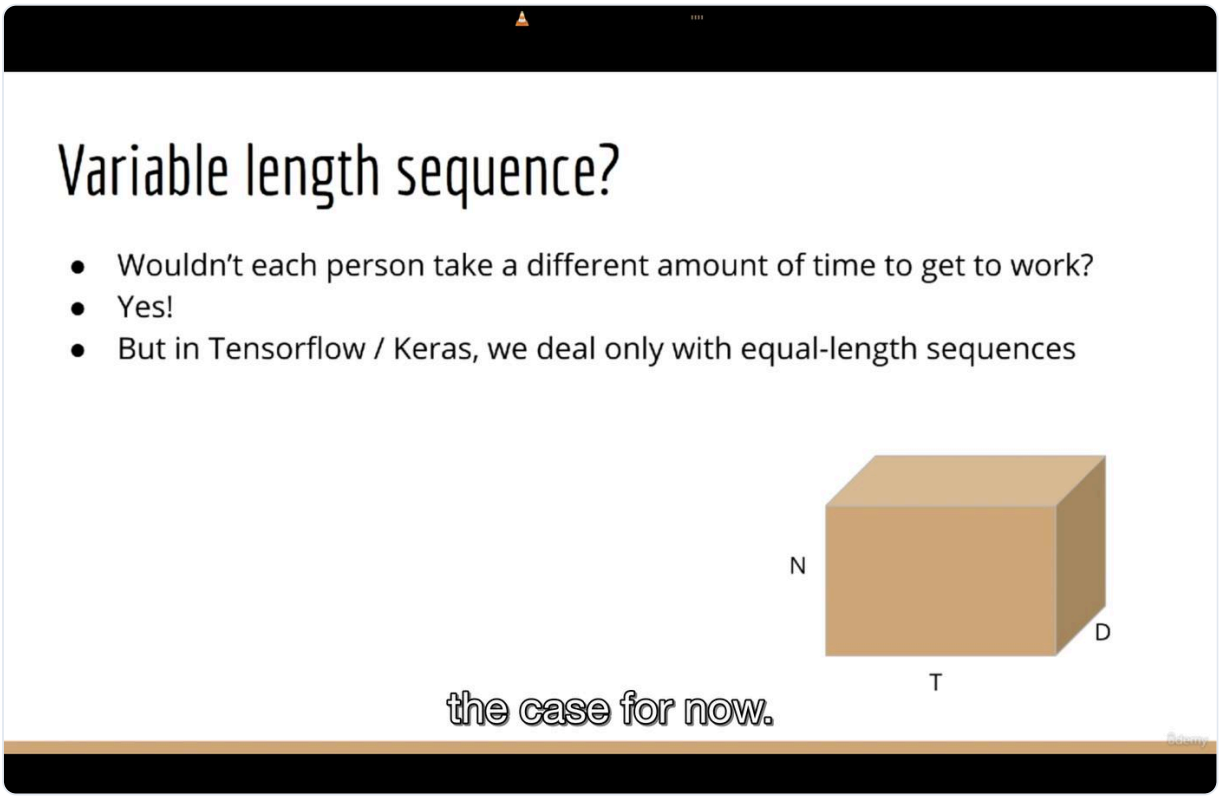


FIGURE 8.17 | Variable Length Sequence?

In TensorFlow and Keras we generally work with equal length sequences so just assume that this is the case for now. We'll discuss this in more detail later. For now let's think of more examples here's an example of where d equals 1.

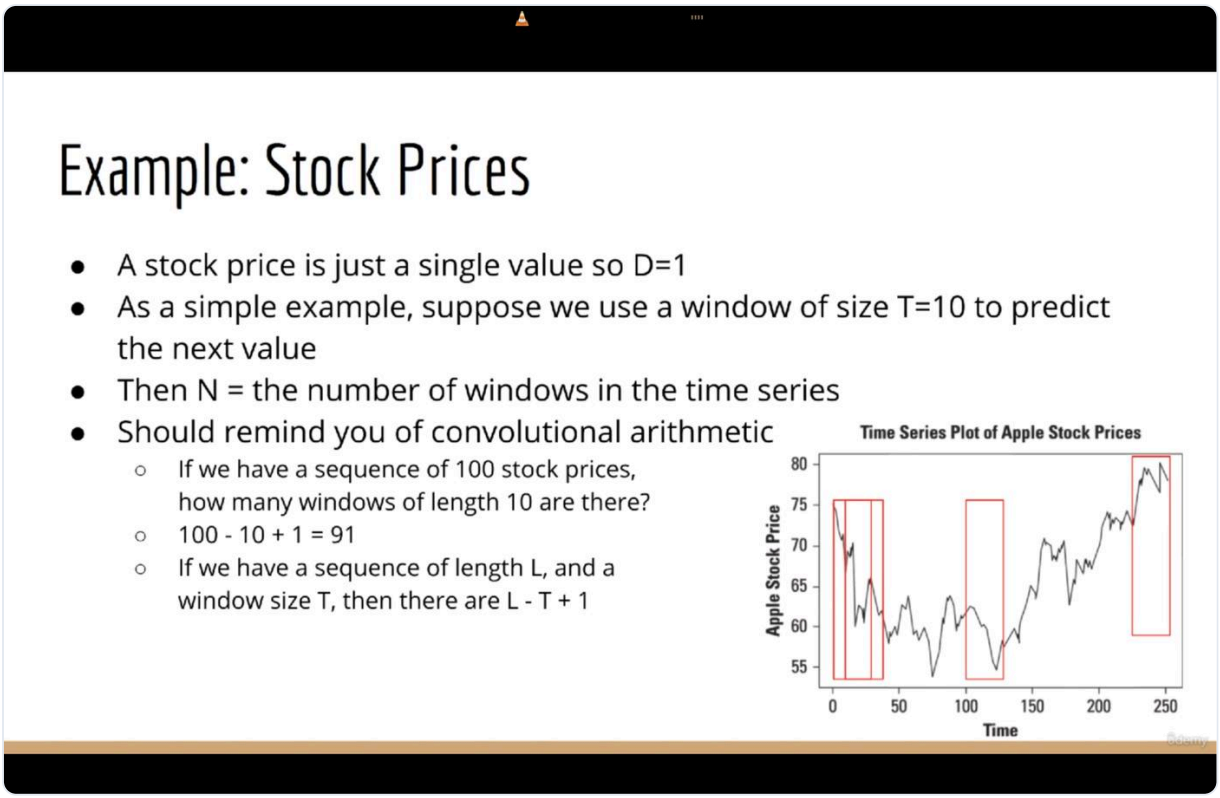


FIGURE 8.18 | Example: Stock Prices

So it's kind of like the simple case analogous to our one dimensional linear regression a simple scenario for this would be stock prices a stock price is a one dimensional signal the vertical axis is price and the horizontal axis is time. In other words to have a signal where D is bigger than one we would have more measurements per unit time for a single stock price. We just have one the price. So what would N be in this case. This is a complicated question actually but let's consider again as something simple. Let's say we want to break up our stock price signal into windows of length 10 so our task might be something like using 10 sequential measurements of the stock price.

Try to predict the next stock price. In this case N would just be the total number of windows of length 10 that we have by the way this should remind you of some of the convolutional arithmetic we did. If your total sequence of stock prices has say length 100 and you want to know how many windows of length 10 can be taken from the sequence. That is just one hundred minus 10 plus one which is ninety one. In other words if you have l consecutive measurements and you have a window of length t then the number of windows is L minus T plus 1 of course with stock prices. You might not want to measure just one stock. There could be multiple stocks. So here's an example with stock prices where D is bigger than one.

Imagine for example that you've taken measurements for 500 stocks just like the S&P 500.

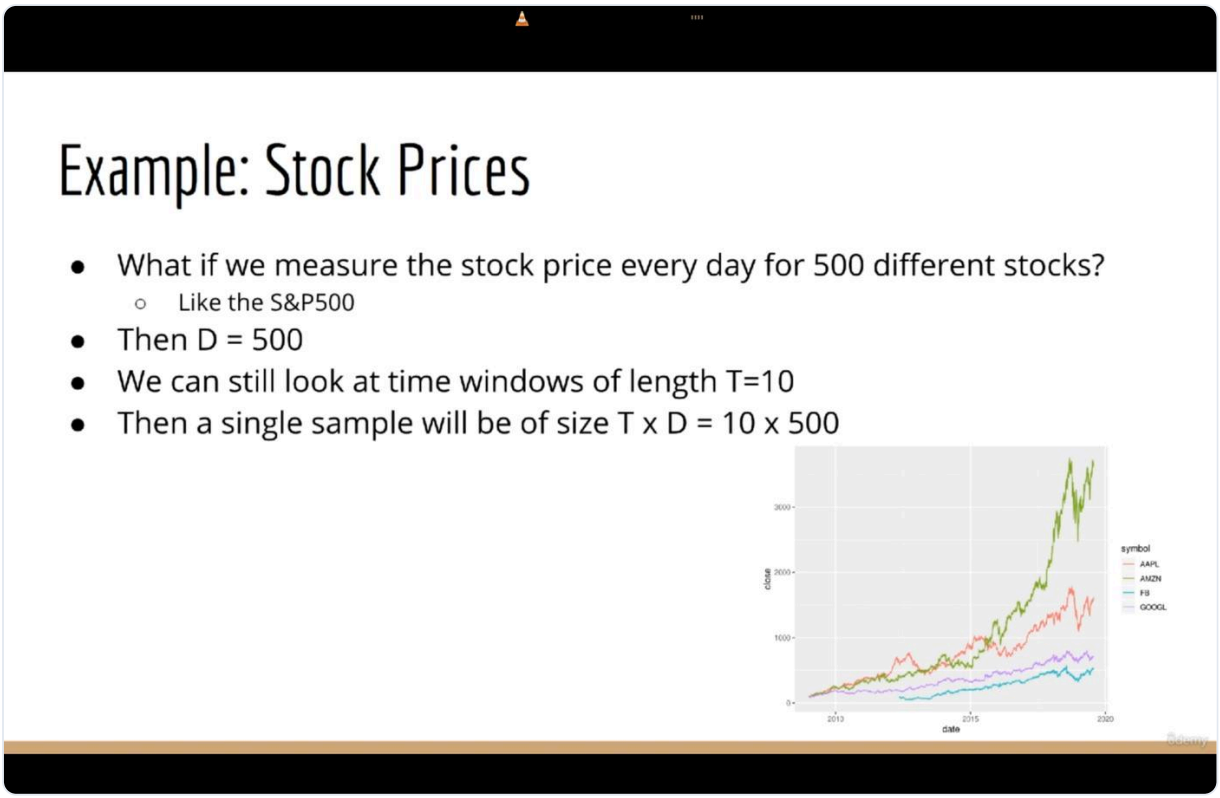


FIGURE 8.19 | Example: Stock Prices

Now you have the stock price for 500 different stocks over time. That means D equals five hundred you can again artificially make a window of length T and then the number of samples you have is just the number of these windows here's another example which is somewhat futuristic.

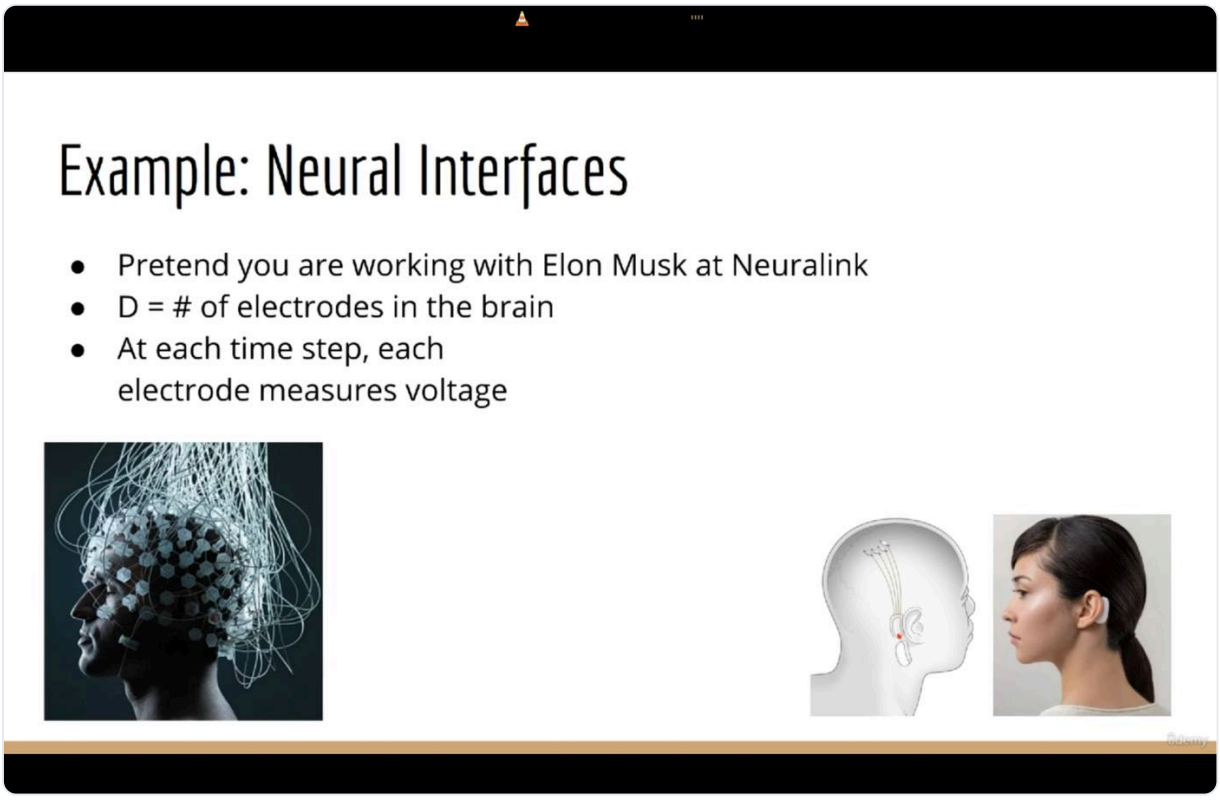


FIGURE 8.20 | Example: Neural Interfaces

Imagine you are working with Elon Musk at Neuralink and you record voltages from electrodes placed in the brain. In this scenario D is equal to the number of electrodes at each point in time you're going to have measured these different voltages at different locations in the brain. One of the early applications of brain computer interfaces was moving a mouse cursor on a screen and typing. In other words everything you need to use a computer. So suppose we are trying to predict to the letter or number you are thinking of or want to type. And this is based on the voltages recorded from your brain for one second. As you think of that letter a number. Now let's say measurements are taken from the electrodes at a sampling rate of one sample per milliseconds.

That's 1000 samples per second. That means for one second in the time we make the recording of you thinking of the letter we've collected 1000 measurements at each electrode. So T equals 1000 and N as usual would be the number of letters that you thought of. So if we did a test where you had to think of a series of letters and collected at ten thousand one second recordings.

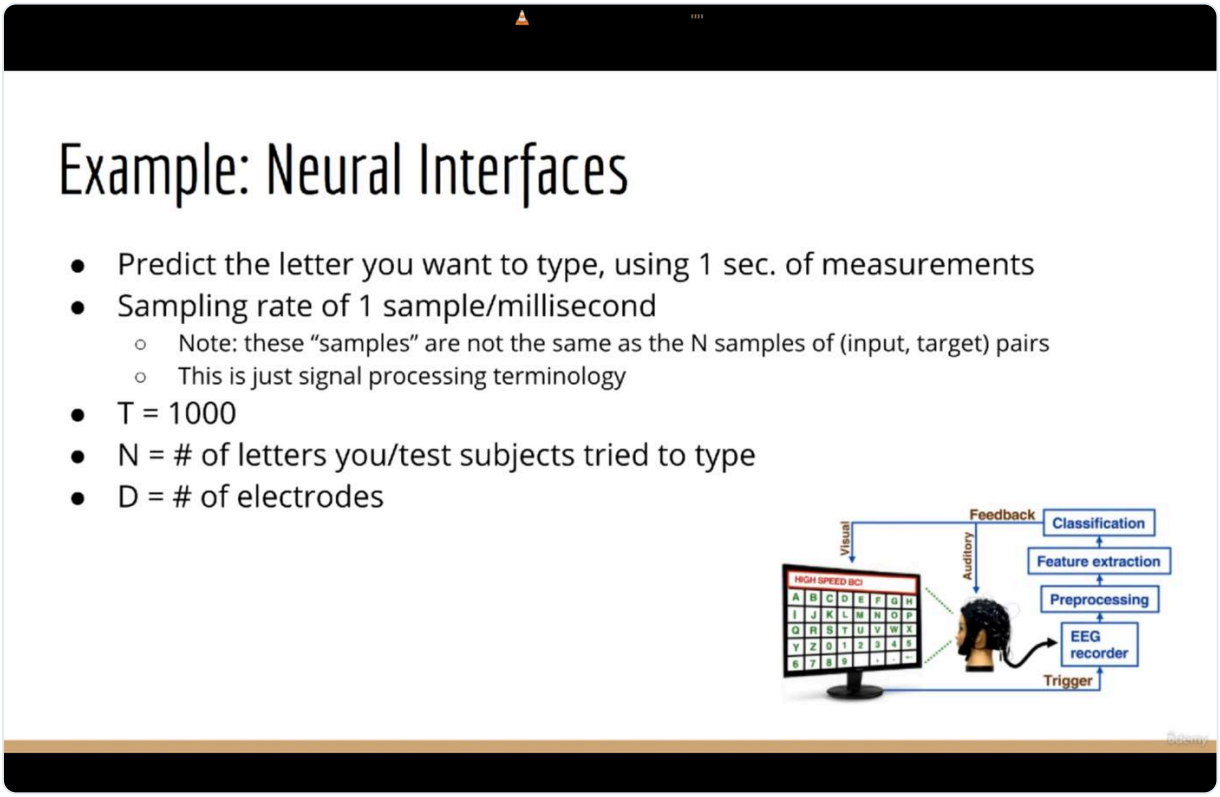


FIGURE 8.21 | Example: Neural Interfaces

That means N equals ten thousand one question you might be pondering is why is our data of shape. And by T. By D. Why not. And by D by T or some other order and in fact there is no particular reason it needs to be this way except for the fact that all Python libraries for machine learning conform to the standard. This is just like our tabular X data where we have N by D. Or our image data where we have n by H by W by C.. This could just as easily be d by N or n by C by h by W. In fact we've already seen that in PyTorch and Theano. The convention is N by C by h by W.. So this stuff is not set in stone.

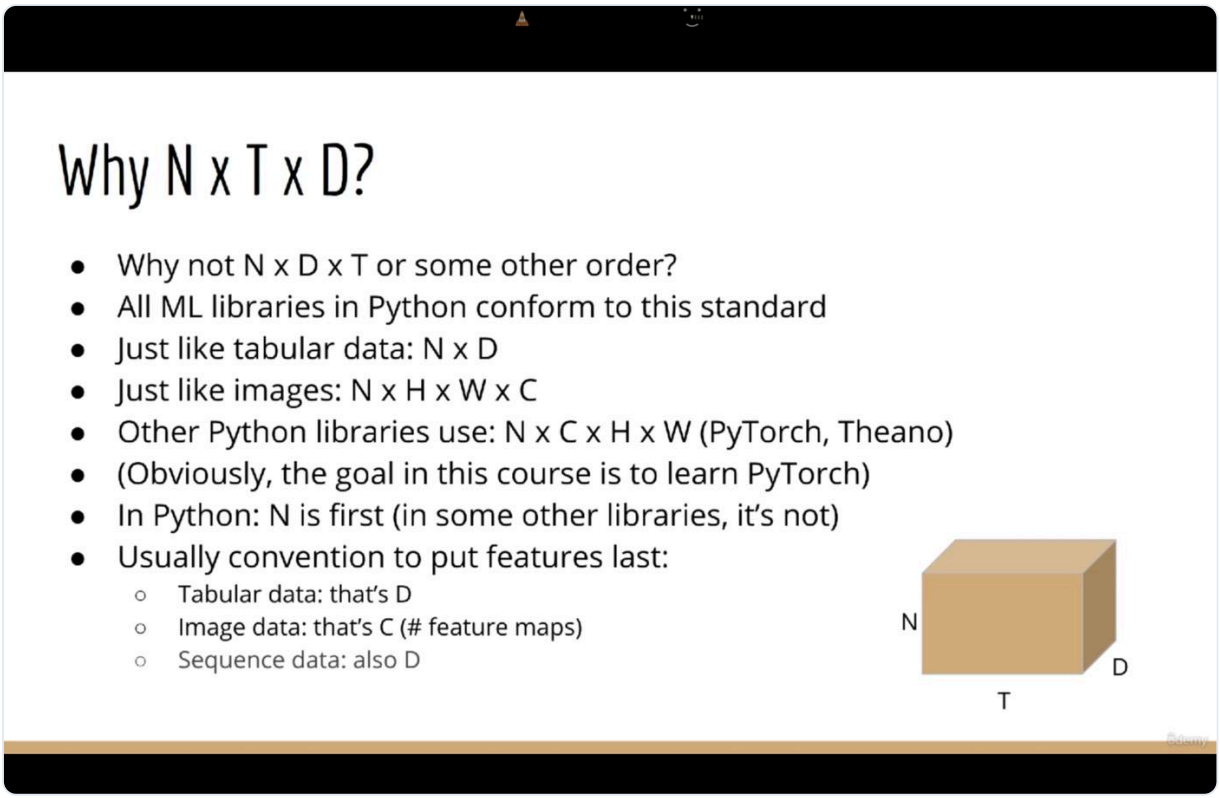


FIGURE 8.22 | All ML Libraries in Python Conform to This Standard

My goal in this course is to show you how we do things in PyTorch in Python the general pattern is that we always put an N first. So that should go without saying. There are certainly other libraries which by convention do not put N first the other convention is that we put the number of features last. So that is why we have N by D. Where D is the number of features. This also makes sense for images where we have end by h by W by C because C is the number of feature maps which is the analogous version of features for image data. And this also makes sense for sequence data because this D still represents the number of features by the way.

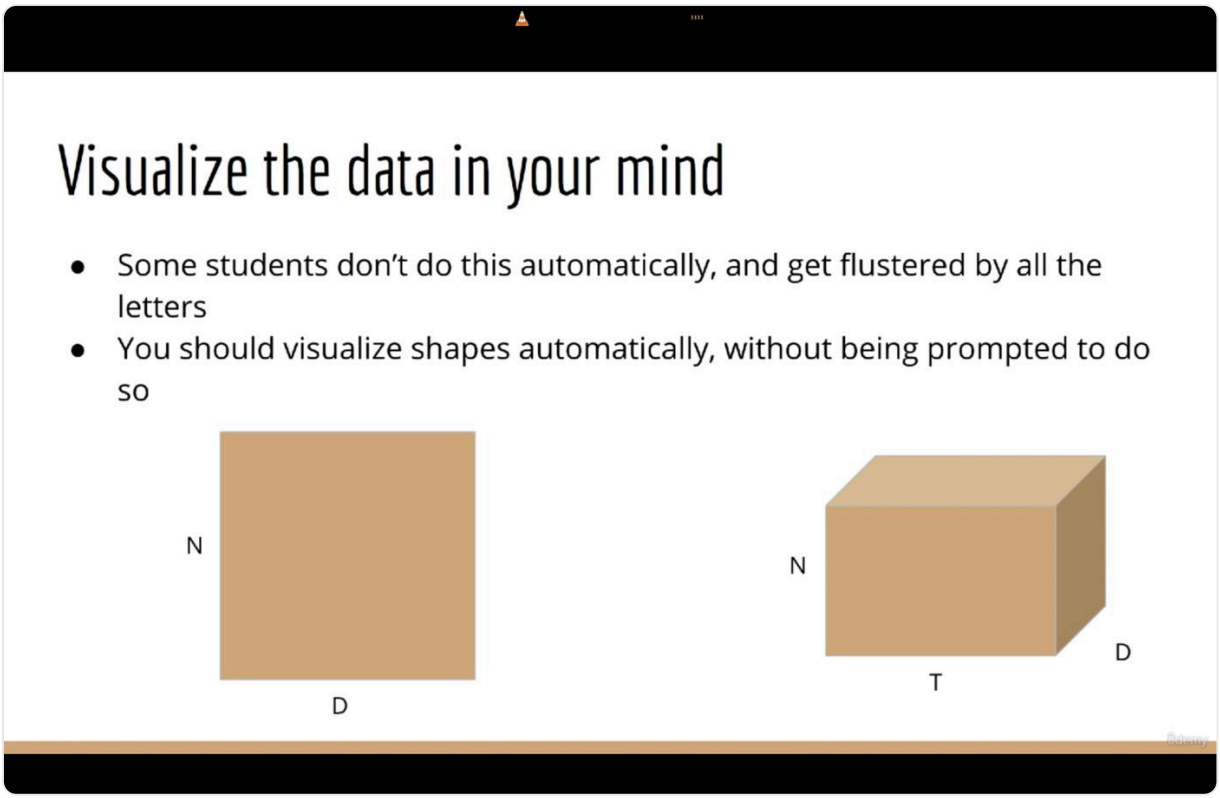


FIGURE 8.23 | Visualize the Data in Your Mind

One important thing to do is visualize the shapes of the data in your mind before where we had N by D inputs we visualize that as a rectangle rectangles are two dimensional so that makes sense. Now we have a three dimensional object so it is no longer a rectangle it's a box. So when you hear me say and by t by D you should automatically think of a box without needing me to tell you to do so. This should be very helpful as you write your code and reason about how RNNs work as promised. One thing we have to consider is what if we have a variable length sequences. It's clear that this is a pretty common scenario. Just think of sentences not all sentences have the same number of words some sentences have one or two words and other sentences have one hundred words.

This is actually a very interesting question. In the past I've written code that can handle sequences of variable length. The problem is this gets complicated fast. Another problem with this is that it means you'll be using inefficient data structures. If you have an N by T by D box where T is constant then we can store this in a single NumPy array that is efficient because NumPy was written to be efficient. But if T is variable and depends on which sample we are looking at we might use the notation T sub n instead. Since t depends on the sample n in this case we cannot store this in a single NumPy array.

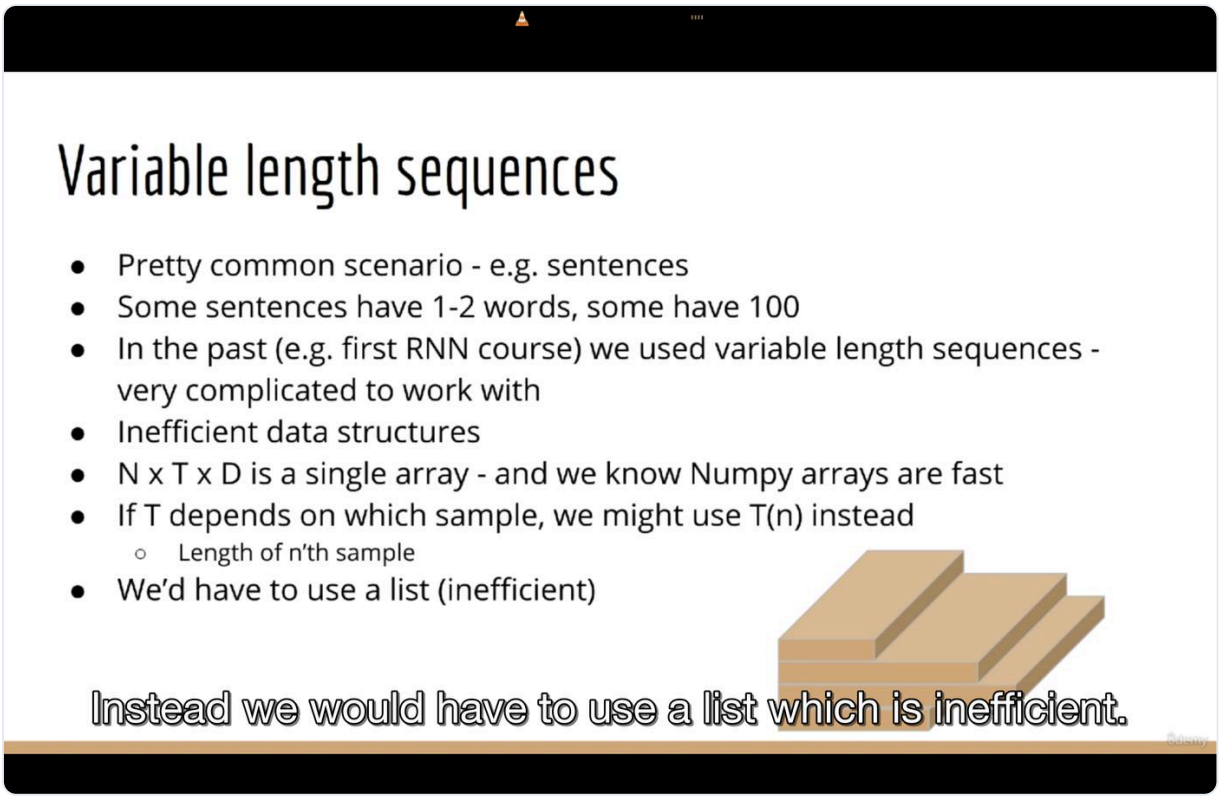


FIGURE 8.24 | Variable Length Sequences

Instead we would have to use a list which is inefficient. On the other hand there are some inefficiencies with constant Length Sequences as well.

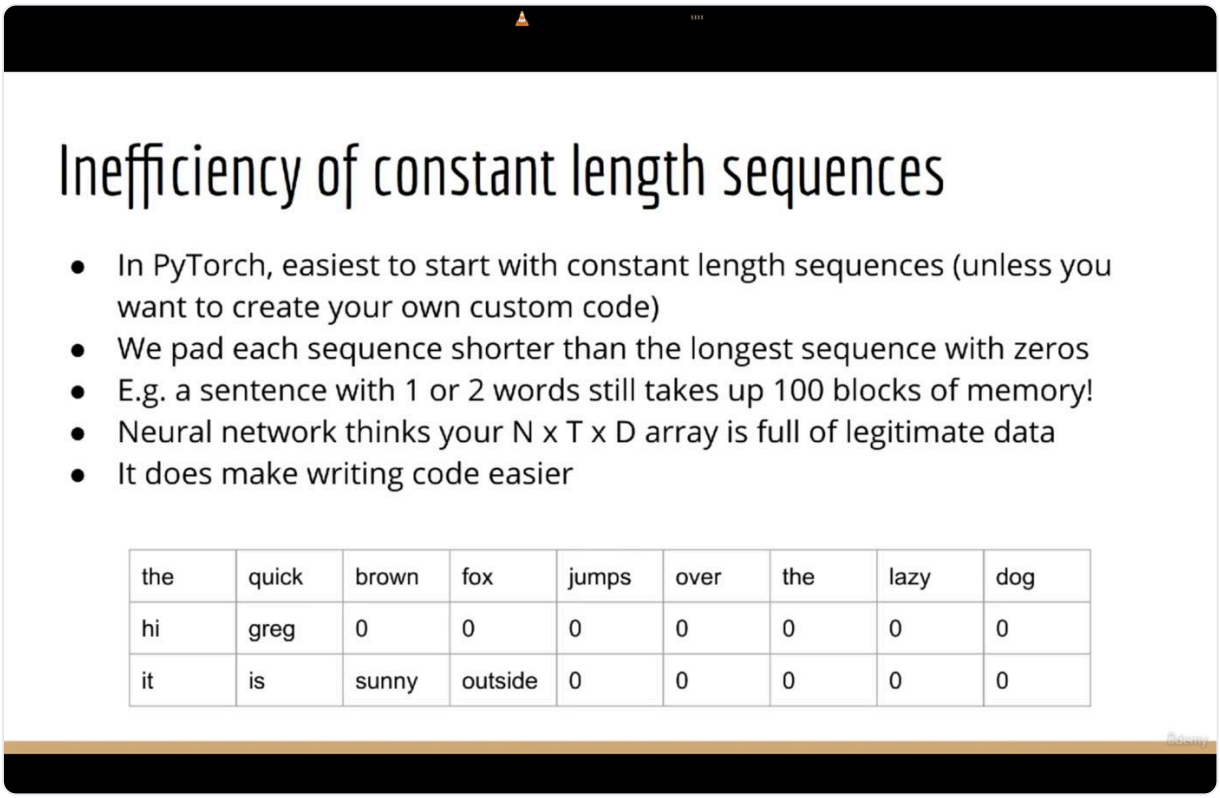


FIGURE 8.25 | Inefficiency of Constant Length Sequences

In PyTorch it is easiest to start with constant length sequences so that's what we'll be doing in this course. This is the case with other modern Deep Learning libraries as well such as TensorFlow and Keras. If we do end up having variable length sequences as input what we can do is pad the sequence with zeros so that they all have the same length as the longest sequence for example if your longest sequence has one hundred words then even sentences with one or two words would be the first one or two words and then 98 or 99 zero values as you can imagine. This is going to take up a lot of unnecessary space and processing time because the neural network still thinks your entire N by T by D box is full of legitimate data simultaneously.

It does make coding RNNs much easier. So it's a tradeoff. Ultimately this is the way we do things in PyTorch.

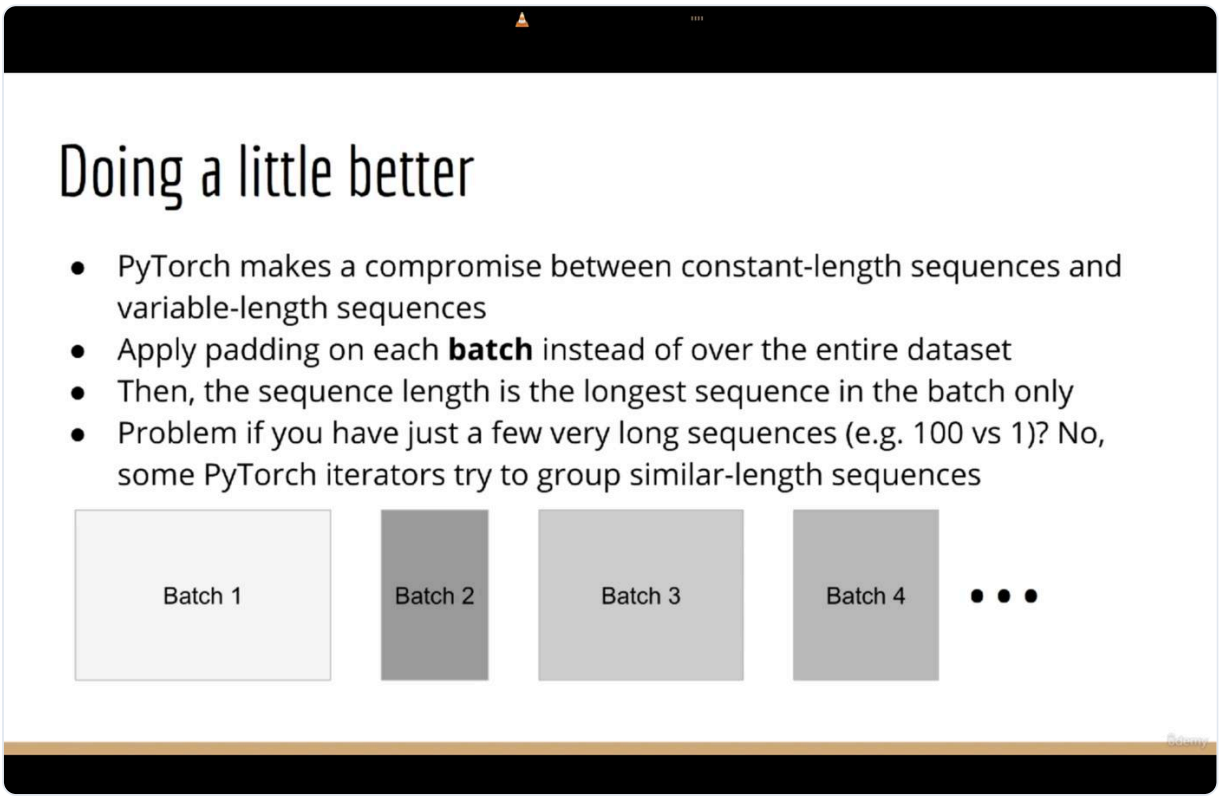


FIGURE 8.26 | Doing a Little Better

So unless you wanted to write your own custom code this will be our data format for sequences luckily what PyTorch does is a kind of compromise between constant length sequences and variable length sequences. The idea is instead of having all your sequences padded to be the same length as the longest sequence. Just do that across single batches. As you know our method of training in deep learning is batch gradient descent so we're only going to look at a subset of our training data at any given step during that step. We can pad the sequences in the batch on the fly so that they are only as long as the longest sequence in that batch instead of the entire training set.

Now you might think this is still going to be a problem if you have one sequence in your batch that has Length one hundred and all the rest are length one. Then you'll have ninety nine padded values in all those shorter sequences but what PyTorch does is for certain generators. It tries to grouped together sequences with similar sequence length so the batches you see will still be random but not as random as they would be if you didn't care about the sequence length. Of course if you're creating your own data

generator you would have to write that kind of custom code yourself. Finally note that it's possible to take things one step further and actually deal with a variable length sequences almost without dealing with padding.

This part of the lecture is optional and outside the scope of this course. So don't feel like you have to understand this to progress through this section the idea with this is that although your batch will be of size n by t by D. Where t is the length of the longest sequence in the batch.

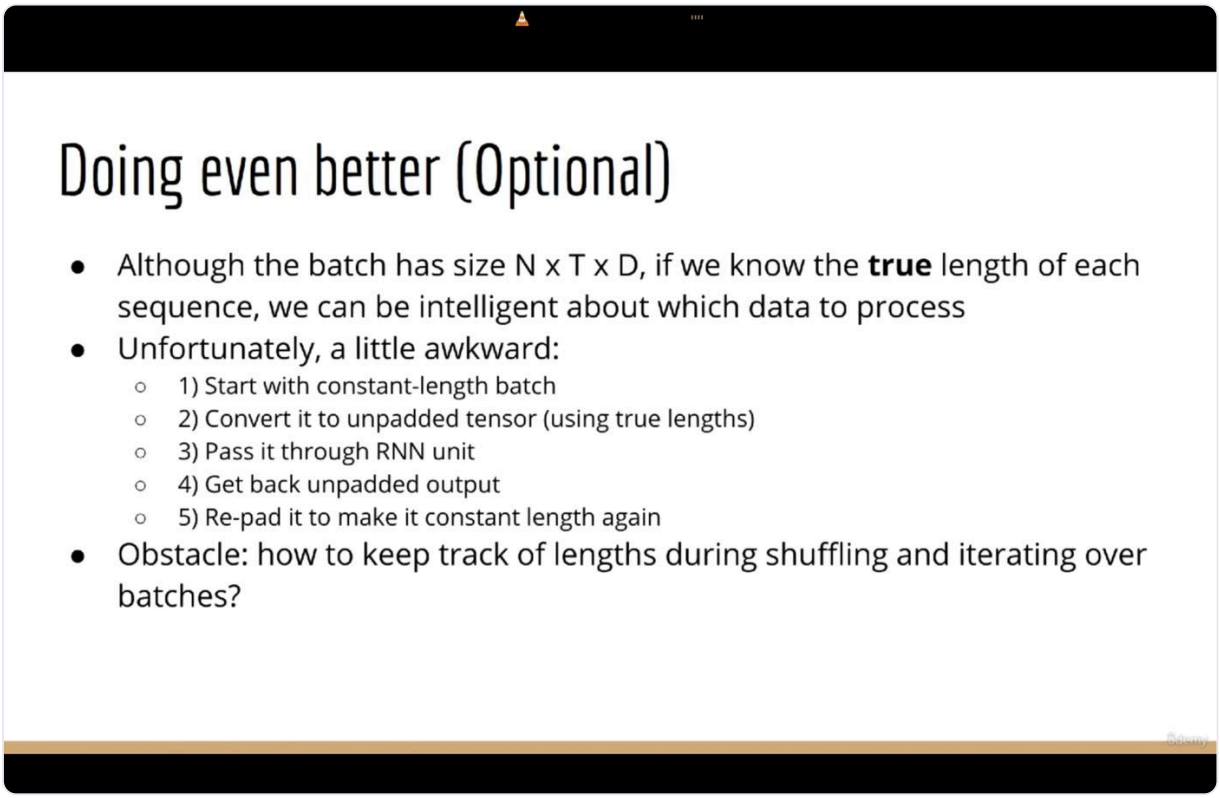


FIGURE 8.27 | Doing Even Better (Optional)

If we know the length of each sequence in the batch then we can be intelligent about which data we process. Ultimately it looks kind of weird because your batch of data will still be a padded tensor but then using the lengths of each sequence you'll convert it into an unpadded tensor. Then you'll pass it through the RNN unit and get back an unpadded output. And finally you still have to reapply the padding and then pass your data through the rest of the neural network. Unfortunately doing this requires you to keep track of the lengths of each of your data sequences even while shuffling and iterating over randomize batches. This approach is more involved.

So at this time it's going to be considered to be outside the scope of this course. All right. So what are the main takeaways from this lecture. Number one when you have sequence data that means your data isn't just a single feature vector but a value or a set of values that can vary with time.

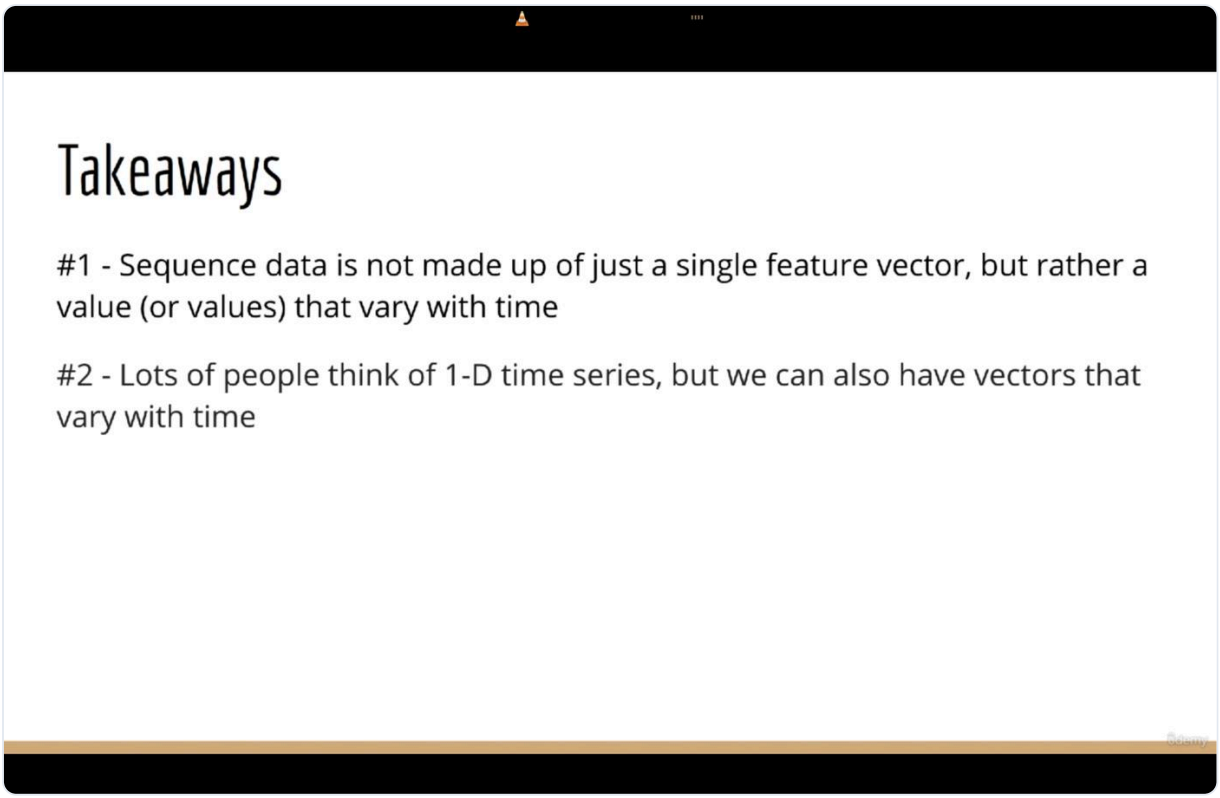


FIGURE 8.28

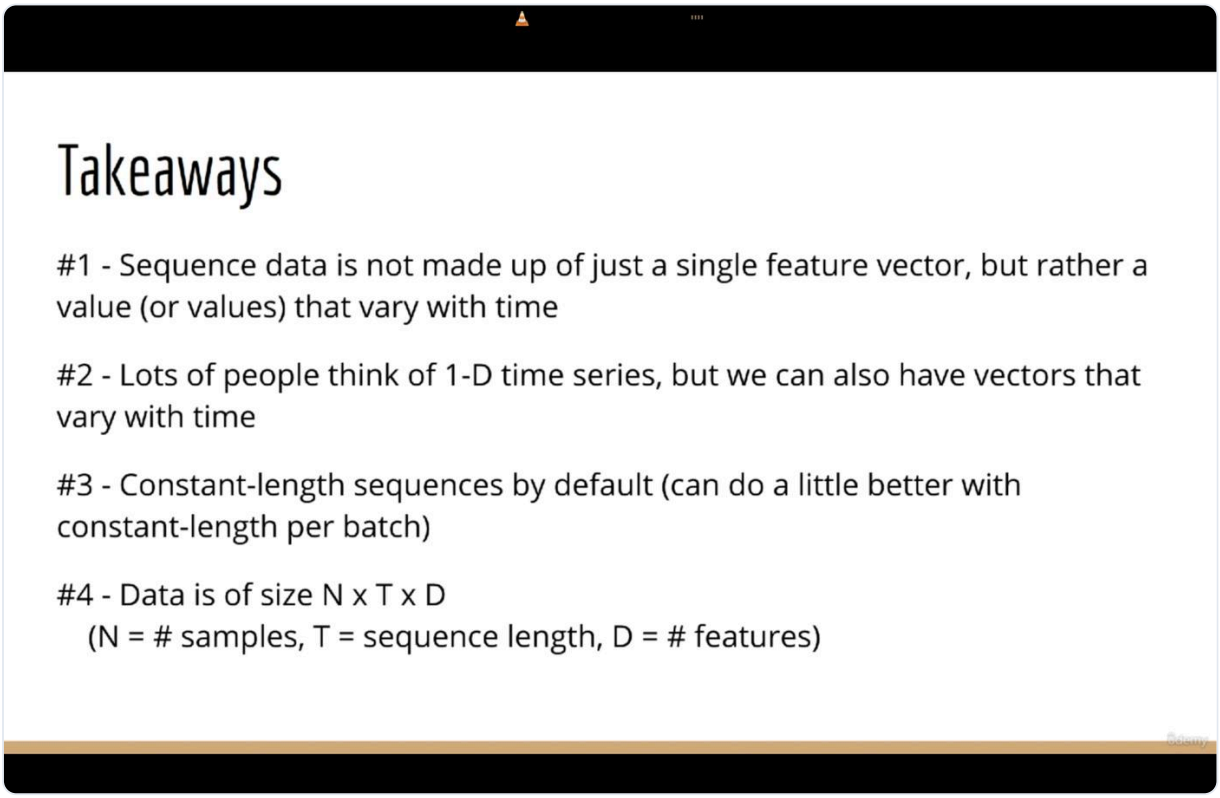


FIGURE 8.29

Number two lots of people think of times series which are one dimensional values that vary with time but we can also have d dimensional vectors that vary with time. Some examples of that are weather readings from multiple weather towers or brain signals from multiple electrodes on the brain. Number three in modern deep learning libraries you're limited to constant Length Sequences. If you want to do better you can use constant length sequences per batch instead of the whole dataset. If you want to do

even better you can convert those constant length batches back into a variable length sequences but that's outside the scope of this course. Number 4 by convention will organize our sequence data to be of shape N by T by D where n is the number of samples t is the sequence length and d the observation dimensionality.

You should always be visualizing these shapes in your mind which will help your understanding immensely. But I want to prompt you to do so and therefore you'll just have to remember to do it yourself.

LECTURE 47

All Machine Learning Interfaces Are the Same

In this lecture, we are going to discuss forecasting, which will lead us into our next coding example. This lecture will focus on how to do a correct forecast because unfortunately, a lot of resources do this in a way that makes your results look really nice, but actually make zero sense. So first, what does it mean to forecast in the way that we mean in this course? It means that we have a time series and our job is to predict the next values of this Time series. Importantly, notice I use the plural there. We want to predict multiple values, not just one value. The length of time that you want to predict is called the horizon, in some applications, your horizon might be three days or five days.

For example, if you're trying to predict the demand for each product at a factory, you might want to forecast the demand over the next three to five days. Another pretty obvious example is the weather. Let's say we want to predict the hourly weather over the next seven days or so. So if we predict the weather for each hour over seven days, that's seven times 24, which is one hundred and sixty eight hours. So if your sequence step size is hours, that would be predicting one hundred sixty eight steps ahead. This is important. So remember, you don't want to just predict one step ahead as a side note. Forecasting is a huge topic.

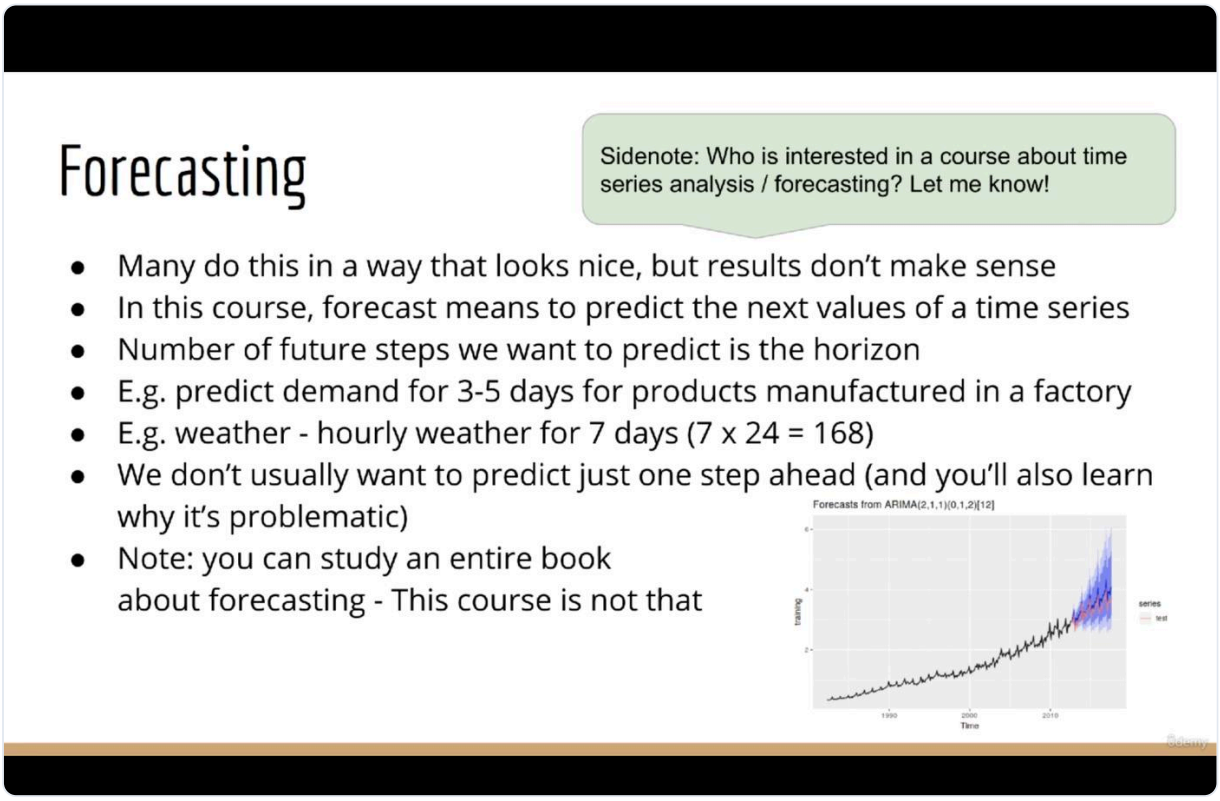


FIGURE 9.1

You could study an entire book about forecasting. This course is not that we're just touching on specific points which are relevant to deep learning and RNNs. Now, you might recognize that this section is all about RNNs, recurrent neural networks. So where are the RNNs? Well, we are not there yet. A common approach in machine learning, especially in the industry, is to do the simplest thing possible. Do not throw an RNN at a problem if there is a simpler method available.

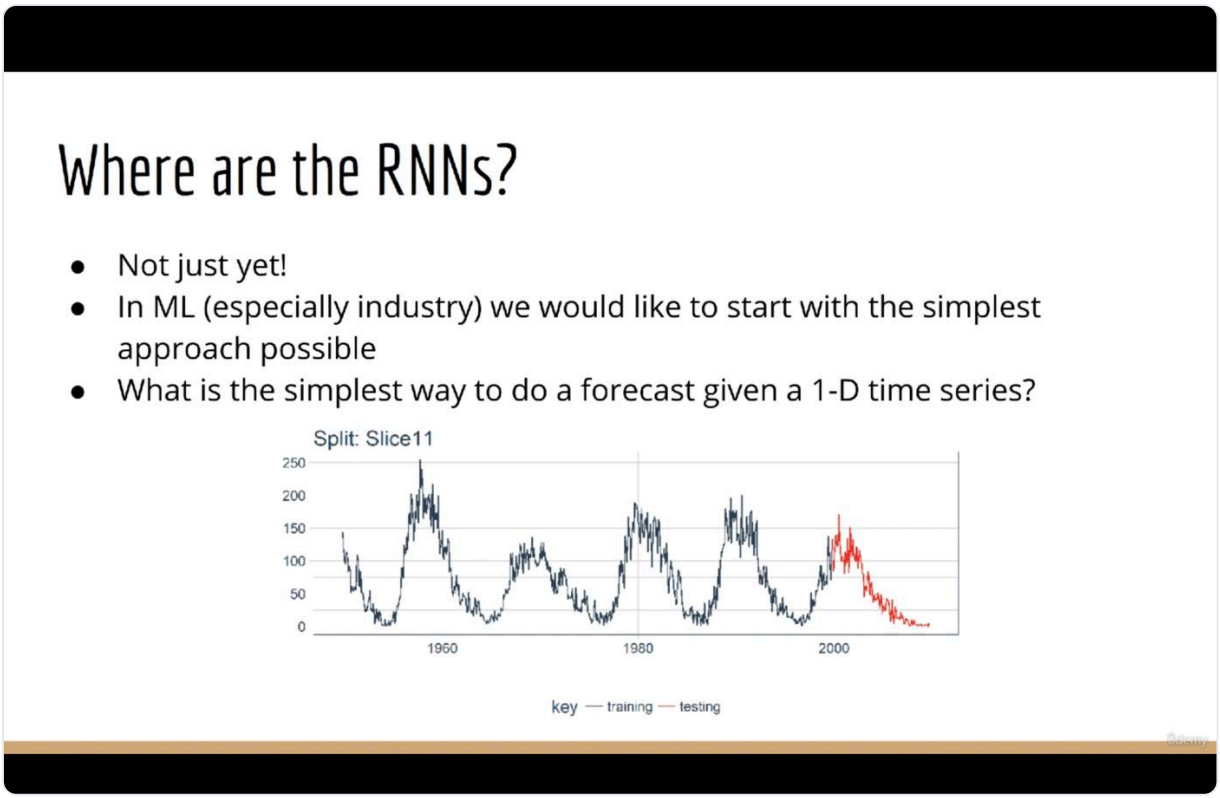


FIGURE 9.2 | Where Are the RNNs?

So my question for you now is what is the simplest way to do a forecast given a single one dimensional time series? Well, how about we go back to our approach at the very beginning of this course, linear regression, linear regression is in fact one of the best models to use to study forecasting because it's very intuitive and there's not that much going on in terms of formulas and manipulating equations.

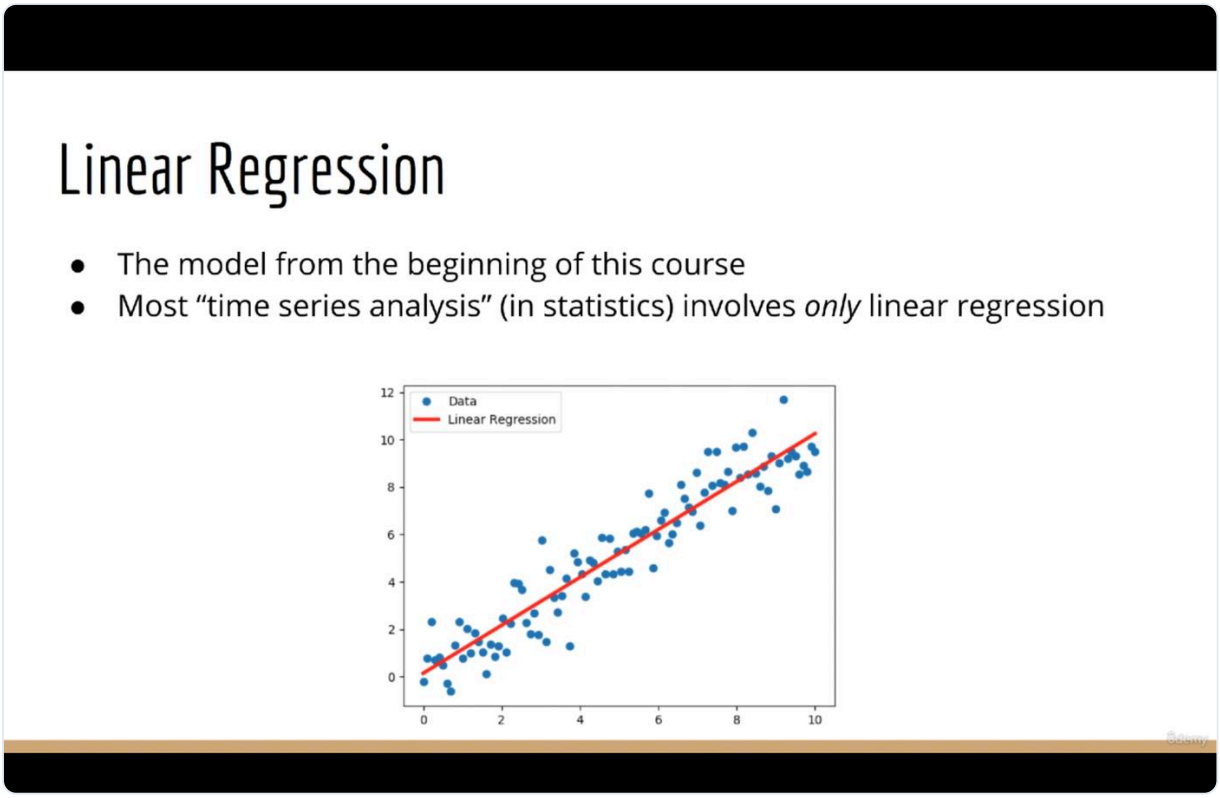


FIGURE 9.3 | Linear Regression

At this point, you might raise an objection if our data is of shape N by T by D. We can't use linear regression because linear regression works on 2D rectangles of shape N by D.

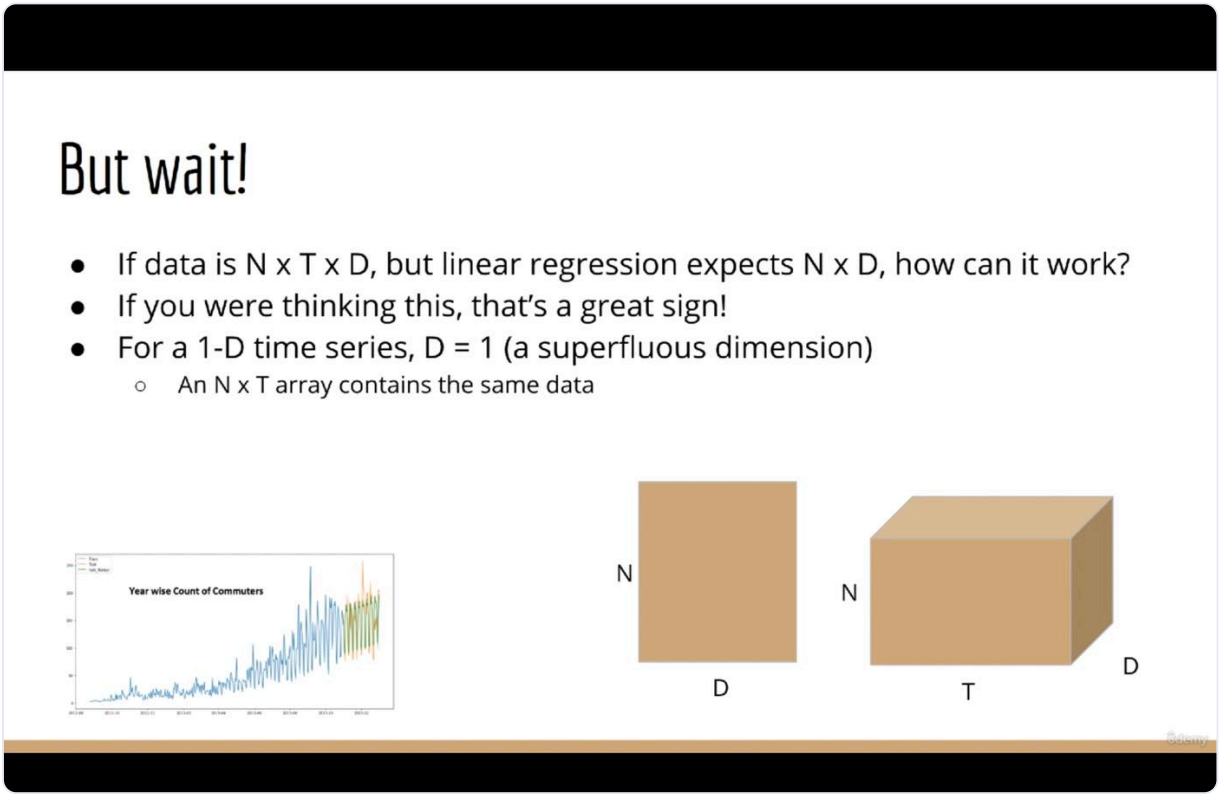


FIGURE 9.4

And if you were thinking this, that's a great sign. That means you're thinking of our shapes. Which is exactly what you should be doing. So how can we reconcile this apparent contradiction? Well, in fact, linear regression does only work on data of shape N by D. But remember that in our example, we are considering only one dimensional time series. So D equals one. As you know, this is just a superfluous dimension. We can represent the exact same data in a two dimensional array of shape N by T. Again, this is analogous to the situation where you have an N length, one dimensional array. Or you can have the same array stored in a two dimensional N by one object.

So how do we use linear regression in this situation? The answer is we pretend that T is D or to put that a different way. In this scenario, linear regression treats T as if it were D. This is another instance of my rule.

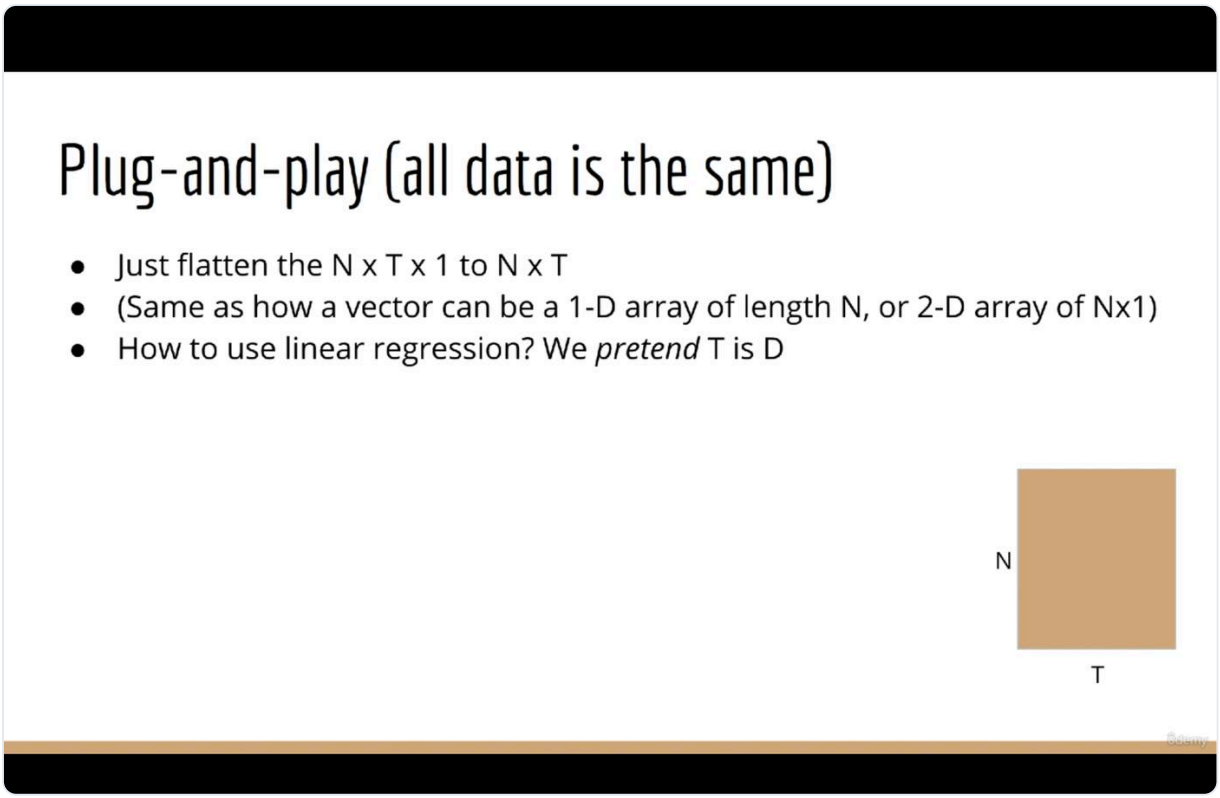


FIGURE 9.5 | Plug-and-play (All Data Is the Same)

All data is the same. Let's think about how our data will look.

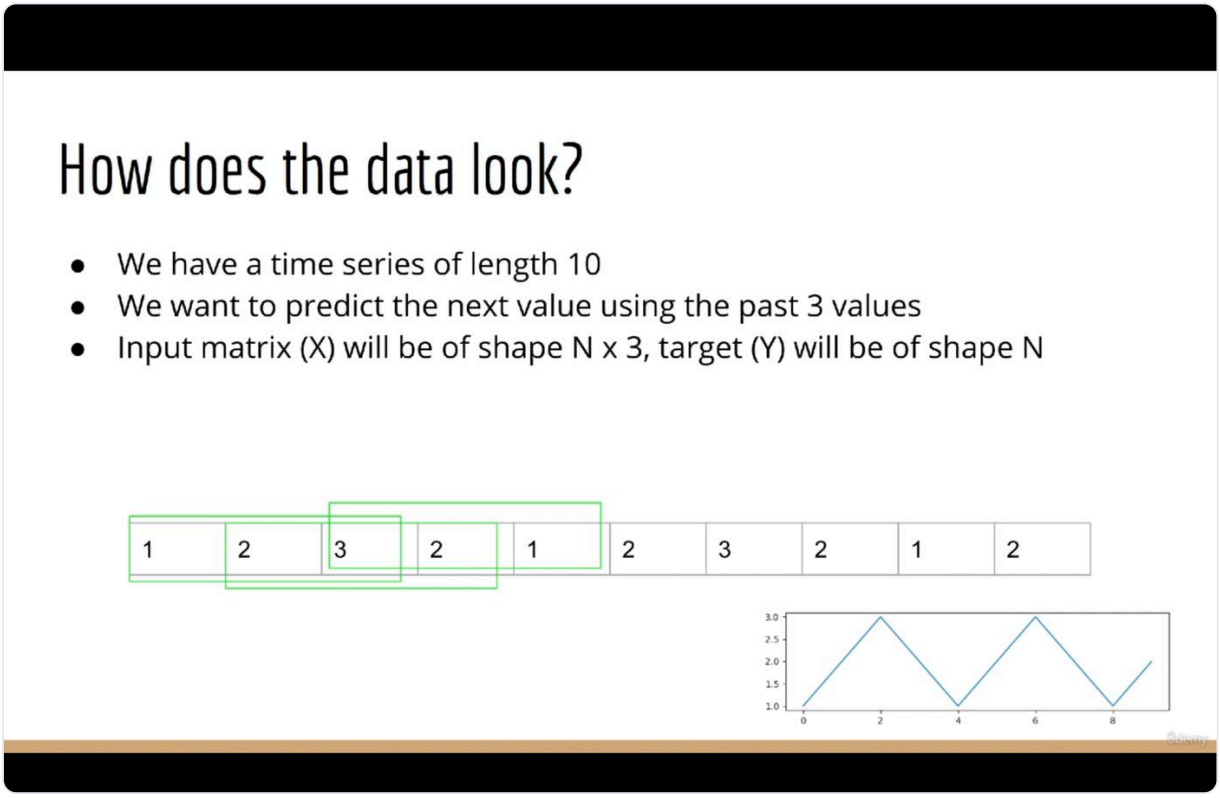


FIGURE 9.6 | How Does the Data Look?

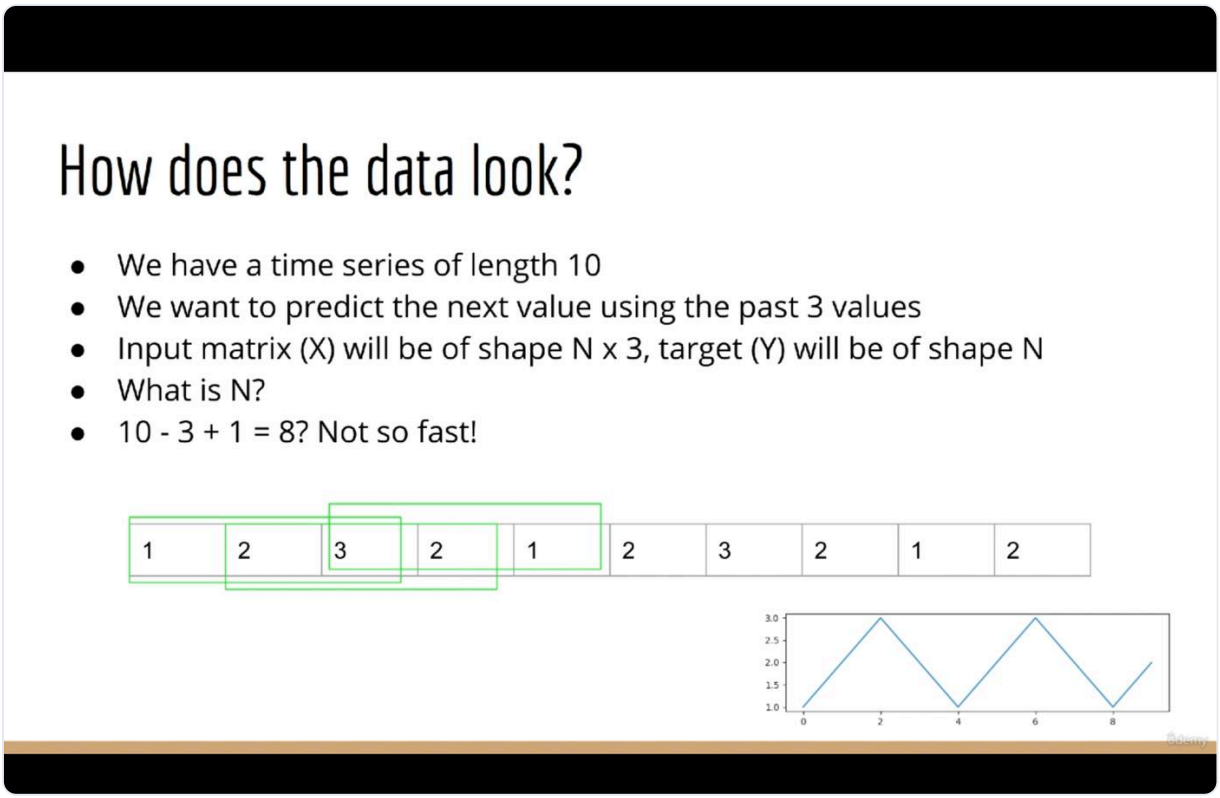


FIGURE 9.7 | How Does the Data Look?

Let's suppose we have a time series of length 10. And we want to train a model to predict the next value in the Time series using the past three time steps. In this case, our input matrix will be of size N by three. And our target vector will be of size N. Now, what is n as mentioned previously, we can calculate N as the number of length three windows that fit into the Time series, which is length 10. So that's ten minus three, plus one, which is eight. But wait. Remember that we would like to be able to predict the next value, which is the target. So while we can fit eight windows into the Time series, we can't actually use the last window in our training set because we don't know the next value.

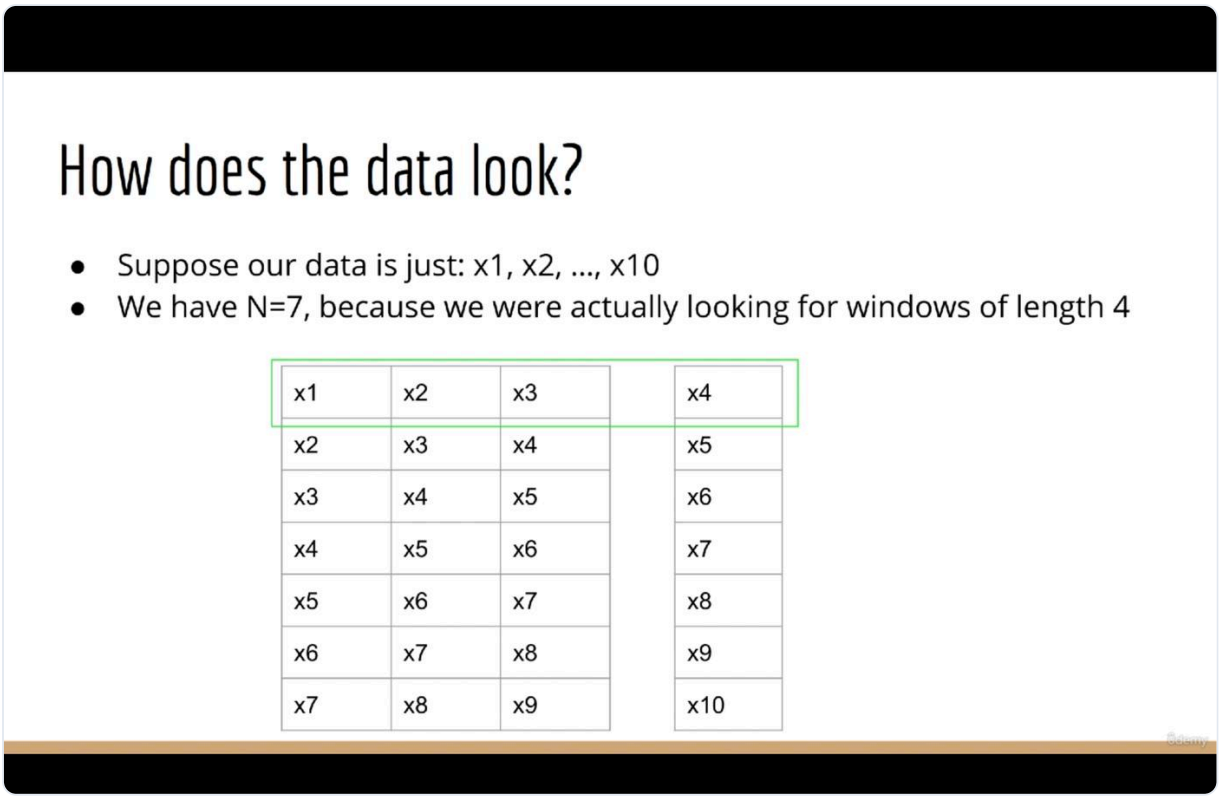


FIGURE 9.8 | How Does the Data Look?

Suppose our data is just x 1 x 2 all the way up to x 10. In this case, if our input is X eight, x nine and 10, we can't have X eleven as a target because we don't have X eleven in our time series. So in actuality, we only have N equals seven. So really, it's like asking how many windows of length four fit into the time series of length. Ten. That's ten minus four plus one which is seven. And this is because the fourth value is the target. So what does a linear regression forecasting model look like at this point? It should be pretty clear it is X hat at time T equal to W zero plus W one times X at time, T minus one plus W two times X at T, minus two plus W, three times X at T minus three.

Well, I'm using time indices here. You should still be able to recognize this as linear regression, as a side note.

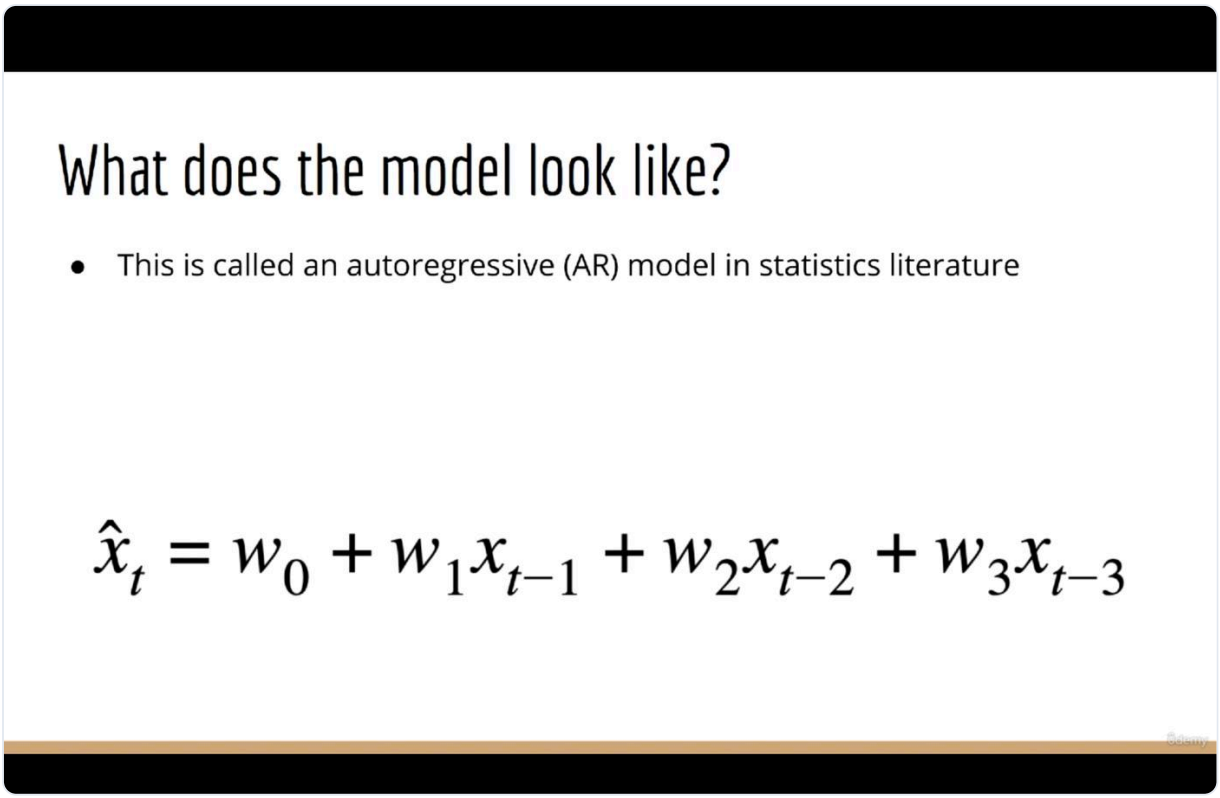


FIGURE 9.9 | What Does the Model Look Like?

This is called an autoregressive or AR model. In statistics nomenclature, the word auto means self. So it's a model that tries to predict the value in a time series using its own values. OK, so using this model, how do we forecast? Here's a common thing I see a lot.

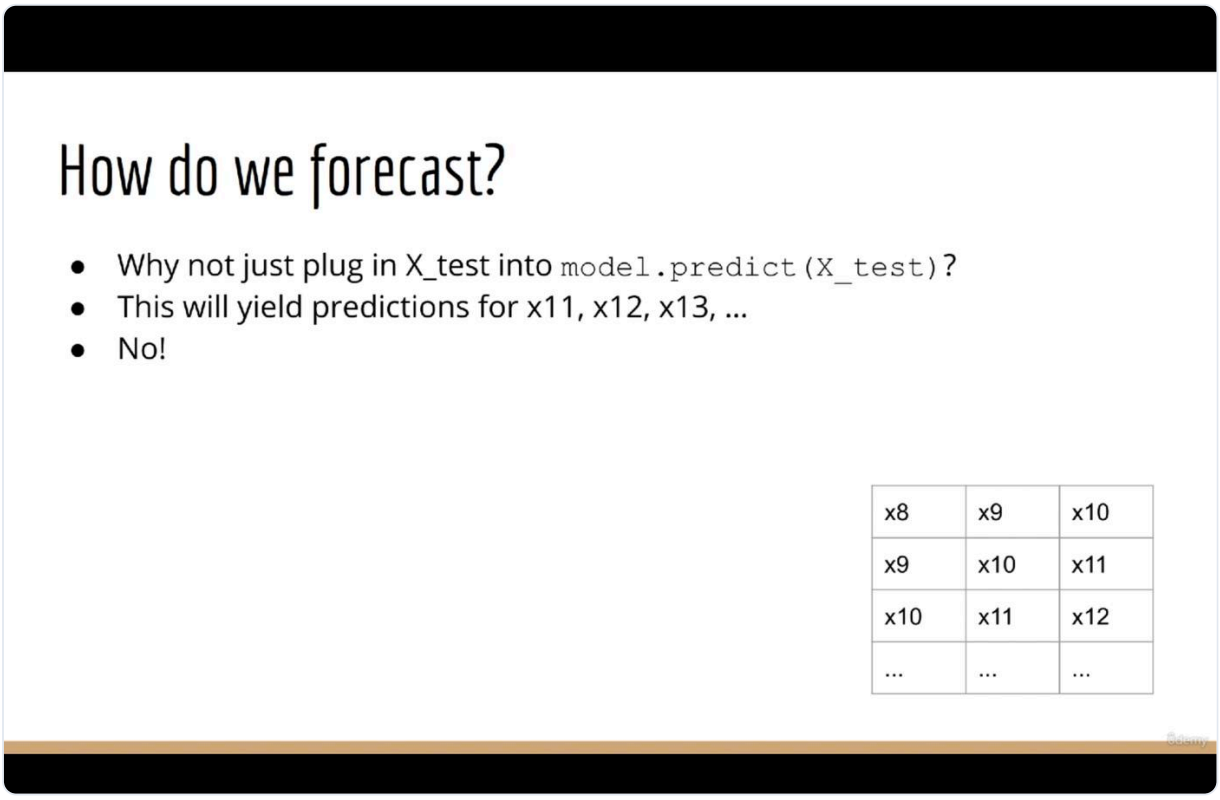


FIGURE 9.10 | How Do We Forecast?

Let's say our test set consists of the data X eleven up to x 20. And we want to calculate the test accuracy on our test set. So you might say take the data matrix that we see here, X say X nine, 10, x nine, x 10x eleven and so on. And plug that into model dot predict. Then we'll get the predictions for X eleven X twelve x 13 and so on.

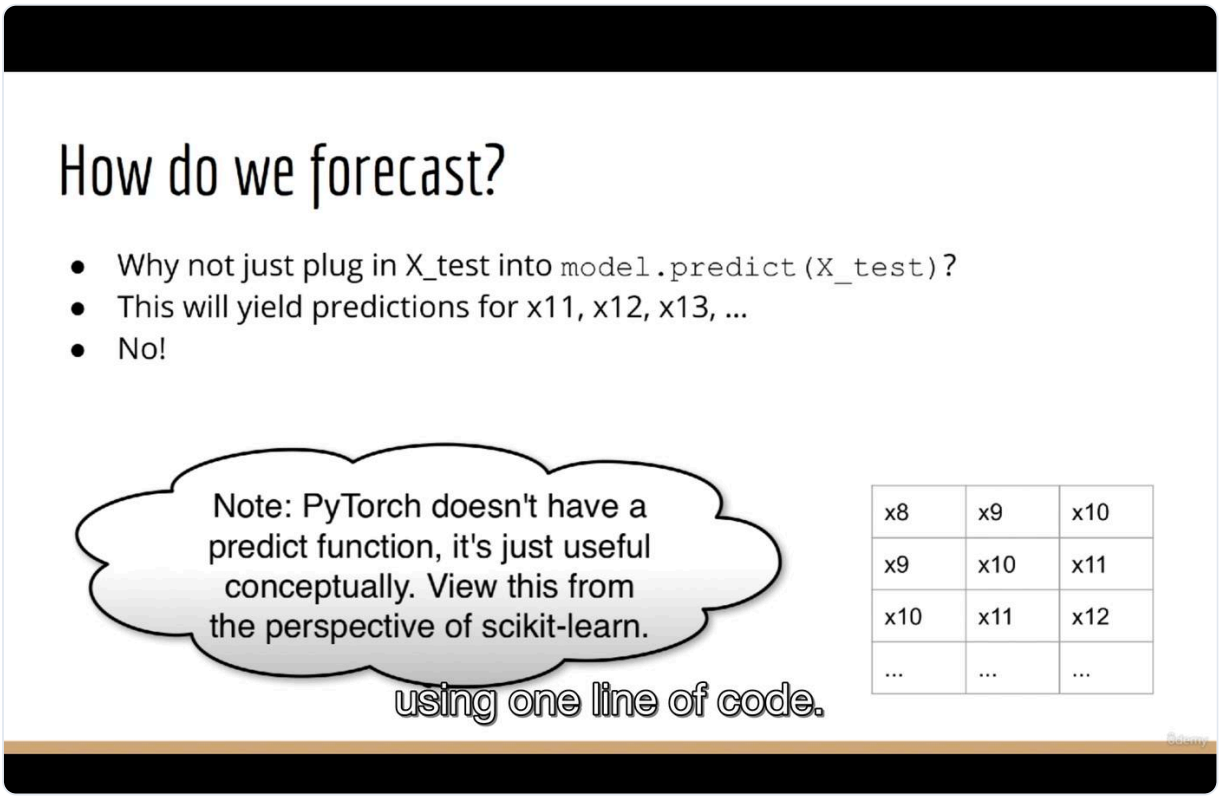


FIGURE 9.11 | How Do We Forecast?

Unfortunately, this is wrong. As a side note, please be aware that models in PyTorch do not have a predict function. We're just using this notation for convenience since it allows us to express how to make the model predictions using one line of code. You should recognize this notation from scikit-learn so you can just frame this lecture in the context of using a scikit-learn model rather than PyTorch. So why is it wrong?

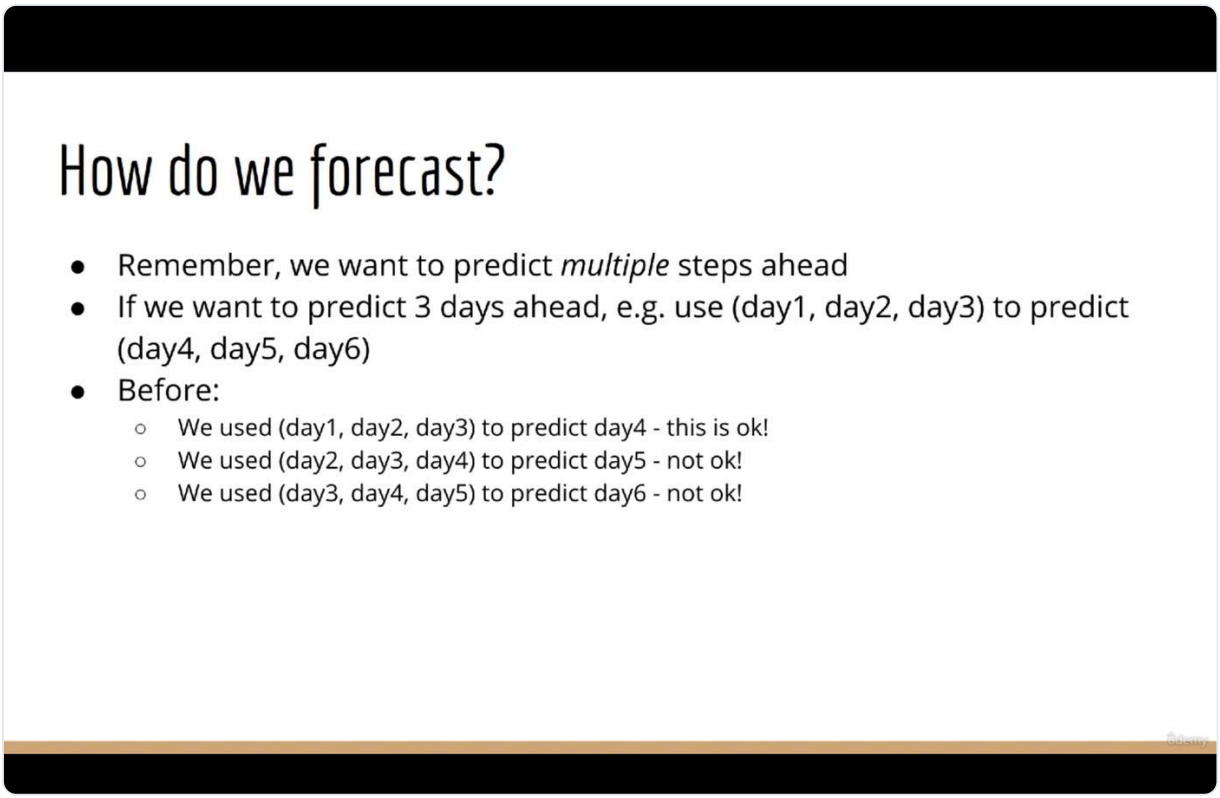


FIGURE 9.12 | How Do We Forecast?

Remember that we would like to use our forecasting model to predict multiple steps ahead. What we just saw was only predicting one step ahead. This is not good enough. Let's say we want to predict three days ahead. We want to use day one day two day three to predict day four, day five and day six. What were we doing before we used day one, day two and day three to predict day four? That's fine. Then we used day two day three and day four to predict day five. That's not fine because remember, we don't know the value on day four yet. Today is only day three. Similarly, we can't use day three, day four, day five to predict day six. So what do we do instead?

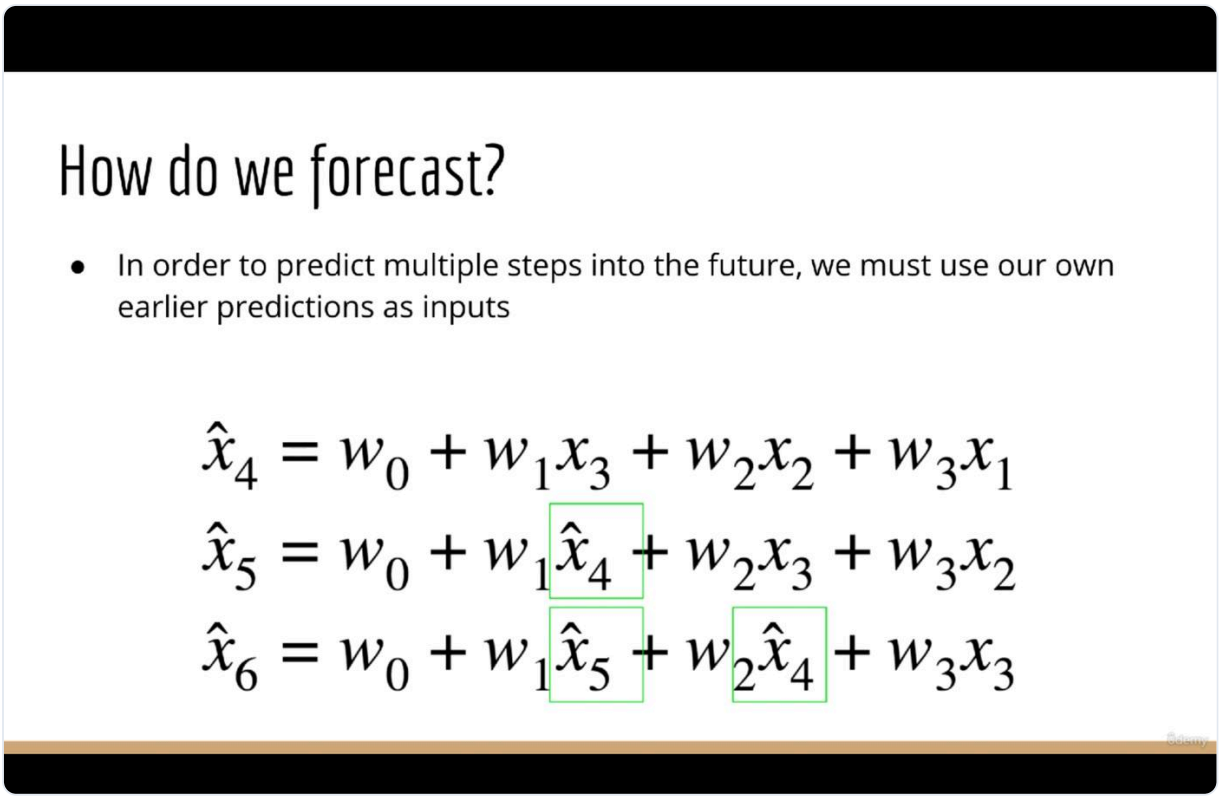


FIGURE 9.13 How Do We Forecast?

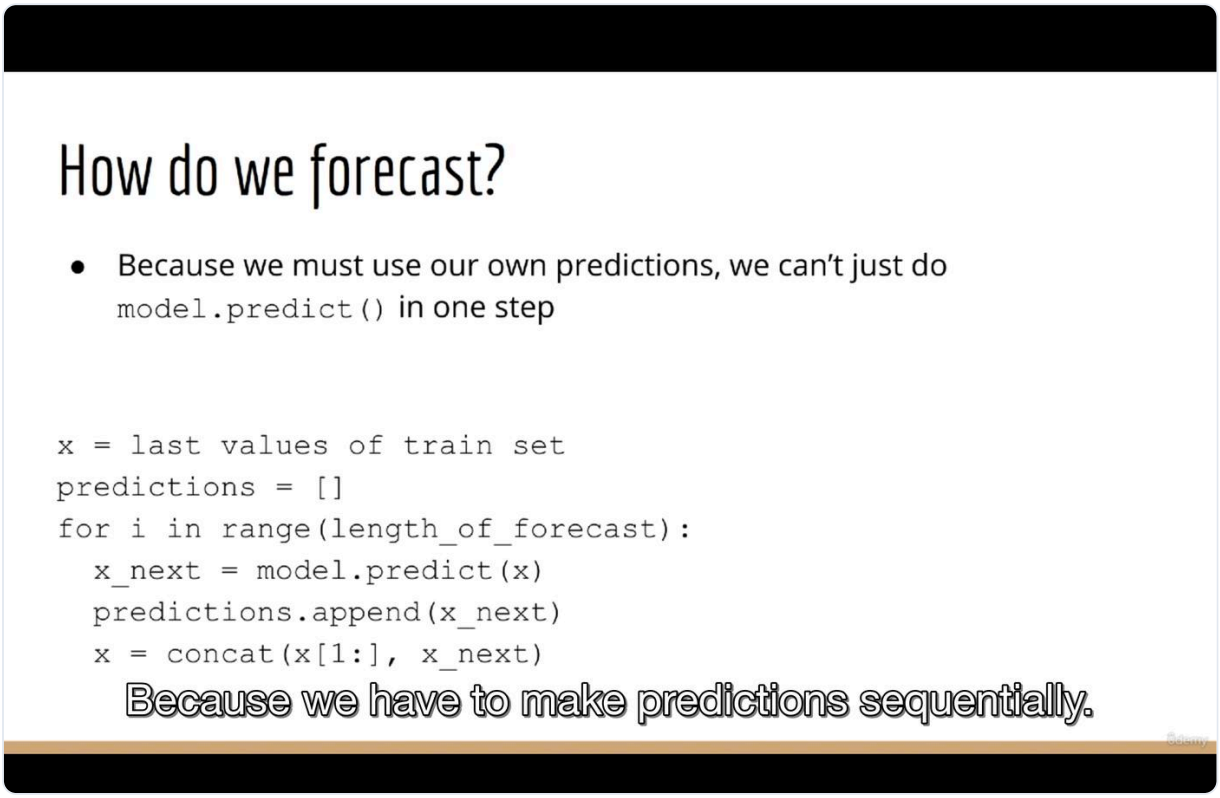


FIGURE 9.14 How Do We Forecast?

If we want to make predictions multiple steps into the future, we must use our own earlier predictions as input, for example, in order to predict X five. We use X two, X three and X hat four. X hat four was calculated from X one X to an X three. To predict X six we will use X three X hat four and X hat five. This is the correct way to forecast. Now, note that this doesn't mean one step ahead. Predictions aren't useful. It

just means you have to know when to use and when not to use them. Because we have to make predictions sequentially. We can't do it all in one call of model dot predict like we did before. Instead, we need to use a loop and call model dot predict only on the most recently generated sequence.

And this pseudocode, I'll attempt to express the main idea, our first sequence X we initialize to the last values of the train set from which we want to predict the first value of the test set. We also initialize an empty list to store our predictions. Then we enter a loop that goes for the number of steps we want to forecast inside the loop. We call model dot predict to get only the next immediate value. We store this in our predictions list then. And this is the important part. We discard the oldest value. And we concatenate the rest of X with the new prediction. We assign that to X. Then when we call model dot predict on the next round, we are using an X that contains a both true values and predictions for the forecasted values.

As a final note in this lecture, I want you to think about how can we apply the rule?

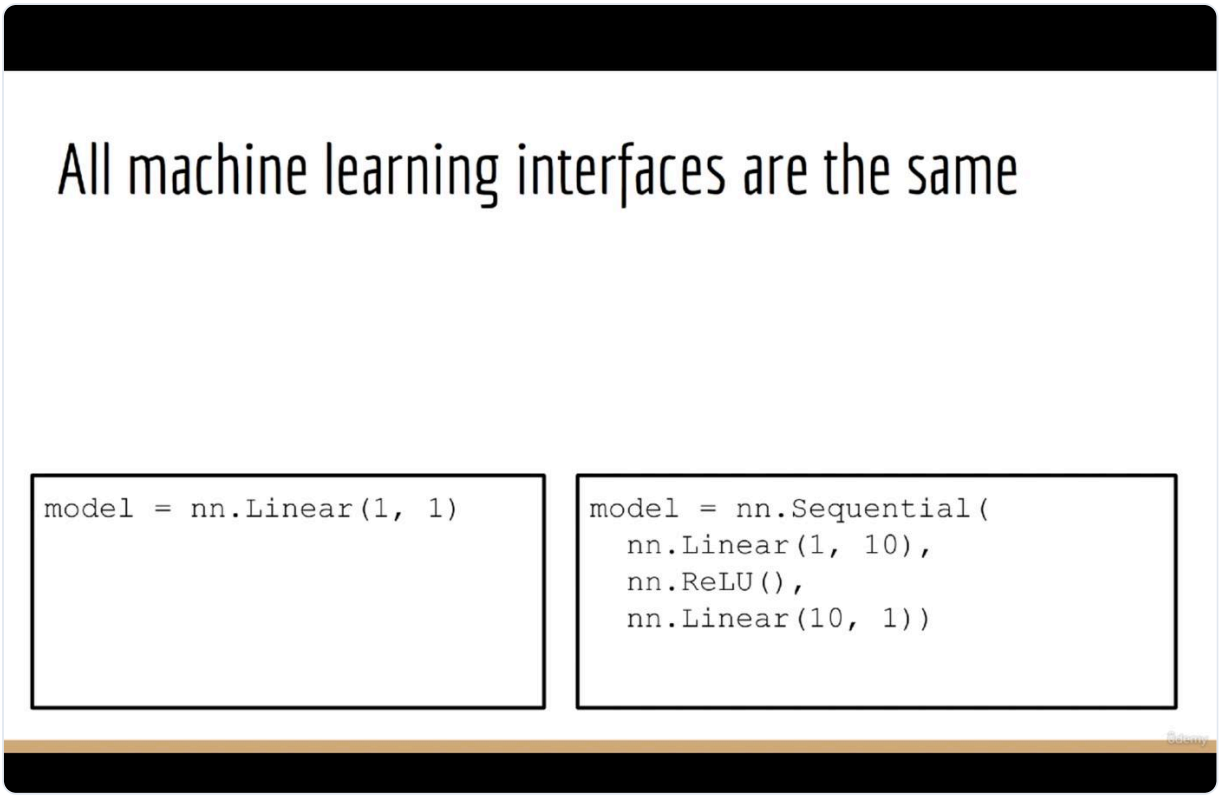


FIGURE 9.15 | All Machine Learning Interfaces Are the Same

All machine learning interfaces are the same in order to build a more powerful predictor.

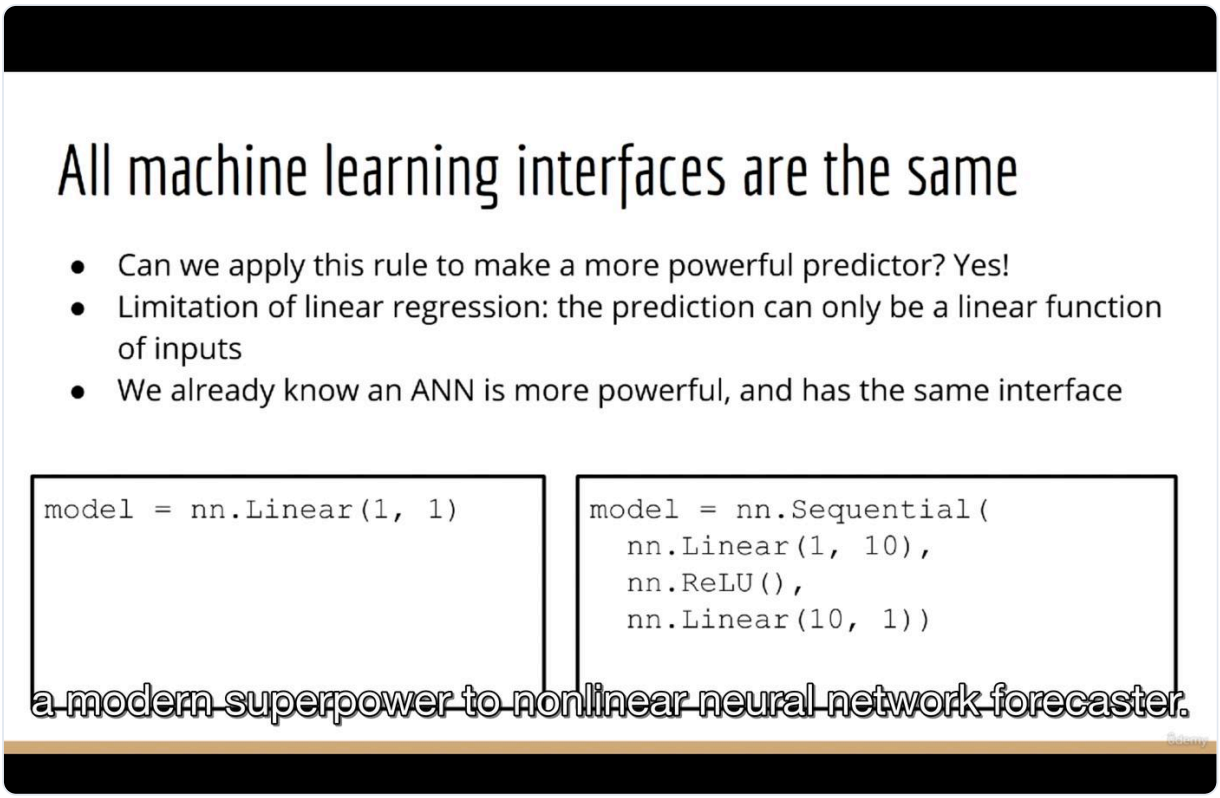


FIGURE 9.16 | All Machine Learning Interfaces Are the Same

One limitation of linear regression is that the prediction can only be a linear function of its inputs. Luckily, we already know how to build a more powerful model that works on the same kind of data an ANN. So by being an expert in deep learning, you can immediately apply this technique without any extra effort. That's a pretty powerful approach. All of a sudden, you are able to convert your classic statistical time series forecasting model into a modern superpowered nonlinear neural network forecaster. And that's all just with one line of code. Not bad.

LECTURE 48

Validation Predictions and Sine Wave

Check the notebook `9.PyTorch-Autoregressive-Model.ipynb` for the code.

In this lecture we are going to look at the code to implement an autoregressive linear model for time series prediction. Since you already know how to write linear regression Using PyTorch that's not going to be much of a challenge in this lecture. The challenge in this lecture mostly has to do with processing the data correctly and making forecasts. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at.

So let's get started. The new thing here is after the imports we're going to start by creating a synthetic dataset. Remember how earlier I explain why synthetic datasets are important. They allow us to carefully study the behaviour of our model under different controlled circumstances. So in this lecture we're going to start with a very basic time series a sine wave to start we're going to create a sine wave with no noise just a pure sine wave with 1000 points we also plot the series so you know what it looks like all right. So there's the sine wave. Next we build our data set and this example we're going to set t equal to 10 which means that we're going to use 10 previous time steps to predict the next time step.

We'll start by populating X and Y as lists and then cast them into arrays. Once we are done you could alternatively pre allocate these as fixed size NumPy arrays but I've decided to take a more lazy approach. So in this loop we go from zero up to the length of the series minus Big T. Now of course you want to double check that this is correct. So that we don't have an off by one error. The reasoning is this. We know that our input x will go from Little T up to Little T plus big T. So for example zero up to 10 in terms of actual indices that will be at 0 1 2 and so on up to 9 inclusive. So when we say zero up to 10 it doesn't actually include the index 10.

Instead the target will be at the index 10. We know that the final index of the original series will be the final target. That is the index len series minus 1 thus in order for the final target to be the length of the series minus 1. That means Little T must go up to the length of the series minus Big T. If we plug in the final value of Little T we get len series minus T minus one plus T which is equal to len series minus one so inside the loop we assign a little X to be the series from Little T up to Little T plus big T then we append this to Big X next we assign a little Y which is the target to be the series at time A little T plus big T then we append this to big Y when we're done that we cast x and y to NumPy arrays and print out the shape so as expected X is of shape N by 10 whereas Y is N by 1.

Next we build our autoregressive linear model as you know it is just an nn.Linear as expected at the next step is to create our loss and optimizer since this is regression we will be using the mean squared error next we split the data into train and test sets. Here's another important point about training time series models you don't want to split up your data randomly say using the scikit-learn function train test split. This wouldn't make sense because a real forecasting model has to predict the future it can't train on points within that future. Thus our model trains only on the first half of the dataset and validates on the 2nd half since sine waves are periodic.

This is a non issue in the scripts but in general this is important next we have our training function which is called full GD because it does full gradient descent instead of batch gradient descent. This is okay for this dataset because this dataset is quite small all right so next we call our full GD function and training is almost instant OK. So next we plot our loss as usual so this looks at it next. We're going to do a forecast using the incorrect method. The reason I want to do this is because I want to show you what not to do. We're also going to do this calculation manually because the general form of the code is going to be helpful when we want to make actual forecasts as an exercise.

You might want to think about how to make this code do the same thing but more efficiently. So first we assign the validation target variable which is just the second half of why we'll create an empty list for the validation predictions and then populate this as we move through the data. Next we're going to set the index variable i which indexes x test to 0. Next we're going to enter a loop that continues while the length of the validation predictions list is less than the length of the validation target list. Once they have the same length we know that we're done because we've made all the predictions to correspond with the targets inside the loop.

We get our model prediction P. Now this might look weird to you. Why not just pass in x test sub i to the model and then assign that to P. Well there are two problems there. First x test sub i does not have the right shape. Remember that the inputs in the model must be a two dimensional array but X test itself is a two dimensional array. So if we index x test we only get a 1D array of size T. Therefore we need to reshape the array to one by T. That means one sample with t features we can accomplish that by calling the view function and passing in one minus one then we get the output of the model prediction. And that's also not the right shape because remember that in general the model returns an N by K output.

If we have N samples and K output nodes in this particular example we happen to have one sample and one output node. So this returns a 1 by 1 2 dimensional array therefore to get the scalar value contained within it. We have to index the array at 0 0 finally since the output value lives in PyTorch land. We want to bring it back to Python land by calling the item function. Next we increment i and we append the prediction P to our validation predictions. Next we're going to plot our predictions against the targets so this looks very good. Pretty much exactly correct. But remember this is still the wrong way of forecasting. Next we're going to do forecasting the correct way.

So if you just glance at this code so it looks almost exactly the same as what we had before we start by grabbing our validation targets as the second half of Y and we initialize validation predictions as an empty list. The next line is different. Before we just had an index i and we used I to index x test the input data. But the problem was we shouldn't have been using the true input data to predict future values of the time series. Instead we'll set the variable last X to be the first input vector from here on out. We will no longer take any new values from the actual data set. So next we enter the loop inside the loop. We do the same prediction as before.

Remember that we have to reshape the input to one by T. Now this is where things change first. After getting the model output which we'll call P I'm not going to immediately grab the value and bring it back to Python land instead I'll do that on the next line where I appends the prediction to the list of predictions so I index p at 0 0 and then I call dot item. So why did I do that. Well I still need the p value to be in PyTorch land in order to make our new input on the next iteration of the loop. We essentially have to shift all the values of the array to the left and then add the new prediction at the end. Of course this entire array must be in PyTorch land and not NumPy land.

So that's why I kept the p variable as it was returned by the model lastly we use the PyTorch cat function which stands for concatenate. I take everything in the previous input x throw away the first value and then concatenate it with P.. The newest prediction. So that's how we update the variable last X with our latest forecasted prediction. So let's run this and finally let's plot the predictions against the targets. So this is very interesting. So it appears even though we are now making the forecast using the correct method we still get a perfect prediction. Now remember that this time series has no noise so you might be wondering what happens if it does have noise.

Let's go back and add noise and then run everything again so let's scroll back up. Let us uncomment that line and then this good at runtime and run after OK. So here's the noisy sine wave here's the other code so you see that the loss is a little higher now loss per iteration still looks good. So here's where things get tricky. We're going to plot our incorrectly made forecast as you can see it appears to look pretty good. Our forecast even appears to smooth out all the noise but remember this is the wrong way to forecast so it should be no surprise that these results are misleading. So let's go down to the correct way of making forecasts. OK so now we see what happens when we do a correct forecast which is what happens in the realistic scenario.

As you can see the results are no longer as good because of all that noise. Interestingly the model does learn that the pattern is periodic and that the true underlying function is smooth. So at least that's pretty encouraging.

LECTURE 49

Perfect Forecast for a Sine Wave?

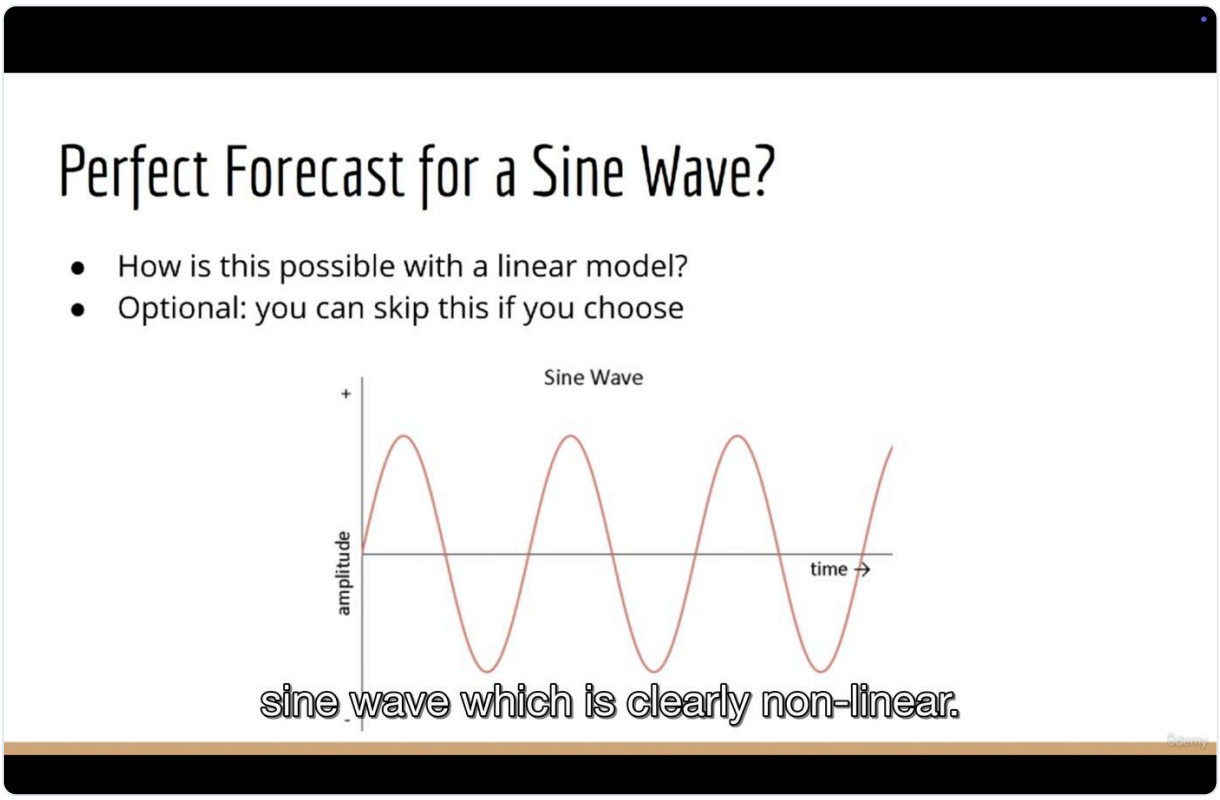


FIGURE 11.1 | Perfect Forecast for a Sine Wave?

Now you might be curious how a linear model like linear regression is able to perfectly forecast the sine wave which is clearly non-linear. For those of you who are interested I'd like to share with you the derivation since I find this pretty interesting myself. If you are not interested you can consider this optional so don't feel like you have to understand this in order to understand the rest of the course. It turns out that it's possible to predict a sine wave perfectly using only two past values of the time series.

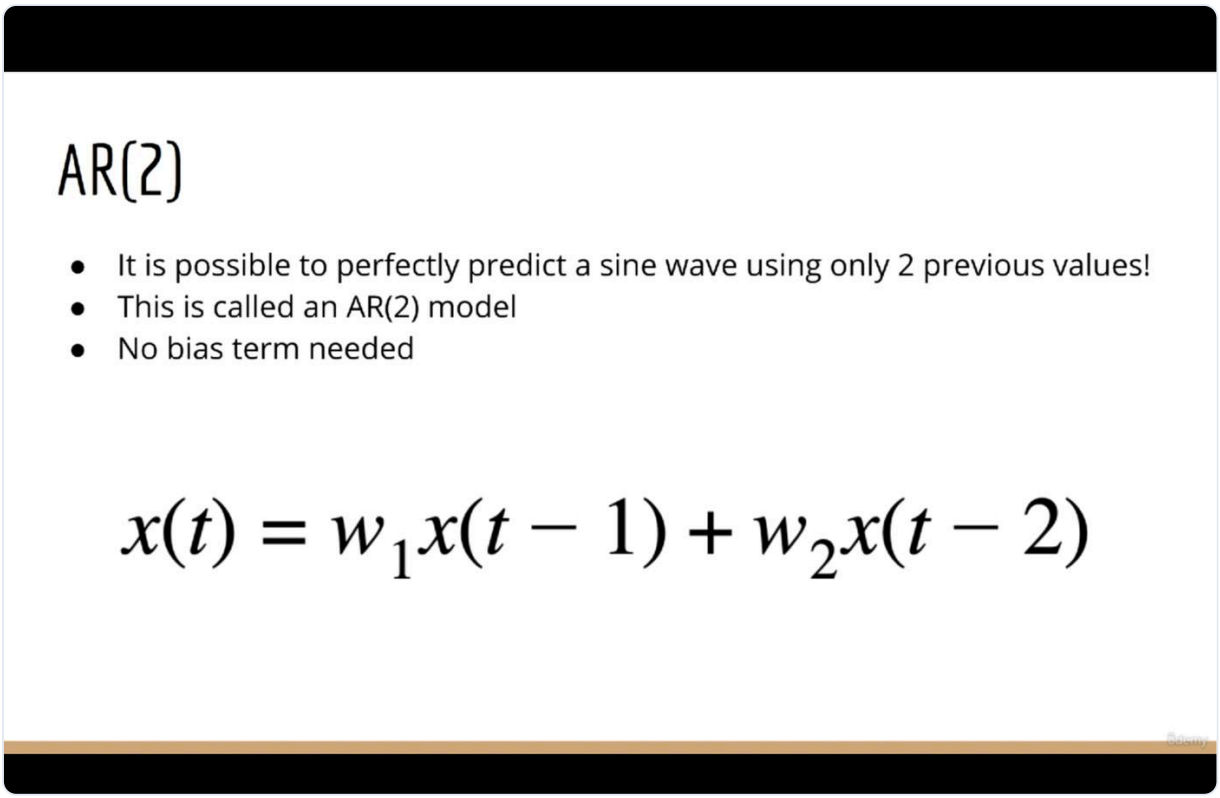


FIGURE 11.2

We call this an AR2 model. Notice we do not even need a bias term so let's see how this might be true. First note that if you want to write a time series which is a function of time in a mathematical form it looks like this. So x of T equals the sine of Omega times t and the s the letter inside the sine is Omega and not w the lowercase Omega just happens to look a lot like a W.

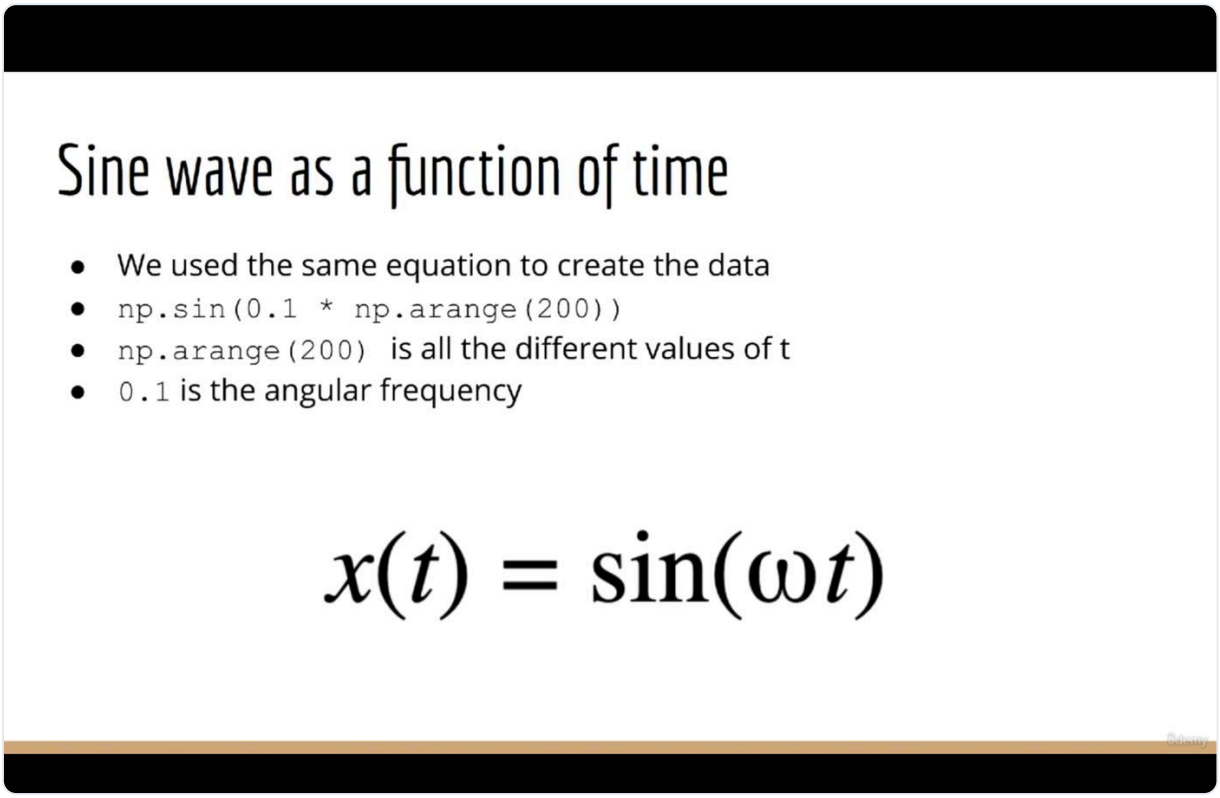


FIGURE 11.3 | Sine Wave as a Function of Time

Now if you recall we actually did something very similar to this when we created the data we said a series is equal to np dot sin of zero point one times np dot arange of two hundred so np dot arange of two hundred it represents all the time points and zero point one was the angular frequency surprisingly. This is all we need to show that our AR2 model can learn a sine wave. Let's just plug in the sine wave function into our recurrence relation.

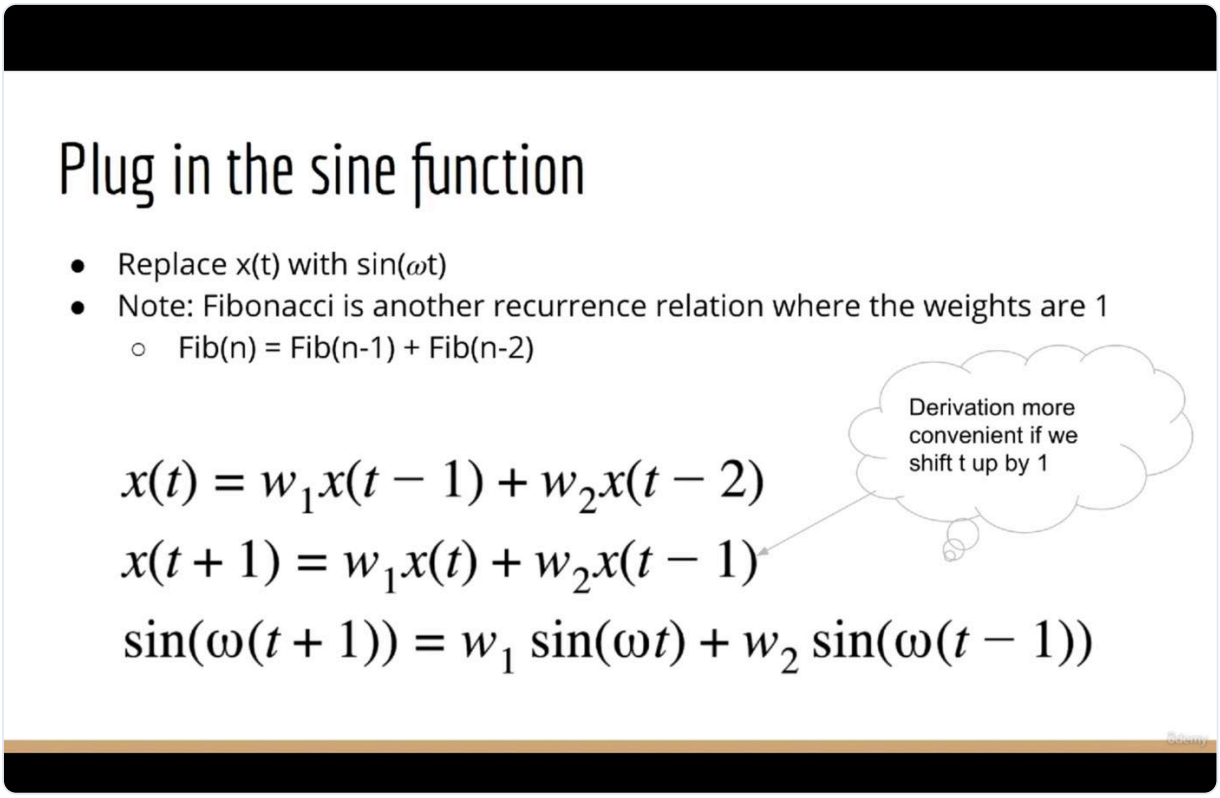


FIGURE 11.4 | Plug in the Sine Function

By the way recurrence relation is just a fancy way of describing the autoregressive model. The Fibonacci equation is also an example of a recurrence relation. In fact it has the exact same form as our AR2 model with weights of 1 for this derivation. It's actually more convenient to shift the time steps up by 1.

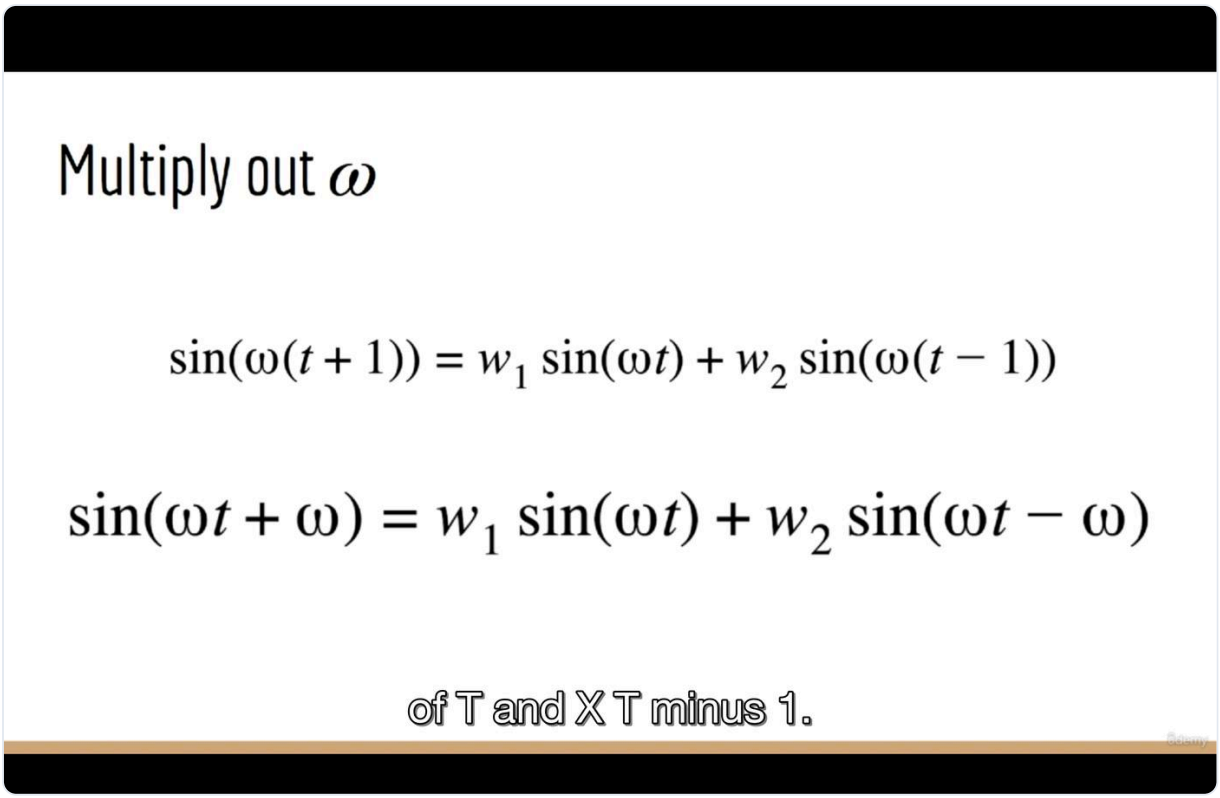


FIGURE 11.5 | Multiply Out a

So instead of predicting x of T using X T minus 1 and X T minus 2 will predict x of T plus 1 using x of T and X T minus 1. The next step is to multiply out the terms inside of the sine functions. This still may not look like much but let's keep going.

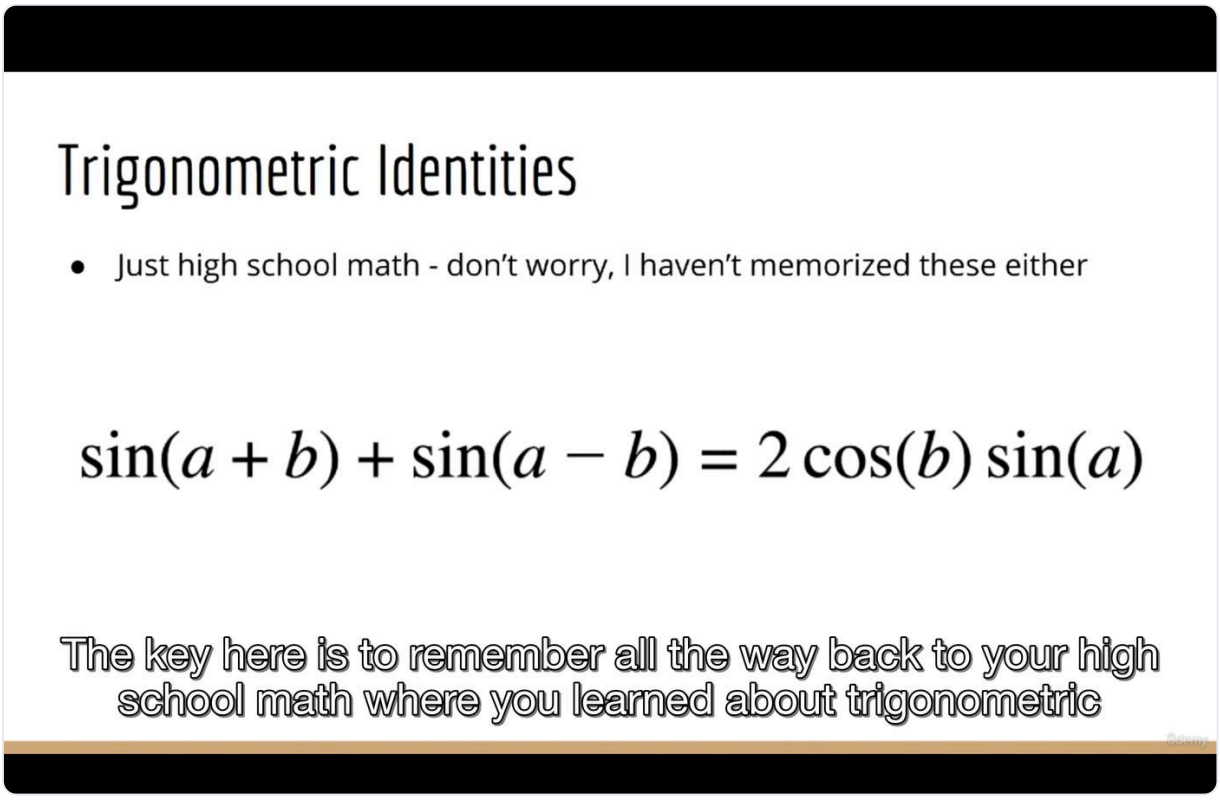


FIGURE 11.6 | Trigonometric Identities

The key here is to remember all the way back to your high school math where you learned about trigonometric identities. Now don't worry I've forgotten these as well. These are not the type of thing you're using your day to day machine learning work so don't be too sad. The important identity is this one.

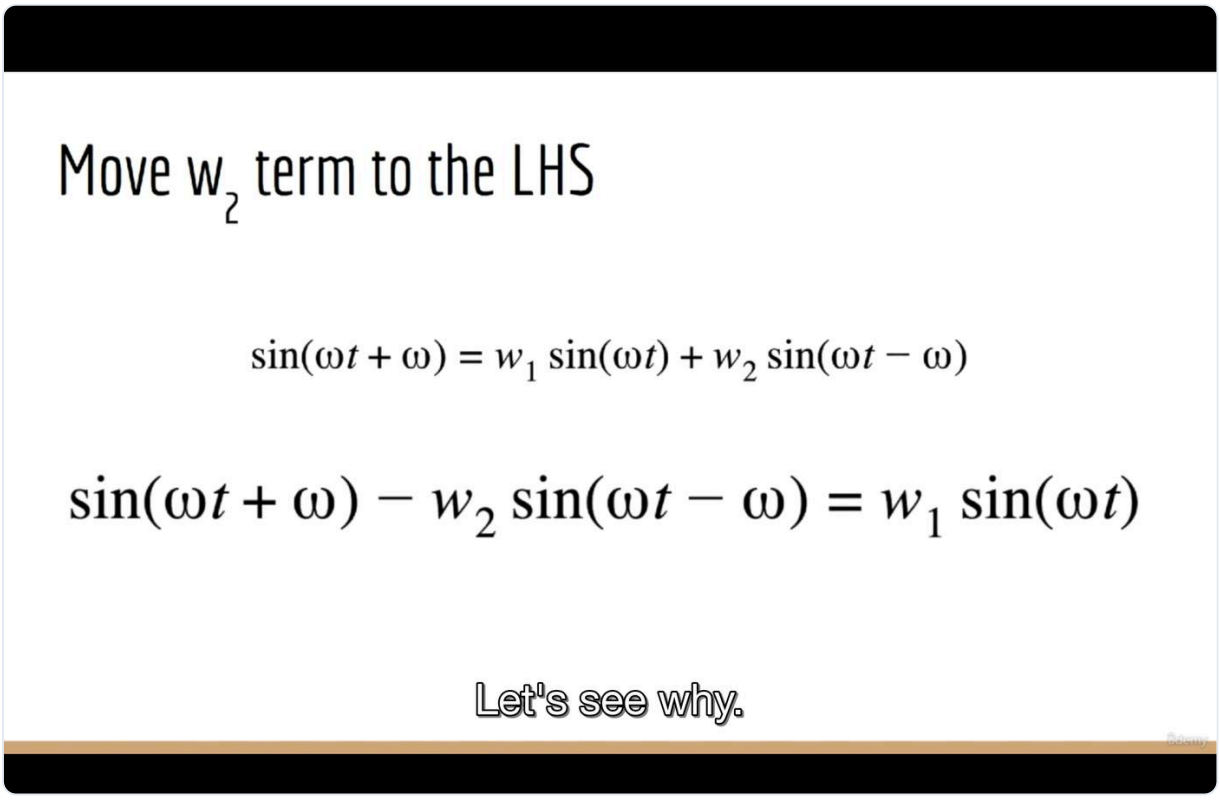


FIGURE 11.7 | Move W, Term to the LHS

It's the sine of a plus b plus the sine of a minus B equals two cos B sine A. Let's see why. First we can manipulate our sine omega t equation by bringing the W term to the left hand side. So now we have sine of omega T plus Omega minus W2 times sine of omega T minus Omega equals w one times sine of omega T.

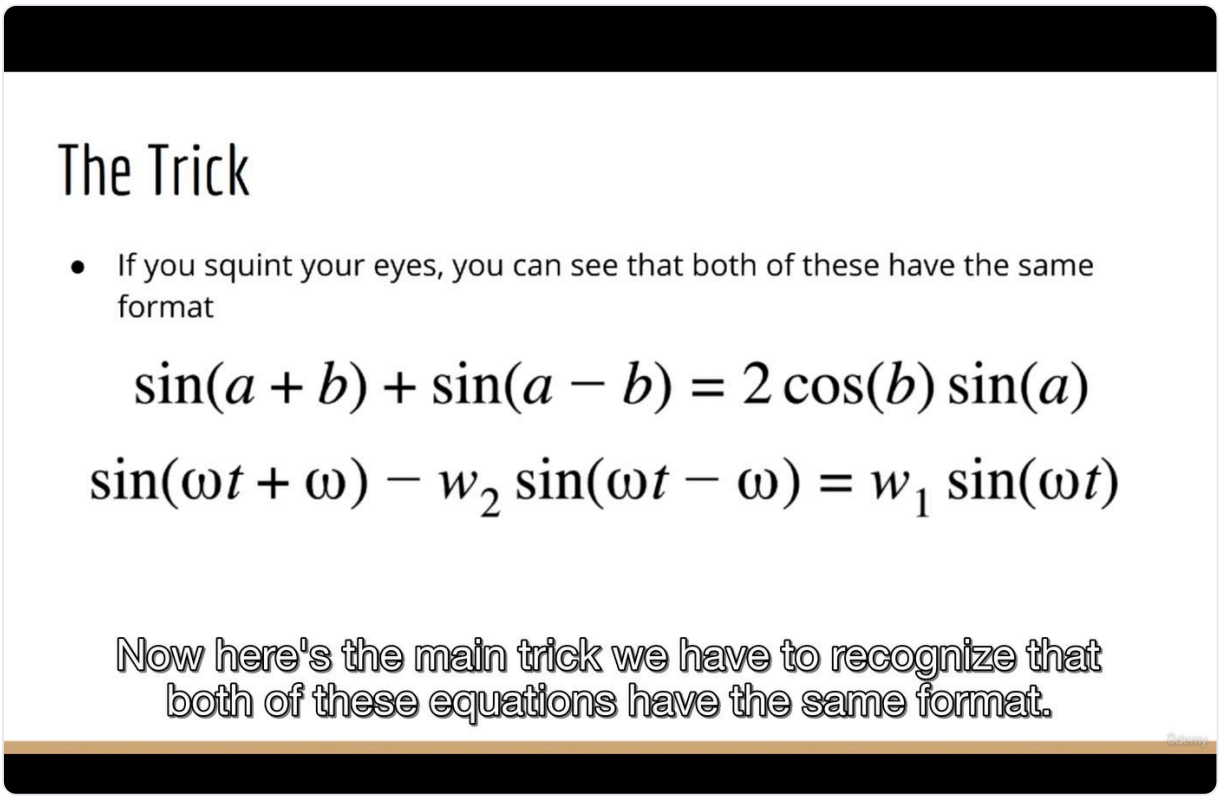


FIGURE 11.8

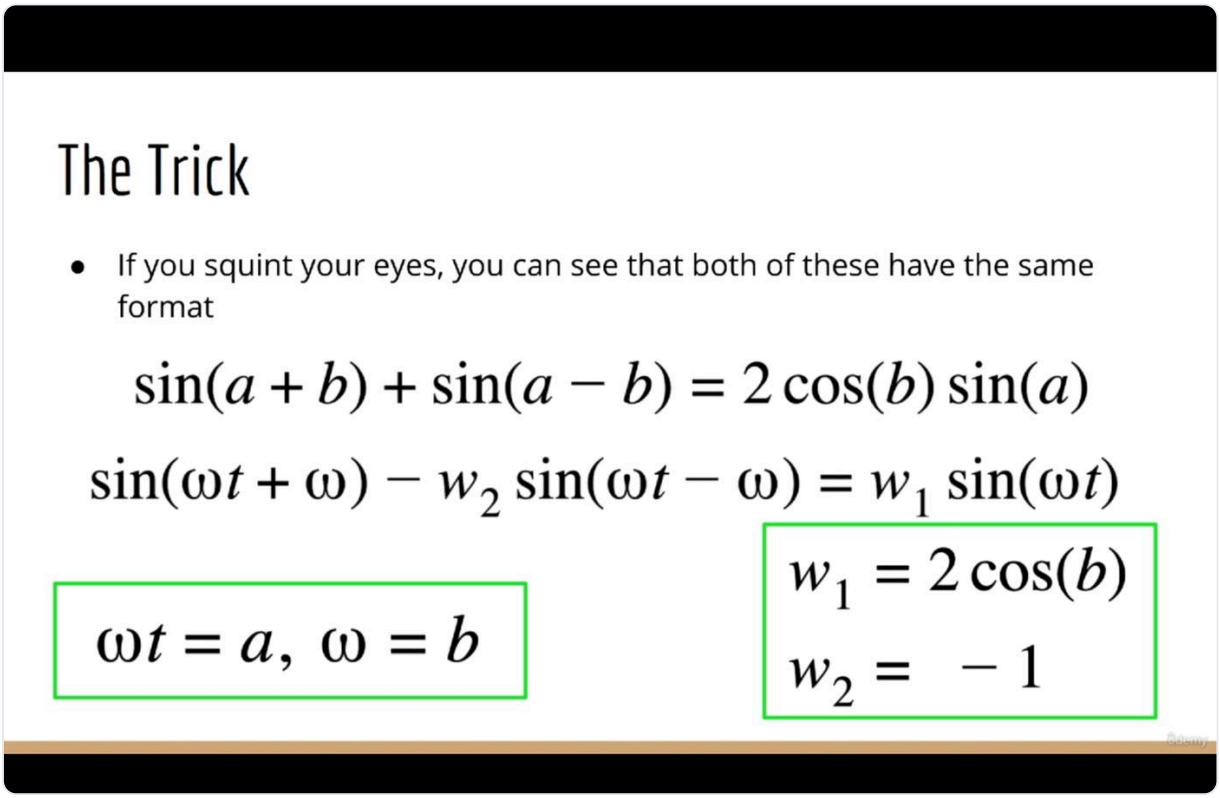


FIGURE 11.9 | The Trick

Now here's the main trick we have to recognize that both of these equations have the same format. If we look inside the sine terms we see that omega T is just a and Omega is just B immediately. This helps us figure out the constants w 1 and W2. We can already see that W2 must equal minus 1. This also means that w 1 is equal to 2 cos B which is equal to 2 cos Omega. Thus we prove that we can represent the sine wave as a recurrence relation.

LECTURE 50

Calculating Our Savings

In this lecture we are going to begin discussing recurrent neural networks. Let's first think about why an RNN would be useful first. As usual we are going to start with our dumb as possible approach. What is the simplest thing we could do. Let's think back to how we approach the image classification. Before we learned about CNNs which are specialized for looking at images we used ANNs. We were able to do this because we took an image and we just flattened it into a feature vector which is the only type of data you can pass into an ANN. Recall that this goes along with our rule. All data is the same. It doesn't matter if your feature vector is a flattened image or some data you collected in a survey.

The ANN doesn't care so let's consider whether we can take the same approach now. In fact we already did when we looked at our time series forecasting model. We just said pretend t is D and pass that N by T matrix into a linear regression model. The model has no idea what you're passing in as a time series. It's just doing the same algorithm it's always done. But here's the question what if your D is not one. In other words you have a multi-dimensional time series recall that one example of this is if you work at Neuralink and you've implanted multiple electrodes into your brain. Now you have multiple recordings at every time step making it a multi-dimensional time series Well here's an idea that requires pretty much no imagination at all.

Why not flatten this data to let us take a T by D matrix and turn it into a T times D sized vector. Let's say you have five electrodes in your head and you make a recording with sequence length one hundred so one hundred times steps. Then if you flatten this you would just get a vector of size 500 in fact this is even easier to visualize than a flattened image. Here we just have five different time series signals. Now all we're going to do is concatenate them together and say it's one big vector. Easy peasy all right but we already know that this approach is possible and we've pretty much done it in code already.

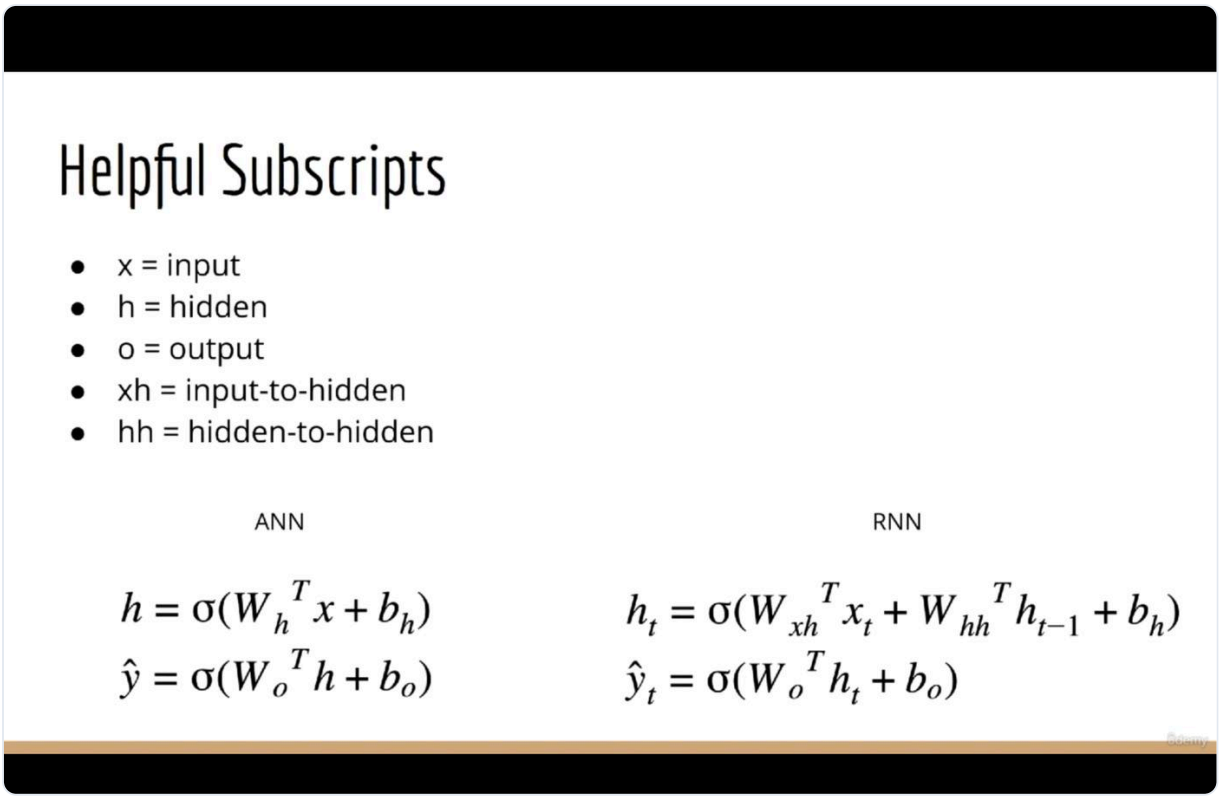


FIGURE 12.1

If you're confused what I mean by that it would be pretty much the same as our M.A. Script which at this point is pretty simple compared to what we've done so far so let's go to the next step and think about why that might not be a good idea. The first problem is the same as with CNNs. If you were to do a full matrix multiplication. It would just take up way too much space if d equals one hundred and T equals ten thousand which by the way is not unrealistic. Then all of a sudden your flattened feature vector is of size 1 million. You probably don't want a one million size feature vector going into your ANN as with CNNs RNNs take advantage of the special structure of the data an ANN is the most general.

It just connects every input to every feature every feature in other words is directly composed of every single input. We know that for images this is not really a great idea because we're actually looking for very tiny patterns relative to the full image. The question is In what way can we exploit the structure of a sequence here's the idea we can take inspiration from forecasting. We know that to predict the next value past values are useful but what if we apply this to the hidden feature vectors. So let's take an ANN where the hidden feature vector is calculated from the input vector. The output is then calculated from the hidden vector now to make an RNN all we need to do is make the hidden vector loop back to itself.

In other words the hidden vector depends not only on the input but also on its own previous value. This is just like our very basic linear regression forecasting model where the prediction was just a linear function of past values. Now we are saying the hidden state is a nonlinear function of past values specifically that the nonlinear function is a neuron. As a side note the loop back implicitly assumes that there's a time delay of one step all right so how do we calculate the output of an RNN. Well we use the calculation you see here as you can see there's only one new thing the hidden state h of T now depends on the hidden state at the previous time step as well.

Also x y and H are index by time but that should be a given. Some other assumptions we made here are that we're using the sigmoid activation for both the hidden and output layers. This is of course not necessary but we need to write something down so that's what I decided to write down. As usual output

activation is dependent on the tasks being done. So a sigmoid for binary classification. Nothing for regression and a softmax for multi class classification.

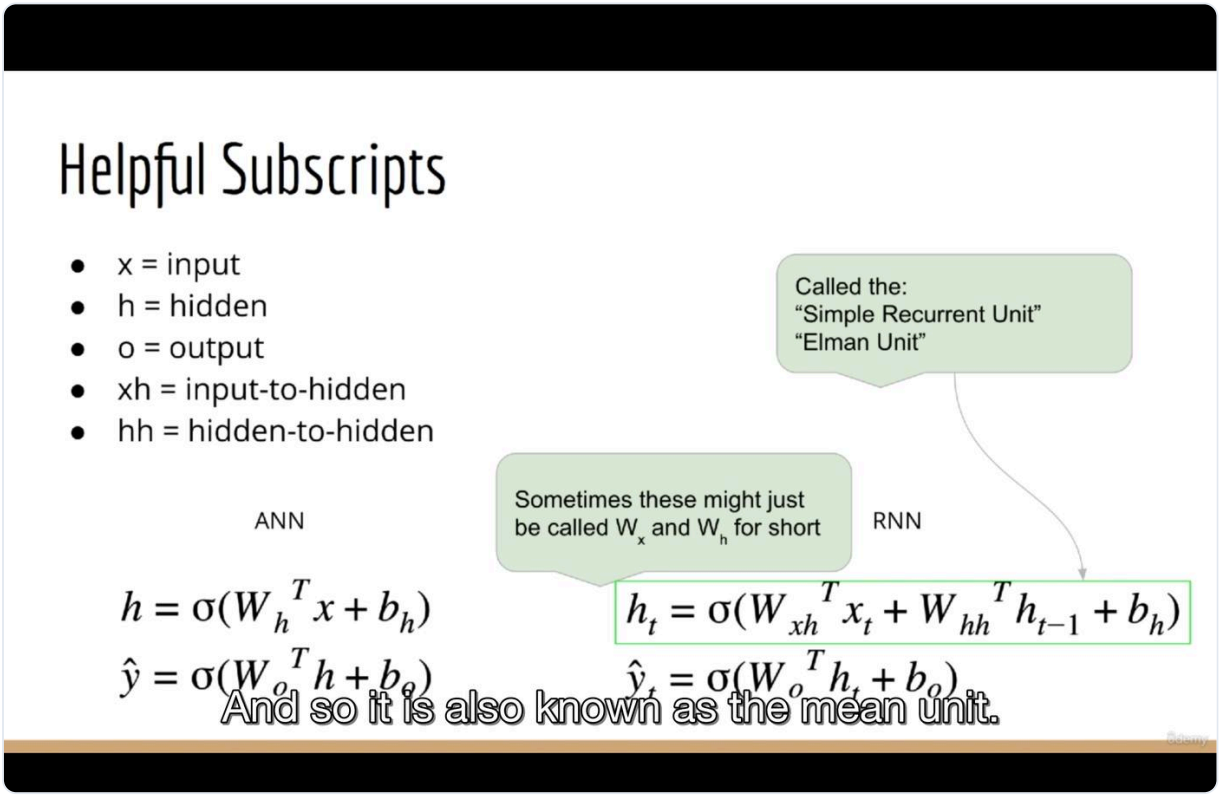


FIGURE 12.2 | Called the:

Another assumption is that this neural network only has one hidden layer. In fact with RNNs this is pretty often the case. You could stack up more hidden layers but I don't see that too often. As usual you'll want to test that out on your particular dataset. If you think it might be a good idea it's helpful to name these weight matrices so we don't get them confused later on. Luckily it's pretty intuitive. The reason we call w x H W H is because the input is X and the output is H. So we call this the input to hidden weight. The reason we call w h h w h h is because the input is the previous H and the output is the next H. So we call this the hidden to hidden weight.

B of h we call the hidden bias and finally w o and b o are the output weight and output bias. The important thing there was to differentiate between w x h and w h h since they are both weights belonging to the recurrent layer. As a side note this layer itself has several names the most common name is just the simple recurrent unit but since this has been studied in the past it's also associated with a person. And so it is also known as the Elman unit. All right so for most people these equations are pretty intuitive but for some it's not. So let's just be super clear about how we were going to calculate the output prediction given a sequence of inputs.

Actually this will help us uncover some hidden details that we need to consider that are not obvious at first. First we assume that we are given a sequence of input vectors x 1 all the way up to X Big T each individual X is of course a vector of size D. So this entire sequence would be stored in code as a matrix of size big T by D. In math however we assume that we are working with individual vectors. So let's start with x 1 from x 1. We can calculate H1 but wait. Now you see one of these hidden details that you probably didn't think about H1 depends on h 0. But what is h zero.

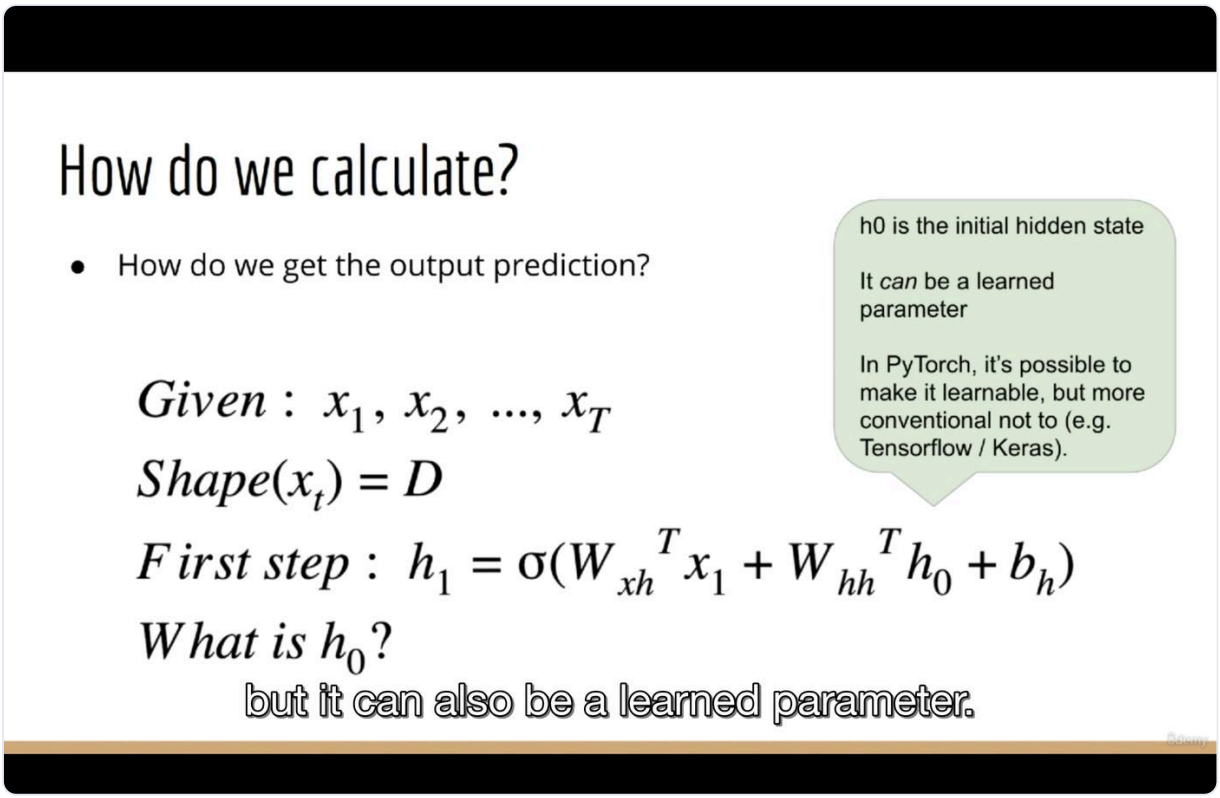


FIGURE 12.3 | How Do We Calculate?

In fact we refer to this as the initial hidden state typically we just set this to an array of zeroes but it can also be a learned parameter. In other words you can learn this vector using gradient descent in PyTorch. It's possible to make the initial state a learnable parameter but it's simpler not to. And in fact quite conventional. So in this course that's the approach we're going to take. All right.

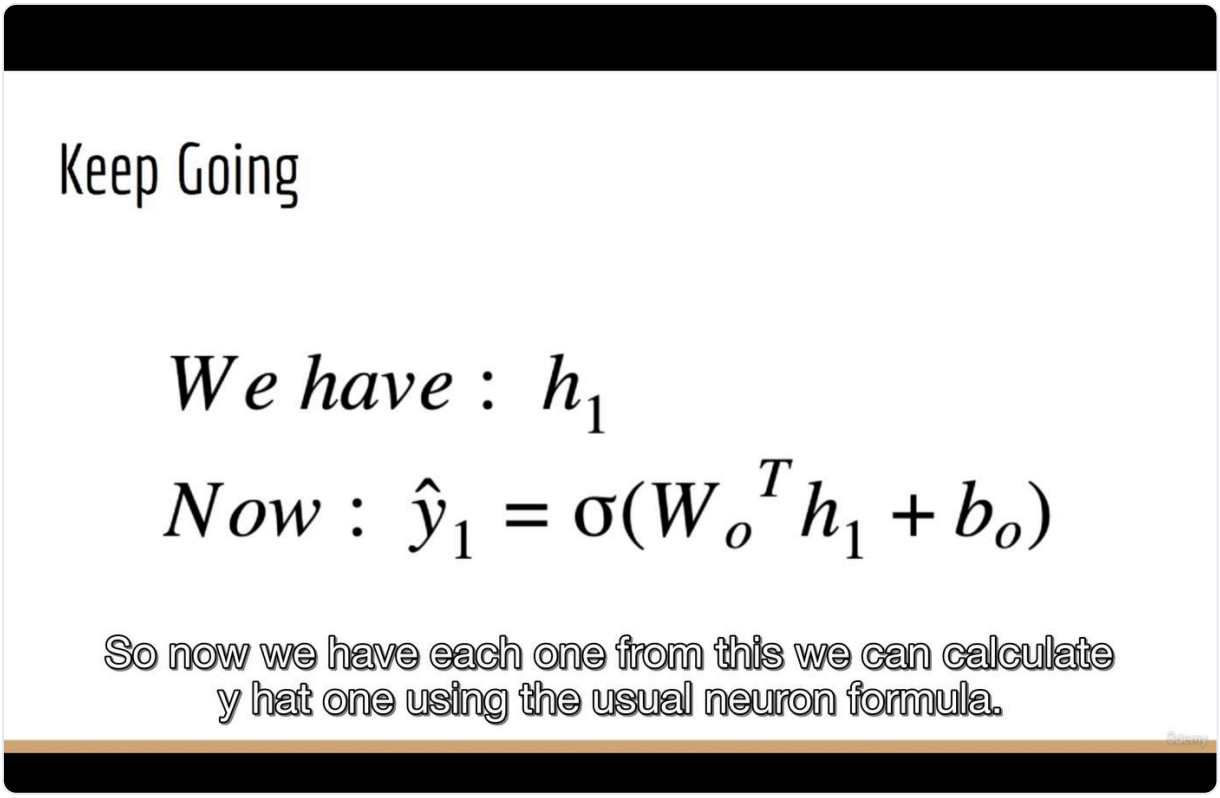


FIGURE 12.4 | Keep Going

So now we have h one from this we can calculate y hat one using the usual neuron formula.

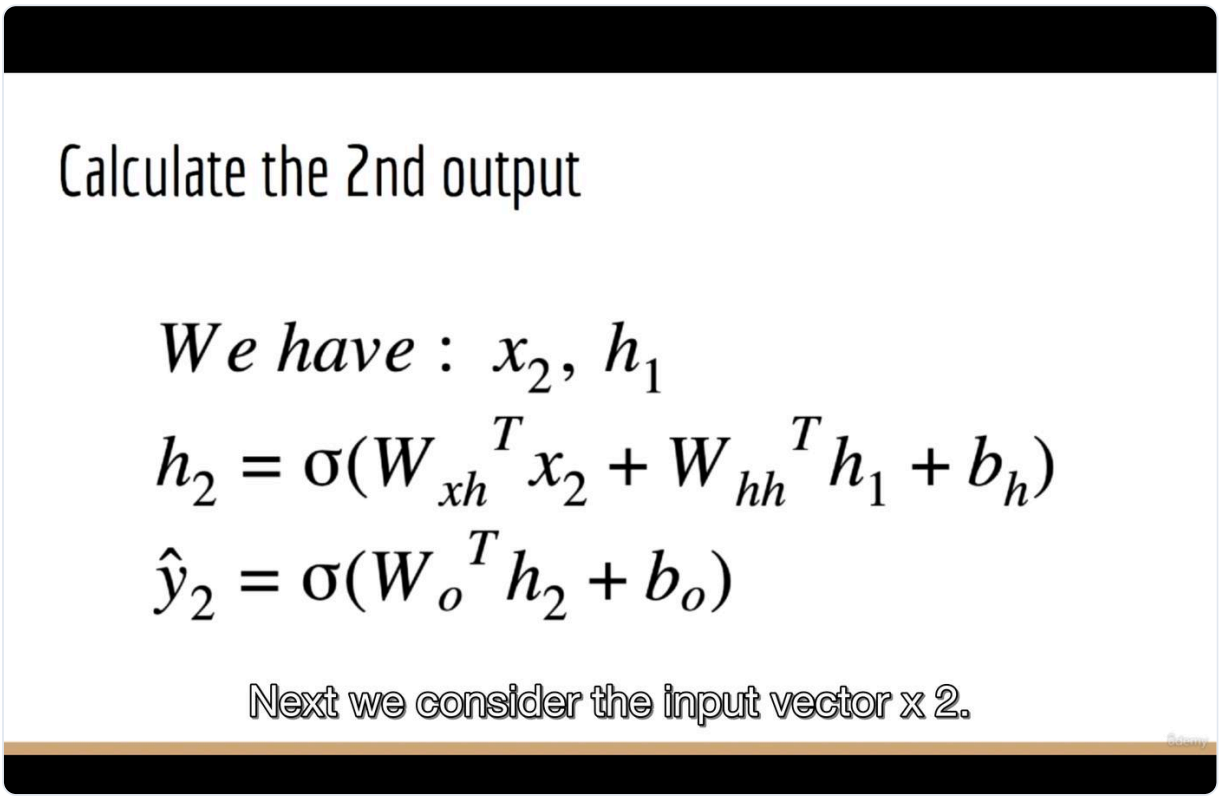


FIGURE 12.5 | Calculate the 2nd Output

Next we consider the input vector x 2. From this we can calculate H2 H2 depends on x2 and H1. We just calculated h one. So that's not a problem. From this we can calculate y hat two now we can repeat this process to calculate h 3 and y hat 3 h 4 and y hat 4 and so on all the way up to h big T.

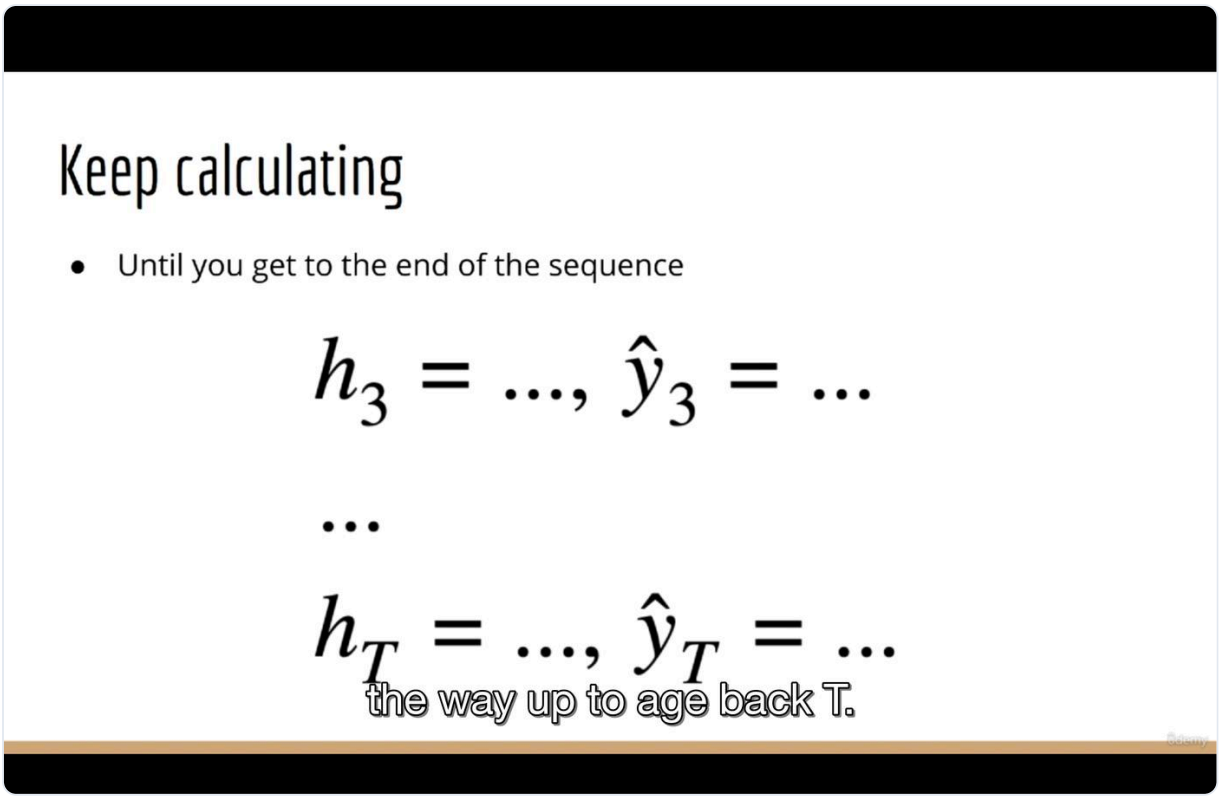


FIGURE 12.6 | Keep Calculating

And y hat big T Okay great. That all makes sense by the way. If you found that a little confusing don't worry because we are also going to do this calculation in code as well. Just as a little extra practice to make sure you understand how it works. But here's something you probably noticed which is strange.

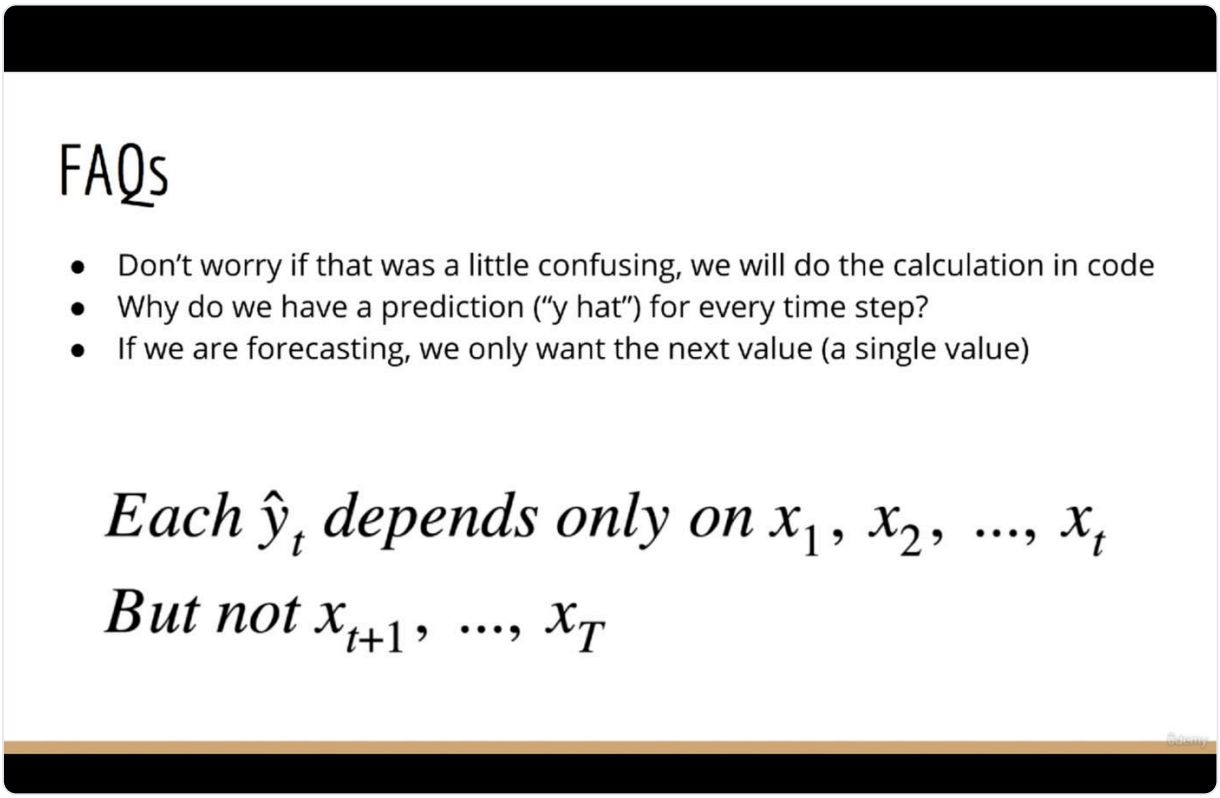


FIGURE 12.7

Why do we have a y hat for every time step if we are doing something like forecasting. We only want to predict the very next value if we are predicting something like the sentiment of a tweet. We only have one answer. Positive sentiment or negative sentiment. So what is the purpose of having a y hat for each time step. Each of the Y hats depends only on the x's up to that point.

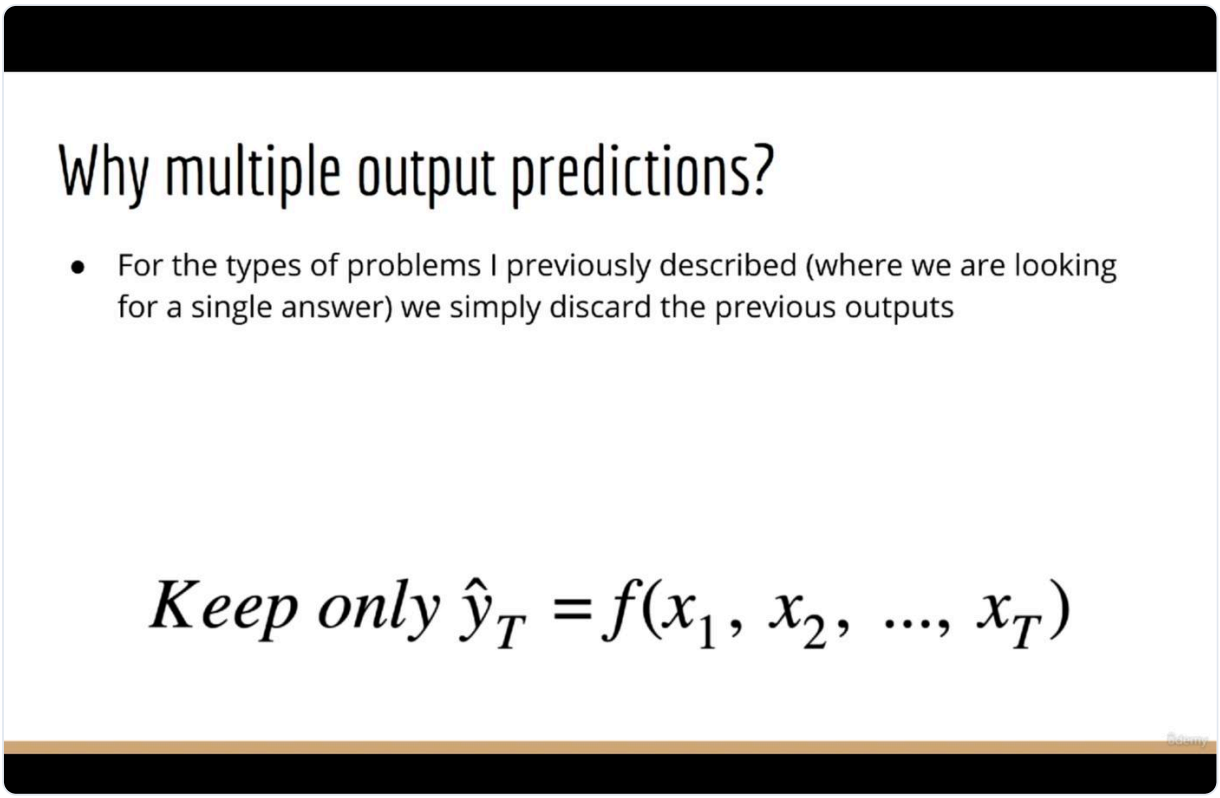


FIGURE 12.8 | Why Multiple Output Predictions?

The answer is that for these types of problems which I just described all the y hats except for the final y hat are ignored. So why hat of Big T gives us the final prediction and all the previous y hats are discarded. You can think of them as like temporary variables at the same time. There are cases where we would want to keep all the y hats at each time. One such example is neural machine translation in a neural machine translation. Both your input and your output are sentences.

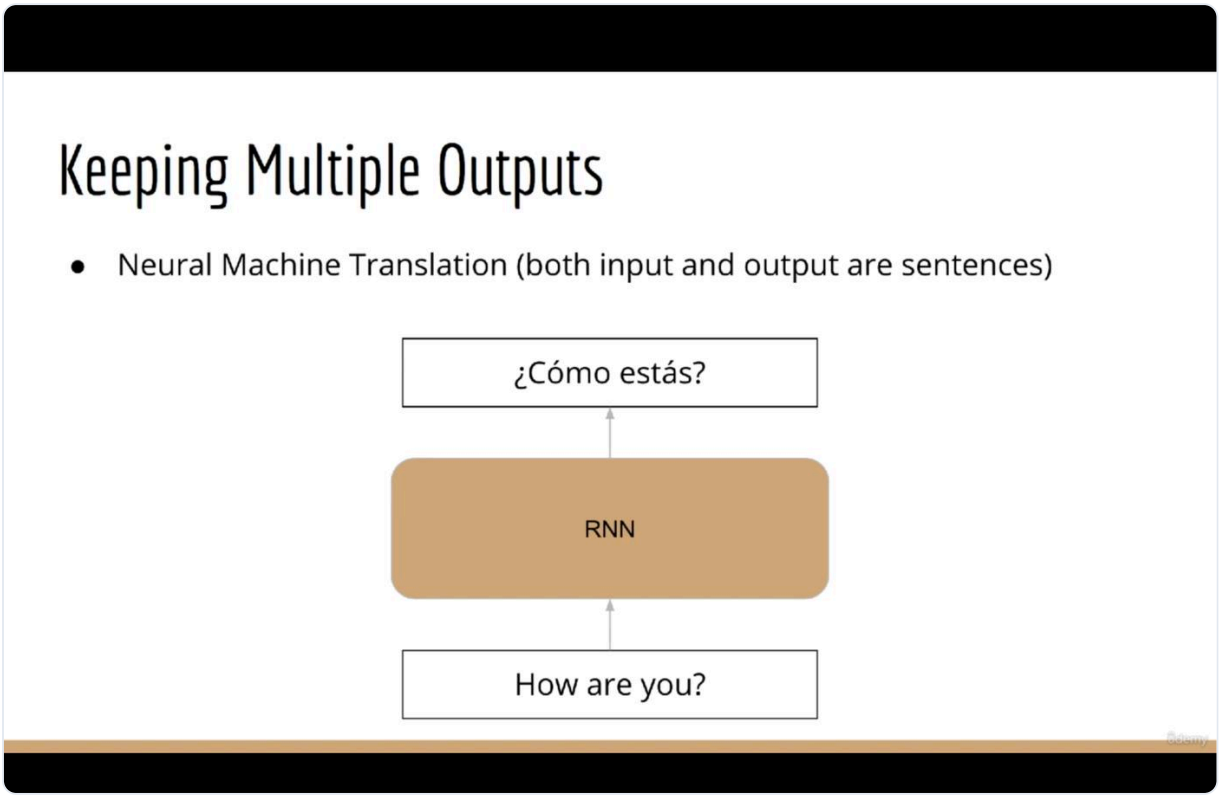


FIGURE 12.9

They are just sentences in different languages. In this case both the input and the target are sequences and therefore you need to capture the predictions at each time point considering neural machine translation. Let's think about how we can reason about what an RNN is predicting in probabilistic terms. If you recall when we look at ANNs and hence this also applies to CNNs that the output prediction after applying the softmax function is a probability distribution for a regular ANN or a CNN. It's the probability that Y equals K given x x is the input vector or in the case of CNNs an image and y is the label for neural machine translation. This is also classification.

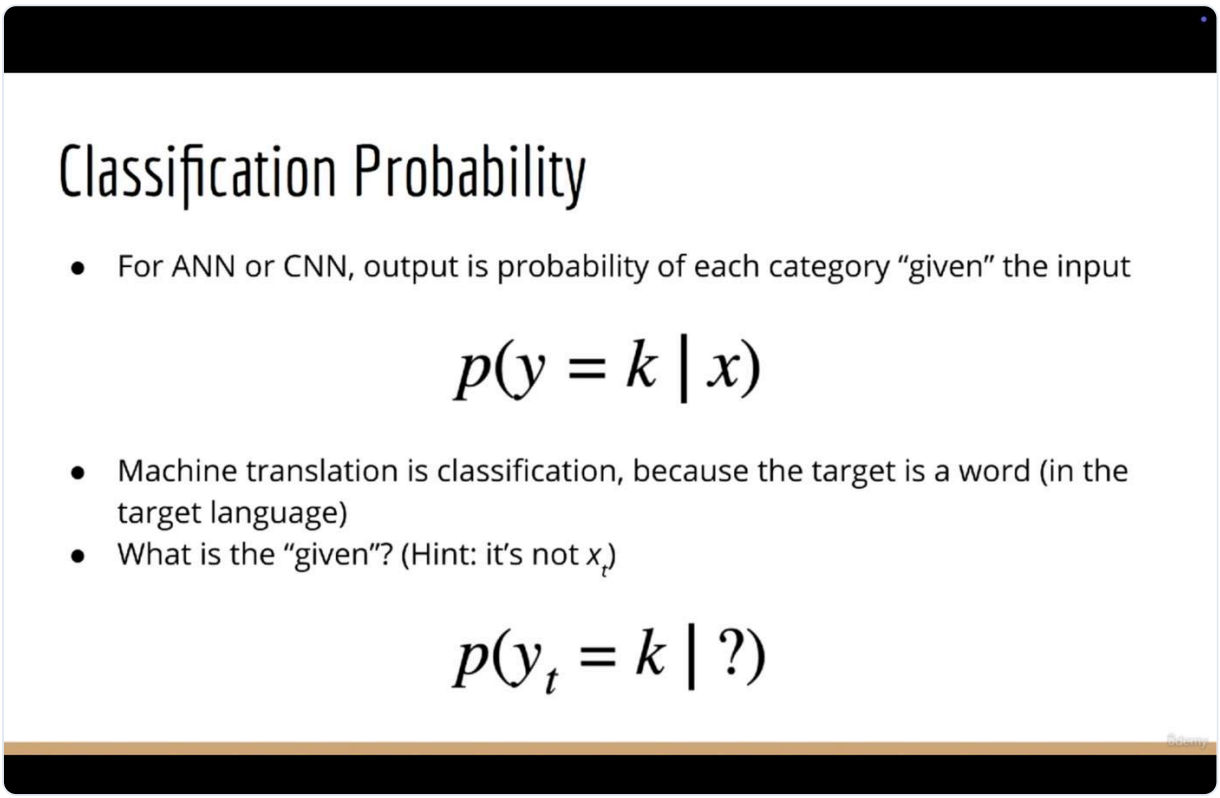


FIGURE 12.10 | Classification Probability

We are trying to predict a category which is a word in the target language. So the question for RNNs is if we are doing classification we will have the probability of Y of T equals K given something.

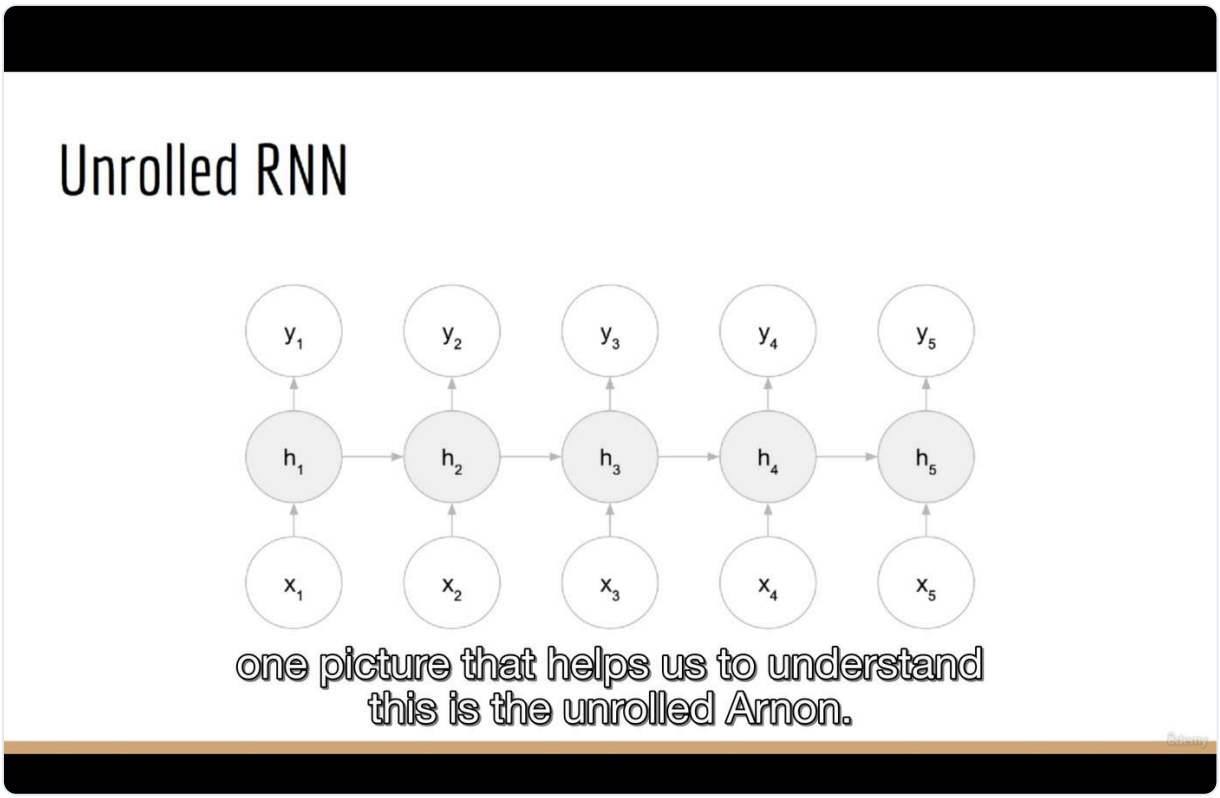


FIGURE 12.11

But what is this something one picture that helps us to understand this is the unrolled RNN. This is like a block diagram of all of our calculations and their dependencies. First we can see that h one depends on h 0 and x 1 y hat one. It depends on h one and therefore it indirectly depends on X1 H2. Depends on H1 and X 2. Y hat two depends on H2.

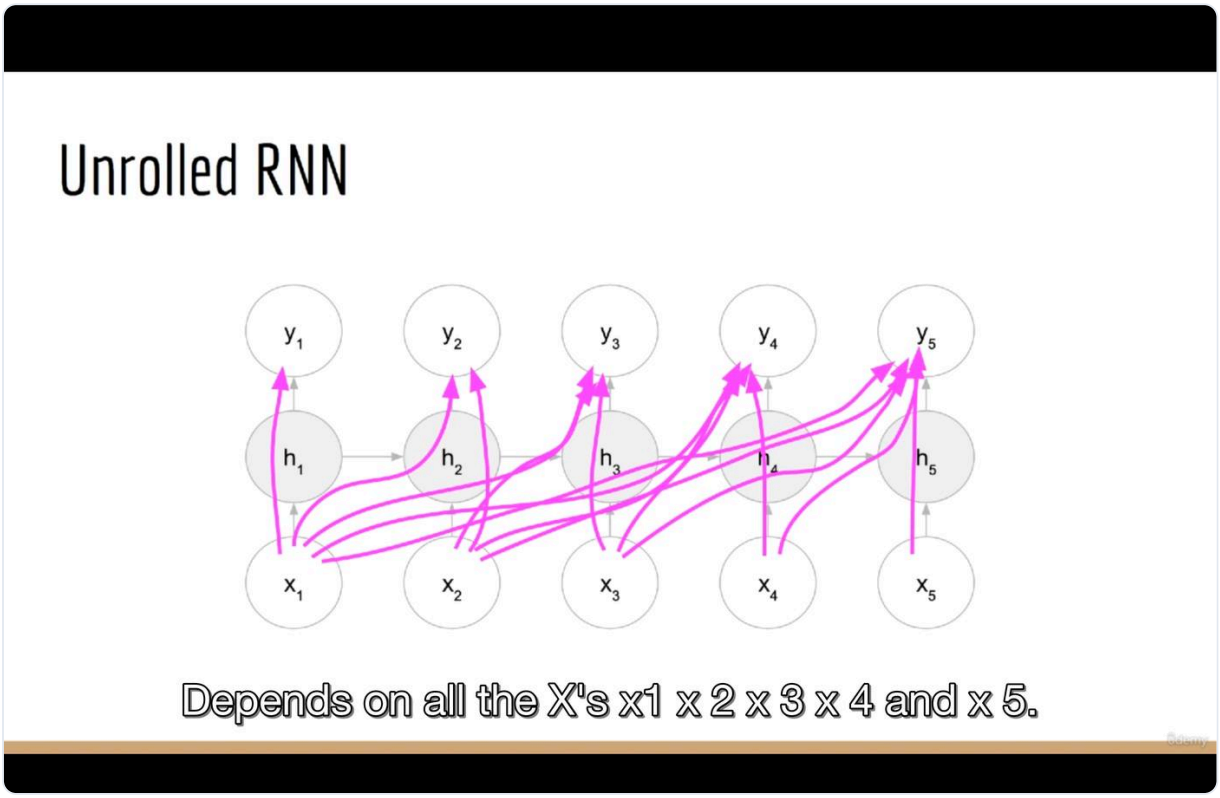


FIGURE 12.12

Thus y hat two indirectly depends on X1 and X2 similarly y hat 3 depends on X1 X2 and X3. Y hat 4 depends on X1 X2 X3 and x 4. Finally y hat 5 depends on all the Xs x1 x 2 x 3 x 4 and x 5. In general we can say that y hat big T depends on every X in the input sequence. It should be clear from this picture that each y of T overall depends only on the current and past values of x. So what does this mean in terms of the probability distribution that the RNN is modelling the probability of Y one equals K given X1 is the first distribution. But now notice that y hat two depends on both x 2 and h one which in turn depends on x 1.

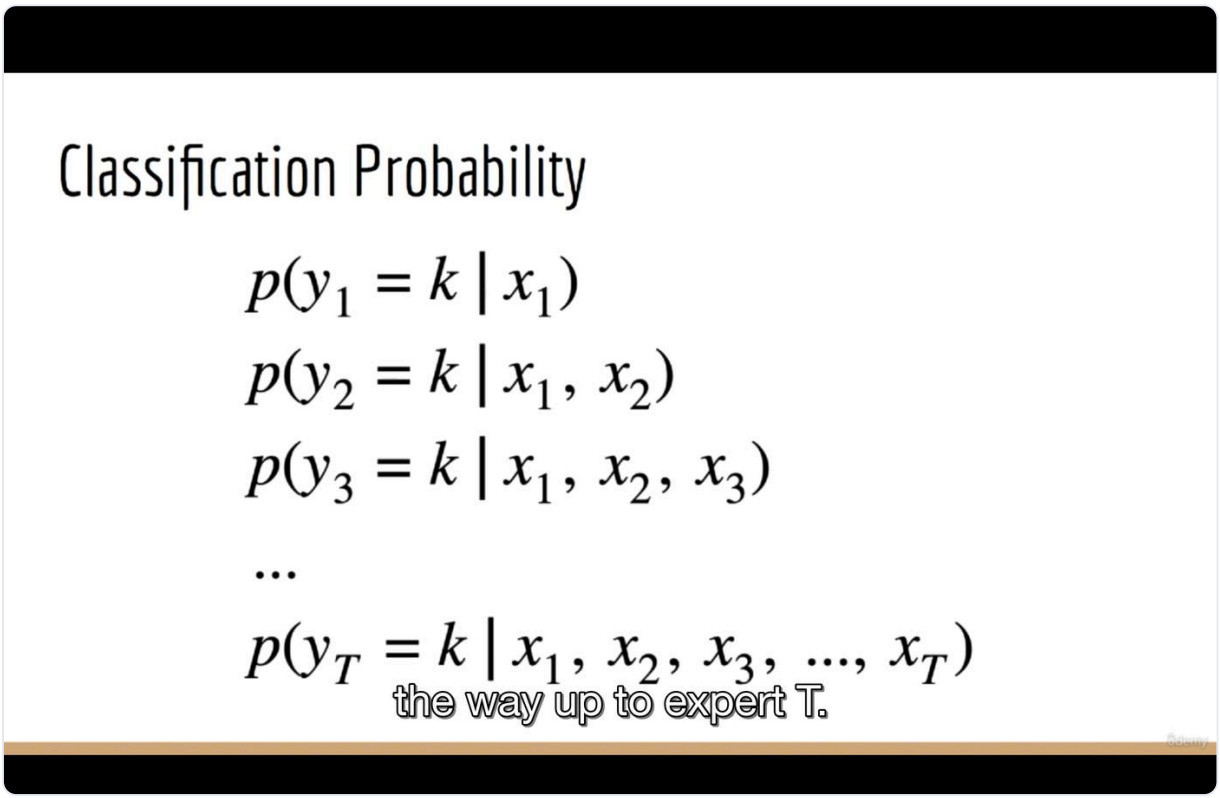


FIGURE 12.13 | Classification Probability

So we write the probability of Y of 2 equals K given x 1 and x 2 Next we have that the probability why three equals K given x 1 x 2 and x 3. And this pattern continues all the way up to the probability of y hat big T equals K given x1 x 2 x 3 all the way up to x big T. Now this is not that important in and of itself. You're not really going to be manipulating these probabilities or doing anything mathematically but it's interesting. If you have ever studied Markov models the idea with the Markov model is that they make the Markov assumption. What is the Markov assumption.

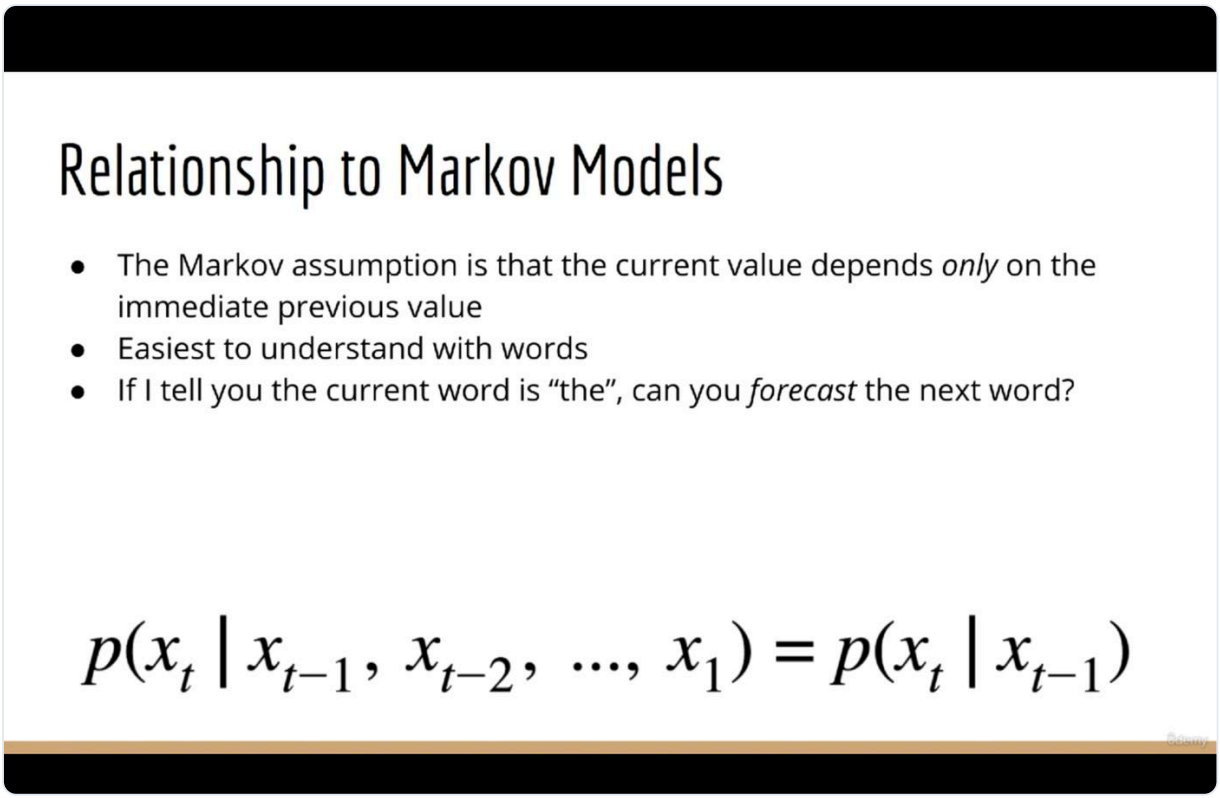


FIGURE 12.14 | Relationship to Markov Models

It's easiest to understand with words the Markov assumption says that the probability of the next word depends only on the current word in the sentence but not on any previous words. You might think that sounds very unrealistic. For example if the current word in a sentence is the how can you possibly predict the next word in a sentence the next where it could be any number of words. So Markov models are pretty weak if you use them that way now consider an RNN for predicting the next word in a sentence. So we're modeling the probability of x of T plus 1 the word at time T plus 1 then our output distribution is the probability of X at T plus 1 given X of one x of 2 all the way up to x of T.

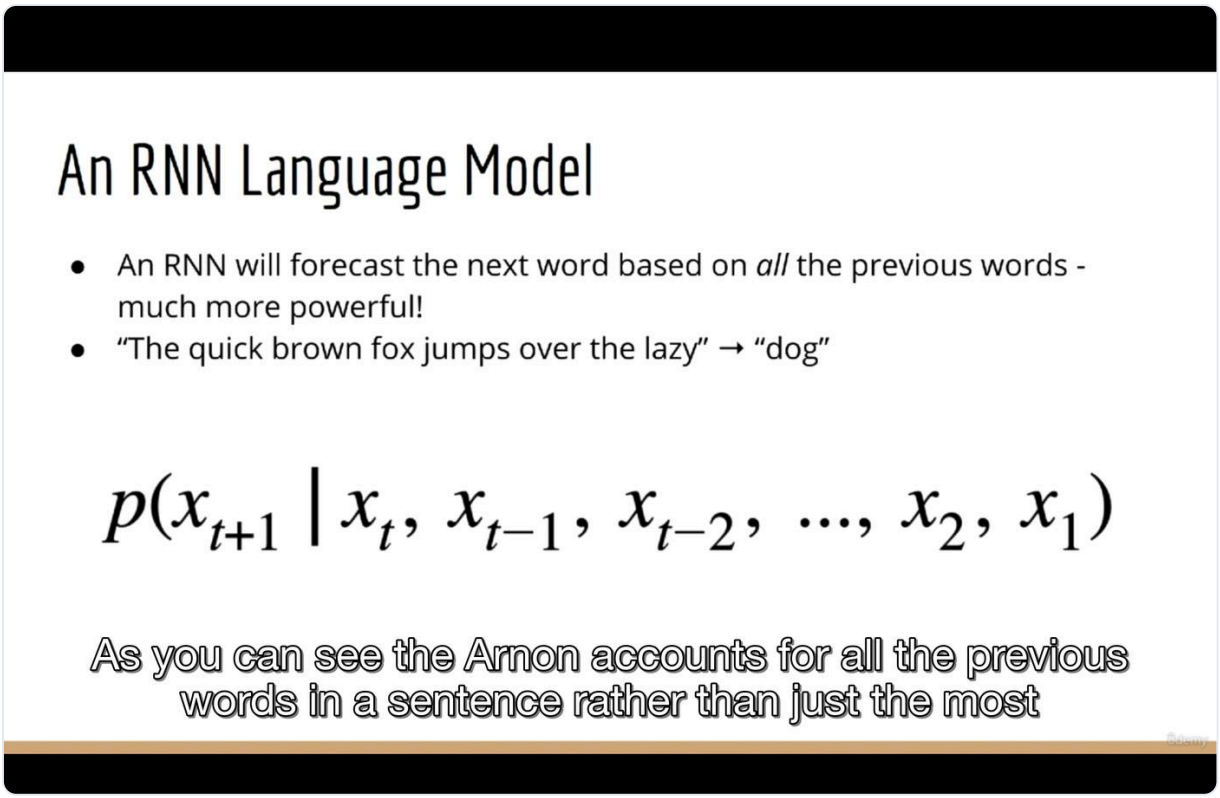


FIGURE 12.15 | An RNN Language Model

As you can see the RNN accounts for all the previous words in a sentence rather than just the most recent word. This is one reason why RNNs are so powerful for modeling sequences as a final thought exercise for helping you understand RNNs. Let's write some pseudocode that would give us the y hats at each time. First we have to consider what we're given. We are given the initial state h 0 and the weights of the RNN so we have w x H which maps from X to h we have w h h which maps from the previous H to the current H. We have B of H the bias term for the recurrent hidden layer we have w o the output weight and b o the output bias and of course we're also given a single input sequence X which has the shape T by D.

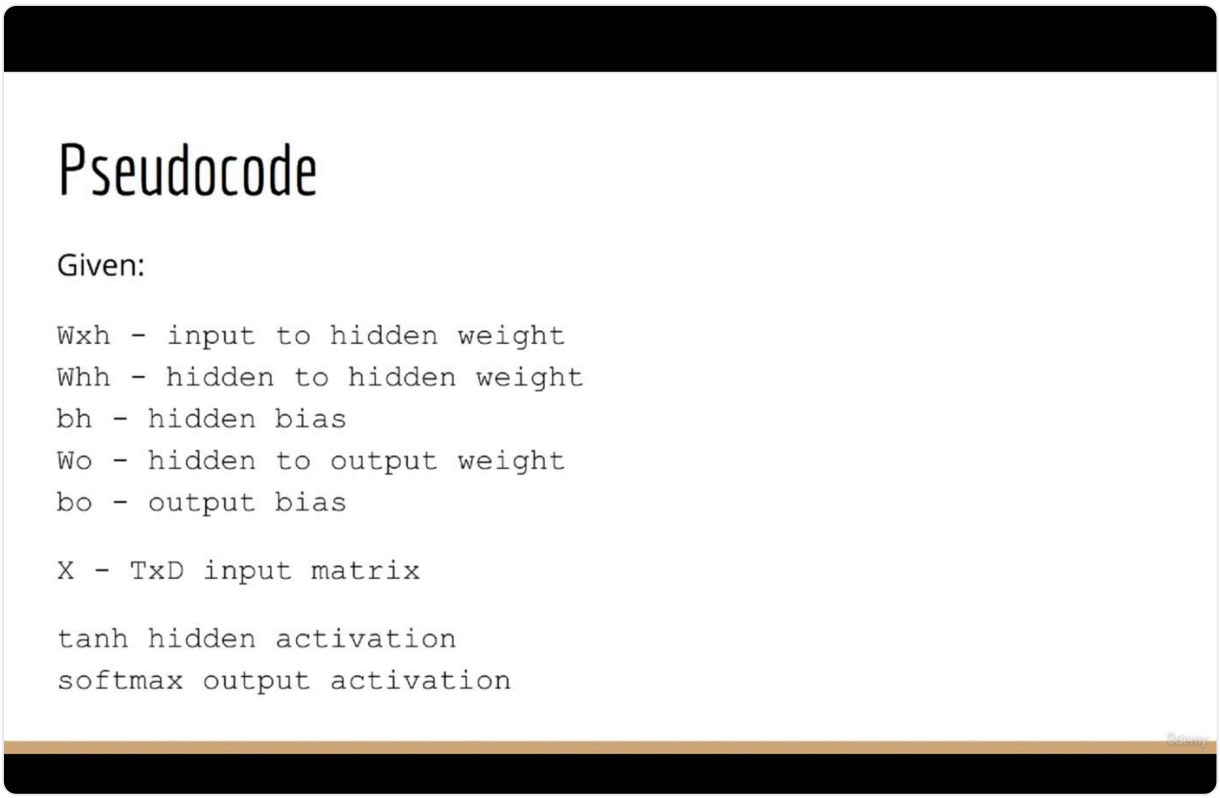


FIGURE 12.16 | Wxh - Input to Hidden Weight

It represents t sequential vectors of size D. Let us assume a tanh activation and a softmax at the output okay. So to calculate the output predictions we're going to start by initializing y hat the output as an empty list. Then we're going to set H last the previous value of H T H zero then we are going to loop through each time step from one up to big T inside the loop. We're going to calculate h of T using the recurrent neuron equation we saw earlier. Then we can use h of t to calculate y hat at time T.

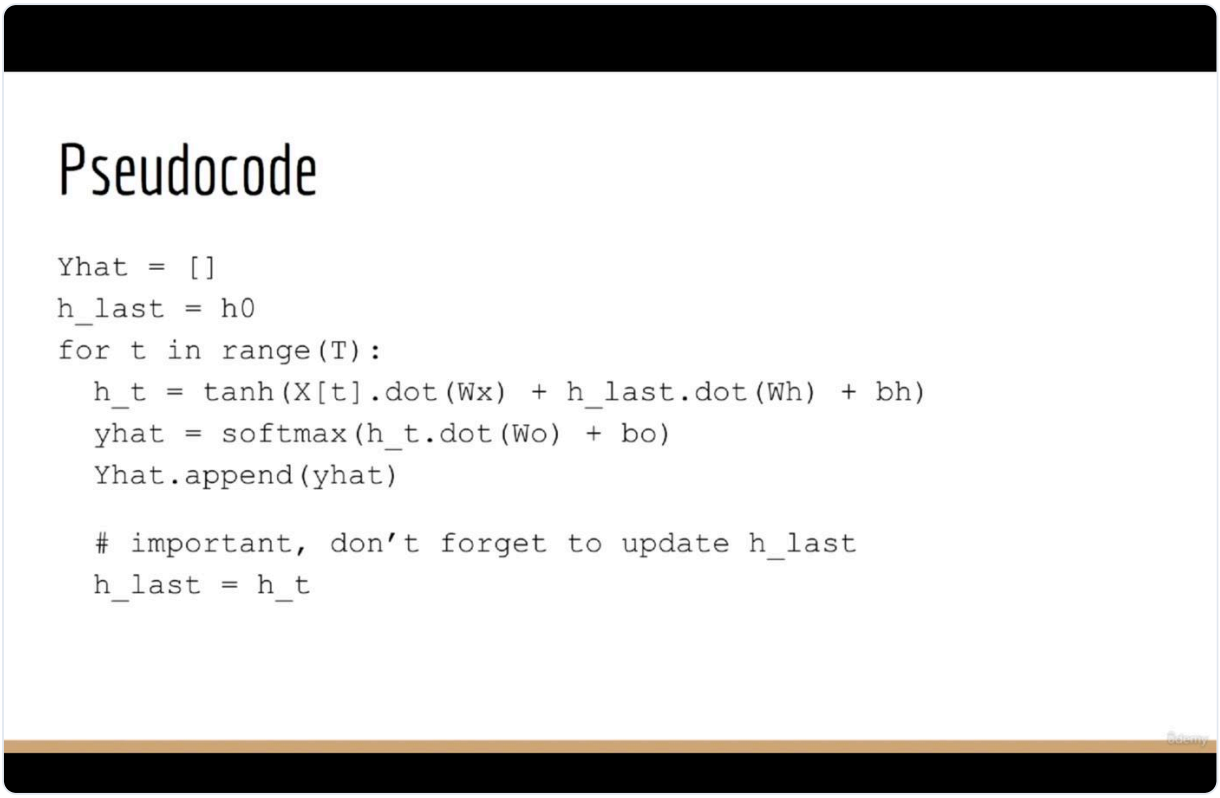


FIGURE 12.17

As usual by the way this is just the dense layer next we append y hat to the list of predictions y finally and this step is important not to forget we assign h of t to H last so that H last always represents the last value of H of t when we do our calculation for the next stage of T at this point we are going to briefly go back to the biological inspiration for neural networks. Recall earlier when we talked about how neurons are organized in the brain we can imagine that there is no reason for the neurons in your brain to all go in one direction. In fact recurrent circuits in the brain have been well studied in biology. The idea that neurons can connect in a loop goes back decades.

And personally I find this topic very interesting. One such example is the Hopfield network one of the methods used to train Hopfield networks is called Hebbian learning invented by Donald Hebb.

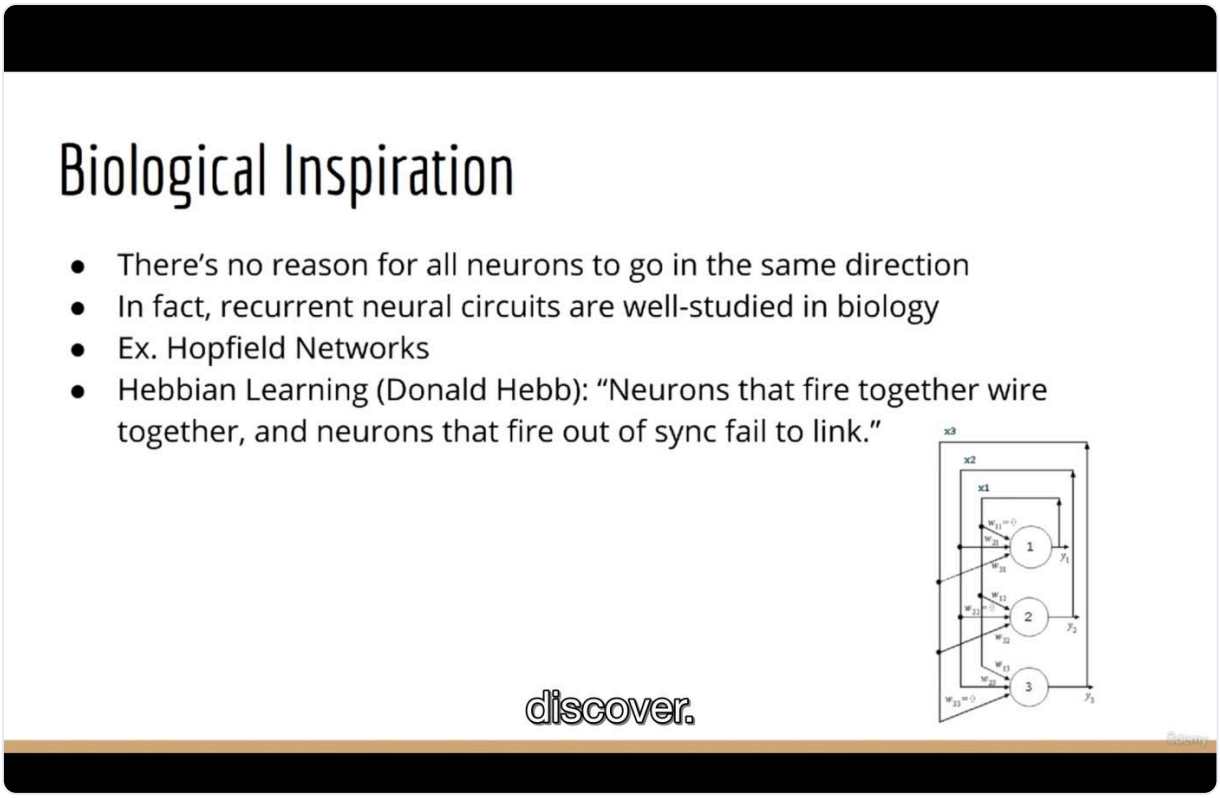


FIGURE 12.18 | Biological Inspiration

He inspired the famous rule neurons that fire together wire together and neurons that fire out of sync fail to link. It's actually quite unfortunate that most people have a very narrowly focused view on deep learning. It would be very interesting to see if we applied modern technology to these old ideas what we could discover. The last thing we are going to do in this lecture now that you understand RNNs is to ask the question what are our savings.

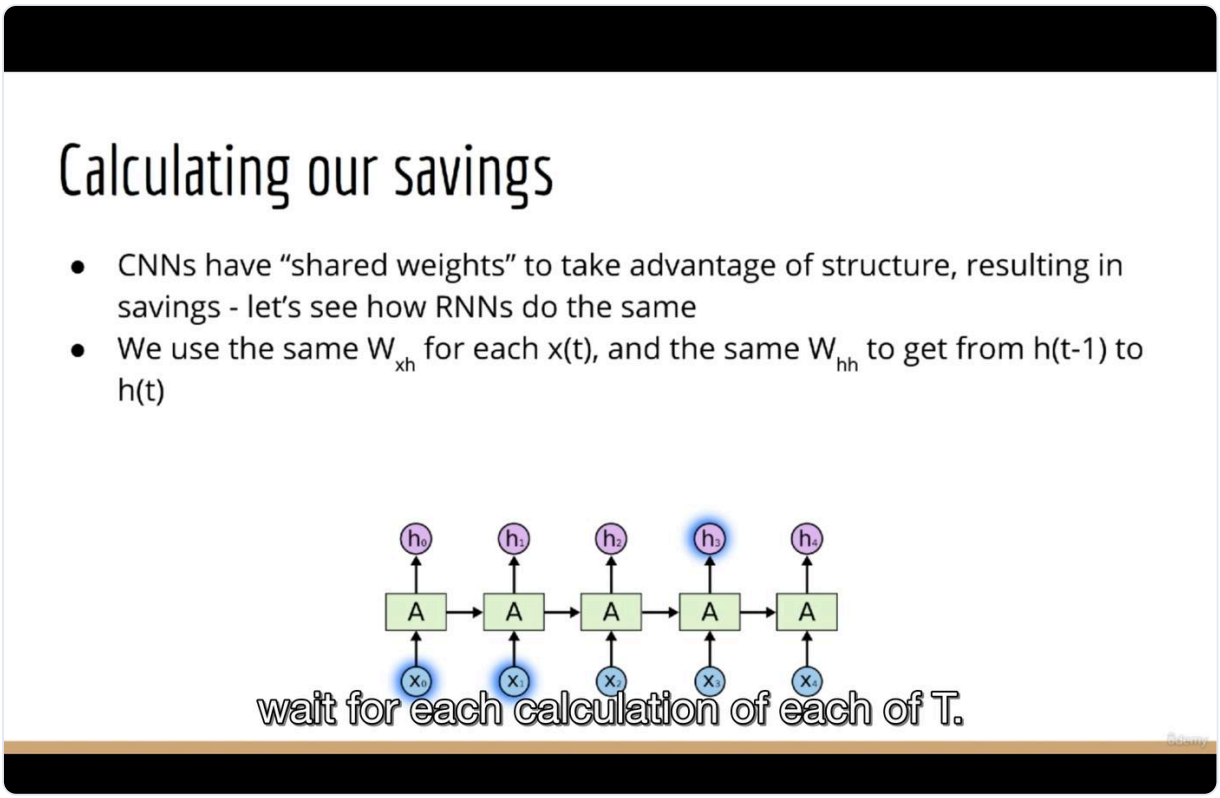


FIGURE 12.19 | Calculating Our Savings

If you recall we did this example with CNNs also the idea was because CNNs take advantage of the structure of images we can use shared weights the same idea applies to RNNs with RNNs. We apply the same weight matrix to each input vector x of t and we apply the same hidden to hidden weight for each calculation of h of T. Thus it's useful to compare how much we would save if we use the RNN instead of an ANN with a flattened time series. So let us do a simple calculation for an ANN for simplicity's sake.

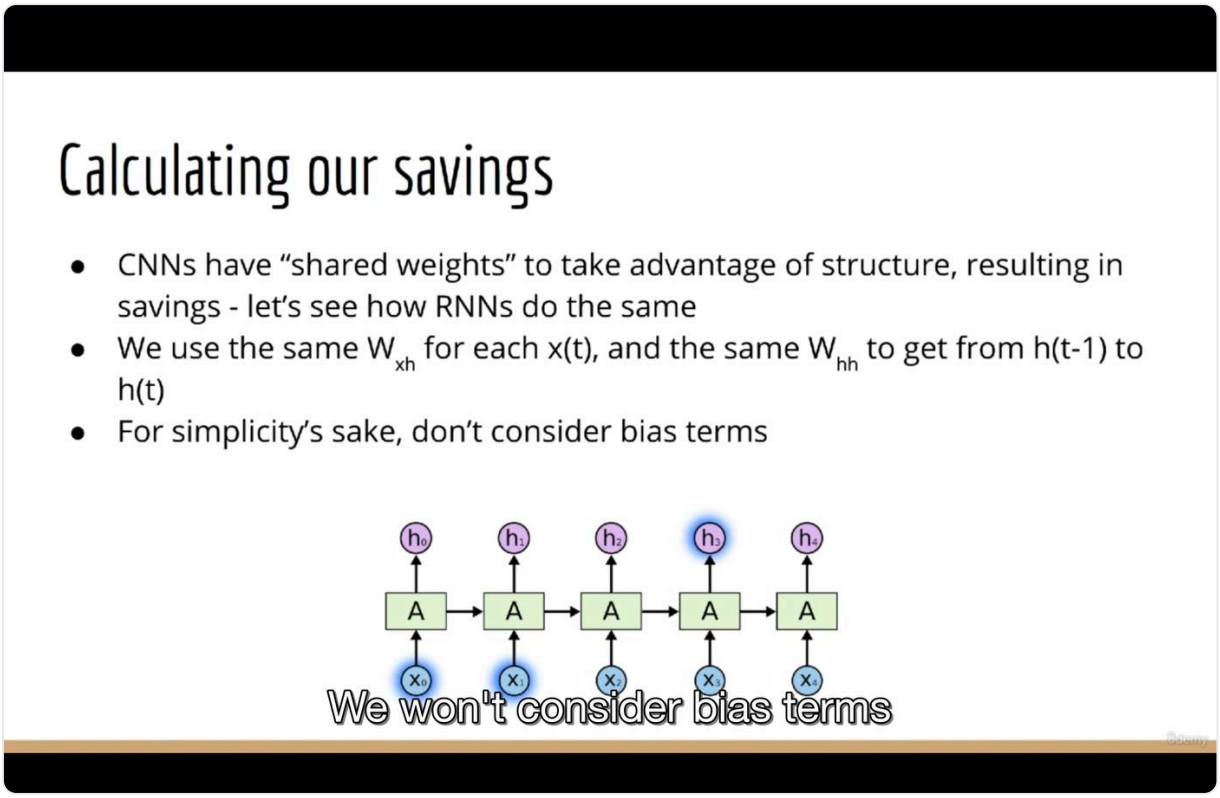


FIGURE 12.20 | Calculating Our Savings

We won't consider bias terms so let's say that t the sequence length is one hundred and D the feature vector size is 10.

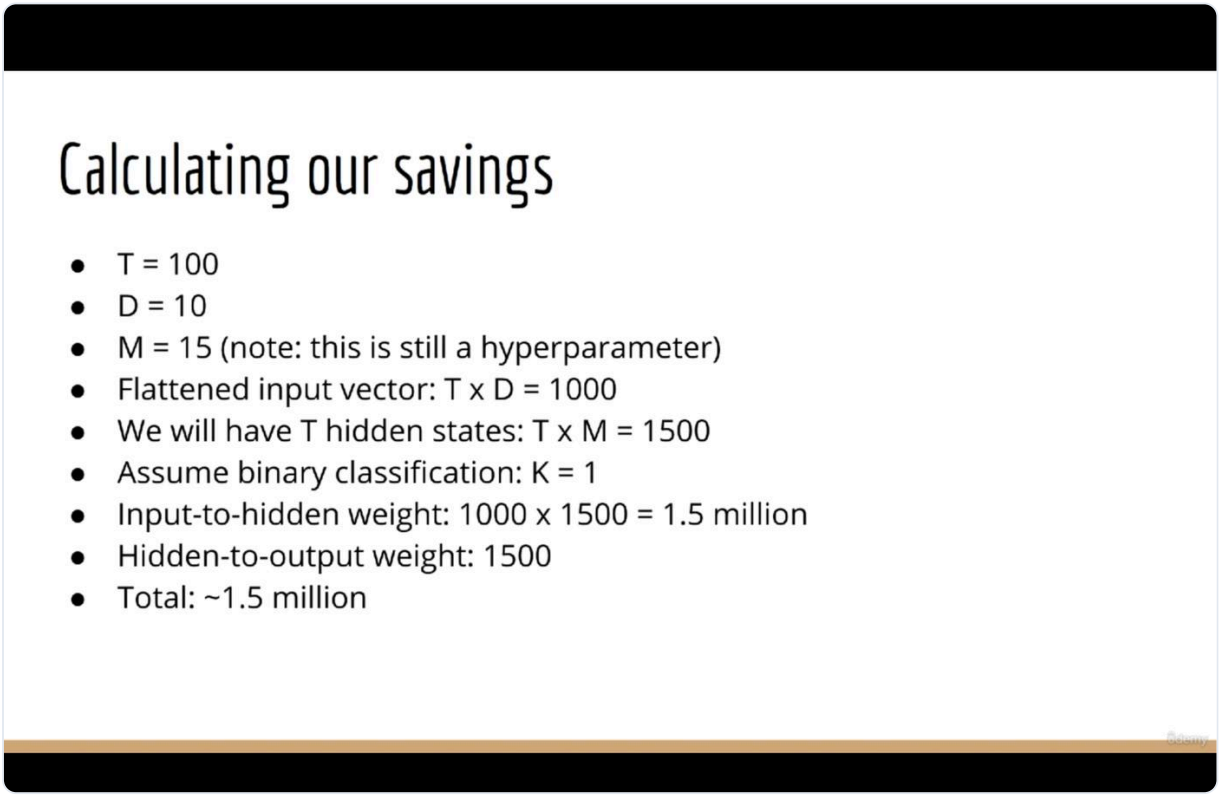


FIGURE 12.21 | Calculating Our Savings

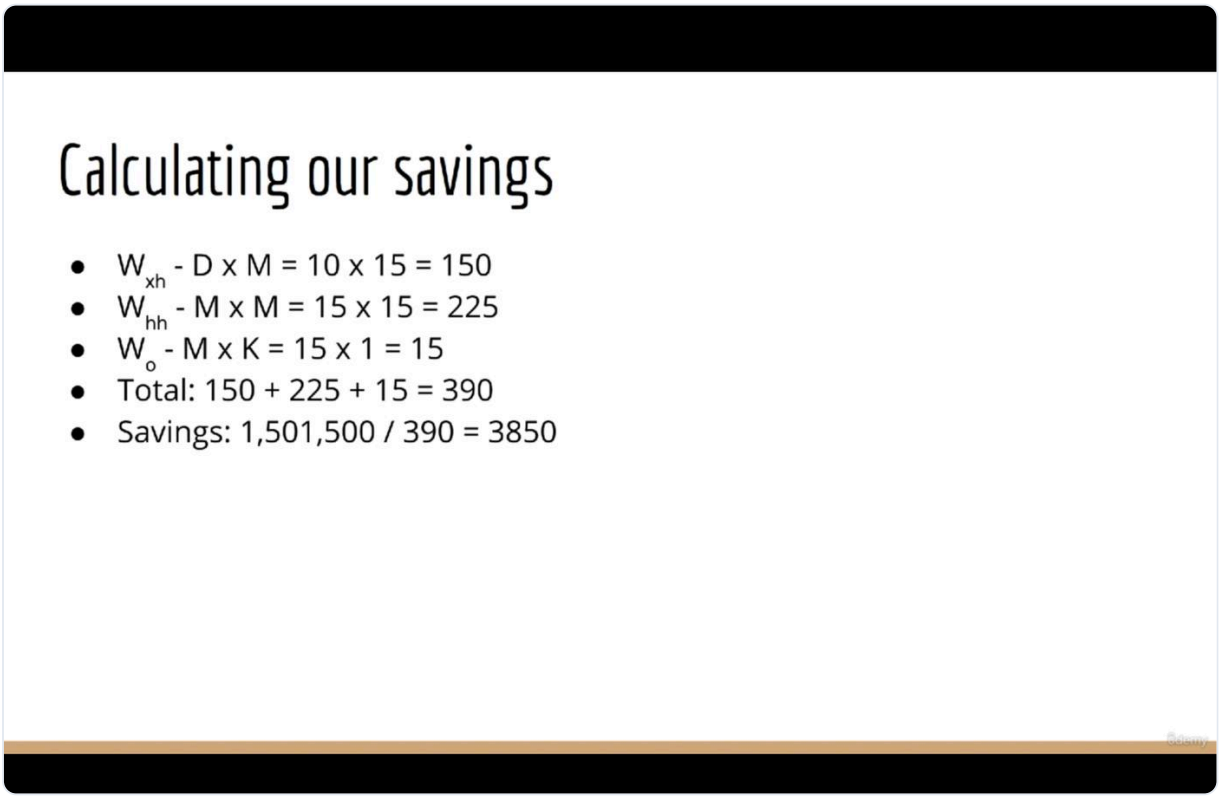


FIGURE 12.22 | Calculating Our Savings

Let us say that the hidden vector size M is 15.

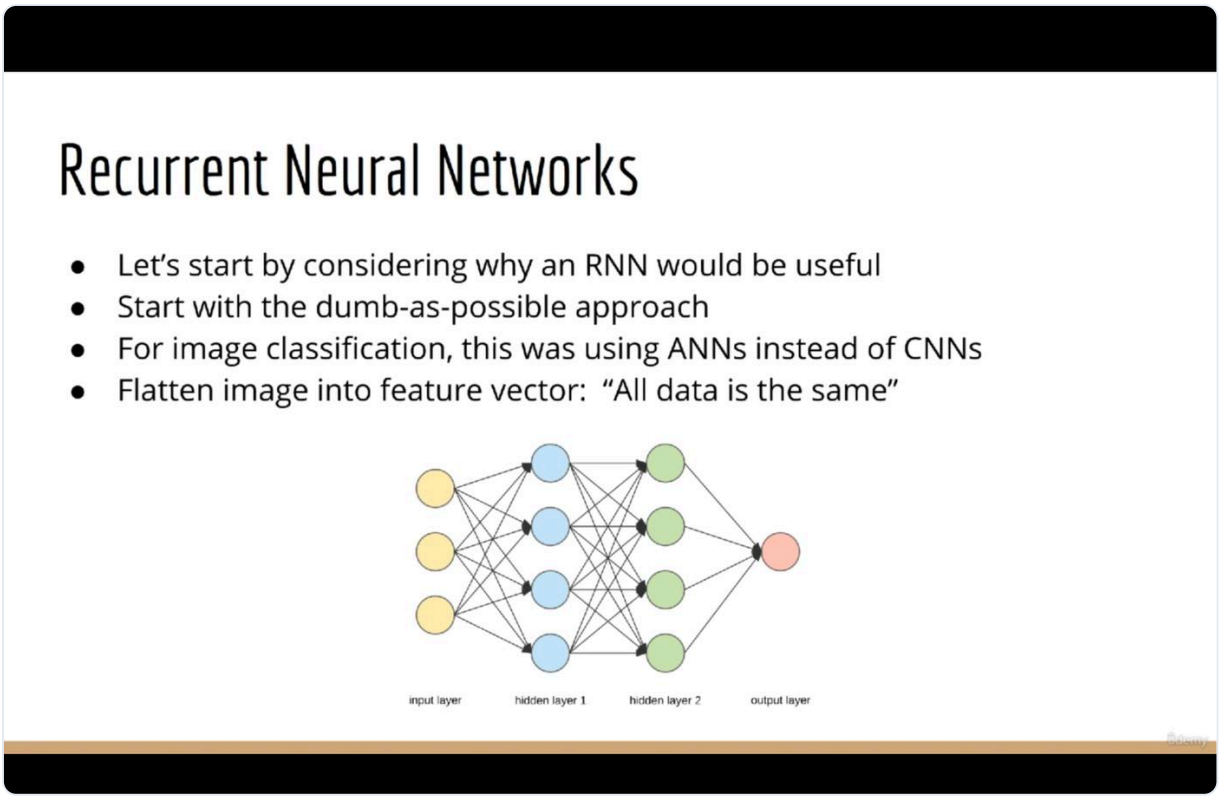


FIGURE 12.23 | Recurrent Neural Networks

Then if we flatten this we would have a 1000 length input vector correspondingly.

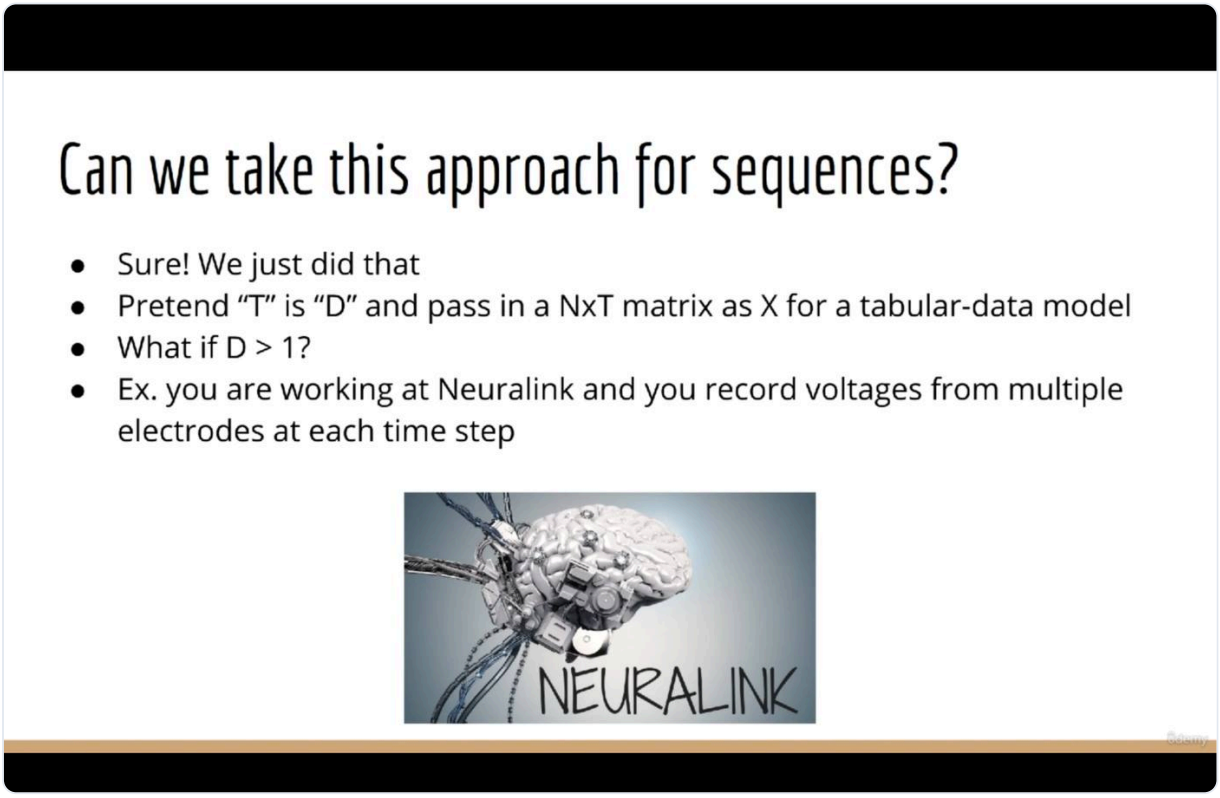


FIGURE 12.24 | Can We Take This Approach for Sequences?

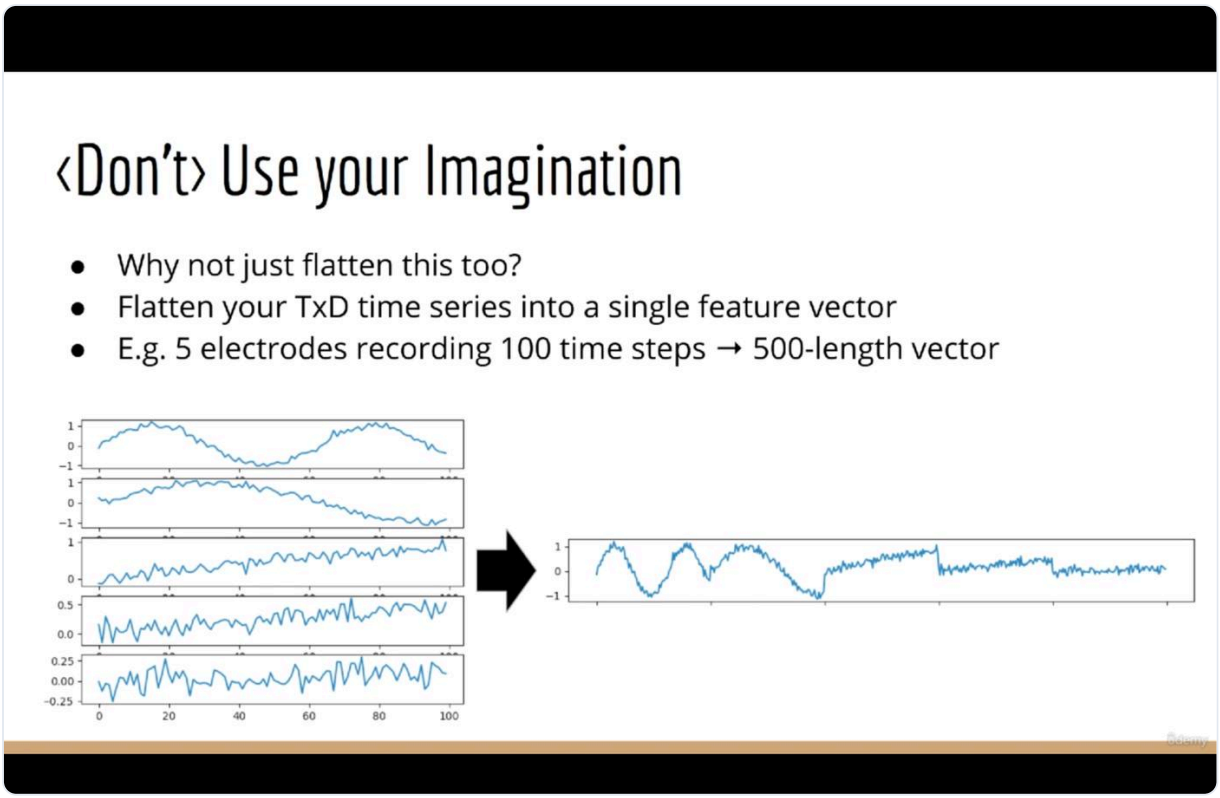


FIGURE 12.25

We would also have T hidden states.

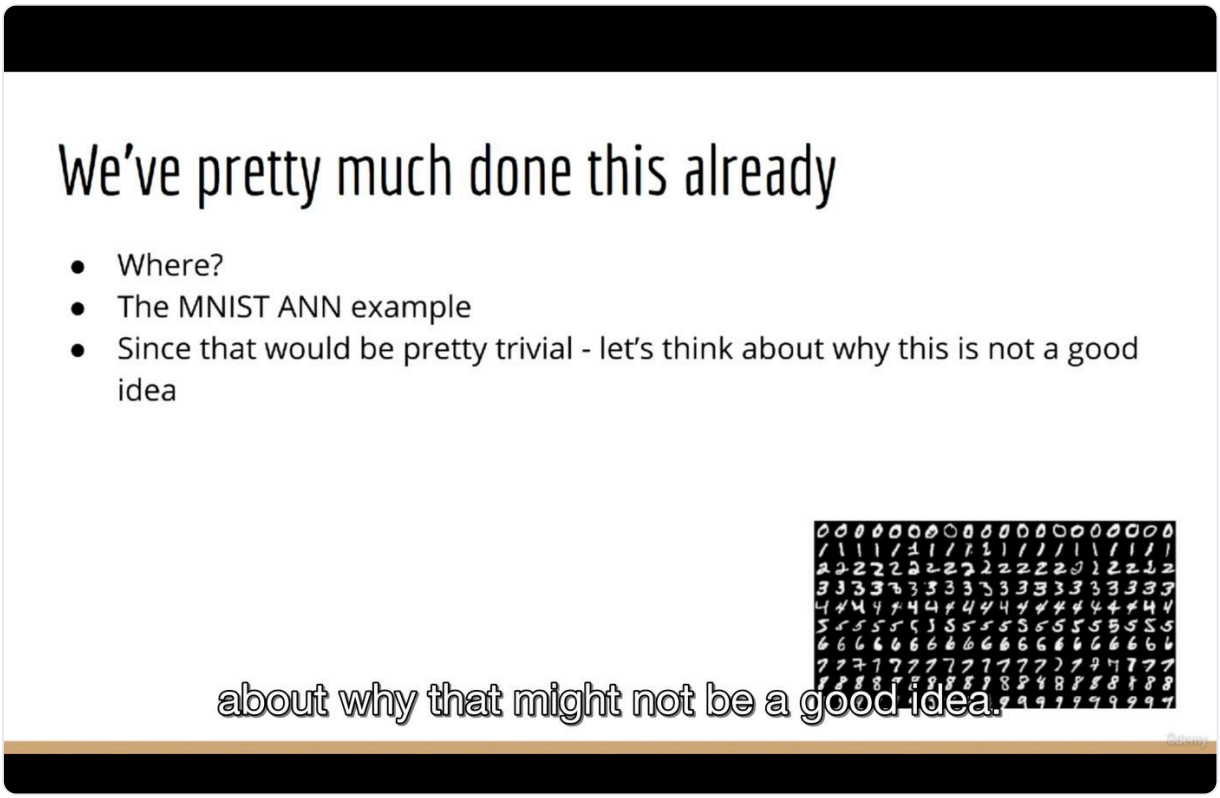


FIGURE 12.26 | We've Pretty Much Done This Already

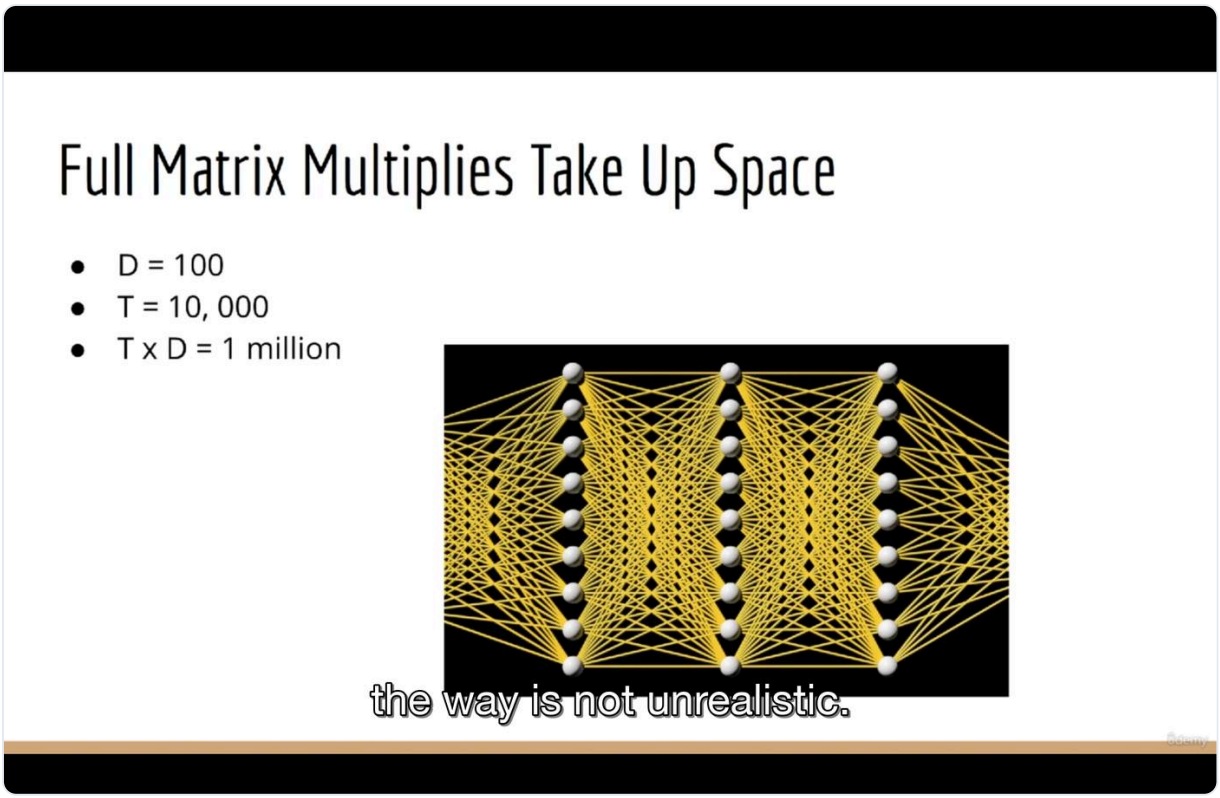


FIGURE 12.27 | Full Matrix Multiplies Take Up Space

So the hidden size in the ANN would be T times M which is fifteen hundred.

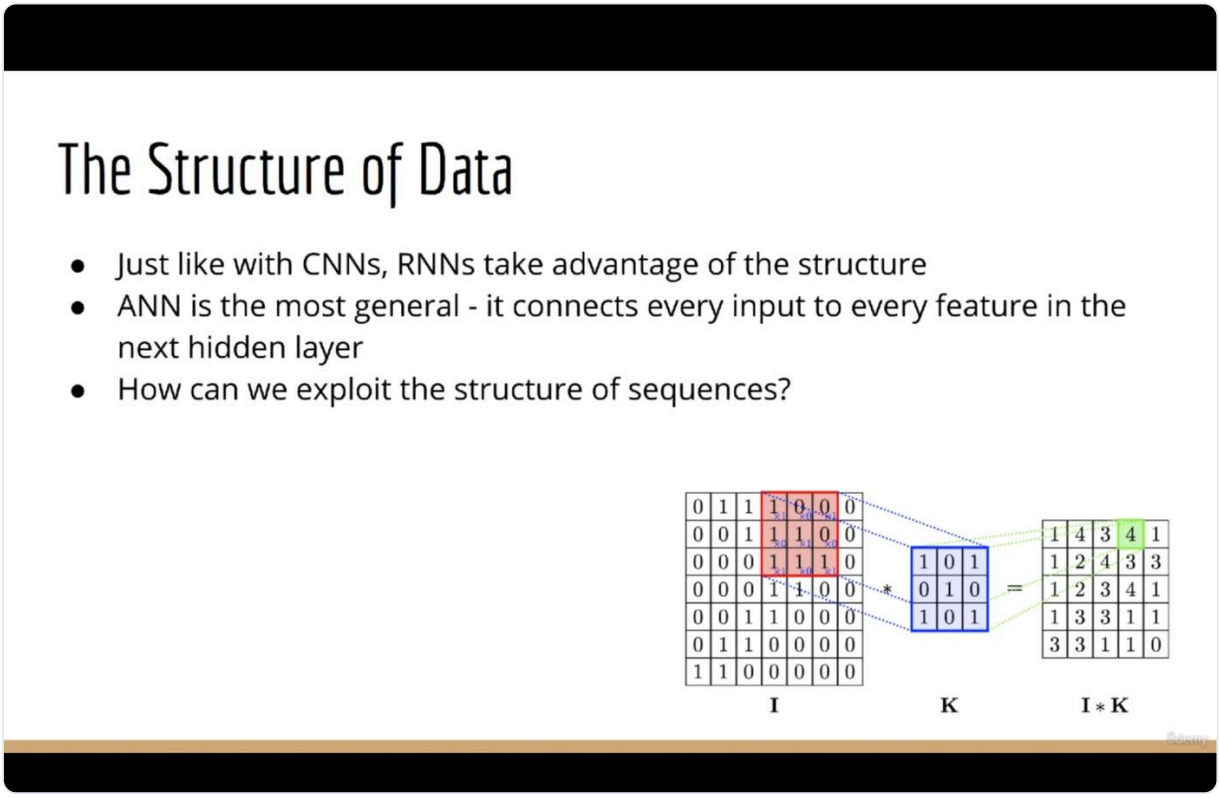


FIGURE 12.28 | The Structure of Data

Finally let's suppose we are just doing binary classification so the number of output nodes is just 1 in this scenario.

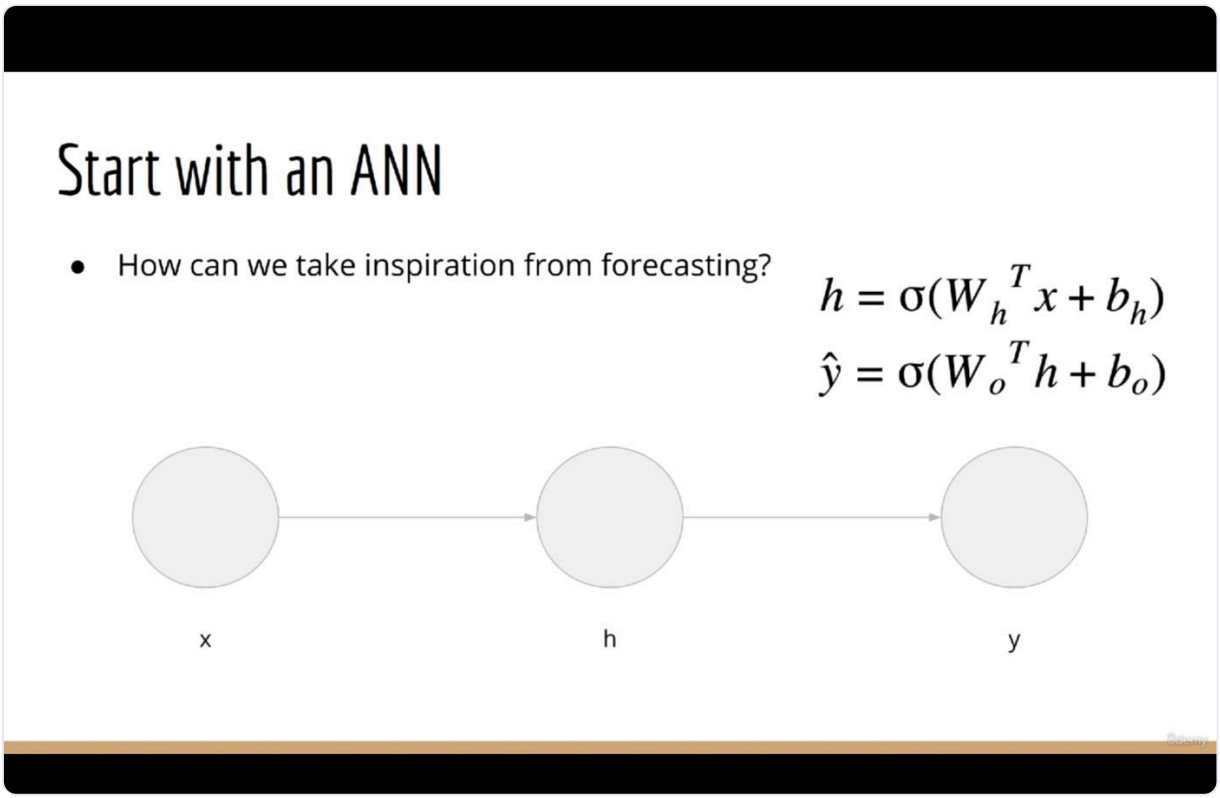


FIGURE 12.29 | Start with an ANN

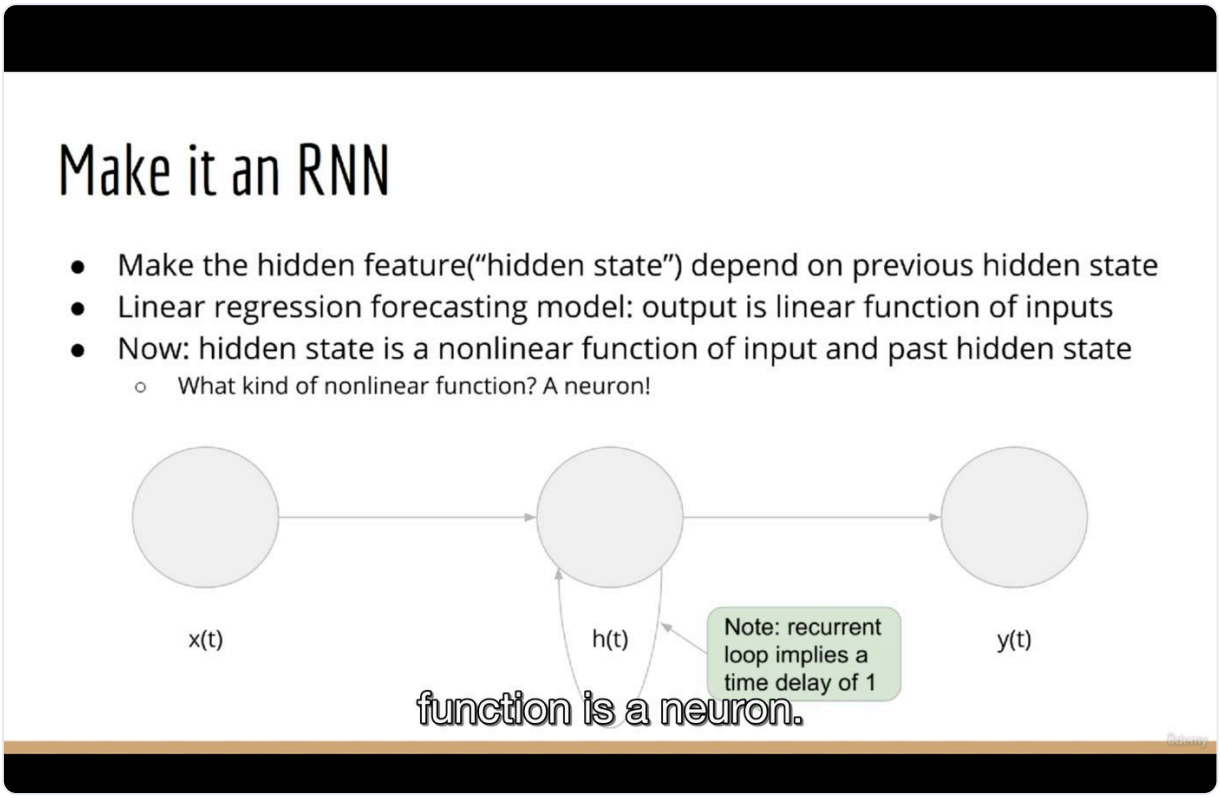


FIGURE 12.30 | Make It an RNN

Our input to hidden weight would be of size one thousand times fifteen hundred which is one point five million our hidden to output weight would be of size fifteen hundred by one which is just fifteen hundred.

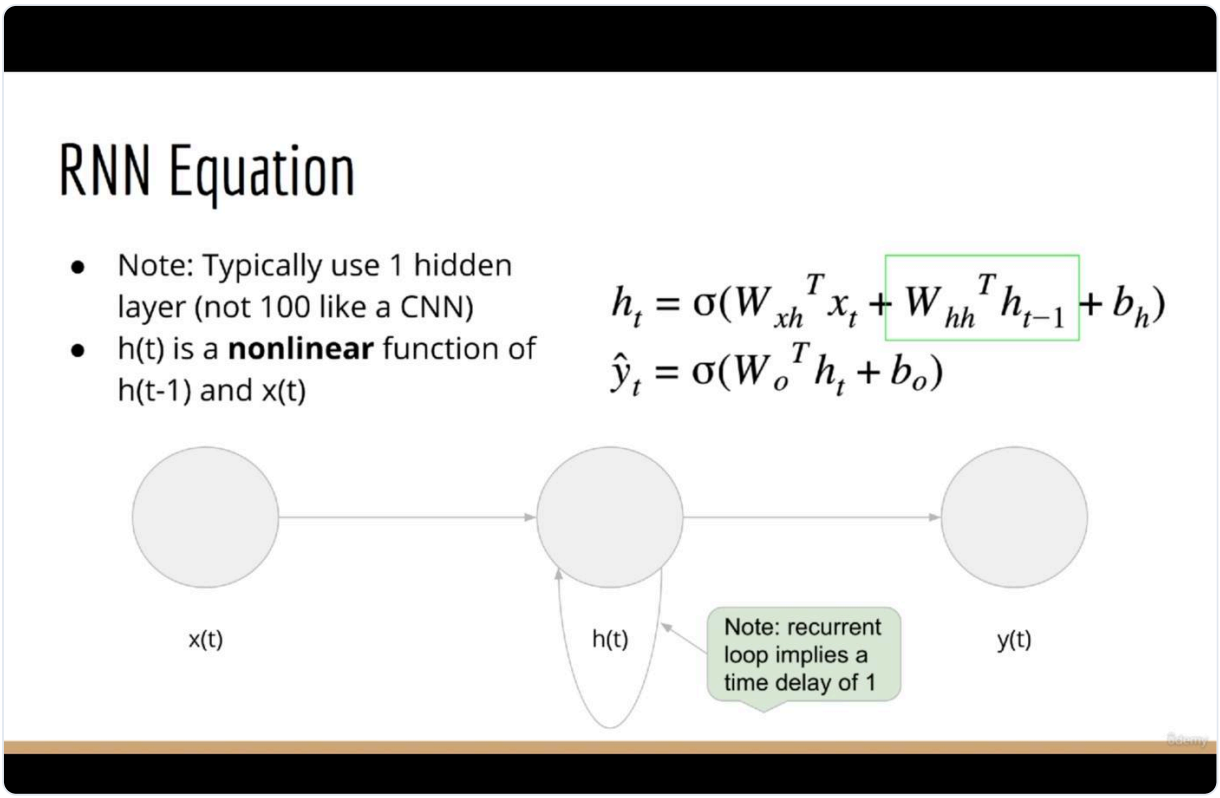


FIGURE 12.31

Therefore in total we have approximately one point five million weights now.

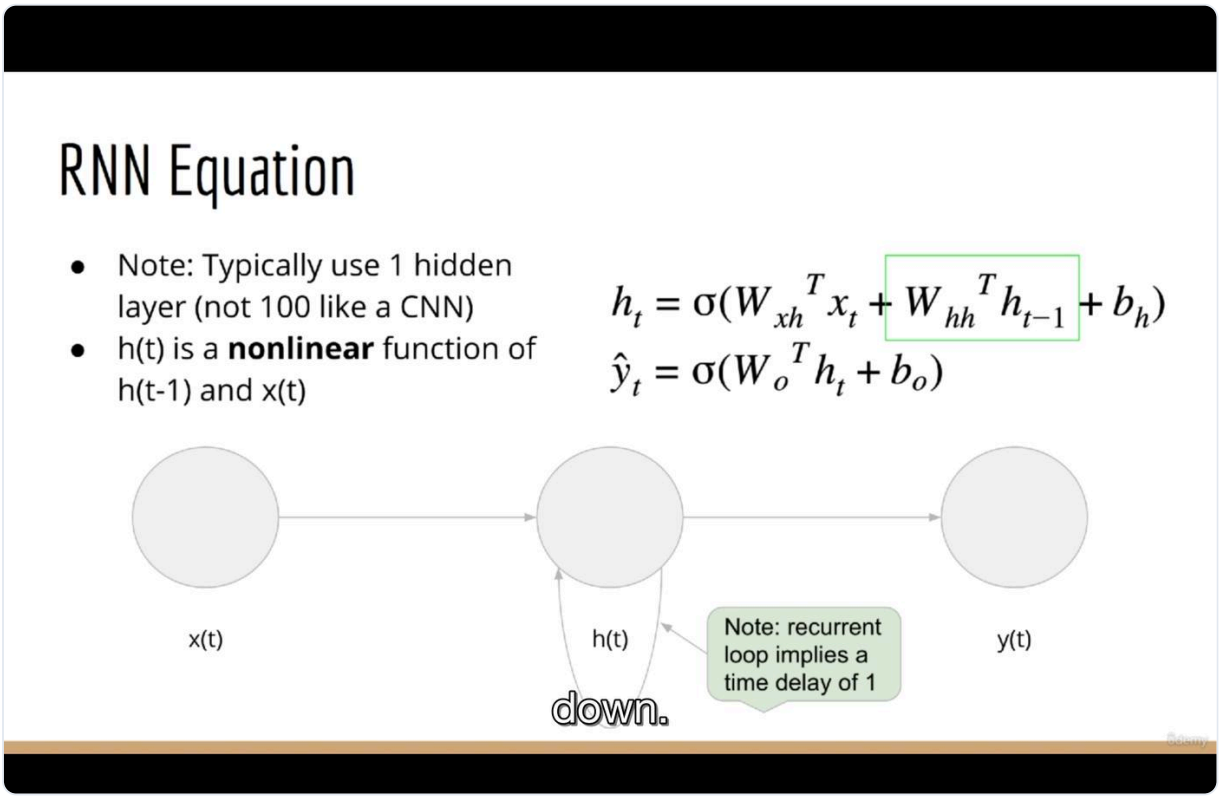


FIGURE 12.32

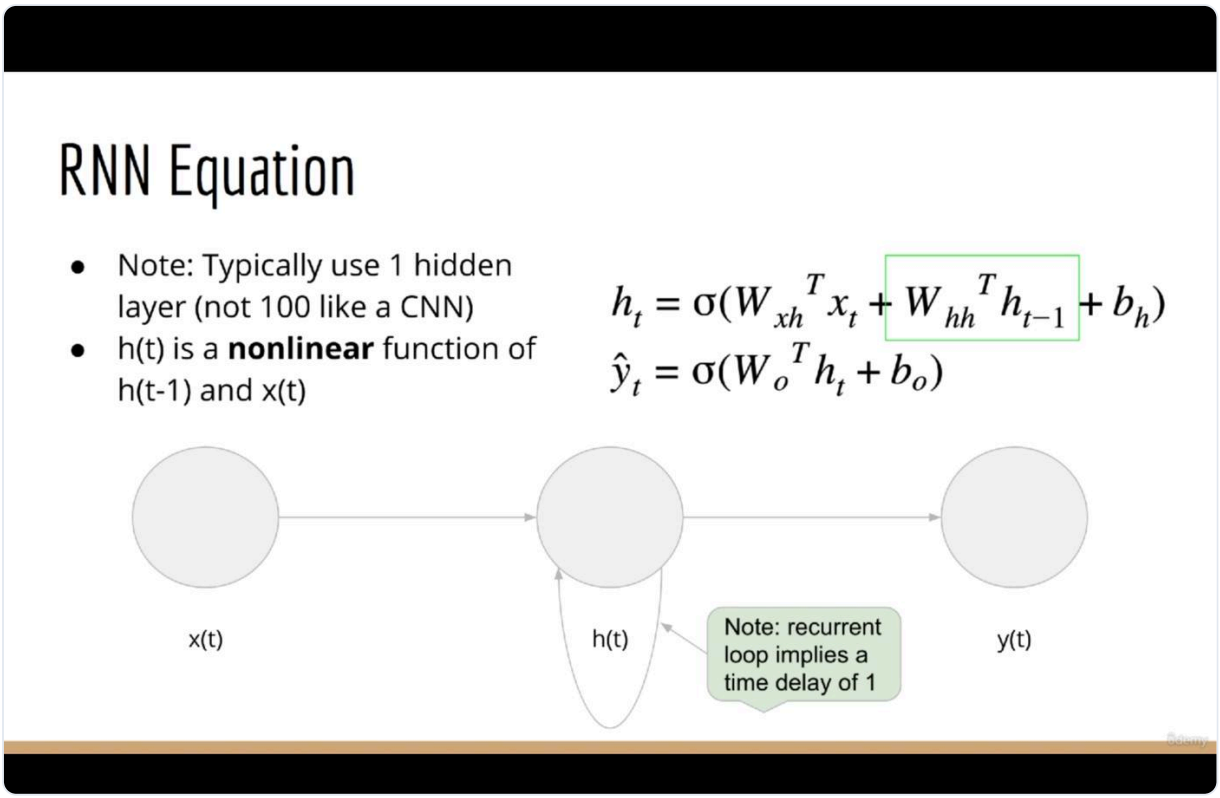


FIGURE 12.33

Compare this to the simple RNN.

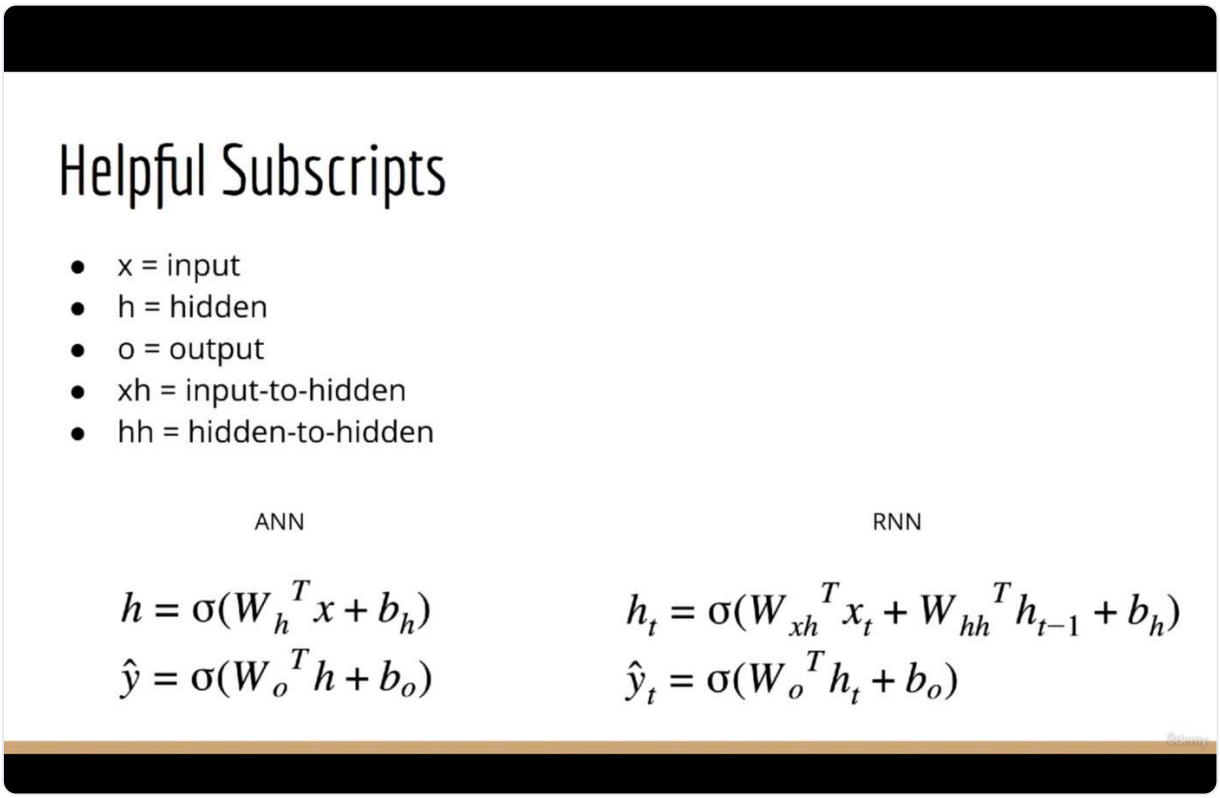


FIGURE 12.34

Remember we are calculating the size of W x H W H H and W O W x H. Since it goes from input to hidden it must be of size D by M which is 10 times 15 which is 150 w h h since it goes from hidden to hidden it must be of size m by m which is 15 times 15 which is two twenty five. Finally we have w o which goes

from hidden to output which must be of size m by 1 which is just 15 so in total we have 150 plus two twenty five plus 15 which is 390. So if we take an N N which has one point five million one thousand five hundred weights and divide that by 390 we find that the ANN has three thousand eight hundred fifty times more parameters which is a huge amount of savings.

Thus we learn again that by taking advantage of the structure of specialized data we can create much simpler and more compact models compared to full ANNs.

LECTURE 51

Forward Function

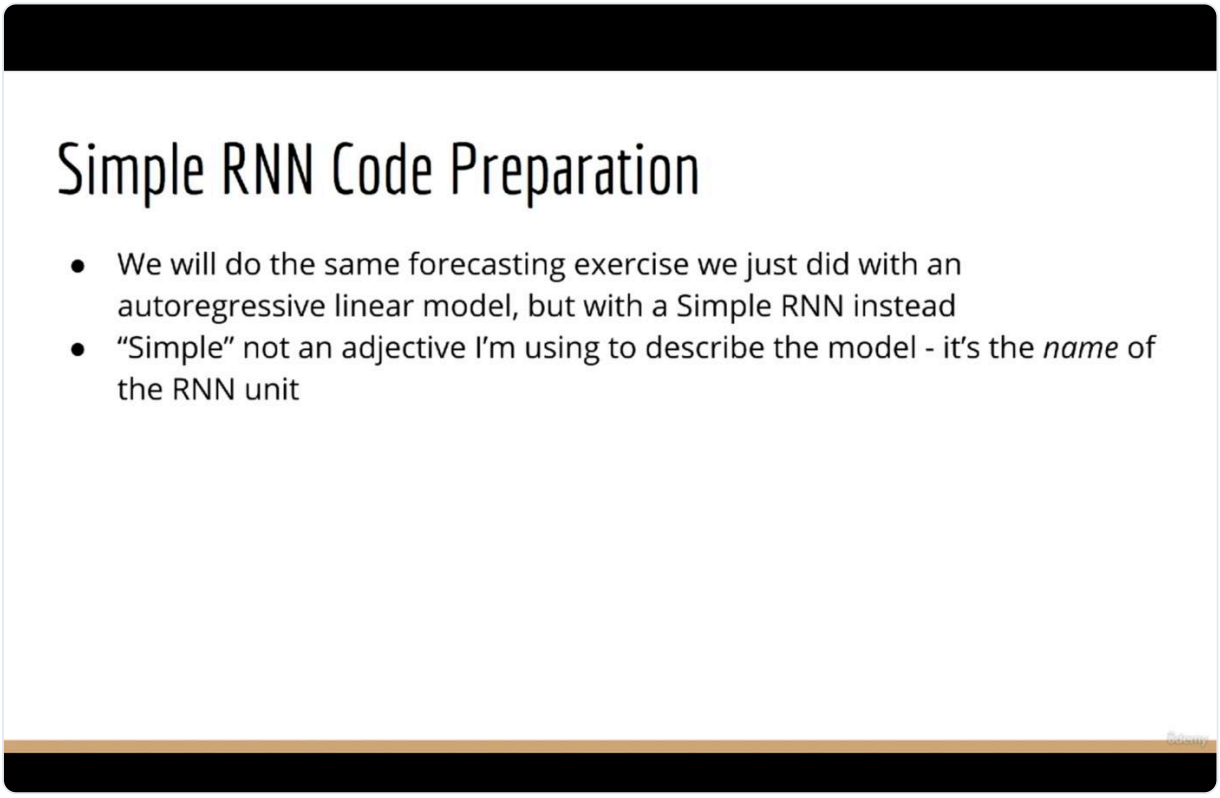


FIGURE 13.1 | Simple RNN Code Preparation

In this lecture we are going to talk about how to write code in PyTorch for a simple RNN this is in preparation for the next lecture where we are going to do the same forecasting exercise that we previously did but we're going to replace the autoregressive linear model with a simple RNN by the way. Remember that I am not saying simple RNN to mean that simple is an adjective to describe the complexity of this kind of RNN rather this is the actual name we use for this kind of RNN. In other words we call this a simple RNN because that is its name. And I'm not saying it's simple as opposed to being complex because in fact what we are doing now is actually quite complex so let's begin again by recalling the basic steps we are going to do in our script.

Luckily we already encountered most of this in the previous script so most of your focus in this lecture is on how to make an RNN as usual. Step number one is to load in the data we are going to use our synthetically generated dataset once again so that remains the same. The only difference is that it is not going to be the right shape for an RNN. Step number two is to instantiate our model. This is the focus of this lecture because we're learning about a new kind of neural network. Step number three is to train the model.

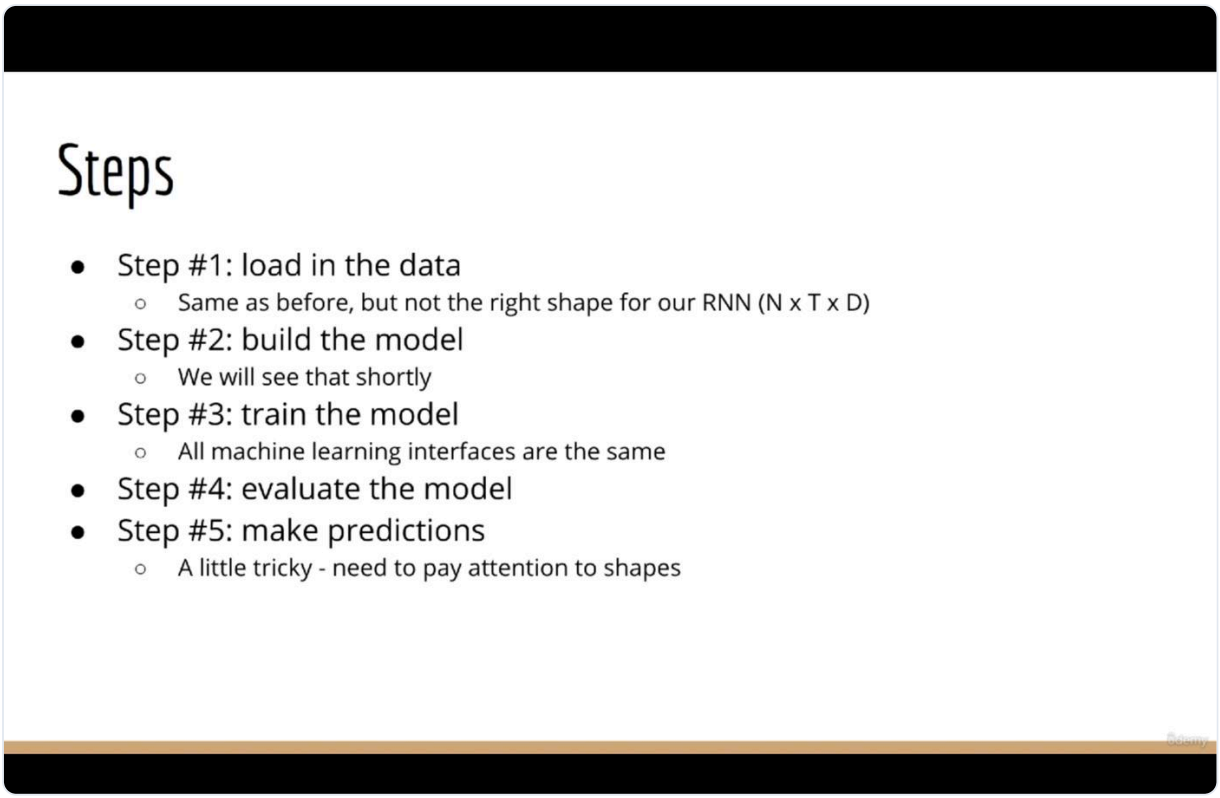


FIGURE 13.2

Luckily this is very simple due to my rule all machine learning interfaces are the same. Step number four is to evaluate the model which again stays the same. Step number five is to make predictions using the model. This will be somewhat tricky again because of the shapes. Have I mentioned how important it is to keep track of the shapes in an RNN. All right. So let's review step number one. Basically is the same setup as before. We're going to create a sine wave with and without noise then we're going to create a supervised learning dataset with both inputs and targets the input being T steps of the sequence and the target being the next step after those T steps.

Remember that we count from zero up to len series minus Big T because we want the final target to be len series minus 1 which is of course the final value of our series. The difference between this script and the previous script is that linear regression expects a 2D array as input so we passed in an N by T array. However as you know an RNN expects an N by T by D array as input.

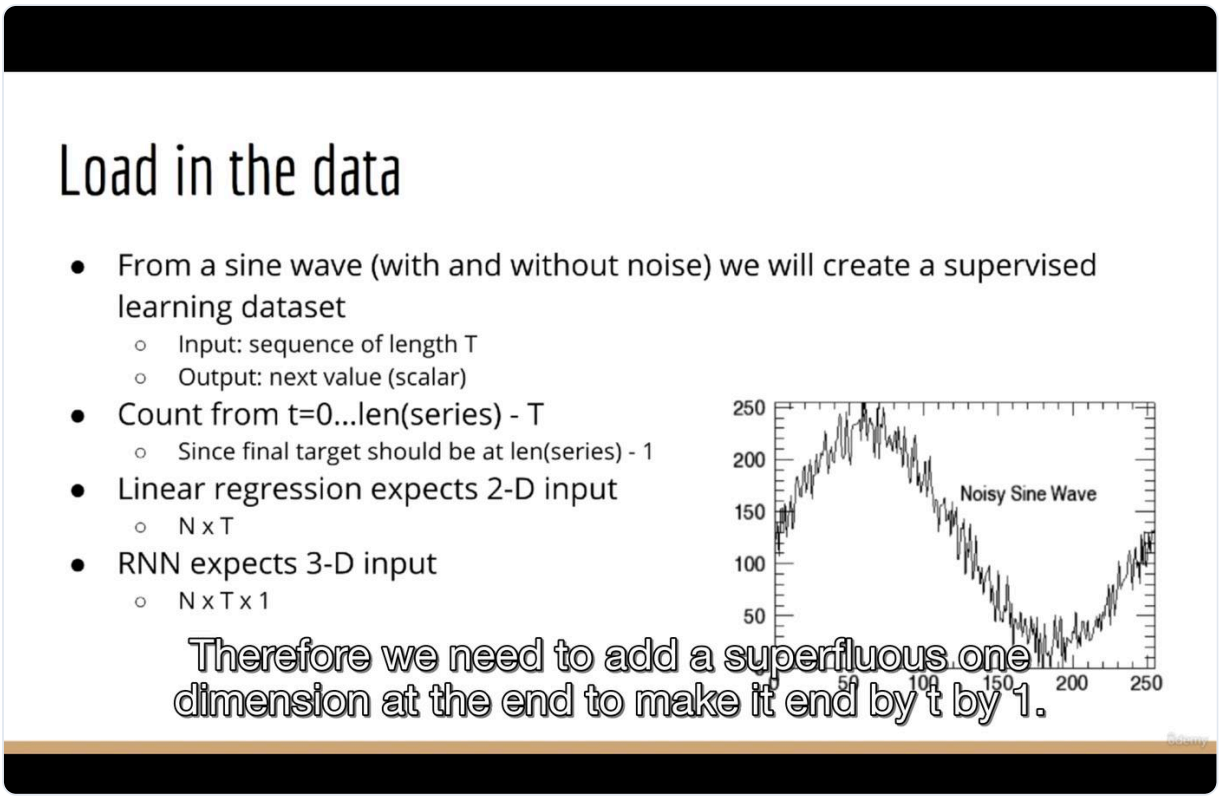


FIGURE 13.3 | Load in the Data

Therefore we need to add a superfluous one dimension at the end to make it N by T by 1. Step two is to instantiate the model. Luckily I've already shown you at the most crucial part of this which is how to create custom models in PyTorch. Now you understand why it was so important to do it early in a simpler environment.

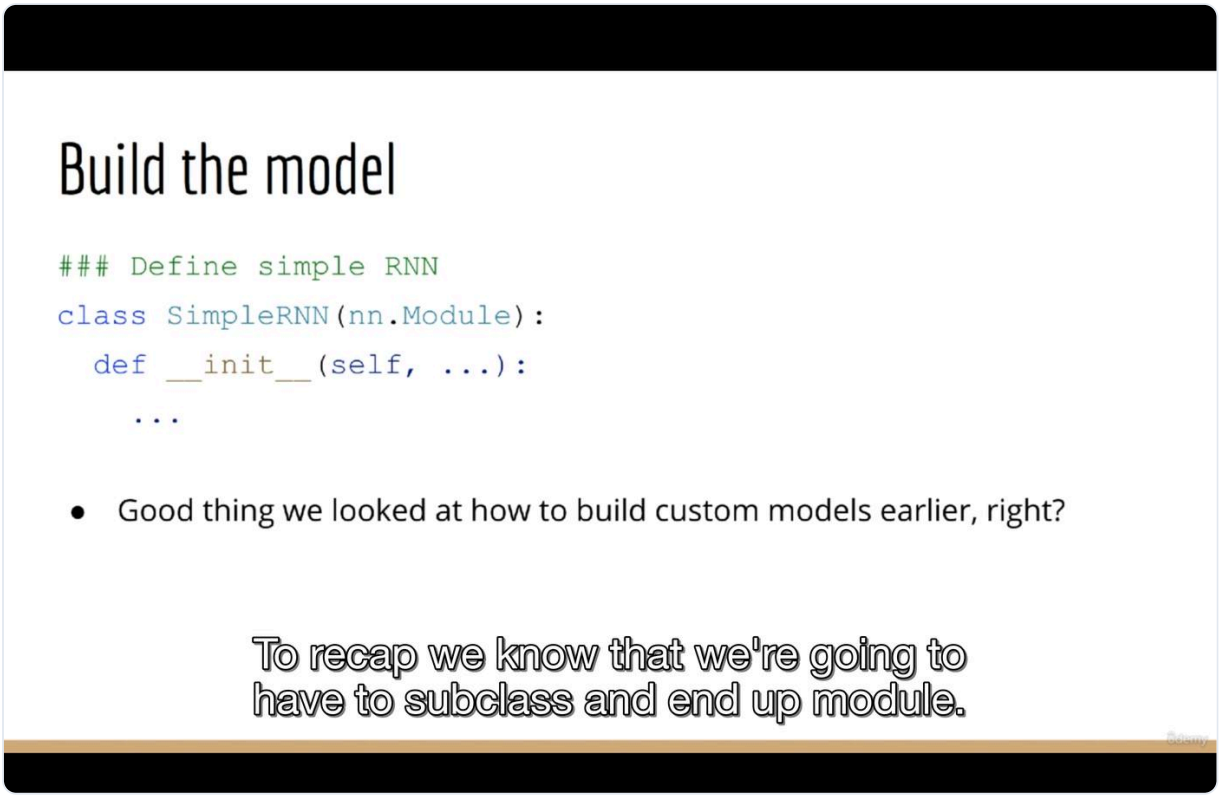


FIGURE 13.4

To recap we know that we are going to have to subclass nn.Module. The next step is to set up our layers. But the question is what layers do we need in PyTorch. The simple RNN layer is simply called RNN. And the reason we need to use a custom model is because RNN layers do not work like the previous layers we've seen in fact with most deep learning libraries RNNs is where things start to get complicated with linear models ANNs and CNNs everything is pretty straightforward and the interface to each layer follows the same kind of pattern. Specifically a feed forward pattern. You have one input and you get one output RNNs are tricky because they have this feedback loop.

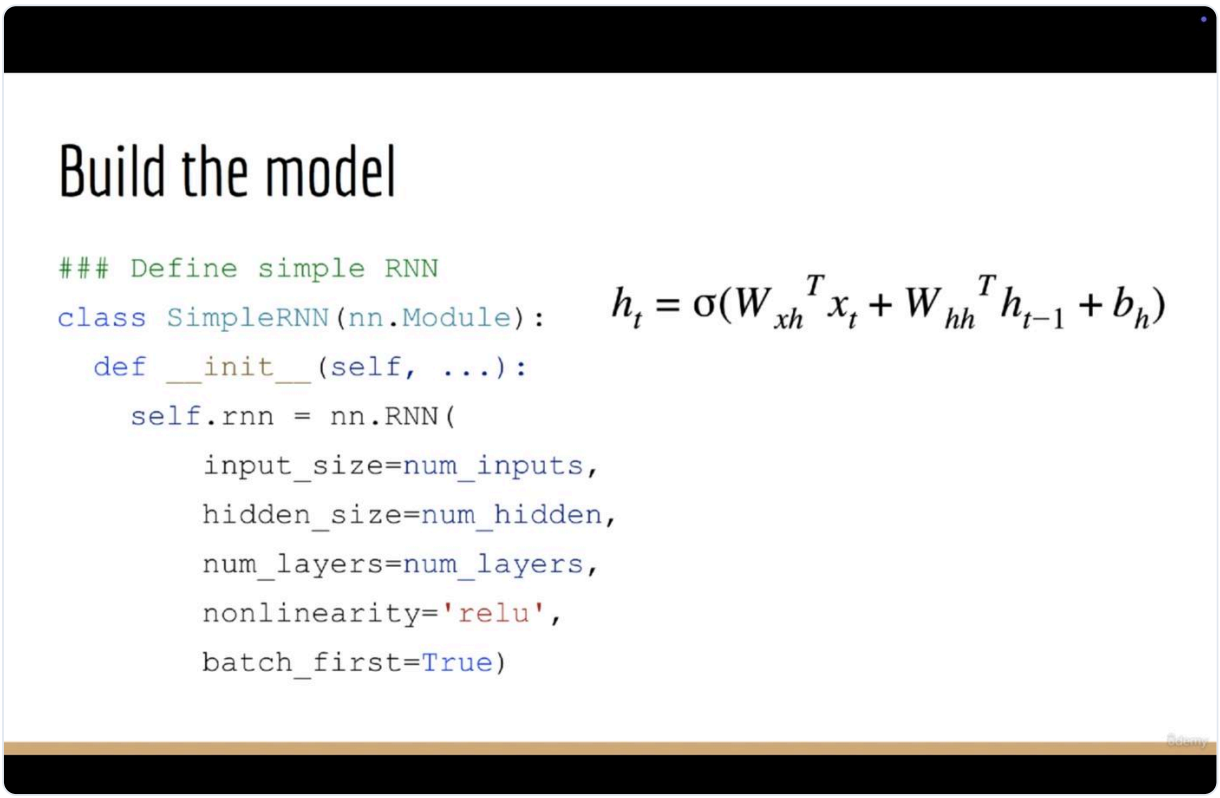


FIGURE 13.5

We know that we want to implement this equation which looks kind of like a logistic regression neuron and we know that the RNN module does just that but it seems to have a lot of interesting arguments. So let's discuss them one by one. First we have the input size and the hidden size which should be pretty easy to understand linear layers and convolution layers have similar arguments. We have the input dimensionality and the output dimensionality but as you can see RNNs have a few more arguments that we have to specify the next one is num layers unlike ANNs and CNNs you do not need to add new layers manually as you recall. If you want more dense layers in an ANN you would add more layers of type nn.Linear.

If you wanted more convolutions you would add more convolution layers but with RNNs.

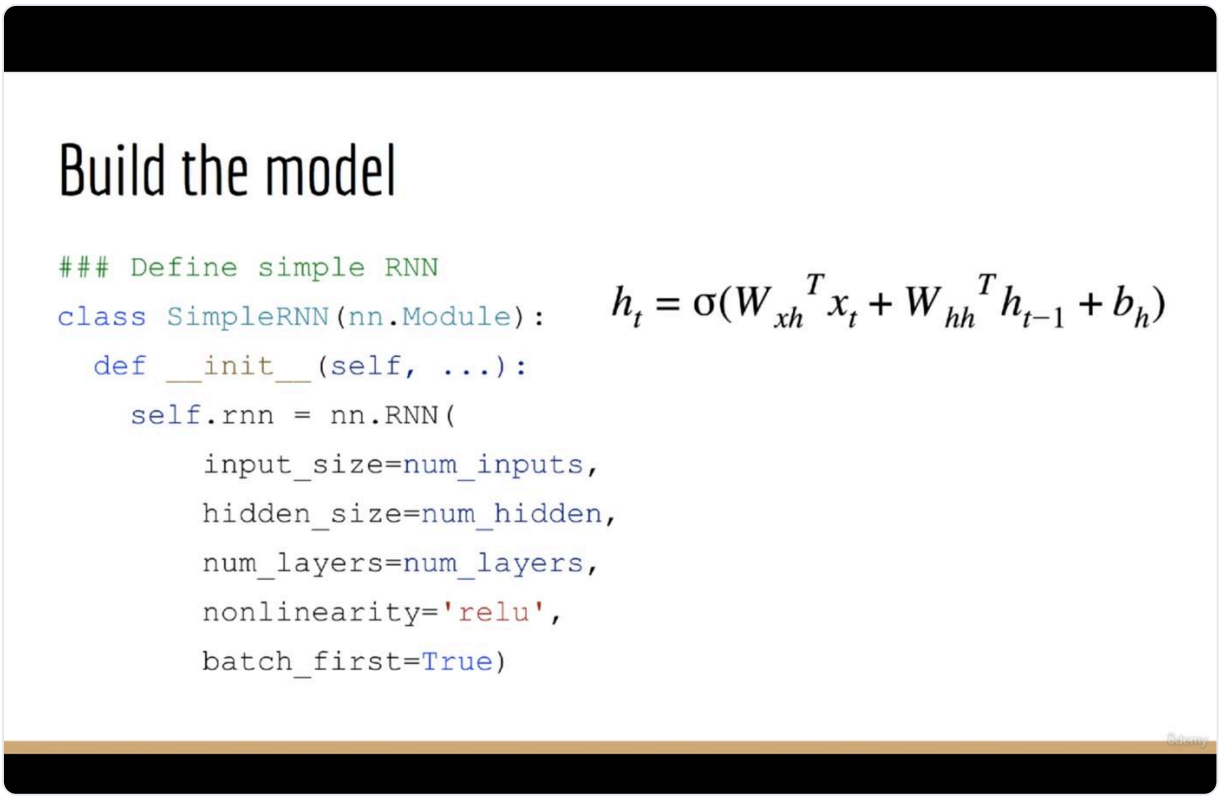


FIGURE 13.6

If you want more RNN layers you do not need to declare more than one RNN object. Instead you just pass in a different number for the argument num layers the next argument is the non linearity where you can pass in something like the ReLU or tanh. Again this is different from ANNs and CNNs where if you want a nonlinear activation function you would specify that manually as a separate object or function with RNNs it is simply an argument also RNNs in PyTorch only accept the ReLU or tanh activation functions. The basic idea is because RNNs are tricky to implement if you need to stack multiple RNN layers. It's more efficient to have them implemented like this rather than gluing multiple RNNs together and doing the computations one at a time.

Finally we set the argument batch first equals true which tells the RNN layer that our sequence data will be a shape N by T by D instead of T by N by D so the full constructor looks like this. First we inherit from nn.Module inside the constructor we call the parent classes constructor.

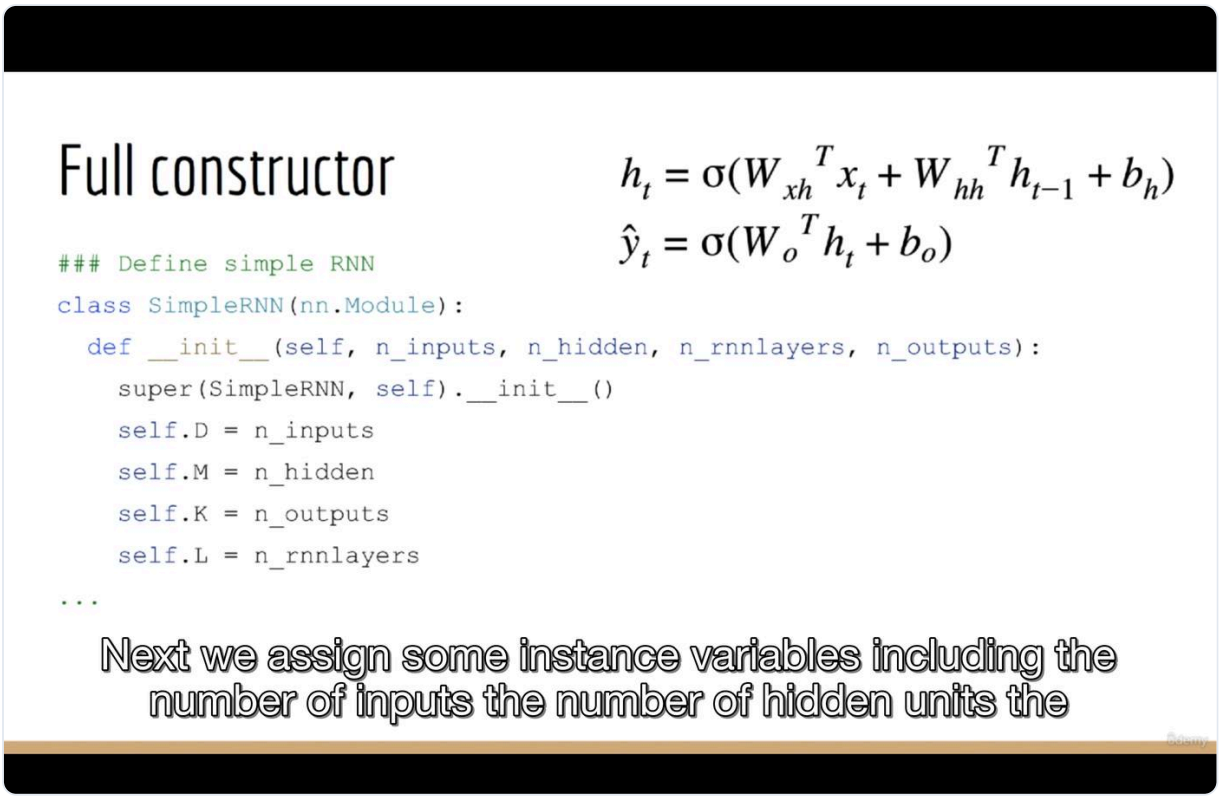


FIGURE 13.7

Next we assign some instance variables including the number of inputs the number of hidden units the number of hidden layers and the number of outputs. Next we declare our RNN layer or layers. And finally we declare our final dense layer which is nn.Linear the forward function is similarly tricky because unlike ANNs and CNNs it is not just a matter of passing the data through one layer after another. Instead we have to pay special attention to the interface of the RNN layer unlike in an ANN or a CNN layer where you only have one input and a set of weights with an RNN layer. You have two inputs and the set of weights specifically the output depends not only on the input x but also on the initial hidden state h naught therefore in the forward function before we even call the RNN.

We have to first define the initial hidden state. As mentioned typically we just set it to all zeros. In PyTorch this is done using the torch dot zeros function which is analogous to the NumPy zeros function. We also have to remember to move all the values to the GPU. One important thing to keep in mind is that due to the interface of the RNN module it may actually comprise multiple RNN layers. If you recall we store the number of layers in an attribute called L and because each RNN layer requires its own initial state. We need to define the initial hidden states to be a three dimensional tensor rather than just a one dimensional vector and not only can we have separate initial hidden states per layer but also per sample.

Because of that the initial hidden state is three dimensional l by n by M where l is the number of RNN layers and is the number of samples or the batch size and m is the number of hidden features as you recall from earlier we can obtain the batch size by calling X dot size of zero. Once we have our initial hidden state h naught we can pass it into our RNN module. So the first argument is X which is the data and the second argument is h naught which is the initial hidden state you can see that the output interface of the RNN module is also a little different. It seems to return two outputs rather than just one output from the PyTorch RNN module is very strange if you haven't worked with it before.

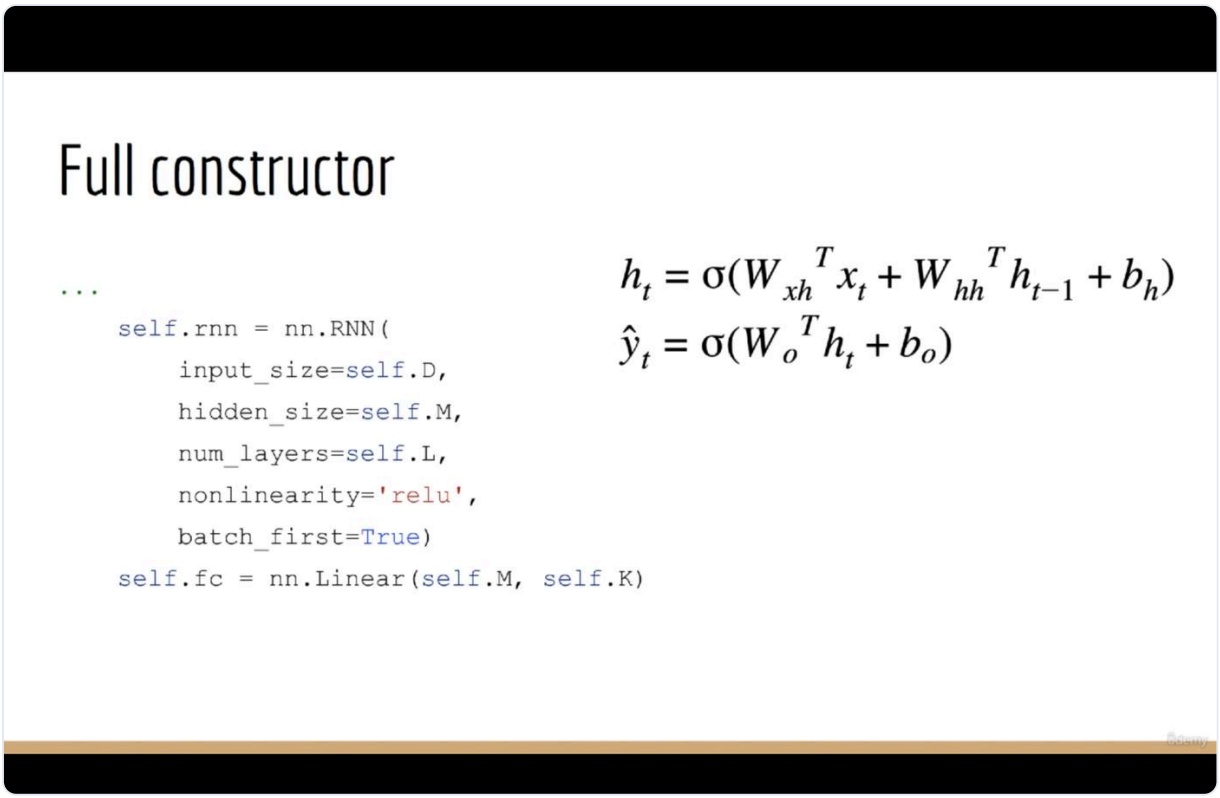


FIGURE 13.8 | Full Constructor

Just to remind you the RNN module in PyTorch is not just a single RNN layer but rather it could be a stack of L layers. If you think about all the possible hidden states we can calculate. We would have quite a few dimensions first we have N which is the number of samples of the batch size. This one we expect because it's always there so small and it tells us which sample next we have L which tells us the number of layers. So the small l index tells us which layer next we have t which tells us the number of time steps or the sequence length. Therefore a little T index is the time step. Finally we have little J which as usual index is the feature dimension.

So overall we have four possible ways to index all the hidden states. If we conceptualize them as being one giant array for each sample each layer each time step and each feature in PyTorch the return values are broken up in kind of a strange way. The first output gives us the hidden states for the final layer at each time step by default. Its shape is t by n by M where t is the sequence length and is the batch size and m is the number of hidden features. But since we passed in the argument batch first equals true earlier that means it is reshaped to be n by T by M the second output gives us the hidden states over all hidden layers but only at the final time step.

So by default a shape is L by n by M where l is the number of layers and is the batch size and m is the number of hidden features curiously most commonly RNNs make use of the first output rather than the second. So in this course we'll be ignoring the second output. However I did want to give you an explanation for why is there and not just ignore it without telling you what it is. The last thing we have to do in the forward function is pass the hidden state through the final dense layer since the hidden state we have obtained from the RNN module is N by T by M.

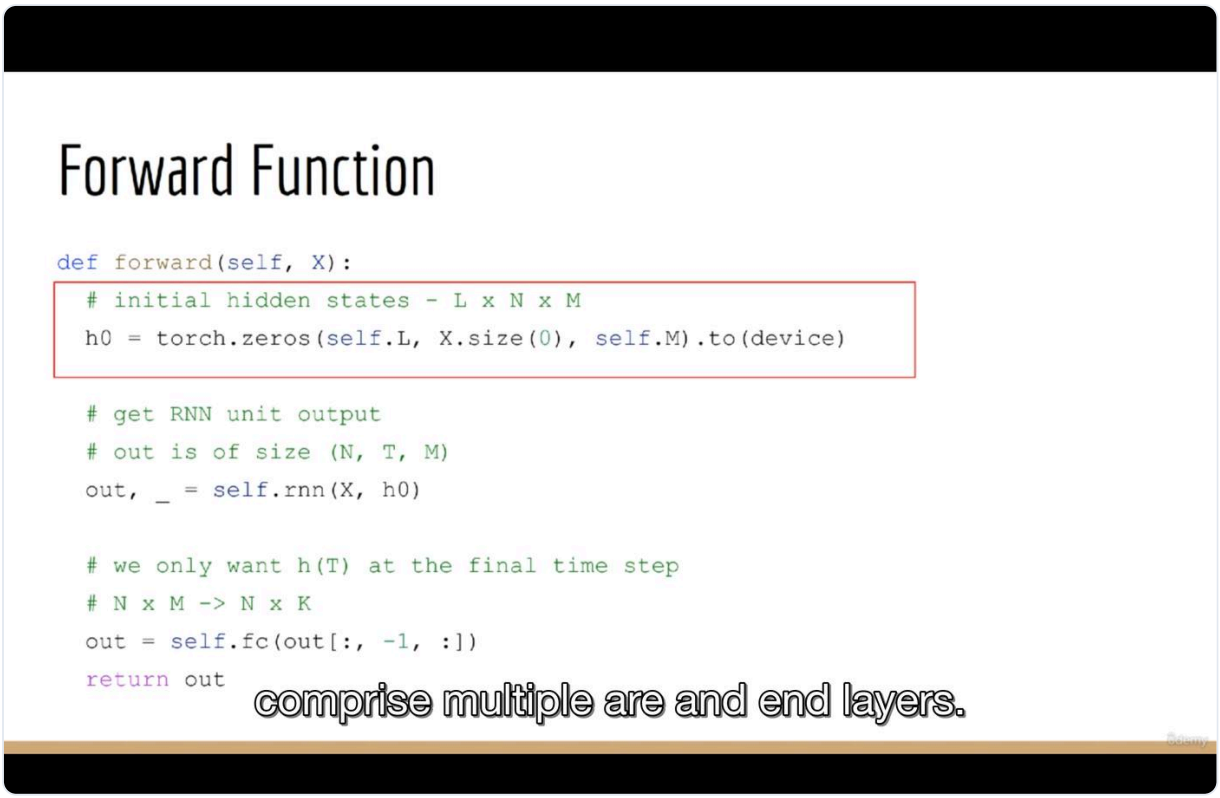


FIGURE 13.9 | Forward Function

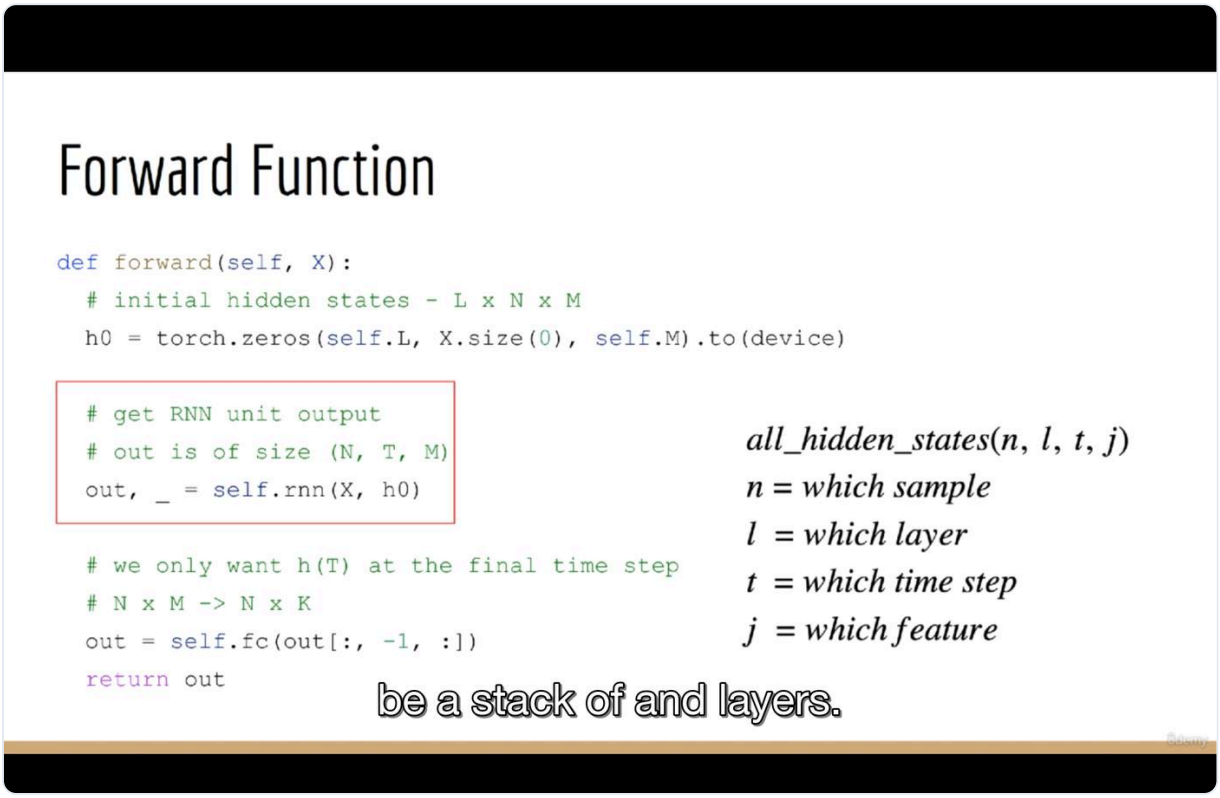


FIGURE 13.10 | Forward Function

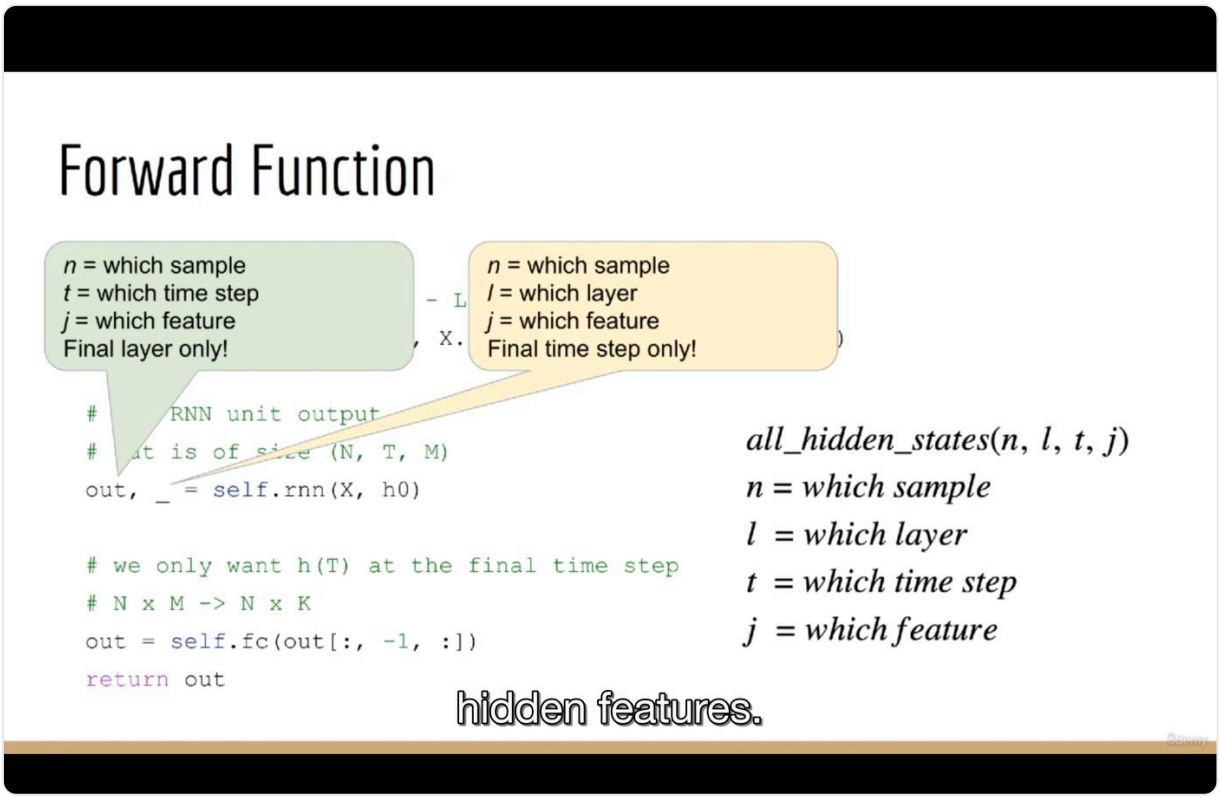


FIGURE 13.11 | Forward Function

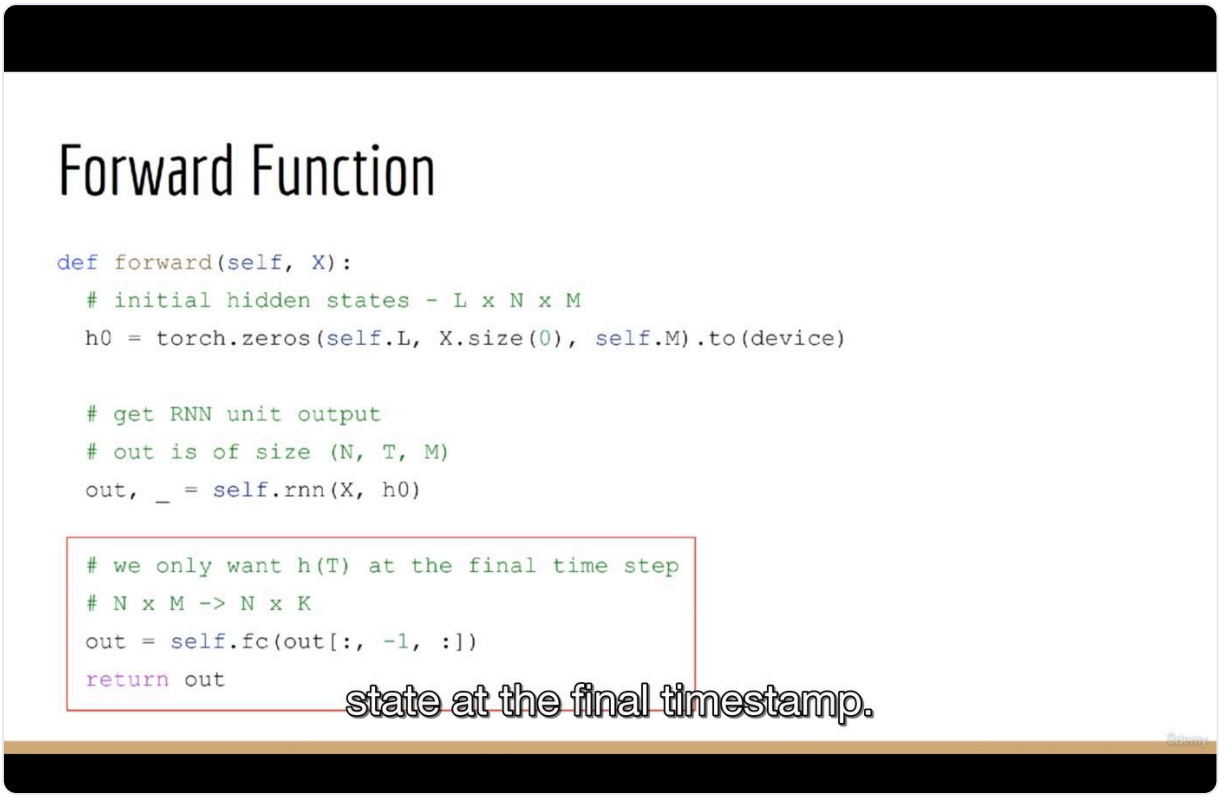


FIGURE 13.12 | Forward Function

It's not necessarily the right shape for our use case remember that this gives us all the hidden states for every time step but sometimes like for the next example we only want the hidden state at the final time step because that takes into account all the previous sequence data in order to obtain the hidden state at the final time step. We do exactly the same thing we would do if it were a NumPy array if the RNN output is called out then I can index it with a colon then a minus one and then another colon the first colon

means grab all the samples in the first dimension the minus 1 means grab only the last item from the second dimension and the final colon means grab all the features in the final dimension after indexing and this way the output will be of size N by M which is the same shape we have when we work with ANNs. So what happens after this should now be trivial for you now that we've done all the hard work the rest of the code will mostly be business as usual.

First we instantiate the model we just created. We also move the model to the GPU we create the loss and optimizer as before we will be using the mean squared error with the Adam optimizer we create the inputs and targets for both the train and test sets.

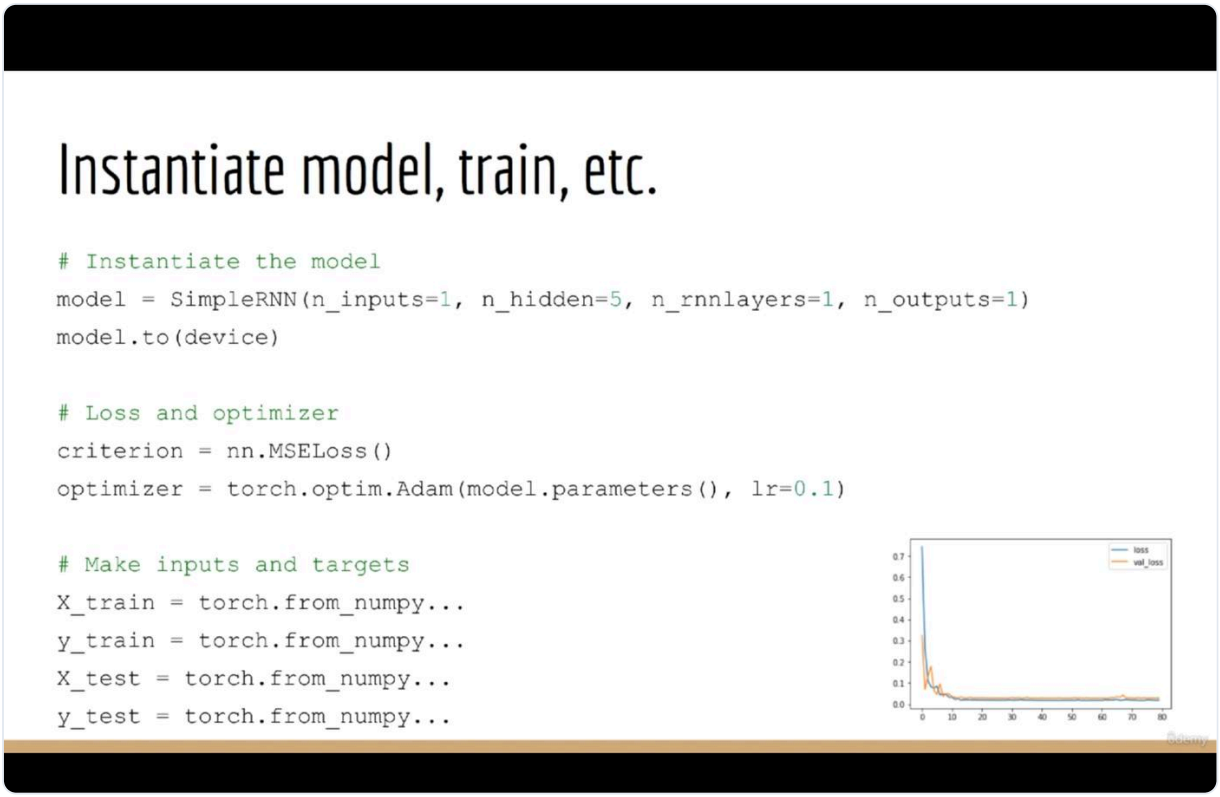


FIGURE 13.13 | Instantiate Model, Train, Etc

We move the data to the GPU. We do full gradient descent and we plot the loss per iteration to make sure the loss converges the last step. Making our predictions might be a little complicated since we have to pay special attention to the shapes. Remember that in general our input is of shape N by T by D where n is the number of samples t is the sequence length and D the number of features the output is of shape N by K where K is the number of output nodes. In our case this is just one because we are doing scalar regression.

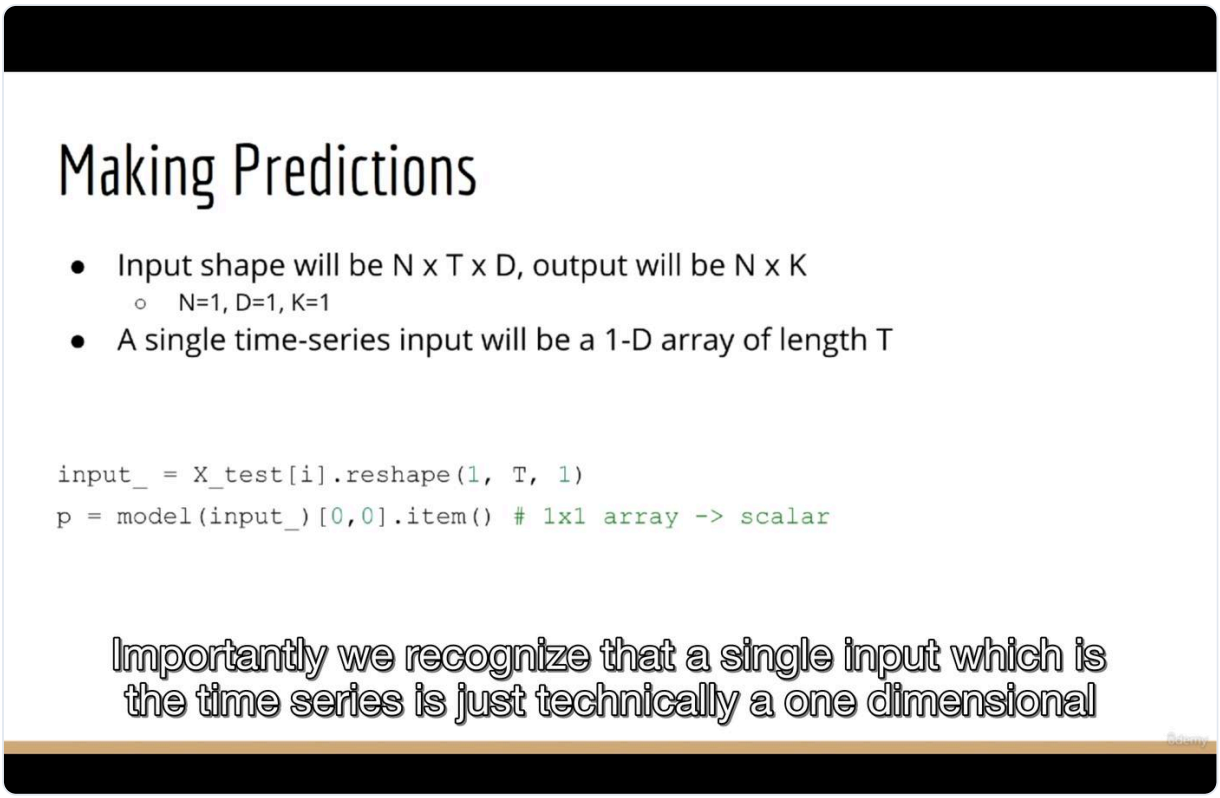


FIGURE 13.14 | Making Predictions

Importantly we recognize that a single input which is the time series is just technically a one dimensional array of length T. So in order to make this into an appropriate input for our RNN we actually have to reshape it to one by t by 1. That's because the number of samples is 1 and the number of feature dimensions is also 1. Then when we get our output it's going to be one by one since again N is 1 and K is 1. Therefore in order to get this value as a scalar we need to index it as 0 twice you saw this already in our linear regression script so it shouldn't be too difficult by now.

LECTURE 52

RNN

See the notebook `10.PyTorch-Simple-RNN-Sine.ipynb` for the code.

In this lecture we are going to use an RNN for time series prediction. The previous lecturer looked at how to use an autoregressive linear model for time series prediction. So in this lecture our main goal is to ask the question does an RNN do better this lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. As you know most of this script is the same as what you saw previously. So first we do all our usual imports.

Next we create our data which is a sine wave with some optional noise so here's the sine wave without noise as with this last lecture we are going to run this notebook twice once without noise and once with noise to see how the RNN performs next we're going to create our data set. This turns our problem into a supervised learning format where we have an N by T by D input and an N length target. So I assume you already understand this loop since it's the same as the last script. The only difference here is at the end we need to reshape X to be of shape N by T by D. Where D is one in order to do this. We can call reshape minus one t 1 so minus one acts as a wildcard here.

So basically means that make it whatever size is necessary so that the second and third dimensions are t and 1. Next we're going to set the device next we are going to build our RNN model. Luckily we saw this in the last lecture so this will just be a review first. Our simple RNN will inherit from nn.Module inside the constructor. We're going to take in several arguments the number of input features the number of hidden units the number of RNN layers and the number of outputs First we assign these arguments to instance variables D M and L as per our conventions in this course. Next we instantiate the RNN object for the input size we pass in D for the hidden size we pass in M for the number of layers we pass an L for the nonlinearity we pass in the ReLU and we also want to set the argument batch first equal to true.

This is to make sure the RNN module uses the convention that the N dimension comes first. Next we have our final dense layer of size m by k. Next we have the forward function as input. We take an x a batch of sequences of shape N by T by D. Inside the function we start by creating the initial hidden state h naught a tensor of zeroes of shape L by N by M. This is because we'll have an initial hidden state for each RNN layer each sample and each hidden feature. Next we pass in X and h naught into the RNN module which returns us two sets of hidden states. We typically only want the first one since it's indexed by time rather than indexed by the hidden layer.

In this script we are building a many to one RNN and in this case we only need the hidden state at the final time step in order to obtain it. We are going to index the RNN output by minus 1 at the second dimension which gives us back an array of size N by M finally we pass this N by M value into our final hidden layer which gives us an N by K output. Next we instantiate our simple RNN and move the parameters to the GPU. Next we create our loss and optimizer the MSE loss and the Adam optimizer now I just want to mention this again since it's important when we do our train test split. We don't want to just randomly pick samples across the entire time period for the validation set.

The purpose of forecasting is to predict the future and thus when you split up your data your validation set should always be future data points. And the train set should only contain points before that. Next we move the data to the GPU next we train the model. Again this is all stuff you've seen before. So there is no need to repeat it. Next we plot our loss. So that looks pretty good. Next we do our one step forecast. This is almost the same as our autoregressive example except the shapes are a bit different. The important part of this is how we shape the input and how we retrieve the output. Since we would like the input to be of shape and by t by D we reshape it to one by T by one since N is one and D is one when we retrieve the output which is n by K we have N equals 1 and K equals 1.

So we index by 0 0 so let's plot the predictions and as expected. This looks pretty nice but of course we know this can be misleading so let's scroll down to the real multi-step forecast. So here we can see that our RNN with the default parameters does not perform as well. This is a very intriguing result. If you recall the autoregressive linear model does this perfectly and in fact we proved that we could find the coefficients of an AR2 model analytically. So why does an RNN not perform as well. The answer is that an RNN has too much flexibility. This is just like how an ANN is more general than a CNN. But just because an ANN is more general and more flexible than a CNN it does not mean that an ANN will perform better than a CNN.

In fact constraining the parameters of a CNN to be shared makes the CNN better and we see the same theme here the AR2 model would work best because that model actually perfectly fits the data set. It is the correct model and the RNN model does not. It has too much flexibility and too many parameters. We say that it is over parameterized although as you'll see later the flexibility of an RNN does allow it to do more powerful things that an AR model cannot. Next we're going to go back and add some noise. This dataset so that it better mimics what you might find in the real world. So let's run all this again. So there is the data loss per iteration and the one step forecast.

So if we look at the one step forecast we can see it it looks pretty good. Which is what we expect but if we look at the multi-step forecast it looks pretty terrible. So in fact the RNN performs worse in this scenario I should mention that sometimes when I run this I see good results from the RNN but generally speaking the results are unpredictable. Sometimes it captures the periodicity and sometimes not. So it may suggest that more tuning would be required.

LECTURE 53

Hidden and Sequence Length

See the notebook `11.PyTorch-Understanding-RNN-Shapes.ipynb` for the code.

In this lecture we are going to go through a Colab notebook that emphasizes the importance of shapes in RNNs. Remember that whenever you hear me say something like N by T by D you should be automatically thinking about a box without me explicitly showing you a box. If you don't have this automatic visualization reflex you'll be at a disadvantage in trying to learn this stuff. This lecture is all about tracking the shapes in an RNN and also we are going to go through the RNN calculation manually to reinforce our understanding of how an RNN works as usual. You can look at the title of the notebook to determine what notebook we are currently looking at.

So the first thing we have here are just some comments where I list out all the important size variables we have to pay attention to. These things should be permanently stored in your memory. You should never be asking what does m mean again if you are. That will significantly slow down your learning so take notes and write things down if you have to. So just to recap N is the number of samples in your dataset. This has been the case since the beginning of this course. T is the sequence length. Remember that in PyTorch we assume constant size sequences D is the input feature dimensionality. We've gone through many examples of this where you might have a D bigger than one.

M is the number of hidden units. This is the same as we have in a regular feedforward ANN so it's a hyperparameter which you can choose. Finally K is the number of output nodes as a side note K being bigger than one does not automatically imply you are doing classification with a softmax. You can do multi-dimensional regression. Imagine for instance you are trying to predict lat long coordinates in that scenario. K would be two but it would still be a regression problem. So next we're going to make some dummy data also going to set our size variables so we are setting N T D K and then we make X to be just a random array of size N by T by D. So N is one.

So for this example we're only going to be working with one sample t is 10. So our sequence length is 10 D equals 3. So our feature dimensionality is 3 M is equal to 5. So our hidden feature size is 5 finally K was two so we have two output nodes. And as you know our input x will be a shape N by T by D. Next we are going to create our model so as before we're going to create a custom class called simple RNN. At this time. I'm not going to have an input argument for the number of hidden layers since the default is 1 and so it does not need to be specified inside the constructor. We have the same steps as before we set all the arguments to be instance variables we instantiate the RNN module and we create a dense layer with the number of output units K and just for fun I will use the tanh activation as the RNN nonlinearity.

Next we have the forward function which is essentially the same as before but with two crucial differences. First we are not going to use the GPU in this notebook. Since we're not training anything and second instead of just taking the final hidden state and passing it through the dense layer I am going to take all the hidden states and pass them through the final dense layer. So now we'll expect our output to be of size N by T by K instead of just N by K next we're going to instantiate our model and then pass it X to get the model's output so obviously both our data and weights are random. So this prediction is not meaningful. These numbers are just for sanity checking as you can see the output shape is as expected one by ten by two.

This is because we have one sample a sequence length of 10 and two outputs taking note of these numbers as this is what we want to compare with later on. Next we're going to detach the output and save it as a NumPy array so we can use it later. Next we're going to grab the parameters of the RNN module as you can see when I call the parameters function it returns four things. Now you might ask how do I know that the parameters will be returned in this order. Well I don't. I had to figure it out by running this script and making sure the outputs were the same as my manual calculation. So as an exercise if you want to close this notebook and try to do it yourself you're strongly encouraged to do so if we inspect w x h by checking a shape and printing out its values.

We can verify that it really refers to the input to hidden weight. If you recall the input dimensionality is three and the hidden dimensionality is five. Next we just have some code to copy each of the parameters to NumPy arrays. Now if we check the shape of all the parameters our ordering seems to make sense w x h we already confirmed w h h should be five by five which it is. What is weird is that PyTorch separates the input to hidden bias and the hidden to hidden bias that won't be a problem as long as we follow the principles. Next we grab the parameters of our final fully connected layer w o and b o if we check the shape the ordering appears to be correct the last step is to do our manual or an N calculation.

This just follows the pseudocode we discussed earlier. So hopefully you were taking notes to start we are going to initialize the initial hidden state to a vector of zeros. Next we get X at index 0 which is our one and only sample. Next we initialize an array of zeros for our y hats. As you know in this example we only have one sample. So for simplicity I'm going to make it an array of size t by k. Next we enter a loop where little T counts up from zero up to Big T inside the loop. We first calculate H. That's the hidden value at the hidden layer. This calculation is a little bit different from the theory lecture since the RNN layer has two bias terms.

Nevertheless we can follow the usual pattern in order to transform x of T. We're going to multiply by W x H and add the bias term b x H. Next we're going to take the previous hidden state H last multiply it by W H H and add the bias term b h h so that transforms H last. So these are both affine transformations on X of T and H last. Next we add both these linear transformations together and apply the tanh activation. Once we have h we can calculate Y which is just the usual equation multiply by W o and add b o and we also store the Y that we get into our y hats array. Finally we assign h h last so that H last has the correct value for the next iteration of the loop.

Once we're outside the loop we can print the final value of the Y hat's array and hopefully this is equal to what we calculated before with the PyTorch model now. Since this is just a big list of numbers and most of us want memorized what we saw before it would be better to check this programmatically by using the allclose function awesome. So we've confirmed that these are indeed the calculations that are done by a simple RNN. One thing that made this exercise simpler was that we only had one sample. As a bonus exercise here's what you can do. Use an N bigger than one. Modify this code so that it still produces the same result even when you have multiple samples.

LECTURE 54

Why Do We Need Fancy RNNs?

In this lecture, we are going to start discussing modern RNN units, you may have heard of these already, so I'll just tell you right now what they are. The LSTM and the GRU.

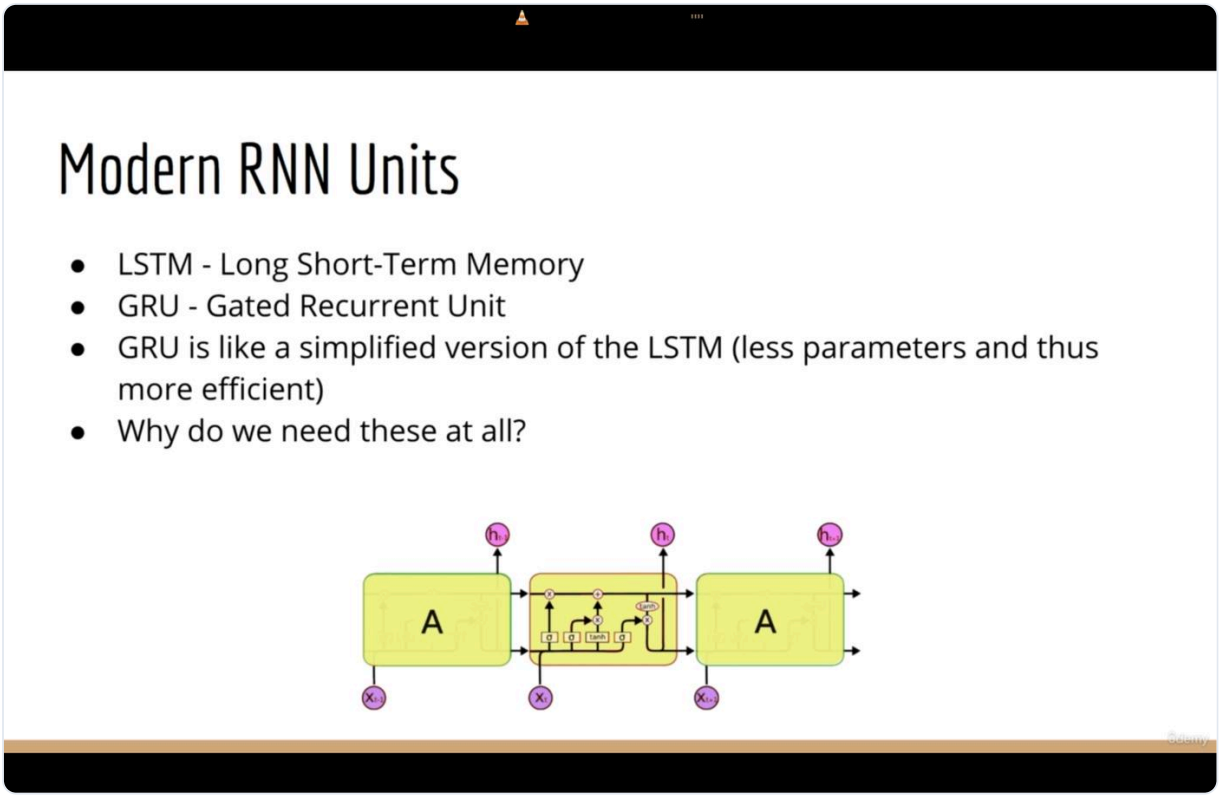


FIGURE 16.1 | Modern RNN Units

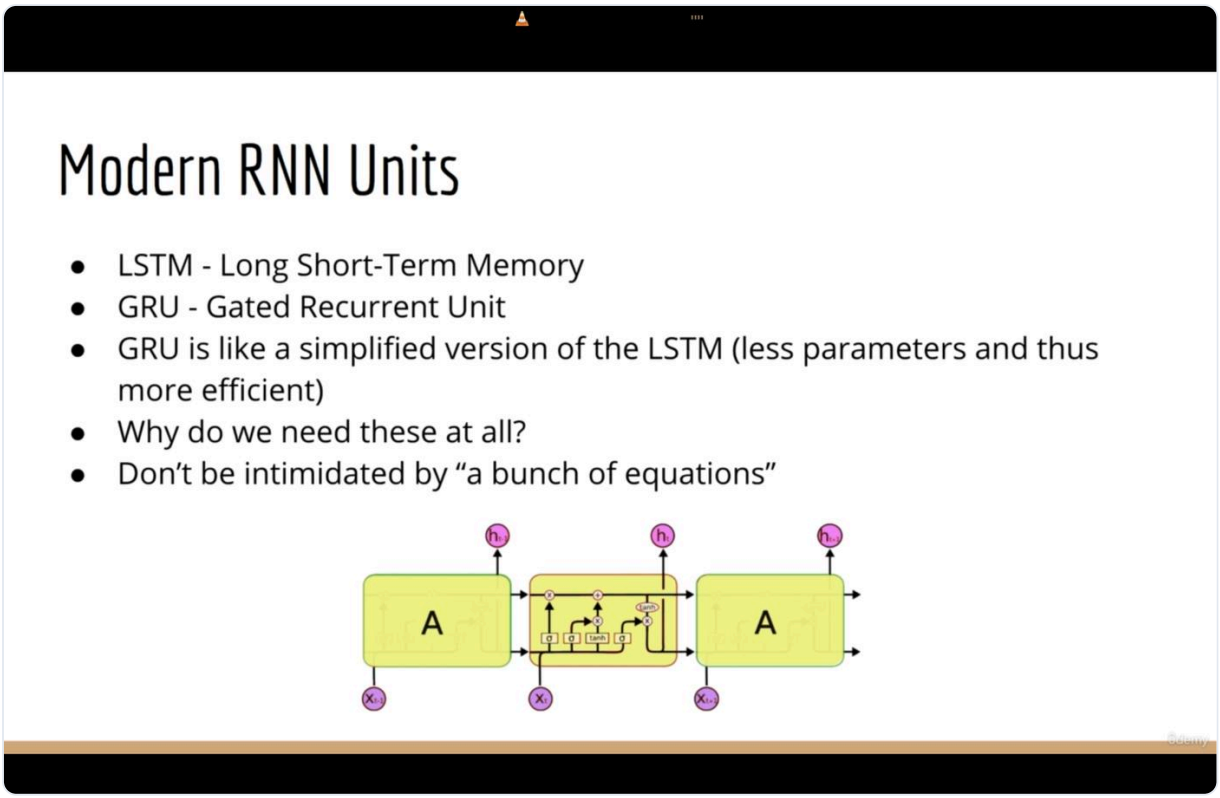


FIGURE 16.2 | Modern RNN Units

LSTM stands for long short term memory and GRU stands for gated recurrent unit. They are both essentially the same thing with the GRU meant to be a more simplified and efficient version of the LSTM. This lecture will discuss why the LSTM and GRU are needed in the first place when the simple RNN seems pretty good already. We'll also look at the underlying equations for the LSTM and GRU and importantly a convenient perspective for looking at them. One mistake a lot of beginners make is that they see a bunch of equations and they get very intimidated. They think, what the heck is this? Just a bunch of equations. What do I do with this?

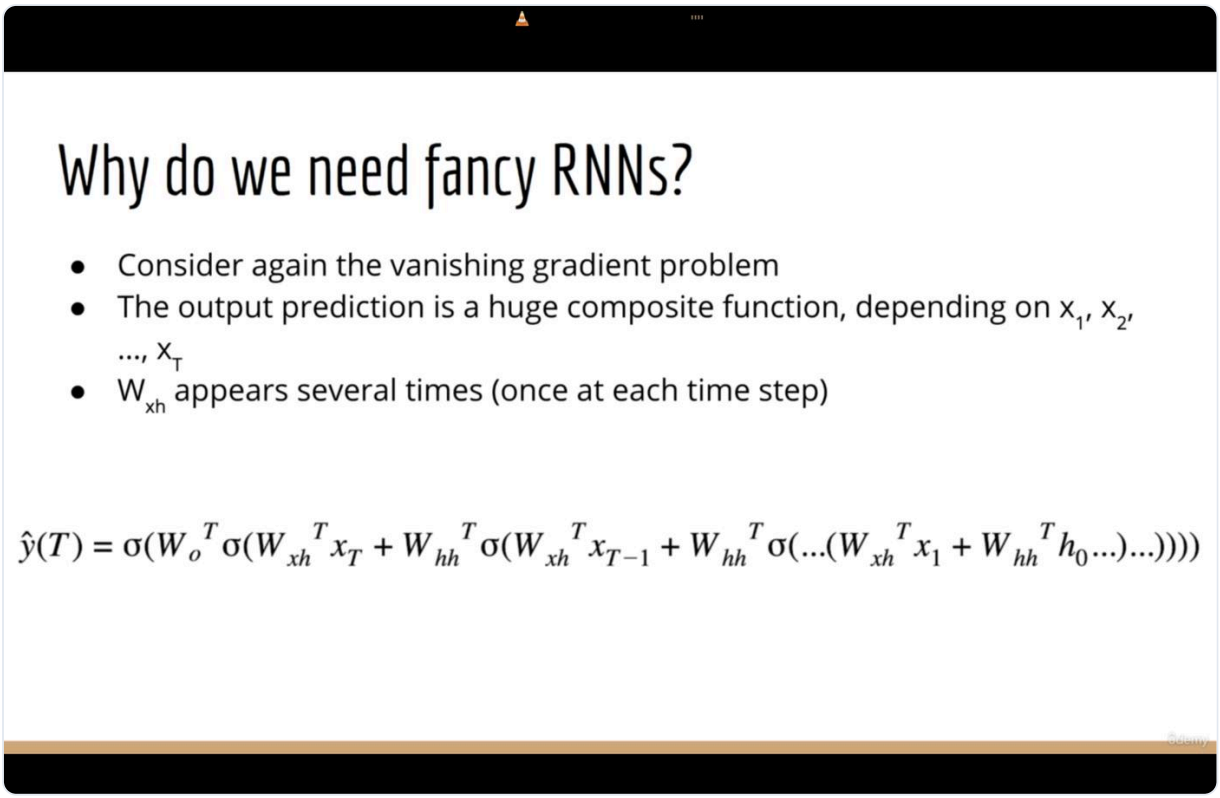


FIGURE 16.3 | Why Do We Need Fancy RNNs?

So hopefully this lecture will take you from that lazy beginner's mindset to how a proper machine learning engineer would think about LSTMs and GRUs. First, we need to discuss why we have these fancy new recurrent units in the first place. Why is a simple recurrent unit not good enough to understand this? We have to go back to the vanishing gradient problem. As with our RNN, we can represent the output prediction y hat big T as a function of the inputs x1 x2 all the way up to x T. Once we do this, we can see that it's just a really big composite function. Now, suppose we want to take the derivative with respect to W x h we can see that W x h appears several times in this equation.

Now, the actual derivatives themselves and how to find them are not what we have to consider here, but what is important is which derivatives are going to show up?

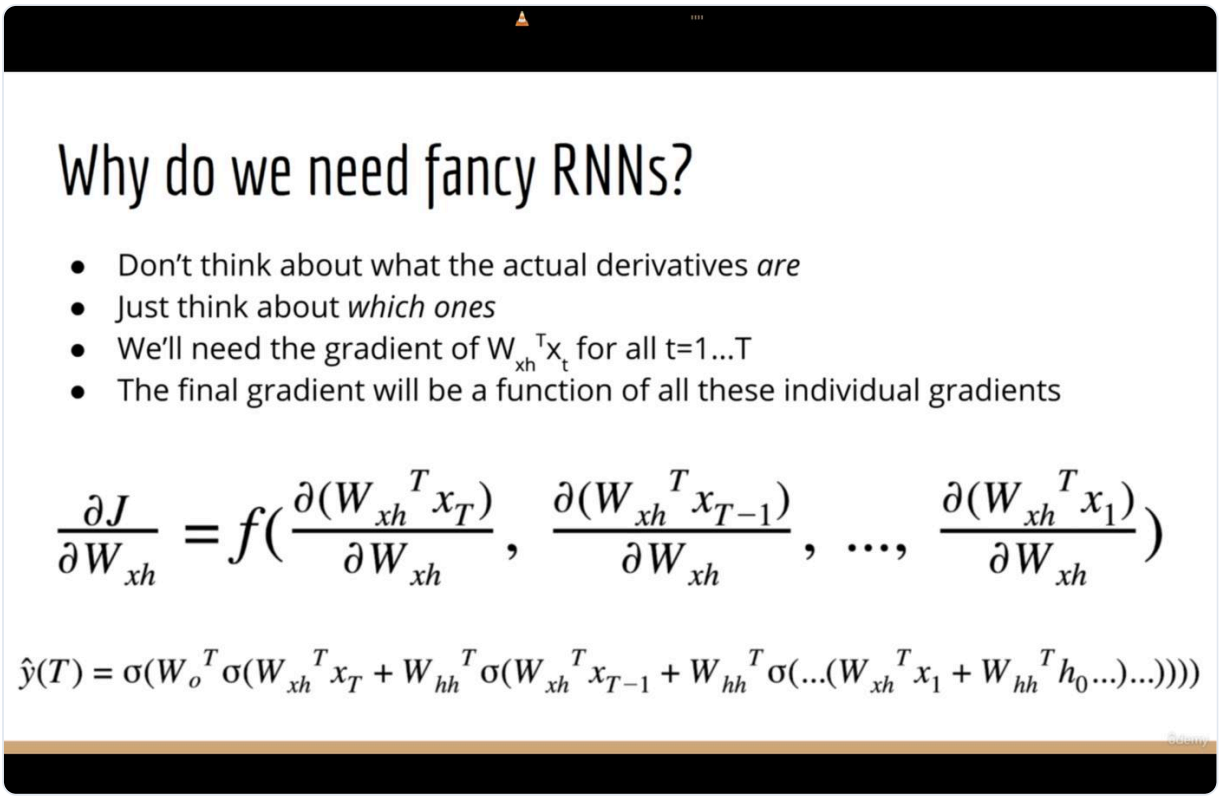


FIGURE 16.4 | Why Do We Need Fancy RNNs?

Well, if W x h appears multiplied by X of T, then we are going to have to find the derivative of W x h times X of T at some point in our expression for the gradient of W x h, it should be clear that W x h appears in front of all the x values for every time step. So ultimately all of these single derivatives are going to appear somewhere in our expression for the gradient of our cost with respect to W x h.

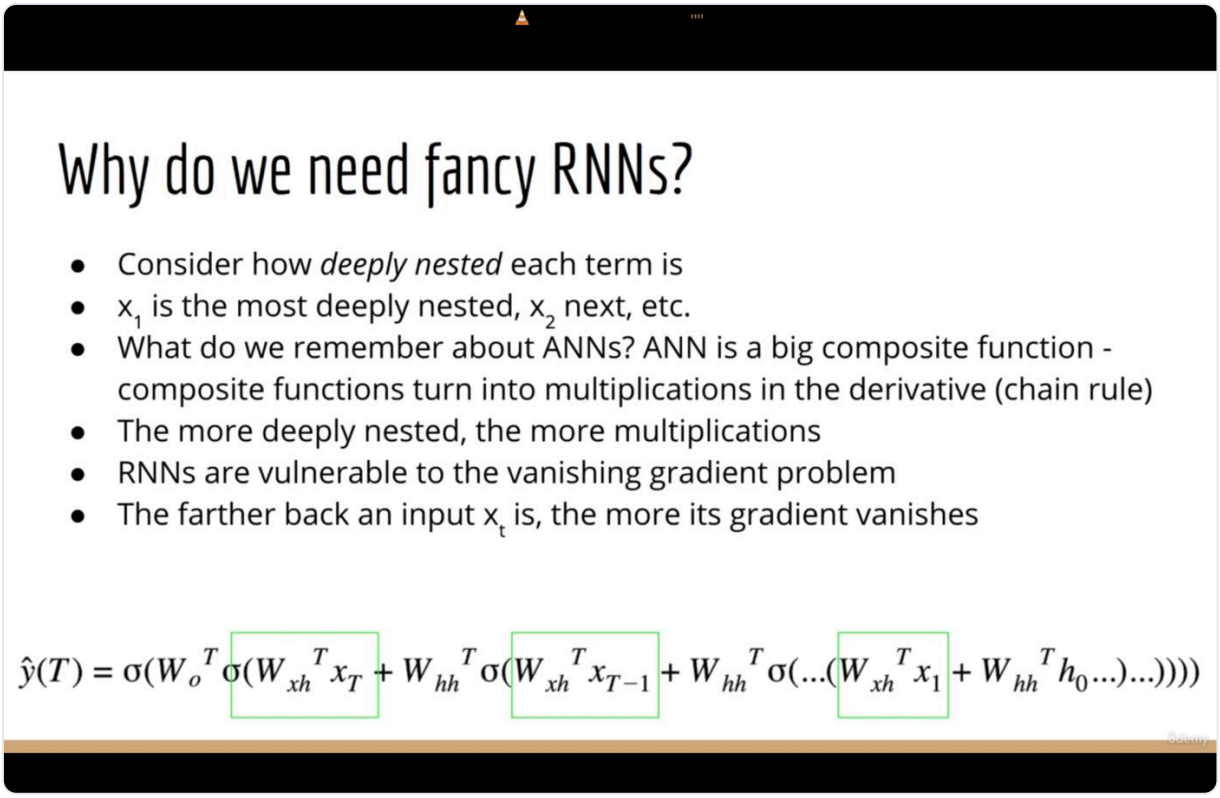


FIGURE 16.5 | Why Do We Need Fancy RNNs?

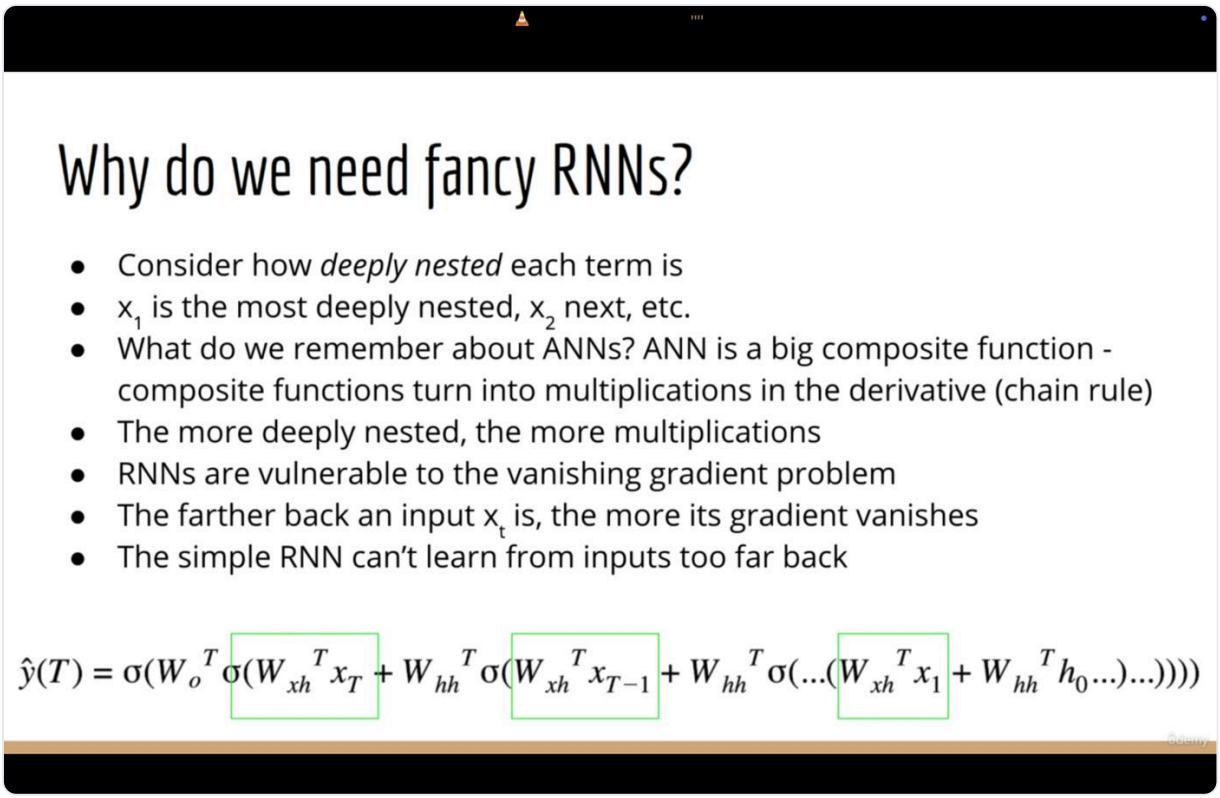


FIGURE 16.6 | Why Do We Need Fancy RNNs?

But another thing that's important to consider is how deeply nested each of these terms is with respect to why hat of Big T, what is clear is that the term involving X1 is the most deeply nested. The term involving X2 is the second most deeply nested and so on. Now, what do we remember about the derivative of composite functions from our theory of ANNs? Well, we know that composite functions turn into multiplication in the gradient. This is due to the chain rule of calculus. Therefore, the more deeply nested you are, the more terms you have to multiply by when you're finding the gradient. In other words, RNNs are particularly vulnerable to the vanishing gradient problem.

The farther back in the sequence the input is, the more vulnerable it is to the effects of the vanishing gradient. This is why it's often said that RNNs have problems learning long term dependencies, the RNN simply cannot learn from inputs that are too far back. So how does the vanishing gradient problem manifest in actual machine learning applications, let's suppose we're doing natural language processing and we would like to extract some information about a document. So to give you a concrete example, suppose an AI is reading the Wikipedia page about Albert Einstein. It has read the entire page.

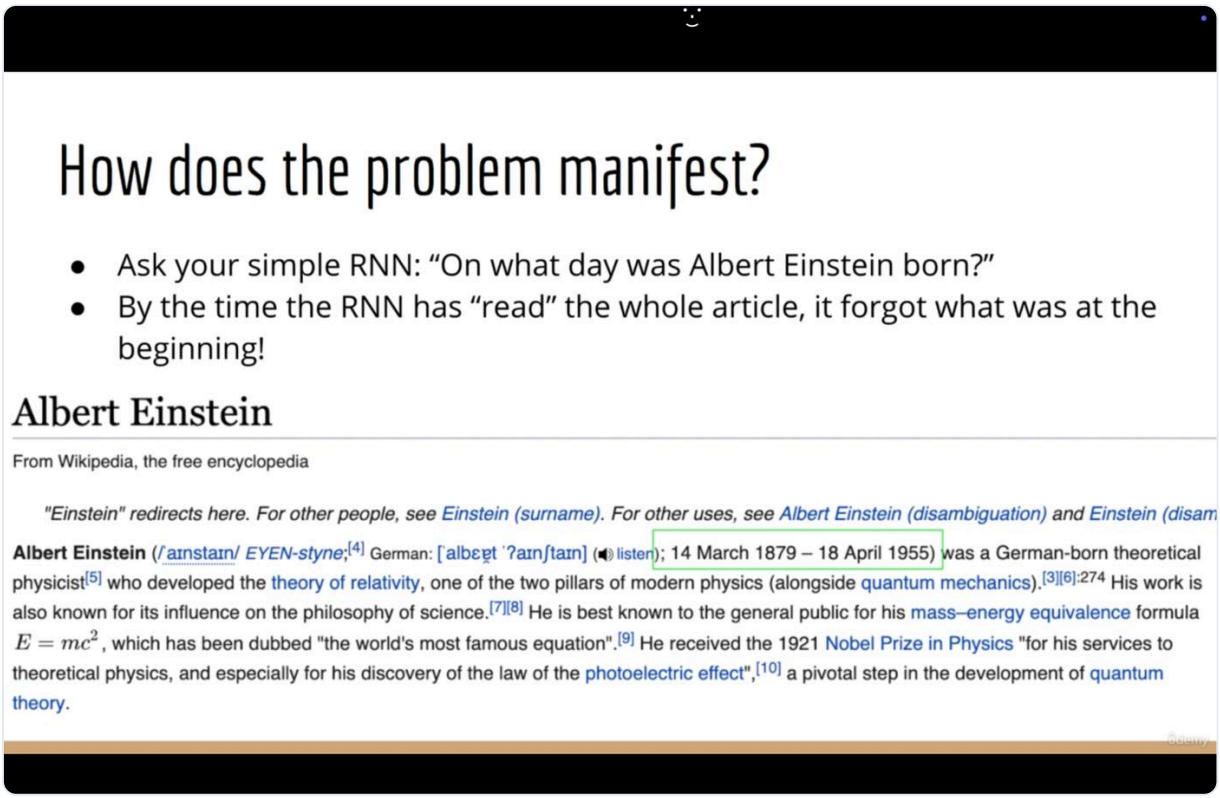


FIGURE 16.7 How Does the Problem Manifest?

Now you ask your AI on what day was Albert Einstein born. Unfortunately, a simple RNN would not be able to answer this, even though this information appears in the article.

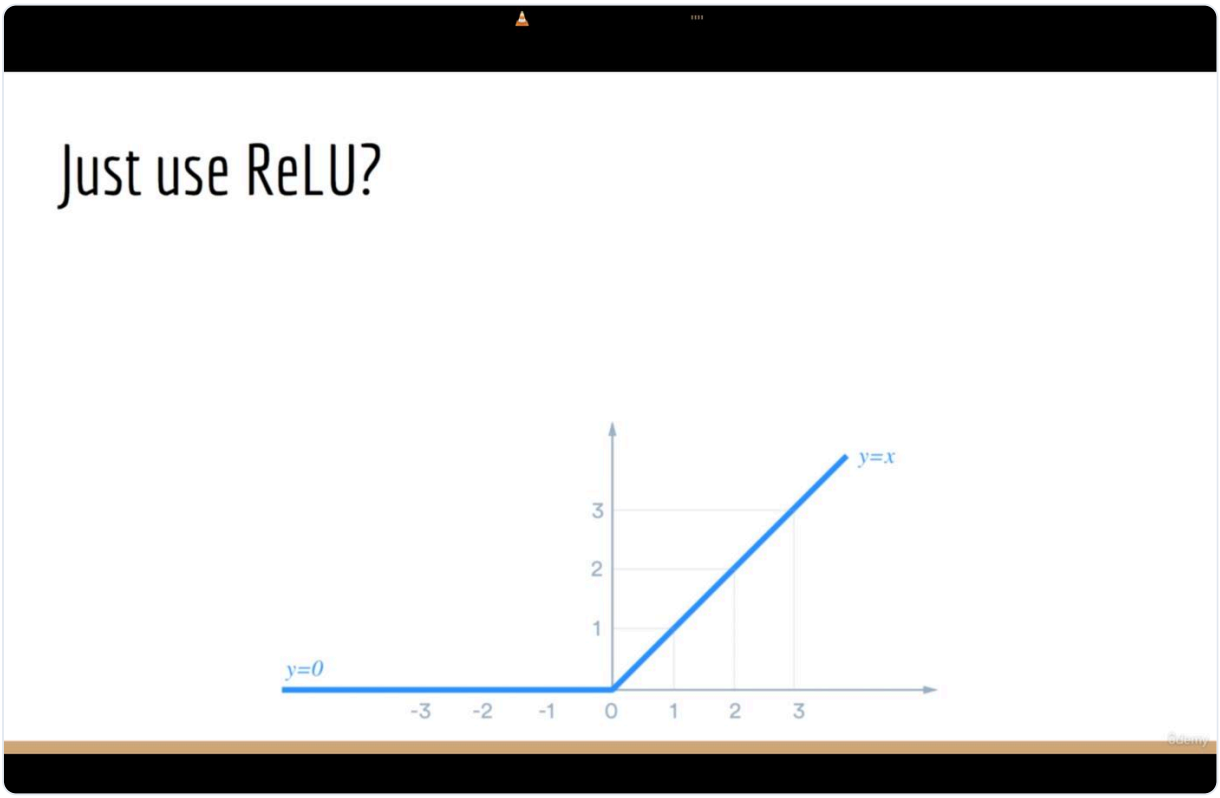


FIGURE 16.8

The problem is it appeared at the beginning of the article. So by the end of the article, the simple RNN has simply forgotten what it had read earlier. Now, you might think, aha, I have a solution to this vanishing gradient problem, it is to use ReLUs.

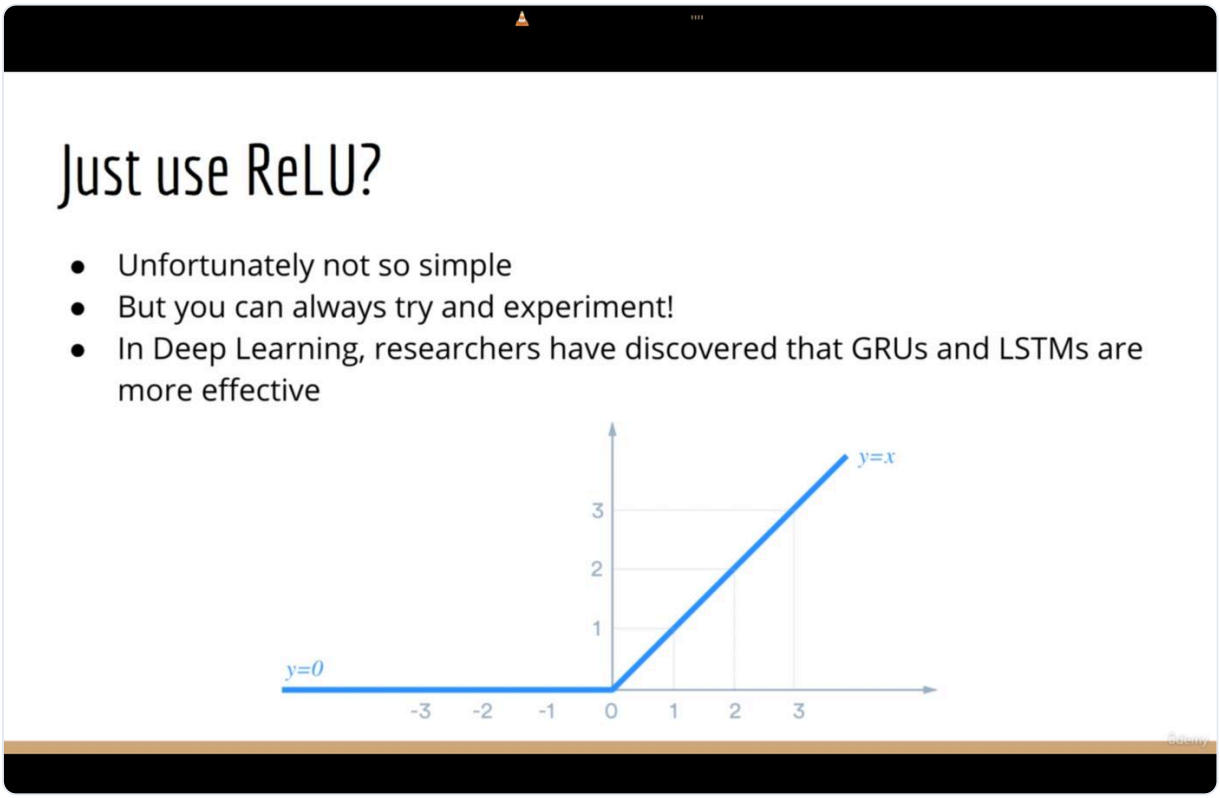


FIGURE 16.9

Unfortunately, with RNNs, things are not so simple, of course, you can always try to use ReLUs in your RNNs and verify experimentally what the results are, but in deep learning, we found that the most effective way of dealing with this problem is to use entirely different units, namely the LSTM and the GRU. One interesting historical fact is that the LSTM was created a very long time ago, 1997. This is more than a decade before deep learning was even called deep learning.

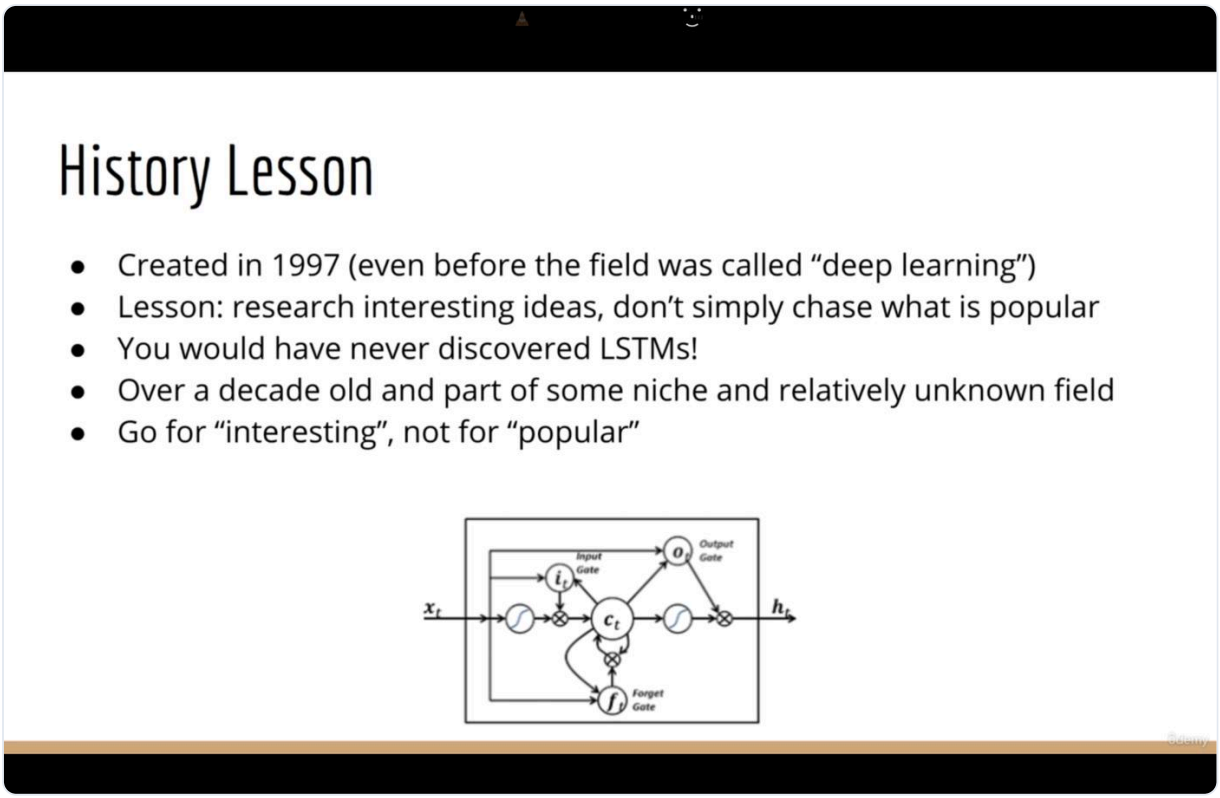


FIGURE 16.10

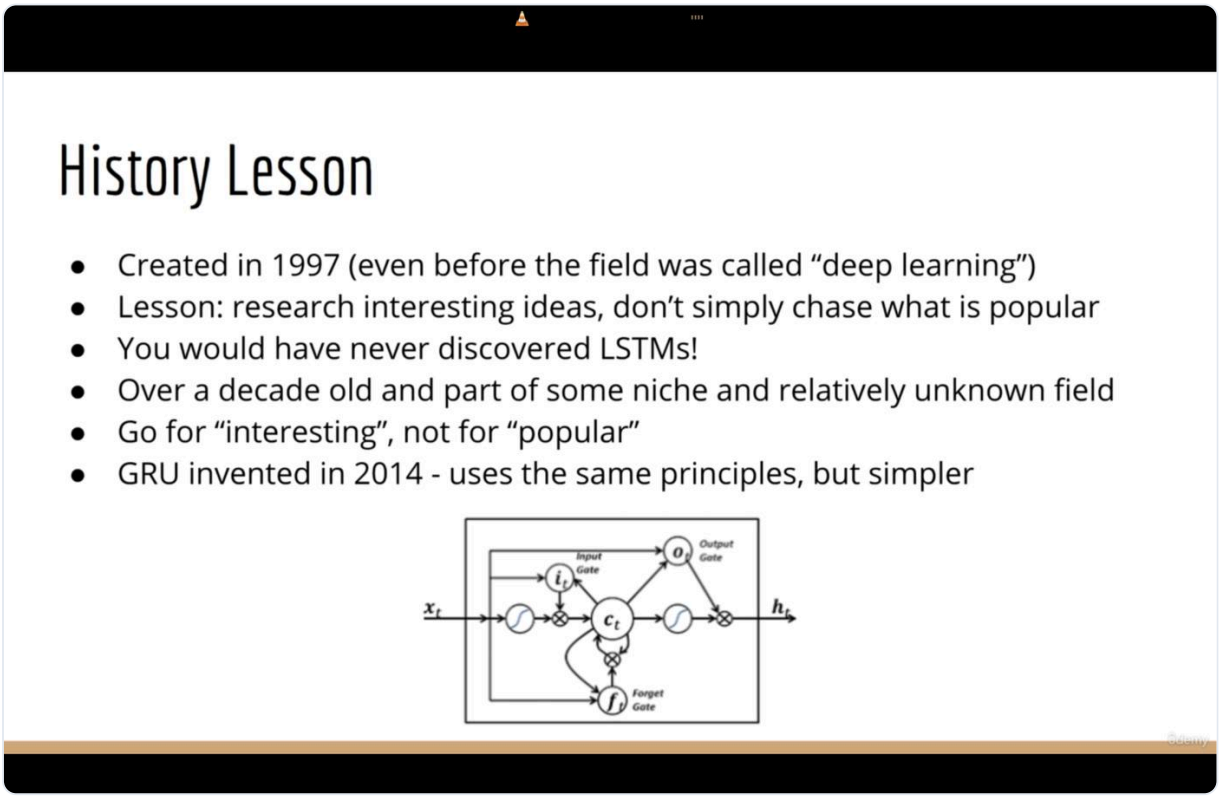


FIGURE 16.11

So this is why I always say to people research, interesting ideas. Don't just go through your career chasing what is popular. By doing that, you were like a hamster running on a spinning wheel, going nowhere. If you're only chasing popular mainstream ideas, you would have never come across this idea of LSTMs. It's more than a decade old part of a field that nobody even cares about. Do what is interest-

ing, not what is popular. Now, the LSTM is an extremely complex unit, the GRU was invented more recently in 2014, but it uses a lot of the same ideas. So I always like to start with the GRU so that you can learn the principles in a simpler setting and then go back to the LSTM.

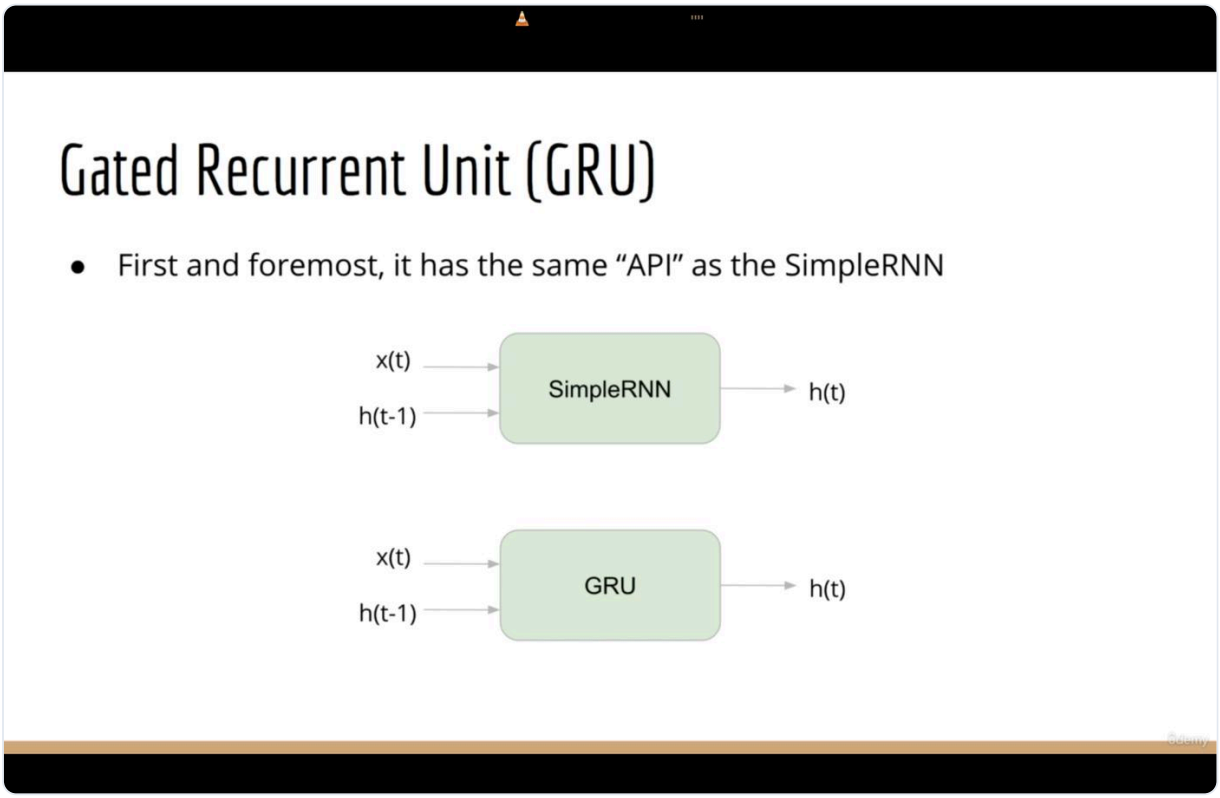


FIGURE 16.12 | Gated Recurrent Unit (GRU)

So what is the gated recurrent unit? First, let's start with the basic principle from a simple RNN. We still want to have some hidden state at time t h of t this hidden state will still depend on x of t the current input and h of T minus one, the previous hidden state. The only thing that will change is how h of T is calculated. Now, there's a little bit of a debate on how to present the GRU, is it more useful to look at the equations or is it more useful to look at diagrams? Now, if you are a beginner, you might automatically say pictures, pictures of the best for learning. Unfortunately for you, that's not what we are going to do.

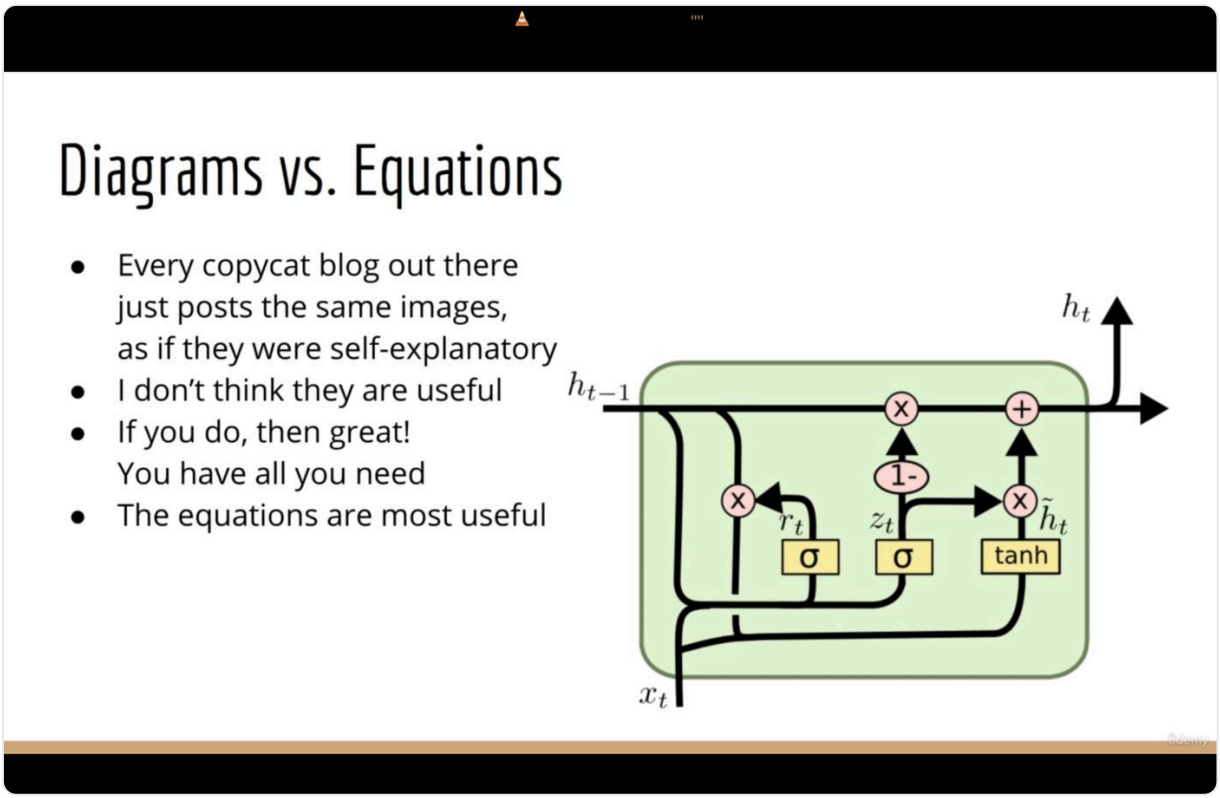


FIGURE 16.13 | Diagrams vs. Equations

A lot of poorly written blogs that I'll copy from each other use these same diagrams as if they were self-explanatory. You look at this picture and all of a sudden, bam, you understand LSTMs and GRUs. Personally, I find this doesn't work for me. I know that other prominent lecturers feel the same way. Of course, if you find these pictures useful, you're welcome to look at them and try to make sense of them. But for me and many others I know, it's actually the equations which are most useful that might seem backwards to a beginner who is very terrified of math, but hopefully you are past that point right now.

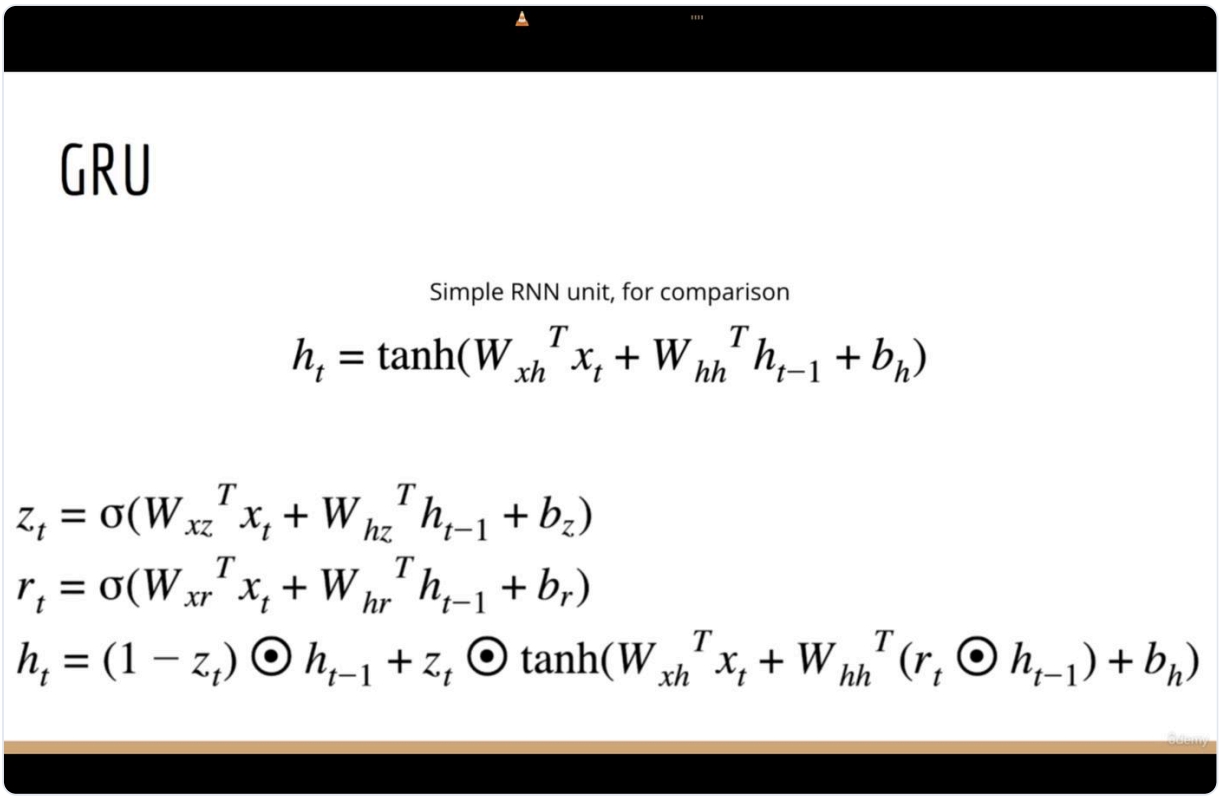


FIGURE 16.14 | Simple RNN Unit, for Comparison

I promise you that there's nothing here that we haven't seen before. OK, so enough chitchat. Where is the GRU? Well, here are the equations for the GRU to sort of give you some perspective on this. I've also included the equation for a simple recurrent unit so that you can compare the complexity of the two. The simple recurrent unit is essentially just one line. h of T depends on x of t and h of T minus one, the GRU is three lines because we have to calculate three different things. First, we calculate z of T, which is called the update gate vector.

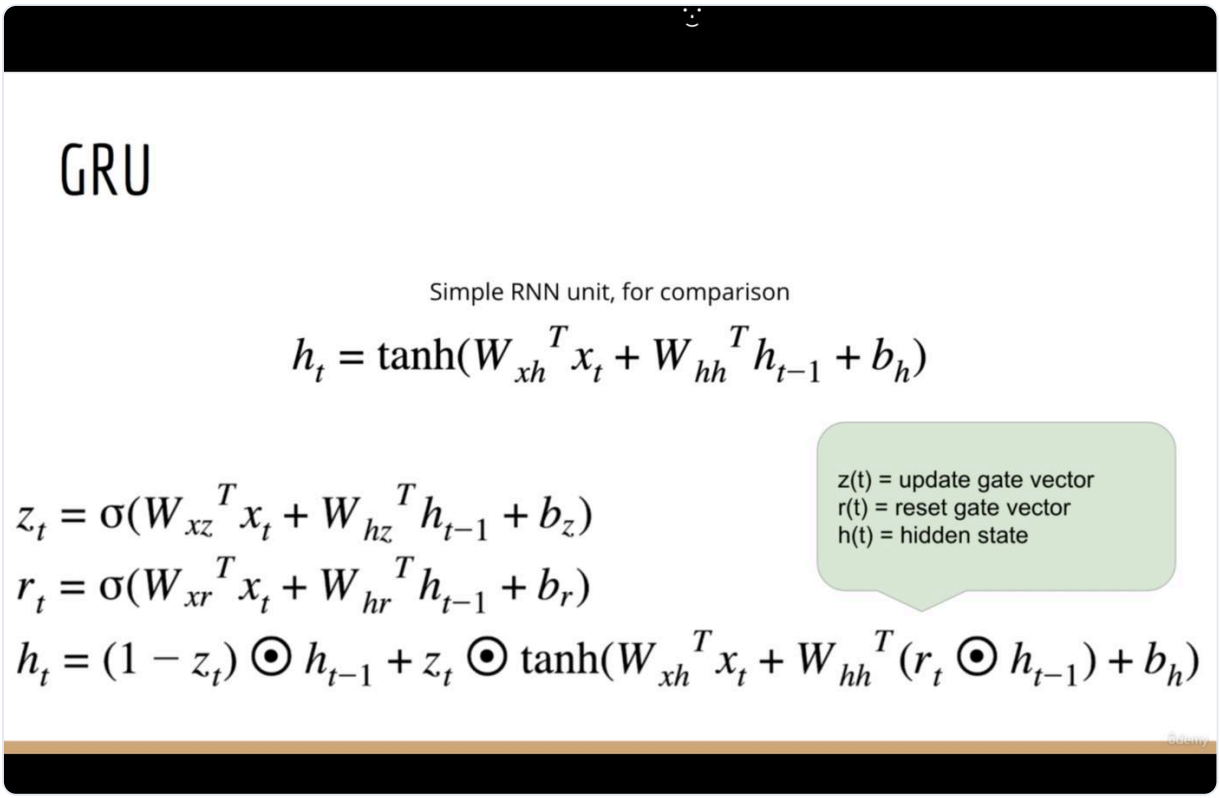


FIGURE 16.15 | Simple RNN Unit, for Comparison

Second, we calculate r of t, which is called the reset gate vector. And finally we calculate h of T, which is the hidden state vector, which is, as with the simple recurrent unit, what gets passed on to the next layer in the neural network. So let's analyze these equations so that we can make sense of the GRU. First, it's important to discuss shapes, you want some visualization of the objects we're dealing with? Well, it's helpful to know that all of these new vectors are just vectors of size m the same size as H of T, as always. This is a hyperparameters, so you can choose how many features the hidden layer should have.

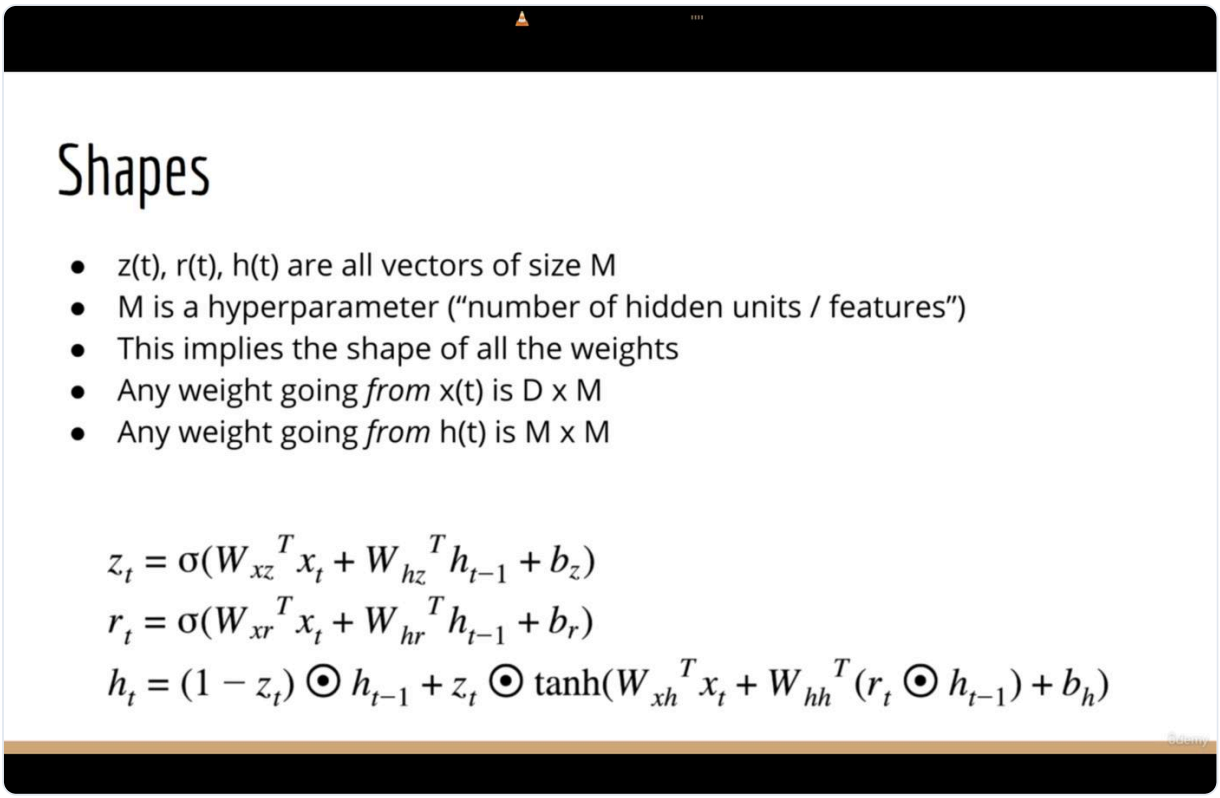


FIGURE 16.16 | This Implies the Shape of All the Weights

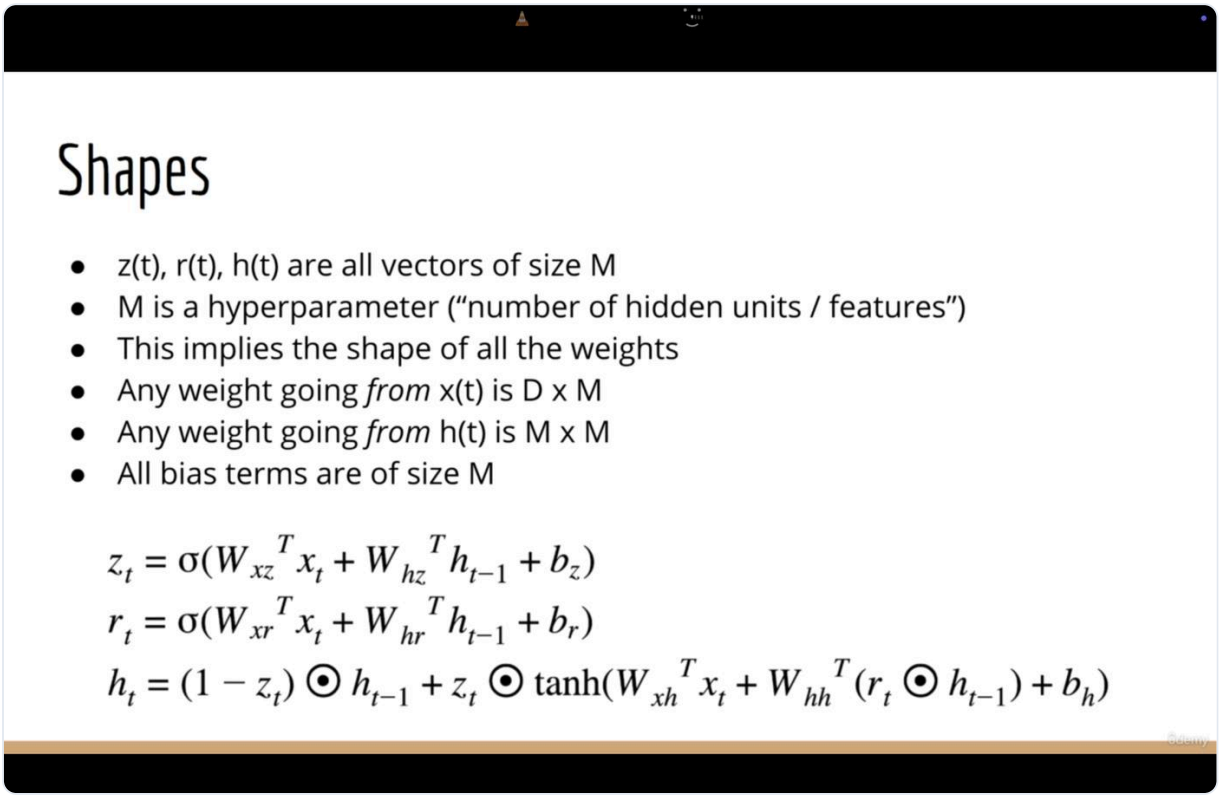


FIGURE 16.17

This also implies the shape of all these weights. If a weight is going from X of T to one of the gate vectors, then it must be of size D by M. If a weight is going from h of T minus one to one of the gate vectors, then it must be of size M by M. And of course, since all the vectors are of size M, then all of the bias terms must also be of size M. Next, let us consider what the GRU is trying to do. I think the names are very

helpful here. z of T is the update gate vector and r of T is the reset gate vector. By the way, if you didn't know the circle with a dot in, it means element-wise multiplication. So one minus Z is an M size vector and h of T minus one is also an M size vector.

When you multiply these together, you are doing element-wise multiplication. So let us start by looking at z of t how is it used? We can see that z of t gets multiplied by one term involving x of t and h of T minus one, then one minus Z gets multiplied by the previous hidden state, h of T minus one. This is very helpful. You can think of ZT as telling us what should we do? Should we take this new value for the hidden state, or should we just remember the previous hidden state h of T minus one? Why is that helpful? Well, remember what we learned about the vanishing gradient. If we use a simple RNN, it is prone to forgetting things that it has seen in the past by using this gate, it explicitly allows us to remember the previous hidden state so that the hidden state can be carried forward to the next hidden state.

By the way, since ZT is the output of a sigmoid, its values are always between a zero and one. Therefore, for h of T, we are always taking a weighted some of the previous H of T minus one and this other function. So if z of t is close to zero, then we will remember the old value h of T minus one. But if it is close to one, then we'll take the new value and forget the old H of T minus one. By the way, you can think of this other function as analogous to what the simple RNN is doing. This is very important, but there's one more piece of the puzzle before we can move on to a helpful interpretation of what's going on. I want you to look very carefully at this equation for z of T, what does this look like to you?

Well, it is a sigmoid of some dot products, plus a bias term. What does that look like?

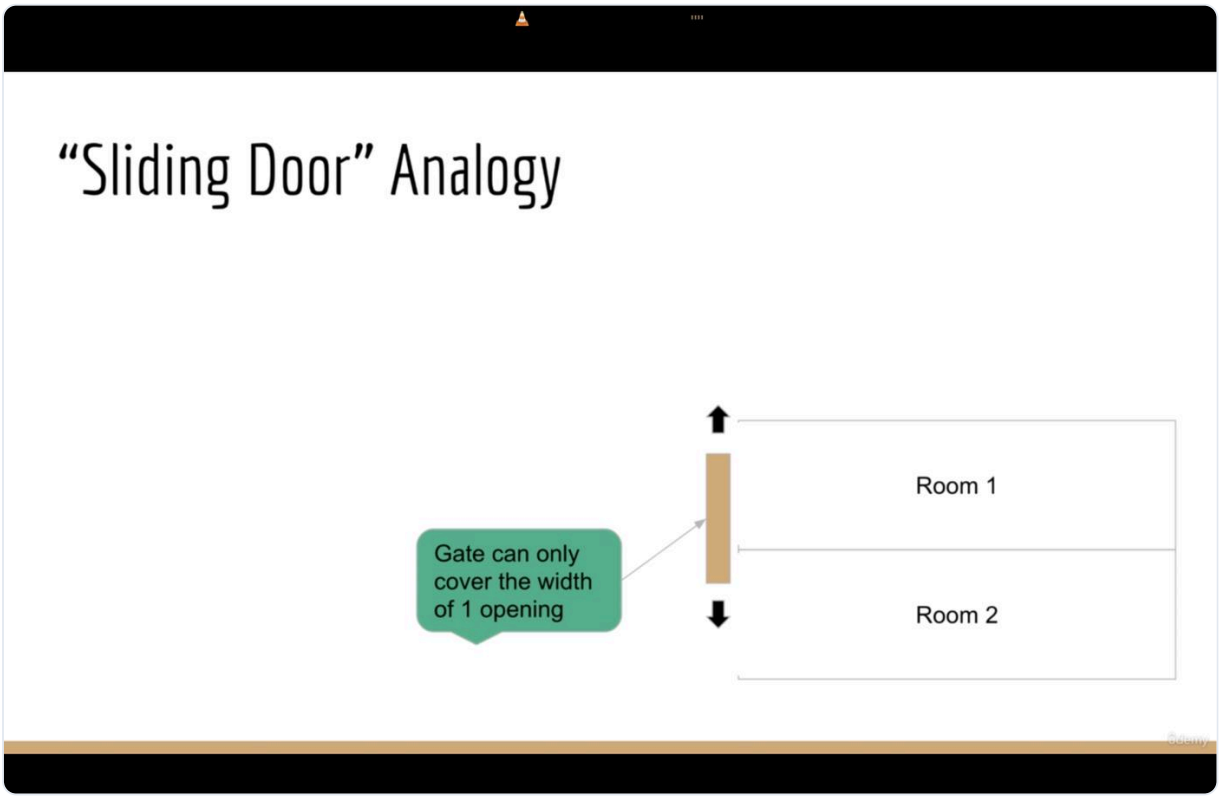


FIGURE 16.18 | Gate Can Only

Well, you might recognize this as exactly like a dense layer. Now, you might object saying that there are two weights. So it's not exactly like a dense layer. But in fact, the way that these units are often implemented, what we do is we concatenate X of T and H of T minus one into a single vector. Let us just call it V for short. If we concatenate those inputs and we concatenate the weights, then we just get back to our

regular old dense layer where we have one input multiplied by one weight matrix plus a bias term. In other words, what is this? This is just a neuron. Now notice that it also ends with a sigmoid. Importantly, this sigmoid is not a hyperparameter.

This is usually always a sigmoid because the output is always a number between a zero and one. That's how we can take Z of T and one minus z of T later on in the GRU. So what is this neuron predicting? It's like a binary classifier telling us essentially the probability that we should take the new value for h of T or if we should keep the old value of T minus one. So if we ignore the reset gate for now, here is how we can interpret the GRU first, remember that h of T is essentially the weighted sum of two things, h of T minus one and the output of a simple RNN. Z of T, the update gate vector is like an output probability, telling us the probability that we should keep the output of the simple RNN in other words, keep h of T minus one with probability one minus C of T and keep the simple RNN output with probability z of T.

The final h of T is then just a mixture of these two, allowing us to keep the most important parts of h of T minus one and discard the rest. Now, I want to make one small clarification, please note that this is really for intuition only. These are not really probabilities of keeping and discarding because we never actually keep or discard. Instead, what do we actually do is take a mixture of the two components. For instance, twenty five percent of component one and seventy five percent of component two. But we don't actually keep or discard. In actuality, the word gate is very helpful. Imagine two openings side by side. Now, imagine a gate which is the same size as each opening, but can only slide between the two openings.

Thus it can only fully cover one of the openings at once, or it can simply cover a part of each. But notice that in total, it always covers an area equal in size to one full opening.

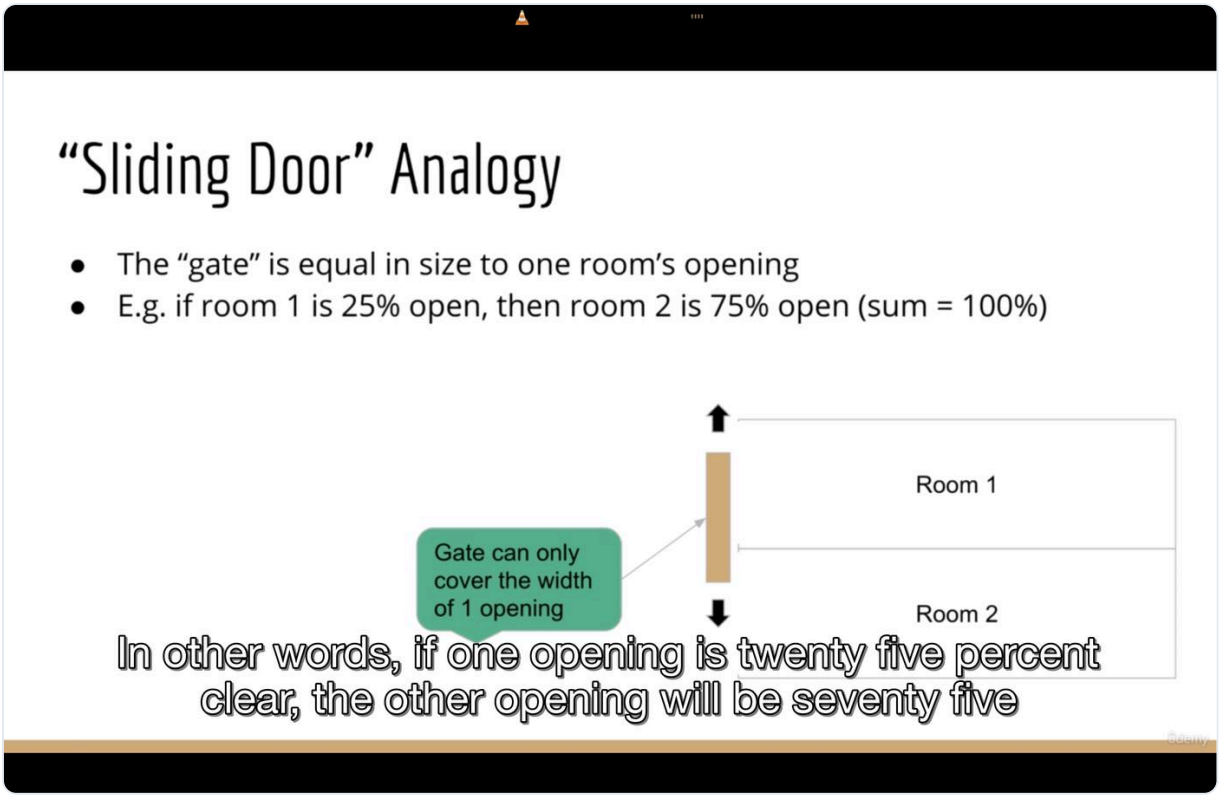


FIGURE 16.19

In other words, if one opening is twenty five percent clear, the other opening will be seventy five percent clear and they always sum up to one hundred percent.

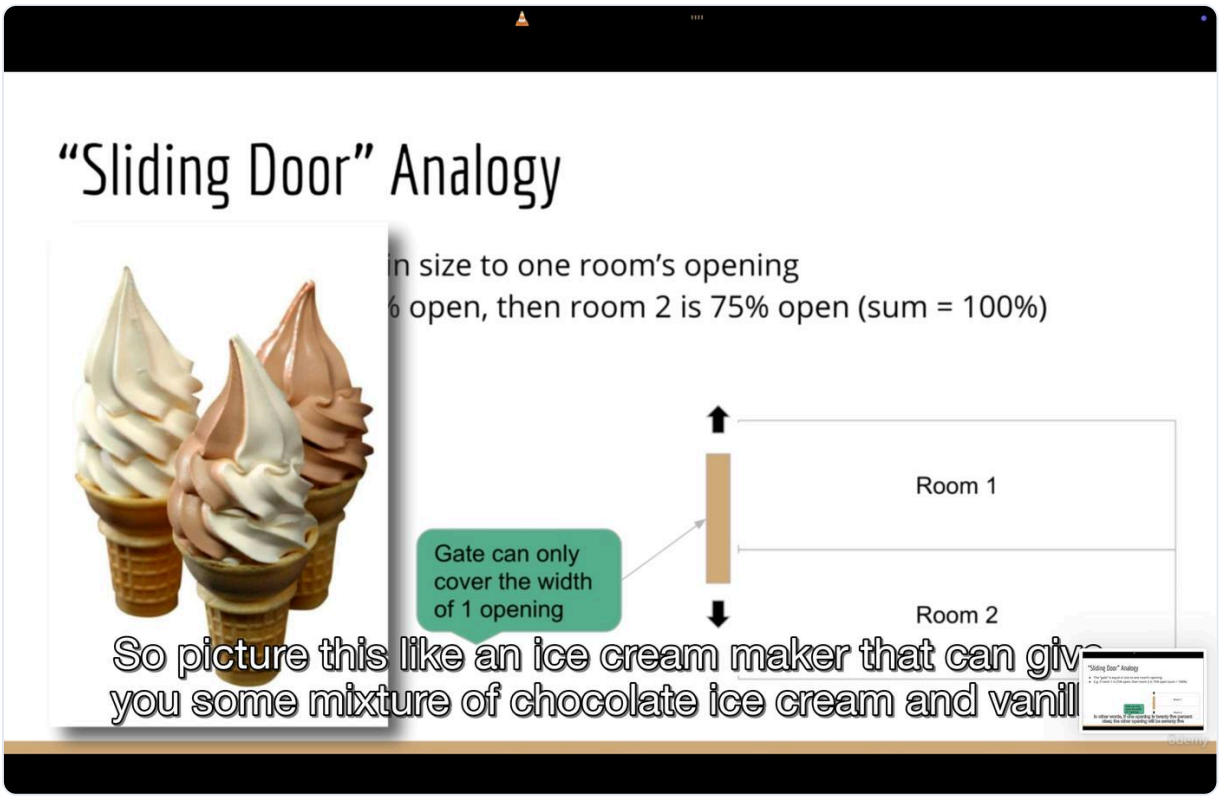


FIGURE 16.20

So picture this like an ice cream maker that can give you some mixture of chocolate ice cream and vanilla ice cream.

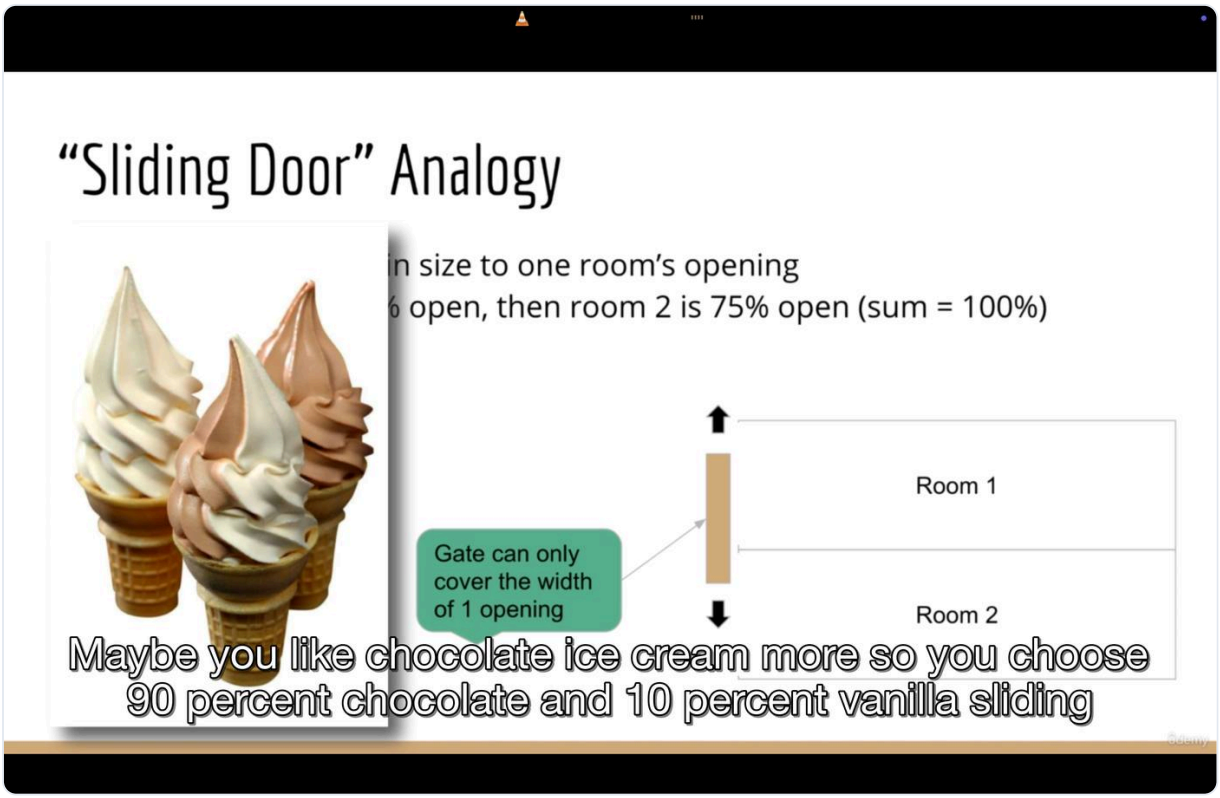


FIGURE 16.21

Maybe you like chocolate ice cream more so you choose 90 percent chocolate and 10 percent vanilla sliding the gate to the appropriate point to achieve this. Note that in practice, this is called a convex combination.

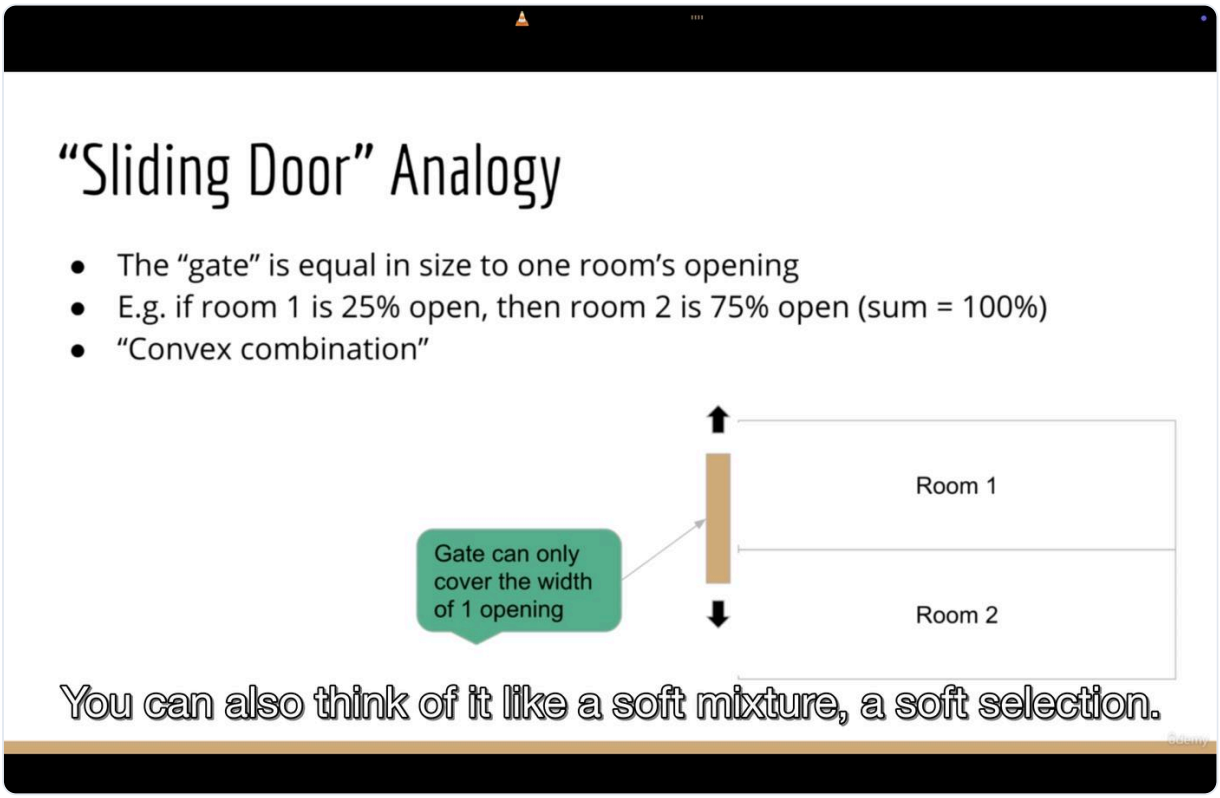


FIGURE 16.22

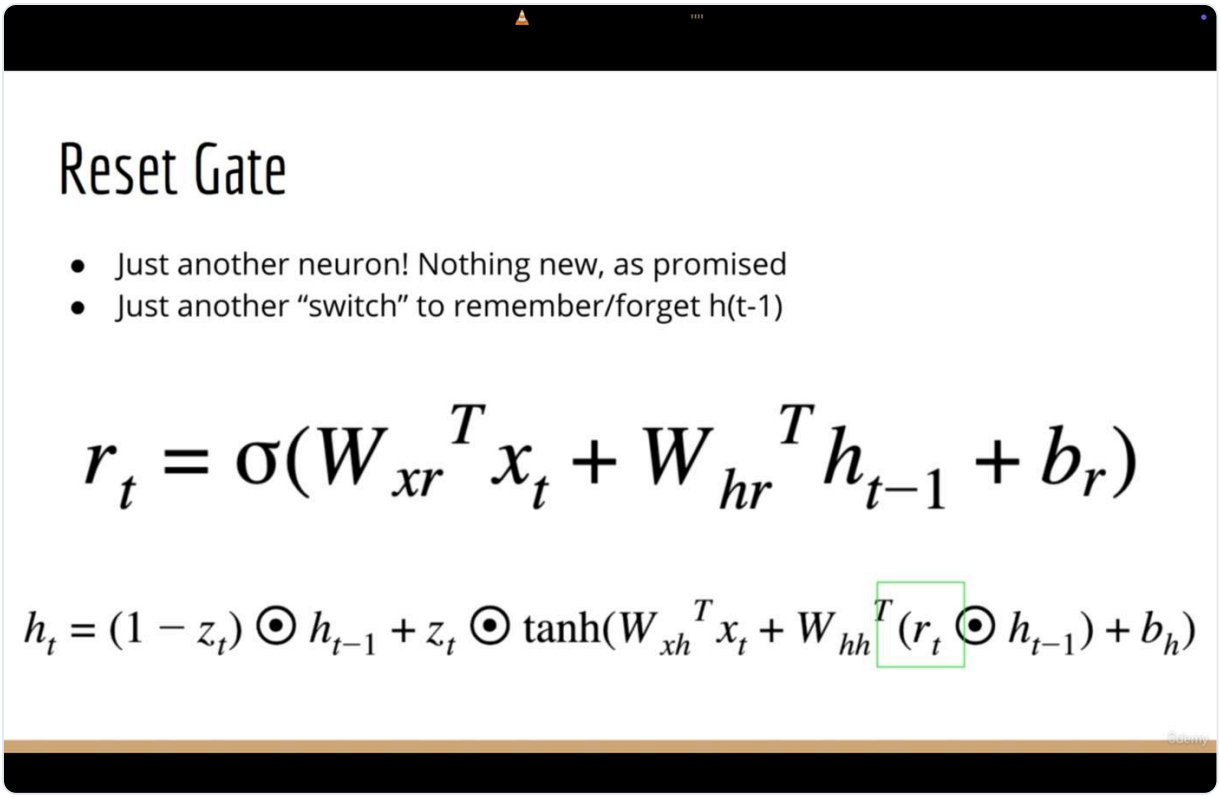


FIGURE 16.23 | Reset Gate

Reset Gate

- Just another neuron! Nothing new, as promised
- Just another "switch" to remember/forget h(t-1)

$$r_t = \sigma(W_{xr}{}^T x_t + W_{hr}{}^T h_{t-1} + b_r)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}{}^T x_t + W_{hh}{}^T (r_t \odot h_{t-1}) + b_h)$$

FIGURE 16.24 | Reset Gate

GRU Summary

- Same API as simple recurrent unit
- Output is h(t), depends on h(t-1) and x(t)
- Has gates to remember / forget each component of h(t-1)
- SimpleRNNs have no choice but to eventually forget, due to the vanishing gradient

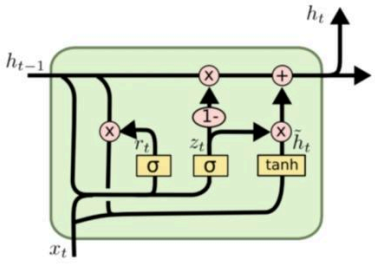


FIGURE 16.25 | Same API as Simple Recurrent Unit

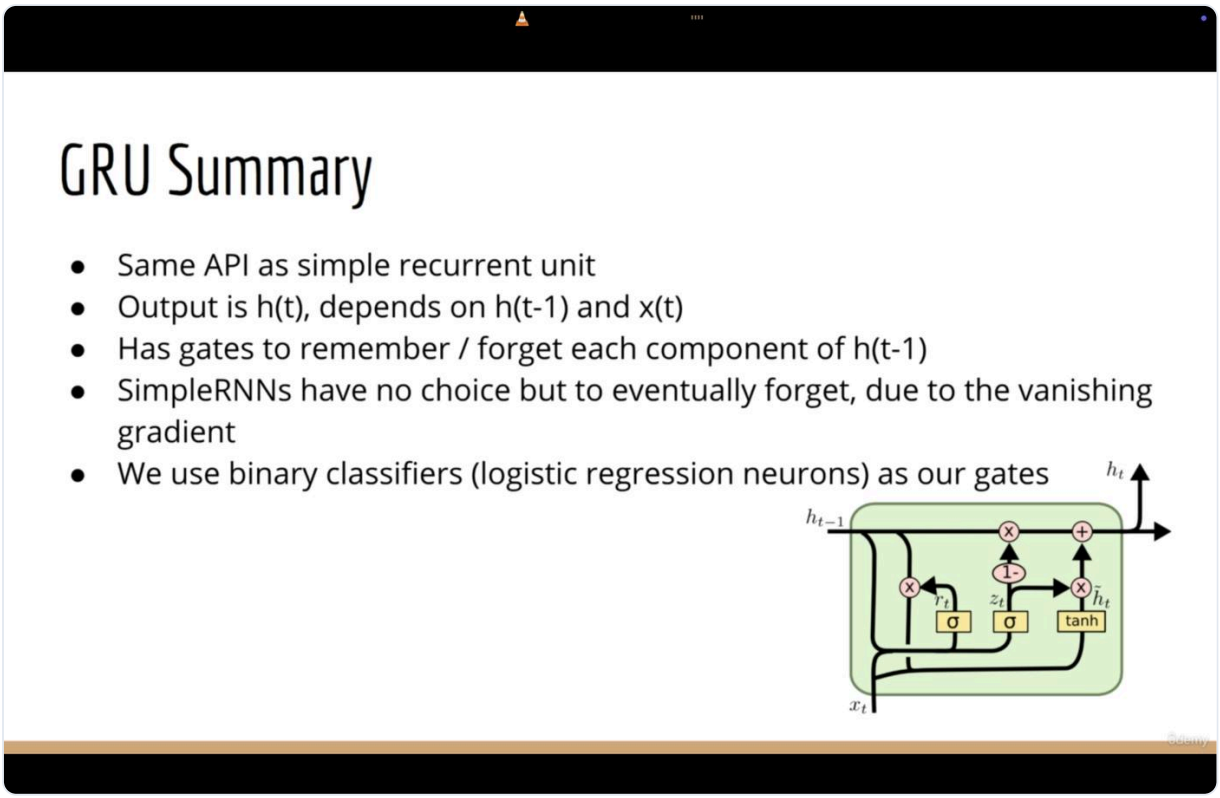


FIGURE 16.26 | Same API as Simple Recurrent Unit

You can also think of it like a soft mixture, a soft selection. Let's now turn our attention to the other gate vector, the reset gate vector, how does this come into play? Well, first, let's consider how this is calculated. As you can see, it's just another neuron, so that should give you some comfort, there is really nothing new here. As I promised, we're just making use of neurons, which we've already learned about, just neurons connected to more neurons. Now it is called the reset gate vector, which is a good hint about what it's for. Remember that we called the second part of the calculation a simple RNN, but this was not entirely true. This is because this is where the reset gate appears.

As you can see, the reset gate is used by doing an element-wise multiplication with h of T minus one. Importantly, remember that the values of r of T are always between zero and one. What happens if we multiply h of T minus one by a value close to zero? Well, it just gets closer to zero. What if we multiply by one? Then it just stays the same as it was before. In other words, before multiplying by the hidden to hidden weight W h h we decide which parts of h of T minus one we want to remember and which parts we want to forget. So we forget them simply by resetting them back to zero, hence the reset gate. Note that this performs a very similar function to the update gate z of T.

It's just in a different place. So it's just reinforcing our ability to remember and forget different parts of T minus one. OK, so to summarize what we've learned so far, what is the GRU and why is it useful? The GRU first and foremost has the same API as the simple recurrent unit. The output is still h of T and it still depends on h of T minus one. And accept the difference between this and the simple recurrent unit is it has functionality for remembering and forgetting what was in h of T minus one. This solves a problem we had with simple RNNs where due to the vanishing gradient problem, it would forget things that it saw earlier in a sequence. We accomplish this by adding little binary classifiers in the form of logistic regression neurons that output a prediction on whether to remember or forget.

LECTURE 55

Return of the Neuron

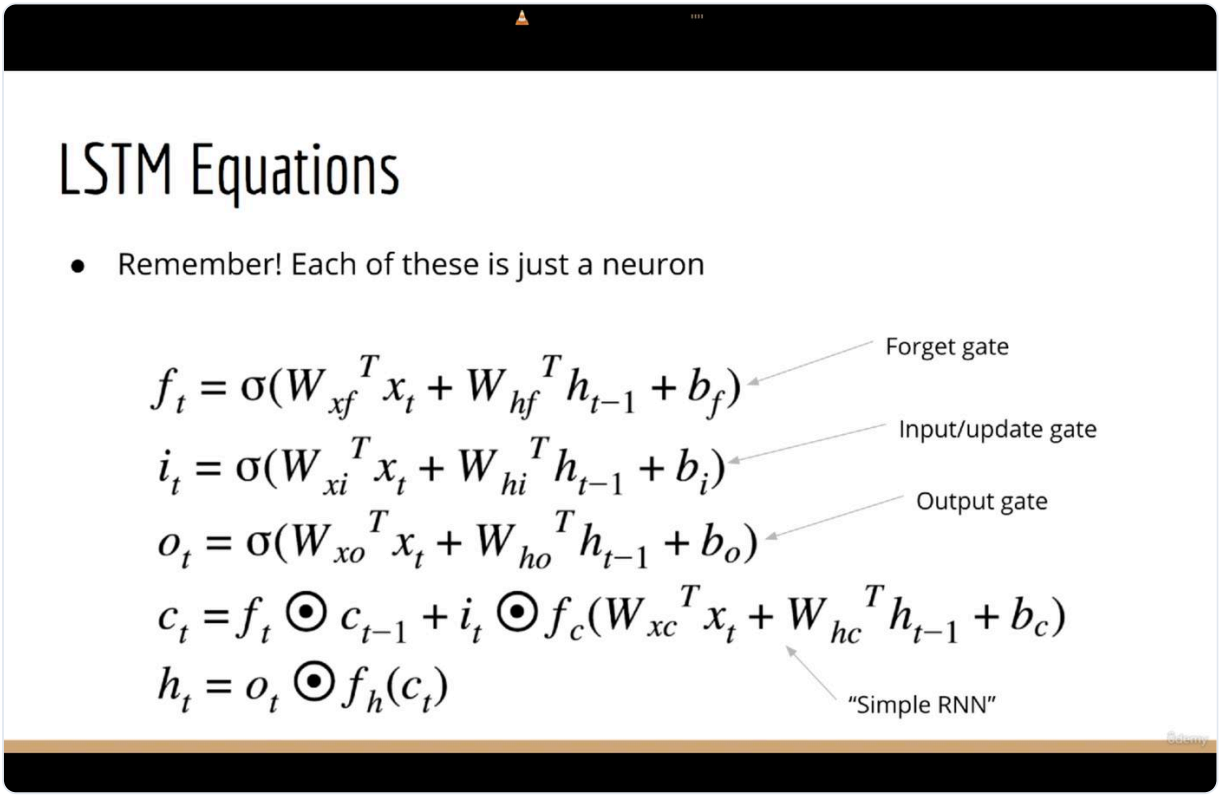


FIGURE 17.1

In this lecture, we are going to continue our discussion of the LSTM and GRU, previously we discussed the GRU and now we are going to move on to the LSTM.

LSTM Equations

- Note: f_c and f_h are tanh by default, and cannot be changed

$$f_t = \sigma(W_{xf}{}^T x_t + W_{hf}{}^T h_{t-1} + b_f)$$
$$i_t = \sigma(W_{xi}{}^T x_t + W_{hi}{}^T h_{t-1} + b_i)$$
$$o_t = \sigma(W_{xo}{}^T x_t + W_{ho}{}^T h_{t-1} + b_o)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot f_c(W_{xc}{}^T x_t + W_{hc}{}^T h_{t-1} + b_c)$$
$$h_t = o_t \odot f_h(c_t)$$

FIGURE 17.2

LSTM is simple!

f(t) = Neuron (binary classifier)

i(t) = Neuron (binary classifier)

o(t) = Neuron (binary classifier)

c(t) = f(t) * c(t-1) + i(t) * SimpleRNN

h(t) = o(t) * tanh(c(t))

tanh() is usually kept even though it's possible to change

Note: if you change it, the LSTM unit won't be GPU-compatible

FIGURE 17.3

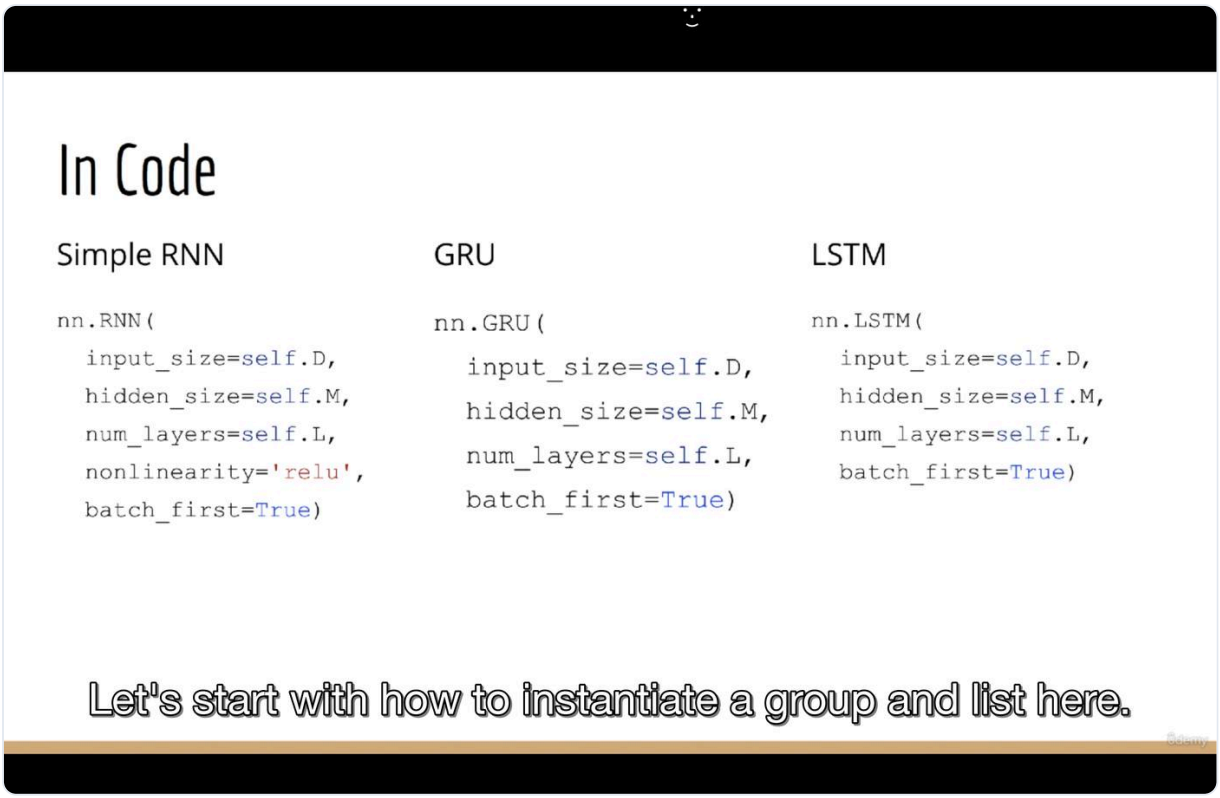


FIGURE 17.4 | In Code

To recap, what did we learn about the GRU?

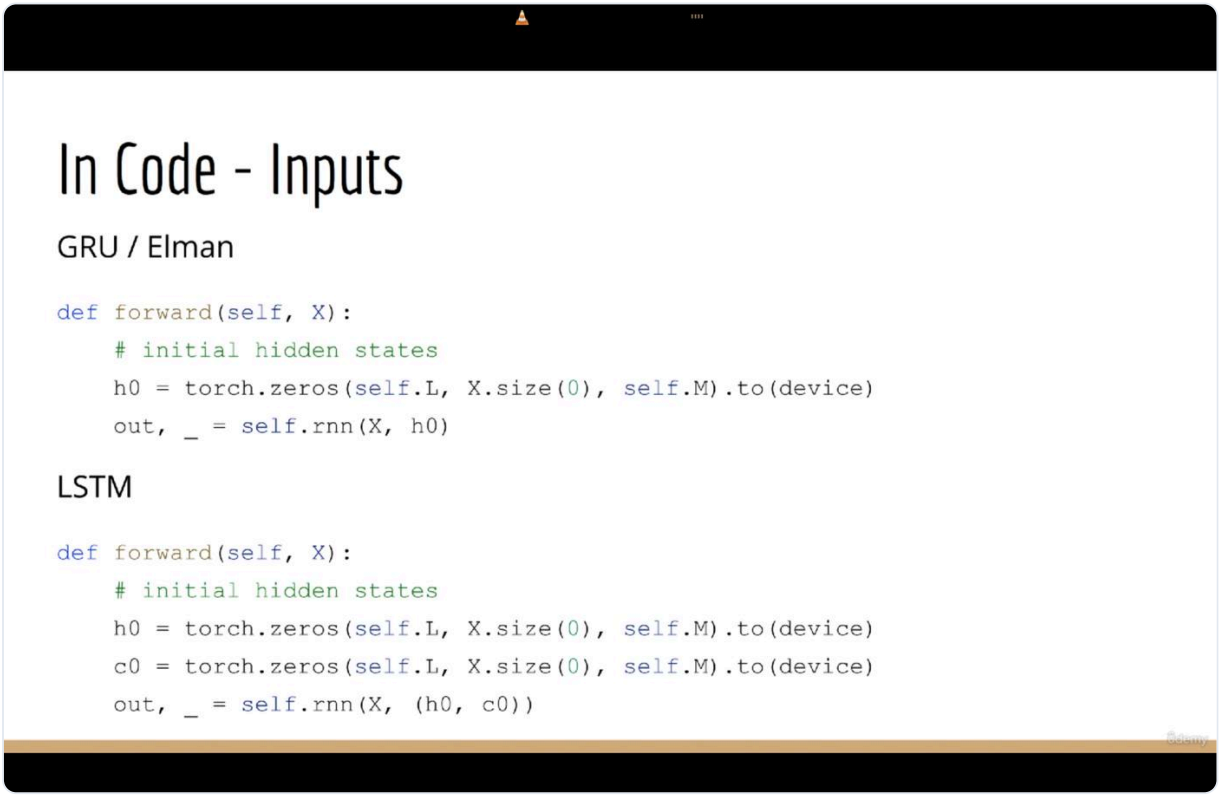


FIGURE 17.5 | In Code - Inputs

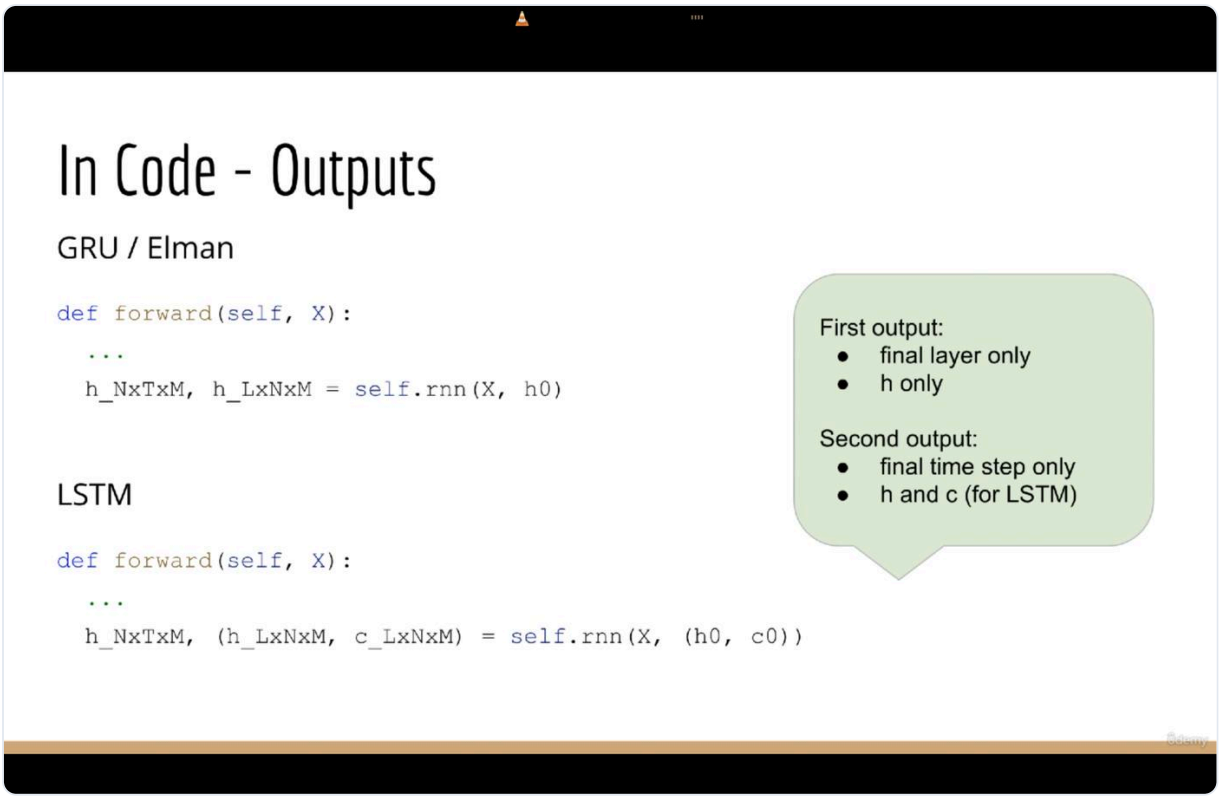


FIGURE 17.6

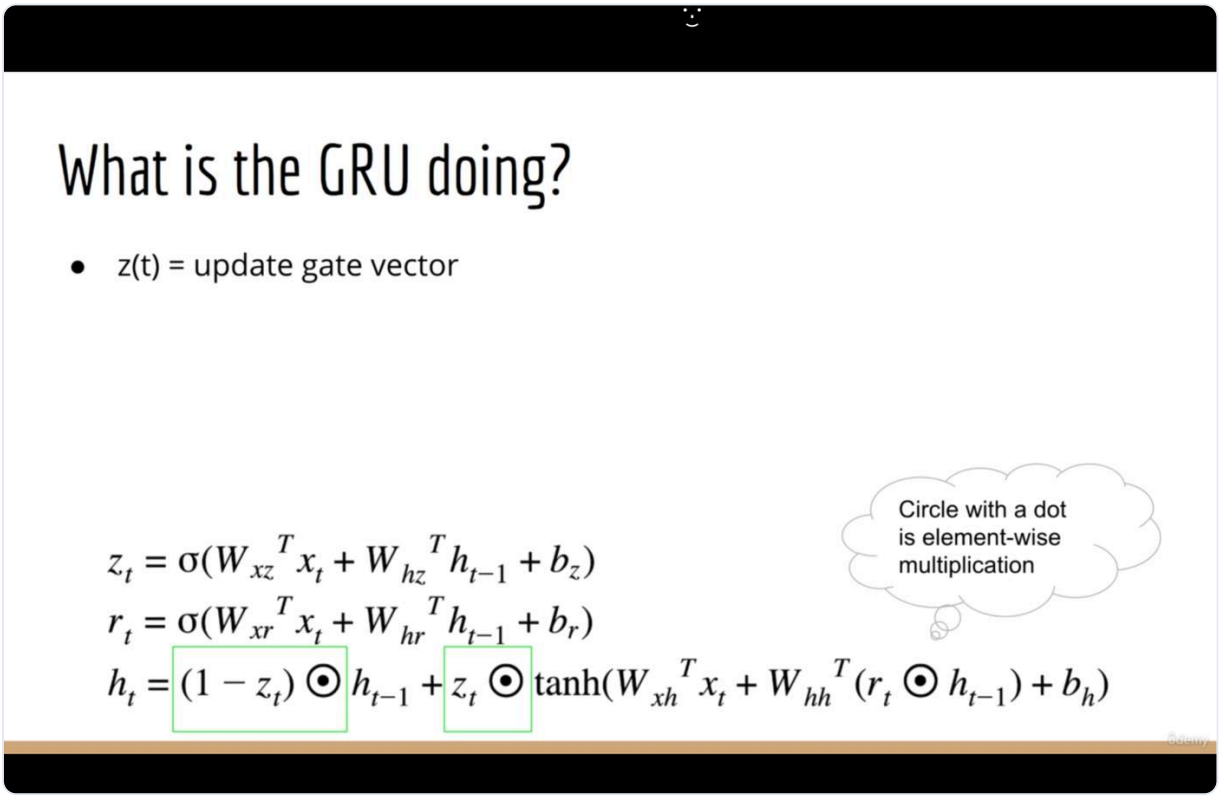


FIGURE 17.7 | What Is the GRU Doing?

First we learn that the simple RNN unit by itself is problematic because it can't remember long term dependencies.

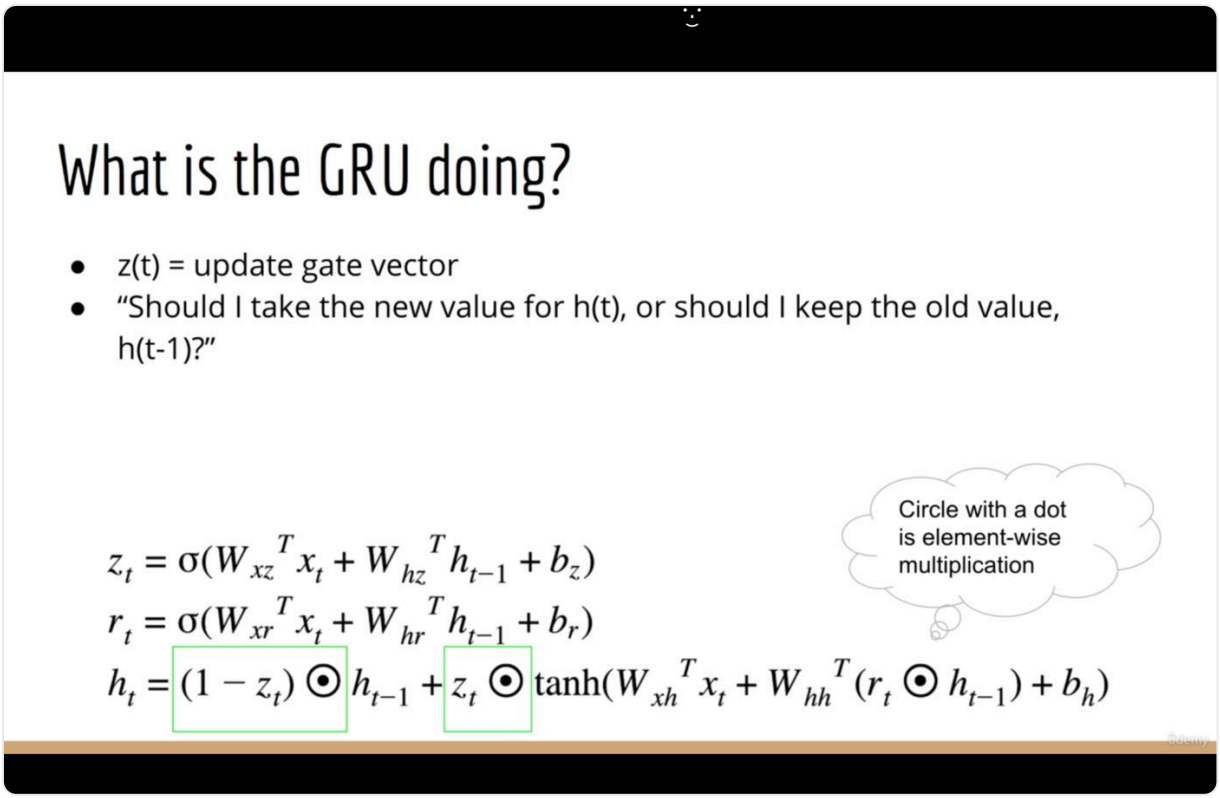


FIGURE 17.8 | What Is the GRU Doing?

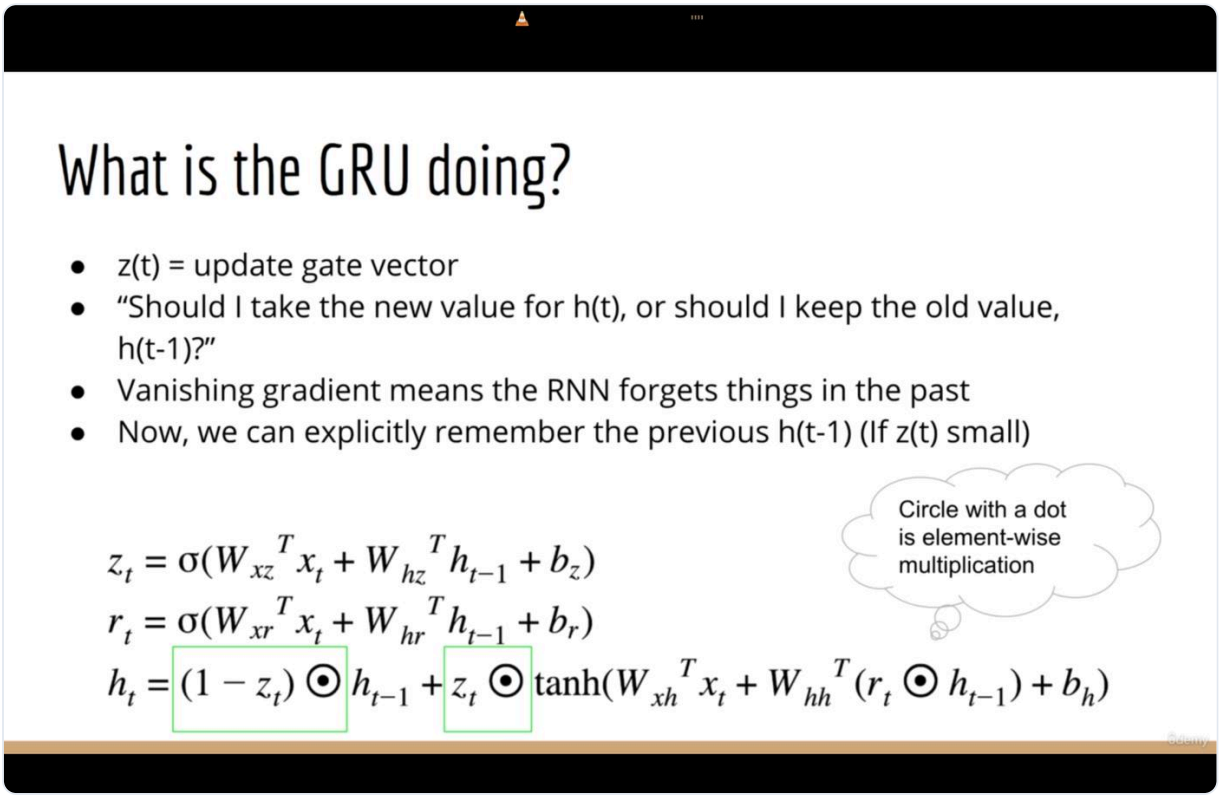


FIGURE 17.9 | What Is the GRU Doing?

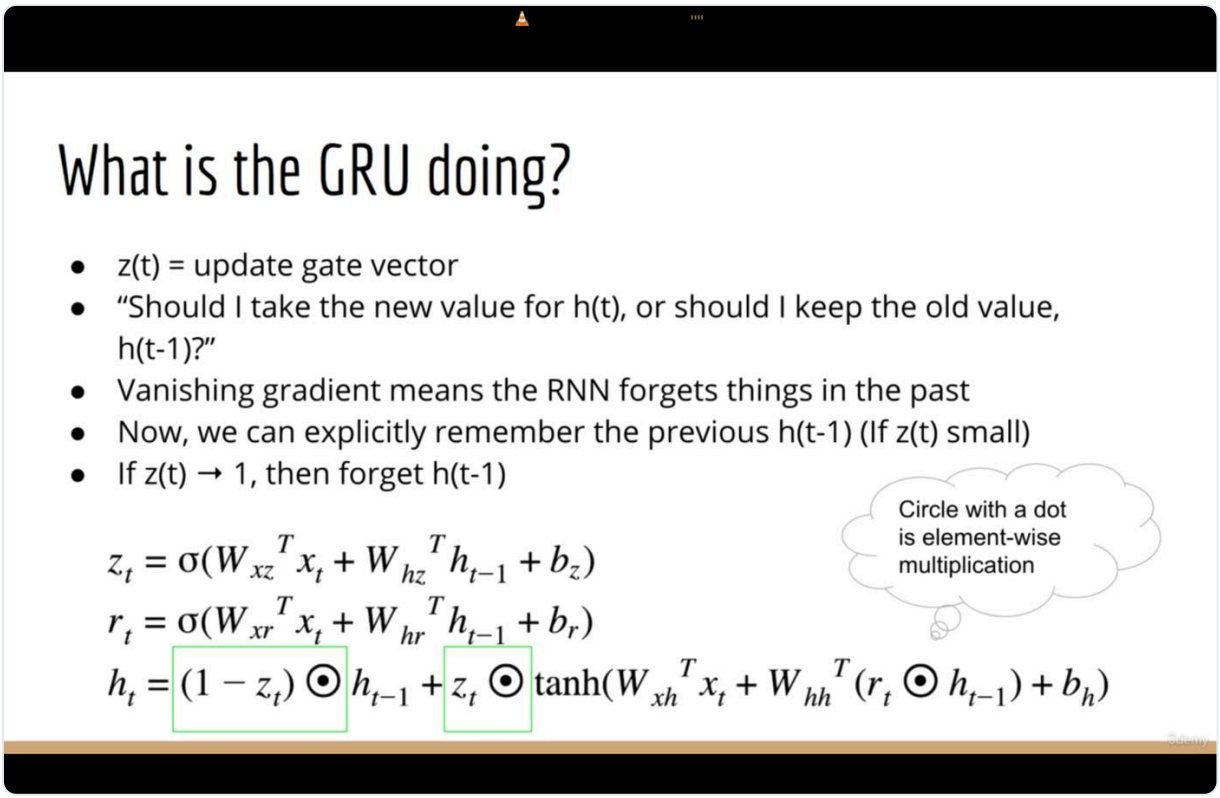


FIGURE 17.10 | What Is the GRU Doing?

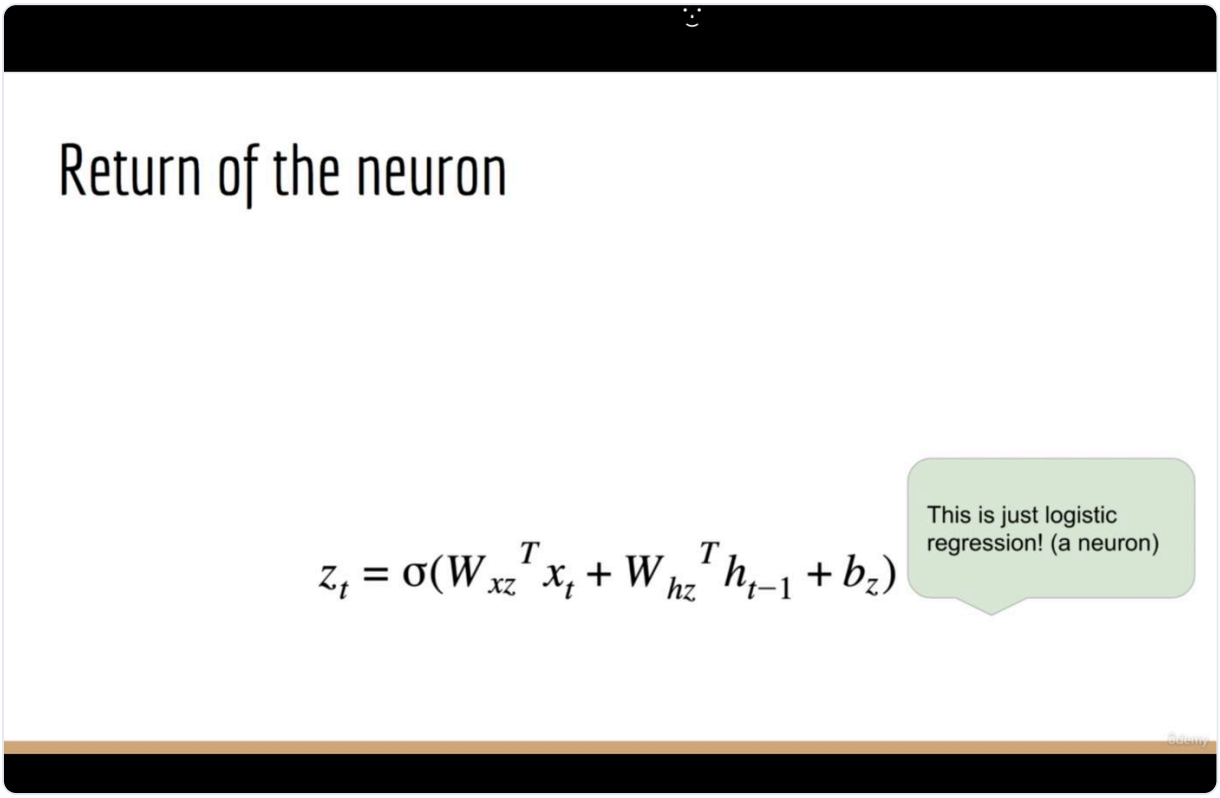


FIGURE 17.11 | Return of the Neuron

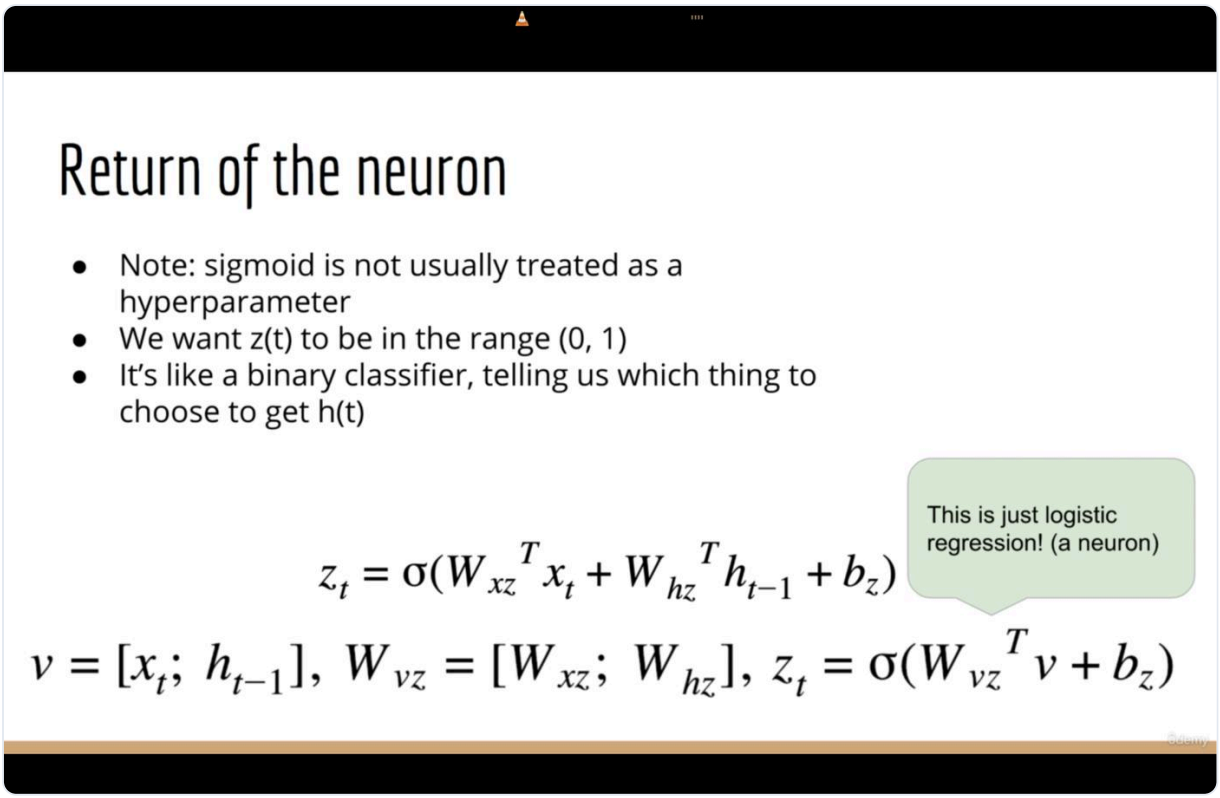


FIGURE 17.12 | Return of the Neuron

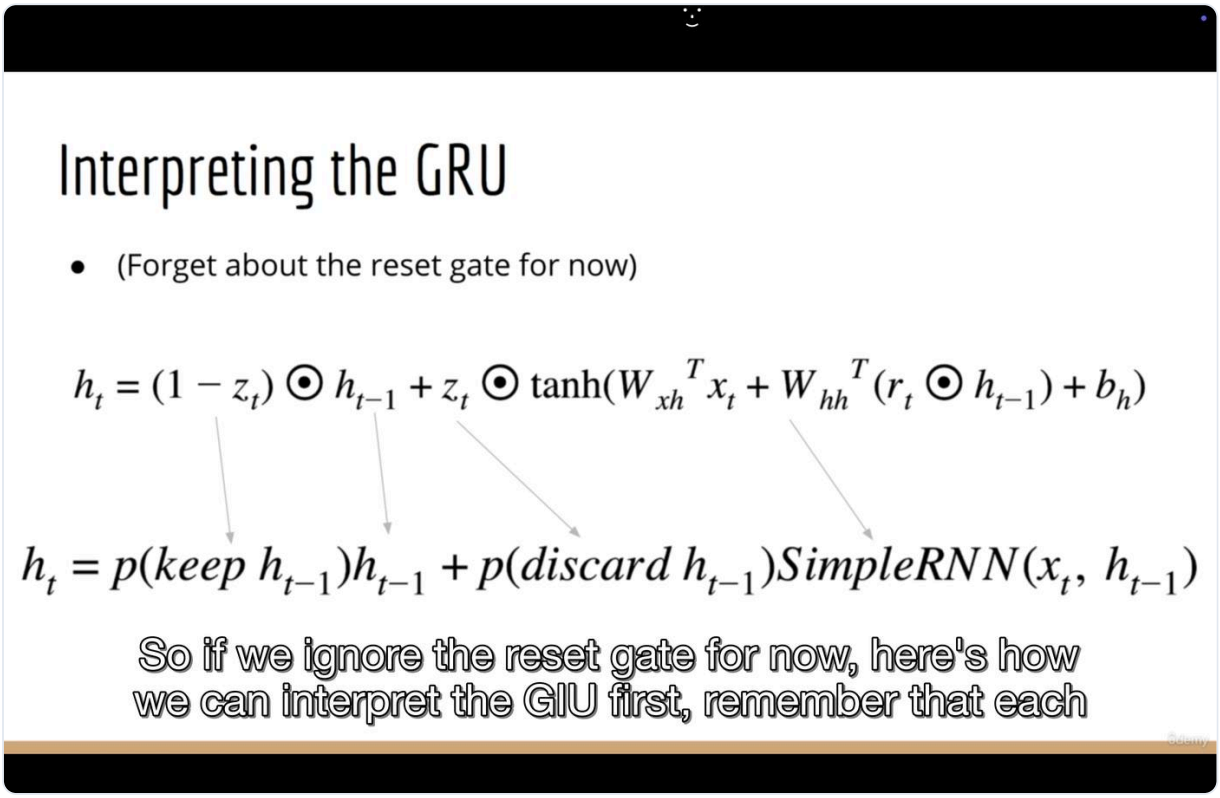


FIGURE 17.13

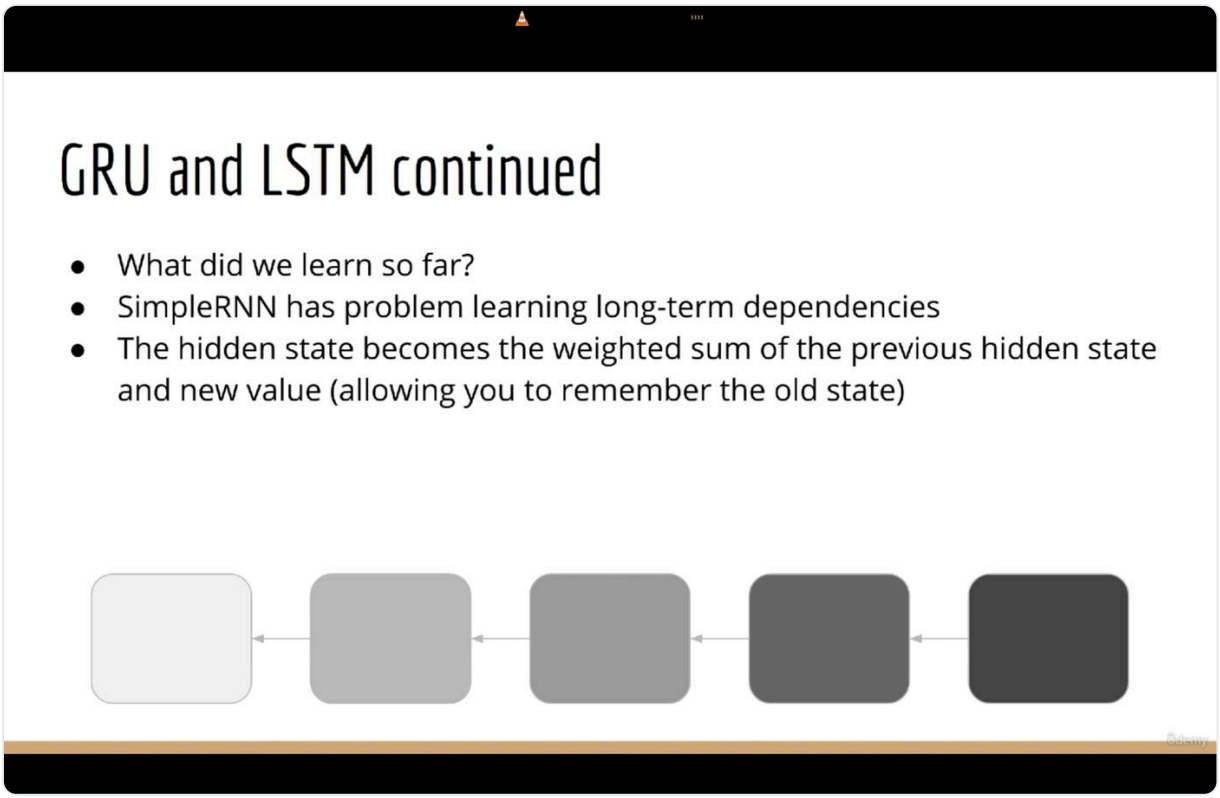


FIGURE 17.14

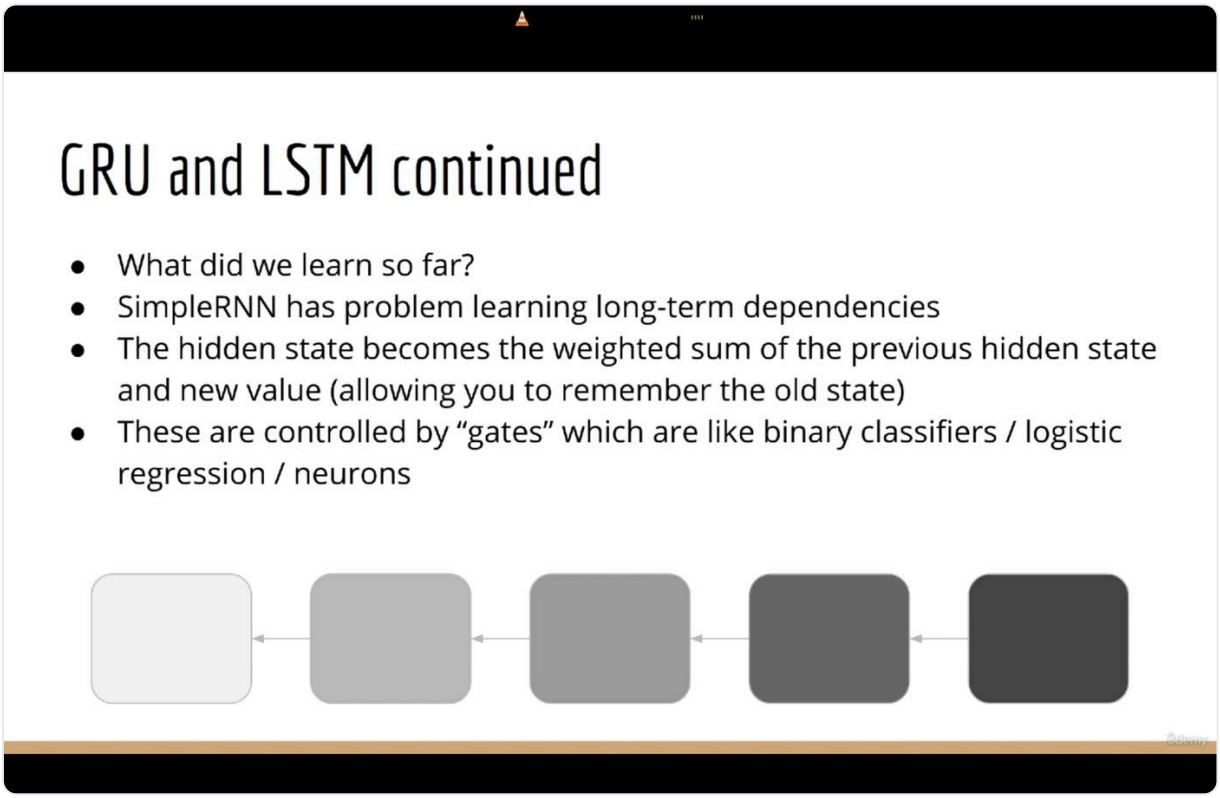


FIGURE 17.15

The way we solve this is we make the hidden state the weighted sum of the previous hidden state and the new value for the state, which is essentially the same calculation as the simple RNN unit in this way the GRU always has the opportunity to remember the previous hidden state allowing it to carry that state forward in time, reducing the chance of forgetting it later. Importantly, we recognize that these

gates, which decide whether we should remember or forget, are just binary classifiers. There are little logistic regression neurons, something we are very comfortable with. Now we are going to move on to the LSTM, which is a little more complicated, but follows the same idea.

Interestingly, when GRUs first came out, the initial research suggested that there was no clear winner between the two. It was reported that the GRU performed comparably with the LSTM. This made the GRU a great choice because it has less parameters and therefore is more performant than the LSTM. These days, the current belief is that this is not true and that the LSTM actually ended up being better. In fact, at the time I am making this course, this latest research only came out about a year ago, which is actually two years after I made my first RNN course. So that just goes to show how fast things move in this field. Experimental results are always changing our thinking about best practices and which techniques to choose over others.

This also is a great example of my rule machine learning is experimentation and not philosophy. I know for a lot of beginners, the approach they take is a philosophical approach. They would try to use logical reasoning to deduce which is better. The LSTM or the GRU. The LSTM has these merits, the GRU has these merits and so on. Such things are useless. Please don't waste your time on philosophy instead follow the example that these researchers have set and use experiments to decide which is best. There are two papers I've attached an extra reading that text that you'll want to check out.

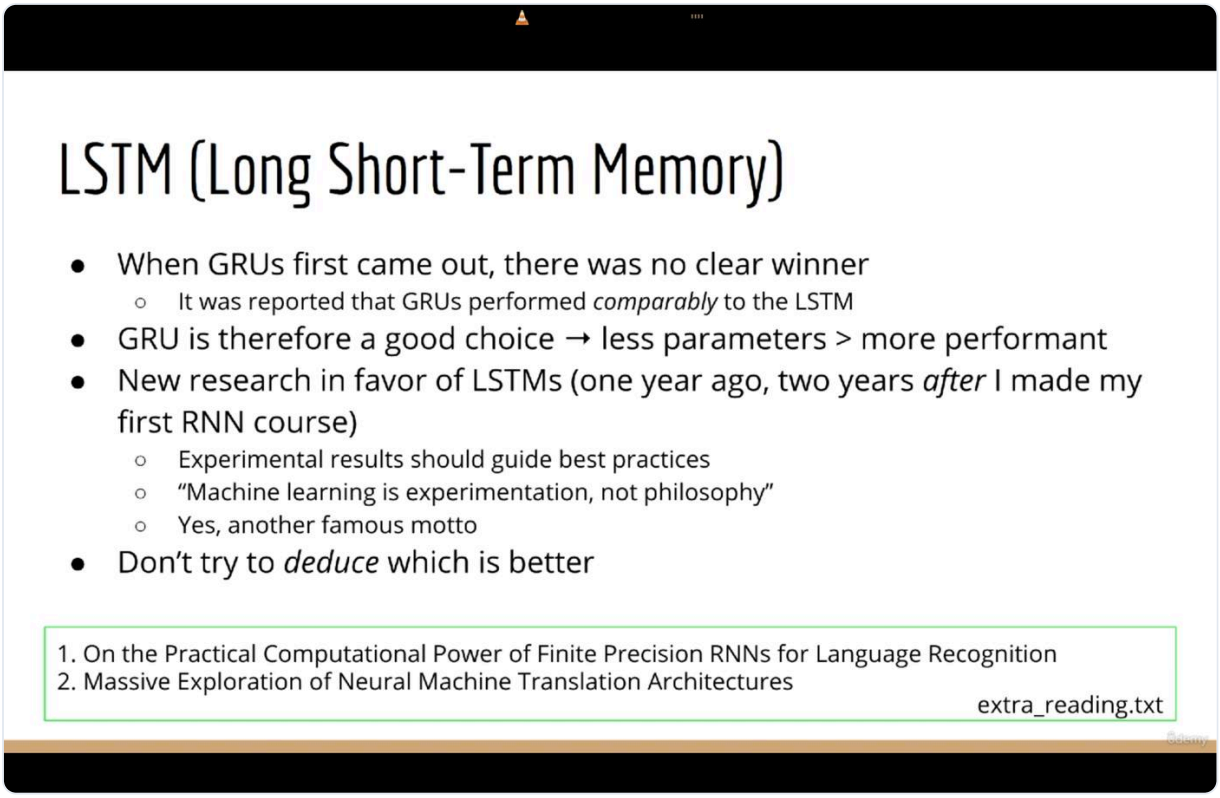


FIGURE 17.16

The first is called on the Practical Computational Power of Finite Precision RNNs for language recognition. And the second is called massive exploration of neural machine translation architectures. They both conclude that the LSTM outperforms GRUs. And so given the choice, I would probably choose the LSTM by default. So how does the LSTM work?

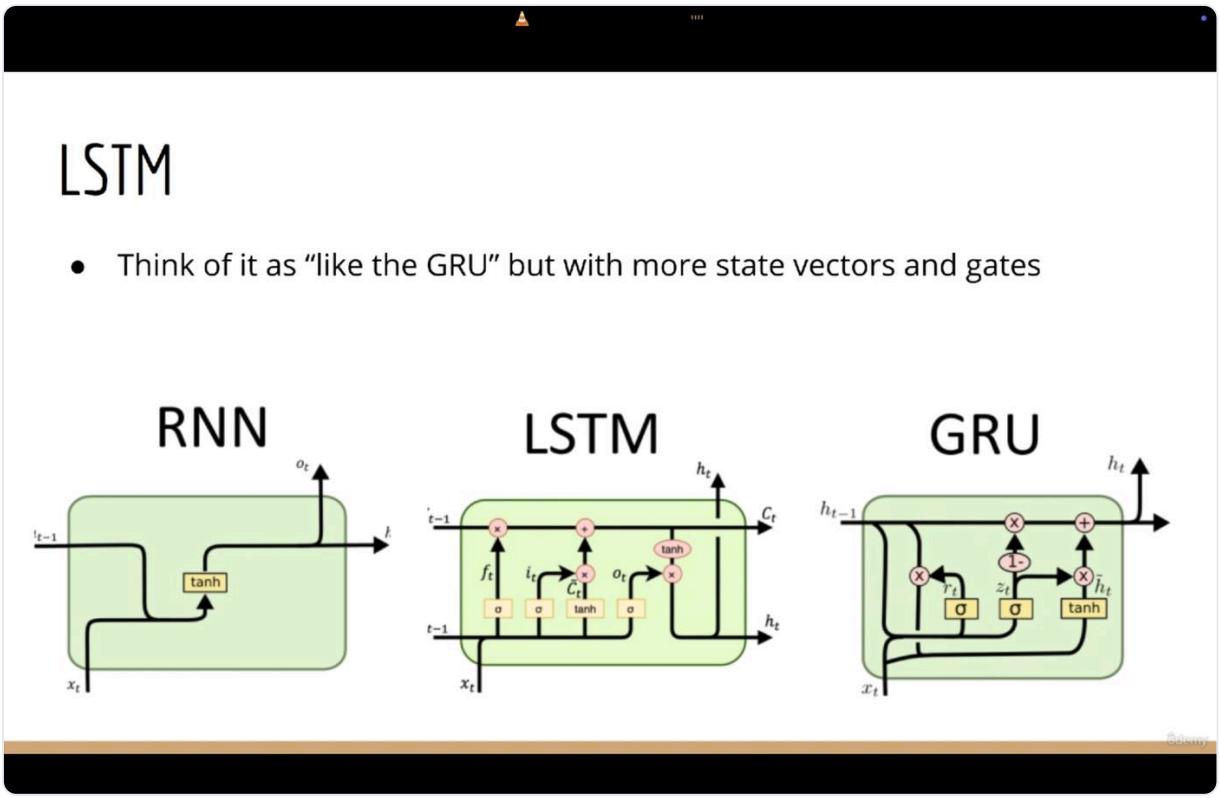


FIGURE 17.17

Basically, you can think of it as like the GRU but with more state vectors and more gates, again, it's my preference to understand the LSTM through the equations. But if you find these visualizations helpful, then you can feel free to look at that as well. The first difference with the GRU and the LSTM is that the LSTM has two states in addition to h of T, which we call the hidden state. We also have c of T, the cell state. Sometimes the cell state is considered an additional head and state. So you actually might pass both of these on to the next layer. For us, we will usually ignore the cell state so you can think of it more like an intermediate value that you calculate, just like the gate vectors.

So you calculate it, but you don't actually use it to pass on to the next layer. So even though the LSTM has an extra step, you can still think of it as having almost the same API as a simple RNN and GRU.

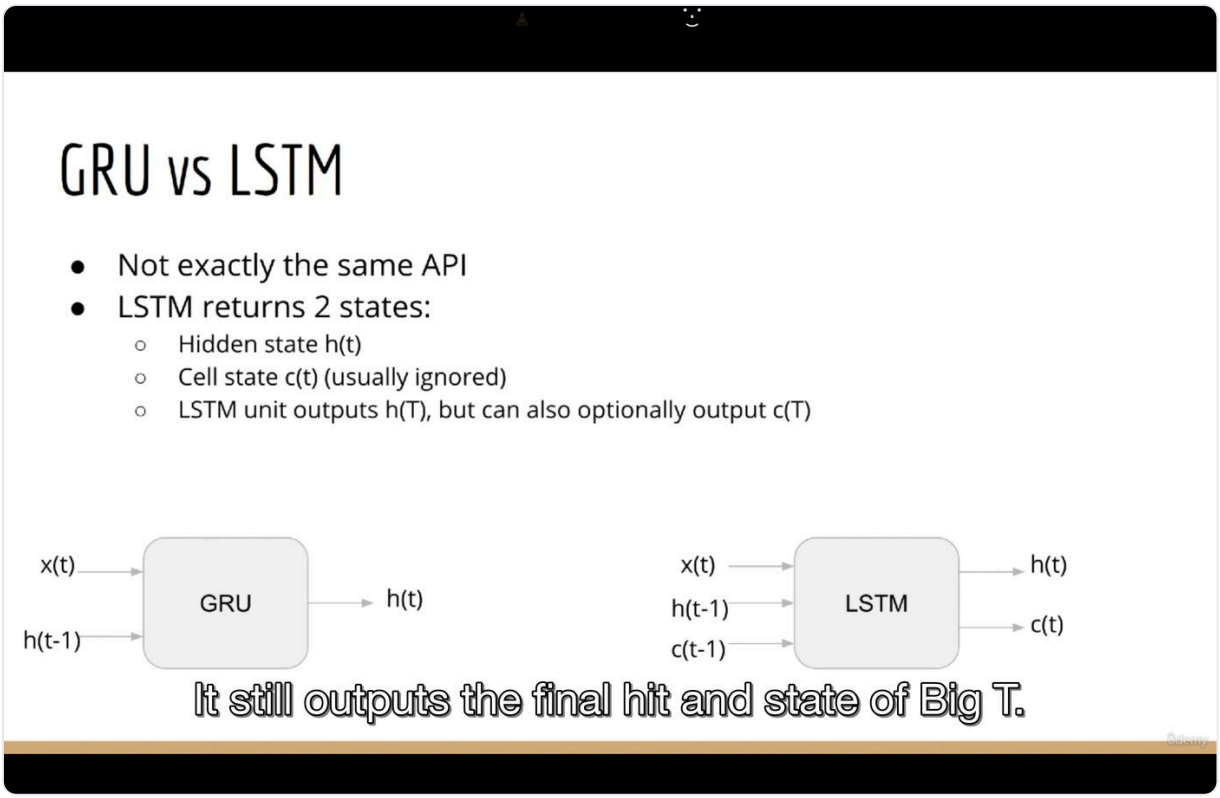


FIGURE 17.18

It still outputs the final hidden state h of Big T. It's just that for the inputs, in addition to x of one all the way up to x of Big T and the initial hidden state h of zero.

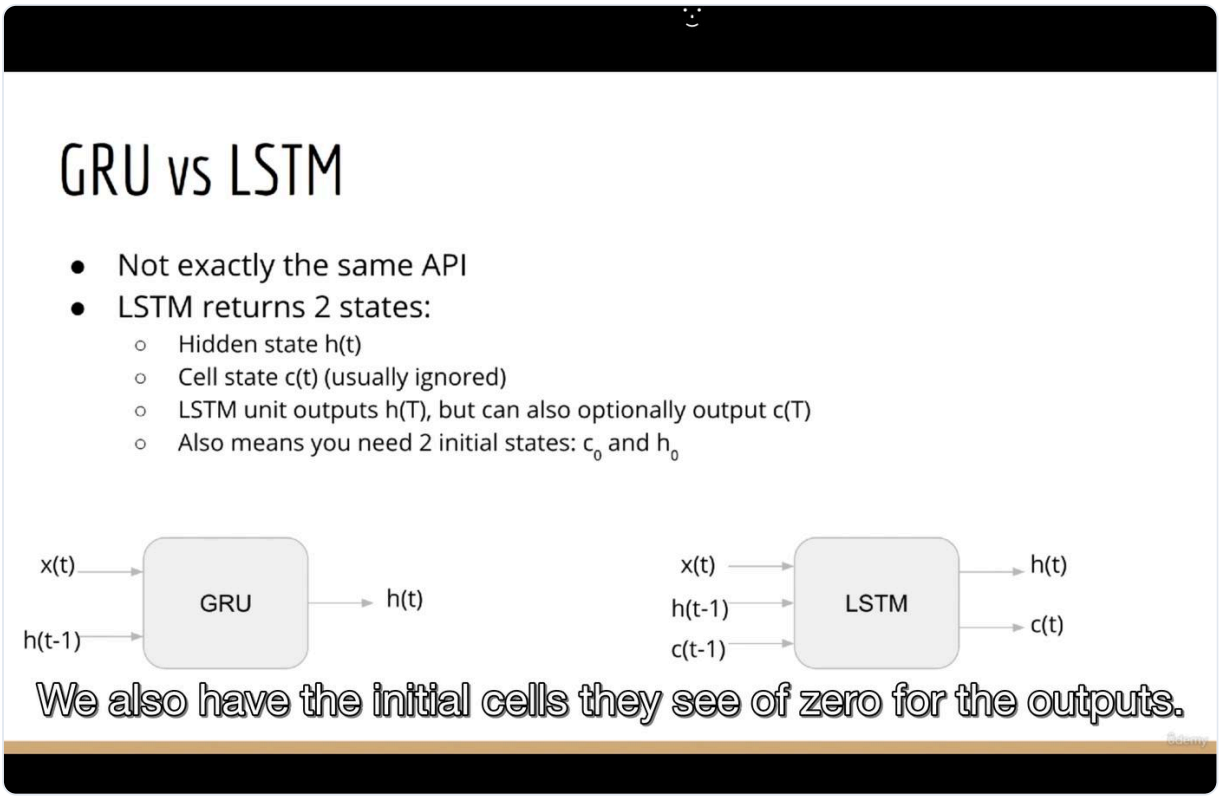


FIGURE 17.19

We also have the initial cell state c of zero for the outputs. It is a little strange due to how PyTorch works, basically will get H of big T and C of Big T, but the details might be surprising, but we'll discuss that more later in this lecture. OK, so let us get to the LSTM calculations. I want to remind you that while this might seem complicated, it's not don't be scared by the sight of equations, but instead try to see the forest for the trees.

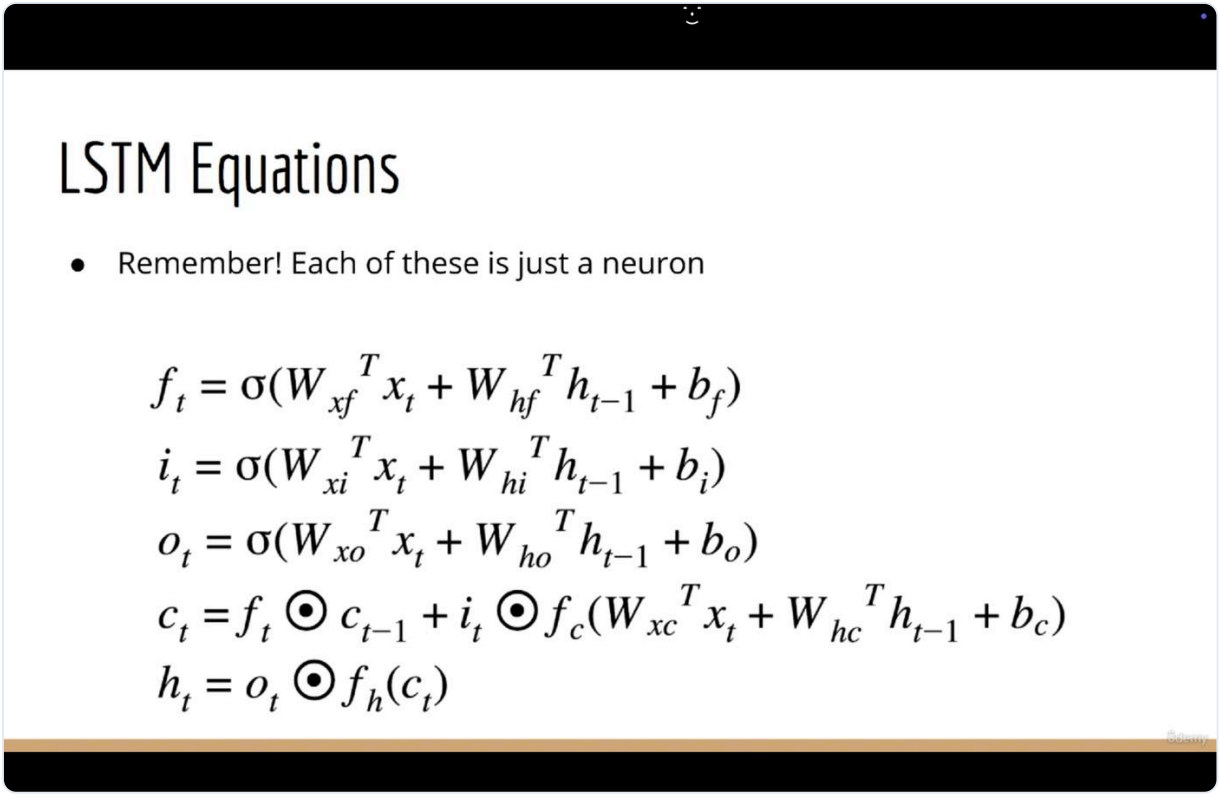


FIGURE 17.20

Each of these is just a neuron. Once you start to realize that each thing here is a binary classifier or logistic regression, it becomes a lot simpler to interpret. So what do we see? First, we see that there are more of these gates. Specifically, we have f of T the forget gate vector. Next we have i of T, the input gate vector, also known as the update gate vector, analogously to the GRU. And next we have o of T the output gate vector. So hopefully this is the easy to remember because the word forget starts with f the word input starts with I and the word output starts with, oh, next we have the cell state c of T. You might notice that this actually takes on the role of h of T from the GRU.

Specifically, we have the weighted sum of two terms. The first term is just the previous cell state c of T minus one. How much of this we end up keeping is controlled by the forget gate f of T. So control is how much of C of T minus one we want to forget. The second term is the simple RNN. How much of this we end up keeping is controlled by the input gate. I have t and notice that this has exactly the same equation as the simple RNN. Unlike the GRU. Now note that for the simple RNN. We have the activation function F of c this can be anything but usually by default is the tanh. Finally we have the hidden state h of T, which is just a simple transformation on the cell state c of T in particular, we apply another activation function F of H, which also can be anything but by default, it is also usually a tanh.

We then multiply this by the output gate o of T. So the output gate controls which values of the cell state we actually pass through to the hidden state h of T. Note that in PyTorch, it is not possible to choose F of c and F of h individually and in fact, you can't even change them from the default, as mentioned before, because RNNs are so complicated, they're usually programmed all in one module for the sake of effi-

ciency, and they're not really composable like the other modules we've learned about. Of course, if you were writing your own LSTM, you could customize this any way you wanted, but its performance would most likely be suboptimal. So you can see that by building up our knowledge first through simple RNNs, then GRUs, then LSTMs, we can make something complex like the LSTM seem very simple.

It's nothing but three logistic regressions or neurons to give us the forget gate, the input gate and the output gate. Then the cell state is the weighted sum of the previous cell state and the simple RNN weighted by the forget gate and the input gate. And remember, we want to offer the cell state the possibility to remember its old state so that we can learn long term dependencies. Finally, the hidden state h of T is just a squashed version of the cell state with the output gate controlling which values are allowed to pass through. By the way, I say squashed because we use the tanh activation by default and it's not possible to change. So this is really not a hyperparameter you have to play around with.

In terms of code, how do the GRU and LSTM work as promised? These have the same or similar API as the simple RNN. So if all you wanted to do is plug in a GRU or LSTM in place of a simple RNN, it can be almost as simple as changing the name. Let's start with how to instantiate a GRU and LSTM here. As you can see, it is almost exactly the same as a simple RNN. We pass on a number of input features d the number of hidden features M and the number of RNN layers to stack L. I also pass in batch first equal to true so that all my data is always N by something instead of having the N dimension somewhere else. The crucial difference is that in a GRU and LSTM, there is no nonlinearity argument because it's not possible to change.

Let us now talk about how the GRU and LSTM will be used in the forward function. First, let us discuss the initial hidden state and what to pass into the RNN. Conveniently, the GRU has the exact same interface as the simple RNN as the input we pass in the data into the first argument and the initial state h naught into the second argument. As before, the initial state h naught is of shape L by N by M number of layers, by number of samples, by a number of features for the LSTM, we also require an initial cell state, luckily because the cell state has the same dimensionality as the hidden state. We will also initialize this to L by N by M.. Now, let's talk about how to interpret the outputs from the GRU and LSTM, as you recall, RNNs in PyTorch return two things, not just one with the Elman unit we get one h, which is of shape N by T by M, which tells us the hidden state at the final RNN layer for each sample, each time step, each feature.

The second h is of shape l by n by m which tells us the hidden state at the final time step for each sample, each RNN layer, each feature. In the GRU it works exactly the same. In the LSTM, we have something similar, but not the same. The first thing that has returned is the same as what we get with the Elman unit, the hidden value at each sample, each time step each feature. The second thing is different. We get the hidden state h and the cell state c, but only at the final time step. So this second h will be of shape L by N by M, which tells us the hidden state at each sample, each layer, each feature, we also get the cell state of the same shape, which means it is the cell state at each sample, each layer and each feature.

Unfortunately, I'm unaware of how you would get the cell states at different time steps.

LECTURE 56

Simple RNN and Linear Model

See the notebook `12.PyTorch-Nonlinear-Sequence-linear.ipynb`, `12.PyTorch-Nonlinear-Sequence-Simple-RNN.ipynb` and `13.PyTorch-Nonlinear-Sequence-LSTM.ipynb` for the code.

In this lecture we are going to do another little demo on a time series. But this time with a much more complicated signal it's still going to be a synthetic signal meaning we can understand its behavior because we know the mathematical formula that represents it. But as you'll see just one small change will make this task much harder for both the autoregressive linear model and the RNN. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at.

OK. So after the imports we create the time series. As you can see it looks very similar to what we had previously but with one crucial difference this is that we are squaring the input argument into the sine function because we square the input argument. This means that the frequency and hence also the period of the wave changes over time. So let's plot the time series and we see exactly that in some parts the wave goes up and down very fast. And in other parts it slows down in the next block. We build our dataset now because of my rule. All data is the same. Nothing about this changes so we don't need to explain this again. Next we're going to build an autoregressive linear model.

You've all seen this before so I'm not going to explain it again. Here is our loss and optimizer here is our train test split here is a training function and here is running the training function so let's see how this performs so it looks like the loss is pretty high about zero point five zero point six. This is much higher than what we had before with the normal sine wave. Next we're going to plot the loss per iteration All right. So this does not look very good. In fact it looks like the test loss does not decrease at all. The train loss appears to decrease which is a sign that all the model is doing is over fitting without actually matching the underlying pattern so for our predictions.

Let's start by doing a one step forecast. As a side note you'll see that I've taken a little shortcut here. If you want to do is a one step forecast we've already built our input data for that. All we need to do is call the model and pass in X test remember that the first half of X is our train set and the second half of X is our test set right. So let's plot the results. So as you can see it does not do very well. It doesn't even get the values in the right range. Knowing this we can deduce that the multi-step forecast is going to be even worse again since we've already gone through this code. I'm not going to explain it again. So let's run this and see if we get as you can see our suspicions are correct.

The linear model does a terrible forecast so we're going to switch notebooks now. Now we're looking at PyTorch non-linear sequence simple art in so in this new notebook we are going to try an RNN model. We're going to start with a simple RNN and then move on to an LSTM later the only different thing we need to do here is reshape our data so that it is three dimensional N by T by D then we create our RNN then fit it and look at the results so we can set the device define our simple RNN you have seen this before instantiate the model create the loss and optimizer make the inputs and targets do our training move to the GPU do full gradient descent.

All right so you can see that a loss is now better than the linear model. Now this is a good lesson for the simple time series like a sine wave with no noise a linear model is perfect and an RNN does worse at least with default parameters it has too much flexibility. But now we have a signal that is much more difficult a signal that a linear model can't solve at all. In this case the RNN does better because it actually has the flexibility to match the signal next. Let's plot the loss per iteration. All right so that seems reasonable. Next we are going to do a one step forecast so as you can see this does a much better job. It looks like it can match the frequency in most places even when the signal slows down significantly in the second half.

This is despite the fact that the model never saw such a low frequency in the training set the values are also approximately in the right range. Now let's go to the multi-step forecast. So as you can see this doesn't look that good but at least at the beginning the model appears to capture the frequency quite well. It doesn't know that in the second half the signal slows down. So it kind of makes sense that the model isn't able to predict that. Also it does a much better job than the linear model at getting the right range of values finally. Let's go to our next script where we use an LSTM instead of a simple RNN so I am gonna run the imports make the series plot the series build the data set.

Set the device. Define our RNN so this uses an LSTM instantiate the model create the loss and optimizer make the train set and test set create the training function move data to the GPU train the model right so the loss values are still small. That's good. All right so the loss goes down pretty smoothly and we get a slightly better loss than the simple RNN let us look at the one step forecast all right. So I feel like the one step forecast looks a tiny bit better and if we go to the multi-step forecast I don't think there is any appreciable difference in the multi-step forecast. So essentially this is pretty much the same as the simple RNN. Now why might this be.

Well don't make the mistake of thinking that LSTMs are more popular and because they are more popular it is probably because they are more powerful and therefore LSTMs are better and they can do anything. LSTMs are better than RNNs at finding long term dependencies. But this does not mean that LSTMs are simply better at everything it does not mean that LSTMs are magic now. Although LSTMs are good at finding long term dependencies it is important to realize that this fact does not hold true for arbitrarily long term dependencies. There is a point where even LSTMs forget of course that point is further than the simple RNN but it exists nonetheless. So avoid absolute generalizations first of all.

And second. Note that this dataset doesn't really have any long term dependencies at least for the time span of the dataset as it exists right now. Data from very far in the past isn't really going to help predict future values. So in fact it makes sense that there's no real advantage to using LSTMs for this problem.

LECTURE 57

RNN for Images

In this lecture we are going to discuss an application of RNNs that I think would be nice because most other resources don't cover anything like this. In particular we are going to look at how to apply RNNs to image classification. Now you might think that's pretty weird. I thought CNNs were for images and RNNs were for sequences and indeed that's generally the case.

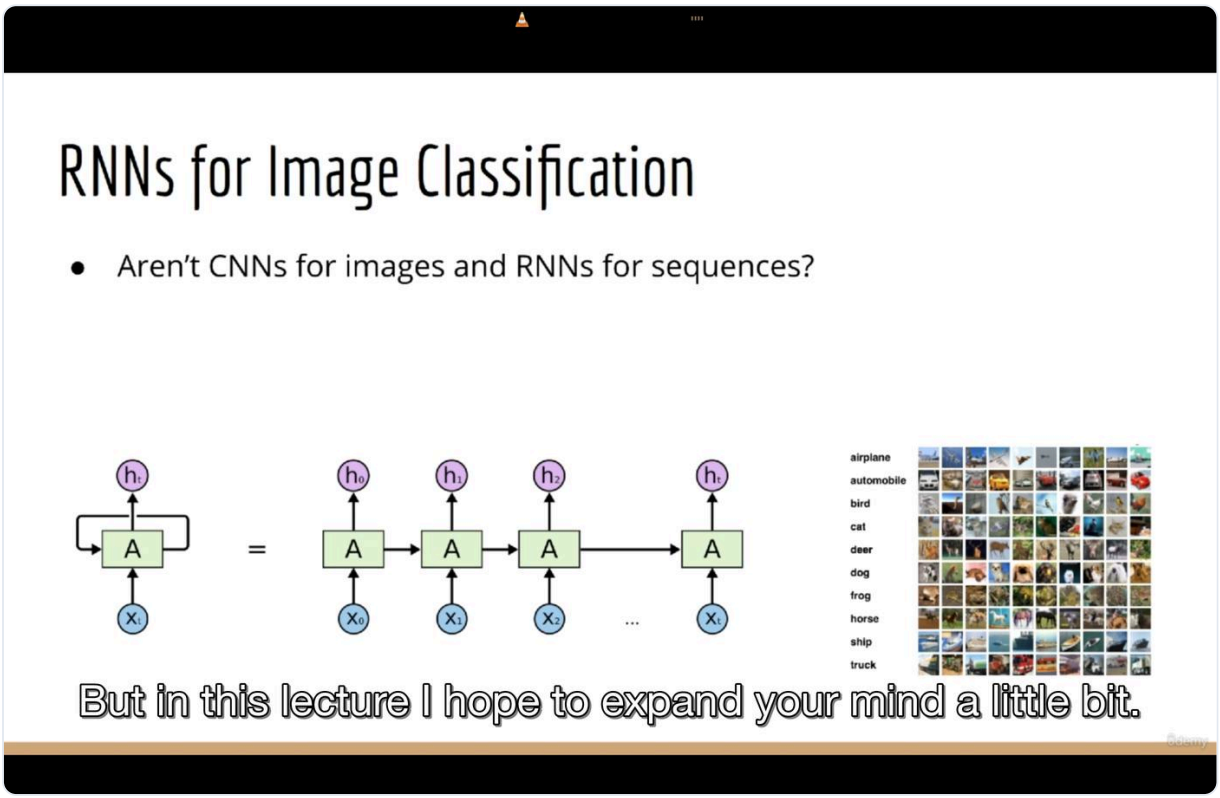


FIGURE 19.1 | RNNs for Image Classification

But in this lecture I hope to expand your mind a little bit. So basically this is all about your imagination. If you recall when we took our dumb as possible approach what did we do. Originally we considered tabular data such as what you might collect in a survey. Each column just represents your answers to my survey questions. How long did you study for my math exam.

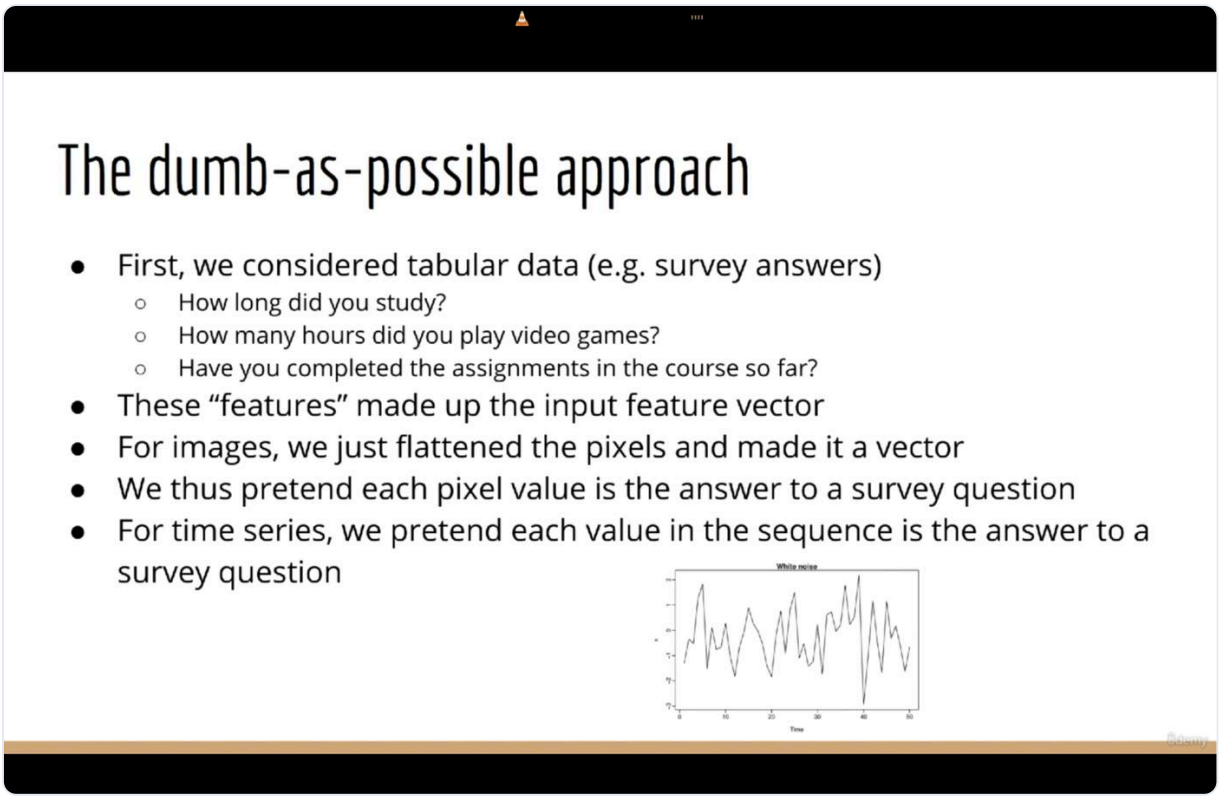


FIGURE 19.2 | How Long Did You Study?

How many hours did you spend playing video games have you completed all the assignments in the course so far and so on. These are called features and they make up the input feature vector. Well the dumbest possible approach for dealing with images was just to flatten the pixels of the image and pretend that it's a feature vector. Pretend that each pixel value is an answer to a survey question. In other words this is all make believe you are just using your imagination. We did the same thing with time series sequences. Instead of thinking of it as a sequence if we just want to pass the data into an ANN we just pretend the sequence is a feature vector and that each value of the sequence is your answer to a survey question.

Again all make believe. Stock prices are not answers to a survey question. It's just imaginary. So how about we think of other kinds of data using this imaginary perspective. Here's what we know. A multi-dimensional time series is a T by D matrix.

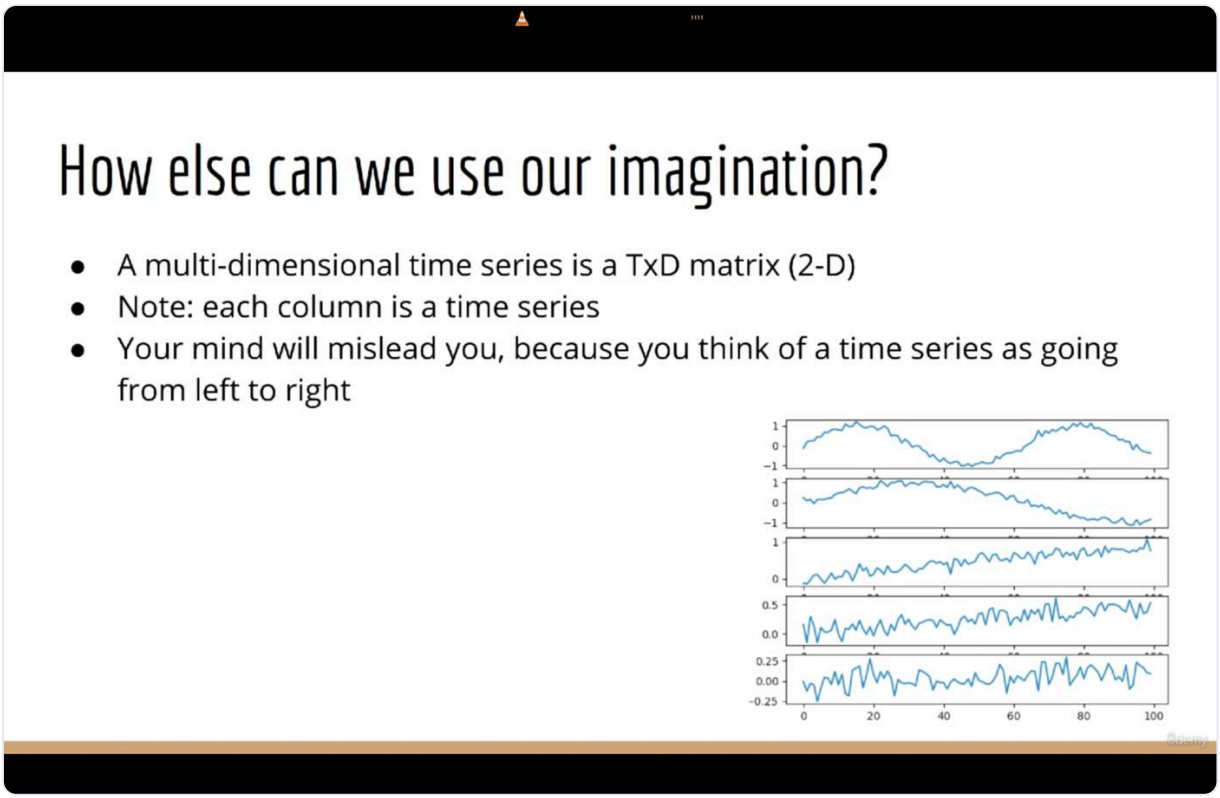


FIGURE 19.3 | How Else Can We Use Our Imagination?

This is a two dimensional matrix where each column is a time series by the way. Note that this image is a little misleading because each time series is a row. It goes from left to right. This is just due to the way we visualize time series. It would look very strange if I gave you a time series that went from top to bottom. So just keep that in mind. A time series stored in a matrix would go along the columns because the number of rows t is the number of time steps now consider a black and white image such as what we have in MNIST and Fashion MNIST. This is a height by width matrix. Importantly it is also a two dimensional matrix.

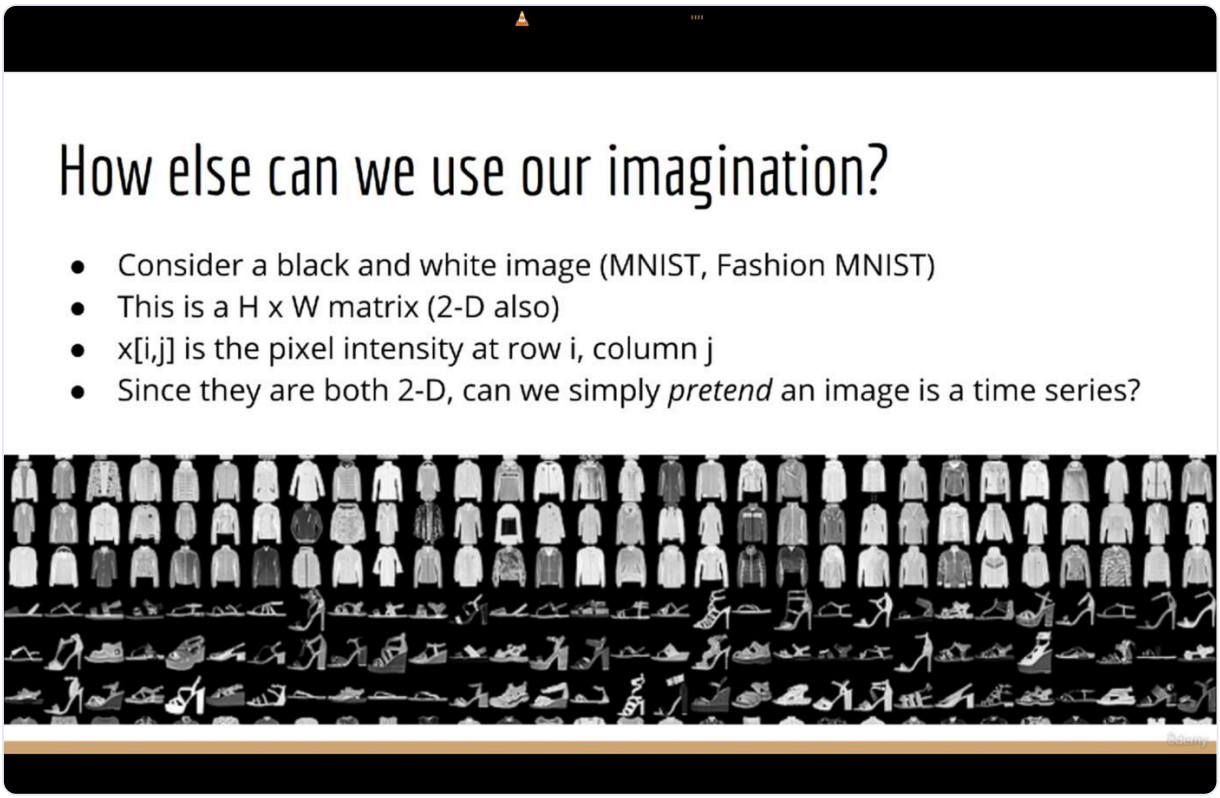


FIGURE 19.4 | How Else Can We Use Our Imagination?

Each i j entry of the matrix is the pixel intensity at row I column J. But here's the trick. Since a multi-dimensional time series is a two dimensional matrix and an image is also a two dimensional matrix.

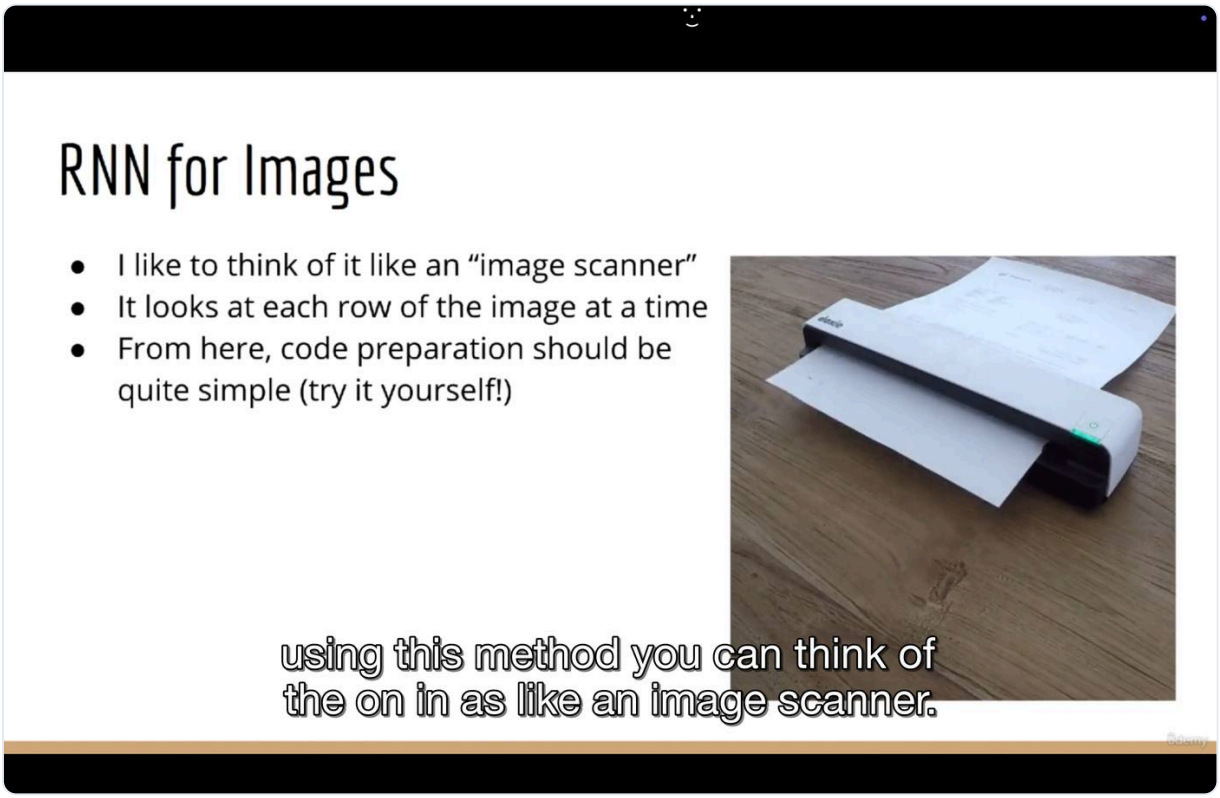


FIGURE 19.5 | RNN for Images

How about we just use our imagination to pretend that an image is a multi-dimensional time series using this method you can think of the RNN as like an image scanner.

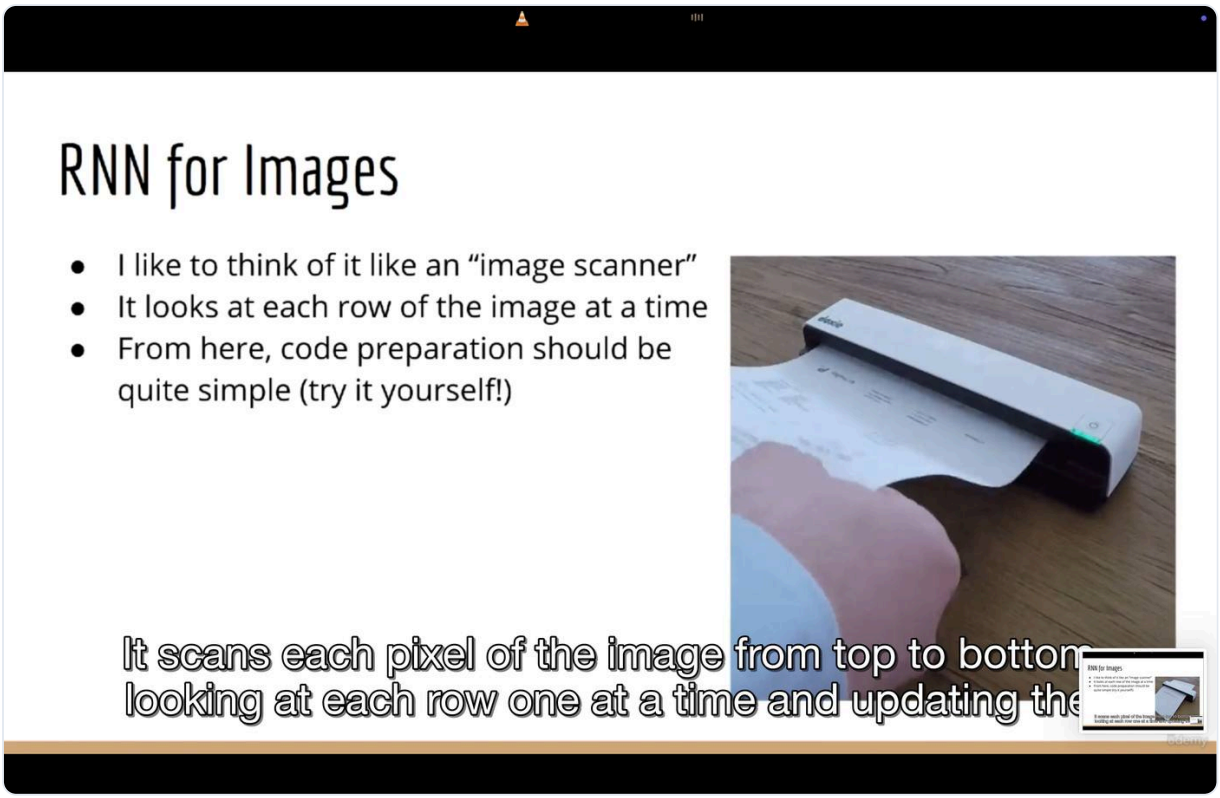


FIGURE 19.6 | RNN for Images

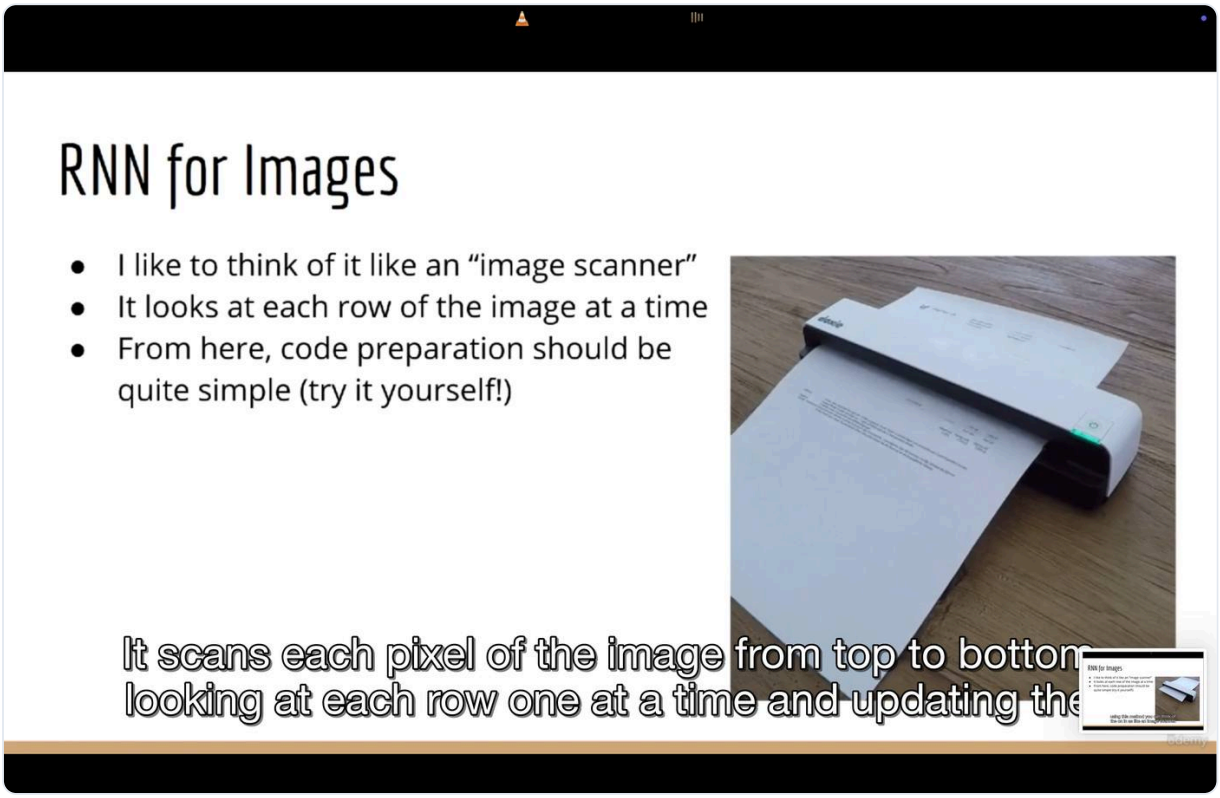


FIGURE 19.7 | RNN for Images

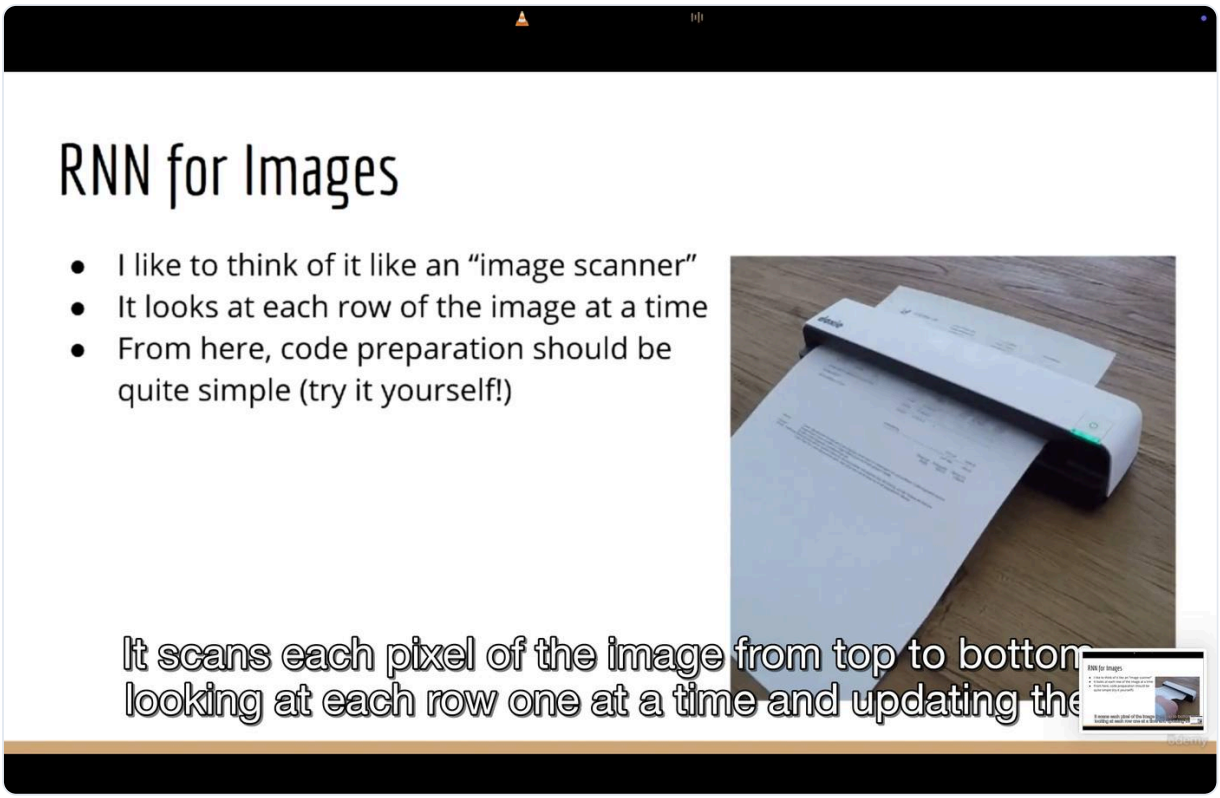


FIGURE 19.8 | RNN for Images

It scans each pixel of the image from top to bottom looking at each row one at a time and updating the hidden state based on each row it encounters from here. Our code preparation doesn't involve anything we haven't seen already step number one is to load in the data. That is just MNIST from this. We get our input data as an array which is of size N by T by D. Here T is 28 and D is also 28. Step number two is to instantiate our model for this we can use an LSTM network and have the final dense layer with 10 outputs and a softmax activation. After this we call fit as usual and plot the loss and accuracy per iteration. So very easy.

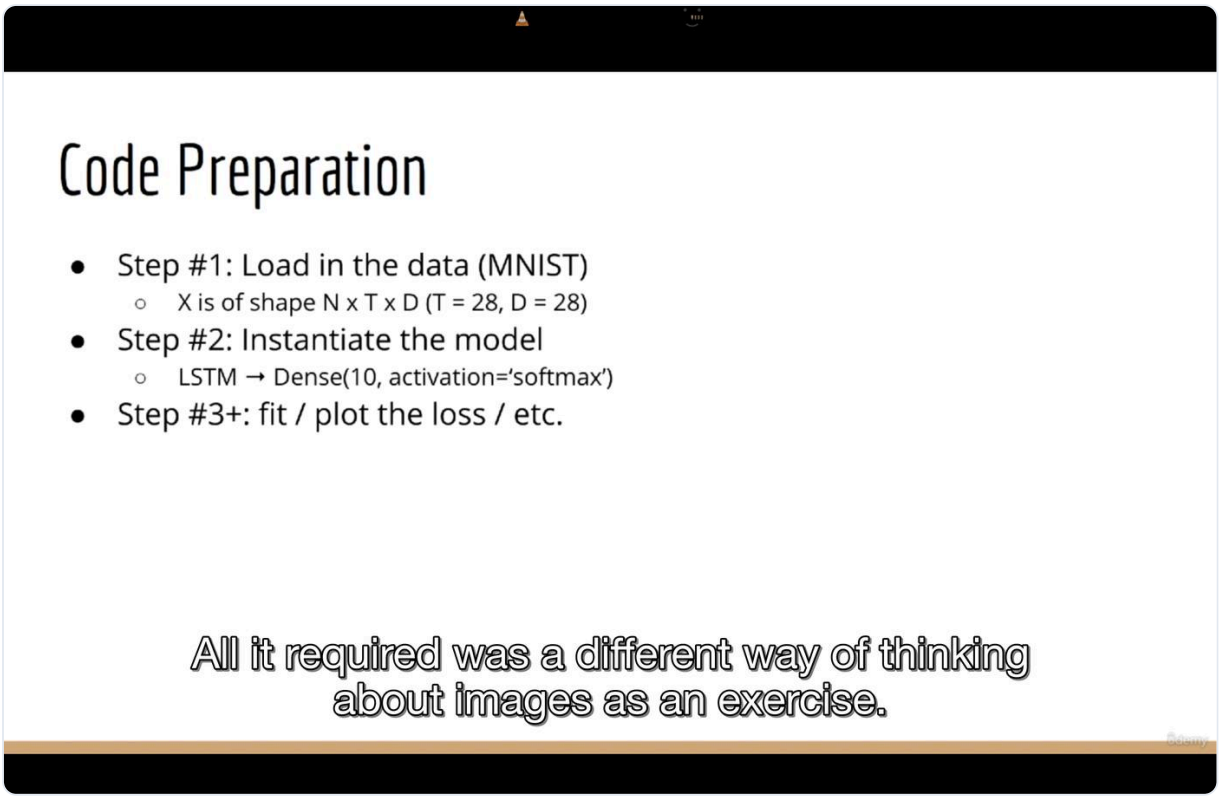


FIGURE 19.9 | Code Preparation

All it required was a different way of thinking about images as an exercise. You might want to try implementing this before looking at my code to practice what you've learned so far. Remember this doesn't require anything new. Just putting together old things in new ways.

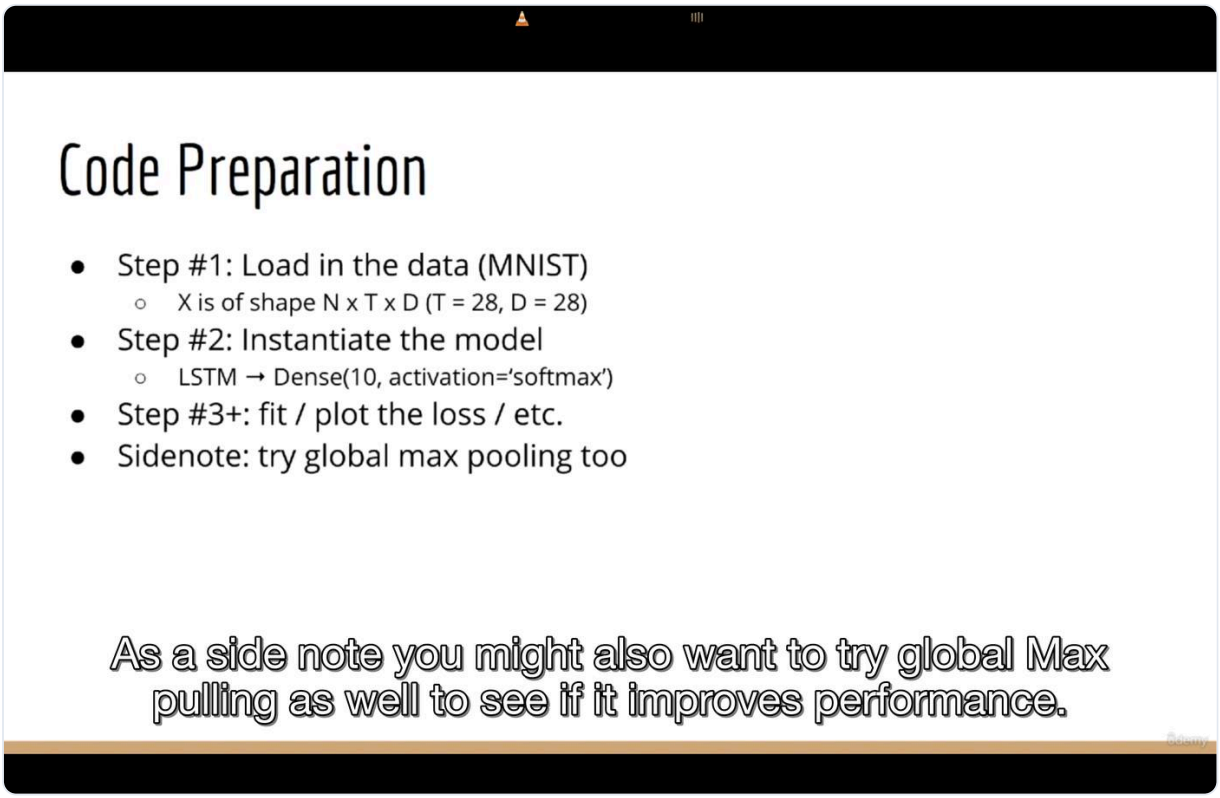


FIGURE 19.10 | Code Preparation

As a side note you might also want to try global max pooling as well to see if it improves performance.

LECTURE 58

Image and Notebook

See the notebook `14.PyTorch-RNN-Image-Classifier.ipynb` for the code.

In this lecture we are going to do image classification using an RNN on the MNIST dataset. This lecture is going to walk you through a prepared Colab notebook although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. At the top we have all our imports. Nothing new for you here. Next we load in the data. Note that this is exactly the same way we loaded in the data previously. So nothing here needs to change. Recall that this data has the shape N by 28 by twenty eight.

Next we build our model. Luckily due to my rule all machine learning interfaces are the same. This is the exact same LSTM model we worked with previously. Next we instantiate the model and we set the device and move the model to the GPU next. We set the loss and optimizer and as you can see I am using the default Adam optimizer here. It seemed to work fine for this architecture and this dataset. Next we train the model all right. So the loss per iteration looks pretty good next we compute the accuracy. All right. So we can see from the results that we get about ninety nine percent accuracy on both the train and test sets after just ten epochs. The neural network appears to generalize well and it doesn't have a problem capturing long distance information in the image.

I would consider this to be long distance because you can't know what digit the images of unless you consider the image as a whole. You can't just look at say the last five rows of the image to determine what the images of most likely the last five rows are just all black pixels anyway since the digit in the image is centered the length of the sequence is 28 which is pretty long in comparison so it is quite surprising that the LSTM is able to learn to classify handwritten digits so efficiently. And there you have it. Yet another lesson in my motto all data is the same.

LECTURE 59

Time Series and Stock Prices

See the notebook `15.PyTorch-Stock-Returns.ipynb` for the code.

Data:
https://raw.githubusercontent.com/lazyprogrammer/machine_learning_examples/master/tf2.0/sbux.csv

In this lecture we are going to do what I think is a lot of students favorite exercise how to predict stock returns. We are going to use the LSTM for this. The most powerful RNN unit we've encountered so far. I hope that as you go through this lecture you will consider everything you've learned so far. Throughout this section this lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at.

First I want to answer the question What is the motivation behind this example. Why are we doing it. Well the answer may surprise you.

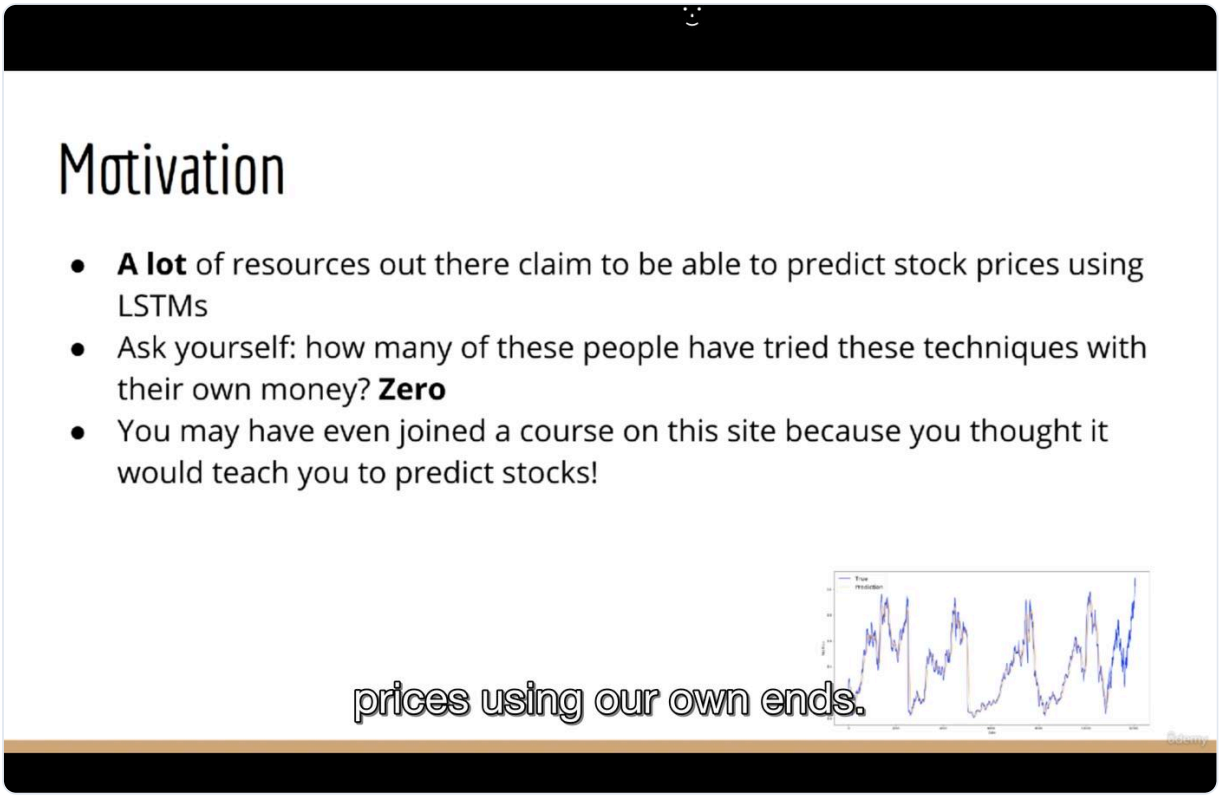


FIGURE 21.1

There are a lot of resources out there that will claim to be able to predict stock prices using LSTMs. Now you must ask yourself how many of these people have actually tried their techniques in the actual stock market using their own money. I can tell you how many 0. Unfortunately the methodology that a lot of these resources take is extremely flawed. You might have even joined the course because you thought it was going to teach you how to predict stock prices using RNNs. I mean it's understandable if you are a beginner all you know is that deep learning is this new field and it's very popular because people have started to realize how powerful it is.

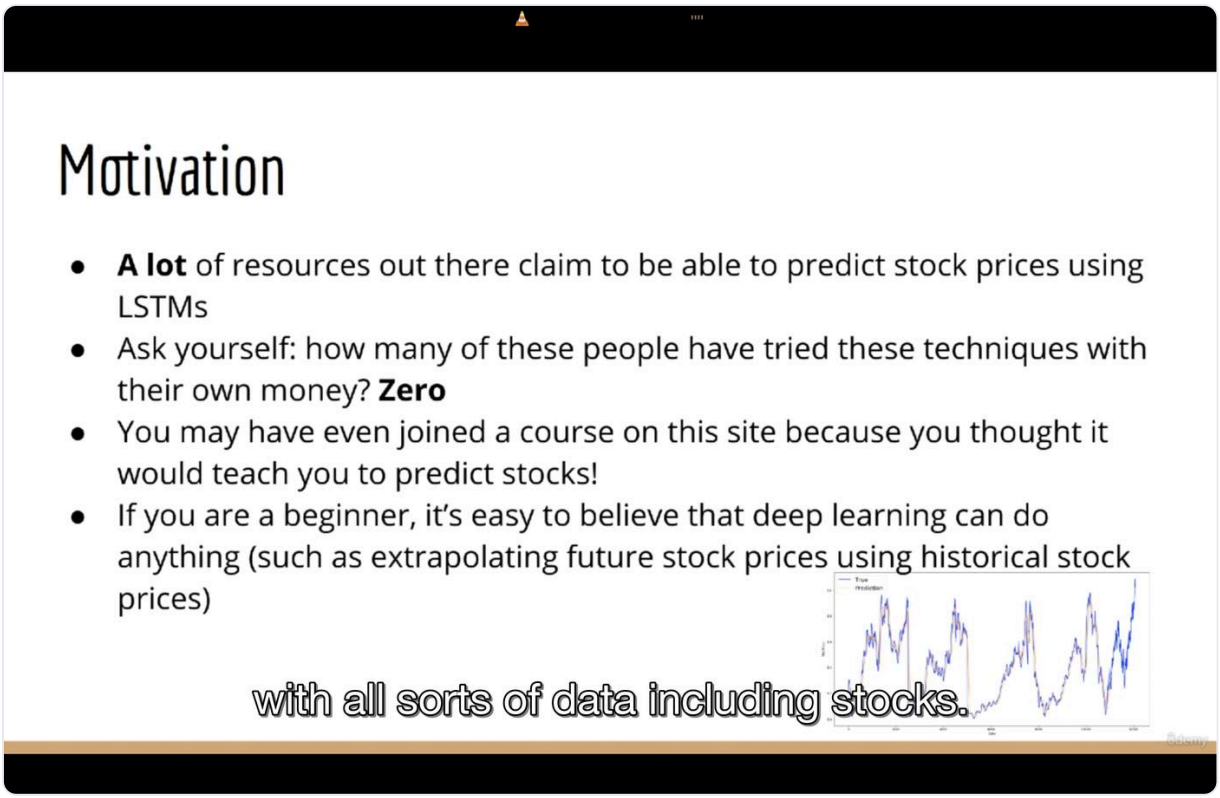


FIGURE 21.2

And so you start to believe that because deep learning is so powerful it must be good at making predictions with all sorts of data including stocks. You see deep learning and you think this will be a game changer for making predictions in the stock market. All you need to do is use this magical LSTM and you will automatically extrapolate future stock prices using historical stock prices.

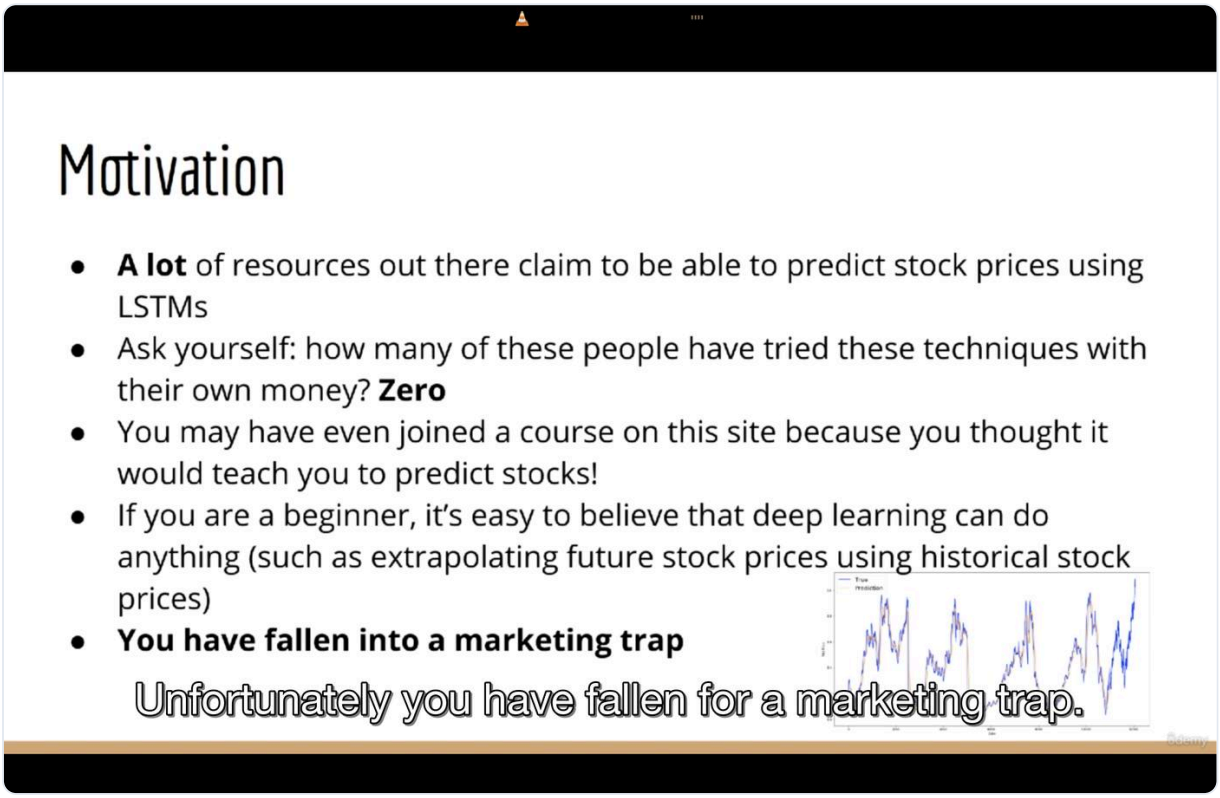


FIGURE 21.3

Unfortunately you have fallen for a marketing trap.

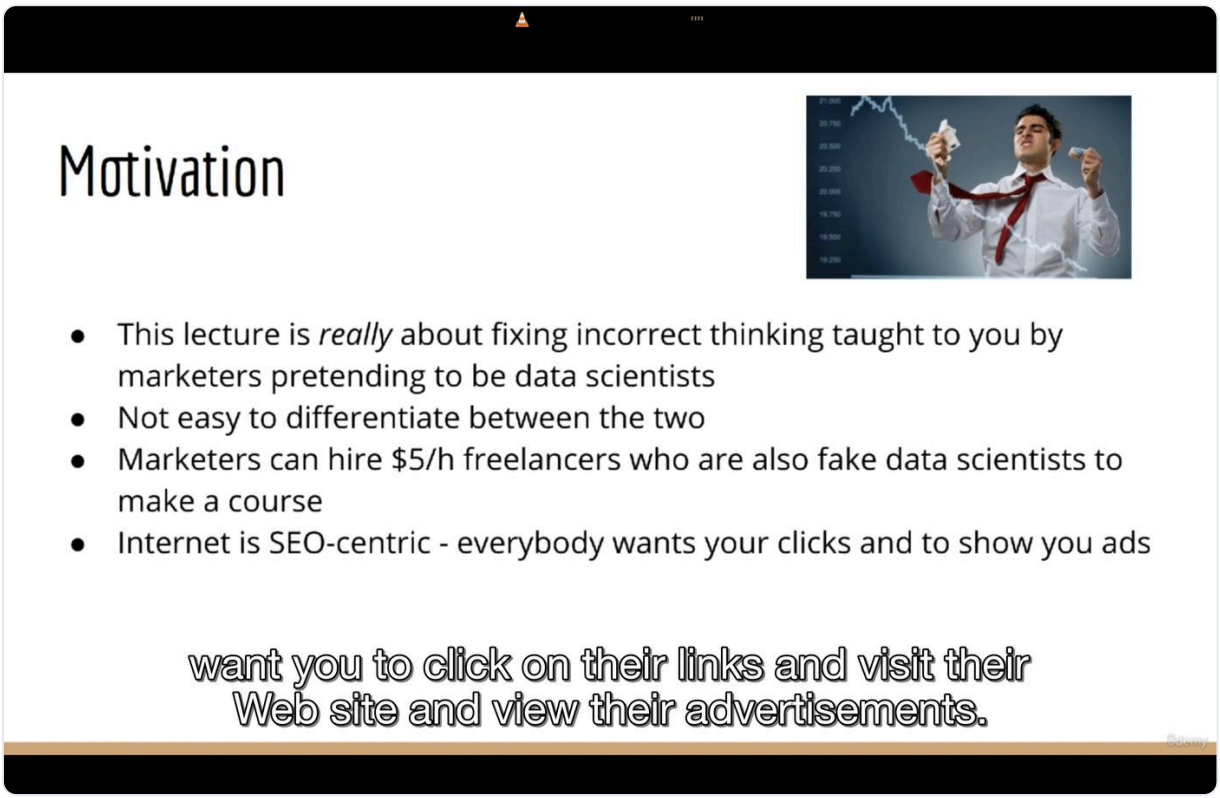


FIGURE 21.4

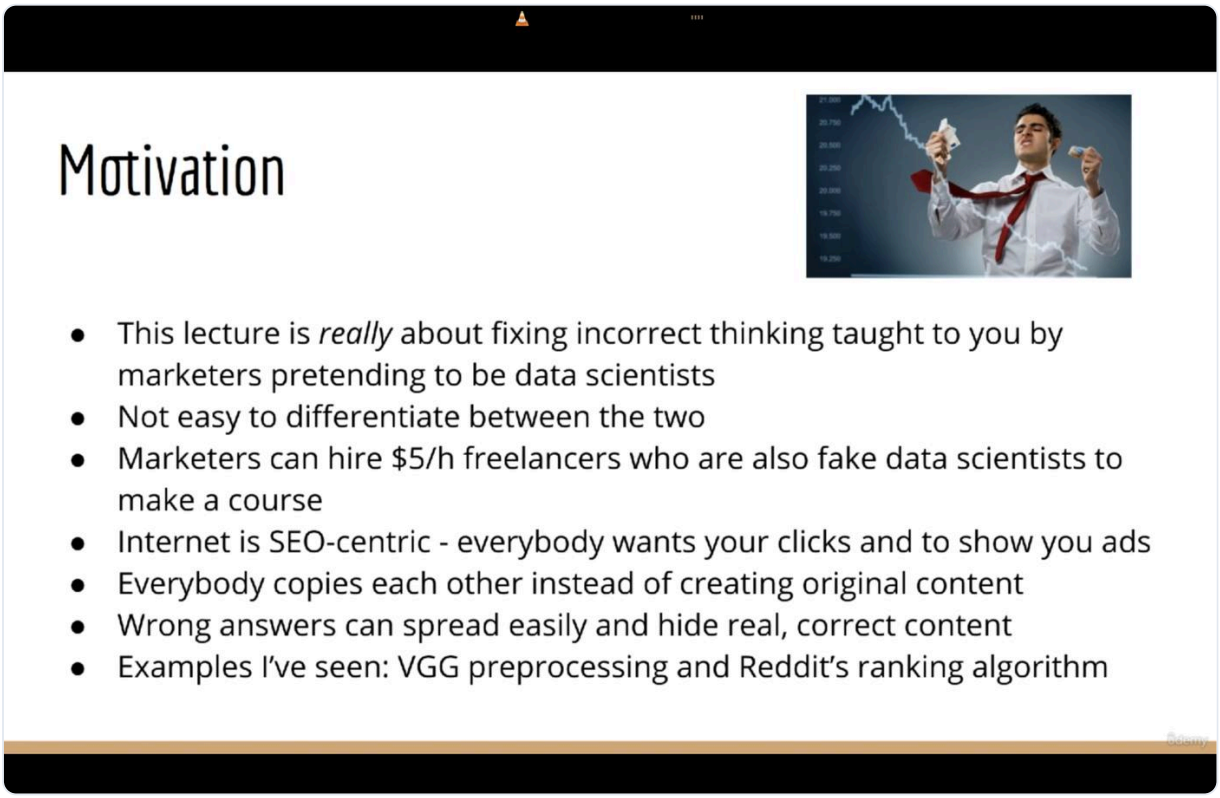


FIGURE 21.5

So what this lecture is really about is fixing incorrect thinking taught to you by all the marketers out there pretending to be data scientists. It's not easy to differentiate between a marketer and a data scientist. I can make a course on deep learning but so can a marketer. Sometimes there may be only subtle

differences in what we teach because marketers can just hire five dollar an hour freelancers online who themselves are also not actual data scientists. The Internet is SEO centric. People write blogs not because they are interested in or knowledgeable about the topic but because they want you to click on their links and visit their Web site and view their advertisements.

What ends up happening is everybody starts copying each other. This has network effects if one guy who has a popular blog ends up writing incorrect code. Guess what happens. Everybody who is a marketer or wants to do SEO is going to copy that guy's incorrect code. It's going to spread everywhere on the Internet which due to SEO will hide the true answer from you. I've seen this in several places throughout my career already. This example not included one example of this is with vague preprocessing and another example is the formula that Reddit uses in their ranking algorithm. Unfortunately when people stop thinking for themselves and resort to simply copying others because they are popular it is easy to make mistakes.

So this lecture is all about undoing those mistakes. Now back to the code. The first step here is to use pandas to grab the data from my github. Yes lucky you. I am not going to make you download this data yourself. This data contains daily stock prices for Starbucks from about February 2013 to February 2018. So let us do a df dot head and see what is in this data frame. Now first of all if you are not an expert in finance you might be surprised that this data when we talk about stock prices we usually think of these stock price. The stock price is a number and each day it's a different number. So you might think of this as a single variable time series.

And yet here we have multiple columns. So what are these columns. The first one is the date. That's pretty obvious. Second we have four different numerical columns open high low and close. Now all of these are stock prices. So the way the stock market works is it starts early in the morning and it stays open for the workday and then it ends and stock prices are tracked at a very granular rate. So it's not just a daily thing. It's pretty much real time. So at any split second you can find out the exact stock price. So to put it in simple terms the open price is the price at the start of the trading day. The closed price is the price at the end of the trading day.

The high price is the maximum price for that day and the low price is the minimum price for that day. Finally the volume column is the number of shares of the stock that were traded that day. If this is new to you this is probably very confusing. But luckily we always remember my rule. All data is the same. The machine learning algorithm doesn't really care what these things are. They are just a table of numbers. All we have to do as machine learning students is make sure those numbers are in the right range for the neural network as you can see the price values are in the tens and the volume values are in the millions. So this is an example of where different columns of our data have different scales.

In any case we can see some of the beginning values in 2013. If we do df dot head and we can see some of the ending values in 2018 if we do df dot tail so it ends on February 7th 2018 all right so that's nice for you. If you've invested in Starbucks your stock approximately doubled in the past five years next. Let's begin our work by demonstrating how some resources out there do this the wrong way. By the way I'm not naming any names but I know that a lot of you taking this course have encountered a course where they do an example just like this so we would like a single variable time series to start with. So let's just focus on the closing price. So we're going to do dot values to get this as a NumPy array and then we are going to reshape it to an N by one matrix.

We need to do this because the next step is to apply the standard scaler from scikit-learn so that our data is standardized. Note that I'm calling the fit function on the first half of the series only. This is because I don't want to include test data in the training pipeline. I call fit transform however on the entire dataset.

Also note that because we're going to be taking windows of this time series you would have to do a few calculations to figure out where exactly the boundary between the train set and the test set is. I kind of hand wave this and said let's just stop in the middle which is probably good enough but it's probably off by a little bit the mean and variance won't change that much if we add a few values or take away a few values so once we're done we flatten the series to get back an N length vector since that's what the code we were working with before expects to see so next we have the same code which turns our time series into a supervised learning dataset.

Basically it's going to take windows of length 10 and the target will be the next value. So conceptually this is no different from our sine wave time series. The only difference is the data itself. It is no longer a sine wave but a real data from a real stock. But again all data is the same. The algorithms we are using don't care about that. The code is still exactly the same next. Right off the bat we are going to use an LSTM network. Now you could attempt to use a linear model or a simple RNN but we are going to skip that in the spirit of following what these other resources are doing as an exercise. You might want to try those approaches and see how well they work so after defining the model I'm going to instantiate it and I am going to move the model to the GPU and create the loss and optimizer then I'm going to define a training function then we are going to do the usual train test split and we are going to move the data to the GPU and finally we train the model all right so let's scroll down and loss is the loss here is the loss per iteration.

So we see that the loss does decrease which is promising. That means our LSTM network can indeed predict somewhere close to the next value in the time series and now we have our one step forecast again this looks pretty darn good if you are just learning deep learning and you encounter something like this. You probably thinking to yourself Man I just hit the jackpot. But then you have to ask yourself. There must be other people out there in the world who are experts in deep learning. Are they making it millions or billions off this approach. You can ask yourself this question even without knowing deep learning. This is just logical reasoning. Thus we can assume that in reality something tricky is happening here.

Remember that one step forecasts are not really that useful. What we would really like to do is forecast for multiple time steps even just two or three. So let's see what happens. So here's our multi-step forecast and that's it for the second half of the dataset. All right. So now we can see what's happening when we do a multi-step forecast. All we see is a pretty straight line. What does that mean. It means the model isn't really predicting the next value in the time series. It's just copying the previous value. And that actually makes a lot of sense for a time series. Recall I said earlier that for images and time series signals this data is highly correlated.

If you're looking at a red car and you find a red pixel what are the neighboring pixel values. Probably also red. And if you're looking at a time series this time series is probably a smooth function over time. And we can see that that's exactly the model we've created. This is a smooth orange line it's not going to jump around from say minus one to plus two and back down to minus three and so forth. Now one thing that is nice about this last time I've run this example is that it did capture the trend that is going to go up but other than that it didn't really do anything useful so it turns out that this model is not really that great. In fact it's not really doing much.

LECTURE 60

Return Intuition

See the notebook `15.PyTorch-Stock-Returns.ipynb` for the code.

OK so that's lesson number one. The lesson is that it's actually very unconventional to try and predict the stock price in the first place. Instead what we really want to try and predict is the stock return.

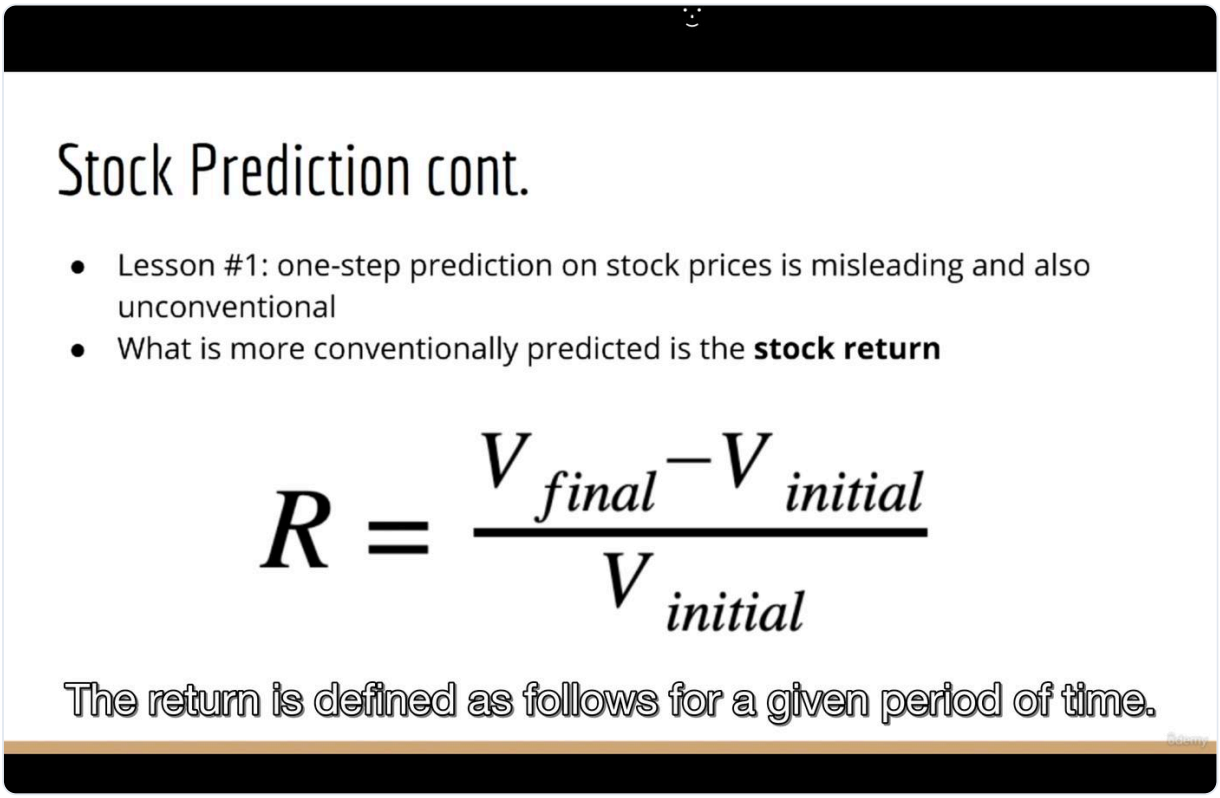


FIGURE 22.1

The return is defined as follows for a given period of time. The return is the final price minus the initial price divided by the initial price not just for some intuition. This is the same formula you would use when you want to calculate a sale.

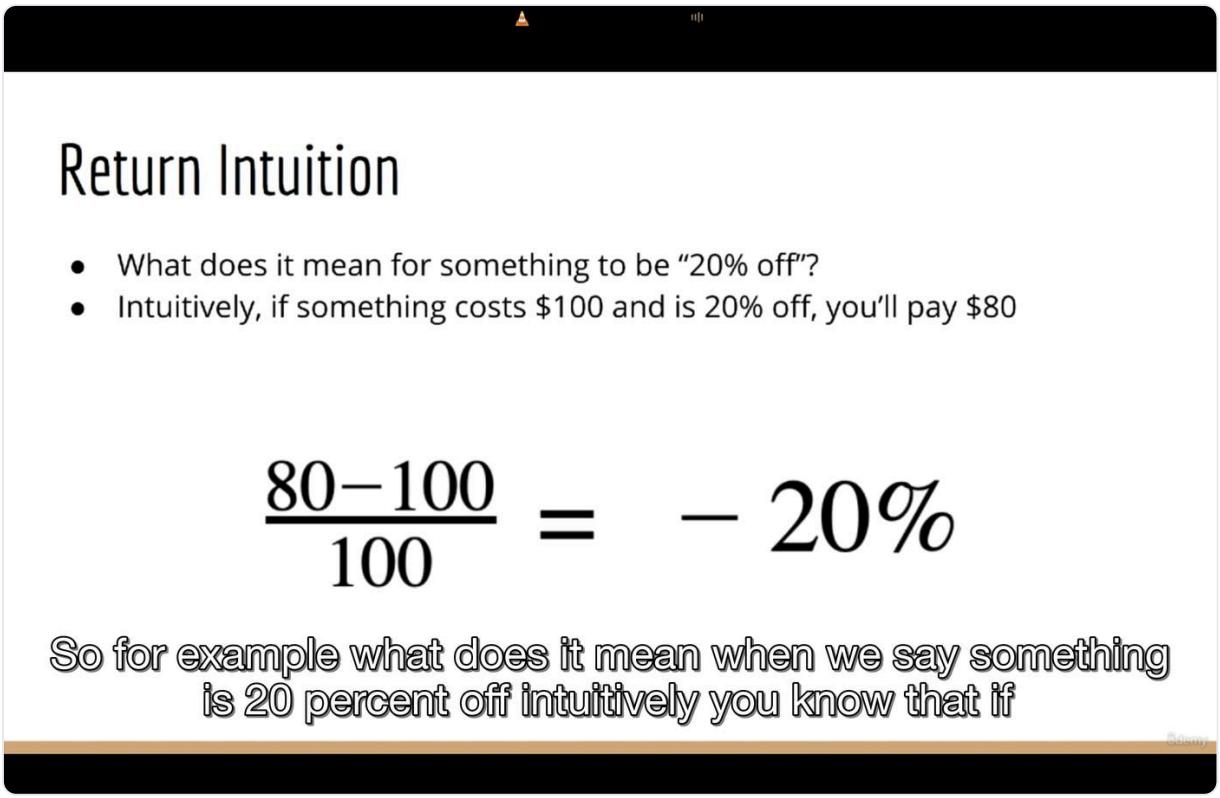


FIGURE 22.2 | Return Intuition

So for example what does it mean when we say something is 20 percent off intuitively you know that if something costs 100 dollars and it's 20 percent off then you'll pay 80 dollars. But we can also calculate this another way. Take 80 subtract one hundred and divide by one hundred. That's minus 20 percent. In other words the price you pay is 20 percent less than the original price. So to summarize when financial engineers do quote unquote stock prediction they are usually looking at some form of return not actual prices. OK so back to the code since we are working with a pandas data frame this is somewhat convenient at least if you know the right functions to call as usual we want to process our data in a vectorized form meaning we do some operation on all the columns at once.

So first we need to shift at the closing price up by 1 so that yesterday's closing price is aligned with today's closing price. So x of 2 is beside x of 1 as a 3 is besides 2 and so on. Then we can subtract them all in one step so let us do df dot head to look at the result now as you can see the first row is NaN again because we don't know the previous price for that row since it's not in our dataset. This is not a problem for us because we are building an RNN it is going to take the first 10 days of data to predict that the eleventh day. So that would be from here up to the tenth row. So we never even use this value. All right so the 11th row is much further away than the first row.

So if the first row has NaNs as long as they are not in the target that's fine. Next we do our return calculation. So we say df return is equal to df close minus df previous close divided by the previous close. Let us do df dot head again to look at these returns. So again the first value is NaN. But we don't care next we do a plot of this time series. Conceptually this is no different from the stock price or a sine wave. It's still just a time series but as you are beginning to see some time series are harder to learn from than others. Next we draw a histogram so we can see the distribution of returns we can see that these are pretty small values. So we might want to normalize them so we're going to go through the same thing again where we assign the returns to a variable called series as an N by one matrix.

Note that here I'm taking dot values at index 1 and up because the 0th index is a NaN as before we create a standard scaler call fit on the first half and call it transform on the whole thing and then flatten it back to an N length vector. Next we're going to go through all the same steps to build our supervised learning dataset and train our autoregressive RNN model this code again isn't any different than the previous code. It's only the data itself that's different in now we are working with the series as the return rather than the actual stock price. So here is the data here is our RNN model and here is the result of training so the loss starts out at about 1 and then by the end of training the loss is about point 8 7 and the test loss is about one point five.

So it's pretty clearly over fitting well we can see from this is that the model has a much harder time learning anything. If we look at the loss per iteration it looks like the loss goes down a tiny bit but only at the cost of the validation loss going up. In other words it's just fitting to the noise. Next we are going to do a one step forecast and so here are the results. Now it's hard to tell whether this is good or not but judging by the previous loss values it's probably not good as an exercise. You might want to try to run this locally so that you can zoom into the plot and check for yourself. Next we're going to do a multi-step forecast. So here are the results for the multi-step so again we have the situation where the model doesn't really match up that well with the actual time series.

LECTURE 61

Only Appears to Work Well

See the notebook `15.PyTorch-Stock-Returns.ipynb` for the code.

Finally we're going to move on to the next example where we see how to actually build this model right. By the way there isn't really anything wrong with the second model. It just wasn't a good model. This is unlike the first model where there actually were things wrong with it because it gave us misleading results and also predicts the wrong thing.

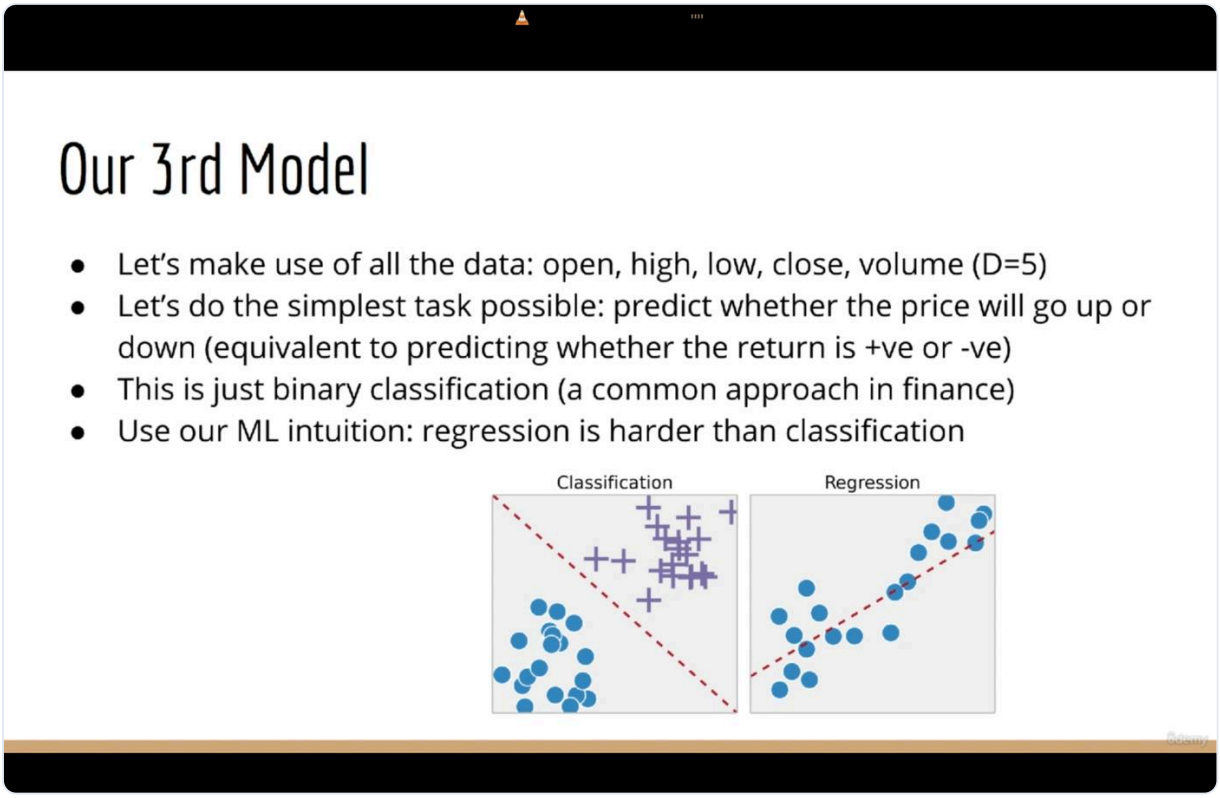


FIGURE 23.1

And this final example we are going to make use of all the data open close high and low prices as well as the volume. So we are going to have a t by D input where t equals 10 and d equals five. And from this we are not going to try and predict a return. Instead we turned this into the simplest problem possible. Just try to predict whether the price will go up or down. In other words binary classification if you've worked in machine learning long enough you should have some intuition about this. Regression is in general a harder problem than classification with regression. You have to predict a continuous value. You can't be too high or too low. You have to be right on the nose classification is easier especially binary classification.

You don't have to predict the exact number. You just have to predict a label. In this case you only have two choices up or down. So this is what we are usually trying to predict when it comes to stock returns. Let's get back to the code. First we are going to start by getting all the data into NumPy arrays so the input data is going to be made up of the open high low and close prices as well as the volume. The target is

going to be based on the return. Next we are going to set N T and D. Since our data is of shape N by T by D. T is simple we can just choose that. So let's stick with 10 although in the future you may want to try different values like 20 30 40 50 and so on D is also simple.

It is just the number of columns in the input data N is a little tricky. It's the length of the series minus Big T although this should remind you of each of our time series examples where we did the same thing. We just never calculated N explicitly next we're going to normalize the inputs. This time we're going to make it a little easier on our model instead of the first half being the train set in the second half being the test set. Let's make the first two thirds the train set and the last third the test set so as usual we're going to normalize our data. This is especially important now since the volume column is very high compared to the price columns.

Next we create our train set X train is of shape N train by T by D and Y train is of shape N train then we do a loop where t counts up from zero up to N train and we populate X train and Y train for x we take the input data from Little T up to a little T plus big T for Y. We have a boolean which is true if the return at Little T plus big T is bigger than 0 next we create the test set. As you might expect N test is N minus N train so x test is of size N minus N train by T by D and Y test is a shape N minus N train. Next we enter a loop counting from zero up to N minus N train or equivalently N test inside the loop. We actually have two indices u refers to the index of X test and Y test which of course will start from zero but remember we are indexing our original dataset which must be offset by N train.

So t is equal to u plus N train and we use t to index the original dataset whereas we use u to index x test and Y test once we are done that we can make our RNN. Importantly remember that we are now doing binary classification what this means is that our model outputs are still the same because no matter what our final layer will be a linear. What makes it different is the loss function. The loss is now the binary cross entropy so let's run this and train our model that we train for quite a while 300 epochs so what do we see here. We see that the train loss goes down a little bit but the validation loss goes up a lot. So what this tells us is that the model is just again over fitting to the noise in the train set so that is what it means when the train loss goes down.

But the validation loss goes up if we look at the accuracy we see the same thing. The train accuracy goes up but the test accuracy stays at 50 percent. Now you might ask why doesn't the test accuracy go down. One important thing to realize about binary classification is that zero accuracy is not the worst accuracy if your accuracy is zero. That means you can just reverse all your predictions and you would get one hundred percent. That would be a perfect model. In other words for binary classification the worst accuracy is actually 50 percent meaning it's no better than random guessing. That's why the test accuracy remains at 50 percent as the model over fits.

All right so that's the end of this script let's now discuss and summarize everything we just did since it was probably a lot to take in we just went through three examples of predicting stock returns in the first example. We used an LSTM to predict future stock prices from past stock prices. While this appeared to work well with the one step forecast it didn't work so well for multi-step forecasts we saw that the model was likely just remembering the previous value and making that prediction. This actually makes sense in the context of our second example as well where we tried to predict stock return. In this example we saw that we could not get the loss to decrease that much.

The third example we did was to consider all the data not just the previous closed prices and to turn our model into a binary classifier we saw that even on the simplest possible task with the most data the models still could not achieve better results than random guessing what this tells us is that there isn't any hope of our first two models working in the first place.

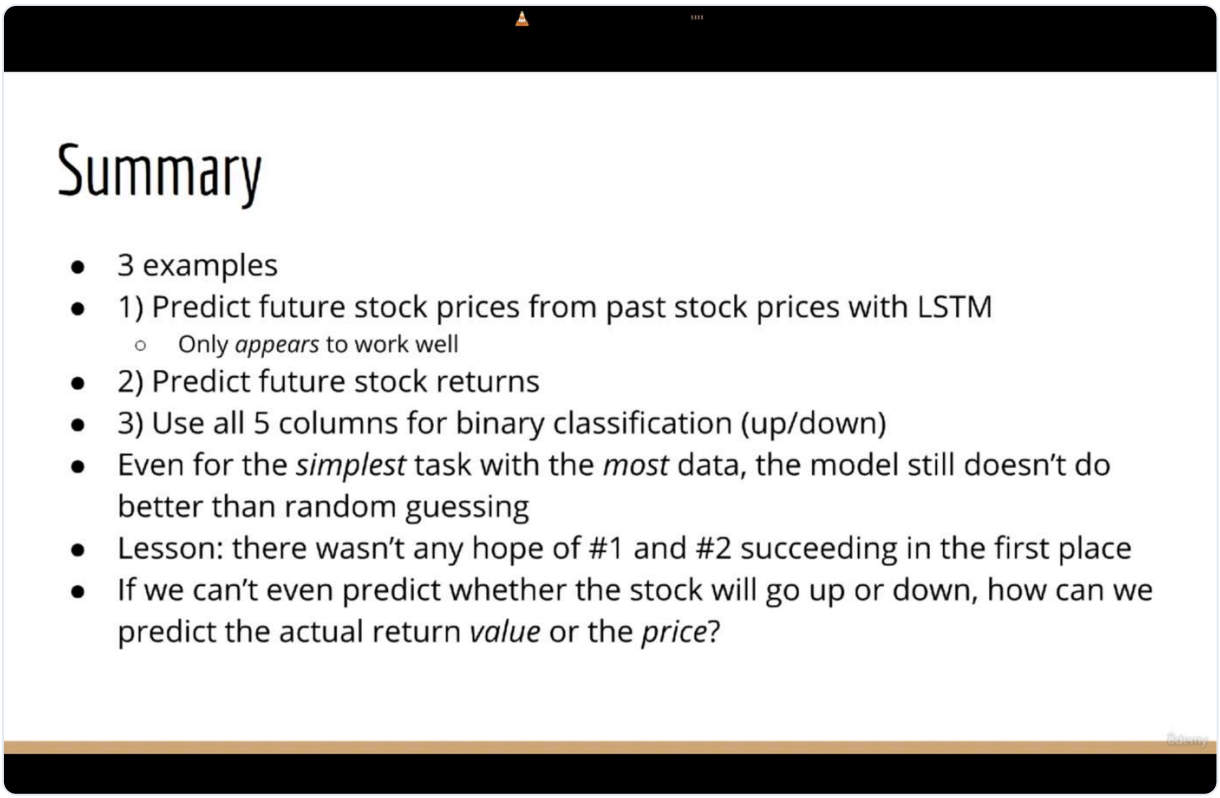


FIGURE 23.2 | Only Appears to Work Well

Imagine if you can't even predict whether the stock price will go up or down. How could you ever predict the numerical value of the return or the next stock price itself. In other words if you can't even do the easy task then surely the harder task is also not going to be solved so be suspicious whenever you encounter an almost perfectly accurate stock price forecast. If we can't even do binary classification then you can be sure that forecasting the actual price will not be so simple by the way these results should not be surprising.

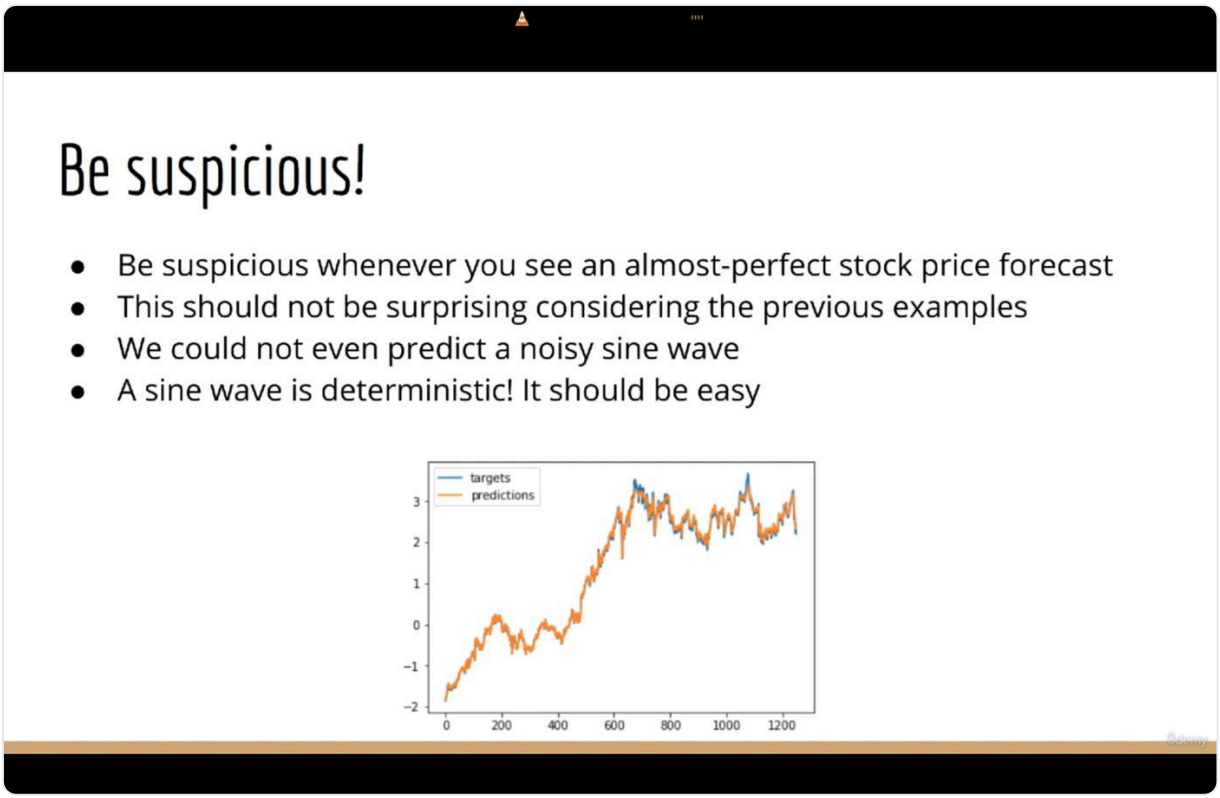


FIGURE 23.3 | This Should Not Be Surprising Considering the Previous Examples

I asked you earlier to keep in mind the previous examples we did in this section if you recall we had trouble getting an RNN to predict a noisy sine wave. The underlying function in a noisy sine wave is a sine wave. It's a deterministic signal you can imagine that if we have problems even with that then in comparison stock prices should be a huge problem the major problem with trying to predict stock prices from stock prices is that it's a fundamentally flawed approach. These approaches don't take into account data about the world itself. Why does a stock price go up or down. Many times it's due to some occurrence in the real world. Imagine how ludicrous it would be to say here is some historical stock prices from this.

The future is totally deterministic. We can predict everything that will happen now. That is a pretty crazy line of thought. Now consider a more realistic scenario where stock prices depend on the emotions of investors that could be based on things like how the company is being portrayed in the media whether the company has invented something new and so on. Even without knowing about deep learning there should be enough to make you suspicious of any course that claims to be able to use LSTMs to predict stock prices using historical data.

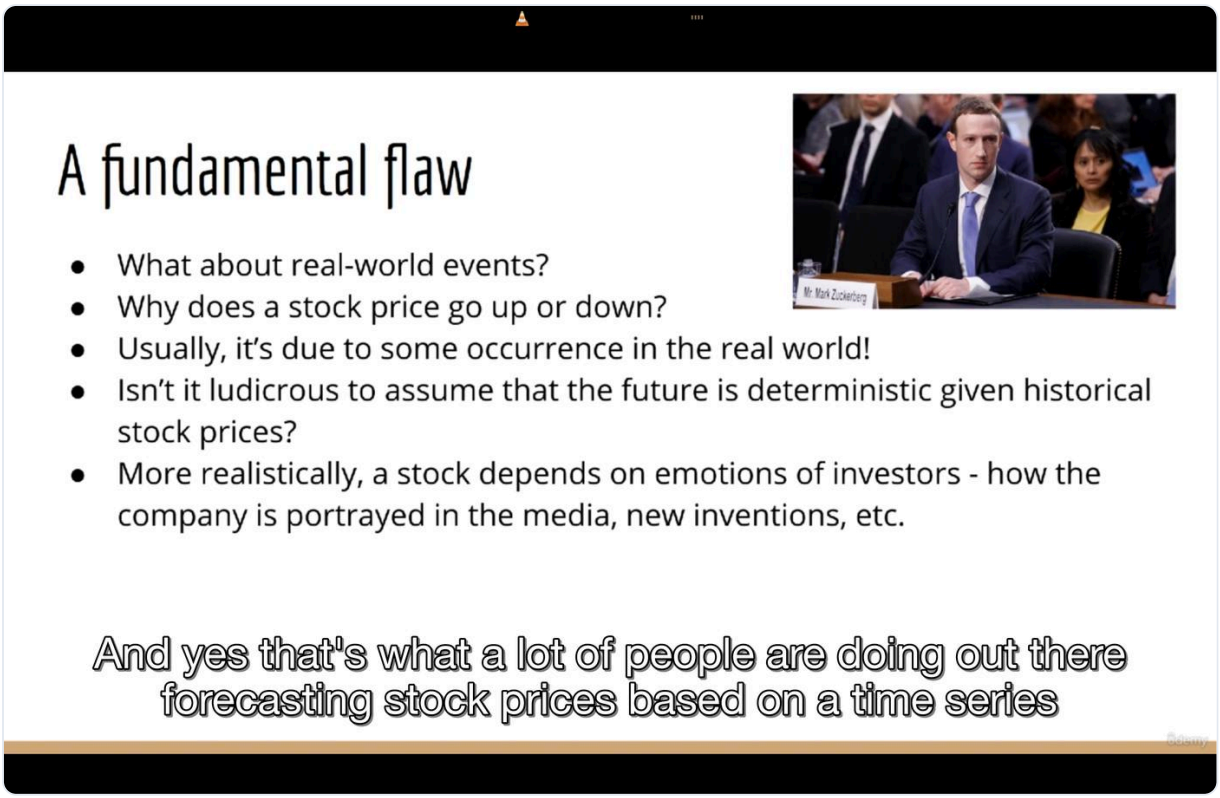


FIGURE 23.4 | What About Real-world Events?

And yes that's what a lot of people are doing out there forecasting stock prices based on a time series alone instead we can think about what actually might cause a stock price to go up or down. We see examples of this all the time. For example when Facebook is being investigated by the government that causes its stock price to go down.

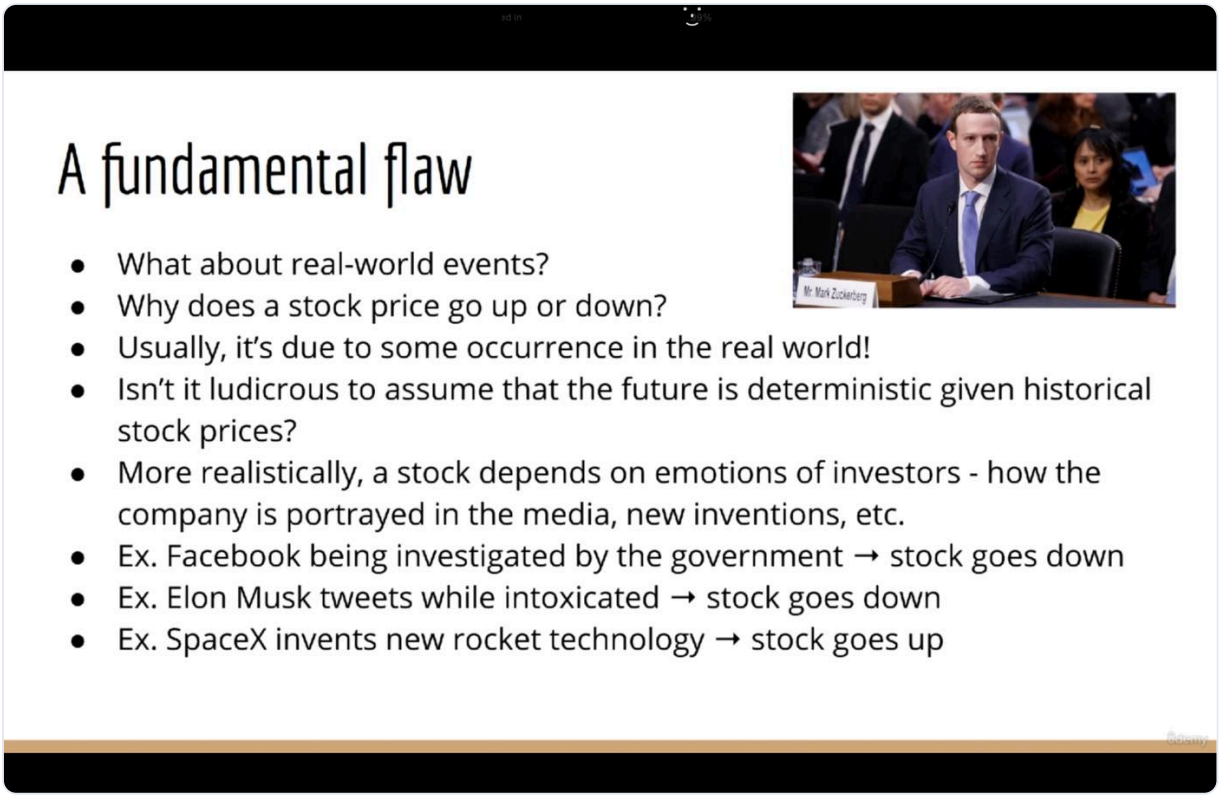


FIGURE 23.5 | What About Real-world Events?

When Elon Musk tweets while intoxicated that makes his company's stock price go down. But when someone at Space X invents a new kind of rocket technology that makes the stock price go up imagine what you are saying. If you try to use historical prices alone to try and predict future stock prices that's like saying using historical stock prices you think you can predict whether Facebook will be investigated or fined by the government. That Elon Musk might make a stupid tweet or that some rocket scientists will invent something to believe that you could predict such events from looking at stock prices is surely a laughable idea. That's like saying different events that happen in the world are preordained by stock prices.

Now that is crazy. If you really want to learn how to do good stock market predictions that could be a course all by itself.

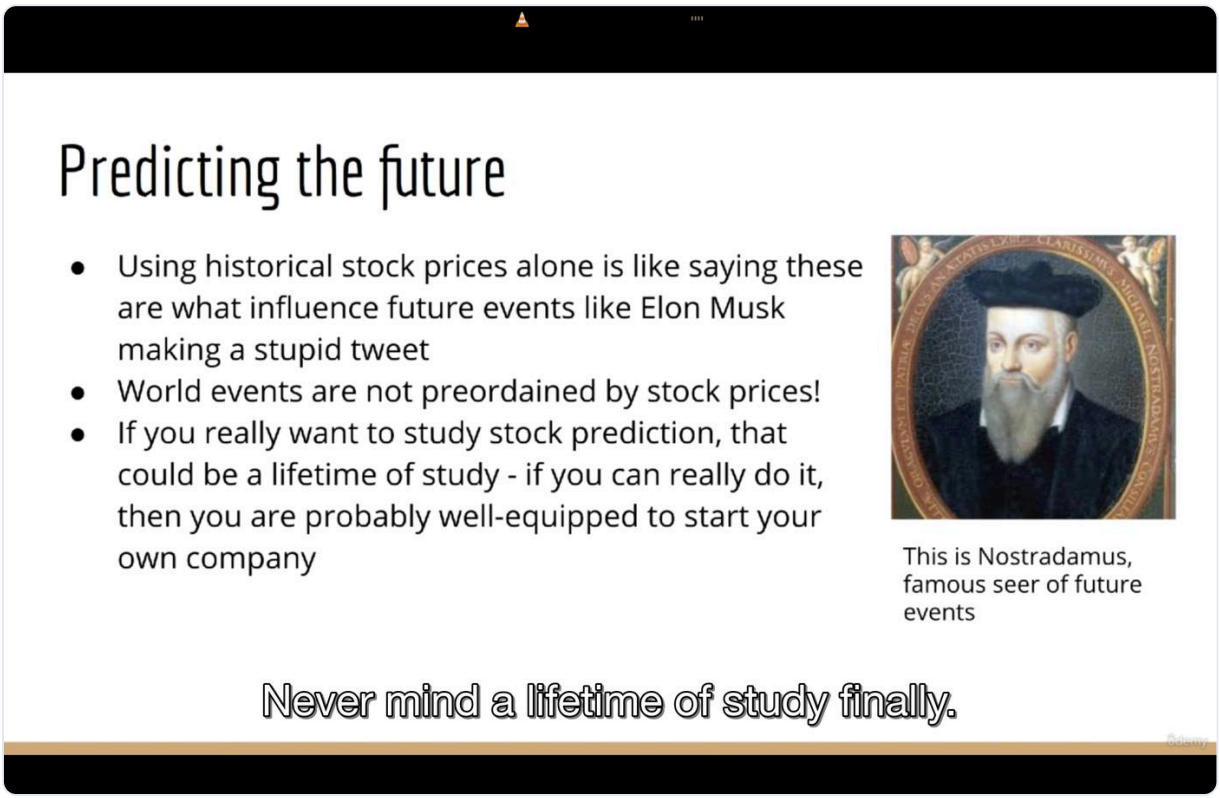


FIGURE 23.6 | Predicting the Future

Never mind a lifetime of study finally. This clearly is not to say that LSTMs are not good.

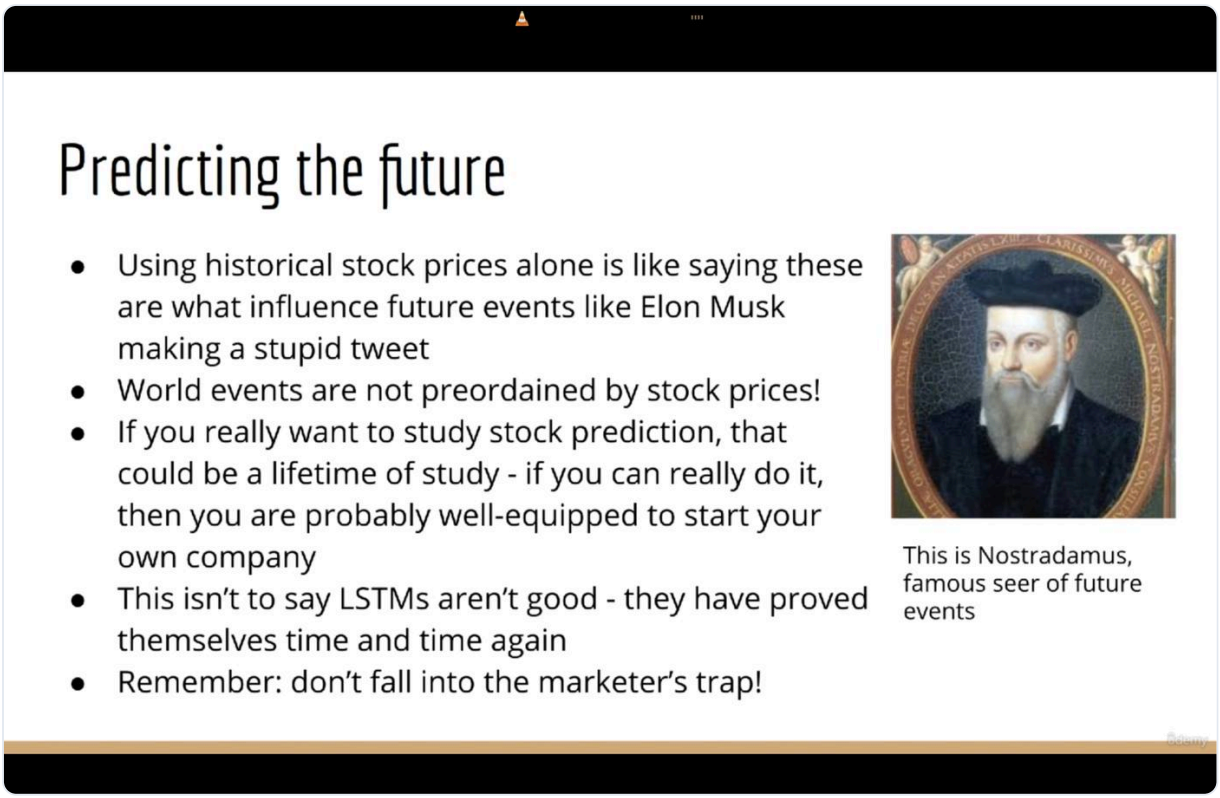


FIGURE 23.7 | Using Historical Stock Prices Alone Is Like Saying These

LSTMs have proven themselves time and time again on tasks such as language modeling and machine translation We have already seen how LSTMs can be used for image classification which is an application you probably hadn't even considered. So I hope this lecture helps you understand sort of a bigger picture around deep learning and how to choose the right resources to learn from. There are a lot of marketers out there trying to get your attention. So try not to fall into their trap.

LECTURE 62

Other Ways to Forecast

In this lecture I want to discuss some other ways you might generate multi-step forecasts. You may have noticed that the two forecasting methods we used in this section demonstrated some pretty extreme results. On one hand we saw that one step forecasts can seem artificially good especially when all they do is just predict the last value or close to it.

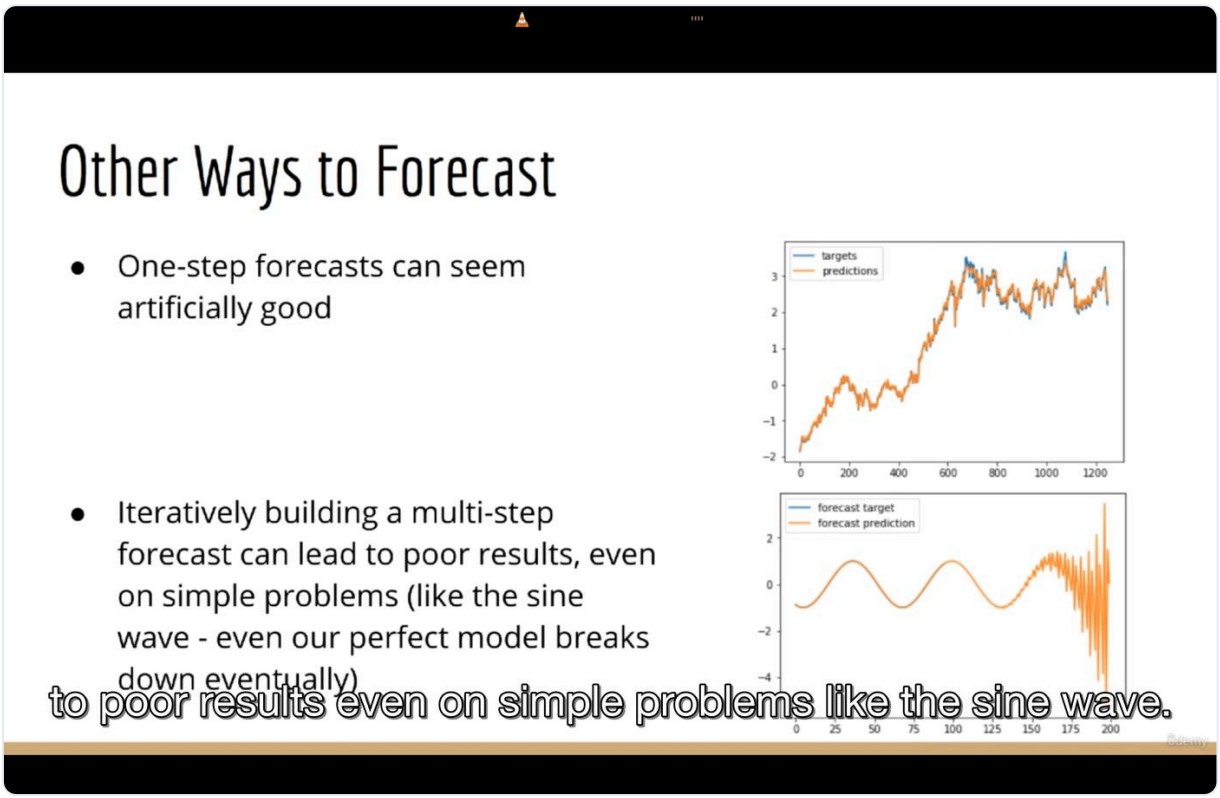


FIGURE 24.1 | Other Ways to Forecast

On the other hand doing a multi-step forecast iteratively using the models own predictions can lead to poor results even on simple problems like the sine wave. Even a nearly perfect sine wave model eventually breaks down due to the build up of numerical precision errors at this point. One issue we have to think about is practicality.

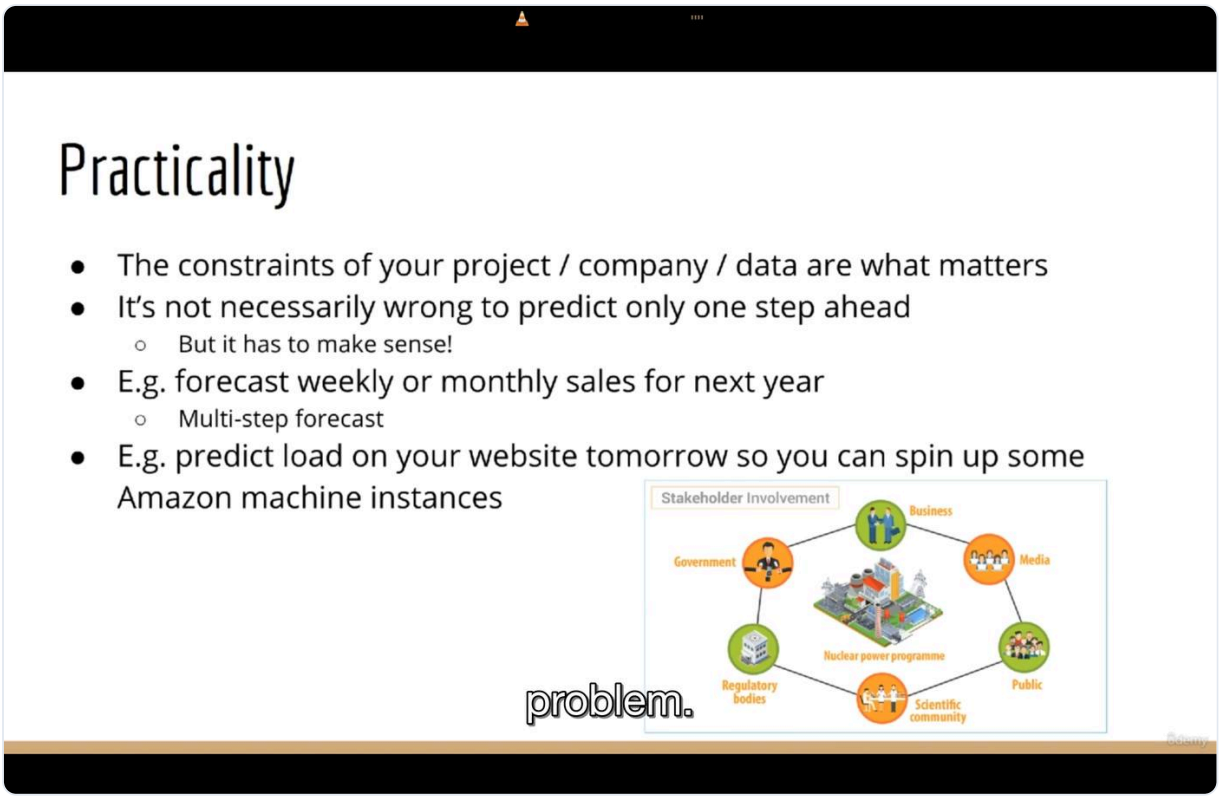


FIGURE 24.2

In fact it's not necessarily wrong to predict only one step ahead. It's certainly wrong when you want to make a conclusion such as Wow look how good my model is at predicting future values. Of course that doesn't make any sense because you used future values to predict the future values. In some cases it may make sense to only want a prediction for one time step ahead. In other cases it may not make sense. For example if you want to forecast weekly or monthly sales for the next year it doesn't make sense to only do a one step forecast. On the other hand maybe you just want to predict the load on your website for tomorrow. So you know how many Amazon machine instances to spin up.

Either way the method you use is highly dependent on context and the real world constraints of your problem. It's also important to benchmark your predictions against a baseline model. The reason you want to do this is well what if your complicated model performs worse than your baseline model. If that happens then you should just use your baseline model as you'll see. Sometimes you can even prove mathematically that your baseline model is better than your complex model. One way to establish a baseline is to use what is called a naive forecast a naive forecast does the dumbest thing possible. You guessed it it just predicts the last value of the time series.

It's a model with zero parameters.

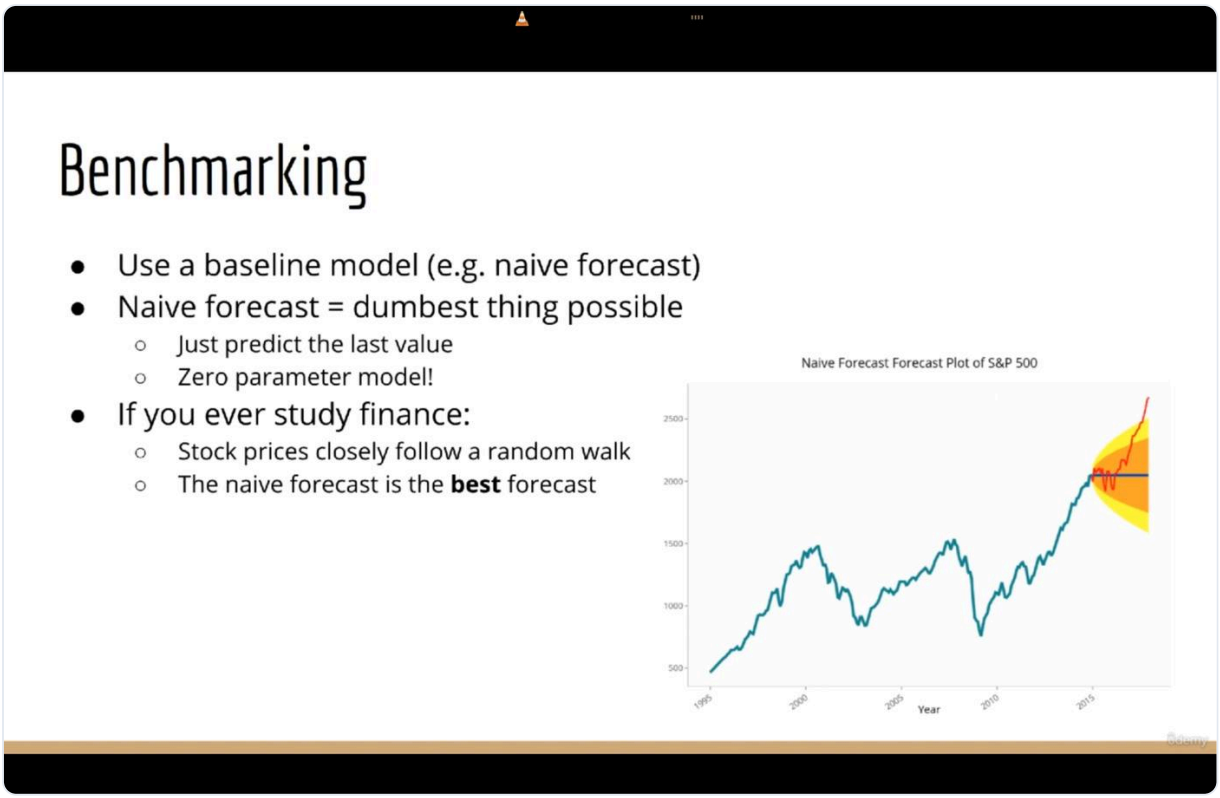


FIGURE 24.3 | Use a Baseline Model (e.g. Naive Forecast)

One interesting fact you will learn about if you ever study finance is that stock prices very closely follow A Random Walk Model in a random walk model. A naive forecast is in fact the best forecasts you can make. So by definition if your time series follows a random walk it doesn't matter how complex your model is. You cannot beat a naive forecast in time series literature classic statistical models such as ARIMA use algorithms that predicts only one step ahead and then use the iterative method we discussed previously in order to do multi-step forecasts into the future. One interesting question to ask is Are there any models that naturally predict multiple steps into the future.

The answer is yes and we can do so quite trivially. As you know it's very easy to specify the number of outputs in your neural network model. You're just passing a number into the dense layer for the number of outputs you want. So what if you want a model that predicts 12 time steps ahead simultaneously. In fact that's entirely possible.

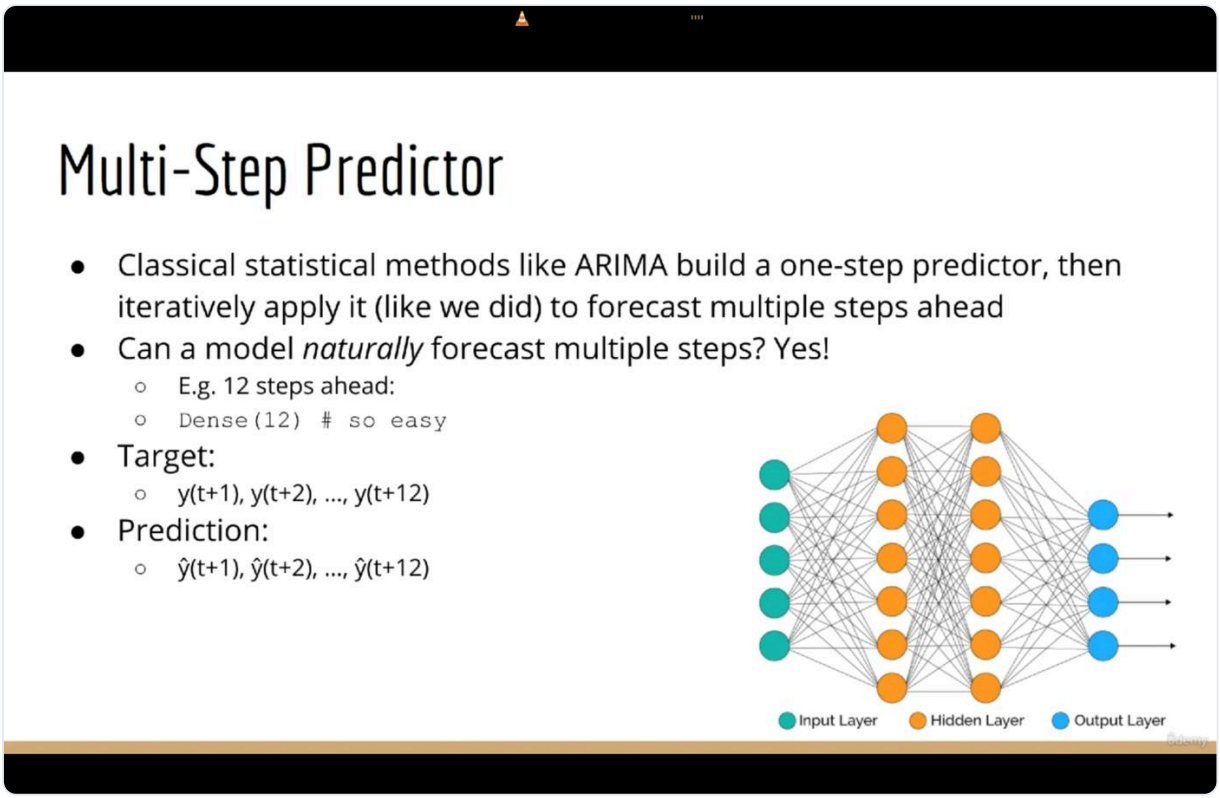


FIGURE 24.4

Just make a neural network with twelve outputs. Then as the target you will use the true values for the next twelve time steps. So whereas in this course the target just had one value for a multi head model. You'll have a target with 12 values. If you're interested in this kind of thing you might want to give this a try as an exercise to summarize this lecture what we wanted to do was clarify the difference between the different kinds of forecasts. The big mistake a lot of people make is they pretend they are predicting the future when in actuality they are only predicting one step ahead. It's not wrong to predict only one step ahead but you must be clear about the results you are presenting.

If you really want to forecast multiple steps into the future then you are not allowed to use data from the future. Of course there are some cases where you actually want to make one step forecasts only. This is dependent on the specifics of your company and your data only you know that and you must design your model accordingly. We also discussed the importance of establishing a baseline which in time series analysis is often the naive forecast where all we do is predict the previous value of the series I mentioned the random walk model which often very closely models stock prices a naive forecast is in the best forecast.

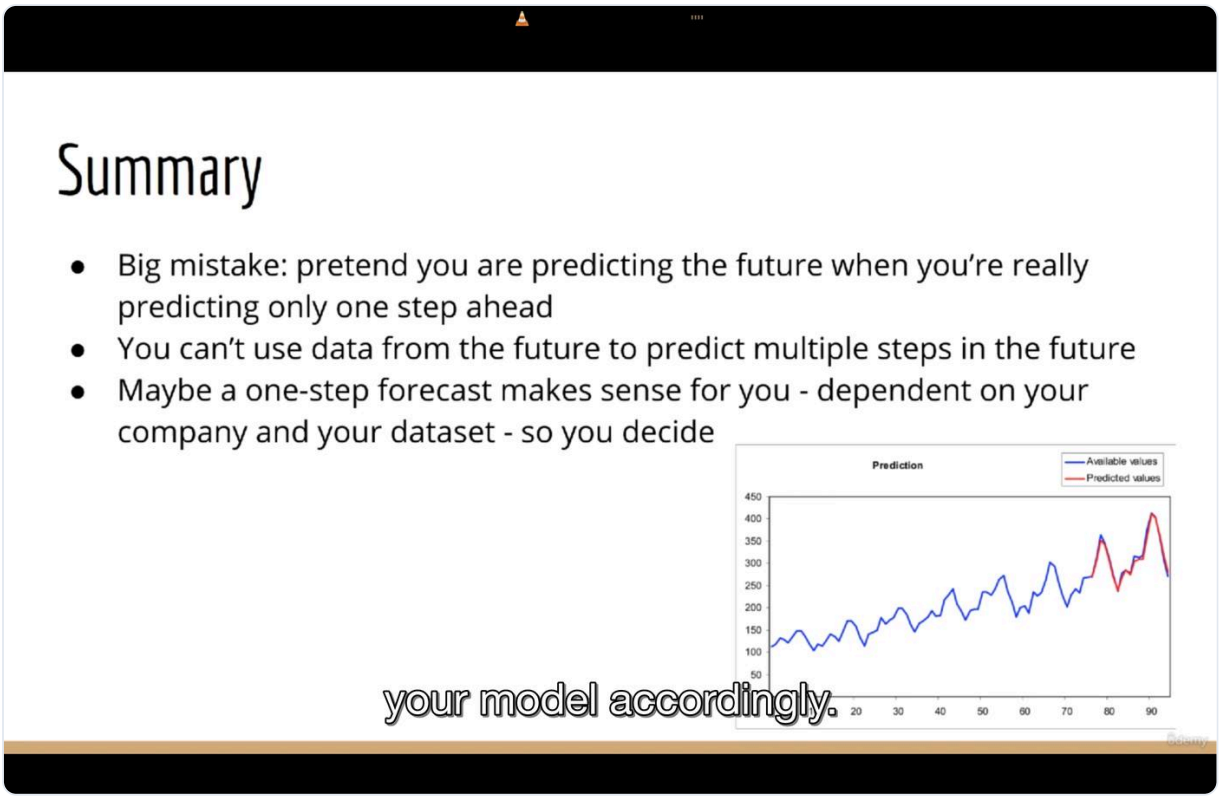


FIGURE 24.5

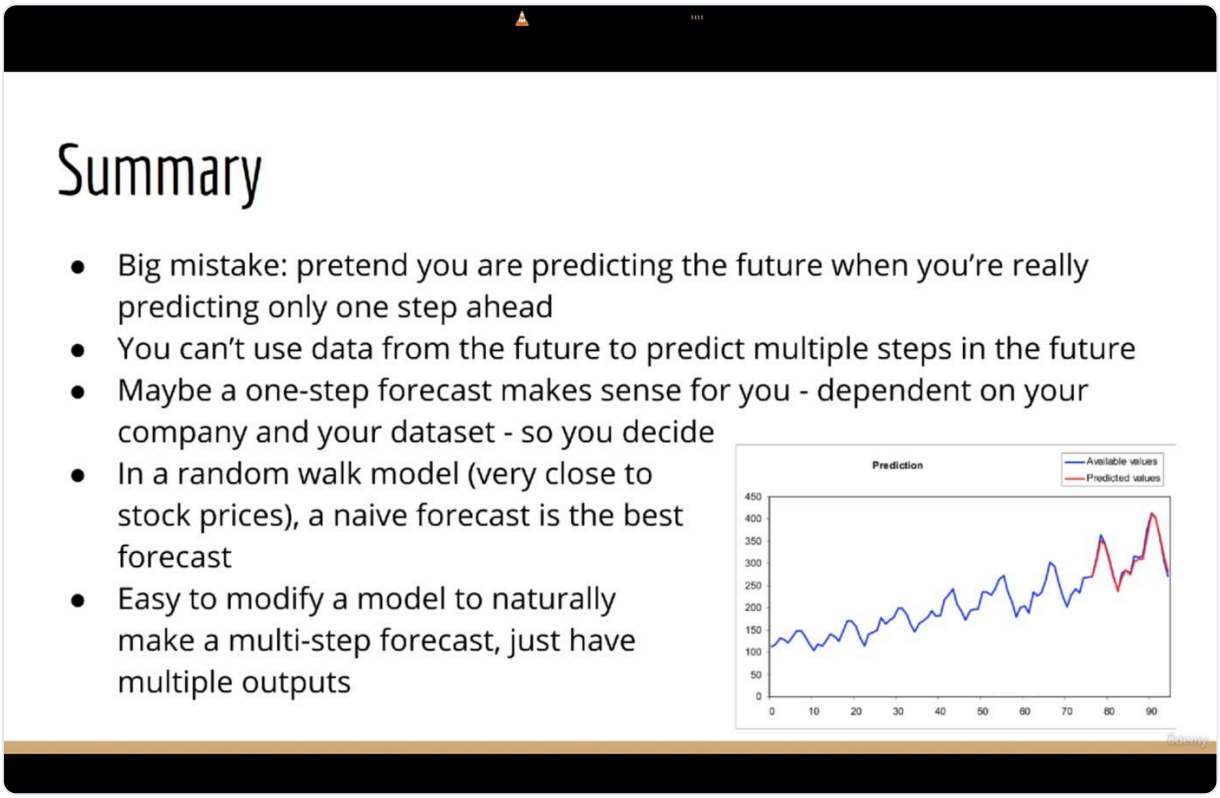


FIGURE 24.6

In addition we saw that aside from building a one step model and then iteratively applying it to predict multiple steps into the future there is an easy modification we can simply use a neural network with multiple output nodes representing the multiple steps in the future that we want to predict.

LECTURE 63

Dealing with Text

In this section of the course we are going to discuss how to deal with text data in natural language processing. This directly ties into the previous section which was on using RNNs for modeling sequence data.

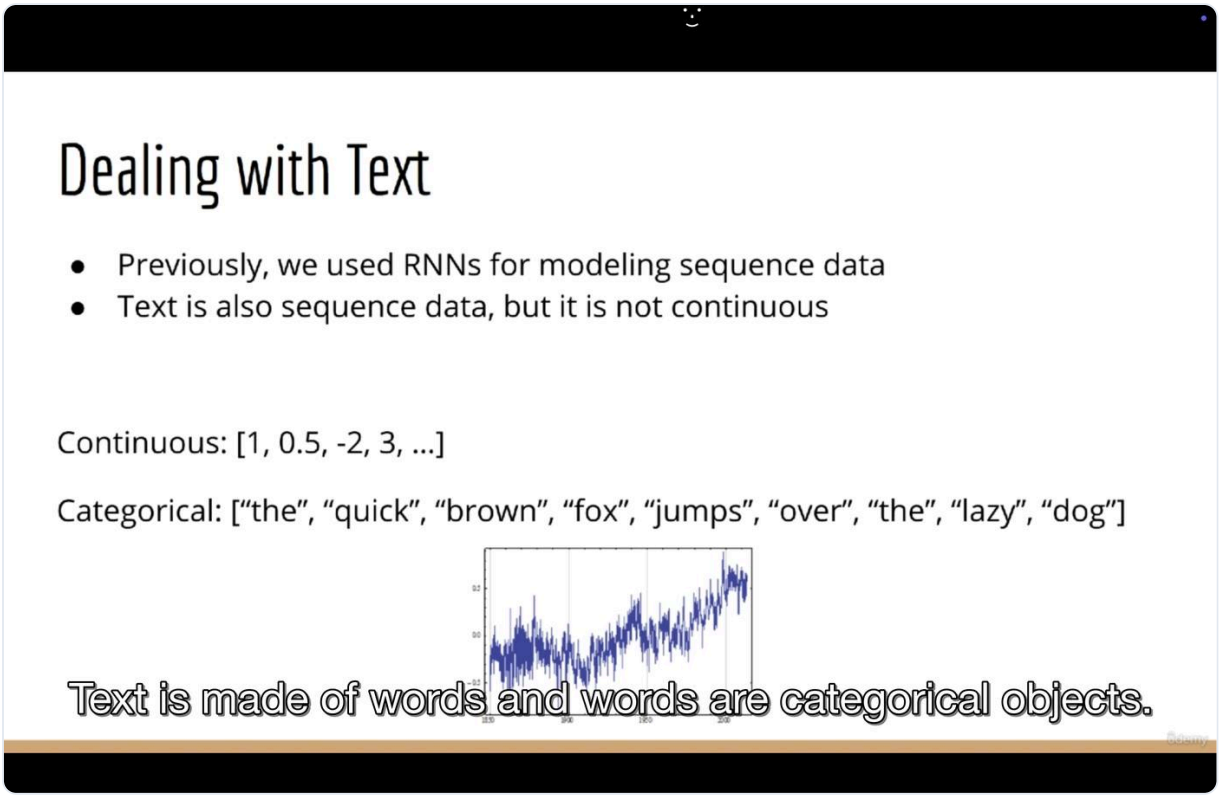


FIGURE 25.1 | Dealing with Text

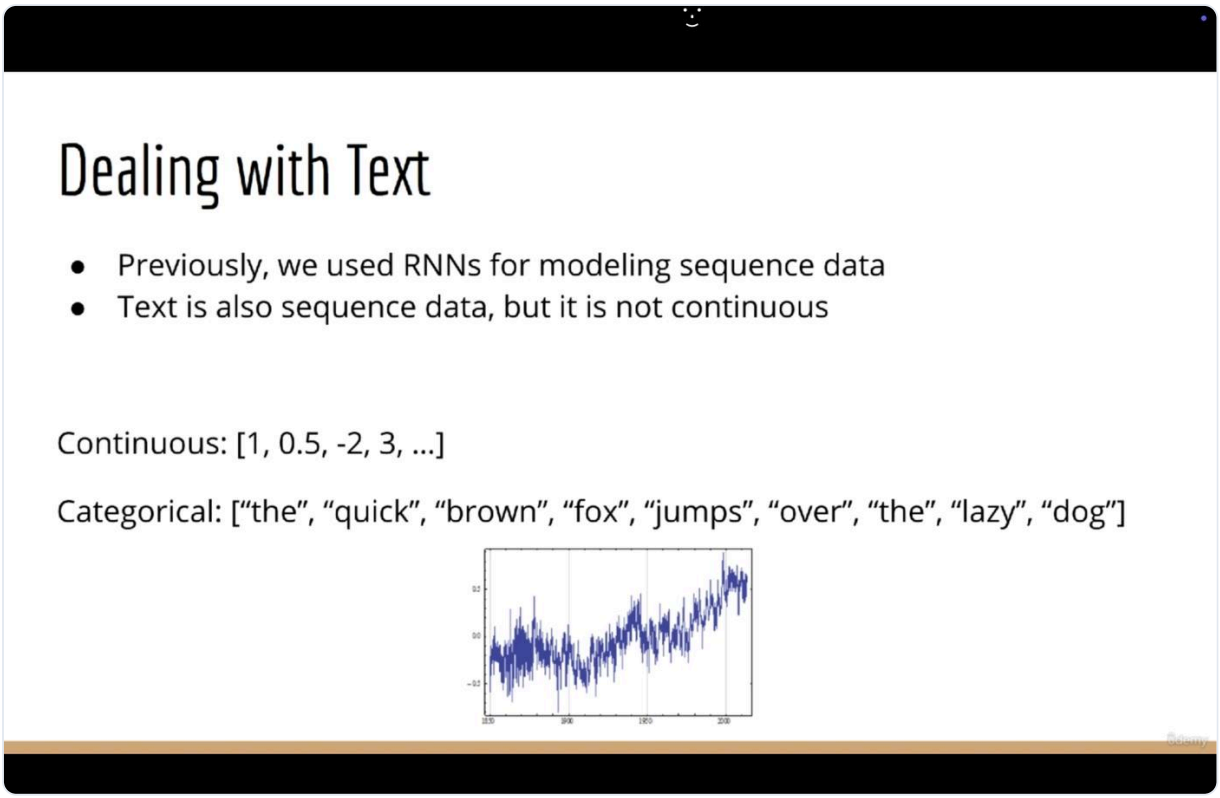


FIGURE 25.2 | Dealing with Text

Of course text is also sequence data but there is a major difference between text sequences and the continuous valued sequences we were dealing with in the previous section. That differences. Text is made of words and words are categorical objects.

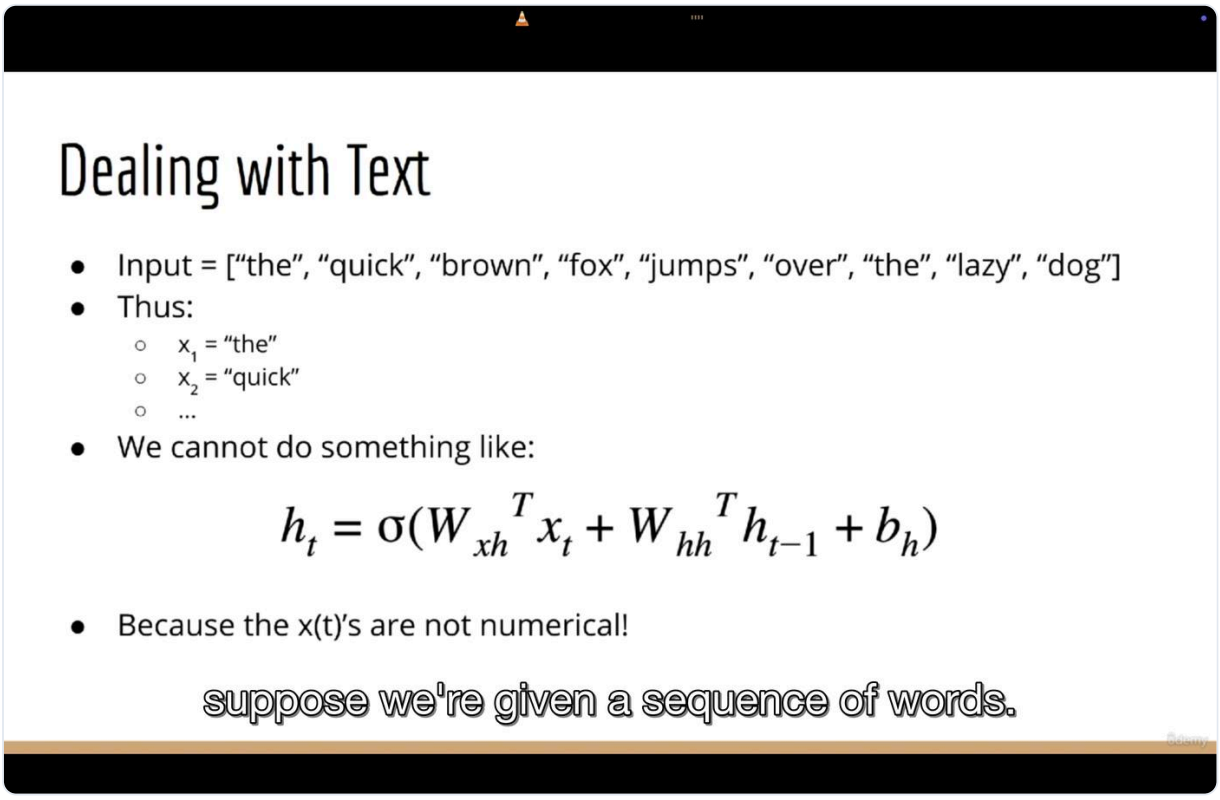


FIGURE 25.3 | Dealing with Text

In other words they are not continuous suppose we're given a sequence of words. The quick brown fox jumps over the lazy dog so x1 equals the X two equals quick and so on. But here's the problem we know that in order to calculate the output of an RNN unit we have to multiply each x by the input to hidden weight W x h. You can see that if X is a category this is not possible you might think Aha I've seen this before when I have categories all I need to do is one hot encode them. So for example what you would do is create an array which has length equal to the size of our vocabulary. In other words the number of words in the English language. Typically we denote that with a capital letter V.

So here is how we are going to one hot encode a word. First we're going to create a vector of size Big V containing all zeros. Then we're going to create a mapping where we have for each word a corresponding integer starting from 1. So for example the word a we'll get the index 1 The word A will get the index two the word activation might get the index 100. And we do this for all other words. So we get all the way up to Zu which maybe has the index 1 million finally.

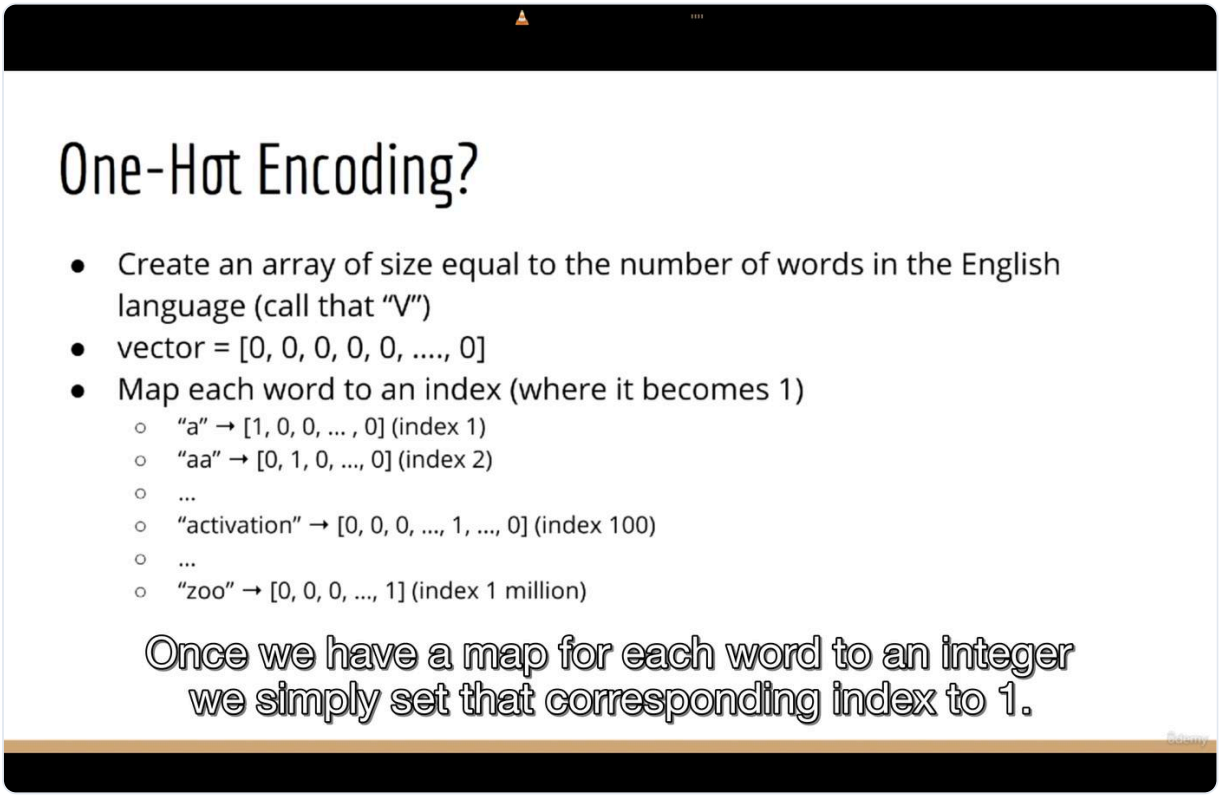


FIGURE 25.4

Once we have a map for each word to an integer we simply set that corresponding index to 1. So as you can see all of these vectors are a bunch of zeros with only a single one at a unique position so thus our final one hot encoded word vector becomes a vector of mostly all zeros except with one one. So what happens after we do this.

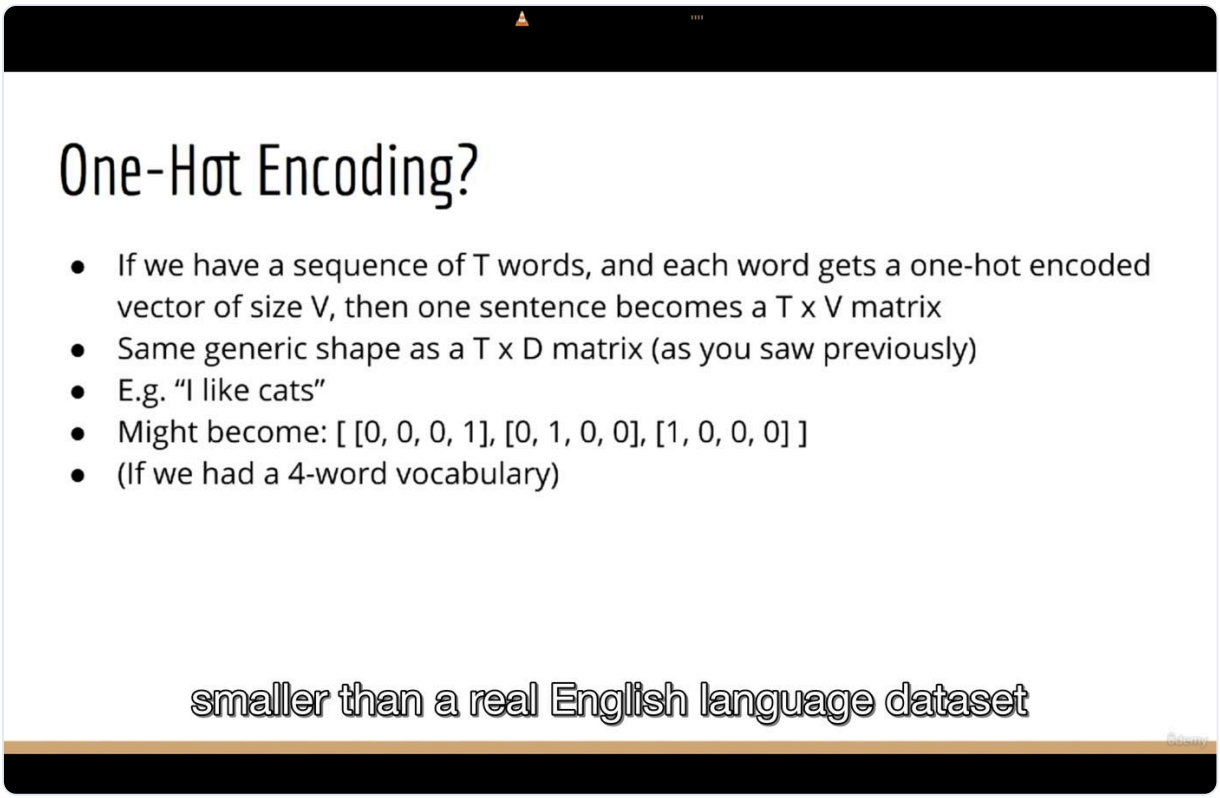
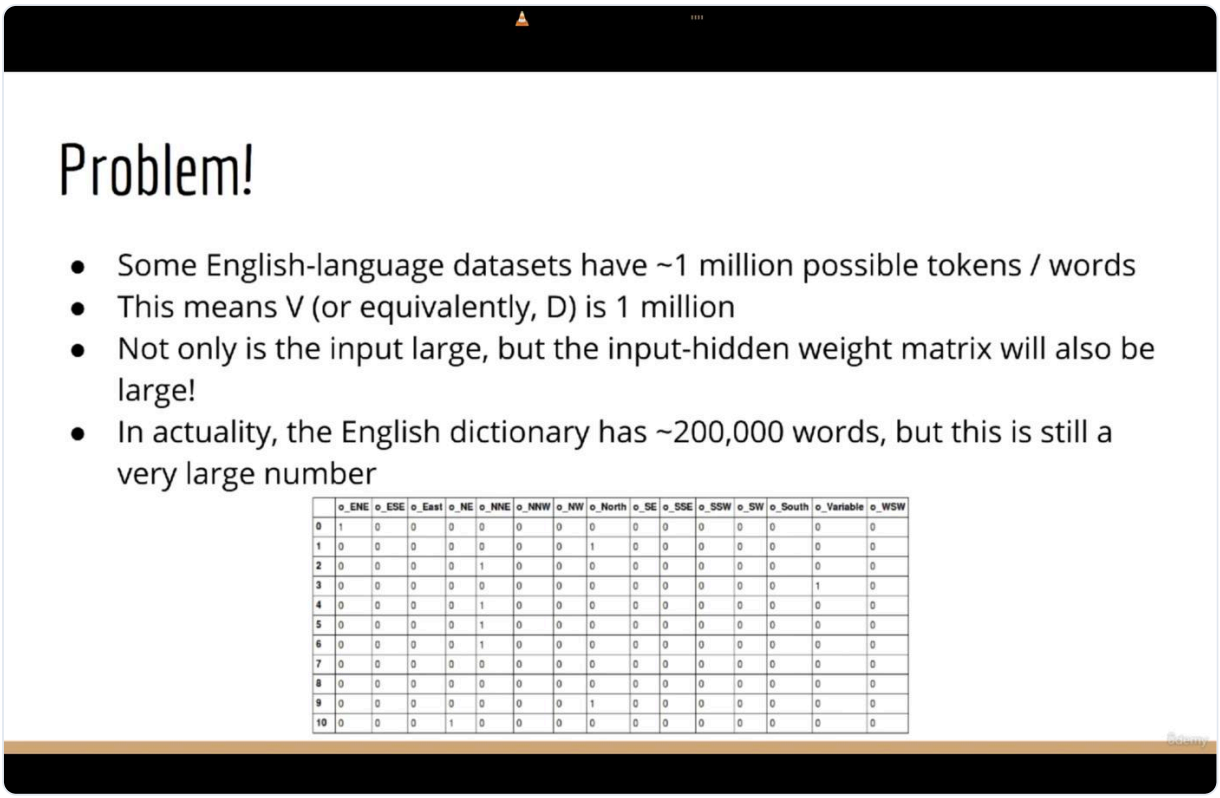


FIGURE 25.5

Well now we can go back to our usual scenario if we have a sequence of T words and each of them becomes a vector of size V then we have a t by V matrix which represents our sequence. This is the same shape as our generic t by D sequence. We can then pass this into our recurrent neural network as normal. So let's do a very simple example. Suppose we had the sentence I like cats. This is a sequence of length 3 because the sentence has three words. This might turn into the sequence of vectors 0 0 0 1 0 1 0 0 1 0 0 0. For this simple example we assume that our vocabulary only contains four words which of course is much smaller than a real English language dataset but there are some problems with this approach.

Some English language data sets have about 1 million possible tokens or words that would lead to an extremely large one hot encoded feature vector which means that your weight matrix would also be very large. So in other words you have a t by V matrix but V is a very big number.



	o_ENE	o_ESE	o_East	o_NE	o_NNE	o_NNW	o_NW	o_North	o_SE	o_SSE	o_SSW	o_SW	o_South	o_Variable	o_WSW
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
2	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
4	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
10	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0

FIGURE 25.6 | This Means V (or Equivalently, D) Is 1 Million

In actuality the English dictionary contains only about two hundred thousand words so it really depends on how you split up your words and where you obtain your vocabulary from. But this is still a significantly large number. Here's another problem with this approach. Remember that we would like our input features to have some structure. If you recall one of the earliest rules we encountered was machine learning is nothing but a geometry problem.

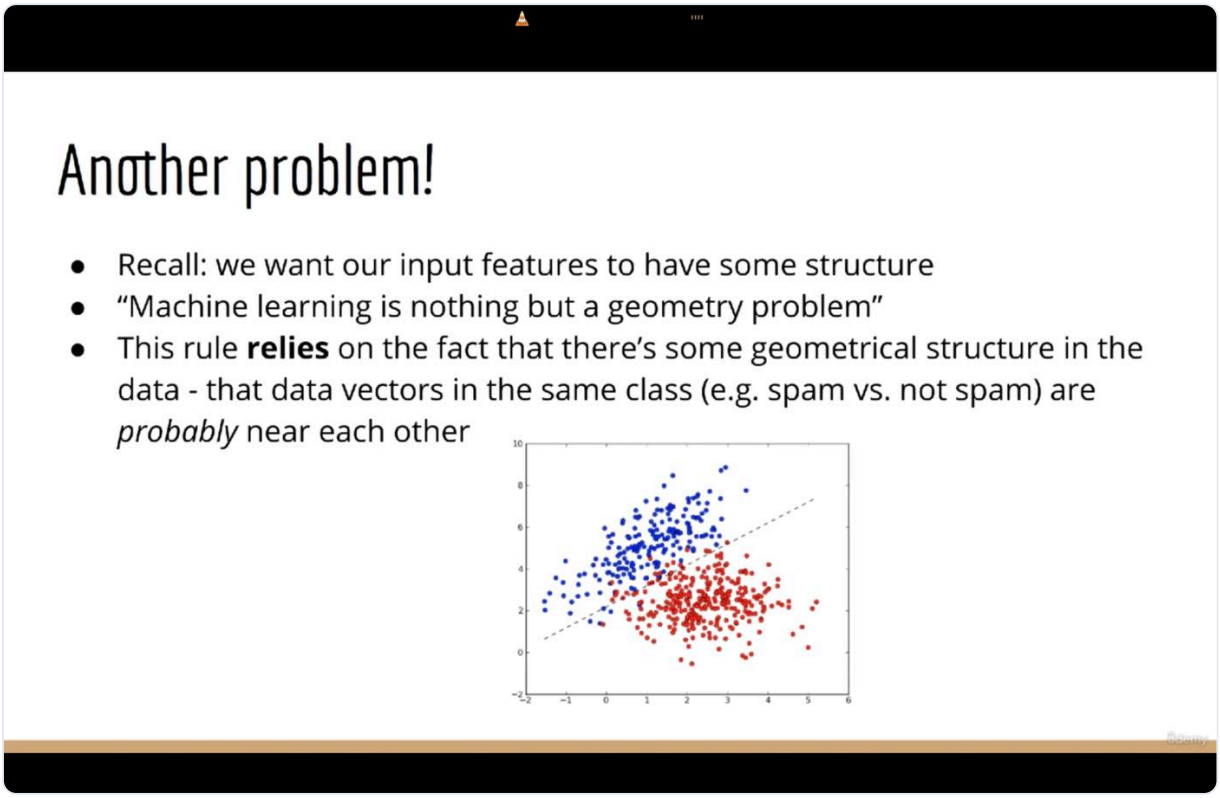


FIGURE 25.7

This rule relies on the fact that there's geometrical structure in our data. Data vectors from the same class are probably close to each other but what do one hot encoded feature vectors look like. Well if we take any to one hot encoded feature vectors and we calculate the Euclidean distance we will always get the square root of 1 squared plus 1 squared which is just the square root of 2. It doesn't matter which two words we select because they are all one hot encoded. For example the distance between Cat and feline would be square root of 2. But the distance between Cat and airplane would also be square root of 2.

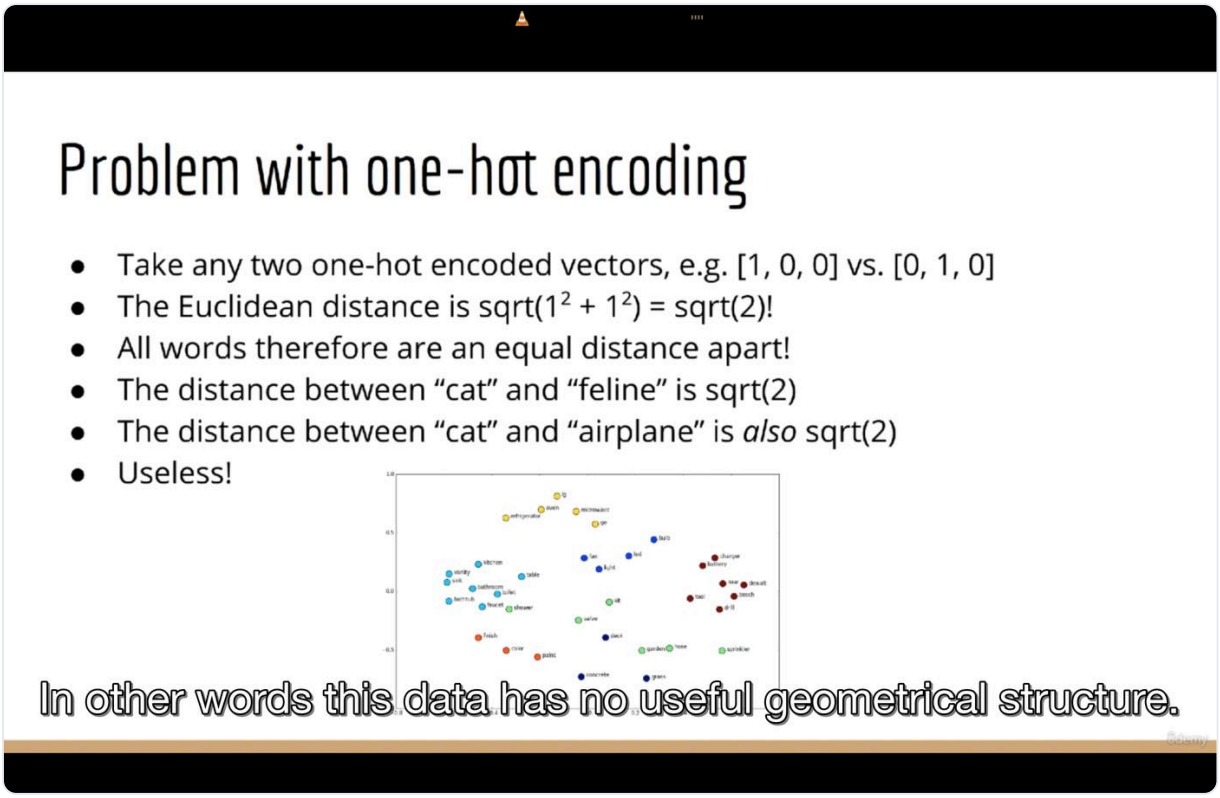


FIGURE 25.8 | Problem with One-hot Encoding

In other words this data has no useful geometrical structure. So what can we do instead.

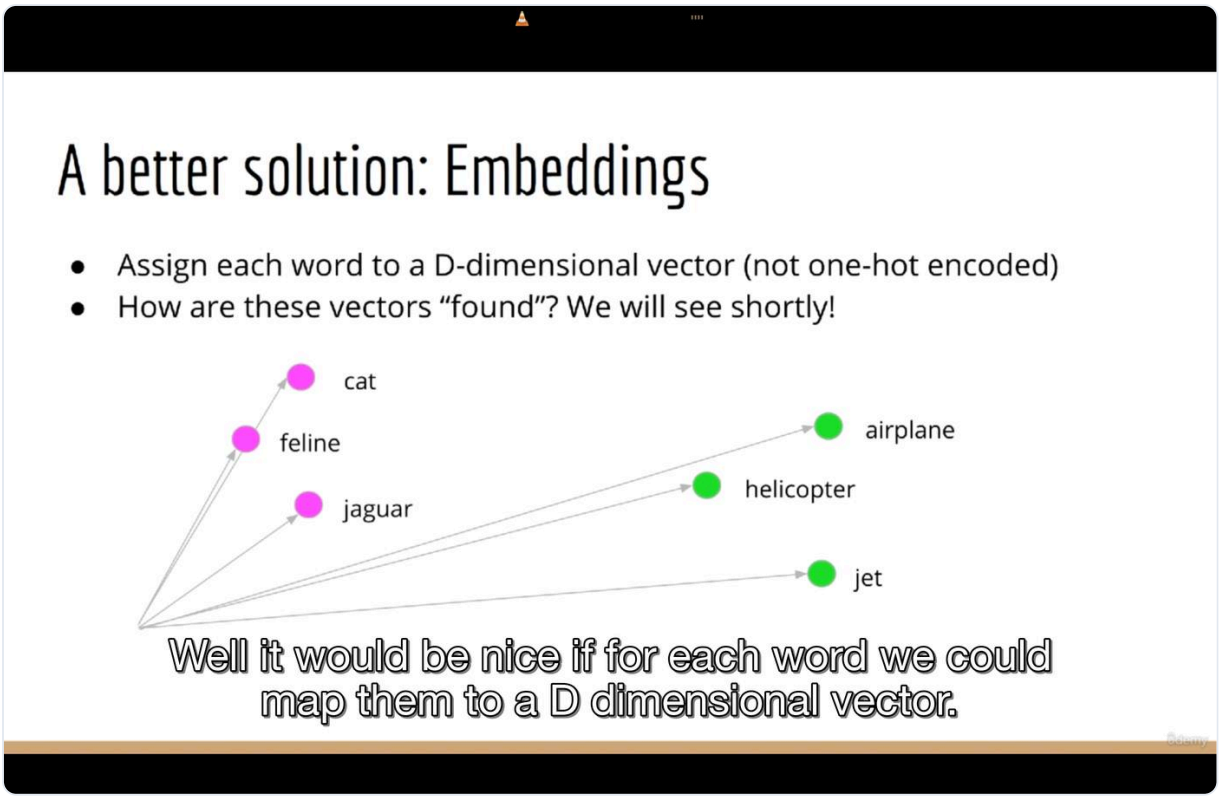


FIGURE 25.9 | A Better Solution: Embeddings

Well it would be nice if for each word we could map them to a D dimensional vector. Luckily we have just the tool for this. It's called an embedding layer. Now you might be wondering how can we actually make these vectors have a useful structure. We'll get back to that later.

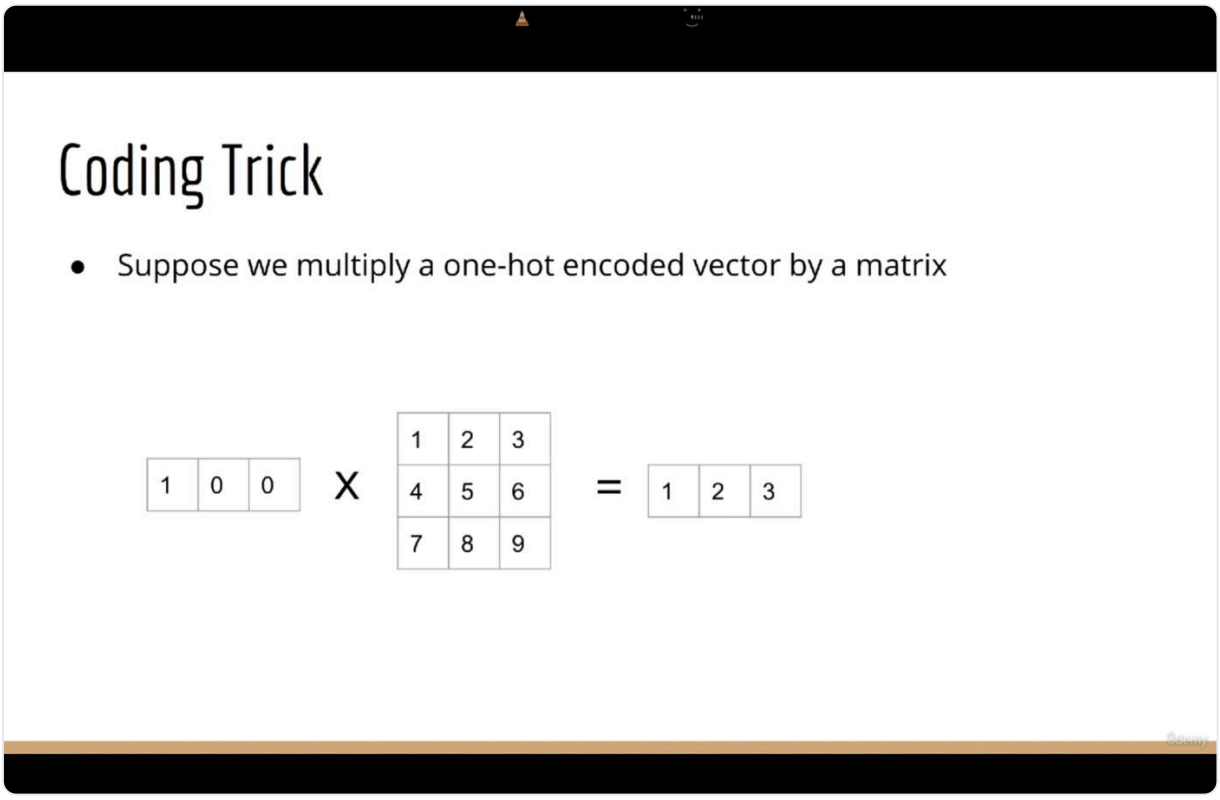


FIGURE 25.10 | Coding Trick

So stay tuned. But before we discuss that I want to show you a little coding trick so consider what happens if we multiply a 1 Hot encoded vector with a weight matrix. Let's suppose the vector is just three dimensional 1 0 0 and the weight matrix just contains the numbers 1 up to 9. You can verify for yourself that when we multiply this vector by this matrix we get the vector 1 2 3 now consider the one hot encoded vector 0 1 0. What happens when you multiply this vector by the same weight matrix.

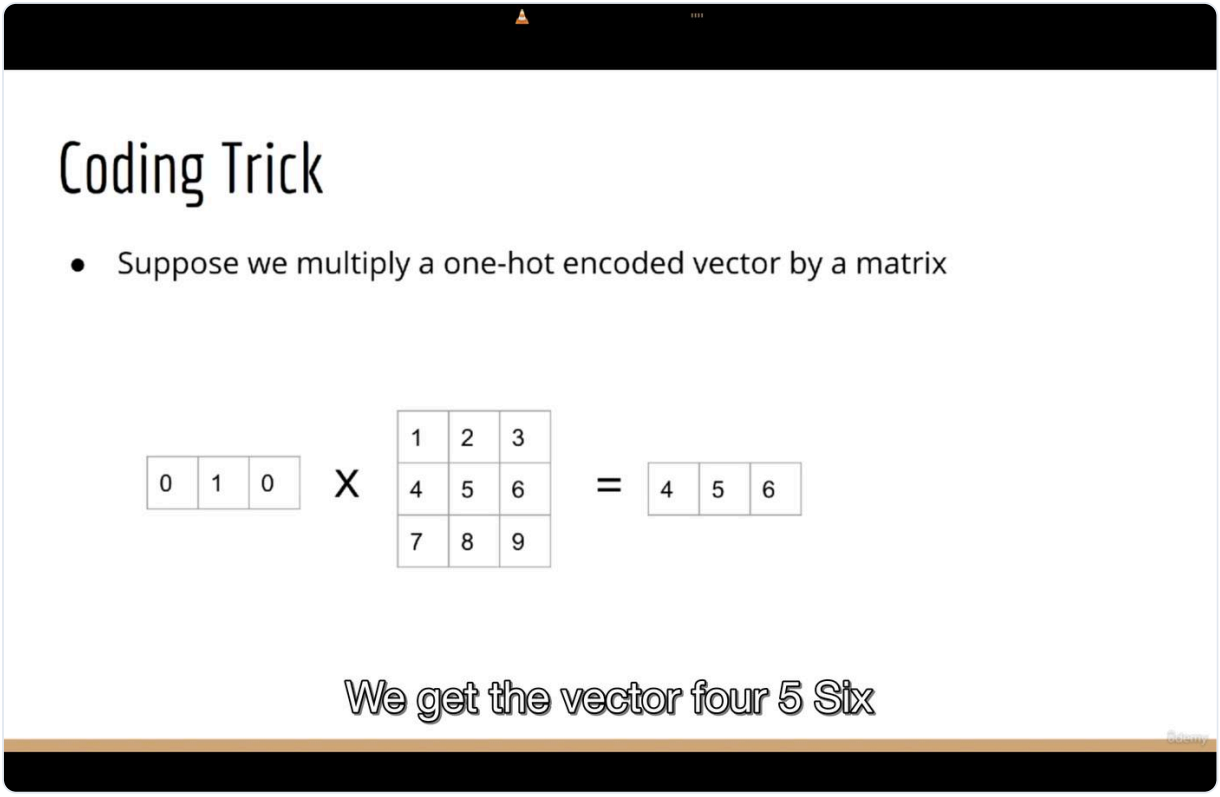


FIGURE 25.11 | Coding Trick

We get the vector four 5 Six now consider the 1 Hot encoded vector 0 0 1.

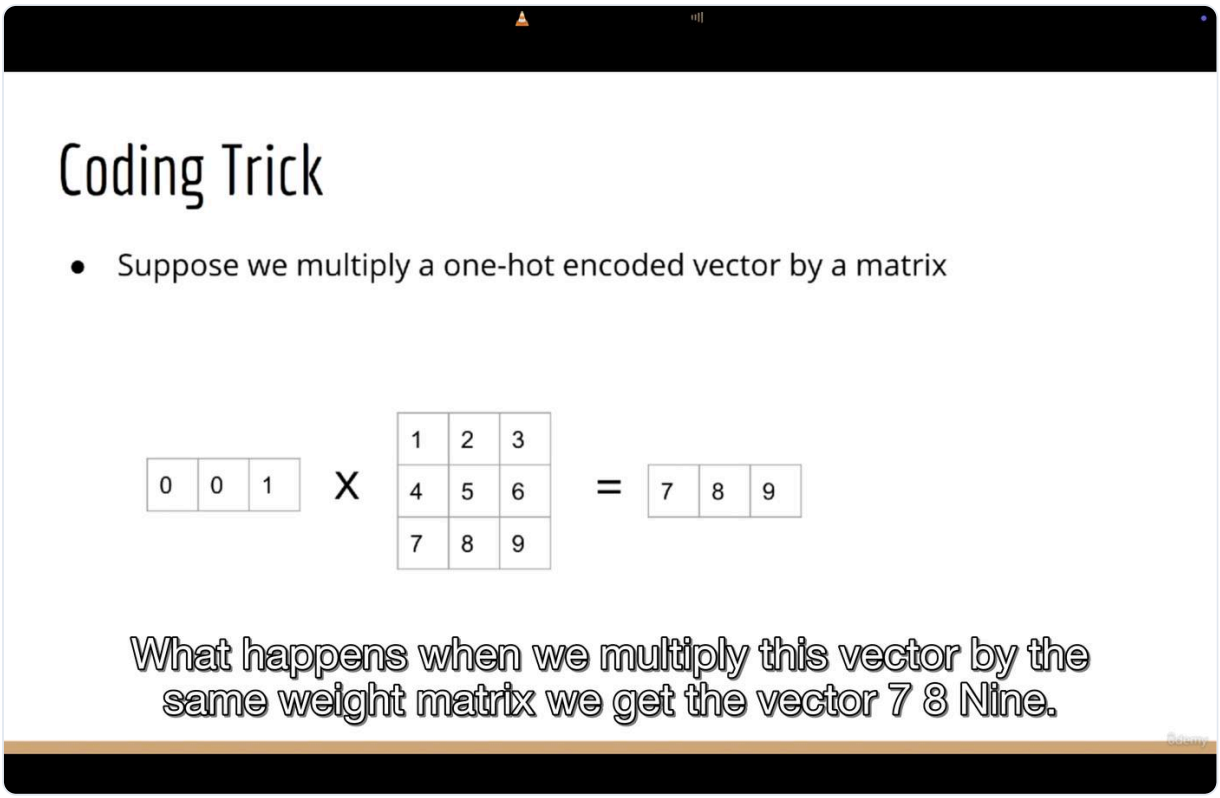


FIGURE 25.12 | Coding Trick

What happens when we multiply this vector by the same weight matrix we get the vector 7 8 Nine. So what is the pattern we see if the index in the one hot encoded vector which was set to 1 was 1. Then we get the first row of the weight matrix.

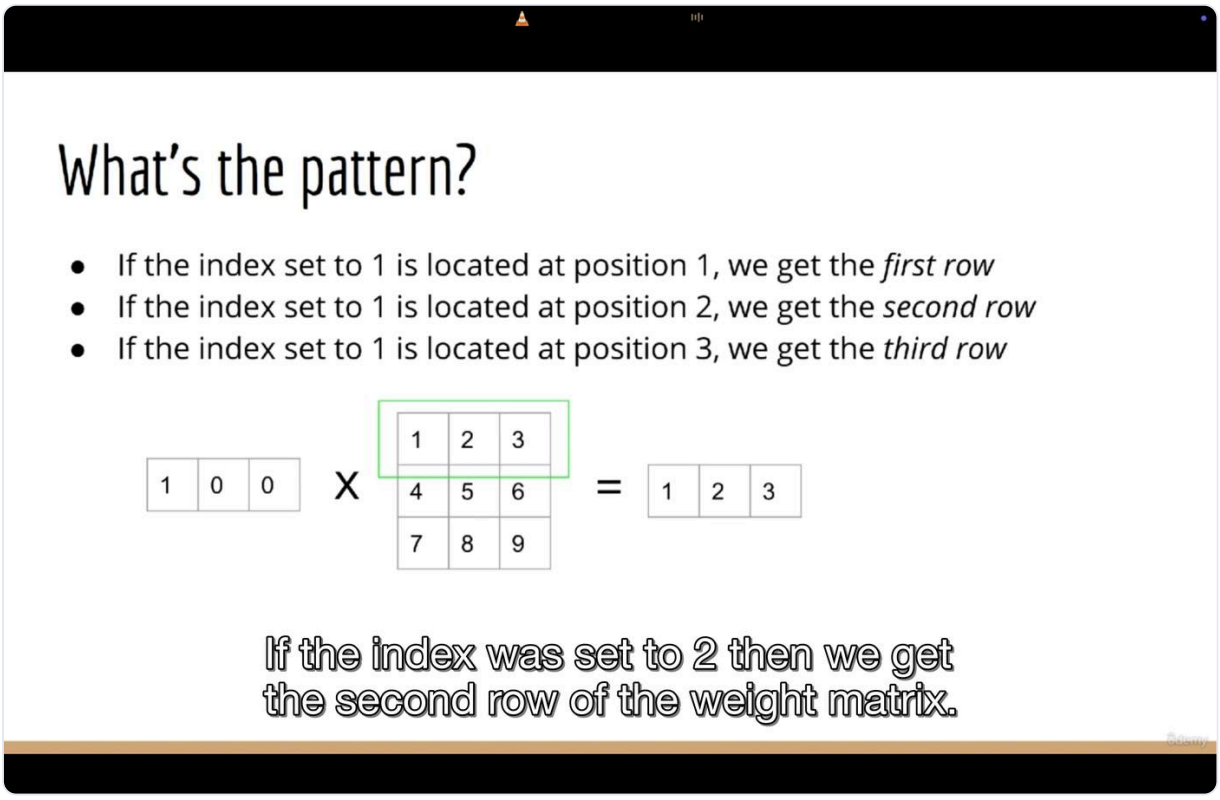


FIGURE 25.13

If the index was set to 2 then we get the second row of the weight matrix. If the index was set to 3 then we get the third row of the weight matrix.

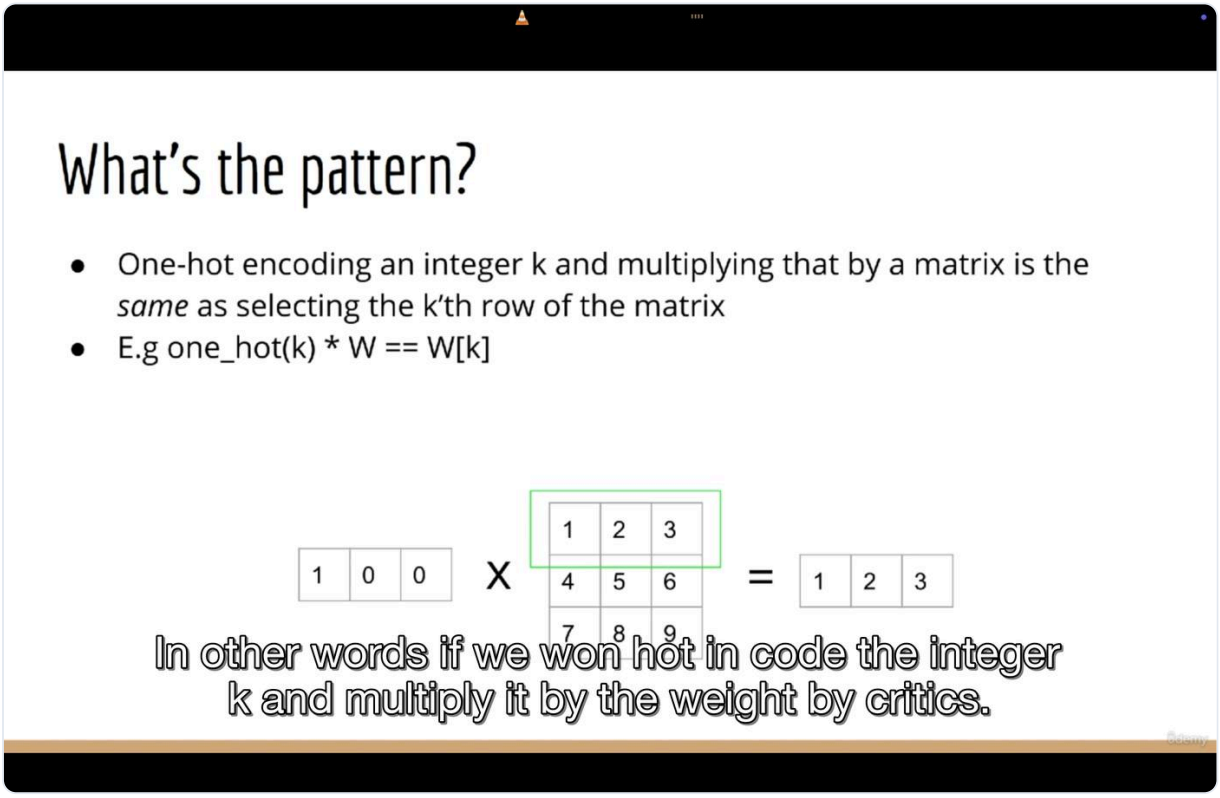


FIGURE 25.14

In other words if we one hot encode the integer k and multiply it by the weight matrix. All that is really doing is selecting the k th row of the weight matrix. So here's a shortcut. The old way of doing this took two steps. First we had to create a one hot encoded vector and set the k th entry to 1.

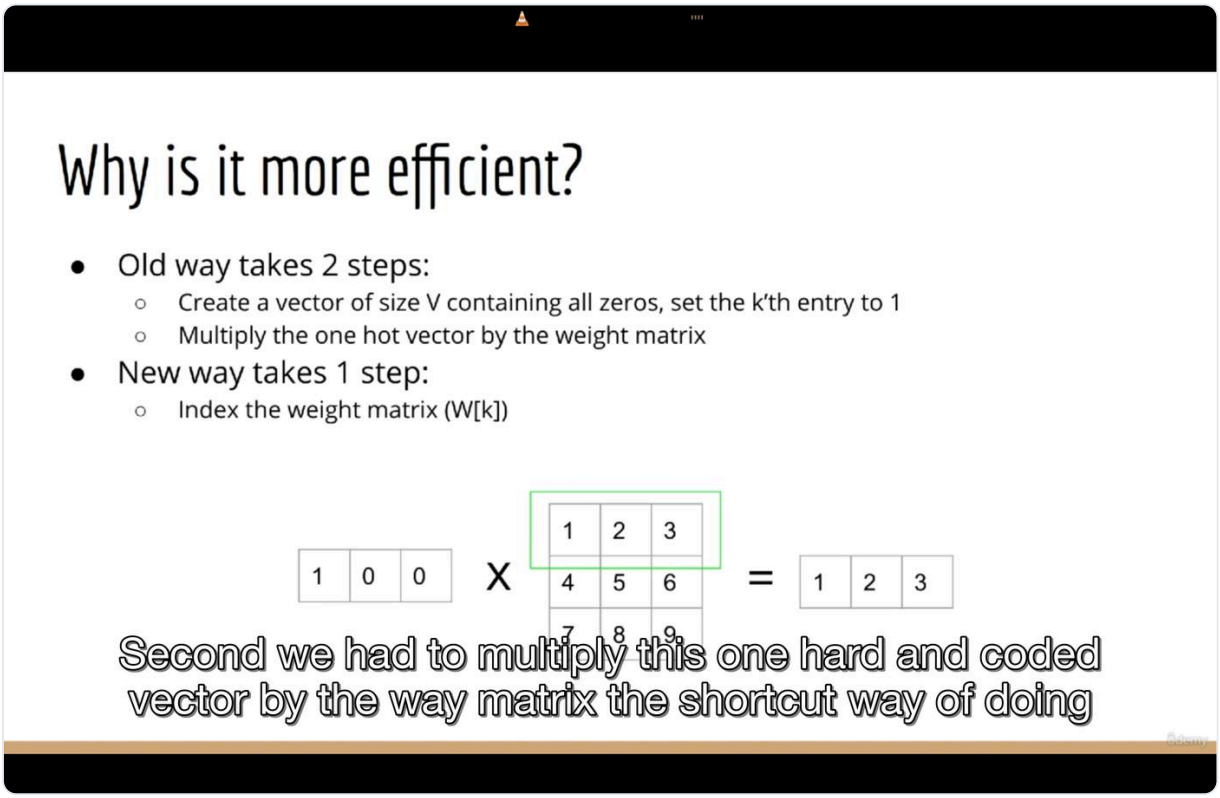


FIGURE 25.15 | Why Is It More Efficient?

Second we had to multiply this one hot encoded vector by the weight matrix the shortcut way of doing this takes only one step. We simply index the weight matrix at row k. This is obviously much more efficient than creating a one hot vector and then doing matrix multiplication. Think of it this way indexing an array is O of one that is constant time. But how long does it take to create a one hot vector and then do matrix multiplication.

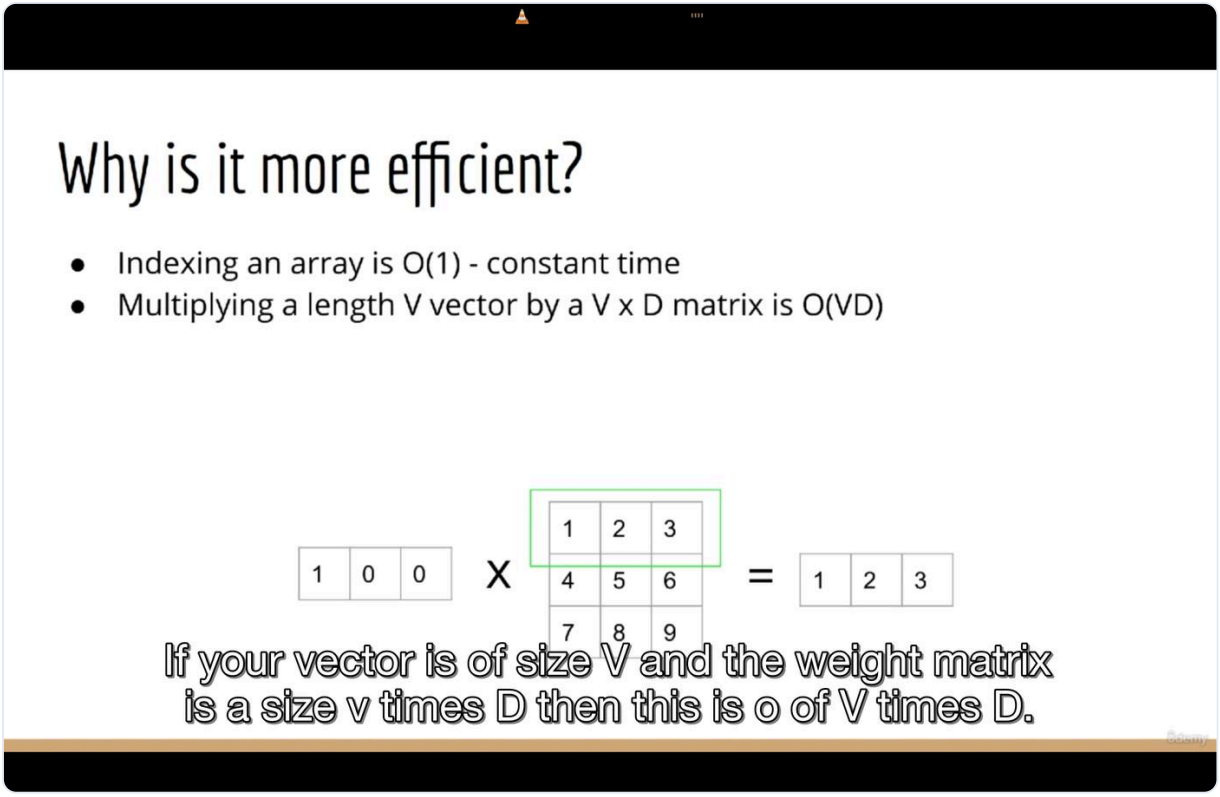


FIGURE 25.16 | Why Is It More Efficient?

If your vector is of size V and the weight matrix is a size v times D then this is o of V times D.

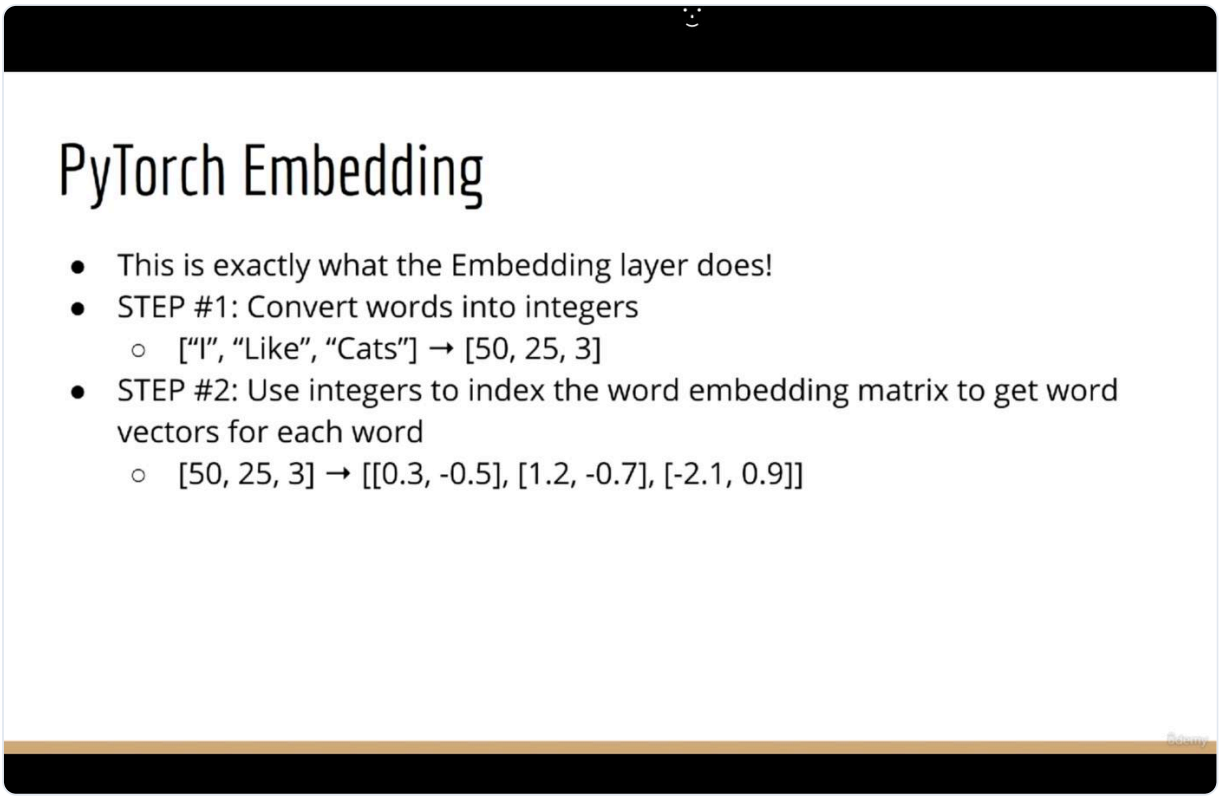


FIGURE 25.17

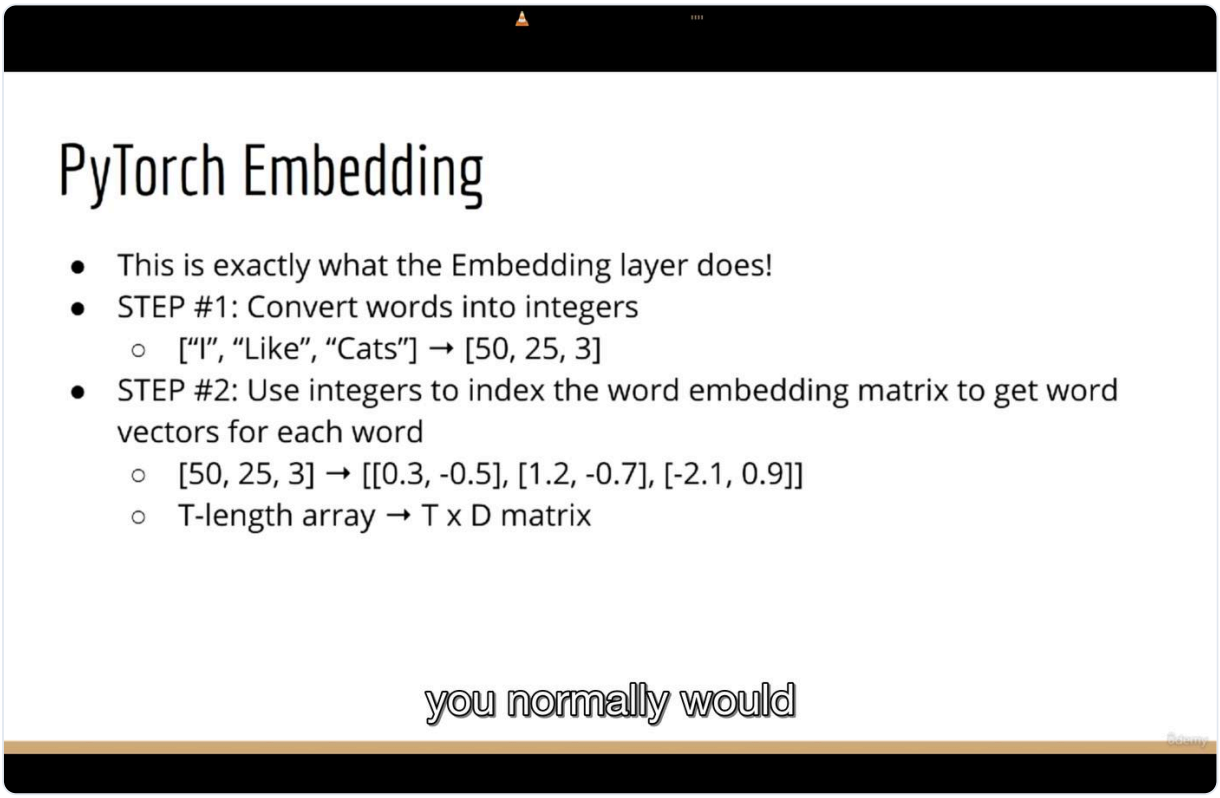


FIGURE 25.18

So this is exactly what the embedding layer in PyTorch does. Instead of doing all the work yourself to create one hot encoded vectors the only step you need to do is map your data set with sequences of words into a data set with sequences of word indices. Now that you have only integers you can use an embedding layer to map each word's integer to a corresponding word vector. From there on out your se-

quence becomes a T by D matrix at which point you can use an RNN as you normally would Conceptually you can think of the process like this. First we have some sentence a sequence of words let's say for simplicity's sake it's I like cats. Then this sequence of words becomes a sequence of integers.

Obviously it doesn't matter which integer we assign to each word as long as they are unique. This is just like when we are doing classification. It doesn't matter if we set dog to 1 and cat to 0 or vice versa.

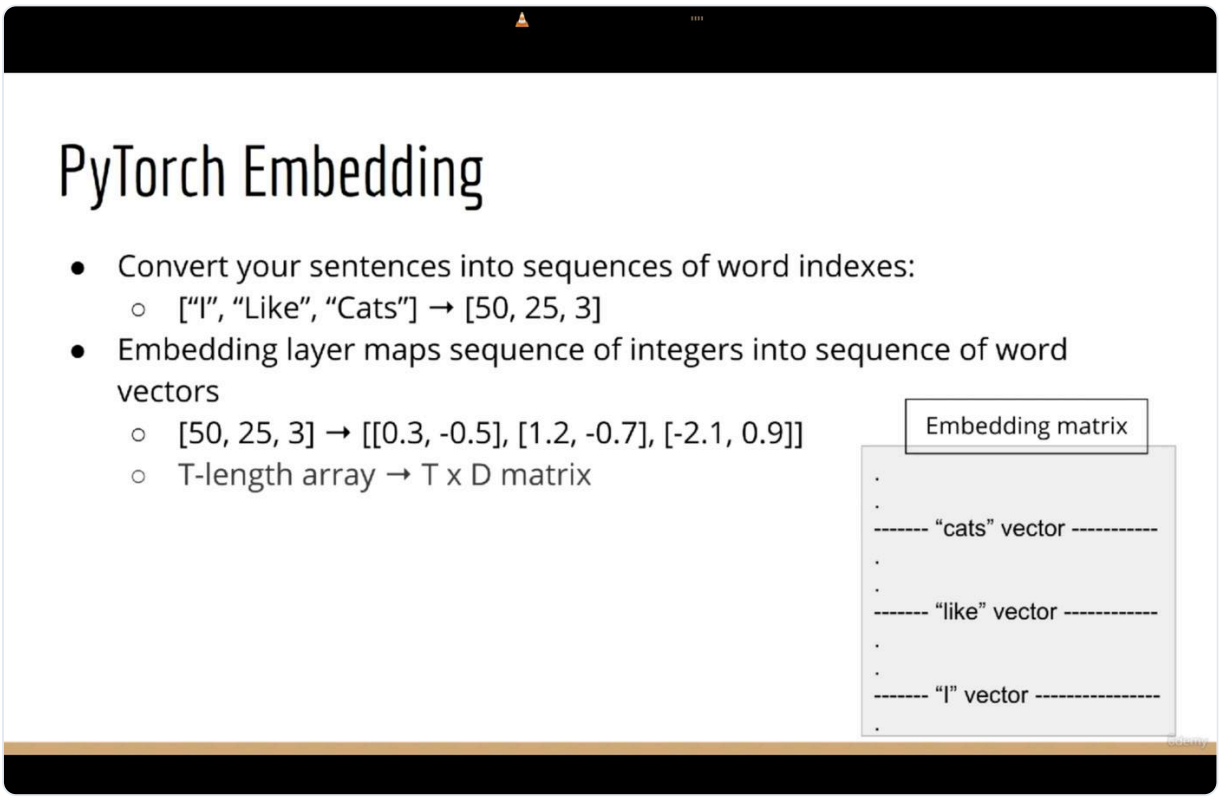


FIGURE 25.19

They are just arbitrary assignments. Finally we can use these integers to index an embedding matrix which will convert each integer into a word vector. So in the end what we have done is two steps. First we converted words to integers. Second we map those integers to vectors one question you might have at this point is we know intuitively that similar words should be closer together than dissimilar words. In other words these feature vectors should exist somewhere meaningful relative to each other. For example King should be close to Queen and car should be close to automobile but Queen should not be close to car and King should not be close to automobile so how can we make sure that that's the case.

Luckily this is the same story as with convolutional filters because they are just weights in a neural network. They all get trained automatically.

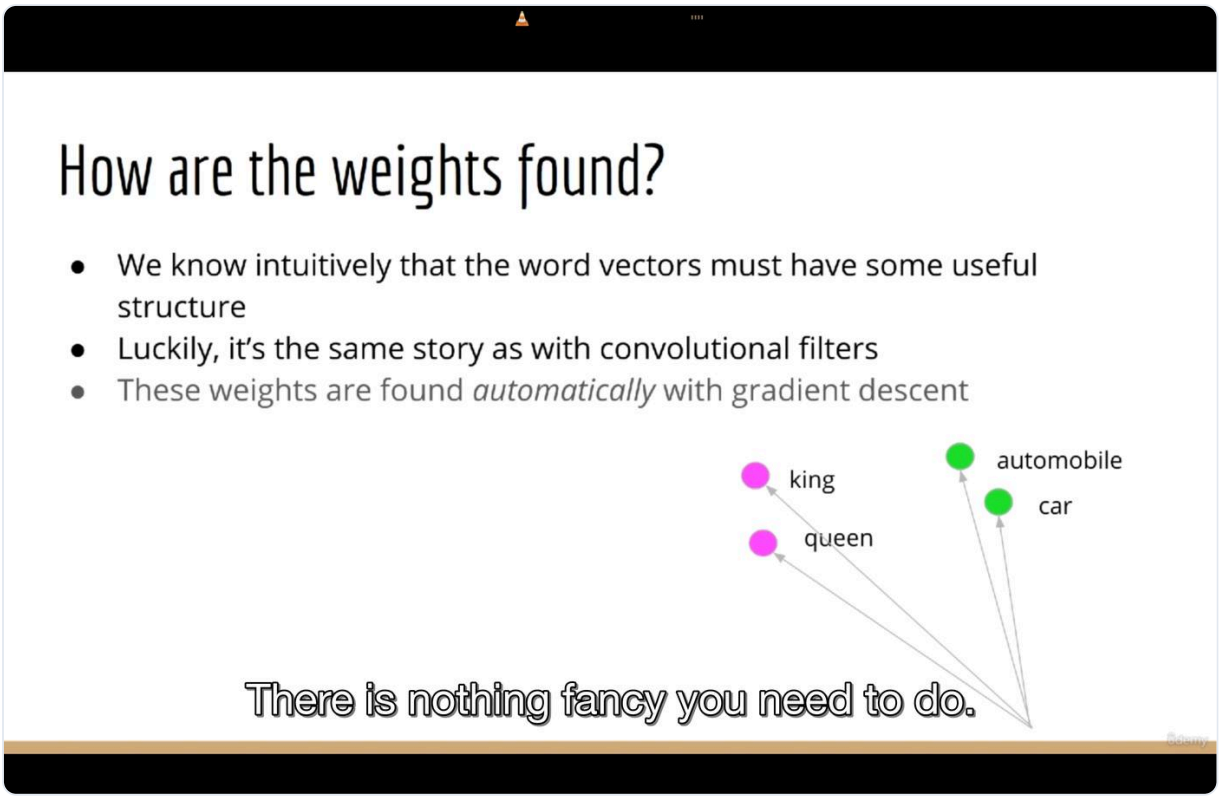


FIGURE 25.20 | How Are the Weights Found?

There is nothing fancy you need to do.

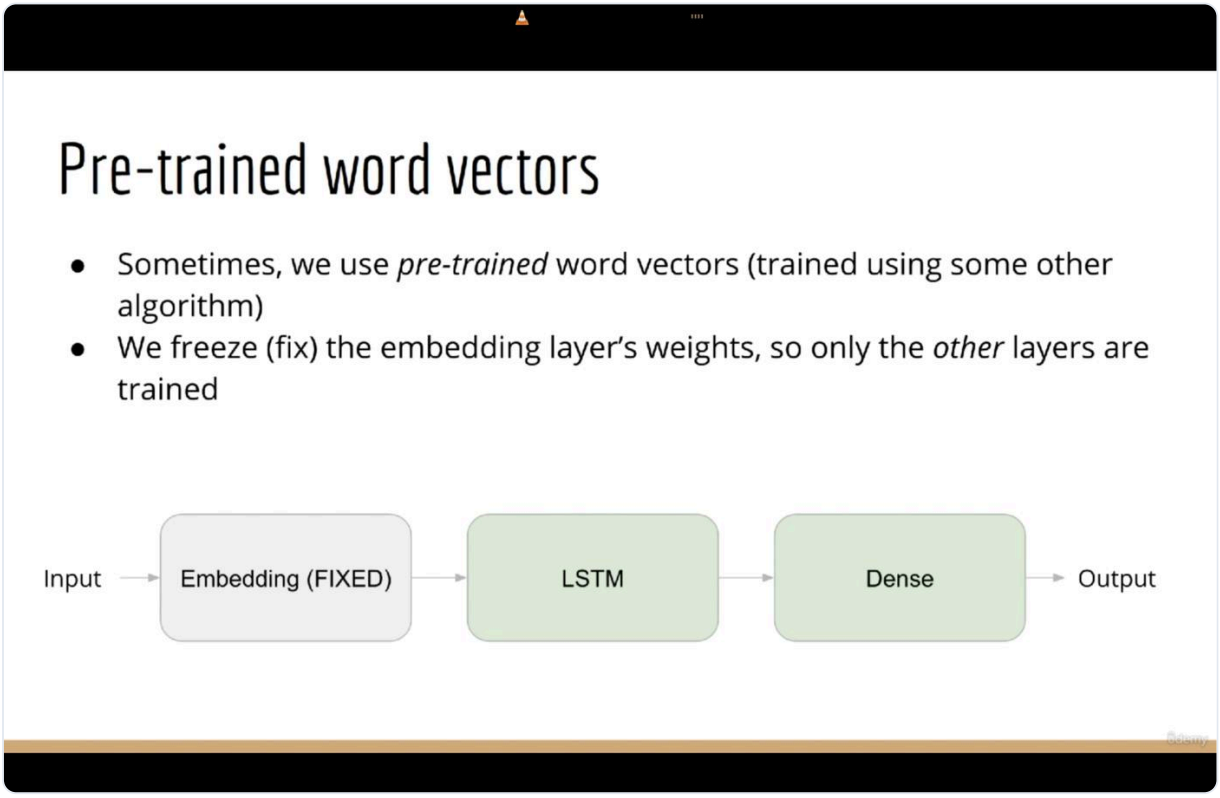


FIGURE 25.21

Now there's one caveat to that which is that what people often do is use pre trained word vectors. How this works is when they create an embedding layer. They set the weights to a set of pre trained vectors that were obtained through some other method then they freeze this layer so that those weights are

never changed. Typically these word vectors are found through algorithms such as word2vec and GloVe. Now that is a deep topic in itself.

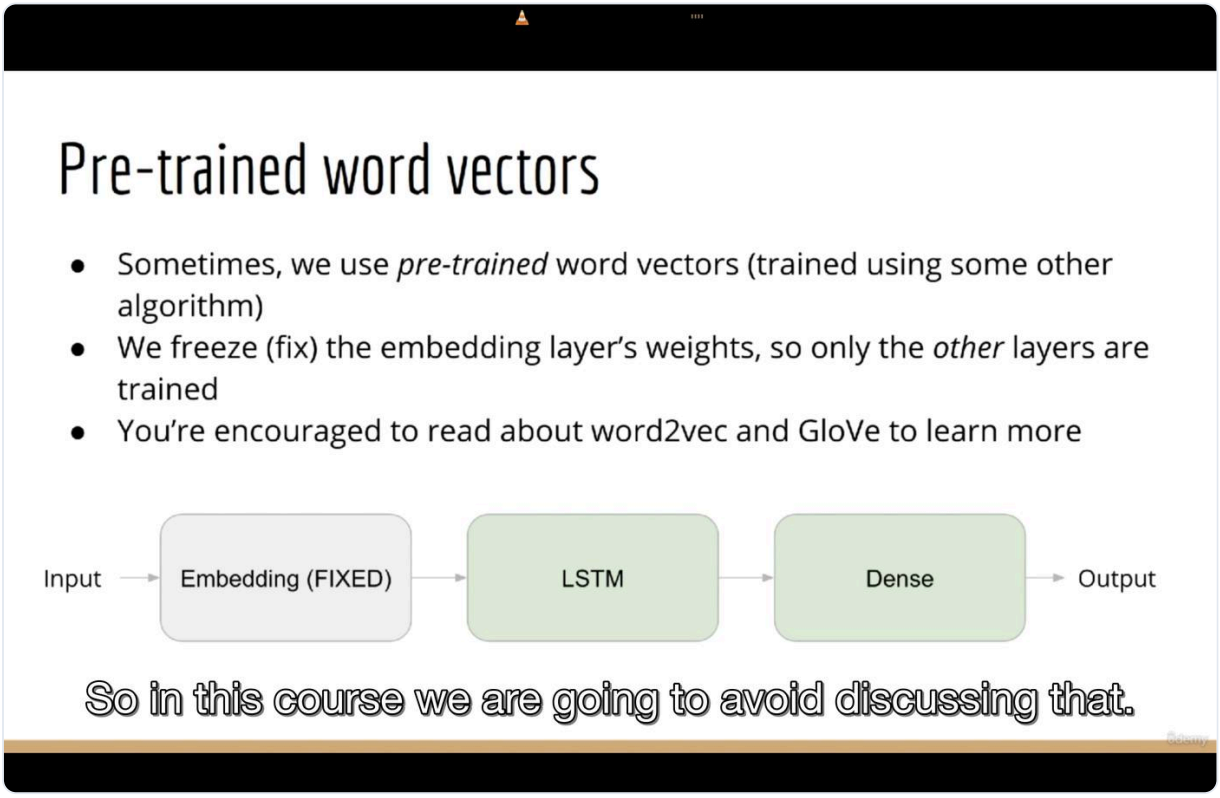


FIGURE 25.22

So in this course we are going to avoid discussing that. Instead we will train embeddings like any other layer by letting gradient descent do its magic if you want. You're encouraged to read the word2vec and GloVe papers since they are generally pretty accessible. To summarize this lecture here's what we discussed.

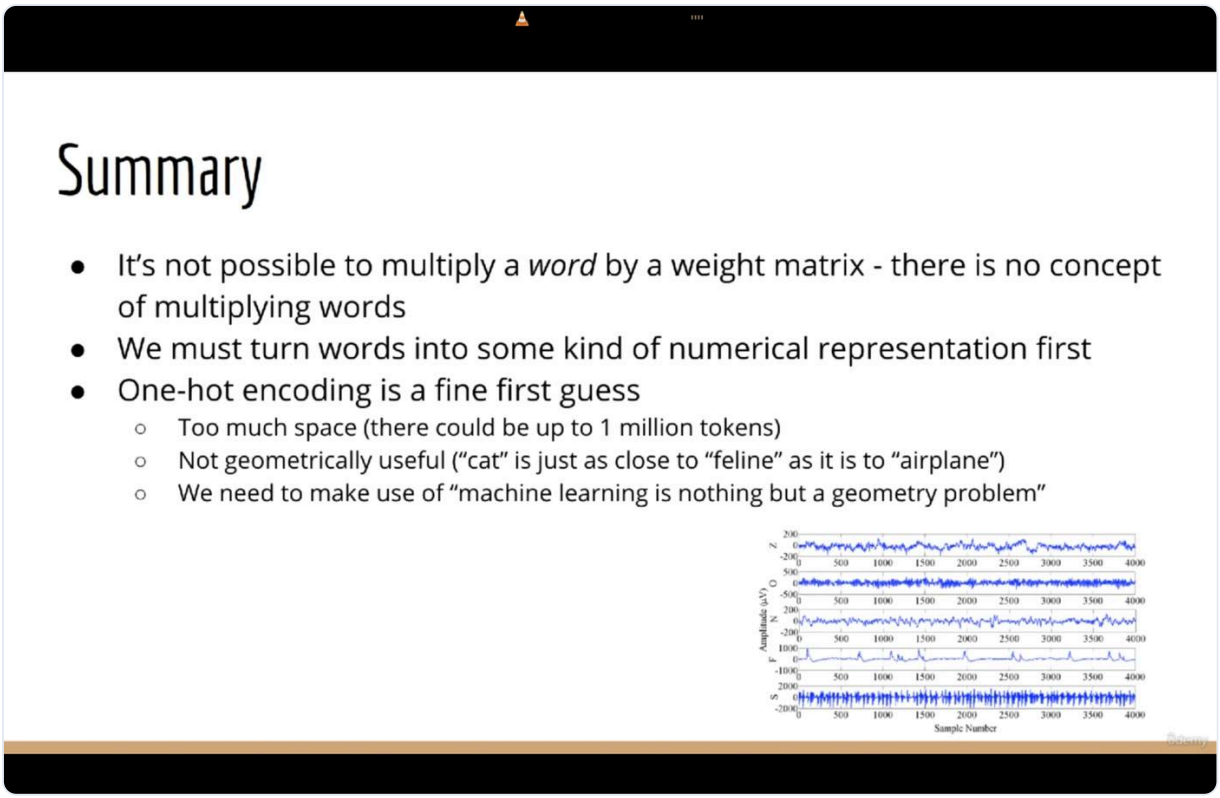


FIGURE 25.23

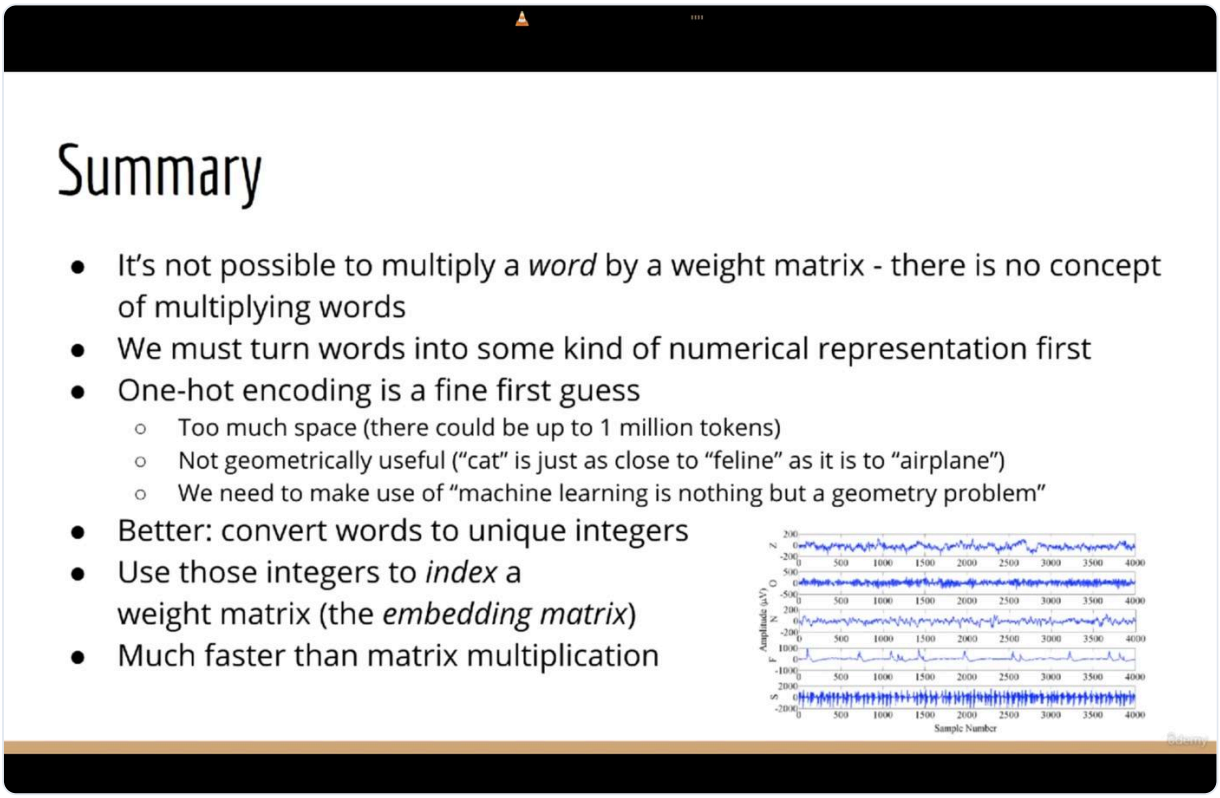


FIGURE 25.24

First we recognize that it is not possible to multiply a word by a weight matrix. There is no concept of multiplying words. Therefore we needed to somehow convert words into a numerical representation we first thought about one hot encoding each word. But we soon realized why that would not be a good idea. First of all it would take up way too much space. There were around 1 million possible tokens in the English language so each word would take up 1 million numbers to represent which is quite silly.

Also these one hot encoded vectors are actually meaningless in terms of being feature vectors. This is because each word vector if it's just a one hot encoded vector would be equal distances apart so cat is the same distance to feline as it is to airplane.

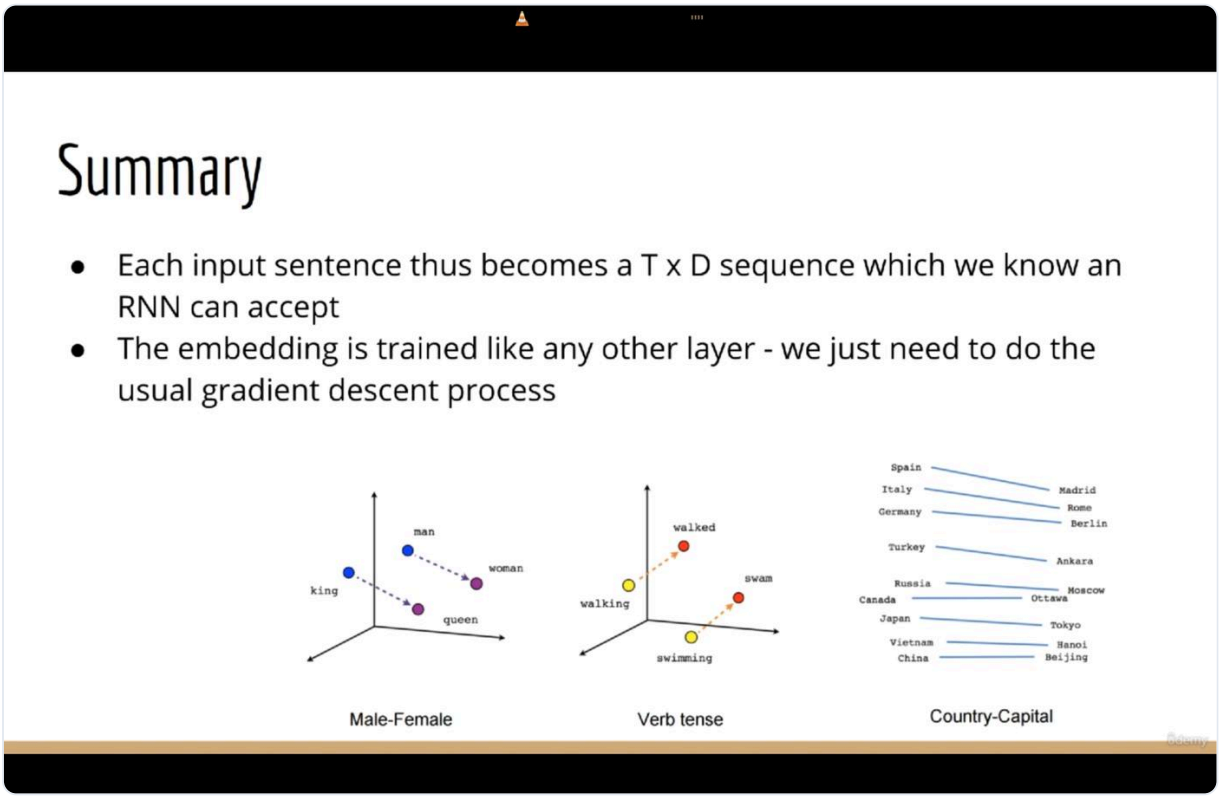


FIGURE 25.25 | RNN Can Accept

So that's not very useful. It doesn't allow us to really make use of the rule. Machine learning is nothing but a geometry problem. And thus we need a better way to convert words into vectors. This method is to take each word turn it into an integer and then use that integer to index an embedding matrix. This is much faster than one hot encoding and then doing matrix multiplication in this way. An input sentence consists of words then becomes a T by D matrix of numbers which you know an RNN can readily accept. We saw that this layers parameters are trained just like any other layer. So nothing other than gradient descent needs to be done.

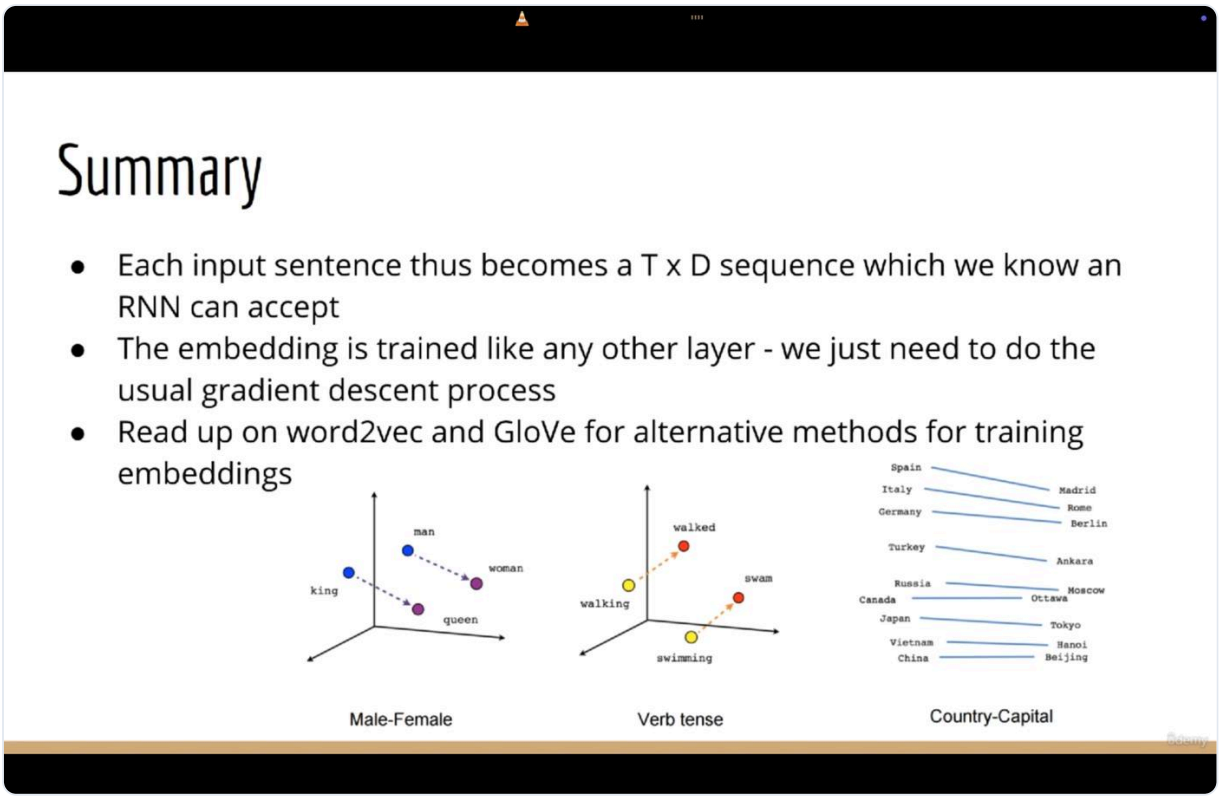


FIGURE 25.26 | RNN Can Accept

Alternatively if you were interested you're welcome to read up on word2vec and GloVe which provide different ways of initializing word embeddings so that you don't need to train them along with the rest of the neural network.

LECTURE 64

Forward Function

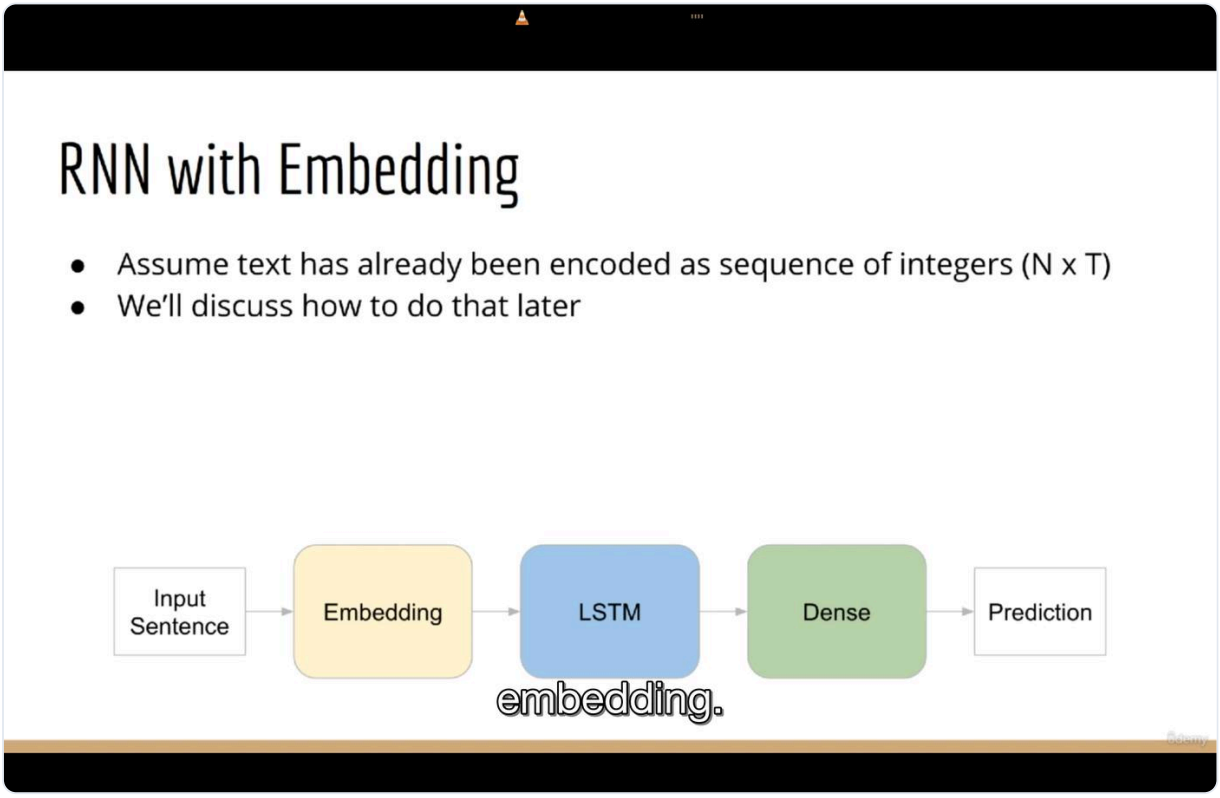


FIGURE 26.1 | RNN with Embedding

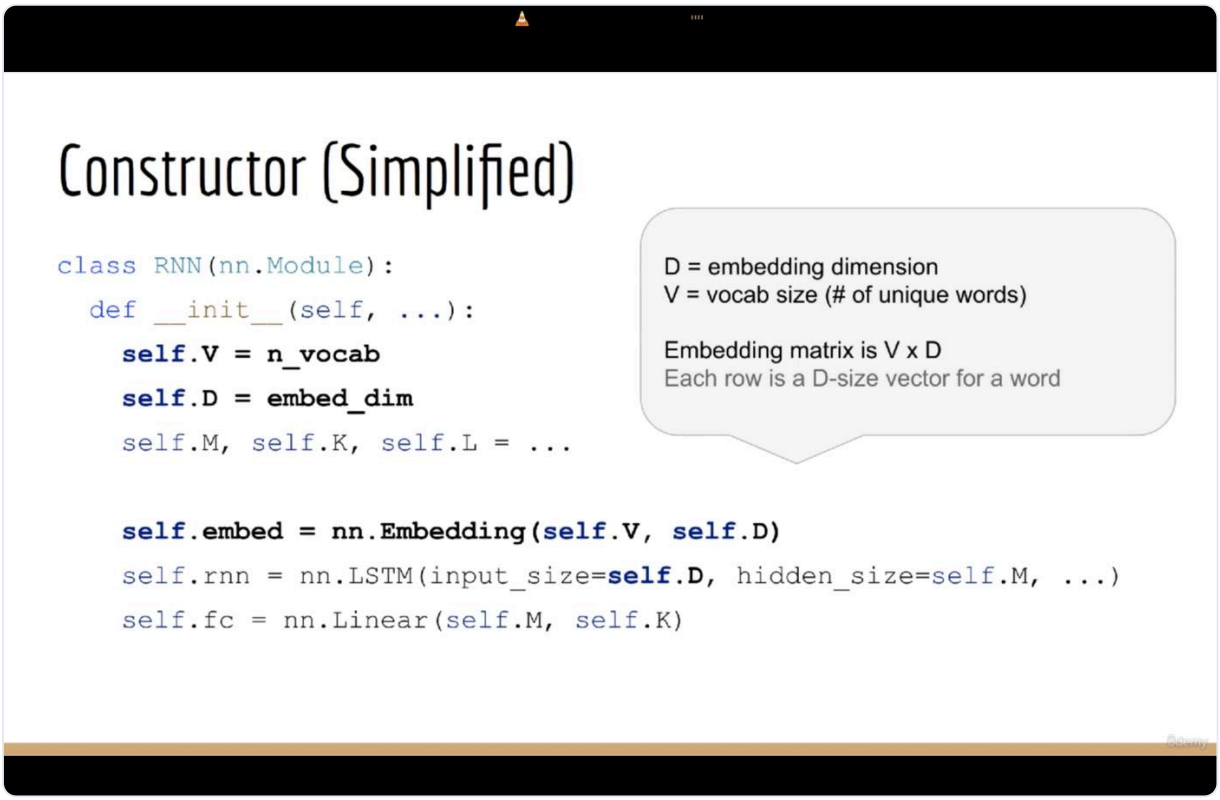


FIGURE 26.2 | Each Row Is a D-size Vector for a Word

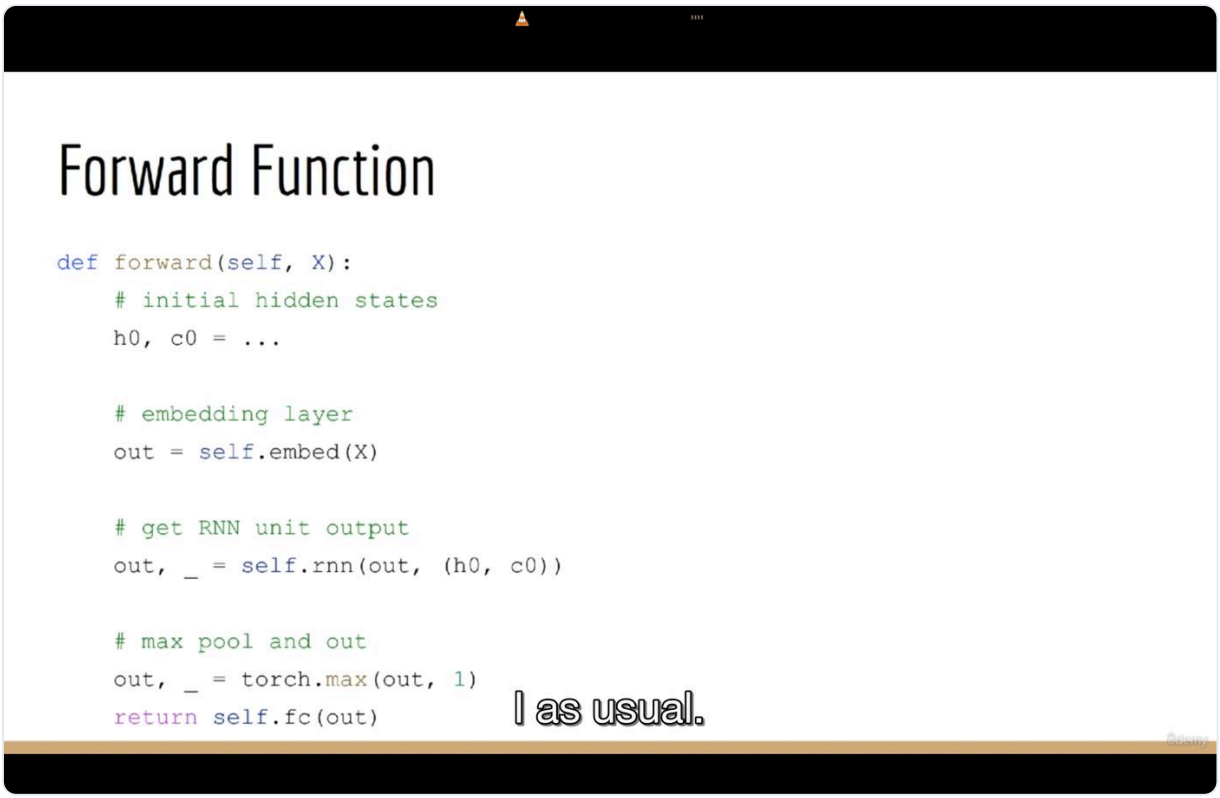


FIGURE 26.3 | Forward Function

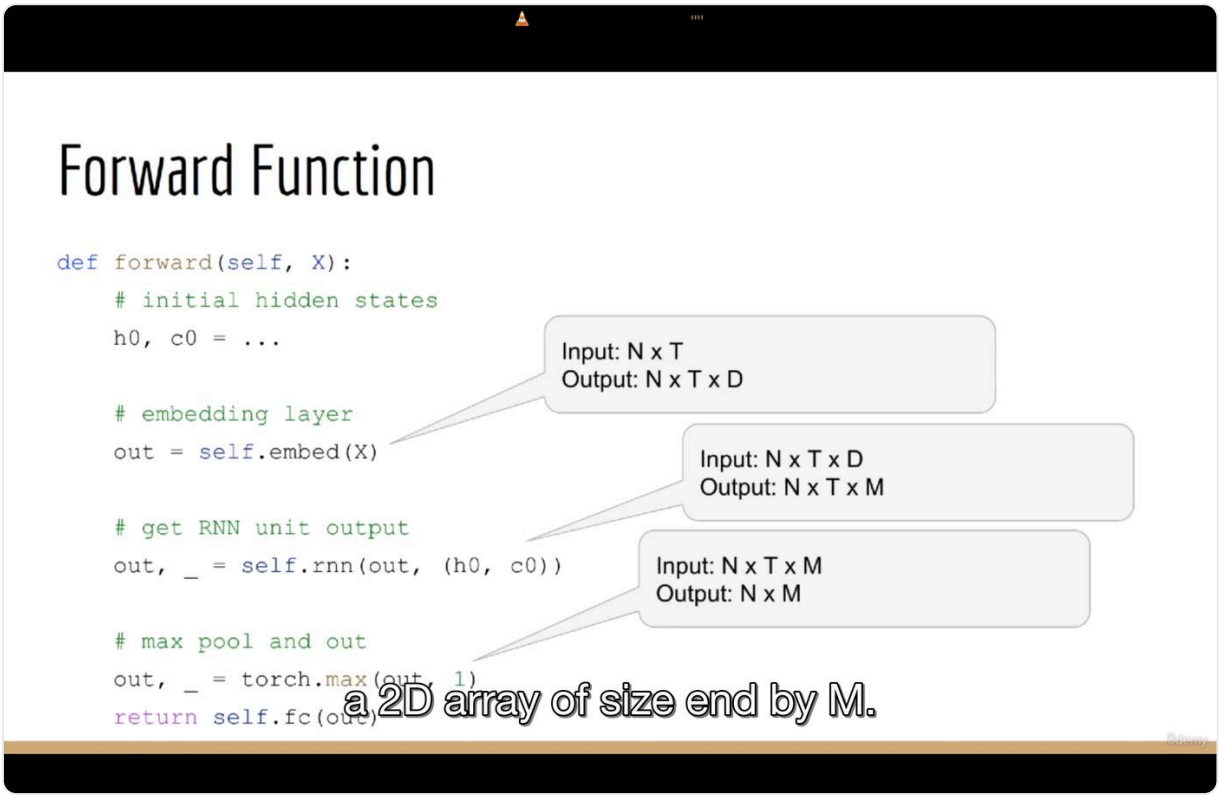


FIGURE 26.4 | Forward Function

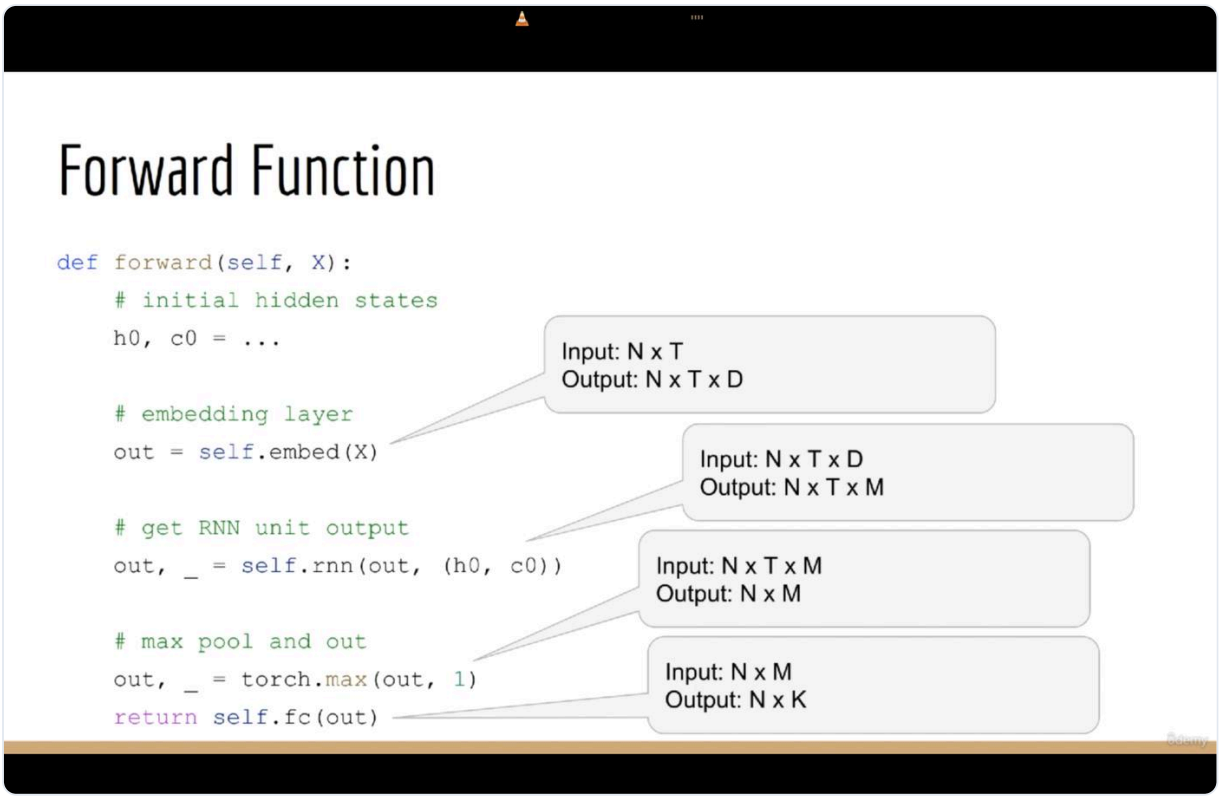


FIGURE 26.5 | Forward Function

In this lecture I'm going to show you the code for how to create an RNN with an embedding layer so that it can take text as input rather than a feature vectors as input to be more specific. We'll assume that this text is already encoded as a sequence of integers that can be used to index and embedding. We haven't discussed how to create such sequences of integers yet but I assume you have some idea in your mind what is cause how exactly that will work in a later lecture. But for now we'll just focus on the model itself. So let's start with the constructor as you might expect. It's pretty similar to a regular RNN but just with one additional embedding layer because we have this additional embedding layer we'll also need additional hyperparameters specifically the size of the embedding dimension which we will denote as D as a hyperparameter.

Do we want the word vectors to be of length 20 or length 50 or maybe length 300. This is a hyperparameter choice to be made by you. We'll also need to know the size of the vocabulary which will denote as V. So why do we need to know V and D well this specifies the size of the embedding matrix which will be of shape V by D. It has V rows and D columns. Why is that is because each row gives us back a D size vector corresponding to a word. This is what we discussed in the previous lecture. So in total we have five different numbers that we pass into the constructor V which is the vocabulary size D which is the embedding dimension M which is the number of hidden units L which is the number of hidden layers and K which is the number of outputs.

Then we instantiate three modules the embedding module the LSTM module and the final linear module. We already know how the LSTM and linear layer work. So I assume this requires no further elaboration. Next we have the forward function in the forward function that we take in some kind of x and then we pass it through each of our layers that we defined earlier since our model includes an LSTM. We know that we need to instantiate an initial hidden state h naught and an initial cell state c naught as usual. The important thing to pay attention to is shapes. First what is the shape of X that we expect as input since each sentence is a sequence of word indexes which are integers.

We expect X to be an array of shape N by T containing integers. Each row is a sequence of length T and each of the T columns contains a word index after passing this through an embedding layer. It is going to map each word index to a D sized vector. Therefore we get back an N by T by D tensor. We should recognize this as the usual shape that we pass through an RNN therefore after this. Nothing else really changes when we pass in an N by T by D through an LSTM we get back an array of size N by T by M if we do a global max pool or we simply take the final hidden state we get back a 2D array of size N by M. Finally after passing this through the final dense layer we get back an array of size N by K.

LECTURE 65

Multiple Words to Single Words (Tokenization)

In this lecture I want to introduce you to the concepts behind text preprocessing in the previous lecture we learn that obviously you can't multiply words by numbers which doesn't make any sense. In order to use deep learning which really means neural networks we must have numbers because a neural network is basically just a series of multiplications and additions. So the question now becomes how do I turn my text into numbers specifically. We need the kind of numbers that can be passed into an embedding layer which means we need integers that correspond to rows in an embedding matrix let's start from basic and hopefully pretty obvious points. First how is text stored.

The answer is text files to be clear I mean in plain text files and not Microsoft Word or anything like that.

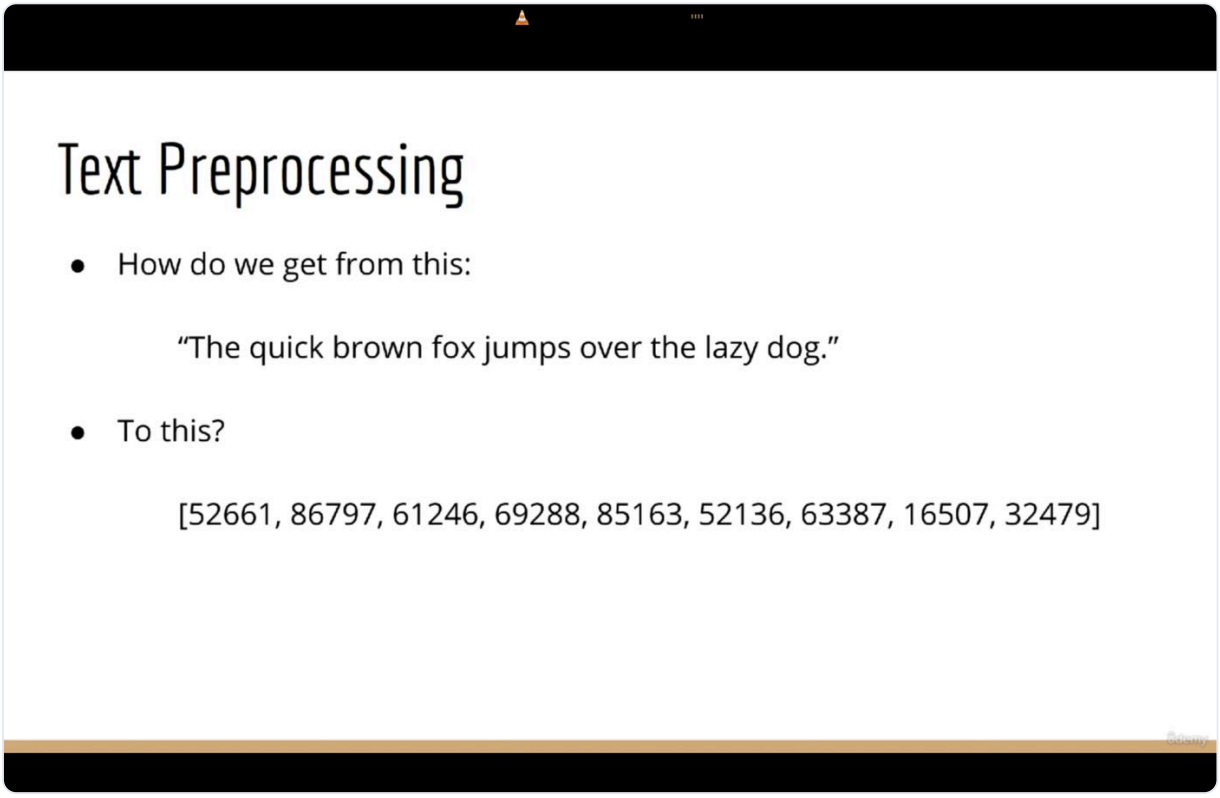


FIGURE 27.1 | Text Preprocessing

Hopefully you've done enough Python so that you know how to open a text file read its contents line by line and so forth. Hopefully you know that Python files are themselves text files so text files are a fundamental ingredient of computer programming. You should love them. OK so let's say I read in a plain text file using Python. How are those file contents represented in Python code. Well the answer is as a string. But wait.

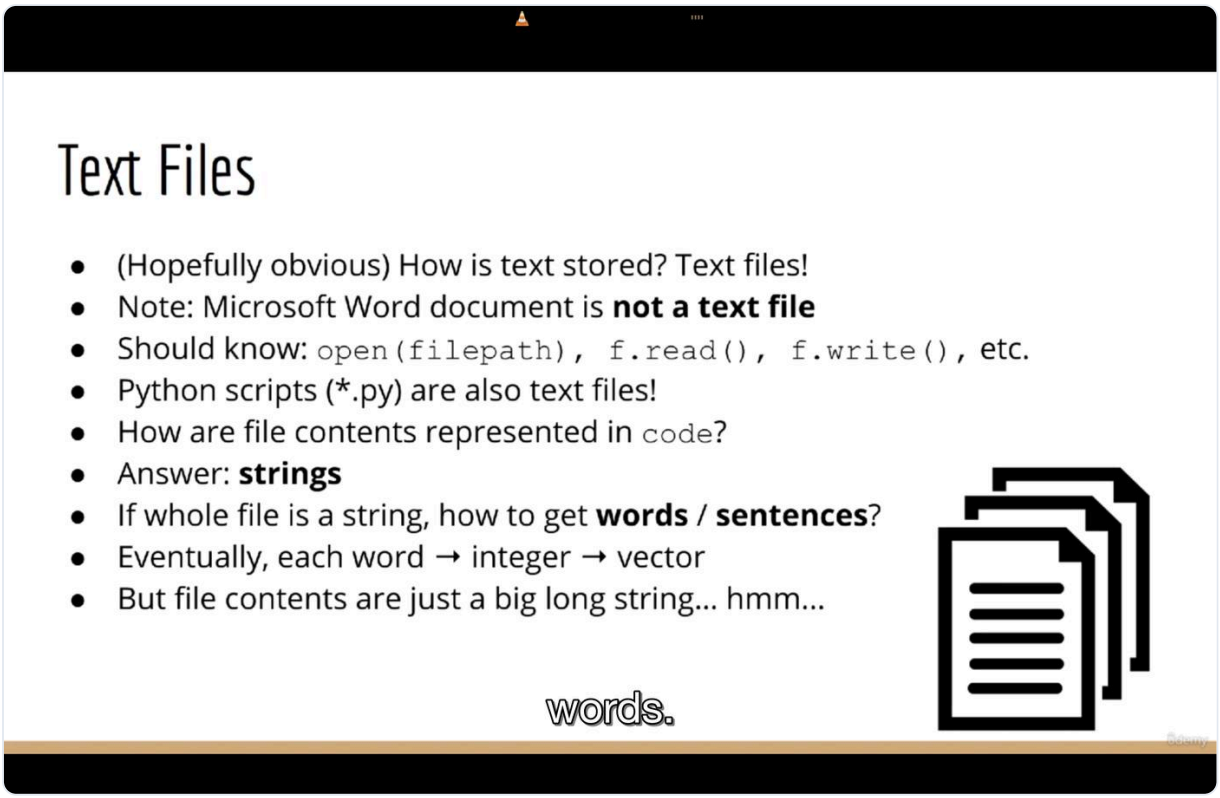


FIGURE 27.2 | Text Files

If the whole file is a string. How can I get the individual words and sentences. Remember that in order to convert my words into word vectors those words must be represented individually as integers but starting out. I have nothing but a humungous string full of multiple sentences and each sentence containing multiple words. No good OK. I'll make it a little easier on you. Suppose we impose some structure on our text file. Let's make it a CSV. We've all seen CSVs before. So I hope there's no need to explain what these are. Let's suppose since this is what we're going to see later anyway that this is a two column CSV where the first column is some string representing an email or a tweet that we want to classify.

And the second column is its corresponding label.

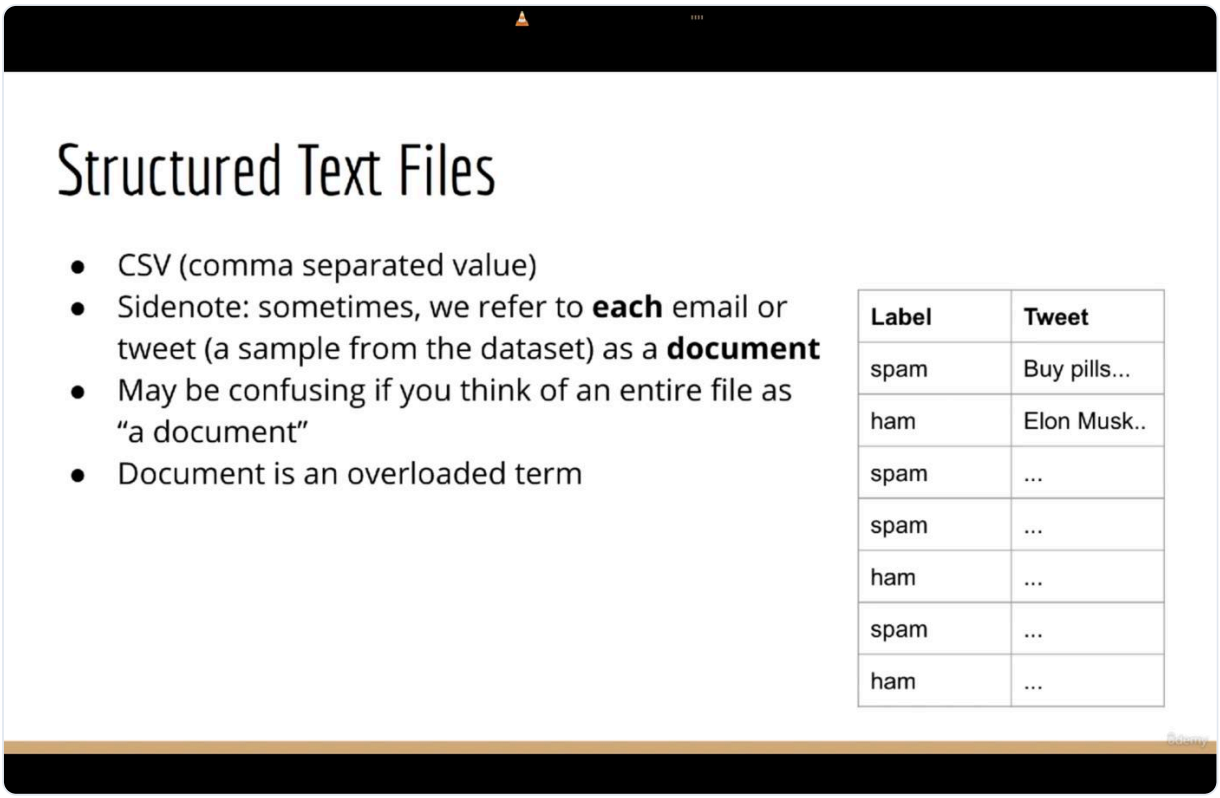


FIGURE 27.3

This makes things slightly easier because each line of the file consists of only two parts an email or a tweet and some corresponding label as a side note. Sometimes people including myself will refer to these emails or tweets simply as documents that may be confusing to you as a beginner because you might think of a document as an entire file. Like a book or a report. But in NLP. When we say document that probably just refers to a single sample which could be a sentence or maybe a few sentences. It could possibly be longer but that depends on the task you're doing so be aware that the word document is kind of an overloaded term in NLP. It doesn't mean a document as an entire file like a word document.

It just means a single sample in your dataset. So from here on out when I say the word document I'm going to assume you understand what I'm talking about. Now that our data is in the form of a CSV maybe we can use pandas to load it into a data frame. But the problem is each of the cells in the first column of this data frame are still documents they contain multiple words when what we really want is individual words in the next portion of this lecture.

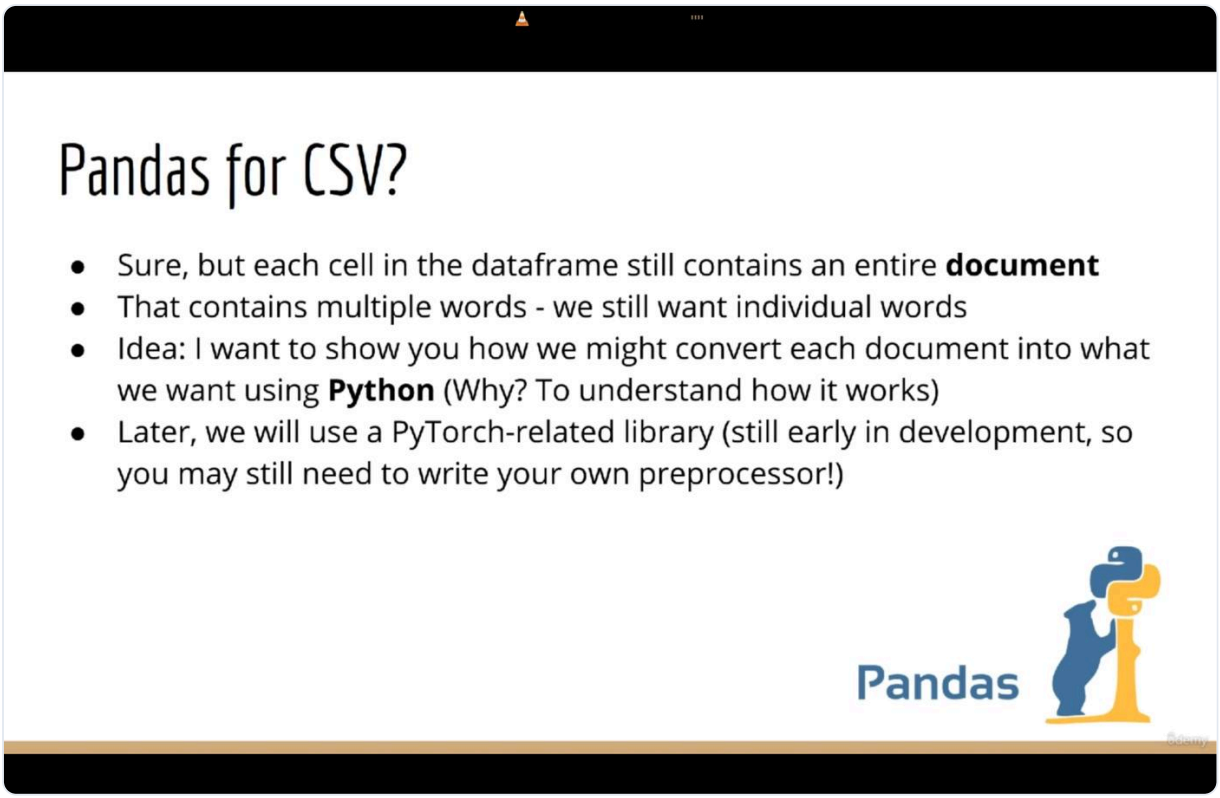


FIGURE 27.4

I want to discuss how we might process these documents in pure Python code to give you an understanding of how this works and how you might implement it yourself in the end we're going to end up using a library that's part of the PyTorch ecosystem but it's quite immature and limited in its capabilities so it's entirely likely that in the future it may not fit your needs and you may need to write your own text preprocessing code so it's good to have some idea of how this works in my more advanced and in-depth NLP courses. We actually go over how to do text preprocessing in Python. So if you're interested in getting practice with that you can ask me about it on the Q and A in any case let's start with what we have a document which contains multiple words and possibly multiple sentences.

We know that we want to split this up into individual words. This is a process called tokenization. If you wanted to take the simplest possible approach you could achieve this by taking your document which is a string and just calling the split function. This is probably fine and a lot of cases but there are some downsides to this too such as that. It doesn't take care of punctuation consider the document.

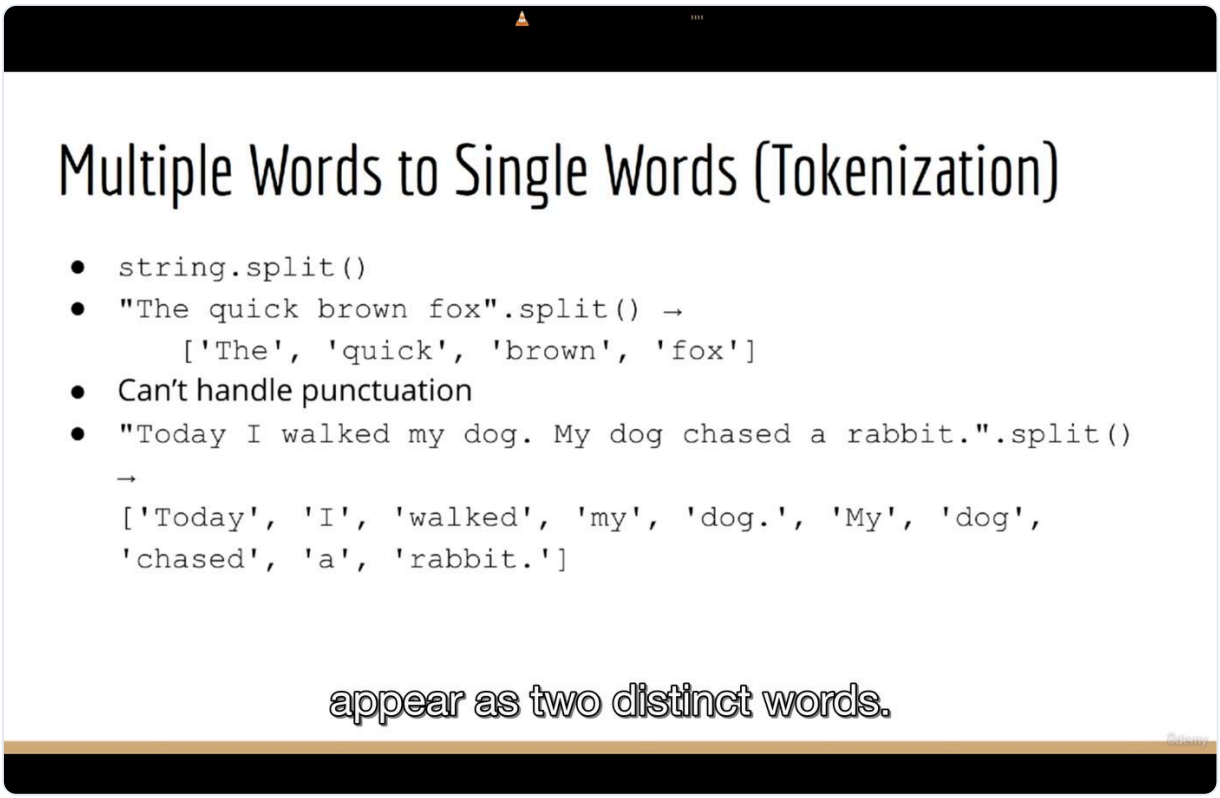


FIGURE 27.5 | Multiple Words to Single Words (Tokenization)

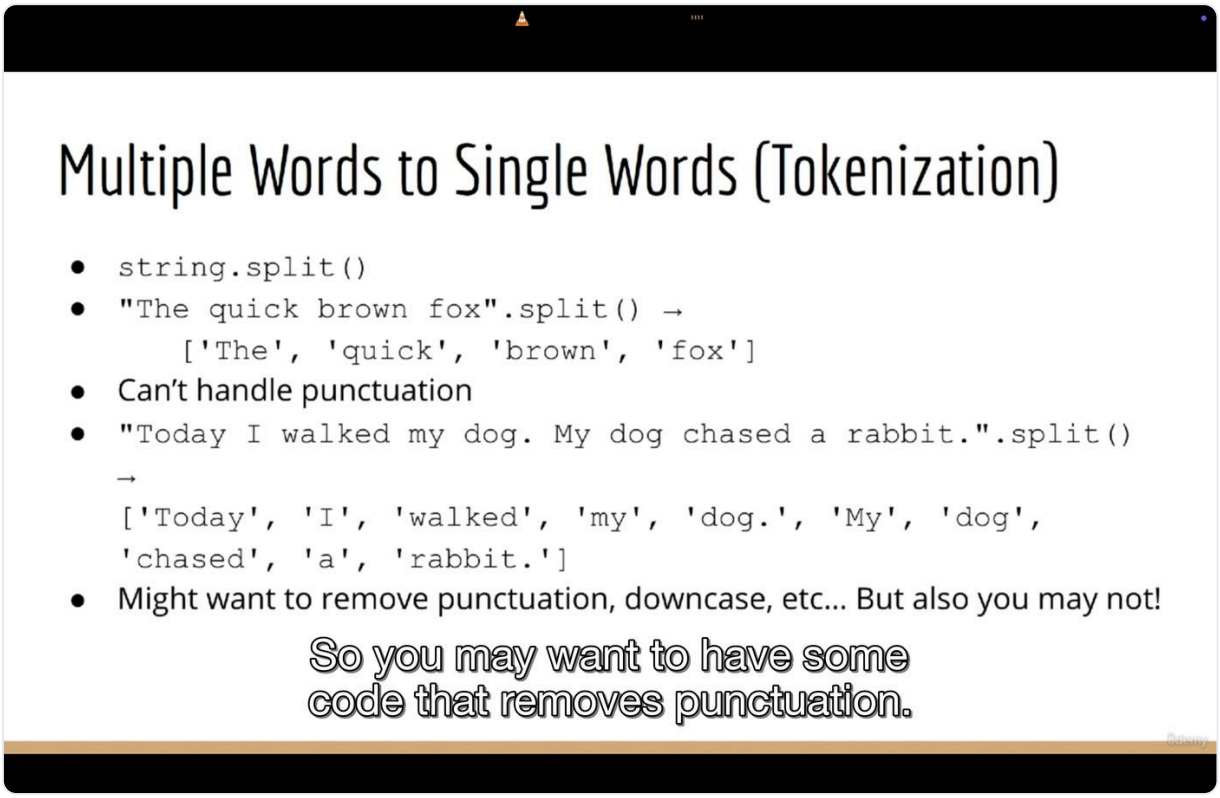


FIGURE 27.6 | Multiple Words to Single Words (Tokenization)

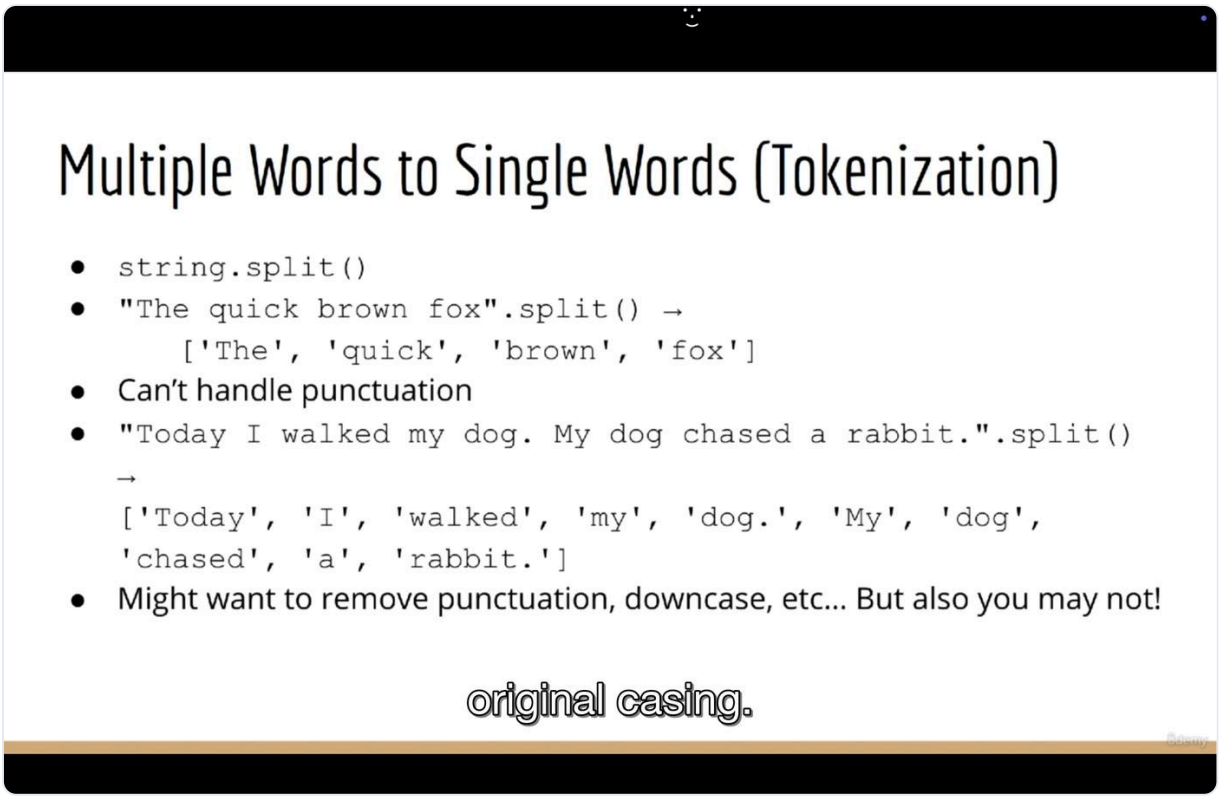


FIGURE 27.7 | Multiple Words to Single Words (Tokenization)

Today I walk my dog my dog chased a rabbit we see here that although the word dog appears twice the two times it appears are different because the first time there is a period appended at the end.

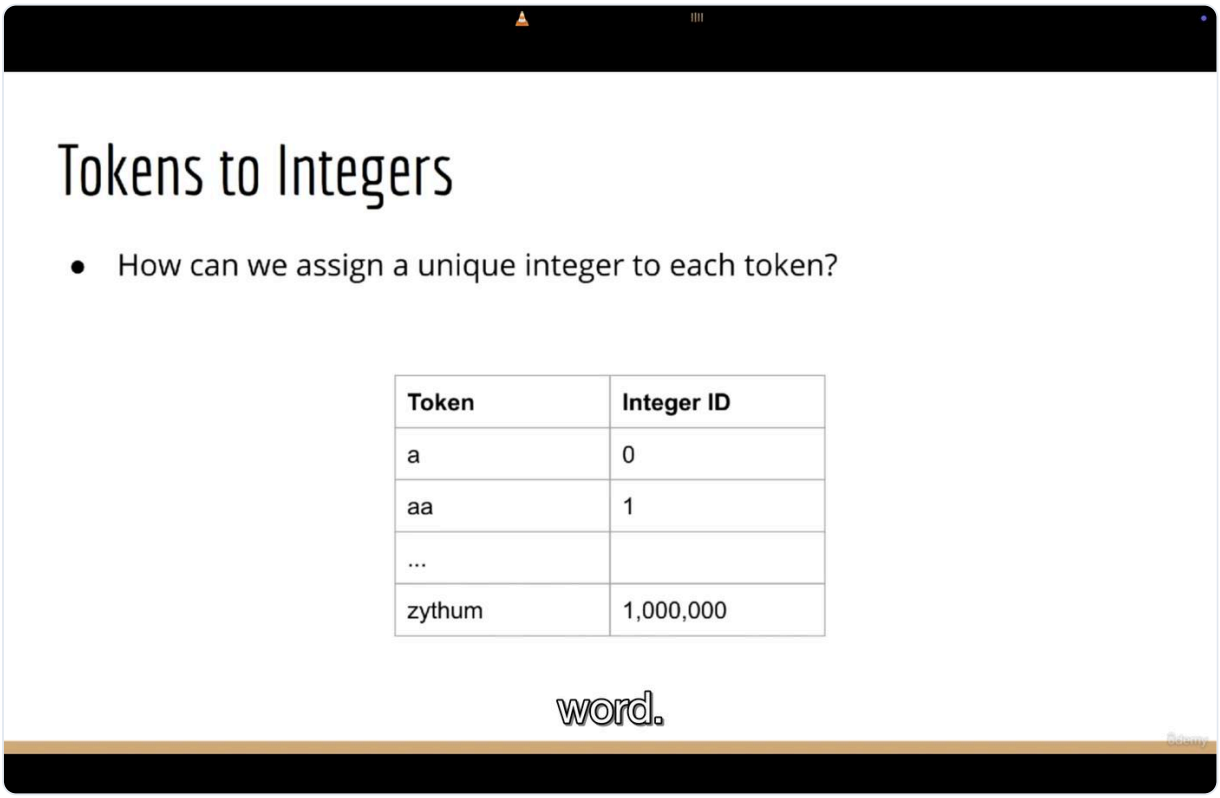


FIGURE 27.8 | Tokens to Integers

So if you wanted to say assign a unique integer to each word that appears in your dataset these would appear as two distinct words. That's possibly not good. So you may want to have some code that removes punctuation. You might also want to down case all the letters as well although there have been models that kept the original casing. Let's suppose that we've made it to the next stage where we've successfully tokenized all our documents so that instead of just single strings they now appear as lists of strings where each string as an individual word. As you know the next step is to convert these words as strings into an integer representation. Clearly each word must have a uniquely assigned integer for this to work.

So the word a might be assigned the integer 0 AA might be assigned one and so on until we get to the last word in the dictionary Zeitung which might be assigned 1 million. The question is how do you write this in code. Again this is all just conceptual for now. But here are some ideas. First you define some counter called current index and you make that zero.

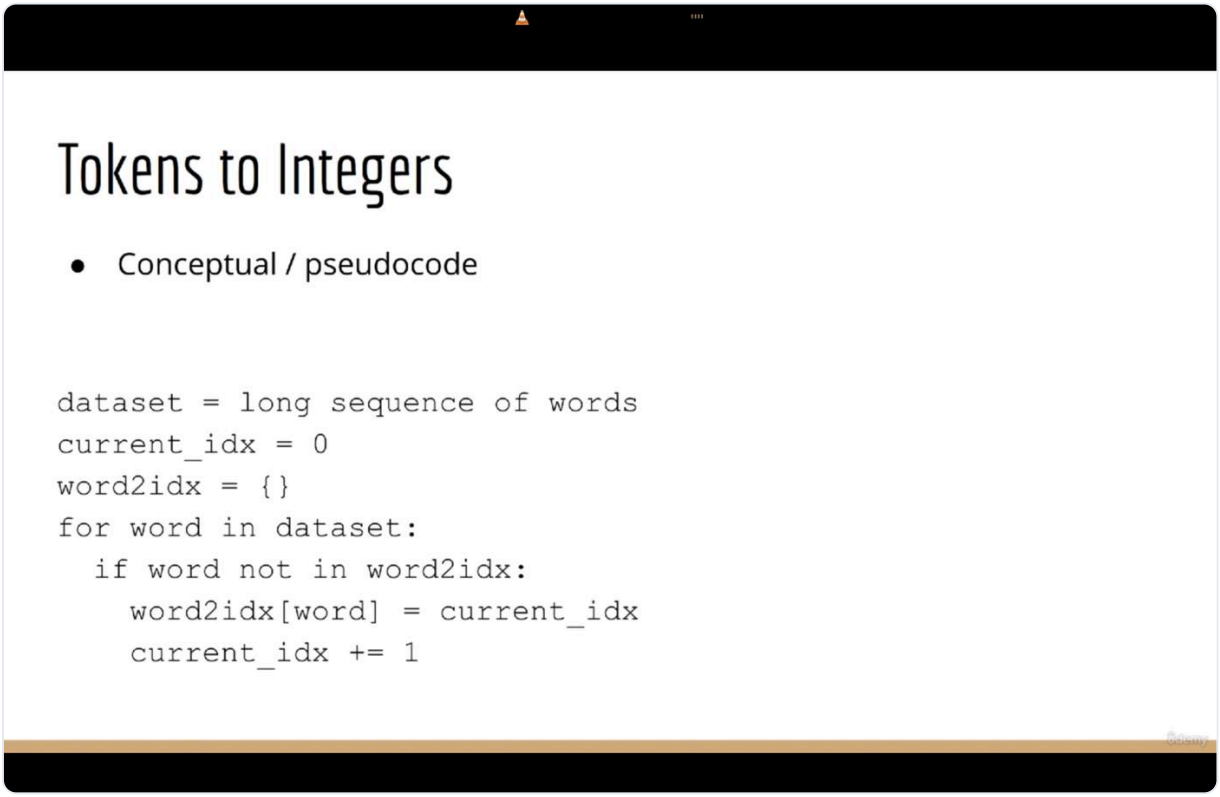


FIGURE 27.9 | Tokens to Integers

You also instantiate a set which keeps track of every word you've seen so far. Then you live through each word and your corpus by the way Corpus is just a fancy term for your text data set. So each time you encounter a word that you haven't seen before. You do two things. First you add this word and the current index which will be assigned to this word to a dictionary. We'll call this the word to index dictionary since the key is a word and the value is its index. Second we increment the current index by 1 so that the next time we encounter a new word it will be assigned a new index value. Pretty simple I hope one caveat to the previous pseudocode is that usually we don't want to start at the index 0.

Instead I propose that we start at the index too. So why would I want to do that. Well if you recall each sentence in our dataset might be of different length. So we need padding padding is usually done with zeros so zero is typically a reserve number for that use case. But why start at 2 and not 1.

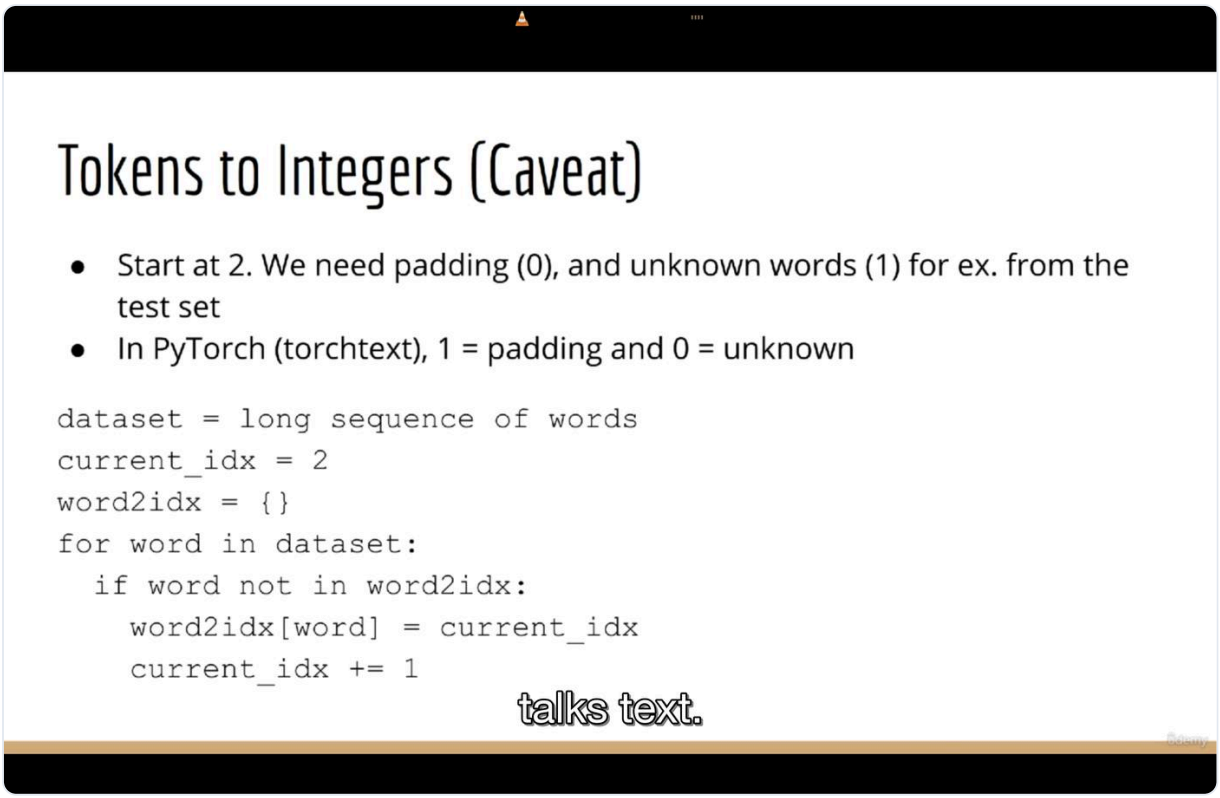


FIGURE 27.10 | Tokens to Integers (Caveat)

Well remember that we're going to have a train set and a test set. What if our test set contains words that didn't exist in the train set then these should be considered unknown words because we couldn't have learned anything about them during the training process. And so for unknown words we should assign that next one and therefore we have two special indices zero represents padding and 1 represents unknown words as a side note in the library we'll use later called torchtext. It actually does the opposite but obviously that doesn't change the point. So in actuality zero is unknown and one is padding. And of course this is only the case if you use torchtext and you don't write this yourself.

OK. So what's the next step. Remember that we would like to have constant length sequences or at least constant Length Sequences per batch. So let's suppose we have a batch of sentences each of different length the first sentence is nine three one hundred twenty seven and six. The second sentence is 14 21 and the third sentence is ninety nine five hundred one eighty seven and to remember at this point we've converted each sentence into a list of tokens which we then converted into a list of integers what we can see that the length of these sentences are five two and four respectively since the longest sentence has length five. The two shorter sentences would have to be padded to be of length five.

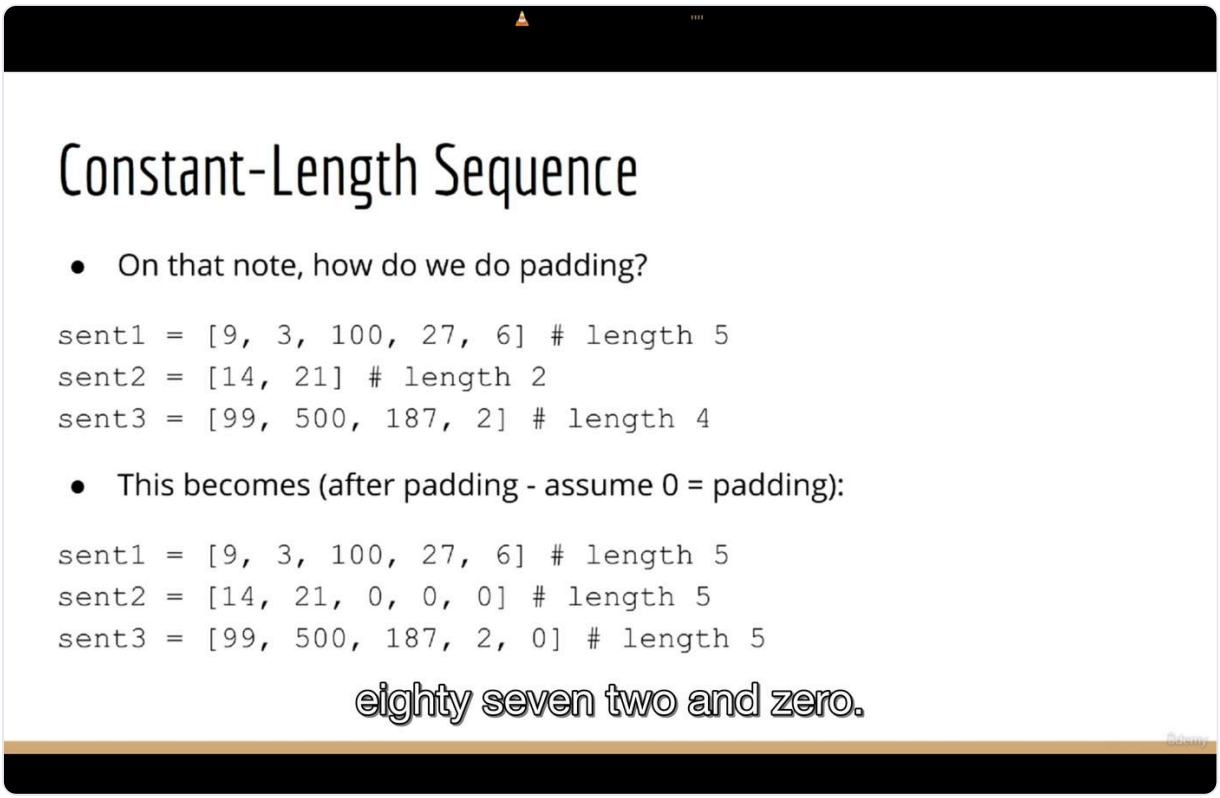


FIGURE 27.11

So the second sentence becomes 14 21 0 0 0 and the third sentence becomes ninety nine five hundred one eighty seven two and zero. And so just a reminder we're using the convention that zero equals padding. Even though this might not be the case in the code but here's an interesting idea. Instead of padding at the end why not pad at the beginning.

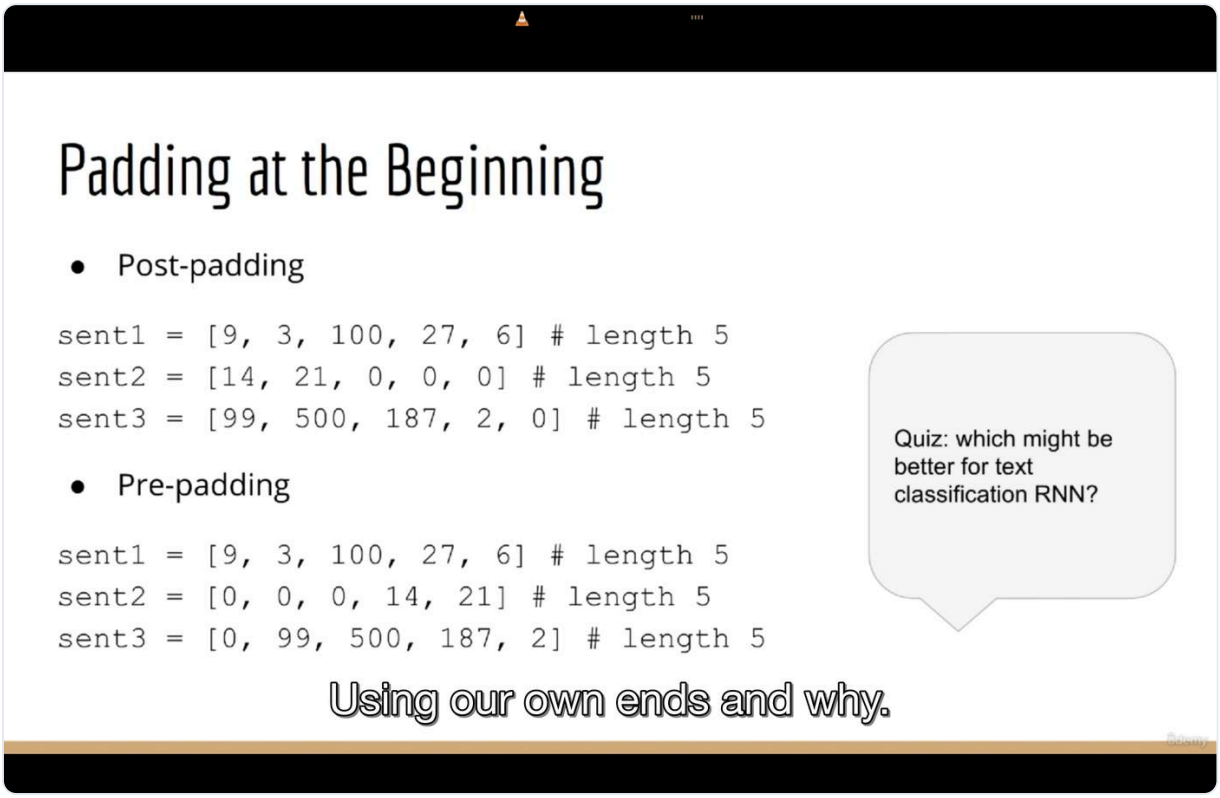


FIGURE 27.12 | Padding at the Beginning

So the second sentence becomes 0 0 0 14 20 one and the third sentence becomes zero ninety nine. Five hundred to one eighty seven and two in the first case we call that post padding because the padding goes at the end of each sentence and in the case we're considering now we call it pre padding because the padding goes at the beginning of each sentence as a quiz. Think about why we might want to choose one over the other which one might be better for text classification. Using RNNs and why. I'll give you a minute to think about it or you can pause the video and come back when you're ready. All right. So hopefully you thought about your answer to the question which might be better for text classification.

Using RNNs pre padding or post padding the answer is pre padding in our case the RNN will read the sequence from left to right.

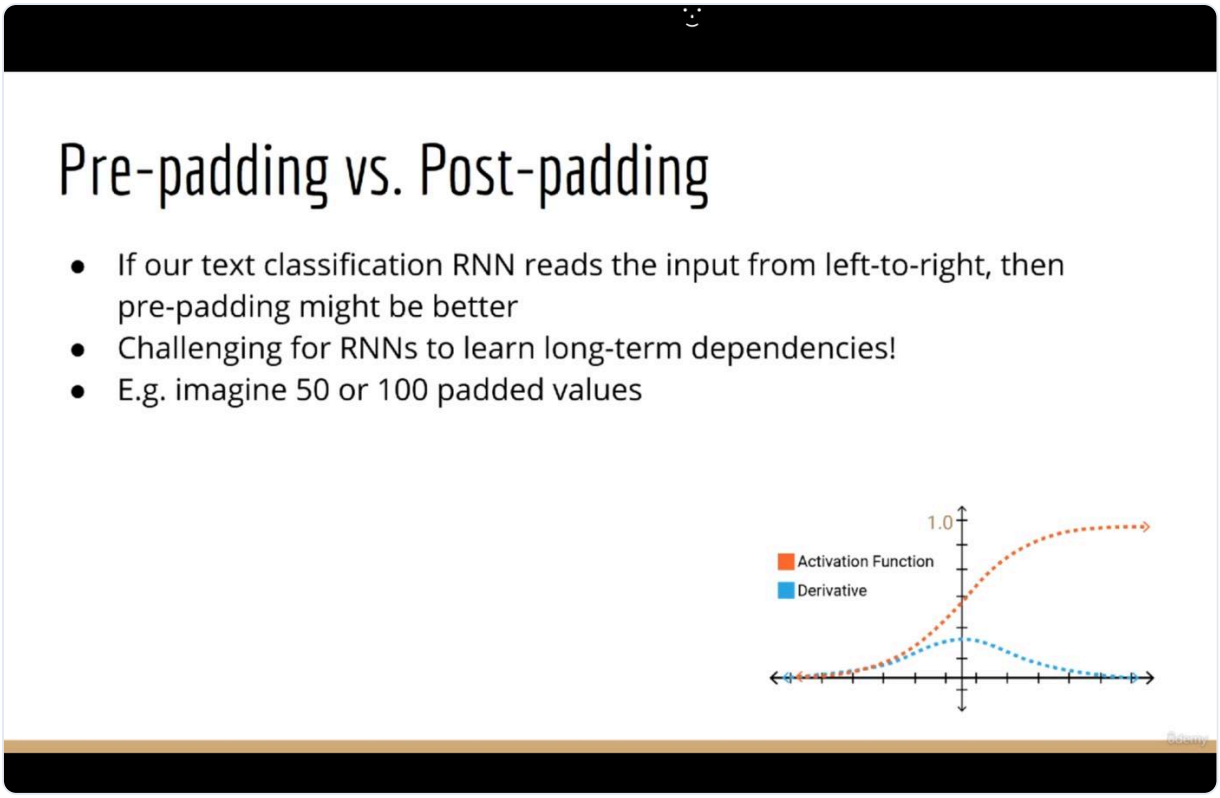


FIGURE 27.13

Or equivalently from beginning to end. That's not a problem in and of itself but recall what challenges do RNNs face. The main challenge for RNNs is learning long term dependencies due to the vanishing gradient problem. So if we put padding at the end what happens. That means if there was any pattern to be learned from the sentence it's going to appear at the beginning and if there's tons of padding say 50 or 100 zeroes then there's a good chance that by the end of all those zeros the RNN will have completely forgotten what it saw earlier and due to the vanishing gradient problem it's going to be very hard for the RNN to learn to capture those patterns that appeared at the beginning of the sentence.

Therefore it's better if you're doing a many to one task in the left to right direction that you put your padding at the beginning. Okay so let's summarize all the conceptual steps that need to happen in a text preprocessing pipeline.

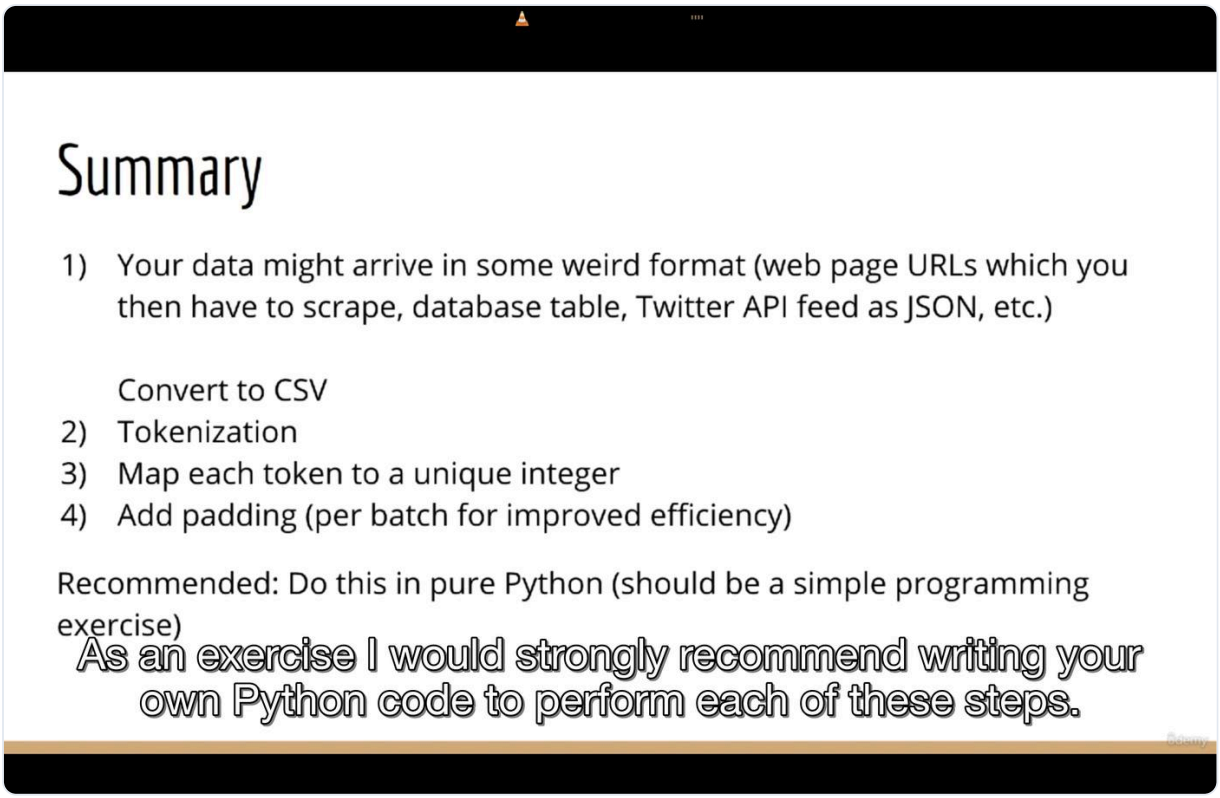


FIGURE 27.14 | Convert to CSV

First your data might arrive in some weird format like for example and maybe be a collection of web page else or a database table or you might be using the Twitter API to download tweets as JSON documents for reasons you'll understand very soon it'll be convenient if you can format these into a CSV. Second when you load in your data whether it's a CSV or not this data won't be in the form of one string per word in order to convert each document into a list of strings where each string contains only a single word or token. We need to perform a process called tokenization. Third once you have your documents encoded as lists of strings you need to convert each of those tokens into an integer representation.

That means you need to assign or map each token a word to a unique integer and in addition you'll need to convert each sentence from a list of strings into a list of corresponding integers. At this point your dataset consists of nothing but lists of integers finally. You'll want to perform padding so that your data set or at least each batch has the same sequence. I hope that none of these steps seems particularly difficult to write by yourself in Python code. As an exercise I would strongly recommend writing your own Python code to perform each of these steps. This isn't strictly required for the upcoming example but it's an extremely useful skill to have especially if you plan on doing something unique and not covered by the basic use cases in existing libraries.

LECTURE 66

Why Bad Programmers Need the Latest Version

https://www.youtube.com/watch?v=BIXH_m6CT2I

So in this lecture, we are going to discuss how to do text preprocessing. Now, you might notice this lecture has a funny name. I've called it beginner blues, PyTorch NLP version. So what do I mean by this? Well, let's begin with the fact that this is a brand new updated lecture, which did not exist when I first released this course. In fact, there have been a few free updates to this course over the years. So why does this lecture exist? Well, you may have noticed if you went through the legacy NLP lectures, they use the library called torchtext. And at the current time, the torchtext library has been updated such that the previous code does not work with the latest versions.

Many beginners got stuck because of this change. Now, the reason I wanted to make this lecture is because over the years I've seen many beginner mistakes being made and many misconceptions that need to be addressed.

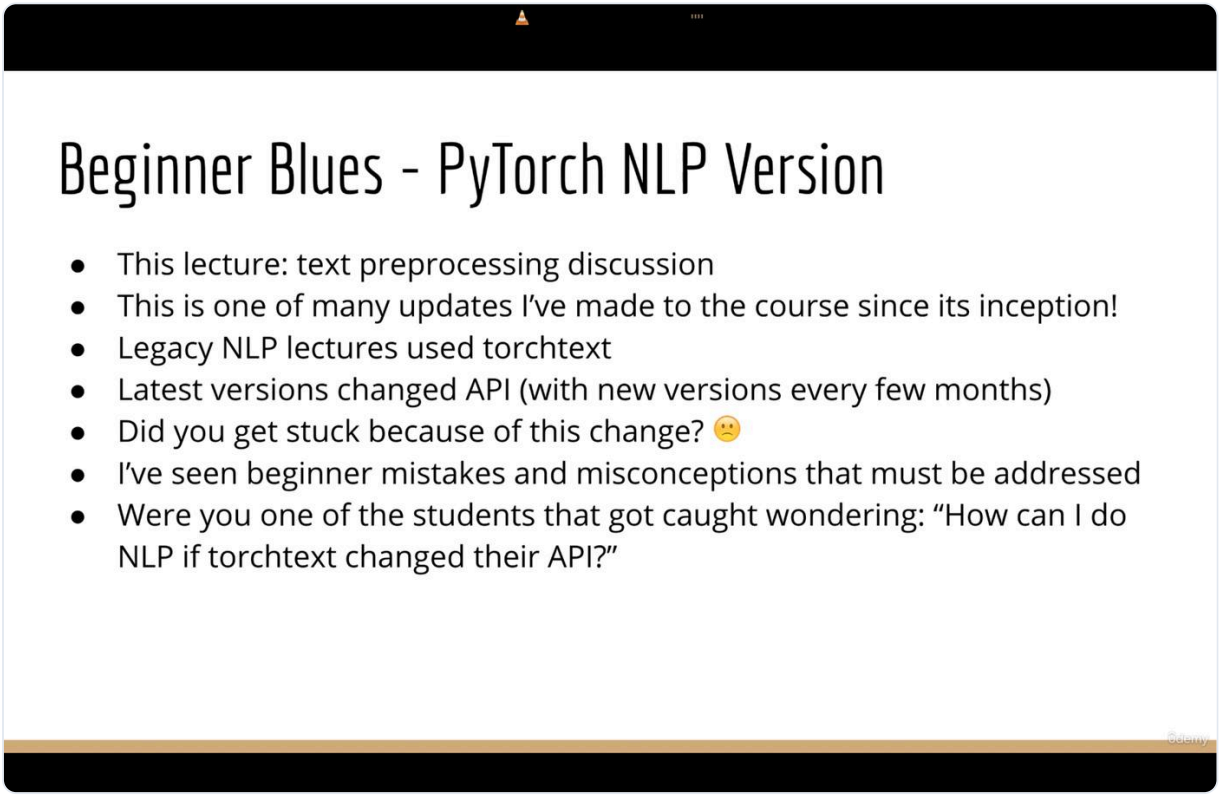


FIGURE 28.1 | Beginner Blues - Pylorch NLP Version

So if you are one of the students that got caught wondering how can I do NLP? If torchtext change their API, then this lecture is for you.

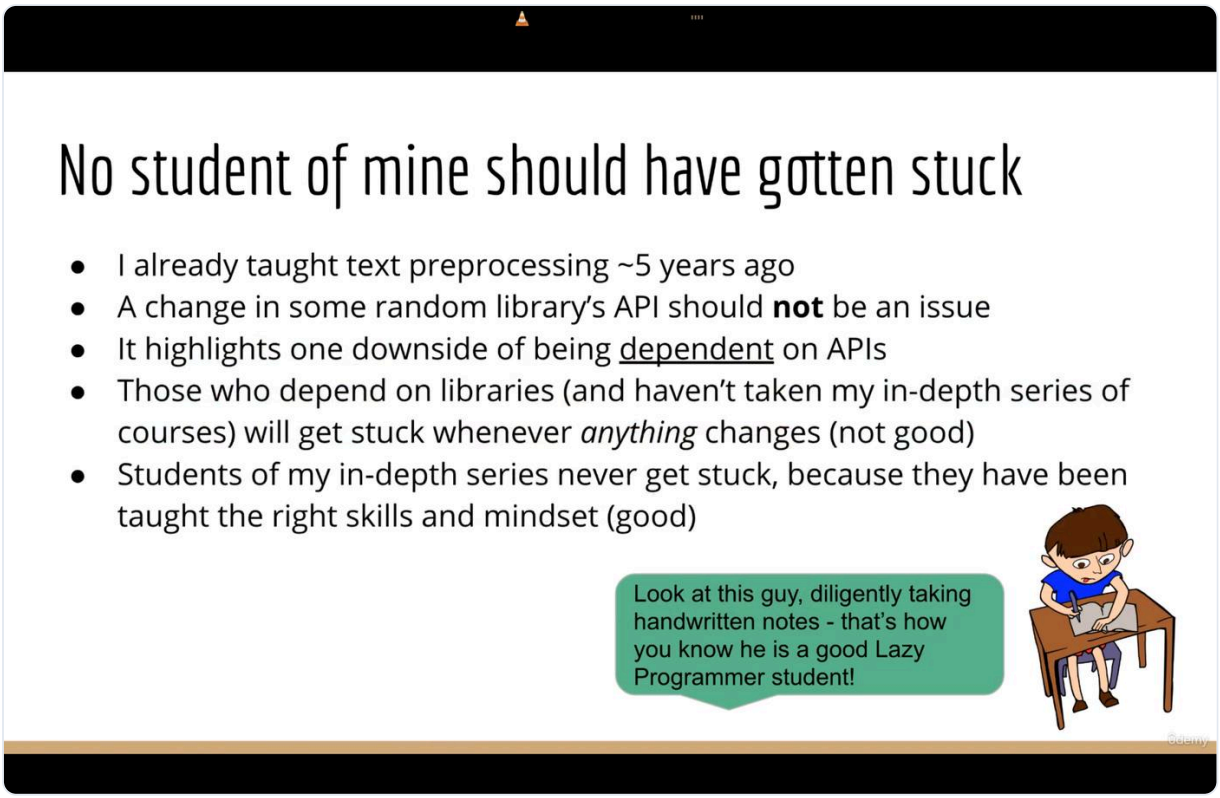


FIGURE 28.2 | No Student of Mine Should Have Gotten Stuck

So let's start this lecture by saying that this is not something anyone who is a student of mine should have gotten stuck with, basically I taught the skills to deal with this sort of thing approximately five years ago. So this is not new technology by any means, merely a change in the API. So that's one of the major downsides of being dependent on libraries. If you haven't studied my in-depth series of courses and you depend on libraries to get things done, then you will pretty much get stuck whenever anything changes. On the other hand, students who have taken my in-depth series of courses tend not to get stuck because they've been taught the skills such that a change like this is not even an issue.

So in this lecture and the coming lectures, we'll be borrowing a bit of content from my in-depth series, specifically deep NLP, RNNs and HMMs, where I first taught this technique many years ago. Now, you might think I'm just being too hard on beginners, but here's the reality, what I'm about to teach you in this lecture is a very practical skill used very often in the real world. In fact, it's so useful. I use this as an interview question for data science employees. Basically, the question is to perform the text preprocessing steps in Python, which in this course we had previously done using torchtext. The hard truth is I would never hire someone who could not write the code to solve this problem.

In fact, my colleagues loved this question as well. And so they are also using this question in their interviews. So don't be surprised if you go to your data science interview and they ask you this question. We use code like this every day. And so if you can't write this kind of code, we will not give you the job. Thus, if you were one of the beginners who saw this and just gave up or got frustrated because the library changed on, you realize that this is not a good attitude for a data scientist to have. If this is your attitude, you are not getting the job.

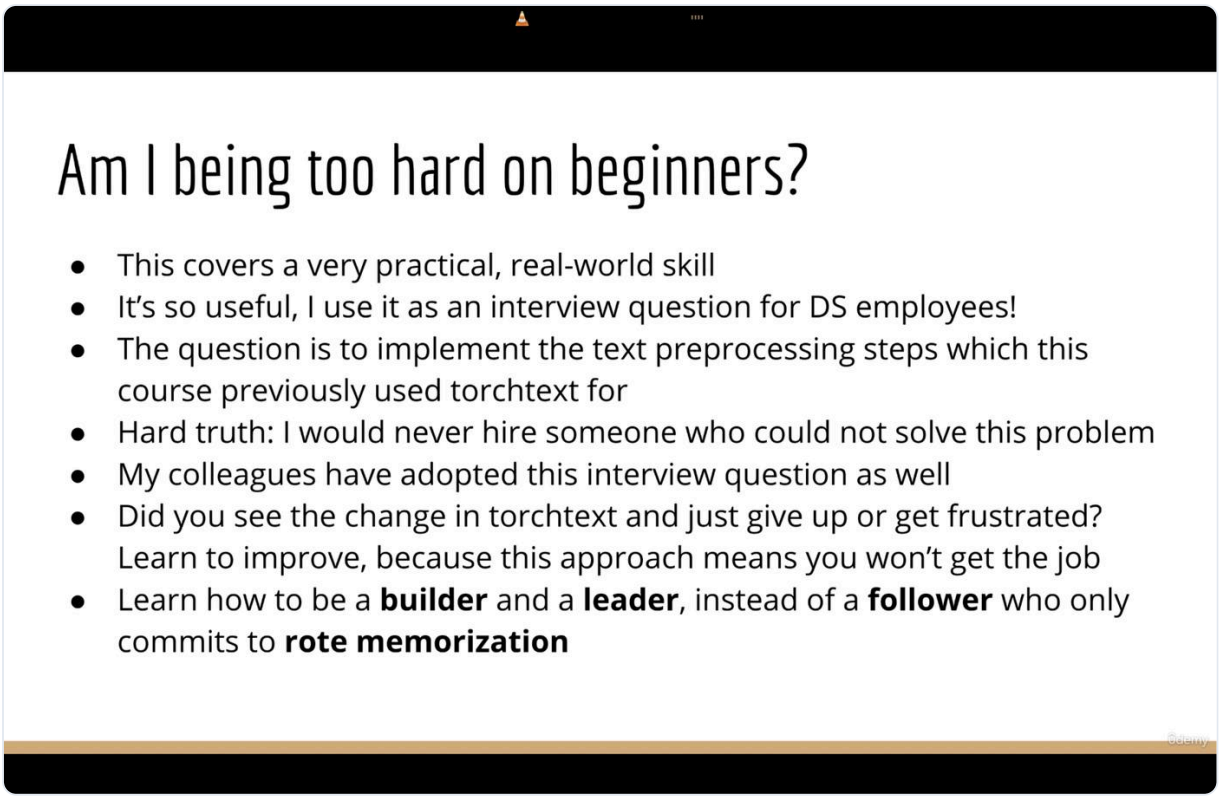


FIGURE 28.3 | This Covers a Very Practical, Real-world Skill

The main theme of this lecture is basically how you can be a builder and a leader instead of a follower and someone who only commits to rote memorization. Now, I do want to mention that most of you did not get stuck. For the most part, I find that the students of my VIP courses tend to be more mature and well-rounded, not always, but to a greater degree. So this is a good thing.

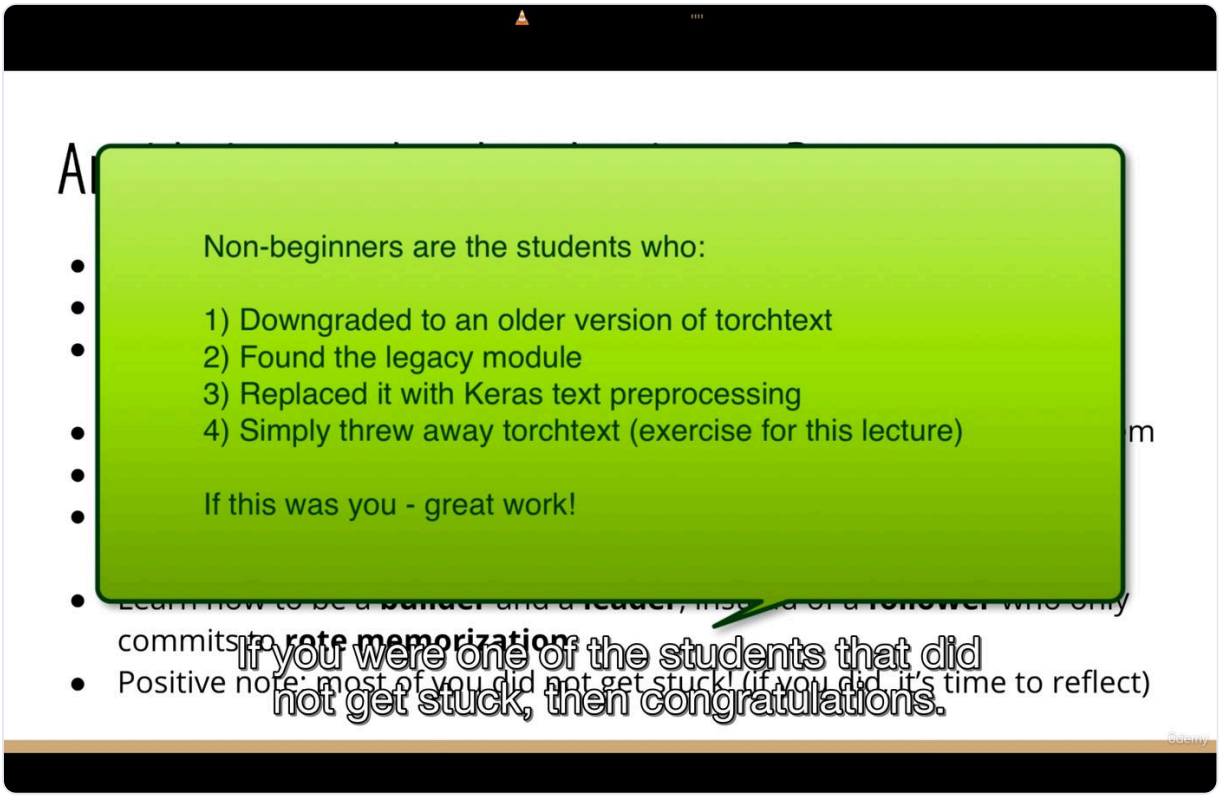


FIGURE 28.4 | Non-beginners Are the Students Who:

If you were one of the students that did not get stuck, then congratulations. You are on the right track. Another falsehood I want to address before getting to the main content is that beginners often think they should always be using the latest version. In fact, I have a YouTube video about this called Why Bad Programmers Always Need the latest version.

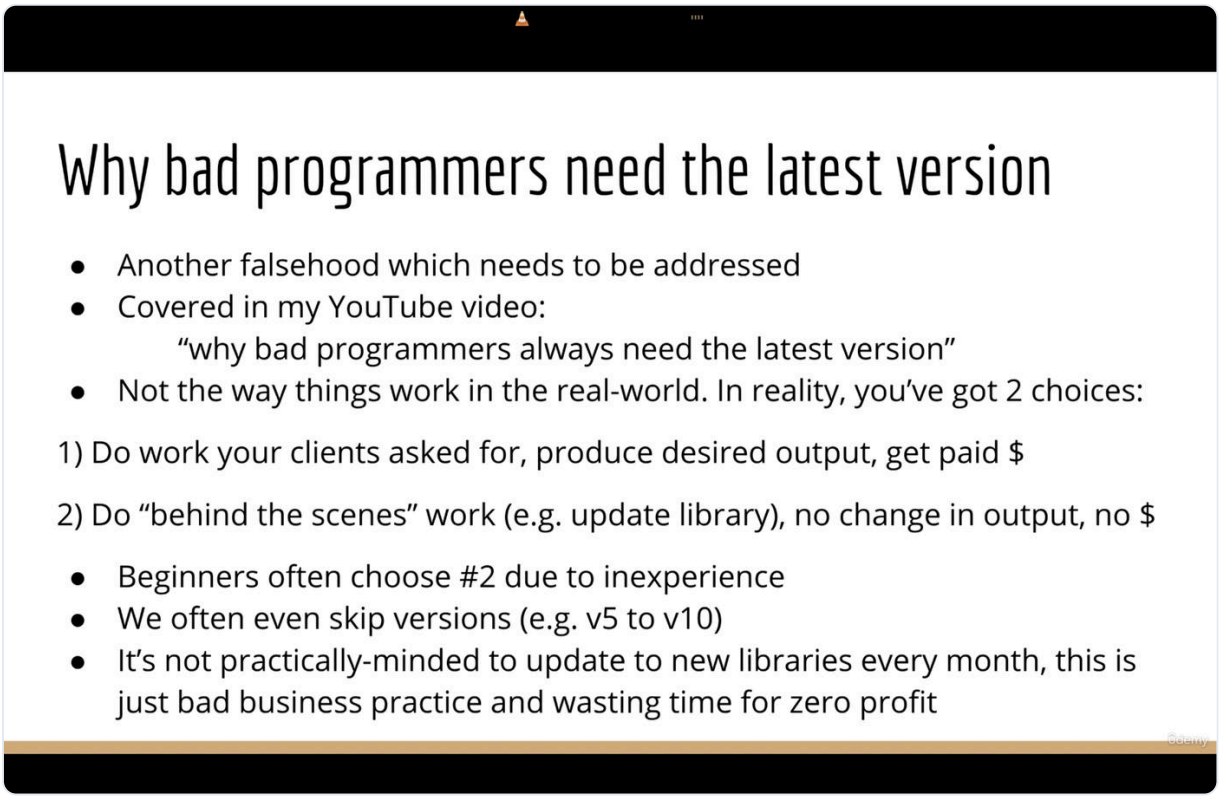


FIGURE 28.5 | Why Bad Programmers Need the Latest Version

The reality is this is not the way things work in the real world. You see, as a business, you really only have two choices. Number one, do the work that your clients have asked for to produce real meaningful output and get paid real money or no to do behind the scenes work, such as updating libraries, which does not produce any change in output for your clients in which you do not get paid for. So as a business, we tend to choose number one rather than number two. Beginners often feel like everything has to use the latest library because they haven't yet had the opportunity to work on real world projects.

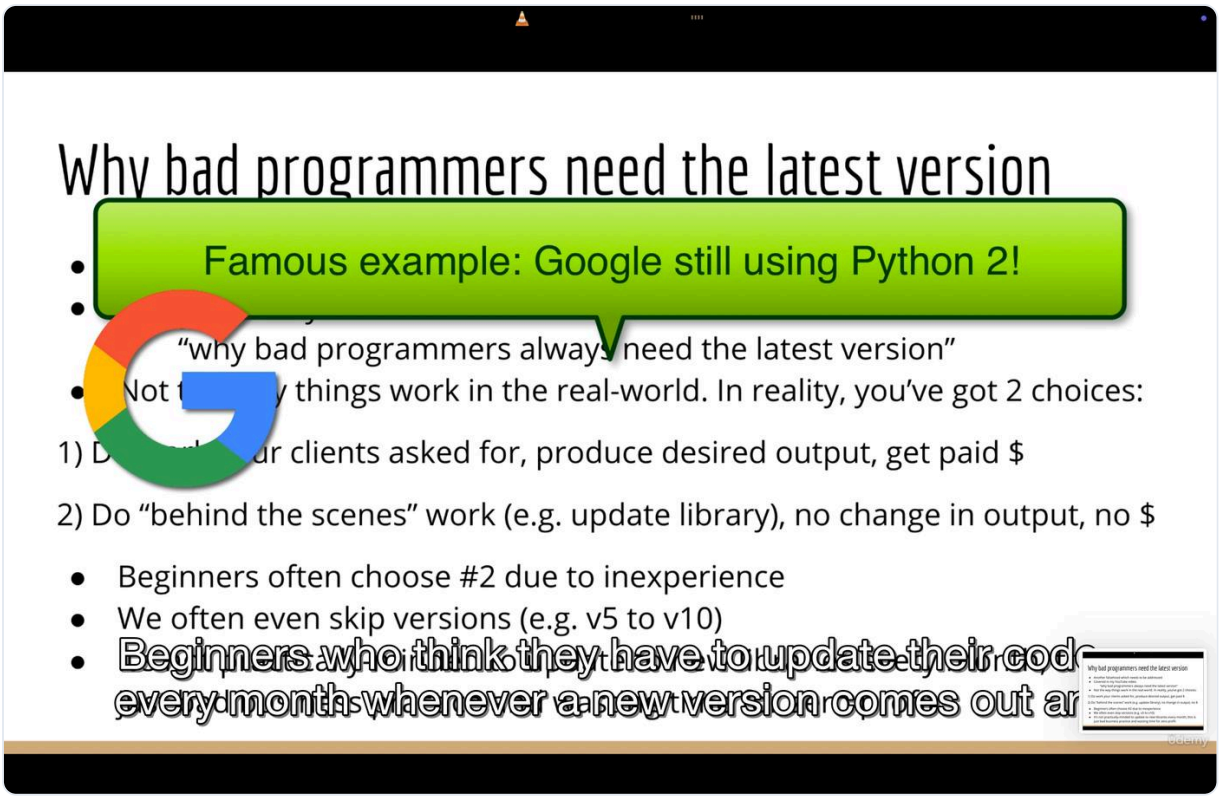


FIGURE 28.6 | Why Bad Programmers Need the Latest Version

Many times we even skip versions. So today we might be using version five and we might not update until version 10. Beginners who think they have to update their code every month whenever a new version comes out are not very practically minded. This is not how we operate in a real business. So for the rest of this section, I'm actually going to leave the legacy lectures up since you'll need to watch them in order to complete the following, exercise the exercises to write your own code, to replicate the text preprocessing steps, that is, write the code to replace the torchtext functionality that we need. Now, you might wonder, how can I do this if I haven't yet taken Lazy Programmer's deep NLP course is true.

This exercise will be a little easier if you've taken my in-depth series of courses. But it's not true that you can't complete it.

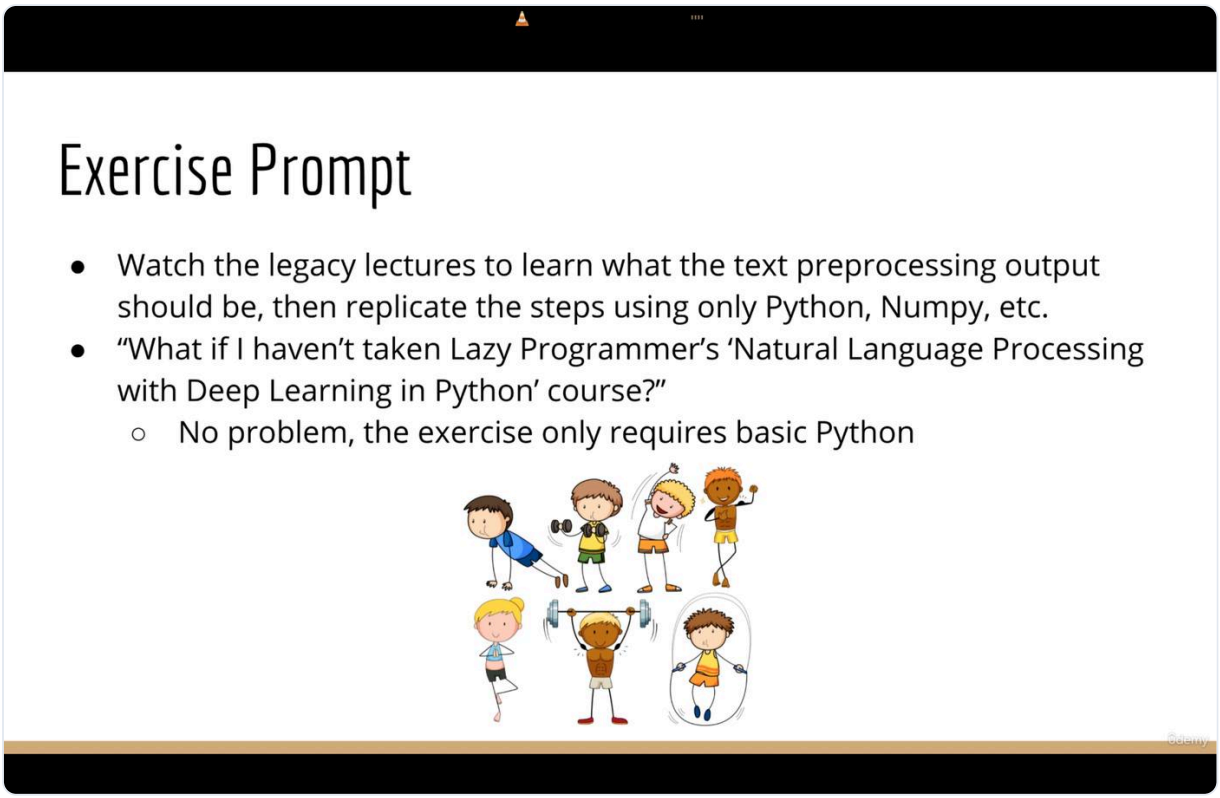


FIGURE 28.7

If you have not taken my in-depth series of courses to complete this exercise, you only need basic python knowledge. And actually this is why it surprised me so much that so many people got stuck. So to summarize the exercise, the steps are roughly as follows. Number one, tokenize each document, you can use any tokenizer you like, but the simplest method is to simply call string that split number two map each token to a word index. As you recall, these will be used as indices to the embedding matrix. Note also that you'll need special indices for padding since each batch needs to have the same length. Number three, once you have your word to index mapping, convert all your documents into integer format.

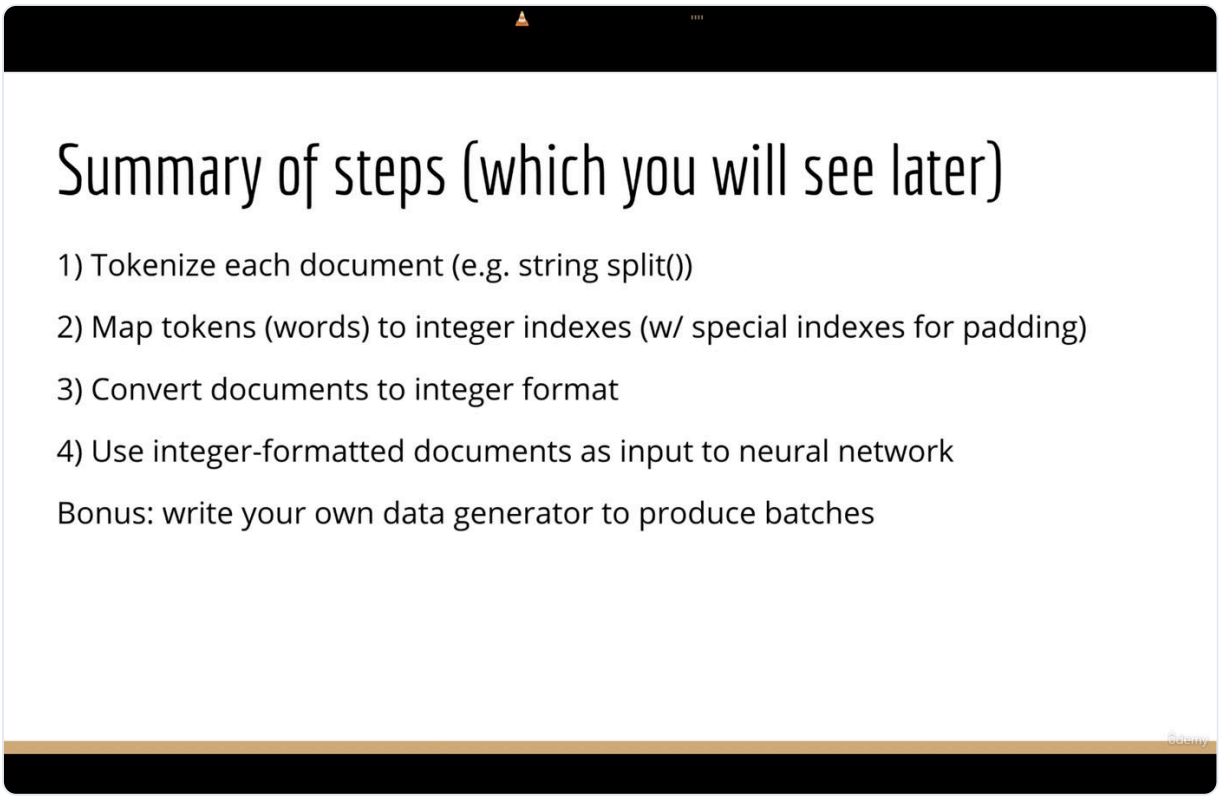


FIGURE 28.8 | Summary of Steps (Which You Will See Later)

Number four, use your integer formatted documents as input into your neural network. Of course, the last step you already know how to do if you've completed the previous sections of this course as a bonus, you should also create your own data generator that produces batches of neural network inputs. Remember to also return these as Torch Tensors.

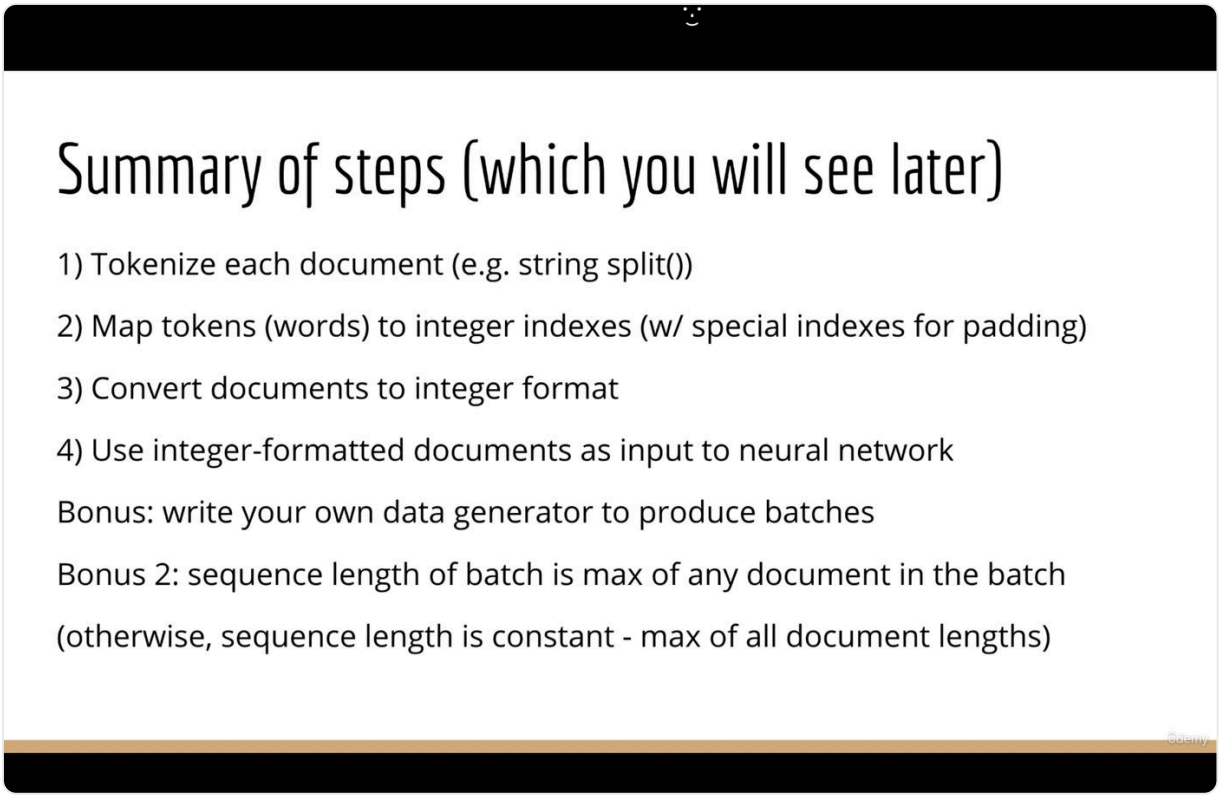


FIGURE 28.9 | Summary of Steps (Which You Will See Later)

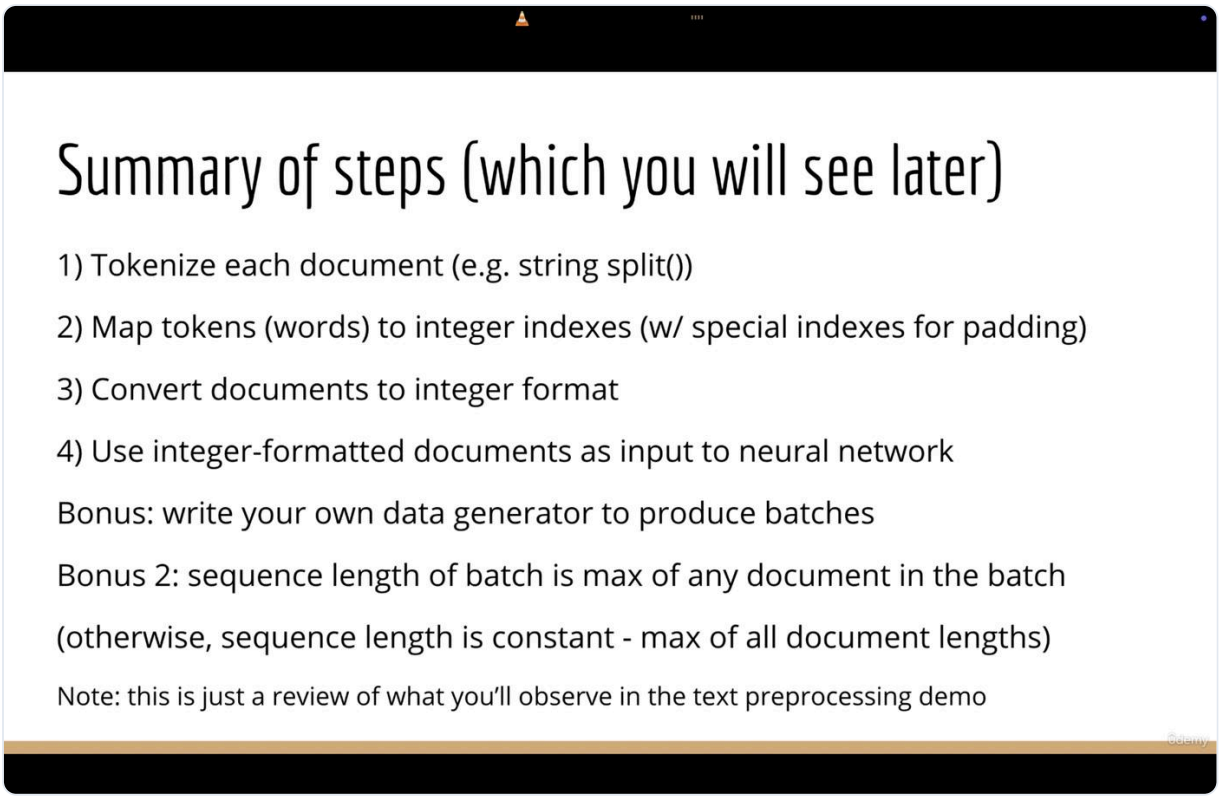


FIGURE 28.10 | Summary of Steps (Which You Will See Later)

As a further bonus, the sequence length of each batch should only be as long as the longest sequence in the batch, otherwise the sequence length can simply be the maximum document length in the whole data set. OK, so as you'll see, this is nothing but a review of what is already shown in the legacy text preprocessing lecture. So watch that lecture to see an example of how your code should behave and then write your own code to reproduce that behavior. Now, after all this, you might be wondering why not simply use the latest version of torchtext and to these students, I say you must watch this whole lecture again because the purpose has evaded you.

You see, your approach of using libraries is the whole reason why you are stuck here in the first place. Of course, if you learn today's library, you're just going to be stuck again three months from now. And then six months later, when it changes again, you're going to be stuck again. So hopefully you see why this approach is a bad idea. Furthermore, the torchtext API is just not that nice, which is, in my estimation, why they decided to change it in the first place. Unfortunately, the latest version is also not that nice, in my opinion. Not really worth learning. In fact, I've seen several courses that simply use the Keras text module instead because it has a much nicer API instead, a better and more powerful approach is to actually learn how these things work.

Getting back to the real world, often when we're hiring people, we don't care which APIs you know how to use, we care that you can write code when we're hiring people.

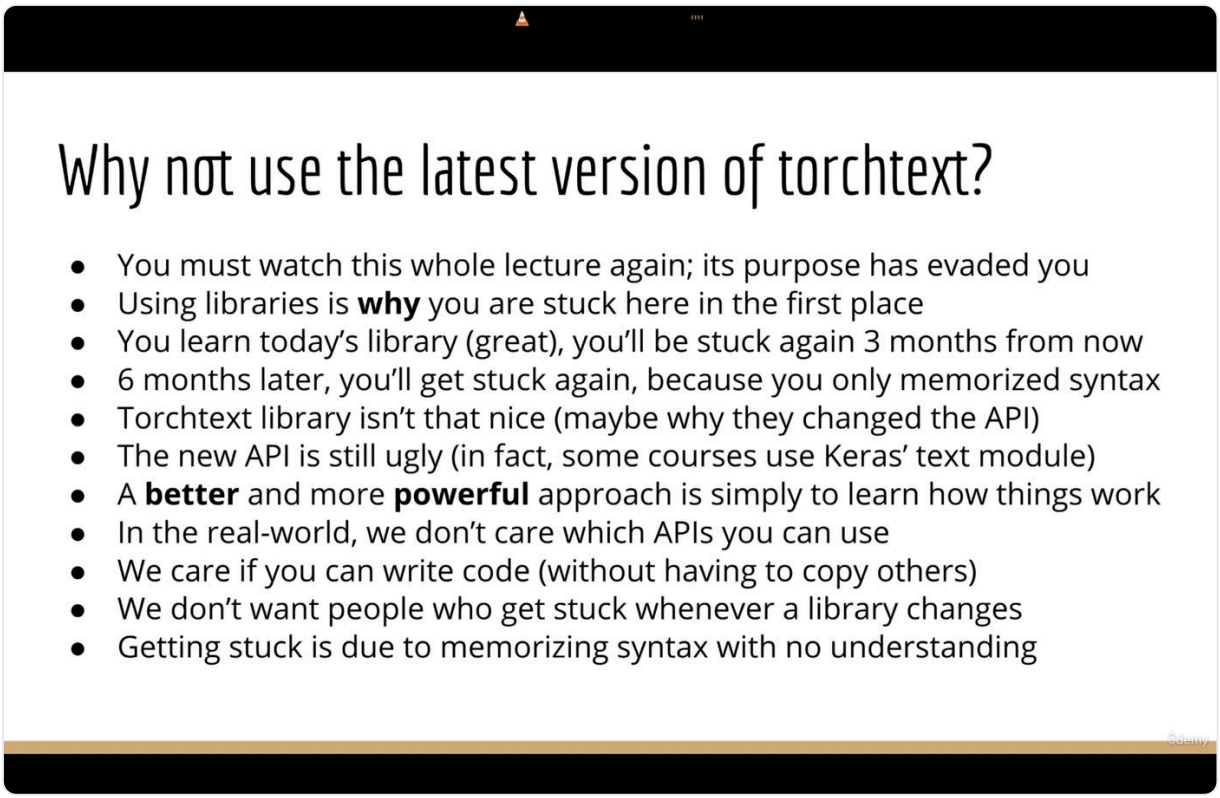


FIGURE 28.11 | Why Not Use the Latest Version of Torchtext?

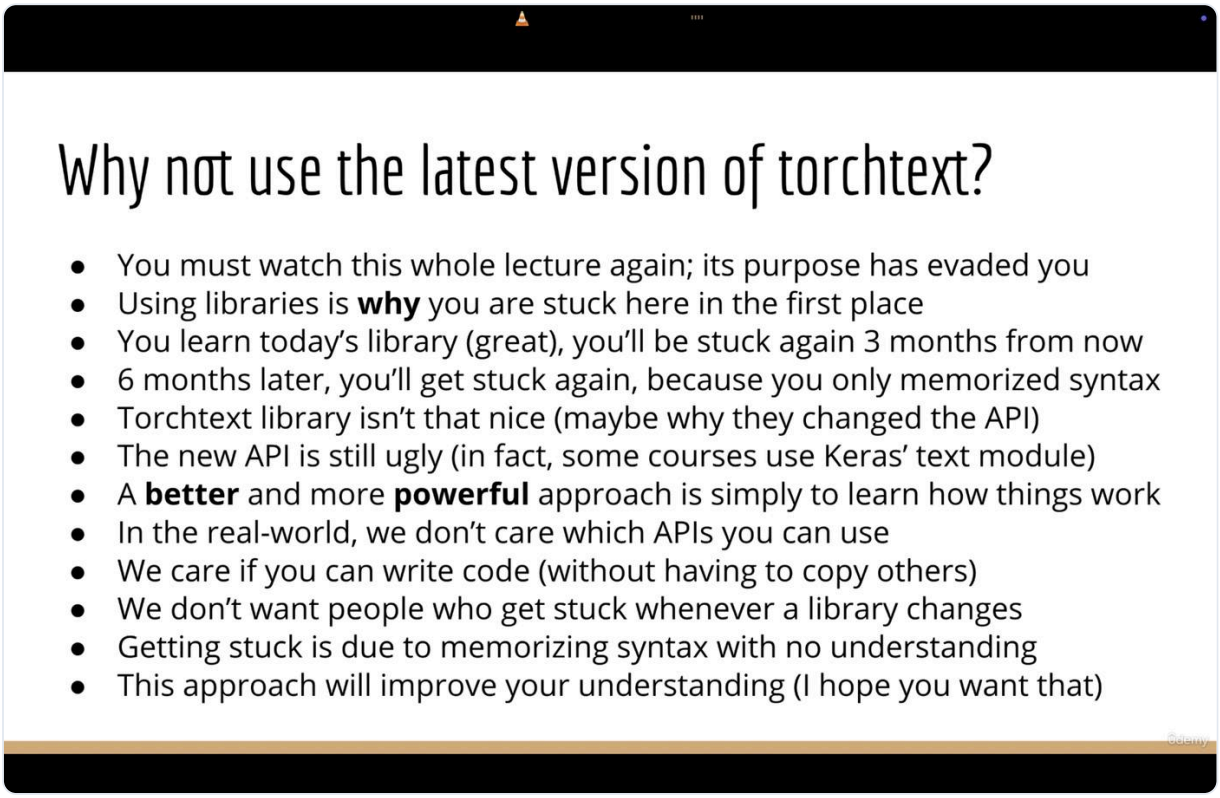


FIGURE 28.12 | Why Not Use the Latest Version of Torchtext?

We don't want people who are going to get stuck every time a library changes. Those employees tend to need too much handholding to get things done. And as such, my recommendation is to not be that kind of employee. The main reason why beginners were getting stuck is because they were just trying to memorize syntax without any understanding of what it was doing. So the approach of rote memoriza-

tion is very bad in programming. So not only is the approach in this exercise nicer, it's also conducive to improving your understanding of how things work. Another question you may have is, isn't it always better to use a tried and true solution instead of rolling your own?

And for this type of question, it's obvious you have to apply some critical thinking and context. Obviously, you're not going to build your own database or a Web server framework. That would be absurd.

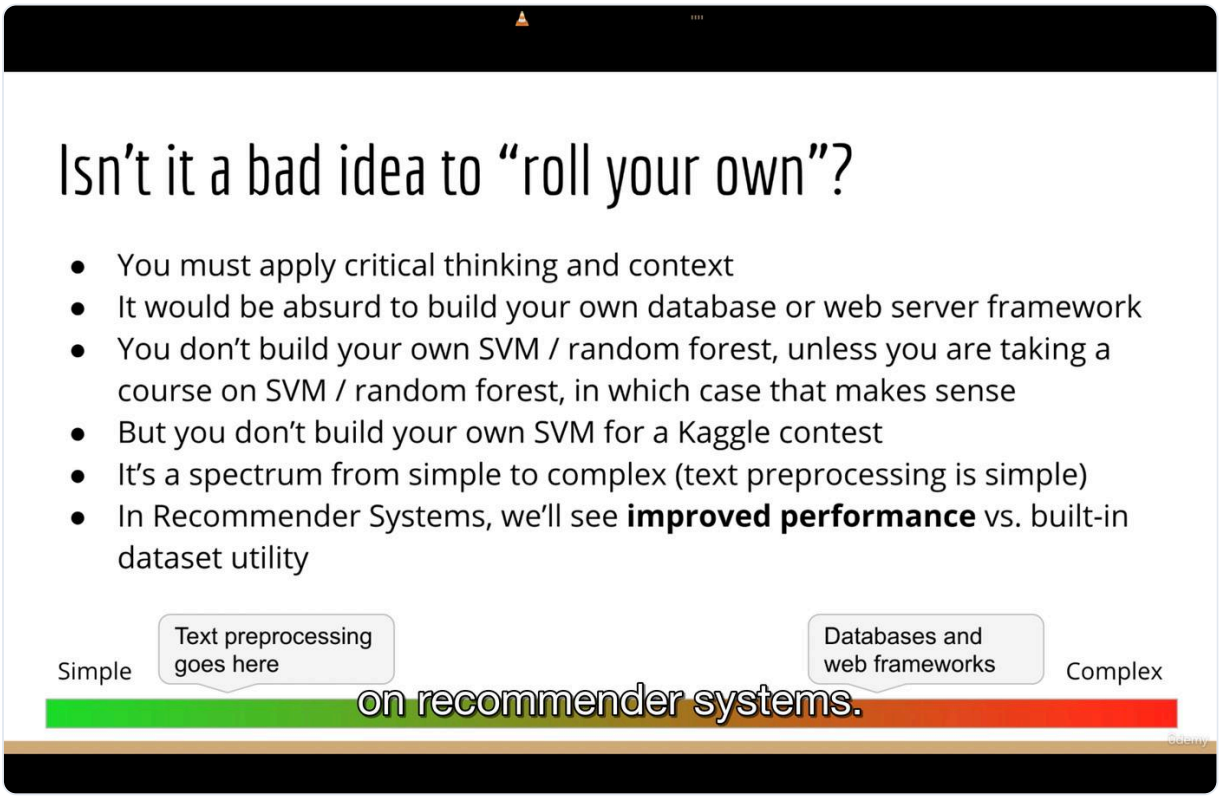


FIGURE 28.13

You won't build your own SVM or random forest unless, of course, you're taking a course on SVM or random forest, in which case that would make sense. If you're doing a Kaggle contest, then of course, you're not building your own SVM for the contest. So it's all about context. Normally, big complex code bases like a database or a web server would not be built from scratch. Simple code like looping through a log file to pass out the important fields would be built from scratch. So there's always a spectrum in the case of text preprocessing. What you'll see is that this actually falls on the simple side instead of the complex side. In fact, another reason, aside from all the good reasons already given, is performance in the section on recommender systems.

We'll see that building your own data loader is actually more performant than using the built in utilities in PyTorch.

LECTURE 67

Field Objects

So in the previous lecture we talked about all the steps that are required to pre process your text in order to convert your documents into lists of integers so that they can be ingested by a neural network.

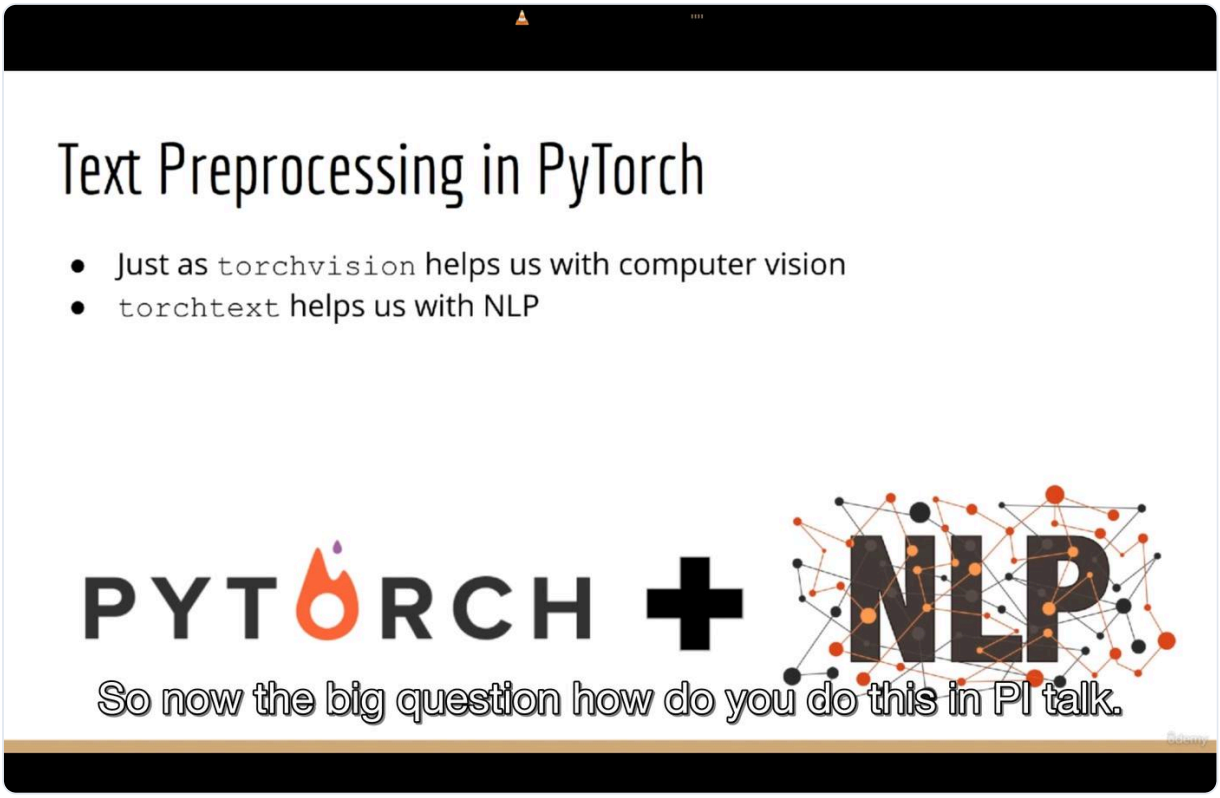


FIGURE 29.1 | Text Preprocessing in PyTorch

So now the big question how do you do this in PyTorch. Well it's not done in PyTorch per say but it's part of the PyTorch ecosystem just like how a specialized computer vision operations are done by the torchvision library specialized NLP operations are done by the torchtext library. This is also automatically installed in Google Colab so there isn't anything special you need to install on your own. If you use Google Colab unfortunately the syntax is quite strange. So bear with me as I try to make it digestible. First we're going to assume that our task is text classification which it is and it's going to be a many to one task. That means for each document which is a sequence of words our prediction is going to be a single label for that entire document that might sound a little bit abstract.

So to give you a concrete example think of spam detection.

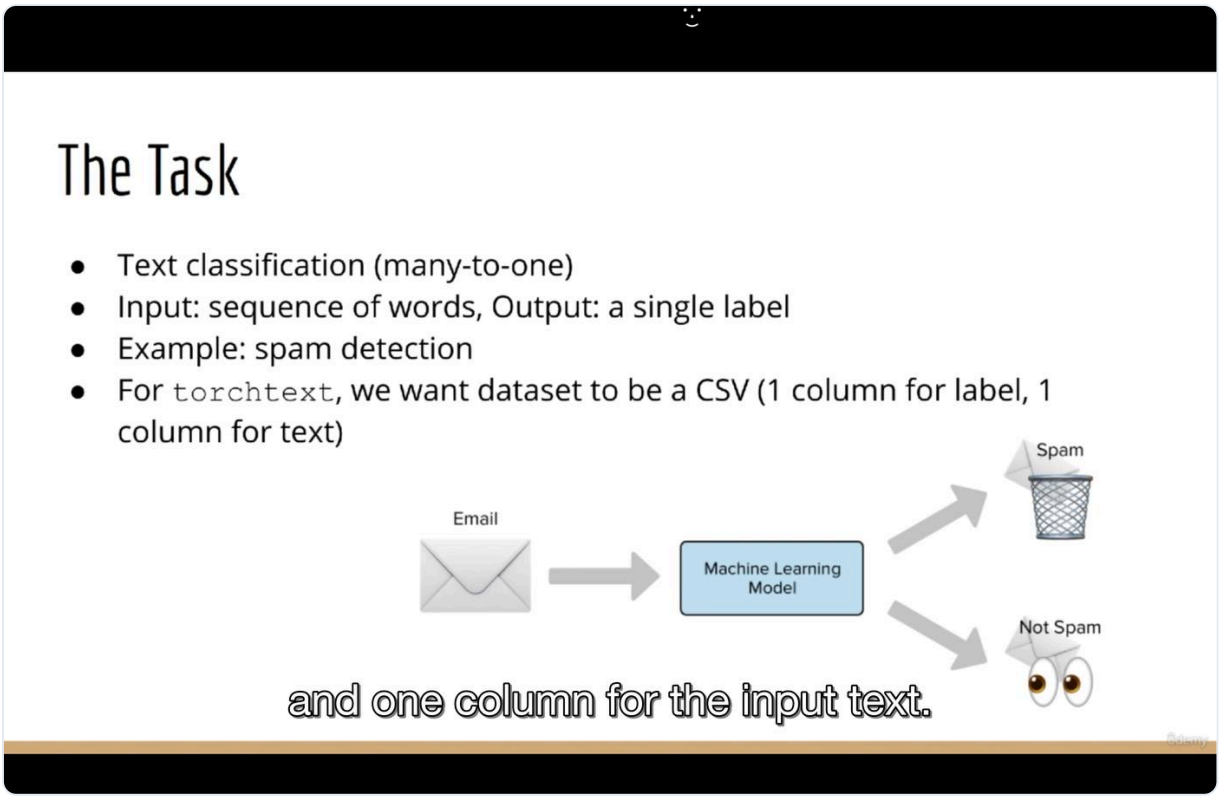


FIGURE 29.2 | The Task

Your input is an email which is a sequence of words and you would like to output a single label spam or not spam for torchtext will want this data to be formatted as a CSV with one column for the label and one column for the input text. I assume that at this stage you know how to write a program in Python so that if your data does not come in this format you know what to do to convert it into the required format. The next step is to create field objects field objects are part of the torchtext data module both the inputs column and the targets column will be field objects but with different arguments for the input data.

FIGURE 29.3 | Field Objects

I want to pass on the arguments sequential equals true batch first equals true lower equals true and pad first equals true. Let's run through these quickly although they should be pretty intuitive already. We set sequential equal to true since the data is sequential each value is a sequence of words. We set batch first equal to true because we want the N dimension to come first. And the T dimension to come last we set lower equal to true because we want to lower case the words. This isn't always necessary but our data set is pretty small in which case it's usually a pretty good idea. And lastly we set pad first equal to true so that we use pre padding instead of post padding.

We already discuss why that might be desirable although you should try both.

FIGURE 29.4 | Field Objects

Just to be sure for the label we're going to create another field objects but with an entirely different set of arguments specifically we're going to set sequential equal to false use vocab equal to False and is target equal to true since the target is not sequential we set sequential equal to false. This should be clear. We set use vocab equal to False which might seem strange but let's think about it. Remember that there are a whole bunch of text preprocessing steps that we need to do. One of which involves assigning a unique integer to each word in the dataset. This is the process of building the vocabulary of words in the dataset. Of course such a process wouldn't make sense to do on the targets because the targets are just numerical labels.

So we set use vocab equal to False. Finally we set is target equal to true. Since this field is for the target column after all. In my opinion this could have a better API but it is what it is. You might also want to consider how these arguments might change if you were doing something like neural machine translation.

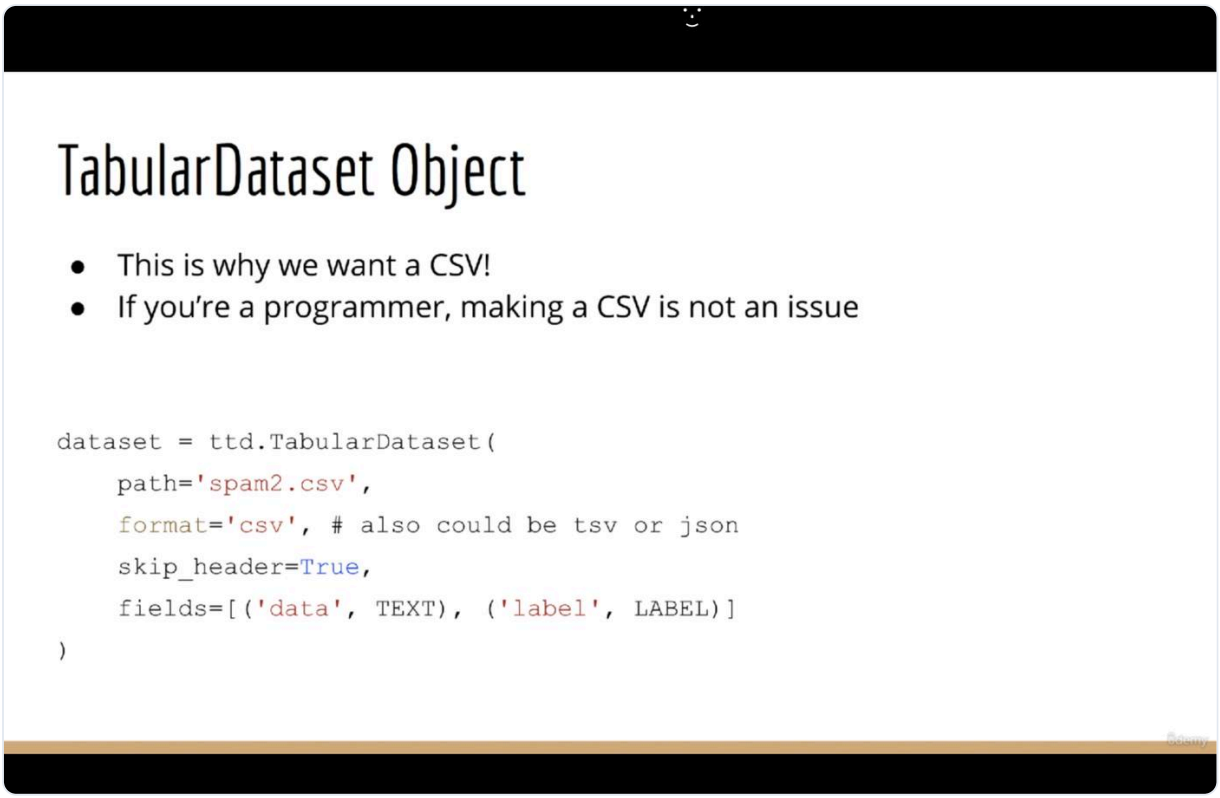


FIGURE 29.5

The next step is to instantiate a tabular data set object which is also part of the torchtext data module. So now you understand why we want to create a CSV to store our data. It's because a CSV is a table and torchtext only knows how to read tabular data sets. Now you might think it might seem kind of limited to only have tabular data sets. What if we need a data set that is not tabular but if you think about it pretty much every text dataset you can think of will probably fit into this framework. Whether that sentiment analysis spam detection parts of speech tagging machine translation and so forth. Even if it's unsupervised learning and all you have is just one document per line that's still technically a CSV.

So all these different kinds of data sets can fit into a CSV format. And as long as you're a competent programmer and you know how to make your own CSV they should not be an issue. So let's go through the arguments one by one so you know what they mean. The first argument which is the path should be clear is the path to the data file where you're CSV data set is stored.

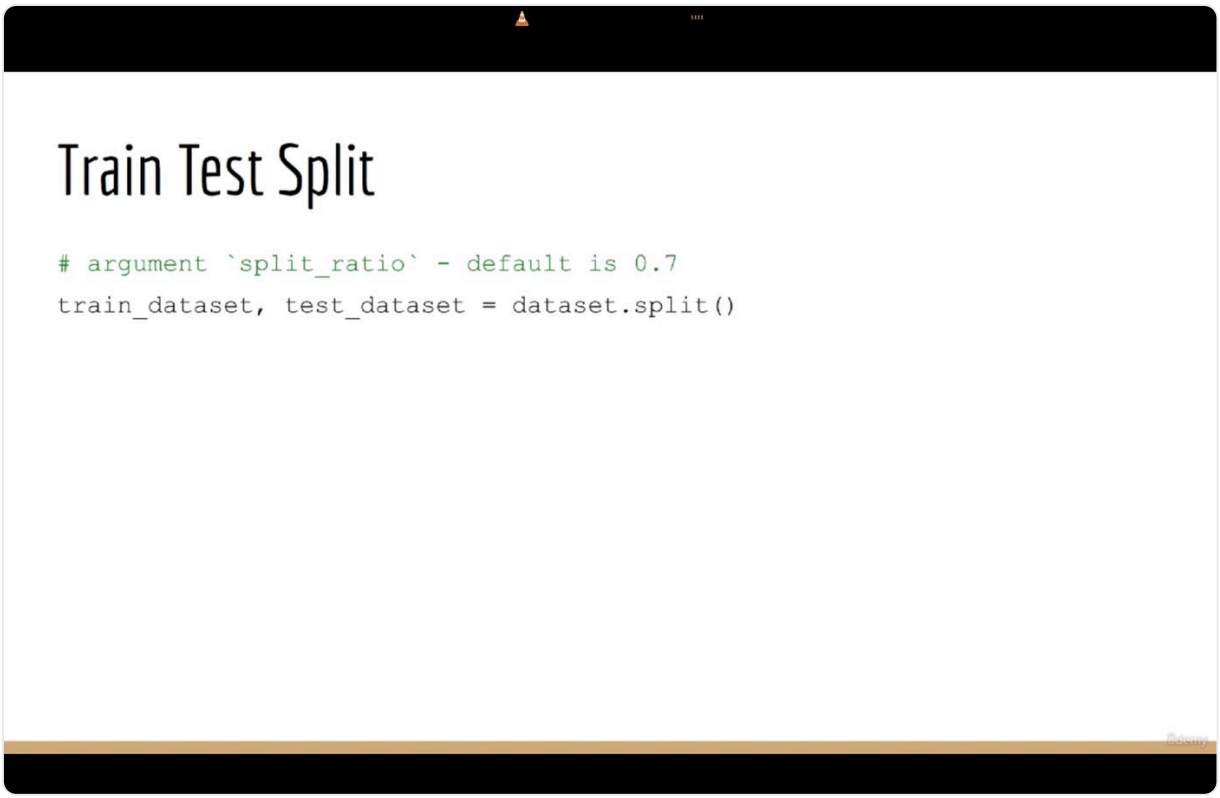


FIGURE 29.6 | Train Test Split

The second argument is the format which I have specified as CSV. Other options are TSV and JSON but these are not so significant in my opinion. A TSV is just like a CSV but with tabs instead of commas it's trivial to switch between them so there's no advantage to using one over the other. JSON is nice but it just adds extra characters to your data file without any real benefit. You can use it if you want but again there's no distinct advantage unless for some reason your data already came in that format. Next we set skip header equal to true since in our dataset there is a header row that will want to ignore. Finally we set the fields argument which is a list of tuples each tuple Contains two items.

The name of the field and the corresponding field object. Called the first one data and the second one label. The next step is to split the data into train and test sets which is nice and easy with torchtext just call dataset dot split and this returns two items your train set and your test set as input. You can pass in an argument called Split ratio to say what percentage of the dataset you want to be part of the train set. By default this is 70 percent so I haven't bothered to change it.

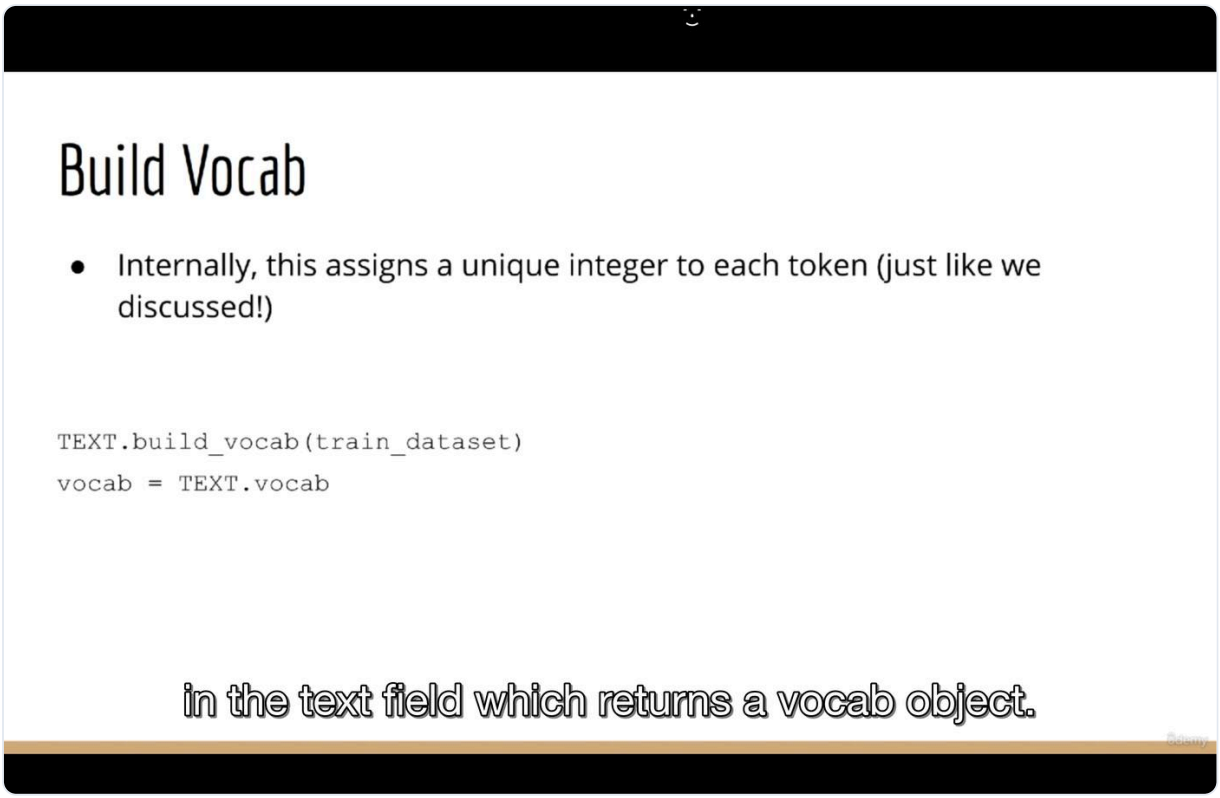


FIGURE 29.7 | Build Vocab

The next step is to call the build vocab function on our text fields from earlier this assigns a unique integer to each unique token in the dataset so you can imagine that internally torchtext is doing the tokenization and all the other text preprocessing steps that we discussed earlier. This function by itself doesn't return anything but after doing this we can access the vocab attribute in the text field which returns a vocab object. So what is a vocab object Well the vocab object contains that words index mapping that we talked about in the previous lecture. You can access it by calling vocab dot stoi so stoi should remind you of C style naming if you've ever coded in the C programming language before.

This returns a dictionary where the key is a token and the value is a unique integer that corresponds to the token. Additionally you can call the vocab dot itos function which returns the opposite. In particular it returns a list of all the unique tokens in the dataset.

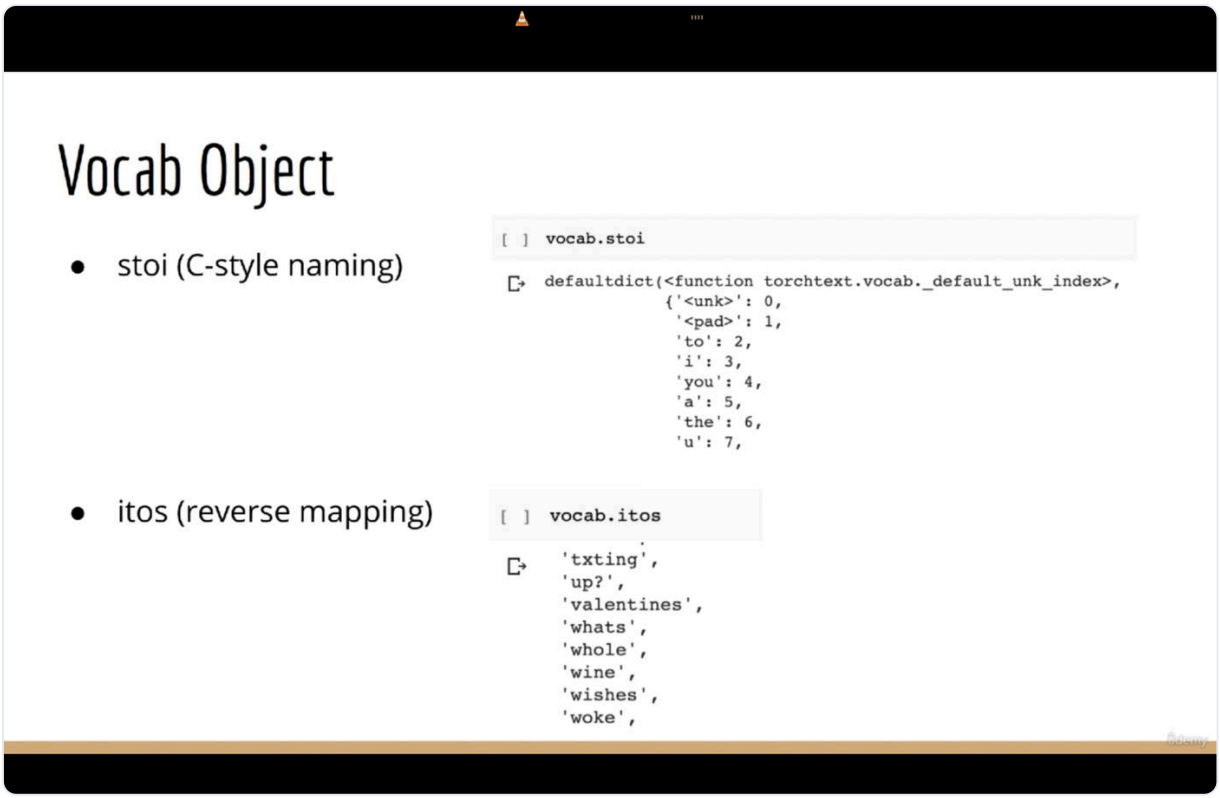


FIGURE 29.8 | Vocab Object

Now you might be wondering how can the opposite of a dictionary be a list well. Remember that a dictionary is nothing but a set of indices and corresponding values. So too is a list. A list has the indices 0 up to K minus 1. If the list has length k the values of a list are just the elements of that list. So if I call vocab dot stoi of dog and I get back two eleven then if I call vocab dot itos of two eleven I will get back the string dog so they are inverse mappings relative to each other. Now you might be wondering who cares and why do we even need these mappings.

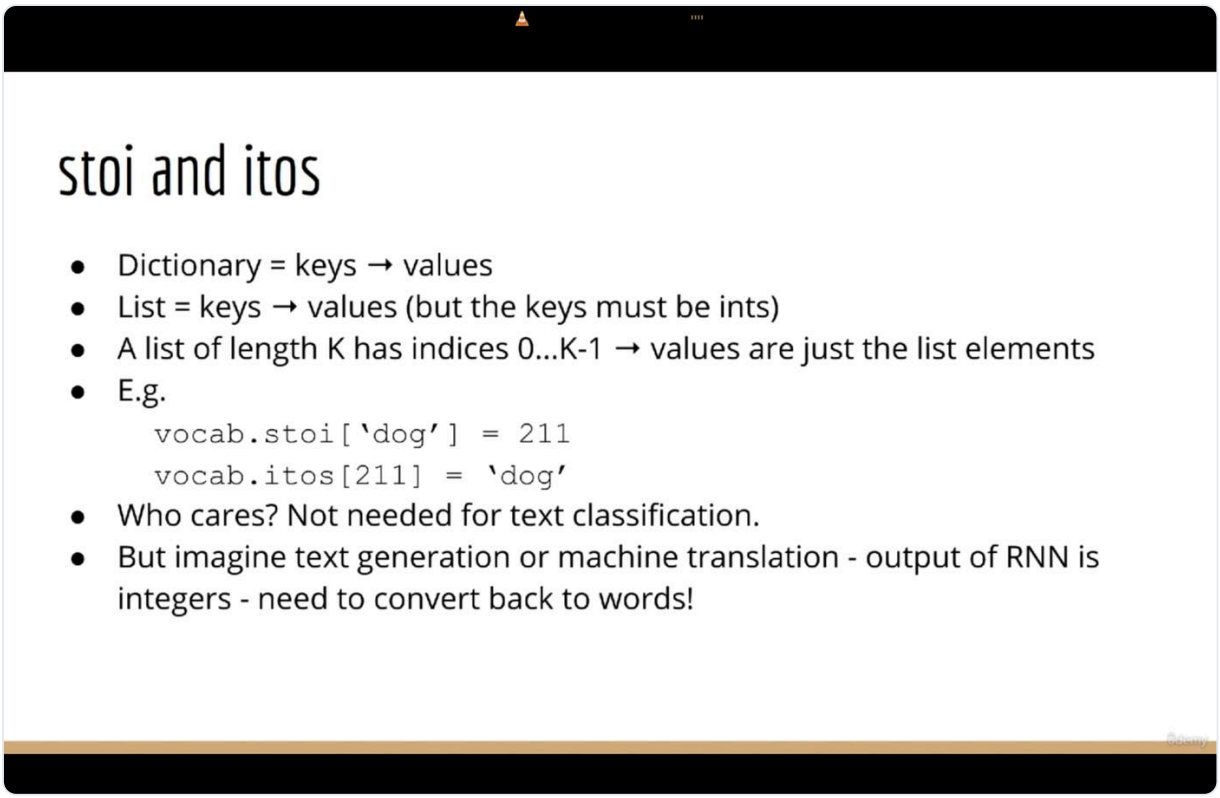


FIGURE 29.9 | Stoi and Itos

Well if all we want to do is text classification then we don't necessarily but there may be instances where let's say you're doing text generation or machine translation where the output of the neural network is a sequence of integers. Of course in order to turn that into text you have to convert that sequence of integers back into a sentence and in order to do that you must know which integer corresponds to which word in your vocabulary. Okay. So at this point we have a train data set and a test dataset. Now what. Well imagine your dataset is huge so huge that just like our image datasets we rather do batch gradient descent rather than full gradient descent.

So just like in torchvision where we had an iterator over our image dataset we would like to use an iterator from torchtext for our text dataset. Here's an example of how we would create an iterator for our text datasets the first argument is a tuple containing the datasets that we previously created. The second argument is a sort key. The idea behind this is that because we'll need to pad each batch of our dataset we would like each of the sentences in each batch to be of similar size. So by specifying the sort key to be a function that returns the length of the text field we can tell torchtext to organize the batches so that each sentence will be of roughly the same length.

Next we pass in batch sizes. This is a tuple containing the same number of elements as we have datasets. Basically the first number represents the batch size of the train data set and the second number represents the batch size of the test data set. Usually we can specify the batch size of the test data set to be larger. Since it doesn't need to do any heavy processing other than just making predictions finally we set the device argument to the device objects that we've been creating in all of our notebooks which refers to the GPU. So this means that our dataset will be automatically placed on the GPU without us having to do so manually all right. So at this point you should be asking what's next.

Since we now have iterators that can loop through our train set and our test set and return batches that are just N by T arrays of integers corresponding to each sentence. There's nothing more to do. We can pass this data directly into our RNN for both training and testing so you can imagine that our training

loop will look like it usually does as well our loop to calculate the accuracy one minor caveat is that the torchtext iterator gives us back a target that is only one dimensional. We know that in PyTorch we like to have two dimensional targets so we can call the view function to convert each batch of targets from an N length one dimensional array to an N by 1 2 dimensional array.

LECTURE 68

Data Frame

Check the notebook `16.PyTorch-Text-Processing.ipynb` for the code.

In this lecture we are going to work on a notebook to just do text preprocessing so we can explore a little bit how it works. This lecture will be useful before we move on to the next script where we will assume that we already know how it works. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. OK so we're going to start by making some fake data since we eventually want this to be saved as a CSV. It's convenient to create a data frame but in order to do that a convenient way to specify the data is in a dictionary. So in this dictionary the key is refer to our eventual columns and the values are lists that correspond to each row in the data.

So we're going to have three samples in our dataset. The labels are 0 1 and 1. The sentences are I like eggs and ham eggs I like in ham and eggs or just ham I've added some punctuation to this dataset so we can see what happens to it in our tokenizer. Next we pass this data into the data frame constructor to instantiate a data frame. Next we do a df dot head so we can look at the data frame. Hopefully this is what you expected to see next. We save the data frame to a CSV file by calling the to CSV function we can also use the command line version of head to see the contents of the file. Next we're going to create our 2 field objects one for the text and one for the label for the text.

I've set sequential equal to true since our data is a sequence of words I've set batch first equal to true since I want the data to be N by something rather than something else I've set lower equal to true so that the data is lowercase I've set tokenize equal to spacy. This might seem odd to you but spacy is another NLP library in Python that is somewhat tangentially related to PyTorch. Basically if you leave this argument unsaid it's just going to use a string that split by default. I would recommend you try both to see what the differences are. I wanted to use spacy here to show you how it deals with punctuation. Finally we set pad first equal to true so that the padding goes at the beginning of each sentence rather than at the end.

Next we set the label field. Here we set sequential equal to False. Since this column does not contain sequential data we say use vocab to false. Since this column contains no vocabulary and it's just integers and finally we set is target equal to true. Since this is the target column next we create a tabular data set object where we pass in the CSV file. We created earlier we said format equal to CSV since it's a CSV and we set skip header equal to true since pandas saves the header row by default.

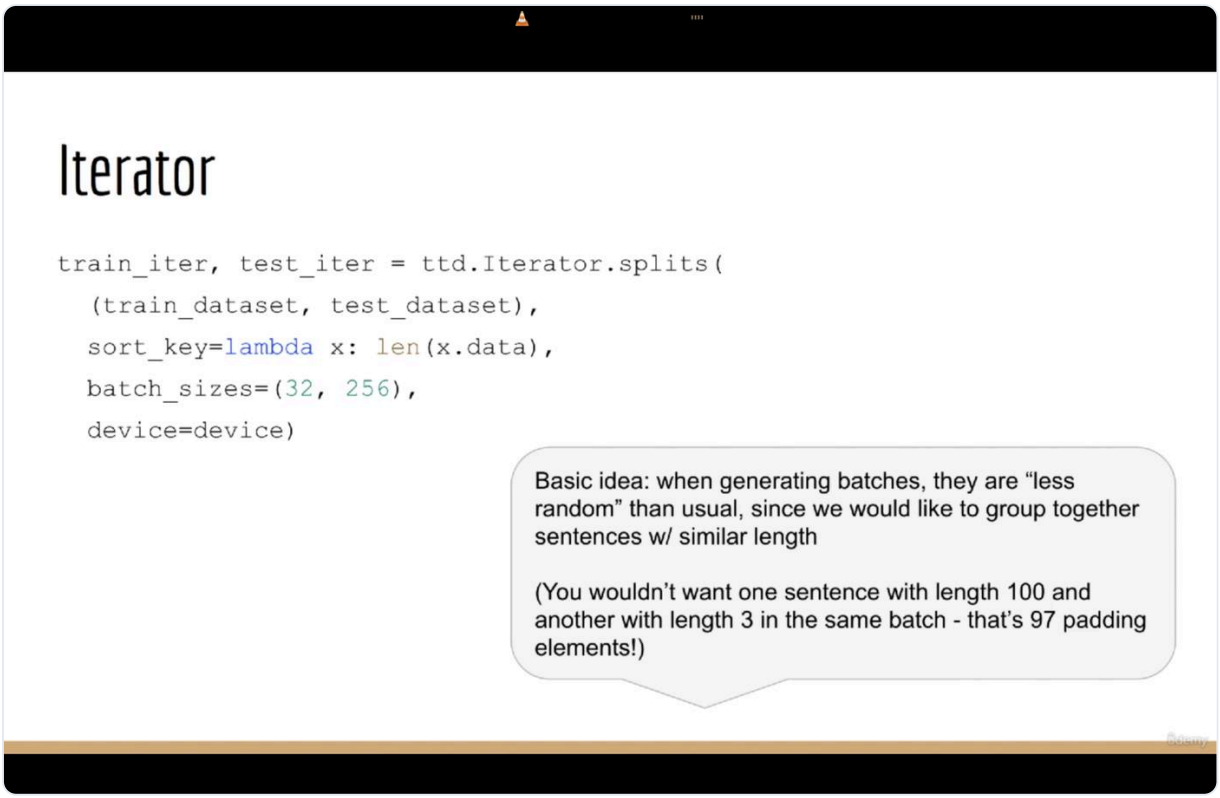


FIGURE 30.1

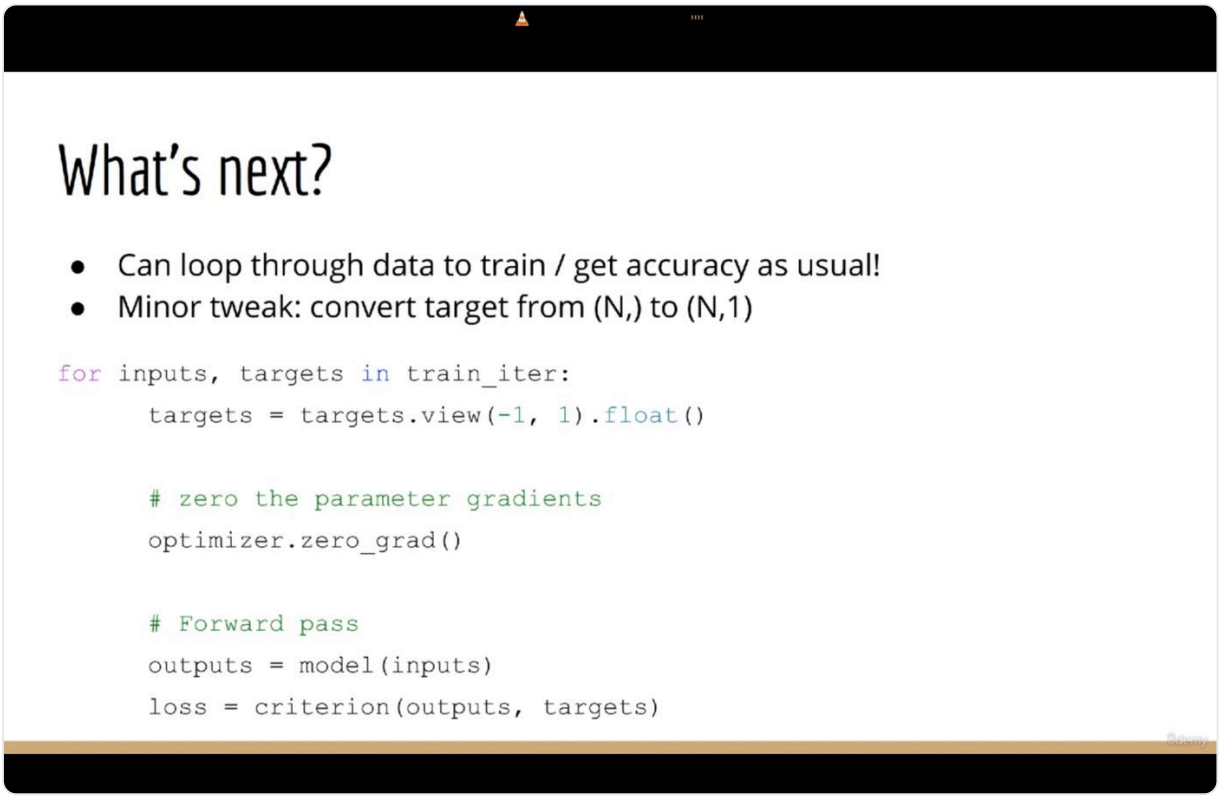


FIGURE 30.2

We set the fields argument to a list containing the fields we just created one important note is that these fields must be specified in the same order as they appear in your file. So if your file contains the data column to the left of the label column you would put the data fields first so if you check the attributes of the data set object you'll see it has one called examples which is a list of example objects. What I've done

here is I've grabbed the first one so we can see what's in it. As you can see there is an attribute called data. And if we look at this attribute we can see that it contains our sentences but not in their regular form. In fact our sentence is now tokenized.

You can see that the period has been separated from the word ham which would not be the case if you just did a plain string that split if we look at the label it just returns zero as we would expect next. I'm going to split the dataset. This is kind of weird in our case since the data only contains three samples. But let's just pretend this is OK. Next we're going to call the build vocab function on our text field and pass in the train dataset. Oddly this is not done automatically when you create the dataset object. Again it's not a mature library next we call the vocab attribute and check its type. We can see that it is a vocab object next we call that attribute stoi which returns a dictionary since our dataset is so small which I did on purpose we can see the entire dictionary as you can see each token in the dataset is assigned a unique integer.

Also punctuation is considered a token since we use the spacy tokenizer we can see that the unknown token gets assigned the index 0 and the pad token gets assigned the index 1. We can also see that all the words are lowercase next. Let's look at the itos attribute. We can see that it contains a list of our tokens. You should confirm to yourself that the indices of each element in this list corresponds to the values in the dictionary above. Next we create a device object as we normally do. Next we create an iterator from our datasets. We set the batch size equal to 2. Since both the train and test set only have a maximum size of 2 next we loop through the train iterator and print the inputs and targets along with their shape.

As you can see the shape of the inputs is 2 by 7. Next we do the same thing for the test iterator and the shape of the inputs is 1 by 6. So what do we learn from this. Well we learned that PyTorch already intelligently makes your batches have the smallest sequence length possible. There is no unnecessary padding in the train batch or the test batch. This is good since for example if our data has a maximum sequence length of 10 but the sentences in our batch only have length 2 or 3 then that would be a huge waste of computation finally if we look at these sequences of integers carefully we can probably deduce which sequence belongs to which original sentence.

The one with all the padding is the easiest because that corresponds to the shortest sentence in our small dataset. As an exercise here's what you want to do instead of just deducing which sentence goes with which sequence make use of the index to token mapping from earlier to programmatically convert these sequences back into their original sentence form.

LECTURE 69

Index Dictionary

See the notebook `17.Custom-PyTorch-Text-Classification-using-RNN.ipynb` for the code.

In this lecture, we are going to look at a Colab notebook that does text classification using RNNs, the specific text classification we're going to work on is spam detection, which, as you can probably surmise on your own, is a very practical application. Note that there are some other examples of a text classification, such as sentiment analysis, where you can try to guess whether a passage is positive or negative and also just document classification. For example, if you're a news site and you have thousands of articles, you may want to automatically classify them into different categories on your site, such as technology, local news, lifestyle, health, weather and so forth.

So there are lots of basic applications of text classification. This lecture is going to walk you through a prepared Colab notebook, although a very good exercise, which I always recommend, is once you know how this is done, to try and recreate it yourself with as few references as possible. As usual, you can look at the title of the notebook to determine what notebook we are currently looking at. To start, we're going to load in our data set, which is a spam detection data set stored as a CSV. We use the get method to download the data first. You'll notice that this is a very strangely formatted file. If we use the head commands, you'll see that there appear to be some invalid characters.

In addition, each line ends with three commas, but none of those columns contain any data. The next step is to load in our CSV using pandas read CSV notice that I've used in encoding called ISO eight eight five nine, dash one. The default is UTF-8, which does not work on this file due to invalid characters. The next step is to do a df dot head to see what our data looks like. It seems that the first two columns are simply called V1 and V2, and there are three completely empty columns after that. This makes sense since each line of our file ended with three commas. So in the next line, I'm going to drop these three unnecessary columns. The next step is to do a df dot head.

And we see that the three columns are gone as expected. Next, because I don't like having my columns named, if you wanted me to, I'm going to name them something a little better labels and data. The next step is to do a df dot head again to check that the columns have been successfully renamed. So currently, the labels are called ham and spam, as you know, in machine learning, we like our columns to be integers from zero up to K minus one. In our case, since K is two, we want our labels to be zero one one. We'll map ham to zero and spam to one will assign this new column to B B labels. The next step is to split our data into train and test. The next step is to check the shape of our data sets just as a sanity check.

So in the next few blocks of code, we're going to begin writing the code for text preprocessing. We'll start by initializing our word to index dictionary. Currently, this will only contain the padding token, which is assigned to have the index zero. So that's the format of this dictionary. The key is the word and the value is its corresponding index. We also have a variable called idx, which I've initialized to one. Basically, this variable will store our current index as we loop through the train set. So notice it's initialized to one because zero is already taken by the padding token. But you'll see precisely how this word to index dictionary is used very shortly.

The next step is to loop through each document in the train set, our goal right now is to populate the word to index dictionary, that is, we want to store all the words in the train set and assign them to all unique integer indices. So inside the loop. We begin by grabbing the data column, lower casing the

string and then calling the split method. This is the simplest way to do tokenization. Lots of people think tokenization is some magical operation, but in reality it's just a fancy string split. There are other things you can do, like slip punctuation or completely remove punctuation, but we won't consider those scenarios. Of course, it's easy to replace this line with anything you like.

For example, the tokenizer from NLTK spacy or Gensim, it's essentially no effort to do that. So if you want to try that on your own, please do. The next step is to loop through each of our tokens inside this loop. We're going to check whether or not the token is currently in our word to index dictionary. If this is not the case, then we need to create a new entry, otherwise we don't need to do anything. So inside the statement, we simply add this token as a key and set idx to be the value. We then increment idx to prepare it for the next token we find. OK, so hopefully you can see how this loop will populate the word to idx dictionary with unique indices assigned to each word in the training corpus.

The next step is to simply print out the word to index dictionary to confirm that it contains what we expect. OK, so hopefully this is what you expected to see. The next step is to check the size of the word to index dictionary to see how many unique tokens we have. So as you can see, we have about ten thousand unique tokens. The next step, as you recall, is that we need to convert our text data into data that can be accepted by a neural network. This means we have to convert each document into a list of word indices in order to do this. We'll start by creating an empty list called train sentences as ints, hopefully for obvious reasons. The next step is to loop through the train set.

Once again, note that I've made a little comment here, which reminds you that we could have done this step and the previous step all at once. That is, we could have populated word to idx and converted the data into lists of integers at the same time. So you may want to try that as an exercise. I think doing it separately makes things a little more clear. OK, so inside the loop, we're going to re tokenize the current document so you can see how this is a bit inefficient since we've tokenized every document twice. Once we have the tokens, we then map them to integers using the word to idx dictionary after we get the result, which I've called sentence as int, we then append that to our list of train inputs.

The next step is to do the same kind of loop for the test set, but this time it's going to be a bit different. The main reason is because we can't be sure that every word that appears in the test set also appeared in the train set one common way of dealing with this is to simply not include those words. Another common method is to have a special token just for unknown words. But then you would have to train. Your neural network weights for unknown words as well. In this case, we're just going to ignore any words that do not appear, in our word to idx dictionary. Other than that, this loop is the same as above. OK, so the next step is to check the size of our train and test inputs.

So as you can see, they are the same as before. The next step is to create a data generator. This is optional since theoretically you could just loop through the data structures we already have. However, this makes the subsequent PyTorch code more similar to what you've seen before. So this function is going to take in three arguments, the inputs X, the targets Y and the batch size inside the function. We're going to start by shuffling the inputs and targets, as you recall, when we do batch gradient descent. We like to shuffle the data on each epoch. We then compute the number of batches, which is the ceiling of the size of Y, divided by the batch size.

The reason we want to use the ceiling is because if they don't divide evenly, we still want to include the leftover points. The next step is to loop through each of our batches. Inside the loop, we're going to start by computing the end index of the batch, so normally this would just be a plus one times batch size. But remember, the final batch may not be a full batch. So we need to account for that by taking the min of

the usual value and the length of y. The next step is to index X and Y to obtain X batch and Y batch using the indices we just found. OK, so please double check these indices and make sure they make sense to you. If they do not make sense to you, then apply my rule when in doubt, print it out.

OK, so currently X batch is a list of documents, each represented as a list of integers. There's one problem with this, which is that each document in our dataset could have a different length. In fact, they most likely do by default. RNNs don't work with sequences of variable length. So unless you want to write some additional code to handle that, then the typical way to deal with this is to make each sample in the batch have the same length. In order to do this, we first have to find the longest sequence in the batch. Of course, this is so that we can include all the data that appears in the batch, so all the shorter sequences will have padding and the longest sequence will not have any padding.

OK, so max len is just the maximum length of the documents in the batch. The next step is to loop through each sample in the batch. Inside this loop, we first grab the sample, which is called Little X. So little X at this point is a list of integers representing the document. As always, when in doubt, print it out. OK, so the next step is to determine the amount of padding. Of course, this is just Max Len minus the length of X. This is because after padding, we want all the documents to have the length max len. So we multiply max len minus the length of X by a list containing zero, which, as you recall, doesn't actually do multiplication. When you use the asterisk operator with a list, it simply repeats the list that many times.

So what we get back is a list of zeros. The final step is to join the padding and X together and store this in X batch. So this will overwrite what was in X batch at the start of this loop. Now, once this inner loop is complete, X batch will be a non jagged array. So at this point we can convert both X and Y back into torch Tensors. The final step in this function is to yield the X batch and Y batch. As you recall, this is a generator. So we want to yield and not return. The next step is to test their new function. This will also show you how our generator is to be used. As expected, we use it in a loop and it takes in our inputs and labels as arguments inside the loop.

We're going to print out one input target pair along with their shapes, will then break out of the loop since we only need to see one. OK, so here you can see the input tensor, which is a 2D array of integers as expected, where most of the rows just start with a bunch of zeros. This makes sense since that's the padding. We can see that the target tensor is just an array of zeros and ones with a size of thirty two as expected. The next step is to do the same thing, but with our test data set, just so you can see how we can loop through that data as well. As you can see, it's pretty much the same thing. The next step is to set the device to be the CPU.

Next, we define the model. This is the same as the RNNs we looked at earlier, but with an embedding layer up front, so as input we take in the vocab size, the embedding dimension, the number of hidden units, the number of hidden layers and the number of outputs, the two relevant things for us in this section are, of course, the vocab size V and the embedding dimension since that specifies the size of our embedding matrix. Next, we create our three layers, the embedding, the LSTM and the final dense layer. In the forward function, we pass our data through each of the layers, first we create our initial LSTM states, H0 and C0. Next, we pass our data through the embedding layer.

This gives us an N by T by D. Next, we pass our data through the LSTM layer. This gives us an N by T by M next we do a global max pool. This collapses the T dimension and gives us an N by M. Finally, we pass our data through the last dense layer and this gives us an N by K. Next, we instantiate the RNN and move the parameters to the GPU as before, I'm going to speed through the usual boilerplate code which

we've seen. The next step is to create the loss and optimizer. The next step is to create simple lambda functions that can be used to call our data generators for training tests, you'll see that these shorter names are much easier to use throughout the notebook.

The next step is to define the training function. The thing to pay attention to here is how we use the train gen and test gen lambda functions also note that we explicitly move the inputs and targets to the GPU. Other than that, you've essentially seen all this before. The next step is to call the training function. The next step is to plot the train and test loss per iteration. OK, so the last iteration looks good. The next step is to calculate the accuracy again, notice that we simply use our train gen and test gen lambda functions. All right, so we end up doing pretty well in the high 90s, so conceivably, if you wanted to say create your own Web server that processes some type of email or digital message, you can now filter spam very accurately.

As usual, please take this opportunity to try different hyperparameters yourself and see if you can improve the results on your own. So please try that as an exercise and I'll see you in the next lecture.

LECTURE 70

1-D Convolution: Example

In this lecture, we are going to discuss how we can use convolutional neural networks for sequences, specifically text, although the type of CNN we are about to discuss will work just as well for generic sequences to. You might be wondering, how can this be if CNNs are for images?

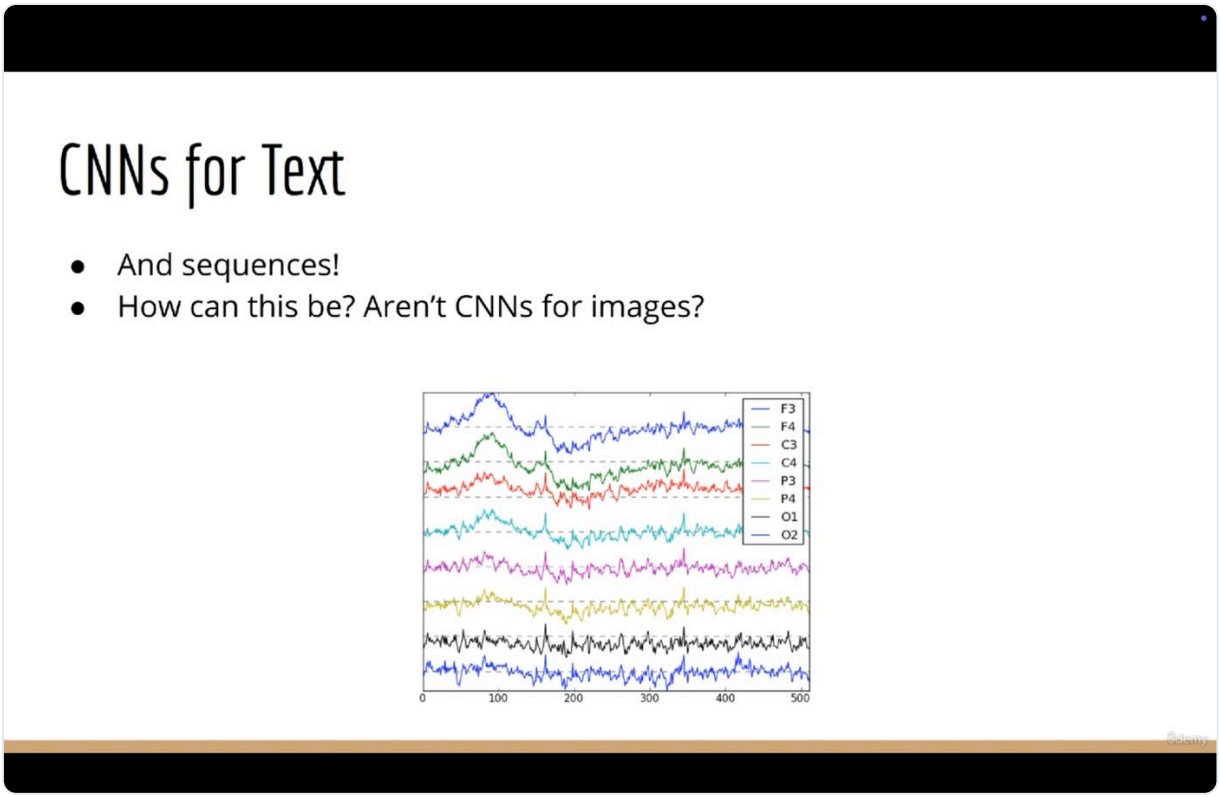


FIGURE 32.1 | CNNs for Text

How is it possible for CNN to also work with sequences? First, let's recall the basics of convolution.

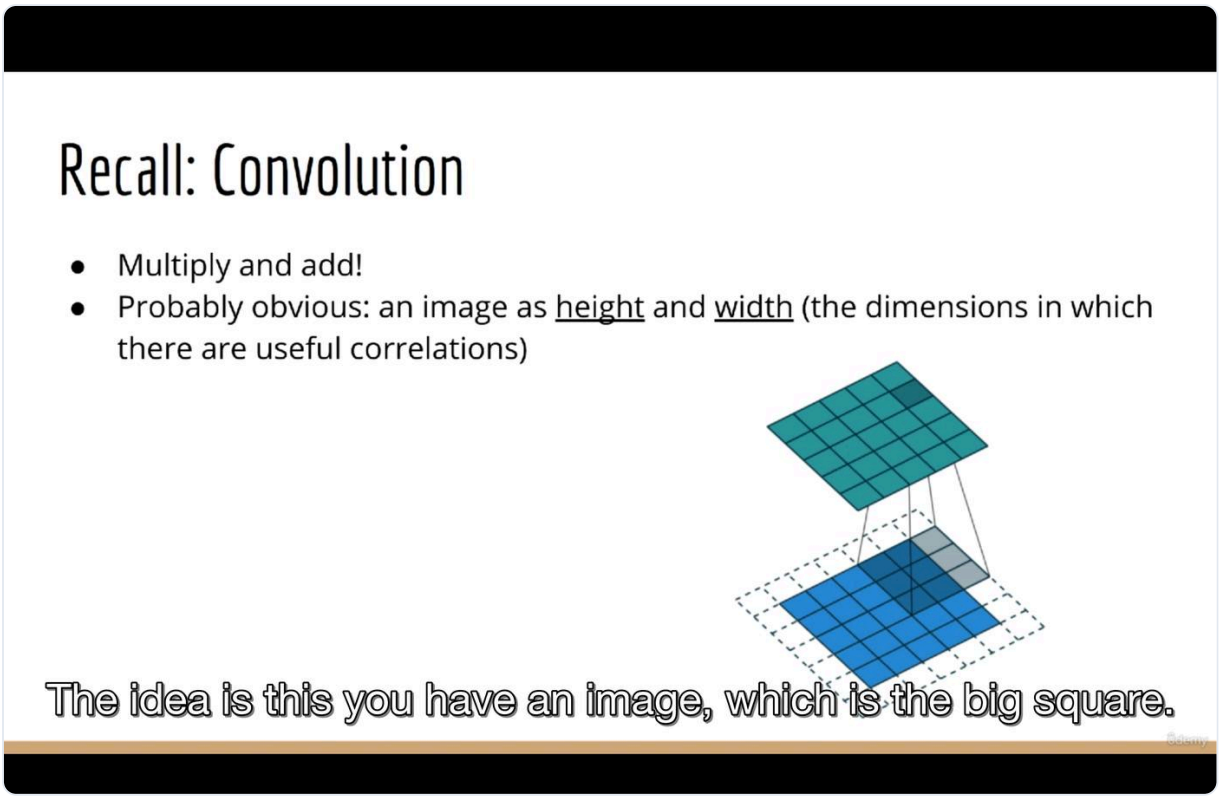


FIGURE 32.2 | Recall: Convolution

The idea is this you have an image, which is the big square.

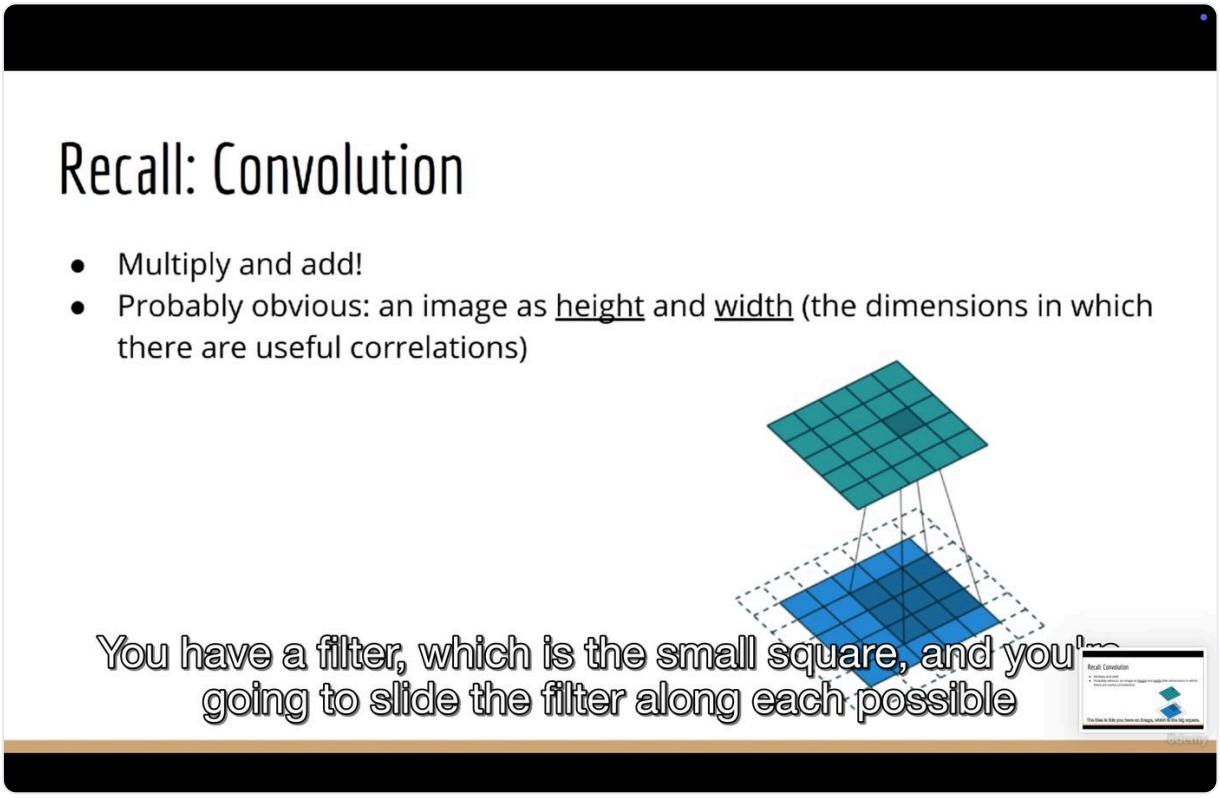


FIGURE 32.3 | Recall: Convolution

You have a filter, which is the small square, and you're going to slide the filter along each possible position in the big square. And at each point, you're going to multiply all the overlapping values and add them together just like a DOT product. Now, I know this is obvious, but it's worth taking note of an im-

age has two dimensions, height and width. There is also the feature dimension. But that's not an actual dimension which has correlation. And recall what I mean by correlation.

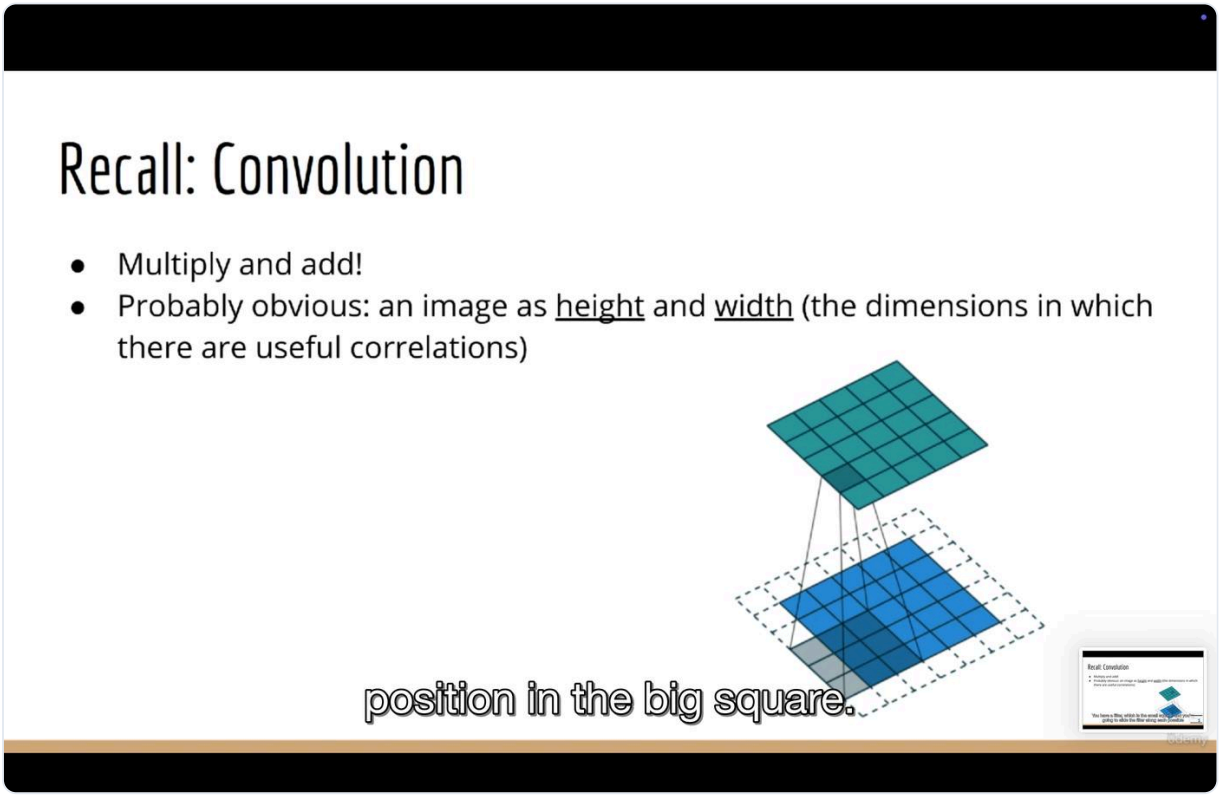


FIGURE 32.4 | Recall: Convolution

If you have a picture of a red car and you find a pixel in the image which is red, probably the neighboring pixels are also red. In other words, pixels beside each other likely have similar values. Now, think of a sequence, unlike an image, a sequence has just one dimension that is not a feature time. So it's time instead of space. But notice one important detail.

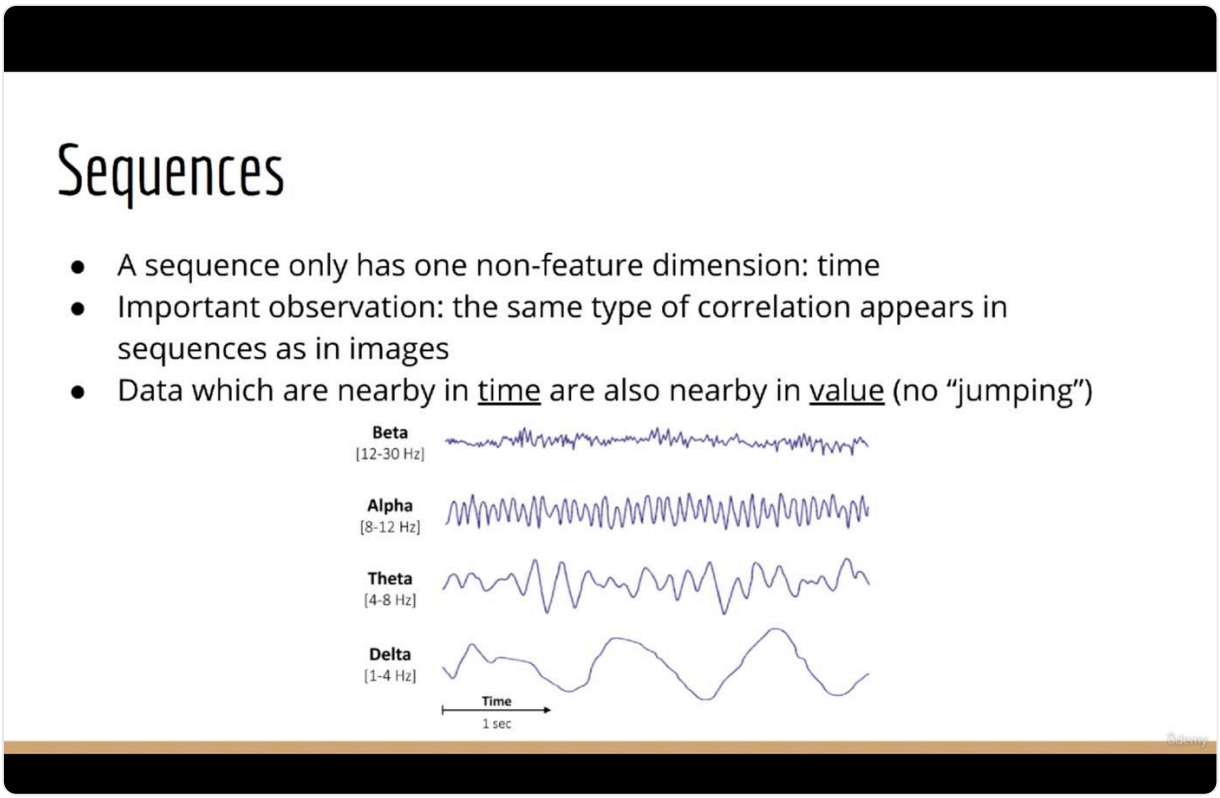


FIGURE 32.5 | A Sequence Only Has One Non-feature Dimension: Time

We have the same type of correlation and sequences, data which are nearby. Other data in time are also close in value. That's why this appears like a smooth curve rather than just noise jumping around with random, uncorrelated values. This suggests that convolution might be useful here as well. Luckily, convolution in one dimension is actually much simpler than a convolution in two dimensions instead of a big square. We just have a big line and the filter is a smaller line.

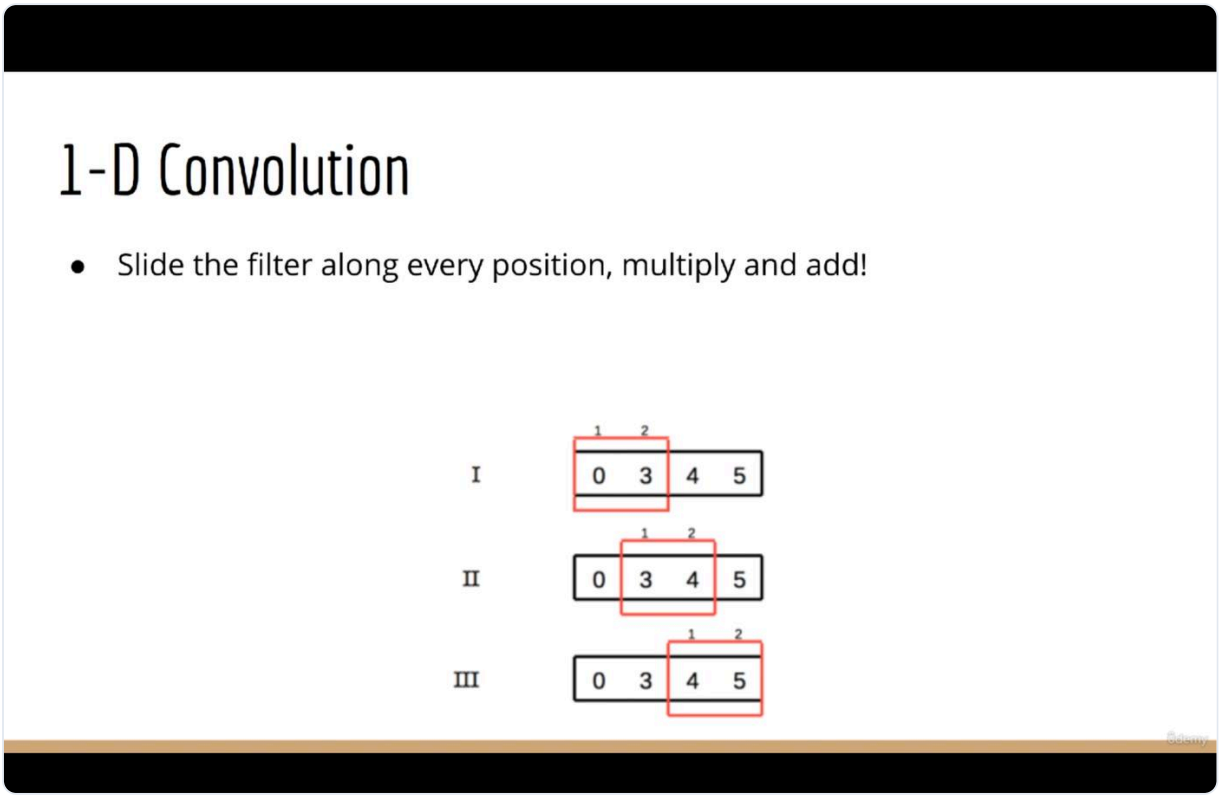


FIGURE 32.6

Then we slide the small line across every position in the big line. Multiply all the overlapping values and add them together. Let's do a simple example for the sequence data we have. One, two, three, two, one. And for the filter, we have simply plus one, minus one as an exercise.

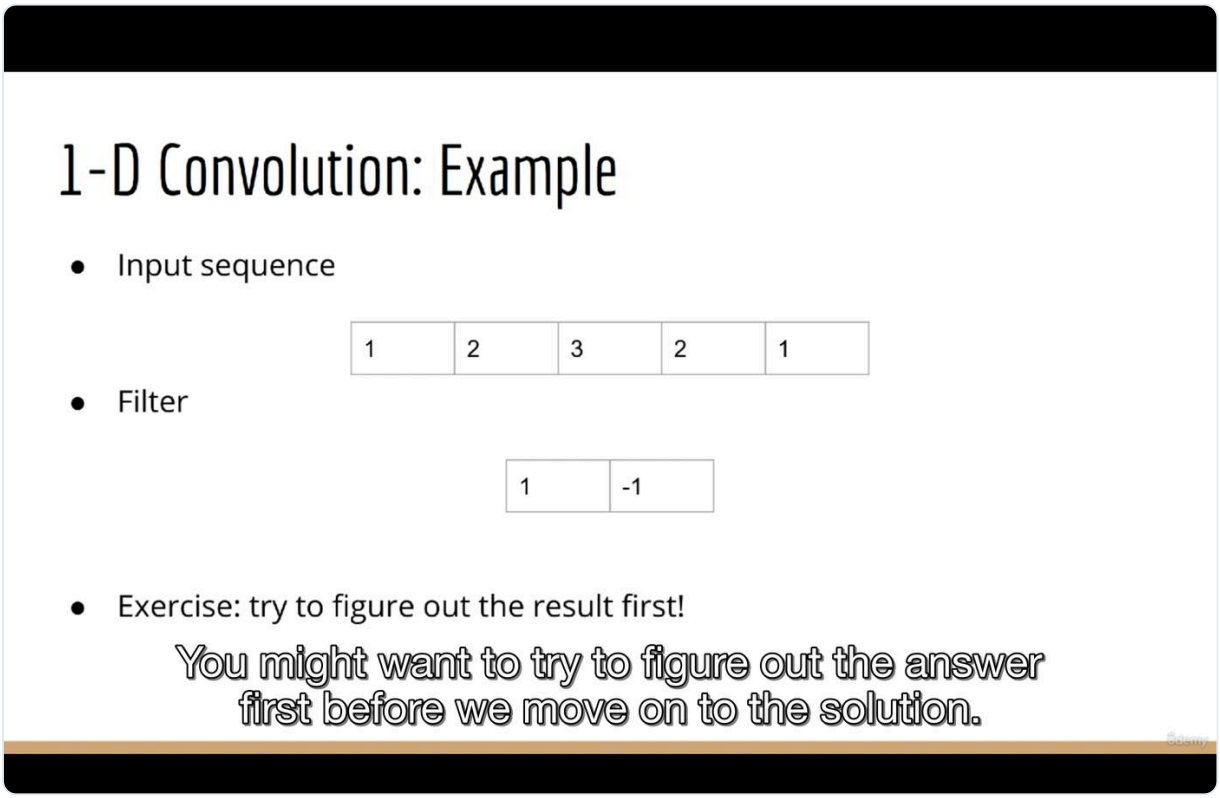


FIGURE 32.7 | 1-D Convolution: Example

You might want to try to figure out the answer first before we move on to the solution.

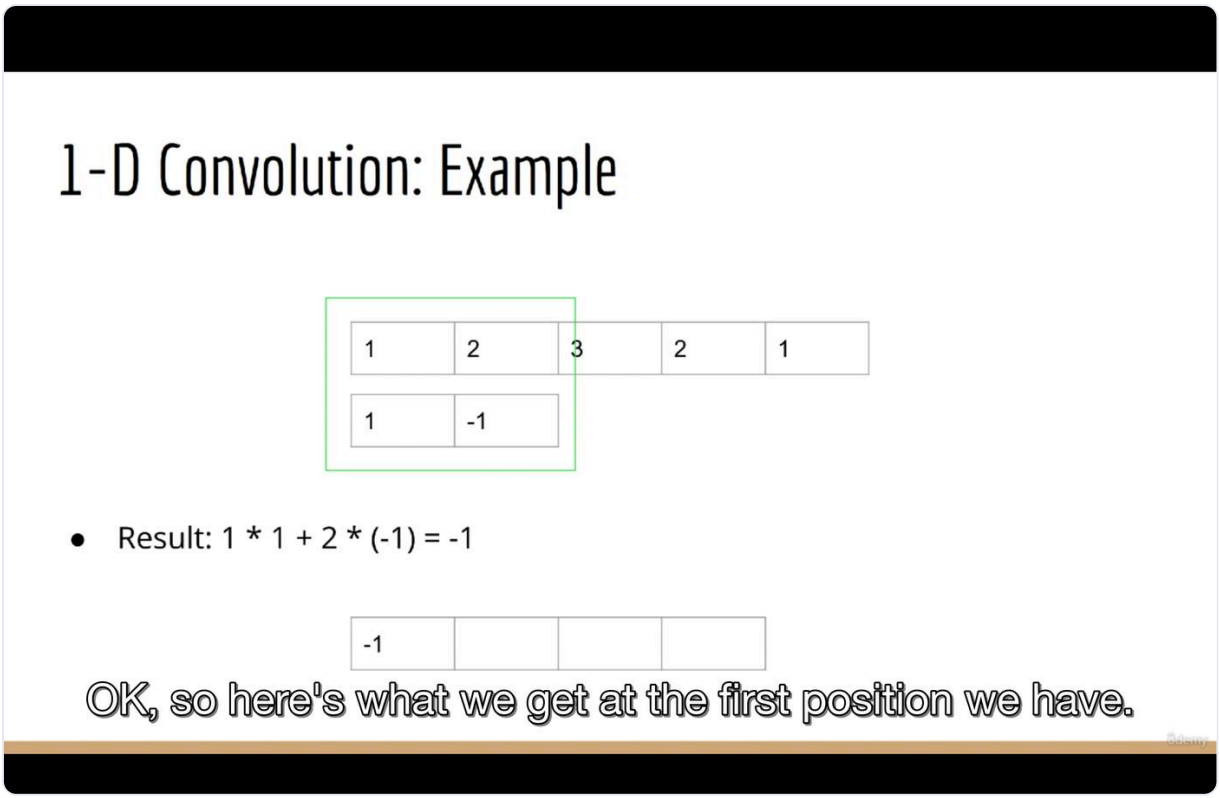


FIGURE 32.8 | 1-D Convolution: Example

OK, so here's what we get at the first position we have. One times one plus two times minus one, which is minus one.

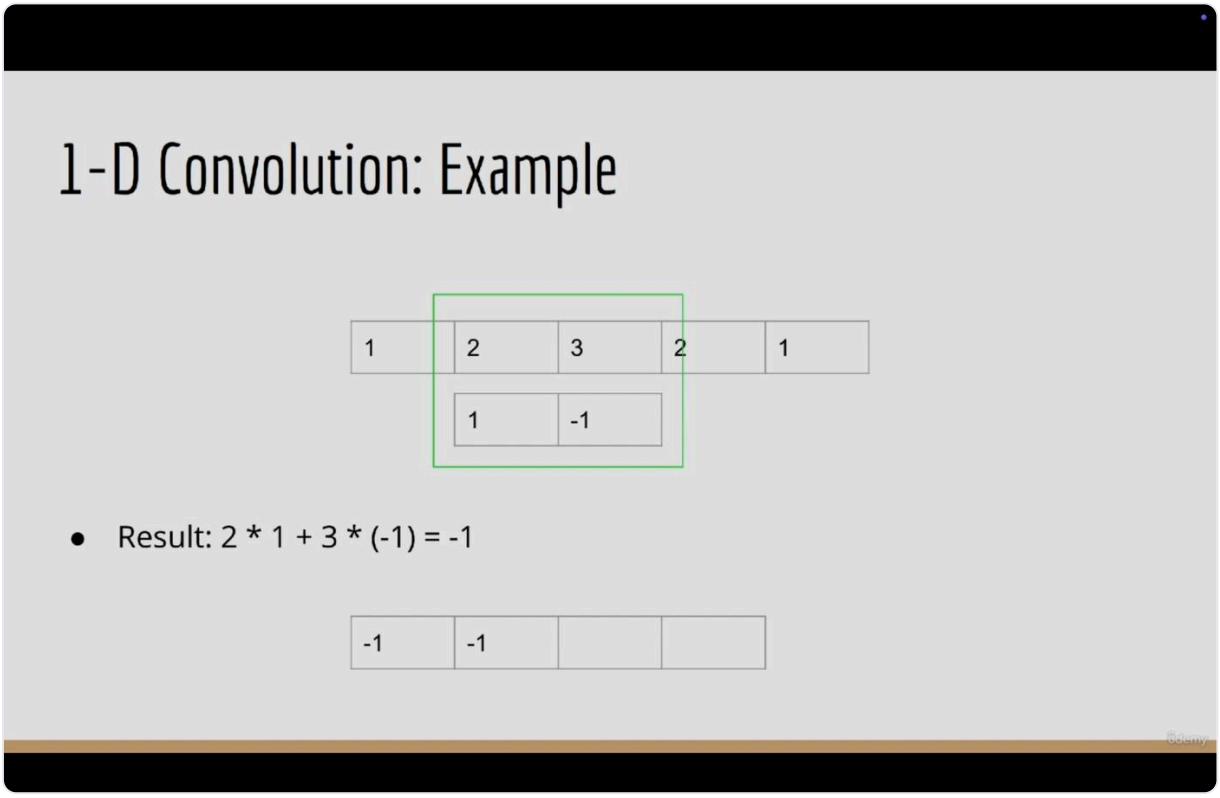


FIGURE 32.9 | 1-D Convolution: Example

At the second position, we get two times one plus three times minus one, which is minus one. At the third position, we get three times one plus two times minus one, which is one.

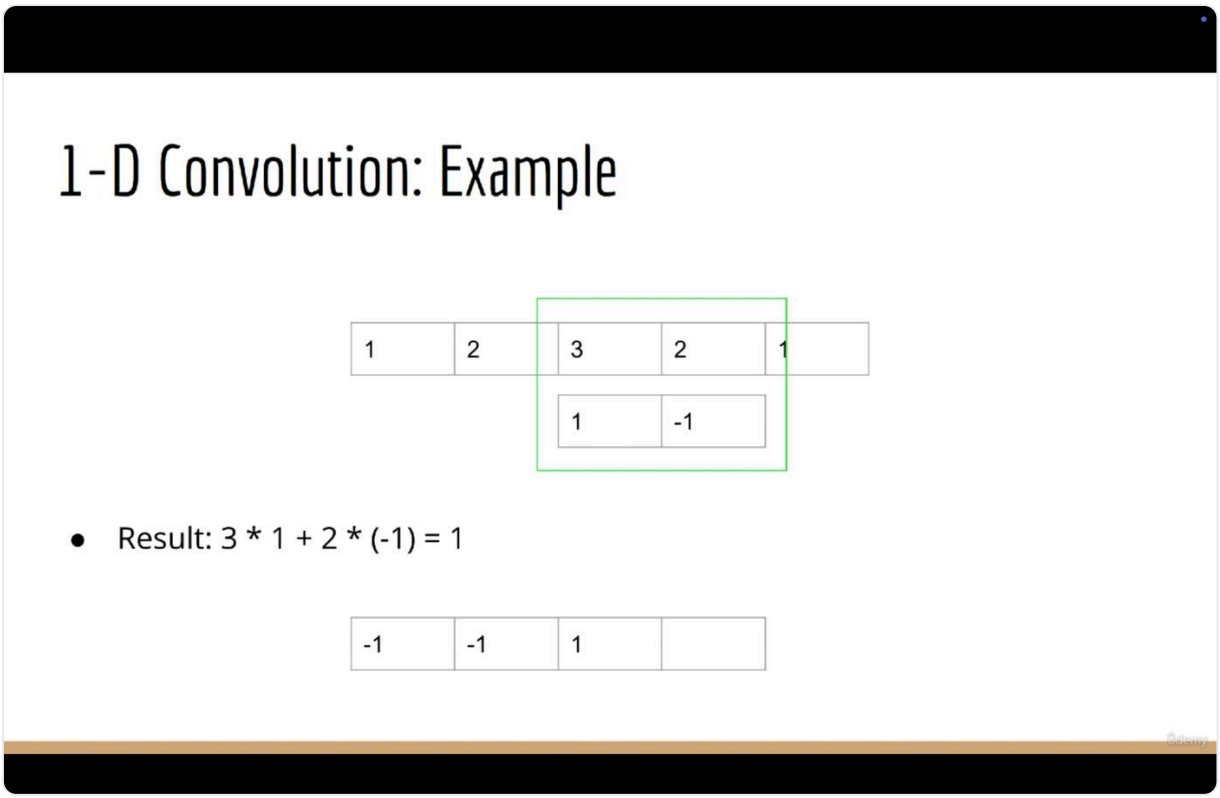


FIGURE 32.10 | 1-D Convolution: Example

And at the fourth and final position, we get two times one plus one times minus one, which is one.

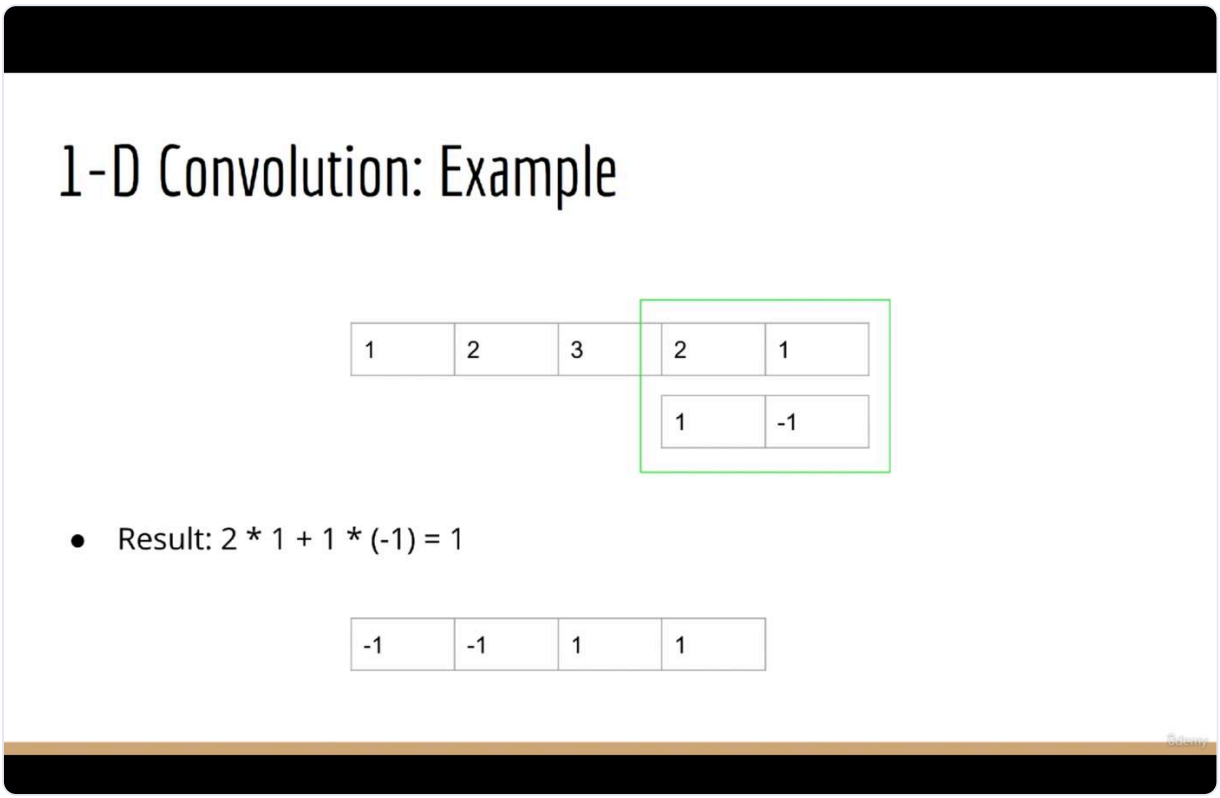


FIGURE 32.11 | 1-D Convolution: Example

So essentially exactly what we would expect. As with images, it's possible to express this operation as an equation.

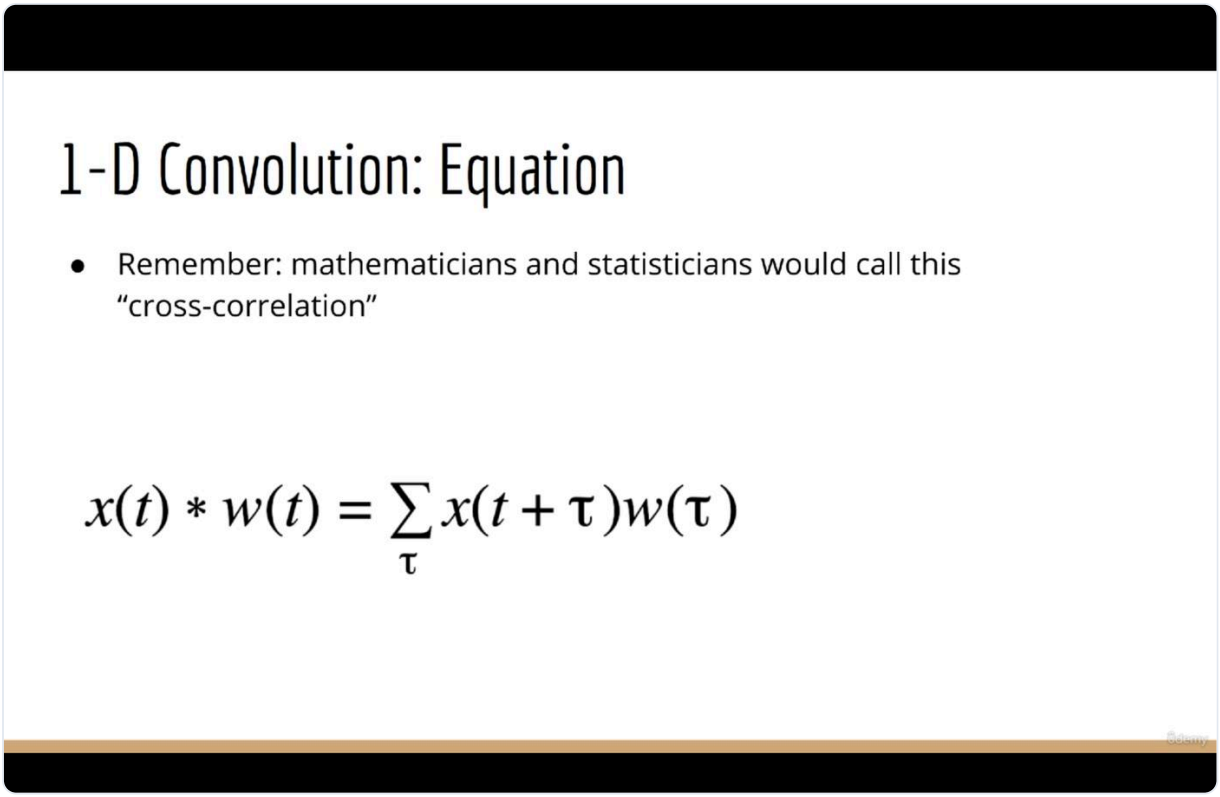


FIGURE 32.12 | 1-D Convolution: Equation

And remember that in deep learning, while we call this convolution, pure mathematicians and statisticians would call this cross correlation. So just keep in mind the sign is reversed as an exercise. You might want to try and confirm to yourself that this equation does, in fact, implement the operation we just performed in the previous example. And remember that another thing we can do is add on features. So imagine X as a T by D array where T is the number of times, steps and D the number of input features.

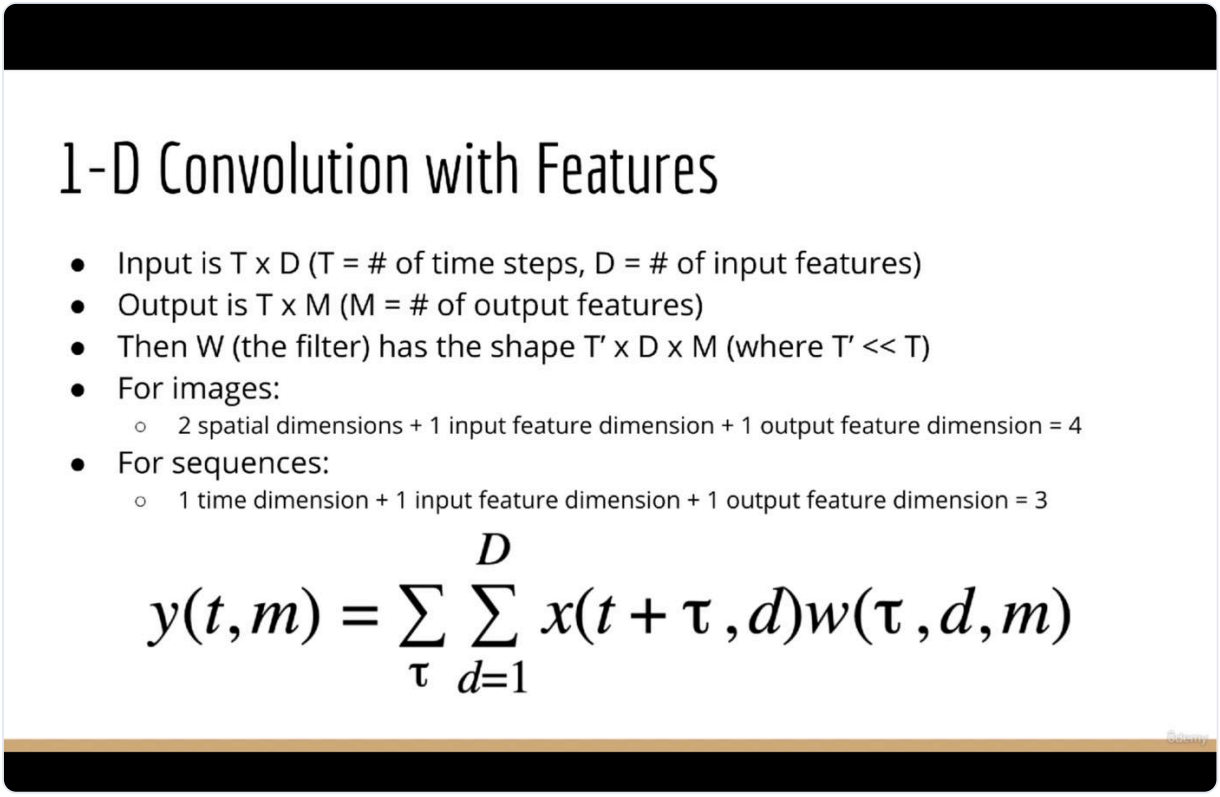


FIGURE 32.13 | 1-D Convolution with Features

Then imagine the output as another two dimensional array of size T by M, where T is the number of times steps and M is the number of output features. Then W must be a three dimensional array T by D by M. And then we would have the equation that we see here. This is just like convolution with images for a two dimensional convolution.

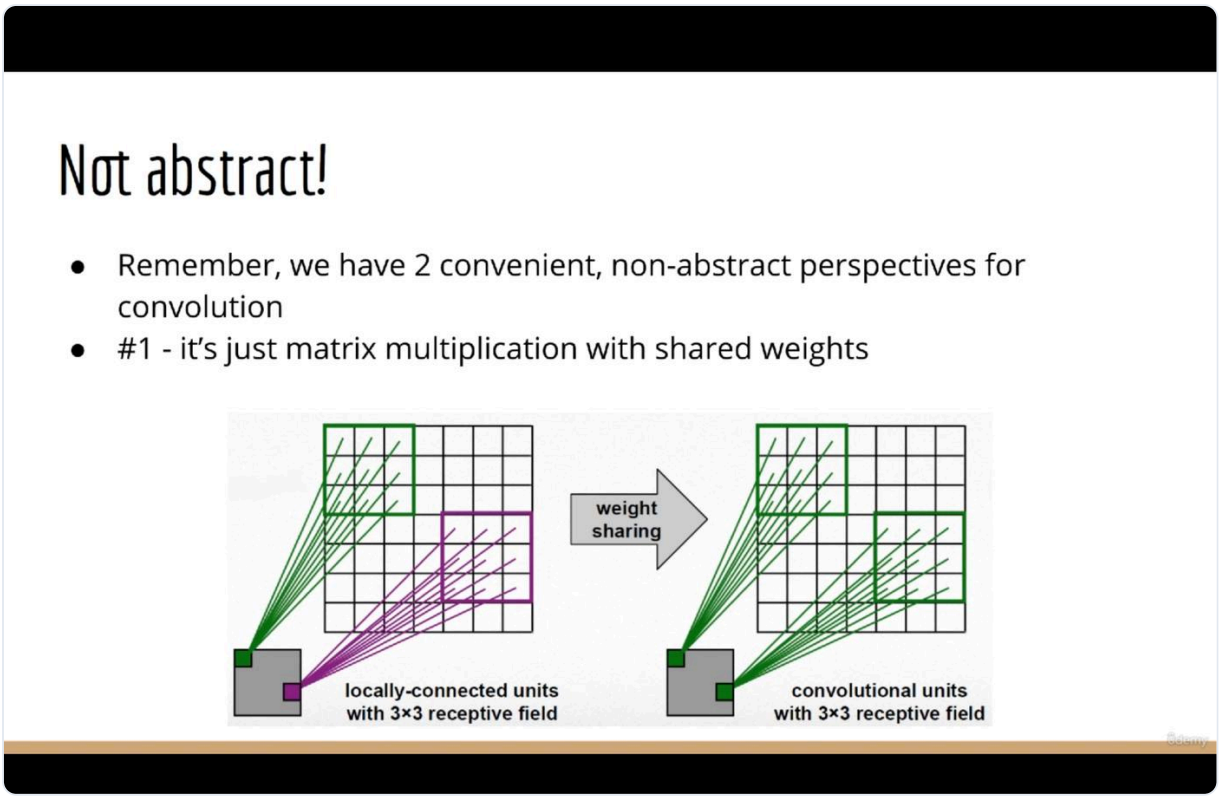


FIGURE 32.14

We have two spatial dimensions, plus one dimension for the input features plus one dimension for the output features. So that's four dimensions in total for a one dimensional convolution. We have one time dimension plus one dimension for the input features plus one dimension for the output features. So that's three dimensions in total.

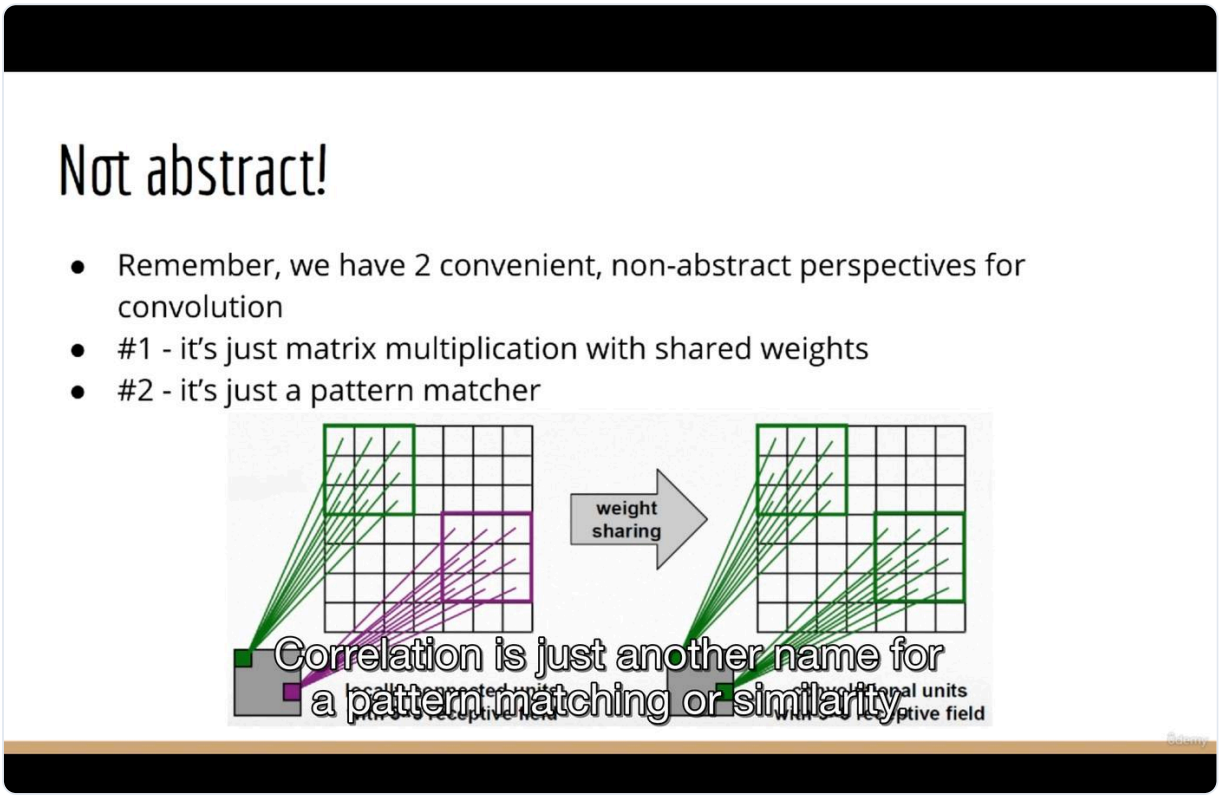


FIGURE 32.15

As usual, while convolution might seem like an abstract concept. Remember that there are multiple convenient perspectives on it that make it seem like things we are already familiar with. So one perspective is that it's just matrix multiplication.

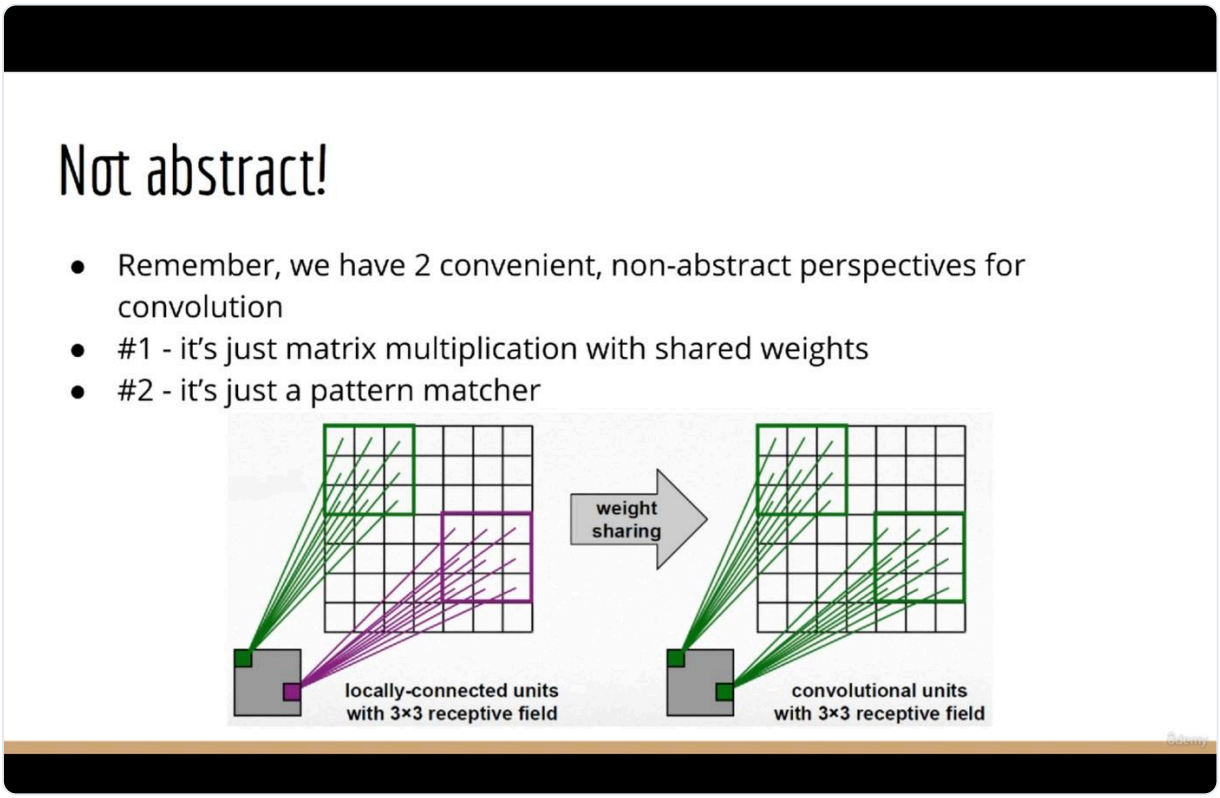


FIGURE 32.16

Just like a regular feedforward neural network. Except that we have shared weights in order to take advantage of the special structure and correlation and the data. Another intuitive perspective is that it's just a sliding dot product and the dot product is just a correlation finder. Correlation is just another name for a pattern matching or similarity.

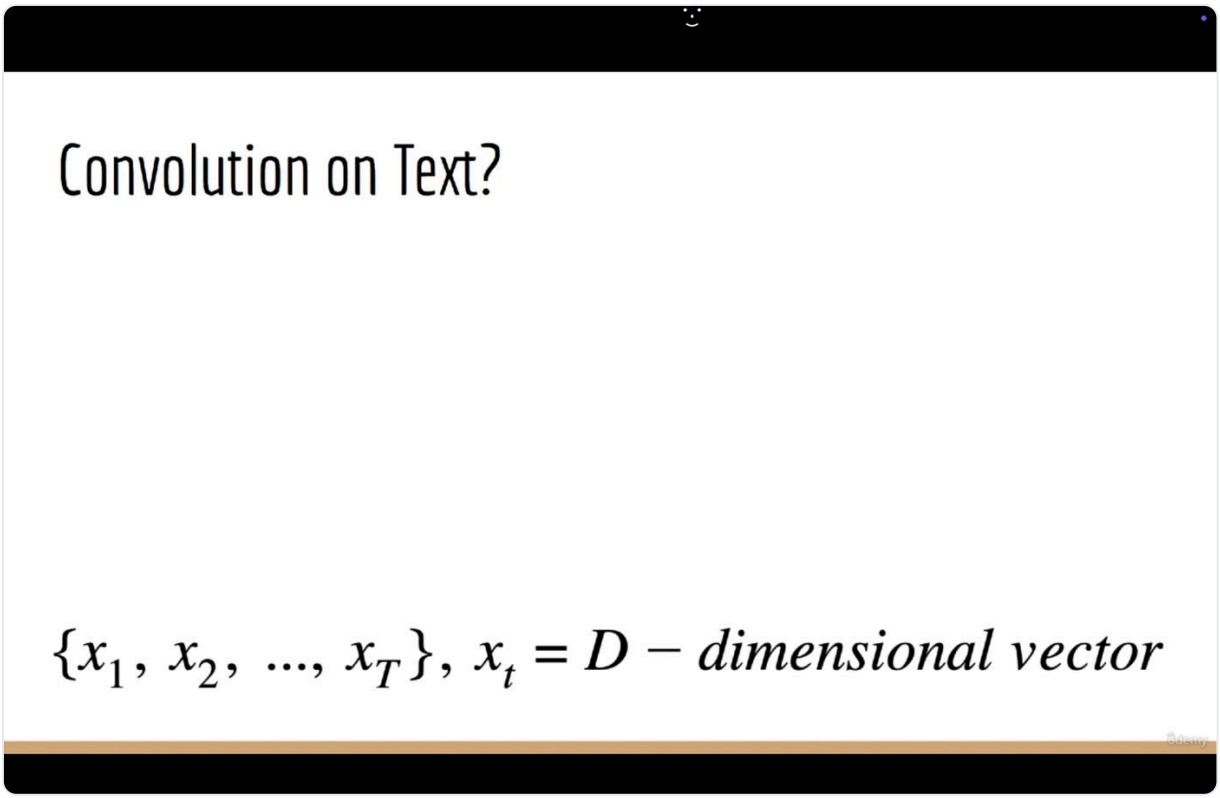


FIGURE 32.17 | Convolution on Text?

So really what we are doing is asking, is this part of the sequence similar to my filter? And thus the filter becomes a pattern matcher or a pattern finder. All these concepts that you learn about convolution before still apply here. All right, so now that you know how Convolution works with sequences, how do we apply these the text?

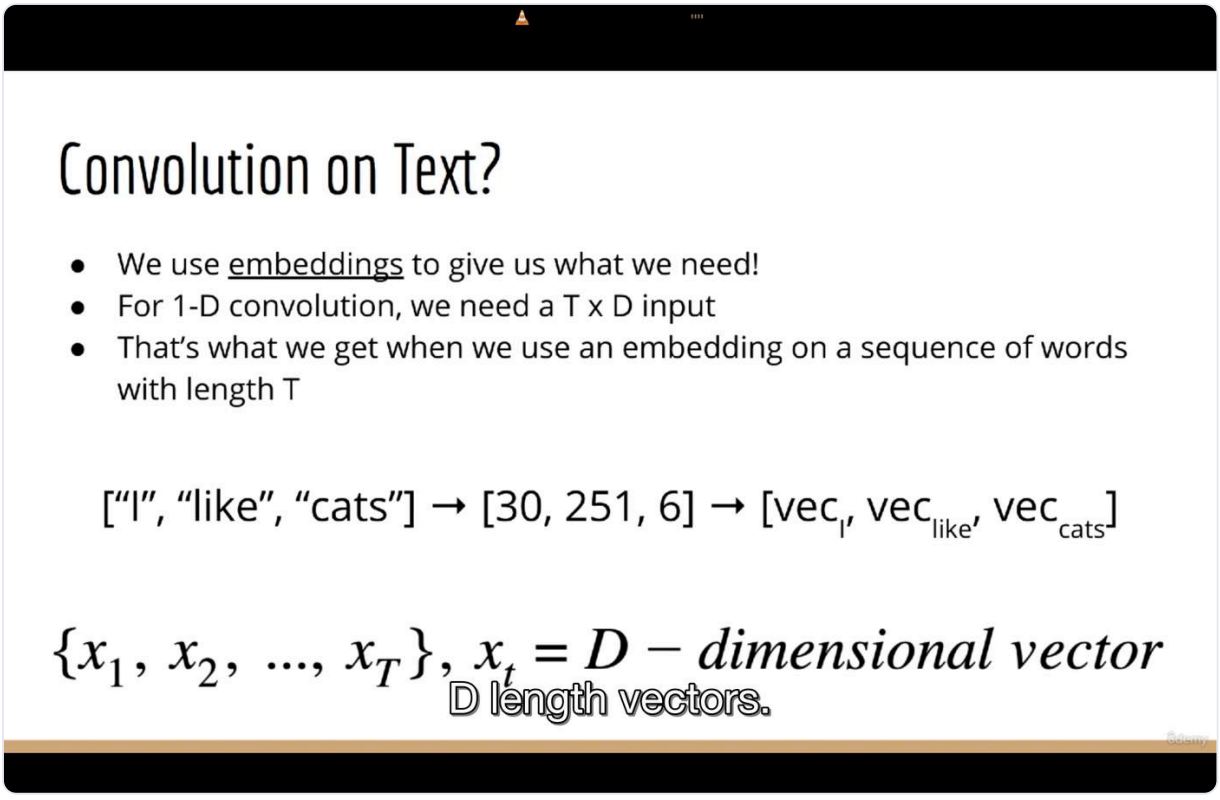


FIGURE 32.18 | Convolution on Text?

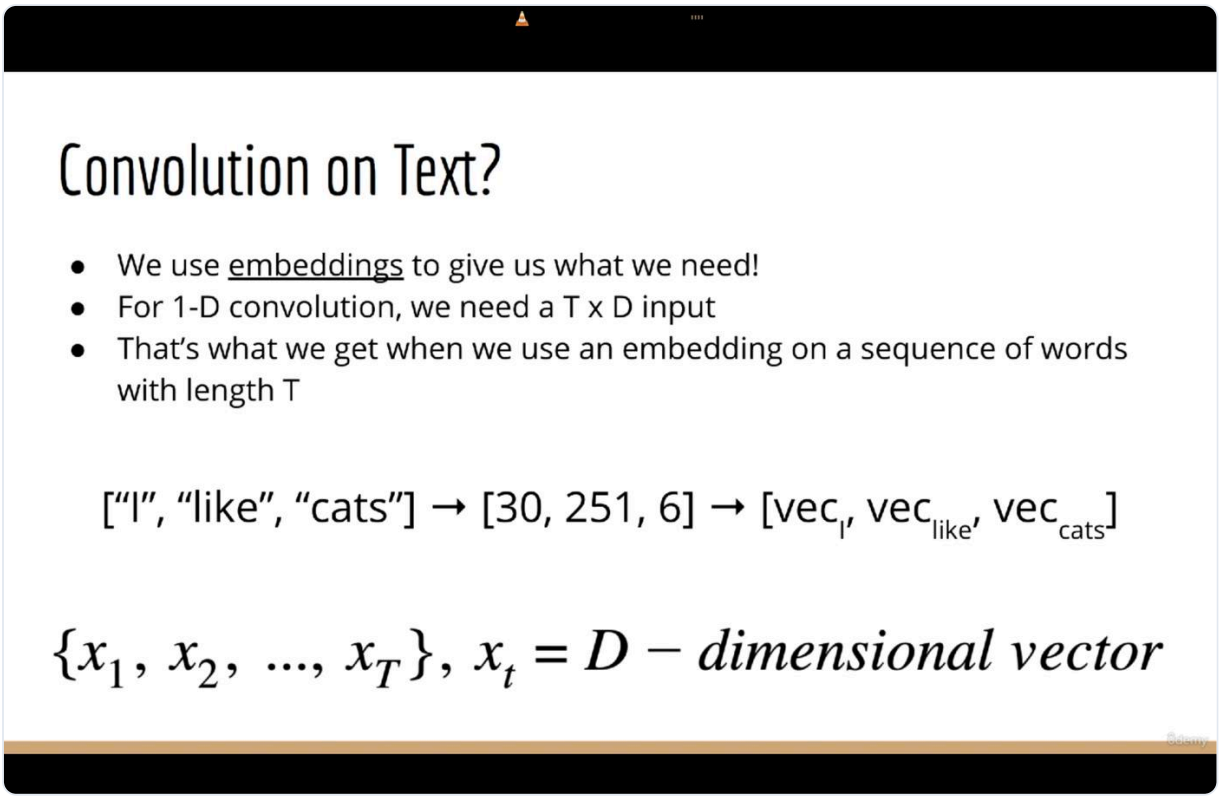


FIGURE 32.19 | Convolution on Text?

Well, luckily when we use embeddings, that already gives us exactly what we need for a one dimensional convolution. We need an input, which is a T by D sequence, where T is the number of timesteps, and D is the number of features. And of course, this is exactly what we get after we pass our sentence through and embedding layer. We go from a length T sequence of words to a length t sequence of integers to a length T sequence of D length vectors. Since we have T vectors each of length D this makes up a T by D matrix. When you stack them all together and thus we have exactly what we need to build a CNN for text. So here's how it all looks in code. First, we have our input.

That's a length T sequence of word indices. Then we have an embedding layer. The output of that is a T by D sequence of word vectors. Then we have a one de convolution. This is just the Conv1d class. Then we follow the same typical CNN architecture where we have convolution followed by pooling and so forth. So the same pattern applies here that we had for images. Generally speaking, the data shrinks in the time dimension, but grows in the feature dimension. So that's why you see the number of feature maps getting larger and larger. Once we've done that, then we can do a flatten or we can do a global max pooling, which will give us a single vector of size M3.

Finally, we pass this through one or more dense layers to get a single scalar, assuming we are doing binary classification. As you can see, this is no different from a CNN meant for images, except that all the convolutions and pooling are one dimensional instead of two dimensional. There is one small caveat to what we just discussed. But it's not obvious until we tried to implement the forward function inside the constructor. There's no real problem because all we're doing is instantiating some objects. But remember that in PyTorch, the general convention is that for convolution features come first. We saw this earlier in the context of two dimensional convolution on images.

PyTorch Convolution works on images of size and by C, by H, by W, which means that color comes before the spatial dimensions. In other words, a feature comes first. However, in TensorFlow, OpenCV and other libraries will use the convention. And by H. By W, by C. Which means that the feature dimension

comes last. torchvision in some sense accepts that this is the convention and makes its interface such that you can work with N by H, by W, by C all throughout your code and never have to worry about the fact that in PyTorch things are backwards. This is because you don't ever interact with the data explicitly torchvision does all that work for you. But now we have this mixture of conventions.

The output of an embedding is N by T. By D. Where T is the sequence length and D the feature dimension. So the feature comes last. When it's the output of an embedding. But the Conv1d function, just like the Conv2d function, expects the features to be first. In other words, it expects the features to come before the time dimension. So it wants to see N by D, by T and not N by T, by D. What we end up having to do is permuting the data right after the embedding and before the first convolution. So after the embedding its features last, but before the first convolution, we reshape it to features first. Then after all of our convolutions and poolings have been done, we permute the data back from features first.

It features last. At that point, we can pass the data through the usual dense layers that make up the rest of the CNN. So here it is in action, since there are many layers in a CNN. I've split this up to make it more readable. Here we take in the input and pass it through and embedding layer. Then we do a series of convolutions and poolings. I'm not going to show all the convolutions and poolings since we just repeat the same pattern over and over. But we can reason about the shapes. So first, the input X is an N by T array of integers. After the embedding it becomes N by T by D word vectors. Next we permute the dimensions. So it becomes N by D. By T. After the first convolution, we change the number of features.

So it becomes N by M by T the ReLU which comes next, doesn't change the shape of the data. After pooling, which changes the length of a sequence we get N by M by T2. You can think of T2 as T divided by two or T divided by three depending on the pooling parameters. Next we go through the second convolution and so on and so forth. So let's pretend that we had a three convolutions. So after the final convolution, our data has the shape N by M3 by T3 at this point, we need the features to be last again.

FIGURE 32.20 | CNN for Text in Code

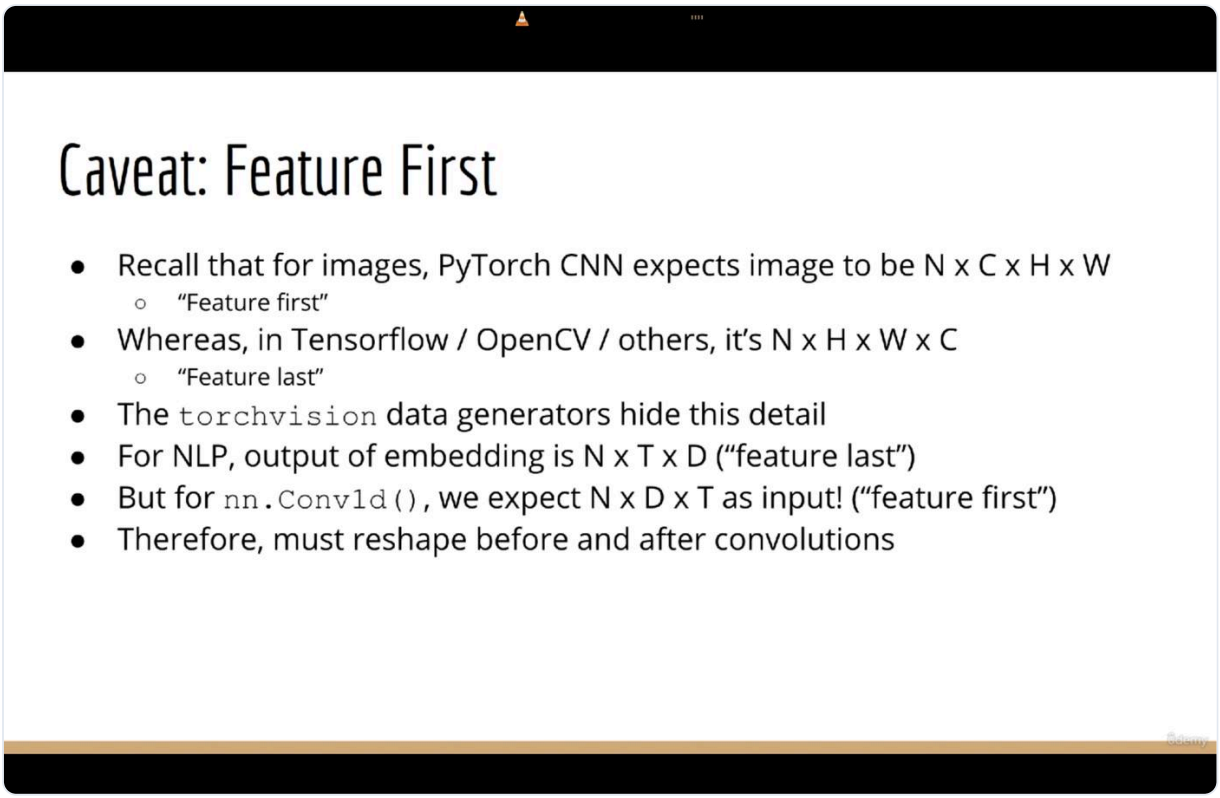


FIGURE 32.21 | Caveat: Feature First

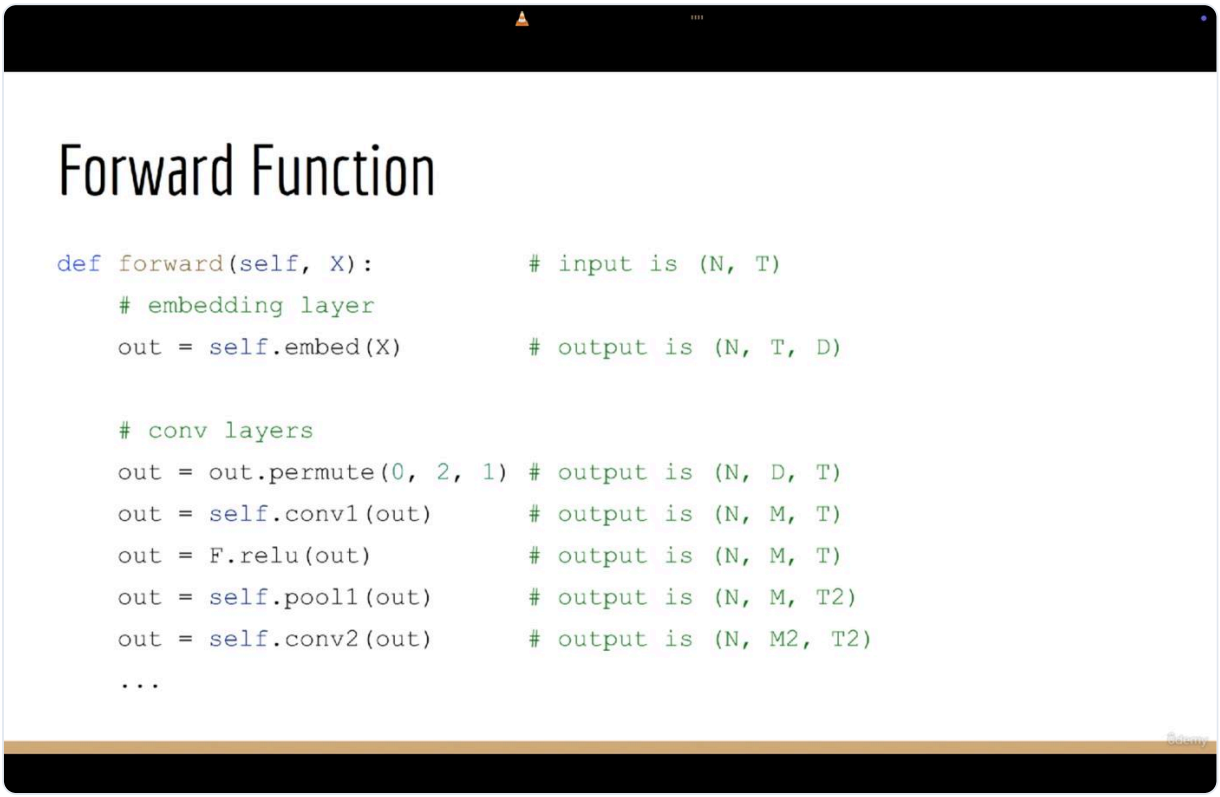


FIGURE 32.22 | Forward Function

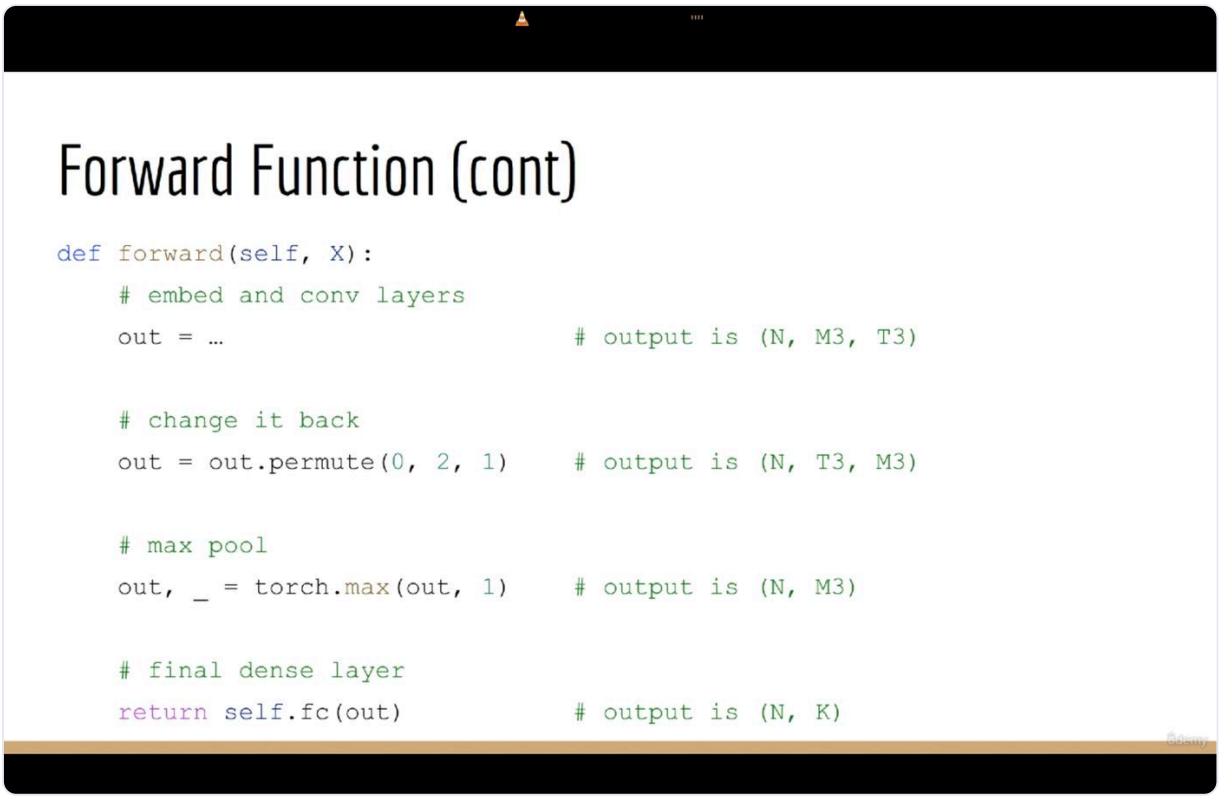


FIGURE 32.23

So we call permute again and we get N by T3 by M3 after doing a global max pool along the time dimension we get N by M3, note that because we can specify the axis to perform the global max pool on, we didn't necessarily have to permute the data after the final convolution. You could just choose a different axis. Although I think this makes it more clear what's going on. Lastly, we passed the data through a final dense layer and as usual, we get back an array of size N by K!

LECTURE 71

Training Corpus and Idx

Check the notebook `18.Custom-PyTorch-Text-Classification-using-CNN.ipynb` for the code.

In this lecture, we are going to look at a notebook that implements convolutional neural networks for text classification. This lecture is going to walk you through a prepared Colab notebook, although a very good exercise, which I always recommend is once you know how this is done, to try and recreate it yourself with as few references as possible. As usual, you can look at the title of the notebook to determine what notebook we are currently looking at. So at a high level, really, this script contains nothing new. You know how to do text preprocessing because we just did that. You know, how to use embeddings because we just did that. And you know how to build and train a CNN because we did that earlier.

The only difference in this script is we're replacing all the 2D operations with 1D operations, not a huge change. So I hope you had the opportunity to try this yourself first before watching this lecture. OK, so since the script is very similar to the previous one, we're going to go through it very fast and only talk about the new thing, which is the CNN with one dimensional convolutions. So first we download Spam dot CSV. The next step is to read in the CSV using pandas. The next step is to look at the CSV using the head. The next step is to drop the junk columns. The next step is to call the head again to make sure they were removed. The next step is to rename the columns to labels and data.

The next step is to call the head again to make sure that we're not. The next step is to create a zero one labels. The next step is to split the data into train and test. The next step is to check the shape of our train and test set just to see what they were. The next step is to perform all the text preprocessing steps. We'll start by initializing idx and word to idx. The next step is to populate word to idx using the training corpus. The next step is to print out word to idx as a sanity check. The next step is to check how many values are in word to idx. The next step is to convert our training corpus into lists of integers. The next step is to do the same thing for the test corpus.

The next step is to check the length of our new model inputs. The next step is to create our data generator, which you've seen before. The next step is to grab our device variable. Now, I've told you why the output of the embedding layer will be N by T, by D. But this kind of thing is no good if you've just memorized it. You have to see it for yourself. So here we create an embedding with Dimension 20. Next we through our data generator and print out the shape of the data before and after the embedding. As you can see, the shape of the data before the embedding is N by T and the shape of the data after the embedding is N by T by 20, which is what we would expect.

This brings us to our custom CNN model in the constructor, we create our embedding layer, a series of convolutions and max pools, and then a final dense layer. Notice that we now use Conv1d instead of Conv2d and MaxPool1d instead of MaxPool2d. But the forward function is where things get strange. First, we pass the data through the embedding, which gives us an N by T by D, as you know, but convolutions and PyTorch use features first, meaning that it expects an N by D by T, not an N by T by D. So we have to permute the data to make it that way. Note that in other libraries, this might be called transpose. It's just a generalisation of the usual two dimensional matrix transpose.

Next, we pass our data through the convolutional and pooling layers. The next step is to permute the data back, to put the features last note that this isn't absolutely necessary since we get to choose which dimension to take the max over in the next step. So maybe as an exercise, you can try to remove that

second permute. Lastly, we pass our data through the final dense layer and then return our output. The next step is to instantiate our CNN and move it to the GPU and after this is just stuff you've already seen. The next step is to create our loss and optimizer. The next step is to create train gen and test gen. The next step is to create the training function.

The next step is to call the training function. The next step is to plot the loss per iteration. OK, so the loss per iteration looks good. The next step is to calculate the accuracy. OK, so the accuracy looks good and we're done. OK, so we get a very high accuracy with a CNN in the high 90s, you might have thought because RNNs are for sequences, they might do a little better than CNN's. But in fact, CNN's are pretty great as well.

LECTURE 72

Pre Process and Single Sentence

See the notebook `17.Custom-PyTorch-Text-Classification-using-RNN.ipynb` for the code.

In this lecture we are going to look at extending our NLP example so that we can use a trained model to make predictions. A lot of people ask this question and at a high level you just need to follow the same steps that we did earlier to pre process the text and call the model predict function. However some students get overwhelmed with the amount of code and so hopefully this lecture will clear up any confusion you may have had. Before we start first recognize that we are in a separate VIP notebook which you can find in the VIP notebooks document. So to start we're going to do a few things differently in this notebook to kind of extend what we discussed earlier.

So if we scroll down to after the model is trained you'll see that I also do a check for how many of our observations are actually spam. As you can see it's not that many. Only about 13 percent. This is an example of a data set with imbalanced classes. So when we see accuracy in the high 90s we might be concerned that our model is just predicting the majority class all the time. Next we're going to look at the confusion matrix to see if our model is doing well on one class but not on the other. So I've pasted the same confusion matrix code from earlier in the course in this block if we scroll down to the results we can see that we actually do well on both classes.

So this goes for the tests. And with the train set so it seems that even though the classes were imbalanced this did not pose a problem for the model next. Let's go onto you making a prediction on a new test sentence. There are two ways and one to show you how to do this. The first way is closer to what we did earlier in the script which I think is easier to understand. That is we want to start with a CSV file containing our data. So first I have a snippet of code here that shows you how to sample from the data frame but only for the rows with a label that spam in the next block. I assign this to a variable called small sample and then I write this to a CSV file called sample test dot CSV.

Note that the only thing I'm writing to this file is the sentences themselves not any of the other columns. So in the first line I say open sample test dot CSV in write mode in the next line. I write the header row which is just the string data. Then I do a loop through the small sample data frame using the iterrows function inside the loop I select the data column from the row and then write back to the file along with the new line next we use the cat command to confirm that our CSV has been written properly. As you can see it contains only the data column with the corresponding spam sentences. The next step is to create a tabular data set from our CSV.

This is almost the same as the code from earlier. But notice that in the fields argument I only have one field which is data. I also pass in the text field object that we created earlier. Since this contains all of the information about tokenization and the vocabulary and so forth from the train set next I create an iterator object from the data set object which is similar to what we did earlier when we called iterator dot splits. As you can see it takes in almost the same arguments a dataset a batch size a sort key and a device but you'll notice that before the splits function it took in multiple datasets whereas this takes in only a single dataset.

Now you might wonder how did I figure out such an object existed. Well guys you have to read the documentation all the information is there in the next block. We loop through our sample iterator so this is the moment of truth where we get to find out whether this will work or not. As you can see it's slightly

different from what we had earlier in the previous loops. We got back the data directly as tensors but it seems that in this loop the data is returned as a batch object. We can access the data by calling inputs dot data. Unfortunately the API for the iterator object versus iterator that splits is inconsistent. Well you just have to observe what's going on and make the necessary changes.

Nobody said you wouldn't have to put in some effort as you can see our predictions come out just fine. Now your next question might be what if I have just a single sentence and I don't want to write it to a CSV. In this case we can still make use of the torchtext documentation to figure out what to do so we'll start with a sentence from the spam set and I'll assign this to a variable called single sentence the next step is to use our text field object and call the pre process function on this single sentence. Here I'm just printing out the result to confirm that it does the right thing. As you can see this only does the tokenization part but it does not map the tokens to integers the next step is to call the numericalize function which converts the above tokenized sentence into a sequence of integers.

Unfortunately the API is a bit inconsistent here again since this accepts a list of tokenized sentences rather than just a single tokenized sentence. Finally the last step is just to put these all together and then use our model to get a prediction. So in the last block of code we call text pre process to convert our sentence into tokens. Next we call numericalize to convert our tokens into a sequence of word indexes. Lastly we call our model and we pass in the sequence of word indexes. After moving the input tensor to the GPU as a final sign for this lecture you may want to check out another notebook I created which uses the Keras API to pre process the text.

It might surprise you that this is possible but in fact since the Keras API just gives you back a NumPy array. There is no mixture of Keras code with PyTorch code. I found that this was advantageous in a few ways namely that it achieves better accuracy perhaps due to smarter default tokenization and the API is easier to work with and more consistent than torchtext. You can find this notebook along with the rest of the VIP notebooks in the extra section.

LECTURE 73

List of Integers

004 Why-bad-programmers-always-need-the-latest-version https://youtu.be/BIXH_m6CT2I

See the notebook `17.Custom-PyTorch-Text-Classification-using-RNN.ipynb` for the code.

OK, so in this lecture, we are going to discuss how to make predictions with our trained NLP neural network. Now, before we start this lecture, I want to mention that what we will do here is totally within your capabilities, using only what you've learned so far. So if you have not yet tried this yourself, please do so before watching this lecture. At this point, you should be able to take a raw string of text, convert it into the right format for a neural network, and then use a trained neural network to make a prediction. So please try that as an exercise before moving on. So one thing I have to mention is that these how to make prediction lectures are really made for the very, very beginner level students.

That is, if you could not figure out how to do this by yourself, take this as a big danger sign. So my recommendation is that if you were waiting for this lecture or you believe that this lecture was necessary in order to teach you how to make predictions, then please take a moment to think about how much you have really learned. Consider whether you are actually learning or if you are just copying me. And if you find that you are the latter student, please pause right now and really put in some more effort into trying this exercise on your own. Now, you'll also notice that I've actually left the legacy version of this lecture in the course, I think this is useful for two reasons.

Firstly, it covers topics like imbalance classes and the confusion matrix, which is unnecessary to repeat again. But secondly, and more importantly, I want you to notice how complex it is to use the torchtext API for a simple prediction. It's actually surprisingly ugly. So I want you to watch that and compare it to this one to get a real understanding for why doing things yourself cannot only be useful, but preferable. Keep in mind that the torchtext library is not even at version one, which means it is likely to change again in the near future. So there are many reasons why it's simply better to avoid using it. Learning it now is just not a good investment.

As mentioned, I've seen multiple instances where courses simply use the Keras text module instead, and with all that complexity you'll see that we can do the same thing here with just five very short lines of code. OK, so the first part of this notebook is all the same as before. So let's assume that the code has been run. The next step is to simply pick a random text that we want to classify. Alternatively, you can make up your own or draw a sample from the data set. The next step is to perform the usual preprocessing steps where we tokenize the text and then convert it into a list of integers. The next step is to convert our list of integers into a torch tensor, as you recall, this needs to be N by T, where N is the number of samples and T is the sequence length since we only have one sample.

This is a one by T array. Now that our text is in the format we need for passing it into a neural network. The next step is to simply make the prediction using our trained model. Note that I've also move the tensor to the GPU since our model is also on the GPU. OK, so the final step, which doesn't really count, is a line of code, is to just print out the output. So as you can see, we get a very large number, which means that this would be assigned to spam. This makes sense, since if you read the above text, you can confirm that it is, in fact, a spam message as an exercise. Think about why this output value is not between zero and one, as you may have expected.

LECTURE 74

Ratings Dataset Must Be Incomplete

In this section of the course we are going to look at how deep learning can be used for recommender systems. This lecture will discuss the theory behind how deep learning can be applied.

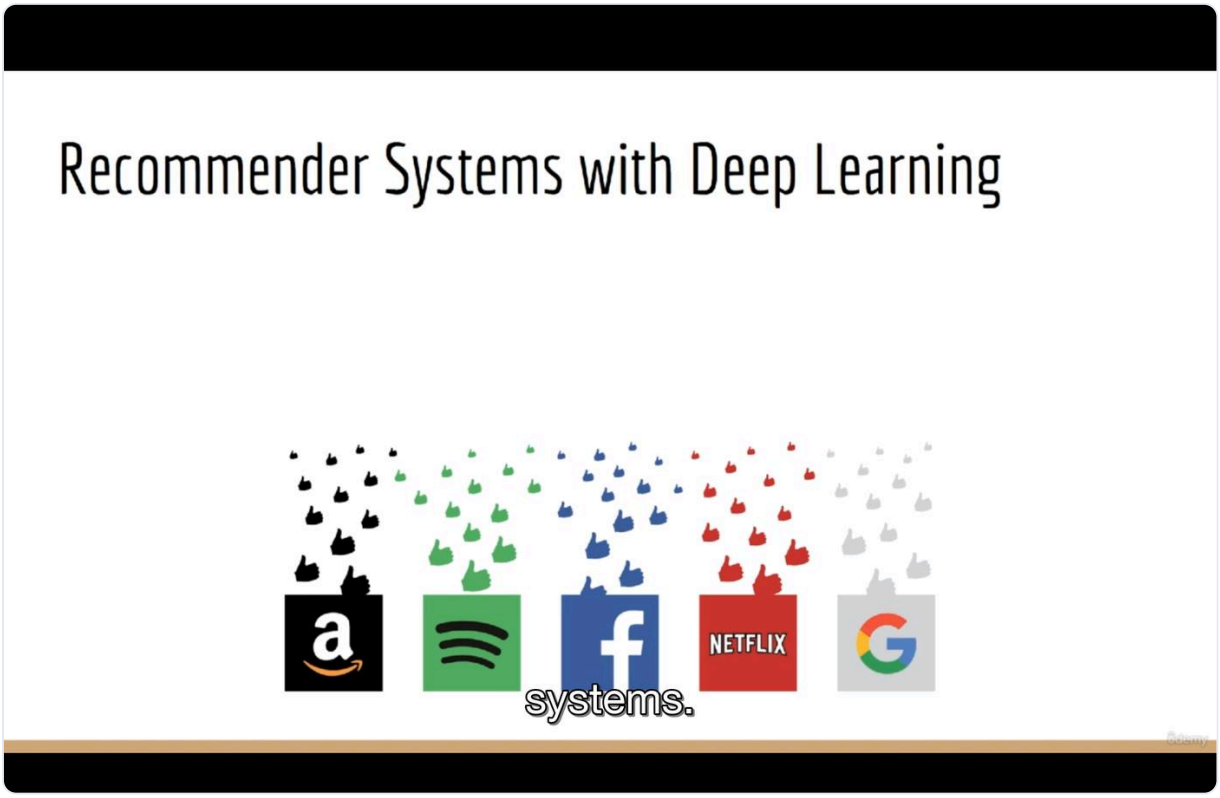


FIGURE 36.1 | Recommender Systems with Deep Learning

And following that we will look at the code to start. I want to talk about recommender systems as a general concept.

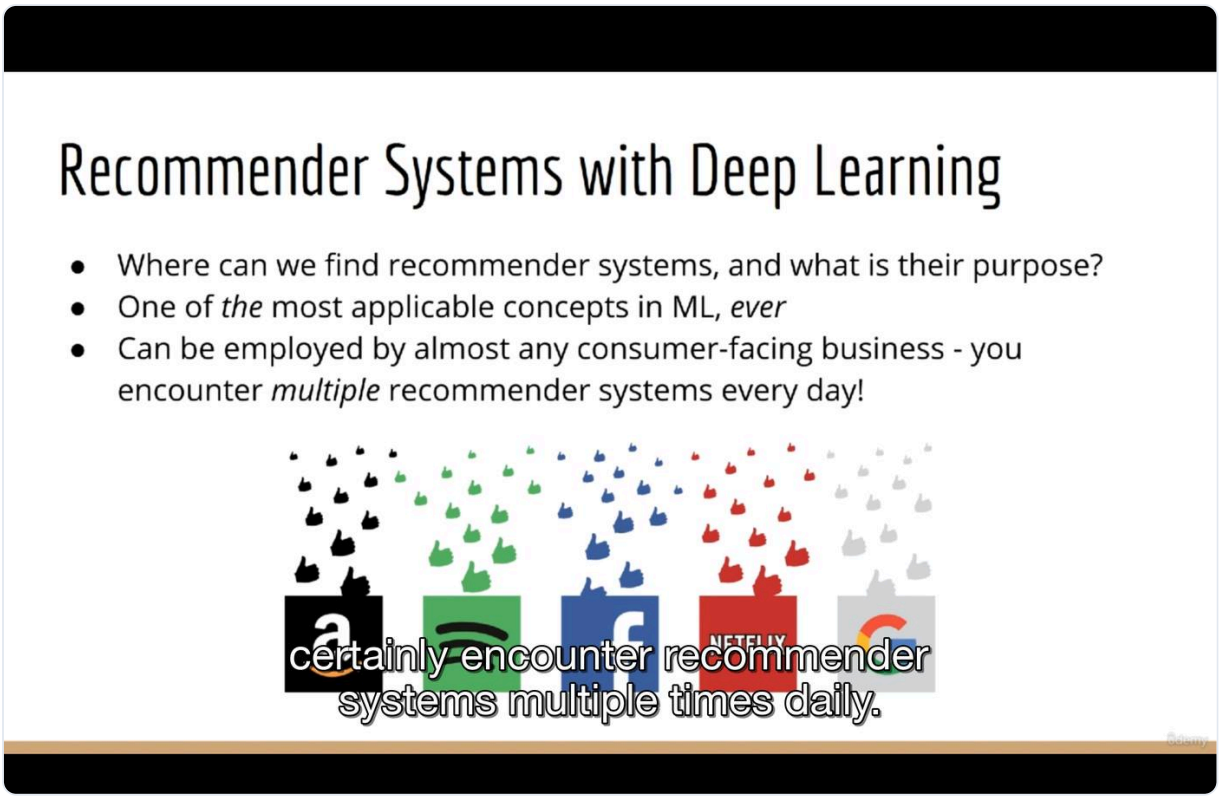


FIGURE 36.2 | Recommender Systems with Deep Learning

Where can we find recommender systems and what is their purpose. I think you'll be surprised to find out that recommender systems are one of the most applicable machine learning concepts ever. It's a type of machine learning that can be used by nearly any consumer facing business and you most certainly encounter recommender systems multiple times daily. Let's start with the first Web site. Everybody goes to when they go online.

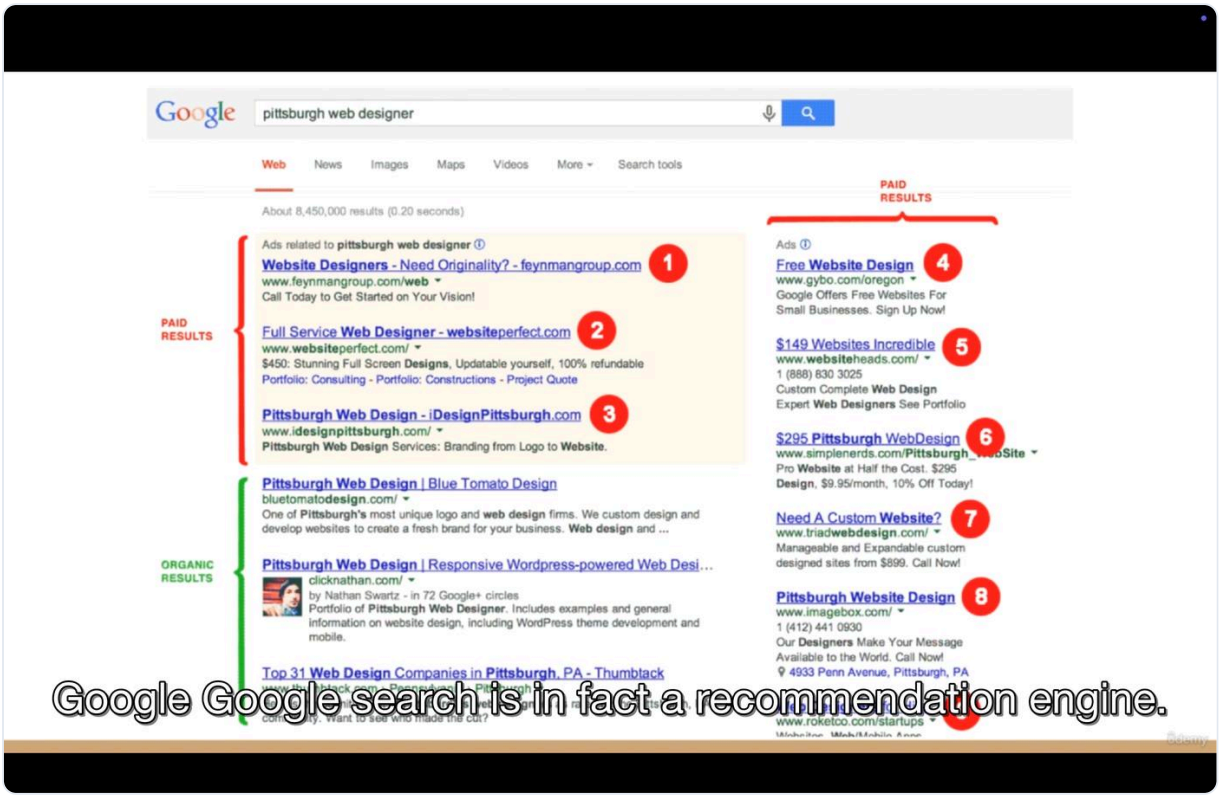


FIGURE 36.3

Google Google search is in fact a recommendation engine. Actually when you perform a search multiple recommendation engines are working to build the page that you see. Not only do you have the search results which are recommendations based on your query your geographical location and maybe even your search history. But don't forget that Google is also an advertising company. The online advertisements you see when you go to Google dot com are also recommendations based on your previous browsing history another great example is Amazon.

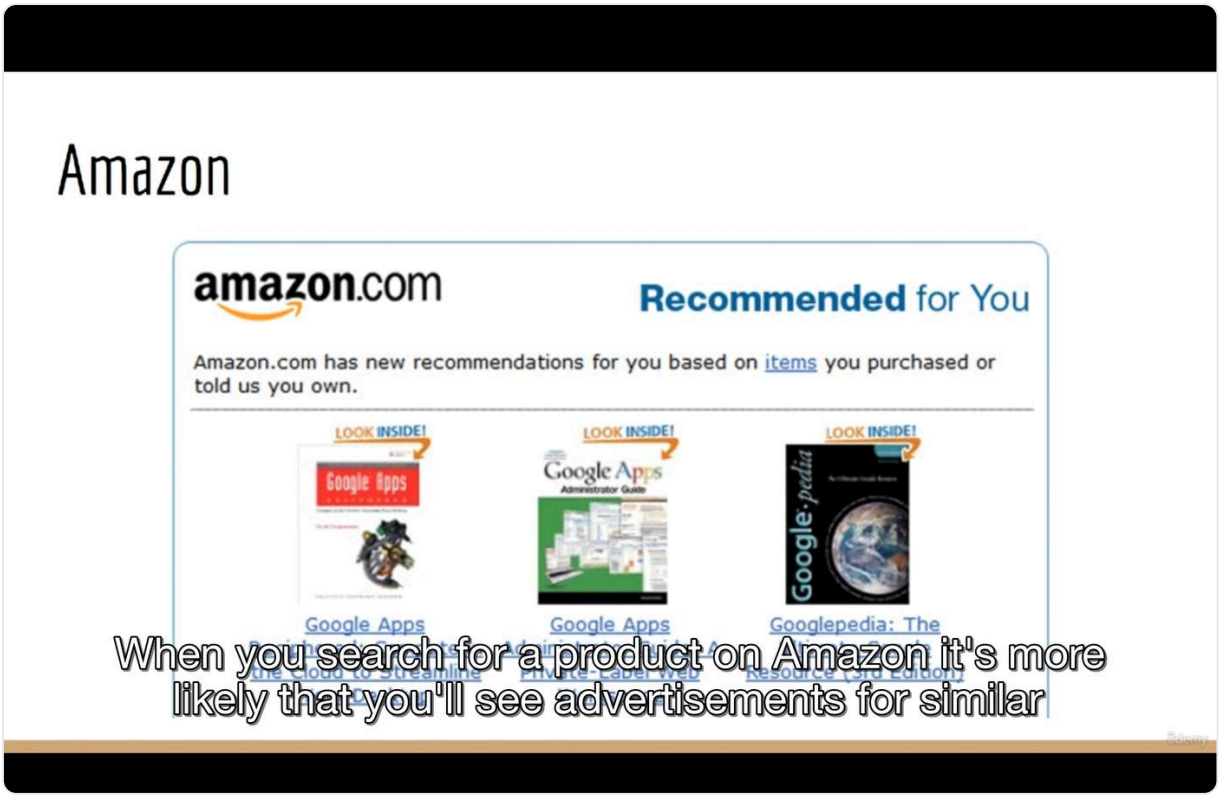


FIGURE 36.4

When you search for a product on Amazon it's more likely that you'll see advertisements for similar products in the future.

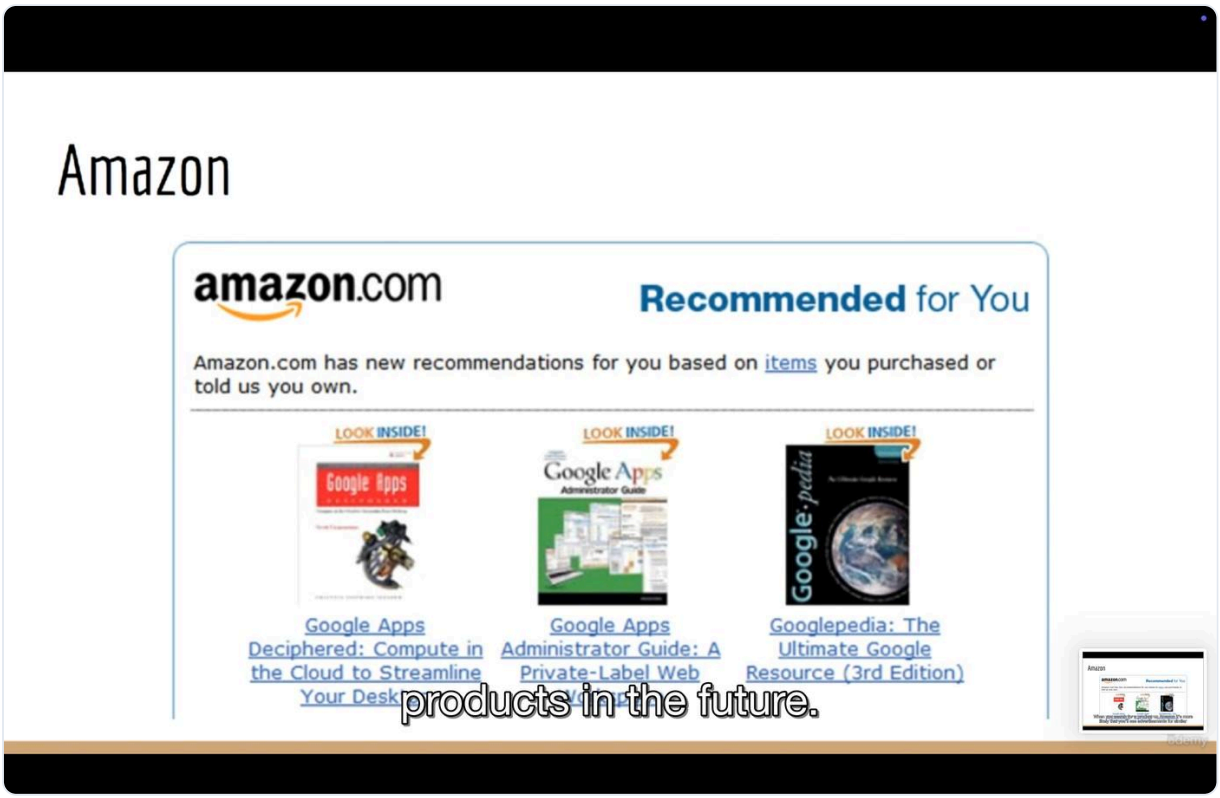


FIGURE 36.5

You might also see recommendations for related products.

FIGURE 36.6

For example if you're buying an iPod you might also be in the market for new headphones a case and a screen protector another great but seriously unfortunate example is the news news sites employ recommendation systems and they even do a b testing on their article headlines to improve click rate. This is

very unfortunate because this means that news has become more biased and more polarizing over time. People don't care about boring news. They want to read something exciting and news companies are more than happy to give that to you in place of real factual and unbiased news there are really too many examples of recommendation systems to mention.

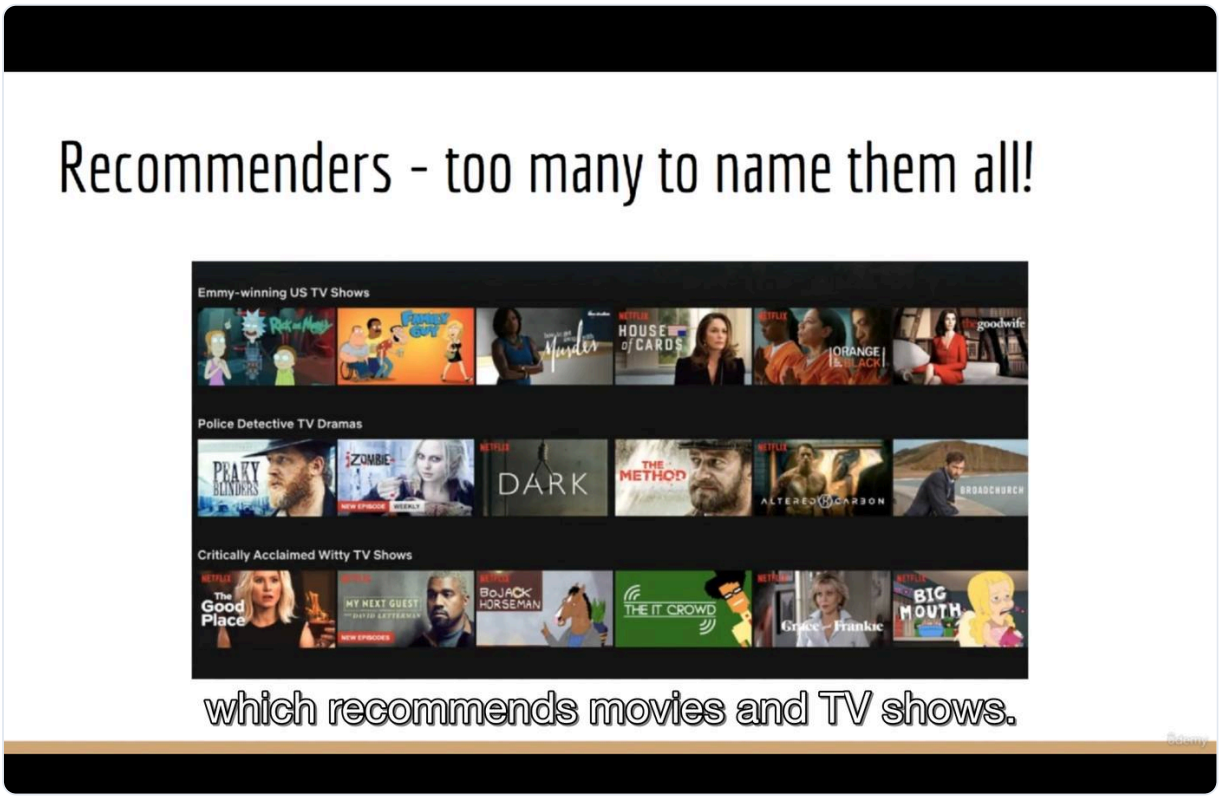


FIGURE 36.7

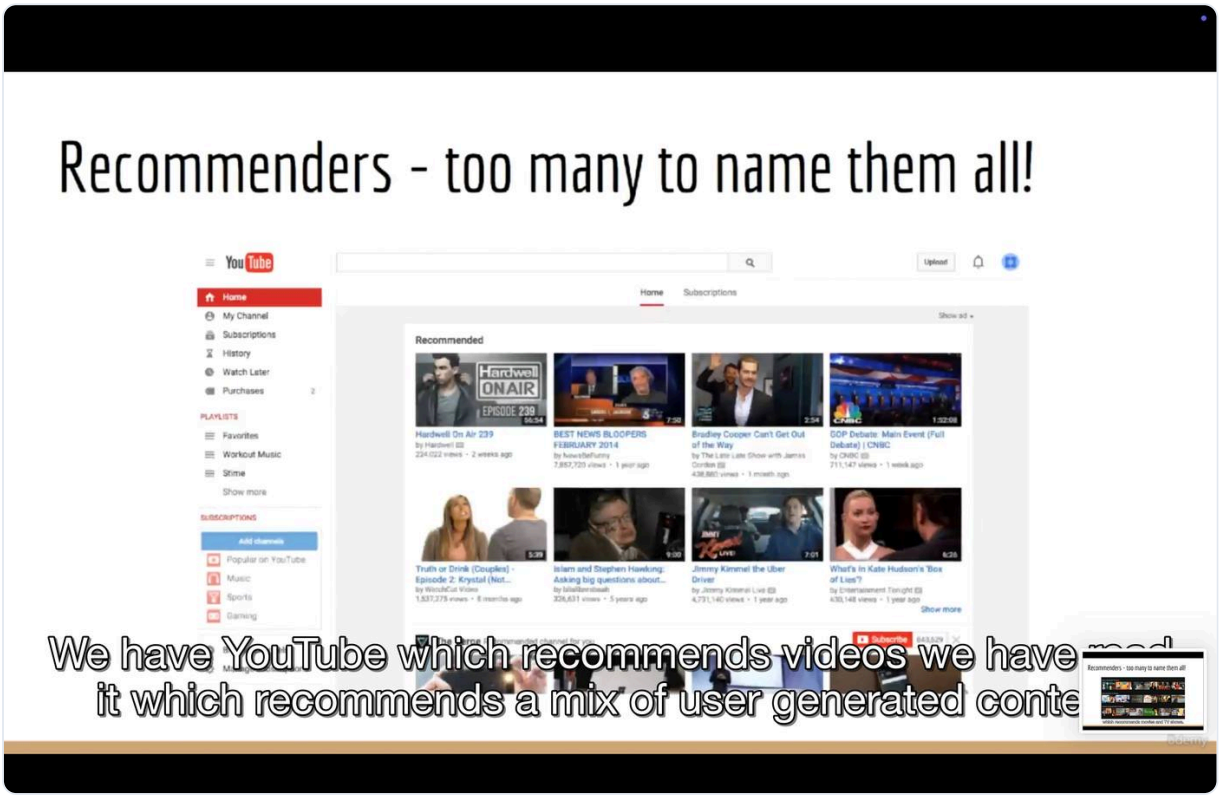


FIGURE 36.8

Here are a few more examples to help you internalize what exactly it means to recommend we have Netflix which recommends movies and TV shows.

FIGURE 36.9

We have YouTube which recommends videos we have Reddit which recommends a mix of user generated content and third party content posted by users.

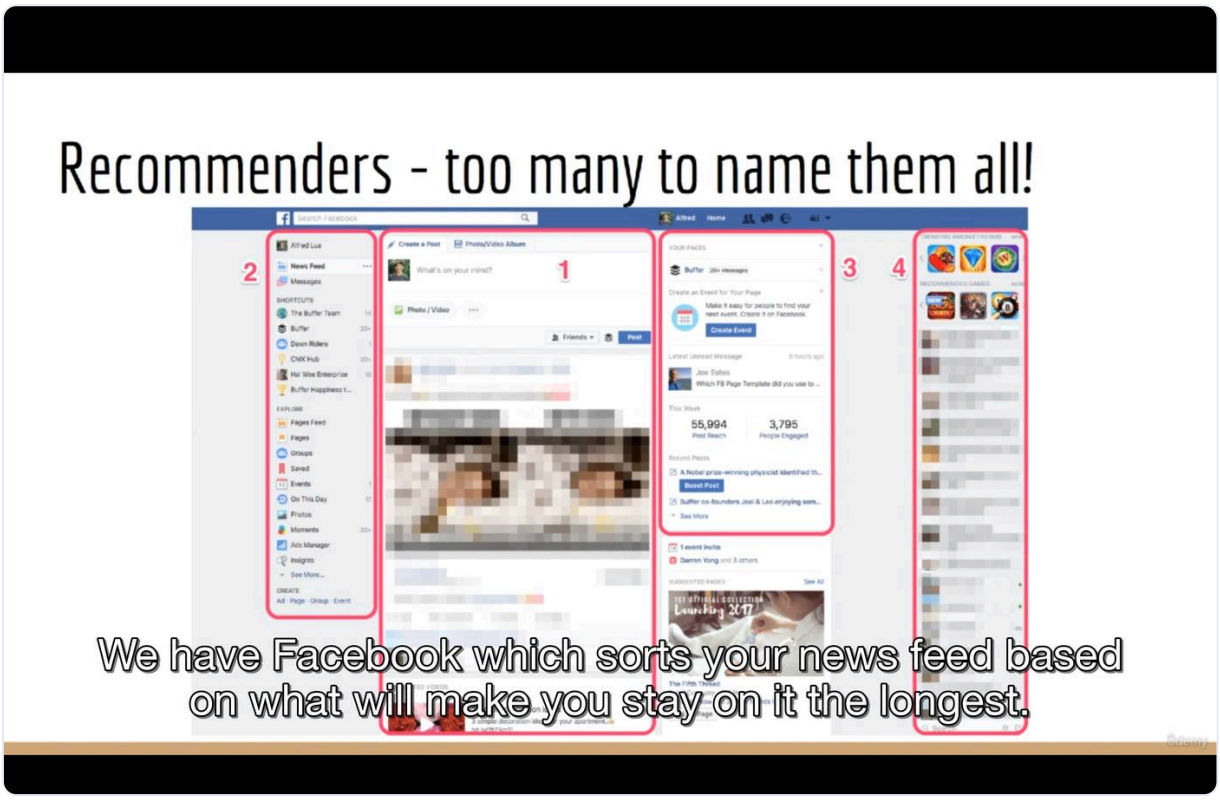


FIGURE 36.10

We have Facebook which sorts your news feed based on what will make you stay on it the longest.

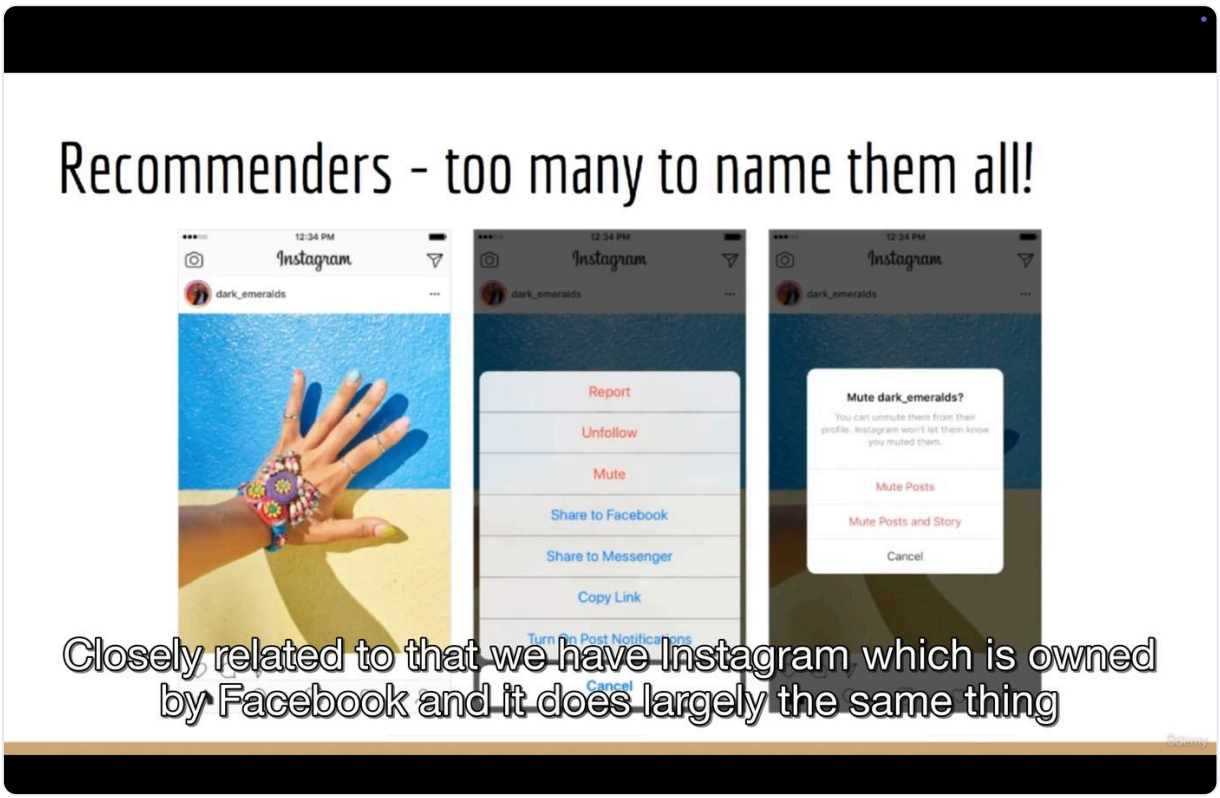


FIGURE 36.11

Closely related to that we have Instagram which is owned by Facebook and it does largely the same thing you're shown a feed of Instagram posts that are recommended in order to keep you engaged. In this lecture we'll be focusing on a specific kind of recommender.

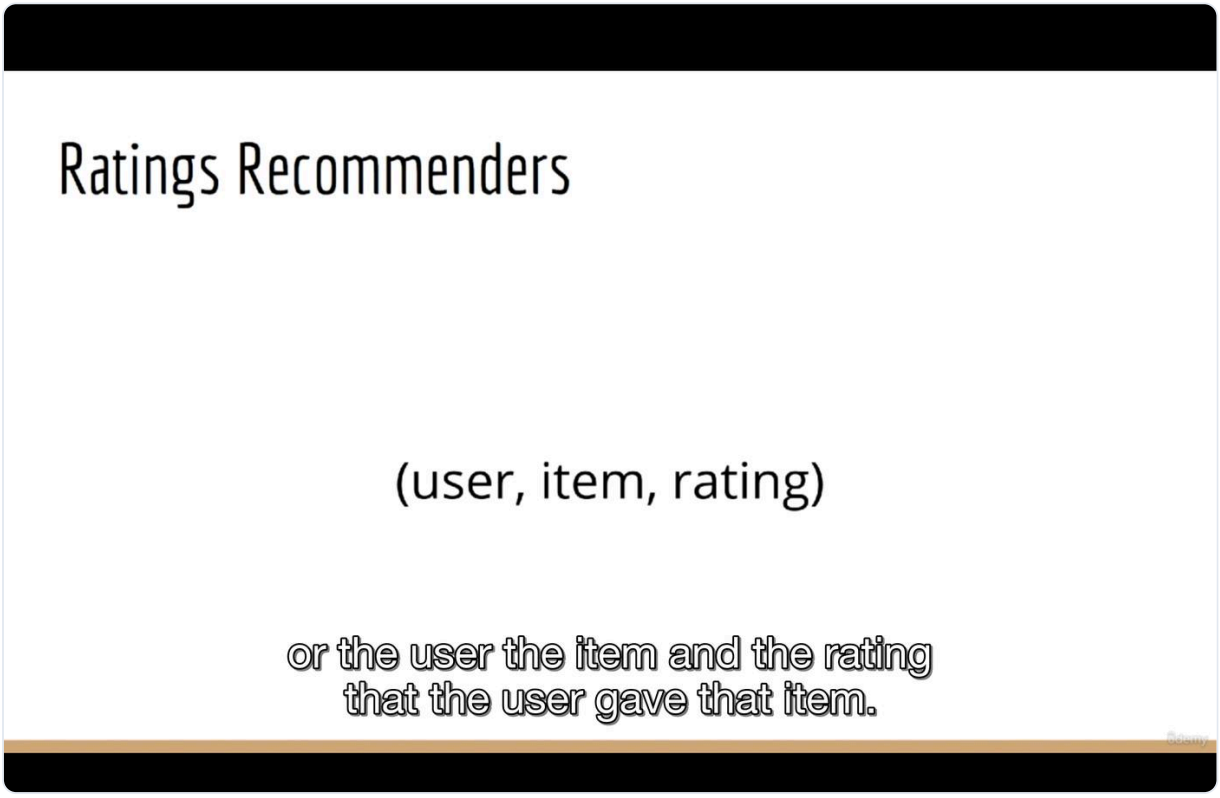


FIGURE 36.12

This recommender works on data that comes in the form of triples the three items that form a sample or the user the item and the rating that the user gave that item.

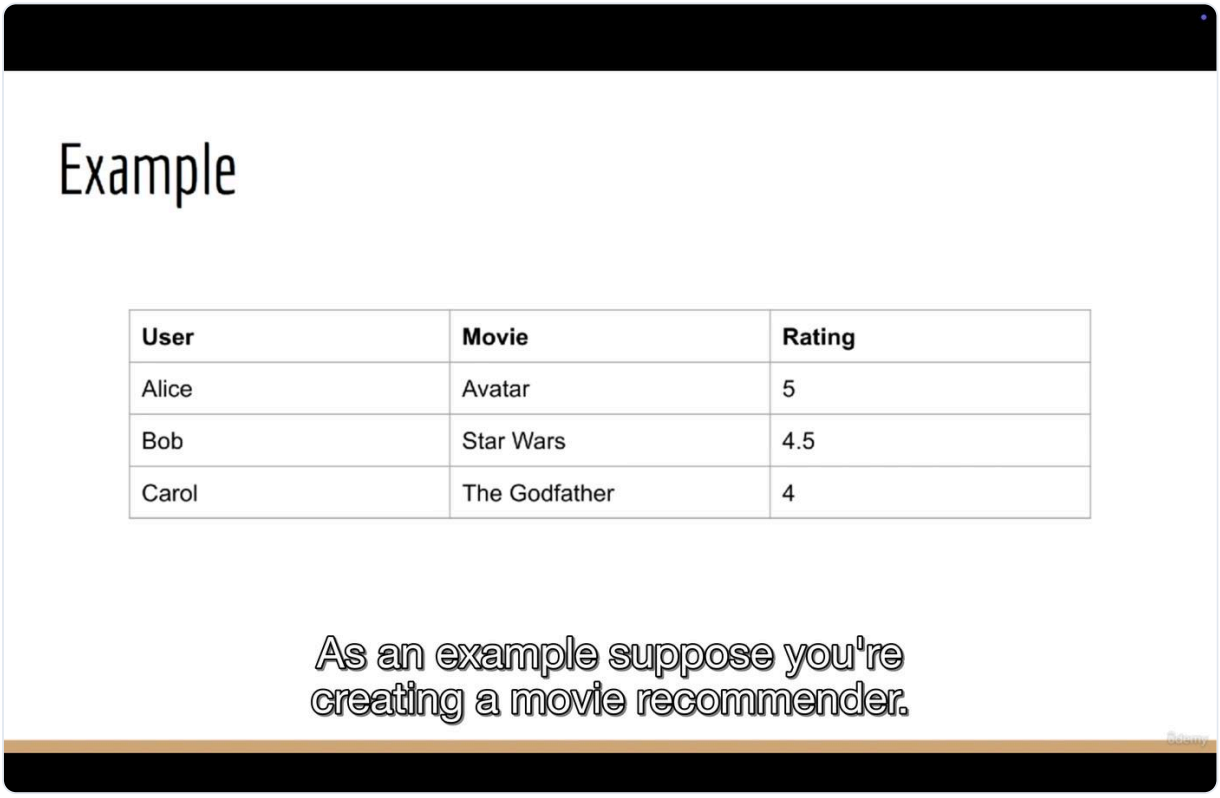


FIGURE 36.13 | User Movie Rating

As an example suppose you're creating a movie recommender. Here are what a few rows of your data set might look like. Alice rates. Avatar 5 Bob rates. Star Wars of 4.5 and Carol rates the godfather of 4. So all of our samples will look like this.

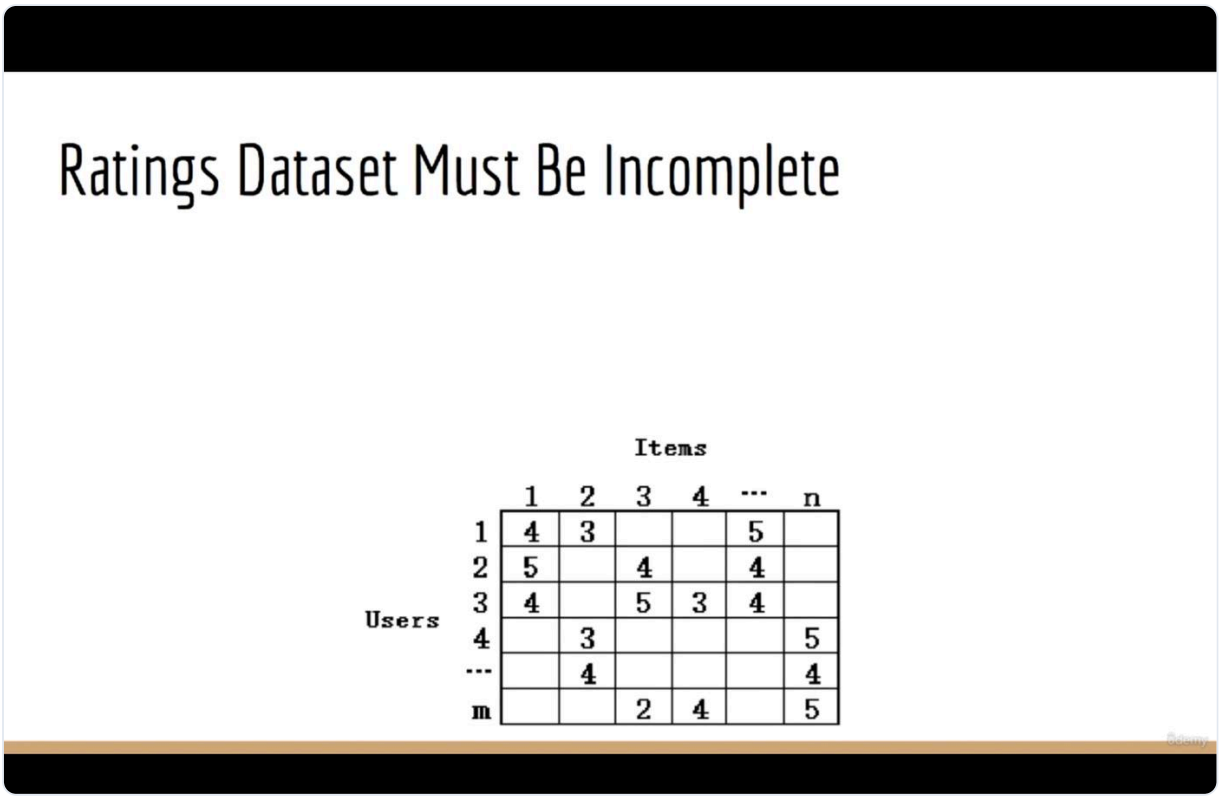


FIGURE 36.14 | Ratings Dataset Must Be Incomplete

The first item is the user. The second item is the movie. And the third item is the rating one important feature of this type of dataset is that it must be incomplete. Why. Well think about it.

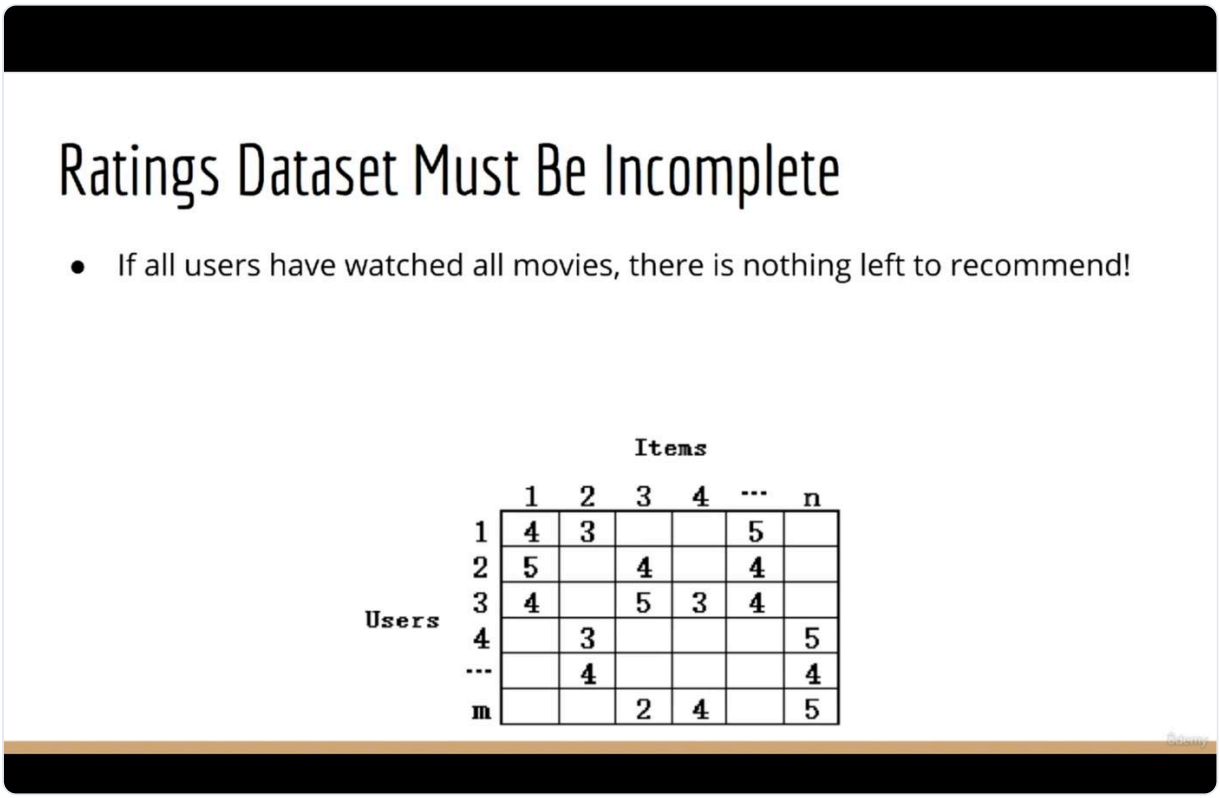


FIGURE 36.15 | Ratings Dataset Must Be Incomplete

If all users have already watched and rated all existing movies then what would there be to recommend if I've already watched and rated every movie in existence. There is no need to recommend anything to me. You can't recommend a movie to me because I've already seen it. Luckily for any real data set. This won't be the case.

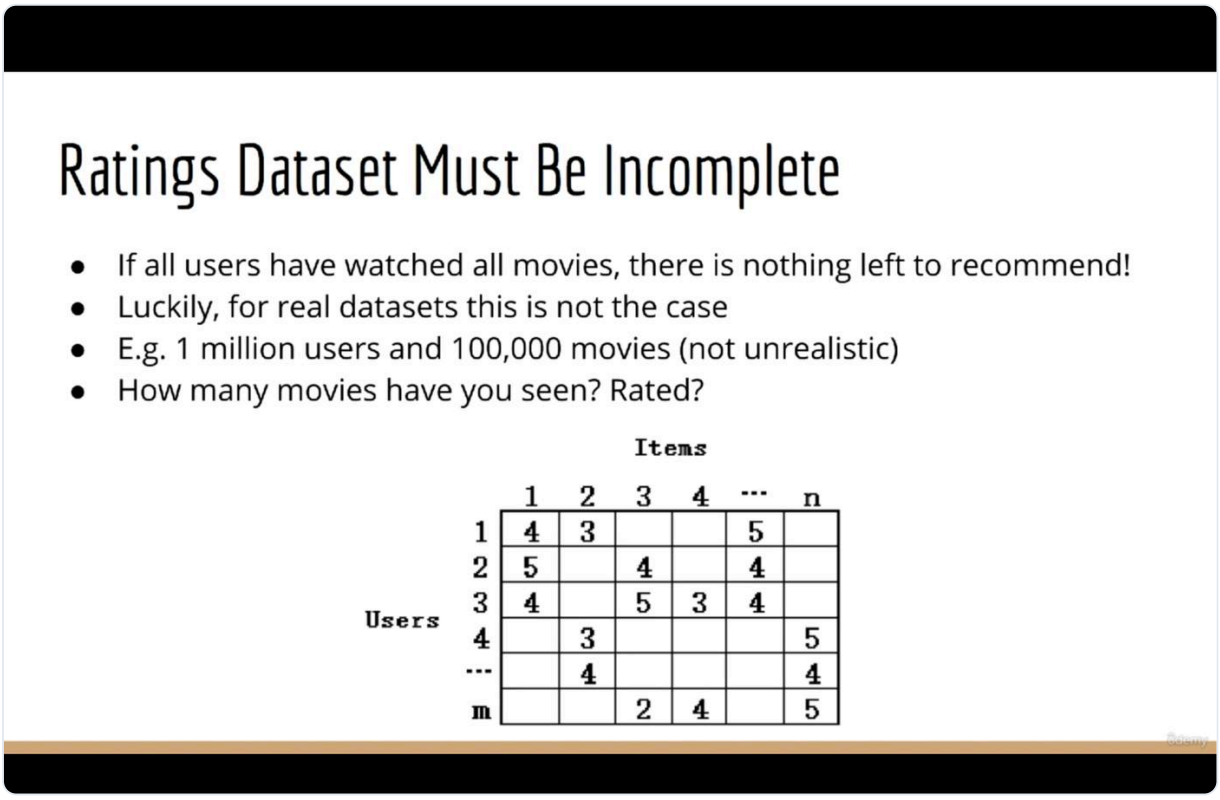


FIGURE 36.16 | Ratings Dataset Must Be Incomplete

Imagine you have 1 million users and one hundred thousand movies which is not unrealistic. You can probably count the number of movies you've seen in your life and it's probably a lot less than one hundred thousand. The number of movies you've actually bother to rate is probably even smaller. The question now is once we have such a dataset how does it help us make recommendations. Let's take a very simple example consider Bob Bob R rated every movie in the Star Wars series of 5 Bob also rated a few of the Star Trek movies of five. And Bob also rated Avatar five.

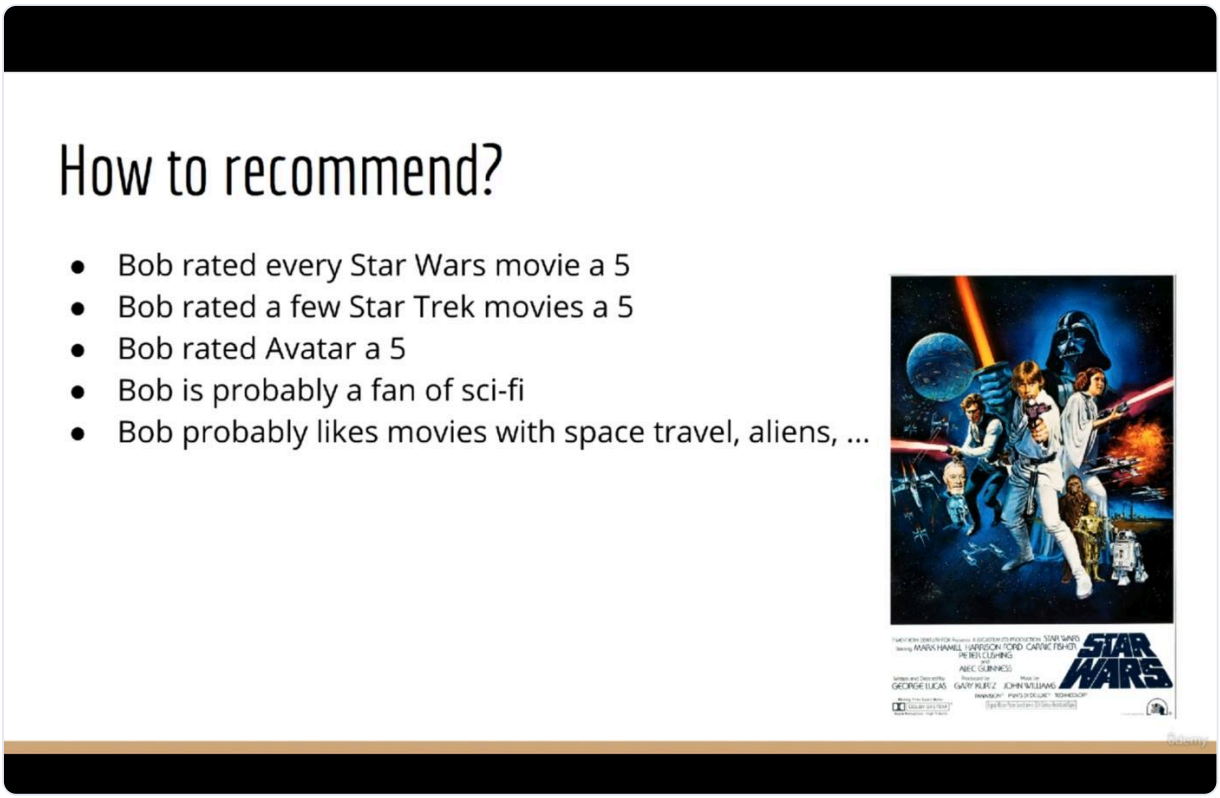


FIGURE 36.17 | How to Recommend?

Common sense tells us that Bob is probably a fan of sci fi. Bob likes movies that have space travel aliens and so forth. It is probable then that Bob would like movies that have the same features. A pretty obvious recommendation for Bob would be a star trek movie or a TV show that he hasn't seen yet. Let's try to generalize this concept. As you know we're given a dataset that consists of triples containing user's items and the user's ratings for those items. Suppose we can fit a model to this data. Let's just call it F F takes in two arguments. A user you and a movie M and it outputs a predicted rating. There are two things that we would like this function to do.

Number one obviously if the user movie and rating appear in our dataset then we would like the predicted rating to be close to the real rating and number two. We would like this function to be able to output a predicted rating even if the rating did not appear in our training set luckily a neural network is a perfect model for this use case.

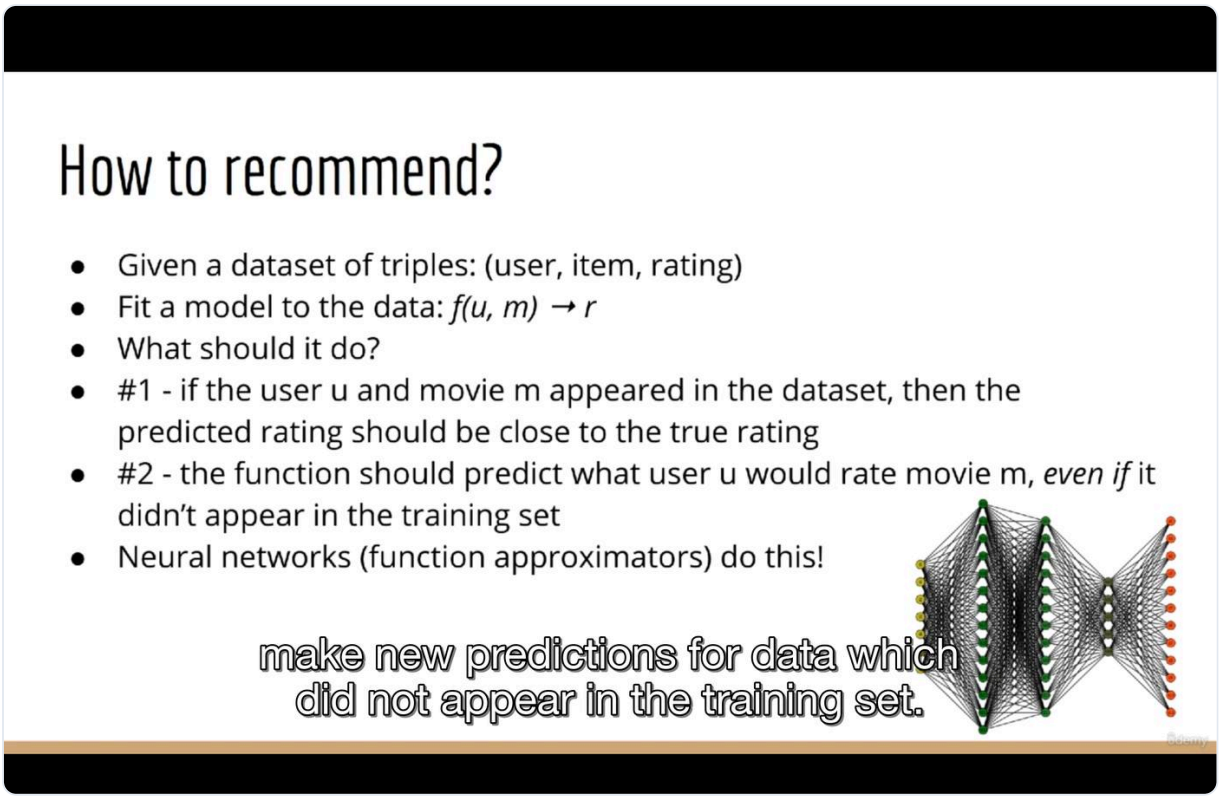


FIGURE 36.18 | How to Recommend?

As you know a neural network is a function approximator it fits to the data set so that when it makes predictions those predictions are close to the training targets but not only that it can also make new predictions for data which did not appear in the training set. So how might we use such a model to make recommendations. Well here's one strategy.

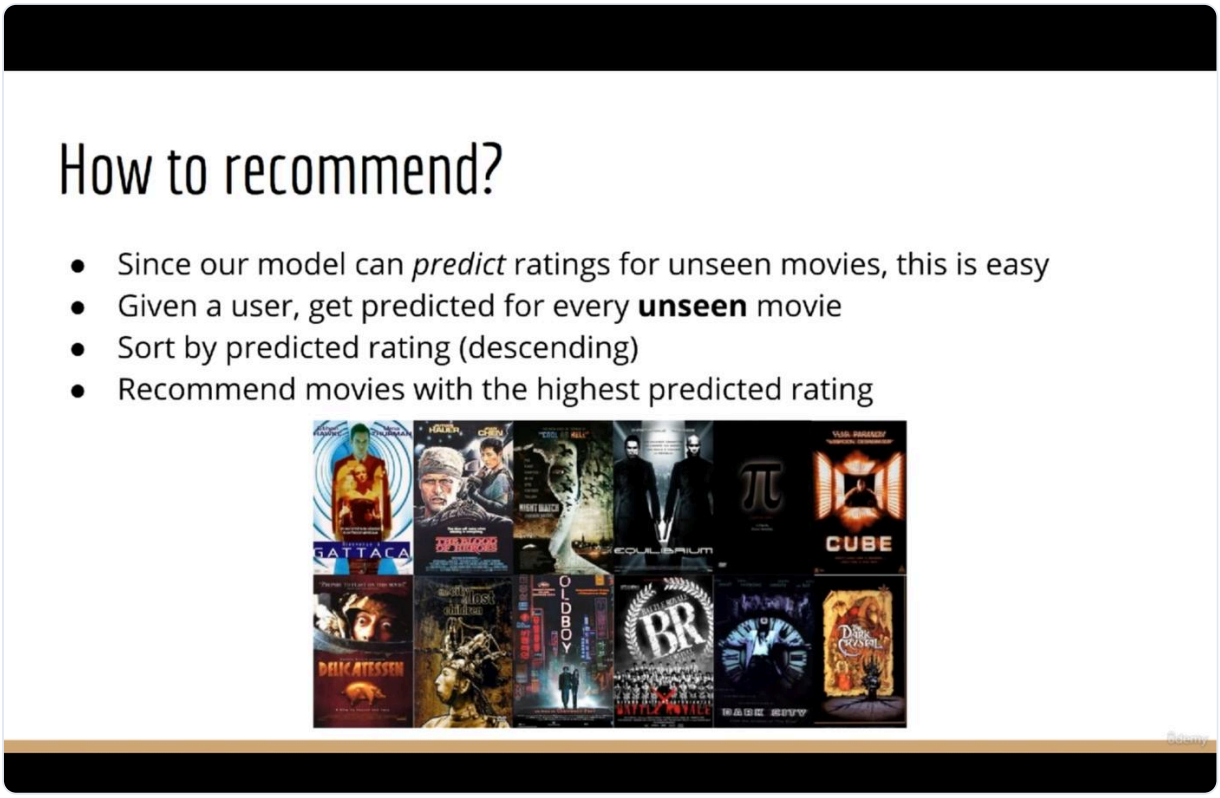


FIGURE 36.19 | How to Recommend?

Once we have a model that can predict ratings this becomes easy for a given user predicts a rating for every movie that they have not yet seen then sort these movies by predicted rating the movies you want to recommend are simply the movies with the highest predicted rating for this user. Pretty simple The next question is how can we go about building such a model. We know that we need a neural network of some sort.

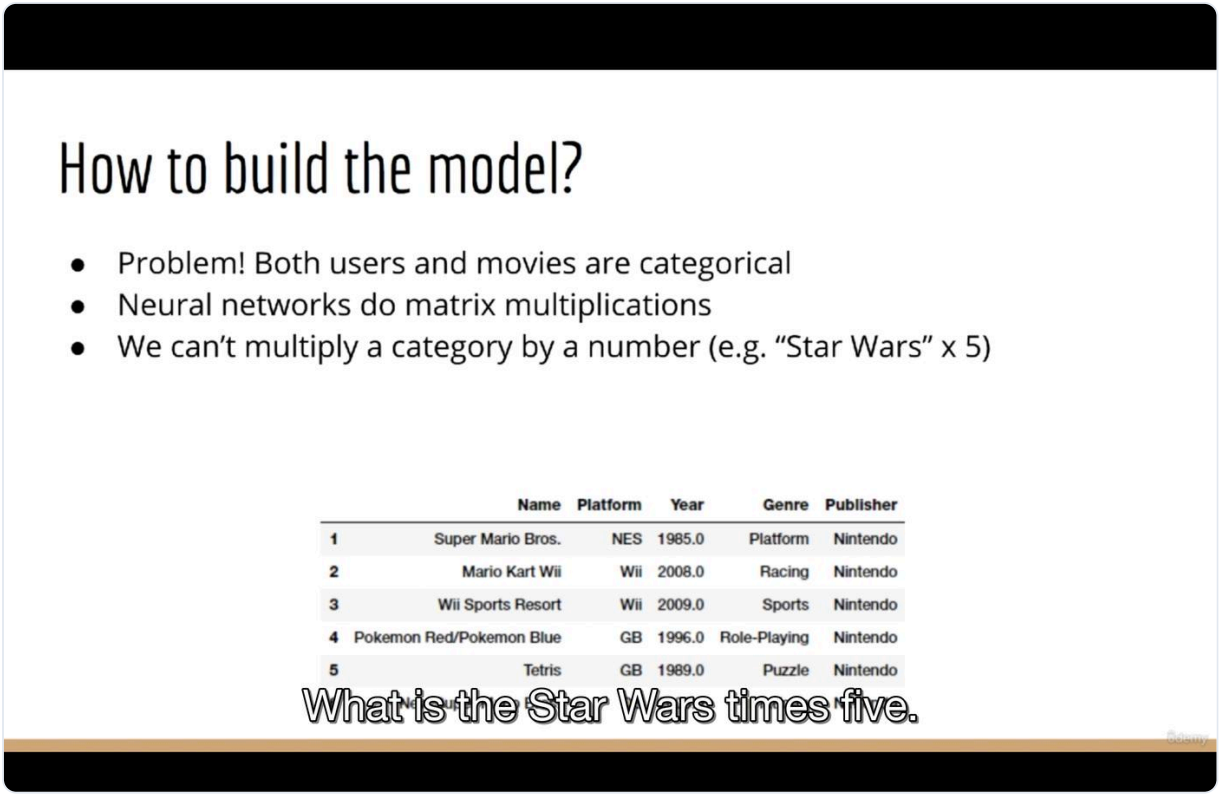


FIGURE 36.20 | How to Build the Model?

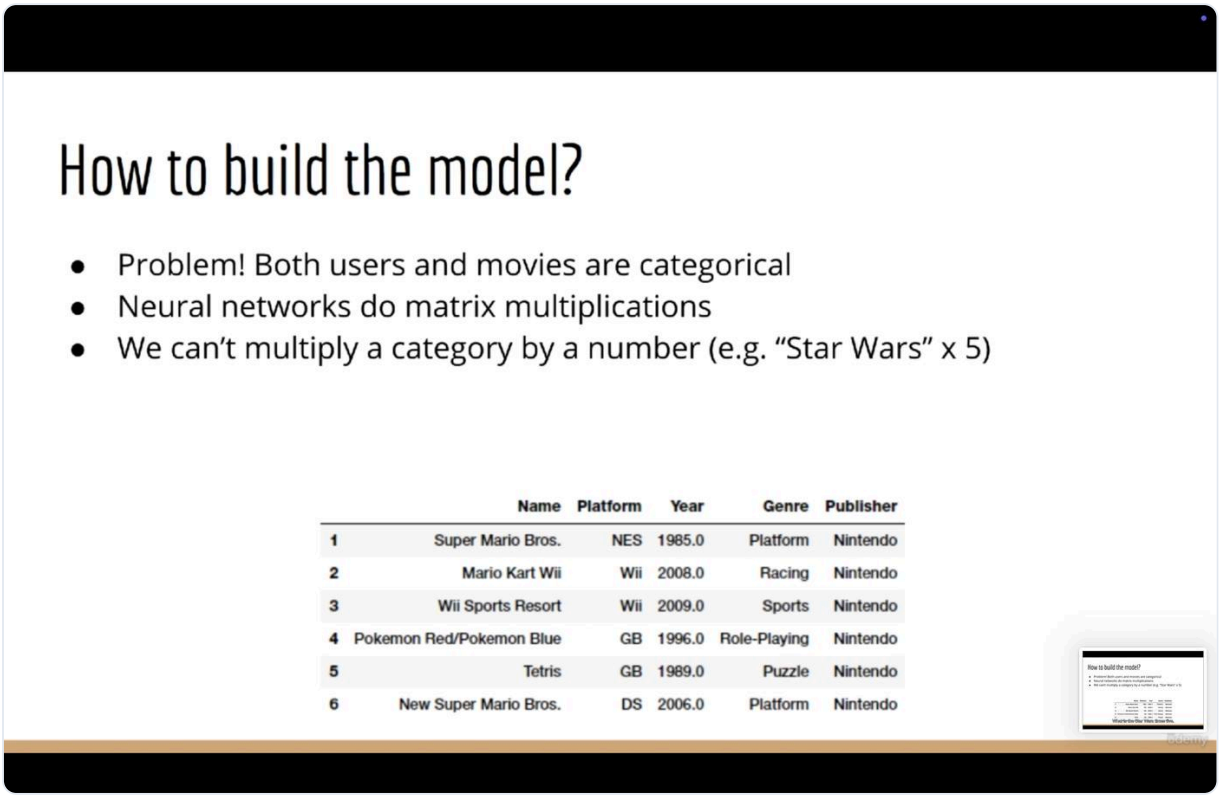


FIGURE 36.21 | How to Build the Model?

But here's a problem both users and movies are categorical objects neural networks are basically a series of matrix multiplications. As you know we can't multiply a categorical object by a number. What is the Star Wars times five. This does not make sense luckily we have encountered this problem before. Strangely natural language processing is the field we draw inspiration from in order to build deep recommender systems. As you know in NLP the main input is words. Words are categorical. They cannot be multiplied by numbers. So what do we do.

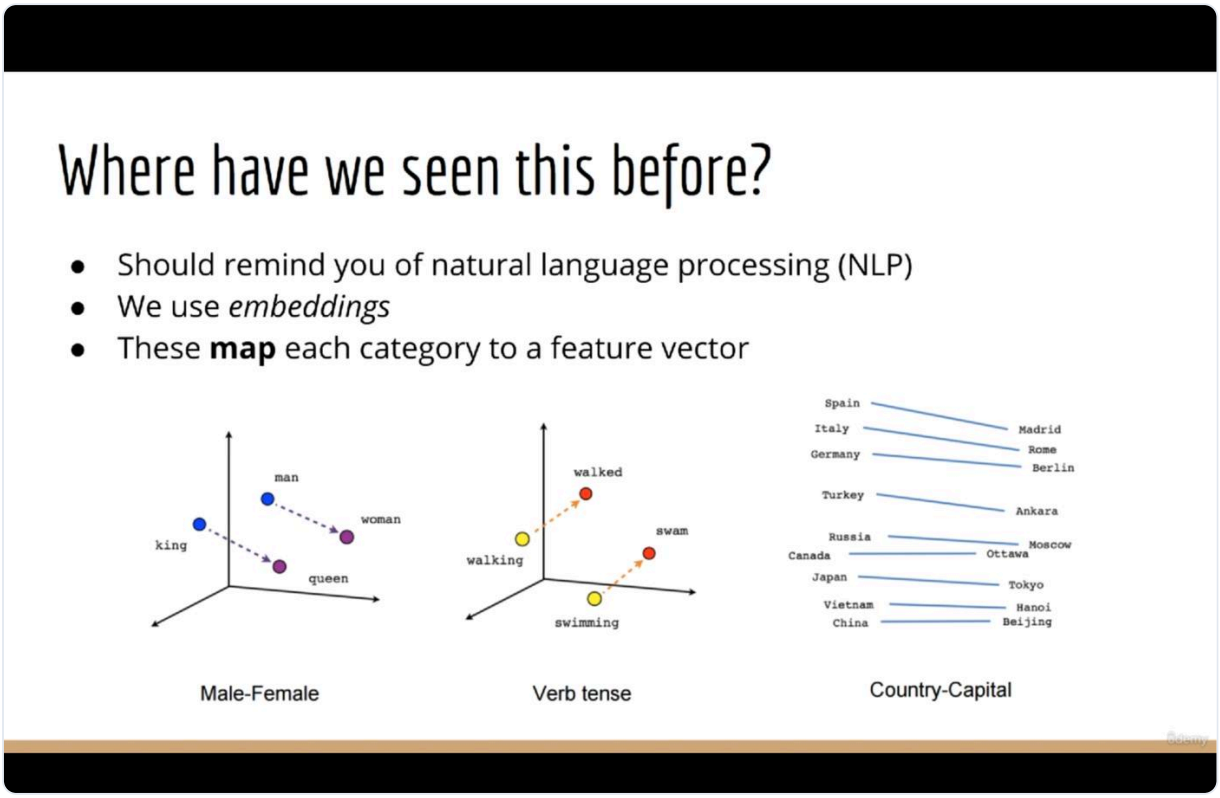


FIGURE 36.22 | Where Have We Seen This Before?

We use an embedding Recall that an embedding is a mapping from each category to a feature vector which is essentially a list of numbers that can be multiplied here's how we might use a neural network for recommender systems. Let's say we have an input user Bob and an input movie Star Wars the first thing we do is map both the user and the movie to their respective embedding vectors. So I have a vector for Bob and I have a vector for Star Wars. Now usually these embedding vectors are the same size but this need not be the case. So now both Bob and Star Wars are represented by feature vectors. The next thing we do is concatenate these two feature vectors into a single feature vector.

Once we have a feature vector the rest is obvious. We just do the same thing we've always done pass this through a neural network since this is a feature vector and not some special object like an image or a sequence.

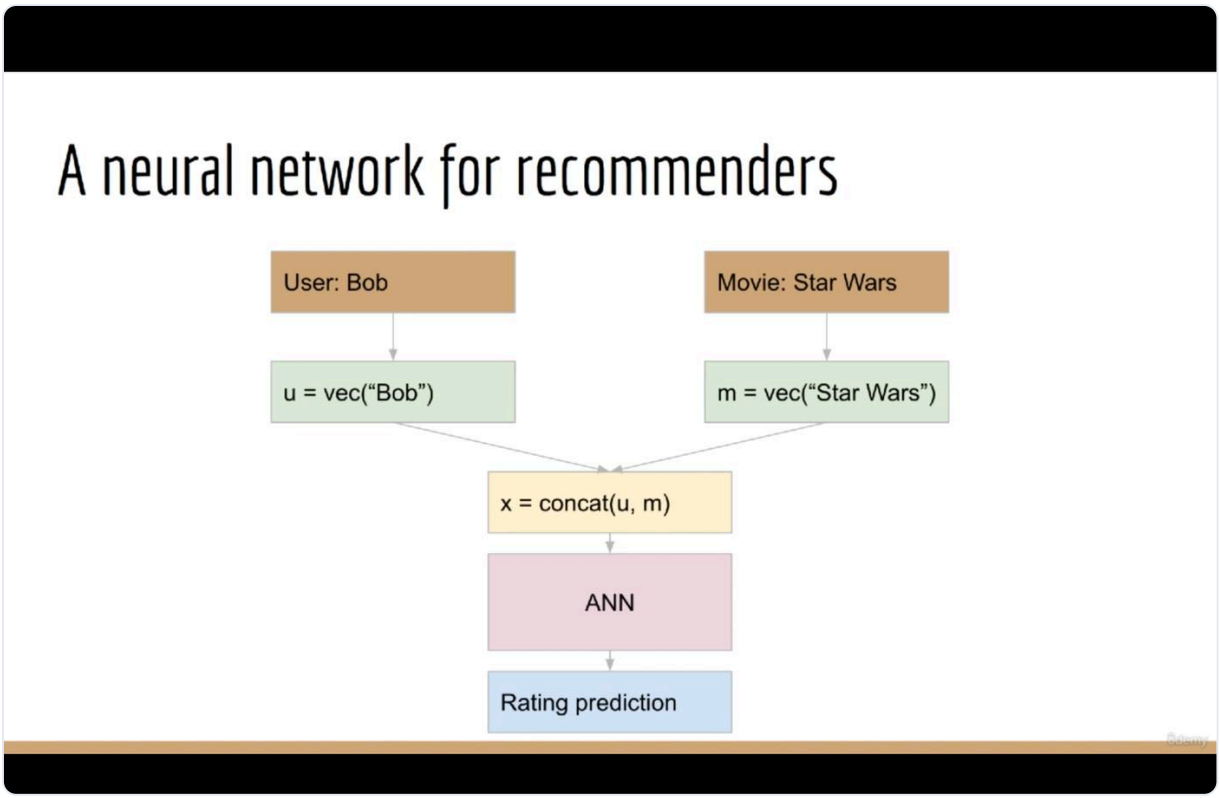


FIGURE 36.23 | A Neural Network for Recommenders

We can use simple ANNs rather than CNNs or RNNs for recommender systems. The final output layer will be a linear regression with no activation function since predicting ratings is a regression task and not a classification task as a point of interest. I said earlier that we can use NLP techniques to inspire models for recommender system. In fact the opposite is true as well recommender techniques have been used in NLP. My favorite example of this is a technique called Matrix Factorization Matrix Factorization which we haven't talked about is an algorithm that has been known for quite a while in the field of recommender systems around the time word embeddings were becoming popular.

Specifically due to the algorithm known as word2vec another similar algorithm came out called GloVe GloVe essentially solves the same task as word2vec. It's an algorithm for finding word meanings using these word embeddings you can do all sorts of fun analogies such as King minus man equals Queen minus woman.

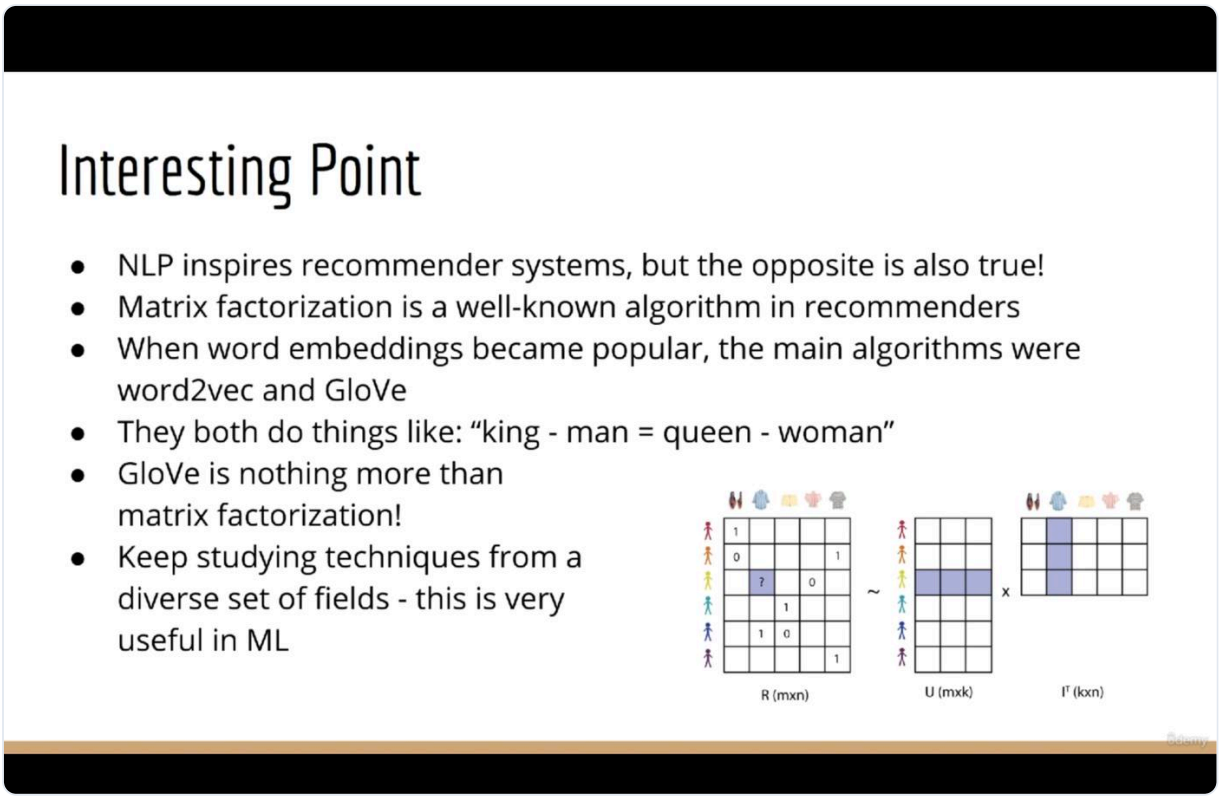


FIGURE 36.24

Well it turns out that GloVe is nothing more than Matrix Factorization. Just an old technique from recommender systems so it's always good to study techniques from different fields because you never know when or what you learn that can be applied to your own work.

LECTURE 75

User Movie Rating

OK so now that you understand how everything works conceptually let's talk about how it will work in code. First of all the data itself is going to be very different from what you're used to before. Whenever we wanted to use ANNs we remembered the rule. All data is the same.

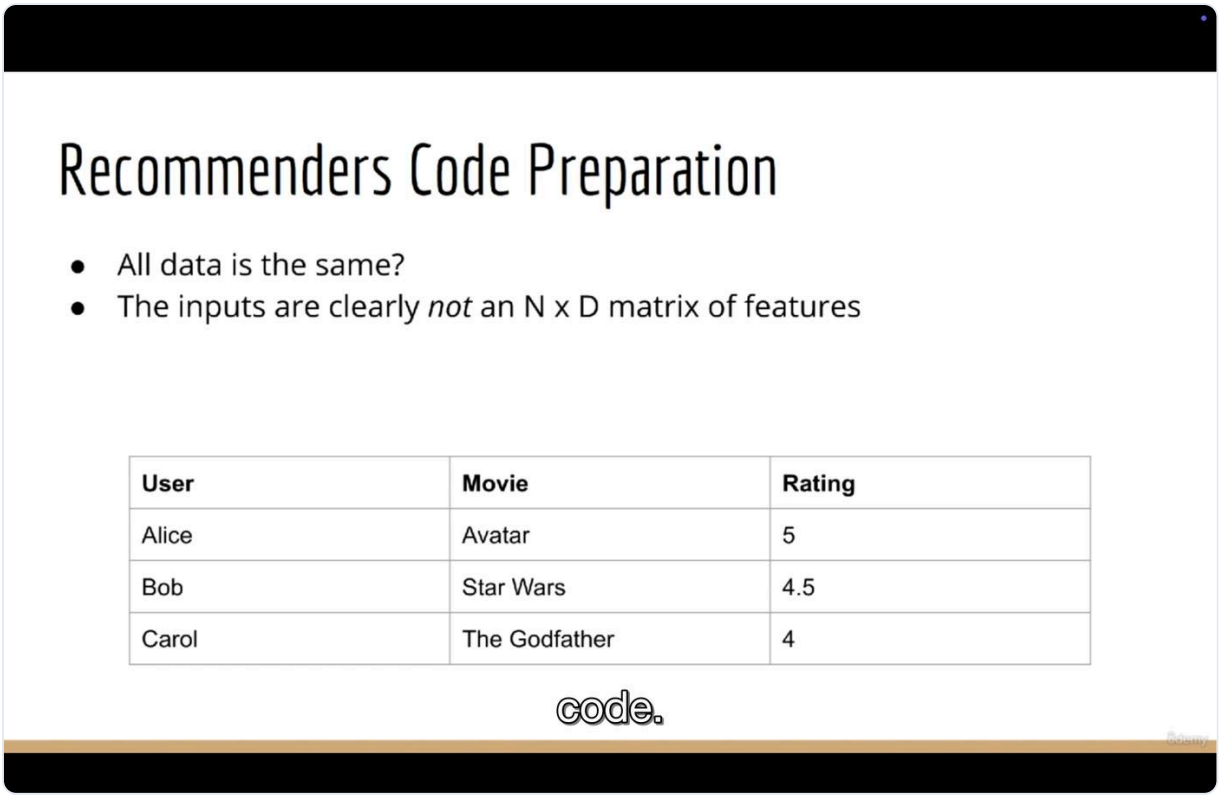


FIGURE 37.1 | User Movie Rating

But now it appears that my rule has been broken. All data is in fact not the same. Recall that with the recommender systems we want to look at all the data comes in the form of triples the inputs or the user and movie while the target is the rating. Clearly a sample of users and movies does not make up an N by D matrix of features so an N by D matrix of features is what we want but what do we actually have.

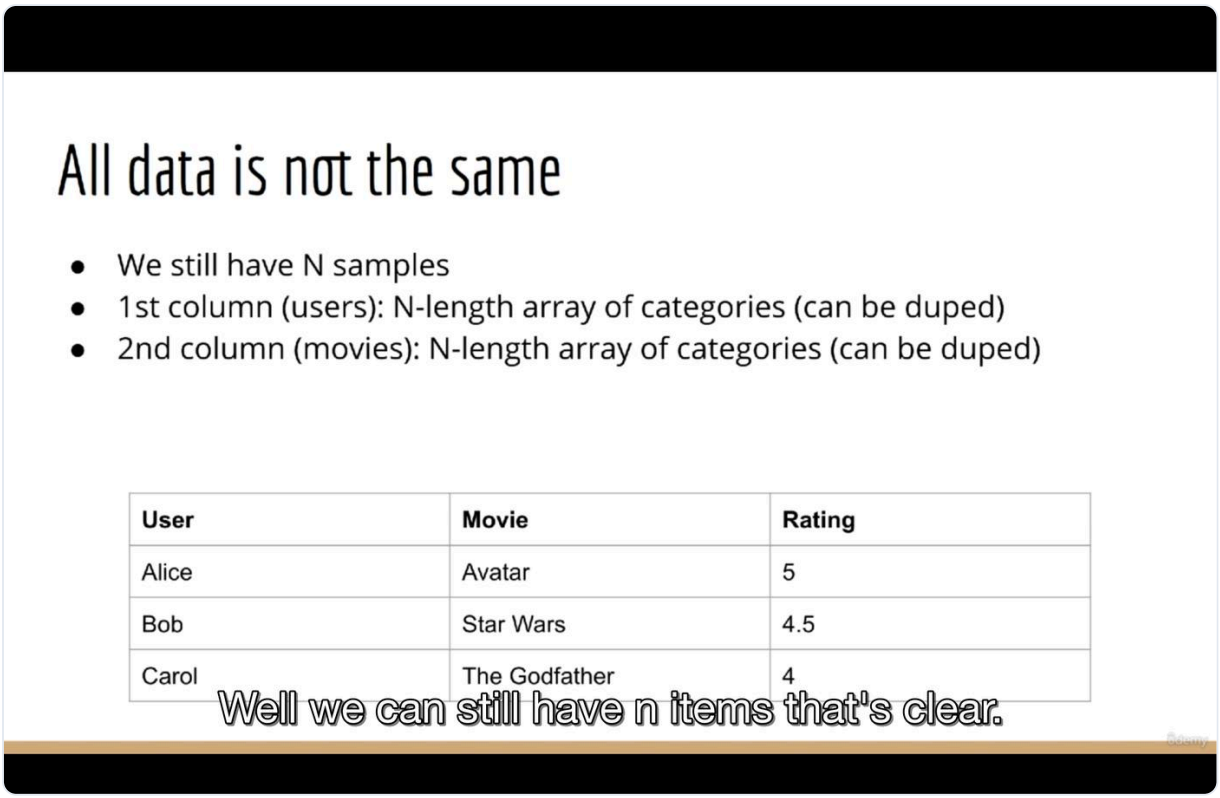


FIGURE 37.2 | All Data Is Not the Same

Well we can still have n items that's clear. That's just the number of samples. But let's look at it this way a list of end users which by the way can contain duplicates will be an N length array of categorical objects. Similarly a list of n movies which also may contain duplicates will be another N length array of categorical objects. Now you might wonder how can the list of users and movies contain duplicates. Of course this must be the case the same user may have watched multiple different movies in which case that user will show up multiple times in the data.

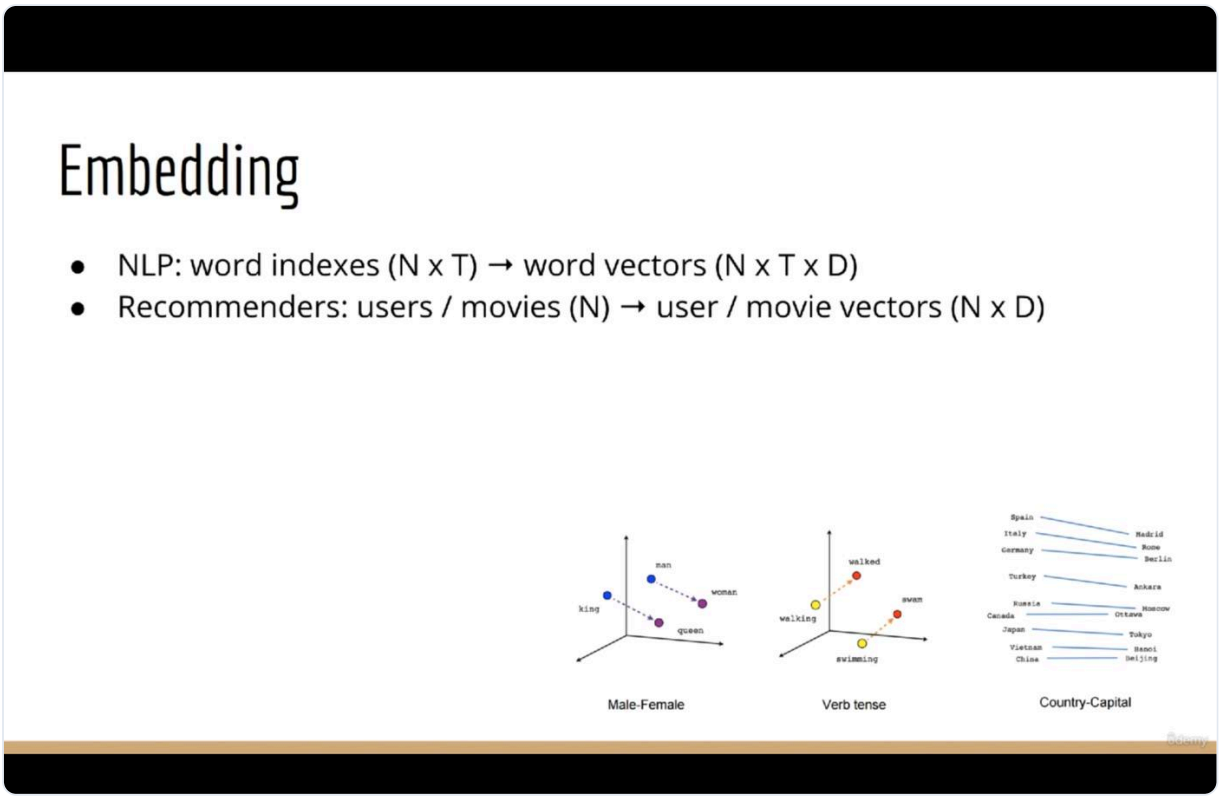


FIGURE 37.3

Conversely multiple users will of course rate the same movie in which case that movie will show up multiple times in the data as we discussed. The key operation inspired by NLP is the embedding in NLP. We have an N by T matrix of word indices which we use to index the embedding matrix mapping each word to a feature vector. We get back an N by T by D array of word vectors. Similarly both users and movies which since they are categorical will be represented by user indexes and movie indexes so the users will be an N length array of integers and the movies will also be an N length array of integers. After we index an embedding matrix we will get back N by D arrays of feature vectors for both users and movies.

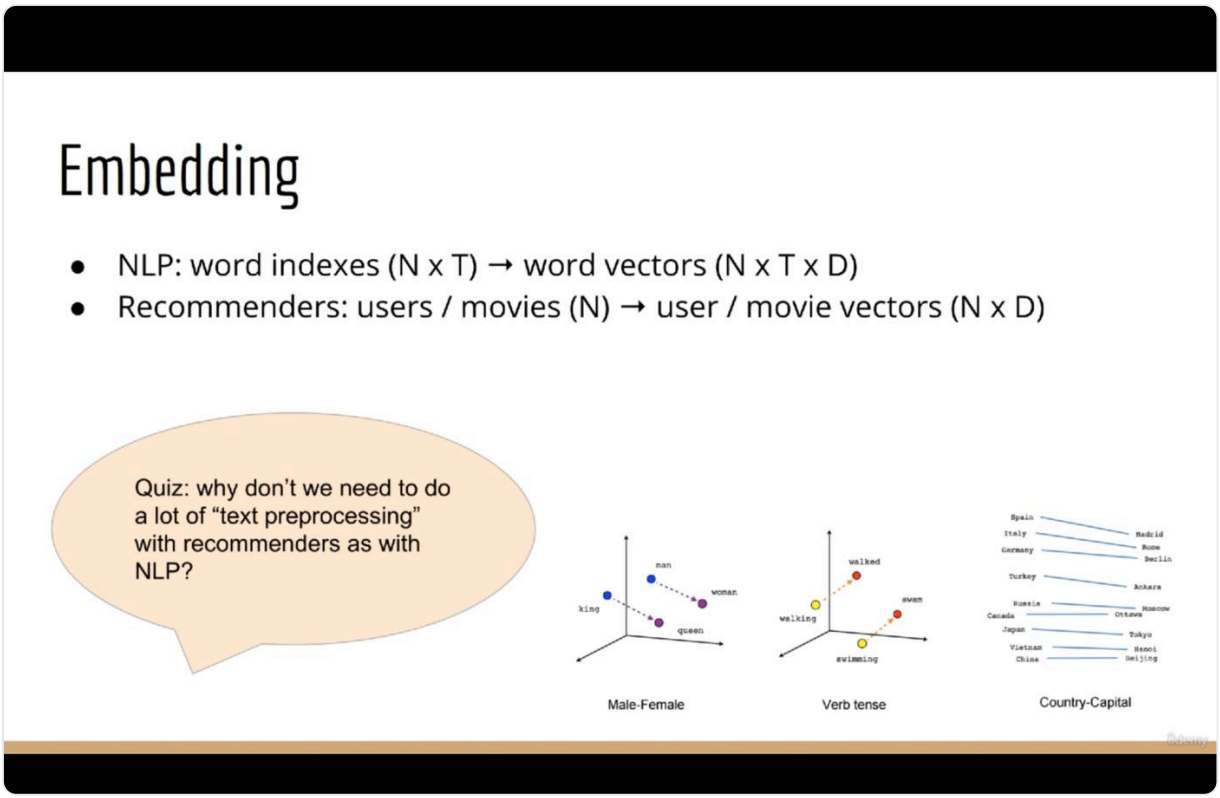


FIGURE 37.4

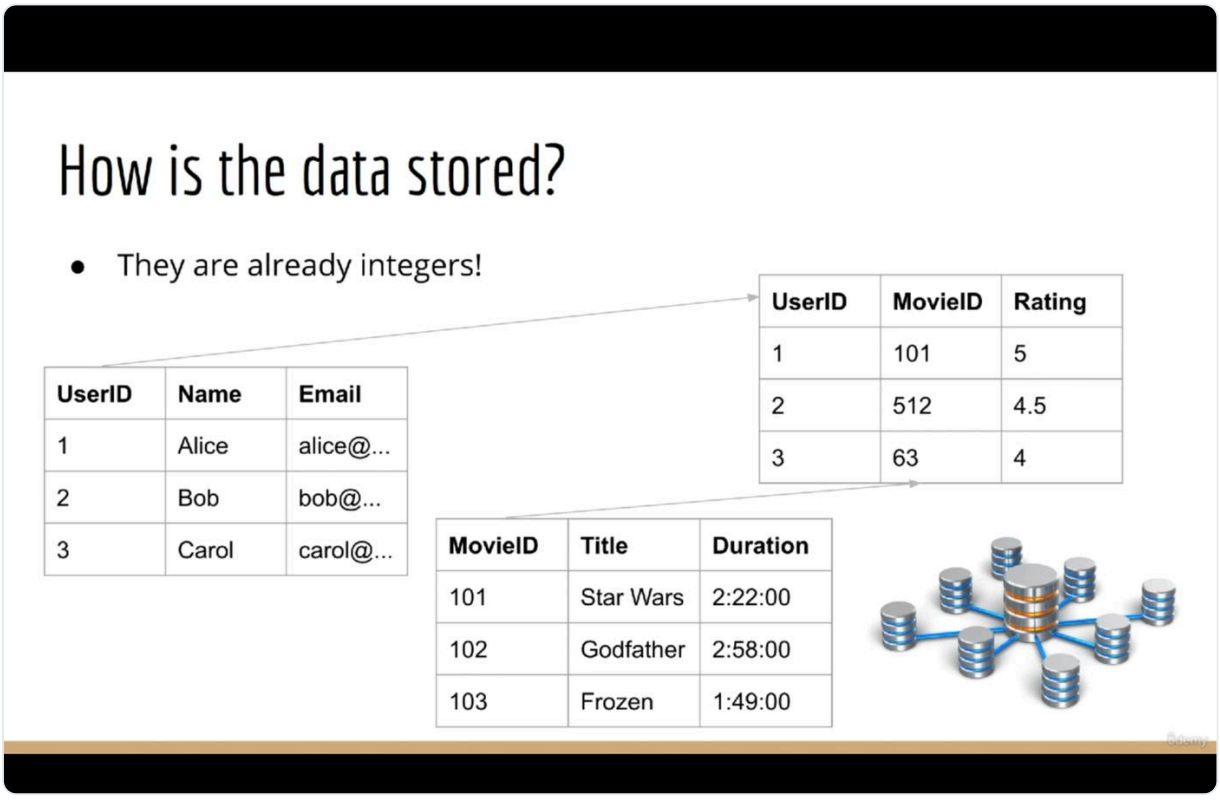


FIGURE 37.5 | How Is the Data Stored?

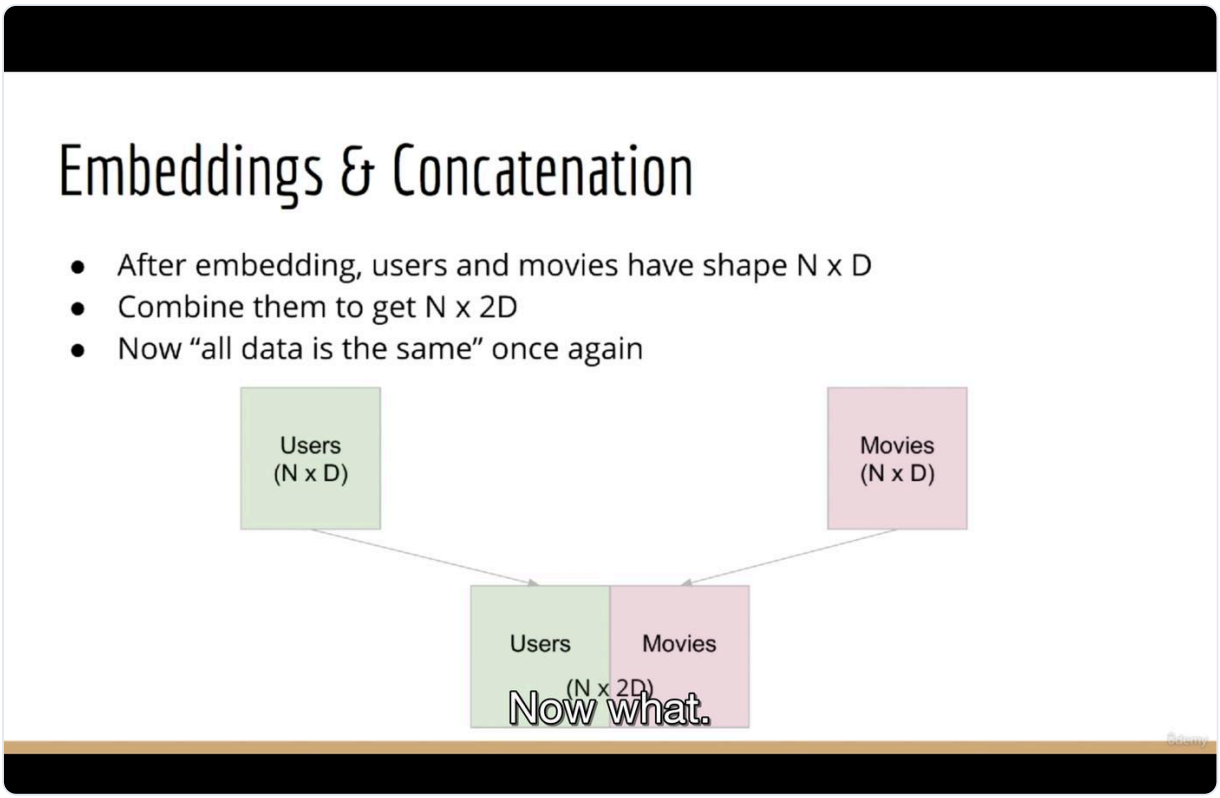


FIGURE 37.6

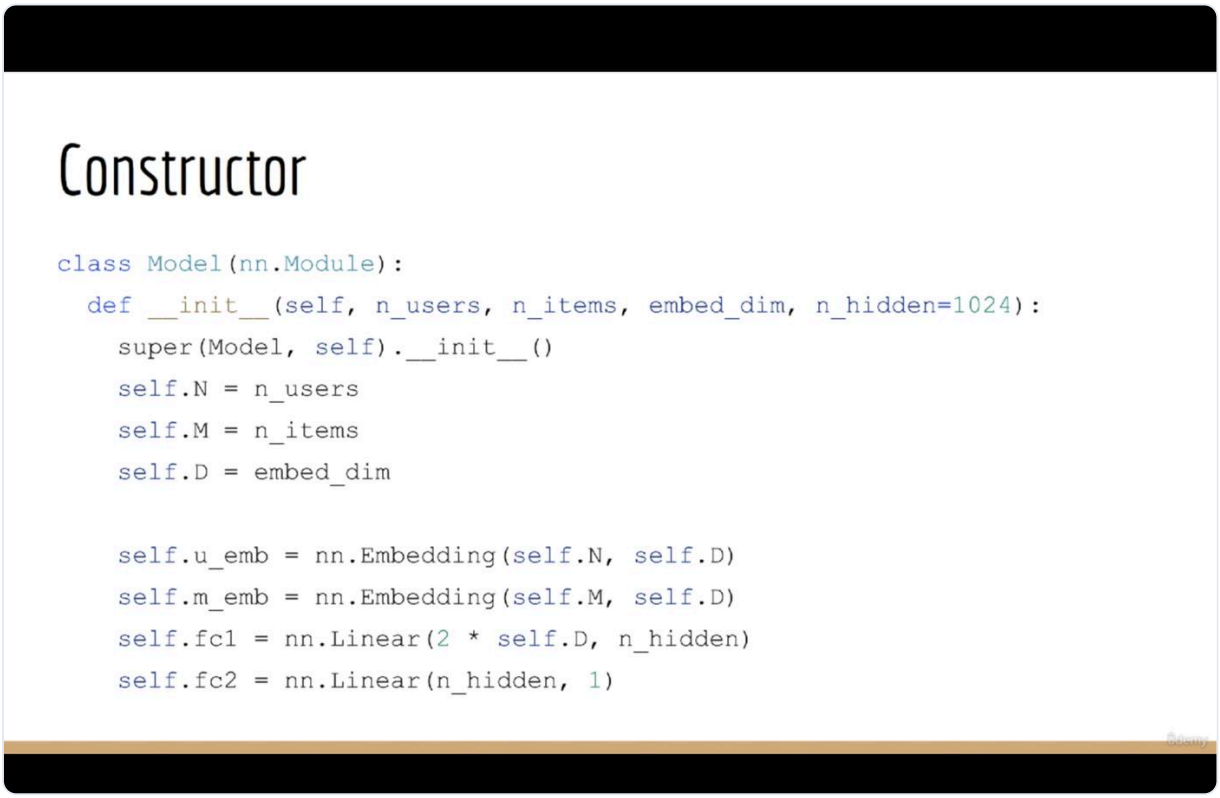


FIGURE 37.7

Forward

```
def forward(self, u, m):
    u = self.u_emb(u) # output is (num_samples, D)
    m = self.m_emb(m) # output is (num_samples, D)

    # merge
    out = torch.cat((u, m), 1) # output is (num_samples, 2D)

    # ANN
    out = self.fc1(out)
    out = F.relu(out)
    out = self.fc2(out)
    return out
```

Something special about this model: it has 2 inputs, not just one!

Probably exactly what you expect.

FIGURE 37.8

Loading in Data

- Data is just a CSV (can use Pandas)
- What if we have millions of ratings? Should use some data generator
- Unlike vision / text - no specialized library

```
# Make datasets
Ntrain = int(0.8 * len(ratings))
train_dataset = torch.utils.data.TensorDataset(
    user_ids_t[:Ntrain],
    movie_ids_t[:Ntrain],
    ratings_t[:Ntrain],
)
```

UserID	MovieID	Rating
1	101	5
2	512	4.5
3	63	4

This is pretty much guaranteed.

FIGURE 37.9

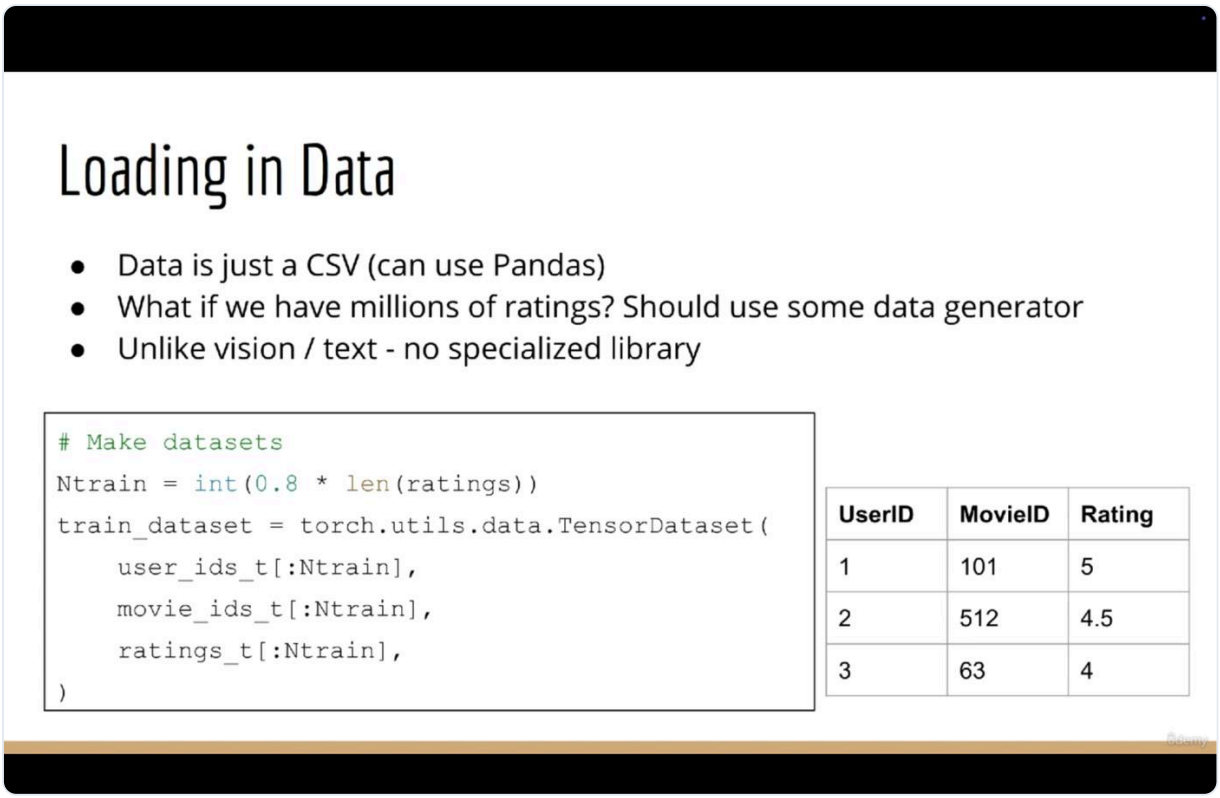


FIGURE 37.10 | Data Is Just a CSV (Can Use Pandas)

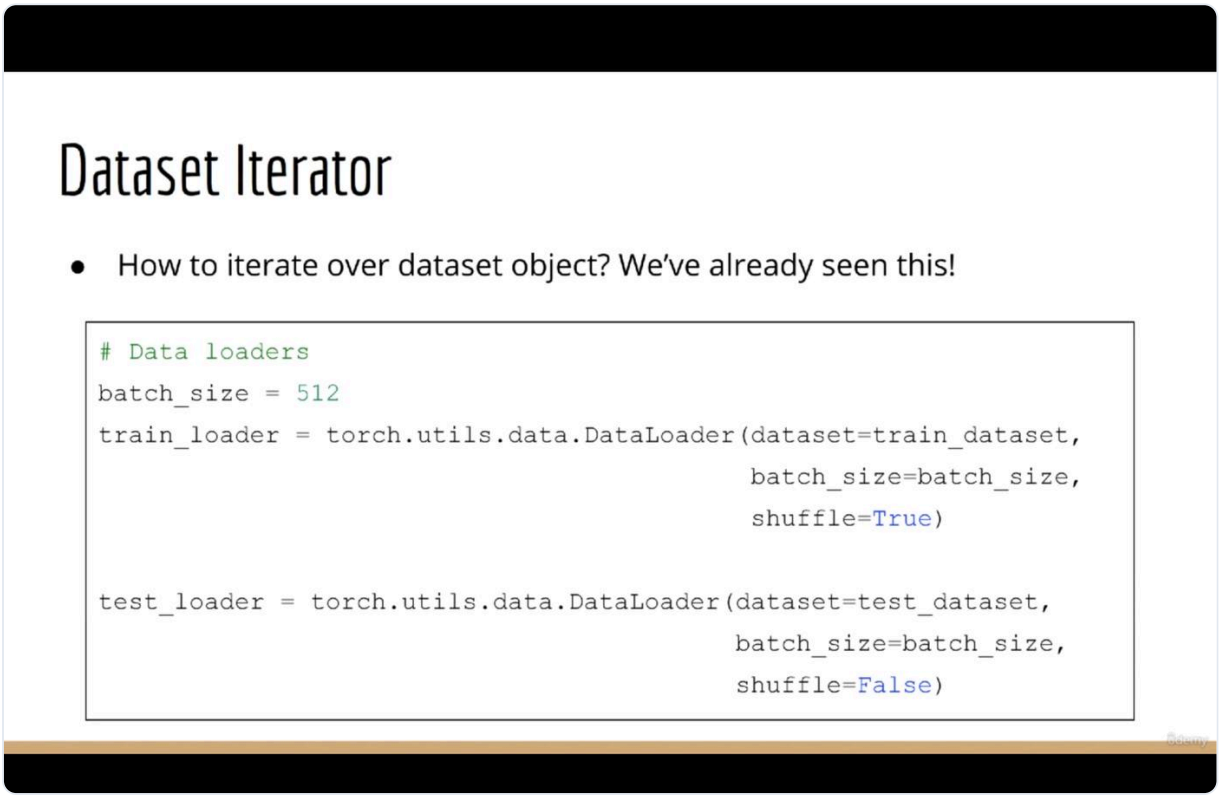


FIGURE 37.11 | Dataset Iterator

As a side note unlike NLP where we have to do a lot of text processing this is not the case with recommenders as a quiz question. I would like you to think about why. Please take a moment to think about this or pause the video until you think you have the answer. OK. So why do we not need all the text preprocessing that we do for NLP. With recommenders. Well we have to think of the natural state of the

data data like this is usually stored in a database in databases. We don't pass around strings all the time because strings take up lots of space and they're inefficient. For example checking whether or not one string is the same as another string. It's not efficient.

On the other hand comparing whether two numbers are equal is efficient. So in a database when you store a movie you you'll probably have a table just for movies where you give each movie an integer ID and then in that table you'll have a column for the movie name. Its synopsis the director the producer. It's duration and so forth. By the way the director and producer will probably also just be integer IDs. Similarly users will be stored the same way so you'll give users a user I.D. And then along with that you'll store their name email address and so on therefore in the ratings table you would not say something like Bob rated Star Wars a 5 although that would be its interpretation.

Instead you'll say user 3 4 5 rates movie 6 7 8 5 okay. So we've established that there won't be any need to do text preprocessing and we've established that the first thing we need to do is map each user and movie to a feature vector using embeddings. Now what. Well now we can concatenate the user vectors and the movie vectors that make up each batch. This will give us a matrix of size N by 2D and so now we come back to the situation where we can again say all data is the same. Since this is nothing but a two dimensional data matrix we can do anything with it anything that we would normally do in machine learning on data of the same kind i.e. We can pass this through a neural network.

So here's what the code might look like for a recommender model in the constructor we take in some arguments for the total number of users the total number of movies the embedding dimension and the number of hidden units. Of course you could make this more fancy and let your model be a neural network with multiple hidden layers but we won't do that. You may want to try that as an exercise you can think of the total number of users and the total number of movies like vocabulary sizes for the users and movies. So if we call the number of users N and the number of movies M and the embedding dimension D then the user embedding will be a matrix of size N by D.

And the movie embedding will be a matrix of size M by D by the way don't confuse this N with the N that we used previously which meant the number of samples in a batch on this slide and within the code. That's not what we mean. And the reason for this sort of overloaded use of the letter N is because that's what people normally use in recommender systems. So just make sure you're paying attention to the context. Once we create our two embeddings all we need to do is create our ANN layers which for our simple model can just be too linear layers. And by the way since this is a scalar regression problem we'll have one output okay. So what does the forward function look like.

Probably exactly what you expect. Well maybe not since there's one thing you haven't seen before. Earlier in the course we started by looking at very simple ways to build models modules themselves our models. So if I create a linear module I can call that a model or I can take a whole sequence of modules and wrap that in a sequential and I can call that a model later we talked about custom models. We saw that with our RNNs. There's a little extra work that we need to do. That can't be done in a sequential model at least not yet. So custom models were necessary. There was another reason I mentioned for why custom models are useful and this is it. This model is unique in this course because it's the first one we've seen that takes in multiple inputs.

As you can see the arguments into the forward function are not just a single X like we had before but now we have U and M U and M in this case are both one dimensional arrays containing user IDs and corresponding movie IDs a sequential model would not be able to handle an input like this. So the first thing we do is pass U and M into their respective embeddings. After that both the users and movies will

be represented by arrays of size num samples by D. The next step is to merge or concatenate these two matrices together along Axis 1 to get back an array of size num samples by 2D finally it's just business as usual to pass this to the ANN layers.

Okay. So the next thing to consider is how will we load in the data the data is going to come in the form of a CSV or a TSV or something like that. So we can use pandas to load in this dataset. We can even just turn it into a NumPy array right away since we don't actually need any pandas functionality. But here is one thing to consider ratings data sets are usually huge. You might have millions of data points if your business has millions of users. This is pretty much guaranteed. So just like with vision and text you probably want to use some kind of data generator or data iterator but unlike vision and text there aren't any specialized libraries that will do all these steps for you luckily there are generic data set objects that we can use that just loop over arrays without any regard to what's in them.

So here's an example of how to use tensor data set which takes in an arbitrary length sequence of data tensors. For us we'll pass in three tensors the users the movies and the ratings later in order to loop over each data set for training. We use the data loader object which we've already seen. The first argument is the data set objects which we just looked at. The second argument is the batch size and the third argument is whether or not to shuffle the data. And once you have this it's just business as usual defined your training function and inside that function to iterate over the data loader.

LECTURE 76

IDs and Num Samples

See the notebook `19.PyTorch-Recommender-20M-Unmodified.ipynb` for the code.

In this lecture we are going to look at how to build a recommender system using deep learning. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. So none of the imports are too surprising so let's start with downloading the data. This data is hosted at group lens dot org and we'll be looking at the famous movie lens 20 million dataset. Note that there are multiple versions of the movie lens dataset and most tutorials and courses use the version of the dataset which only has 100000 ratings.

But that's kind of lame so we're going to demonstrate that we can do better. Now since this is a zip file we're going to unzip it so you'll see that it comes with quite a few files. I'd recommend looking at these since I've already done the hard work for you by figuring out which is which. But you want to do that yourself at some point since you know that's part of being a data scientist. So next we do an ls to see what we now have in our working directory. Next I load in the data using pandas and of course the data is a CSV so the first column is the user I.D.. The second column is the movie ID. The third column is the rating and the fourth column is the timestamp.

So if you wanted to make some kind of really complex recommender system that maybe recommends the right thing at the right time you could take into account the timestamp or the order of the events as they occur. But that's not really a thing that people usually teach in recommender systems. So we're not going to discuss it. Next we recognize that although the dataset already uses integers for the user IDs and movie IDs they might not be in the format that we want. Remember what these integers are eventually used for they're used to index an embedding so user ID 0 will index the first row user ID 1 will index the second row and so on. So think of a really not ideal scenario.

Let's say you have one hundred users but these one hundred users have the IDs 1 million 1 million and 1 1 million and 2 and so forth. Well what happens is that since each ID must be an index into an array that array must have size 1 million and one hundred that's about 1 million rows of wasted space space that you won't be using. Therefore you would like your user I.D. To be numbered from zero up to N minus one. The thing is we can't just trust that the data is formatted in this way. In fact I would recommend checking to verify this for yourself as an exercise. In any case that's what this next block of code does. So I've commented out some code that you might use if you wanted to implement an index assignment from first principles using the apply function.

Of course if you read the documentation for long enough eventually you'd stumble upon this useful thing where you can just cast the entire column to be a PD dot categorical object which automatically assigns integer IDs starting from zero. Then we can grab those IDs by calling dot cat dot codes. So next it's the same story for the movie IDs. Again as an exercise you want to verify that the original IDs were or were not in the format. We would like them next since our data is currently in a data frame. We're going to split them into individual NumPy arrays we can do that by calling dot values on the relevant columns. Also we're going to center the ratings data which is part of the standardization process.

The reason I'm not also dividing by the standard deviation is because in recommender systems literature results are often reported in terms of the root mean squared error. The RMSE of course this metric is reported on the scale of the original ratings. So if we change the scale our results won't be directly comparable to other results. One alternative is just scale your predictions and targets back before calculating the metric. But I leave that up to you. Next we're going to find N the number of users and m the number of movies. We'll also set D the embedding dimension. The next step is to define our custom neural network for recommenders in the constructor.

We'll take in the number of users the number of movies the embedding dimension and the number of hidden units as an exercise. You could extend this to have multiple hidden layers inside the constructor. We create all the necessary modules we have two embeddings and two linear layers in the forward function. We define how to get a prediction given some input user and some input movie. As usual it's important and useful to pay attention to the shapes the U variable starts out as a num samples length array as does the M variable. The first step is to pass the user IDs and the movie IDs into their respective embeddings. This gives us two things of size num samples by D next we concatenate these along the 1 axis and we get back a thing of size num samples by 2D.

After this it's just a regular neural net where we pass it through a dense layer to a ReLU and then a final dense layer next we create a device object. Next we instantiate our model and move it to the GPU. Next we create the loss and optimizer and we shuffle the data. This is so that we effectively randomize the train test split. Next we convert the data to torch tensors next we create tensor data set objects for both the train set and the test set. We defined the train set to be the first 80 percent of the shuffled data and the test set to be the last 20 percent. Next we create data loader objects from our data sets after this. Things are more or less the same as usual.

We create our training function which by the way has to move the data to the GPU manually now previously the special vision and text libraries were doing that for us. Next we run the training function. You'll notice that I've got this special prun magic command. This is a profiler that tells us things like how much time we spent any function. So why did I want to do this. Well you'll notice that each epoch took quite a long time over five minutes. I thought this was very strange. And so if you look at the outputs from the profiler you can see that a majority of the time spent isn't coming from our for loops or the model since our model is quite small.

It's actually coming from the PyTorch data set functions. So while you might assume that if you're using PyTorch you should use PyTorch data loaders. Maybe that's not always the best idea. Sure you could try to figure out if maybe there's some arguments we could have set to improve the performance or something like that but for a basic data set like this you shouldn't have to it should just work which is the whole point of built in functions in the first place. In any case we'll discuss that more in the next lecture so the final step is to plot the loss per iteration and that shows us that the model has converged. Although you should pay attention to the final MSE here because we are going to improve this value in the next lecture.

LECTURE 77

Software Engineer and TensorFlow Model

See the notebook `19.PyTorch-Recommender-20M1-Modified.ipynb` for the code.

In this lecture we are going to look at a modified version of the recommenders code. This notebook will make some major improvements on the previous code and also I will talk about how I discovered what modifications to make. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. Okay so before we look at the code I want to make a special note about why it was rewritten and why it might confuse a lot of beginners.

So from a general software engineering perspective everything we did in the old script was totally right. We were using the functions that PyTorch had given us and we were doing things the PyTorch way. One thing I really advocate against is when students come to me and say hey lazy programmer Why did you write your code this way. Why aren't you using this built in function. Instead of writing this complicated code that's hard for me to understand. Why are you reinventing the wheel. And if you're a beginner or maybe you don't have that much experience yet but you are a newly minted software engineer who is very intent on following the rules and being a good well-behaved software engineer.

Then you would tend to agree but I hope to teach you why this is a bad idea and I hope that you will learn that being a good well-behaved software engineer is not always optimal. So first of all if you're taking a course because you're trying to learn how something works then you should expect to implement some things yourself. That's what learning is. As always my rule goes if you can't implement it then you don't understand it. Okay. So maybe it's hard for you to understand but this is not a valid reason for you to not do something. If it were easy then you wouldn't need to take a course on how to do it in the first place. If it were easy you should be able to figure it out yourself.

So if you're looking for things to be easy then you're probably taking the wrong approach. I'm not a fan of programmers who excel at Reading documentation but are poor at problem solving. Programming isn't an exercise in memorization but rather it's a task that involves building and putting things together from basic building blocks it's better to ask how can I build this rather than asking how do I spell some function name that does this one is a spelling exercise and one is a thinking exercise. Some people say hey you're the lazy programmer aren't you supposed to advocate the lazy approach. And to that I say being lazy is good. If you're that good kind of lazy good lazy is when you're being more efficient.

Bad lazy is when you're trying to be more efficient but that leads to you being less effective and less skilled and less knowledgeable. So keep these ideas in mind as we go through the lecture okay. So in this script since it's so similar to the previous one I'm just going to go over the things that are different so let's go down to where we define the model now it's pretty obvious that I've trained this model many many times in the past. In fact I have an entire course on recommender systems so I noticed that when I implemented this model aside from the fact that it was training very slowly it was also not getting the mean squared error I expected.

So you might wonder how do I debug something like that if I know my TensorFlow model performs very well. But my PyTorch model does not. What can I do. Well here's one thing you can do. First you want to make sure that both models are equivalent. So if I create both models side by side and I copy the

weights from one model to the other I should get the same model predictions given the same inputs. If I don't then something is wrong so you check that and it's a good sanity check. And let's say that it works. The next thing to ask is Well maybe the optimizers are implemented a little differently or they have different hyperparameters. This is totally possible.

So you can test that as well. Basically you write your own custom training loop and for both models look at the data at the same time and do the exact same updates. If both optimizers are the same then both the loss per iteration and the model parameters should be the same after each update. So you can plot the loss per iteration for both models and if they're the same then that means the optimizer is the same. So that's a good sanity check as well. Of course before you start the training process you have to make sure that both models start with the exact same weights. Luckily you already know how to copy weights from one model to the other since that was your first sanity check.

Okay so let's say you do that and the loss per iteration is the same for both models. Now what. Well you have to ask are you getting the worse mean squared error or the better mean squared error. If you copy the PyTorch weights to the TensorFlow model and both the optimizers are the same then you should see the worse mean squared error. If you copy the TensorFlow weights to the PyTorch model and both optimizers the same then you should see the better mean squared error. So at this point you're narrowing down the source of the problem. Basically it has to do with weight initialization This is a topic that's outside the scope of this course. So if you're not familiar with it please don't worry about it.

You can learn more in my in-depth series but basically there are different kinds of weight initialization. For example you might sample from a uniform distribution or a Gaussian distribution. From there you can choose the limits on the uniform distribution or the variance of the gaussian sometimes programmers will make this depending on the dimensionality of the input or the output. Well long story short if you plot a histogram of the initial weights of your PyTorch model and your TensorFlow model you'll notice that they are not the same. In other words that's the source of the discrepancy is that PyTorch initializes their weights differently from TensorFlow luckily we have only two different kinds of layers in our network embedding and dense layers.

So what we have to figure out is where are these differences coming from. How does PyTorch initialize those layers by default. And how does TensorFlow initialize those layers by default. And if they're both different do they both matter. Or maybe it's that only one of them matters. Lucky for you I've done all this work. So what you get after doing a little digging is that although both the linear layers and the embedding layers are initialized differently between PyTorch and TensorFlow it's really the embedding layer that makes a huge difference specifically if you check the PyTorch documentation you'll see that PyTorch initializes the embedding matrix to come from the standard normal regardless of the dimensionality.

It's actually very surprising that this makes a huge difference since as you know we like to standardize our inputs before putting them into a neural network. Well if you sample from n 0 1 then our inputs will be exactly standardised. So it's kind of surprising that this leads to poor results. In any case what I ended up doing was manually initializing the embedding layers to come from a normal distribution with a standard deviation of my choice. So this is how you would do that. So for each of the embeddings we can access the weight attribute which then has a data attribute which we can set simply by using the equals operation on the right side. We create a parameter object which wraps a tensor object which wraps a NumPy array and that NumPy array is a random matrix I generated myself all right so that takes care of one part which is the accuracy of the model but we also have this problem that the model

trains extremely slow so you'll notice that in the script there are no more data loaders or data set objects so we just go straight to the training function which takes in two new inputs the train data and the test data.

These are just tuples that store the NumPy array versions of the data and if you've taken my in-depth deep learning courses then you're in luck because we've written this loop many times for ANNs CNNs RNNs and so forth and back in the old days you would actually have to write this kind of thing yourself. And by the old days I really mean like two or three years ago. So I don't think what I'm doing is some kind of ancient secret or something. This is still in the Deep Learning era. OK. So basically we're going to iterate through the batches manually. Now most of this is the same but some of it is a little different. So I'll walk you through the different parts.

And by the way if you already have some idea of how you would do this yourself you should try to do that before looking at this. So one thing that we need to do is calculate how long the inner loop will even be. It's going to be the number of batches but how many batches do we have. Well that's roughly the number of samples divided by the batch size. Of course they may not divide evenly. So we need to take the ceiling and then cast the result to an integer so inside the main training loop we first shuffle the data. We do this on every iteration to avoid unwanted correlations. Then we do our inner loop through each batch inside the loop. We grab the current batch.

The index range J times batch size up to J plus 1 times batch size. You want to verify this for yourself by writing it out on paper. Next we convert the batch to tensors and move the data to the GPU and we do our usual gradient descent step in down below we do a similar loop over the test set since this also may be too large to fit into memory. You should confirm that for yourself OK. So next we call our training function after splitting up the data. This is almost the same as what we did earlier except that this is in pure NumPy and we only need to convert the tensor when we're inside the training function. All right. So here are the results. So if we look at the output from our training function we notice two things.

First the accuracy is much better. These results are on par with what you would get with TensorFlow. But note that TensorFlow works right out of the box whereas you had to do some customization in PyTorch to achieve the same results. Second we notice that the duration per epoch is much lower than before before it took over five minutes per epoch. And this takes just under one minute per epoch so that's over a five x speed up just from writing our own training loop and not using the built in data sets or data loaders which is very interesting. So let that be a lesson to you. Don't blindly take the approach of everyone must use libraries in order to be a proper software engineer and don't reinvent the wheel.

Most people who say these kinds of things do so because they can't cope and because they can't code they don't want anyone else acting like they can code. They want everyone else to be just like them to appear that they have the same skill level. So don't be like these people instead know how to implement things yourself know how to investigate and debug and things like that. Otherwise you'll just be a slave to APIs and libraries.

LECTURE 78

Movie IDs

See the notebook `20.PyTorch-Recommender-20M-with-Predictions.ipynb` for the code.

In this lecture we are going to answer a very common question I get which is how do I use the train model to make predictions before we start this lecture. I want to say that this should be completely possible using only what you've learned so far. You should be able to apply what you've learned to do this since no new code is necessary. You have all the tools you need so if you would like to stop this video and try on your own I would strongly recommend doing so. To start first recognize that we are in a separate VIP notebook which you can find in the VIP notebooks document. Once you've reached the notebook we are going to scroll all the way down to where we plot the loss per iteration.

Since this is where our new work begins the first step here is to think about which movies do I actually want to make predictions for to make this simple. Let's suppose we want to make predictions for the user with user I.D. One. Well we of course only want to recommend movies that user one has not watched and in order to do that we first have to figure out what movies User 1 has watched in order to calculate this. We first need to select all the rows in the data frame where new user ID is one. Next we grab the new movie ID column and then call dot values to get this as a NumPy array in the next block. I've printed out all the movies that user 1 has rated so you can have a look in the next block.

We need to find the movies that user one has not rated which I'll call potential movie IDs in order to find these. We need to make use of the isin function in pandas by using the tilde in front of the isin function call. We are basically saying find all the rows where the new movie ID is not in the array watched movie IDs. After selecting these rows we do dot new movie ID to get the movie IDs column and then we call the unique function since obviously there will be multiple users who have watched the same movies. Next we check how many movies there are. That user 1 has not rated and it's about twenty six thousand. Of course we expect this number to be close to the total number of movies for all users unless of course there's someone out there who does nothing but watch and rate movies all day.

This is going to be the case by the way this array of potential movie IDs forms one of the inputs into our model. As you know the other input is just the user I.D. Which should be one for every movie so in order to get an array like that we want to use the ones like function to create an array of ones with the same shape as the movie IDs. As an exercise you might want to think about how you would do this if you wanted to make predictions for a different user I.D. Say user ID 10 obviously there's no such function as tens like the next step is to convert our data into torch tensors and move them to the GPU which I'm sure you know how to do by now. The next step is to pass these inputs into our model and assign this to a variable called predictions in the next block.

We convert our predictions into a NumPy array and we also flatten them since we want a 1D array and not a 2D array. The next step is to sort our predictions in descending order since we actually want the sort index. We can use the argsort function. We also have to make sure that we negate the predictions since the argsort function sorts ascending. Next I print the predictions and the sort index just for your information and because it's a nice sanity check. Finally let's suppose I want the top 10 predictions for this user. All I need to do is take the first 10 elements of the sort index which tells me where those movie IDs are located in my input array.

I can also index the predictions array to get the corresponding scores. Next I use a for loop to loop through the top 10 movies and their scores and I prints out the result inside the loop as you can see we do in fact get a list of movies with scores sorted in descending order.

LECTURE 79

Transfer Learning Intuition

In this section of the course we are going to cover one very important topic in a modern deep learning which is called transfer learning. This lecture will introduce you to both the intuition and implementation of transfer learning. So by the end of this you'll be able to understand that the subsequent code.

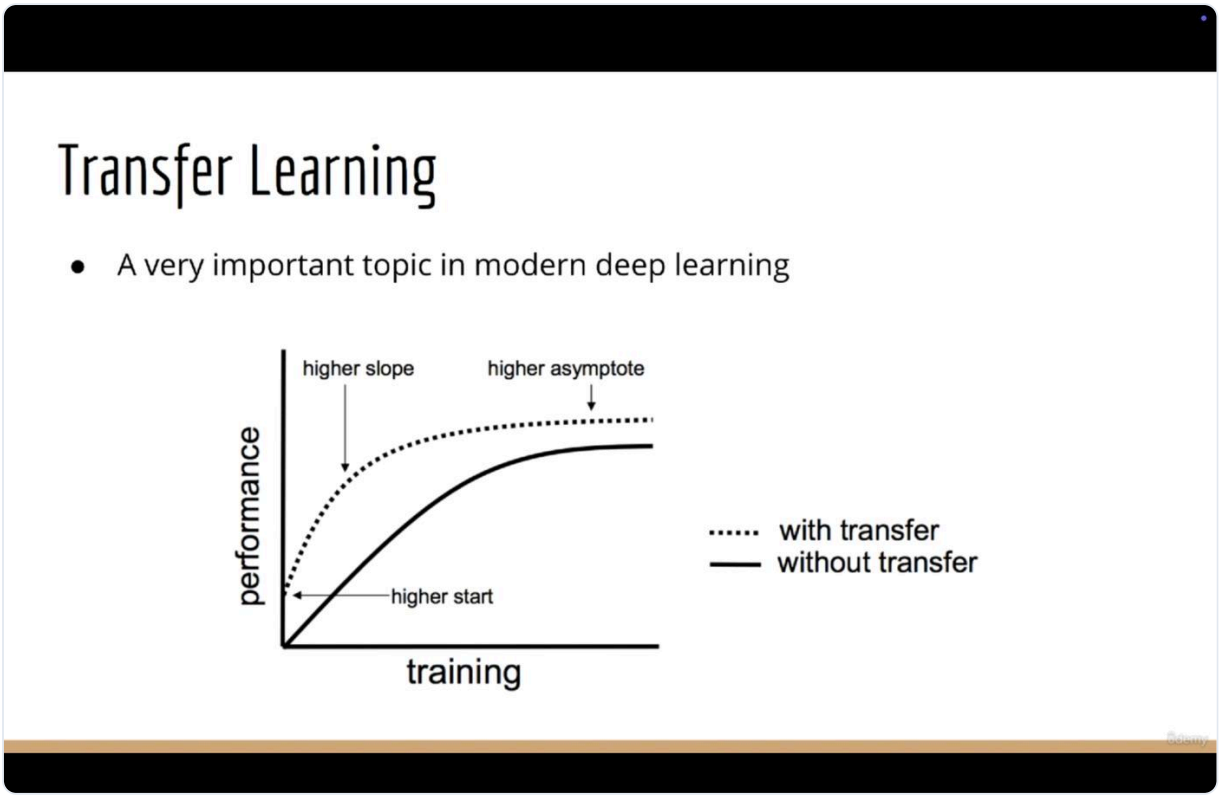


FIGURE 41.1 | Transfer Learning

The nice thing about transfer learning is it allows you to obtain very good off often the state of the art results with less effort and less resources spent the intuition behind transfer learning is this. Let's start with the basic observation that in a CNN the features that are learned at each layer are hierarchical. In the beginning layers the CNN only learned some basic lines and strokes in the later layers.

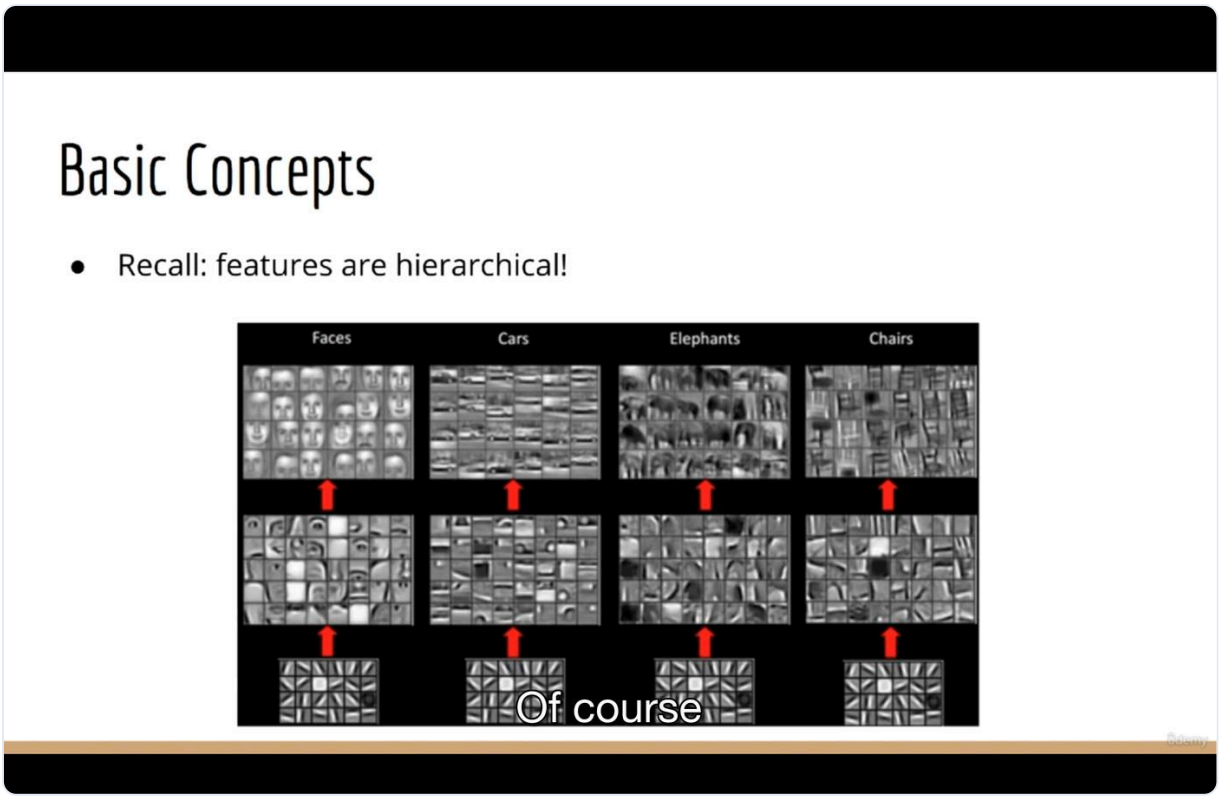


FIGURE 41.2

The features get progressively more complex from lines and strokes to parts of faces eventually to full faces. This is of course if you train your CNN on images of faces. You can also train your CNN on other types of images as well. Of course but here's another basic observation.

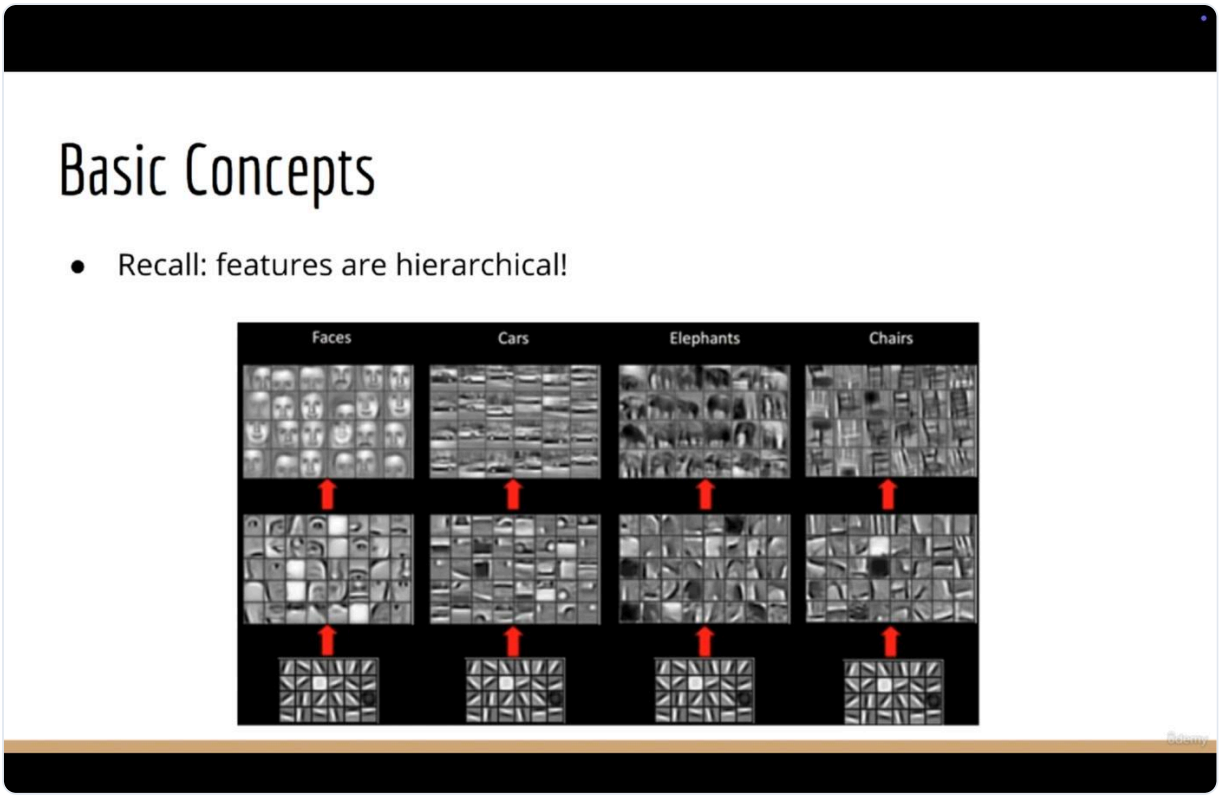


FIGURE 41.3

We recognize that the initial layers learn to recognize just simple lines and strokes is a plausible that other task that don't involve looking at images of faces would still find a line and stroke features useful. The answer is yes because probably all pictures will have lines and strokes. The difference between a car and a face being that the way that you put together those lines and strokes will be different in this way.

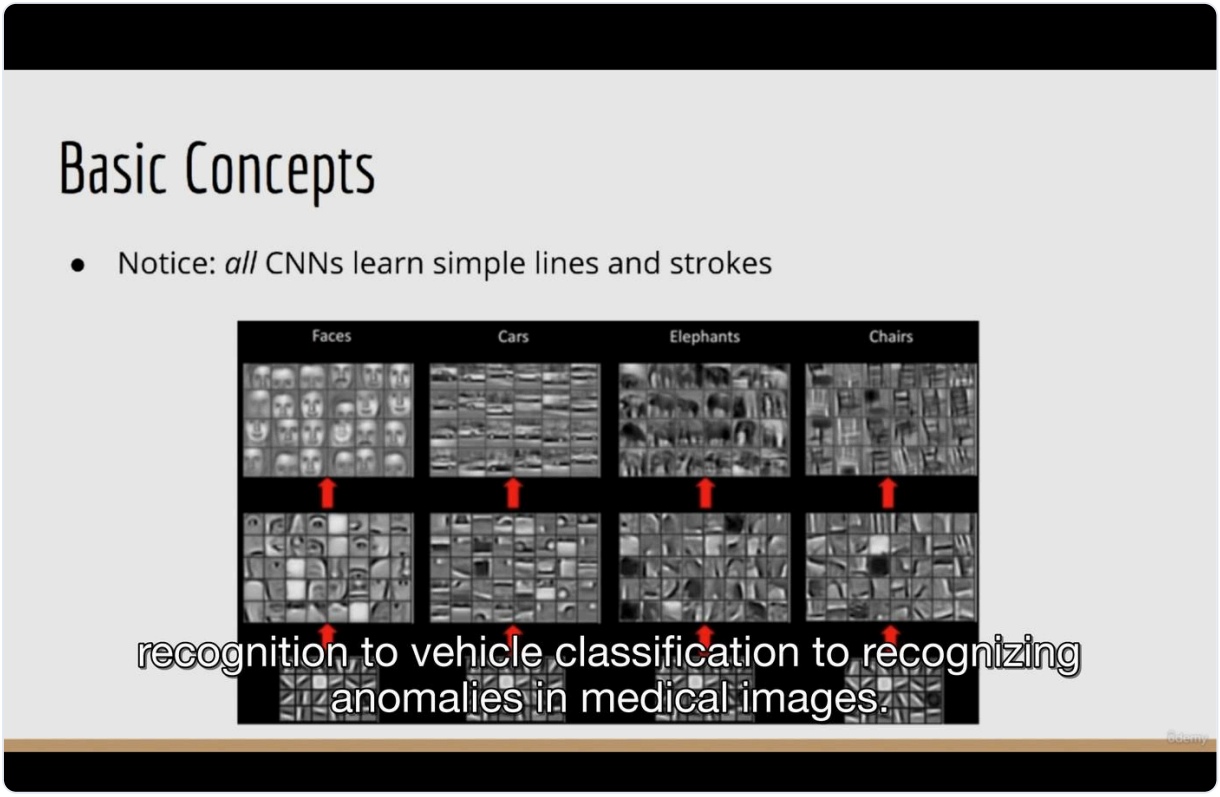


FIGURE 41.4

We can imagine that the first few layers of a CNN might be useful for a wide variety of tasks from facial recognition to vehicle classification to recognizing anomalies in medical images. This is the basic idea behind transfer learning.

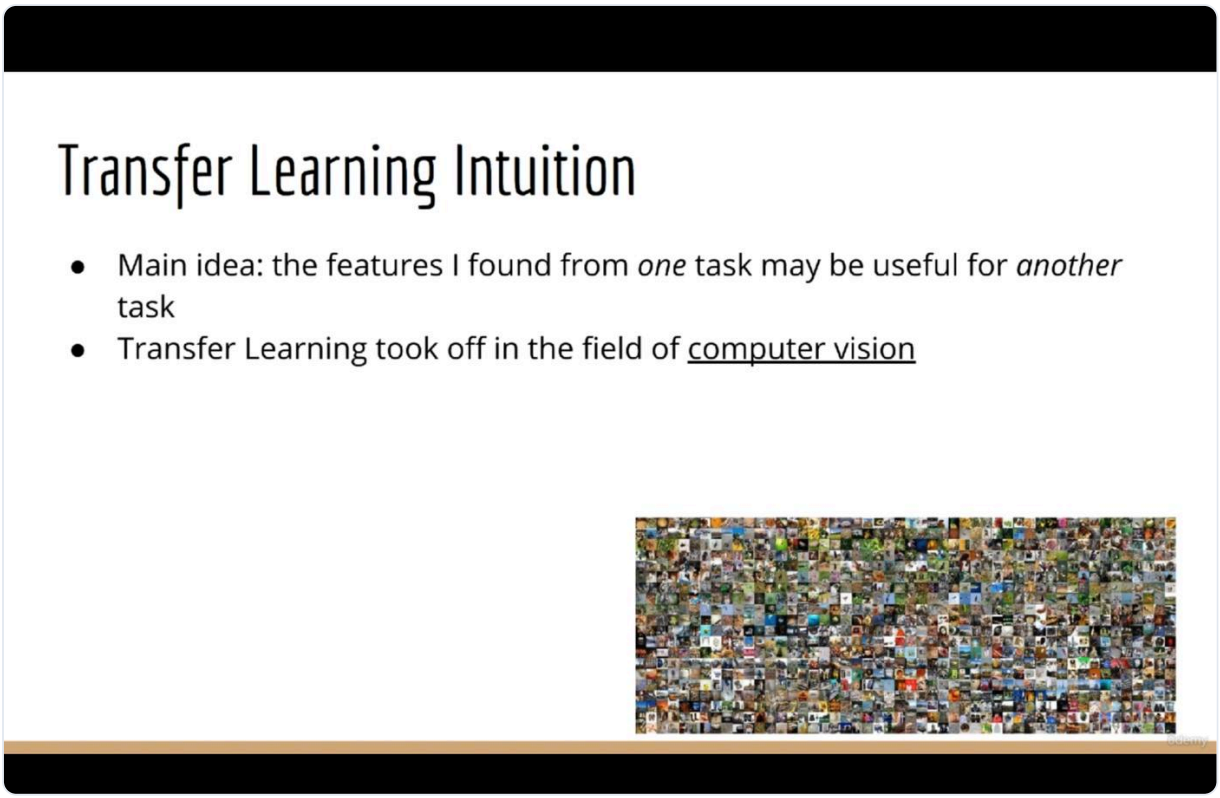


FIGURE 41.5 | Transfer Learning Intuition

It's the idea that the features that I found for one task could be useful for some other task transfer learning really took off in the field of computer vision so that's what we were going to focus on in this course. Probably the most widely known large scale image dataset is called ImageNet. This is a dataset that contains millions of images and 1000 categories meaning it can recognize 1000 different objects. Every year there is a contest to see who can build the best image classifier on this dataset.

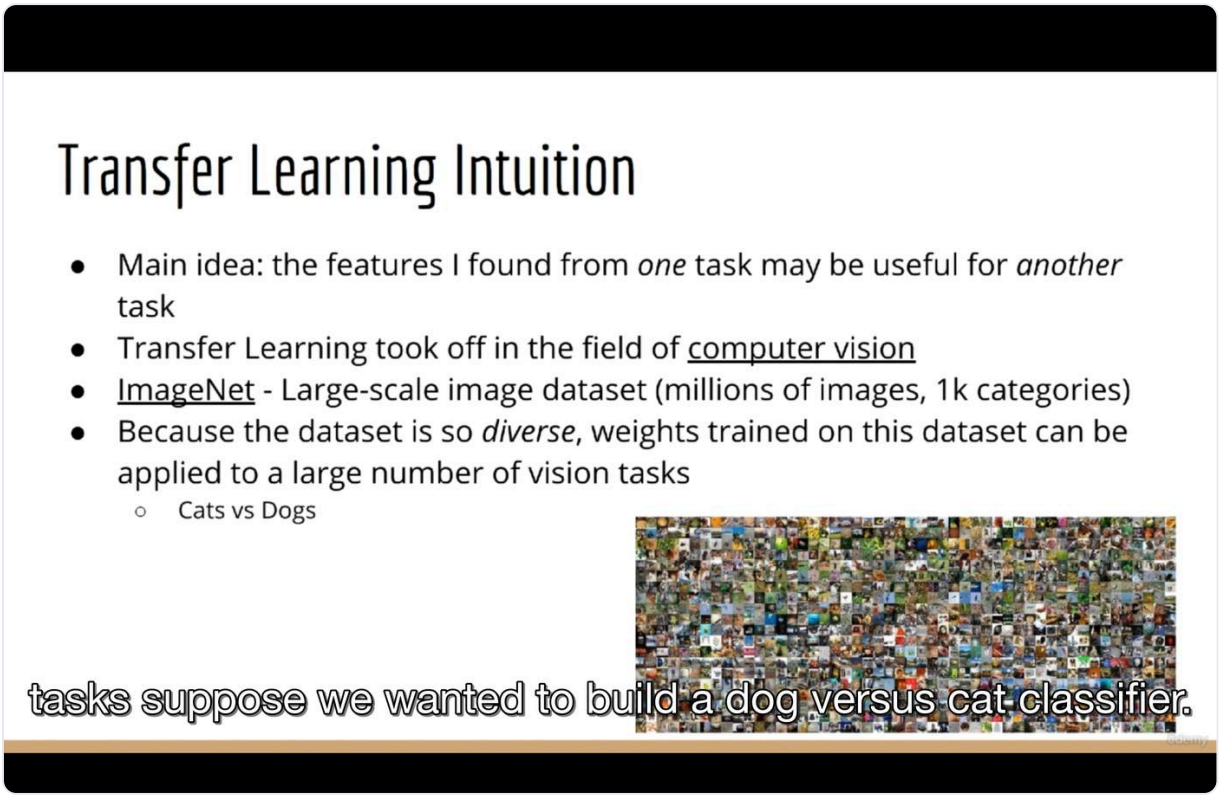


FIGURE 41.6 | Transfer Learning Intuition

The nice thing about the ImageNet dataset is because it's such a large dataset containing so many different kinds of images we can transfer the features learned from this dataset to a wide variety of tasks suppose we wanted to build a dog versus cat classifier.

FIGURE 41.7

That's easy because ImageNet already knows how to classify dogs and cats. Suppose we wanted to build a car versus truck classifier.

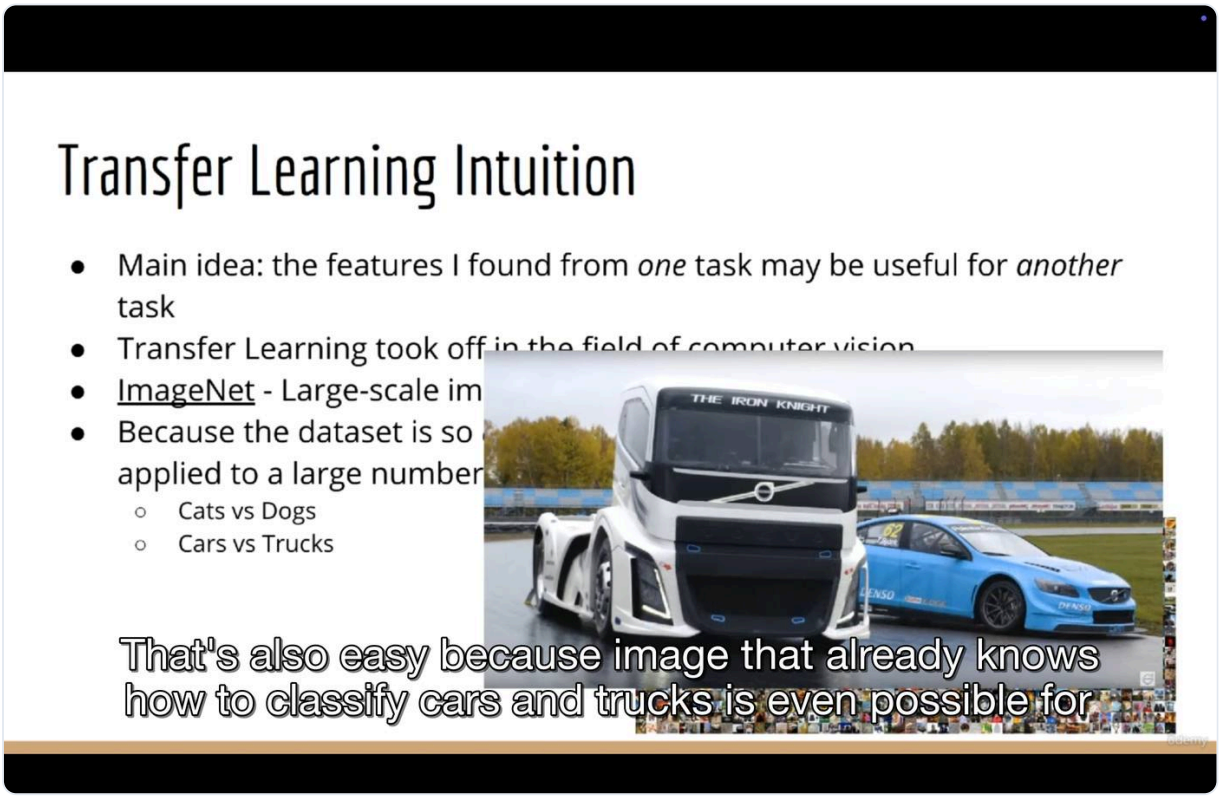


FIGURE 41.8 | Transfer Learning Intuition

That's also easy because ImageNet already knows how to classify cars and trucks is even possible for us to use the same features for things ImageNet has not seen before like say images from a microscope all right.

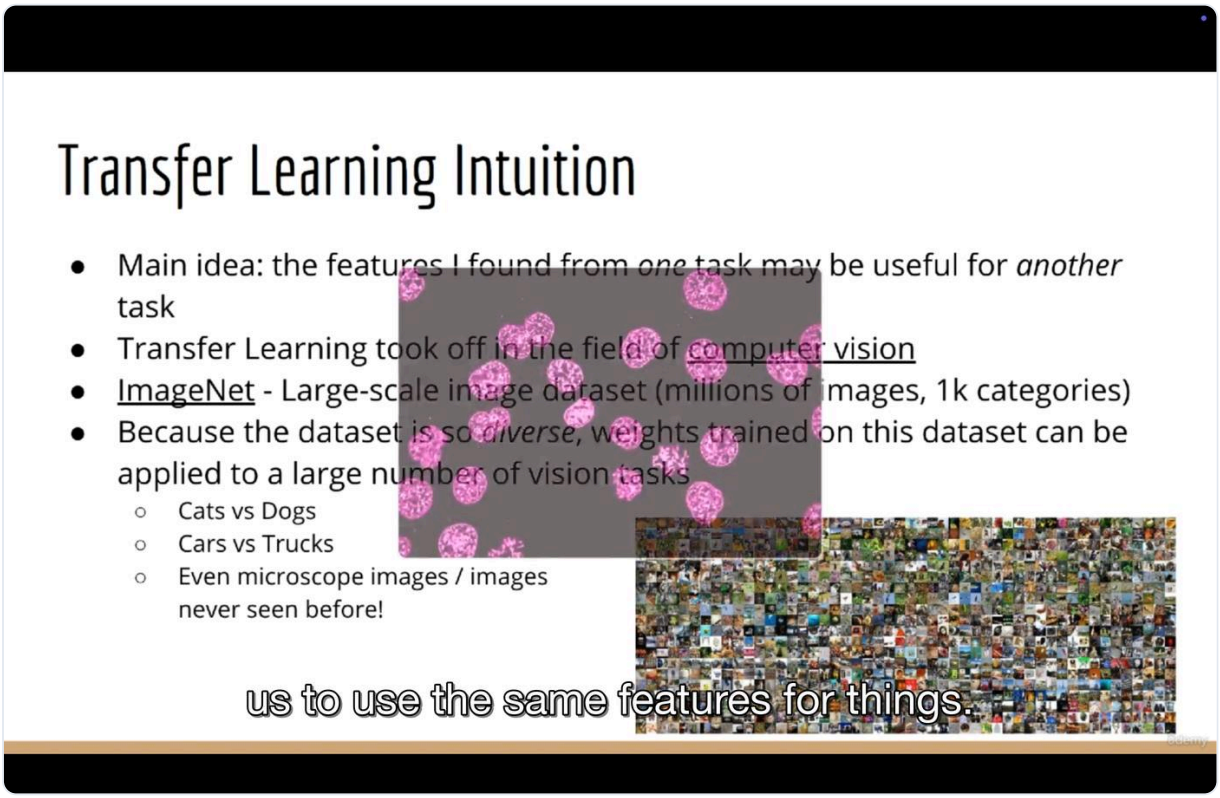


FIGURE 41.9 | Transfer Learning Intuition

So that's just the high level intuition.

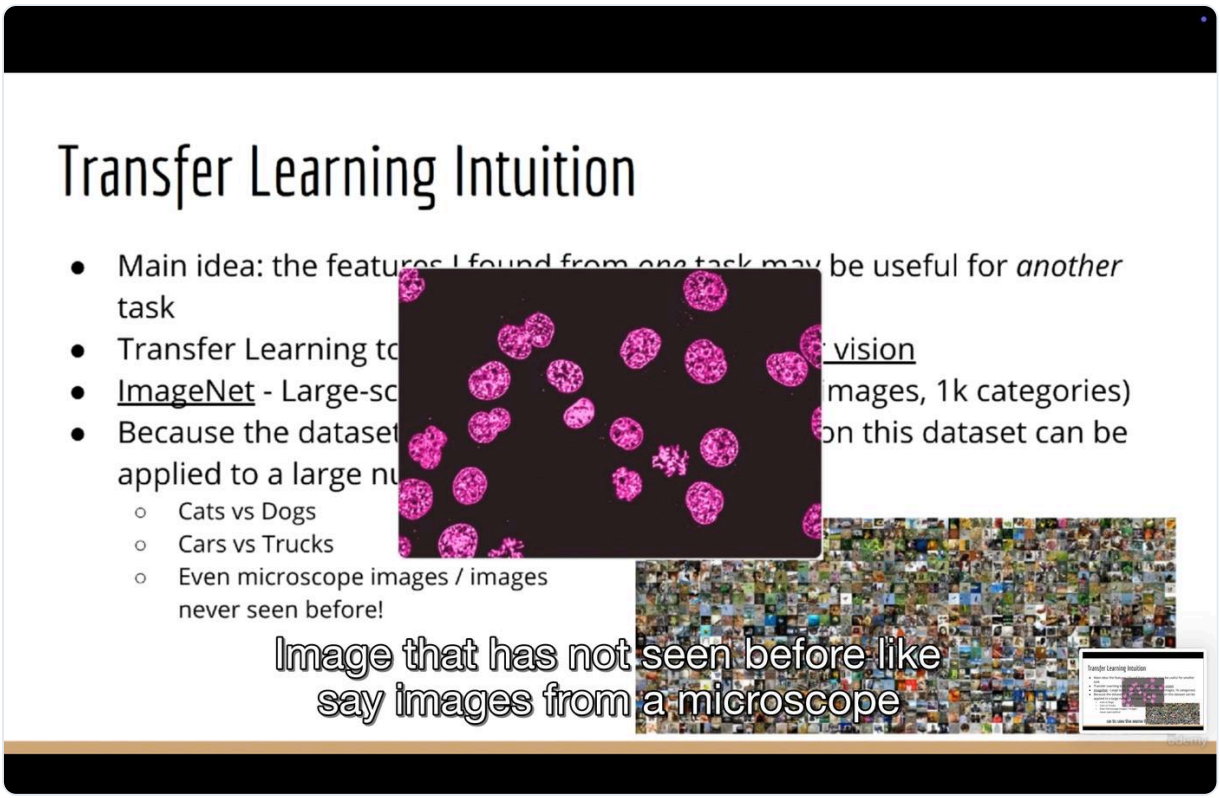


FIGURE 41.10 | Transfer Learning Intuition

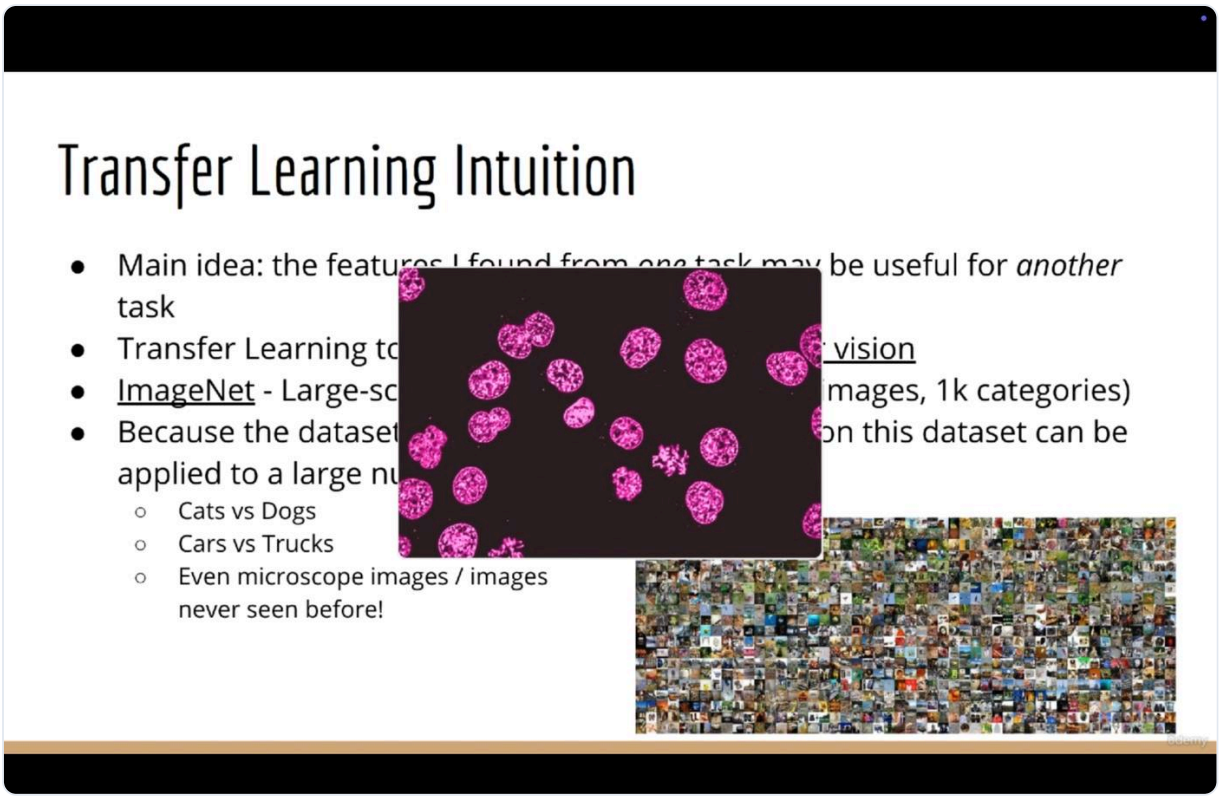


FIGURE 41.11 | Transfer Learning Intuition

Let's go deeper and discuss how this will actually work conceptually.

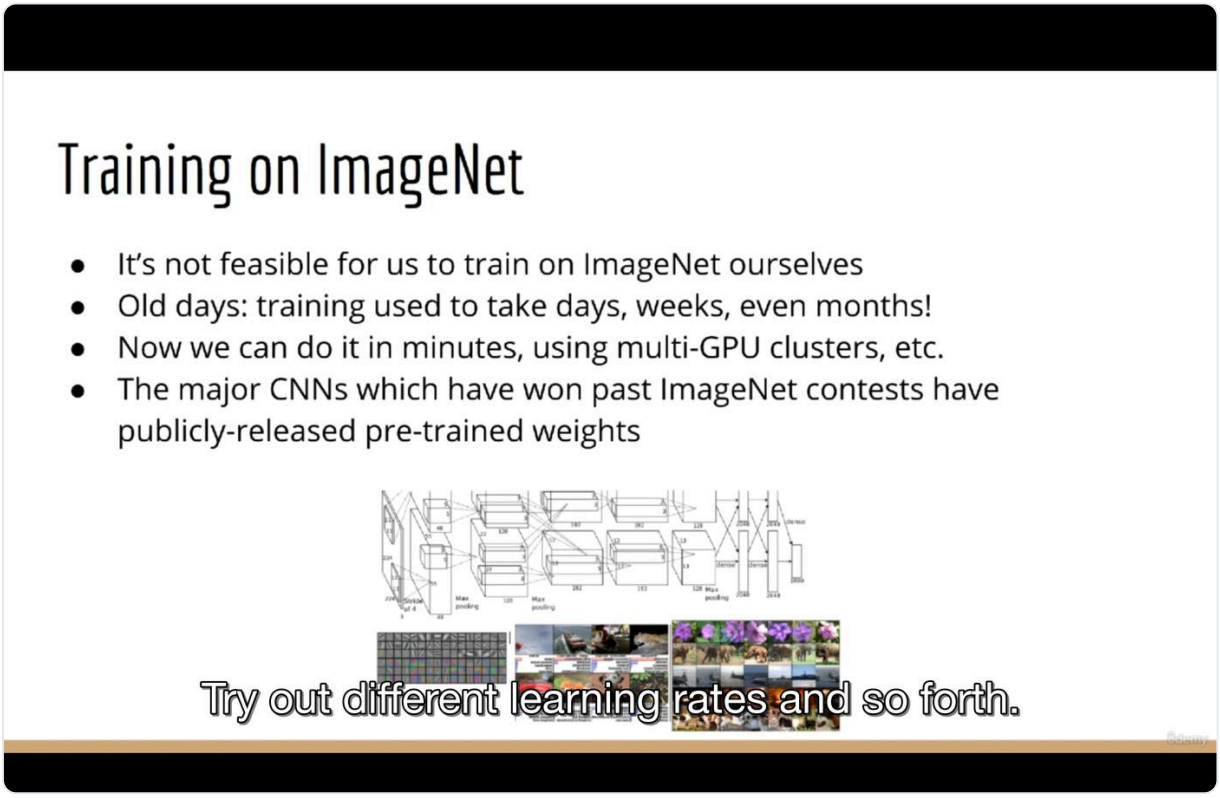


FIGURE 41.12 | Training on ImageNet

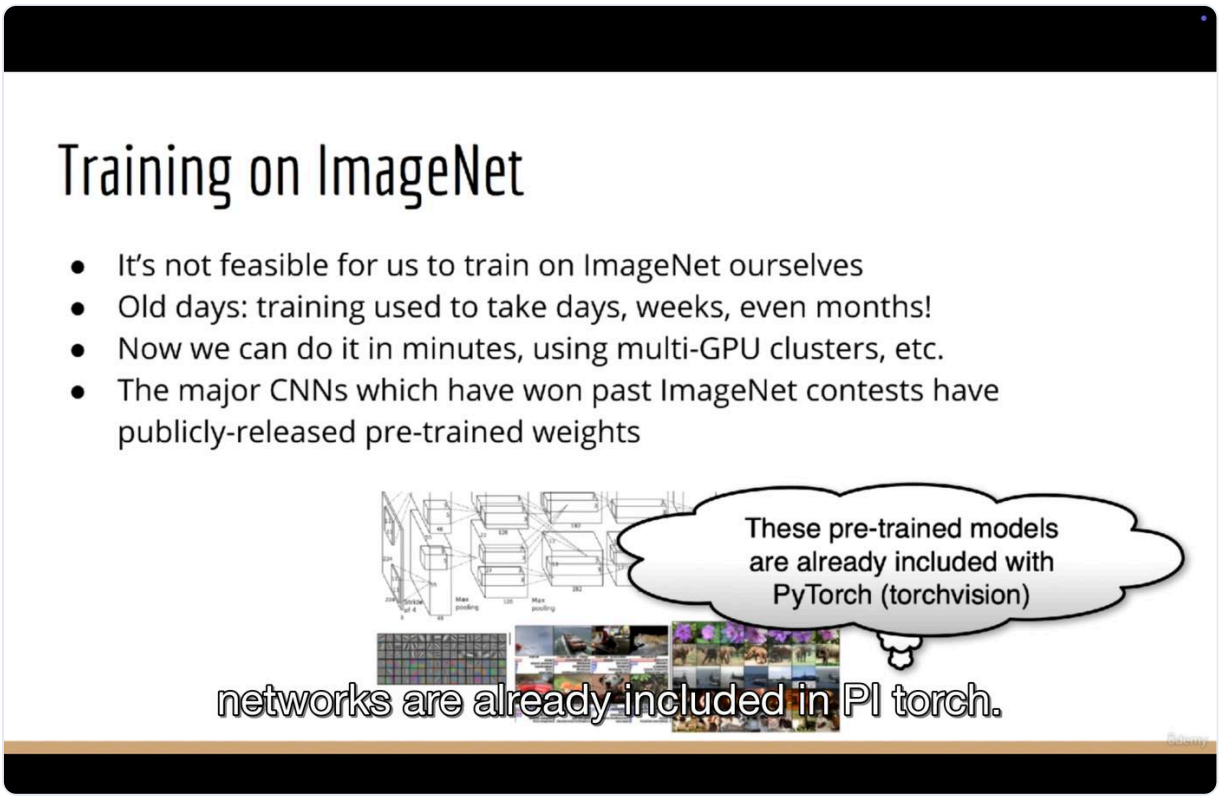


FIGURE 41.13 | Training on ImageNet

First we recognize that it's not feasible for us to train a CNN on ImageNet ourselves although it's a lot less expensive and time consuming these days compared to the earlier days of deep learning in the old days it was common for CNN training to take on the order of days weeks or even months. Nowadays we have distributed training multi GPU clusters and so forth but at the same time it's still not really something you want to do on your own. Luckily many of the major CNN architectures that have won the ImageNet contest in the past have released their pre trained CNNs to the public so there's no need for you to do any hyperparameter tuning. Try out different learning rates and so forth.

All that has been done for you and the results are open source doubly awesome is that these pre train networks are already included in PyTorch. We'll discuss what they are and how to get them later let's say you found a super powerful state of the art CNN trained on the ImageNet data set and now you want to use it to do transfer learning for your specific task.

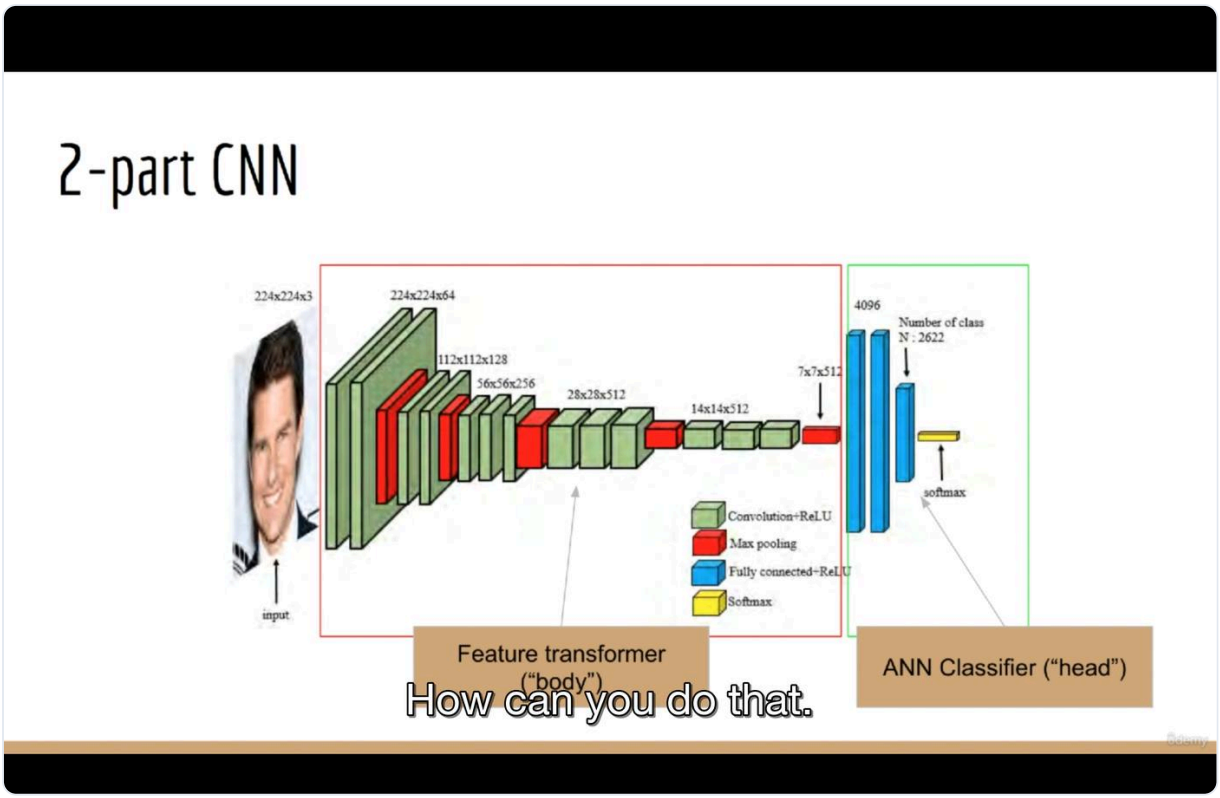


FIGURE 41.14 | Feature Transformer

How can you do that. Well let's start with the picture. This is a picture of a typical CNN. It's a series of convolutions and poolings followed by a series of dense layers. One perspective on the CNN is that it's made up of two parts part one which involves convolutions and pooling makeup a feature extractor or a feature Transformer. It takes your raw image and converts it into a feature vector which is a useful representation for ANNs and other machine learning models. Part 2 is just a feedforward neural network or ANN which we learned about earlier in this course.

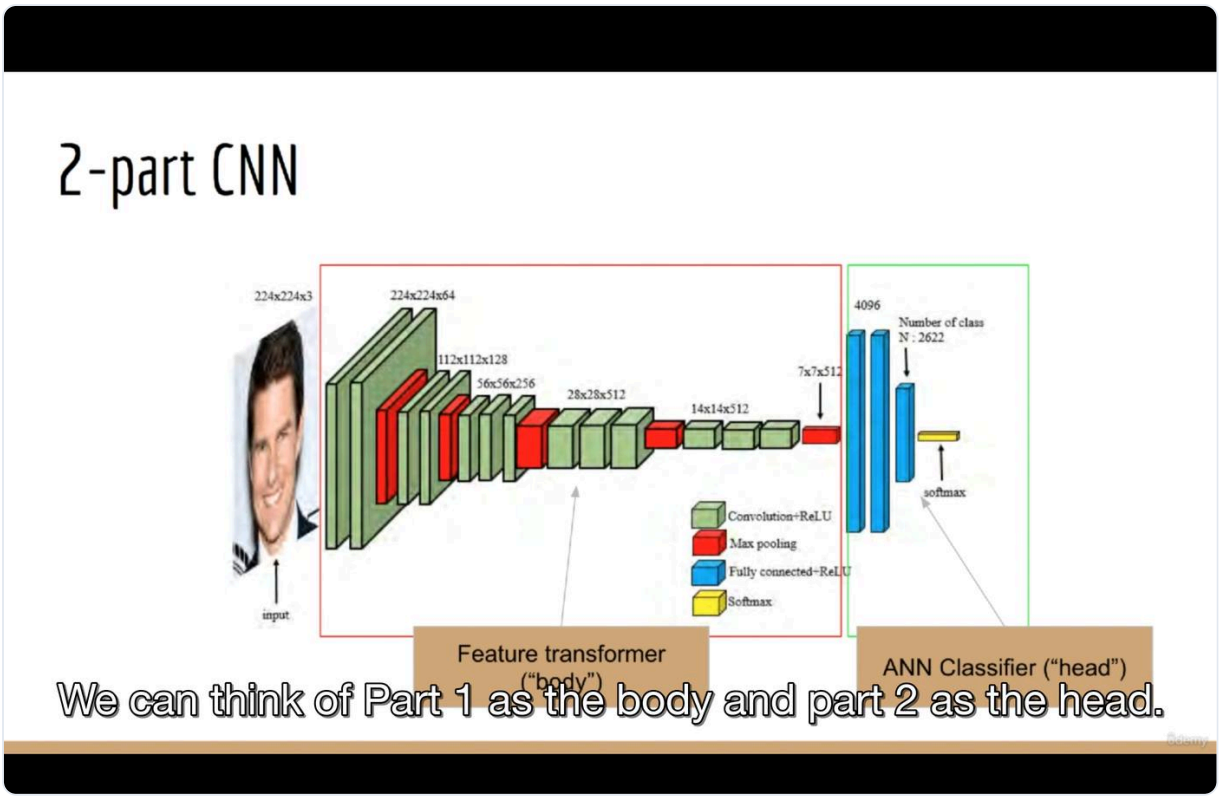


FIGURE 41.15

We can think of Part 1 as the body and part 2 as the head. So here's how transfer learning works. What we do is we chop off the head keep the body which is a useful feature transformer and then attach our own head. This head can be used for whatever task we want to perform for example identifying cars and trucks.

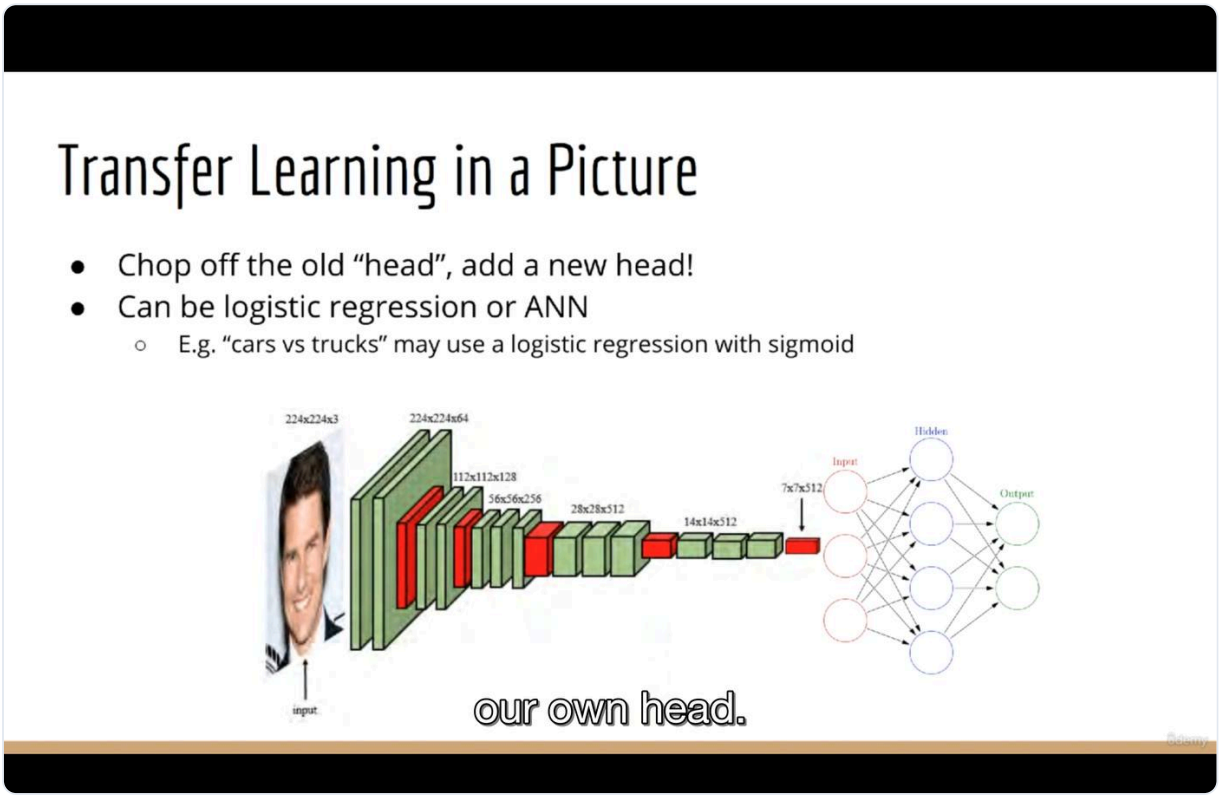


FIGURE 41.16 | Transfer Learning in a Picture

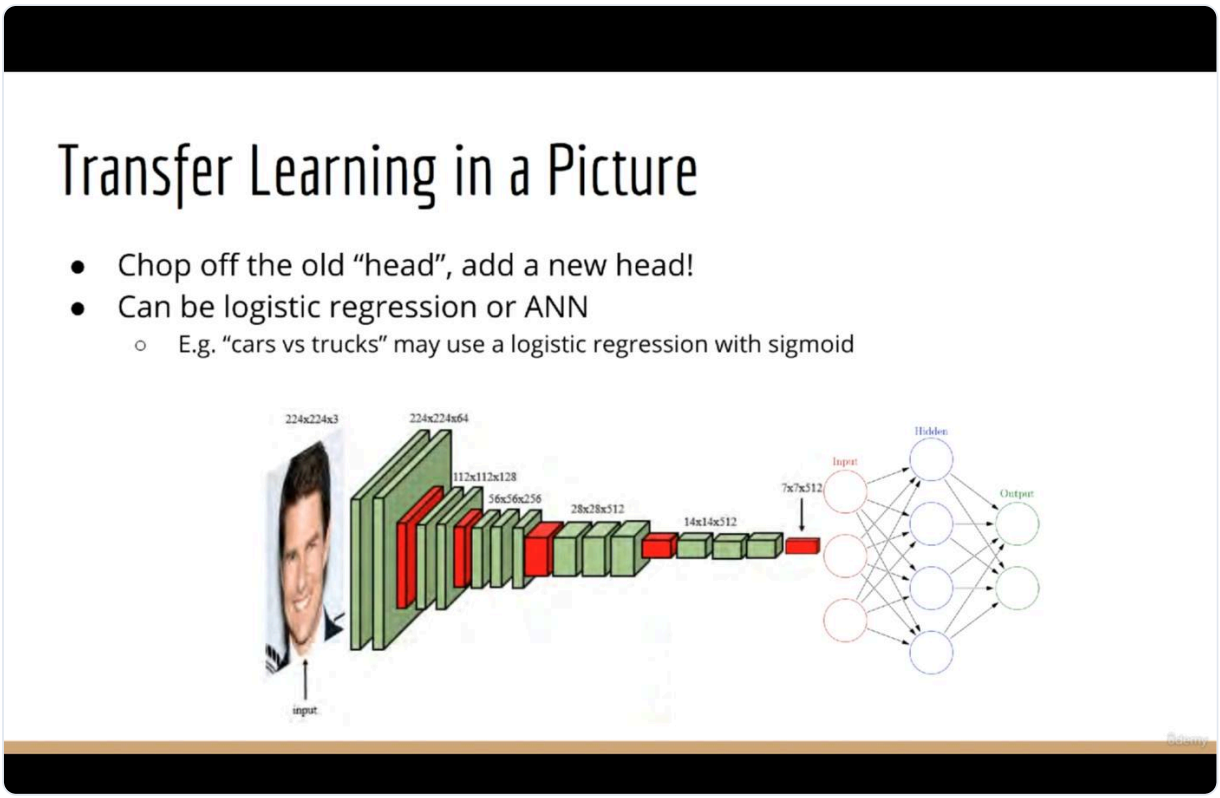


FIGURE 41.17 | Transfer Learning in a Picture

In that case we might use a logistic regression with a sigmoid output since as you recall the sigmoid activation is the appropriate activation function for binary classification optionally you could add more layers to make it a feedforward ANN then here's another important detail about transfer learning. There are a few approaches to this and I'm going to teach you about the most important and the most basic version. The idea is we're going to freeze the weights of the body and only train the head. In other words only train the logistic regression layer. You can imagine that if we use that gigantic pre trained CNN with 100 layers it would train very slowly because we would have to update all the weights of a 100 layers CNN.

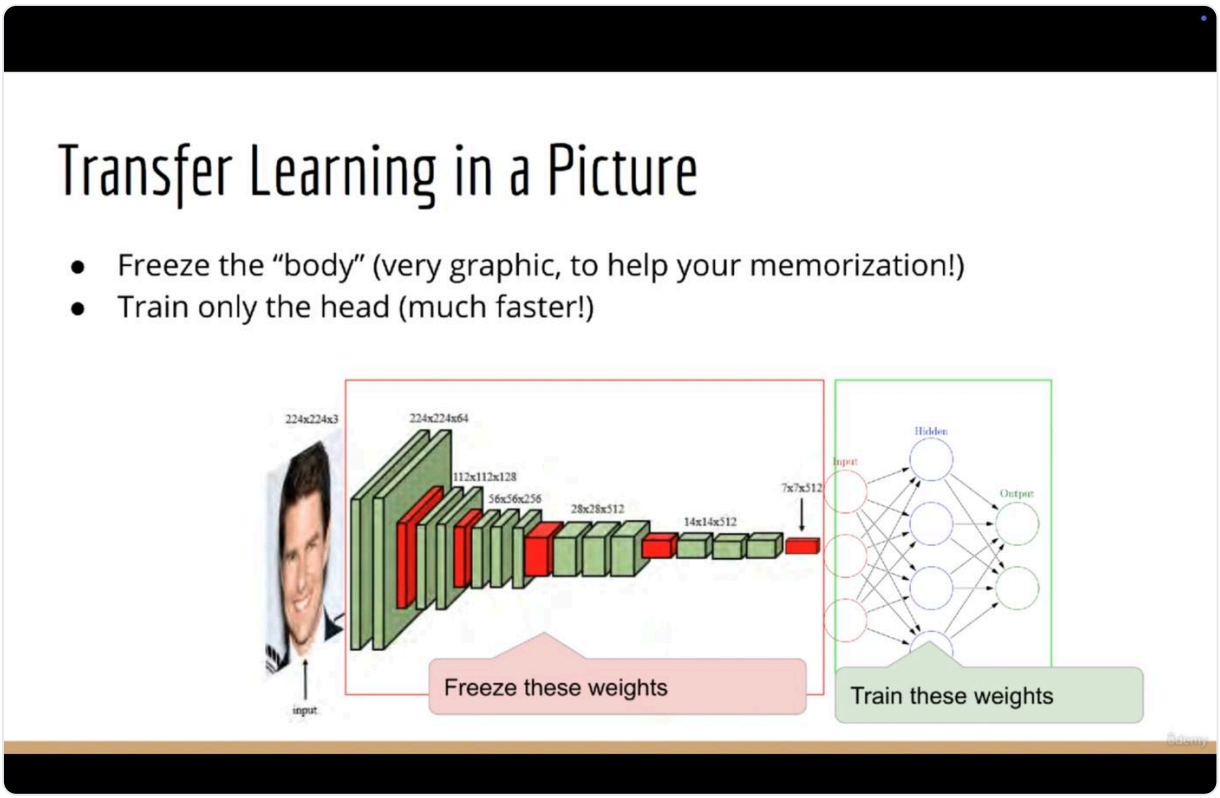


FIGURE 41.18 | Transfer Learning in a Picture

But by freezing the body we can simply ignore those weights and only train the weights of our own linear model. The important point is we don't need to train the convolution layers because they have already been trained on a data set much larger than anything we could have gathered ourselves. The weights we already have for those convolutions are very good. There is no need to try to make them better.

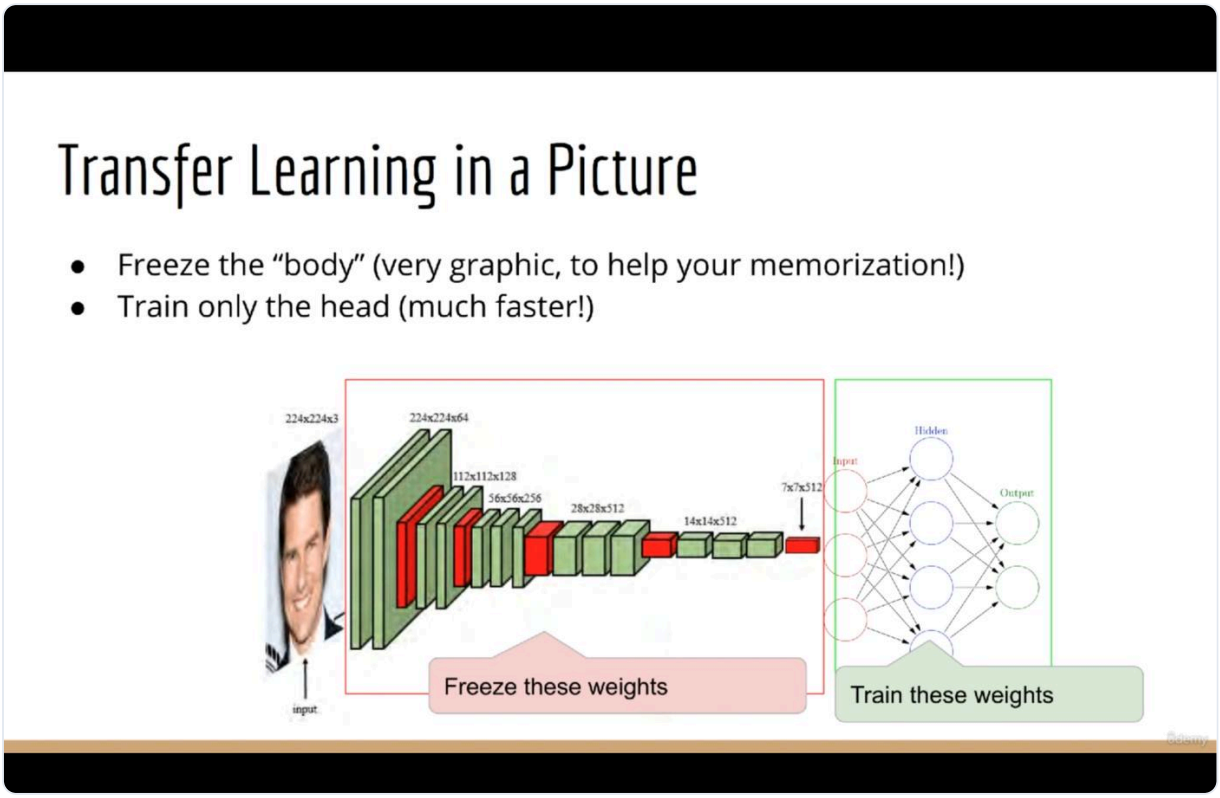


FIGURE 41.19 | Transfer Learning in a Picture

Instead we can assume that they give us good features and worry about training the top one major advantage of using transfer learning is that you don't need a lot of data to build a good model. You might recognize this graph which basically says that the more data you have the better your deep learning model is. This is opposed to classic machine learning methods which tend to plateau after a certain point. So you might think in order to build a good a deep learning model you must have a lot of data which will in turn make your neural network take a long time to train but with transfer learning. This work has been done for us.

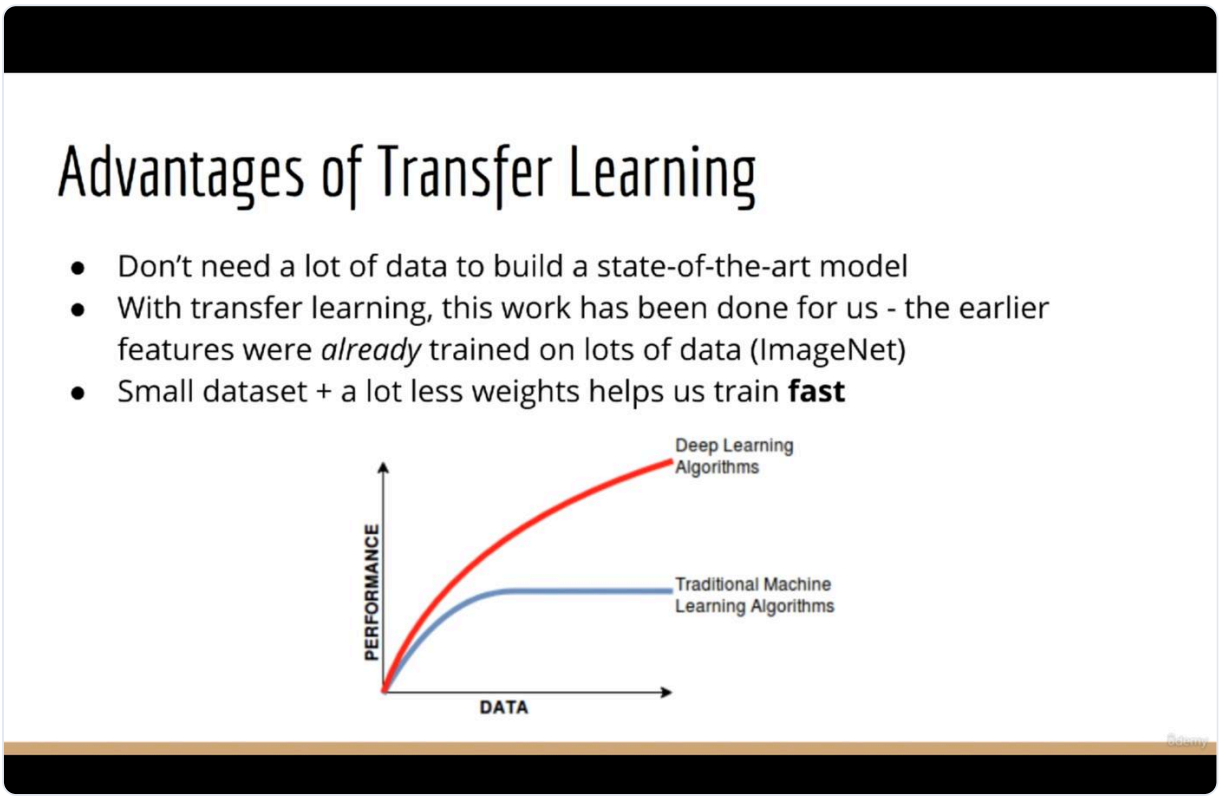


FIGURE 41.20 | Advantages of Transfer Learning

We don't need a lot of data because the earlier features we're already trained on a lot of data more data than we could have practically handled ourselves and so transfer learning allows us to get the benefits of deep learning without needing to have a ton of data ourselves. And on top of that it means we can train our model a lot faster. Not only are we just training a logistic regression but we have a smaller dataset compared to what we would need if we trained our model from scratch.

LECTURE 80

ResNet

Let's briefly discuss a few of the popular architectures that I use for transfer learning. The first and most popular is probably VGG which stands for Visual Geometry Group. This is just the name of the research team that created it. When you look at the architecture for VGG.

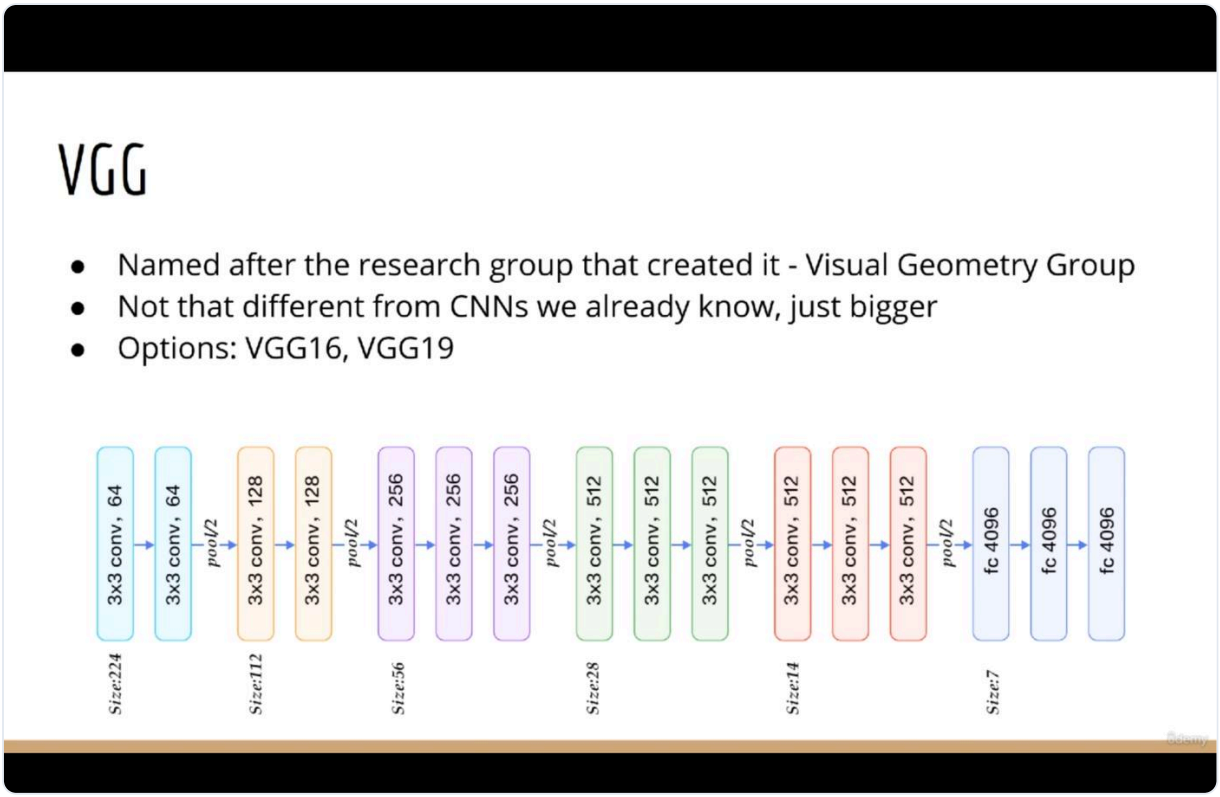


FIGURE 42.1

Although it got state of the art results at the time it's not that different from what we already know. It's just the usual CNN with an unusually large number of layers. At least it was a large number of layers compared to other neural networks. At the time it was created. So here's how it looks. First we have two convolutions followed by pooling. We repeat that once then we have three convolutions followed by pooling and we repeat that block three times. Finally we have three fully connected or dense layers. As you can see it's pretty much the same thing as LeNet just bigger. We have a series of convolutions and pooling followed by a few dense layers.

Note that there are a few variations of the VGG network such as VGG16 and VGG19 which have 16 layers and 19 layers respectively. Another common choice is ResNet ResNet is an even larger network than VGG. We won't go into too much detail in this course since our objective is not to understand ResNet but just to know what our options are. The idea behind ResNet is that it's a neural network with branches each branch is responsible for learning something different.

ResNet

- A CNN with *branches* (one branch is the identity function, so the other learns the *residual*)
- Variations: ResNet50, ResNet101, ResNet152, ResNet_v2, ResNeXt

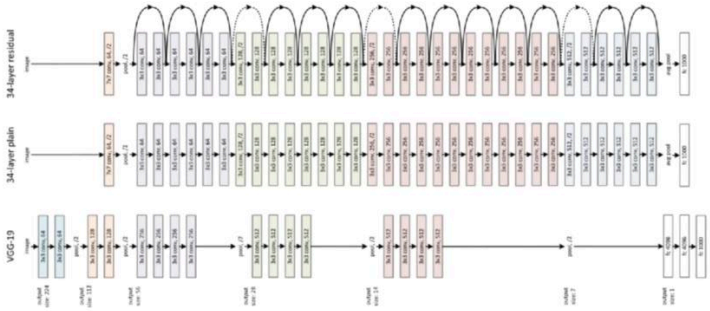


FIGURE 42.2

Inception

- Multiple convolutions in parallel branches
- Instead of trying to choose different filter sizes (1x1, 3x3, 5x5, etc.) just try them all!

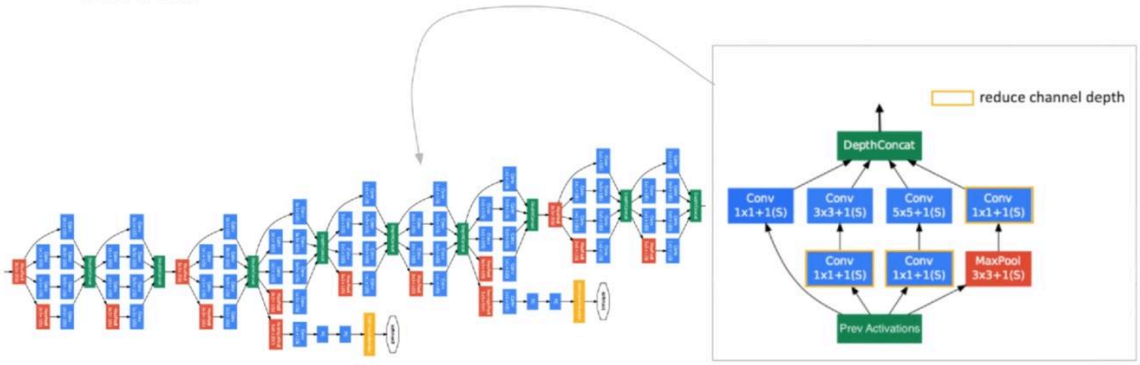


FIGURE 42.3

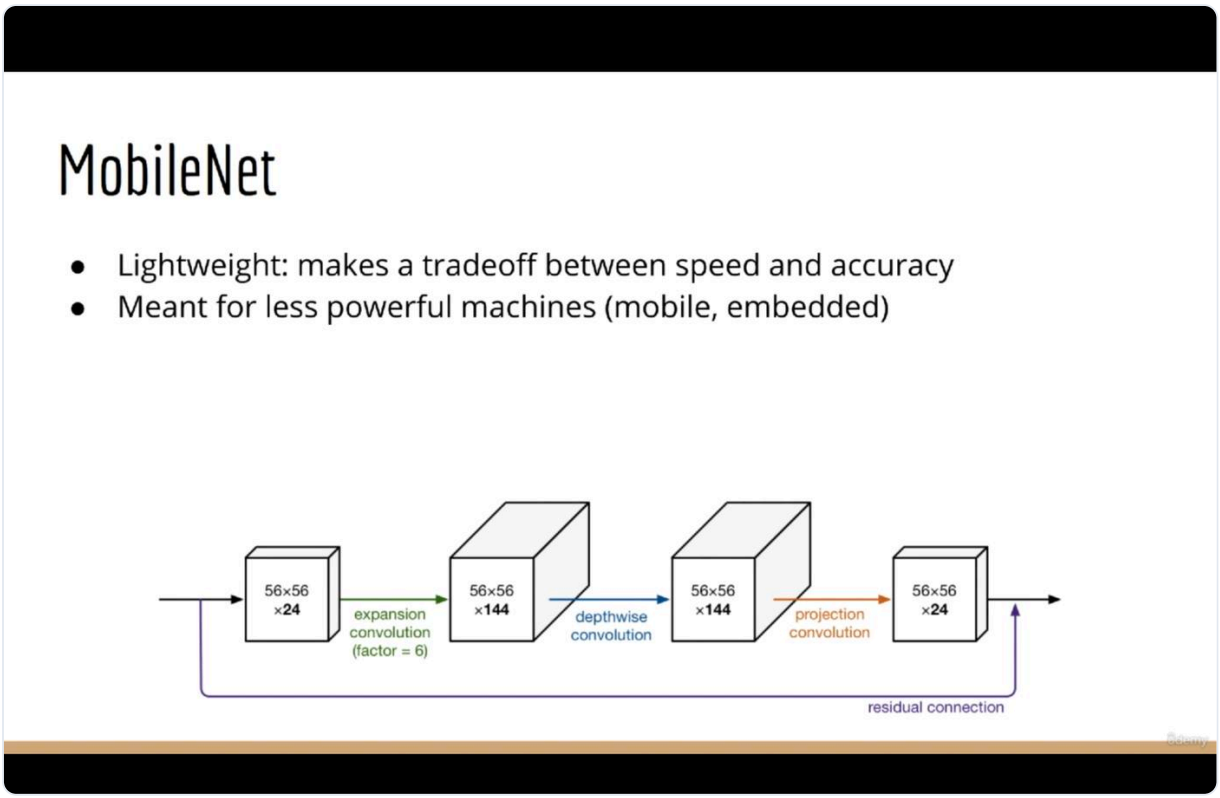


FIGURE 42.4

It turns out that one of the branches is just the identity function. So the other branch is responsible for learning the residual hence the name residual network like VGG ResNet also has different variations with different numbers of layers. There's ResNet 50 with 50 layers ResNet 101 with 101 layers and ResNet 152 which has 152 layers. There are also some ResNets with different architectures such as ResNet V2 and ResNeXt yet another common choice is inception. The layout of Inception is similar to ResNet in that you have multiple parallel branches but with Inception The concept is a little different with Inception instead of just a single convolution going into another convolution you'll do multiple convolutions in parallel.

You recall that one of the hyperparameters we have to choose is the filter size. Well inception says why not simply try them all and concatenate the result. So for example you'll have a one by one filter a three by three filter and a five by five filter and convolve the same input image by all these filters. Once you've done that you'll take the resulting images and stack them all together. The idea behind the inception network is that you'll have multiple layers of Inception blocks just like how the ResNet has multiple layers of residual learning blocks. Now I know a lot of people are going to ask Well which one should I choose VGG ResNet or Inception.

And the answer is as always you have to try it on your specific dataset with your specific task. There is no way to predict which one will work better for each use case another popular pre trained model used for transfer learning is called MobileNet which specializes in being lightweight. So this one actually does have a specific purpose because it's lightweight. There's a tradeoff made between a speed and accuracy. Generally speaking the ideal use case for MobileNet is if you want to use convolutional neural networks on less powerful machines such as mobile devices and embedded devices.

LECTURE 81

Large Image Datasets

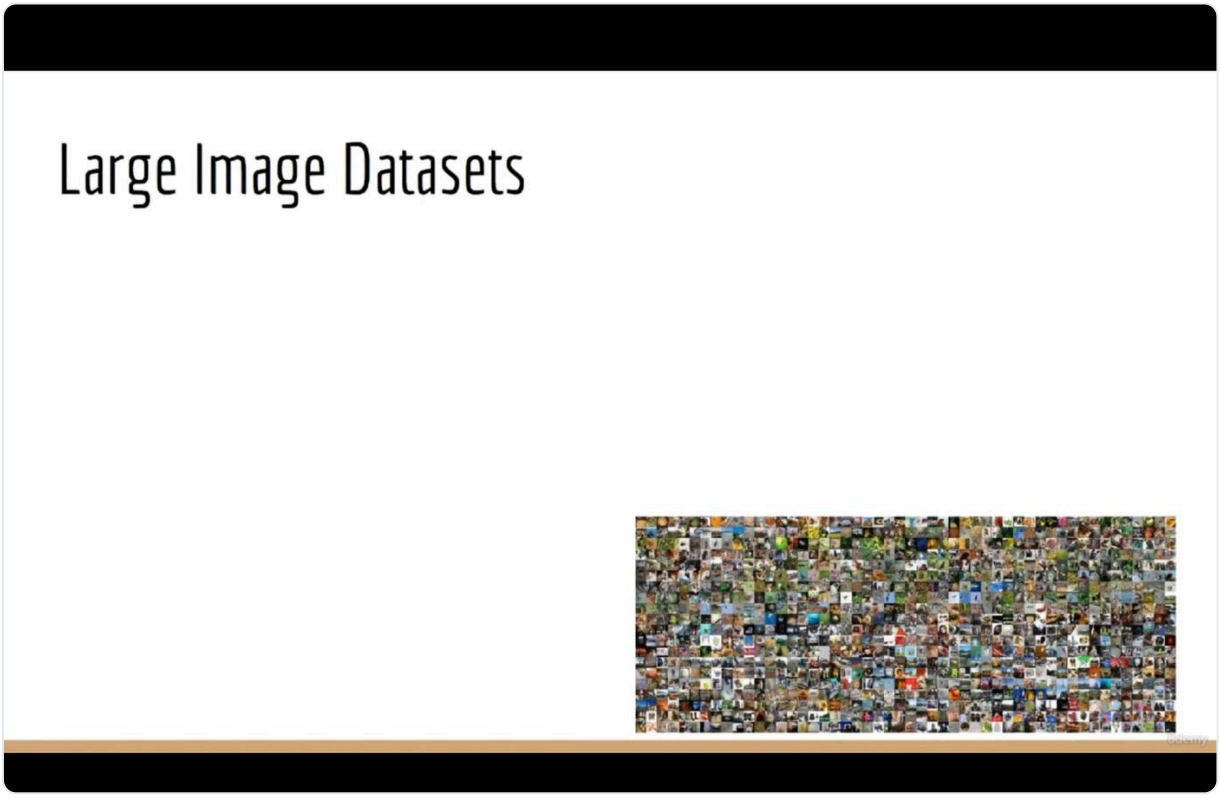


FIGURE 43.1 | Large Image Datasets

In this lecture we are going to discuss how to work with large data sets in PyTorch specifically large image data sets.

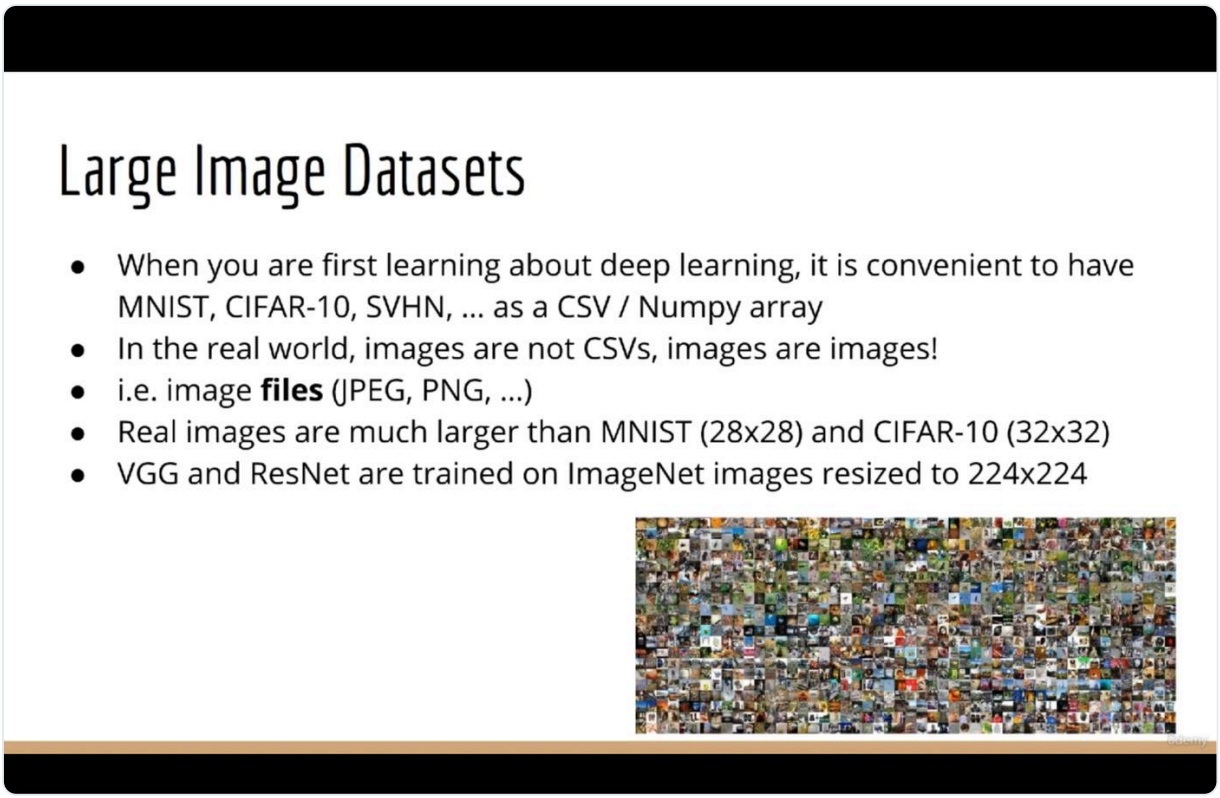


FIGURE 43.2 | Large Image Datasets

When you're first learning about deep learning it's convenient to have data sets like MNIST CIFAR-10 and SVHN preloaded into a CSV or a NumPy array. That way you can focus on the machine learning aspects and not the data processing aspects but in the real world images don't come in the form of CSVs instead images or images. In other words actual image files like JPEG PNG and so forth. These are files that sit on your computer. They are not organized in a nice NumPy array for you. On top of that images are usually much larger than what you see in MNIST or CIFAR-10 where you have a 28 by 28 image or a 32 by 32 image ImageNet models like VGG and ResNet are trained on images of size 224 by 224 which is an order of magnitude larger than MNIST as an exercise.

Think about how much space you would need to store one million images each of size 224 by 224. Please pause the video until you've calculated and answer all right.

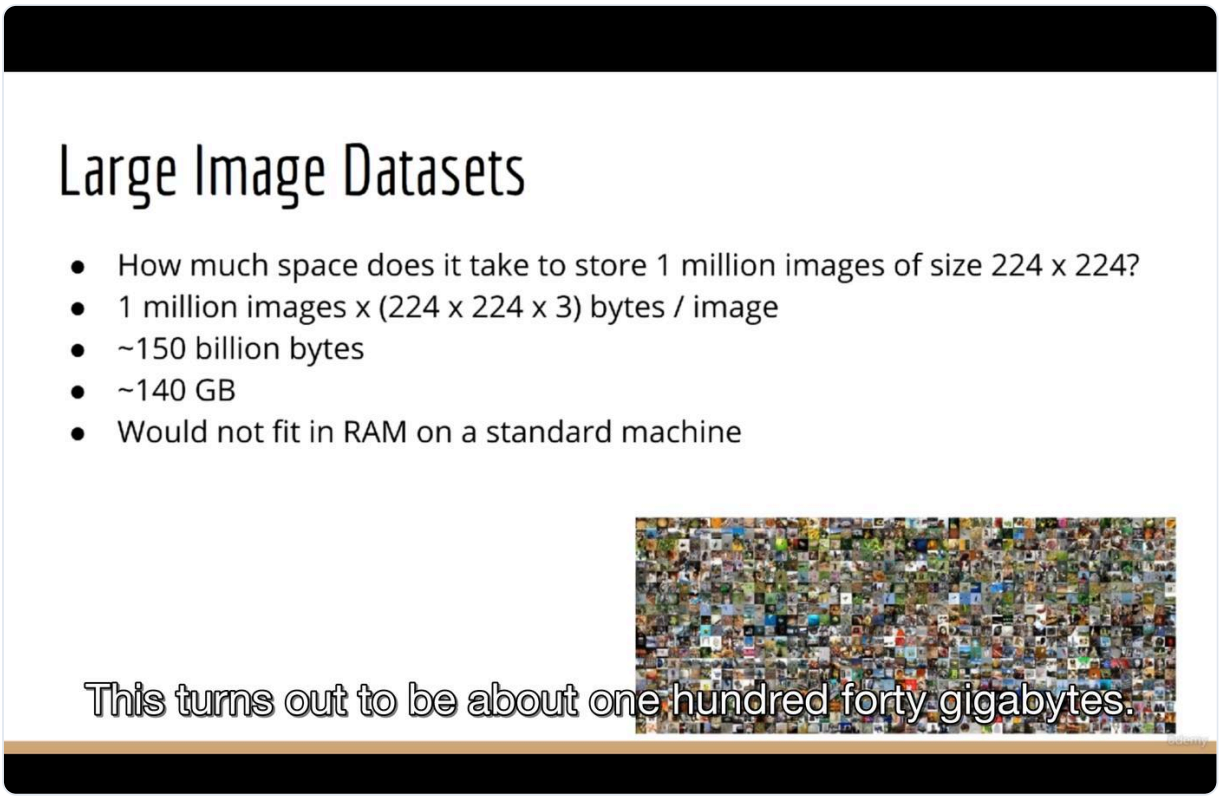


FIGURE 43.3 | Large Image Datasets

So hopefully you thought about how much space it would take to store one million images of size 224 by 224. Here's how much space it would take. First we have one million images. Multiply that by 224 by 224 by 3. The number of bytes per image. This is the total number of bytes. Our dataset would take up. This is about one hundred fifty billion bytes. This turns out to be about one hundred forty gigabytes. It should be clear that you cannot fit a dataset of this size into memory on a standard machine. So what should we do. As always. I don't like to give people code without having them first think about how to approach the problem themselves. So let's first take a moment to think about how to approach this problem.

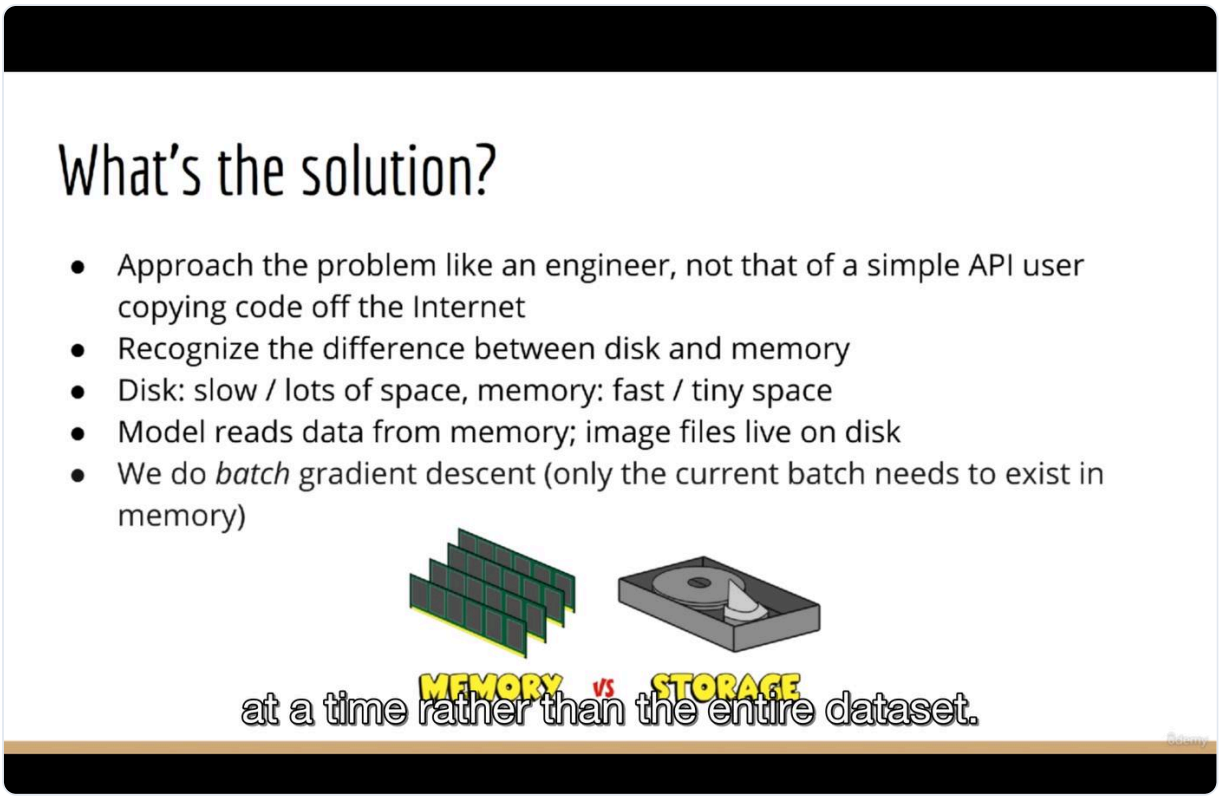


FIGURE 43.4 | Recognize the Difference Between Disk and Memory

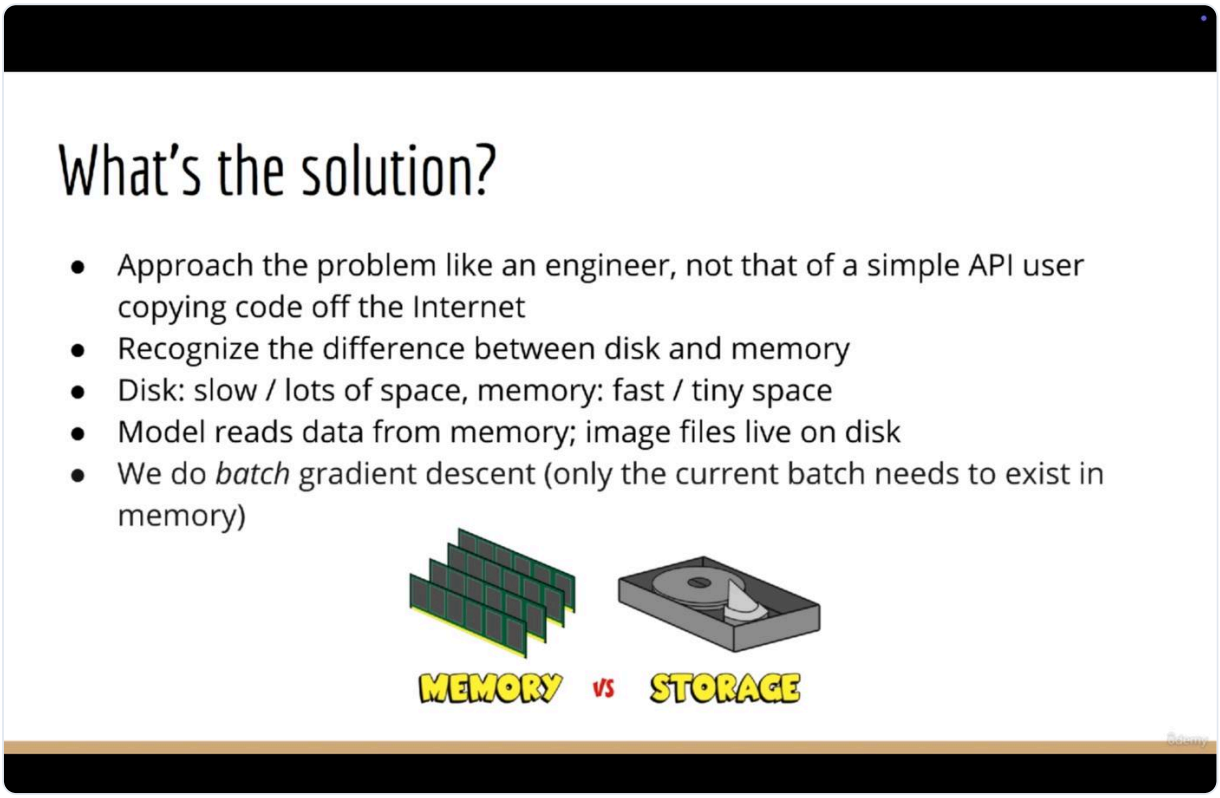


FIGURE 43.5 | Recognize the Difference Between Disk and Memory

As an engineer rather than as a simple API user a copying code off the Internet first recognize the difference between a disk and memory disk is generally slow and memory is generally fast. The tradeoff is that disk generally has lots of space. While memory has much less space your model reads data from memory but the images live on disk. Second recognize that our approach is Batch gradient descent where we only look at one batch of data at a time rather than the entire dataset. So technically only that

batch of data needs to exist in memory for the time that it's needed. In fact that's all we need to know. Let's suppose we have two arrays to represent our dataset. a list of file names where the images live and a list of target labels for those images.

Let's say our batch size is 32 in order to do a batch gradient descent. All we need to do is loop through each of the arrays 32 items at a time inside the loop. We load in those 32 images and store them in a NumPy array. Of course thirty two images is a totally feasible amount to store in memory.

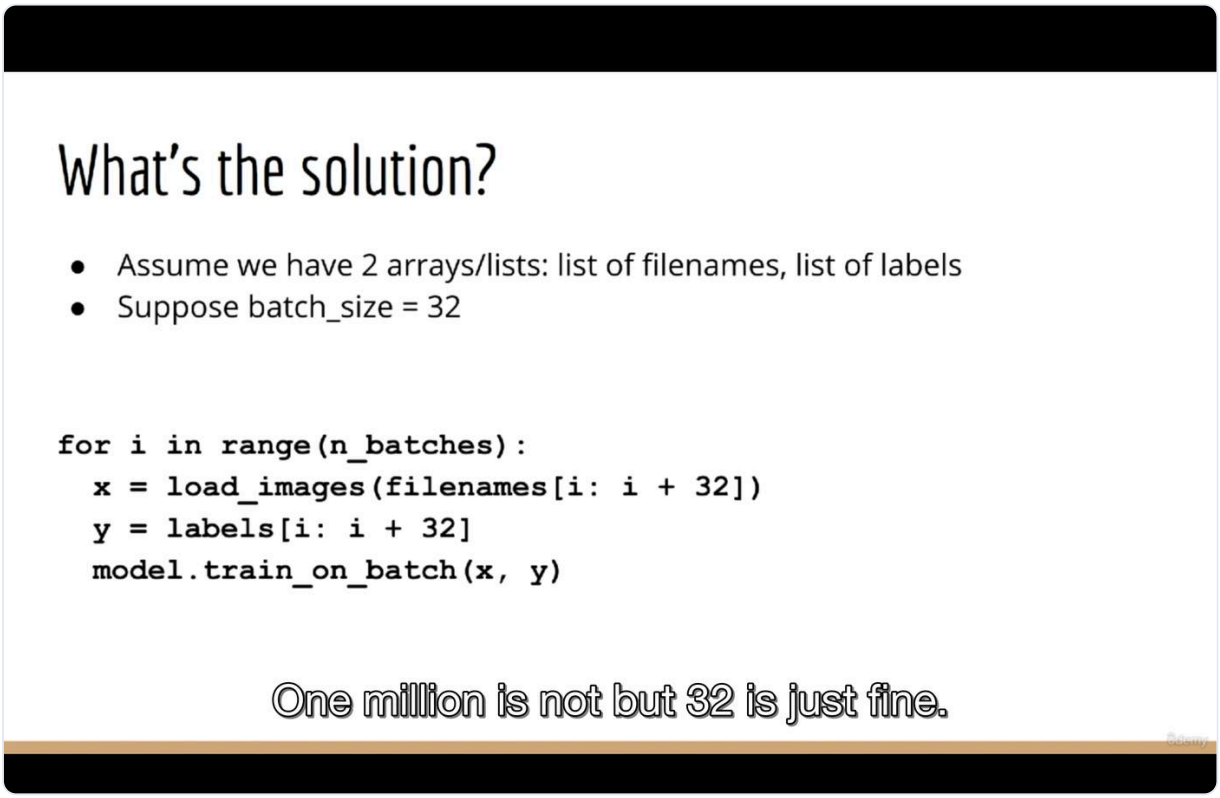


FIGURE 43.6

One million is not but 32 is just fine. Then we can call a function like model dot train on batch passing in X and Y which will do one iteration of gradient descent on this batch of data on the next iteration of the loop. We throw at the old data and reassign x and y to the next 32 images. Thus at any point in time only 32 images exist in memory. Once they are used that they are forgotten until the next epoch luckily in code although you could do this manually you don't have to. We're going to make use of a few special functions and classes that make it very easy for us. First I want to show you all the main ingredients and then I'll describe how to put them all together.

What you should recognize is that this is just like the data set and data loader combinations we've seen throughout the course previously the image data set objects we use before loaded in images into single arrays. These images were very tiny. So that was OK they could all live in memory at the same time in the same array. We had specialized data set objects for different datasets such as MNIST fashion MNIST and CIFAR.

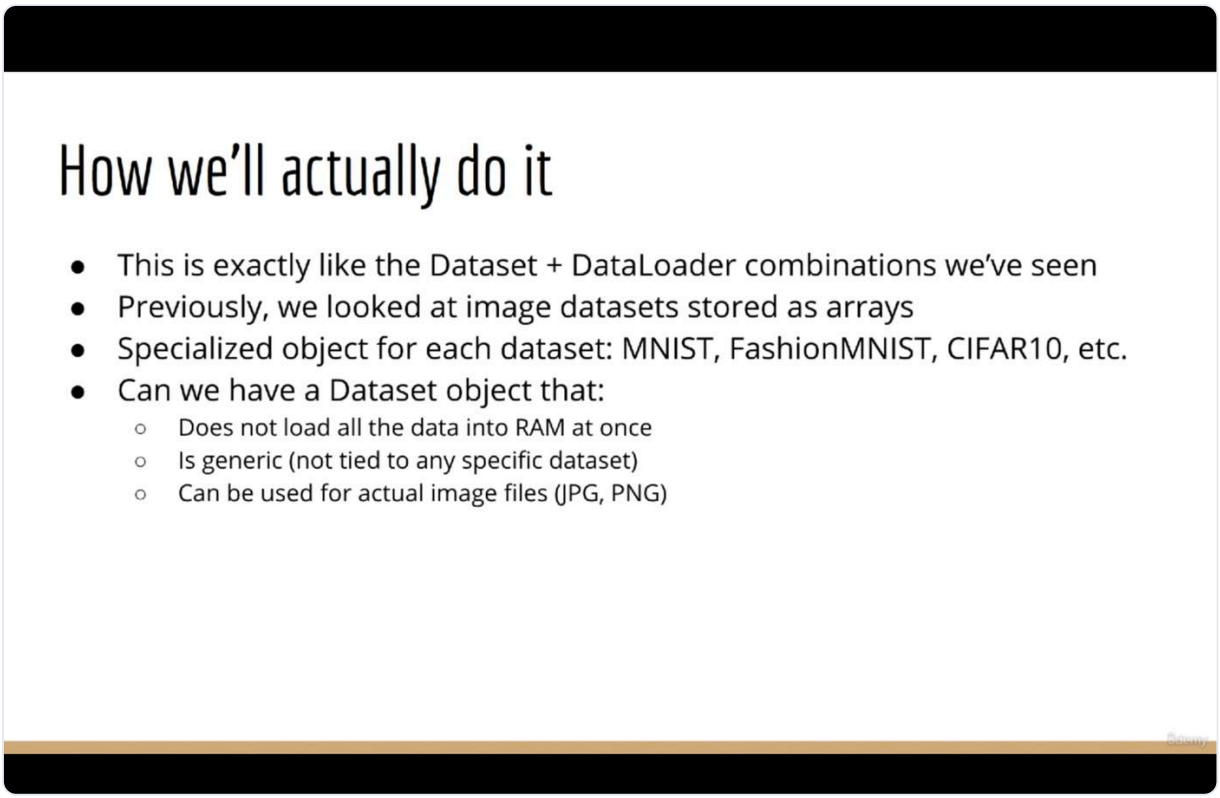


FIGURE 43.7 | Previously, We Looked at Image Datasets Stored as Arrays

So your question should be is there a kind of data set object that does not load all the data into memory All at once that is generic.

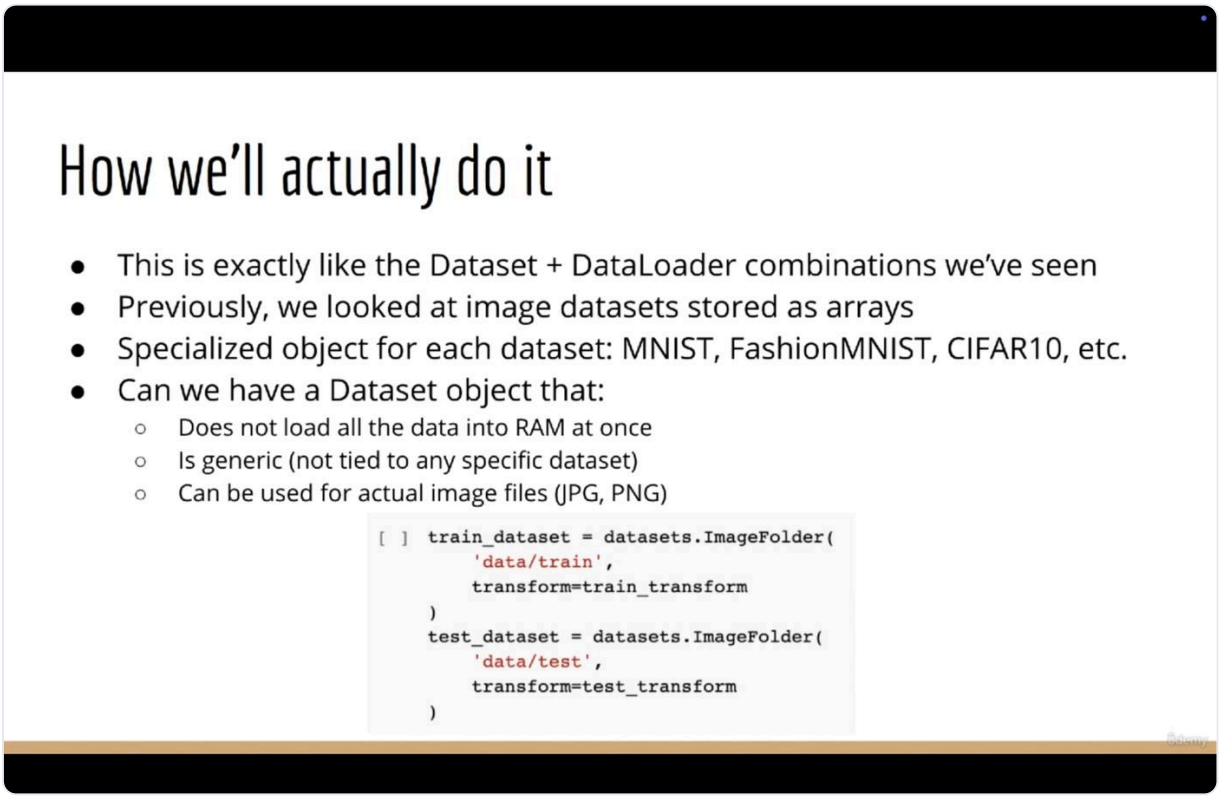


FIGURE 43.8 | Previously, We Looked at Image Datasets Stored as Arrays

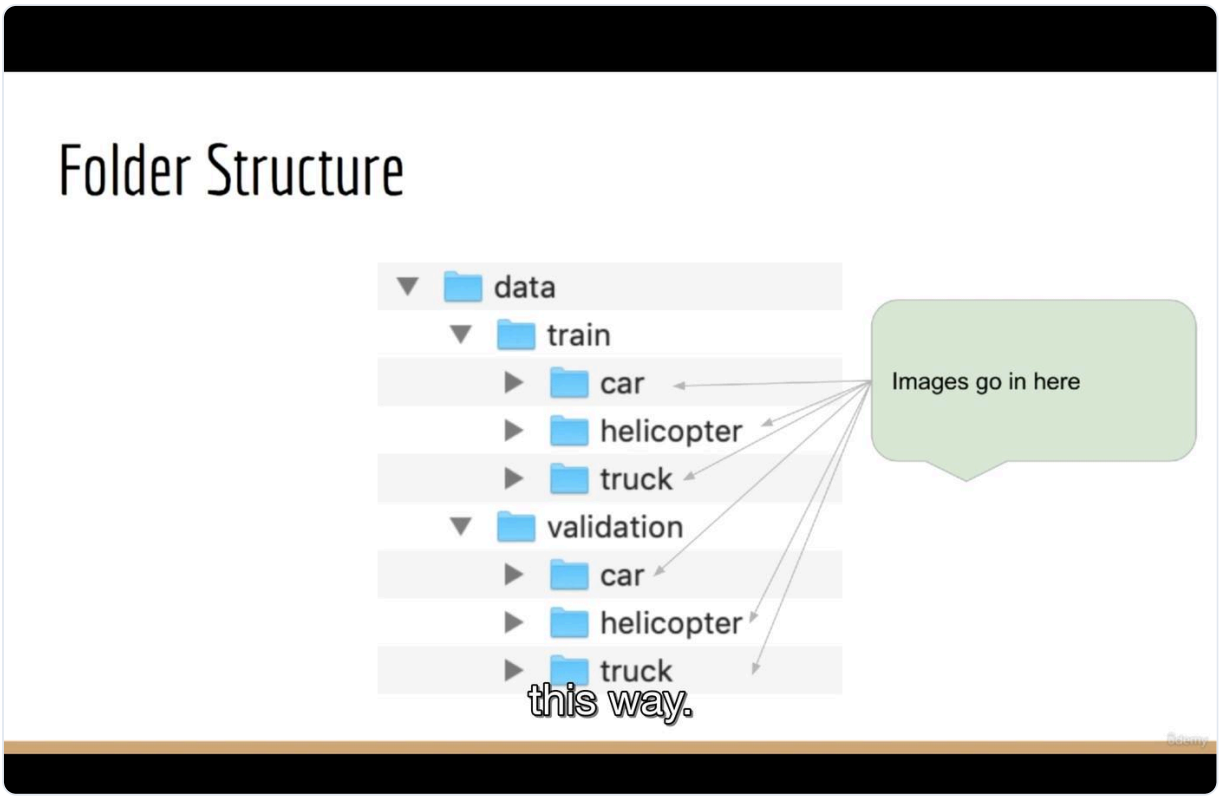


FIGURE 43.9 | Folder Structure

So it's not tied to any specific data set and that can be used for actual image files like JPEGs and PNGs instead of arrays and of course as you would expect this functionality is included in the torchvision library. So how does it work well quite appropriately. It's an object called image folder. Basically it means your dataset is a set of files that exists in some folder specified by some path and the arguments are generally what you would expect from a data set object. The first argument is the path to the Data folder and the second argument is a list of image transformations as you recall from earlier. These could be used for data augmentation and for simple scaling like making the pixel values go from 0 to 1.

The last thing I want to mention in this lecture is that using the image folder data set object necessitates a very specific folder structure. This folder structure is very reasonable so there's no reason for you to not be able to store your data this way. Here's how it goes. First assume your train and validation data live in different folders. Let's call them train and validation. Next within each of these folders we have an individual folder for each class. The name of the folder should be the class name. So for example if you want to classify cars trucks and helicopters then we should have three folders car truck and helicopter next. Within those nested folders is where we would store the actual images.

So inside the car folder we would have images of all cars inside the truck folder. We would have images of all our trucks and inside the helicopter folder. We would have images of helicopters. Again this is just how torchvision expects your data to be organized. So if you want to use those built in functions then you must conform to this format.

LECTURE 82

Pros and Cons

In this lecture I want to discuss two different approaches to transfer learning both of which we will cover in the subsequent coding lectures. The main issue I want to cover is this Imagine that the body of our neural network is very large. Let's say hundreds of layers the head as discussed previously can just be a single logistic regression dense layer.

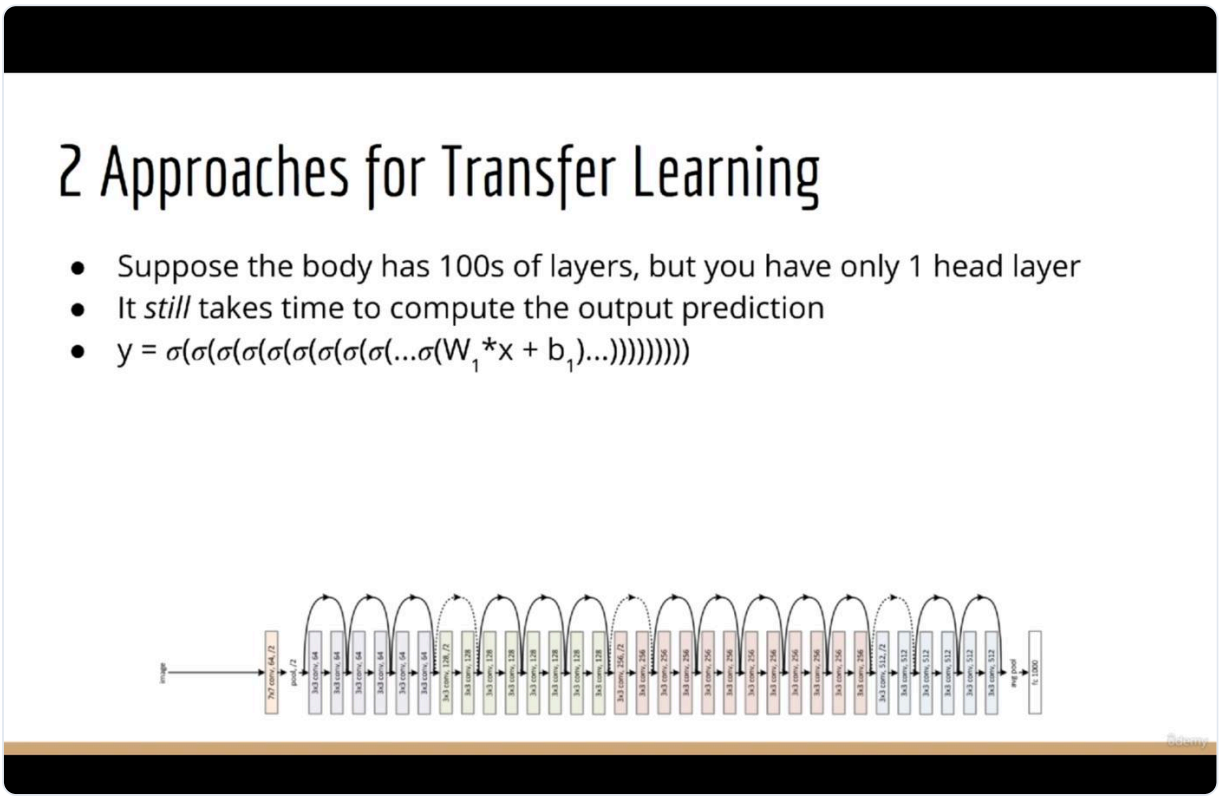


FIGURE 44.1 | 2 Approaches for Transfer Learning

The issue is even though we are not training any of the weights in the body it still takes time to compute an output prediction. Just imagine a really big long nested function at each of these layers. We have to do a convolution which is faster than matrix multiplication but still can take a non-trivial amount of time. Imagine this computation in two parts in order to calculate the output prediction. We first calculate a feature vector z using the body of the neural network. Then in order to calculate the output we take Z and pass it through a logistic regression. My issue is the first part of calculating Z is what takes a long time. Now consider that our gradient descent loop works as follows.

We loop through each batch of data for each batch. We calculate the output prediction y.

2-part CNN

- Imagine the computation in 2 parts:
- Part 1: z = f(x) # pre-trained CNN - slow
- Part 2: y_hat = softmax(Wx + b) # logistic regression - fast

```
for epoch in epochs:
  shuffle(batches)
  for x, y in batches:
    z = vgg_body(x)
    y_hat = softmax(z.dot(w) + b)
    gw = grad(error(y, y_hat), w)
    gb = grad(error(y, y_hat), b)
    update w and b using gw and gb
```

FIGURE 44.2

All data is the same

- Z = f(X) # use CNN to precompute all z's at once
- Turn (Z, Y) into a tabular dataset
- Fit (Z, Y) to Logistic Regression, never have to look at X again!

FIGURE 44.3 | All Data Is the Same

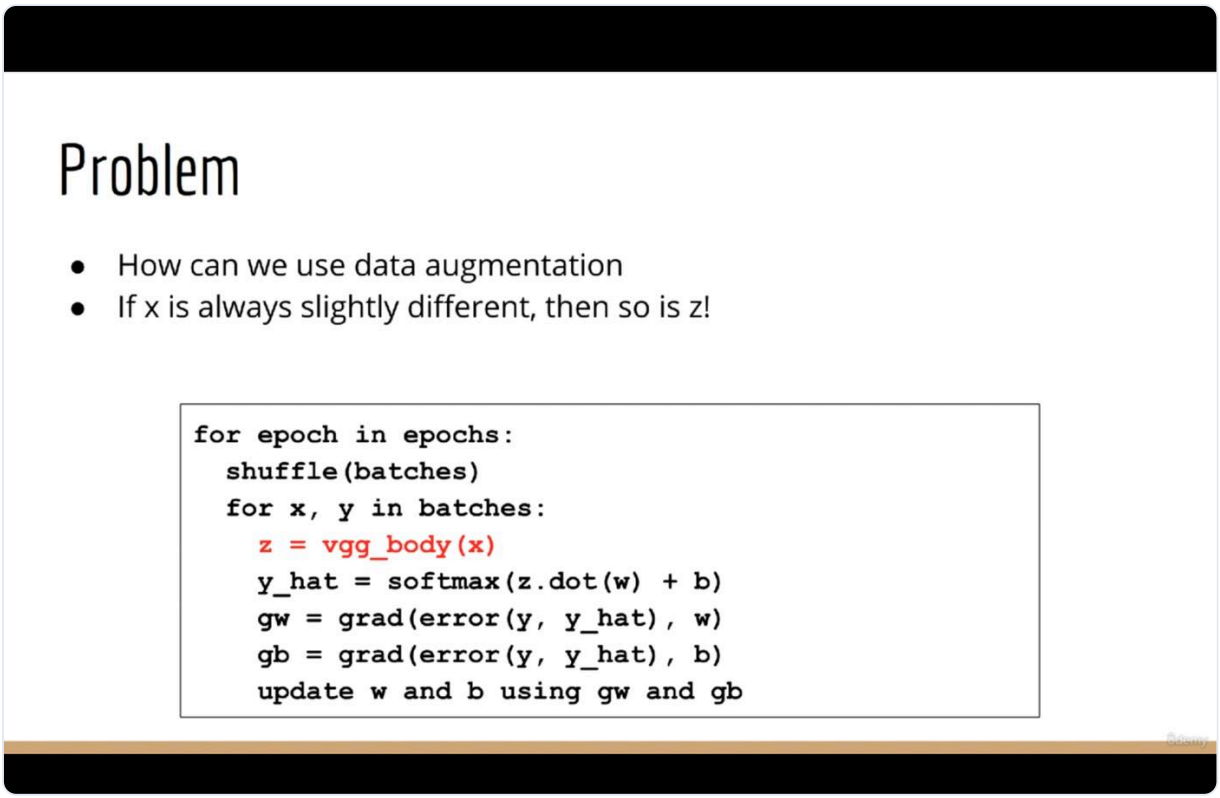


FIGURE 44.4

Then we calculate the gradient of the error with respect to the logistic regression parameters W and B then we update a W and B using those gradients. The thing we want to think about is this Does the calculation of Z ever actually change since Z is the output of the VGG network. After passing in our input data it doesn't make sense to put this inside our loop because it's going to be the same every time. Remember the VGG weights are not going to be trained. So here's what I propose. Remember my rule all data is the same. The idea is before we even begin training let's convert our image data into a tabular matrix of feature vectors z then all we need to do is run logistic regression on Z without ever looking at the VGG network again.

By doing this we can avoid having to pass our data through any pre trained network which can take a lot of time if it has a lot of parameters. All right so there is a problem with what I just proposed.

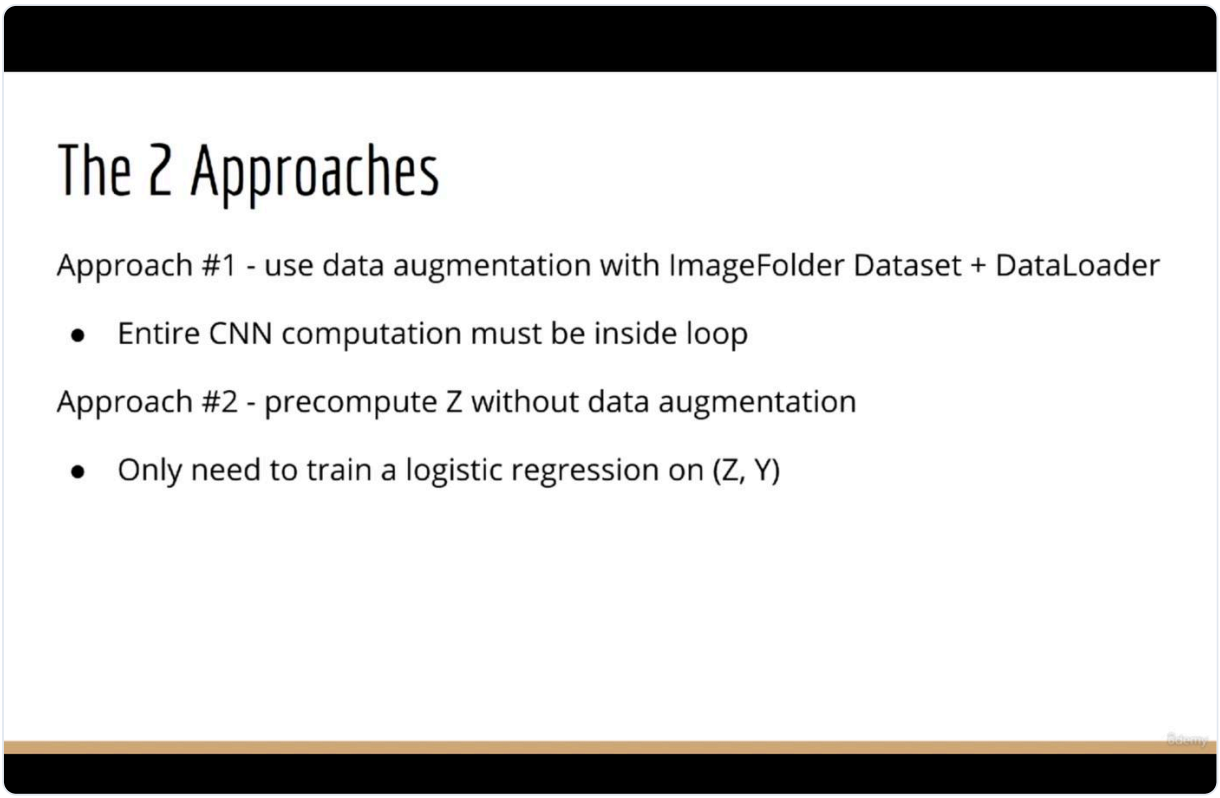


FIGURE 44.5 | The 2 Approaches

What is it. Well remember that when we create our image data generator it gives us the option to do data augmentation with data augmentation. Every time we loop over the data set the generator modifies the original images just a little bit to help our neural network generalize better. Of course if X is different on every iteration of the loop then Z will also be different on every iteration of the loop. In this case we cannot pre compute Z before the training loop begins. The only time it makes sense to do that is if we don't care about doing data augmentation so these are the two approaches approach number one will be to use the image data generator with data augmentation.

This means we have to put the entire CNN computation inside the training loop approach. Number two will be to pre compute the feature vector z on the original data X without data augmentation. This means that after transforming the data all we need to do is train a logistic regression model.

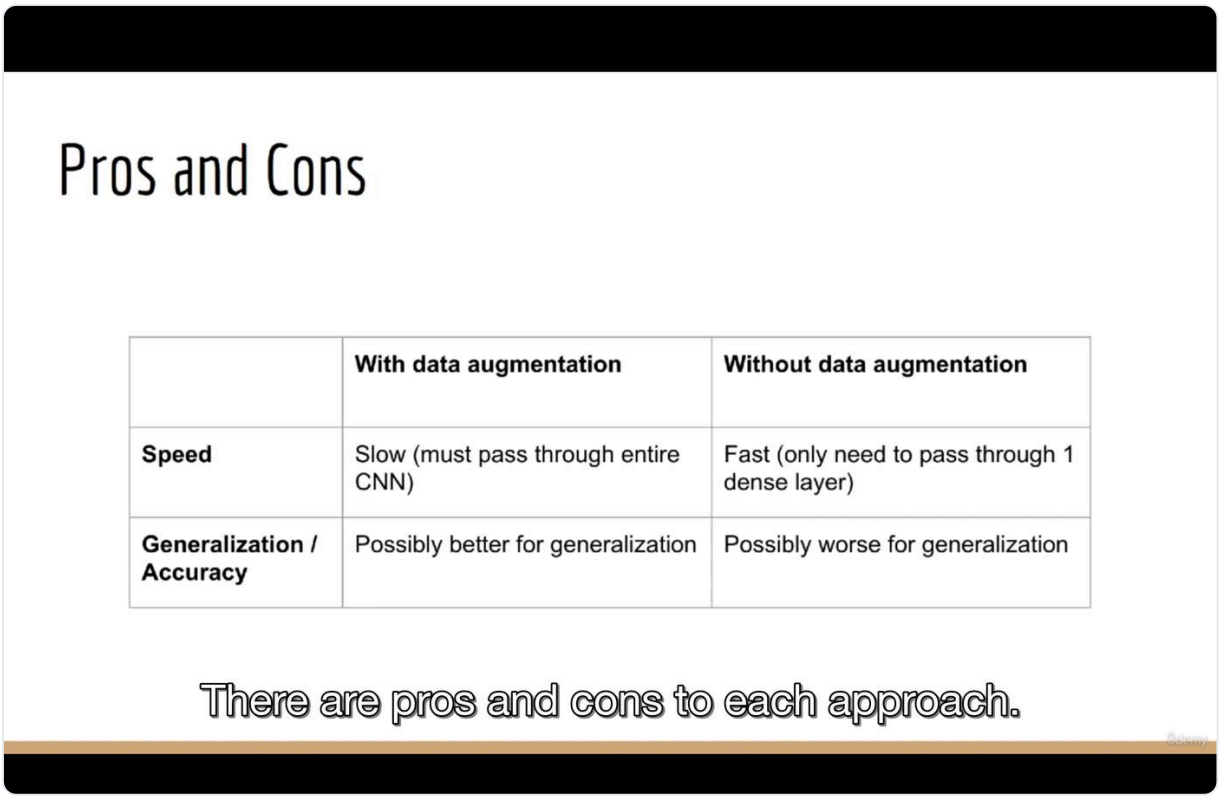


FIGURE 44.6 | Pros and Cons

There are pros and cons to each approach. Let's list them out as discussed earlier. When we do use data augmentation that means we have to recompute the features for every batch we see. This is slow especially if you have a large network. Yes it's faster than training the entire network but still slower than only training a logistic regression on the other hand. Data augmentation may help your model generalize better without data augmentation. It's basically the opposite situation. You can pre compute the features before training which will save you a lot of time training logistic regression is extremely fast yet. This also means that you cannot use data augmentation which may cause your model to perform sub optimally.

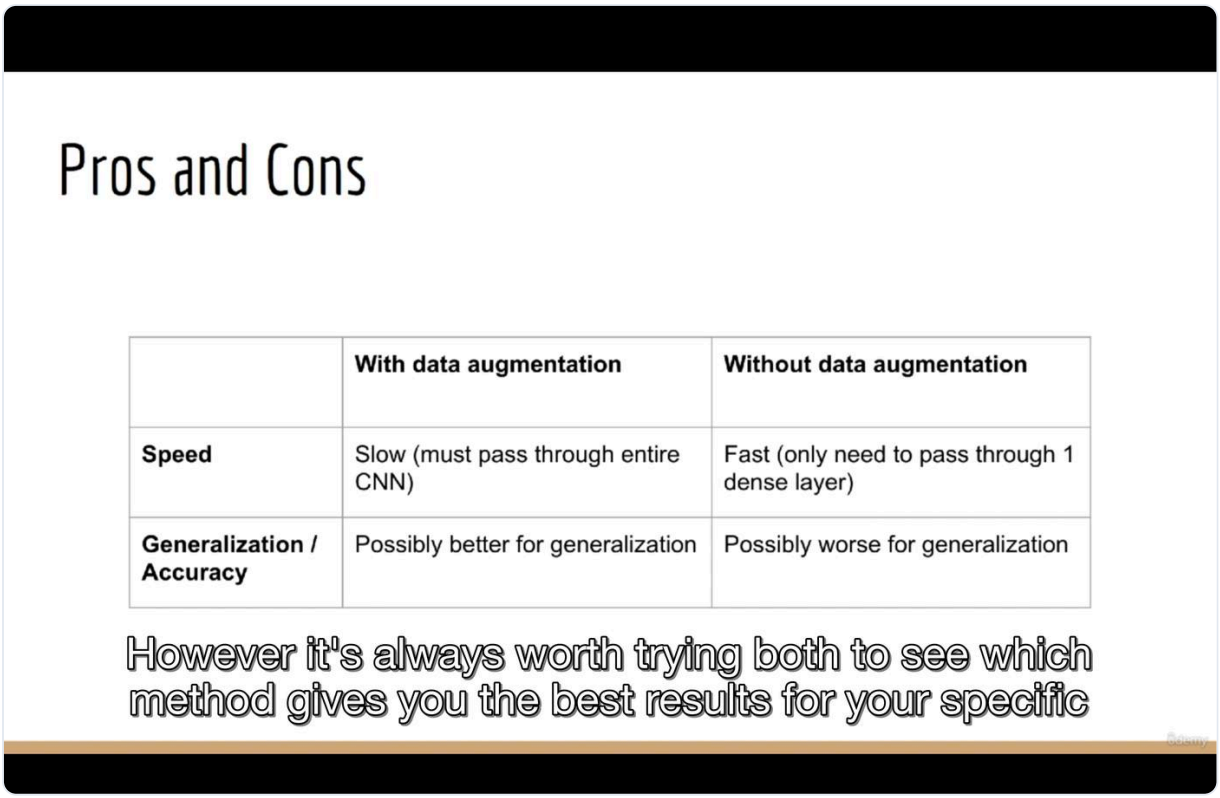


FIGURE 44.7 | Pros and Cons

However it's always worth trying both to see which method gives you the best results for your specific dataset.

LECTURE 83

Food and Data Augmentation

See the notebook `21.PyTorch-Transfer-Learning-with-Data-Augmentation.ipynb` for the code.

In this lecture we are going to look at a Colab notebook that does transfer learning with data augmentation. This lecture is going to walk you through a prepared Colab notebook although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. It's called PyTorch transfer learning with data augmentation so at the top we have all our usual imports. We have one extra special thing which comes from the torchvision library and that's the models module. This is of course where pre trained image networks are accessible from.

We have some other minor new imports as well but you'll see how those are used as we go along. Next we download the data which is called the Food five k data set. This dataset is a set of images of food and non-food items so it's a binary classification dataset and your job is to classify whether an image is food or not food. The next step is to unzip the file since there are a lot of files we're going to use the qq option to suppress the output. Next we're going to do an ls to see what got unzipped. As you can see there are a few new folders here specifically evaluation training and validation. So let's check and see what's in the training folder. So as you can see it's just a bunch of image files.

So what do we know right away. Well we can see that this is not in the format we need in order to use the image folder data set object. What we are looking for is one folder for each class of images what it looks like though is that all the images for one class start with a zero in the file name whereas all the images for the other class start with a one. Of course we shouldn't try to guess whether that's true or not we should at least do a sanity check to make sure that this is actually true so let's print an image that starts with zero. Just to be sure. So I've chosen 0 8 0 8 but you should try other images yourself too. All right so this is a flower which is not food next let's try an image that starts with one I've chosen a 1 6 1 6.

As you can see this appears to be food there looks like pizza. Again try a few other images yourself to see if all the zeros are non-food and other ones or food since these images are not organized the way we need. We're going to have to move the files around a little bit. So let's start by making a new folder called data inside this folder. We're going to have two more folders train and test. Now the original dataset came with three folders evaluation and training and validation but we don't have any need for three separate datasets. So what we're going to do is just use the training folder as the train set and the validation folder as the test or validation set will ignore that evaluation folder inside the train and test folders.

We need to have a folder for each class so you can see here that I've created two folders inside each of the train and test folders food and non-food next. It's just a matter of moving the images moving is better than copying in this case since we don't want to take up too much space and waste time replicating the data on disk so you can see here that I'm taking all the files that start with zero and end in JPEG and moving these to the corresponding non-food folders. I'm taking all the files that start with the one and end in JPEG and moving these to the corresponding food folders next I'm going to specify the transformations to perform on both the train and test sets.

As usual you should try to play around with these data augmentation parameters to see what effects they may have if any. You'll notice that the final transformation I apply happens to both the train and test sets. It's a normalize which as you might expect does normalization and I've specified what appears

to be just a bunch of random numbers. So what are these random numbers and why are they there. Well you have to remember that these are pre train networks with very specific weights. So we can't just pass in any image in any way we want into this neural network. We have to transform our image in the same way that the original authors of the neural network did.

Otherwise our input won't make any sense. So this is more or less like that. There are some additional details that I don't want to get into now you'll notice that there are many more transformations we're applying to the test set which may seem odd you might think why are we applying data augmentation to the test set. And actually we're not. So these transformations are static transformations they don't augment the data they're just necessary for the neural network to work. So the first transformation which of course has to be applied to both datasets is the resize. CNN's work on image batches of constant size. So first we'll resize the 256 by 256. After that we cropped the image to be of size 224 by 224.

OK. So that's why we have some transformations on the test set. Now you notice that the train data has a few additional steps in between. So random rotation color jitter and so forth. As mentioned you should feel free to play around with these. Change the order and so on next we create our image folder datasets. This is pretty trivial. We pass in the paths to the image files and the transforms that we just created. Next we create our data loader objects. As usual next we define our model. So this is going to look very different from what you're used to. First we grab the VGG16 model which is located in the torchvision models module we set pre trained equal to true because we want pre trained weights and not random weights.

Next we freeze the weights so that these weights are not updated during training. We can accomplish this simply by looping over the model parameters one by one and setting the requires grad attribute to false. Next we print the model so you can see that it's a very large model. Importantly though it has a structure we've seen before so the first part is a sequential. That wraps all the convolution and pooling layers as you can see this is attached to an attribute called features. Next we have an average pool 2D. And finally we have another sequential that wraps the dense layers and this is attached to an attribute called the classifier so if we print out model dot classifier we see just the ANN part of the CNN.

This is the part we want to get rid of because we want to attach our own head so next we're going to get the number of features that are output from the first part of the model and call that n features. We need this because it's going to be the number of inputs into our head so we can get this by calling classifier of 0 which gives us the first layer and then calling the attribute in features. So we have twenty five thousand eighty eight features. Next we're going to create our own classifier which will just be a regular linear layer logistic regression and we'll assign this to the classifier attribute to replace the old one so if we print the model again we can see that the classifier attribute now refers to our own linear layer and actually move the model to the GPU as usual actually after this everything is the same as before.

So we create a loss and optimizer we create the training function we call the training function we look at the loss per iteration and we calculate the accuracy and these are the results so we get ninety nine percent accuracy on this large image dataset using transfer learning.

LECTURE 84

Data Augmentation

See the notebook `22.PyTorch-Transfer-Learning-Fast.ipynb` for the code.

In this lecture we are going to look at a Colab notebook that does transfer learning without data augmentation. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. And this notebook is called PyTorch transfer learning fast so our main goal in this script is to see if we pre compute the features a Will training B faster and B Well we get an unacceptable reduction in accuracy I'll assume that in this lecture you've already seen the code that does transfer learning with data augmentation.

So I'm not going to explain any of the lines that I've already explained so at the top everything is the same as before. Same imports same data same process to move the data into the folders which are structured the right way for the image folder data set object OK. So when we get to the transforms you'll notice that we only have one. Why do we only need one. Well since we're not going to do any data augmentation we can apply the same transformations to both the train and test that and these are all non data augmentation and transformations so next we create our image folder objects. Next we create our data loader objects. Next I grab the pre train model with pre trained equals true so the next part is also new since the VGG operation is separate from the rest of the network.

We can define a separate model for it so I'll define a class called VGG features in the constructor. The only thing I need to do is save the VGG model in the forward function. I take X and I pass it through the different parts of the VGG one at a time until right before the final classifier. At that point I just flatten and then return next I instantiate this class and I call it VGG next. As a sanity check just to make sure this works we pass in a random input output a shape one by twenty five thousand eighty eight. So it works. Next we're going to transform our data using the feature transformer that we just instantiated. So first we need to find the length of each dataset.

Luckily we can use the lend function on the data set objects so we have N train three thousand and N test one as. Next we're gonna grab D the feature dimensionality. This is the size of the output of the VGG transformer which is equal to twenty five thousand eighty eight. Next we're going to work on creating our tabular data set consisting of transform the features from the original data. Once we have this we no longer have to work with the original dataset. So we're going to start by instantiating empty arrays to store X train y train x test and Y test. Next we're going to set the device to the GPU and move the feature transformer to the GPU next we're going to populate X train and Y training.

We can do this by looping over the train at dataset. So on each iteration we start by moving the inputs to the GPU it's not necessary to move the targets to the GPU because we only want to make a prediction with the model. Next we get the output by passing the inputs through VGG. Next we get the size of the output. Note that this is not equal to the batch size since the number of samples may not divide evenly into the batch size. So the final batch is less than or equal to the batch size next we assign the features and the targets to X train and Y train at the current index i. Then we increment i by the current batch size and go to the next iteration of the loop so since we print i on each iteration we can confirm that the number of samples in our new dataset is equal to the original number of images.

All right so we can see that the final value of i is three thousand which is the correct number of samples. Next we do the same thing for X test and Y test. So one thing that's interesting to check is the min and the max of the feature values. As you can see the max in X train is about sixty two which should seem weird to you because you should expect it values in a neural network to be somewhat normalized. Since we're using a ReLU activation at the minimum value with zero so anyway since the data is at this weird scale we're going to use a standard scaler object to transform it so we'll call the scale data X train to an X test to now because the next step is to do logistic regression.

There's no actual reason we need to use PyTorch. In fact we can just use the logistic regression that's built into scikit-learn so let's do that. As usual it's just fit and predict which I've taught you for free in the past so everybody knows this. And as you can see we do very well just as well as when we did use data augmentation in fact. So maybe data augmentation wasn't so useful in this instance. Next we're going to complete this script by doing transfer learning the usual way which is to use the same library for the classifier. So let's start by creating a linear model and the rest of these steps you've seen before so we move the model to the GPU create the loss and optimizer make some tensor data set objects create the data loaders define the training function call the training function plot loss per iteration and calculate the accuracy and so we end up with pretty much the same result also important.

Notice how much faster this is with data augmentation. Each epoch takes about 40 seconds without data augmentation each epoch is nearly instant about zero point two seconds. So that's a two hundred x speed up by not using data augmentation and not having to go through the pre train network on each pass.

LECTURE 85

See for Yourself

Refresh the site `https://thispersondoesnotexist.com/` again and again.

`https://www.whichfaceisreal.com/` is a game to check which face is real and which is fake.

In this lecture we are going to discuss a very popular topic in modern deep learning. GANs GANs stands for generative adversarial network.

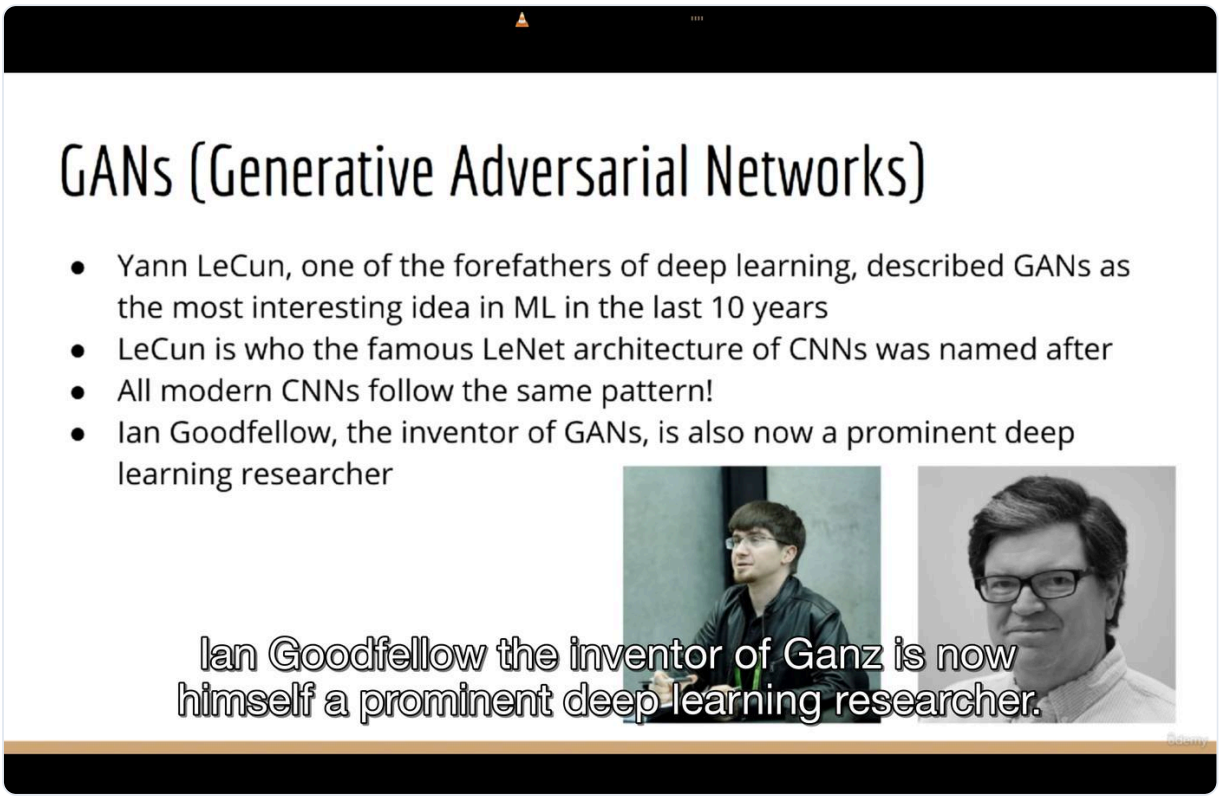


FIGURE 47.1

Yann LeCun who is one of the forefathers of Deep Learning described GANs as the most interesting idea in machine learning in the past 10 years. By the way this is also who the famous LeNet architecture of convolutional neural networks was named after the CNN from which all other CNN's are based. Ian Goodfellow the inventor of GANs is now himself a prominent deep learning researcher. My goal in this lecture is to introduce you to the fundamental theory behind GANs as you'll see that although they sound complicated they are nothing but a new and interesting way of combining things we already know how to build. This is a common pattern in deep learning neural networks themselves are nothing but a bunch of logistic regressions chain together.

CNNs and RNNs are nothing but neural networks with shared weights.

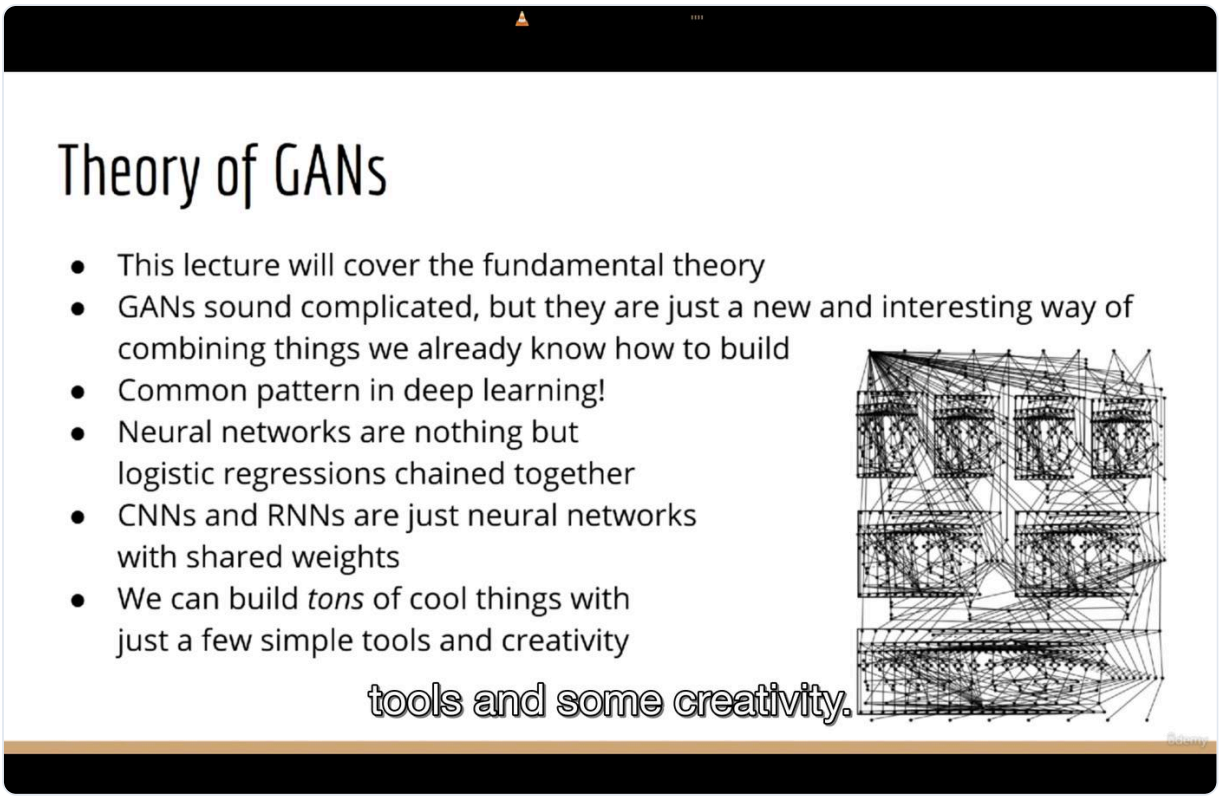


FIGURE 47.2 | Theory of GANs

I like this approach because deep learning allows us to build tons of cool things with only a few simple tools and some creativity. First let's discuss what GANs are even used for in the first place. Since this may not be immediately obvious the main use case for GANs is to generate data. Ninety nine percent of the time.

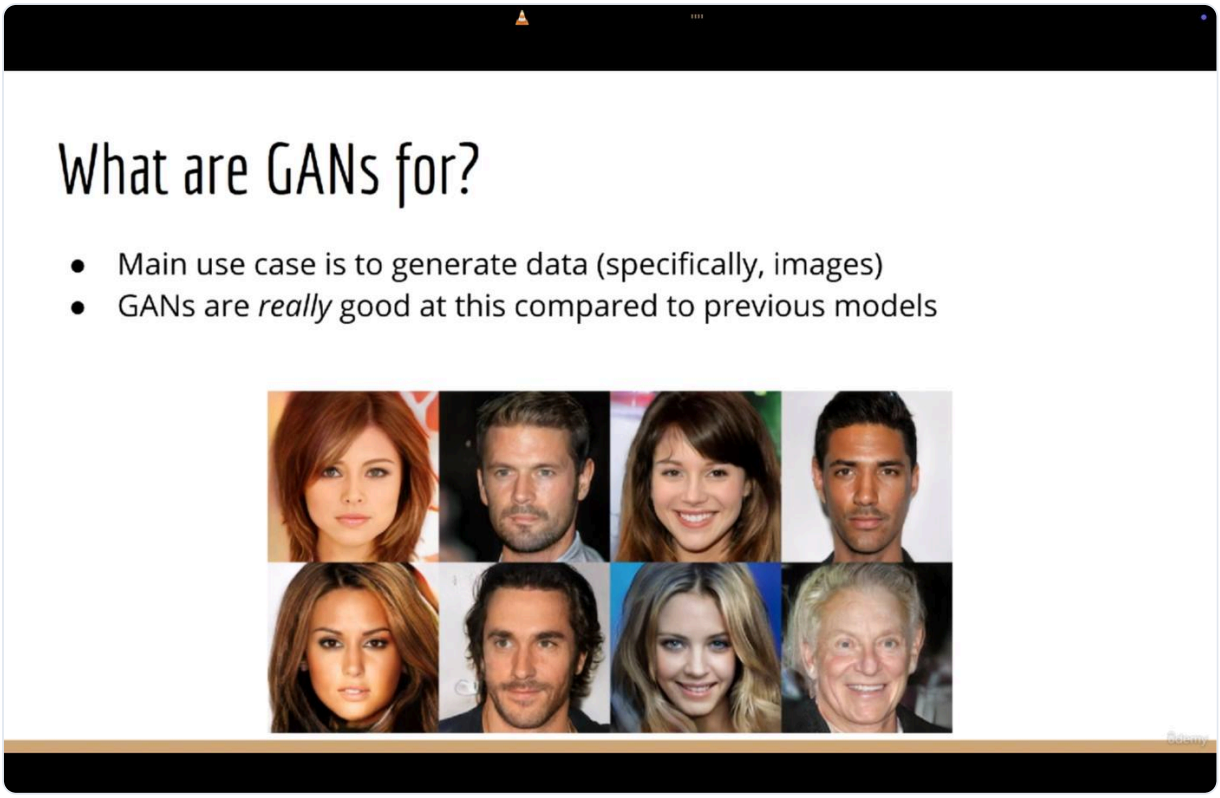


FIGURE 47.3 | What Are GANs for?

This means generating images. The reason GANs have become so popular is because unlike the models that came before it GANs are exceptionally good at this. Here is some example images produced by state of the art GANs. Believe it or not these are not real people but rather images generated by a computer one thing you can do if you want to see a few more examples and observe for yourself the power of GANs is to check out a few Web sites that display such examples. One Web is.

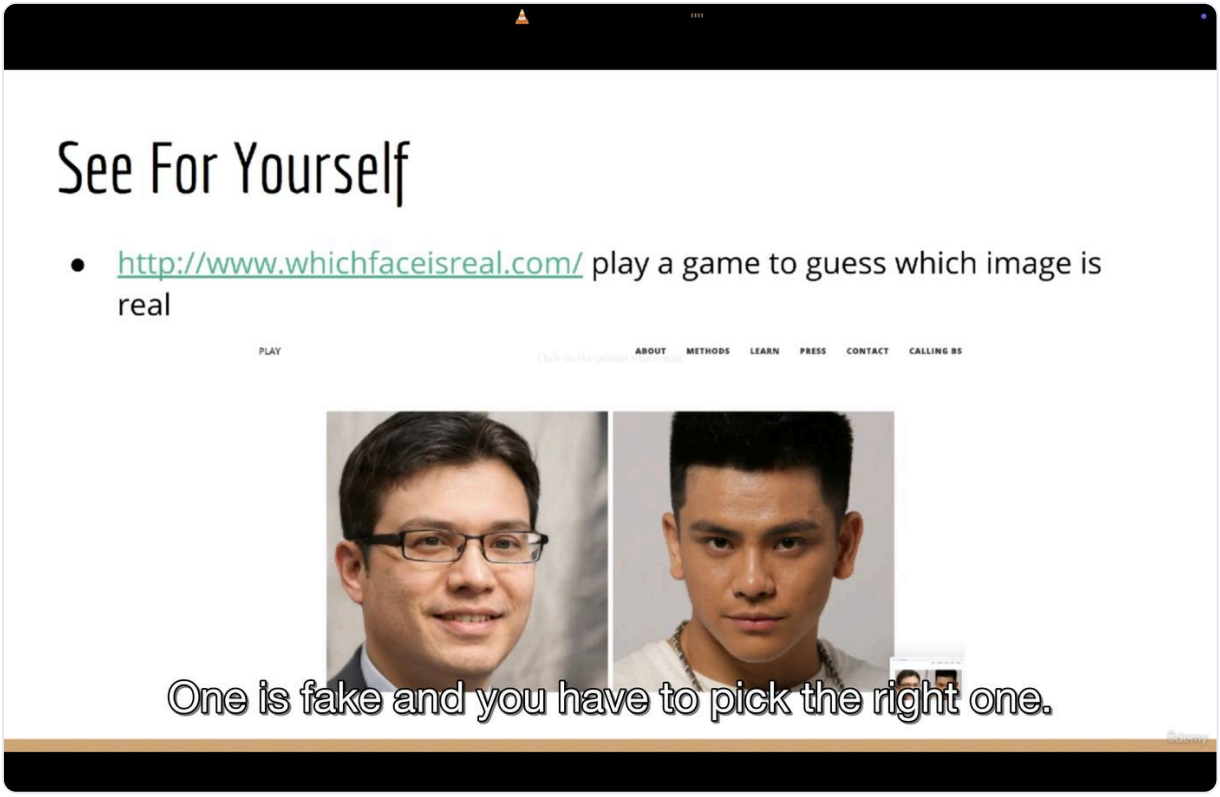


FIGURE 47.4 | See for Yourself

This person does not exist. Dot com. This one is simple. You go to the page and it shows you a randomly generated face.

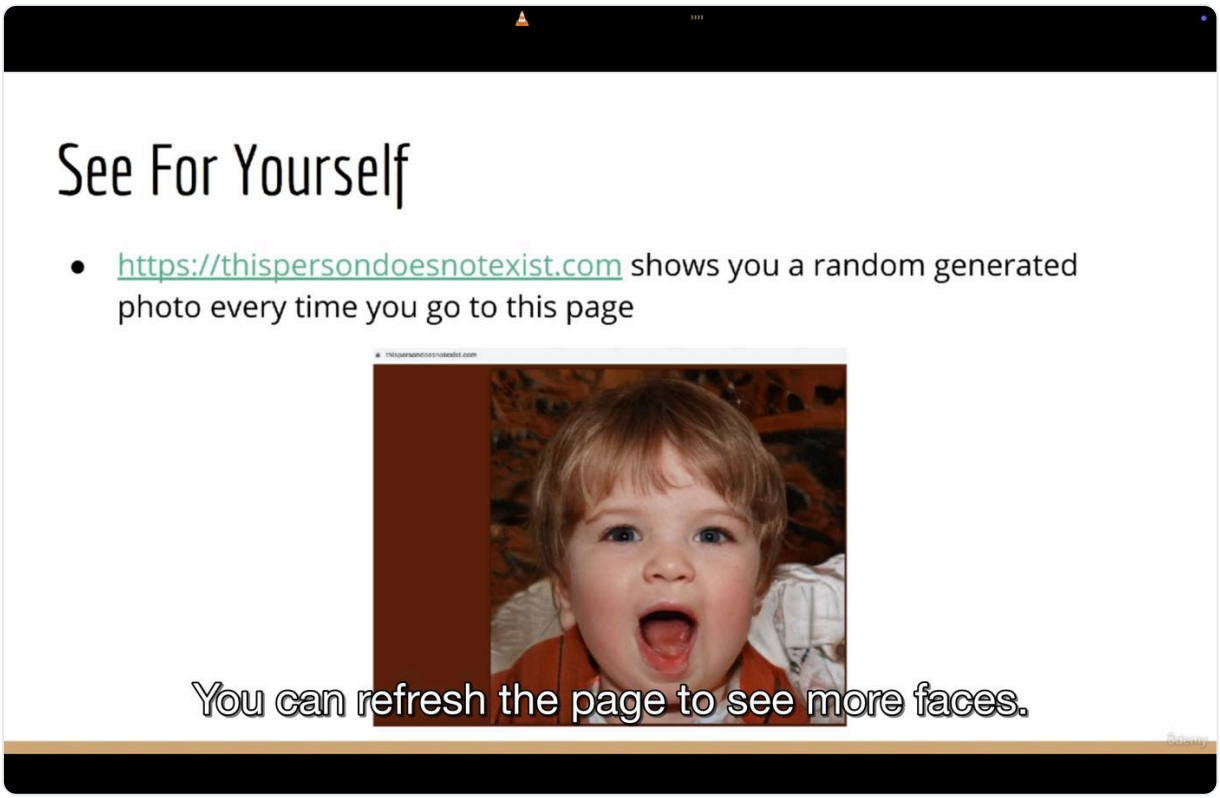


FIGURE 47.5 | See for Yourself

You can refresh the page to see more faces. Another one which I really like is which faces real dot com. This one is more like a game. So you're shown two images and one is real. One is fake and you have to pick the right one. So check these out for yourself. If you want to see more examples of how good GANs are at generating realistic images OK. So what's the main idea behind GANs GANs are a system of two neural networks each of which has their own objective the objective of the first neural network is to generate images.

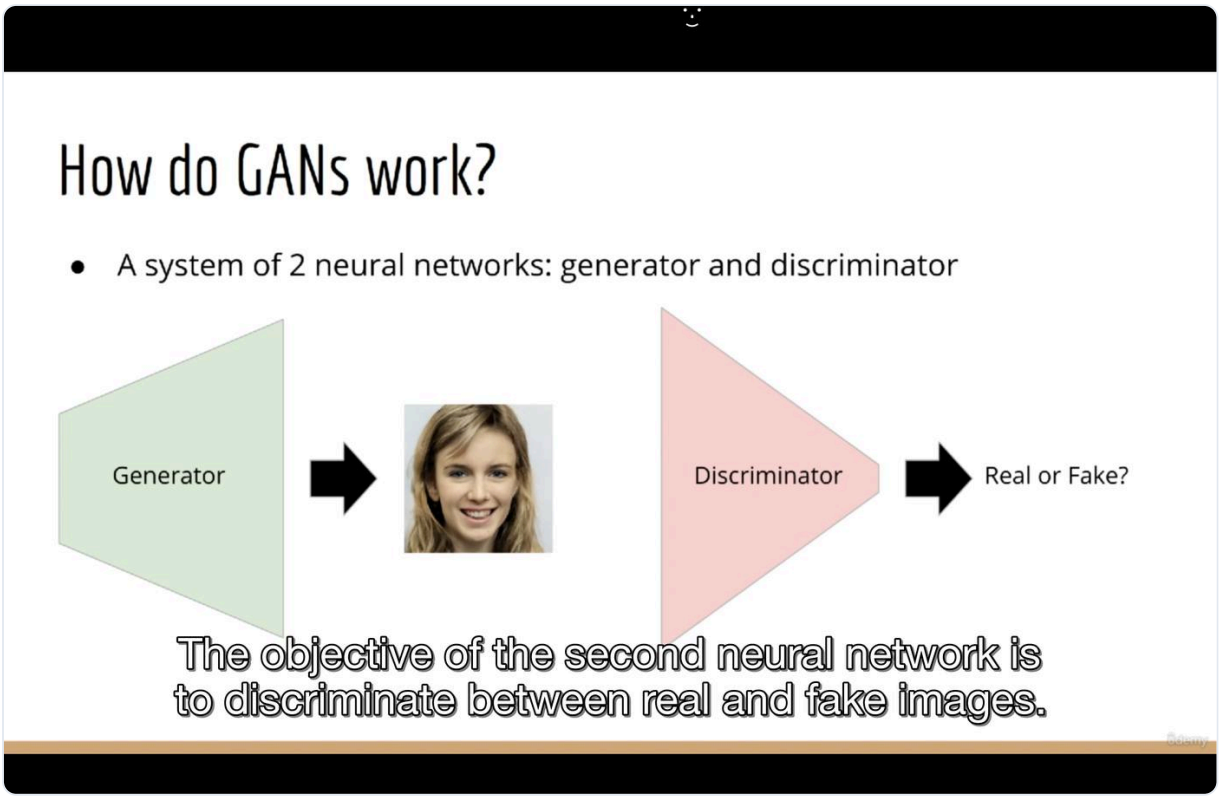


FIGURE 47.6 | How Do GANs Work?

The objective of the second neural network is to discriminate between real and fake images. This is all you need to understand how GANs get their name. They are generative because you are generating images. They are adversarial because you have two neural networks which oppose each other in an adversarial manner. They have opposite goals. The goal of the first neural network is to generate images that look real and the goal of the second neural network is to detect real and fake images intuitively. You can think of GANs like a counterfeiter and a shop owner. The counterfeiter goes to the shop and tries to use fake cash to buy an item. The shop owner has to detect whether the cash being given by customers is real or fake.

Perhaps at the beginning both the counterfeiter and the shop owner are not experts at what they do.

FIGURE 47.7 | Intuition: Counterfeiter vs. Shop Owner

The counterfeiter is not that good at making fake bills but the shop owner is also not that good at detecting fake bills. However once the shop owner learns to detect the fakes the counterfeiter may then improve on his counterfeit design and trick the shop owner once more. After a while the shop owner may again realize he is being duped and learn to inspect the bills more carefully to detect these improved fake bills. The cycle continues as the counterfeiter gets better at tricking the shop owner and the shop owner gets better at avoiding being tricked. Of course these analogies only take us so far. How do we actually build a GAN. Let's recall all the ingredients we need for a basic neural network that we want to train first.

We need a neural network that much is obvious. Well in our case we actually have two neural networks in order to train the neural network. We need an objective or a loss function.

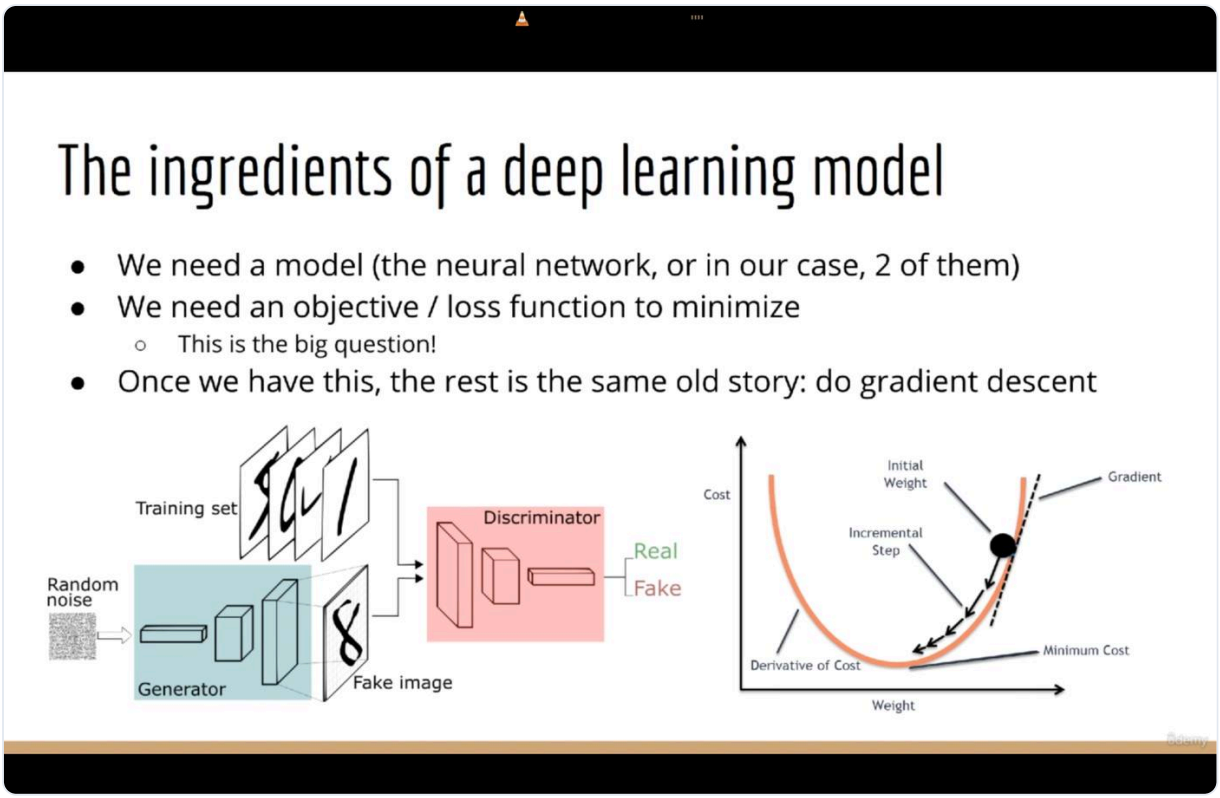


FIGURE 47.8 | The Ingredients of a Deep Learning Model

This is the big question with GANs. What is the loss function. But once we have these two things everything else is nearly trivial. We call model dot fit which does gradient descent and that's that OK. So let's dig a little deeper on what the loss function is. And in fact for a GAN we're going to have two loss functions one for the generator and one for the discriminator. Let's first look at the discriminator since that's more in line with what we are used to already. Since the discriminator must be able to tell the difference between real and fake images. This is a binary classification problem for every input the discriminator can only predict one of two categories real or fake.

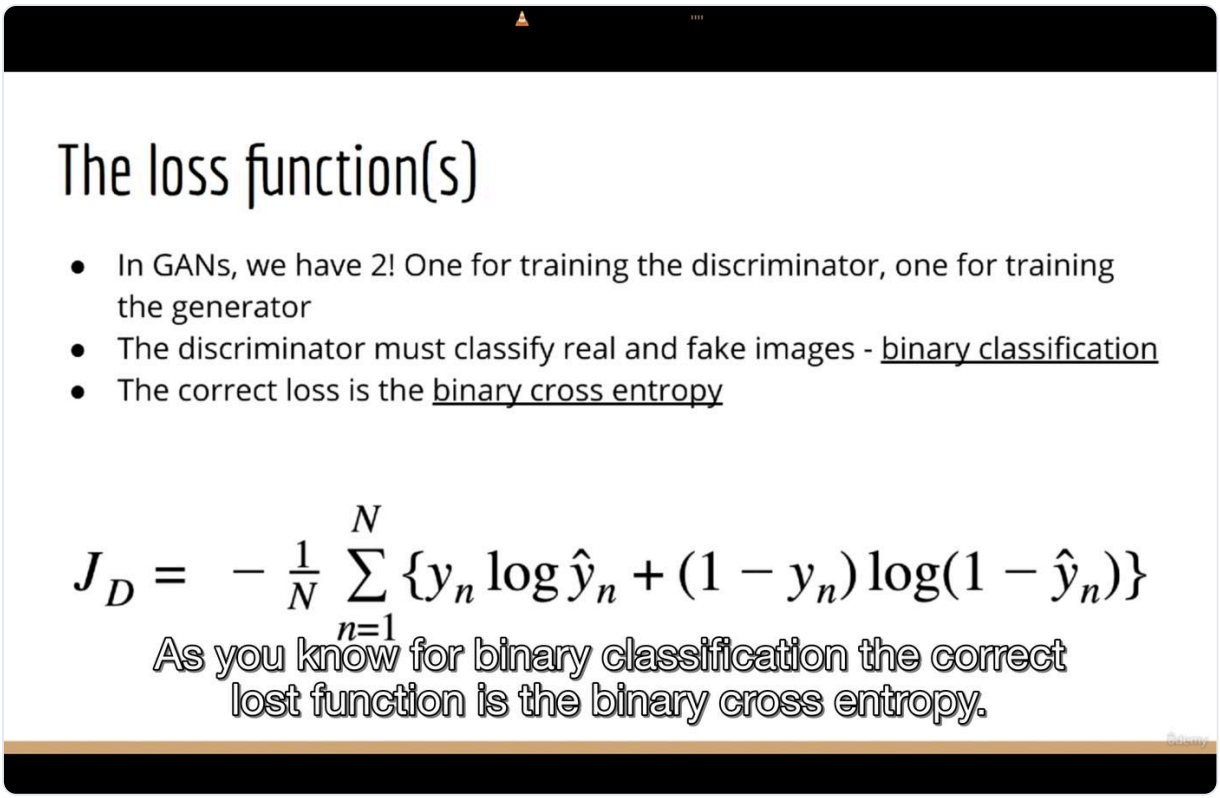


FIGURE 47.9 | The Loss Function(s)

As you know for binary classification the correct loss function is the binary cross entropy. What about for the generator. What is the loss function. This is what I call applying creativity to the basic tools that we already know about intuitively. The basic idea is this we have our generator network in our generator network feeds into our discriminator network. From this perspective this is just one gigantic neural network.

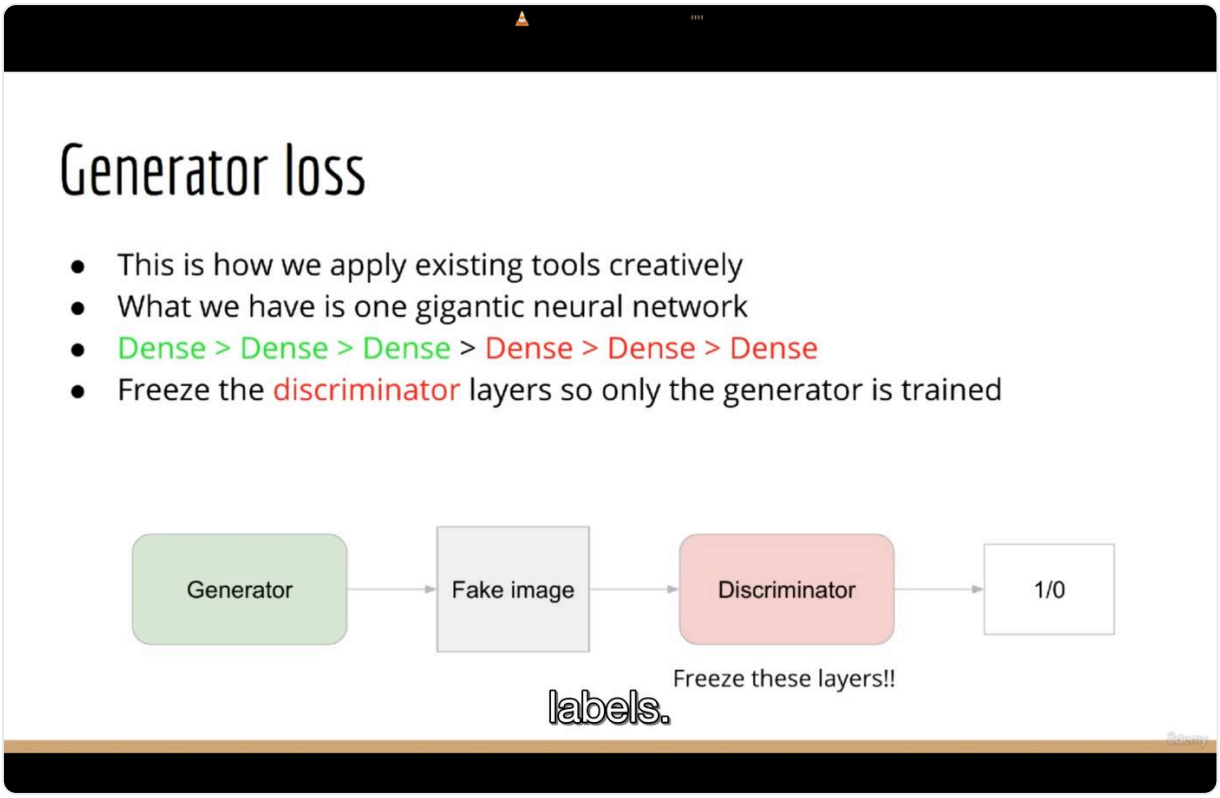


FIGURE 47.10 | Generator Loss

What we can do is just like with transfer learning we can freeze the layers in the discriminator so that when we run gradient descent only the generator network gets trained next. And here is the ingenious part. We want the generator to get better at producing real looking images. And so our loss function will remain the binary cross entropy except that we are going to switch the labels. In other words let's say for the discriminator real is one and a fake a zero. We're going to pass in fake images but make the target one such that we're encouraging the discriminator to identify these fake images as real but the discriminator weights are frozen so they won't change when we do this only the generator weights are trained here's what the loss would look like mathematically.

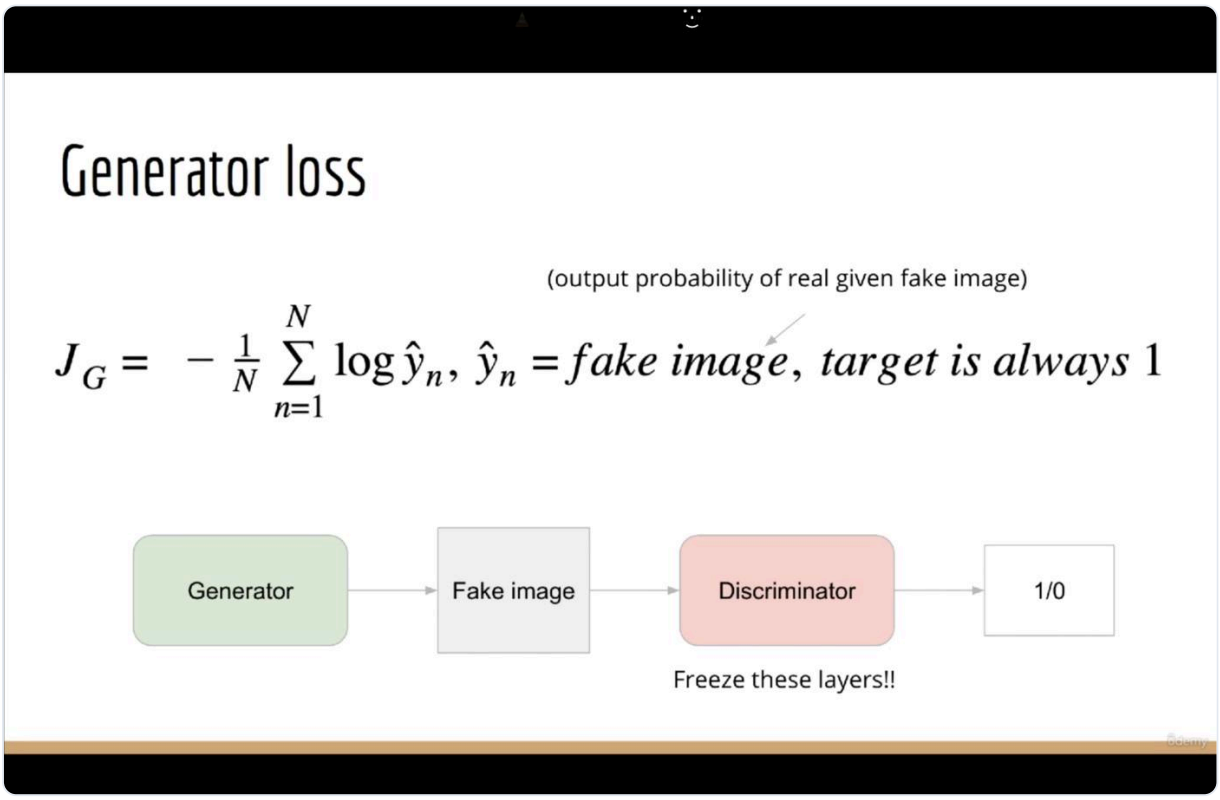


FIGURE 47.11 | Generator Loss

If you're interested here all the y hats are predictions on fake images. Since the generator is now the input into the discriminator because we always want the target to be one we only care about the first half of the binary cross entropy. So essentially it's just the sum over the log of all the y hats. Or as stated another way the negative log likelihood of the y hats now that you understand the basics of how GANs work. Let's look at some details that will help us in our implementation. We know that neural networks must have both inputs and outputs for the discriminator.

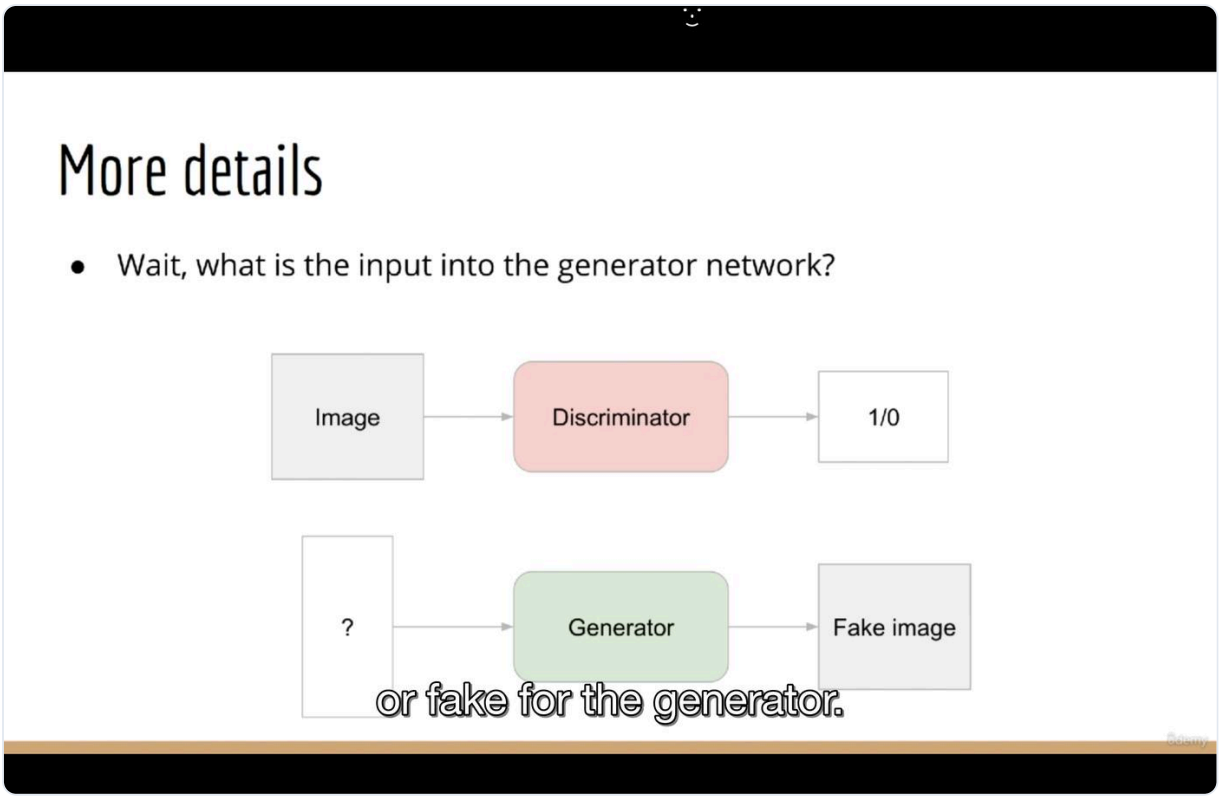


FIGURE 47.12 | More Details

This is easy to input as an image and the output is a prediction about whether the input image is real or fake for the generator. This will be a little strange. We know that the output should be an image because its job is to generate images but what is its input. In fact the input to the generator is nothing but noise. What we are going to do is generate noise from a multivariate standard normal and the generator will learn to map this noise to an image. Mathematically what we are doing is saying we have some random vector z which comes from a standard normal.

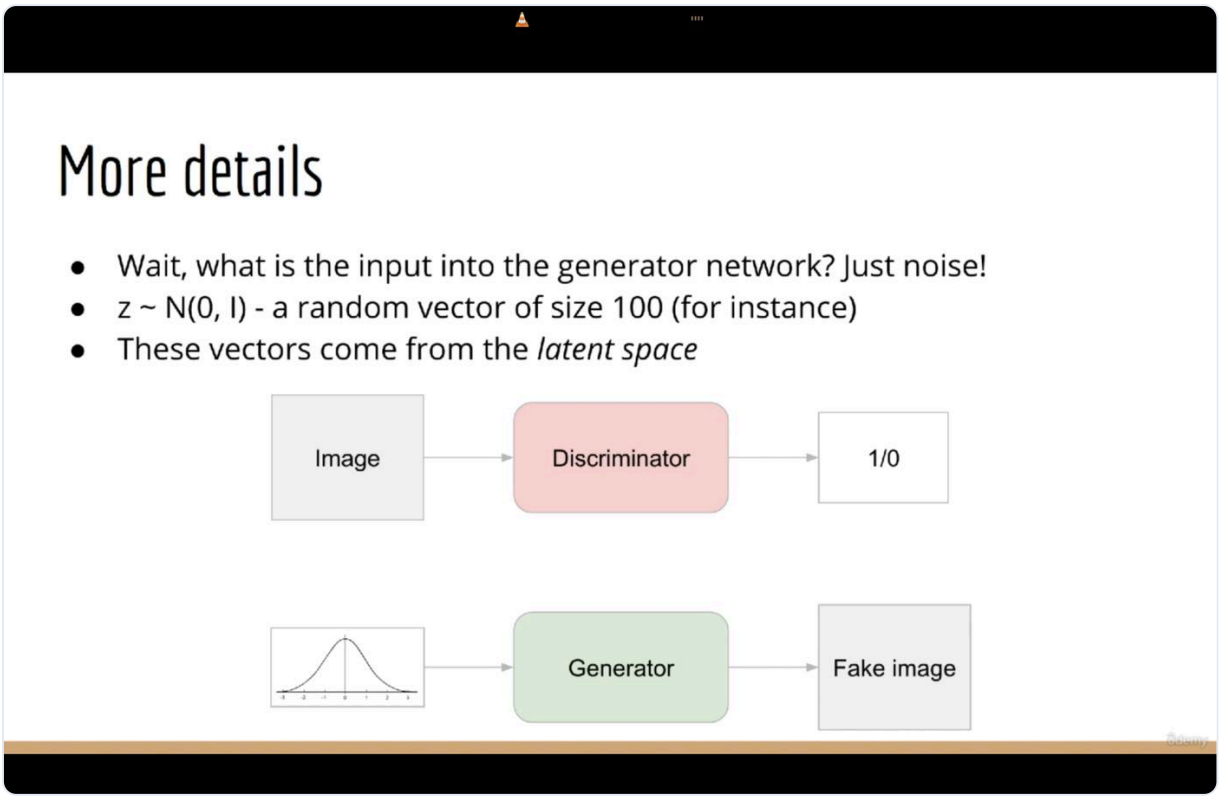


FIGURE 47.13 | More Details

Let's say it has size D equals 100 although this is a hyperparameter so you can choose what you like. We call this one hundred dimensional space. The latent space in essence you can think of it as an imaginary space where the generator believes all the images of digits live. So maybe the ones are over here. All the sevens are right beside it because seven's kind of look like ones near the Sevens or the nines because nines look like sevens if you draw the loop very small so you can see how these images kind of morph into one another when they are drawn in different ways. So this latent space maps all the different kinds of images that you can have and of course this is a source of confusion for both computers and humans.

When an image is right on the boundary between say 7 or 1 it's difficult for us to tell what digit it should be.

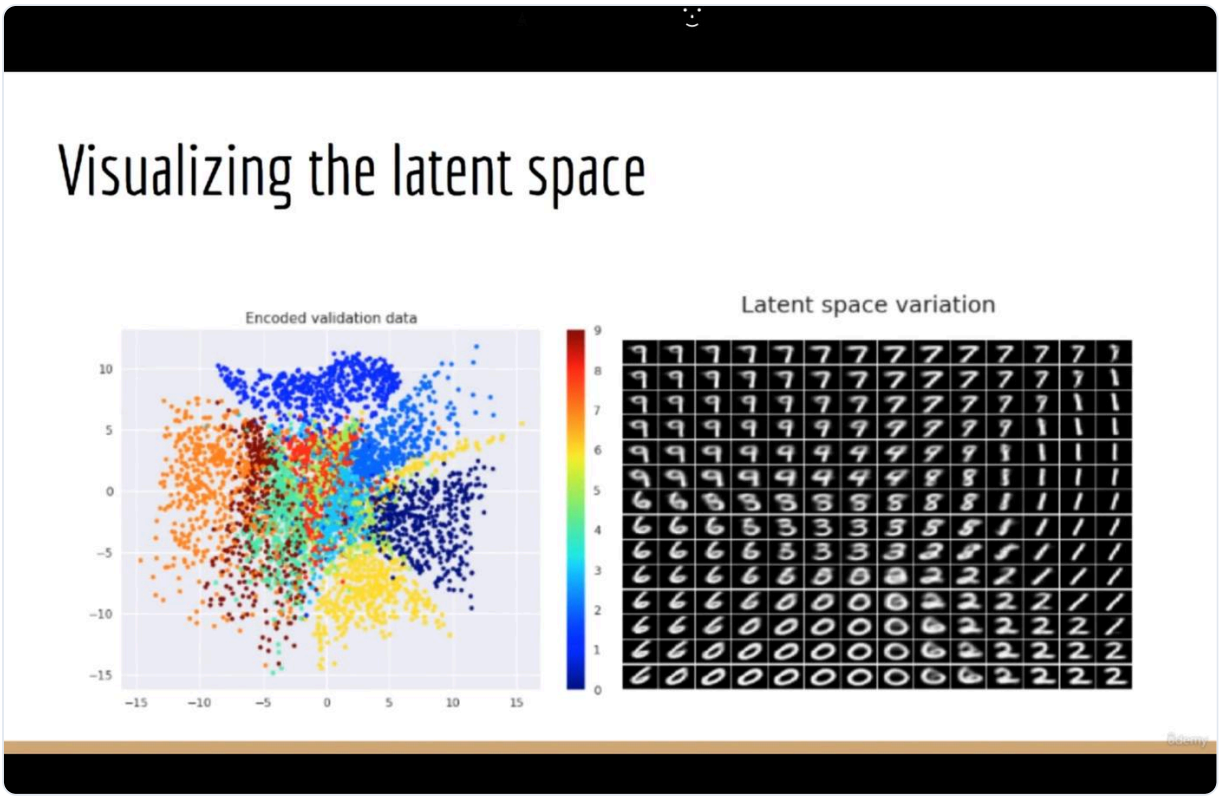


FIGURE 47.14 | Visualizing the Latent Space

Should it be a 7. Or should it be a one. In any case what the generator is doing is it's learning to associate each part of the latent space with different images in a continuous manner. Then later when we generate images we're just picking a random part of the space and generating an image from whatever that part of the space represents. You can think of the generator as kind of the reverse of a feature transformer or an embedding with a feature transformer or an embedding or taking an input image or an input text or some other kind of input and we're mapping it to a vector. But for a generator in a GAN we're doing the opposite.

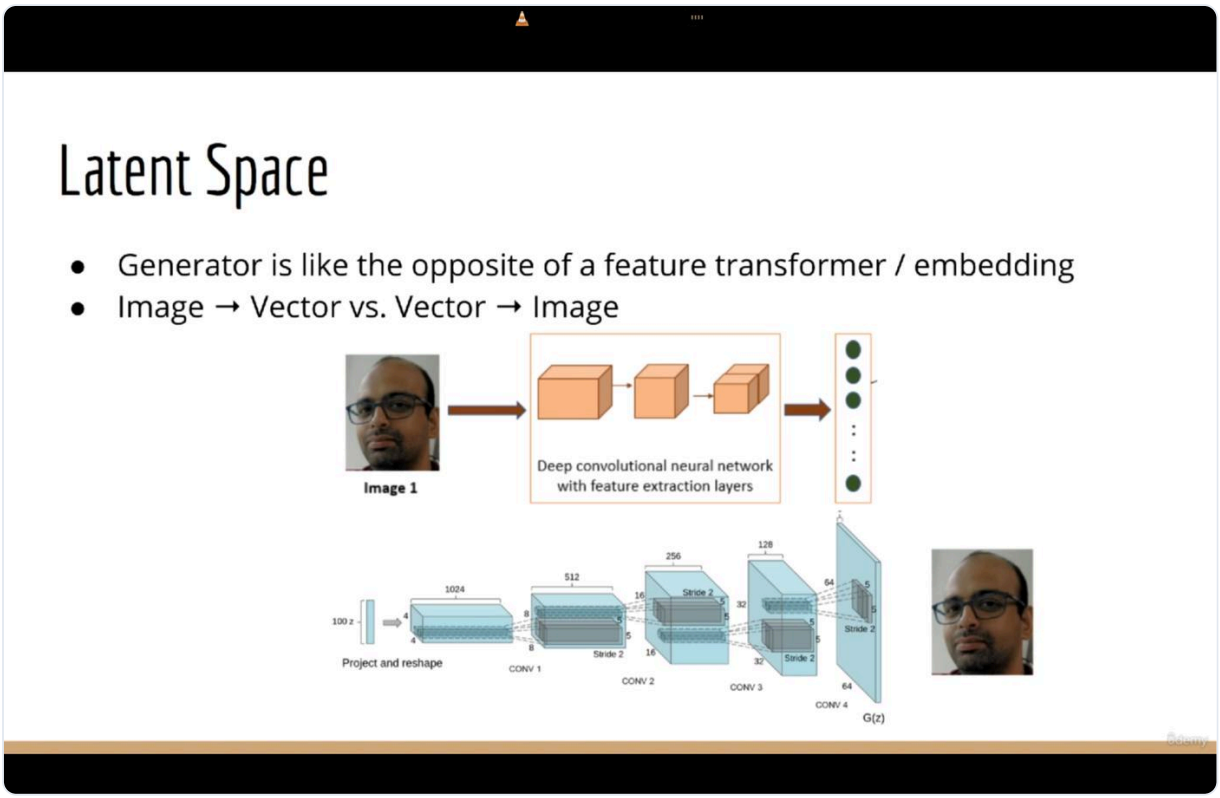


FIGURE 47.15 | Latent Space

We start with a vector and then we map that to an image as always it's going to be helpful to look at some pseudocode before we move on to the actual implementation. So here's how it's going to look. So the first step is the load in our data set. We're going to use MNIST for this example in a future courses. You'll learn about more advanced types of GANs which will work on more complex types of data. Next we're going to create our discriminator and generator networks. These are just standard neural networks some dense layers some ReLUs and so forth.

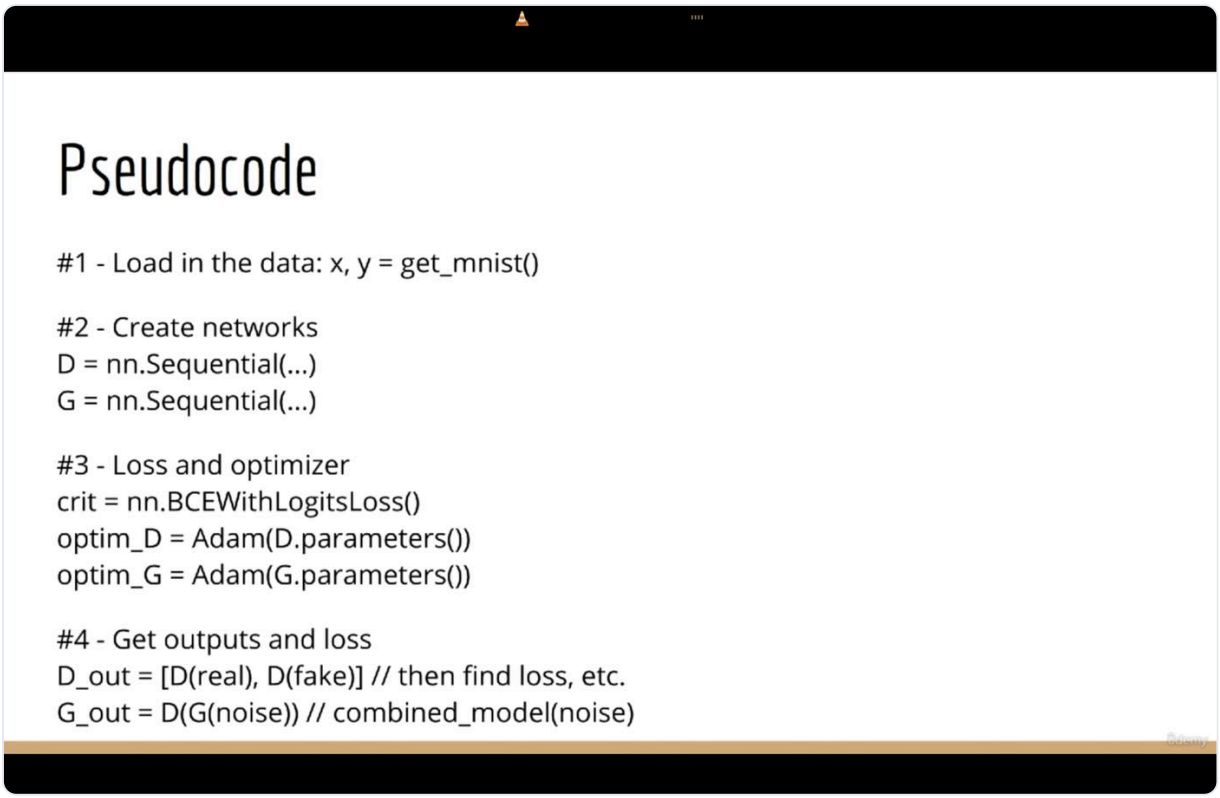


FIGURE 47.16

The trick is we're going to have two optimizers one that optimizes the parameters of the discriminator and one that optimizes the parameters of the generator. Conceptually you can imagine that the loss function is always applied to the output of the discriminator only in either case we will use the binary cross entropy but with flipped labels when we train the discriminator half of the data will be real images from the dataset and half of the images will be generated from the generator. These will use the true labels when we train the generator we will generate random noise pass that through the generator then pass that through the discriminator and use a flipped label meaning that we say it's a one even though it's really a zero we can conceptualize this chain of generator and discriminator as a kind of combined model where only the generator weights our parameters although that's not exactly how we'll look at code.

It's a good conceptual tool. Once we've built our two networks we're going to run a gradient descent loop as usual inside the loop. We're going to train the discriminator and generator in alternating order in order to train the discriminator we'll grab a random sample of images from my dataset and a random sample of images from our generator.

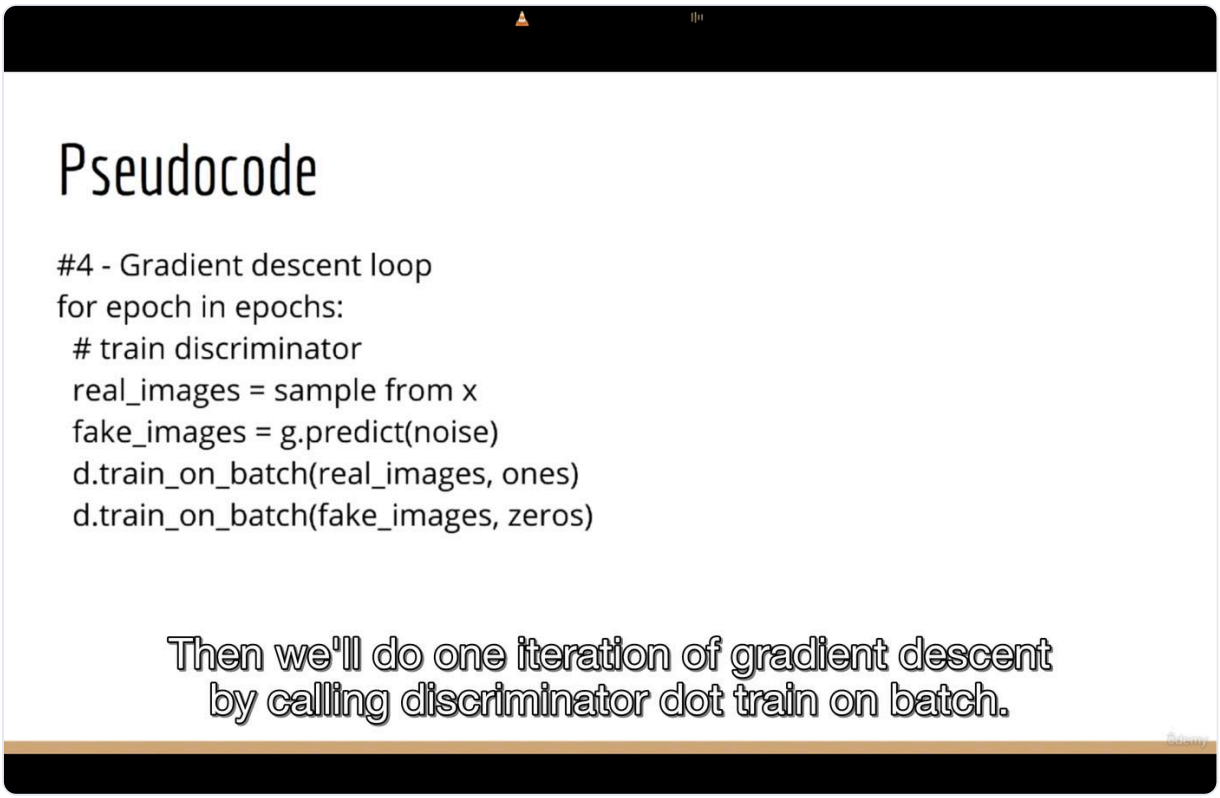


FIGURE 47.17

Then we'll do one iteration of gradient descent by calling discriminator dot train on batch. We do this for both the real and fake images then for generator training we'll do the following.

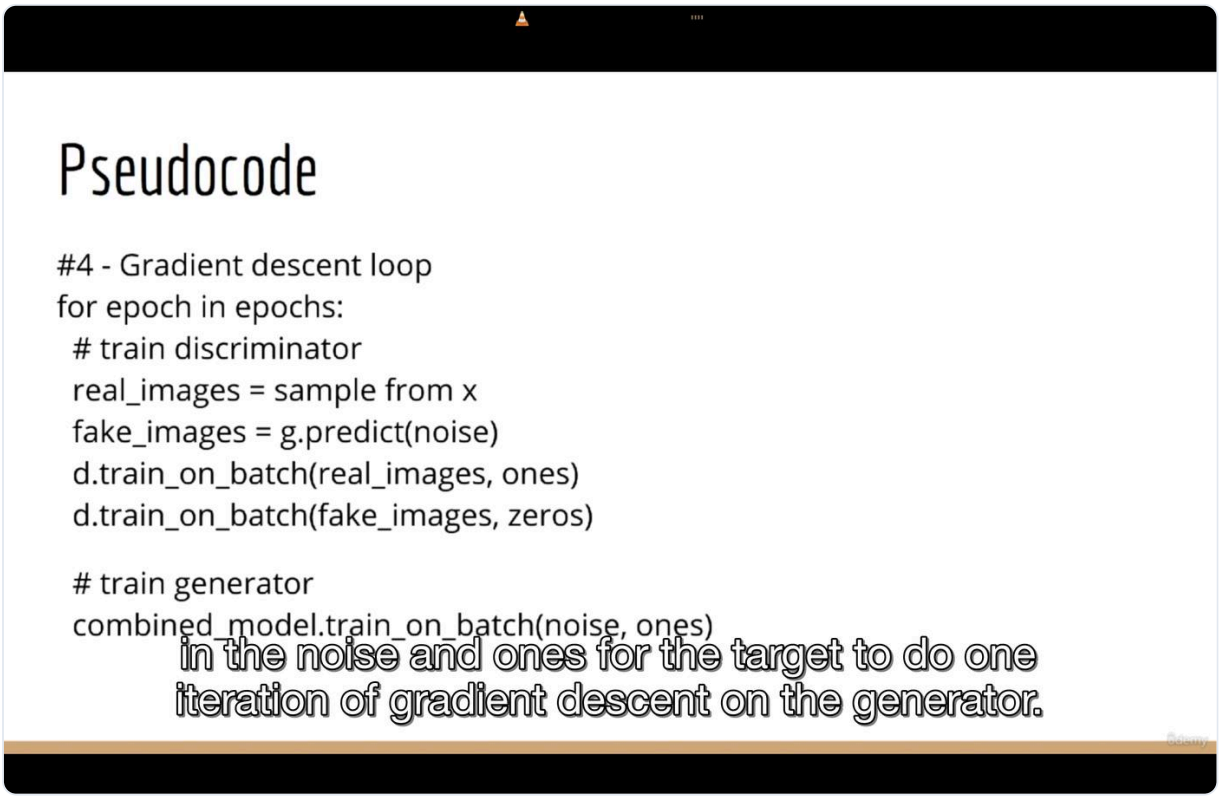


FIGURE 47.18

First of all generate a batch of noise samples then we'll call combined model dot train on batch passing in the noise and ones for the target to do one iteration of gradient descent on the generator. After several thousand iterations our generator should be good enough to produce images that look like the MNIST data set. As a final note for this lecture I want to mention that in terms of model evaluation we're going to take a bit of a departure from what we normally do. You recall that for most models what we're looking for after training is that the loss goes down and the accuracy goes up.

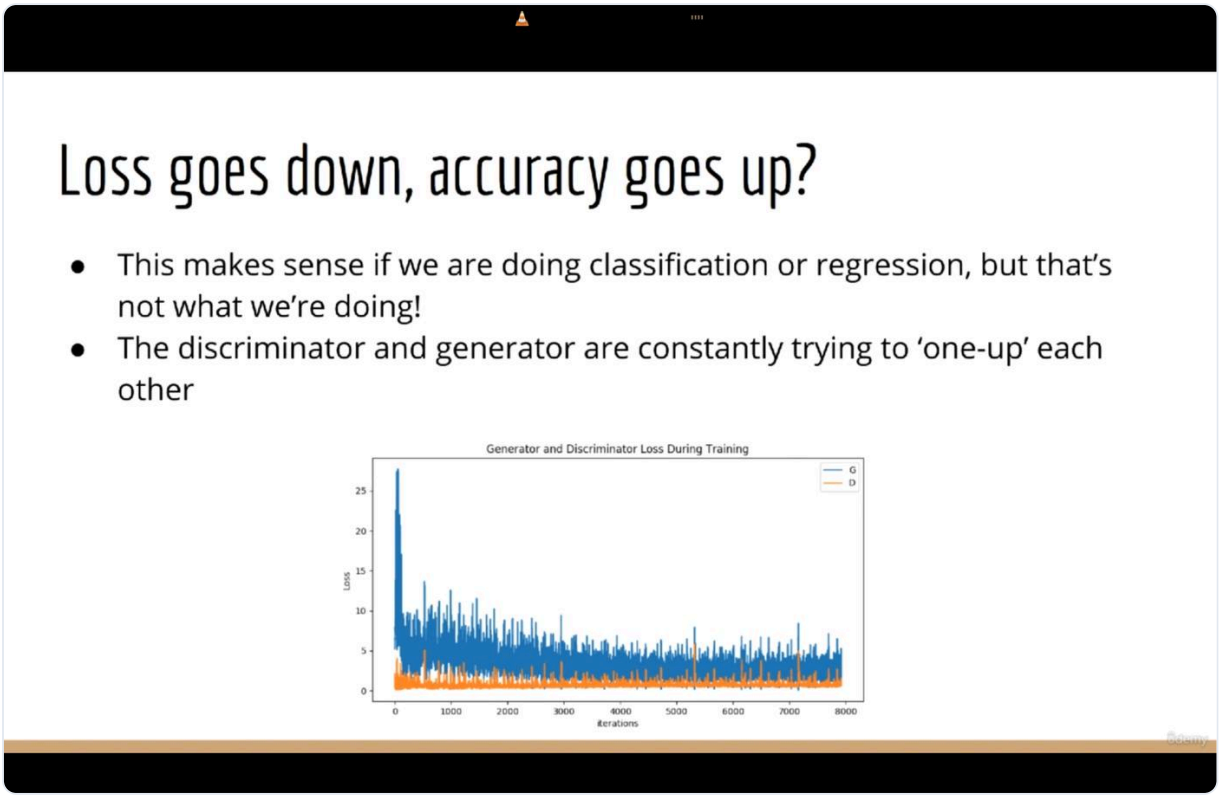


FIGURE 47.19 | Loss Goes Down, Accuracy Goes Up?

This makes sense if you are doing classification or regression but now we are no longer doing classification or regression. Remember that in our case the generator and discriminator are constantly trying to one up each other. And so what is interesting is that the loss functions for both the generator and discriminator should look just like noise. It should appear that if you only look at the loss per iteration that nothing is happening at all. But in fact something profound is happening both the generator and discriminator are improving. You don't see this in the loss per iteration because ideally both the generator and discriminator are improving at the same rate.

Let's now summarize everything you learned in this lecture. First you learn that GANs are not just a single neural network but a system of two neural networks a generator and a discriminator. The discriminator is responsible for telling the difference between a real and fake images. The generator is responsible for creating realistic looking images both the generator and discriminator learned in tandem. The loss function for the discriminator is just the binary cross entropy because there are two classes real and fake the loss function for the generator is not directly applied to the generator but rather a combined model where the generator feeds into the discriminator.

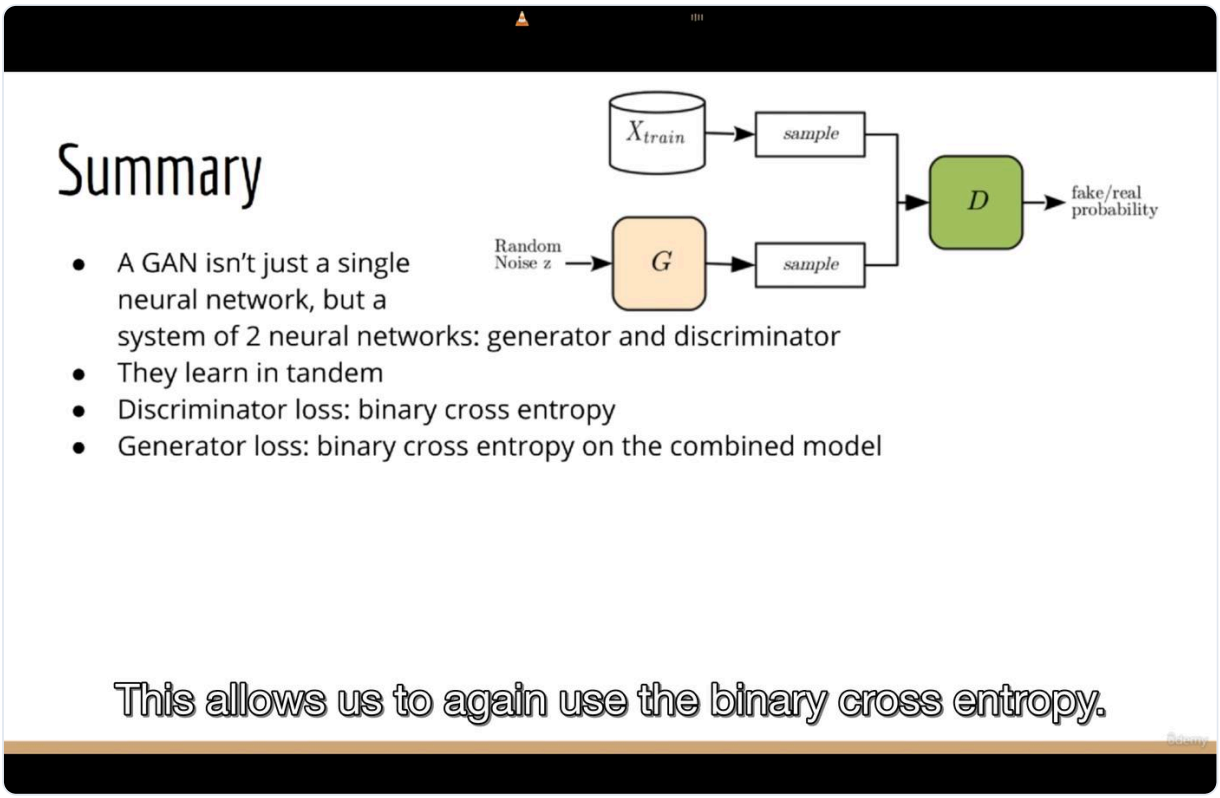


FIGURE 47.20

This allows us to again use the binary cross entropy. But there are two important details first we freeze the weights of the discriminator so that only the generator is trained. Second we flip the labels so that image is generated by the generator are labeled as real. Next we learn that the input to the generator is nothing but noise sampled from the latent space the latent space is where the encodings for each digit live.

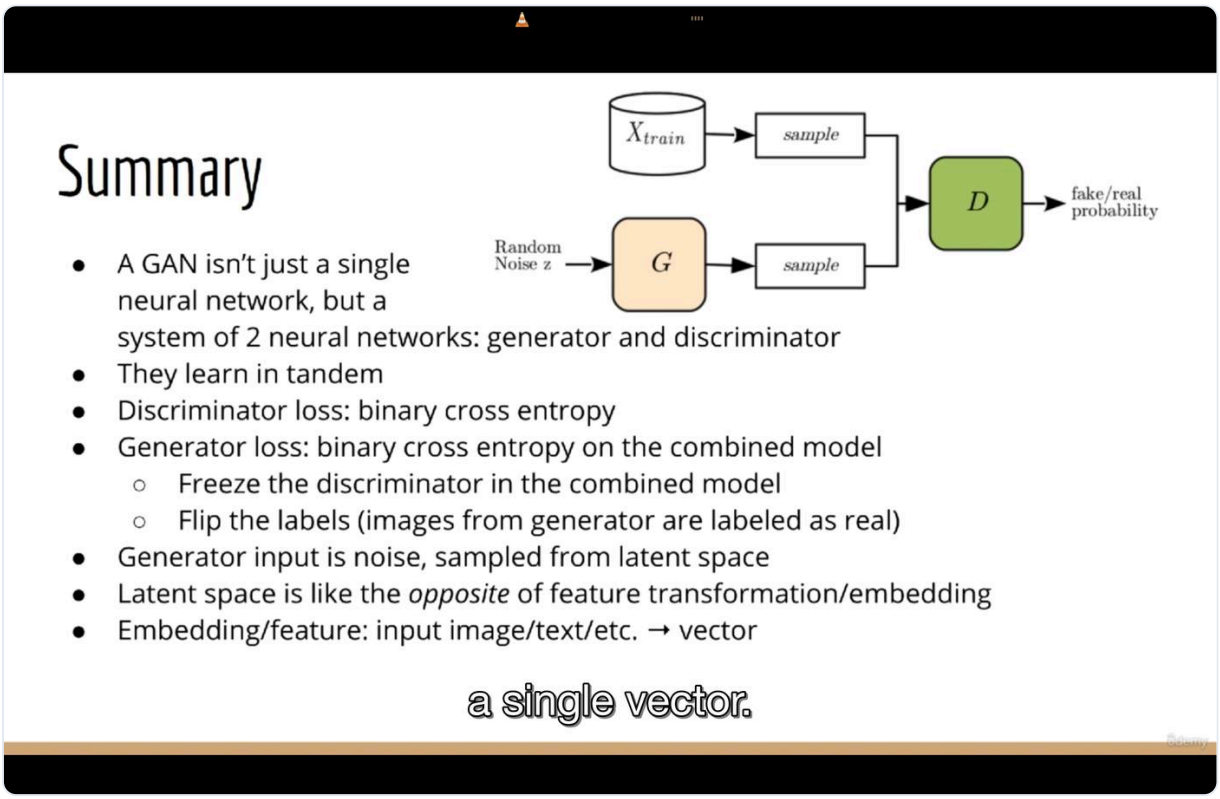


FIGURE 47.21 | A GAN Isn't Just a Single Noise Z

You can think of this as the opposite of a feature transformation or an embedding and embedding or feature transformation is like a mapping from an input image or an input word or any other kind of input into a single vector. The generator does the opposite. It takes a vector which lives in the latent space or the embedding space and produces an image.

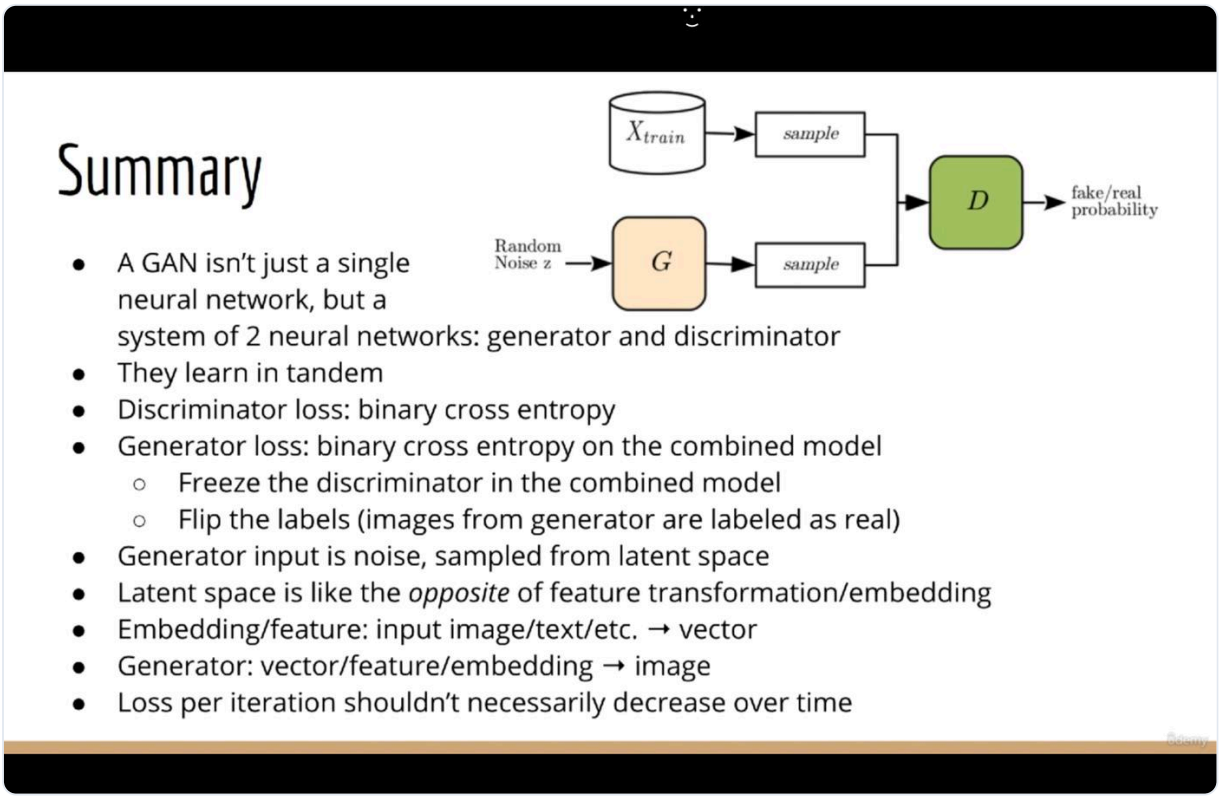


FIGURE 47.22

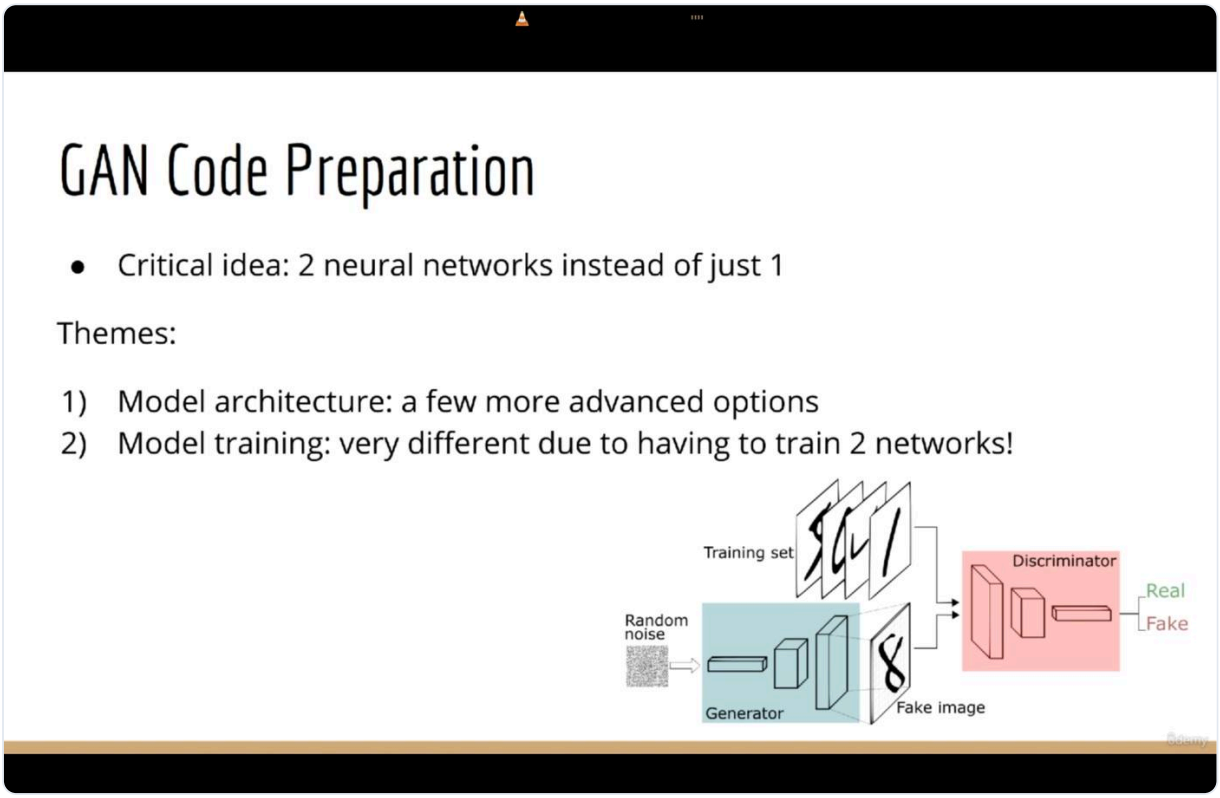


FIGURE 47.23

Lastly we learn that when training GANs the loss per iteration should not be informative in the sense that it should not decrease over time as it does when we are training other models. This is because both the discriminator and the generator should be learning at the same rate continually helping each other improve.

LECTURE 86

Model Architecture

In this lecture we are going to take a closer look at how the GAN code will work and hopefully iron out a few details before we move on to the actual notebook. This is another interesting model in this course because unlike all the other algorithms we looked at now we're dealing with two neural networks simultaneously rather than just one. So it's kind of an odd concept. What I want to focus on in this lecture are a few overarching themes. First we'll look at the architecture of each model. Just for fun we'll use a few more advanced layers and activation functions that we didn't use earlier in the course. Second we'll look at how to train them. This is the centrepiece of this section because unlike all the previous examples you saw in this course this training loop will be very different from what you saw before you may have assumed that because the generator is a kind of a.

Discriminator that they would have opposite architectures. What I mean by that is if the generator had hidden sizes of one hundred three hundred and seventy four that the discriminator would have hidden sizes of seventy four three hundred and one hundred. In fact this need not be the case in our example we'll make the generator a more powerful model than the discriminator. The intuition is binary classification is an easy task whereas generating images is a hard task for the discriminator. All you have to do is map the input to 0 or 1. There aren't that many degrees of freedom for the generator. You have to generate seven hundred eighty four different pixel values and they all have to be coherent.

So there are many degrees of freedom.

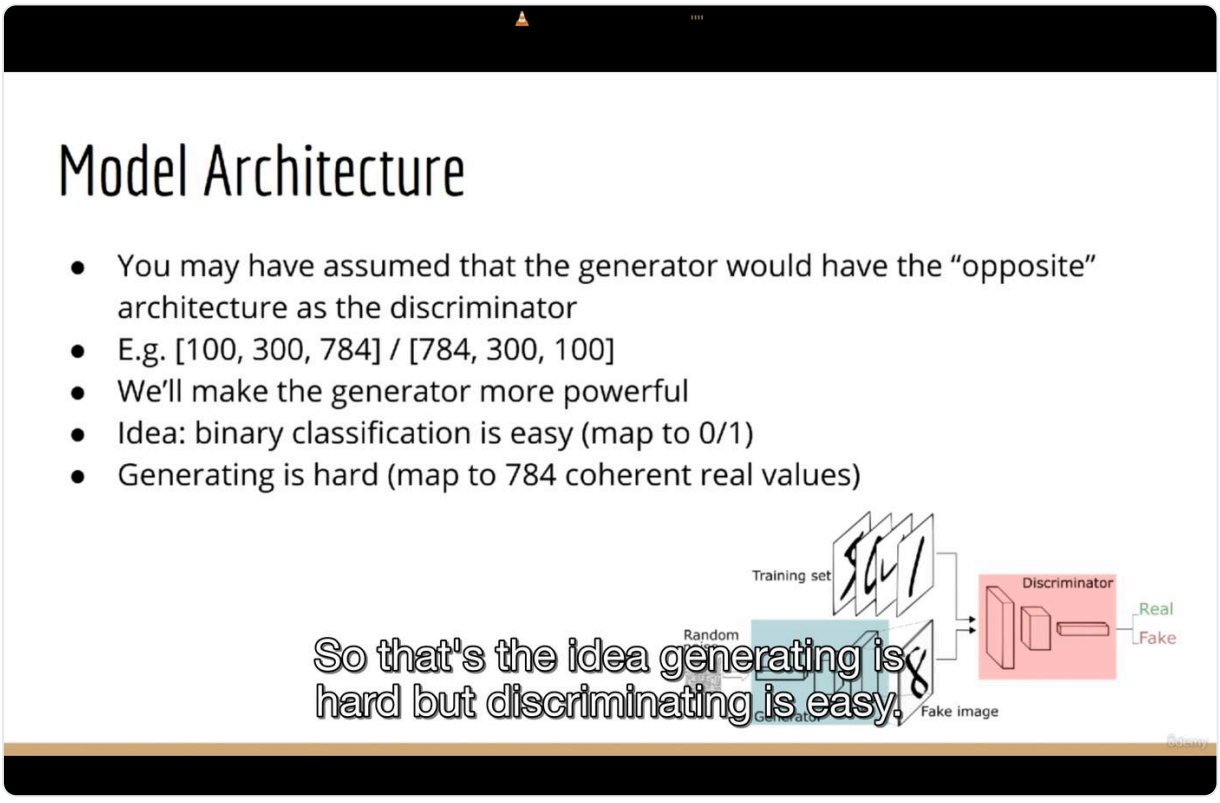


FIGURE 48.1 | Model Architecture

So that's the idea generating is hard but discriminating is easy. So here's what we'll be using for the discriminator although you should feel free to play around and design your own.

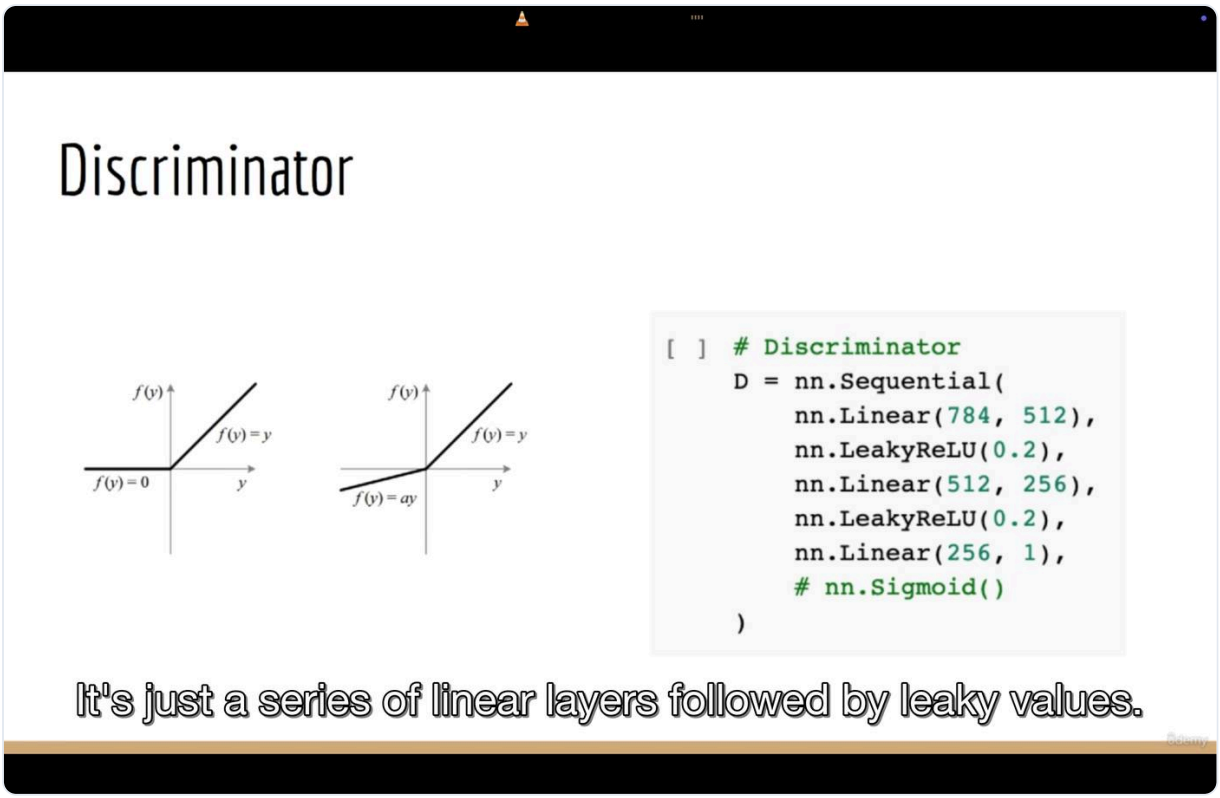


FIGURE 48.2

It's just a series of linear layers followed by leaky ReLUs. Recall that a leaky ReLU is just like a normal ReLU except it has a small positive slope for values less than zero. This is an attempt to avoid the problem of dead neurons where the slope becomes zero and therefore the gradient becomes zero and therefore no error can back propagate here's what the generator looks like. Again feel free to play around. So in addition to leaky ReLU using linear layers we'll also add batch norm to the mix. We've also added a tanh activation at the end which implies that we'll be working with images in the range minus one plus one rather than zero to one experimentally.

Researchers have found that these changes tend to yield better results. OK so here's the real meat of this code. After we define the discriminator D and the generator G we're going to define the loss and optimizer the loss is just the usual binary cross entropy. But we're going to have two separate optimizers one for each network. You'll notice that when we instantiate the optimizers we only pass in the parameters that we want the optimizer to update. So when we call D optimizer dot step it's only going to update the parameters of the discriminator when we call G optimizer dot step. It's only going to update the parameters in the generator inside the training block.

We're going to start by instantiating constant arrays to store zeros and ones. These will act as our targets. So we don't have to keep creating new arrays on each iteration of the training loop inside the training loop. We're going to loop over the data loader. Remember that we don't need the image labels so we can use an underscore to ignore them what we're going to do now is two training blocks one for the discriminator and one for the generator. This slide will focus on the discriminator. Now one question you probably have is does it matter which order you train them in the discriminator first or the generator first. I would recommend you try it out as an experiment so how do we train the discriminator.

FIGURE 48.3

First we start by passing real images into the discriminator. That's just the outputs from the data loader. From this we can calculate the loss using the ones target for the fake images. We start by generating a batch of random noise and then passing that through the generator. From that we get fake images which we then pass through the discriminator. And from that we get fake outputs. We pass the fake outputs through the loss function using the zeros target next. We take the average of the two losses to get a single value. We can optimize. After this we would like to call d loss dot backward and d optimizer dot step. But before that we have to remember to zero the gradients next.

Let's look at the generator training block. One interesting technique discovered by researchers was that doing multiple steps of training for the generator for each single step of the discriminator seemed to work well. This goes along with a theme that the generator is harder to optimize than the discriminator. Therefore it requires more training so to accomplish that we can just do a loop inside the loop. We generate some fake images once again using a batch of latent noise and passing this through the generator. Since this is for training the generator there is no need to use any real images. We just passed the fake images through the discriminator and then passed that output through the loss.

Importantly this time we use the ones target to tell the loss function that we want the weights to go towards these images being classified as real. Next we zero the gradients and do a gradient descent step as usual.

LECTURE 87

Loss and Optimizers

FIGURE 49.1

See the notebook `23.PyTorch-GAN.ipynb` for the code.

In this lecture we are going to look at how to implement a GAN in code. This lecture is gonna walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. So at the top we have all our usual imports. We've also imported the Save image function from torchvision which is a useful utility for saving an entire batch of images to a single image file. This will let us check how our model progresses over each iteration.

Next we define a set of transformations to perform on the data. Remember that researchers have found that using pixel values in the range minus one to plus one tends to work better. So this code might seem strange efforts but if you check the calculation you can verify that it works. First remember that the ToTensor transformation already scales the image to be between 0 and 1. So if we take one subtract a mean of zero point five and divide by a standard deviation of zero point five we get zero point five divided by zero point five which is one if we take zero subtract the mean of zero point five and divide by a standard deviation of zero point five we'll get minus zero point five divided by zero point five which is minus one.

Therefore we map 1 to 1 and 0 to minus 1. So this works. Next we load in the MNIST train dataset from torchvision. You've seen this before. Next we create a data loader from our data set next we create our discriminator network. So this is an ANN with layer sizes 784 512 256 and 1 and with leaky ReLU activa-

tions. Next we create our generator network. This is an ANN with input dimension one hundred and hidden dimensions 256 512 1024 and 784.

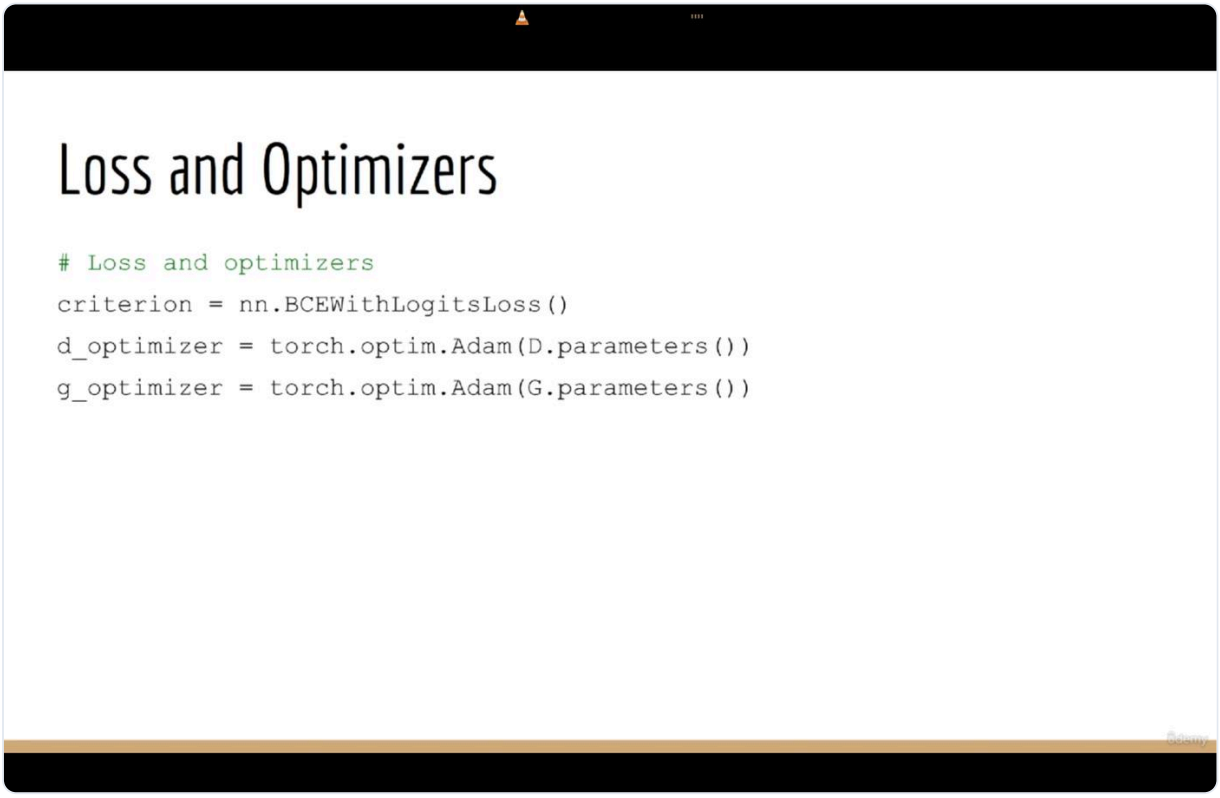


FIGURE 49.2 | Loss and Optimizers

Notice how we've also added a batch norm after each leaky ReLU and that the neural network ends with a tanh activation which maps the output values to be between minus 1 and plus 1. Next we set the device to be the GPU and we move both the discriminator and generator to the GPU. Next we create a loss and optimizers for the loss we can use the binary cross entropy for both cases. The only thing that changes is the label for the optimizer as we make sure to pass in only the parameters for that specific network. So the D optimizer only optimizes the parameters in the D model and the G optimizer only optimizes the parameters in the G model. Next we have a convenience function for scaling images back from minus one plus one to zero one.

This is useful for plotting and saving and so forth. Next we create a directory to store the images generated during training is called GAN images. Next we have the training area. So this will be slightly more complex than the code preparation section. Since I left out a few minor details but hopefully you understand the main idea and so these minor details are just that minor. Sort of start we're going to create one dimensional arrays just containing ones and zeros. These will be used as targets during training. The targets are constant values so instantiating these inside the loop would be wasteful. Next we create two lists to store the losses for the generator and discriminator.

Next we enter a loop that will go for two hundred epochs and inside that loop we have another loop to go over the data loader notice we ignore the targets by using an underscore since they are not used inside the loop. The first thing we do is get in the batch size. Remember that since the batch size may not divide evenly into the total number of samples the actual batch size may not be equal to the specified batch size. Next we take the inputs and flatten them into an N by D matrix and move it to the GPU and now that we have N which is the current batch size we can also index the ones array and the zeros array to get the correct size targets next we have the train discriminator block.

The first thing we do is get the loss for the real images so we pass the inputs through the D model and call that real outputs. Then we pass real outputs into our criterion with ones as the target.

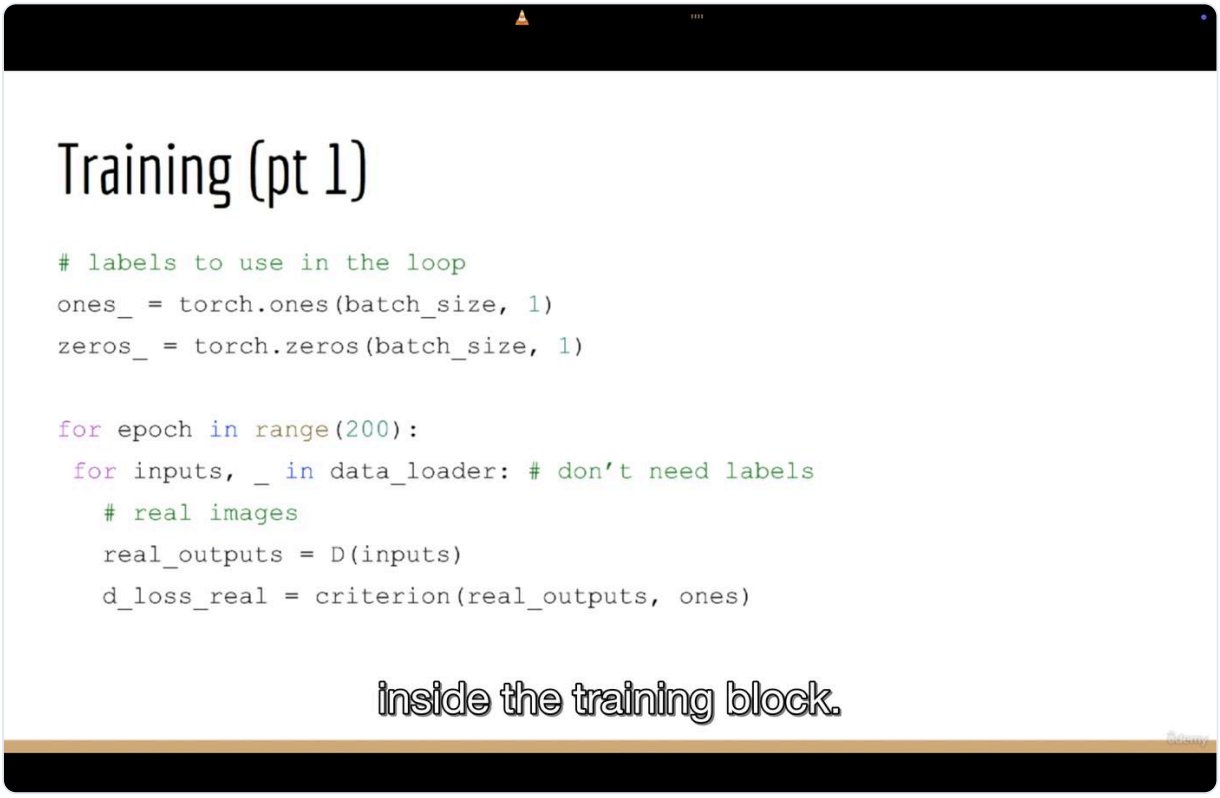


FIGURE 49.3

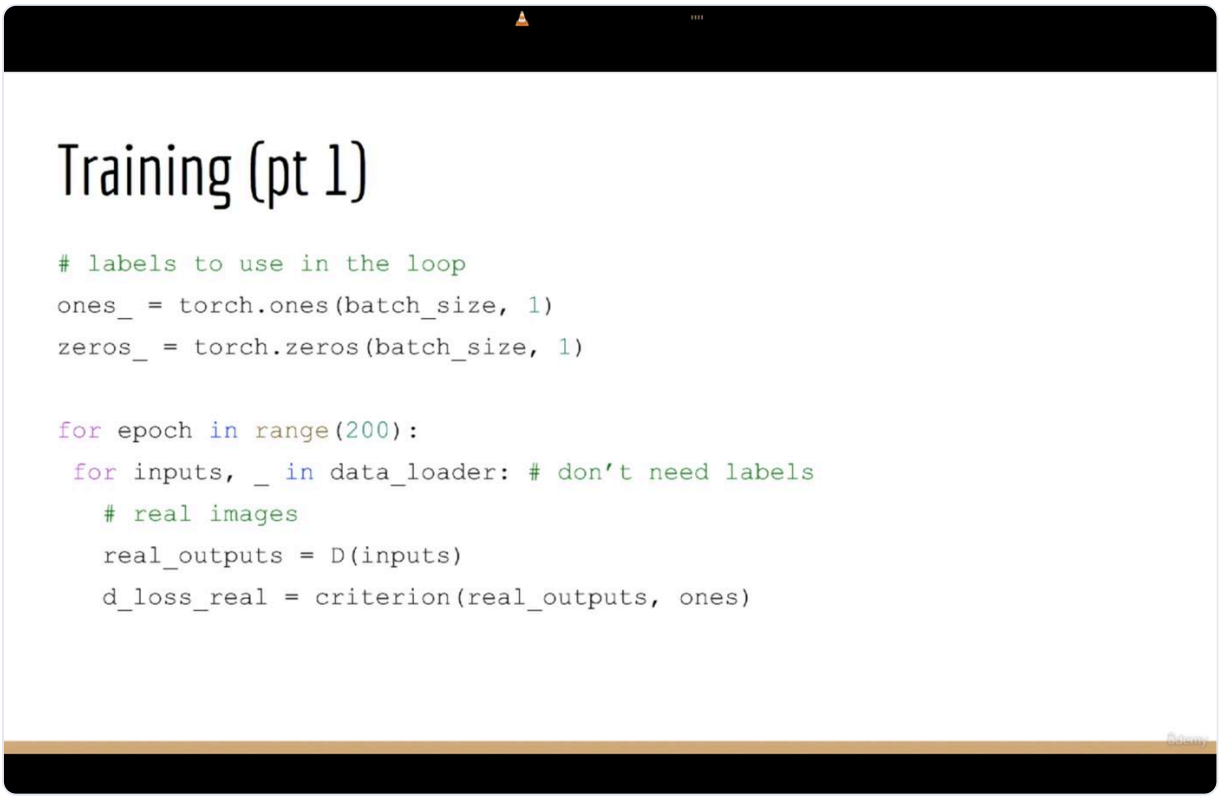


FIGURE 49.4

We'll call this D loss real. Next we calculate the loss for the fake images. So we start by generating some noise which goes in a matrix of size N by 100 we move that to the GPU. Next we create our fake images by passing this noise through the generator. Next we pass the fake images into the discriminator and we get the fake outputs. We pass the fake outputs into our criterion with zeros as the target.

FIGURE 49.5

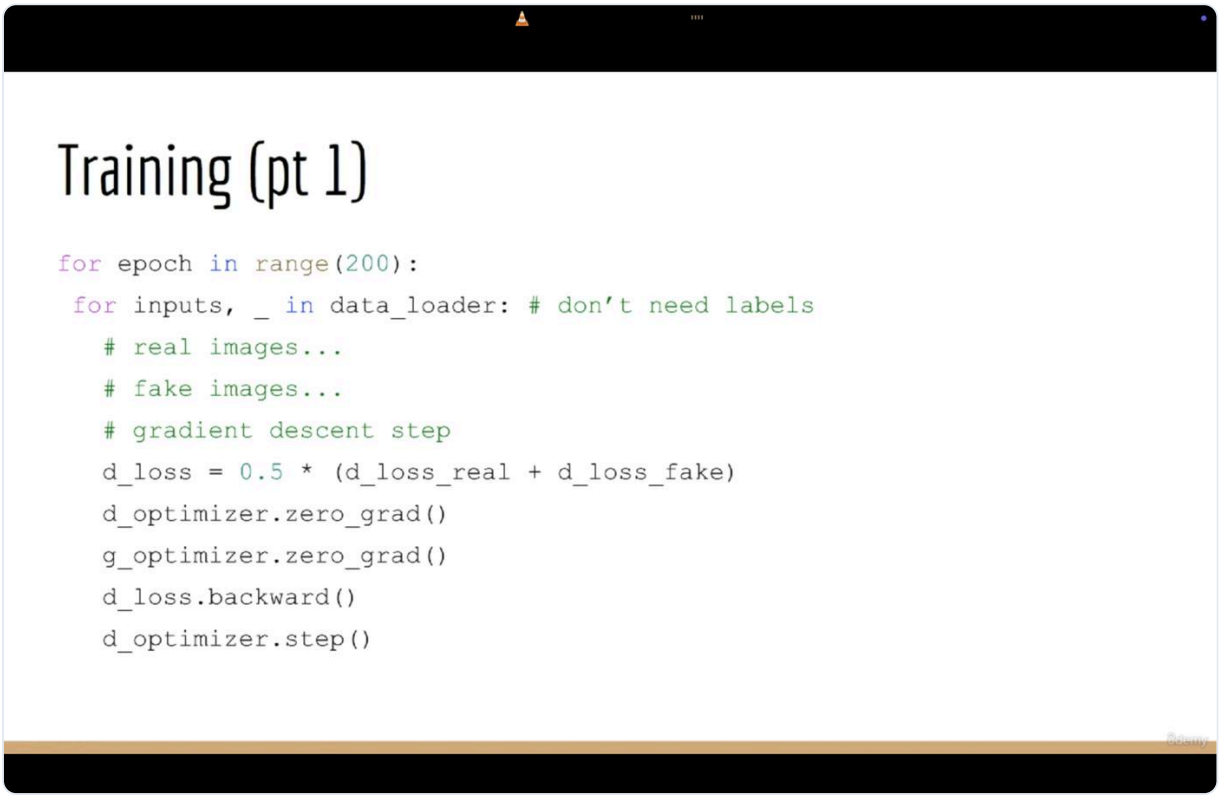


FIGURE 49.6

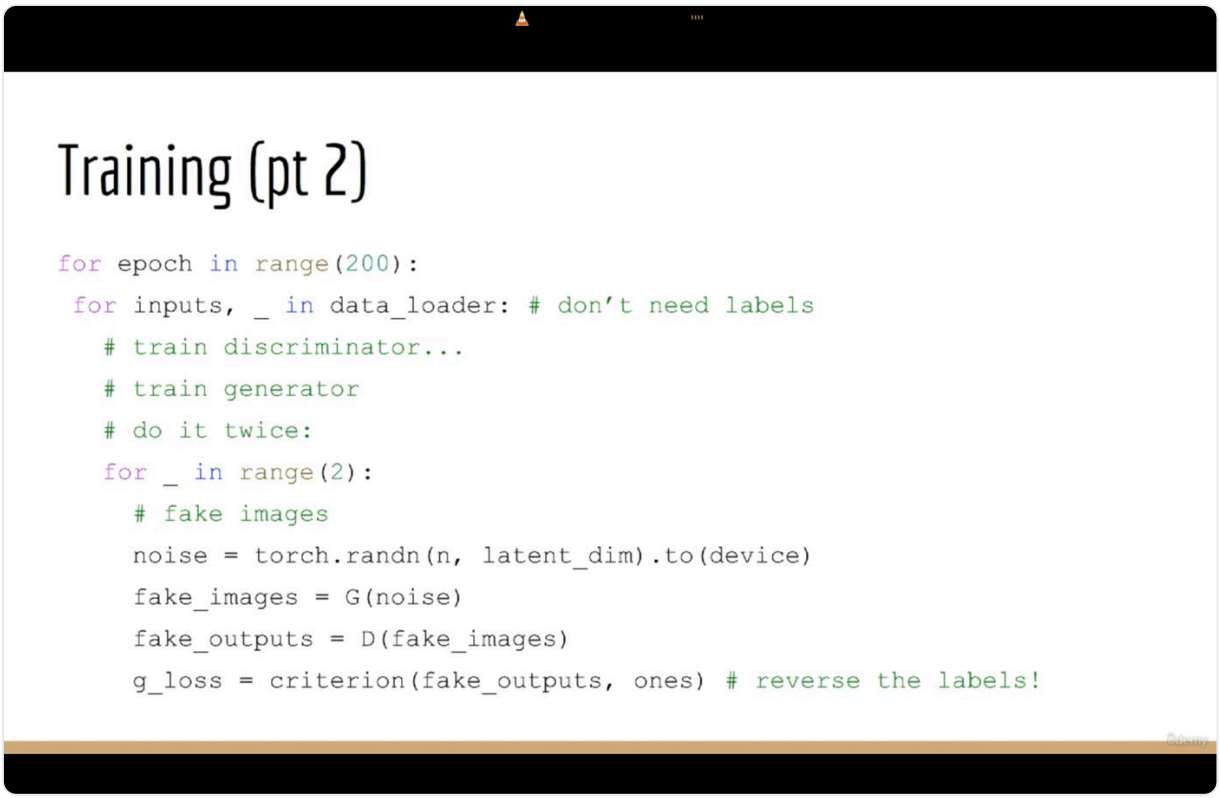


FIGURE 49.7

FIGURE 49.8

We'll call this loss D loss fake. Next we combine the losses by taking the average then we zero the gradients in both optimizers. Then we call the loss dot backward and then D optimizer dot step to do one step of gradient descent on the discriminator. Next we have the train generator block. As mentioned it's been found that training the generator more than once for each iteration is useful. So we do a for loop that

goes twice inside the loop. We generate some random noise of size N by one hundred and move that to the GPU. Next we pass this to the G model to get a set of fake images. Next we pass the fake images through the D model to get a set of fake outputs.

Next we pass the outputs through the criterion this time using ones as the targets. This is because we would like the discriminator to think these are real. Next we do a gradient descent step. First we zero the gradients. Then we call G loss dot backward and then we call G optimizer dot step and remember that even though the outputs that go into the loss function are outputs from the discriminator we are running G optimizer which only knows about the G models parameters so only the G models parameters are updated. Finally we save the losses to our list of losses. We print the epoch number and the losses and then at the end of each epoch we save the fake images that were last generated using the Save image function that we imported earlier.

Note that before we save the images we have to scale them back to the correct range using the scale image function OK. So let's check out the loss per iteration so as promised it's not the nice downward converging pattern that you're used to. Okay so let's look at the images we generated so here are the fake images after epoch 1. We can see that at this point they are already starting to resemble the MNIST dataset after epoch 50 it pretty much looks like MNIST. After epoch 100 it looks even more like MNIST after epoch 150. It still looks like MNIST maybe even a little better and after epoch 200 it still looks like MNIST. So as you can see the results are pretty much what we expect at the beginning.

We didn't see anything at all. And by the end it looks pretty much like MNIST. The generator learns to generate real looking digits quickly so there's an aspect of diminishing returns here. After the first third of the way through things already look pretty good.

LECTURE 88

Reinforcement Learning

In this section of the course we are going to discuss the theory behind deep reinforcement learning this lecture.

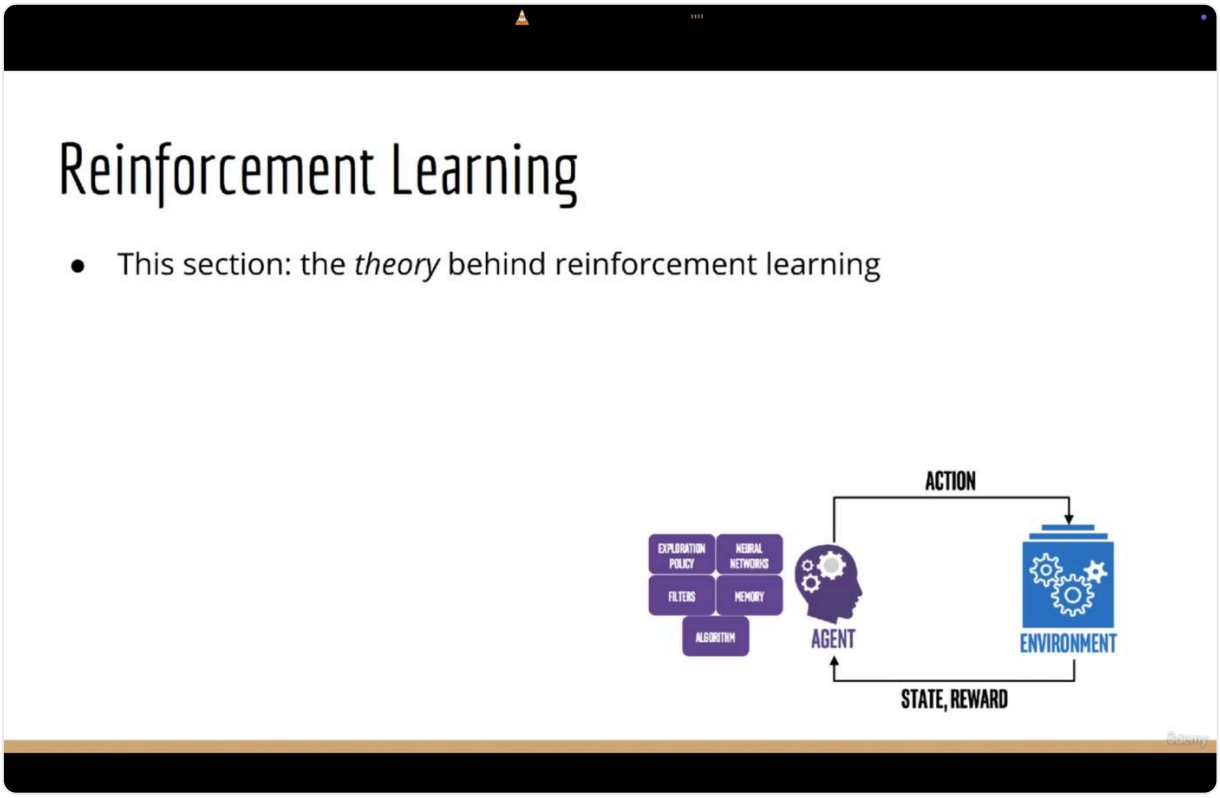


FIGURE 50.1 | Reinforcement Learning

We'll give you an introduction to reinforcement learning and we will talk about it in general terms without any math or terminology one thing you have to brace yourself for when you're learning about reinforcement learning is how different it is to supervised and unsupervised learning.

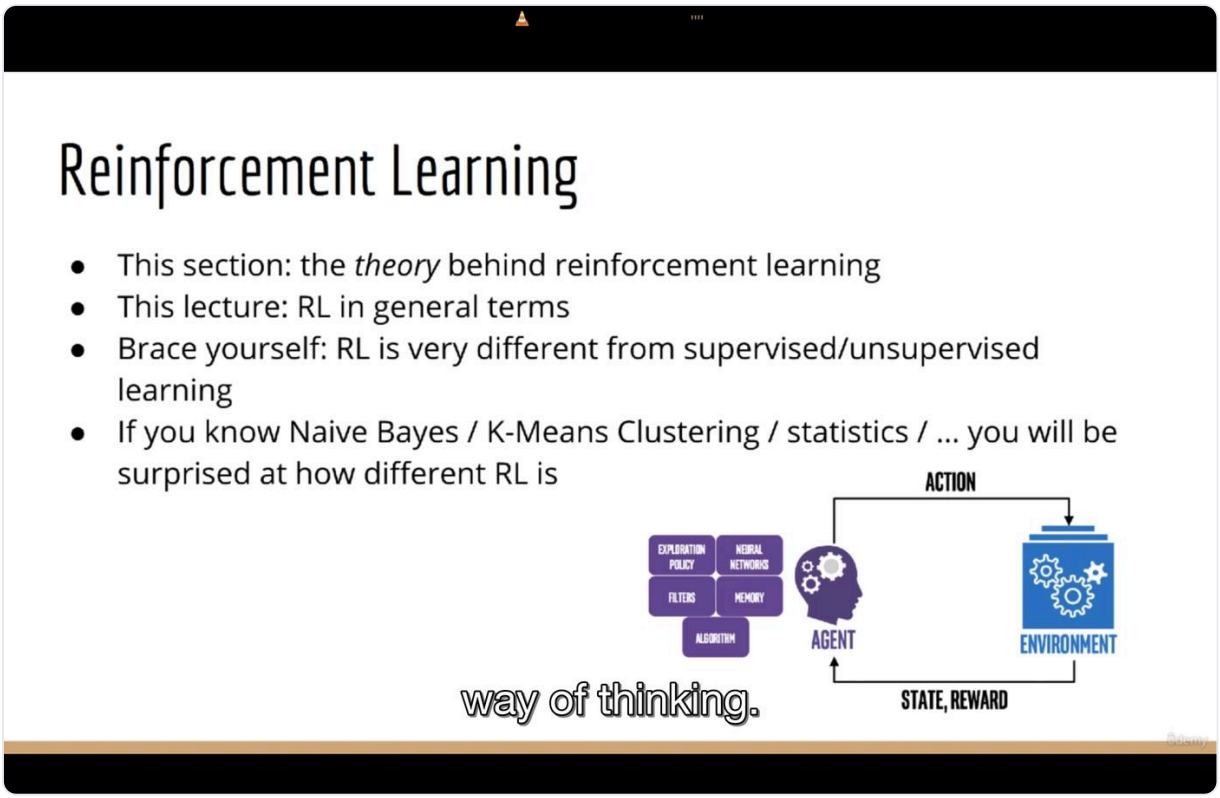


FIGURE 50.2 | Reinforcement Learning

So if you just took an introduction to machine learning where you learned about models such as naive bayes and K Means clustering or you come from a statistics background you will be surprised at how different reinforcement learning is compared to what you are used to. So try to take some time to soak up these concepts and don't feel intimidated by this new and different way of thinking. I want to begin by thinking about supervised learning when we think about say an image classifier. You can think of it as a static function. I pass on an image and I get a prediction. It tells me what kind of object is in the image.

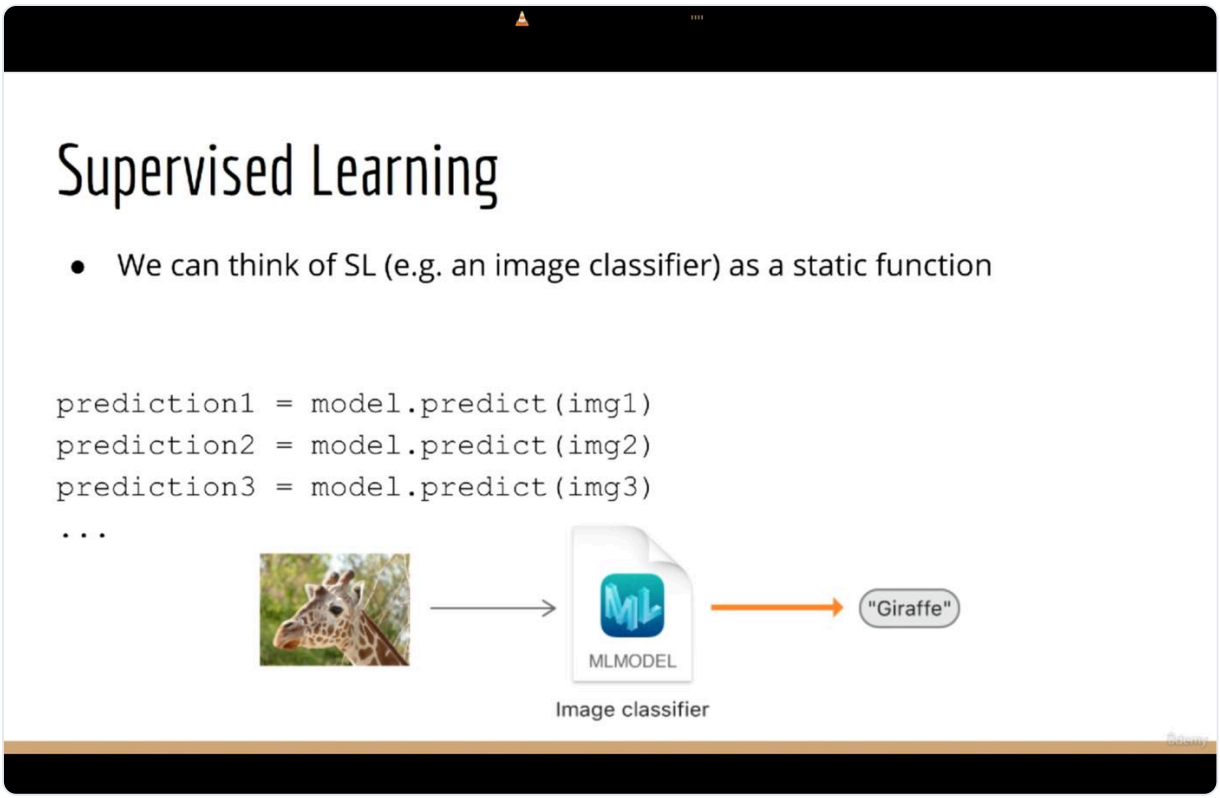


FIGURE 50.3 | Supervised Learning

For example there is no notion of time I pass in another image. I get another prediction. The image classifier is just a function I given an input and it produces an output so what do I mean by static. And what do I mean by time. You might immediately think of recurrent neural networks which are neural networks that can handle sequences inputs that vary with time. However this is not the kind of time I am thinking about if I pass in some stock prices for a given time period and my model predicts whether the stock will go up or down tomorrow.

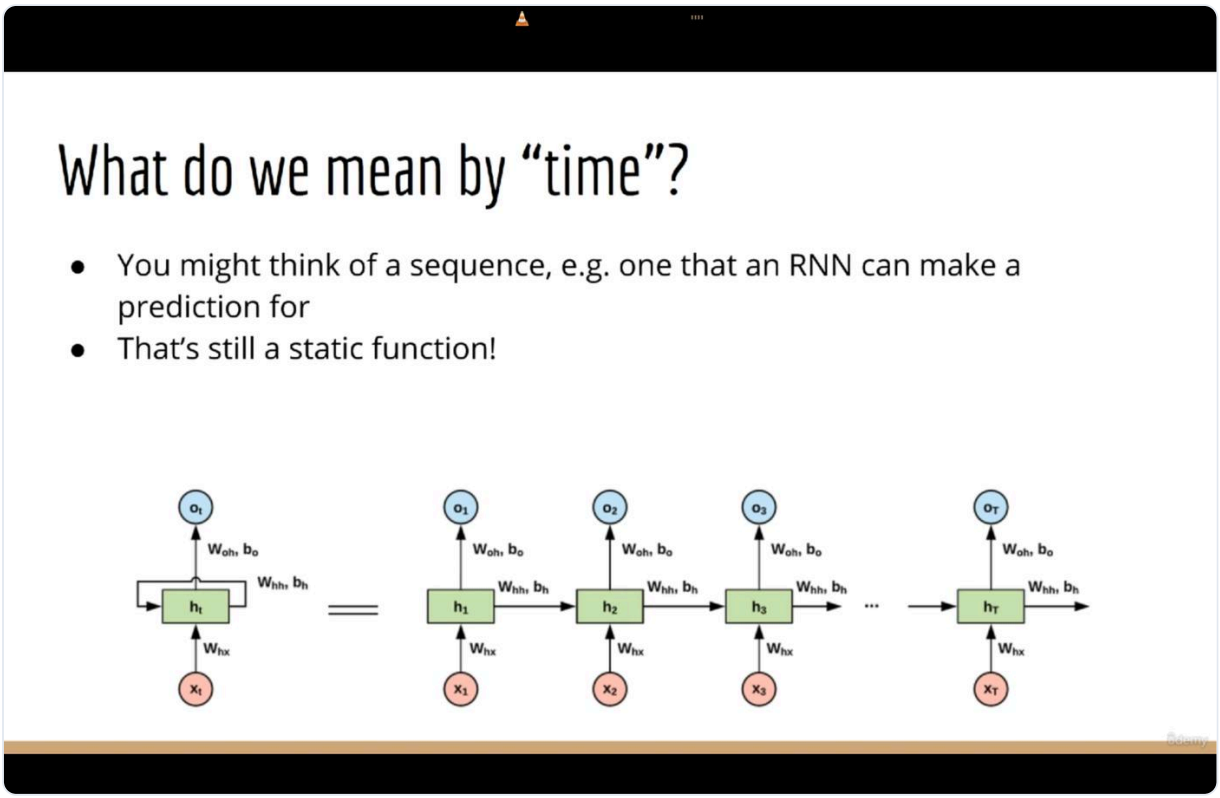


FIGURE 50.4

That's still a static function. Here's what I mean by time.

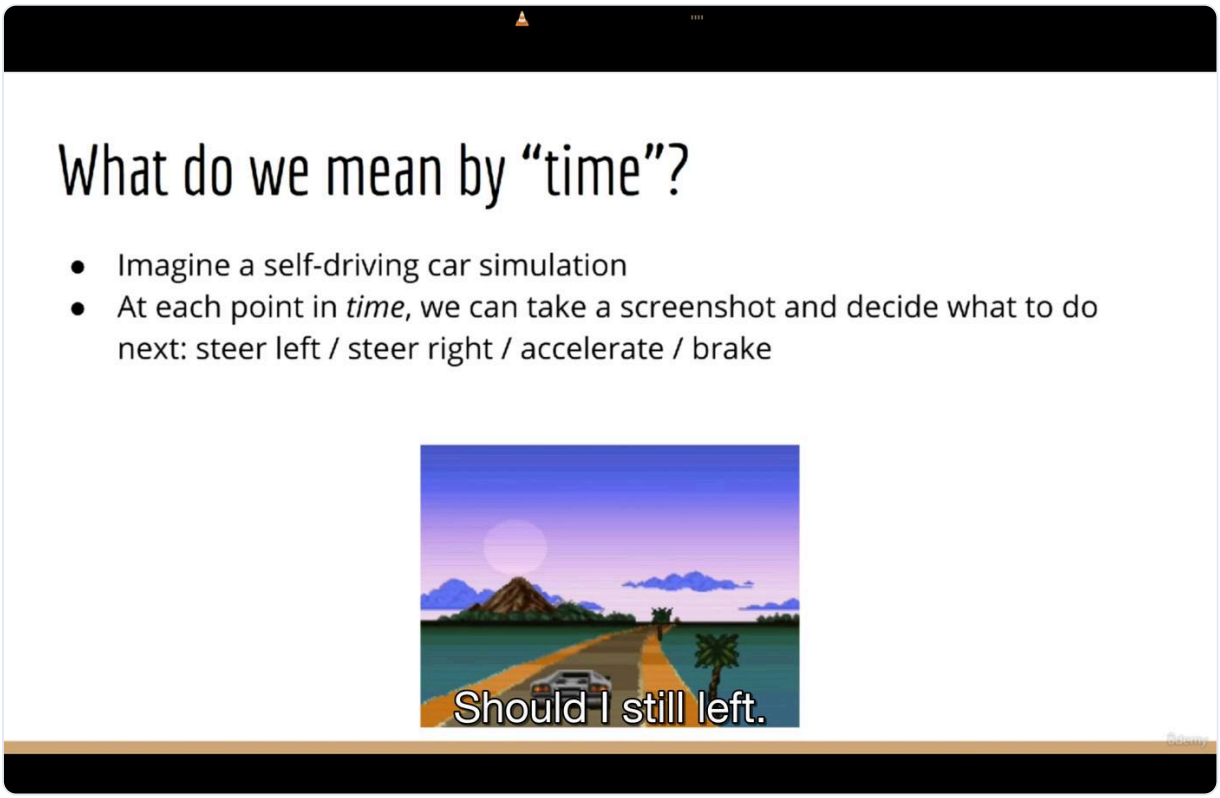


FIGURE 50.5

Imagine you are building a self-driving car simulation at each moment in time your neural network can take a snapshot of the screen and decide what to do next. Should I steer left. Should I steer right accelerate or brake. So this is the difference between a supervised and reinforcement learning supervised

learning is just a function you can call this function repeatedly but it's still just a function you pass in an image and it produces an output reinforcement learning is more like a loop. It exists to achieve some goal. For example driving you to your desired destination inside the loop yes it still takes in an image and produces an output that specifies how to control the car but importantly this reinforcement learning program has the concept of time.

It doesn't think about just what is this image.

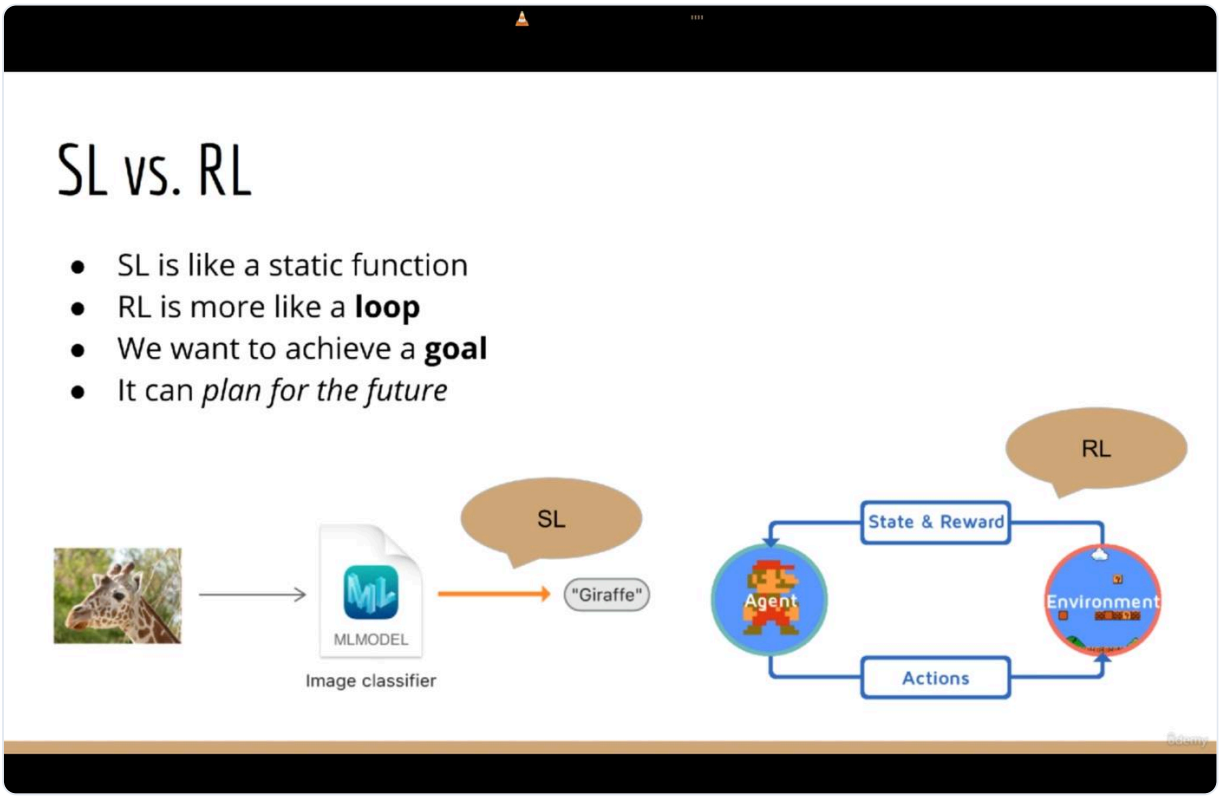


FIGURE 50.6

How do I translate this image into an output prediction. Instead it has the capacity to plan for the future even though at this moment the car may only see where it is on the road right now. It knows that there is some sequence of actions it must take in the future that will lead it towards its goal. All right so that's the major difference between a supervised learning and reinforcement learning with supervised learning. We have no concept of a goal or the future or planning. We just take an input and produce an output. It's a static function. With reinforcement learning we have a plan and that plan can be carried out in the future to reach some predefined goal here's another way to think about reinforcement learning.

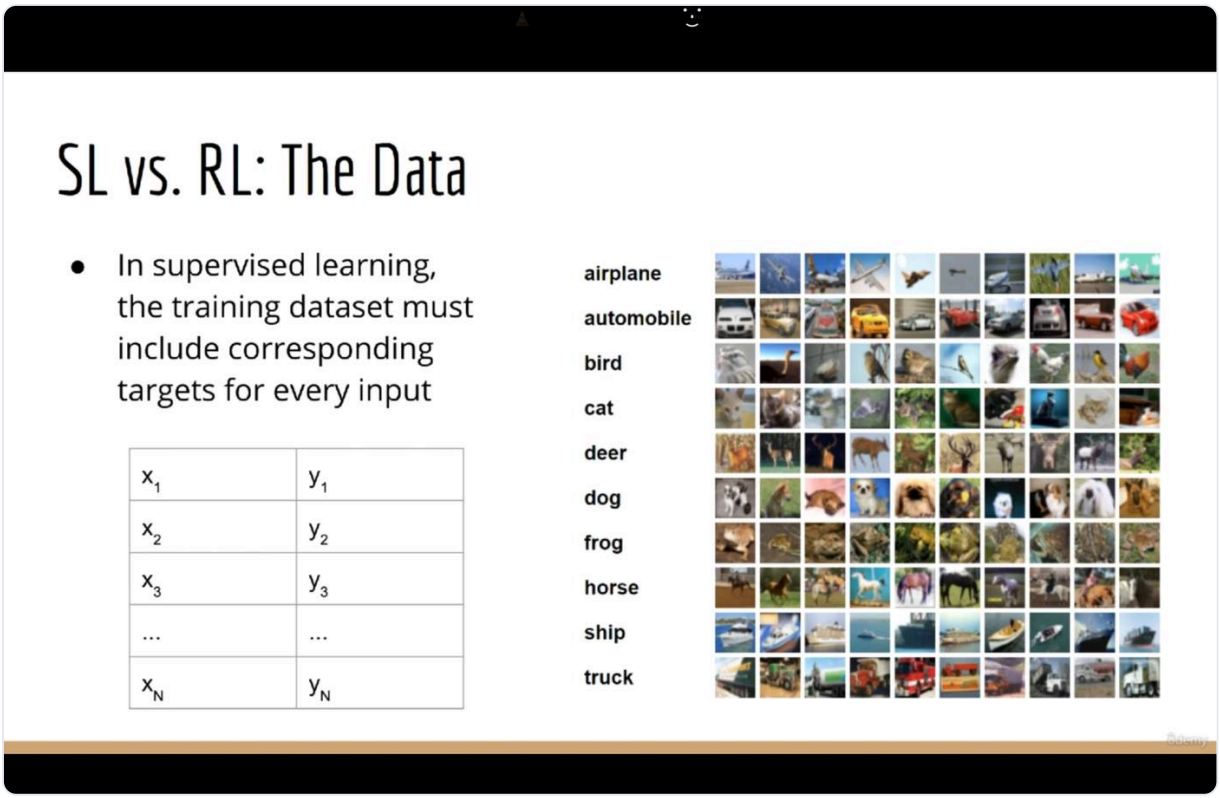


FIGURE 50.7 | SL vs. RL

Think about the data with supervised learning. Again let's use image classification as our example. We must have a label for every input in our training set. So if we have an image of a dog we must have another item specifying the class dog. If we have an image of a cat we must have another item specifying the class cat. In other words for every x we must have a Y it's important to remember that label data sets must be made by humans. Sometimes students have this really funny idea that we should just automate the creation of labeled datasets guys.

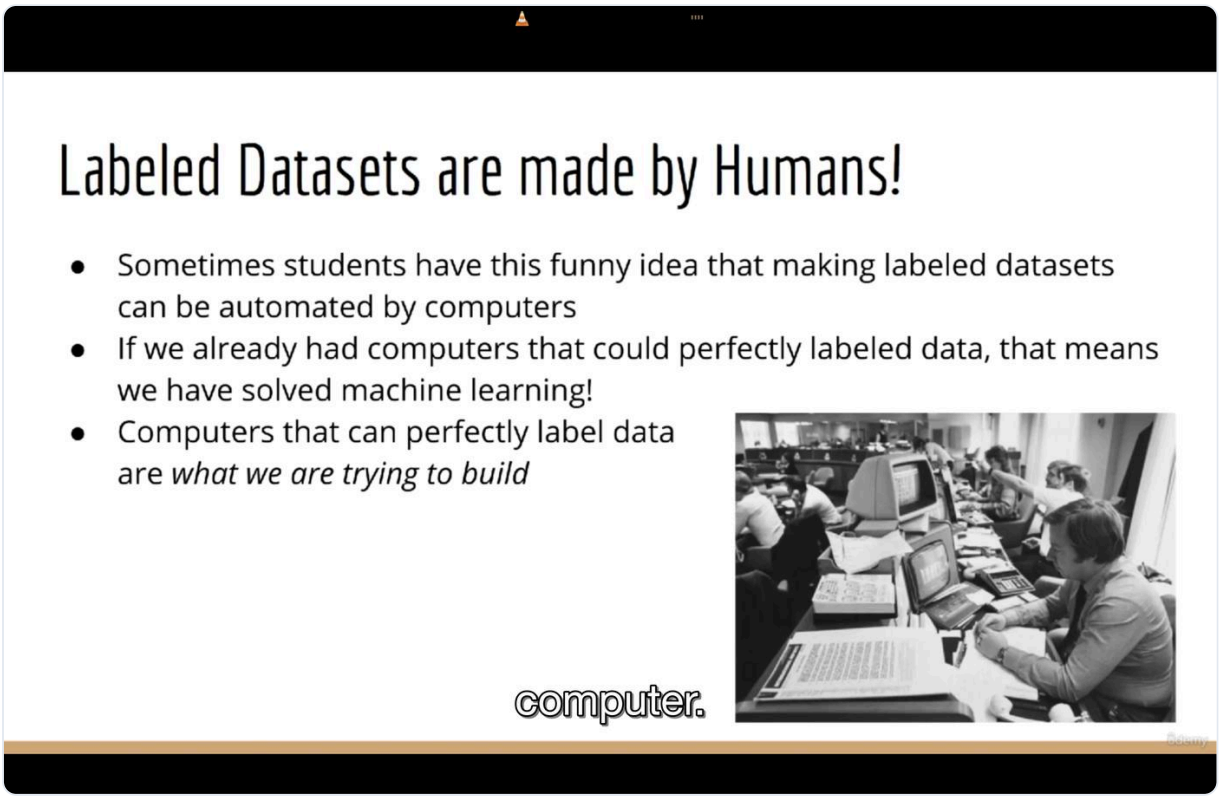


FIGURE 50.8

If we already had computers that could perfectly label data then that would mean we have already solved machine learning computers that can perfectly label data are actually what we are trying to build. If such computers already existed then we would not need to build them. All right so hopefully you are convinced that label data comes from humans and not some super smart computer. Why is this important. Well think about our self-driving car again. Take this image of the road. If we were to use supervised learning for this data point we would need to give it a target. But what's the target. Should I steer left. Should I steer right. Should I accelerate. Should I brake.

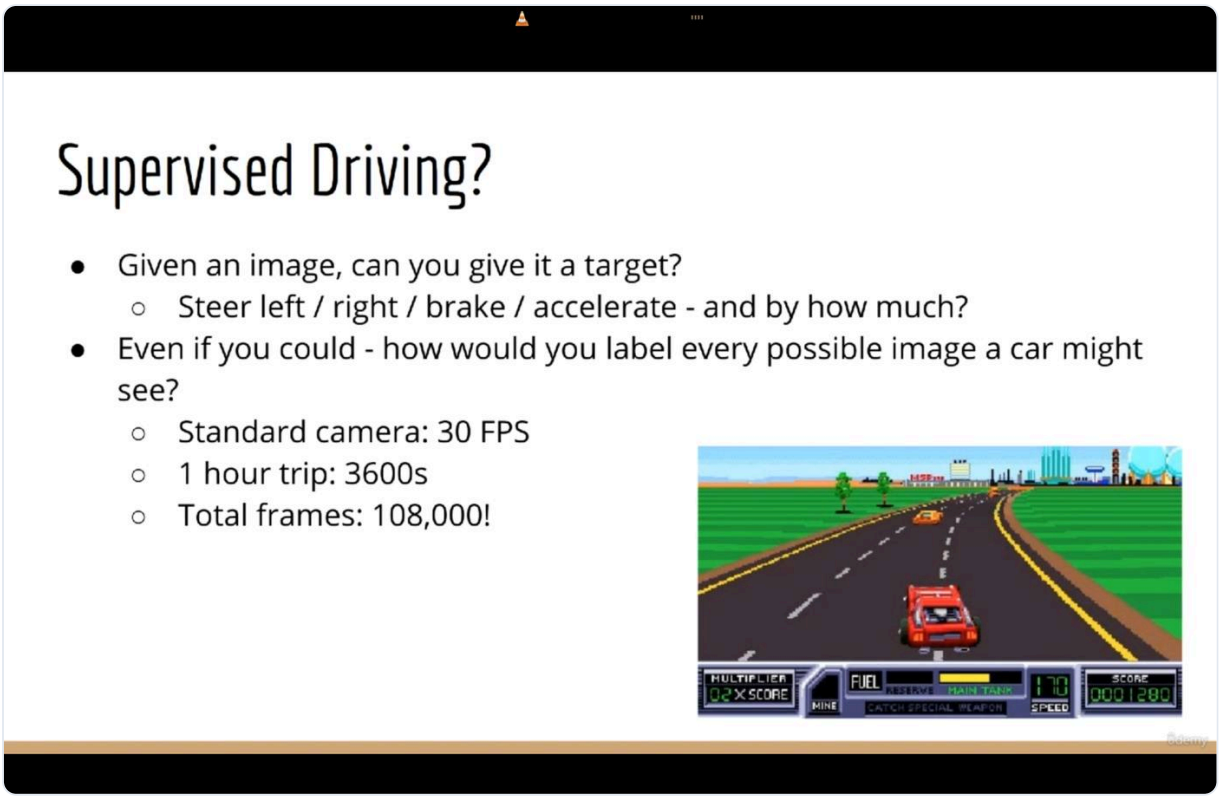


FIGURE 50.9 | Supervised Driving?

In fact you probably cannot give this image a target and even if you could how could you possibly label every single frame that the car will encounter along its journey. Imagine if you have a standard camera that captures images at 30 frames per second and you have a one hour drive as your data set. One hour is thirty six hundred seconds which means you would have to label one hundred eight thousand images from just one trip instead. Reinforcement learning learns using goals rather than targets. For example suppose you wanted to teach a reinforcement learning algorithm to solve a maze.

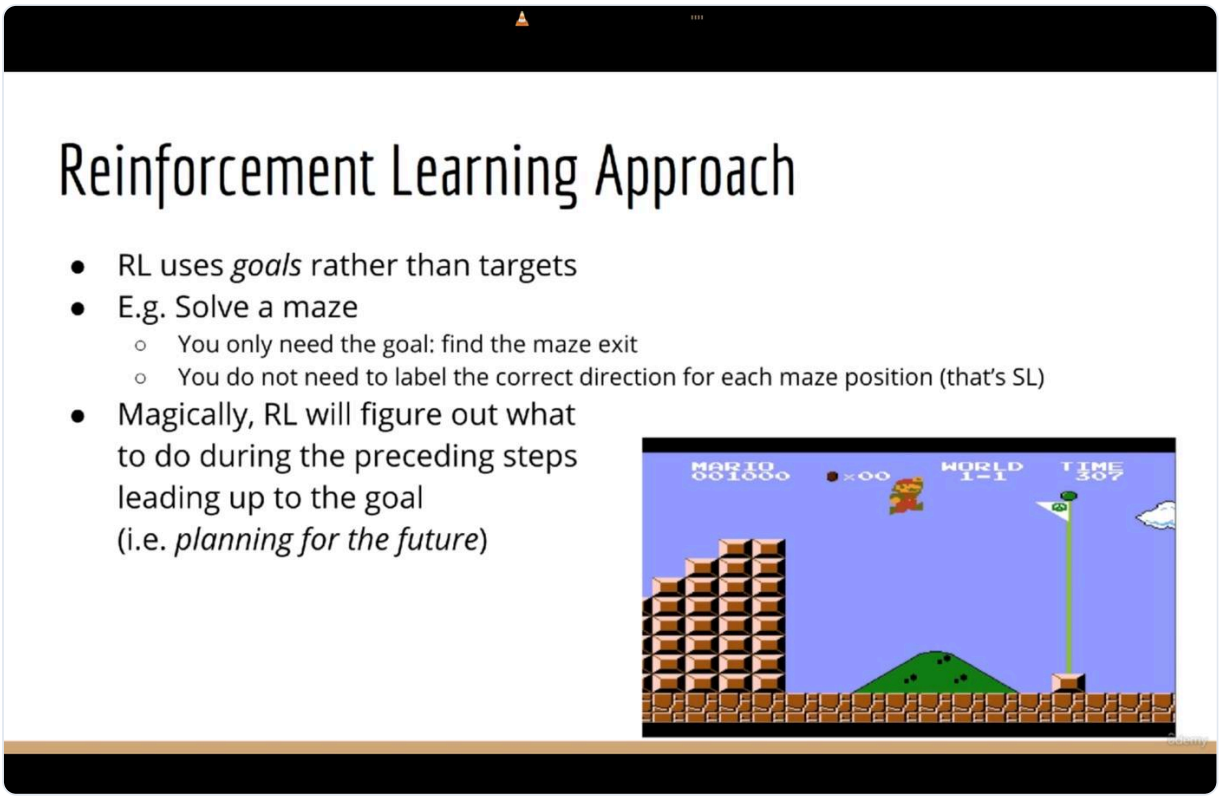


FIGURE 50.10 | You Only Need the Goal: Find the Maze Exit

In this scenario the goal would be finding the maze exit. You do not need to tell the algorithm what the correct thing to do is for each position on the Maze. Since that would be supervised learning instead the only thing the reinforcement learning algorithm needs to know is what is the goal from there. Figuring out what to do in each position of the maze can be found by reinforcement learning. That is the power of this new paradigm.

LECTURE 89

Large State Spaces

In this lecture we are going to discuss reinforcement learning from a more technical standpoint and this will allow us to define most of the terminology involved in reinforcement learning problems. I'm a big believer in learning by example. So in this lecture it's not going to be so much about abstract and technical definitions as it is about providing examples of everything. Let's start with the main objects in a reinforcement learning problem. The agent and the environment. The best example of this is yourself. You are the agent and the world is your environment.

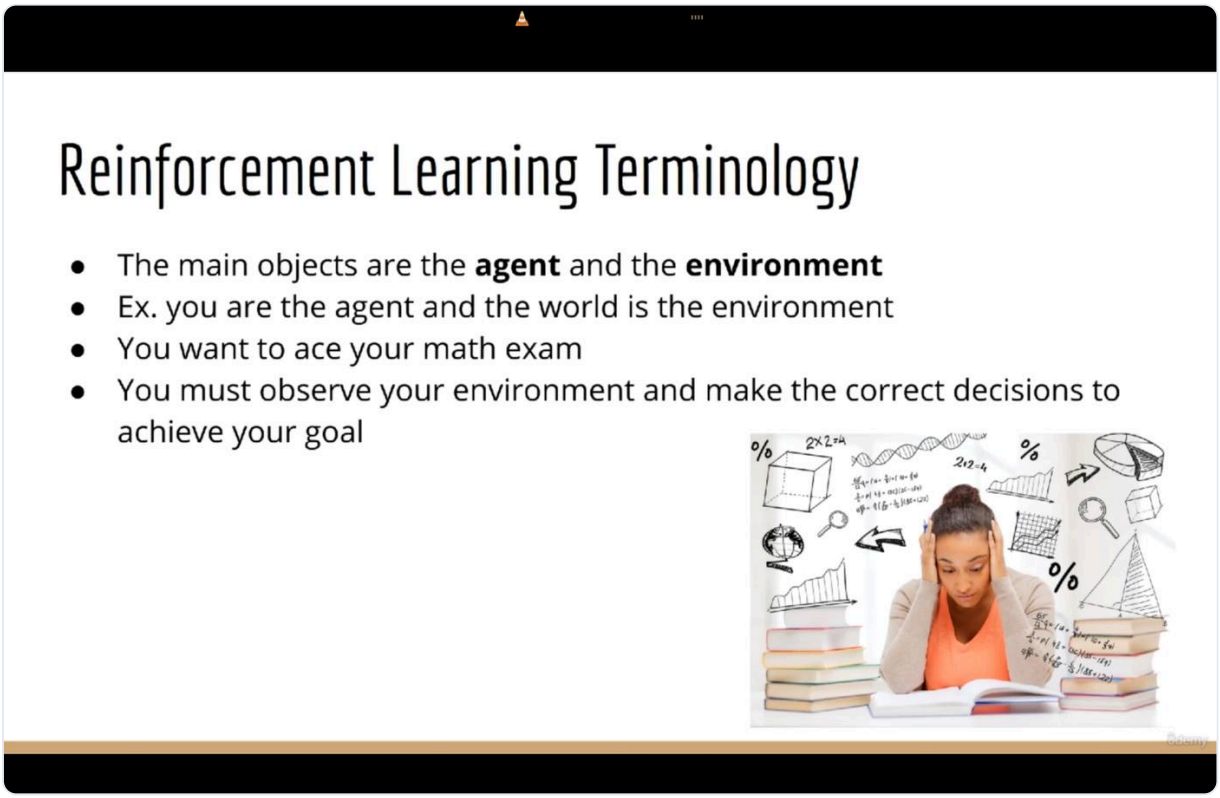


FIGURE 51.1 | Reinforcement Learning Terminology

Maybe your long term goal is to ace your math exam and so just like an autonomous vehicle driving to a destination you must observe your environment and make the correct decisions every day in order to achieve your goal. That means studying going to class taking notes doing your homework asking questions when you are confused and so on. So that's one basic example here's another example that's closer to what we might actually use reinforcement learning for. Suppose you're writing a program that plays tic tac toe. What's the agent and what's the environment in this case. The environment is composed of the computer program that implements this tic tac toe game.

Of course this computer program may also involve some form of A.I. That will be the other player in the game. But for all intents and purposes just pretend it's a bunch of if statements and predefined rules so it's not intelligence per say.

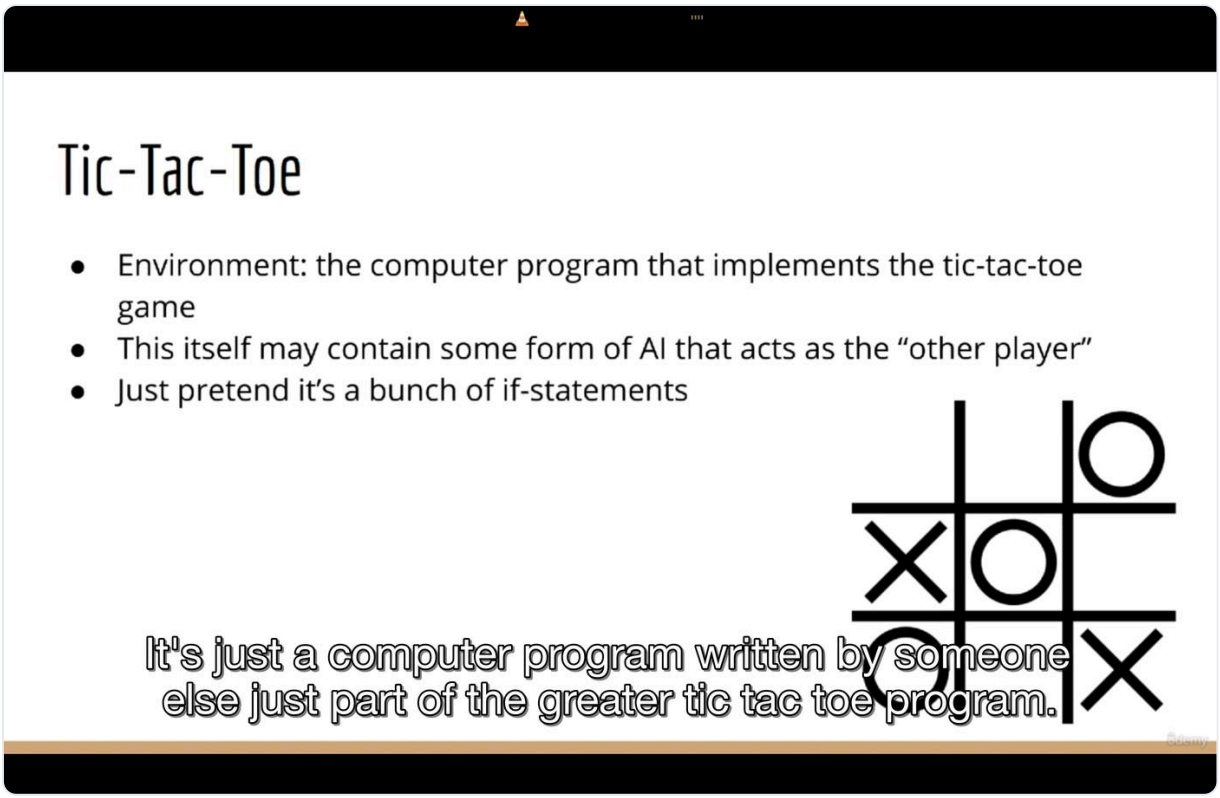


FIGURE 51.2

It's just a computer program written by someone else just part of the greater tic tac toe program. You can imagine that this tic tac toe program will have an API that will allow you to interact with it programmatically. So for example there might be a function to start a new game. There might be a function to place your X or your O at some location on the board.

FIGURE 51.3 | Imagine the Tic-tac-toe Game Has an API

There might be a function to read the State of the board so you can see where all the X's and O's have been placed so far. There might be a function to check whether the game is over and if so who won the game. The computer or your agent.

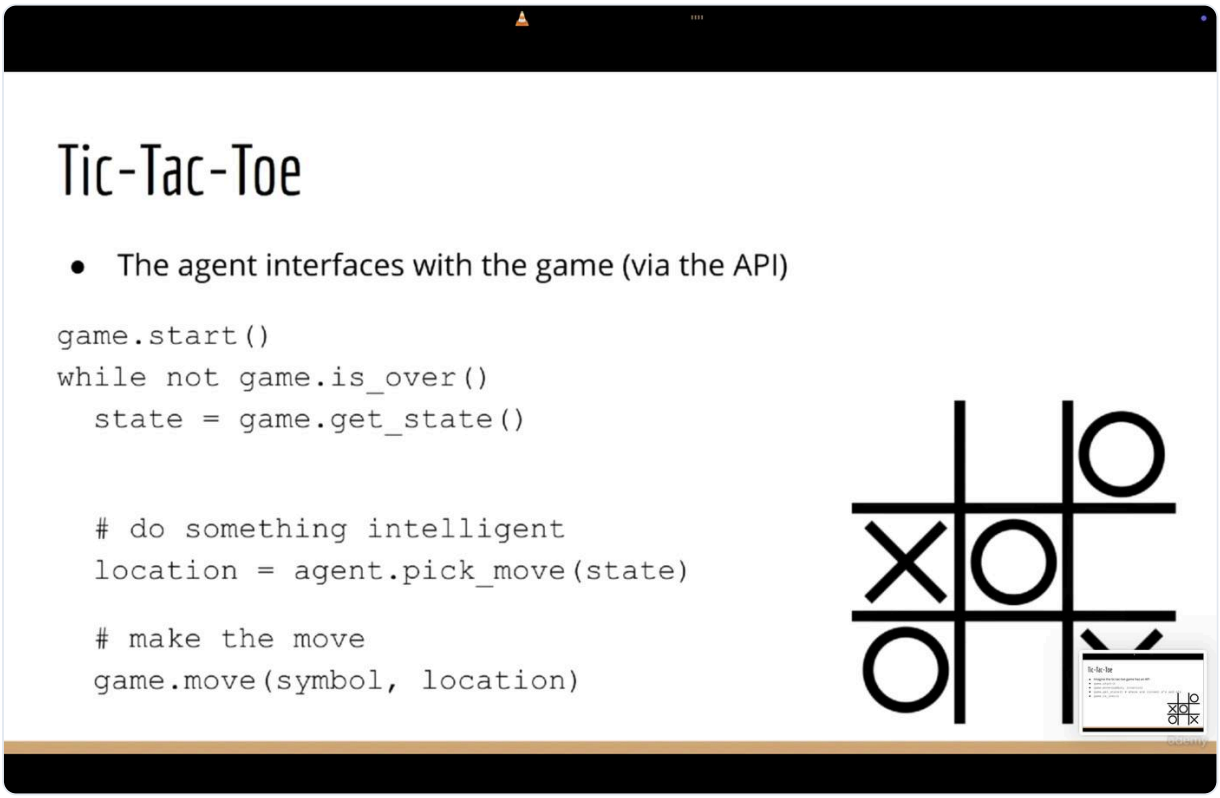


FIGURE 51.4

So that's the environment your agent on the other hand will be another computer program that interfaces with the tic tac toe game. Your agent may use an algorithm such as one from reinforcement learning in order to learn how to play tic tac toe from experience. It makes use of the API to interface with the environment.

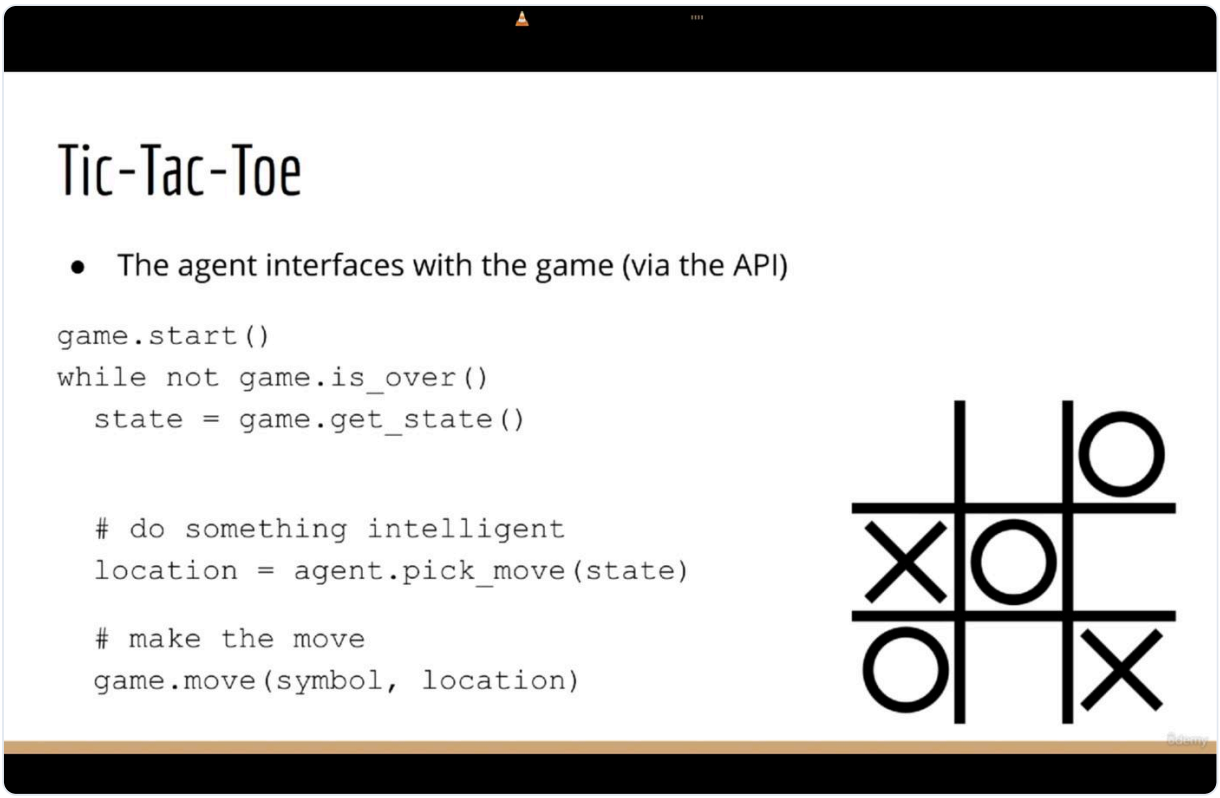


FIGURE 51.5

So here we have a program that plays the game. Part of this code is where the agent reads the state of the game board and chooses the most intelligent action. That's our A.I. Represented by our agent here's another popular example video games.

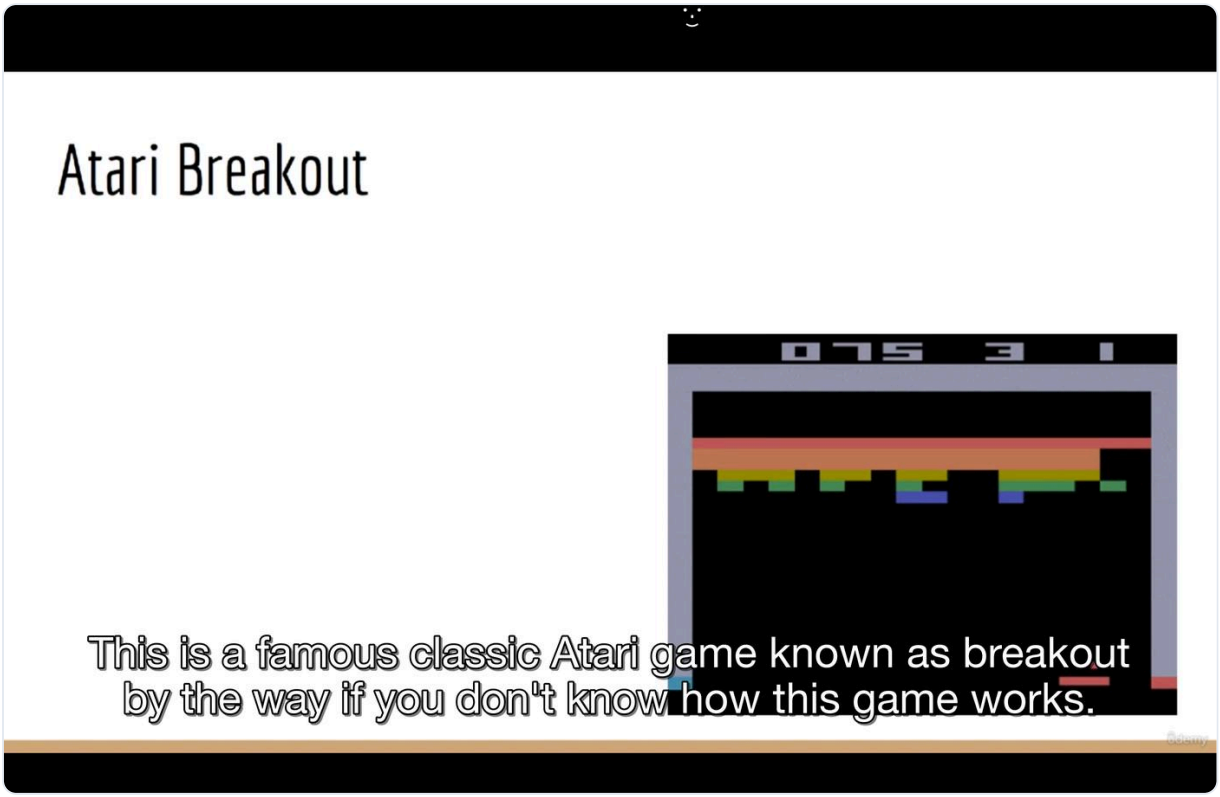


FIGURE 51.6 | Atari Breakout

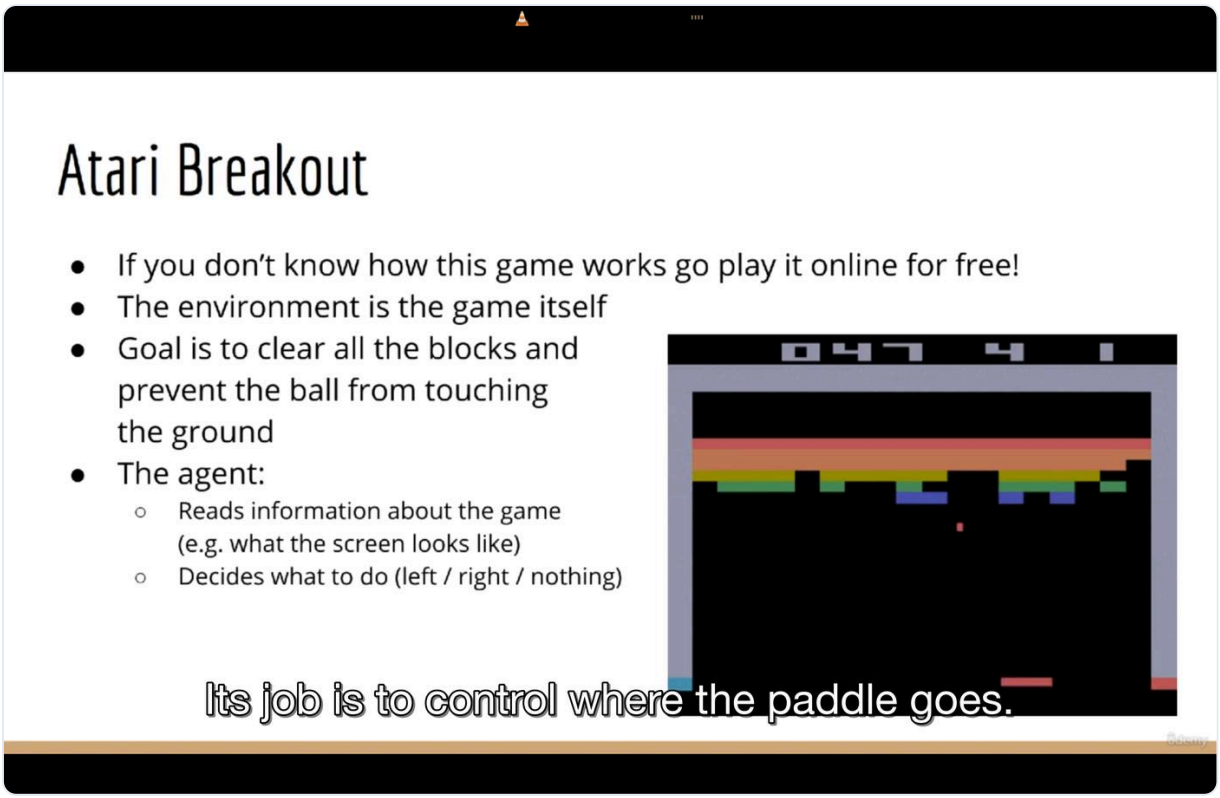


FIGURE 51.7 | Atari Breakout

This is a famous classic Atari game known as breakout by the way if you don't know how this game works. There are many places where you can play this game online for free. So if you've never played this game before please go and give it a try. In this game the environment is obviously the game itself. The goal is to clear all the blocks and your job is to move the paddle in such a way that the ball destroys the blocks but never falls to the ground. The agent would be your computer program which can read information from the game like what the screen currently looks like so it can figure out where the blocks are where the paddle is where the ball is going and so forth.

Its job is to control where the paddle goes. So basically you can move left right or do nothing next. Let's continue defining more terms. So far you know about the agent and the environment. The next term I want to define is episode what happens when I play a game of tic tac toe or breakout. Well some sequence of events will occur and then at the end I will win or lose with my math exam example. You will take your math exam and then you'll get your grade. Now we know that with learning algorithms the way that they learn is with data.

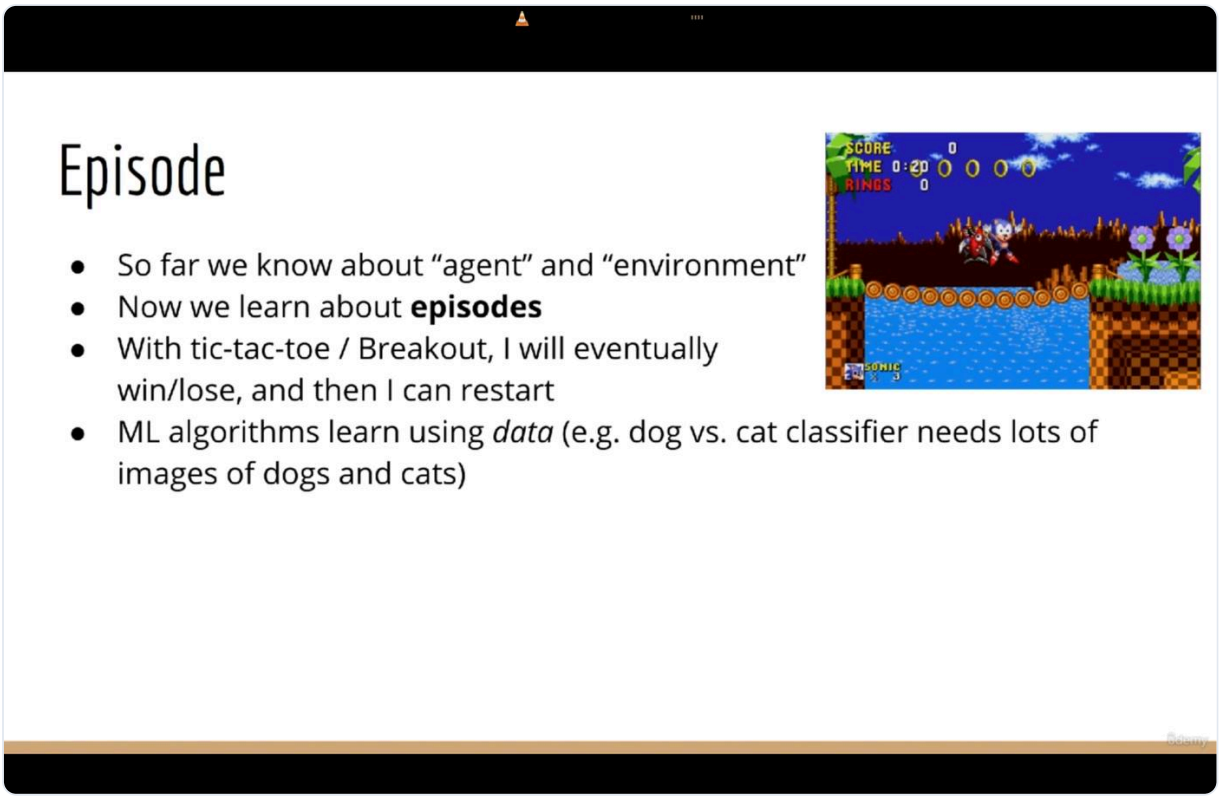


FIGURE 51.8 | Now We Learn About Episodes

So if you're training a dog versus cat classifier you'll need lots of labeled images of dogs and cats similarly with tic tac toe or breakout. Once the game is over I can opt to play again.

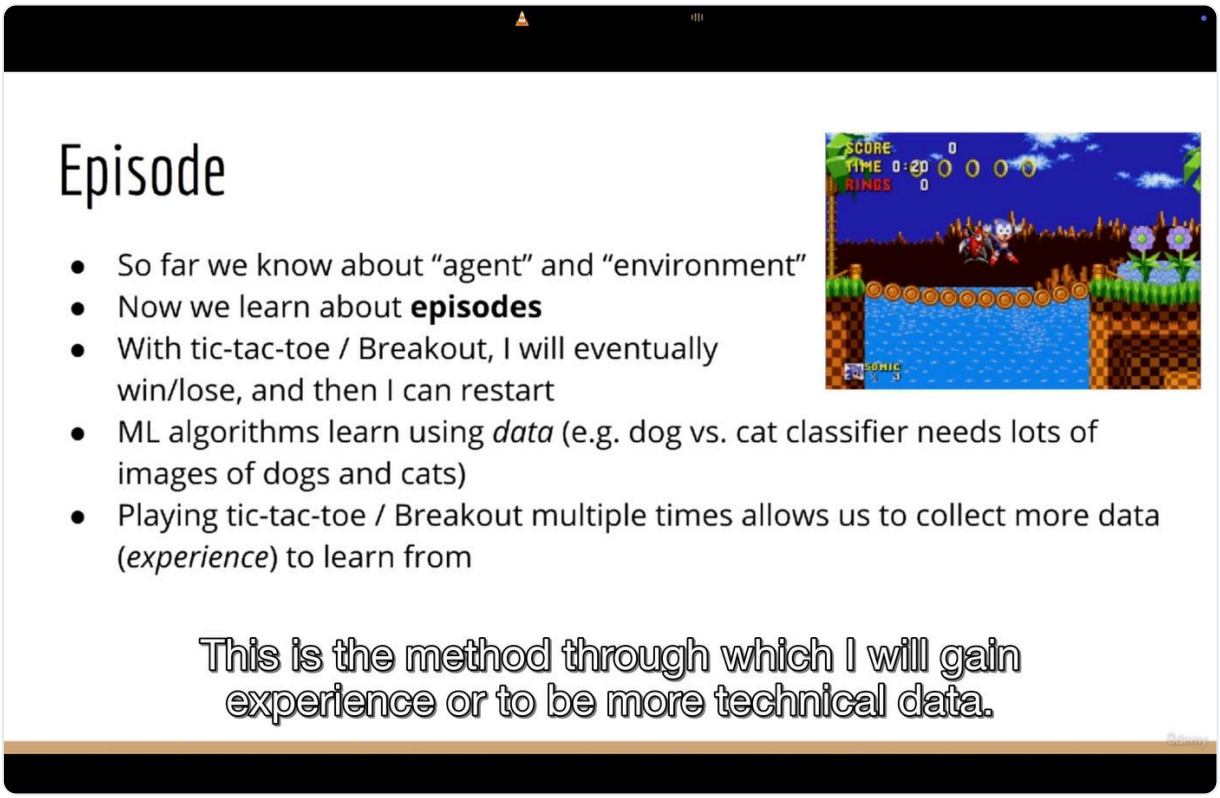


FIGURE 51.9

This is the method through which I will gain experience or to be more technical data.

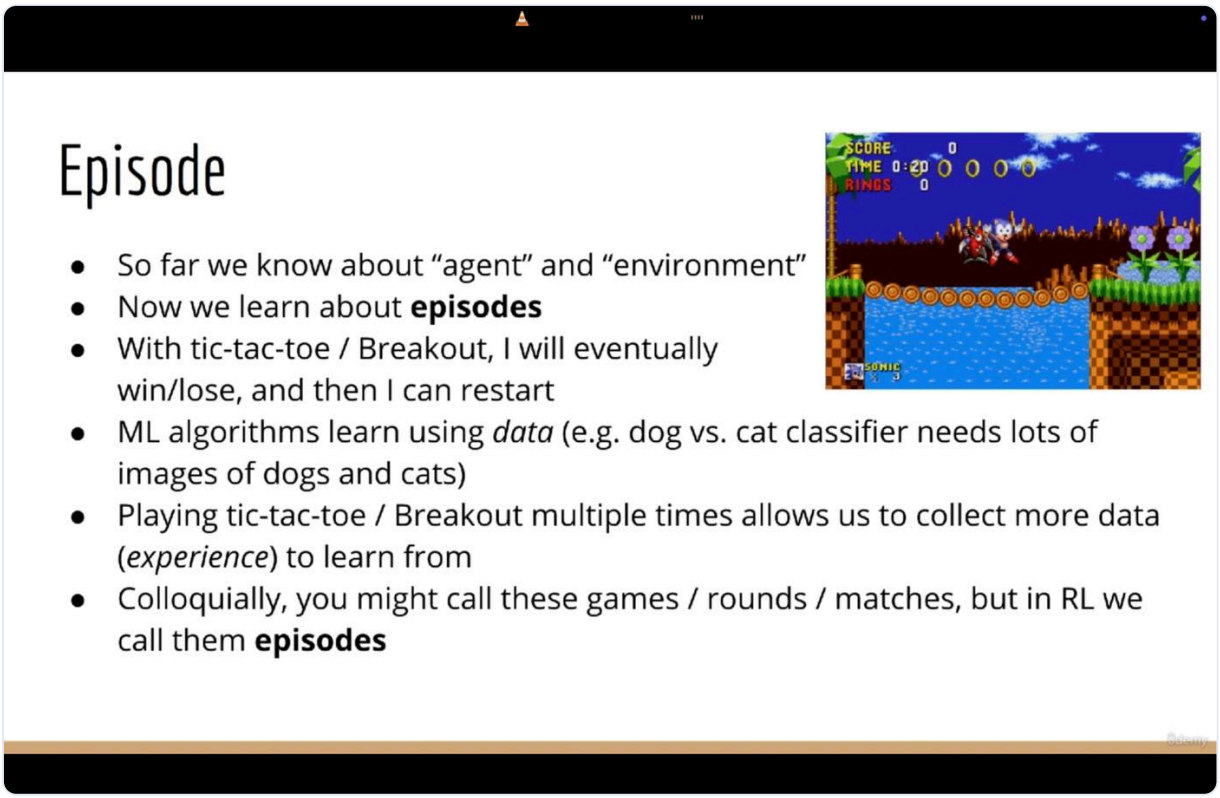


FIGURE 51.10 | Now We Learn About Episodes

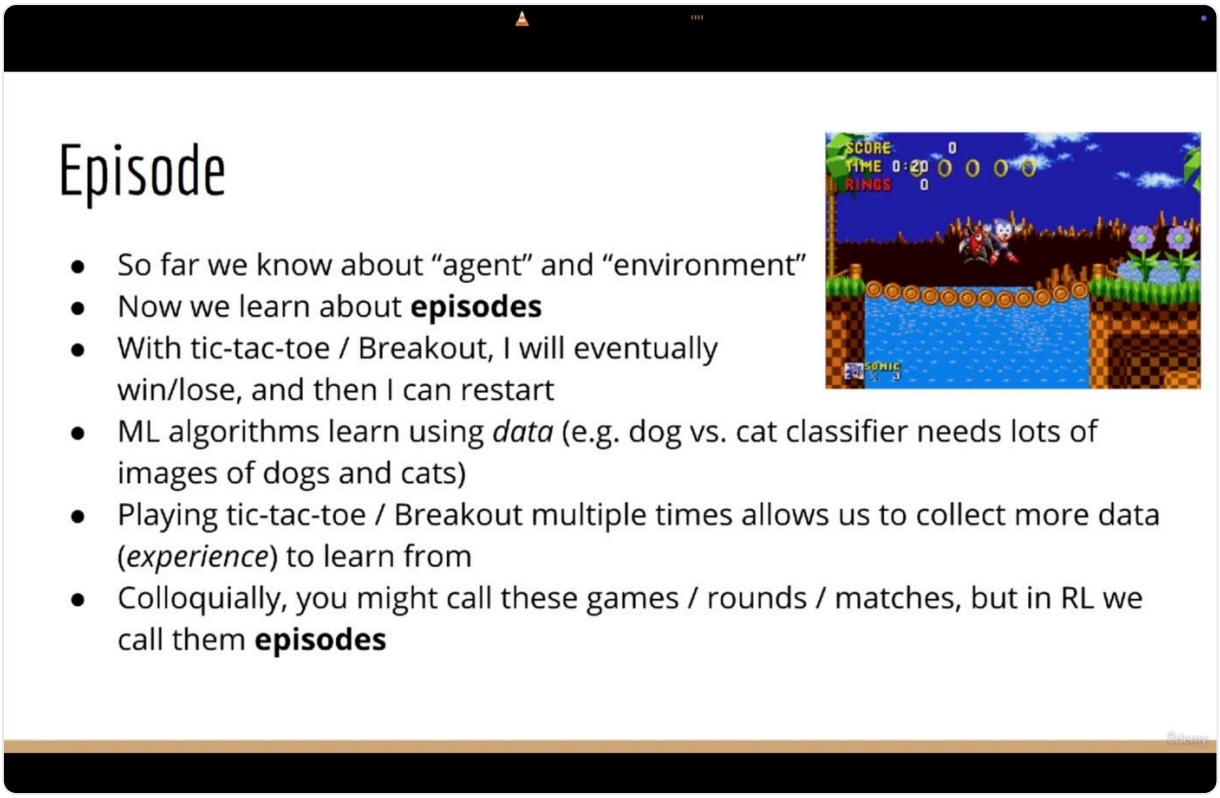


FIGURE 51.11 | Now We Learn About Episodes

You might call these games or rounds or matches but in reinforcement learning the official term is episode so when you're training an agent to play tic tac toe you're going to play multiple episodes and at the end of each episode your agent will have won or lost. Hopefully by the end of the training process or

in other words after many episodes your agent will be winning more than losing. Of course not all reinforcement learning environments are episodic to say an environment is episodic means that they end at some point and you can start again with a new fresh episode.

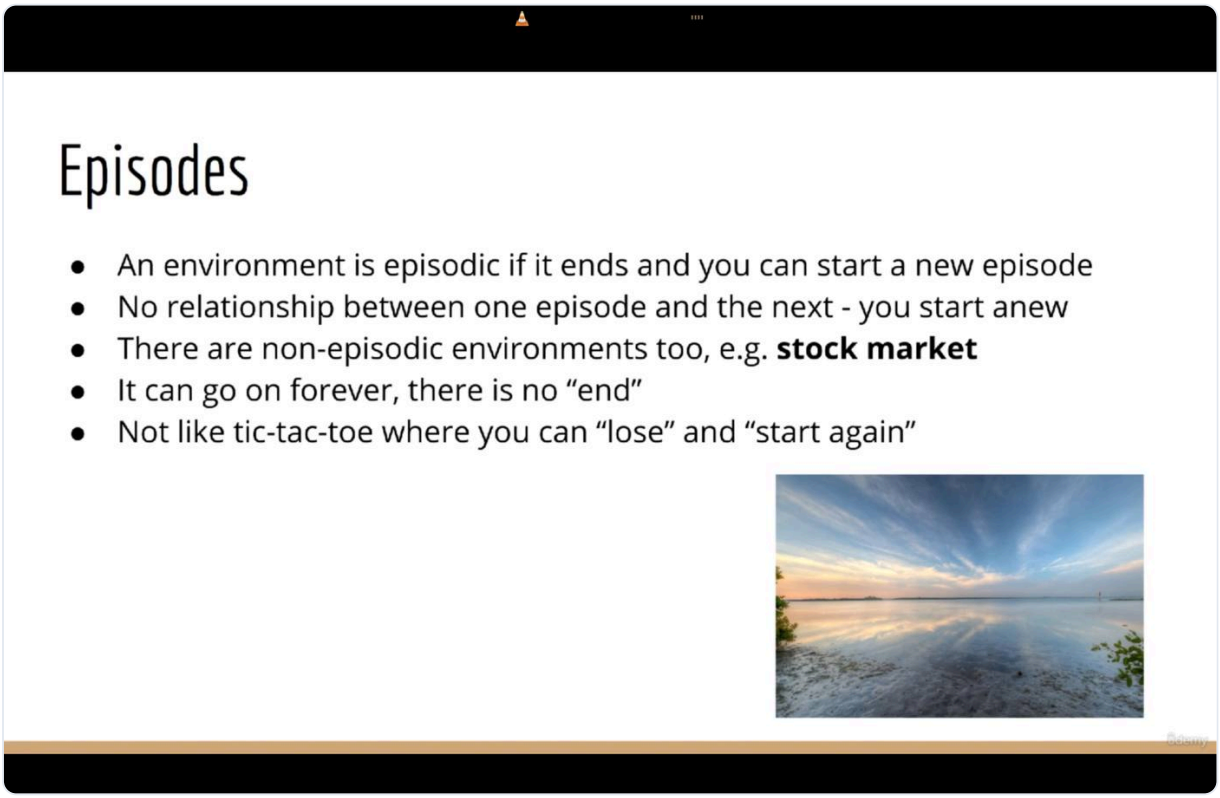


FIGURE 51.12

Furthermore there is no relationship between one episode and the next so the fact that I lost the previous tic tac toe episode will have no effect on the environment in the next episode. However there are examples of non episodic environments. Take for example the stock market. For all intents and purposes this can go on forever. There is no real notion of the end. Well if you lose all your money then technically there is nothing more you can do. But it's not the same as losing a game of tic tac toe and starting again. You can't go back in time and restart the stock market. Another example is an online advertising system your agent's job will be to choose the right advertisements to show to users at any given moment in order to maximize your company's revenue.

It should do this continuously.

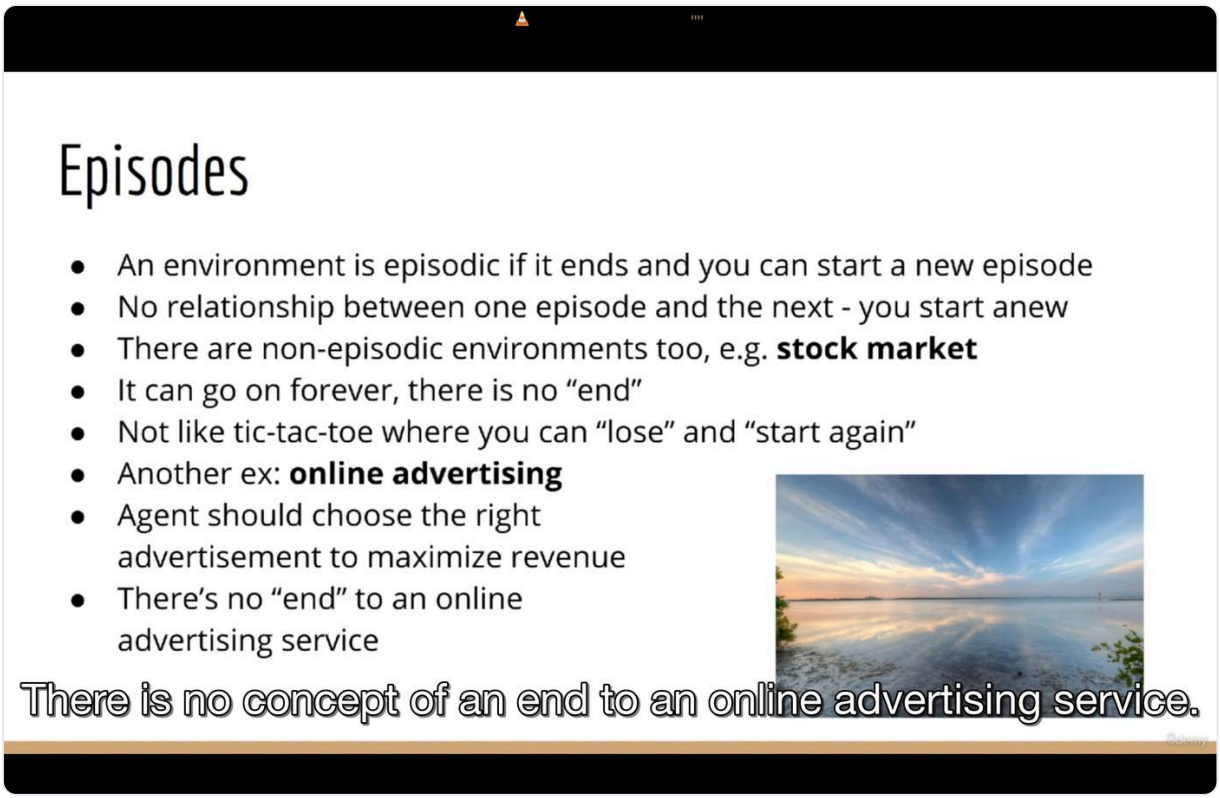


FIGURE 51.13

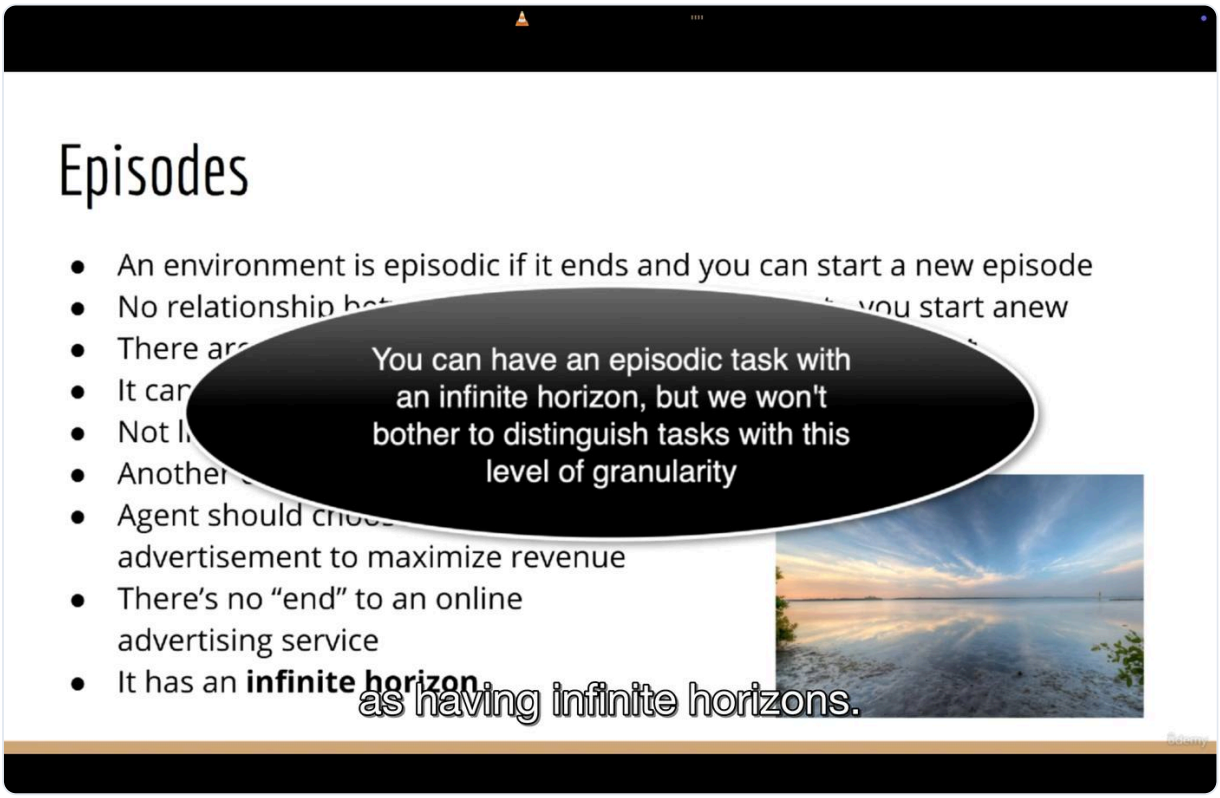


FIGURE 51.14 | There Ame You Can Have an Episodic Task with

There is no concept of an end to an online advertising service. All right so those are some examples of some non episodic environments we can refer to such environments as having infinite horizons. All right. So to recap the terms we've discussed so far now we have Agent environment and episode the next few terms I would like to think about are the state action and reward. These items help us describe

what goes on when the agent and environment interact let's again use our tic tac toe example. In this scenario the state would be the configuration of the board. So for each position on the board I want to know is there an x there. Is there an O there or is it empty.

This information is all that I need in order for my agent to make an intelligent decision about what move to play next. Speaking of which the moves that the agent makes are what we refer to as the action. So in tic tac toe to take an action would mean placing a new X or O somewhere on the board finally the reward is just a number that you can receive at any moment as you play an episode of the game by the way. Keep in mind when I say the word game I don't necessarily mean a board game like tic tac toe or a video game like breakout.

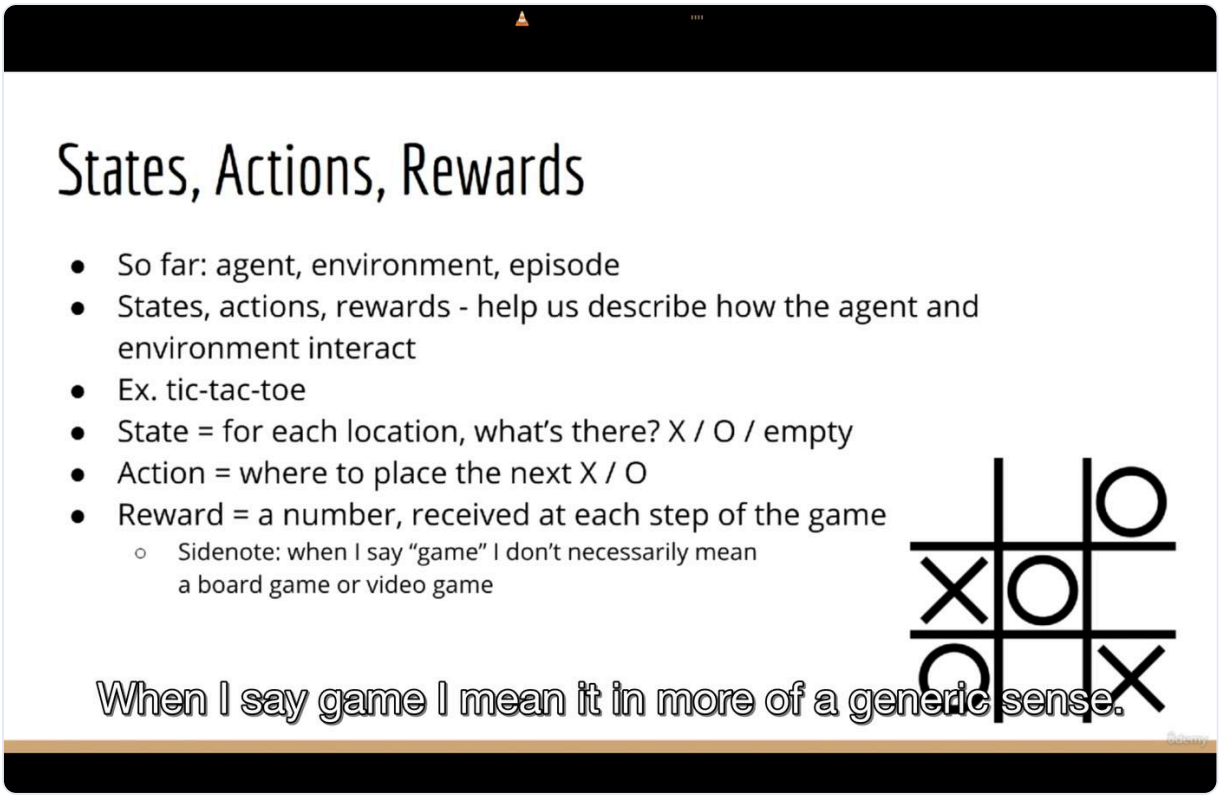


FIGURE 51.15 | States, Actions, Rewards

When I say game I mean it in more of a generic sense. In any case perhaps the reward you get in tic tac toe maybe plus one for winning minus one for losing and 0 for a draw. Although this is just an example. In general you can always assign rewards yourself in order to improve the training of your reinforcement learning agent. Here's another example of states actions and rewards.

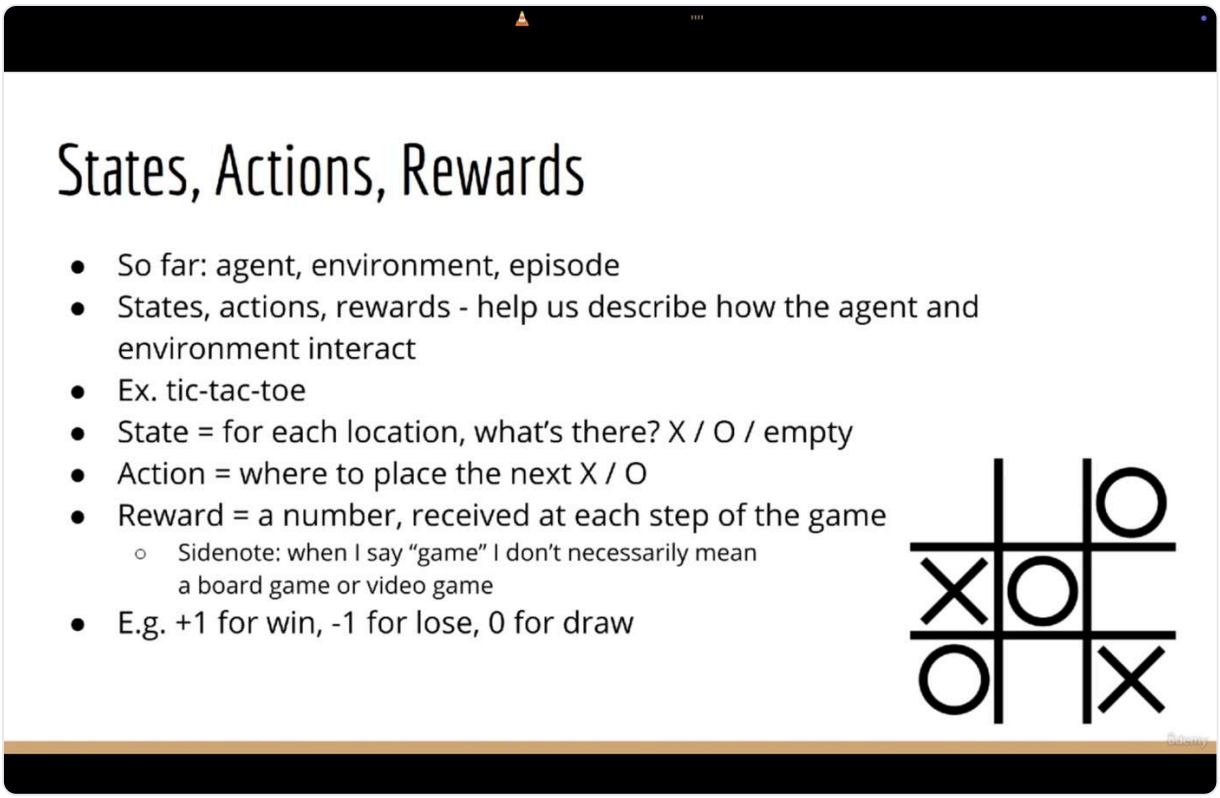


FIGURE 51.16 | States, Actions, Rewards

Think of a maze in this maze. The state is your position in the maze. Your actions may consist of the various directions you can go for example up down left or right. The reward is tricky.

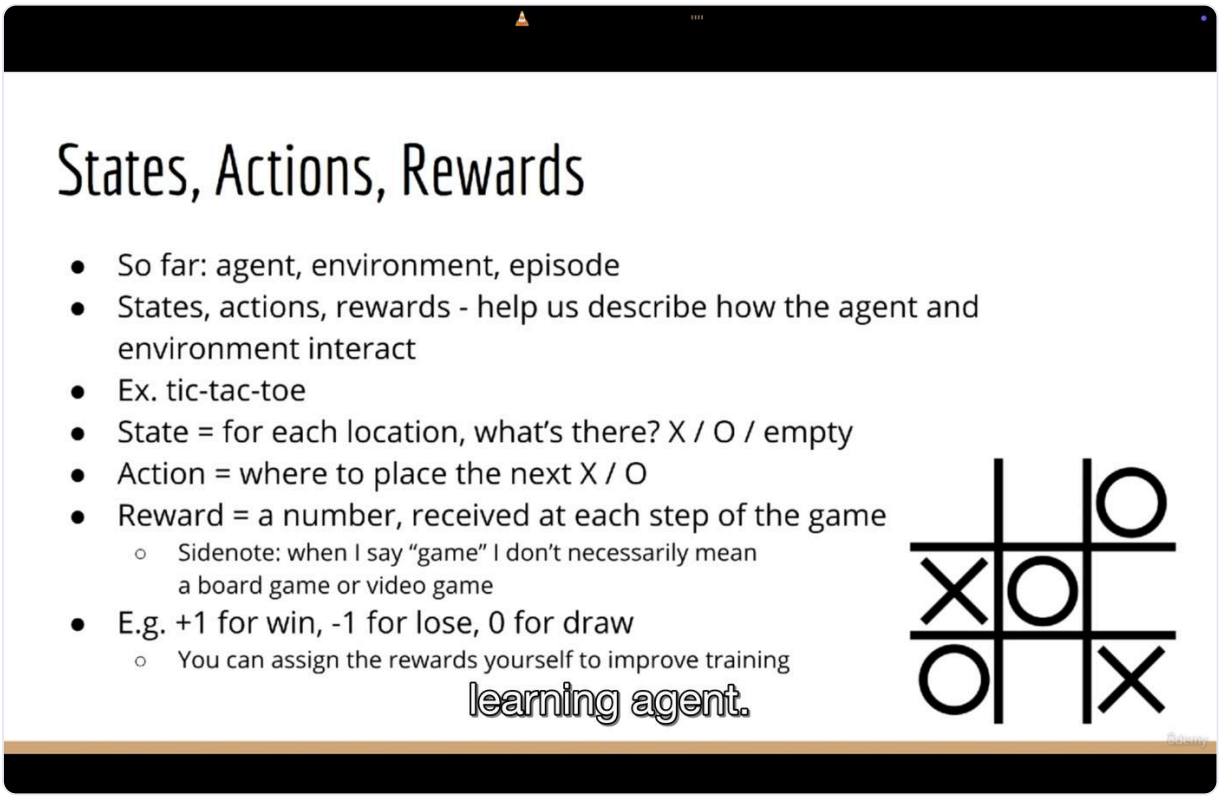


FIGURE 51.17 | States, Actions, Rewards

Remember I said it's up to you to think of a good reward to assign to your agent to encourage it to learn how to solve the environment you might say plus one for solving the maze and zero otherwise. But ask yourself Is this a good strategy. Imagine you throw your agent into this maze and it has to learn what to do. Imagine your agent has played this game ten thousand times and has never solved the maze. We can pretend the environment is episodic so that after taking one hundred steps you reach a terminal state and the game is over.

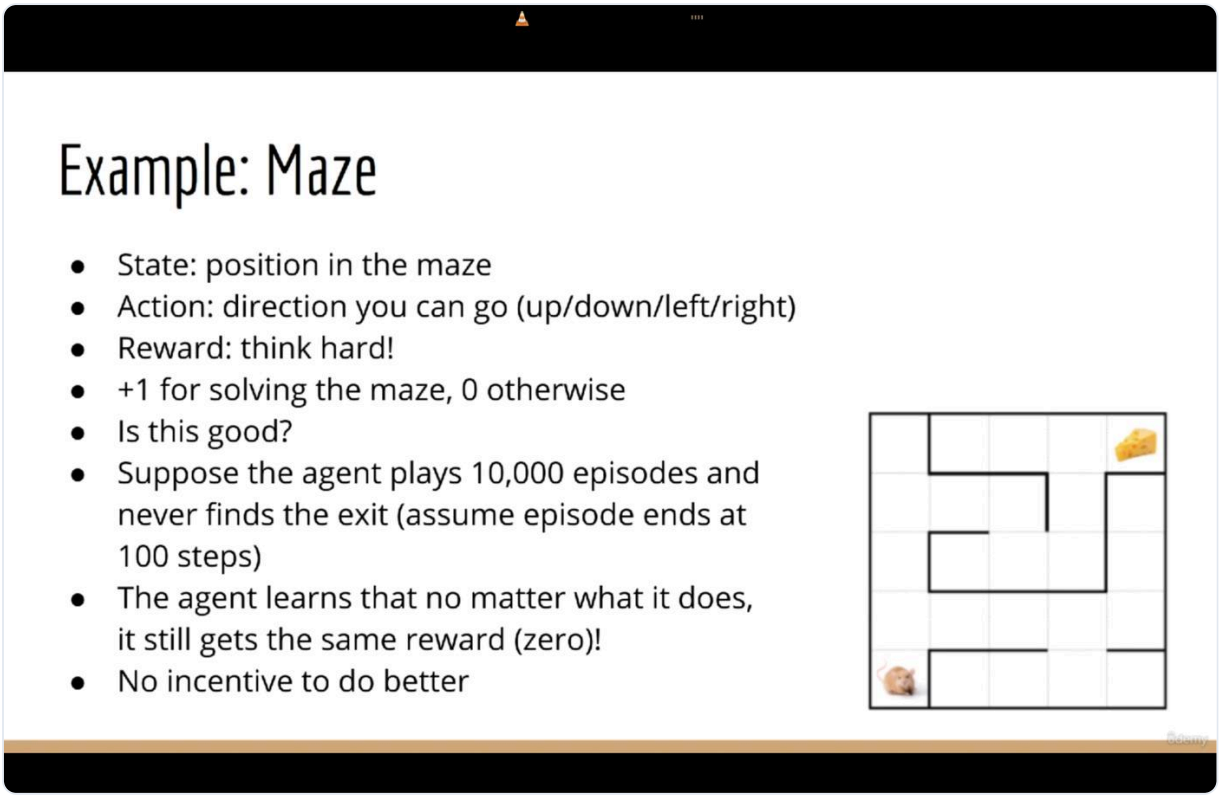


FIGURE 51.18 | Example: Maze

What happens if we get zero reward each time well in that case the agent learns that it does not matter at all what it does because doing anything always leads to the same reward. Zero in this case the agent has no incentive to solve the maze. Your agent will never prioritize one action over another because it knows that no matter what it does it always gets zero reward. In this case all actions are equal perhaps a better reward structure would be to assign a minus one reward at every stage in this case. You can maximize your reward by solving the maze as fast as possible. Doing any extraneous actions will lead to a more negative reward not solving the maze will lead to the most negative reward.

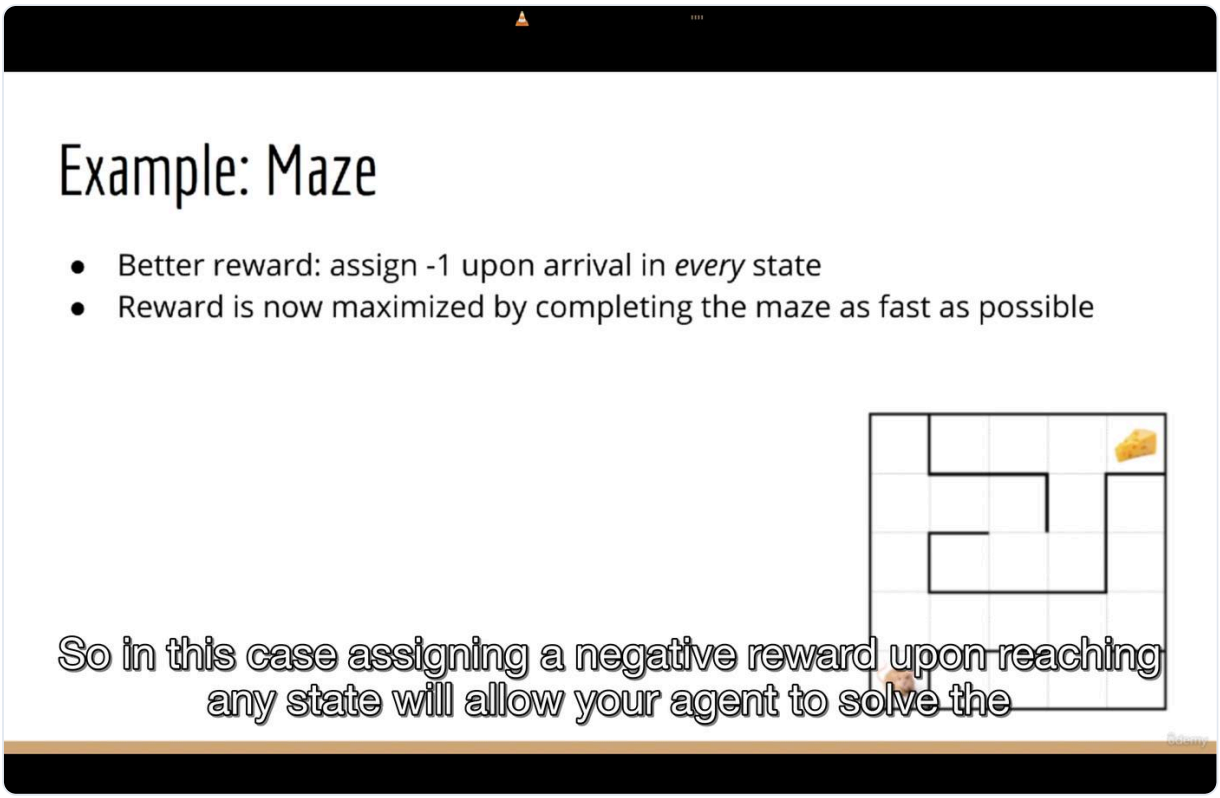


FIGURE 51.19 | Example: Maze

So in this case assigning a negative reward upon reaching any state will allow your agent to solve the environment in now you have to keep in mind this is not English class so you have to remove any bias you may have about what connotations are associated with the term reward. You might think of reward as a good thing like a prize. For example if you're a dog and you've just successfully completed a trick your owner may give you a treat as a reward. But in reinforcement learning this is not what we mean by reward. The only constraint is that the reward is a real number.

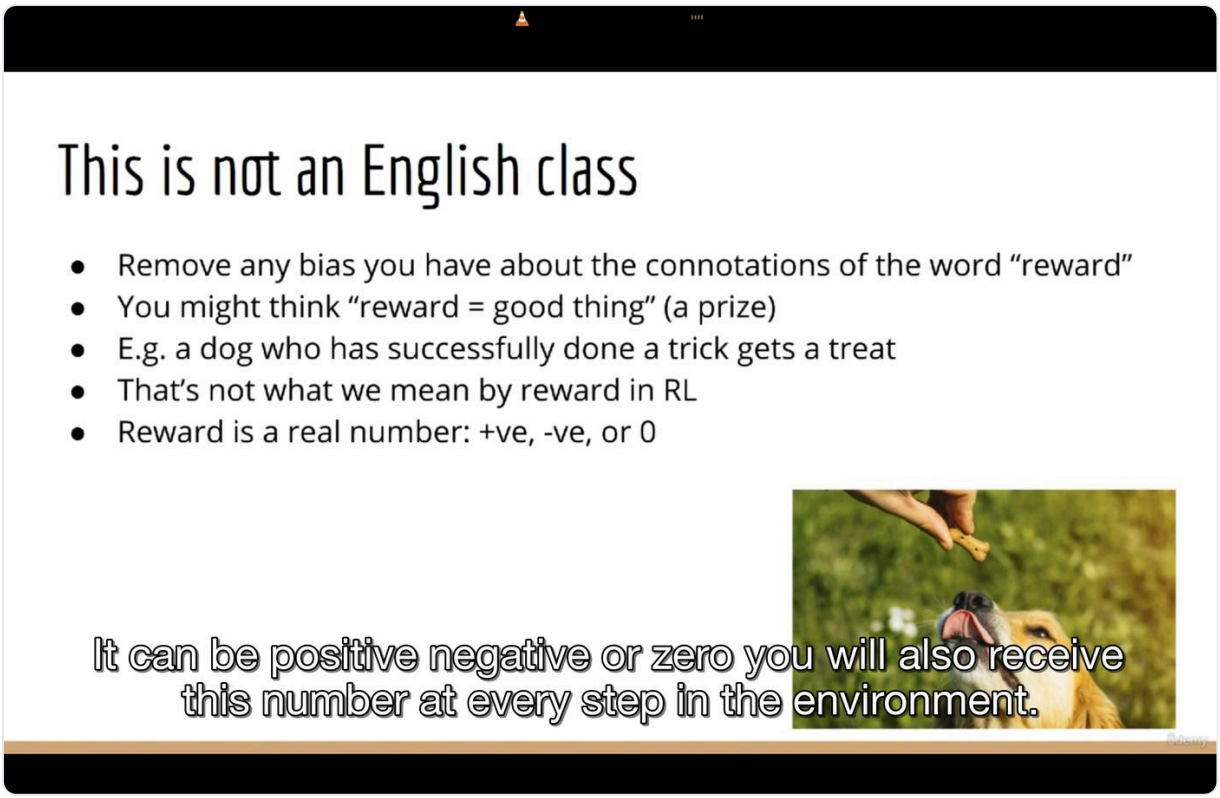


FIGURE 51.20 | This Is Not an English Class

It can be positive negative or zero you will also receive this number at every step in the environment. Not just when you reach some goal or definitively fail to achieve it. The agent as you will learn later in this section will try to maximize its reward over each episode. For example you may get a reward of minus one hundred.

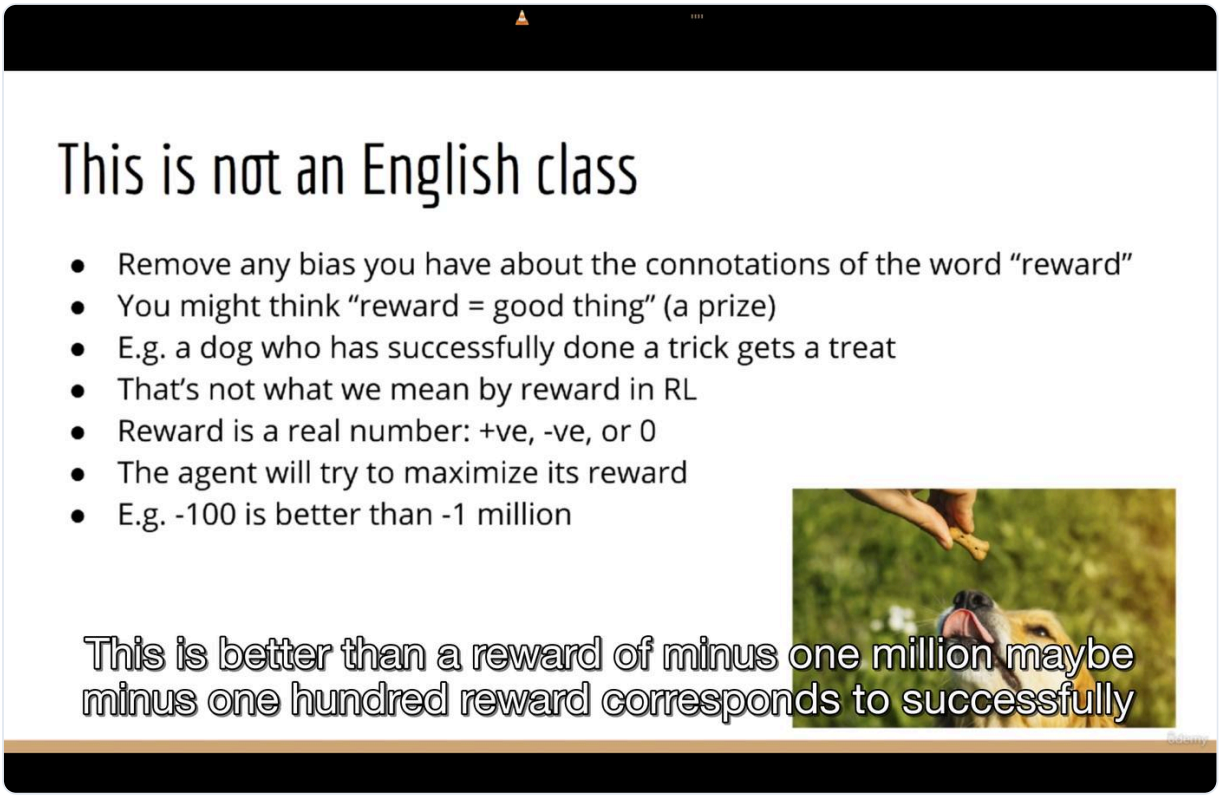


FIGURE 51.21 | This Is Not an English Class

This is better than a reward of minus one million maybe minus one hundred reward corresponds to successfully solving the environment but in the end it's just a number.

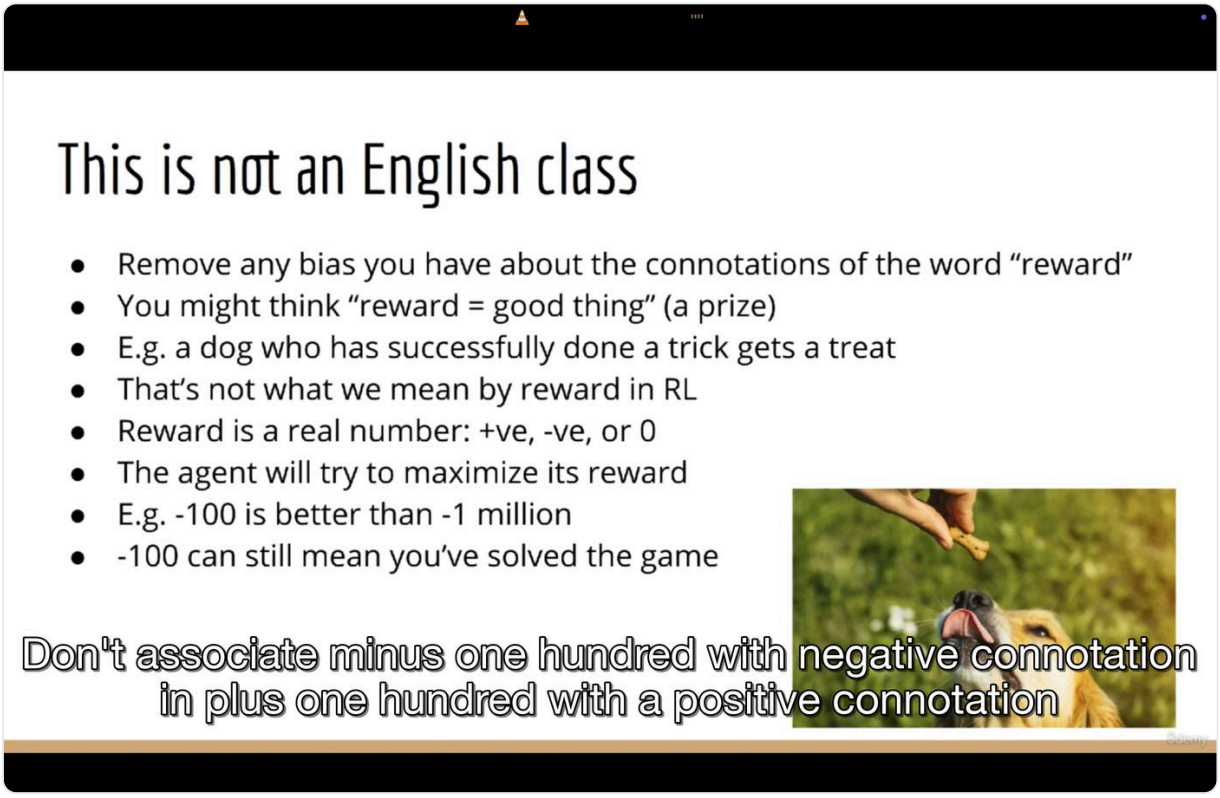


FIGURE 51.22 | This Is Not an English Class

Don't associate minus one hundred with negative connotation in plus one hundred with a positive connotation so just remember this. The reward is not a prize. The reward is a number which is to be maximized.

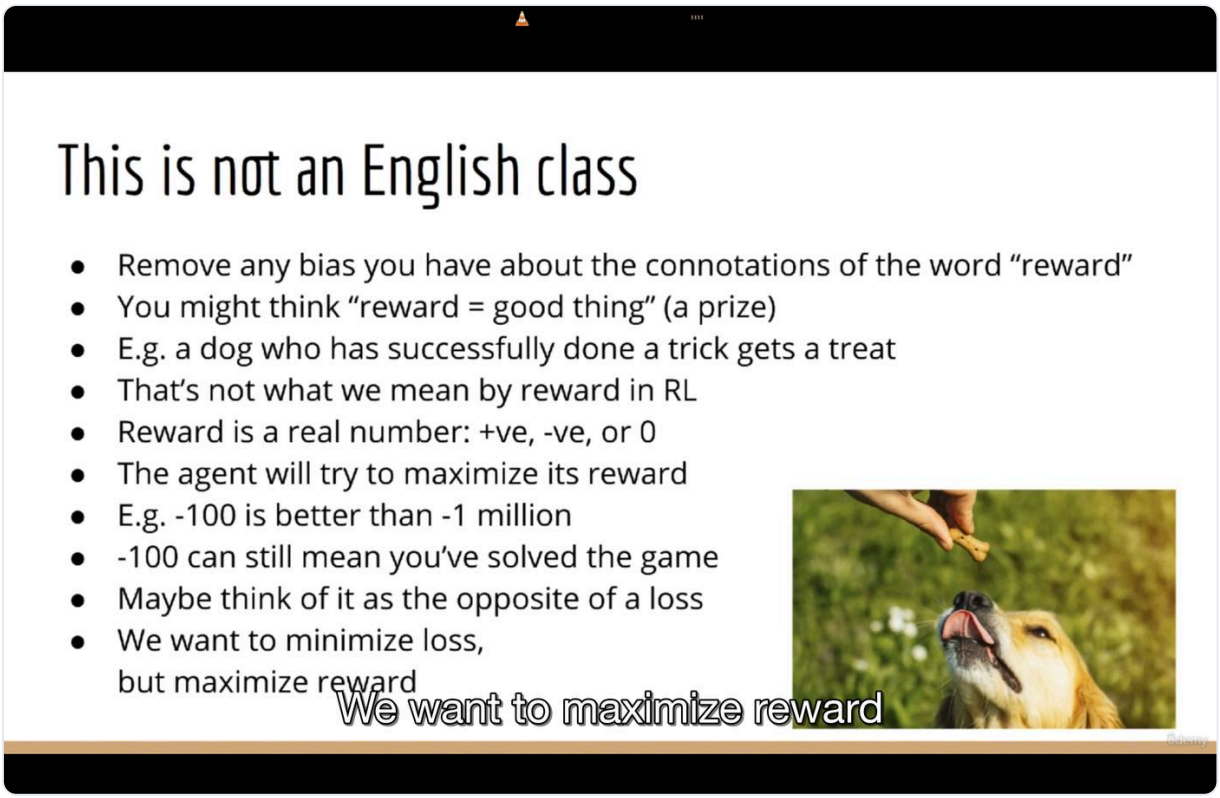


FIGURE 51.23 | This Is Not an English Class

You can think of it as kind of the opposite of a loss function. Whereas we want to minimize the loss in a supervised or unsupervised learning problem in reinforcement learning. We want to maximize reward since I love examples. Here's one more. Imagine again the game breakout. In this case we actually have several options for the state.

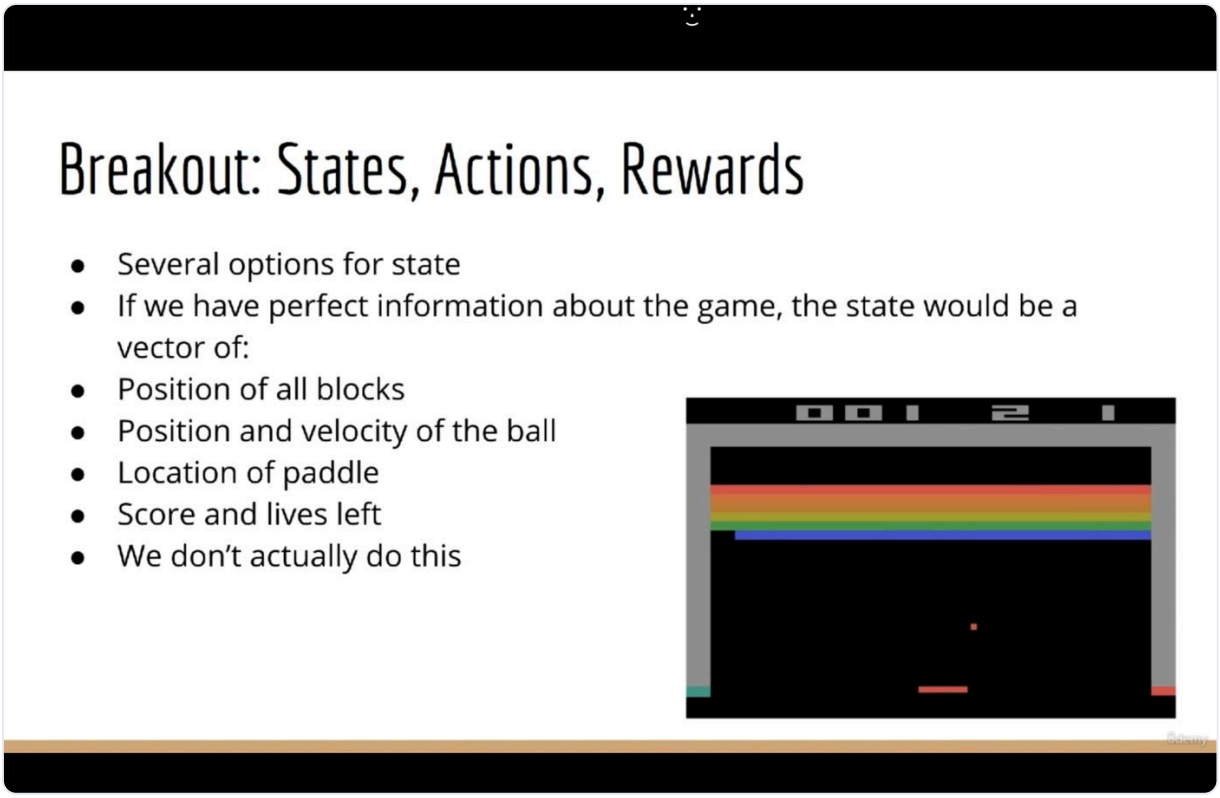


FIGURE 51.24 | Breakout: States, Actions, Rewards

For example we may have perfect information about the game. We could be told the exact positions of all the blocks we could be told the position and velocity of the ball. We could be told the location of our paddle and we could be told our current score and the number of lives we have left. Although I think you'll find that most reinforcement learning applications do not make use of such information another way you can read the information about the state of the environment in breakout is to look at the game's RAM. In other words look at the values it has stored in memory. In contrast to the above this actually is one method used in modern reinforcement learning applications.

It's a proxy to the above perfectly defined state.

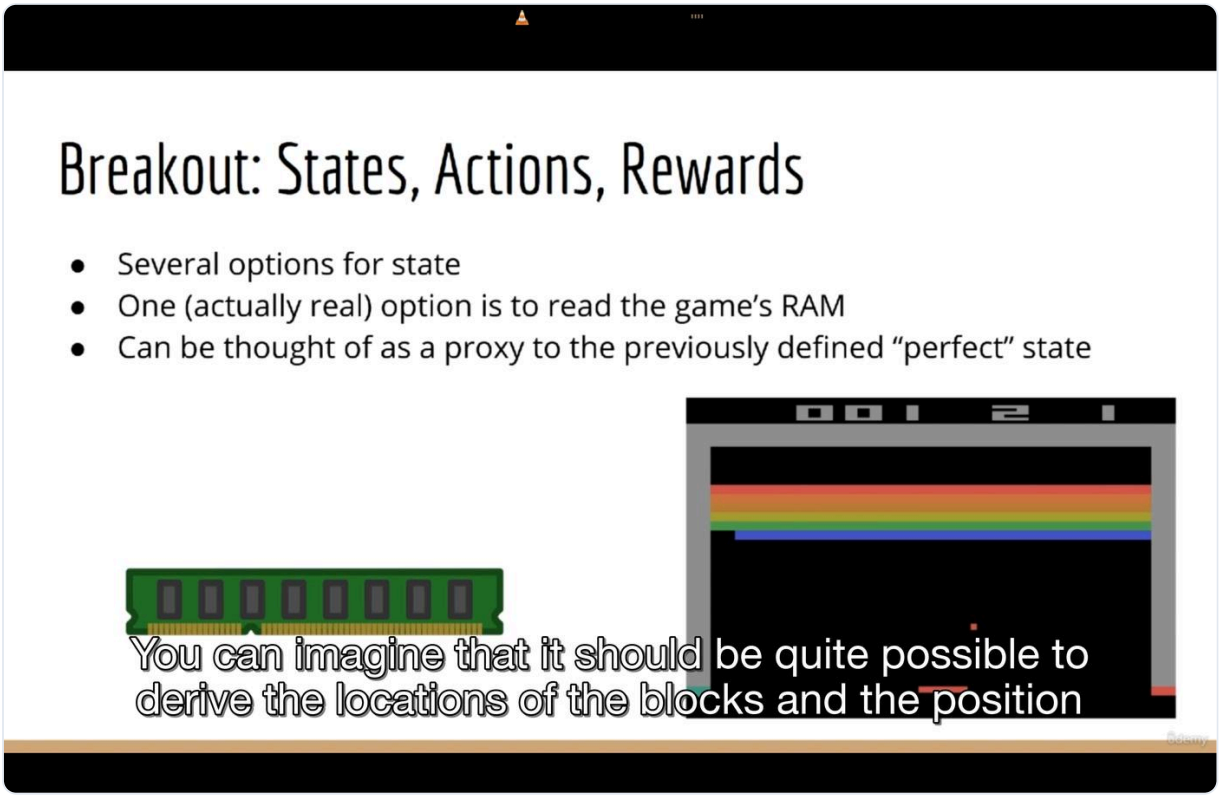


FIGURE 51.25 | Breakout: States, Actions, Rewards

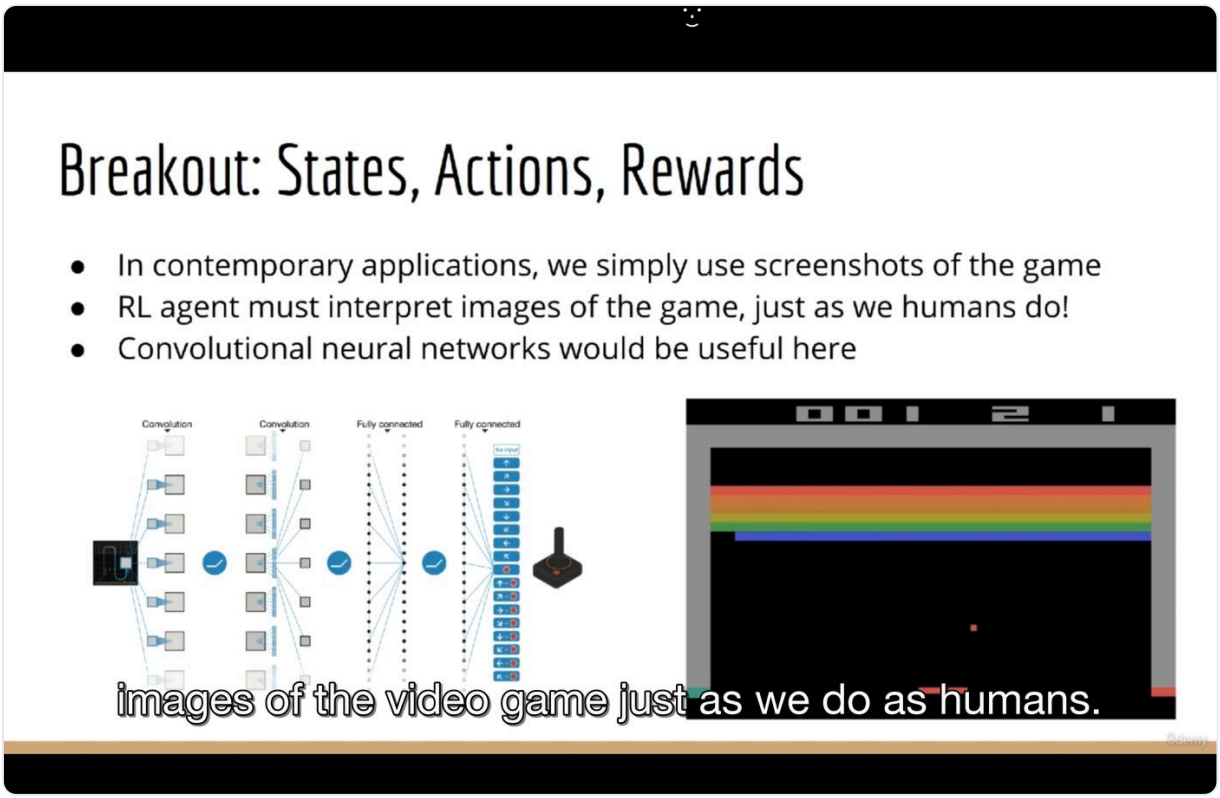


FIGURE 51.26 | Breakout: States, Actions, Rewards

You can imagine that it should be quite possible to derive the locations of the blocks and the position and velocity of the ball and so forth from the values stored in RAM although I think the most common way to represent the state in contemporary reinforcement learning is to use screenshots from the game in this way our reinforcement learning agent is learning to interpret images of the video game just as we do as humans. I think this is the most meaningful because it's the closest match to how you and I play video games. We look at the screen you can imagine that models like convolutional neural networks would be useful here one complication that can arise from only looking at images of the screen is that you don't really have information about movement an image is only a frozen picture of the game at a single point in time.

Looking at this image how can I tell which direction the ball is moving and so this allows us to consider an important point the state need not be what I observe in the environment. It can also be information derived from both current and past observations. So one way of dealing with the problem of frozen pictures is to simply include past frames as well.

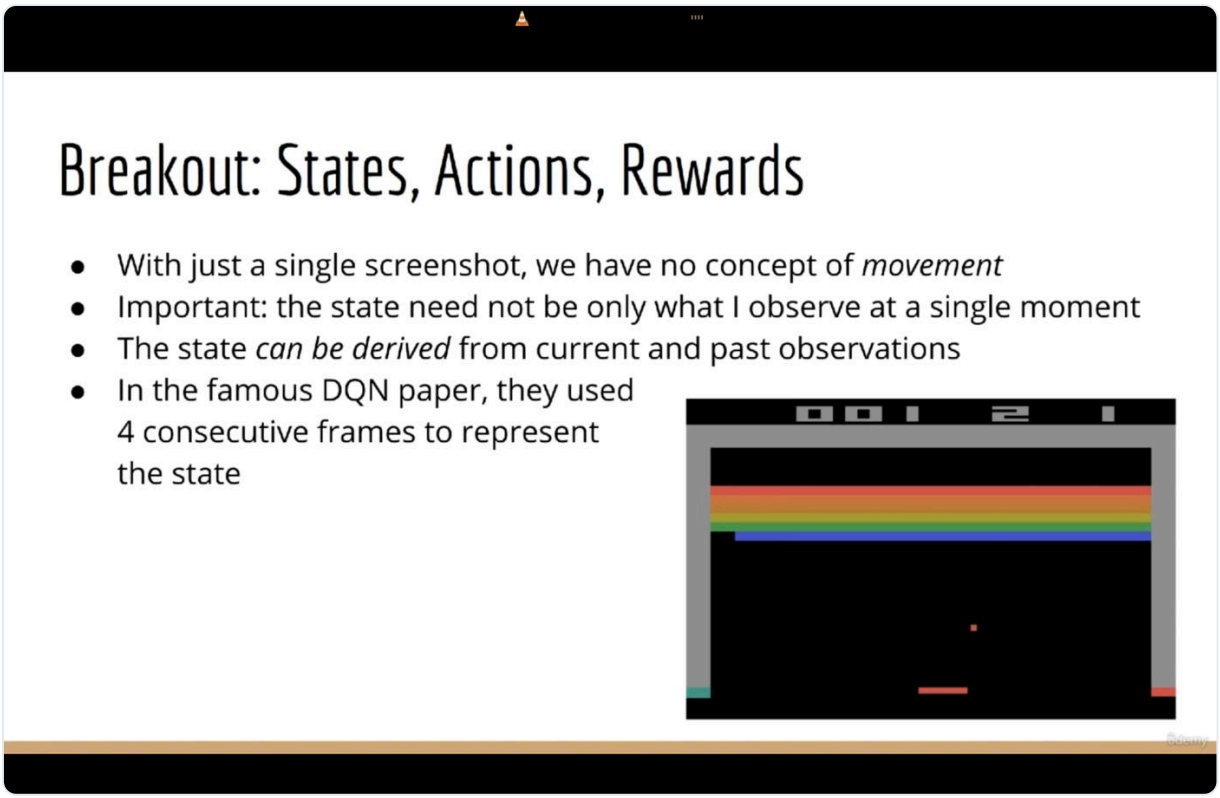


FIGURE 51.27 | Breakout: States, Actions, Rewards

In the famous DQN paper they use the most recent four consecutive frames of the game to represent a single state. To get back to our states actions and rewards the rest of it is quite basic. The actions consist of the various moves you can do in the game. You can think of this in terms of pressing buttons on a joystick or a control pad in breakout.

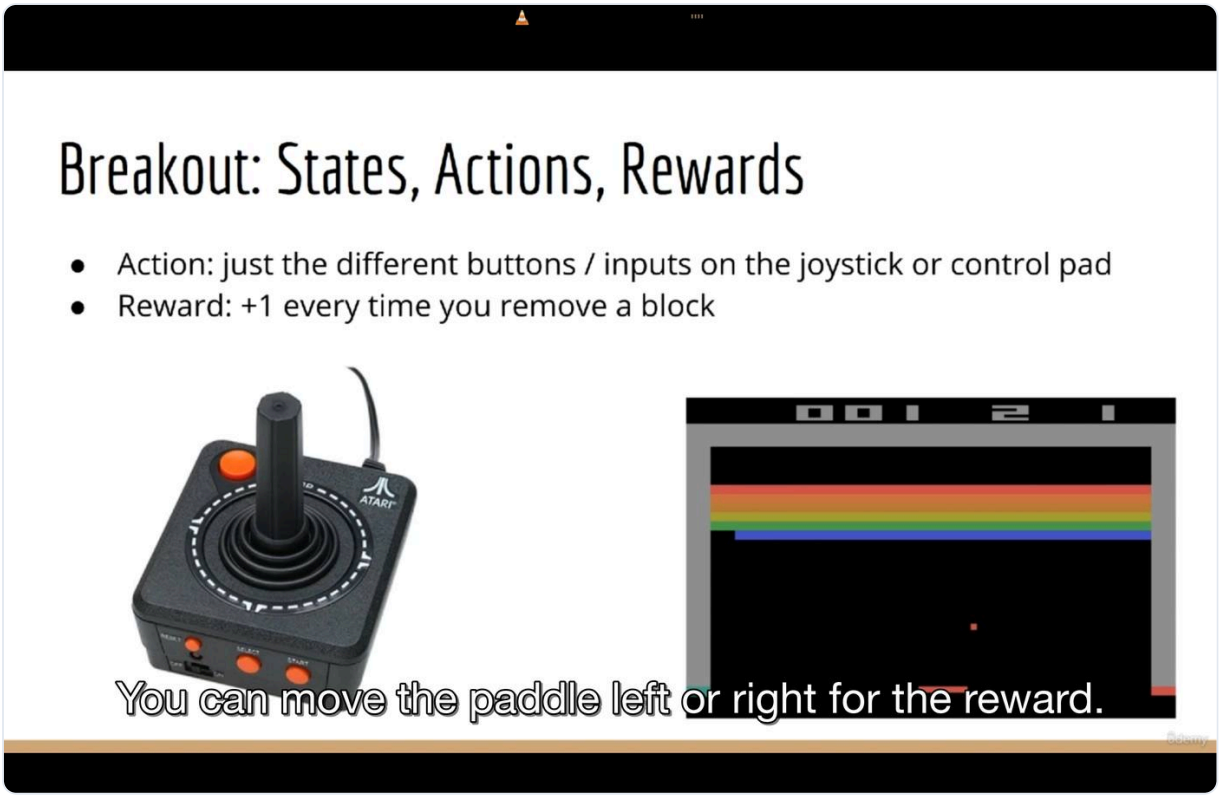


FIGURE 51.28 | Breakout: States, Actions, Rewards

You can move the paddle left or right for the reward. As an example you might get plus one reward every time you destroy block as a final note in this lecture. I want to introduce you to the concept of state spaces and action spaces.

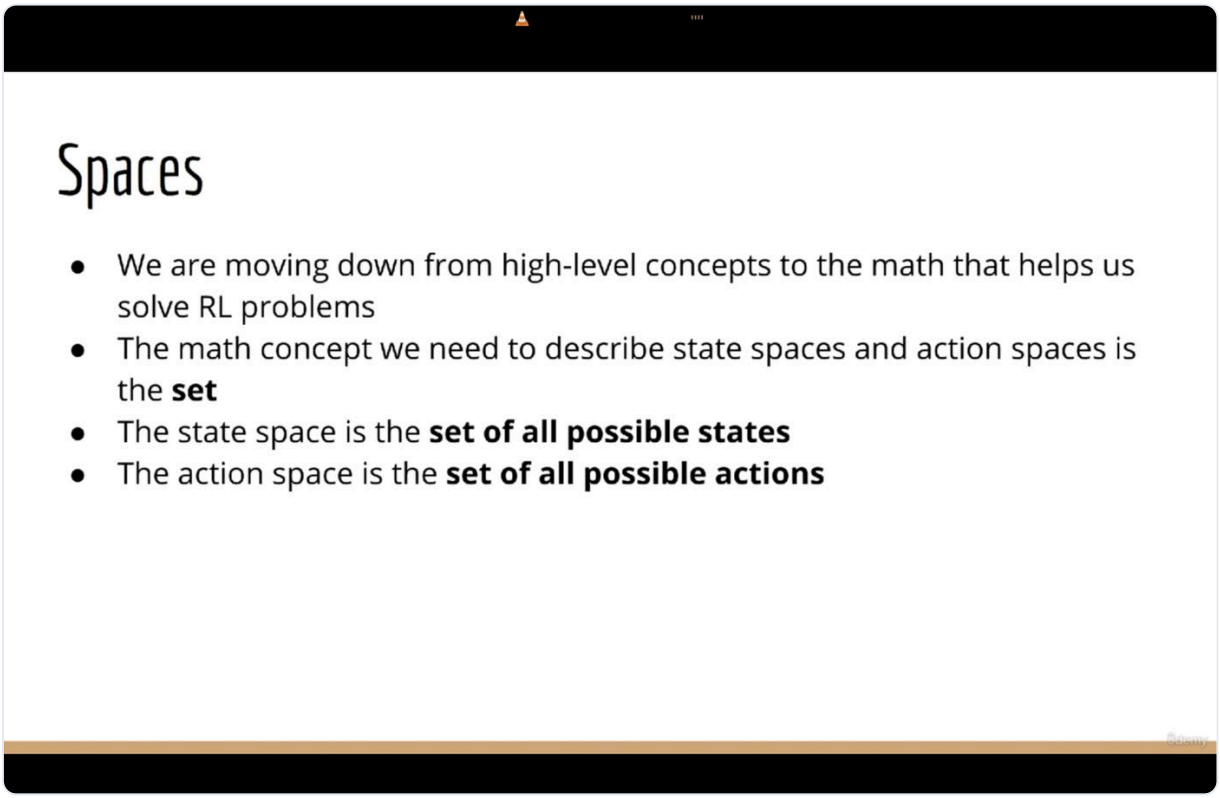


FIGURE 51.29

This is important as we move down from high level ideas and concepts to the actual math that will allow us to solve reinforcement learning problems the particular math concepts that we need to describe state spaces and action spaces is the set the state space is the set of all possible states and the action space is the set of all possible actions. We don't need to go further than this. We just need to know what it means.

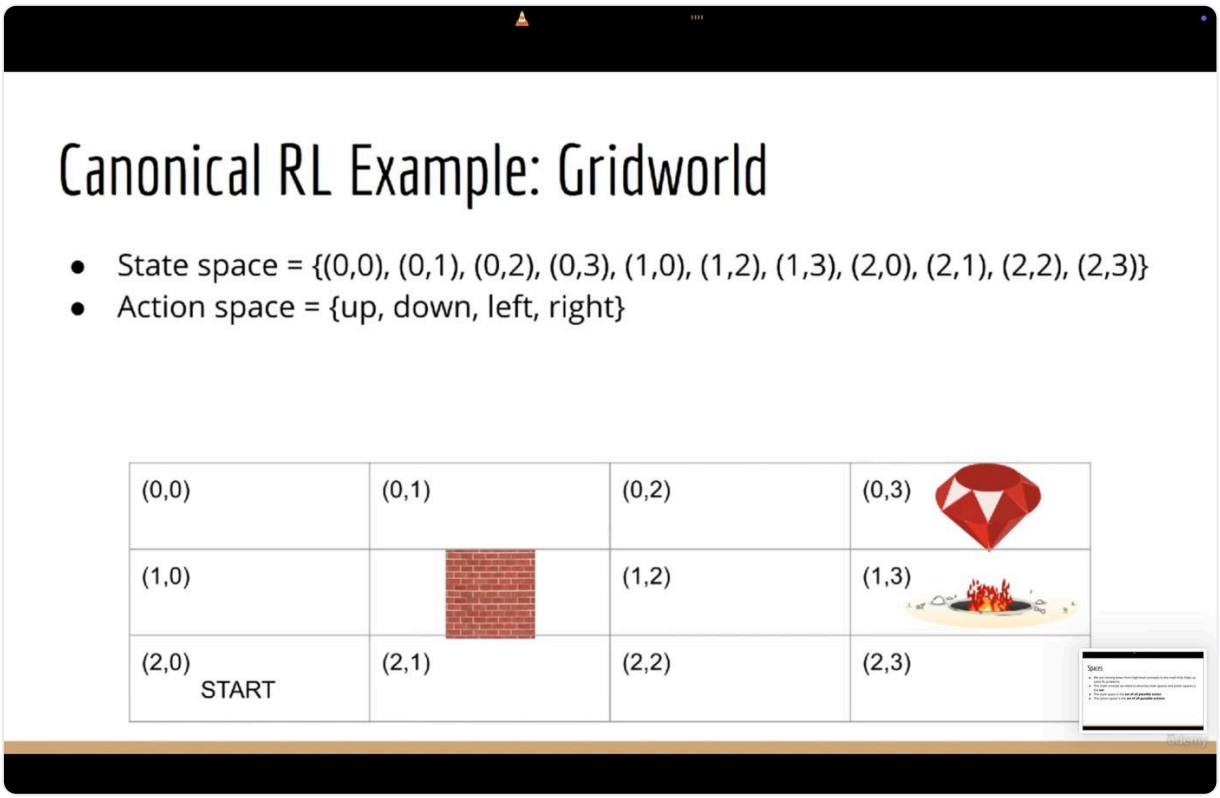


FIGURE 51.30 | Canonical RL Example: Gridworld

So as an example consider the canonical example of a reinforcement learning problem known as grid world in grid world. The idea is you're going to start in the bottom left square and your goal is to arrive at the top right square with a ruby is if you make it there you get a reward of plus 1 below that there is a losing state where if you arrive there you'll get a reward of minus 1 and in the second row second column there is a wall meaning that your agent cannot go to that square so that's the basics of the game to describe the state space. That's simply the set of all possible positions on the board. So you may want to pause this video and look closely at this list of coordinates to confirm that they correspond to positions on the board the action space consists of the actions up down left and right.

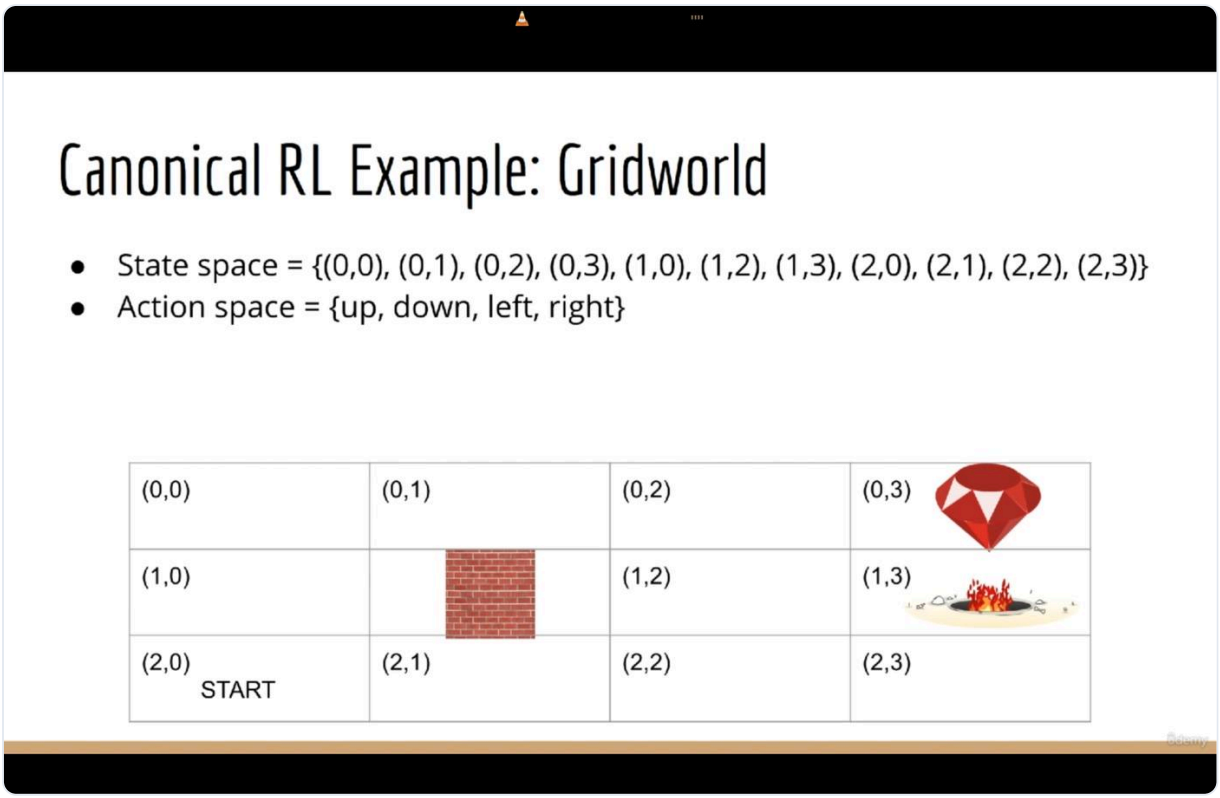


FIGURE 51.31 | Canonical RL Example: Gridworld

Now the reason we had to talk about grid world for a bit is because for our other examples such as tic tac toe and breakout the state space is much more complicated. The action spaces are pretty simple since for tic tac toe it consists of all the possible positions you can draw an X or O. And for breakout it consists of moving left right or doing nothing but for tic tac toe the state space is quite large as there are many possible configurations of the board as an exercise.

FIGURE 51.32 | Large State Spaces

I would strongly recommend trying to write a computer program that can enumerate all the possible configurations of a tic tac toe board. This should give you some intuition about why games like chess and go are very difficult. You can imagine that if a three by three board with only two possible characters can have thousands of states imagine how many states are involved in chess and go for breakout. The number of states is even larger. It's equal to the screen resolution multiplied by the number of possible colors per pixel two to the power twenty four or two to the power eight to the power of three. But for all intents and purposes we can consider images just like Time series to be continuous valued.

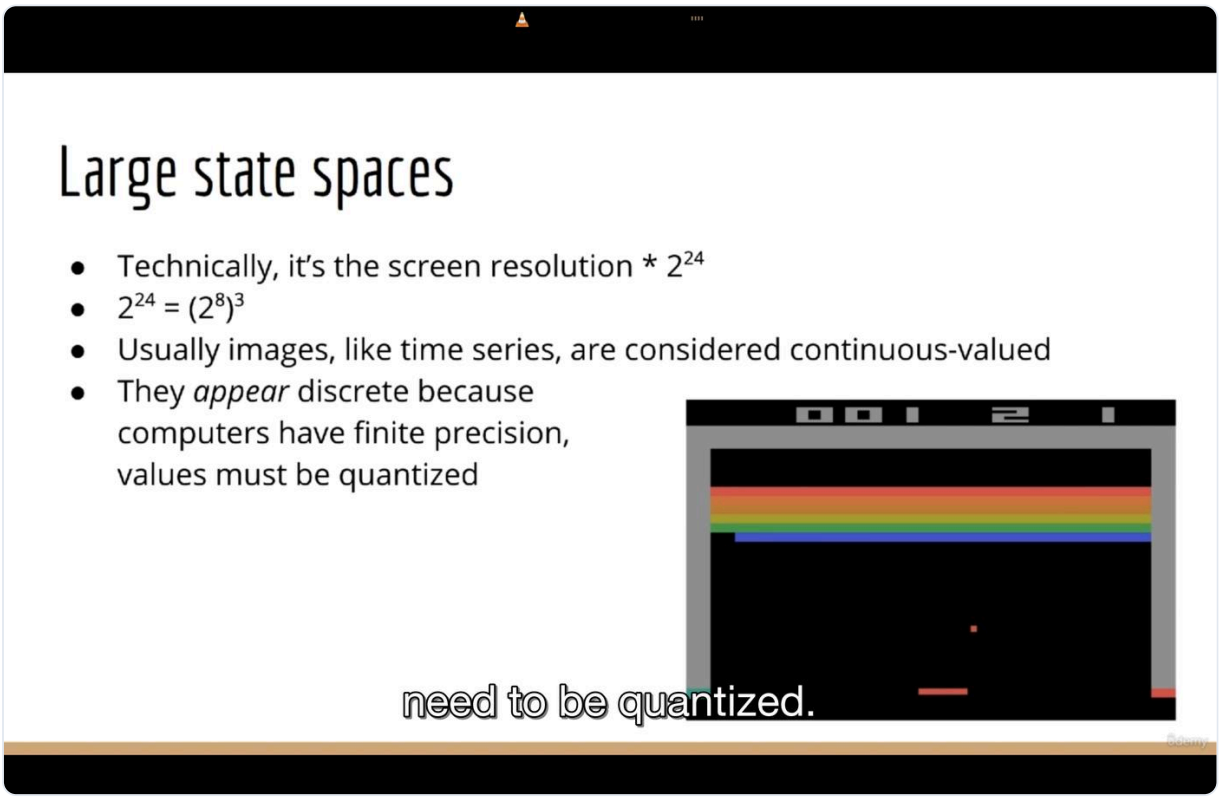


FIGURE 51.33 | Large State Spaces

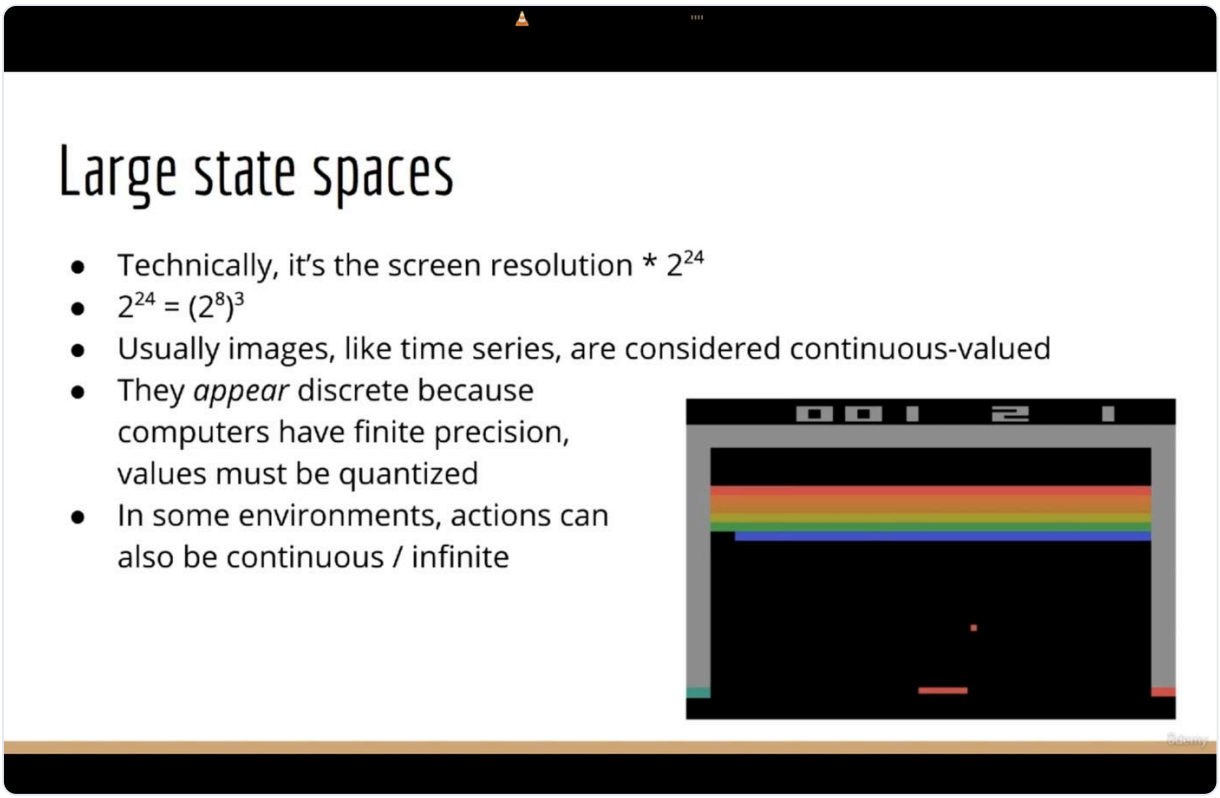


FIGURE 51.34 | Large State Spaces

The only reason they appear to be the screen is because computers have finite precision and so the values need to be quantized. When we have continuous values this means that the number of possible values is actually infinite. In fact it's possible for actions to be continuous also so that the action space is also infinite since this lecture was quite long. Let's summarize everything we learned. This lecture was all about defining some reinforcement learning terminology to help us in our discussion of reinforcement learning. First we define the terms agent and environment. You can think of the environment as the world or whatever computer game you are teaching your agent to win.

You can think of your agent as your computer program the one that does the learning. Next we define the term episode. This is like one round or one match of a game as you know machine learning models learn through data or in reinforcement learning parlance experience and so you can imagine that in order to sufficiently learn how to play a game.

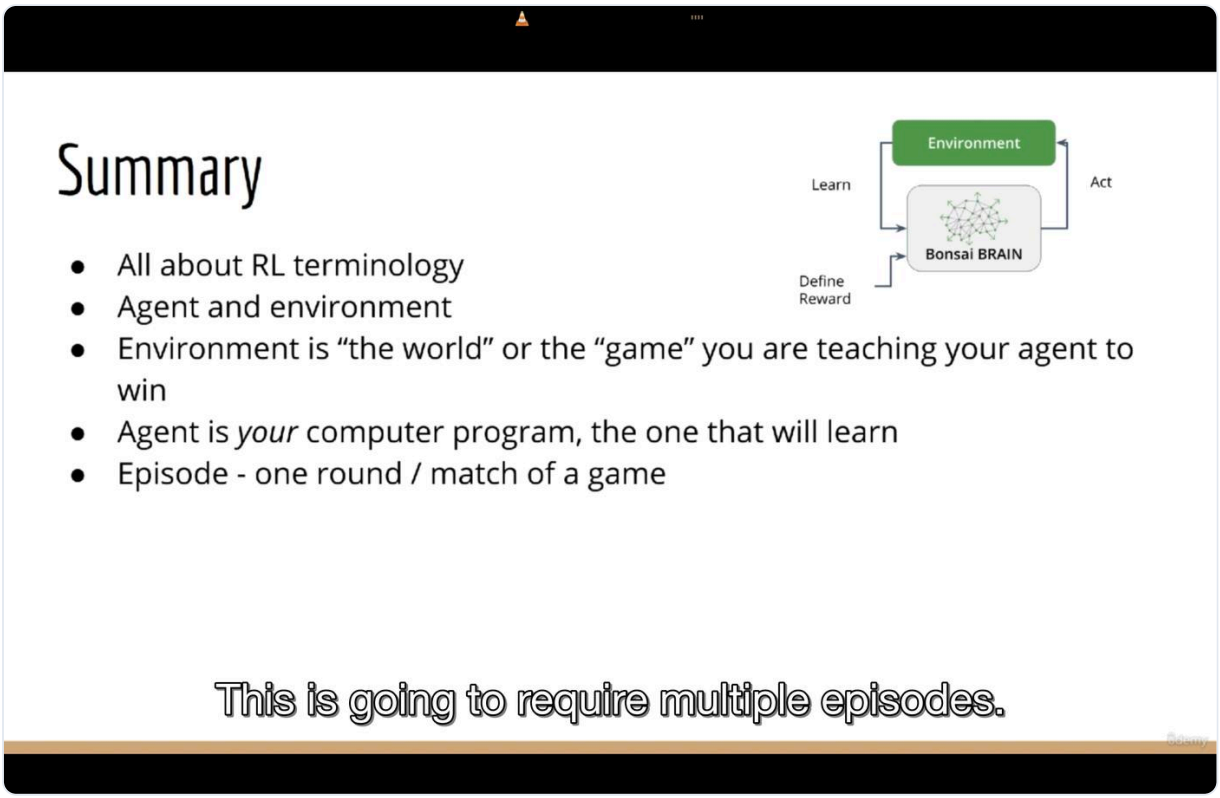


FIGURE 51.35

This is going to require multiple episodes. Next we learn about states actions and rewards. Rewards are a number which can be any number positive or negative. The job of a reinforcement learning agent is to maximize its reward actions are what an agent does in an environment. For example playing a move in tic tac toe are going left or right.

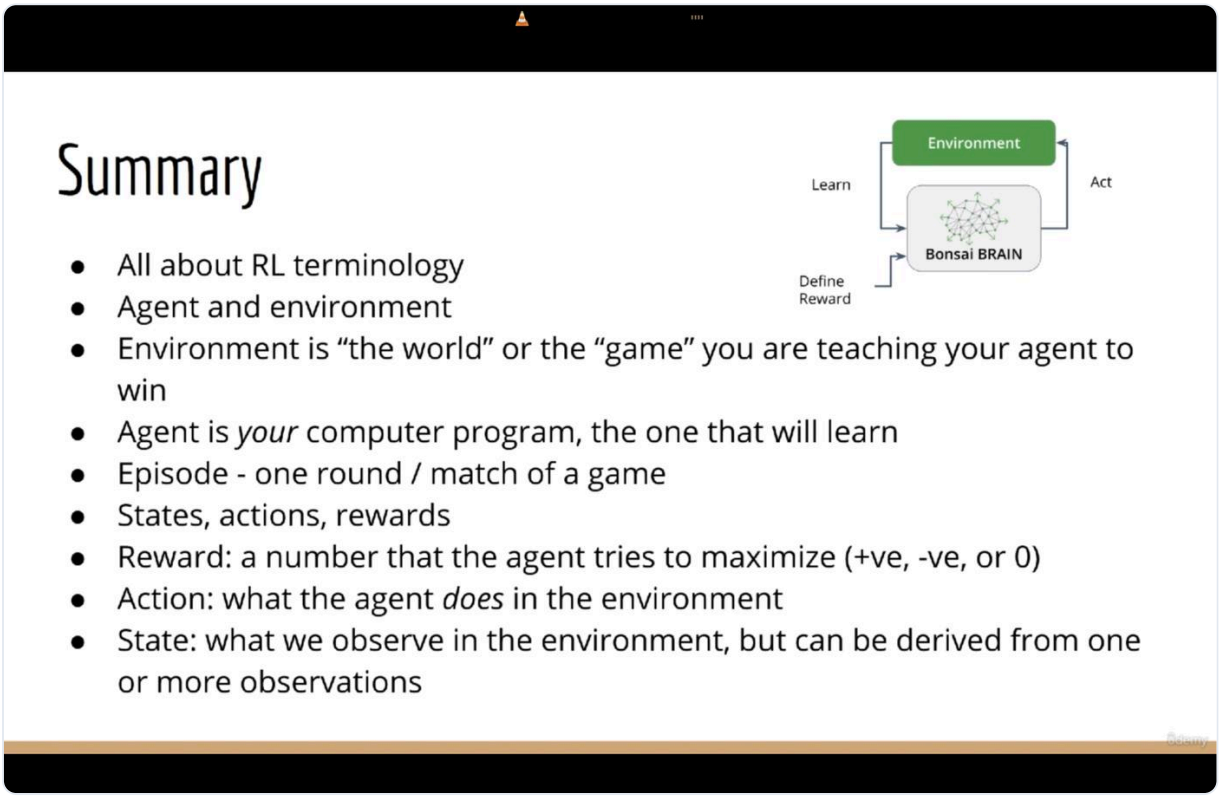


FIGURE 51.36 | All About RL Terminology Cm LO

In a video game states are what we observe from the environment but they can also be values derived from those observations or even a sequence of past observations.

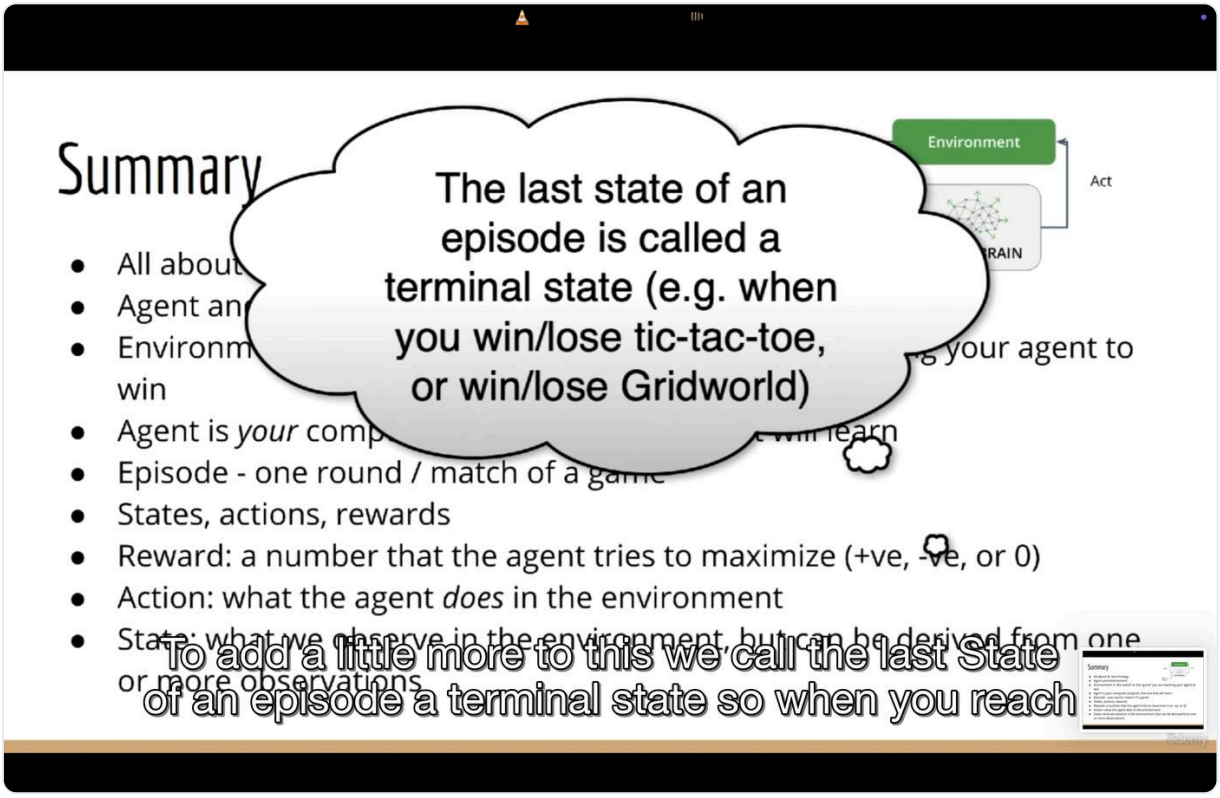


FIGURE 51.37 | The Last State of an

To add a little more to this we call the last State of an episode a terminal state so when you reach a terminal state that is the end of your episode.

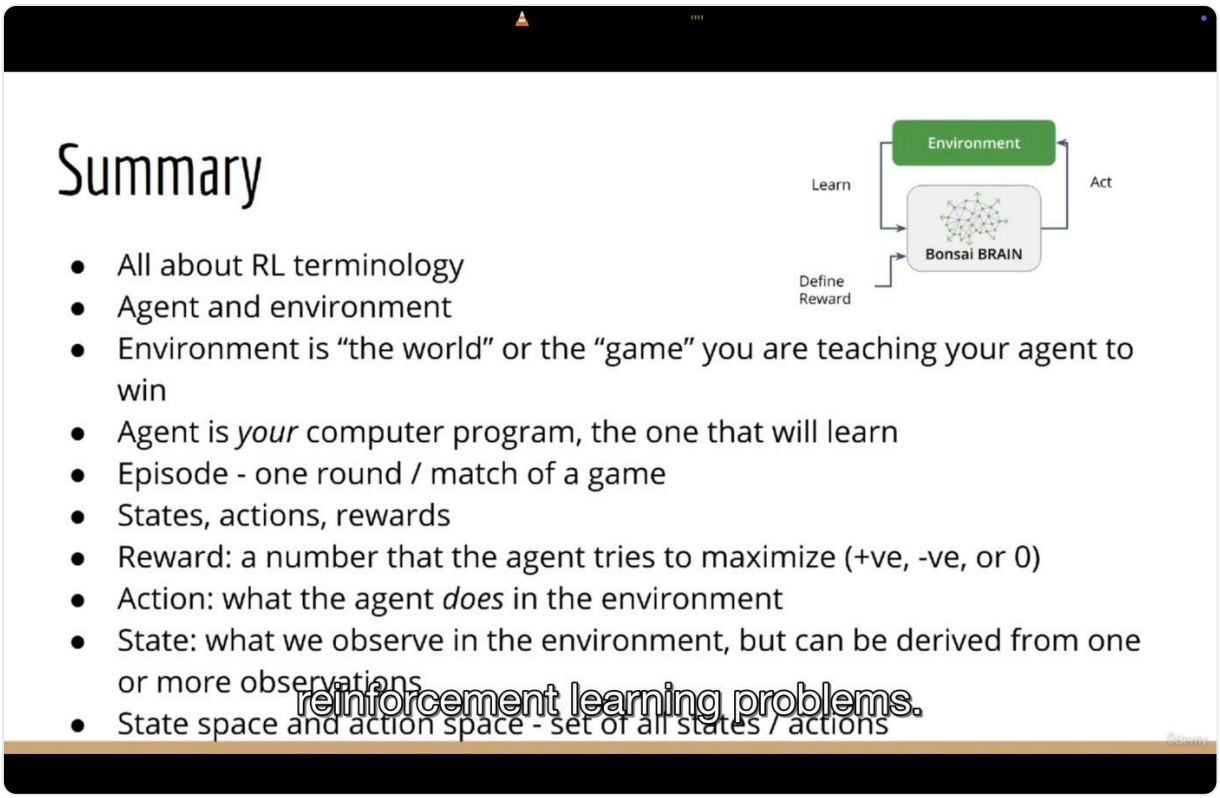


FIGURE 51.38

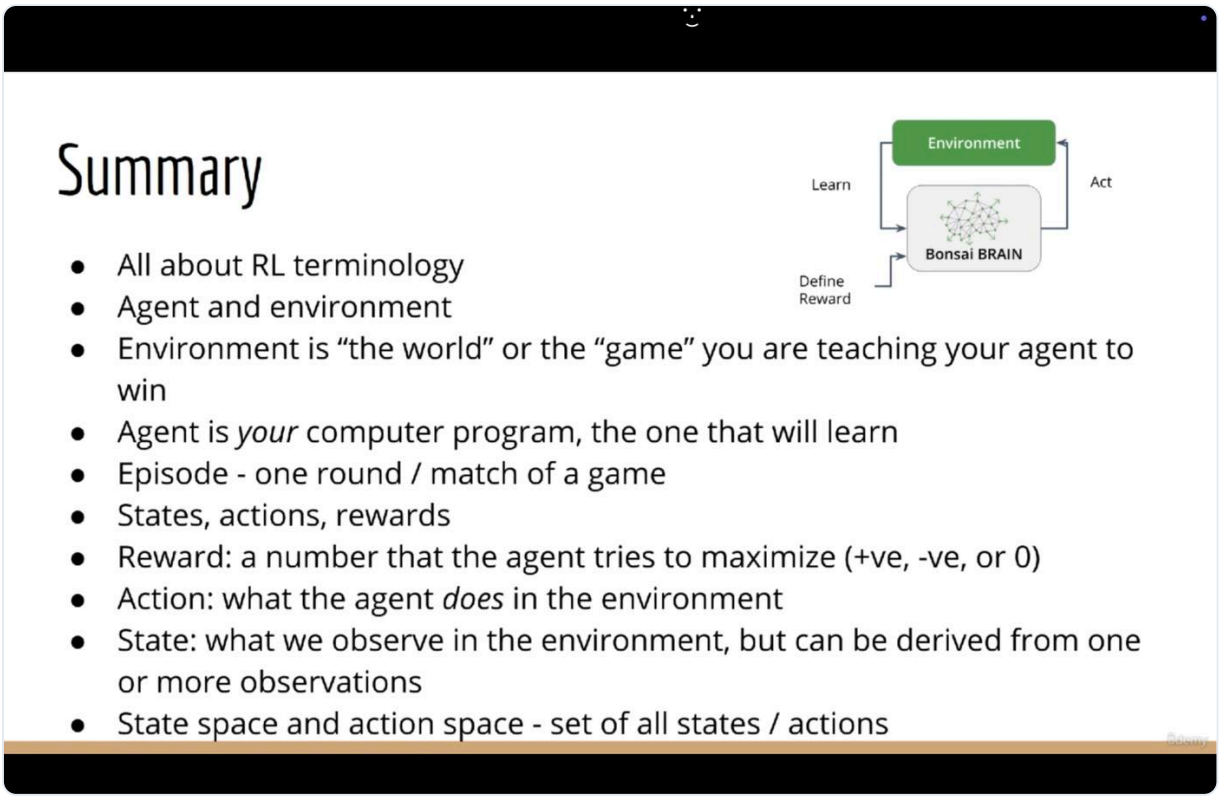


FIGURE 51.39

Lastly we define the terms state space and action space. These are the set of all states and the set of all actions respectively using these terms. We can now talk about reinforcement learning coherently and build a framework that allows us to solve reinforcement learning problems.

LECTURE 90

Policy as a Function or Mapping

In this lecture we are going to discuss the concept of states actions and rewards. More in-depth this lecture is about how we encode states and actions when we are programming talking about states actions and rewards theoretically is nice but eventually we're going to have to put this into code as a side note.

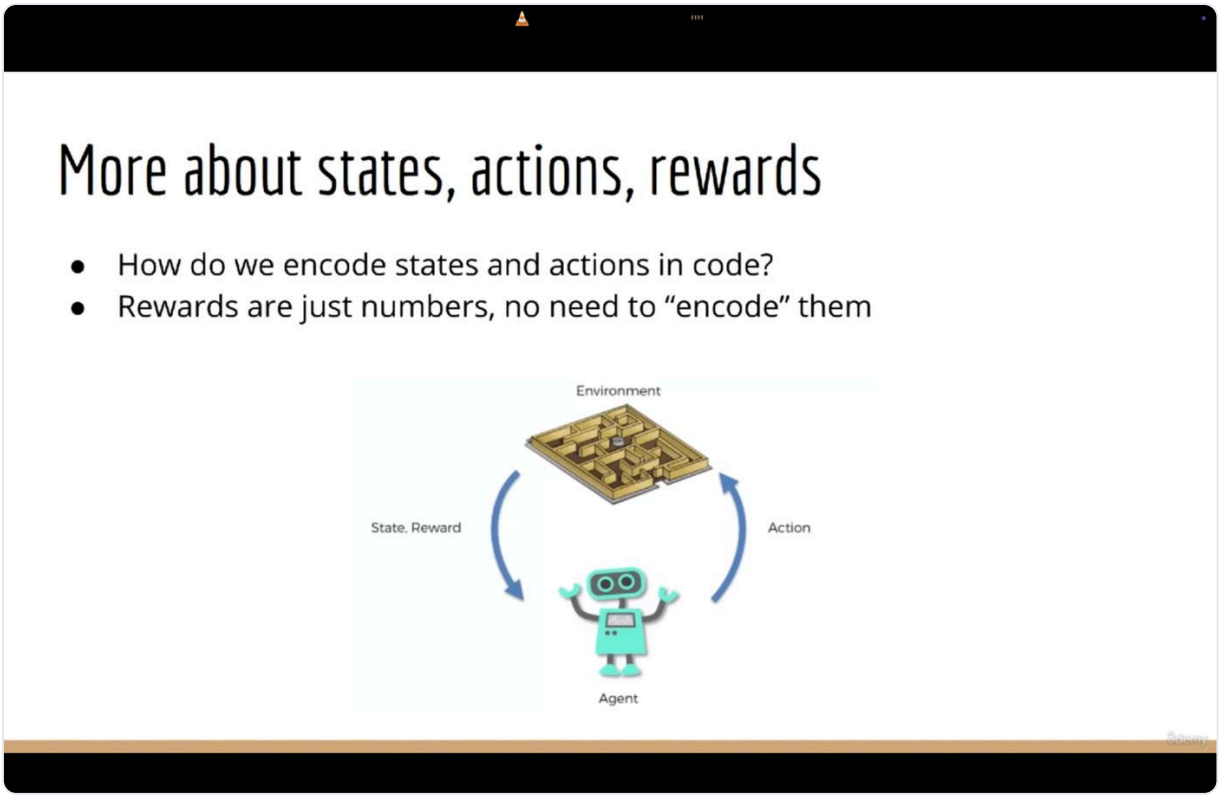


FIGURE 52.1 | More About States, Actions, Rewards

Since rewards are just numbers this is trivial. There is no need to discuss how we represent numbers in code let's start with the state.

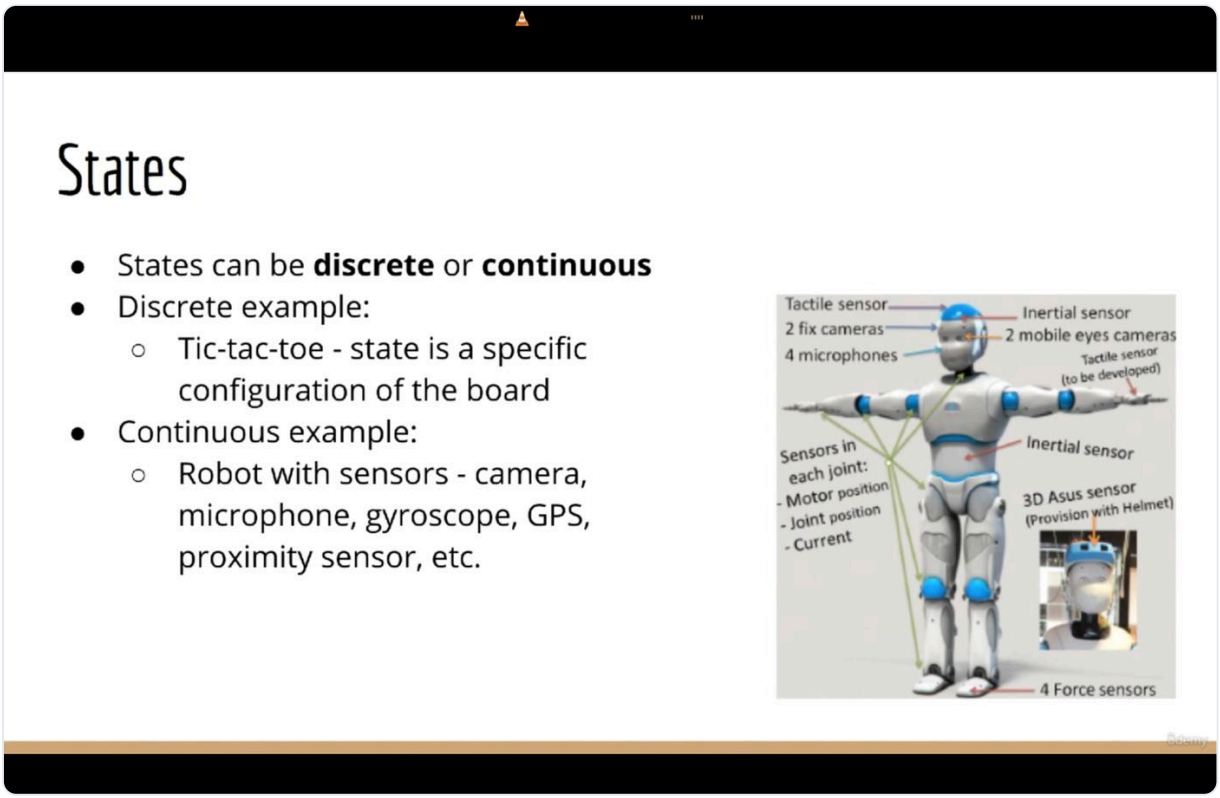


FIGURE 52.2

As mentioned previously the state can be discrete or continuous in a game like tic tac toe. The states are discrete because they are just different configurations of the tic tac toe board. If we make a robot that has sensors like a camera a microphone a gyroscope a proximity sensor and so forth. Those would all be continuous values so you would end up with a vector of continuous values. In fact this takes us back to regular supervised learning. If our targets are categorical how do we represent them in code. If there are K categories we will use the integers 0 up to K minus 1. So we might say that dog equals zero cat equals one mouse equals two and so forth.

Obviously it does not matter which category is assigned to which number. And the reason we want to do this is because at some point we're going to have to use these categories as Array indices. And so similarly if we have big S states then we'll represent them in code using the integers 0 up to a big S minus 1 for continuous values. It makes sense to store them in a vector although if you have something like an image then it will be a three dimensional tensor.

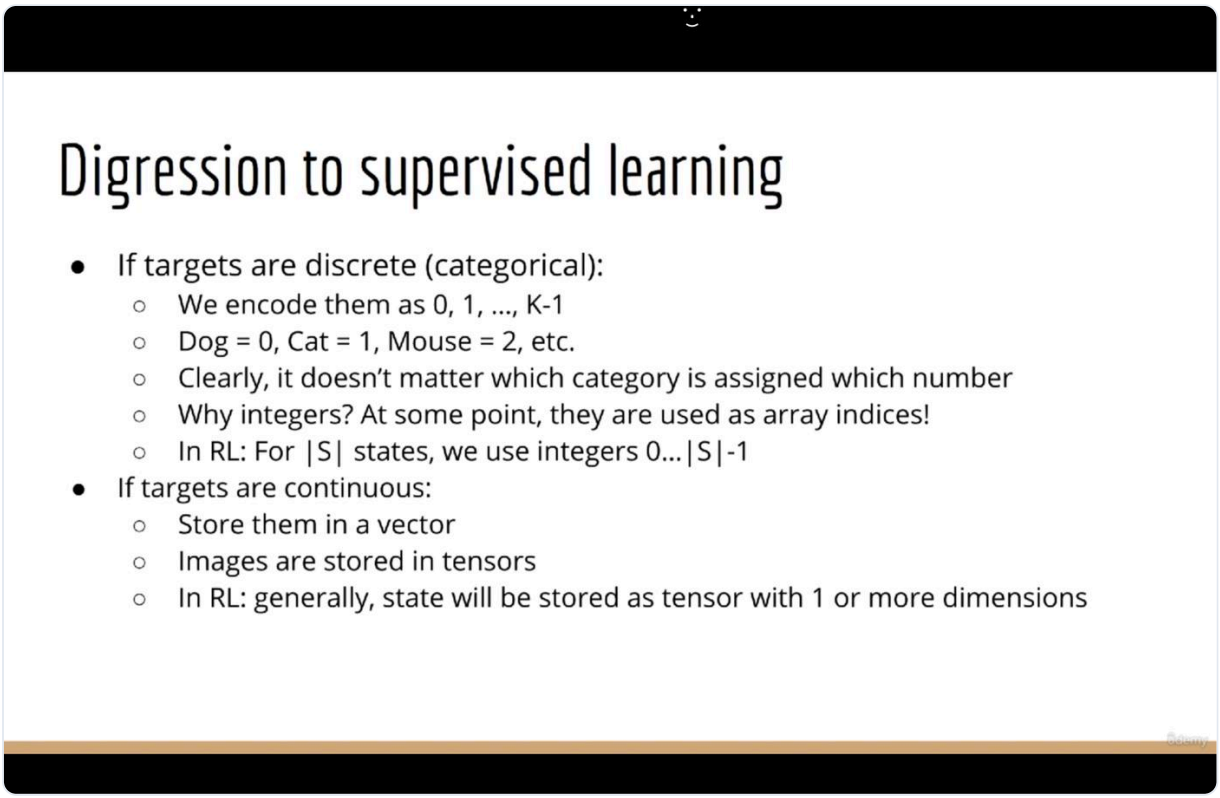


FIGURE 52.3 | Digression to Supervised Learning

So more generically if you have continuous values you can think of them as a tensor with one or more dimensions now that you know how to represent states and actions in code. It's time to talk about policies. When I first started learning about reinforcement learning I found policies to be kind of an odd concept. Well actually I found all of reinforcement learning to be kind of odd and foreign but you'll get used to it. The idea of a policy makes sense at a high level but it becomes somewhat ambiguous when you start thinking about how to represent it mathematically or in code.

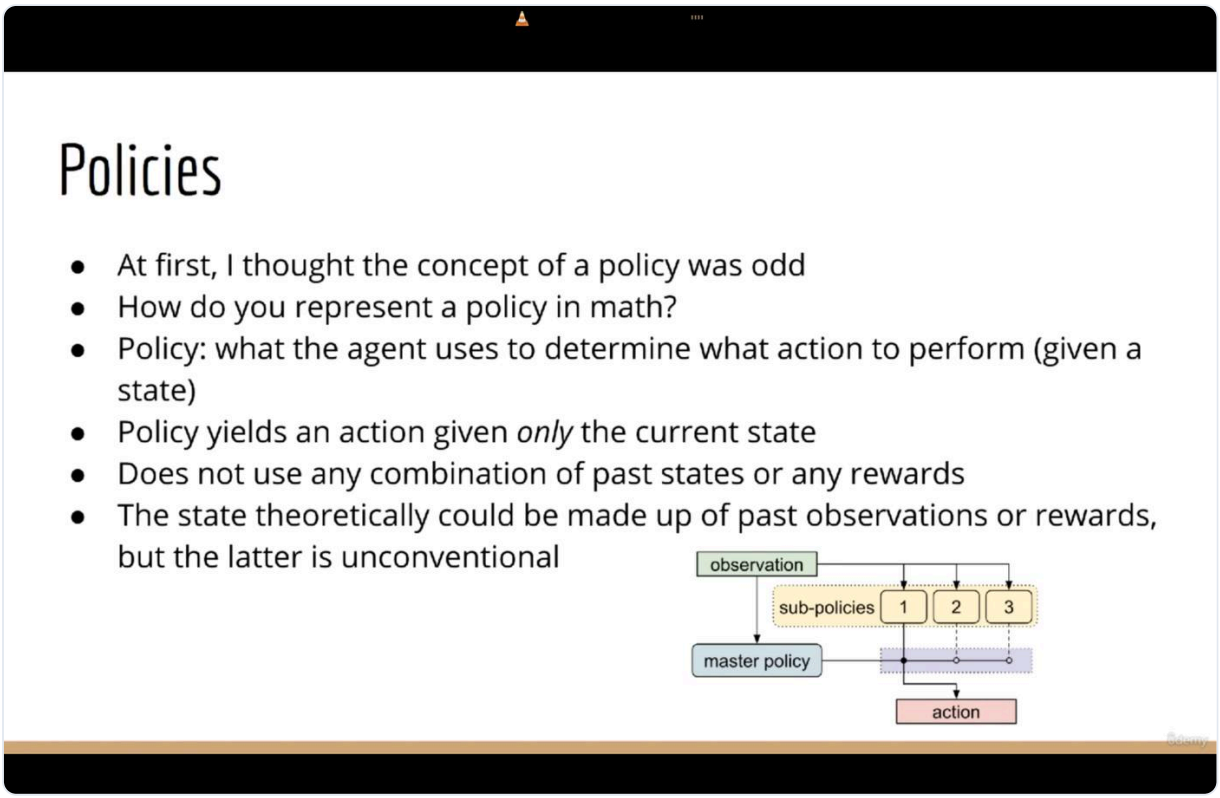


FIGURE 52.4 | How Do You Represent a Policy in Math?

The policy is what the agent uses to determine what action to perform given a state. It's important to keep in mind that the policy yields and action using only the current state it doesn't use any combination of the current state and previous states and it doesn't use any information about rewards technically as I mentioned earlier. The state may be made up of multiple observations and that may also include the reward although that is unconventional but strictly speaking the policy will yield an action using only the current state the simplest way to think about a policy is that it's a dictionary mapping or a function that returns in action.

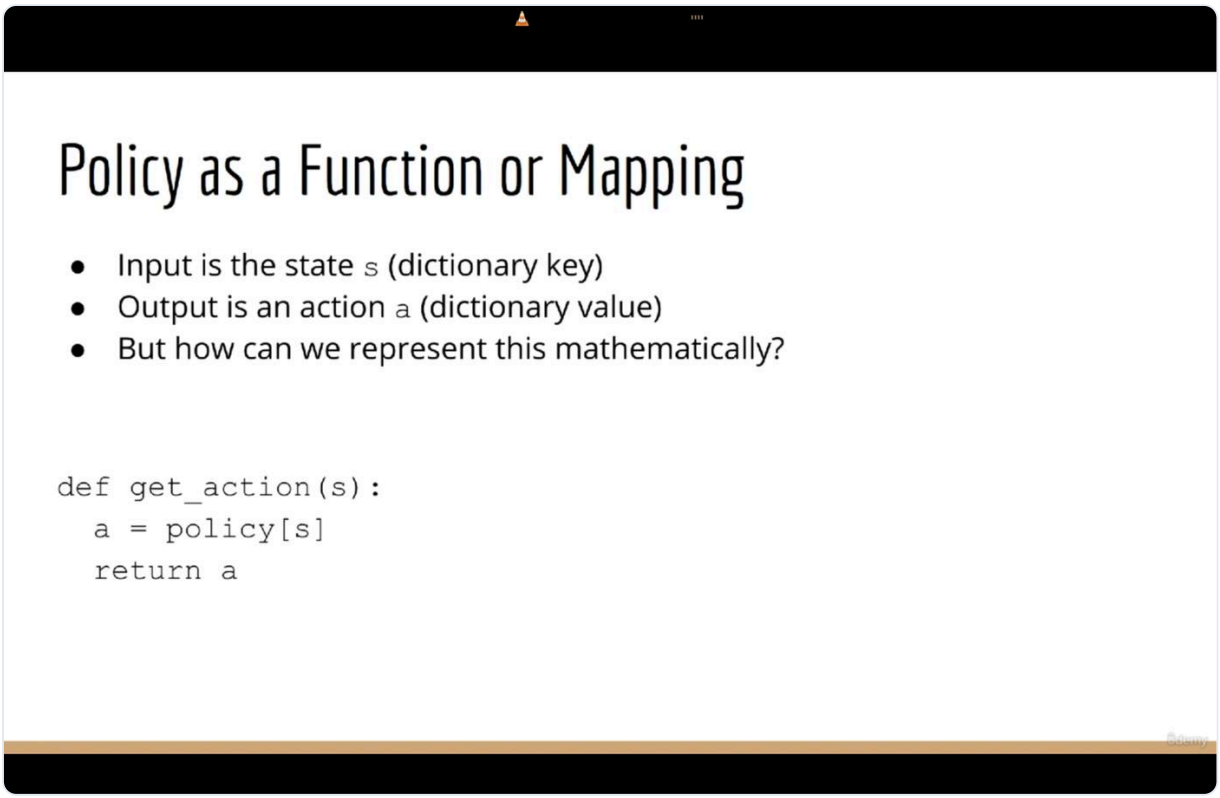


FIGURE 52.5 | Policy as a Function or Mapping

Given a state here is such a function as you can see the only input is a state s and it returns an action a where the state s was the key to the dictionary and the action a was the value the real question is how can we represent this mathematically this is why it's useful to talk about how we encode states and actions first. So imagine again we are in grid world your agent now has a dictionary representing what action to perform given the state as you can see here. So for example if we're to the left of the goal state then the appropriate action is to move right and just to be clear the left of the goal state is 0 2 if we're at the initial state then the appropriate action is to move up and just to be clear that is the state 2 0 now from the initial state.

Moving right is just as valid since we can still reach the goal from that point. You can see that although I've encoded the states here explicitly as tuples we could gain more efficiency by encoding them as integers corresponding to the tuples and using those integers to index an array.

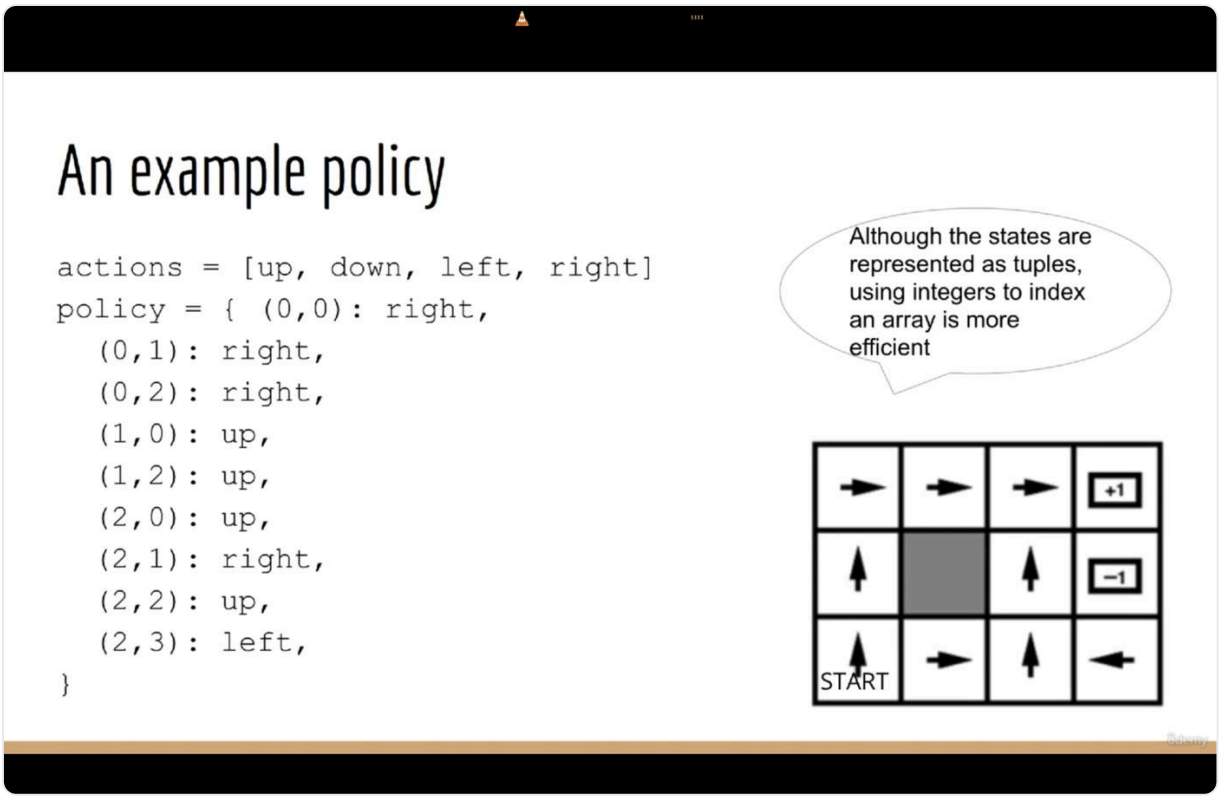


FIGURE 52.6 | An Example Policy

As you may recall from your computer science studies indexing arrays is faster than indexing dictionaries thinking about policies as dictionary mappings is somewhat limited. There are two reasons for this. The first is that this won't work. If you have an infinite state space you would need an infinite sized dictionary which is not possible to have. Second is that it doesn't allow our agent to explore its environment.

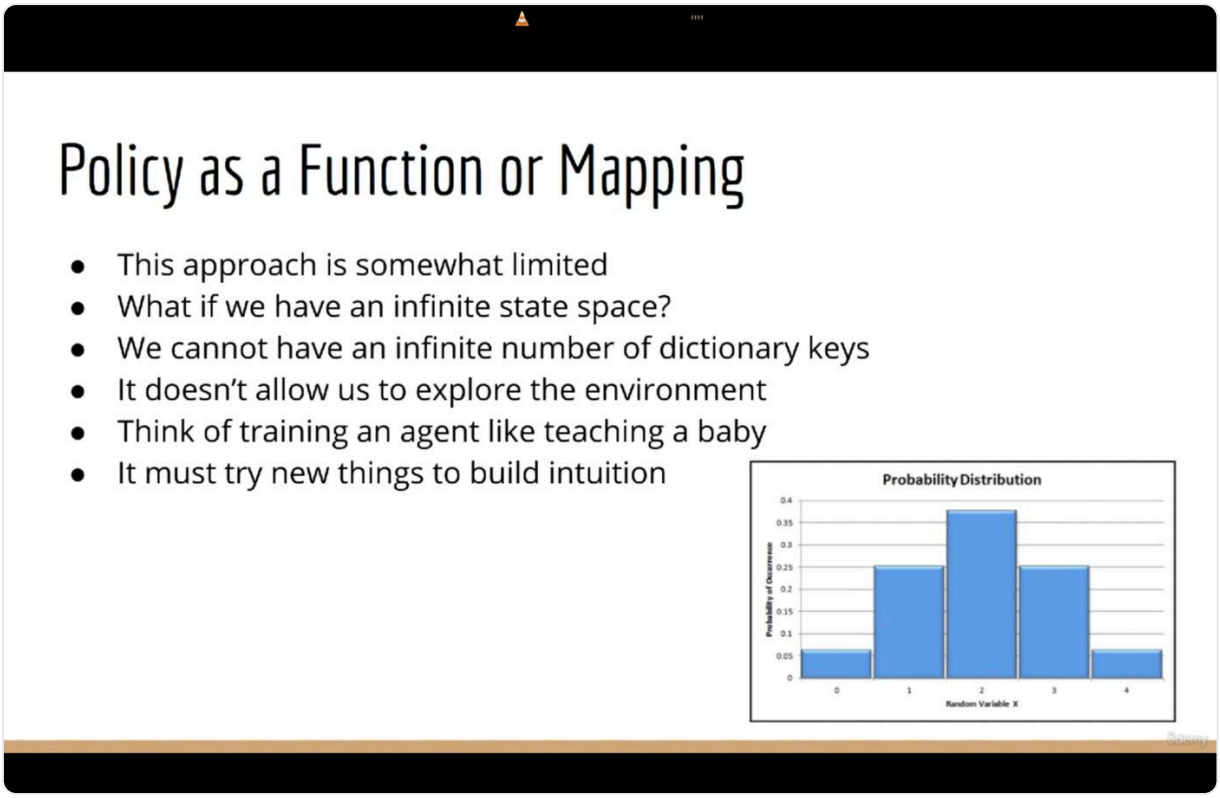


FIGURE 52.7 | Policy as a Function or Mapping

Think of training your agent like teaching a baby at first. A baby knows nothing. It must try new things in order to figure out how the world works and build up its intuition a reinforcement learning agent is the same way. If it has a fixed policy and it only does the same thing all the time then it can't gain new experiences. Thus it makes sense for policies to be stochastic stochastic is just a fancy word for random.

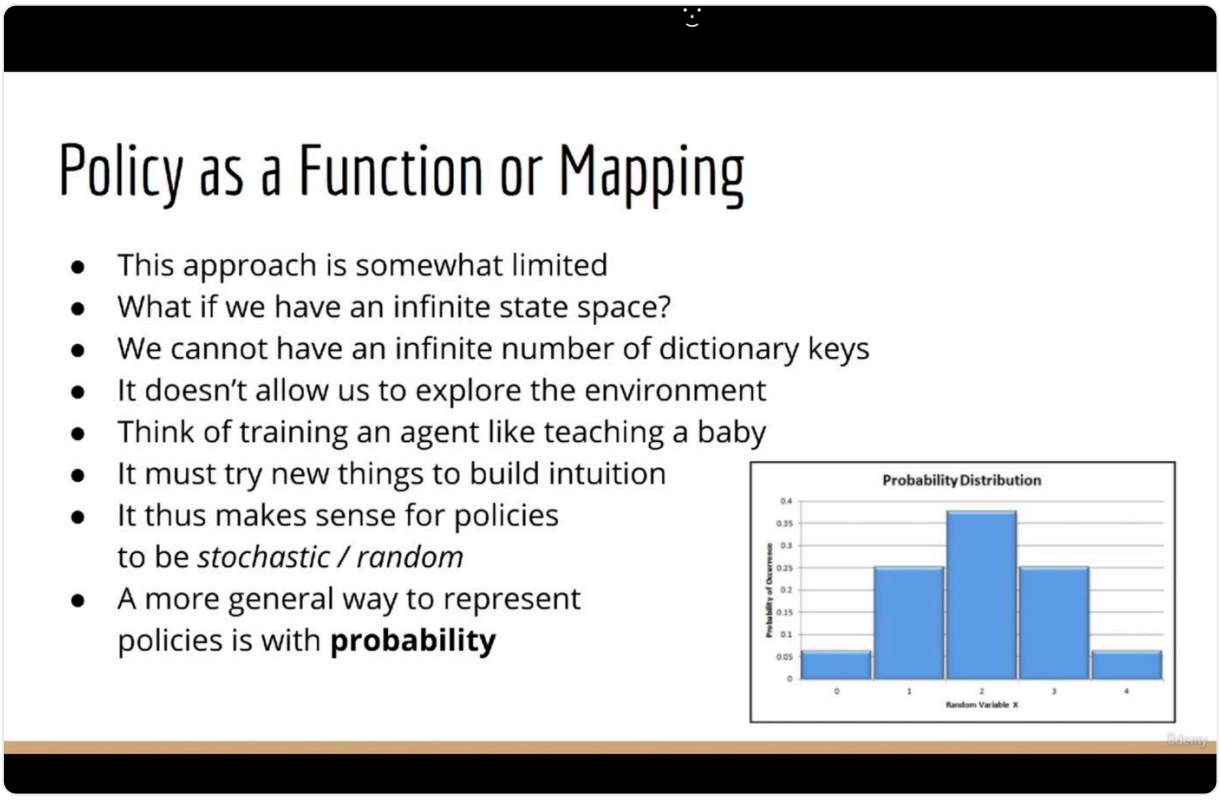


FIGURE 52.8 | Policy as a Function or Mapping

In other words a more general way to represent policies is to represent them as probabilities representing policies as probabilities actually solves both of the problems I posed above. Let's see how first it deals with this problem of randomness. The common way of dealing with this in reinforcement learning to allow the agent to explore is to give it a small chance of performing a random action. So here is a python function that can accomplish this. We first generate a random number. If this number is less than some small number Epsilon let's say zero point one then it will choose an action at random from the action space otherwise we will grab an action from our fixed policy dictionary mapping.

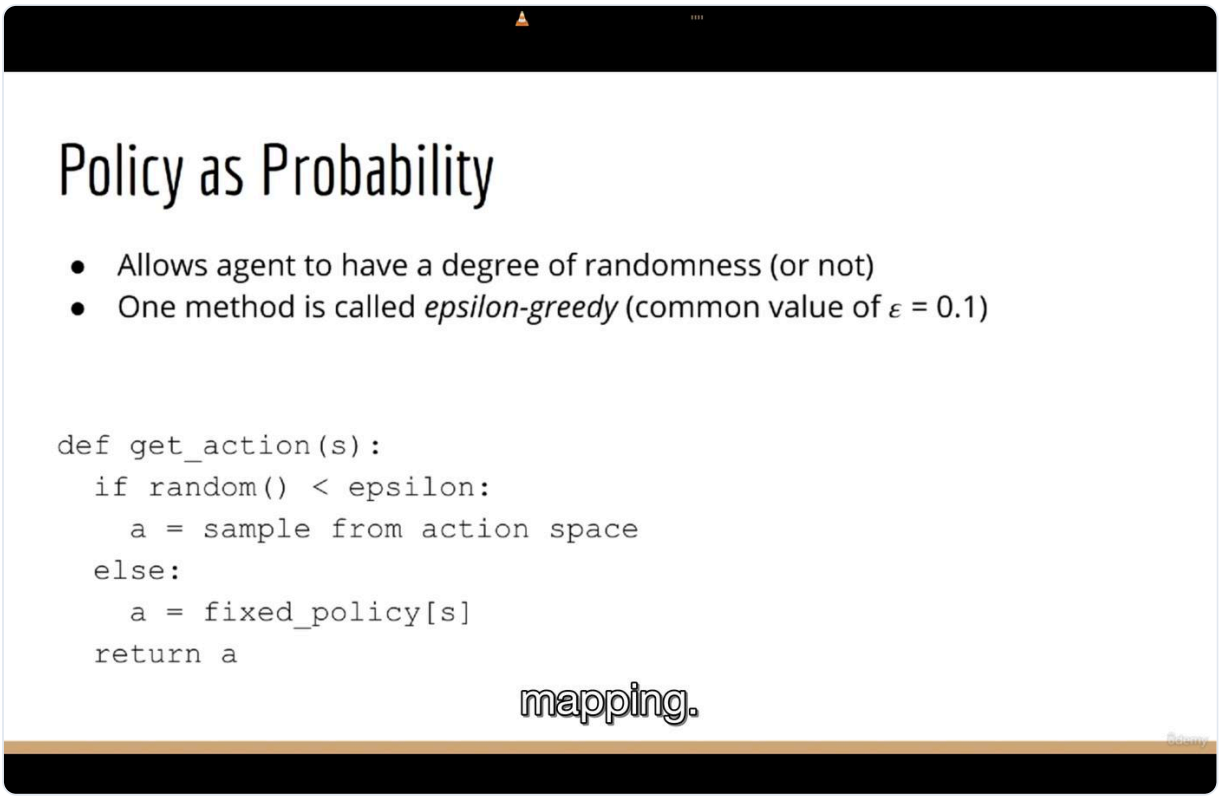


FIGURE 52.9 | Policy as Probability

This method is called Epsilon greedy and you'll learn later in this section of why it's useful and what the relevance of exploration is. So what about continuous state spaces. In fact seeing policies as probabilistic easily lends itself to continuous or infinite state spaces. Imagine your state is a vector s. Now imagine we have some policy parameters w the shape of W is the dimensionality of the state space by the size of the action space. For now we'll presume that the action space is still categorical So what do we do when we want to output probabilities for a set of categories. In fact this is just like classification we can use the softmax function.

So now our policy is the softmax of W dotted with the state s as you can see.

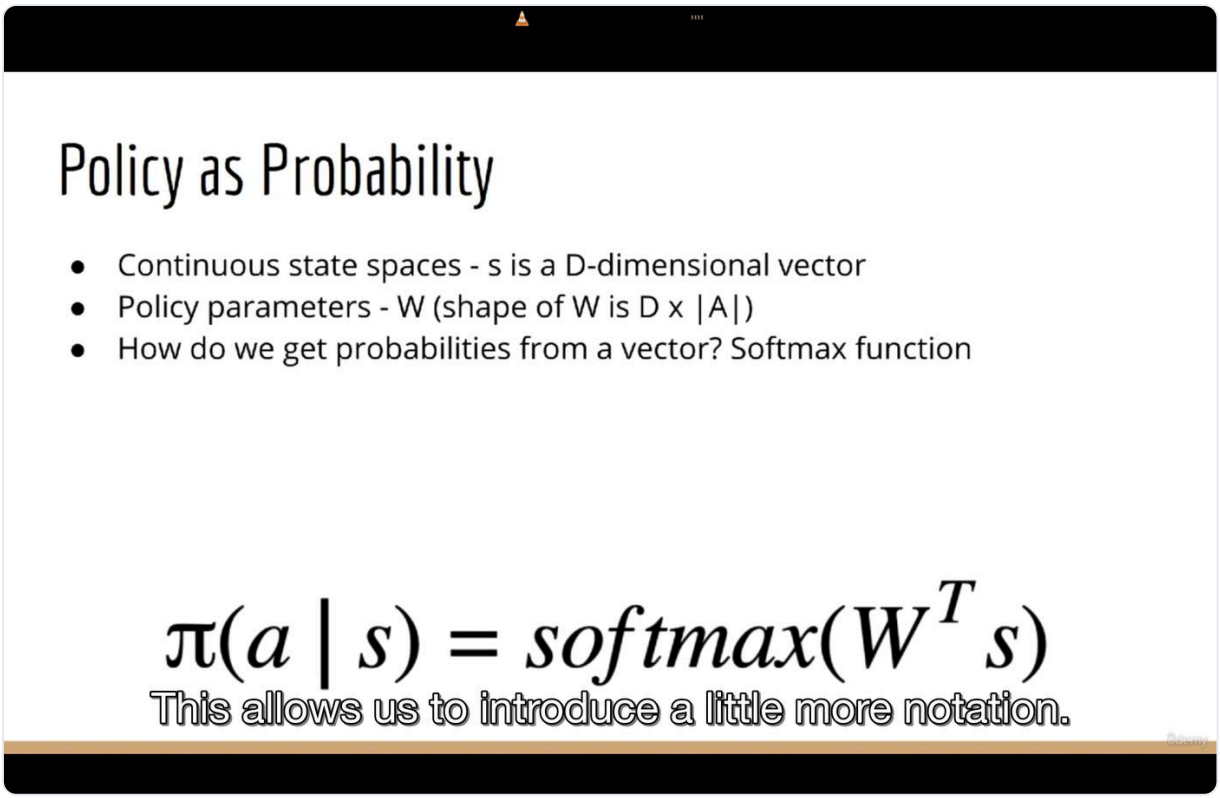


FIGURE 52.10 | Policy as Probability

This allows us to introduce a little more notation. It's common in reinforcement learning to denote the policy with the symbol pi not to be confused with the number pi. For any given state we can calculate a probability distribution over the actions base Pi of a given S. Then to decide which action to perform in the environment. We can simply sample from this distribution.

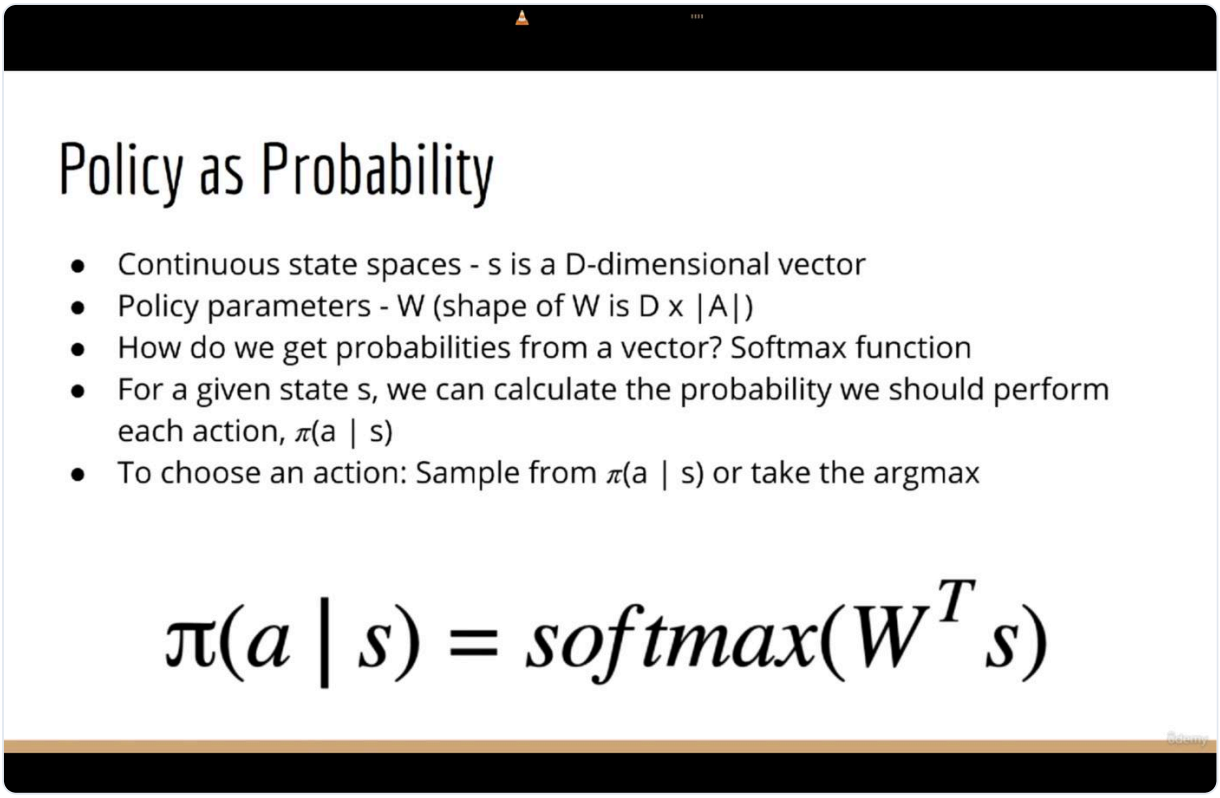


FIGURE 52.11 | Policy as Probability

This allows us to explore if necessary but we can still treat this policy deterministically. If we want simply by using the argmax also note that it's not necessary to use a linear model as we are doing here. We can in fact use any function approximator such as a neural network now at this point. You may be wondering how can an intelligent agent possibly know what to do using only the current state. This is the problem we described before just by looking at a still image of the road how can I know what the correct action is if you're more inclined to think inside a supervised learning paradigm.

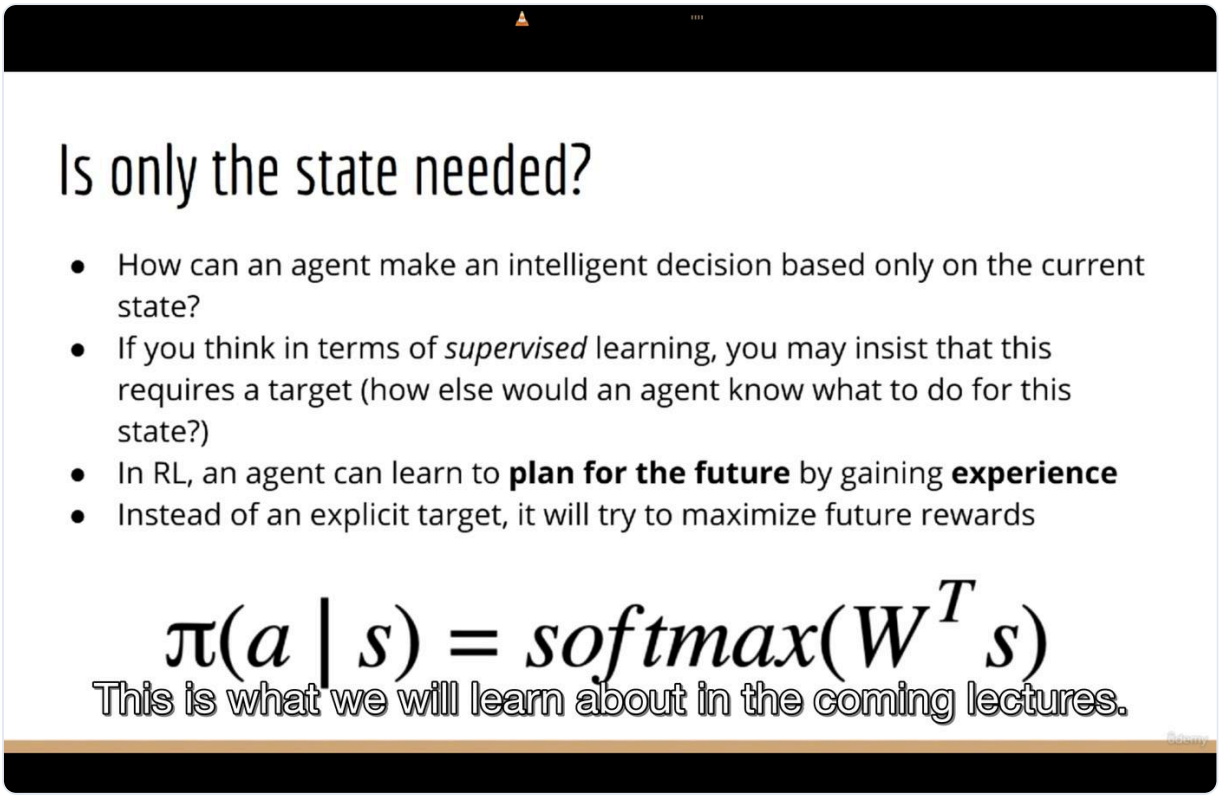


FIGURE 52.12

You may insist that there should be a target here so that the agent learns to associate this state with this action but in fact it's quite possible for an agent to learn how to plan for the future using experience collected by playing multiple episodes even without an explicit target for a given state. The agent can still learn what action to perform such that it maximizes rewards in the future. This is what we will learn about in the coming lectures.

LECTURE 91

The Markov Assumption

Previously we learned about terminology the various words we use to describe reinforcement learning problems which allow us to solve reinforcement learning problems. Now that you understand concepts such as agents environments policies states actions and rewards we can build on this. The goal is to have a framework which we can then use to find solutions. So at this stage we are still working to more accurately and more narrowly define a problem.

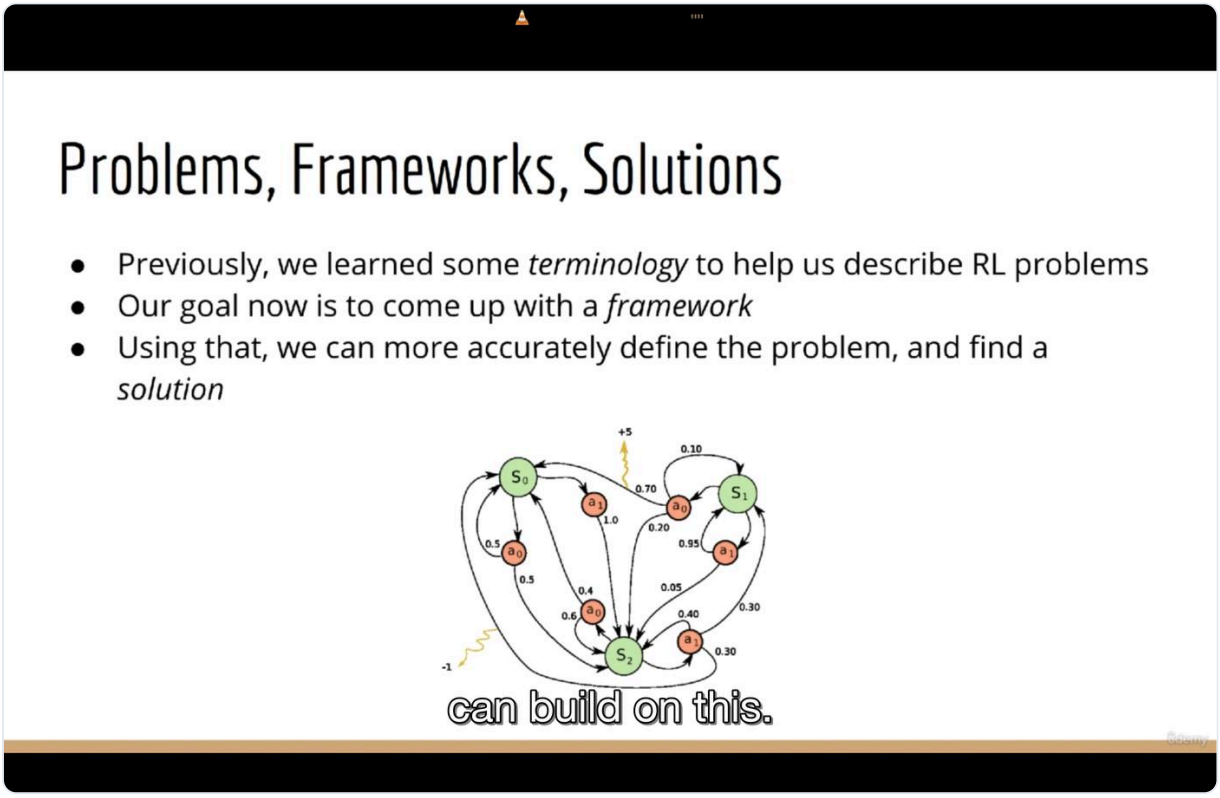


FIGURE 53.1

Once we have an accurately defined problem we can work within this framework and whatever assumptions it involves to derive solutions. The main assumption we make in the reinforcement learning is the Markov assumption. This is something we often discuss in terms of Markov models and sequence modelling.

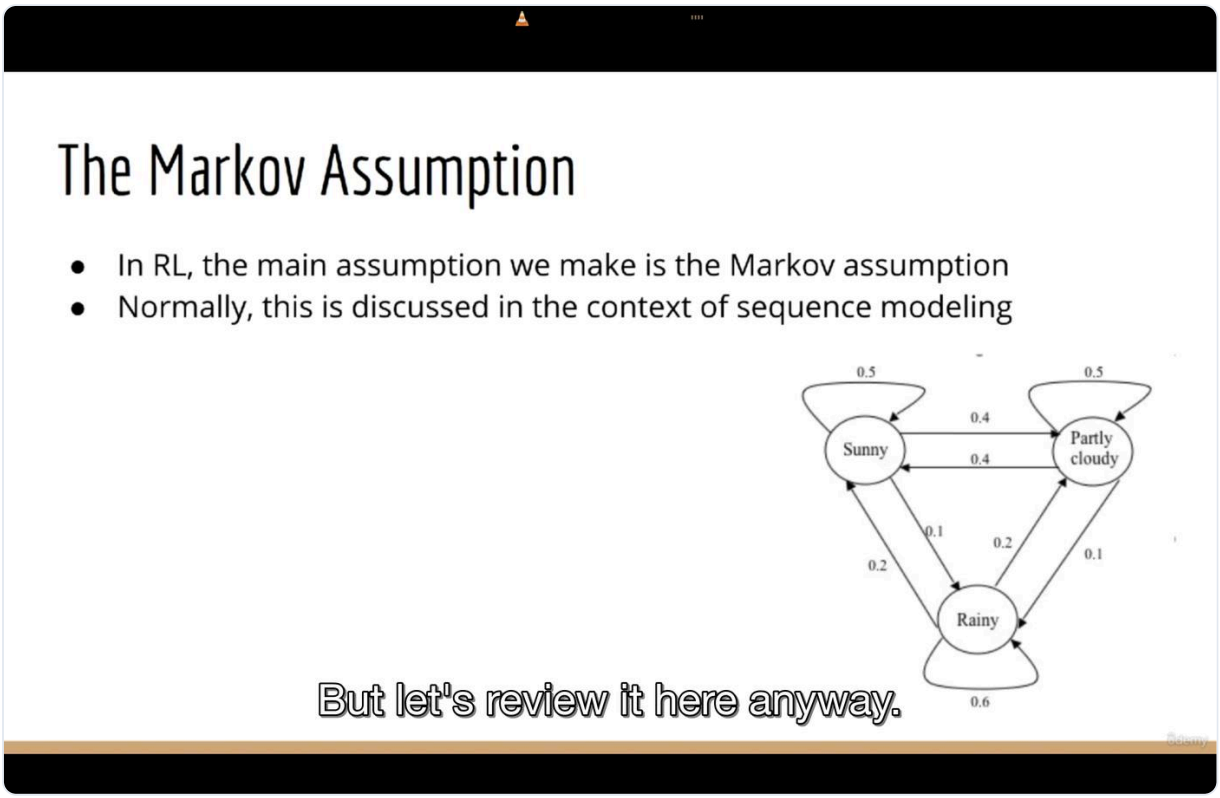


FIGURE 53.2 | The Markov Assumption

But let's review it here anyway. The Markov assumption goes like this.

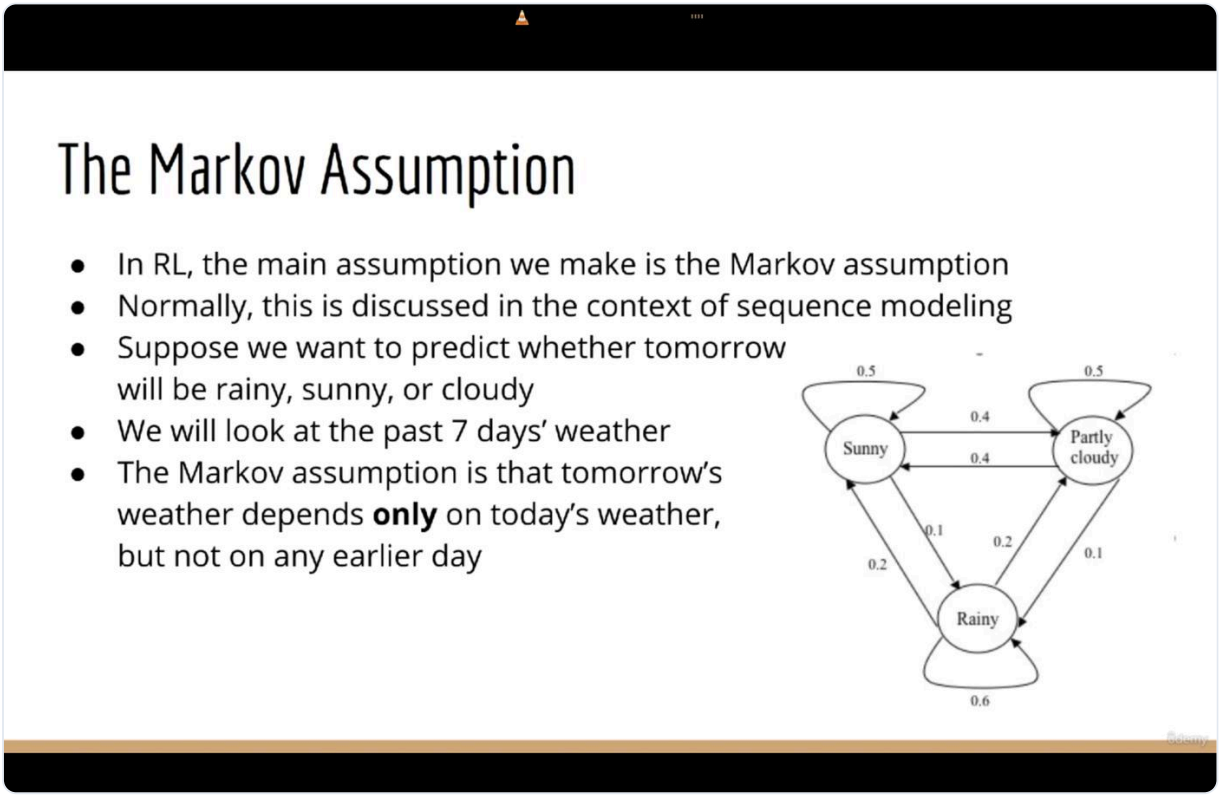


FIGURE 53.3 | The Markov Assumption

Suppose we want to predict whether tomorrow will be rainy sunny or cloudy. Perhaps your idea may be to base this on whether it was raining sunny or cloudy in the past seven days. Well the Markov assumption is that tomorrow's weather it doesn't depend on all of the past seven days. Only the immediate pre-

vious day here's another example of the Markov assumption. Suppose I want to predict the next word of a sentence. I tell you that the previous word in the sentence is lazy.

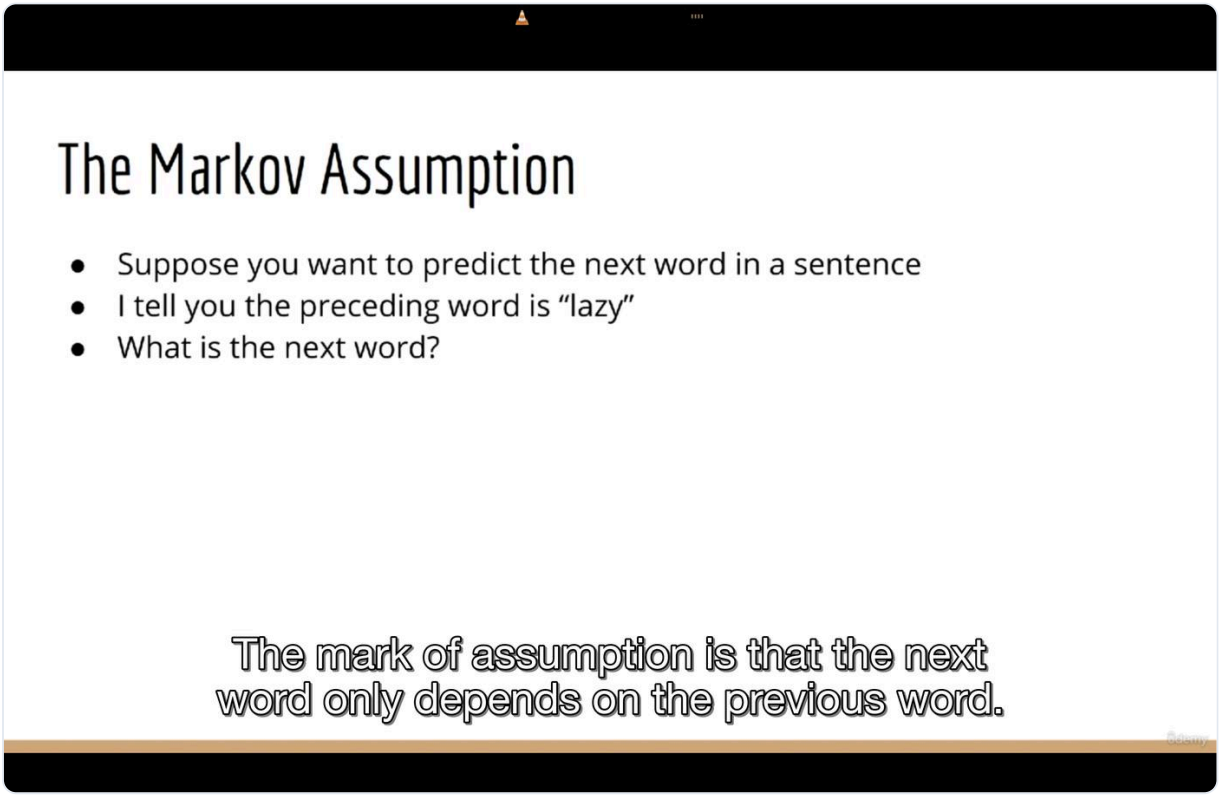


FIGURE 53.4

The Markov assumption is that the next word only depends on the previous word. Therefore if the Markov assumption is true then you should be able to predict the next word in my sentence. Of course things are not so easy you might think because you are taking a course by the lazy programmer that the next word in the sentence is programmer but in fact that's not what I had in mind. Now let's suppose I tell you the full sentence so far is the quick brown fox jumps over the lazy.

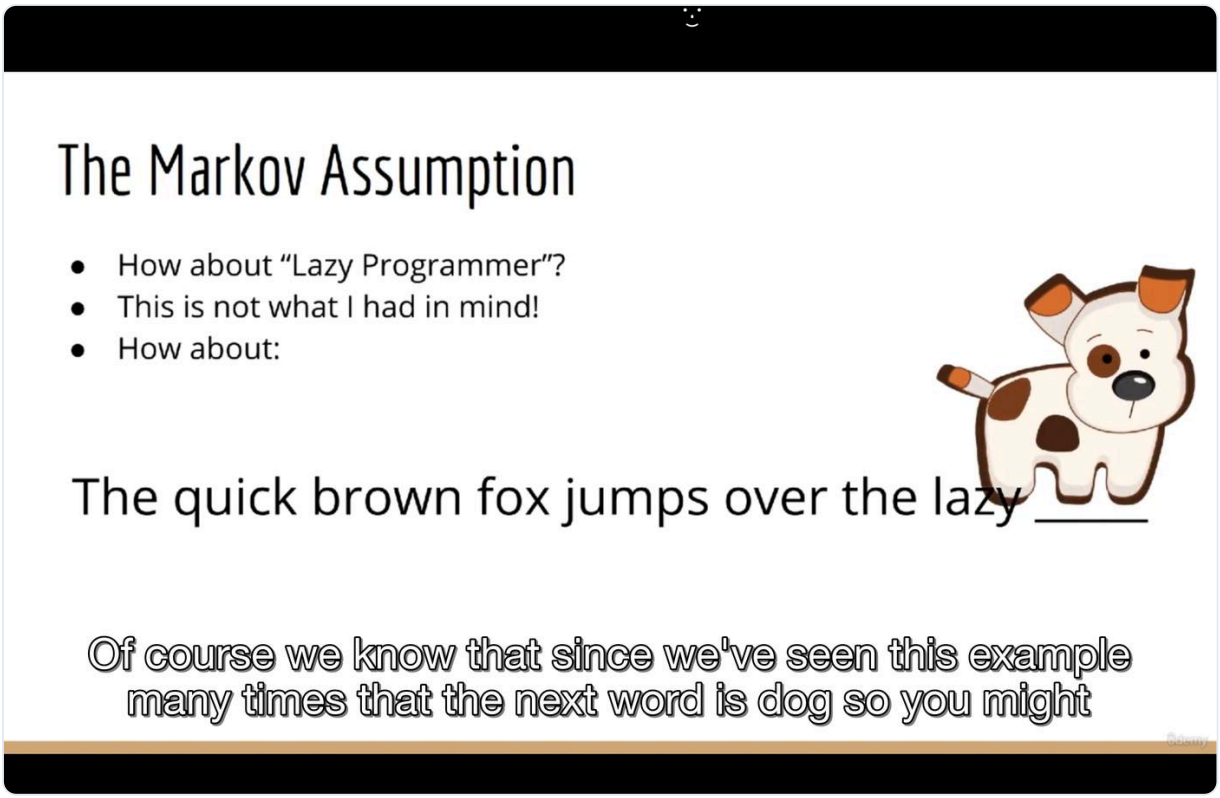


FIGURE 53.5 | The Markov Assumption

Of course we know that since we've seen this example many times that the next word is dog so you might think this Markov assumption thing doesn't really seem to be a great idea. In fact there has been some work in reinforcement learning where they do not make use of the Markov assumption although that is outside the scope of this course.

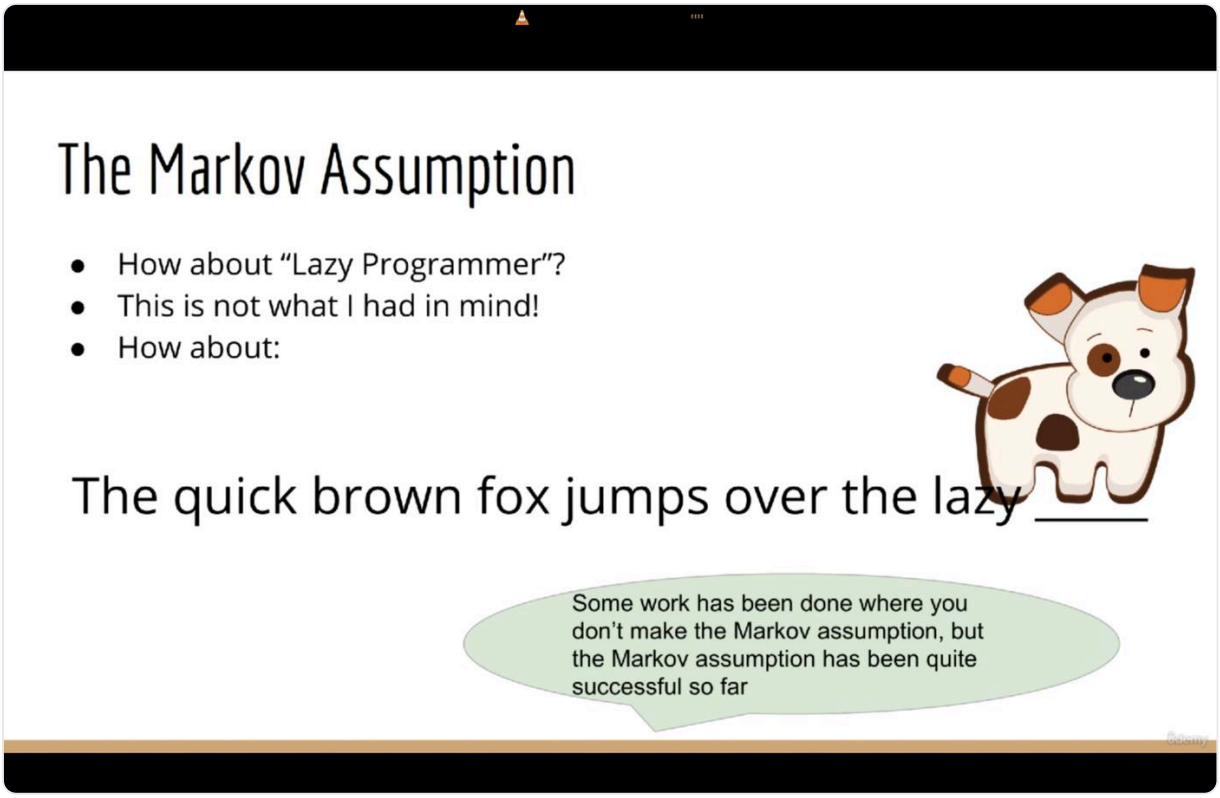


FIGURE 53.6 | The Markov Assumption

The Markov assumption has actually been quite successful so far.

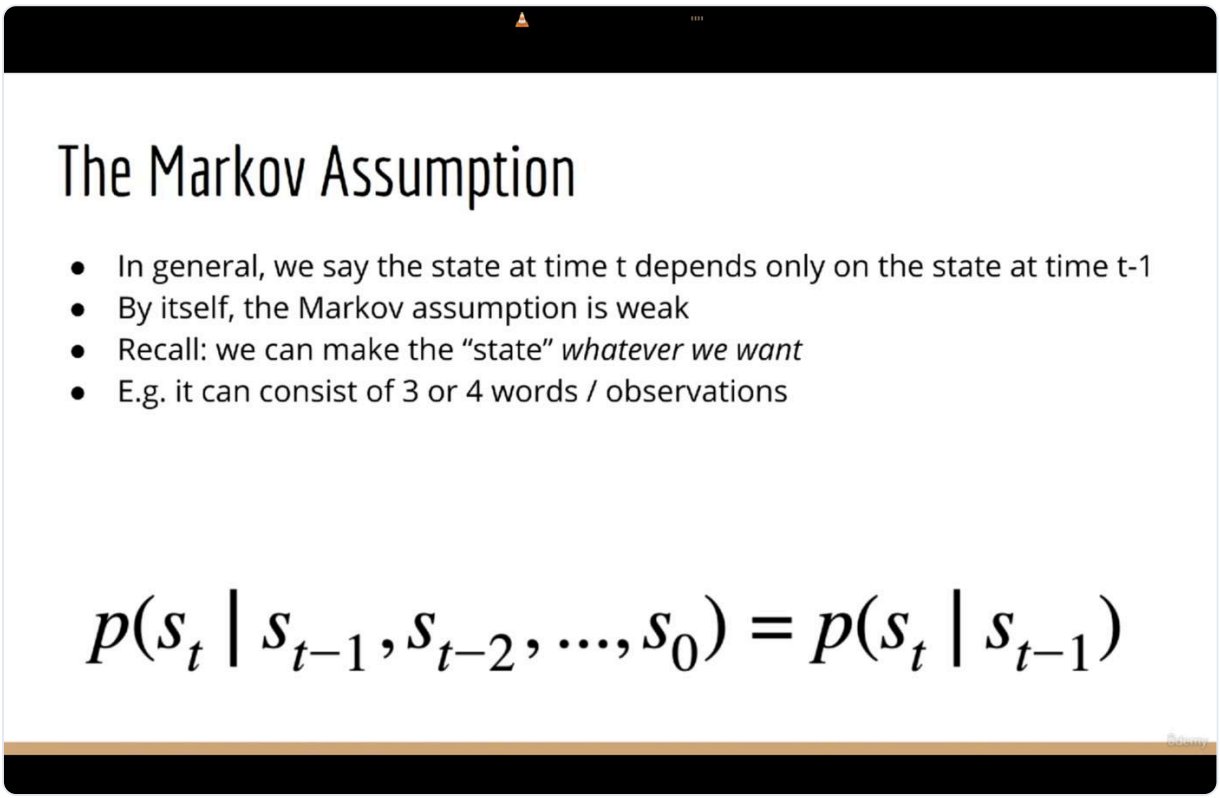


FIGURE 53.7 | The Markov Assumption

In general the Markov assumption states that the probability of the state at time t depends only on the state at time T minus one and not on any state that came before that now by itself. The Markov assumption is weak but as you recall I said earlier that we can make the state whatever we want. So if we want to make the state three or four words long that's fine too. The words are merely observations but the state is made up of a sequence of observations. In this way the Markov assumption is not as bad as you might initially think. So why do we need to know about the Markov assumption. This is because reinforcement learning problems are commonly described as a Markov decision process or MDP.

Previously we discussed the Markov assumption in terms of a state only but as you know reinforcement learning problems involve other objects as well namely actions and rewards. So the way we describe an MDP is to use the state transition probability it's the probability of arriving in the state at time T plus 1 and getting the reward at times plus one given the state at time t and taking the action at time t another simpler way of writing this without time indices is just to write P of s prime and r given s and a note that because the reward R has no prime symbol the prime symbol does not indicate time T plus 1 you get the reward at time T plus 1 for arriving in state as prime.

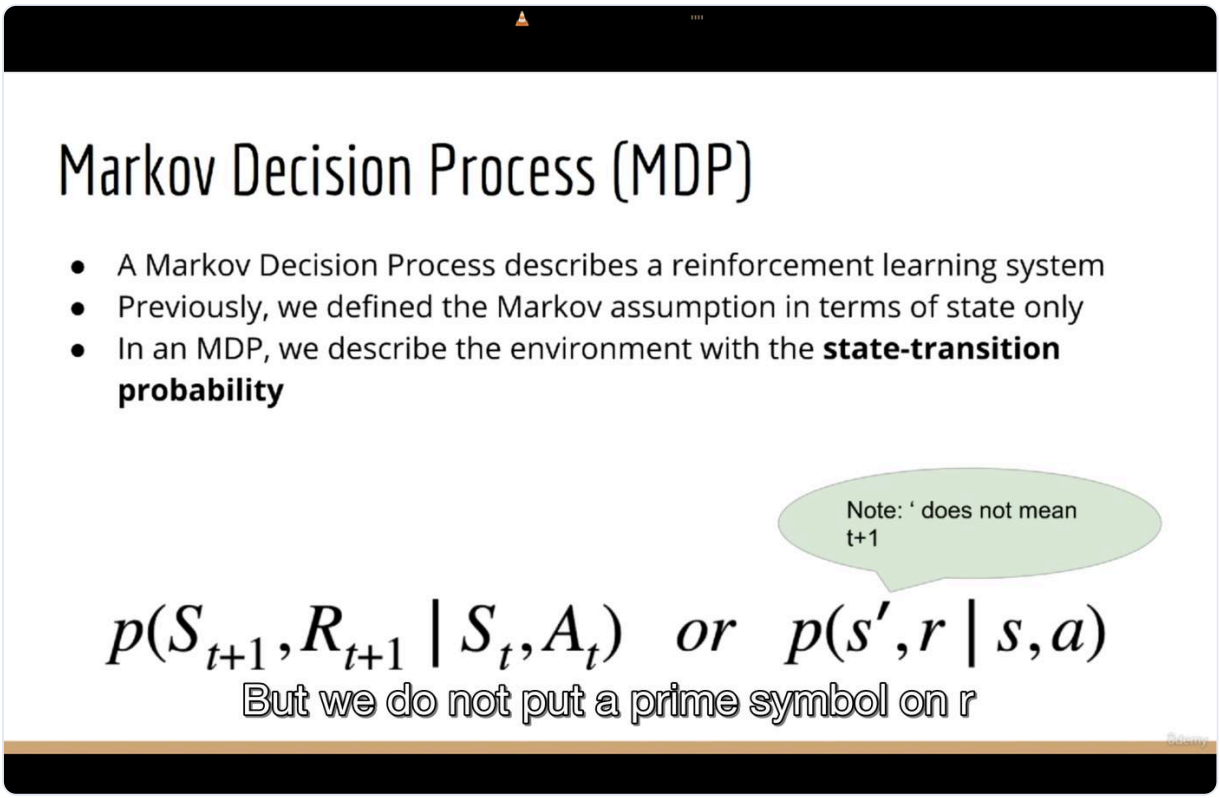


FIGURE 53.8

But we do not put a prime symbol on r so I just showed you the most general way of writing down the state transition probability but often we can make it less general. For example if we are solving a maze then most likely we are going to make the reward deterministic. In other words there's no need to represent it as a probability distribution.

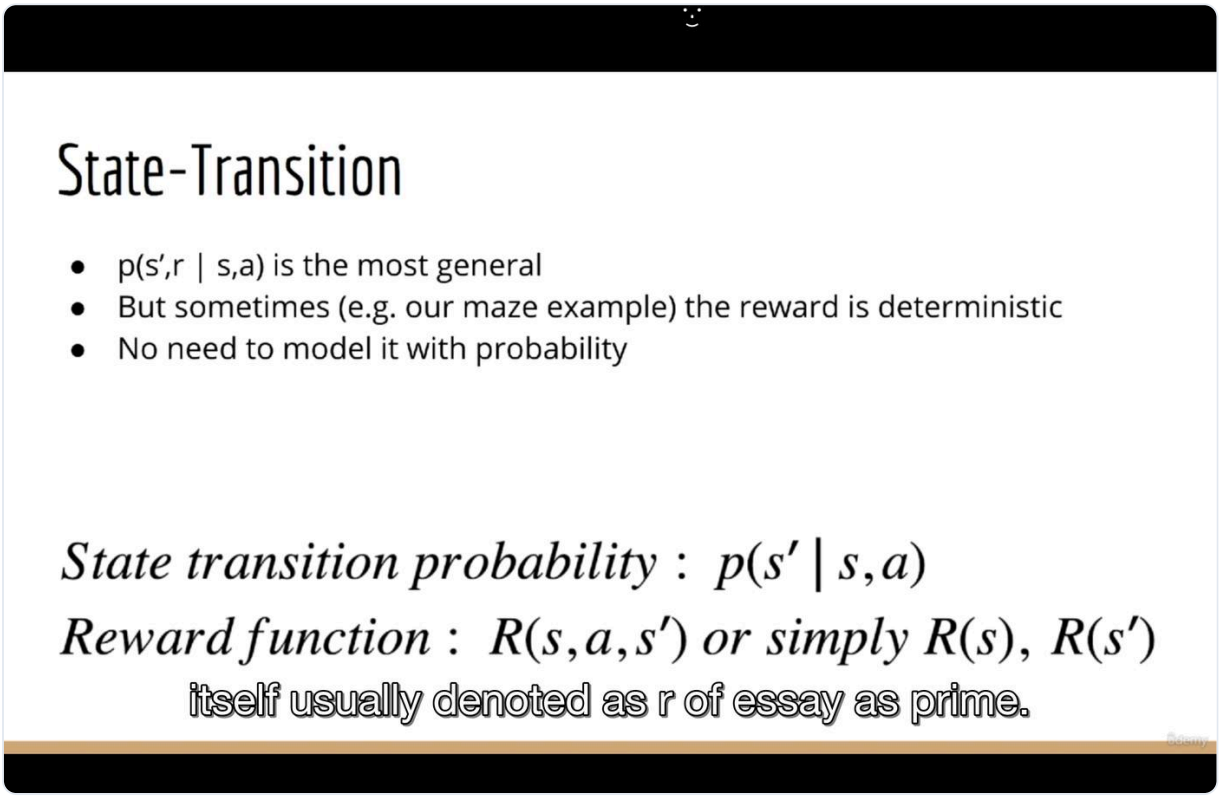


FIGURE 53.9

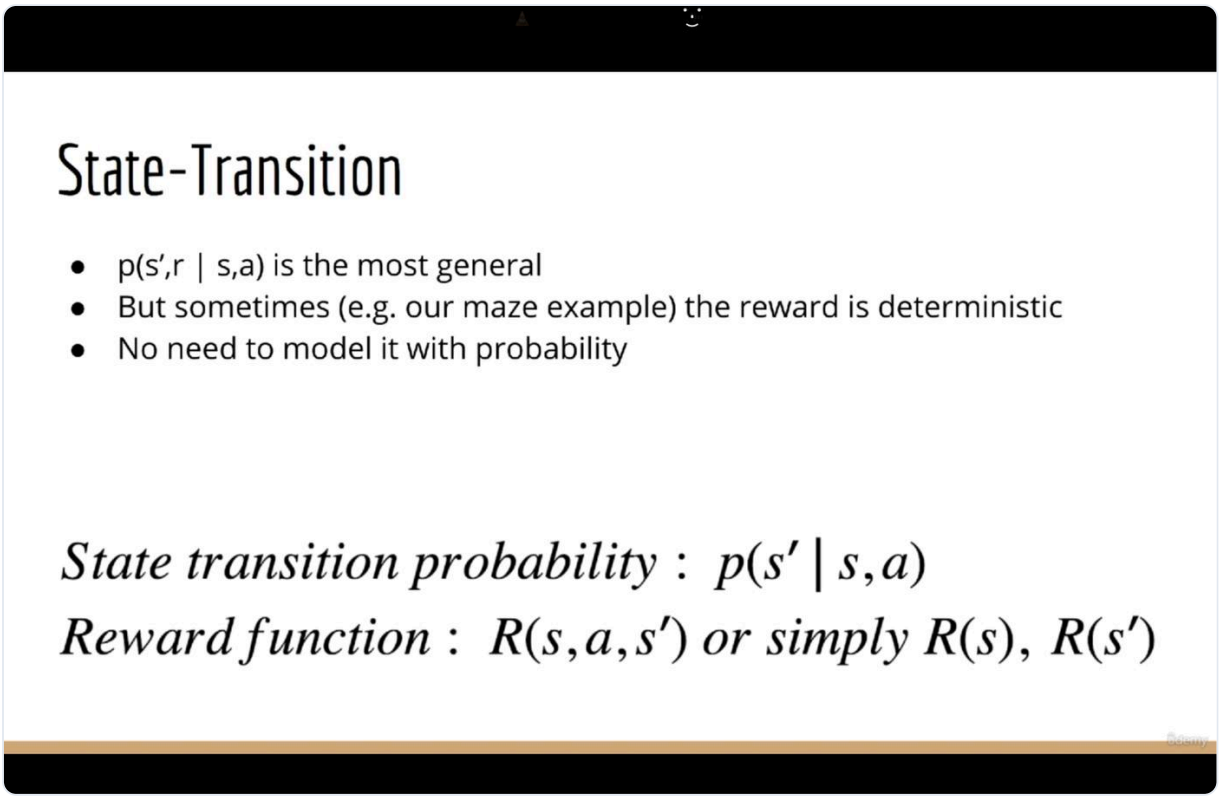


FIGURE 53.10

In this case we can use the notation P of s prime given s and a and the reward it can be a symbol all by itself usually denoted as r of s a s prime. This encodes the idea that we were in state s. We performed action a and we arrived in the next state as prime we can even just say r of S or R of as prime.

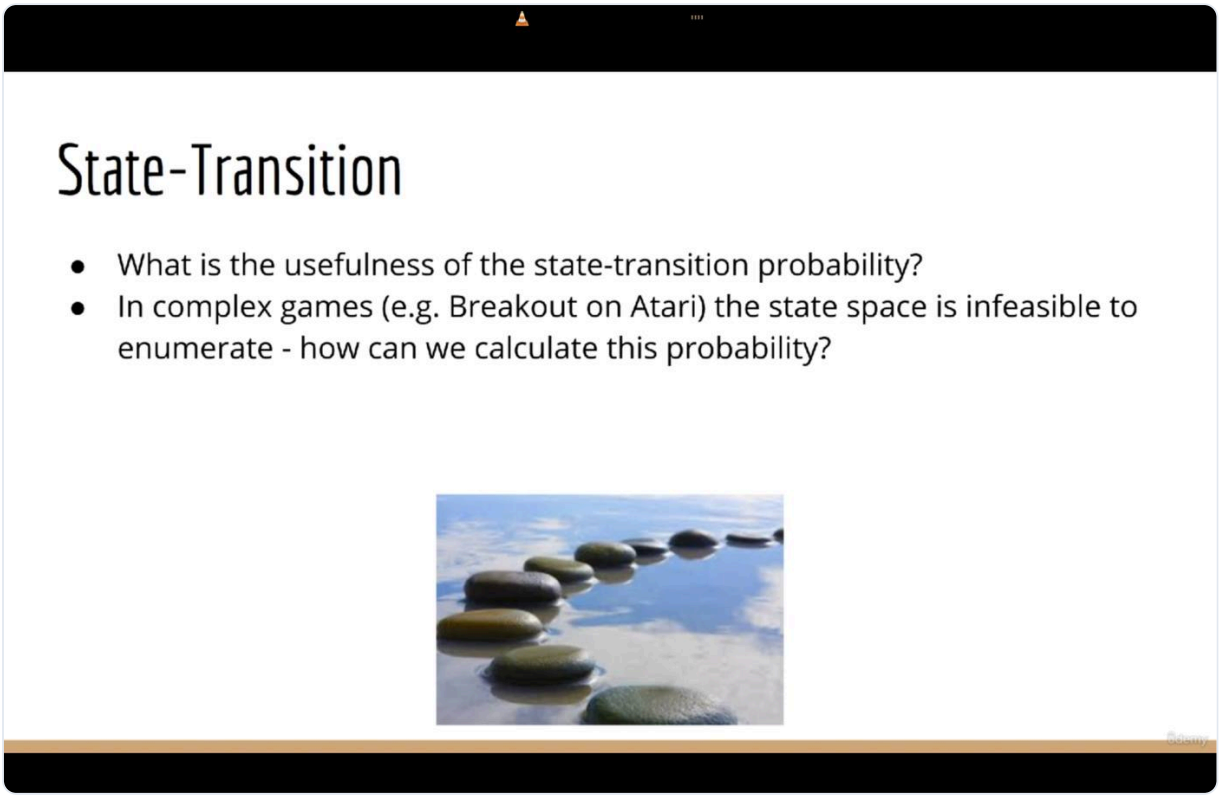


FIGURE 53.11

In the case where the reward depends only on the state where you arrive at which is actually quite common an important point to consider is what is the usefulness of the state transition probability.

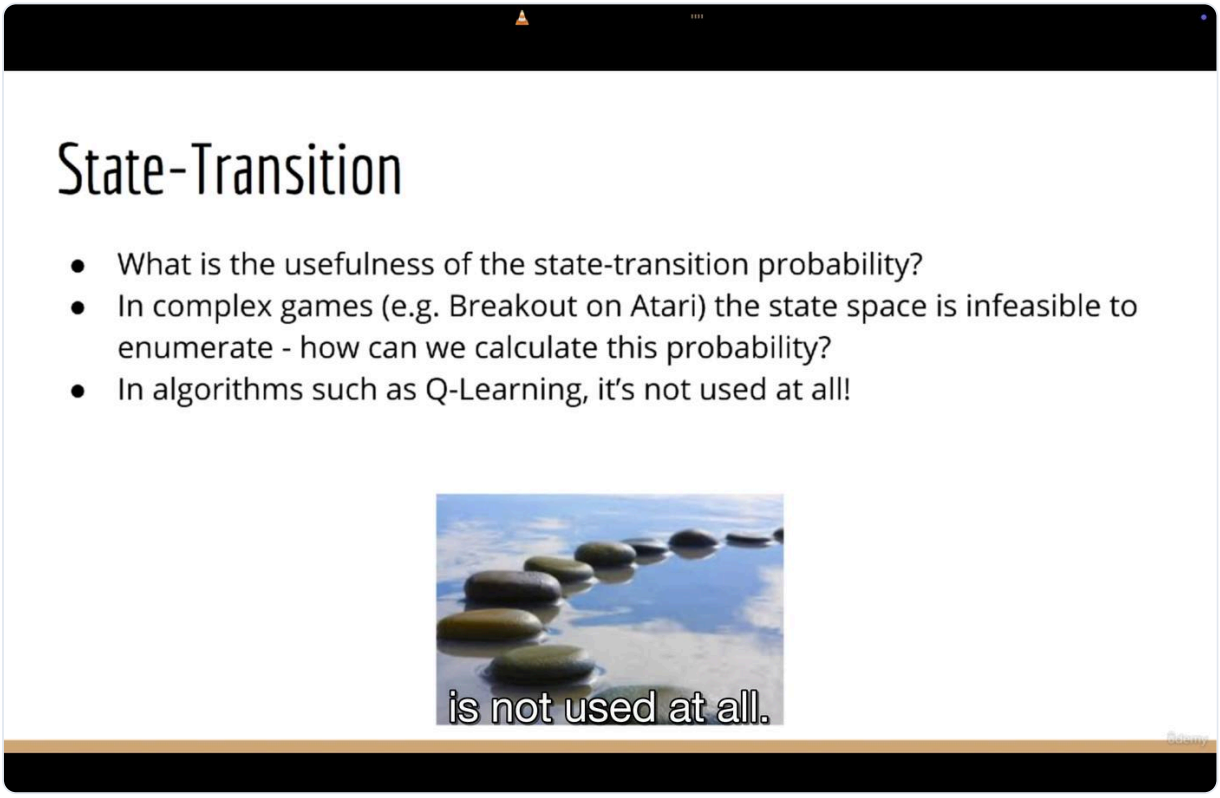


FIGURE 53.12

You can imagine that if we are playing some game like breakout on Atari it is very unlikely we will ever be able to calculate these probabilities given that the state space would be infeasible to enumerate and in fact for Q learning the main algorithm we're going to discuss in the section this probability is not used at all.

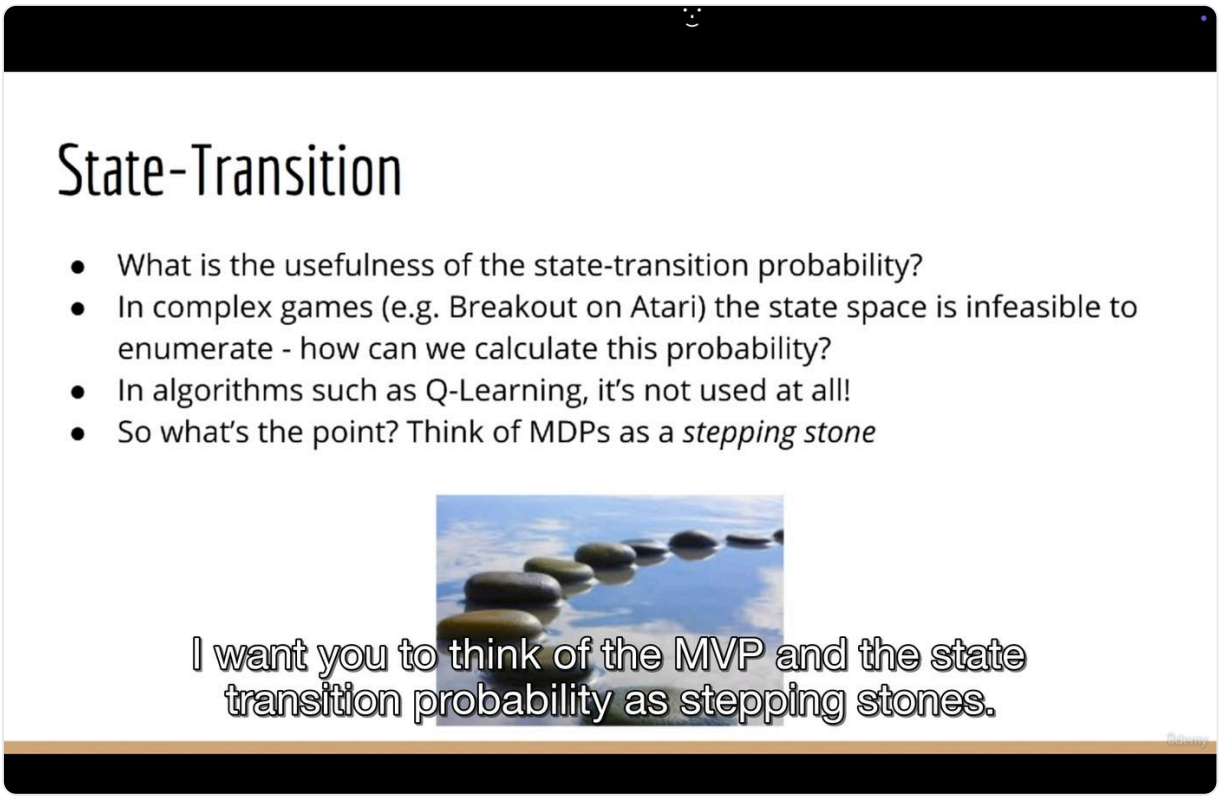


FIGURE 53.13

I want you to think of the MDP and the state transition probability as stepping stones.

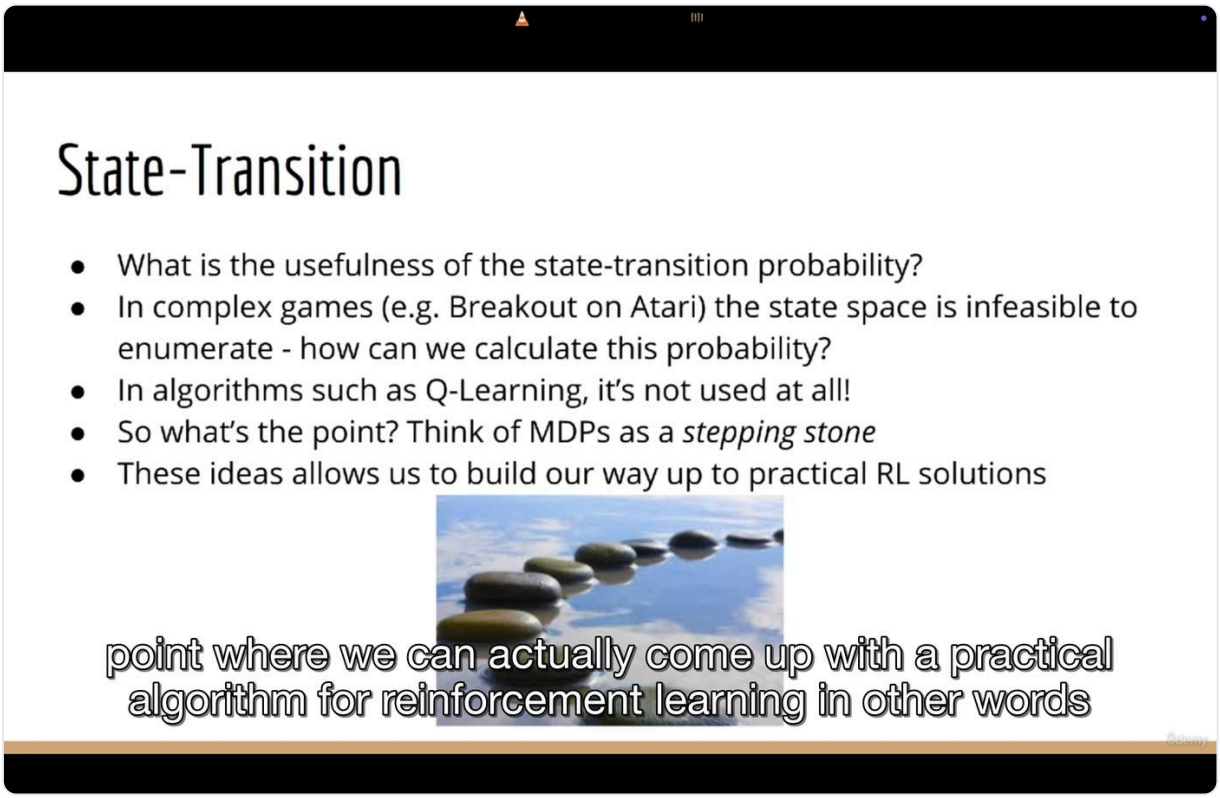


FIGURE 53.14

They are simply conceptual tools which we will use to further advance our knowledge and take us to a point where we can actually come up with a practical algorithm for reinforcement learning in other words while we're not going to be using state transition probabilities directly in Q learning. They do

help us build on what we've done so far so that we can actually arrive at Q learning in a logical manner.

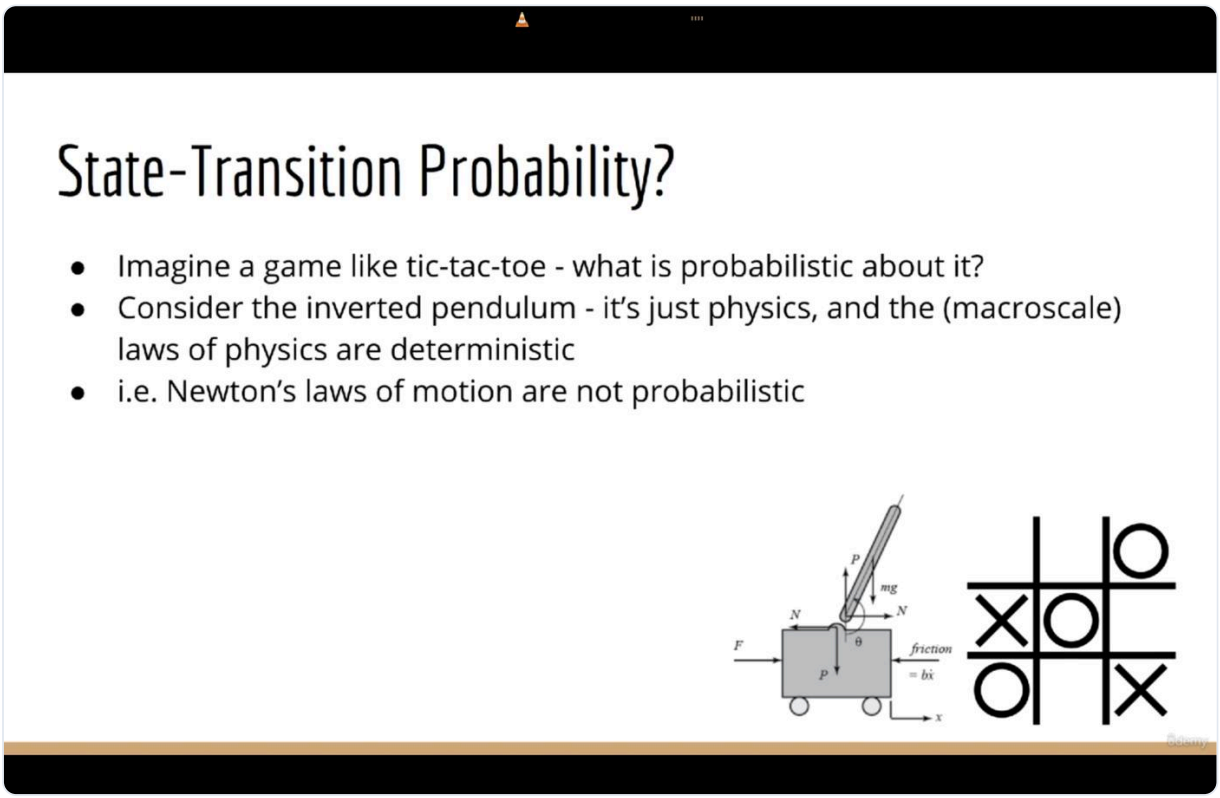


FIGURE 53.15

Why else is the state transition probability useful Imagine a game like tic tac toe. You might think there is nothing probabilistic about this game. When I write down an X I know that's where the X or the O goes. Why is there a probability associated with that. Why is my action not deterministic. And in fact it is entirely possible for your action to deterministically bring you to the next state imagine for example a classic task known as the inverted pendulum. In this reinforcement learning task your job is to control an upside down pendulum so that it does not fall down by moving the cart left or right as necessary. Now you might think to yourself how do we describe such a system.

Well we use the laws of physics and now think to yourself are the laws of physics not deterministic. For example when we learn Newton's three laws of motion do those laws of motion involve probability the answer is no. So then what in the world do we need. Probability for the answer is that your state may not completely capture all the possible information about the environment.

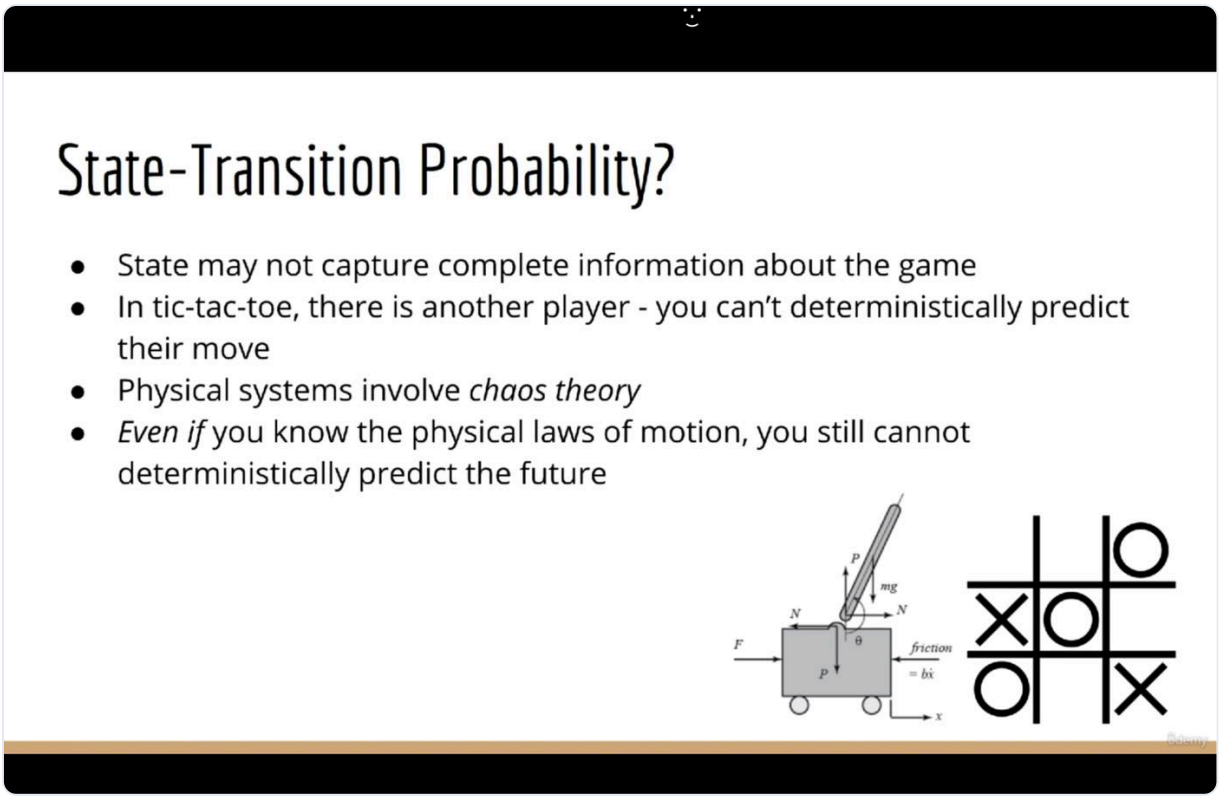


FIGURE 53.16

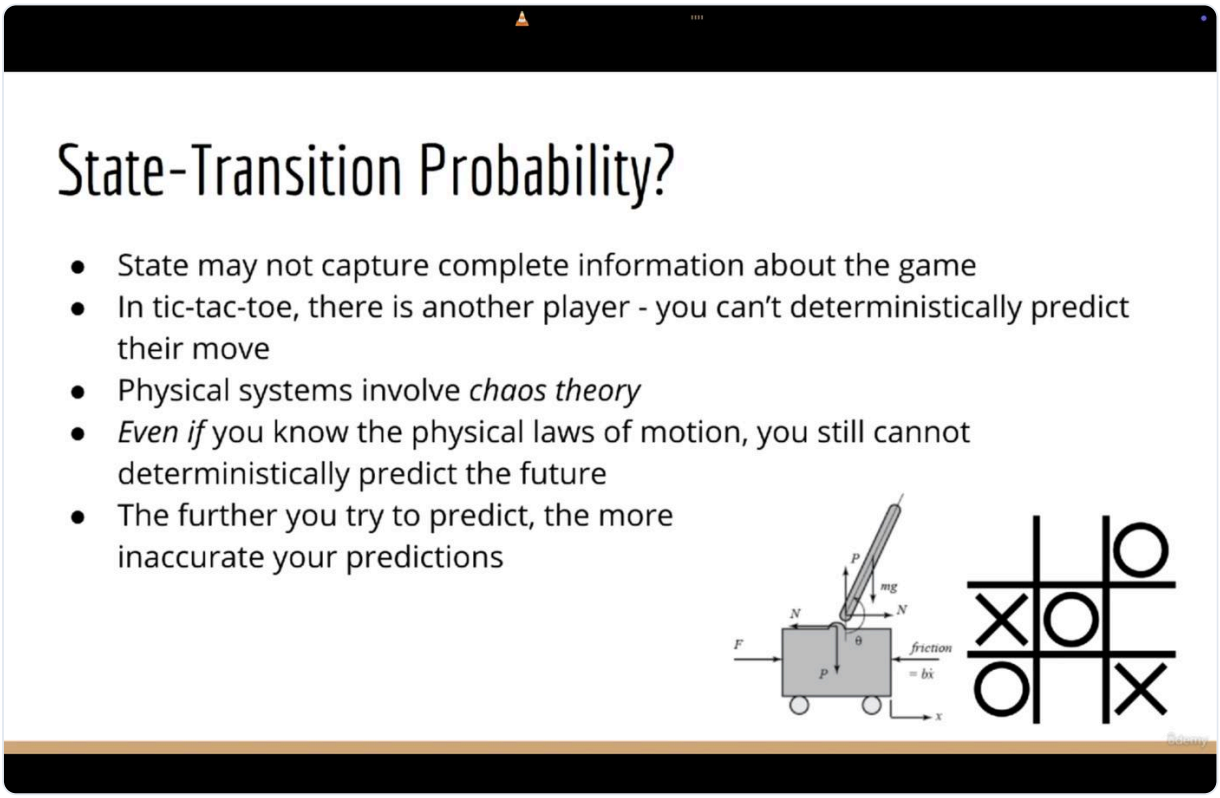


FIGURE 53.17

Consider tic tac toe again in tic tac toe. There is another player that players moves cannot be predicted by a tic tac toe agent. Therefore there are multiple possible moves that could occur between the agents previous move and the agent's next move. If we're talking about physical systems we also have to take

into account chaos theory that is even if you know the exact laws of motion. This does not mean you can accurately predict the future. In fact the further into the future you try to predict the more unreliable your predictions become.

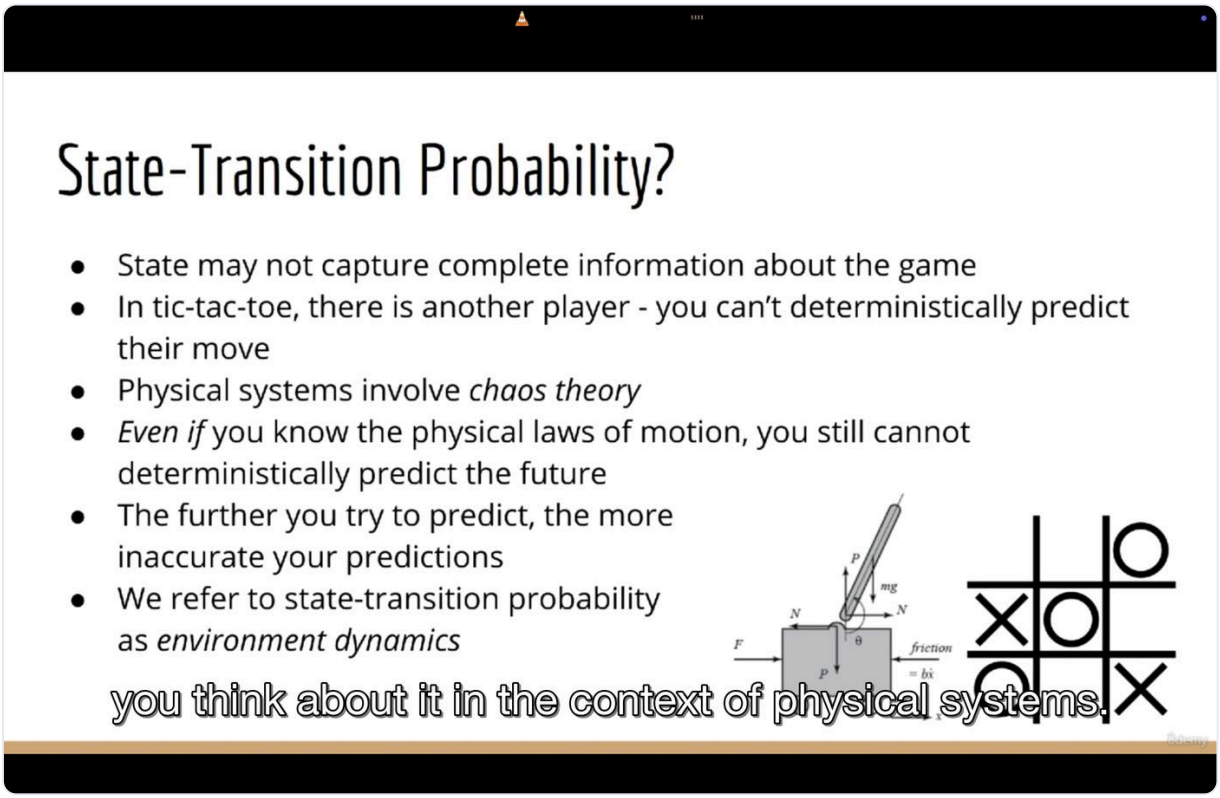


FIGURE 53.18

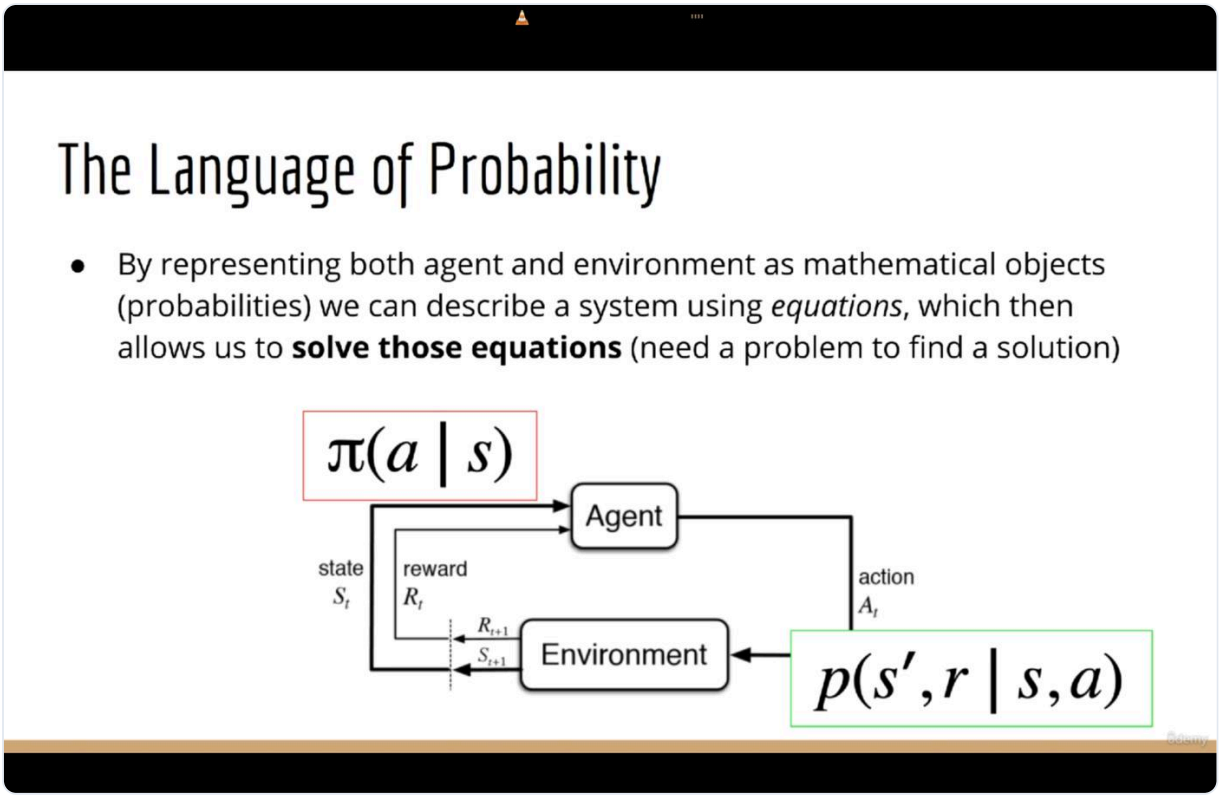


FIGURE 53.19 | The Language of Probability

Sometimes we refer to the transition probability as the environment dynamics which makes sense when you think about it in the context of physical systems. A system like an inverted pendulum is in fact a dynamical system. The last thing I want to mentioned in this lecture is to bring us back to this picture which you've probably seen several times at this point. An MDP or a reinforcement learning problem consists of these two objects the agent and the environment going back and forth. The agent reads the state from the environment and decides what action to take. It takes that action in the environment and the environment is updated based on that action and brings the agent to the next state while also returning an associated reward.

The agent can then read this next state take the next action and so forth. So they just go back and forth in the circular pattern. What we've done so far is to represent both of these objects with probabilities the environment as represented by the state transition probability P of s prime and are given us and a. The agent is represented by the probability Pi of a given s this is more helpful than you probably realize at this point by representing both the agent and the environment as probabilities it allows us to describe reinforcement learning problems mathematically in particular. Once we have an equation we can solve that equation without an equation.

There isn't really anything to solve. That's a pretty deep insight. In order to come up with a solution we have to have a well-defined problem using mathematics specifically probability allows us to create this well-defined problem. And that's the first step towards finding a solution.

LECTURE 92

Future Rewards

In this lecture we are going to define a one more term which deserves its own lecture so far. I told you that the goal of the agent is to maximize the reward it gets.

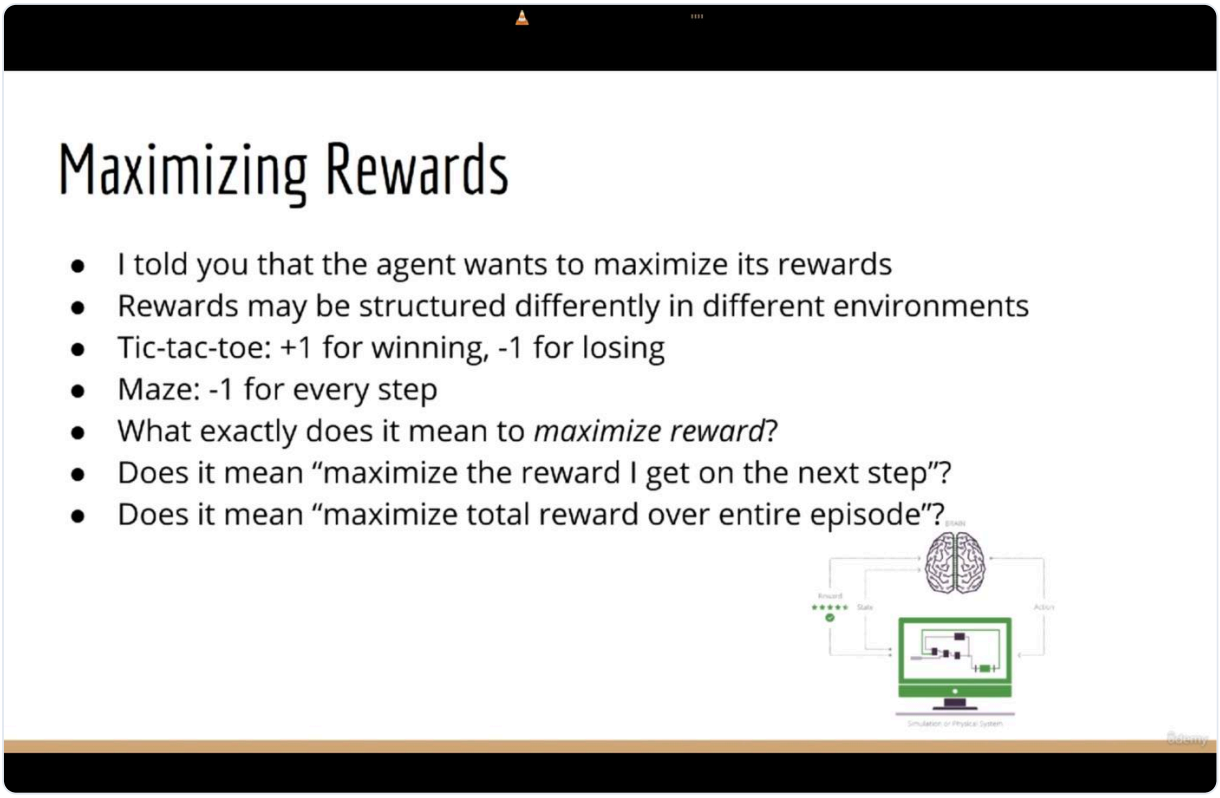


FIGURE 54.1 | Rewards May Be Structured Differently in Different Environments

But as you recall the reward may be structured differently in different games. For example in tic tac toe you may receive a plus one for winning or a minus one for losing on the other hand. If you're solving a maze you might get a minus one at every step. So what does it mean to maximize the reward. Does it mean to maximize the reward at the next step. Or does it mean to maximize the total reward over an entire episode. Here's the answer to stated more accurately the goal of the agent is to maximize the sum of future rewards. Why is that. Well it can't maximize the rewards it got already. Those are in the past. They cannot be changed.

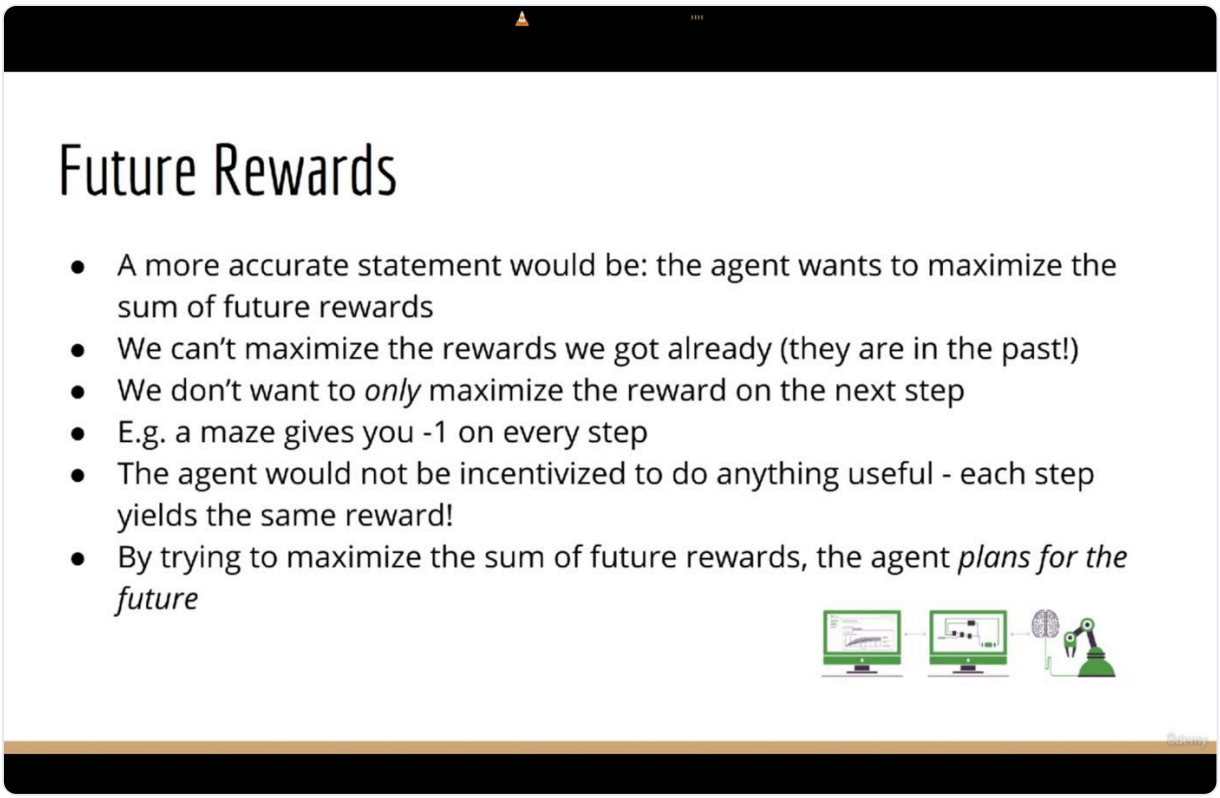


FIGURE 54.2 | Future Rewards

Furthermore we don't want to only maximize the reward on the next step. What if we are solving a maze and we get a minus one for any step we take. In that case the agent isn't incentivized to do anything useful because no matter what it does the immediate reward is still minus one. Thus the agents true goal is to maximize the sum of future rewards until the episode is over. In this way the agent is planning its future steps as well. It must have some concept of where it will end up because that's the only way it will know how to maximize rewards in the future to take a real world example. Consider again the idea of preparing for a math exam for a math exam.

You don't receive any reward until you've completed the exam. The reward signal is your grade on the exam. But imagine all the actions it will take to actually maximize that reward. You'll have to study you'll have to do homework. You have to forgo socializing with your friends. In fact all of those actions do not sound very rewarding at all and thus if your only incentive is immediate gratification. In other words the rewards you receive immediately. Then you will not do well on your math exam. Instead you must make use of long term planning. Sure I may not want to study today. It may be very annoying and I will miss my favorite TV show.

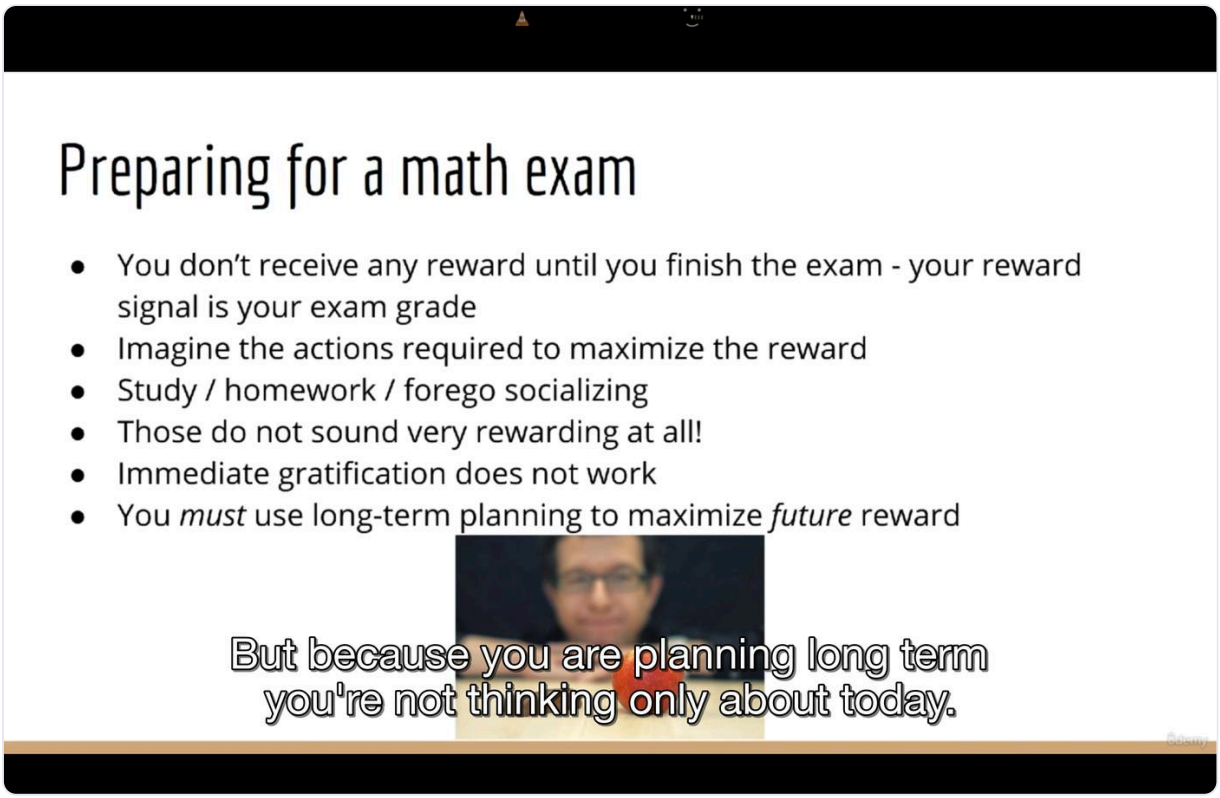


FIGURE 54.3 | Preparing for a Math Exam

But because you are planning long term you're not thinking only about today. You're thinking about the results of your math exam. The desire to maximize the total future reward is necessary for long term planning we call the sum of future rewards the return we describe the return mathematically. Using the symbol G because it depends on future rewards only it is time dependent. So we index it with a T.

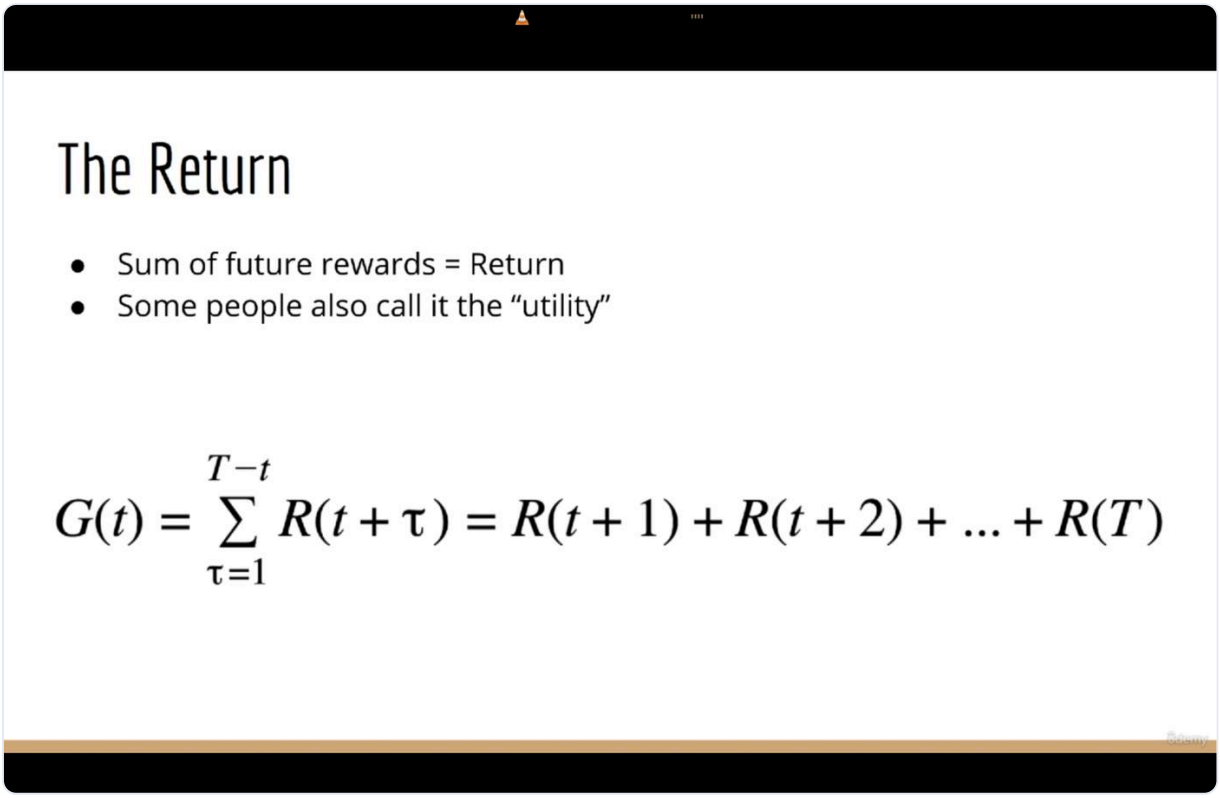


FIGURE 54.4 | The Return

We can say the return at time t is the sum of rewards at time T plus one up to the terminal state at time big T now you might be wondering what happens if we have an infinite horizon MDP a game that never ends. In this case your return might be infinity. Therefore we introduce a concept known as discounting discounting is used for infinitely long tasks but it's also used for episodic tasks as well. We introduce a discount factor called gamma. Each future reward is weighted by gamma to some power gamma is usually a number close to one like zero point nine zero point nine nine or zero point nine nine nine. It's a hyperparameter so you'll have to choose its value based on the performance of your agent.

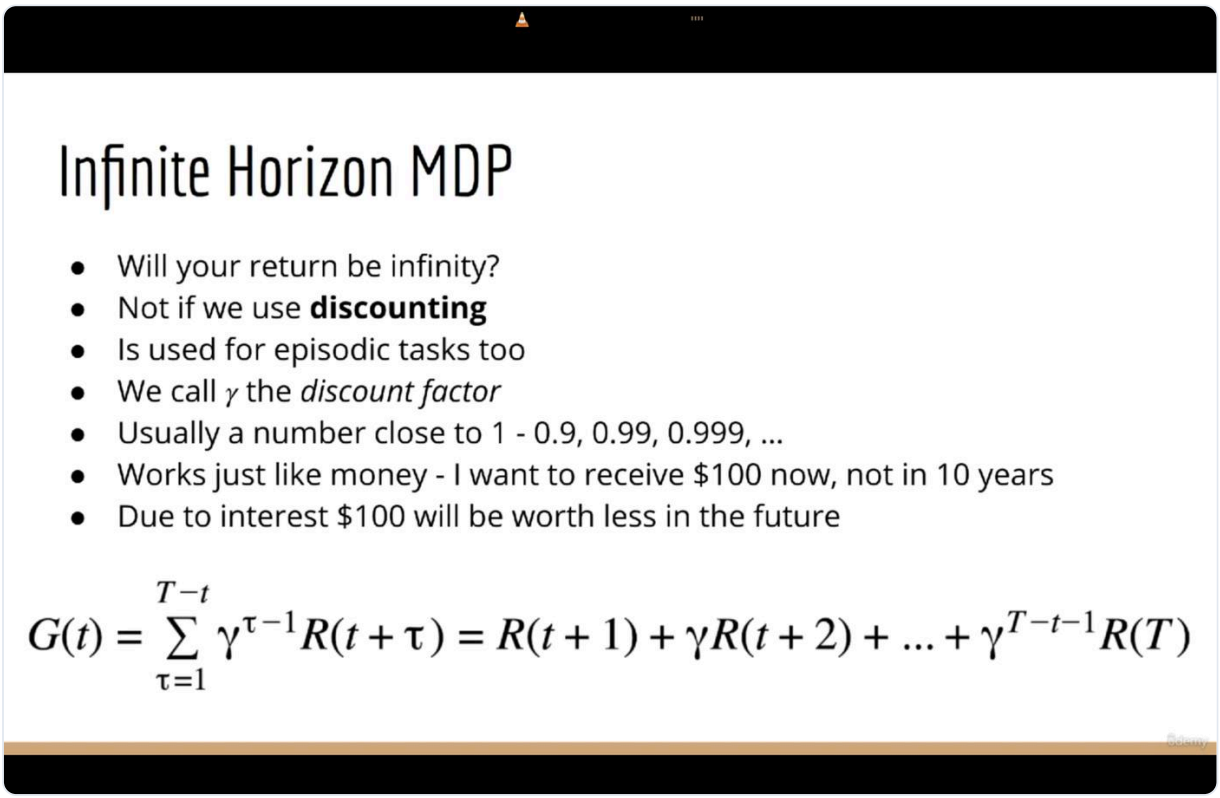


FIGURE 54.5 | Infinite Horizon MDP

The idea is the further you go into the future the harder it is to predict. Therefore we care a little more about getting rewards now than we do later. Intuitively this works just like money. I would rather receive one hundred dollars today than receive one hundred dollars ten years from now. In ten years 100 dollars will be worth much less than it is today due to interest one important feature of the return which we will make use of throughout the rest of the section is that it can be defined recursively.

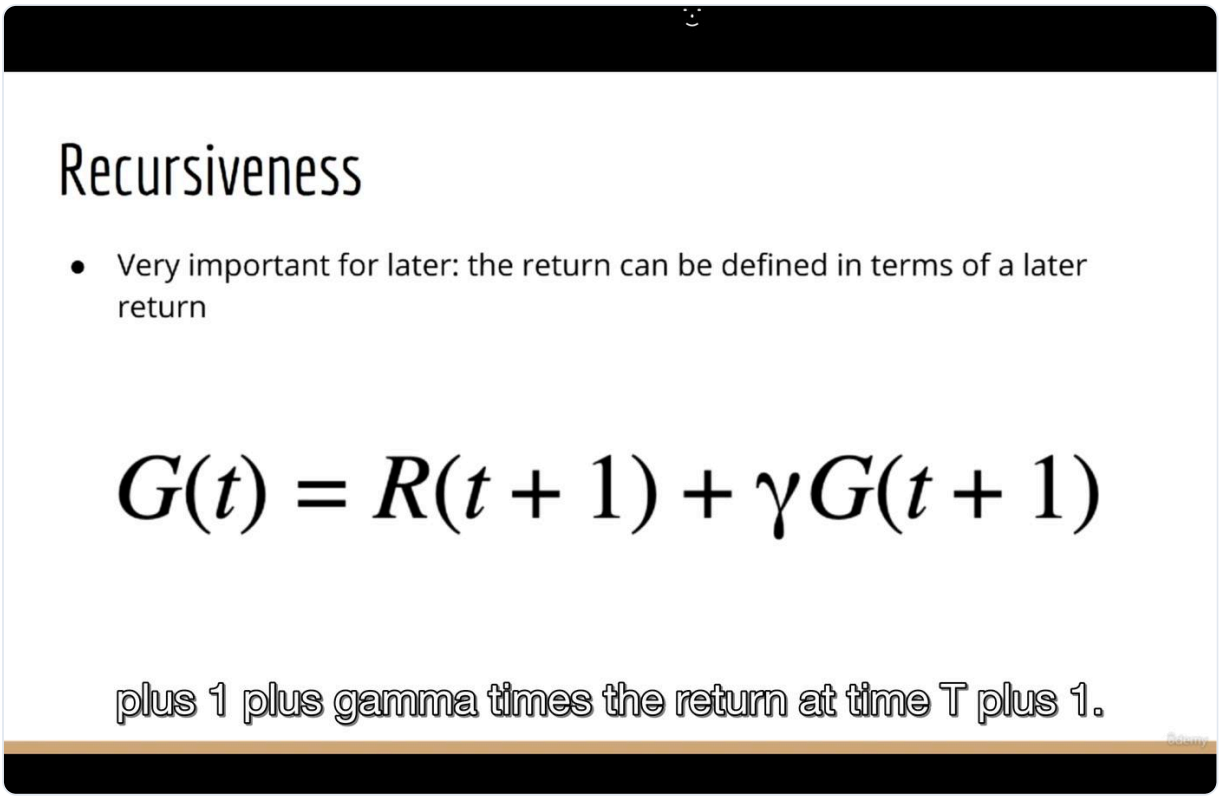


FIGURE 54.6

In other words in terms of itself specifically the return at time t is equal to the reward at time T plus 1 plus gamma times the return at time T plus 1. This may not seem like much more than a simple math substitution now but you'll see how it will become very useful later on.

LECTURE 93

Expressing the Problem as an Equation

In this lecture we are going to finally form the equation which represents a generic reinforcement learning problem. And from this we can derive a solution. Let's begin by discussing expected values. By the way if you've never taken a probability course then you've probably never heard of expected values in which case if you want to understand this you'll want to take a course in probability to think about it simply the expected value is just the mean. So for example Suppose I've measured the heights of 1000 students and I want to model this as a Gaussian distribution. I find that the average height is 70 inches with a standard deviation of 4 inches.

In this case the mean of the distribution is 70 and thus the expected value is 70. Let's look at a coin as the next example a coin has a binary outcome.

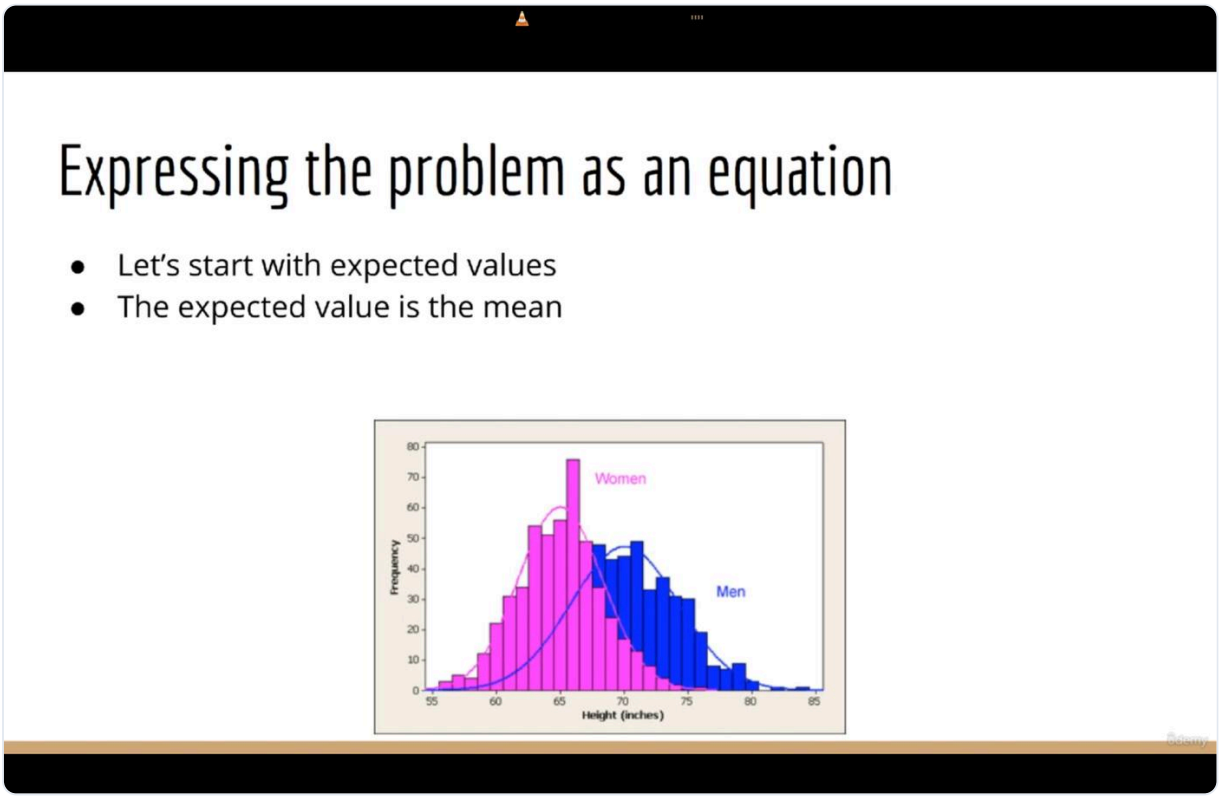


FIGURE 55.1 | Expressing the Problem as an Equation

If I flip a coin I may get one or I may get zero. Suppose the probability of both 1 and 0 is 50 percent. In this case what's the expected value. Well since it's the mean. That means the expected value is zero point five. Sometimes this terminology confuses people. They ask how can the value I expect to get be zero point five. If the result can only be zero or 1.

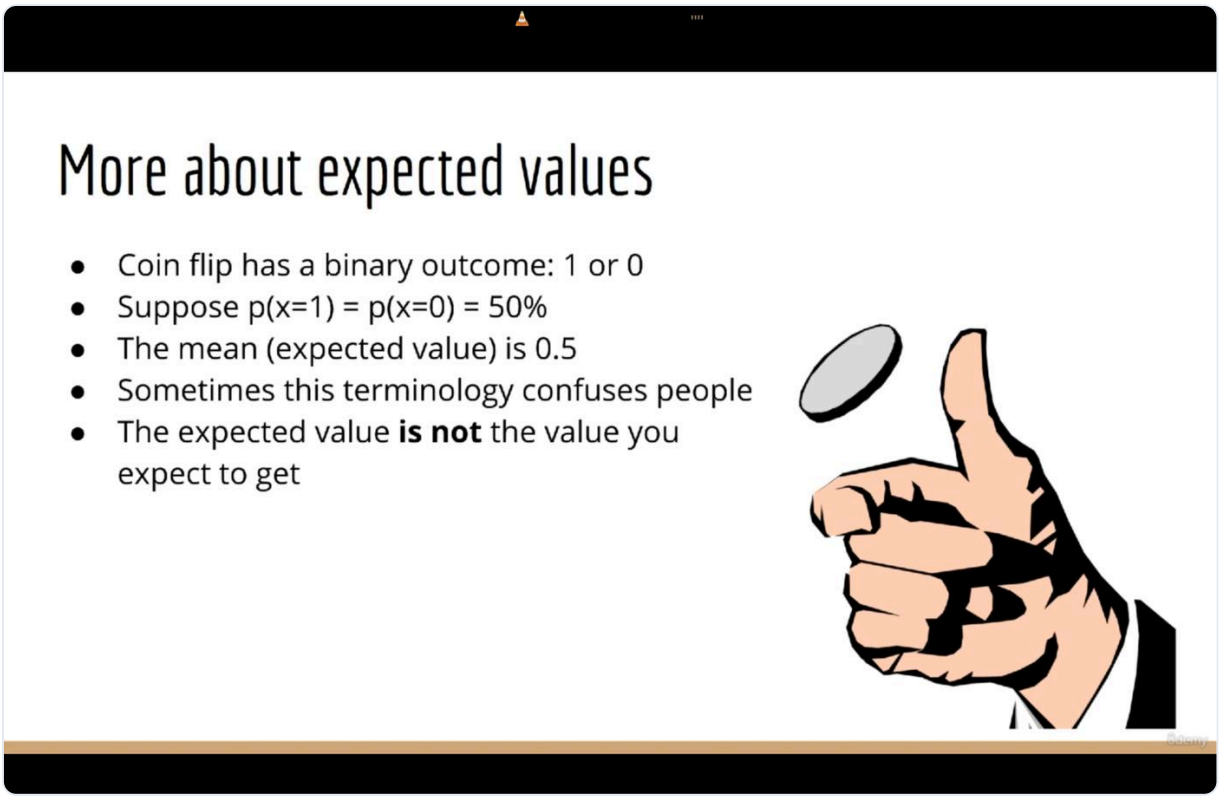


FIGURE 55.2 | More About Expected Values

It's important to realize the expected value despite its name is not the value you expect to get. So if I flip a coin I don't actually expect to see the value zero point five the expected value actually has a precise meaning for both continuous and discrete distributions although for most of this section we will assume that we are working with discrete distributions.

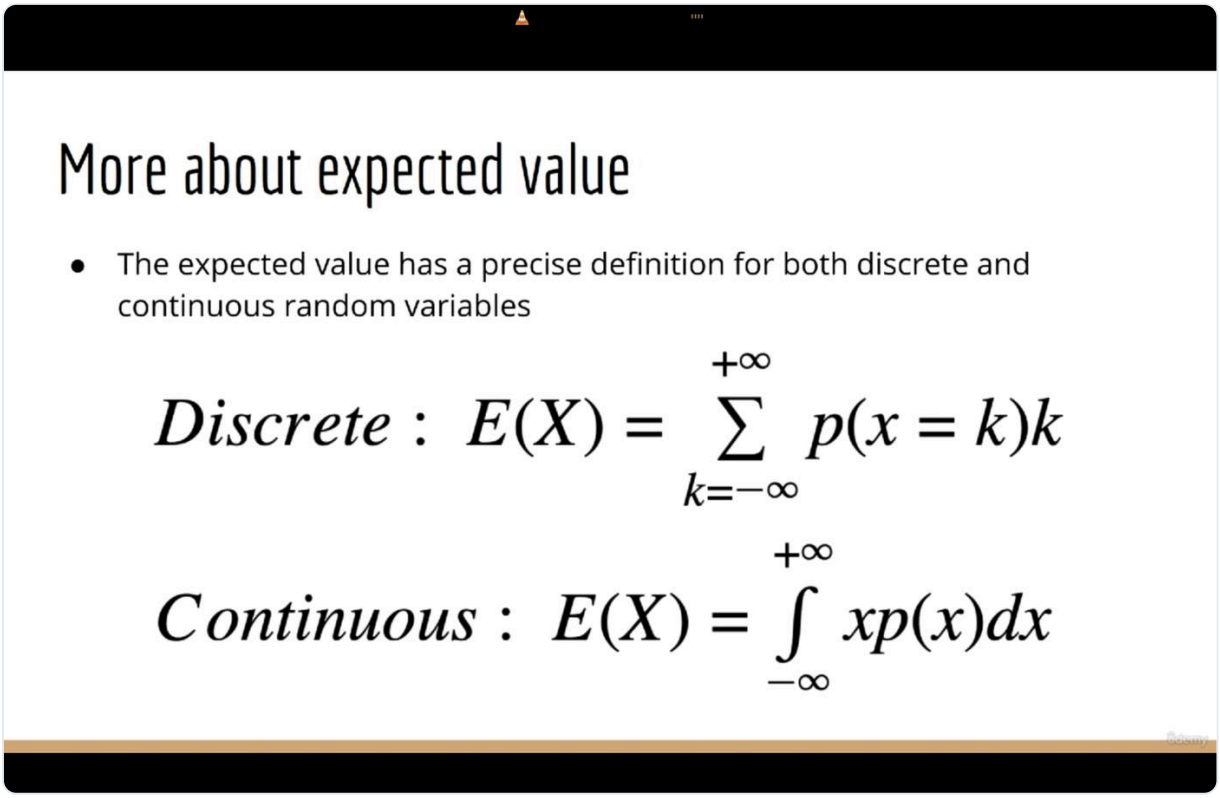


FIGURE 55.3 | More About Expected Value

The idea is that the expected value is a weighted sum of all the possible values of a random variable where the weights are the probabilities of those values as a simple example we can return to our coin example if we have a biased coin so that the probability of heads is zero point six.

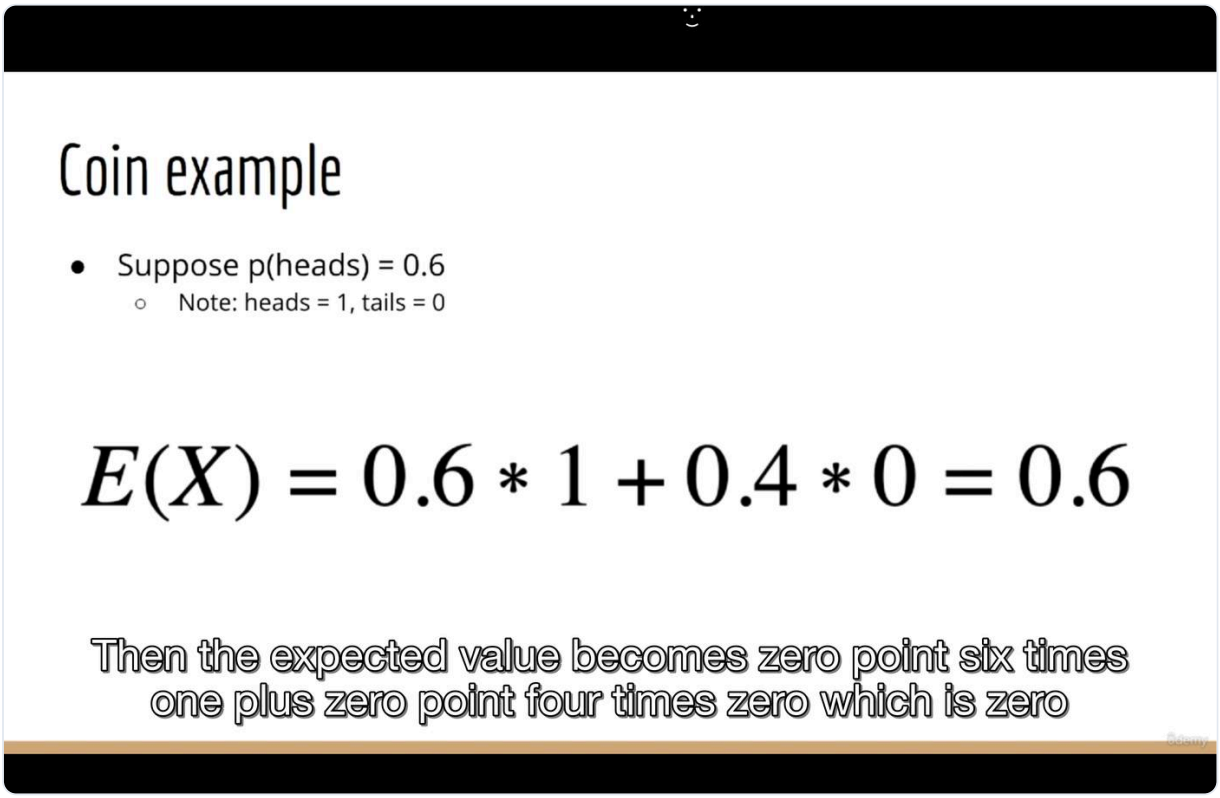


FIGURE 55.4 | Coin Example

Then the expected value becomes zero point six times one plus zero point four times zero which is zero point six because the probability of one is now a little higher than the probability of zero the expected value also becomes a little higher. All right. So what's the significance of expected values. Well let's remember that our reward and hence also our return which is the sum of rewards are random variables at a high level. What this means is that if I play a game one hundred times using the same policy in the same environment I will get different rewards. This is because both the environment dynamics and my policy are probabilistic and thus it makes sense to not think about a single return by itself but the expected value of the return.

I would like to maximize the sum of future rewards but because the result is probabilistic.

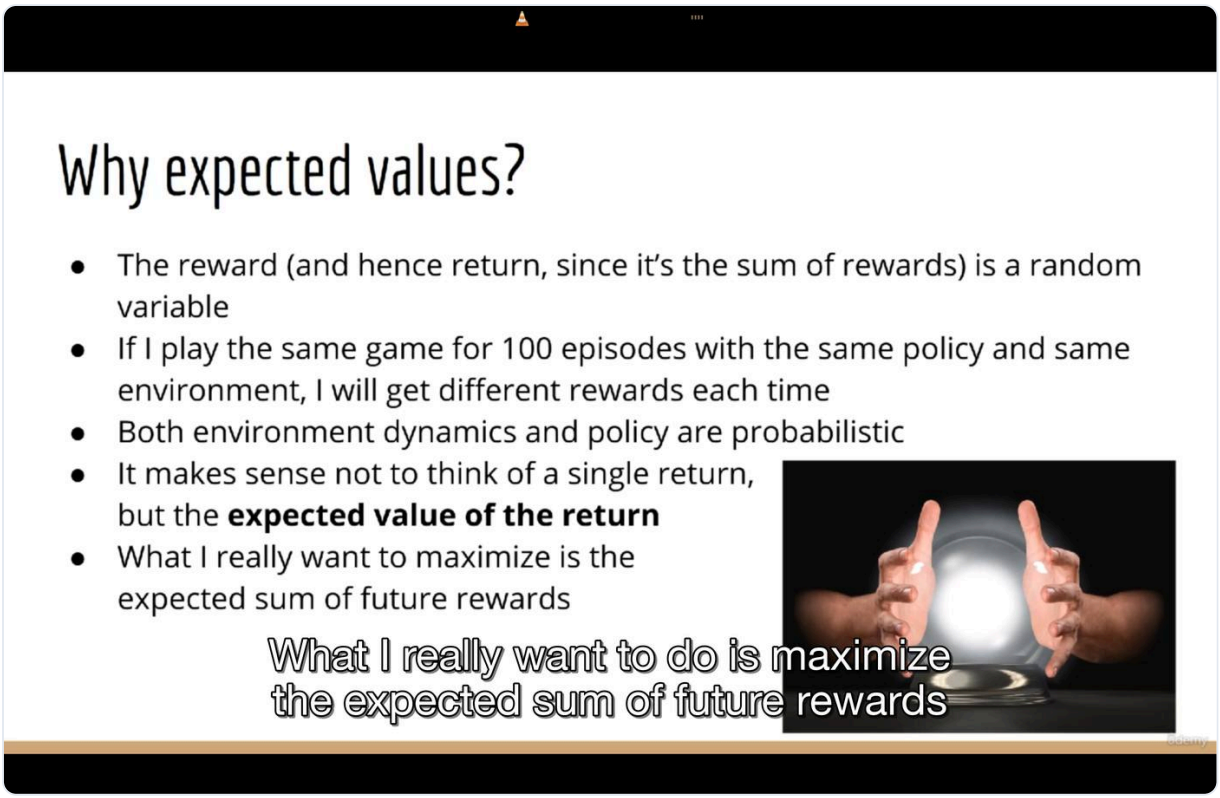


FIGURE 55.5 | Why Expected Values?

What I really want to do is maximize the expected sum of future rewards the expected sum of future rewards has a special name and reinforcement learning. We call it the value function. I always mentioned that this is a very unfortunate name because the word value is such a generic word. Nonetheless this is what it is called because the return is the sum of future rewards after we have arrived in some state. So too is the value function as you can see it's not just a plain expected value but rather it's conditioned on the fact that we arrived in the state s at time t. Thus we denote this as V of s the value of the state s as a side note we sometimes referred to the return as simply the reward.

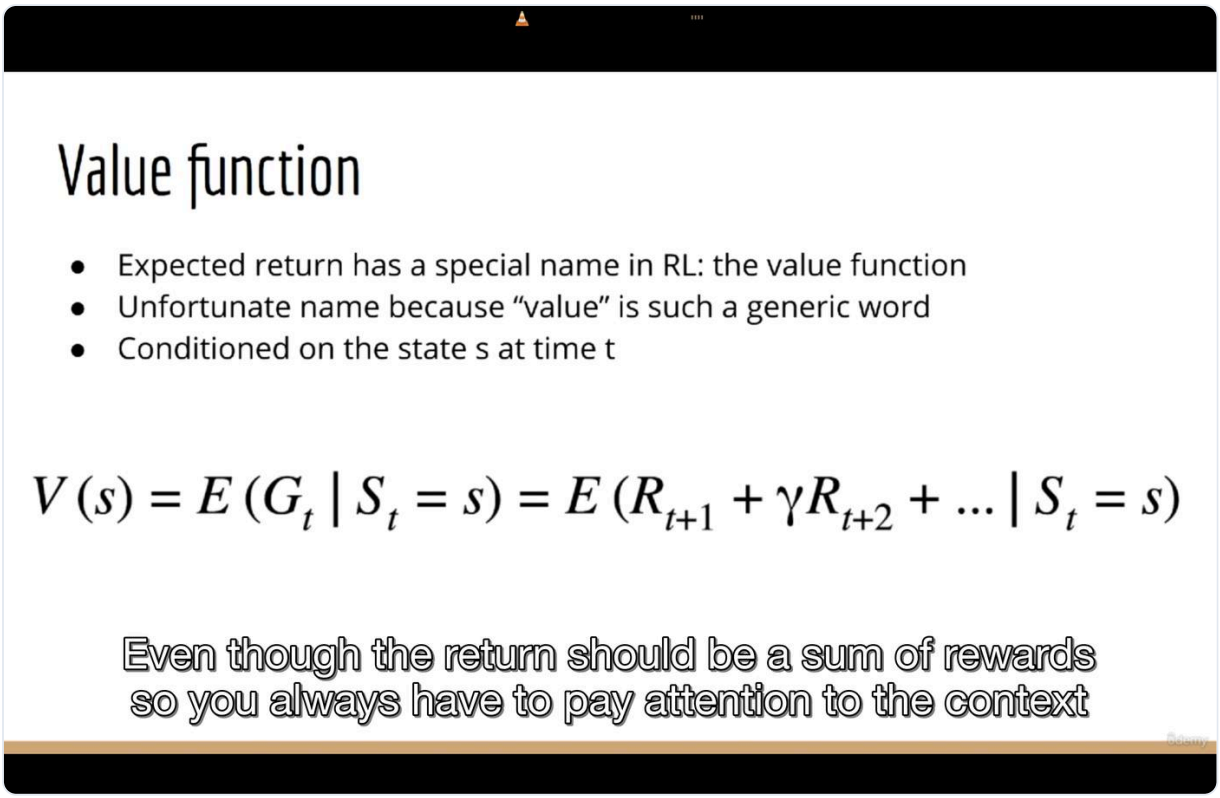


FIGURE 55.6 | Value Function

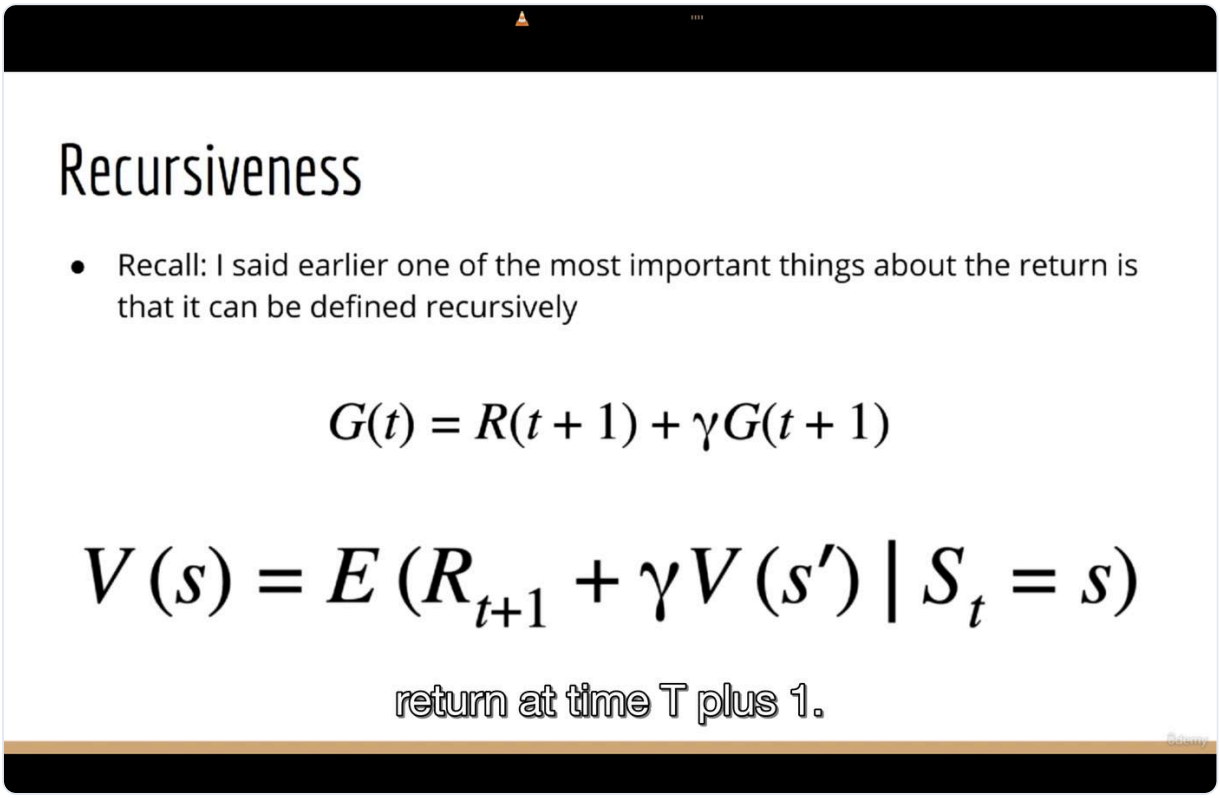


FIGURE 55.7

Even though the return should be a sum of rewards so you always have to pay attention to the context as you recall I said that one of the most important things about the return is that it can be defined recursively the return at time t g of T is equal to the reward at time T plus 1 plus gamma times the return at time T plus 1. And because of this we can also define the value function recursively V of s is the expected

value of the reward at time T plus gamma times V of s prime the value at the next state s prime. Now you must be wondering what is the purpose of all this. Are we just defining equations for the sake of it. Of course the answer is No.

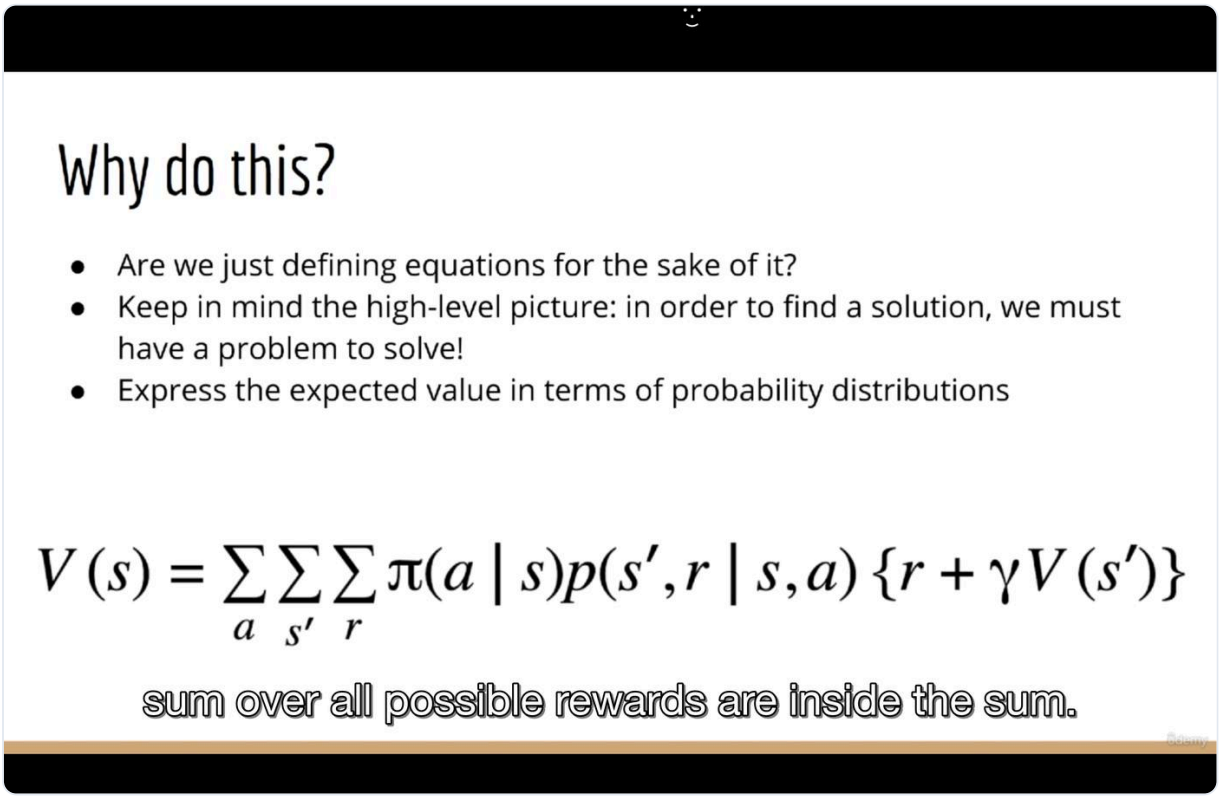


FIGURE 55.8 | Why Do This?

Just keep in mind the high level picture we are setting up a problem from which we can find a solution in order to find a solution. We must have a problem and thus our job right now is to continue building up the problem so that it is well-defined. The next step is to remember that the expected value is really just the weighted sum where the weights are the probabilities. So if we write it out in full. This is what we get. This is the sum over all possible actions a and the sum over all possible next states s prime and the sum over all possible rewards r inside the sum. We have the probability of performing action a given that we are in state s multiplied by the probability of landing in the next state s prime and receiving the reward R.

And then this is multiplied by the reward R Plus gamma times V of s prime. In other words it's r plus gamma times V of s prime multiplied by the probability of actually getting the reward R and arriving in the state s prime In other words the expected value of the return. This is called the Bellman equation.

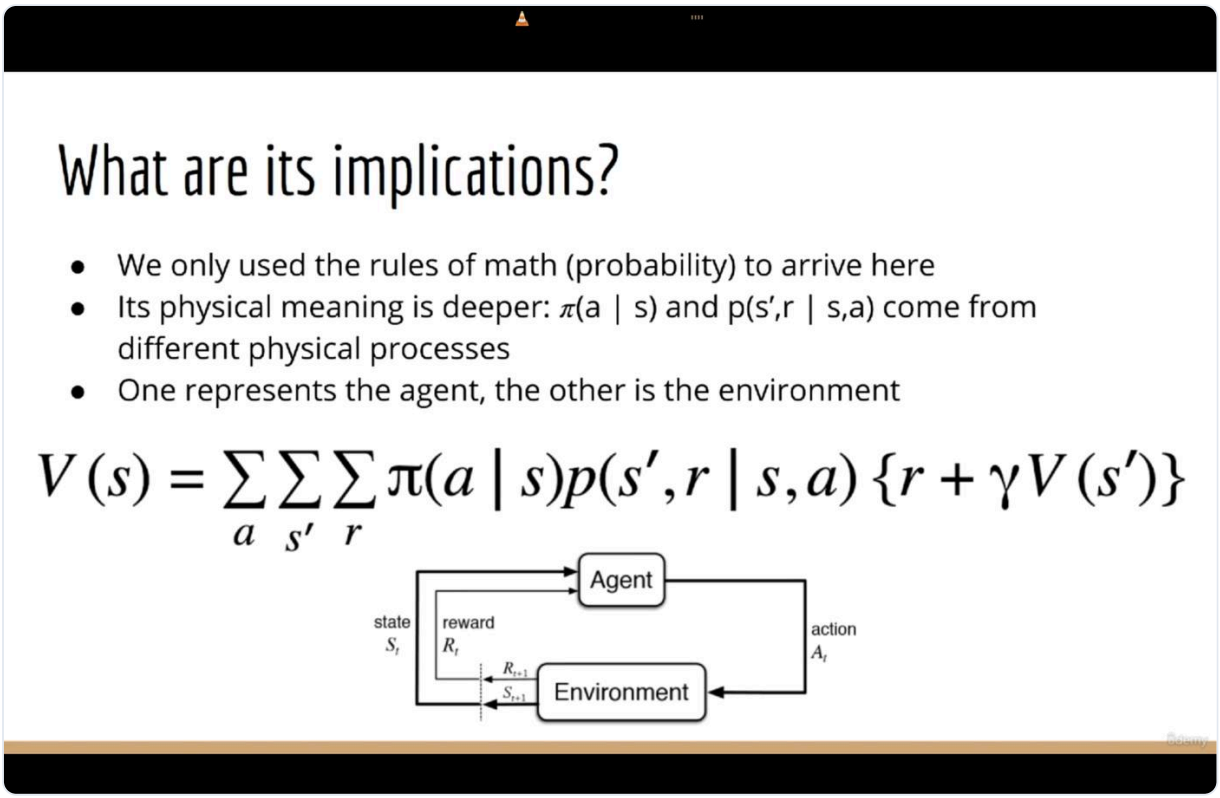


FIGURE 55.9 | What Are Its Implications?

It is the centerpiece of all the subsequent solutions to reinforcement learning that we will discuss it's worth thinking about the Bellman equation a little more. Notice that in order to come up with this equation we only use the rules of mathematics. The rules of probability it seems like just a regular old expected value. However the implications are deep. It's interesting to consider that the two probabilities here Pi of a given S and P of s prime and r given s and a come from totally different of physical processes. Pi is our policy and it represents our agent. So you can think of that as like an animal. The state transition probability represents our environment so you can think of that as the world.

So it's interesting that while we can freely manipulate this equation mathematically these two objects are actually completely distinct physically at this point. You must be extremely tired.

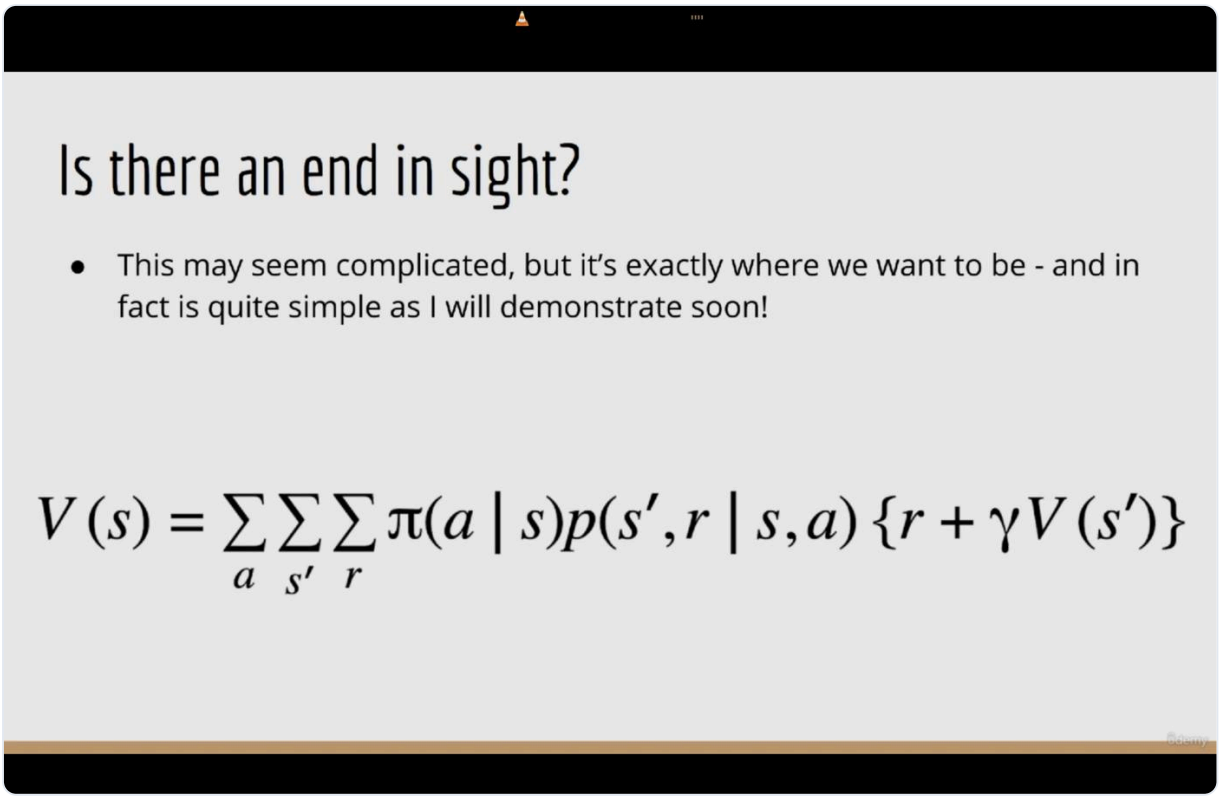


FIGURE 55.10 | Is There an End in Sight?

All we are doing is making our formulas progressively more complicated with no end in sight but in fact this is exactly where we want to be. Although this may seem complicated I will demonstrate to you that in fact what we have arrived at is pretty simple we can now define the first problem in reinforcement learning from which we can find a solution. Remember that in a reinforcement learning problem you can have multiple policies. Some may be good but some may be bad. How can we tell what is a good policy and what is a bad policy. Well how about the value function.

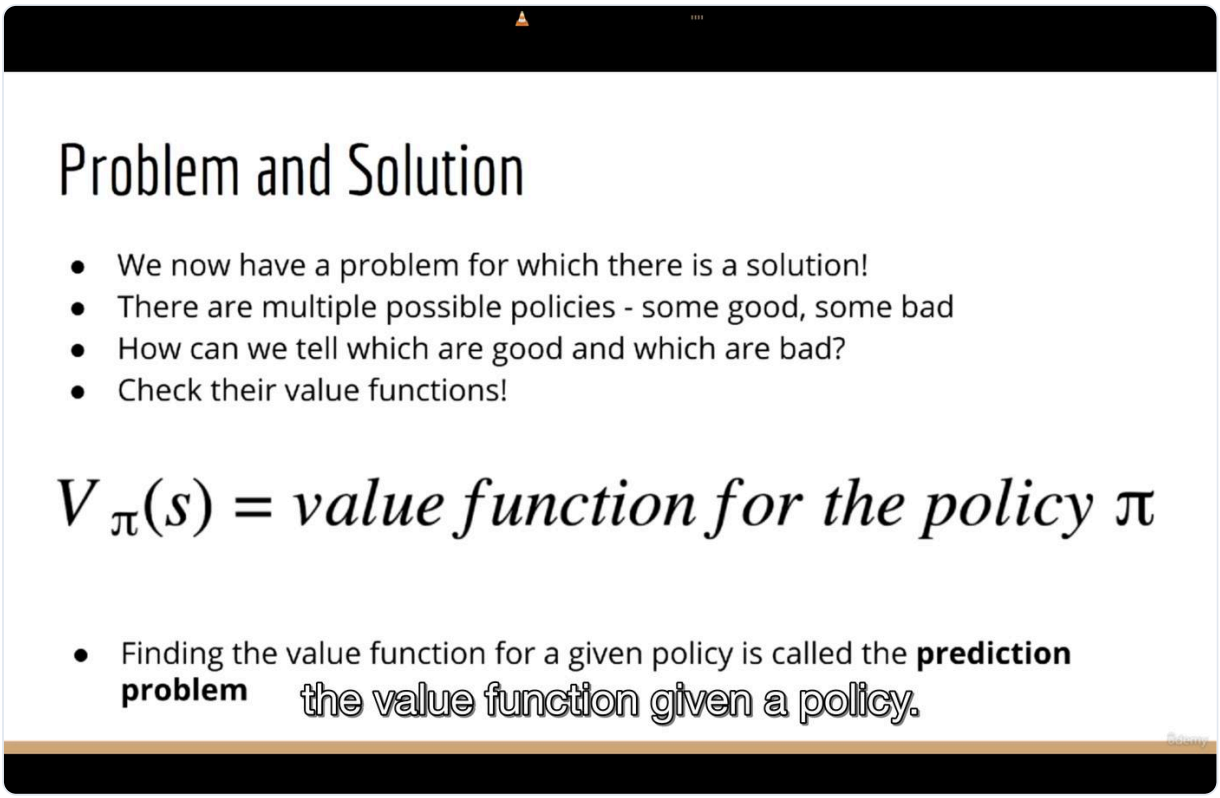


FIGURE 55.11 | Problem and Solution

In other words if I can find the value function for a given policy that tells me how good that policy is. We call this V subscript pi of s the value function given the policy pi we call the problem of finding the value function given a policy. The prediction problem as promised. I said that we've arrived at something pretty simple. Let's look at the Bellman equation again. These probabilities are a little deceptive. They have complicated symbols but they are not complicated. Remember that the policy pi is just a function which we implement in our own code. We know this probability. Therefore it's not a problem. How about the state transition probability.

Let's assume this is known as well. This is entirely possible.

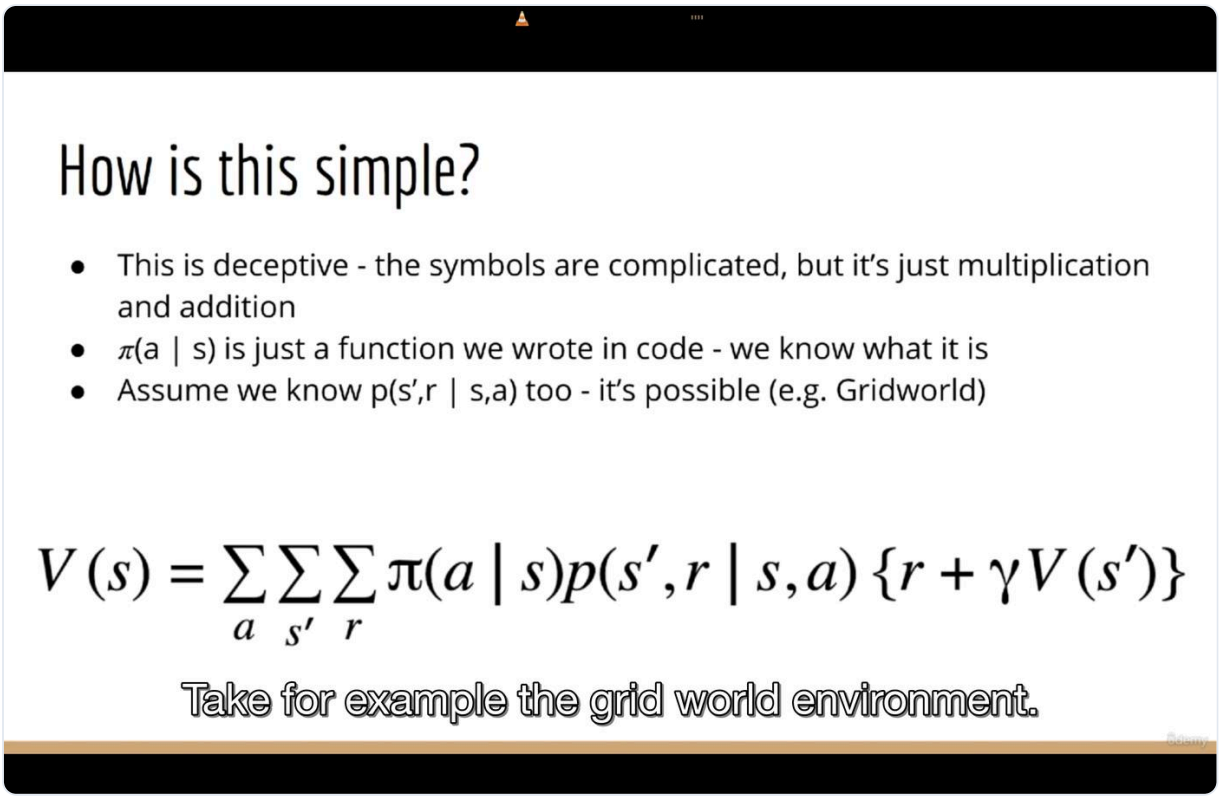


FIGURE 55.12 | How Is This Simple?

Take for example the grid world environment. Now what happens if we know these two probabilities surprisingly.

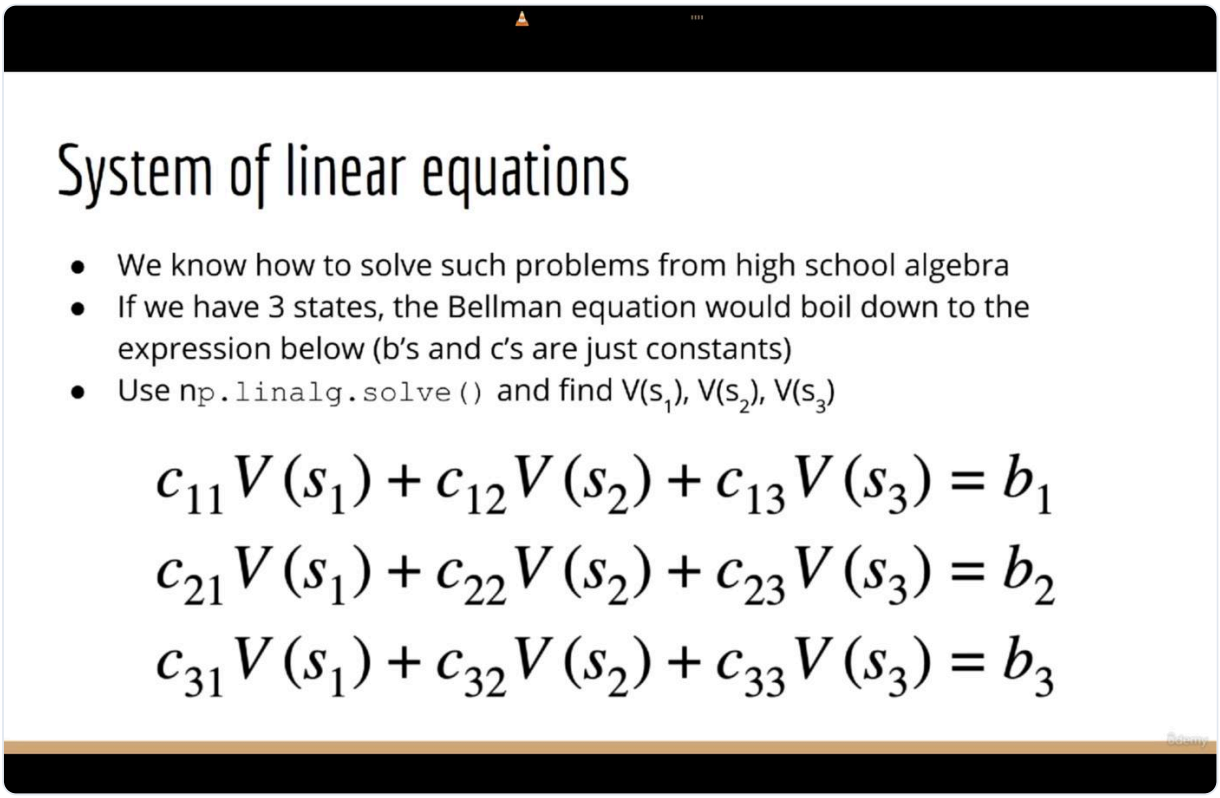


FIGURE 55.13 | System of Linear Equations

This just becomes a system of linear equations which you know how to solve from high school algebra suppose we have three states. Then if we simplify the summation from the Bellman equation we would arrive at something like this. So here the b's and the c's are just constants which are derived from the probabilities which we've assumed that we know from this we can call a simple function like np dot linalg dot solve and this would tell us V of s1 V of s2 and V of s3. In other words if we know both our policy and the environment dynamics we can solve for the value function using only linear algebra.